

# Indice

|                                                                                      |               |
|--------------------------------------------------------------------------------------|---------------|
| <b>Walkthrough del codice – materiale di studio per l'orale (DCLV)</b>               | <b>14</b>     |
| 1 - HM0 – Falcon 9, ascesa 3-DoF (HM0_falcon9_ascent/) . . . . .                     | 14            |
| 2 - HM1 – ascesa planare, metodo indiretto (HM1/) . . . . .                          | 14            |
| 3 - HM2 – powered descent, metodi diretti (HM2_powered_descent/) . . . . .           | 15            |
| 4 - HM3 – il modello del lanciatore a max-q (HM3/) . . . . .                         | 15            |
| 5 - HM3 – margini e progetto: <b>il cuore metodologico</b> . . . . .                 | 16            |
| 6 - HM3 – gli entry point (quelli che il professore fa girare) . . . . .             | 16            |
| 7 - HM3 – track Simulink . . . . .                                                   | 17            |
| 8 - LTV_FULL_ASCENT – l'estensione LPV (oltre la traccia) . . . . .                  | 17            |
| 9 - Test e benchmark . . . . .                                                       | 17            |
| Consiglio di studio . . . . .                                                        | 18            |
| <br><b>HM0_falcon9_ascent/main.m</b>                                                 | <br><b>19</b> |
| Ruolo del file nel progetto . . . . .                                                | 19            |
| Intestazione e <code>clear</code> (righe 1-6) . . . . .                              | 19            |
| CONSTANTS AND VEHICLE PARAMETERS (righe 8-54) . . . . .                              | 20            |
| INITIAL CONDITIONS – Launch from Kennedy Space Center (righe 56-71) . . . . .        | 21            |
| PARAMETER STRUCTURE (righe 73-90) . . . . .                                          | 21            |
| NUMERICAL INTEGRATION ( <code>ode45</code> ) (righe 92-98) . . . . .                 | 22            |
| POST-PROCESSING (righe 100-188) . . . . .                                            | 22            |
| CONSOLE SUMMARY (righe 190-212) . . . . .                                            | 25            |
| PLOTS (righe 214-350) . . . . .                                                      | 25            |
| EXPORT FIGURES TO <code>figures/</code> (righe 352-369) . . . . .                    | 25            |
| <code>eom</code> – equazioni del moto (righe 375-466) . . . . .                      | 25            |
| Possibili domande d'esame . . . . .                                                  | 29            |
| <br><b>HM0_falcon9_ascent/main2.m</b>                                                | <br><b>31</b> |
| Ruolo del file nel progetto . . . . .                                                | 31            |
| Intestazione: il contratto della riparametrizzazione (righe 1-35) . . . . .          | 31            |
| CONSTANTS AND VEHICLE PARAMETERS (righe 37-83) e LAUNCH SITE (righe 85-90) . . . . . | 32            |
| REFERENCE QUANTITIES (righe 92-108) . . . . .                                        | 32            |
| NON-DIMENSIONAL PARAMETERS (righe 110-123) . . . . .                                 | 33            |
| NON-DIMENSIONAL INITIAL CONDITIONS (righe 125-135) . . . . .                         | 34            |
| ARC DURATIONS (righe 137-147) . . . . .                                              | 34            |
| PARAMETER STRUCTURE (righe 149-169) . . . . .                                        | 34            |
| SINGLE <code>ode45</code> CALL (righe 171-184) . . . . .                             | 35            |
| RECOVER GLOBAL ND TIME AND DIMENSIONAL QUANTITIES (righe 186-268) . . . . .          | 36            |
| CONSOLE SUMMARY (righe 270-300) . . . . .                                            | 37            |
| PLOTS (righe 302-462) . . . . .                                                      | 37            |
| <code>eom_nd</code> – equazioni del moto adimensionali (righe 468-578) . . . . .     | 37            |
| <code>arc_boundary_events</code> (righe 582-589) . . . . .                           | 39            |
| Possibili domande d'esame . . . . .                                                  | 39            |

|                                                                                   |               |
|-----------------------------------------------------------------------------------|---------------|
| <b>HM1/ode_burn.m</b>                                                             | <b>41</b>     |
| Ruolo del file nel progetto . . . . .                                             | 41            |
| ode_burn – intestazione e contratto (righe 1-14) . . . . .                        | 41            |
| Unpack dello stato (riga 16) . . . . .                                            | 42            |
| Costati: ricostruzione analitica (righe 18-21) . . . . .                          | 42            |
| Legge di controllo ottima (righe 23-24) . . . . .                                 | 43            |
| Derivate dello stato (righe 26-33) . . . . .                                      | 44            |
| Possibili domande d’esame . . . . .                                               | 45            |
| <br><b>HM1/main_task1.m</b>                                                       | <br><b>47</b> |
| Ruolo del file nel progetto . . . . .                                             | 47            |
| Header e %% Parameters (righe 1-18) . . . . .                                     | 47            |
| %% Storage (righe 20-24) . . . . .                                                | 48            |
| %% Solve BVP for each yf, sweeping Q with continuation (righe 26-94) . . . . .    | 48            |
| %% Optimal Q* per target altitude (righe 96-105) . . . . .                        | 50            |
| %% PLOT 1a: Final mass vs Q (righe 107-118) . . . . .                             | 50            |
| %% PLOT 1b: Velocity losses vs Q for yf = 0.04 (righe 120-153) . . . . .          | 50            |
| %% PLOT 1c: Trajectory and angles for optimal Q (righe 155-205) . . . . .         | 51            |
| %% EXPORT FIGURES (righe 207-223) . . . . .                                       | 52            |
| shooting1 – il residuo del BVP (righe 227-274) . . . . .                          | 52            |
| set_costates (righe 276-287) . . . . .                                            | 54            |
| ode_burn_losses (righe 289-323) . . . . .                                         | 54            |
| Possibili domande d’esame . . . . .                                               | 54            |
| <br><b>HM1/main_task2.m</b>                                                       | <br><b>56</b> |
| Ruolo del file nel progetto . . . . .                                             | 56            |
| Richiamo: le condizioni necessarie (PMP) usate dall’arco propulso . . . . .       | 56            |
| Intestazione e parametri (righe 1-17) . . . . .                                   | 57            |
| Fase 1 – salita verticale (righe 19-37) . . . . .                                 | 57            |
| Fase 2 – impostazione e soluzione del BVP (righe 39-64) . . . . .                 | 58            |
| Estrazione della soluzione e risultati (righe 66-87) . . . . .                    | 59            |
| Plots (righe 89-153) . . . . .                                                    | 59            |
| Export figure (righe 158-174) . . . . .                                           | 59            |
| ode_vertical (righe 178-187) . . . . .                                            | 59            |
| event_altitude (righe 189-199) . . . . .                                          | 60            |
| shooting2 (righe 201-247) . . . . .                                               | 60            |
| Limiti noti / punti onesti da dichiarare . . . . .                                | 61            |
| Possibili domande d’esame . . . . .                                               | 61            |
| <br><b>HM1/main_task3.m</b>                                                       | <br><b>63</b> |
| Ruolo del file nel progetto . . . . .                                             | 63            |
| Richiamo: la funzione di switching e il problema bang-bang . . . . .              | 63            |
| Intestazione e parametri (righe 1-18) . . . . .                                   | 64            |
| Fase 1 – salita verticale (righe 20-31) . . . . .                                 | 64            |
| Fasi 2+3 – impostazione del BVP burn + coast (righe 33-61) . . . . .              | 64            |
| Loop dei guess e filtro sulla radice fisica (righe 63-77) . . . . .               | 65            |
| Post-processing: ricostruzione del coast in forma chiusa (righe 79-119) . . . . . | 66            |
| Plots (righe 121-195) . . . . .                                                   | 66            |
| Export figure (righe 201-217) . . . . .                                           | 66            |
| ode_vertical (righe 221-230) e event_altitude (righe 232-242) . . . . .           | 66            |
| shooting3 (righe 244-305) . . . . .                                               | 67            |
| Limiti noti / punti onesti da dichiarare . . . . .                                | 69            |
| Possibili domande d’esame . . . . .                                               | 69            |
| <br><b>HM1/main_task4.m</b>                                                       | <br><b>71</b> |
| Ruolo del file nel progetto . . . . .                                             | 71            |

|                                                                                |            |
|--------------------------------------------------------------------------------|------------|
| %% Parameters (righe 7-17) . . . . .                                           | 71         |
| %% First solve single-stage (Task 1) as reference (righe 19-36) . . . . .      | 72         |
| La matematica dello staging: salto di massa, costati, Hamiltoniana . . . . .   | 72         |
| %% Sweep staging time (righe 38-112) . . . . .                                 | 75         |
| %% Find optimal staging + plot (righe 114-191) . . . . .                       | 76         |
| %% EXPORT FIGURES (righe 195-211) . . . . .                                    | 76         |
| shooting_single (righe 215-246) . . . . .                                      | 76         |
| shooting_twostage (righe 248-298) . . . . .                                    | 77         |
| Limiti e onesta' sul file . . . . .                                            | 78         |
| Possibili domande d'esame . . . . .                                            | 78         |
| <b>HM1/validate_rocket_sled.m</b>                                              | <b>80</b>  |
| Ruolo del file nel progetto . . . . .                                          | 80         |
| Header e dati del problema (righe 1-22) . . . . .                              | 80         |
| Single shooting: fsolve sui costati iniziali (righe 24-27) . . . . .           | 82         |
| Report e cross-check sul controllo (righe 29-48) . . . . .                     | 82         |
| sled_ode (righe 51-61) . . . . .                                               | 83         |
| sled_residual (righe 63-74) . . . . .                                          | 83         |
| Possibili domande d'esame . . . . .                                            | 84         |
| <b>HM1/validate_staging_corner.m</b>                                           | <b>86</b>  |
| Ruolo del file nel progetto . . . . .                                          | 86         |
| La condizione che stiamo validando . . . . .                                   | 86         |
| %% Parameters (righe 15-27) . . . . .                                          | 87         |
| %% Step 1: warm-start from a fixed-ts inner BVP (righe 29-36) . . . . .        | 87         |
| %% Step 2: augmented solve (righe 38-41) . . . . .                             | 88         |
| %% Extract and report (righe 43-60) . . . . .                                  | 88         |
| %% Check: the burnout reference lam_m(tf) is essential (righe 62-72) . . . . . | 89         |
| propagate (righe 75-94) . . . . .                                              | 90         |
| shooting_inner (righe 96-121) . . . . .                                        | 90         |
| shooting_corner (righe 123-154) . . . . .                                      | 91         |
| shooting_corner_wrong (righe 156-187) . . . . .                                | 91         |
| Cosa questo script valida davvero, e cosa no . . . . .                         | 92         |
| Possibili domande d'esame . . . . .                                            | 92         |
| <b>HM2_powered_descent/ode_descent.m</b>                                       | <b>94</b>  |
| Ruolo del file nel progetto . . . . .                                          | 94         |
| ode_descent (righe 1-16) . . . . .                                             | 94         |
| Perche' due RHS diversi (ode_descent vs ode_descent_uacc) . . . . .            | 96         |
| Nessun blocco arguments: e' voluto . . . . .                                   | 96         |
| Possibili domande d'esame . . . . .                                            | 97         |
| <b>HM2_powered_descent/ode_descent_uacc.m</b>                                  | <b>98</b>  |
| Ruolo del file nel progetto . . . . .                                          | 98         |
| ode_descent_uacc (righe 1-21) . . . . .                                        | 98         |
| Perche' esistono entrambi i RHS . . . . .                                      | 100        |
| Nessun blocco arguments: e' voluto . . . . .                                   | 100        |
| Possibili domande d'esame . . . . .                                            | 100        |
| <b>HM2_powered_descent/rk4_zoh.m</b>                                           | <b>102</b> |
| Ruolo del file nel progetto . . . . .                                          | 102        |
| rk4_zoh (righe 1-25) . . . . .                                                 | 102        |
| Perche' ZOH e non un'interpolazione di ordine superiore . . . . .              | 104        |
| Costo computazionale e assenza del blocco arguments . . . . .                  | 104        |
| Possibili domande d'esame . . . . .                                            | 105        |

|                                                                            |                |
|----------------------------------------------------------------------------|----------------|
| <b>HM2_powered_descent/lti_zoh.m</b>                                       | <b>106</b>     |
| Ruolo del file nel progetto . . . . .                                      | 106            |
| lti_zoh (righe 1-28) . . . . .                                             | 106            |
| “Esatta”: in che senso, e quando conviene davvero rispetto a RK4 . . . . . | 109            |
| Cosa resta non convesso (e perche’ c’e’ ancora una SCvx) . . . . .         | 110            |
| Possibili domande d’esame . . . . .                                        | 110            |
| <br><b>HM2_powered_descent/main_task1.m</b>                                | <br><b>112</b> |
| Ruolo del file nel progetto . . . . .                                      | 112            |
| Header e setup (righe 1-12) . . . . .                                      | 112            |
| Problem data (Table 1, dimensional) (righe 14-29) . . . . .                | 113            |
| Non-dimensionalisation (righe 31-35) . . . . .                             | 113            |
| Sensitivity sweep on flight time (righe 37-57) . . . . .                   | 114            |
| Plots / Export figures / Summary table (righe 59-85) . . . . .             | 115            |
| Grid-convergence study (righe 87-101) . . . . .                            | 115            |
| nondim (righe 109-134) . . . . .                                           | 115            |
| dim_sol (righe 136-164) . . . . .                                          | 116            |
| solve_trapcol (righe 166-267) – il cuore della trascrizione . . . . .      | 116            |
| trap_nonlcon (righe 269-304) – defect e vincoli di percorso . . . . .      | 118            |
| dyn_rhs (righe 306-315) . . . . .                                          | 121            |
| diagnostics (righe 317-349) . . . . .                                      | 121            |
| fwd_integrate_pwl (righe 351-377) e node_err (righe 379-387) . . . . .     | 123            |
| plot_results (righe 389-460) . . . . .                                     | 123            |
| Possibili domande d’esame . . . . .                                        | 123            |
| <br><b>HM2_powered_descent/main_task2.m</b>                                | <br><b>126</b> |
| Ruolo del file nel progetto . . . . .                                      | 126            |
| Il problema di partenza e perche’ NON e’ convesso . . . . .                | 126            |
| Teoria: la trascrizione ZOH per sistemi LTV (Appendice A) . . . . .        | 127            |
| Teoria: Successive Convexification (SCvx) . . . . .                        | 128            |
| Teoria: lossless convexification e il cambio di variabili GFOLD . . . . .  | 130            |
| %% Problem data (Table 1, dimensional) (righe 25-39) . . . . .             | 132            |
| %% Non-dimensionalisation (righe 41-49) . . . . .                          | 132            |
| %% YALMIP + ECOS availability check (righe 51-55) . . . . .                | 133            |
| %% Solve all three transcriptions (righe 57-110) . . . . .                 | 133            |
| %% Forward-integration validation (righe 112-137) . . . . .                | 133            |
| %% Replay landing accuracy + wall-time summary (righe 139-165) . . . . .   | 134            |
| %% Plots e %% Export figures (righe 167-193) . . . . .                     | 134            |
| nondim (righe 203-228) e dim_sol (righe 230-253) . . . . .                 | 134            |
| jacobians (righe 255-277) . . . . .                                        | 134            |
| ltv_aug_rhs (righe 279-303) . . . . .                                      | 135            |
| compute_ltv_zoh (righe 305-339) . . . . .                                  | 135            |
| solve_ltv_nlp (righe 341-412) e ltv_nonlcon (righe 414-427) . . . . .      | 136            |
| solve_scvx (righe 429-537) . . . . .                                       | 136            |
| ternary (righe 539-548) . . . . .                                          | 137            |
| ref_to_z (righe 550-569) e apply_trust (righe 571-600) . . . . .           | 137            |
| solve_trap (righe 602-624) e solve_zoh (righe 626-650) . . . . .           | 138            |
| init_guess (righe 652-676) e box_bounds (righe 678-700) . . . . .          | 138            |
| bcs (righe 702-720) . . . . .                                              | 138            |
| fmincon_opts (righe 722-736) . . . . .                                     | 138            |
| unpack (righe 738-757) . . . . .                                           | 139            |
| trap_nonlcon (righe 759-781) e zoh_nonlcon (righe 783-803) . . . . .       | 139            |
| path_ineq (righe 805-820) . . . . .                                        | 139            |
| fwd_integrate (righe 822-857) . . . . .                                    | 139            |
| node_err (righe 859-868) . . . . .                                         | 140            |
| plot_compare3 (righe 870-963) . . . . .                                    | 140            |

|                                                                         |            |
|-------------------------------------------------------------------------|------------|
| solve_ltv_nlp_yalmip (righe 965-1040) . . . . .                         | 140        |
| solve_scvx_yalmip (righe 1042-1121) . . . . .                           | 141        |
| solve_gfold_socp (righe 1123-1196) . . . . .                            | 141        |
| solve_gfold_scvx (righe 1198-1283) . . . . .                            | 142        |
| fwd_integrate_uacc (righe 1285-1303) . . . . .                          | 143        |
| plot_compare5 (righe 1305-1420) . . . . .                               | 143        |
| Cosa misura davvero il confronto, e cosa emerge . . . . .               | 143        |
| Limiti, hack e deviazioni dalla teoria – da dichiarare . . . . .        | 144        |
| Possibili domande d’esame . . . . .                                     | 145        |
| <b>HM2_powered_descent/proto_gfold_logmass.m</b>                        | <b>147</b> |
| Ruolo del file nel progetto . . . . .                                   | 147        |
| Header e cambio di variabile (righe 1-27) . . . . .                     | 147        |
| Dati del problema e non-dimensionalizzazione (righe 31-48) . . . . .    | 149        |
| Controllo disponibilit  solver (righe 50-53) . . . . .                  | 149        |
| Chiamata al solver SCvx (righe 55-57) . . . . .                         | 149        |
| Validazione in variabili originali (righe 59-86) . . . . .              | 149        |
| nondim (righe 103-113) . . . . .                                        | 150        |
| dim_sol (righe 115-123) . . . . .                                       | 150        |
| solve_gfold_socp (righe 125-177) . . . . .                              | 151        |
| solve_gfold_scvx (righe 179-239) . . . . .                              | 153        |
| fwd_integrate_uacc (righe 241-254) . . . . .                            | 154        |
| ternary (righe 256-258) . . . . .                                       | 154        |
| Limiti noti, in chiaro . . . . .                                        | 154        |
| Possibili domande d’esame . . . . .                                     | 154        |
| <b>HM3/load_hw3_params.m</b>                                            | <b>157</b> |
| Ruolo del file nel progetto . . . . .                                   | 157        |
| load_hw3_params – firma e blocco arguments (righe 1-23) . . . . .       | 157        |
| Letterali Table 1: geometria, masse, forze (righe 25-35) . . . . .      | 158        |
| L’incongruenza su a4 (righe 36-38, 44) . . . . .                        | 159        |
| Bending, sensore INS, attuatore TVC (righe 47-57) . . . . .             | 159        |
| Override dal dataset LPV (righe 59-80) . . . . .                        | 159        |
| Pressione dinamica derivata (righe 82-84) . . . . .                     | 160        |
| Scaling di incertezza Task 3 (righe 86-92) . . . . .                    | 160        |
| Possibili domande d’esame . . . . .                                     | 161        |
| <b>HM3/build_plant_rigid.m</b>                                          | <b>162</b> |
| Ruolo del file nel progetto . . . . .                                   | 162        |
| Firma e contratto (righe 1-9) . . . . .                                 | 162        |
| La matrice A: le equazioni del moto (righe 11-14) . . . . .             | 163        |
| Le colonne di ingresso (righe 16-17) . . . . .                          | 164        |
| Le uscite: misure e segnali di plot (righe 19-28) . . . . .             | 165        |
| L’oggetto ss e i suoi nomi (righe 30-34) . . . . .                      | 166        |
| Il polo instabile: da dove viene esattamente . . . . .                  | 166        |
| Possibili domande d’esame . . . . .                                     | 167        |
| <b>HM3/build_plant_full.m</b>                                           | <b>168</b> |
| Ruolo del file nel progetto . . . . .                                   | 168        |
| Firma, arguments, contratto (righe 1-16) . . . . .                      | 168        |
| La matrice A: le due righe nuove (righe 20-25) . . . . .                | 168        |
| Le colonne di ingresso (righe 27-28) . . . . .                          | 169        |
| L’osservazione INS: Eq. (2) (righe 30-44) . . . . .                     | 170        |
| Da dove vengono i +29 dB (la derivazione che serve all’orale) . . . . . | 170        |
| Possibili domande d’esame . . . . .                                     | 171        |

|                                                                                 |            |
|---------------------------------------------------------------------------------|------------|
| <b>HM3/build_tvc.m</b>                                                          | <b>173</b> |
| Ruolo del file nel progetto . . . . .                                           | 173        |
| build_tvc (righe 1-23) . . . . .                                                | 173        |
| Possibili domande d'esame . . . . .                                             | 175        |
| <b>HM3/build_notch_filter.m</b>                                                 | <b>177</b> |
| Ruolo del file nel progetto . . . . .                                           | 177        |
| build_notch_filter (righe 1-25) . . . . .                                       | 177        |
| Possibili domande d'esame . . . . .                                             | 181        |
| <b>HM3/assemble_loop.m</b>                                                      | <b>182</b> |
| Ruolo del file nel progetto . . . . .                                           | 182        |
| Firma e contratto (righe 1-21) . . . . .                                        | 182        |
| Il controllore come guadagno statico (righe 23-27) . . . . .                    | 183        |
| La catena attuatore (righe 29-32) . . . . .                                     | 183        |
| Chiusura dell'anello (righe 34-35) . . . . .                                    | 184        |
| Loop aperto e riduzione (righe 37-39) . . . . .                                 | 184        |
| PUNTO CRITICO – perche' margin() da solo non significa niente . . . . .         | 185        |
| Possibili domande d'esame . . . . .                                             | 186        |
| <b>HM3/load_wind_profile.m</b>                                                  | <b>188</b> |
| Ruolo del file nel progetto . . . . .                                           | 188        |
| Firma, docstring e blocco arguments (righe 1-28) . . . . .                      | 188        |
| Il ramo strongwind: uscita anticipata (righe 30-34) . . . . .                   | 189        |
| Ampiezza della raffica dalla dispersione di drywind.mat (righe 36-49) . . . . . | 189        |
| I tre profili analitici (righe 51-65) . . . . .                                 | 190        |
| Dal vento all'angolo d'attacco: alphaw = vw/p.V (righe 67-69) . . . . .         | 190        |
| local_strong_wind (righe 71-112) . . . . .                                      | 193        |
| Possibili domande d'esame . . . . .                                             | 193        |
| <b>HM3/simulate_gust_response.m</b>                                             | <b>195</b> |
| Ruolo del file nel progetto . . . . .                                           | 195        |
| simulate_gust_response (righe 1-35) . . . . .                                   | 195        |
| Il segno di alpha_w (riga 29) – perche' e' un MENO . . . . .                    | 197        |
| Possibili domande d'esame . . . . .                                             | 199        |
| <b>HM3/classify_margins.m</b>                                                   | <b>201</b> |
| Ruolo del file nel progetto . . . . .                                           | 201        |
| Docstring e contratto (righe 1-25) . . . . .                                    | 201        |
| Blocco arguments e setup (righe 27-37) . . . . .                                | 202        |
| allmargin e l'estrazione degli attraversamenti (righe 39-41) . . . . .          | 202        |
| Gain margins: aero, rigid, flex (righe 43-46) . . . . .                         | 202        |
| Phase margins (righe 48-50) . . . . .                                           | 203        |
| Attenuazione del bending: LwBM_dB (righe 52-58) . . . . .                       | 204        |
| Attraversamenti di deriva (righe 60-62) . . . . .                               | 204        |
| Delay margin e flag di stabilita' (righe 64-65) . . . . .                       | 204        |
| Funzione annidata pick (righe 67-79) . . . . .                                  | 204        |
| Possibili domande d'esame . . . . .                                             | 205        |
| <b>HM3/design_controller.m</b>                                                  | <b>207</b> |
| Ruolo del file nel progetto . . . . .                                           | 207        |
| Firma, docstring e contratto (righe 1-23) . . . . .                             | 207        |
| Blocco arguments (righe 25-37) . . . . .                                        | 208        |
| Estrazione di A_6, K_1 e definizione delle bande (righe 40-48) . . . . .        | 208        |
| Silenziamento del warning MarginUnstable (righe 50-52) . . . . .                | 209        |
| Il re-tune: seed canonico + fminsearch (righe 54-57) . . . . .                  | 209        |

|                                                                                |            |
|--------------------------------------------------------------------------------|------------|
| Applicazione dei guadagni e verdetto di stabilita' (righe 59-69)               | 210        |
| Stampa diagnostica (righe 71-76)                                               | 211        |
| Funzione annidata <code>cost</code> (righe 78-90)                              | 211        |
| Possibili domande d'esame                                                      | 212        |
| <b>HM3/plot_nichols_lv.m</b>                                                   | <b>214</b> |
| Ruolo del file nel progetto                                                    | 214        |
| Docstring e contratto (righe 1-22)                                             | 214        |
| Blocco <code>arguments</code> (righe 24-30)                                    | 215        |
| Scelta della frequenza di riferimento (riga 35)                                | 215        |
| Nichols nativa e phase matching a -180 (righe 37-44)                           | 215        |
| Replica manuale dello shift per gli overlay (righe 46-49)                      | 216        |
| Marker dei margini classificati (righe 51-56)                                  | 217        |
| Attraversamenti di deriva marcati come non-margini (righe 58-67)               | 217        |
| Legenda (righe 68-72)                                                          | 217        |
| Funzione annidata <code>addmark</code> (righe 74-82)                           | 218        |
| Possibili domande d'esame                                                      | 218        |
| <b>HM3/make_pz_figures.m</b>                                                   | <b>220</b> |
| Ruolo del file nel progetto                                                    | 220        |
| Intestazione, stile grafico, cartella di uscita (righe 1-35)                   | 220        |
| Punto di progetto condiviso: quali guadagni entrano nelle figure (righe 37-41) | 221        |
| Anatomia del piano s: da dove viene ogni polo e ogni zero (righe 43-58)        | 221        |
| (1) I tre luoghi delle radici (righe 60-65)                                    | 224        |
| (2) Task 1: mappa dei poli in anello chiuso (righe 67-79)                      | 224        |
| (3) Task 2: la coppia del bending con e senza notch (righe 81-96)              | 226        |
| (4) Task 3: migrazione dei poli sul box +/-30% (righe 98-117)                  | 226        |
| Export (righe 119-128)                                                         | 227        |
| Helper (righe 131-168)                                                         | 227        |
| Possibili domande d'esame                                                      | 228        |
| <b>HM3/main_task1.m</b>                                                        | <b>230</b> |
| Ruolo del file nel progetto                                                    | 230        |
| % Model and parameters (Table 1 @ t = 72 s) (righe 12-18)                      | 230        |
| % Controller design (PD pitch + weak negative drift feedback) (righe 20-31)    | 231        |
| % Rigid-body stability margins (righe 33-41)                                   | 233        |
| % Wind-gust time response (righe 43-55)                                        | 234        |
| % Figures (righe 57-99)                                                        | 235        |
| Possibili domande d'esame                                                      | 236        |
| <b>HM3/main_task2.m</b>                                                        | <b>238</b> |
| Ruolo del file nel progetto                                                    | 238        |
| % Model and parameters + %% Step A (righe 19-26)                               | 238        |
| % Step B - full plant + TVC + delay, NO bending filter (righe 28-34)           | 239        |
| % Step C - bending filter trade study (righe 36-129)                           | 239        |
| % Step C decision - deep notch, then RE-TUNE the PD (righe 131-161)            | 242        |
| % Step D - sensitivity to the bending-frequency knowledge (righe 163-179)      | 243        |
| % Time response to a wind gust (righe 181-189)                                 | 244        |
| % Figures (righe 191-267)                                                      | 245        |
| Possibili domande d'esame                                                      | 245        |
| <b>HM3/main_task3.m</b>                                                        | <b>247</b> |
| Ruolo del file nel progetto                                                    | 247        |
| % Fixed controller (Task 2 retained design) (righe 16-26)                      | 247        |
| % Cases: box vertices + one-at-a-time sensitivities (righe 28-67)              | 248        |
| La matematica dietro i numeri (derivazione, non nel codice)                    | 249        |

|                                                                                  |            |
|----------------------------------------------------------------------------------|------------|
| %% Figures (righe 69-105) . . . . .                                              | 250        |
| Cosa il Task 3 NON copre (limiti dichiarati) . . . . .                           | 251        |
| Possibili domande d'esame . . . . .                                              | 251        |
| <b>HM3/main_montecarlo.m</b>                                                     | <b>253</b> |
| Ruolo del file nel progetto . . . . .                                            | 253        |
| RIQUADRO DI ONESTA': lo script E' STALE rispetto ai Task 1-3 . . . . .           | 253        |
| Intestazione e configurazione (righe 1-37) . . . . .                             | 255        |
| Controller fisso e modello nominale (righe 39-54) . . . . .                      | 256        |
| Pre-campionamento (righe 56-63) . . . . .                                        | 256        |
| Propagazione Monte Carlo (righe 65-118) . . . . .                                | 257        |
| Statistiche (righe 120-145) . . . . .                                            | 257        |
| Figura 1 – istogrammi (righe 147-157) . . . . .                                  | 258        |
| Figura 2 – nuvola di Nichols (righe 159-186) . . . . .                           | 258        |
| Figura 3 – sensibilita' (righe 188-216) . . . . .                                | 259        |
| Export (righe 218-237) . . . . .                                                 | 259        |
| Funzioni locali (righe 239-315) . . . . .                                        | 259        |
| Senso statistico: vertici (Task 3) contro Monte Carlo . . . . .                  | 261        |
| Possibili domande d'esame . . . . .                                              | 262        |
| <b>HM3/init_simulink_hm3.m</b>                                                   | <b>264</b> |
| Ruolo del file nel progetto . . . . .                                            | 264        |
| Firma e arguments block (righe 1-26) . . . . .                                   | 264        |
| Parametri e controllore (righe 28-38) . . . . .                                  | 265        |
| Matrici del plant (righe 40-51) . . . . .                                        | 266        |
| Attuatore TVC + notch (righe 53-57) . . . . .                                    | 267        |
| Vento come timeseries (righe 59-64) . . . . .                                    | 268        |
| Push nel base workspace (righe 66-75) . . . . .                                  | 268        |
| Discrepanze verificate fra i due track . . . . .                                 | 268        |
| Possibili domande d'esame . . . . .                                              | 269        |
| <b>HM3/run_simulink_closed_loop.m</b>                                            | <b>271</b> |
| Ruolo del file nel progetto . . . . .                                            | 271        |
| Firma, arguments e guardia sul modello (righe 1-29) . . . . .                    | 271        |
| Init del workspace e baseline degli script (righe 31-47) . . . . .               | 272        |
| Simulazione del modello (righe 49-57) . . . . .                                  | 273        |
| Overlay e figura (righe 59-83) . . . . .                                         | 274        |
| get_logged_signal (righe 86-102) . . . . .                                       | 275        |
| Discrepanze e limiti verificati . . . . .                                        | 276        |
| Possibili domande d'esame . . . . .                                              | 276        |
| <b>HM3/LTV_FULL_ASCENT/ode_lpv_ascent.m</b>                                      | <b>278</b> |
| Ruolo del file nel progetto . . . . .                                            | 278        |
| Firma e contratto (righe 1-13) . . . . .                                         | 278        |
| Perche' il sistema e' LPV (e non solo "non lineare") . . . . .                   | 279        |
| Vento: aw = M.windfun(t) (riga 14) . . . . .                                     | 279        |
| Il controllore chiuso dentro la RHS (righe 17-22) . . . . .                      | 280        |
| Il plant tempo-variante (righe 25-28) . . . . .                                  | 280        |
| Che cosa viene interpolato davvero (init_simulink_lpv.m, righe 49-107) . . . . . | 281        |
| La fallacia del frozen-time (il punto centrale) . . . . .                        | 282        |
| Limiti e punti fragili (riepilogo onesto) . . . . .                              | 283        |
| Possibili domande d'esame . . . . .                                              | 283        |
| <b>HM3/LTV_FULL_ASCENT/ode_lpv_flex.m</b>                                        | <b>285</b> |
| Ruolo del file nel progetto . . . . .                                            | 285        |
| Firma, stato e contratto (righe 1-16) . . . . .                                  | 285        |



|                                                                                        |            |
|----------------------------------------------------------------------------------------|------------|
| Coefficienti tempo-varianti: qui si interpola il <b>grezzo</b> (righe 19-21) . . . . . | 286        |
| Misure INS contaminate dal bending (righe 24-25) . . . . .                             | 286        |
| Il notch tempo-variante (righe 30-33) . . . . .                                        | 287        |
| TVC e ritardo: unico blocco LTI (righe 34-35) . . . . .                                | 288        |
| Il plant flessibile (righe 38-40) . . . . .                                            | 288        |
| Assemblaggio (riga 42) . . . . .                                                       | 289        |
| Rigido vs flessibile: cosa cambia davvero . . . . .                                    | 289        |
| La fallacia del frozen-time, vista da qui . . . . .                                    | 289        |
| Limiti e punti fragili (riepilogo onesto) . . . . .                                    | 290        |
| Possibili domande d'esame . . . . .                                                    | 290        |
| <b>HM3/LTV_FULL_ASCENT/build_hm3_full_ascent.m</b>                                     | <b>292</b> |
| Ruolo del file nel progetto . . . . .                                                  | 292        |
| Firma, header e <b>arguments</b> (righe 1-18) . . . . .                                | 292        |
| Percorsi, generatore del professore, modello nuovo e solver (righe 20-37) . . . . .    | 293        |
| Le scorciatoie di authoring: A, W, LK (righe 39-42) . . . . .                          | 293        |
| Clock e le lookup dei coefficienti effettivi (righe 44-54) . . . . .                   | 294        |
| Il generatore di vento e <b>alpha_w</b> (righe 56-64) . . . . .                        | 295        |
| Il plant: prodotti -> somme -> integratori (righe 66-100) . . . . .                    | 296        |
| Il controllore: frozen contro scheduled (righe 102-133) . . . . .                      | 296        |
| Logging (righe 135-146) . . . . .                                                      | 297        |
| Salvataggio (righe 148-153) . . . . .                                                  | 298        |
| Limiti, hack e codice stale (riepilogo onesto) . . . . .                               | 298        |
| Possibili domande d'esame . . . . .                                                    | 299        |
| <b>HM3/LTV_FULL_ASCENT/build_hm3_full_ascent_flex.m</b>                                | <b>300</b> |
| Ruolo del file nel progetto . . . . .                                                  | 300        |
| Firma, header e setup (righe 1-41) . . . . .                                           | 300        |
| Clock e le 14 lookup (righe 43-51) . . . . .                                           | 301        |
| Il generatore di vento e <b>alpha_w</b> (righe 53-58) . . . . .                        | 301        |
| Plant z/theta (righe 60-83) . . . . .                                                  | 302        |
| Bending (righe 85-96) . . . . .                                                        | 302        |
| Accoppiamento INS (righe 98-114) . . . . .                                             | 302        |
| Controllore a guadagni congelati (righe 116-123) . . . . .                             | 303        |
| Il notch tempo-variante (righe 125-141) . . . . .                                      | 303        |
| TVC (righe 143-147) . . . . .                                                          | 305        |
| La disciplina dei coefficienti effettivi, rotta (discrepanza reale) . . . . .          | 305        |
| Logging (righe 149-158) . . . . .                                                      | 306        |
| Salvataggio (righe 160-165) . . . . .                                                  | 307        |
| Rigido contro flessibile: il riassunto . . . . .                                       | 307        |
| Limiti, hack e codice stale (riepilogo onesto) . . . . .                               | 308        |
| Possibili domande d'esame . . . . .                                                    | 308        |
| <b>HM3/LTV_FULL_ASCENT/main_full_ascent.m</b>                                          | <b>310</b> |
| Ruolo del file nel progetto . . . . .                                                  | 310        |
| Intestazione e preambolo (righe 1-20) . . . . .                                        | 310        |
| <b>% Setup</b> – coefficienti LPV, gain schedule, vento (righe 22-24) . . . . .        | 311        |
| <b>% LTV closed-loop integration</b> – frozen vs scheduled (righe 26-35) . . . . .     | 311        |
| <b>% Frozen-time margin sweep</b> (righe 37-47) . . . . .                              | 312        |
| <b>% Consistency check at t_ref</b> (righe 49-61) . . . . .                            | 313        |
| <b>% Summary</b> (righe 63-73) . . . . .                                               | 313        |
| <b>% Figures</b> (righe 75-111) e <b>% Export</b> (righe 113-120) . . . . .            | 313        |
| <b>make_model</b> (righe 123-134) . . . . .                                            | 314        |
| <b>unpack</b> (righe 136-163) . . . . .                                                | 314        |
| <b>loop_margin</b> (righe 165-176) . . . . .                                           | 314        |
| Cosa produce oggi il codice (verifica numerica) e divergenze dal README . . . . .      | 315        |

|                                                                                                                                                 |            |
|-------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Il generatore di vento del professore: come e' cablato e perche' usarlo . . . . .                                                               | 315        |
| La fallacia del frozen-time e come questo script la mette alla prova . . . . .                                                                  | 316        |
| Possibili domande d'esame . . . . .                                                                                                             | 316        |
| <b>HM3/LTV_FULL_ASCENT/main_flex.m</b>                                                                                                          | <b>319</b> |
| Ruolo del file nel progetto . . . . .                                                                                                           | 319        |
| Intestazione e preambolo (righe 1-19) . . . . .                                                                                                 | 319        |
| %% Frozen-time detuning sweep (righe 21-33) . . . . .                                                                                           | 320        |
| %% Time-domain flexible response (righe 35-41) . . . . .                                                                                        | 321        |
| %% Summary (righe 43-51) . . . . .                                                                                                              | 322        |
| %% Figures (righe 53-81) e %% Export (righe 83-90) . . . . .                                                                                    | 322        |
| make_flex (righe 93-106) . . . . .                                                                                                              | 323        |
| La fallacia del frozen-time, vista qui . . . . .                                                                                                | 323        |
| Possibili domande d'esame . . . . .                                                                                                             | 324        |
| <b>HM3/LTV_FULL_ASCENT/main_q_scheduling.m</b>                                                                                                  | <b>325</b> |
| Ruolo del file nel progetto . . . . .                                                                                                           | 325        |
| Intestazione e preambolo (righe 1-21) . . . . .                                                                                                 | 325        |
| %% Build a q-keyed gain lookup (righe 23-37) . . . . .                                                                                          | 326        |
| Perche' q non basta – la derivazione (il cuore dello studio) . . . . .                                                                          | 326        |
| %% Closed-loop responses (righe 39-50) . . . . .                                                                                                | 327        |
| %% Frozen-time margin sweep (righe 52-61) . . . . .                                                                                             | 328        |
| %% Summary (righe 63-74) . . . . .                                                                                                              | 328        |
| %% Figures (righe 76-115) e %% Export (righe 117-124) . . . . .                                                                                 | 329        |
| make_model (righe 127-144) . . . . .                                                                                                            | 329        |
| unpack (righe 146-161) . . . . .                                                                                                                | 329        |
| loop_margin (righe 163-174) . . . . .                                                                                                           | 329        |
| Possibili domande d'esame . . . . .                                                                                                             | 329        |
| <b>HM3/LTV_FULL_ASCENT/init_simulink_lpv.m</b>                                                                                                  | <b>331</b> |
| Ruolo del file nel progetto . . . . .                                                                                                           | 331        |
| Firma, contratto e opzioni (righe 1-37) . . . . .                                                                                               | 331        |
| Caricamento dati di riferimento (righe 44-47) . . . . .                                                                                         | 332        |
| Coefficienti efficaci e breakpoints (righe 49-68) . . . . .                                                                                     | 332        |
| Controllore congelato di riferimento (righe 70-72) . . . . .                                                                                    | 333        |
| Gain schedule (righe 74-85) . . . . .                                                                                                           | 333        |
| Vento: il generatore del professore, una volta sola (righe 87-90 e 141-178) . . . . .                                                           | 334        |
| Struct di ritorno e interpolanti (righe 92-117) . . . . .                                                                                       | 335        |
| Push al base workspace (righe 119-138) . . . . .                                                                                                | 335        |
| Possibili domande d'esame . . . . .                                                                                                             | 336        |
| <b>HM3/LTV_FULL_ASCENT/run_full_ascent_simulink.m</b>                                                                                           | <b>338</b> |
| Ruolo del file nel progetto . . . . .                                                                                                           | 338        |
| Firma e opzioni (righe 1-26) . . . . .                                                                                                          | 338        |
| Setup: dati, tolleranze, modello (righe 28-37) . . . . .                                                                                        | 339        |
| Ciclo sulle due varianti (righe 39-70) . . . . .                                                                                                | 339        |
| Perche' l'accordo e' 1e-7 rad – cosa deve essere identico . . . . .                                                                             | 340        |
| pack_model (righe 80-93) . . . . .                                                                                                              | 341        |
| Possibili domande d'esame . . . . .                                                                                                             | 342        |
| <b>HM3/LTV_FULL_ASCENT/run_flex_simulink.m</b>                                                                                                  | <b>343</b> |
| Ruolo del file nel progetto . . . . .                                                                                                           | 343        |
| Firma, rebuild, setup (righe 1-27) . . . . .                                                                                                    | 343        |
| Simulazione e replay ode45 (righe 29-43) . . . . .                                                                                              | 344        |
| Il notch variabile: perche' e' realizzato “a mano” (contesto, ode_lpv_flex righe 30-33 e<br>build_hm3_full_ascent_flex righe 125-141) . . . . . | 345        |

|                                                                                |            |
|--------------------------------------------------------------------------------|------------|
| Residui e figura (righe 45-67) . . . . .                                       | 345        |
| Cosa deve essere identico perche' l'overlay funzioni (riepilogo) . . . . .     | 346        |
| Possibili domande d'esame . . . . .                                            | 347        |
| <b>HM0_falcon9_ascent/tests/falcon9AscentTest.m</b>                            | <b>348</b> |
| Ruolo del file nel progetto . . . . .                                          | 348        |
| Dichiarazione della classe e docstring (righe 1-8) . . . . .                   | 348        |
| properties (righe 10-13) . . . . .                                             | 348        |
| properties (TestParameter) (righe 15-17) . . . . .                             | 349        |
| TestClassSetup – runScripts (righe 19-26) . . . . .                            | 349        |
| runAscentScript – il pattern “run + harvest” (righe 65-74) . . . . .           | 350        |
| testPropellantBookkeeping (righe 29-34) . . . . .                              | 351        |
| testMachOneBeforeMaxQ (righe 36-40) . . . . .                                  | 352        |
| testAltitudeStaysAboveGround (righe 42-45) . . . . .                           | 352        |
| testFinalAltitudeInPlausibleBand (righe 47-52) . . . . .                       | 352        |
| testDimensionalVsNonDimensionalAgreement (righe 54-61) . . . . .               | 353        |
| Cosa la suite NON verifica (limiti dichiarati) . . . . .                       | 354        |
| Possibili domande d'esame . . . . .                                            | 354        |
| <b>HM1/tests/odeBurnTest.m</b>                                                 | <b>356</b> |
| Ruolo del file nel progetto . . . . .                                          | 356        |
| addHm1ToPath – TestClassSetup (righe 7-12) . . . . .                           | 356        |
| testKinematicsAndMassFlow (righe 15-25) . . . . .                              | 357        |
| testThrustAlongConstantCostate (righe 27-36) . . . . .                         | 358        |
| testLinearTangentSwitch (righe 38-49) . . . . .                                | 358        |
| testCostateMassEquation (righe 51-59) . . . . .                                | 359        |
| testBallisticLimitMatchesAnalytic (righe 61-74) . . . . .                      | 360        |
| testVerticalBurnTsiolkovskyWithGravity (righe 76-89) . . . . .                 | 361        |
| Lacune di copertura (onesta' richiesta) . . . . .                              | 362        |
| Possibili domande d'esame . . . . .                                            | 363        |
| <b>HM1/tests/odeBurnPerformanceTest.m</b>                                      | <b>364</b> |
| Ruolo del file nel progetto . . . . .                                          | 364        |
| Perche' ode_burn e' un hot loop – il conto degli ordini di grandezza . . . . . | 364        |
| Properties e fixture (righe 8-31) . . . . .                                    | 365        |
| testRhsEvaluation (righe 34-41) . . . . .                                      | 366        |
| testBurnArcIntegration (righe 43-52) . . . . .                                 | 367        |
| Limiti del file (onesta' richiesta) . . . . .                                  | 368        |
| Possibili domande d'esame . . . . .                                            | 368        |
| <b>HM2_powered_descent/tests/odeDescentTest.m</b>                              | <b>370</b> |
| Ruolo del file nel progetto . . . . .                                          | 370        |
| TestClassSetup – addHm2ToPath (righe 5-10) . . . . .                           | 370        |
| testDerivativeDefinition (righe 13-21) . . . . .                               | 371        |
| testBallisticCoast (righe 23-28) . . . . .                                     | 372        |
| testHoverEquilibrium (righe 30-37) . . . . .                                   | 372        |
| testMassFlowDependsOnlyOnThrustMagnitude (righe 39-47) . . . . .               | 373        |
| Cosa questa suite NON copre (onesta' sulla copertura) . . . . .                | 373        |
| Possibili domande d'esame . . . . .                                            | 373        |
| <b>HM2_powered_descent/tests/rk4ZohTest.m</b>                                  | <b>375</b> |
| Ruolo del file nel progetto . . . . .                                          | 375        |
| Proprieta' costanti (righe 5-10) . . . . .                                     | 375        |
| TestClassSetup – addHm2ToPath (righe 12-17) . . . . .                          | 376        |
| ode45Reference – metodo statico (righe 44-54) . . . . .                        | 376        |
| testMatchesOde45Reference (righe 20-24) . . . . .                              | 377        |

|                                                                                                       |            |
|-------------------------------------------------------------------------------------------------------|------------|
| testMassRowIsExact (righe 26-32) . . . . .                                                            | 377        |
| testFourthOrderConvergence (righe 34-41) . . . . .                                                    | 378        |
| Cosa questa suite NON copre (onesta' sulla copertura) . . . . .                                       | 379        |
| Possibili domande d'esame . . . . .                                                                   | 379        |
| <b>HM2_powered_descent/tests/gfoldLogMassTest.m</b>                                                   | <b>381</b> |
| Ruolo del file nel progetto . . . . .                                                                 | 381        |
| properties (Constant) (righe 6-9) . . . . .                                                           | 381        |
| addHm2ToPath (righe 11-16) . . . . .                                                                  | 381        |
| testUaccDerivativeDefinition (righe 20-27) . . . . .                                                  | 382        |
| testAccelerationIsDirect (righe 29-37) . . . . .                                                      | 382        |
| testMassFlowScalesWithMass (righe 39-46) . . . . .                                                    | 383        |
| testBallisticCoast (righe 48-52) . . . . .                                                            | 383        |
| testZohClosedForm (righe 55-69) . . . . .                                                             | 383        |
| testZohMatchesOde45 (righe 71-83) . . . . .                                                           | 384        |
| testMassRowConsistency (righe 85-97) . . . . .                                                        | 385        |
| Cosa NON e' coperto (onesta') . . . . .                                                               | 385        |
| Possibili domande d'esame . . . . .                                                                   | 386        |
| <b>HM2_powered_descent/tests/descentDynamicsPerformanceTest.m</b>                                     | <b>387</b> |
| Ruolo del file nel progetto . . . . .                                                                 | 387        |
| Il framework: matlab.unittest.TestCase vs matlab.perftest.TestCase . . . . .                          | 387        |
| classdef e proprieta' (righe 1-20) . . . . .                                                          | 389        |
| addHm2ToPath (righe 22-27) . . . . .                                                                  | 390        |
| setupState (righe 29-35) . . . . .                                                                    | 390        |
| testOdeDescentEvaluation (righe 38-48) . . . . .                                                      | 391        |
| testRk4ZohPropagation (righe 50-61) . . . . .                                                         | 392        |
| testOde45ZohReplayInterval (righe 63-71) . . . . .                                                    | 393        |
| Il nesso con il design: perche' le RHS non hanno arguments – e perche' lti_zoh invece ce l'ha . . . . | 393        |
| Cosa significherebbe una regressione qui . . . . .                                                    | 395        |
| Limiti onesti del benchmark . . . . .                                                                 | 395        |
| Possibili domande d'esame . . . . .                                                                   | 396        |
| <b>HM3/tests/hm3PlantTest.m</b>                                                                       | <b>398</b> |
| Ruolo del file nel progetto . . . . .                                                                 | 398        |
| Setup di classe (righe 7-20) . . . . .                                                                | 398        |
| testParamsMatchTable1 (righe 23-31) . . . . .                                                         | 399        |
| testDynamicPressureSelfConsistent (righe 33-38) . . . . .                                             | 400        |
| testUncertaintyScalingAppliesToA6K1 (righe 40-47) . . . . .                                           | 400        |
| testRigidPlantDimensionsAndNames (righe 49-55) . . . . .                                              | 401        |
| testRigidAirframeUnstablePole (righe 57-63) . . . . .                                                 | 401        |
| testRigidMeasurementsEqualTrueStates (righe 65-70) . . . . .                                          | 402        |
| testFullPlantBendingMode (righe 72-80) . . . . .                                                      | 403        |
| testInsMeasurementsContaminatedByBending (righe 82-88) . . . . .                                      | 404        |
| testTrueMeasurementsBypassBending (righe 90-94) . . . . .                                             | 404        |
| testFullPlantRejectsUnknownMeas (righe 96-99) . . . . .                                               | 404        |
| Cosa NON e' coperto (limiti noti) . . . . .                                                           | 405        |
| Possibili domande d'esame . . . . .                                                                   | 405        |
| <b>HM3/tests/hm3FilterTest.m</b>                                                                      | <b>407</b> |
| Ruolo del file nel progetto . . . . .                                                                 | 407        |
| Setup di classe (righe 7-19) . . . . .                                                                | 407        |
| testTvcUnityDcGain (righe 22-26) . . . . .                                                            | 407        |
| testTvcOrderIsActuatorPlusPade (righe 28-32) . . . . .                                                | 408        |
| testTvcDelayPhaseAtLowFrequency (righe 34-43) . . . . .                                               | 408        |
| testNotchUnityGainFarFromCentre (righe 45-50) . . . . .                                               | 409        |

|                                                                                               |            |
|-----------------------------------------------------------------------------------------------|------------|
| testNotchDepthIsZetaRatio (righe 52-59) . . . . .                                             | 410        |
| testDefaultVariantIsNonMinimumPhase (righe 61-65) . . . . .                                   | 411        |
| testMinimumPhaseVariantHasLhpZeros (righe 67-70) . . . . .                                    | 412        |
| Test negativi sull'arguments block (righe 72-85) . . . . .                                    | 413        |
| Cosa NON e' coperto (limiti noti) . . . . .                                                   | 413        |
| Possibili domande d'esame . . . . .                                                           | 413        |
| <b>HM3/tests/hm3LoopTest.m</b>                                                                | <b>415</b> |
| Ruolo del file nel progetto . . . . .                                                         | 415        |
| Kref – i golden value del progetto (righe 7-13) . . . . .                                     | 415        |
| Setup (righe 15-35) . . . . .                                                                 | 416        |
| testGustProfileShape (righe 38-47) . . . . .                                                  | 417        |
| testStepProfileShape (righe 49-53) e testWindProfileRejectsUnknownProfile (righe 61-65) . . . | 417        |
| testDefaultGustAmplitudeFromDrywind (righe 55-59) . . . . .                                   | 418        |
| testRigidLoopMeetsMarginTargets (righe 67-76) . . . . .                                       | 418        |
| testDesignControllerMeetsTargets (righe 78-86) . . . . .                                      | 419        |
| testDesignControllerRestoresWarningState (righe 88-96) . . . . .                              | 420        |
| testBareFullModelIsBendingUnstable (righe 98-104) . . . . .                                   | 420        |
| testDeepNotchStabilisesFullModel (righe 106-114) . . . . .                                    | 420        |
| testGustResponseAngleOfAttackBudget (righe 116-128) . . . . .                                 | 421        |
| I golden value di questo file, in chiaro . . . . .                                            | 423        |
| Possibili domande d'esame . . . . .                                                           | 423        |
| <b>HM3/tests/hm3LoopPerformanceTest.m</b>                                                     | <b>425</b> |
| Ruolo del file nel progetto . . . . .                                                         | 425        |
| Kref – un golden value che era diventato STALE (righe 8-13) . . . . .                         | 426        |
| TestClassSetup e TestMethodSetup (righe 24-45) . . . . .                                      | 427        |
| testAssembleLoopRigid (righe 48-57) – livello <i>unit</i> . . . . .                           | 428        |
| testAssembleLoopFullChain (righe 59-67) – livello <i>unit</i> , caso pesante . . . . .        | 428        |
| testTunerCostEvaluation (righe 69-80) – livello <i>system</i> . . . . .                       | 429        |
| testGustResponseLsim (righe 82-89) – livello <i>system</i> . . . . .                          | 429        |
| testDesignControllerSearch (righe 91-98) – livello <i>workflow</i> . . . . .                  | 430        |
| Cosa questa suite NON fa (limiti noti) . . . . .                                              | 431        |
| Il commento “~400x” (righe 3-4) . . . . .                                                     | 431        |
| Possibili domande d'esame . . . . .                                                           | 431        |

# Walkthrough del codice – materiale di studio per l'orale (DCLV)

Documento **locale, per la preparazione dell'orale** – non fa parte della consegna. Un file `.md` per ogni file di codice dei quattro homework: ruolo del file nel progetto, poi spiegazione funzione-per-funzione / riga-per-riga, con la matematica derivata (non solo nominata) e riquadri “**Possibili domande d'esame**” sui punti delicati.

**Come usarlo.** Se il professore indica un file o una riga, apri il `.md` corrispondente qui sotto e cerca il sotto-titolo della funzione: ogni sezione riporta i **numeri di riga reali** del sorgente.

**Copertura:** 55 file (~8.700 righe di MATLAB) su HM0, HM1, HM2, HM3 e l'estensione LPV. Scritto contro il commit **b673640**. I numeri di riga invecchiano quando il codice cambia: se una sezione non torna, ricontrolla il sorgente prima di fidarti della pagina.

**Onesta?** Le pagine documentano il codice **com'e'**, non come dovrebbe essere. Dove il codice ha un bug, un pezzo stale, un commento che contraddice l'implementazione o una deviazione dalla teoria, la pagina lo dice. Sono esattamente i punti su cui un esaminatore mette il dito: meglio arrivarci sapendolo.

---

## 1 - HM0 – Falcon 9, ascesa 3-DoF (HM0\_falcon9\_ascent/)

Propagazione in avanti, niente ottimizzazione: e' il banco su cui si impara il modello.

| file                    | cosa spiega                                                                                                                             |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">main.m</a>  | EOM in coordinate sferiche derivate da zero, termini di trasporto vs Coriolis, spinta corretta per contropressione, gravity turn, max-Q |
| <a href="#">main2.m</a> | adimensionalizzazione, riparametrizzazione in <b>tau</b> a tre archi con chain rule, e l'event function inerte smascherata              |

## 2 - HM1 – ascesa planare, metodo indiretto (HM1/)

PMP, costati, funzione di switching, BVP risolto con shooting. La teoria pesante.

| file                         | cosa spiega                                                                                                                |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <a href="#">ode_burn.m</a>   | <b>il cuore:</b> Hamiltoniana, Eulero-Lagrange, primer vector, legge linear-tangent; perche' niente <b>arguments</b> block |
| <a href="#">main_task1.m</a> | shooting a 4 incognite, $H(0)=0$ algebrica, continuazione su $Q$ e $yf$ , tolleranze strette                               |

| file                                      | cosa spiega                                                                                                                            |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">main_task2.m</a>              | salita verticale imposta + arco propulso: stesso shooting 4x4, $H(0)=0$ generalizzata, e perche' NON serve una condizione di giunzione |
| <a href="#">main_task3.m</a>              | il coast balistico: shooting 5x5, switching function $S=0$ , corner conditions di Weierstrass-Erdmann, la radice spuria                |
| <a href="#">main_task4.m</a>              | staging: salto di massa, costati continui, Hamiltoniana <b>discontinua</b> , sweep sull'istante di separazione                         |
| <a href="#">validate_rocket_sled.m</a>    | validazione dello shooting contro il rocket sled a energia minima (soluzione analitica nota)                                           |
| <a href="#">validate_staging_corner.m</a> | corner condition come quinto residuo, invarianza di gauge di <code>lam_m</code> , confronto con lo sweep                               |

### 3 - HM2 – powered descent, metodi diretti (HM2\_powered\_descent/)

Dalla collocazione trapezoidale alla convessificazione lossless. La parte piu' moderna.

| file                                  | cosa spiega                                                                                                                                                   |
|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">ode_descent.m</a>         | RHS punto-materiale 2D con spinta ZOH: derivazione adimensionale, $V_c = V_{ref}/c$ , il kink di <code>abs(T)</code> in $T=0$                                 |
| <a href="#">ode_descent_uacc.m</a>    | RHS con <b>accelerazione</b> $u = T/m$ tenuta costante: omogeneita' in massa, il ponte verso il log-mass                                                      |
| <a href="#">rk4_zoh.m</a>             | RK4 a 4 stadi su intervallo ZOH: ordine quattro, riga di massa esatta, perche' passo fisso                                                                    |
| <a href="#">lti_zoh.m</a>             | ZOH <b>esatto</b> via <code>expm</code> : trucco di van Loan a blocchi, $A$ nilpotente, forma chiusa                                                          |
| <a href="#">main_task1.m</a>          | collocazione trapezoidale: decision vector, <b>defect constraints</b> , costo min-fuel, glide slope, <code>fmincon</code> /SQP, diagnostiche KKT              |
| <a href="#">main_task2.m</a>          | <b>il file piu' grosso della repo</b> (1420 righe): quattro trascrizioni ZOH – RK4 nonlineare, LTV+SCvx ( <code>fmincon</code> e YALMIP/ECOS), GFOLD log-mass |
| <a href="#">proto_gfold_logmass.m</a> | <b>convessificazione lossless</b> : cambio di variabile log-mass, dinamica esattamente LTI, slack <b>sigma</b> , tangente sul bound di spinta                 |

### 4 - HM3 – il modello del lanciatore a max-q (HM3/)

Pitch-plane, corpo rigido instabile, bending, attuatore. Le fondamenta di tutto il resto.

| file                                | cosa spiega                                                                                                                        |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">load_hw3_params.m</a>   | i parametri Greensite a $t = 72$ s: Table 1 vs dataset LPV, derivazione di $\mu_{alpha}/\mu_c$ , $qbar$ , scaling del Task 3       |
| <a href="#">build_plant_rigid.m</a> | EOM pitch-plane a 4 stati, il polo instabile <b>+sqrt(A6)</b> , il polo lento di deriva, <b>la convenzione di segno di alpha_w</b> |

| file                                     | cosa spiega                                                                                                              |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <a href="#">build_plant_full.m</a>       | il modo di bending forzato dal TVC e <b>la contaminazione INS</b> (Eq. 2): da dove escono i +29 dB di risonanza nel loop |
| <a href="#">build_tvc.m</a>              | attuatore TVC del 2o ordine + ritardo di 20 ms via <b>Pade</b> (all-pass a fase non minima)                              |
| <a href="#">build_notch_filter.m</a>     | notch / lead-lag (Eq. 4): <b>gain vs phase stabilization</b> , il prezzo del notch profondo, il detuning                 |
| <a href="#">assemble_loop.m</a>          | chiusura dell'anello PD: L per Nichols e T per la simulazione; <b>l'integratore di deriva</b> che avvelena i margini     |
| <a href="#">load_wind_profile.m</a>      | la raffica 1-coseno, la conversione $\alpha_w = v_w/V$ , perche' $\max-qbar$ e' la condizione di carico                  |
| <a href="#">simulate_gust_response.m</a> | risposta alla raffica, budget di $\alpha$ , indicatore di carico $qbar*\alpha$                                           |

## 5 - HM3 – margini e progetto: il cuore metodologico

Il lanciatore e' open-loop instabile, quindi l'anello e' **condizionalmente stabile** e un `margin()` secco mente. E' la storia che devi saper raccontare.

| file                                | cosa spiega                                                                                                                                                 |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">classify_margins.m</a>  | <b>perche' margin() e' privo di senso</b> qui, e i margini classificati per banda (aero / rigid / flex / delay) via <b>allmargin</b>                        |
| <a href="#">design_controller.m</a> | i guadagni canonici $Kp0 = 2*A6/K1$ , $Kd0 = \sqrt{A6}/K1$ <b>derivati</b> dai requisiti 6 dB / 30 gradi; auto-tuner <code>fminsearch</code> sul loop pieno |
| <a href="#">plot_nichols_lv.m</a>   | Nichols col punto critico a (-180, 0 dB) da $1+L=0$ (convenzione del corso), la curva che "viene dall'alto", marker per banda                               |
| <a href="#">make_pz_figures.m</a>   | mappe poli-zeri: polo aero instabile, bending quasi sull'asse, zero RHP del Pade                                                                            |

## 6 - HM3 – gli entry point (quelli che il professore fa girare)

| file                              | cosa spiega                                                                                                                      |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">main_task1.m</a>      | <b>Task 1:</b> corpo rigido, attuatore ideale, PD sul loop pieno verso 6 dB / 30 gradi, Nichols, risposta alla raffica           |
| <a href="#">main_task2.m</a>      | <b>Task 2:</b> TVC + ritardo + bending; il trade fra quattro filtri, il notch profondo, e la <b>ri-sintonizzazione</b> del PD    |
| <a href="#">main_task3.m</a>      | <b>Task 3:</b> robustezza +/-30% su $\mu_\alpha$ e $\mu_c$ , i quattro vertici, il margine aerodinamico come vincolo che stringe |
| <a href="#">main_montecarlo.m</a> | ( <i>extra</i> ) Monte Carlo N=1500 su cinque incertezze – e <b>la sua staleness rispetto ai Task 1-3, verificata</b>            |



## 7 - HM3 – track Simulink

| file                                       | cosa spiega                                                                                                  |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <a href="#">init_simulink_hm3.m</a>        | il ponte script -> Simulink: ogni parametro di blocco precalcolato dagli stessi moduli dei <b>main_task*</b> |
| <a href="#">run_simulink_closed_loop.m</a> | driver di validazione: modello vs baseline analitica degli script, e cosa significherebbe uno scostamento    |

## 8 - LTV\_\_FULL\_\_ASCENT – l'estensione LPV (oltre la traccia)

Il design congelato a max-q sollevato all'intera ascesa (0-140 s), con gain scheduling.

| file                                         | cosa spiega                                                                                                                                   |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">ode_lpv_ascent.m</a>             | RHS LTV rigida: coefficienti interpolati <b>dentro</b> il loop di <b>ode45</b> , PD chiuso in linea, <b>la fallacia del frozen-time</b>       |
| <a href="#">ode_lpv_flex.m</a>               | RHS LTV flessibile: bending con <b>omega(t)</b> , misure INS contaminate, notch variabile                                                     |
| <a href="#">build_hm3_full_ascent.m</a>      | authoring del <b>.slx</b> rigido da script: <b>il trucco dei coefficienti effettivi</b> (l'interpolazione non commuta con la moltiplicazione) |
| <a href="#">build_hm3_full_ascent_flex.m</a> | modello flessibile da script: notch che insegue <b>omega(t)</b> , e la discrepanza tabelle-vs-RHS                                             |
| <a href="#">main_full_ascent.m</a>           | propagazione dell'ascesa completa e il limite del design congelato lontano da max-q                                                           |
| <a href="#">main_flex.m</a>                  | il gemello flessibile: <b>omega_BM non e' una costante</b> , e cosa succede al notch fisso                                                    |
| <a href="#">main_q_scheduling.m</a>          | <b>schedulare in qbar invece che nel tempo</b> : perche' il tempo e' barare                                                                   |
| <a href="#">init_simulink_lpv.m</a>          | tabelle dei coefficienti effettivi, gain schedule, generatore di vento, contratto col base workspace                                          |
| <a href="#">run_full_ascent_simulink.m</a>   | overlay Simulink vs <b>ode45</b> : cosa deve coincidere <b>esattamente</b> per arrivare a 1e-7 rad                                            |
| <a href="#">run_flex_simulink.m</a>          | overlay flessibile a 13 stati e il mismatch fra tabelle e RHS dietro il residuo                                                               |

## 9 - Test e benchmark

| file                                         | cosa verifica                                                                                            |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <a href="#">HM0 falcon9AscentTest.m</a>      | il pattern run+harvest contro il <b>clear</b> iniziale degli script, e cosa resta <b>non testato</b>     |
| <a href="#">HM1 odeBurnTest.m</a>            | <b>ode_burn</b> : linear tangent law, costato di massa, limiti balistico e di Tsiolkovsky                |
| <a href="#">HM1 odeBurnPerformanceTest.m</a> | benchmark della RHS hot-loop; <b>giustifica l'assenza dell'arguments block</b>                           |
| <a href="#">HM2 odeDescentTest.m</a>         | la RHS del descent: derivata analitica vs differenze finite, segni fisici                                |
| <a href="#">HM2 rk4ZohTest.m</a>             | RK4: verifica dell' <b>ordine di convergenza</b> (dimezzo <b>dt</b> , l'errore cala di 16x)              |
| <a href="#">HM2 gfoldLogMassTest.m</a>       | il cambio di variabile log-mass: velocita' indipendente dalla massa, <b>z</b> lineare, <b>ZOH</b> esatta |

| file                                                 | cosa verifica                                                                                                                  |
|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">HM2 descentDynamicsPerformanceTest.m</a> | <code>matlab.perftest</code> sull'hot loop – e perche' chiamarlo <i>test</i> di regressione e' generoso                        |
| <a href="#">HM3 hm3PlantTest.m</a>                   | congela la <b>fisica</b> : il polo a <code>+sqrt(A6)</code> , il bending a <code>omega_BM</code> , <code>zeta_BM</code>        |
| <a href="#">HM3 hm3FilterTest.m</a>                  | congela i blocchi di compensazione: profondita' del notch, fase del Pade                                                       |
| <a href="#">HM3 hm3LoopTest.m</a>                    | congela <b>le conclusioni</b> : i margini del Task 1, l'instabilita' senza filtro, il notch che salva l'anello (golden values) |
| <a href="#">HM3 hm3LoopPerformanceTest.m</a>         | il costo dell'assemblaggio del loop: hot path dell'auto-tuner e del Monte Carlo                                                |

## Consiglio di studio

Non leggerli in ordine. I file ad alto rendimento, nell'ordine in cui conviene affrontarli:

1. [hm3\\_build\\_plant\\_rigid.md](#) – da qui parte tutto. Se sai derivare  $\theta_{ddot} = A6 \cdot \theta + (A6/V) \cdot \dot{z} + K1 \cdot \delta$  e spiegare **perche'  $A6 > 0$  significa instabilita' statica** (centro di pressione davanti al baricentro) e perche' il polo cade esattamente in `+sqrt(A6)`, hai in mano meta' di HM3.
2. [hm3\\_classify\\_margins.md](#) – il punto metodologico su cui l'homework si gioca. Il velivolo e' open-loop instabile, quindi l'anello e' **condizionalmente stabile**: la curva di Nichols deve passare *fra* i due punti critici e un singolo numero di `margin()` non significa niente. Saper dire questo, e perche' i margini vanno classificati per banda, e' la differenza fra 28 e 30.
3. [hm3\\_build\\_notch\\_filter.md](#) + [hm3\\_main\\_task2.md](#) – **gain stabilization vs phase stabilization** del modo di bending, e il prezzo che si paga per il notch profondo (devi conoscere `omega_BM` con precisione, e il ritardo di fase che aggiunge ti mangia il phase margin rigido, tanto da costringere a ri-sintonizzare il PD).
4. [hm2\\_proto\\_gfold\\_logmass.md](#) – la **convessificazione lossless**. Il cambio di variabile  $z = \ln(m)$ ,  $u = T/m$ ,  $\sigma = \text{abs}(T)/m$  che rende la dinamica esattamente LTI, e il rilassamento  $\text{abs}(u) \leq \sigma$  che all'ottimo e' attivo. E' il pezzo di teoria piu' elegante di tutta la repo, ed e' quello su cui e' piu' facile fare bella figura.
5. [hm1\\_ode\\_burn.md](#) – Hamiltoniana, Eulero-Lagrange, primer vector. Poche righe di codice, tutta la teoria del controllo ottimo indiretto.

# HM0\_\_falcon9\_\_ascent/main.m

## Ruolo del file nel progetto

`main.m` e' l'entry point canonico di HM0 e l'unico file della repo che implementa per intero la simulazione richiesta dalla traccia (LVdynamics\_Homework\_0.pdf): propagare in avanti l'ascesa del primo stadio del Falcon 9 dal Kennedy Space Center fino al MECO, con un modello 3-DoF a punto materiale, atmosfera esponenziale, Terra sferica rotante e gravita' newtoniana.

E' importante inquadrare bene cosa NON e': **non c'e' ottimizzazione, non c'e' controllo, non c'e' un problema ai limiti**. Il profilo di assetto e' *assegnato* (verticale -> pitchover -> gravity turn) e il codice si limita a integrare le equazioni del moto con `ode45` a partire dalle condizioni iniziali. Tutta la "intelligenza" del file sta (a) nella derivazione corretta delle equazioni in coordinate sferiche con velocita' in terna UEN, (b) nella modellazione delle forze (spinta corretta per la contropressione, drag, gravita'), (c) nel post-processing che ricostruisce Mach, pressione dinamica, angoli e ground track.

Il file e' **autocontenuto**: costanti, condizioni iniziali, integrazione, post-processing, otto figure e l'export PNG stanno tutti qui; le equazioni del moto sono una *local function* `eom` in coda al file (righe 375-466). Non chiama nessuna funzione della repo. E' chiamato da `tests/falcon9AscentTest.m` tramite il pattern `run()` + `harvest` del workspace, e le sue figure sono quelle referenziate dal README di HM0.

`main2.m` risolve lo stesso identico problema fisico in forma adimensionale con una parametrizzazione a tre archi: e' una variante preparatoria per gli homework di ottimizzazione, non un'alternativa a `main.m`.

---

## Intestazione e clear (righe 1-6)

```
% main.m - Falcon 9 First Stage Trajectory Simulation
% 3-DoF point-mass model in spherical coordinates
% Velocity in Up-East-North (UEN) frame
```

```
clear; close all; clc;
```

- Righe 1-4: la testata dichiara le due scelte di modellazione che governano tutto il resto: **stato in coordinate sferiche** (`r`, `theta`, `phi`) e **velocita' proiettata in terna UEN** (Up-East-North). Sono due scelte indipendenti: si potrebbe benissimo avere posizione sferica e velocita' in componenti cartesiane inerziali. La combinazione scelta e' quella che rende le equazioni "leggibili" per un lanciatore (`u` = rateo di salita, `v` = velocita' verso Est, `w` = verso Nord).
- Riga 6: `clear` all'inizio. E' un dettaglio innocuo per l'uso interattivo, ma ha una conseguenza pesante sulla testabilita': un test che chiami `run('main.m')` dal proprio workspace se lo vede cancellare, `testCase` incluso. E' esattamente il motivo per cui `falcon9AscentTest.m` deve isolare la `run()` dentro una funzione locale usa-e-getta (vedi `hm0_test_falcon9Ascent.md`).

## CONSTANTS AND VEHICLE PARAMETERS (righe 8-54)

Blocco puramente dichiarativo, ma diversi valori meritano una difesa all'orale.

- Righe 13-14:  $\mu = 3.986004418e14 \text{ m}^3/\text{s}^2$  e  $RE = 6378137 \text{ m}$ . Sono i valori WGS-84.  $RE$  e' il raggio **equatoriale**: usarlo come raggio di una Terra sferica sovrastima leggermente il raggio locale al KSC (lat 28.57 deg): il raggio geocentrico dell'ellissoide WGS-84 a quella latitudine vale ~6373 km, quindi l'errore e' di ~5 km su 6378, irrilevante rispetto alle altre approssimazioni del modello.
- Righe 17-20: modello atmosferico.  $\rho_0 = 1.225 \text{ kg/m}^3$ ,  $p_0 = 101325 \text{ Pa}$ ,  $H_{scale} = 8000 \text{ m}$ ,  $T_{amb} = 288.15 \text{ K}$ . **Onesta': queste quattro costanti non sono mutuamente consistenti.** Per un'atmosfera isoterma in equilibrio idrostatico con gas ideale vale  $H = R*T/g$ , che con  $R = 287.058$  e  $T = 288.15$  da'  $H = 8435 \text{ m}$ , non 8000 m.  $H_{scale} = 8000$  e' un valore ingegneristico "arrotondato" molto usato in letteratura. La conseguenza pratica e' che l'atmosfera decade un po' piu' in fretta del previsto dal gas ideale isoterma; l'effetto e' un max-Q leggermente anticipato e piu' basso. Va detto, non nascosto.
- Riga 24:  $\omega_E = 2\pi/T_{sid}$  con  $T_{sid} = 86136 \text{ s}$  (riga 23). **Il giorno sidereo standard e' 86164.09 s**, non 86136. La differenza (28 s) porta a  $\omega_E = 7.29449e-5 \text{ rad/s}$  invece di  $7.29212e-5$ : un errore relativo di  $3.2e-4$ . E' del tutto trascurabile sui risultati (la velocita' iniziale Est cambia di ~0.1 m/s su 408.6), ma e' un errore reale e conviene saperlo ammettere. Il valore e' ripetuto identico in `main2.m` riga 52.
- Riga 29:  $a_{sound} = \sqrt{\gamma_{air} * R_{gas} * T_{amb}} = 340.30 \text{ m/s}$ , calcolato **una volta sola a  $T = 288.15 \text{ K}$**  e poi usato a tutte le quote. Questa e' una conseguenza diretta dell'ipotesi di atmosfera isoterma: se  $T$  e' costante, anche  $a$  lo e'. E' una semplificazione forte: nell'atmosfera standard reale  $a$  scende a ~295 m/s a 11 km, quindi **questo modello sottostima il numero di Mach in quota** (di circa il 13-15% attorno alla tropopausa). L'attraversamento di Mach 1 riportato ( $t = 61.8 \text{ s}$ ,  $h = 8.04 \text{ km}$ ) e' quindi un po' *tardivo* rispetto alla realta'.
- Righe 32-38: primo stadio.  $T_{vac1} = 8227 \text{ kN}$ ,  $c_{vac1} = 3244 \text{ m/s}$ ,  $A_{ex1} = 11.039 \text{ m}^2$ ,  $t_{b1} = 162 \text{ s}$ . La portata di massa e' **derivata**, non assegnata:  $\dot{Q}_{dot1} = T_{vac1}/c_{vac1} = 2536.07 \text{ kg/s}$  (riga 38). Questa e' la definizione di velocita' di efflusso *efficace* in vuoto:  $c_{vac} = T_{vac}/\dot{m}$ , cioe'  $c_{vac}$  ingloba gia' il termine di spinta di pressione  $p_e A_e$ . Se si usasse la velocita' di efflusso "vera"  $u_e$  si otterrebbe una portata diversa.
- **Onesta' sul bilancio di massa:**  $\dot{Q}_{dot1} * t_{b1} = 410\,843 \text{ kg}$ , mentre il propellente dichiarato e'  $mp1 = 410900 \text{ kg}$  (riga 33). I due valori differiscono di ~57 kg. La terna ( $T_{vac}$ ,  $c_{vac}$ ,  $t_b$ ) non e' perfettamente consistente con  $mp1$ ; il codice non se ne accorge perche'  $mp1$  entra solo nella massa iniziale (riga 69) e mai nella dinamica, che usa esclusivamente  $\dot{Q}_{dot1}$ . In pratica il veicolo arriva a MECO con 57 kg di propellente non bruciato. Irrilevante, ma se all'orale viene chiesto "perche'  $m(t_b)$  non e' esattamente  $m_0 - mp1$ ?", questa e' la risposta.
- Righe 45-46:  $CD = 0.329$  **costante** e  $S_{ref} = 10.52 \text{ m}^2$ . Nessuna dipendenza da Mach. Nella realta' il  $C_D$  di un lanciatore raddoppia abbondantemente nel transonico. Questo modello quindi **sottostima le perdite di drag proprio nella regione di max-Q**, che e' dove contano di piu'.
- Righe 53-54:  $t_{end1} = 5 \text{ s}$ ,  $t_{end2} = 15 \text{ s}$ . Sono i due switch di fase, deterministici e noti a priori: e' il motivo per cui non serve una event function nell'integratore (vedi la sezione sull'integrazione).

**Possibile domanda d'esame** – Perche' la portata  $\dot{Q}_{dot}$  si calcola come  $T_{vac}/c_{vac}$  e non come  $T_{vac}/(I_{sp}g_0)$  o con la velocita' di efflusso vera? *Risposta:* Perche'  $c_{vac}$  e' definita come velocita' di efflusso *efficace* in vuoto, cioe' l'impulso specifico in vuoto moltiplicato per  $g_0$ : per definizione  $T_{vac} = \dot{m} * c_{vac}$ . Le tre scritture sono la stessa cosa. Usare invece la velocita' di efflusso *fisica*  $u_e$  sarebbe sbagliato, perche'  $T = \dot{m}u_e + (p_e - p_{amb})A_e$ :  $u_e$  non ingloba il termine di pressione,  $c_{vac}$  si'. Il codice e' coerente: usa  $c_{vac}$  per la portata e sottrae separatamente  $p_{amb}A_{ex}$  per la correzione di contropressione.

## INITIAL CONDITIONS - Launch from Kennedy Space Center 56-71)

(righe

```
r0      = RE;                % Radius [m]
theta0  = lon0;              % Right ascension [rad]
phi0    = lat0;              % Declination [rad]
u0      = 0;                 % Up velocity [m/s]
v0      = omegaE * RE * cos(lat0); % East velocity [m/s]
w0      = 0;                 % North velocity [m/s]
```

- Righe 60-61: KSC, lat 28.573469 deg N, lon -80.651070 deg.
- Riga 64: `theta0 = lon0`. Questo e' il punto piu' sottile di tutto il file. `theta` e' etichettata “**right ascension**”, cioe' un angolo **inerziale**, non la longitudine terrestre. Al tempo  $t = 0$  le due coincidono per convenzione (si sceglie l'origine dell'ascensione retta coincidente con il meridiano di Greenwich a  $t = 0$ ), ma da li' in poi divergono: la Terra ruota sotto il veicolo. La prova che `theta` sia davvero inerziale sta a riga 187, dove la longitudine di ground track viene recuperata come `theta - omegaE*t` - cioe' sottraendo esplicitamente la rotazione terrestre.
- Riga 67: `v0 = omegaE * RE * cos(lat0) = 408.6 m/s`. Il veicolo, fermo sulla rampa, ha gia' una velocita' **inerziale** verso Est pari a quella del punto di lancio trascinato dalla rotazione terrestre. Questa e' l'unica porta d'ingresso della rotazione terrestre nello *stato* del sistema: nelle equazioni non ci sara' nessun termine di Coriolis esplicito (vedi la derivazione piu' sotto). Conseguenza fisica immediata: nel gravity turn la spinta si allinea alla velocita' *relativa*, quindi il veicolo “eredita” e sfrutta questi 408.6 m/s, che e' il regalo del lancio verso Est da bassa latitudine.
- Riga 69: `m0 = mdry1 + mp1 + mdry2 + mp2 + mfair + mpay = 569 100 kg`. Il primo stadio porta in volo *tutta* la pila: secondo stadio pieno, fairing e payload. Corretto, perche' fino al MECO nulla e' stato sganciato.

Un numero utile da avere in tasca per l'orale, ricavabile dalle righe 60-69 piu' la `eom`:

$$\begin{aligned} du(0) &= v0^2/RE - \mu/RE^2 + T\_SL/m0 \\ &= 0.026 - 9.798 + 12.491 = 2.72 \text{ m/s}^2 \end{aligned}$$

con  $T\_SL = T_{vac} - p0 \cdot A_{ex} = 7.108 \text{ MN}$ . Il rapporto spinta/peso al decollo e' **1.275**: il veicolo stacca, ma senza abbondanza. Il termine centrifugo ( $0.026 \text{ m/s}^2$ ) e' il “regalo” della rotazione terrestre ed e' visibilmente trascurabile al lift-off.

**Possibile domanda d'esame** – Se `theta` e' inerziale, perche' la si inizializza con la longitudine geografica del KSC? *Risposta:* Perche' e' una scelta di origine dell'asse x inerziale: si fissa il frame inerziale in modo che a  $t = 0$  coincida con l'ECEF. Da quel momento `theta(t)` continua a essere ascensione retta e la longitudine terrestre si recupera come `theta(t) - omegaE*t`. Se si volesse un frame inerziale allineato all'equinozio vernale basterebbe aggiungere il GMST iniziale: la dinamica non cambierebbe di una virgola, cambierebbe solo l'offset del ground track.

---

## PARAMETER STRUCTURE (righe 73-90)

- Righe 77-90: tutti i parametri fisici confluiscono in una `struct par` passata alla `eom` per riferimento. Nessun uso di variabili globali, nessun `evalin/assignin`: la `eom` e' una funzione pura di (`t`, `y`, `par`).
- Nota che la struct **non** contiene `a_sound`, `Tamb`, `mp1`, `tb1`: queste servono solo prima o dopo l'integrazione, non dentro la dinamica. E' una separazione pulita fra “cosa serve al modello” e “cosa serve al post-processing”.
- `par.t1` e `par.t2` (righe 89-90) sono i tempi di switch: la logica di fase e' quindi dentro la `eom` e non fuori.

## NUMERICAL INTEGRATION (ode45) (righe 92-98)

```
opts = odeset('RelTol', 1e-10, 'AbsTol', 1e-12, 'MaxStep', 1);
tspan = [0, tb1];
[t, Y] = ode45(@eom(t, y, par), tspan, y0, opts);
```

- Riga 98: `ode45` = Dormand-Prince esplicito 5(4) a passo adattivo. Scelta corretta: il problema e' **non-stiff** (le scale temporali sono tutte dell'ordine dei secondi, non ci sono modi veloci) e il RHS e' economico da valutare, quindi un metodo esplicito e' l'ottimo.
- Riga 96 (`odeset`): `RelTol = 1e-10`, `AbsTol = 1e-12`: tolleranze molto piu' strette del necessario per una semplice propagazione (per un grafico basterebbe `1e-6`). Sono la convenzione della repo (`CLAUDE.md`) ereditata dal lavoro di shooting/indiretto degli homework successivi, dove l'accuratezza della propagazione entra direttamente nel residuo del BVP. Qui il costo e' solo tempo di calcolo. Il vantaggio collaterale e' che rende sensato il test di bookkeeping del propellente a `RelTol 1e-8` (vedi `hm0_test_falcon9Ascent.md`).
- `MaxStep = 1 s`: **non** e' una tolleranza, e' un vincolo geometrico sul passo. Serve a garantire che la griglia di output sia abbastanza fitta da (a) risolvere bene il picco di max-Q e l'attraversamento di Mach 1, che vengono localizzati in post-processing con `max/find sui campioni`, e (b) impedire che un singolo passo salti a pie' pari uno degli intervalli di fase – in particolare la fase 2 dura solo 10 s.
- Riga 97: `tspan = [0, tb1]`. Il MECO e' **imposto per tempo**, non rilevato: si integra fino a  $t = 162$  s e basta. Non ci sono event function in `main.m`.

**Perche' nessuna event function?** Perche' i due switch di fase ( $t = 5$  s,  $t = 15$  s) sono *deterministici e noti a priori*: non c'e' nulla da localizzare. Le event function servono quando l'istante dell'evento e' esso stesso incognito (impatto al suolo, esaurimento propellente, attraversamento di una quota).

**Cosa succede allora agli switch?** Va guardato in faccia: - A  $t = 5$  s (fine ascesa verticale) il RHS e' in realta' **continuo**: la legge di pitchover a riga 435 vale  $gT = 90 - 0.05*(t - 5)$ , che per  $t = 5$  da' esattamente 90 deg, quindi  $T_u = T_{mag}*\sin(90) = T_{mag}$ ,  $T_v = 0$  – cioe' la stessa cosa della fase 1. Lo switch e' invisibile all'integratore. - A  $t = 15$  s (fine pitchover) il RHS e' **discontinuo**: la direzione di spinta passa di colpo da elevazione 89.5 deg / azimuth Est alla direzione della velocita' relativa, che in quell'istante ha un'elevazione simile ma non identica. `ode45` non sa che c'e' una discontinuita': semplicemente il suo stimatore d'errore locale esplode sul passo che la attraversa, il passo viene rifiutato e dimezzato ripetutamente finche' non rientra in tolleranza. Il risultato e' corretto ma pagato con qualche decina di passi buttati. La cura "pulita" sarebbe spezzare l'integrazione in due chiamate `ode45` con restart a  $t = 15$  s.

**Possibile domanda d'esame** – `ode45` e' a passo adattivo: perche' allora imporre `MaxStep = 1`?  
*Risposta*: Perche' l'adattivita' controlla l'errore, non la *risoluzione dell'output*. Con tolleranze cosi' strette e una dinamica dolce, `ode45` potrebbe comunque scegliere passi lunghi in tratti "facili", producendo una griglia rada. Ma max-Q e Mach 1 non vengono localizzati con event function: sono estratti con `max(qdyn)` e `find(Mach>=1,1)` *sui campioni*. Una griglia rada sposterebbe l'istante riportato di max-Q anche di parecchi secondi. `MaxStep = 1` e' quindi un vincolo di *post-processing*, non di accuratezza.

---

## POST-PROCESSING (righe 100-188)

### Estrazione e quantita' derivate (righe 104-132)

- Riga 105-111: unpacking delle sette colonne di Y.
- Riga 114:  $h = r - RE$ . Quota su Terra sferica.
- Riga 117:  $V_{mag}$  = modulo della velocita' **inerziale**.
- Riga 120-123: velocita' **relativa** all'atmosfera. Qui c'e' della fisica:

```
urel = u
vrel = v - omegaE*r*cos(phi)
```

wrel = w

L'atmosfera e' assunta **co-rotante rigidamente con la Terra** (nessun vento). Un punto solidale alla Terra a raggio  $r$  e latitudine  $\phi$  ha velocita' inerziale puramente verso Est di modulo  $\omega E r \cos(\phi)$  (e' la velocita' del moto circolare attorno all'asse polare, il cui raggio e'  $r \cos(\phi)$ ). Sottraendola dalla componente Est della velocita' inerziale si ottiene la velocita' rispetto all'aria. Le componenti Up e Nord **non** sono toccate, perche'  $\omega \times r$  non ha componenti in quelle direzioni. Verifica di coerenza: a  $t = 0$ ,  $v = \omega E R E \cos(\text{lat0})$  e  $r = R E$ ,  $\phi = \text{lat0}$ , quindi  $v_{\text{rel}}(0) = 0$  **esattamente**. Il razzo fermo sulla rampa ha velocita' relativa nulla. E' proprio questo che rende necessarie le guardie `if Vrel > 1e-10` nella `eom`.

- Riga 126: `rho_traj = rho0*exp(-h/Hscale)`. Atmosfera esponenziale isoterma. (In post-processing la densita' si chiama `rho_traj`; il nome `rho` esiste solo come variabile locale dentro `eom`, riga 399.)
- Riga 129: `qdyn = 0.5*rho_traj.*Vrel.^2`. **Con la velocita' relativa, non con quella inerziale.** E' l'unica scelta fisicamente sensata: la pressione dinamica e' quella che il veicolo *sente*, cioe' rispetto all'aria. Usare `Vmag` gonfierebbe `q` al lift-off (dove `Vmag = 408 m/s` ma il razzo e' fermo rispetto all'aria) – errore da matita blu.
- Riga 132: `Mach = Vrel / a_sound`, con `a_sound` costante (vedi sopra).

**La struttura di max-Q.**  $q = 0.5 \rho_0 \exp(-h/H) V_{\text{rel}}^2$  e' il prodotto di un fattore che crolla\* esponenzialmente con la quota e di uno che *cresce* col quadrato della velocita'. Derivando rispetto al tempo e ponendo a zero:

$$\frac{d}{dt}[\ln q] = -(1/H) \frac{dh}{dt} + 2 \cdot (1/V_{\text{rel}}) \frac{dV_{\text{rel}}}{dt} = 0$$
$$\Rightarrow \frac{dV_{\text{rel}}}{dt} / V_{\text{rel}} = u / (2 \cdot H)$$

cioe' il max-Q cade nell'istante in cui il **rateo relativo di accelerazione** uguaglia **meta' del rateo relativo di caduta della densita'**. Prima di quel punto vince  $V^2$ , dopo vince  $\exp(-h/H)$ . Nella run nominale (README) esce  $t = 74.9$  s,  $h = 12.78$  km,  $q = 29.5$  kPa.

**Perche' Mach 1 (61.8 s) viene PRIMA di max-Q (74.9 s)?** Sono due condizioni diverse: Mach 1 e' una soglia su  $V_{\text{rel}}$  (una velocita' fissata, 340.3 m/s), max-Q e' un massimo di un prodotto. Il veicolo raggiunge 340 m/s molto prima che il prodotto  $\rho V^2$  giri. *Non c'e' nessun legame causale fra i due eventi – capita in quest'ordine per qualunque lanciatore con questo profilo, ed e' il motivo per cui il transonico e il max-Q sono due criticita' distinte\** nel progetto strutturale.

## Angoli di spinta e aerodinamico (righe 134-155)

```
Vh = sqrt(vrel(k)^2 + wrel(k)^2);
if Vrel(k) > 1e-6
    gammaA(k) = atan2d(urel(k), Vh);
else
    gammaA(k) = 90;
end
```

- Righe 140-145: `gammaA` e' l'elevazione della velocita' relativa sul piano orizzontale locale: `atan2d`(componente Up, modulo della componente orizzontale). 90 deg = salita verticale, 0 deg = volo orizzontale. L'uso di `atan2d` invece di `atan2d(u/Vh)` e' corretto perche' gestisce  $V_h \rightarrow 0$  (salita verticale pura) senza divisione per zero.
- Righe 141-145: la guardia sul caso  $V_{\text{rel}} \sim 0$  (che, come visto, e' esattamente il caso a  $t = 0$ ) assegna convenzionalmente 90 deg. E' una convenzione, non un risultato: a velocita' relativa nulla l'angolo di traiettoria e' indefinito.
- Righe 148-154: `gammaT` viene **ricostruito replicando la logica di fase della eom**, non estratto dall'integrazione.

**Attenzione, punto onesto e importante.** Alla riga 153, in fase 3, `gammaT(k) = gammaA(k)` – cioe' l'angolo di spinta e' *assegnato uguale* a quello aerodinamico. Nella Figura 7 le due curve coincidono perfettamente dopo  $t = 15$  s, ma **questo non e' una verifica del gravity turn: e' una tautologia**. Le curve coincidono perche' il codice le ha rese uguali per costruzione in post-processing. E' vero che la `eom` implementa davvero la spinta lungo

Vrel in fase 3 (righe 443-449), quindi il grafico non *mente*; semplicemente non dimostra nulla. Se si volesse una vera verifica bisognerebbe ricalcolare l'elevazione del vettore spinta dalla `eom` in modo indipendente e confrontarla.

C'è un secondo aspetto notevole: fra  $t = 0$  e  $t = 15$  s, `gammaT != gammaA`. Il veicolo vola quindi con un **angolo d'attacco non nullo** durante l'ascesa verticale e il pitchover (banale al decollo, dove  $V_{rel} \sim 0$  e  $\alpha$  è indefinito, ma reale durante il pitchover). Il modello però **non produce nessuna forza di portanza o laterale**: c'è solo drag lungo  $-V_{rel}$ . È una semplificazione consapevole del 3-DoF a punto materiale. Il “zero-lift gravity turn” è letteralmente a  $\alpha = 0$  solo in fase 3.

## Eventi di missione (righe 157-161)

- Righe 158-159: `ip1`, `ip2` = indici dei campioni *piu' vicini* a  $t = 5$  e  $t = 15$  s. Non esatti: è proprio per questo che `MaxStep = 1` serve.
- Riga 160: `im1 = find(Mach >= 1, 1, 'first')` – **primo campione** oltre Mach 1, non interpolazione. L'istante riportato ha quindi un'incertezza pari al passo locale ( $\leq 1$  s). Restituisce `[]` se il veicolo non supera Mach 1: da qui tutte le guardie `if ~isempty(im1)` nei plot.
- Riga 161: `[qmax, imQ] = max(qdyn)` – massimo **sui campioni**, non del polinomio interpolante. Il valore di picco è quindi leggermente sottostimato (il vero massimo cade fra due campioni). Con `MaxStep = 1` s è una curva  $q(t)$  molto piatta attorno al picco, l'errore è minimo, ma esiste.

## Da ECI a ENU locale (righe 163-184)

- Righe 165-167: coordinate cartesiane inerziali dalla terna sferica:

```
X = r*cos(phi)*cos(theta)
Y = r*cos(phi)*sin(theta)
Z = r*sin(phi)
```

Notare che con `phi` = declinazione (non colatitudine) compare `cos(phi)` nelle componenti equatoriali e `sin(phi)` in quella polare. Se `phi` fosse la colatitudine sarebbero scambiati: è l'errore di segno/funzione *piu' comune* in questi codici.

- Righe 175-177: la matrice `R_enu` ha come **righe** i versori East, North, Up del sito di lancio espressi in ECI:

```
e_East = [-sin(theta0),      cos(theta0),      0      ]
e_North = [-sin(phi0)*cos(theta0), -sin(phi0)*sin(theta0), cos(phi0) ]
e_Up    = [ cos(phi0)*cos(theta0),  cos(phi0)*sin(theta0), sin(phi0) ]
```

Moltiplicare per `R_enu` proietta un vettore ECI sulla terna locale: è esattamente la definizione di matrice di rotazione costruita per righe di versori. Si verifica facilmente che `e_East x e_North = e_Up` (terna destrorsa).

- Righe 179-180: si sottrae la posizione del sito di lancio **prima** di ruotare – cioè è una trasformazione affine (traslazione + rotazione), non una semplice rotazione.
- **Sottigliezza**: `R_enu` è costruita con `theta0`, cioè con la terna locale **congelata all'istante del lancio nel frame inerziale**. Il sito di lancio, però, *ruota* con la Terra. Quindi la Figura 1 non è “la traiettoria vista dal sito di lancio” in senso ECEF: è la traiettoria inerziale proiettata su una terna inerziale fissa che *coincideva* con la terna locale a  $t = 0$ . Su 162 s la Terra ruota di  $\sim 0.68$  deg, quindi la differenza è piccola ma non nulla. Il codice non esplicita questa scelta.

## Ground track (righe 186-188)

- Riga 187: `lon_ground = rad2deg(theta - omegaE*t)`. Qui invece la rotazione terrestre è tolta correttamente: si passa da ascensione retta (inerziale) a longitudine (ECEF). E' la conferma definitiva che `theta` è inerziale.
- Riga 188: la latitudine `phi` non ha bisogno di correzioni (la rotazione è attorno all'asse polare, non cambia la latitudine).



## CONSOLE SUMMARY (righe 190-212)

Blocco di `fprintf`. Non fa fisica, ma i suoi output sono gli stessi numeri riportati nel README e su cui si appoggiano le asserzioni “di banda” del test (quota finale fra 50 e 120 km). Riga 207: la guardia `if ~isempty(im1)` evita di stampare Mach 1 se non c’è stato.

---

## PLOTS (righe 214-350)

Otto figure. Meritano un commento solo tre cose.

- Righe 219-222: la palette dei quattro eventi (fine verticale / fine pitchover / Mach 1 / max-Q) è definita una volta e riusata su tutte le figure: gli eventi hanno sempre lo stesso colore e lo stesso marker in ogni grafico. È una buona pratica di leggibilità e la traccia lo richiede esplicitamente (“mark the key mission events”).
  - `HandleVisibility','off'` sulle `xline` degli switch di fase: le linee tratteggiate verticali compaiono nel grafico ma non nella legenda. Senza questo, ogni legenda avrebbe due voci inutili.
  - Figura 7 (righe 326-333): come discusso sopra, la sovrapposizione di  $\gamma_T$  e  $\gamma_A$  dopo  $t = 15$  s è costruita, non misurata.
- 

## EXPORT FIGURES TO figures/ (righe 352-369)

```
slugify = @(s) lower(regexprep(s, '[^a-zA-Z0-9]+', '_'));
fig_handles = findobj(groot, 'Type', 'figure');
for kk = 1:numel(fig_handles)
    try
        theme(fig_handles(kk), 'light');
        drawnow;
    catch
        fig_handles(kk).Color = 'w';
    end
    ...
end
```

- Riga 355: la cartella di destinazione è risolta da `mfilename('fullpath')`, non dal working directory: lo script si può lanciare da qualunque cartella.
  - Riga 358: `slugify` trasforma il Name della figura in un nome file (`'3D Trajectory' -> 3d_trajectory.png`). È il motivo per cui i nomi delle figure alle righe 225, 249, ... non sono cosmetici: **sono i nomi dei file** e quindi i link del README.
  - Righe 361-366: `theme(fig, 'light')` forza il tema chiaro anche se il desktop MATLAB è in dark mode – altrimenti i PNG uscirebbero con sfondo nero e finirebbero così nel report LaTeX. Il `try/catch` degrada su `fig.Color = 'w'` per versioni pre-R2025a, dove `theme` non esiste.
  - **Onesta**: `findobj(groot, 'Type','figure')` esporta **tutte** le figure aperte nella sessione, non solo le otto di questo script. In pratica non è un problema perché la riga 6 fa `close all`, ma è una dipendenza implicita dallo stato della sessione. Se qualcuno rimuovesse il `close all`, l’export sputerebbe fuori anche figure estranee.
- 

## eom – equazioni del moto (righe 375-466)

È il cuore del file. La firma:

```
function dydt = eom(t, y, par)
% State vector: y = [r, theta, phi, u, v, w, m]
```

- Riga 375: funzione pura, chiamata  $\sim 10^{4-105}$  volte da ode45. Coerentemente con la convenzione della repo (CLAUDE.md), **non ha blocco arguments**: e' una hot loop, la validazione costerebbe piu' del calcolo. Non e' una svista.

## Derivazione delle equazioni (righe 452-460)

Questo e' il punto su cui vale la pena spendere tempo, perche' e' *l'unica vera matematica del file* e all'orale e' la domanda quasi certa.

**Setup.** La posizione e'  $R = r * e_r$ , dove la terna locale (in **frame inerziale**, con **theta** = ascensione retta e **phi** = declinazione) e':

```
e_r = ( cos(phi)*cos(theta), cos(phi)*sin(theta), sin(phi) ) [Up]
e_e = (-sin(theta), cos(theta), 0 ) [East]
e_n = (-sin(phi)*cos(theta), -sin(phi)*sin(theta), cos(phi) ) [North]
```

Si verifica che  $e_e \times e_n = e_r$ : la terna (East, North, Up) e' destrorsa.

**Cinematica (righe 452-455).** Derivando  $R = r * e_r$  nel tempo e usando

$$de_r/dt = \cos(\phi) * \theta_{dot} * e_e + \phi_{dot} * e_n$$

si ottiene

$$V = r_{dot} * e_r + r * \cos(\phi) * \theta_{dot} * e_e + r * \phi_{dot} * e_n$$

Confrontando con  $V = u * e_r + v * e_e + w * e_n$  si legge subito l'identificazione delle componenti UEN, e quindi le tre equazioni cinematiche del codice:

```
u = r_dot -> dr = u
v = r*cos(phi)*theta_dot -> dtheta = v / (r*cos(phi))
w = r*phi_dot -> dphi = w / r
```

Sono **esattamente** le righe 453-455. Notare che  $d\theta$  diverge ai poli ( $\cos(\phi) \rightarrow 0$ ): e' una singolarita' **della parametrizzazione**, non della fisica. Per un lancio da 28.57 deg non e' un problema, ma per una missione polare lo sarebbe (e sarebbe uno degli argomenti per passare a una formulazione cartesiana o a quaternioni).

**Dinamica (righe 457-460).** Qui viene il bello. Derivando  $V = u * e_r + v * e_e + w * e_n$  bisogna derivare **anche i versori**, che ruotano perche' il veicolo si muove. Le derivate della terna sono:

```
de_r/dt = cos(phi)*theta_dot * e_e + phi_dot * e_n
de_e/dt = -cos(phi)*theta_dot * e_r + sin(phi)*theta_dot * e_n
de_n/dt = -phi_dot * e_r - sin(phi)*theta_dot * e_e
```

Quindi

$$a = u_{dot} * e_r + v_{dot} * e_e + w_{dot} * e_n + u * de_r/dt + v * de_e/dt + w * de_n/dt$$

Raccogliendo componente per componente e sostituendo  $\theta_{dot} = v / (r * \cos(\phi))$  e  $\phi_{dot} = w / r$ :

```
a_Up = u_dot - (v^2 + w^2)/r
a_East = v_dot + (u*v - v*w*tan(phi)) / r
a_North = w_dot + (u*w + v^2*tan(phi)) / r
```

Invertendo ( $a = F/m$ ) si ottengono **esattamente** le righe 458-460:

```
du = (v^2 + w^2)/r + g_u + (Tu + Du)/m
dv = (-u*v + v*w*tan(phi))/r + (Tv + Dv)/m
dw = (-u*w - v^2*tan(phi))/r + (Tw + Dw)/m
```

**Interpretazione dei termini di trasporto** (la domanda d'esame):

- $(v^2 + w^2)/r$  in  $du$ : e' il termine **centrifugo/di curvatura**. Il veicolo si muove tangenzialmente con velocita' orizzontale  $\sqrt{v^2+w^2}$  lungo una superficie di raggio  $r$ : per restare su quella superficie servirebbe un'accelerazione centripeta  $V_h^2/r$  verso il basso. Nella forma risolta per  $u_{dot}$ , compare col segno + come *accelerazione apparente verso l'alto*. Fisicamente e' **il termine che alleggerisce il veicolo mano che accelera in orizzontale** – al limite  $V_h^2/r = \mu/r^2$  si ha l'orbita circolare e  $du = 0$  anche a spinta nulla. Se lo si togliesse, il razzo non potrebbe *mai* andare in orbita: ricadrebbe sempre.
- $-u*v/r$  in  $dv$  e  $-u*w/r$  in  $dw$ : conservazione del **momento angolare**. Se il veicolo sale ( $u > 0$ ) e non ci sono coppie, la velocita' tangenziale deve diminuire, perche'  $r*v \sim \text{cost.}$  E' lo stesso effetto per cui una pattinatrice rallenta aprendo le braccia.
- $+v*w*\tan(\phi)/r$  in  $dv$  e  $-v^2*\tan(\phi)/r$  in  $dw$ : accoppiamento **Est-Nord dovuto alla convergenza dei meridiani**. Andando verso i poli i meridiani si stringono, quindi muoversi verso Nord con una componente Est fa ruotare il vettore velocita'. Sono i termini che degenerano ai poli ( $\tan(\phi) \rightarrow \text{inf}$ ), stessa singularita' di prima.

**Il punto piu' importante di tutti: dov'e' Coriolis?** Non c'e', e **non deve esserci**. Le equazioni sono scritte nel **frame inerziale**:  $\theta$  e' ascensione retta,  $u$ ,  $v$ ,  $w$  sono componenti della velocita' *inerziale*. Coriolis e la centrifuga *planetaria* sono accelerazioni **apparenti**, che compaiono solo se si scrive  $F = ma$  in un *frame rotante* (ECEF). Qui non lo si fa. I termini di trasporto sopra **non sono** Coriolis: nascono dalla derivazione dei versori di una terna che ruota perche' il veicolo si sposta, non perche' la Terra gira\* – infatti  $\omega_E$  non compare in nessuno di essi.

La rotazione terrestre entra in `main.m` in **tre** punti soltanto, tutti espliciti: 1. la condizione iniziale  $v_0 = \omega_E * R_E * \cos(\text{lat}_0)$  (riga 67); 2. la velocita' relativa usata per drag e direzione di spinta (righe 403-407); 3. la conversione a longitudine di ground track (riga 187).

Se si riscrisse tutto in ECEF si otterrebbero le *stesse traiettorie*, ma le equazioni avrebbero termini  $-2*\omega \times V_{rel}$  (Coriolis) e  $-\omega \times (\omega \times R)$  (centrifuga planetaria) espliciti, e le condizioni iniziali sarebbero a velocita' nulla. E' pura contabilita': la fisica e' identica.

## Atmosfera e spinta (righe 398-423)

```
rho = par.rho0 * exp(-alt / par.H);
patm = par.p0 * exp(-alt / par.H);
...
Tmag = par.Tvac - patm * par.Aex;
```

- Righe 399-400: densita' e pressione decadono con la **stessa** scala  $H$ . Non e' un'assunzione gratuita: con gas ideale  $p = \rho RT$ , se  $p$  e  $\rho$  hanno lo stesso  $H$  allora  $T = \text{cost.}$  E' precisamente l'ipotesi isoterma. E' anche cio' che giustifica `a_sound` costante. La derivazione: equilibrio idrostatico  $dp/dh = -\rho g$ , con  $p = \rho RT$  e  $T, g$  costanti, da'  $dp/p = -dh/(RT/g) = -dh/H$ , quindi  $p = p_0 * \exp(-h/H)$ .

- Riga 423: **la correzione di contropressione**, l'unico pezzo di propulsione "vera" del file. L'equazione della spinta di un ugello e':

$$T = \dot{m} * u_e + (p_e - p_{amb}) * A_e$$

dove  $u_e$  e' la velocita' di efflusso,  $p_e$  la pressione al piano d'uscita,  $A_e$  l'area d'uscita. In vuoto ( $p_{amb} = 0$ ):

$$T_{vac} = \dot{m} * u_e + p_e * A_e$$

Sottraendo membro a membro:

$$T(h) = T_{vac} - p_{amb}(h) * A_e$$

che e' esattamente la riga 423. Il fatto notevole e' che **non serve conoscere ne'  $u_e$  ne'  $p_e$  separatamente**: bastano  $T_{vac}$ ,  $A_e$  e la pressione ambiente. Il termine  $\dot{m} * u_e$  (spinta di quantita' di moto) e' invariante con la quota, quindi tutta la dipendenza da  $h$  e' nel termine di pressione. Numericamente: al livello del mare  $T_{SL} = 8.227 - 0.10133 * 11.039 = 7.108$  MN, cioe' **il 13.6% di spinta in meno** rispetto al vuoto. La spinta cresce poi monotonamente durante la salita fino a  $T_{vac}$ .

- Riga 463:  $dm = -\text{par.Qdot} \cdot \text{costante}$ . La portata non dipende dalla quota: e' l'ugello che e' "choked", la portata e' fissata dalla gola e dalla pressione in camera, non dall'ambiente. Solo la *spinta* varia con  $h$ , non il consumo. Conseguenza diretta e verificabile:  $m(t) = m_0 - Qdot \cdot t$  esattamente, che e' proprio cio' che il test `testPropellantBookkeeping` sfrutta come oracolo analitico.

## Drag (righe 412-420)

```
if Vrel > 1e-10
    Dcoeff = -0.5 * rho * par.Sref * par.CD * Vrel;
    Du = Dcoeff * ur;
    ...
```

- Il drag e' scritto in **forma vettoriale**:

$$D_{\text{vec}} = -0.5 \cdot \rho \cdot S \cdot CD \cdot |V_{\text{rel}}| \cdot V_{\text{rel\_vec}}$$

Il modulo e'  $0.5 \cdot \rho \cdot S \cdot CD \cdot |V_{\text{rel}}|^2$  (la formula classica) e la direzione e' automaticamente  $-V_{\text{rel}}/|V_{\text{rel}}|$ . Scriverlo cosi' evita di calcolare esplicitamente il versore (e quindi una divisione) ed e' numericamente piu' robusto:  $D_{\text{coeff}}$  va a zero linearmente con  $V_{\text{rel}}$ , quindi anche senza la guardia il drag sarebbe corretto a  $V_{\text{rel}} = 0$ .

- La guardia  $V_{\text{rel}} > 1e-10$  e' quindi **ridondante per il drag** ma indispensabile per la fase 3 (righe 443-449), dove si divide per  $V_{\text{rel}}$ . Ha senso averla lo stesso, per simmetria.
- **Non c'e' portanza**, non c'e' forza laterale, non c'e' dipendenza di  $CD$  da Mach. Il modello aerodinamico e' "drag-only,  $CD$  costante".

## Logica di fase e legge di pitchover (righe 425-450)

- **Fase 1** (righe 425-429),  $t \leq 5$  s:  $T_u = T_{\text{mag}}$ ,  $T_v = T_w = 0$ . Spinta puramente radiale. Serve a guadagnare quota e velocita' prima di iniziare a ruotare, in una regione dove la pressione dinamica e' bassa e il veicolo e' ancora lento (quindi poco controllabile aerodinamicamente).
- **Fase 2** (righe 431-439), pitchover:

```
gT = deg2rad(90 - 0.05 * (t - par.t1));
psi = deg2rad(90);
Tu = Tmag * sin(gT);
Tv = Tmag * cos(gT) * sin(psi);
Tw = Tmag * cos(gT) * cos(psi);
```

L'elevazione scende **linearmente** da 90 deg a 89.5 deg in 10 s: un rateo di **0.05 deg/s**, per un totale di mezzo grado. E' un "kick" minuscolo, ed e' intenzionale: serve solo a *innescare* il gravity turn, non a eseguirlo. L'azimuth  $\psi = 90$  deg significa **Est**, il che spiega perche' il ground track si incurva verso Est e perche' il veicolo sfrutta la rotazione terrestre. Le formule  $T_u = T \cdot \sin(gT)$ ,  $T_v = T \cdot \cos(gT) \cdot \sin(\psi)$ ,  $T_w = T \cdot \cos(gT) \cdot \cos(\psi)$  sono la decomposizione standard di un versore dato elevazione (dal piano orizzontale) e azimuth (misurato **da Nord verso Est**, visto che  $\psi = 0$  da' spinta puramente Nord e  $\psi = 90$  puramente Est).

**Onesta' numerica**:  $\cos(\text{deg2rad}(90))$  in floating point non fa 0 ma  $6.1e-17$ . Quindi  $T_w$  in fase 2 non e' esattamente zero: vale  $\sim 6e-17 T_{\text{mag}} \cos(gT)$ , cioe' al massimo  $\sim 4e-12$  N (caso peggiore a fine pitchover, dove  $\cos(gT) = \cos(89.5 \text{ deg}) = 8.7e-3$  e  $T_{\text{mag}} \sim 7.1$  MN; a  $t = 5$  s, con  $gT = 90$  deg, il fattore  $6.1e-17$  compare due volte e  $T_w$  e' ancora piu' piccolo). Fisicamente irrilevante, ma e' una differenza *reale* rispetto a `main2.m`, che pone  $T_w = 0$  esattamente (riga 550 di quel file). Se qualcuno chiedesse perche' i due script non danno bit-per-bit lo stesso risultato, questo e' uno dei motivi (minore).

- **Fase 3** (righe 441-450), gravity turn:

```
Tu = Tmag * ur / Vrel;
Tv = Tmag * vr / Vrel;
Tw = Tmag * wr / Vrel;
```

La spinta è allineata al **versore della velocità relativa**. Questo è il gravity turn (o “zero-lift turn”): dato che l’assetto segue la velocità relativa, l’**angolo d’attacco è identicamente nullo**, e quindi (in un modello con portanza) non ci sarebbe nessuna forza aerodinamica normale. La rotazione della traiettoria è allora prodotta **unicamente dalla componente di gravità normale alla velocità** – da cui il nome.

La ragione ingegneristica e’ strutturale, non di prestazione: volare ad  $\alpha = 0$  attraverso la regione di alta pressione dinamica azzerava i carichi laterali ( $q\alpha$ ), che sono il vincolo dimensionante del corpo del lanciatore. Un gravity turn è *subottimale* dal punto di vista della perdita di gravità – un profilo di pitch ottimo (HM1) fa meglio – ma è quello che la struttura sopporta.

Nota: dopo l’ingresso in fase 3 il veicolo continua a inclinarsi da solo perché la gravità ha una componente normale a V; ma il **pitchover di mezzo grado della fase 2 è ciò che rende non nulla quella componente**. Senza il kick, la velocità resterebbe esattamente radiale, la gravità sarebbe esattamente antiparallela a V, la componente normale sarebbe nulla e il veicolo salirebbe verticalmente per sempre. **Il gravity turn è un equilibrio instabile che va innescato.**

**Possibile domanda d’esame** – Il pitchover dura 10 s e ruota il vettore spinta di appena 0.5 deg. Come fa un’inclinazione così piccola a produrre una traiettoria che a MECO è quasi orizzontale? *Risposta:* Perché il pitchover non deve fare la rotazione, deve solo *innescarla*. Dopo il kick, la velocità non è più esattamente radiale, quindi la gravità acquista una componente normale a V. Da quel momento, in fase 3, la spinta segue V e la gravità normale ruota V continuamente e in modo autosostenuto per i restanti 147 s. La rotazione totale è l’integrale di  $-g\cos(\gamma)/V$  nel tempo, che è grande; il kick iniziale ne fissa solo il segno e l’innescò. Con 0 deg di kick, la componente normale di gravità sarebbe identicamente nulla e il razzo salirebbe verticalmente per sempre.

**Possibile domanda d’esame** – Perché in fase 3 la spinta segue la velocità **relativa** e non quella inerziale? *Risposta:* Perché l’obiettivo del gravity turn è azzerare l’**angolo d’attacco**, che è un angolo *aerodinamico*: è definito rispetto al flusso d’aria, cioè rispetto alla velocità relativa all’atmosfera. Allineare la spinta a  $V_{\text{inerziale}}$  lascerebbe un  $\alpha$  residuo pari all’angolo fra  $V_{\text{in}}$  e  $V_{\text{rel}}$  – che al lift-off è 90 deg ( $V_{\text{rel}} = 0$ !) e resta di parecchi gradi nella fase densa. Sarebbe autolesionistico proprio dove i carichi  $q\alpha$  contano.

## Possibili domande d’esame

**D: Le equazioni del moto sono scritte in un frame inerziale o rotante? Come si riconosce dal codice, e dove finisce la rotazione terrestre?** R: Inerziale. Si riconosce da tre indizi: (1) nelle equazioni dinamiche (righe 458-460) non compare mai  $\omega E$ ; (2)  $\theta$  è etichettata “right ascension” e la longitudine di ground track si ottiene sottraendo  $\omega E t$  (riga 187); (3) la condizione iniziale ha  $v_0 = \omega E R E \cos(\text{lat}_0)$  diversa da zero (riga 67), cosa che in ECEF sarebbe nulla. I termini  $(v^2 + w^2)/r$ ,  $-u \cdot v/r$ ,  $v \cdot w \tan(\phi)/r$  **non sono** Coriolis: sono termini di trasporto che nascono dal derivare i versori della terna UEN, che ruota perché il veicolo si sposta sulla sfera. La rotazione terrestre entra solo in tre punti espliciti: condizione iniziale, velocità relativa (per drag e direzione di spinta), e conversione a longitudine.

**D: Da dove viene  $T = T_{\text{vac}} - p_{\text{atm}} A_{\text{ex}}$ ? E perché la portata di massa non dipende dalla quota?** R: Dall’equazione della spinta di un ugello,  $T = \dot{m} u_e + (p_e - p_{\text{amb}}) A_e$ . Scrivendola in vuoto ( $T_{\text{vac}} = \dot{m} u_e + p_e A_e$ ) e sottraendo, tutti i termini incogniti ( $u_e$ ,  $p_e$ ) si cancellano e resta  $T(h) = T_{\text{vac}} - p_{\text{amb}}(h) A_e$ . *Il termine di quantità di moto è indipendente dalla quota, quindi tutta la variazione è nel termine di pressione: al livello del mare la perdita è  $p_0 A_{\text{ex}} = 1.12$  MN, cioè il 13.6% (7.108 MN contro 8.227 MN).* La portata invece è costante (riga 463) perché l’ugello è in condizioni critiche:  $\dot{m}$  è fissata dalla gola e dalla pressione in camera, non dall’ambiente esterno. Solo la spinta “sente” l’atmosfera, il consumo no.

**D: Mach 1 a 61.8 s, max-Q a 74.9 s. Perché in quest'ordine? Sarebbe potuto essere il contrario?** R: Sono due condizioni indipendenti. Mach 1 è una **soglia** su una velocità fissa (340.3 m/s con **a\_sound** costante). Max-Q è lo **stazionario** del prodotto  $\rho(h) V^2$ , e cade dove  $dV/dt / V = u/(2H)$ , cioè dove la crescita relativa di velocità non riesce più a compensare metà della caduta relativa di densità. Con un lanciatore che accelera rapidamente in un'atmosfera con  $H = 8$  km, la soglia dei 340 m/s viene raggiunta molto prima che il prodotto giri: l'ordine Mach 1 -> max-Q è quindi robusto per qualunque lanciatore di questa classe. In linea di principio un veicolo che accelerasse molto lentamente in quota potrebbe invertirlo, ma non è il caso di un primo stadio.

**D: Il modello usa a\_sound costante e CD costante. Cosa si perde?** R: Due cose diverse. **a\_sound** costante discende coerentemente dall'ipotesi di atmosfera isoterma (se  $T$  è costante,  $a$  lo è), ma quell'ipotesi è falsa: nella troposfera reale  $T$  scende e  $a$  arriva a  $\sim 295$  m/s a 11 km. Quindi il **Mach in quota è sottostimato di circa il 13-15%** e l'attraversamento di Mach 1 è riportato più tardi del vero.  $CD = 0.329$  costante è invece un errore di modello puro: il coefficiente di resistenza di un lanciatore ha un picco transonico marcato. Il codice quindi **sottostima il drag proprio attorno a max-Q**, che è dove pesa di più: la quota e la velocità a MECO risultano leggermente ottimistiche.

**D: Perché non è stata usata una event function di ode45 per gli switch di fase, mentre MaxStep = 1 c'è?** R: Le event function servono a localizzare istanti **incogniti** (impatto, raggiungimento di una quota). Qui gli switch sono a  $t = 5$  s e  $t = 15$  s, noti a priori: non c'è nulla da cercare. **MaxStep = 1** risponde a un problema diverso: max-Q e Mach 1 sono estratti in post-processing con **max()** e **find()** sui **campioni** (righe 160-161), quindi la loro accuratezza dipende dalla densità della griglia di output, non dalla tolleranza dell'integratore. Detto questo, il RHS è **realmente discontinuo a  $t = 15$  s** (la direzione di spinta salta dalla legge di pitchover alla direzione di  $V_{rel}$ ) e **ode45** la attraversa a forza di passi rifiutati. Una soluzione più pulita sarebbe stata spezzare l'integrazione in due chiamate con restart a  $t = 15$  s.

**D: La Figura 7 mostra gammaT che coincide con gammaA dopo  $t = 15$  s. E' una verifica che il gravity turn è implementato bene?** R: **No, è una tautologia.** In post-processing, alla riga 153, il codice *assegna*  $\gamma T(k) = \gamma A(k)$  per  $t > 15$  s. Le due curve coincidono per costruzione, non come risultato dell'integrazione. È vero che la **eom** implementa davvero la spinta lungo  $V_{rel}$  (righe 443-449), quindi il grafico è coerente con il modello – ma non lo verifica. Una verifica vera richiederebbe di ricalcolare l'elevazione del vettore spinta in modo indipendente dalla dinamica integrata e confrontarla.

**D: Cosa succederebbe se si togliesse il termine  $(v^2 + w^2)/r$  dall'equazione di  $du$ ?** R: Il veicolo non potrebbe **mai** andare in orbita. Quel termine è l'accelerazione centrifuga associata al moto orizzontale su una superficie di raggio  $r$ : è ciò che alleggerisce progressivamente il veicolo man mano che guadagna velocità orizzontale. La condizione di orbita circolare è esattamente  $(v^2 + w^2)/r = \mu/r^2$ , cioè il termine di trasporto che cancella la gravità. Rimuovendolo,  $du$  resterebbe negativa per sempre a spinta nulla e qualunque traiettoria ricadrebbe. È il termine che rende “vera” la meccanica orbitale in queste coordinate.

# HM0\_\_falcon9\_\_ascent/main2.m

## Ruolo del file nel progetto

main2.m risolve lo stesso identico problema fisico di main.m – ascesa del primo stadio del Falcon 9, stesso veicolo, stesso sito, stesse tre fasi, stesse forze – ma con due cambiamenti strutturali che non toccano la fisica:

1. **Adimensionalizzazione completa** dello stato e dei parametri, con scale di riferimento  $L_{ref} = R_E$ ,  $V_{ref} = 7800$  m/s (prima velocità cosmica),  $T_{ref} = L_{ref}/V_{ref}$ ,  $m_{ref} = m_0$ .
2. **Riparametrizzazione del tempo**: al posto del tempo fisico si integra su una variabile  $\tau$  in  $[0, 3]$ , in cui ciascuno dei tre archi di volo occupa esattamente un'unità di  $\tau$ . La durata dimensionale di ciascun arco entra nelle equazioni come un fattore moltiplicativo  $\Delta_k$  davanti al RHS.

Il file **non è una versione migliore** di main.m, ne' un doppiopione. E' un **file preparatorio**: entrambe queste scelte sono esattamente cio' che serve nei problemi di ottimizzazione di traiettoria che seguono (HM1: shooting indiretto, HM2: collocation diretta). L'adimensionalizzazione condiziona bene il problema numerico (tutti i termini  $O(1)$ , cosa che conta enormemente quando i costati o il Jacobiano di un NLP mescolano metri, secondi e chilogrammi); la struttura a tre archi con  $\Delta_k$  in ingresso e' la forma **canonica dei problemi a durata di arco libera**, dove i  $\Delta_k$  diventano *variabili di decisione* e la griglia di integrazione resta fissa.

Nella pipeline di HM0, main2.m ha un ruolo preciso: e' la **seconda implementazione indipendente** contro cui tests/falcon9AscentTest.m cross-valida main.m. Due codici scritti con scalature e parametrizzazioni diverse che convergono agli stessi numeri sono una verifica molto piu' forte di qualunque singolo test di regressione.

Differenze operative rispetto a main.m: main2.m **non esporta le figure in PNG** (non ha il blocco di export), aggiunge una nona figura con lo stato adimensionale contro  $\tau$ , stampa a console le scale di riferimento e lo stato finale adimensionale, e usa una event function (che, come si vedra' piu' sotto, **non fa quello che i commenti dichiarano**).

---

## Intestazione: il contratto della riparametrizzazione (righe 1-35)

```
% Integration variable:
%   A single parameter tau in [0, 3] covers all three arcs.
%   tau in [0,1] -> Arc 1: t* = tau      * Delta1
%   tau in [1,2] -> Arc 2: t* = (tau - 1) * Delta2 + t1*
%   tau in [2,3] -> Arc 3: t* = (tau - 2) * Delta3 + t2*
%   The EOM is scaled accordingly: dy*/dtau = Delta_k * (dy*/dt*)
```

- Righe 6-16: le scale di riferimento e quelle **derivate** (accelerazione, forza, pressione). Il punto chiave e' che si scelgono liberamente **quattro** scale indipendenti ( $L$ ,  $V$ ,  $m$  – e  $T$  che ne discende), e tutte le altre sono *obbligate* per consistenza dimensionale:

```
a_ref = V_ref^2 / L_ref      (= 9.539 m/s^2)
F_ref = m_ref * a_ref       (= 5.43 MN)
p_ref = F_ref / L_ref^2 = m_ref*V_ref^2 / L_ref^3
```



Non c'è libertà di scelta qui: se si sbaglia una sola di queste derivate, il bilancio delle forze non torna e la traiettoria è sbagliata di un fattore.

- Riga 32: **la chain rule**, il cuore matematico del file:

$$dy*/d\tau = (dy*/dt) * (dt*/d\tau) = \Delta_k * (dy*/dt)$$

perché entro l'arco  $k$  si ha  $t^* = t_{start} + (\tau - (k-1)) * \Delta_k$ , e quindi  $dt*/d\tau = \Delta_k$  costante a tratti. È l'unica modifica alle equazioni: si moltiplica tutto il RHS per  $\Delta_k$ .

- Riga 35: `clear`, con la stessa conseguenza sulla testabilità vista in `main.m`.

**Perché fare tutto questo?** La risposta onesta è che **per HM0 non serve a niente**: il problema è ben condizionato anche in forma dimensionale, e la riparametrizzazione (come si vedrà) *peggiora* il RHS introducendo due discontinuità artificiali. Il guadagno arriva dopo:

- In un problema a **durate di arco libere** (tipico dell'ottimizzazione: quando finisce il pitchover? quando avviene lo staging?), i  $\Delta_k$  sono incogniti. Con la parametrizzazione  $\tau$ , l'intervallo di integrazione resta **[0, 3] qualunque siano i  $\Delta_k$** : la griglia non si muove, i nodi di collocation non si muovono, e i  $\Delta_k$  entrano come semplici parametri moltiplicativi nel RHS – quindi si possono differenziare rispetto a essi in modo banale. Se invece si integrasse in  $t$  con estremi variabili, ogni valutazione dell'obiettivo cambierebbe il dominio di integrazione: un incubo per un NLP.
- L'adimensionalizzazione con tutte le variabili  $O(1)$  è ciò che rende utilizzabile un `fsolve/fmincon`: con  $r \sim 6.4e6$ ,  $m \sim 5.7e5$  e  $u \sim 1e2$  nello stesso vettore, il Jacobiano ha numeri di condizionamento assurdi e le tolleranze relative/assolute perdono significato.

---

## CONSTANTS AND VEHICLE PARAMETERS (righe 37-83) e LAUNCH SITE (righe 85-90)

Identici a `main.m` righe 12-54 e 60-61: stessi valori numerici, stessi commenti. Valgono quindi **le stesse osservazioni oneste** fatte in `hm0_main.md`:

- Riga 52: `Tsid = 86136 s`, contro il giorno sidereo standard di 86164.09 s (errore relativo  $3.2e-4$ , trascurabile ma reale).
- Riga 48: `Hscale = 8000 m` non è consistente con `Tamb = 288.15 K` sotto l'ipotesi di gas ideale isoterma, che darebbe  $H = R*T/g = 8435$  m.
- Riga 58: `a_sound = 340.30 m/s` **costante a tutte le quote** – il Mach in quota è sottostimato.
- Riga 67: `Qdot1 = Tvac1/cvac1 = 2536.07 kg/s`; `Qdot1*tb1 = 410 843 kg` contro `mp1 = 410 900 kg` dichiarati.

Il fatto che le costanti siano **duplicate** anziché condivise (per esempio in un `params_falcon9.m` chiamato da entrambi) è un debito tecnico reale: qualunque correzione (per esempio `Tsid`) va fatta in due posti, e se se ne dimentica uno il test di cross-validazione `main.m` vs `main2.m` fallirebbe segnalando una “differenza fisica” che invece è un errore di copia-incolla.

---

## REFERENCE QUANTITIES (righe 92-108)

```
L_ref = RE;           % [m]
V_ref = 7800;         % [m/s]
T_ref = L_ref / V_ref; % [s]
m_ref = m0;          % [kg]
```

- Riga 96: `L_ref = RE`. Scelta con una conseguenza elegante: `r0* = 1` **esattamente** e la quota adimensionale è semplicemente `r* - 1` (riga 507 nella `eom_nd`). Nessuna sottrazione fra numeri grandi quasi uguali – che in forma dimensionale è un vero rischio di cancellazione catastrofica: al lift-off  $h = r - RE$  è una



differenza fra due numeri da 6.4e6 che vale 0. In doppia precisione non e' un problema serio, ma in singola lo sarebbe.

- Riga 97:  $V_{ref} = 7800$  m/s, la “prima velocita' cosmica” (velocita' orbitale circolare rasente). E' una scelta **fisicamente motivata**, non arbitraria: e' la velocita' che il veicolo dovra' *comunque* raggiungere. Così la velocita' adimensionale si legge come “frazione di orbita gia' guadagnata”: con la velocita' relativa a MECO di 2773.6 m/s (README), il primo stadio consegna circa il **36%** di  $V_{ref}$ .
- Riga 98:  $T_{ref} = L_{ref}/V_{ref} = 817.71$  s. Non e' libera: discende dalle altre due. Notare che il tempo di combustione, 162 s, corrisponde a  $t^*_b = 0.198$ : tutta la missione HM0 avviene in un quinto del tempo di riferimento.
- Riga 101:  $m_{ref} = m_0$ , quindi  $m_0^* = 1$  e la massa adimensionale e' la frazione di massa iniziale rimasta. A MECO vale  $\sim 0.278$ .

## NON-DIMENSIONAL PARAMETERS (righe 110-123)

```
mu_nd      = mu      / (V_ref^2 * L_ref);
omegaE_nd  = omegaE * T_ref;
rho0_nd     = rho0    * L_ref^3 / m_ref;
p0_nd      = p0      * L_ref^3 / (m_ref * V_ref^2);
H_nd       = Hscale / L_ref;
Tvac_nd     = Tvac1   * L_ref / (m_ref * V_ref^2);
Aex_nd     = Aex1    / L_ref^2;
Qdot_nd    = Qdot1   * T_ref / m_ref;
Sref_nd     = Sref    / L_ref^2;
```

Ogni riga e' una divisione per la scala di riferimento della grandezza corrispondente. Vale la pena verificarle una per una, perche' un solo esponente sbagliato produce una traiettoria sbagliata e silenziosamente plausibile.

- Riga 114:  $[\mu] = L^3/T^2$ . La scala e'  $L_{ref}^3/T_{ref}^2 = L_{ref}^3 * V_{ref}^2/L_{ref}^2 = L_{ref} * V_{ref}^2$ . Quindi  $\mu_{nd} = \mu/(V_{ref}^2 * L_{ref}) = 1.0272$ . E' un numero bellissimo: dice che  $V_{ref}$  e' *quasi esattamente* la velocita' circolare rasente, perche' quella vera sarebbe  $\sqrt{\mu/RE} = 7905$  m/s (contro i 7800 assunti). Il fatto che  $\mu_{nd} \sim 1$  conferma che la scala di velocita' e' stata scelta bene.
- Riga 115:  $\omega_{nd} = \omega_{ref} * T_{ref} = 0.05965$ . Una frequenza si adimensionalizza moltiplicando (non dividendo) per il tempo di riferimento.
- Riga 116:  $[\rho] = m/L^3 \rightarrow \rho_{nd} = \rho * L_{ref}^3/m_{ref}$ . Verifica:  $1.225 * (6.378e6)^3 / 5.691e5$  e' un numero enorme ( $\sim 5.6e14$ ). **Non tutti i parametri adimensionali sono  $O(1)$**  – e non e' un errore: cio' che deve essere  $O(1)$  sono i termini delle equazioni, non i singoli parametri. Nel termine di drag  $\rho_{nd} * S_{ref_{nd}} * CD * V_{rel_{nd}}^2$ , il fattore  $L_{ref}^3$  di  $\rho_{nd}$  si semplifica contro  $L_{ref}^2$  di  $S_{ref_{nd}}$ , e il risultato e'  $O(1)$ . Se all'orale si fa notare che  $\rho_{nd}$  e'  $5.6e14$ , la risposta e': guarda il prodotto\*, non il fattore.
- Riga 117: la pressione.  $p_{ref} = F_{ref}/L_{ref}^2 = m_{ref} * V_{ref}^2/L_{ref}^3$ , quindi  $p_{nd} = p_0 * L_{ref}^3/(m_{ref} * V_{ref}^2)$ . Corretto.
- Riga 119:  $T_{vac_{nd}} = Tvac * L_{ref}/(m_{ref} * V_{ref}^2) = 1.5155$ . Una forza si divide per  $F_{ref} = m_{ref} * V_{ref}^2/L_{ref}$ . Il valore  $\sim 1.5$  e' significativo: poiche'  $\mu_{nd} \sim 1$  (cioe' l'accelerazione di gravita' adimensionale a  $r^* = 1$  e'  $\sim 1$ ),  $T_{vac_{nd}}/m_{0_{nd}} \sim 1.5$  e' **direttamente il rapporto spinta/peso in vuoto**. L'adimensionalizzazione, quando le scale sono ben scelte, fa emergere i numeri di merito del problema come valori dei parametri.
- Riga 121:  $Q_{dot_{nd}} = Q_{dot} * T_{ref}/m_{ref} = 3.644$ . Significa che, se il motore potesse funzionare per un intero  $T_{ref}$ , brucerebbe 3.64 volte la massa iniziale del veicolo. Confrontato con  $t^*_b = 0.198$ , spiega perche' viene consumato il 72% della massa:  $3.644 * 0.198 = 0.722$ .
- Riga 123: il commento “CD is already dimensionless” e' corretto e non banale: e' l'unico parametro che non richiede conversione.

**Possibile domanda d'esame** – L'adimensionalizzazione dovrebbe rendere tutti i termini  $O(1)$ , ma  $\rho_{0_{nd}}$  vale circa  $5.6e14$ . Non e' una contraddizione? *Risposta:* No, perche' cio' che deve essere

$O(1)$  sono i **termini delle equazioni**, non i parametri presi singolarmente.  $\rho_{0\_nd}$  compare solo dentro il prodotto  $\rho_{0\_nd} * S_{ref\_nd} * CD * V_{rel\_nd}^2 / m_{nd}$ , dove il fattore  $L_{ref}^3$  di  $\rho_{0\_nd}$  si cancella contro  $l^3/L_{ref}^3$  di  $S_{ref\_nd}$  lasciando un singolo  $L_{ref}$ , che a sua volta e' assorbito dalla scala di forza. Il risultato e' un'accelerazione adimensionale  $O(1)$ . Il criterio corretto e' guardare il RHS assemblato, non i coefficienti.

---

## NON-DIMENSIONAL INITIAL CONDITIONS (righe 125-135)

```
r0_nd = 1;
u0_nd = 0;
v0_nd = omegaE * RE * cos(lat0) / V_ref;
w0_nd = 0;
m0_nd = 1;
y0_nd = [r0_nd; lon0; lat0; u0_nd; v0_nd; w0_nd; m0_nd];
```

- Riga 129 e riga 133:  $r_{0*} = 1$  e  $m_{0*} = 1$  per costruzione, come voluto.
- Riga 131:  $v_{0*} = 408.6/7800 = 0.0524$ . La rotazione terrestre regala poco piu' del 5% della velocita' orbitale.
- Riga 135: **theta e phi restano dimensionali** (radianti). Corretto: gli angoli sono gia' adimensionali, non c'e' nulla da scalare. E' facile sbagliare qui e dividere anche loro per qualcosa. Da notare che questo rende il vettore di stato **eterogeneo**: 5 componenti adimensionalizzate e 2 angoli in radianti. Non e' un problema (i radianti sono  $O(1)$ ), ma va detto.

---

## ARC DURATIONS (righe 137-147)

```
Delta1 = t1_nd;           % arc 1 nd duration
Delta2 = t2_nd - t1_nd;   % arc 2 nd duration
Delta3 = tb_nd - t2_nd;   % arc 3 nd duration
```

Numericamente (calcolati con  $T_{ref} = 817.71$  s):

```
Delta1 = 5 / 817.71 = 0.006115      (ascesa verticale, 5 s)
Delta2 = 10 / 817.71 = 0.012229     (pitchover, 10 s)
Delta3 = 147 / 817.71 = 0.179770    (gravity turn, 147 s)
```

- Notare  $\Delta_3/\Delta_2 = 14.7$  e  $\Delta_2/\Delta_1 = 2$ . Questi due rapporti sono **esattamente l'ampiezza dei salti che il RHS subisce** attraversando  $\tau = 1$  e  $\tau = 2$  (vedi sotto): l'arco 3 e' quasi 15 volte piu' lungo dell'arco 2, e il fattore Delta davanti al RHS salta di altrettanto.
- **Qui il codice mostra il suo limite come "file preparatorio"**: i  $\Delta_k$  sono *calcolati* da tempi fissi (5, 15, 162 s), non sono variabili. La struttura e' pronta a riceverli come incognite (basterebbe metterli in par dall'esterno) ma in HM0 non lo sono. Il beneficio della parametrizzazione  $\tau$  e' quindi **potenziale, non realizzato** in questo file.

---

## PARAMETER STRUCTURE (righe 149-169)

- Righe 153-162: i parametri adimensionali.
- Righe 168-169: qui c'e' un dettaglio che merita attenzione. La struct porta **par.T\_ref** e **par.t\_end1** in **unita' dimensionali**, in mezzo a parametri altrimenti tutti adimensionali. Il motivo e' alle righe 545-546:

la legge di pitchover e' definita in **gradi al secondo dimensionali** (0.05 deg/s), quindi dentro l'EOM adimensionale bisogna **ri-dimensionalizzare il tempo** per valutarla.

E' una piccola inconsistenza di design: la legge di guida non e' stata adimensionalizzata insieme al resto. La scelta alternativa "pulita" sarebbe stata esprimere il rateo di pitchover in **rad** per unita' di  $t^*$  (cioe'  $0.05 * \pi/180 * T_{ref} = 0.7135$  rad per unita' di  $t^*$ ) e togliere  $T_{ref}$  dalla struct. Funzionalmente e' identico; ma la forma attuale garantisce che `main.m` e `main2.m` usino **esattamente la stessa legge di pitchover**, il che e' importante per il test di cross-validazione.

---

## SINGLE ode45 CALL (righe 171-184)

```
opts = odeset('RelTol', 1e-10, 'AbsTol', 1e-12, ...  
             'Events', @arc_boundary_events);  
  
[tau_sol, Y_nd] = ode45(@(tau, y) eom_nd(tau, y, par), [0 3], y0_nd, opts);
```

- Riga 184: **una sola chiamata** a `ode45` copre tutta la missione, su `[0 3]`.
- Rispetto a `main.m`: stesse tolleranze (`RelTol 1e-10`, `AbsTol 1e-12`), ma **nessun MaxStep**. Non serve, perche' l'unita' di `tau` vale al massimo 147 s dimensionali: `ode45` sceglie passi in `tau` che si traducono automaticamente in passi dimensionali di grana fine.

## La event function NON fa quello che i commenti dicono

Il commento alla riga 178 afferma:

```
% Arc boundaries are exact (Events function stops the step there).
```

e `documentazione.txt` (sezione 10) rincara: *"il solver non si ferma ma inserisce un punto esatto della soluzione in  $\tau = 1$  e  $\tau = 2$  [...] senza eventi, un singolo step di Runge-Kutta potrebbe attraversare il cambio di scala producendo un errore di integrazione."*

**Entrambe le affermazioni sono false, e questo e' stato verificato sperimentalmente.** Con `istterminal = [0; 0]`, `ode45`:

- rileva l'evento e ne calcola l'istante esatto per interpolazione;
- lo restituisce nelle uscite **te, ye, ie** – cioe' nella *quarta, quinta e sesta* uscita di `ode45`;
- **non modifica ne' la sequenza dei passi ne' la griglia di output  $t/y$** ;
- **non impedisce a un passo di attraversare il confine.**

Riproducendo lo schema esatto di `arc_boundary_events` su un problema di prova, la griglia di output con e senza event function risulta **bit-per-bit identica** (369 punti in entrambi i casi, `isequal(t, t_senza_eventi) = 1`), e **nessun punto di output cade esattamente su  $\tau = 1$  o  $\tau = 2$**  (il piu' vicino a  $\tau = 1$  e' 1.00241...).

In piu' – colpo di grazia – la riga 184 chiama `ode45` con **due sole uscite**:

```
[tau_sol, Y_nd] = ode45(...);
```

quindi **te, ye, ie non vengono nemmeno raccolte**. La event function e' quindi completamente **inerte**: rimuoverla non cambierebbe di un bit il risultato. E' codice morto documentato come se fosse essenziale.

**Cosa protegge davvero la soluzione ai confini degli archi**, allora? Solo il controllo di passo adattivo di `ode45`: attraversando  $\tau = 1$  e  $\tau = 2$  il RHS salta (di un fattore 2 e di un fattore 14.7 rispettivamente), lo stimatore d'errore locale esplode, il passo viene rifiutato e ridotto ripetutamente finche' non rientra in `RelTol = 1e-10`. Funziona – i risultati sono corretti, come conferma il test di cross-validazione contro `main.m` – ma e' costoso, non e' "esatto", e non e' merito della event function.

**La correzione corretta** sarebbe `isterminal = [1; 1]` con **tre chiamate separate** a `ode45` (una per arco), riavviando ogni volta dallo stato finale del precedente. Così' nessun passo attraverserebbe mai una discontinuita'. E' esatta- mente la struttura *multi-arco* che serve comunque negli homework di ottimizzazione.

**Possibile domanda d'esame** – La parametrizzazione in `tau` e' presentata come un miglioramento numerico. Lo e' davvero, per HM0? *Risposta:* No, per HM0 e' un **peggioramento**. In `main.m` il RHS e' continuo a  $t = 5$  s (la legge di pitchover a  $t = 5$  restituisce esattamente 90 deg, riproducendo la fase 1) e ha una piccola discontinuita' di direzione a  $t = 15$  s. In `main2.m`, invece, il fattore `Delta_k` moltiplica l'intero RHS e cambia a gradino: a  $\tau = 1$  il RHS **raddoppia** di colpo ( $\Delta_2/\Delta_1 = 2$ ) e a  $\tau = 2$  viene **moltiplicato per 14.7** ( $\Delta_3/\Delta_2 = 14.7$ ). La riparametrizzazione *introduce* due discontinuita' grandi che nella formulazione dimensionale non esistevano, costringendo `ode45` a rifiutare passi ai confini. Il guadagno arriva solo quando i `Delta_k` diventano **variabili di decisione** in un problema di ottimizzazione a durate di arco libere: lì il dominio di integrazione fisso  $[0, 3]$  vale ampiamente il prezzo.

## RECOVER GLOBAL ND TIME AND DIMENSIONAL QUANTITIES (righe 186-268)

### Ricostruzione del tempo (righe 190-200)

```
mask1 = tau_sol <= 1;
mask2 = tau_sol > 1 & tau_sol <= 2;
mask3 = tau_sol > 2;
t_nd(mask1) = tau_sol(mask1) * Delta1;
t_nd(mask2) = (tau_sol(mask2) - 1) * Delta2 + t1_nd;
t_nd(mask3) = (tau_sol(mask3) - 2) * Delta3 + t2_nd;
t = t_nd * T_ref;
```

- Righe 195-197: e' l'inversa **esatta** della mappa  $\tau \rightarrow t^*$ . Il tempo dimensionale  $t(\tau)$  risulta **lineare a tratti con tre pendenze diverse**, proporzionali a  $\Delta_1$ ,  $\Delta_2$ ,  $\Delta_3$ . La Figura 9, pannello (3,2), lo mostra esplicitamente ed e' il modo piu' rapido per verificare che la mappa sia giusta: se le tre pendenze non stanno nel rapporto 5 : 10 : 147, c'e' un errore.
- **Nota sulla coerenza:** la stessa mappa e' implementata **due volte** – qui in post-processing (righe 195-197) e dentro `eom_nd` (righe 480-496). Sono duplicate a mano. Se una delle due venisse modificata senza l'altra, il codice produrrebbe silenziosamente risultati sbagliati (la dinamica userebbe un tempo, il post-processing un altro). E' un rischio reale di manutenzione.

### Ri-dimensionalizzazione e post-processing (righe 202-268)

- Righe 203-209: si moltiplica per le scale. `theta` e `phi` (righe 204-205) **non** vengono scalati, coerentemente con il fatto che erano gia' radianti.
- Righe 212-268: da qui in poi il post-processing e' **riga per riga identico a quello di main.m** (righe 114-188): `quota`, `Vrel`, `qdyn`, `Mach`, `gammaT/gammaA`, `eventi`, `ENU`, `ground track`. Valgono le stesse osservazioni:
  - `qdyn` con la velocita' **relativa** (riga 224), corretto;
  - `gammaT(k) = gammaA(k)` in fase 3 (riga 242) e' **assegnato**, quindi la coincidenza delle due curve nella Figura 7 e' una tautologia, non una verifica;
  - `im1` e `imQ` (righe 249-250) sono estratti **sui campioni**, senza interpolazione. Qui pero' **non c'e' MaxStep**: la griglia e' scelta liberamente da `ode45`. E' plausibile che la griglia in  $\tau$  sia piu' rada, in termini dimensionali, di quella di `main.m` – motivo per cui il test tollera `RelTol = 1e-2` sul confronto di `qmax` fra i due script (vedi `hm0_test_falcon9Ascent.md`).
- **Riga 223:** `p_traj = p0 * exp(-h/Hscale)` viene calcolata e **mai usata** nel seguito. E' una variabile morta (in `main.m` non c'e' affatto). Innocua, ma e' rumore.

## CONSOLE SUMMARY (righe 270-300)

Identico a `main.m` (righe 194-212) fino alla riga 292, poi aggiunge (righe 293-299) lo **stato finale adimensionale**: `r*`, `u*`, `v*`, `w*`, `m*`, `t*`. E' l'output piu' utile per la verifica: `m*(end)` deve valere  $1 - Q_{dot\_nd} * t_{b\_nd} = 1 - 3.644 * 0.198 = 0.278$  e `r*(end)` deve valere  $1 + h_{end}/RE \sim 1.013$ . Sono controlli che si possono fare a mente.

---

## PLOTS (righe 302-462)

- Figure 1-8 (righe 311-415): **le stesse otto figure di `main.m`**, con gli stessi eventi marcati e la stessa palette, ma con [ND] nel Name e "(ND simulation)" nel titolo. Servono al confronto visivo fra i due script.
  - **`main2.m` non ha il blocco di export PNG**. Le figure restano a schermo e non vengono scritte su disco. E' una scelta deliberata e corretta: `figures/` e' di proprieta' della pipeline di `main.m` (come dice il README), e due script che scrivono gli stessi nomi file si sovrascriverebbero a vicenda in modo imprevedibile. Il commento della classe di test (`falcon9AscentTest.m`, righe 7-8) dice "running the scripts regenerates the PNGs": e' vero solo per `main.m`.
  - **Figura 9** (righe 417-462): l'unica figura che `main.m` non ha. Un `tiledlayout(3,2)` con lo **stato adimensionale contro tau**: `r*`, le componenti `u*`, `v*`, `w*`, `m*`, `h* = r*-1, |V*|`, e `t_dim(tau)`. I pannelli hanno `xline(1)` e `xline(2)` a marcare i confini d'arco. E' il pannello **diagnostico** della riparametrizzazione: guardando (3,2) si legge immediatamente la mappa lineare a tratti  $\tau \rightarrow t$ ; guardando (1,2) si vede che in  $\tau$  i tre archi occupano lo stesso spazio orizzontale pur avendo durate fisiche di 5, 10 e 147 s – e' proprio l'effetto che si voleva.
- 

## eom\_nd – equazioni del moto adimensionali (righe 468-578)

### Identificazione dell'arco e ricostruzione del tempo (righe 479-496)

```
if tau <= 1
    Delta = par.Delta1; t_arc_start = 0;          tau_k = tau;          phase = 1;
elseif tau <= 2
    Delta = par.Delta2; t_arc_start = par.t1_nd; tau_k = tau - 1; phase = 2;
else
    Delta = par.Delta3; t_arc_start = par.t2_nd; tau_k = tau - 2; phase = 3;
end
t_nd = t_arc_start + tau_k * Delta;
```

- E' la stessa cascata di `if` di `main.m` (righe 425-450), ma con una funzione in piu': oltre a scegliere la fase, deve **ricostruire il tempo adimensionale globale `t*`** dalla variabile di integrazione `tau`, perche' la legge di pitchover ne ha bisogno.
- **Confini**: i test sono `tau <= 1` e `tau <= 2`, quindi il valore esatto `tau = 1` e' assegnato all'**arco 1** e `tau = 2` all'**arco 2**. E' arbitrario ma innocuo: nei punti di confine le due definizioni coincidono comunque (a  $\tau = 1$ , la legge di pitchover con  $t_{dim} = 5$  s restituisce 90 deg, che e' esattamente la fase 1). A  $\tau = 2$ , invece, le due fasi **non** coincidono, quindi il valore del RHS esattamente in  $\tau = 2$  e' quello della fase 2. Poiche' e' un insieme di misura nulla, non ha effetto sull'integrazione.

### Corpo delle equazioni (righe 498-574)

Da riga 498 a riga 574, la struttura e' **identica** a quella di `main.m` (righe 387-463). Questa e' la proprieta' piu' importante e vale la pena enunciarla esplicitamente:

**Le equazioni adimensionali hanno la stessa forma algebrica di quelle dimensionali.**

Non e' un caso: e' la conseguenza di aver scelto le scale in modo **auto-consistente** ( $a_{ref} = V_{ref}^2/L_{ref}$ ,  $F_{ref} = m_{ref} * a_{ref}$ , ecc.). Concretamente:

- riga 507: `alt = r - 1` invece di `alt = r - RE` (perche' `R_E* = 1`);
- riga 523: `g_u = -par.mu/r^2` – stessa formula, `mu` adimensionale;
- riga 528: stesso coefficiente di drag vettoriale;
- riga 535: `Tmag = par.Tvac - patm*par.Aex` – **la correzione di contropressione ha la stessa identica forma**, con tutti i simboli adimensionali;
- righe 564-571: cinematica e dinamica **letteralmente identiche** (stessi termini di trasporto  $(v^2+w^2)/r$ ,  $-u*v/r$ ,  $v*w*\tan(\phi)/r$ , ...).

Se una qualunque delle scale derivate fosse sbagliata, **questa proprieta' si romperebbe** e comparirebbero fattori spuri nelle equazioni. Il fatto che il corpo di `eom_nd` sia un copia-incolla di `eom` e' quindi la miglior prova che le adimensionalizzazioni delle righe 114-122 sono corrette.

La derivazione dei termini di trasporto (perche'  $(v^2+w^2)/r$ , perche' non ci sono termini di Coriolis, ecc.) e' identica a quella dimensionale ed e' svolta per esteso in `hm0_main.md`, sezione `eom`.

## La legge di pitchover (righe 543-550)

```
case 2
    t_dim = t_nd * par.T_ref;
    gT     = deg2rad(90 - 0.05 * (t_dim - par.t_end1));
    Tu     = Tmag * sin(gT);
    Tv     = Tmag * cos(gT);
    Tw     = 0;
```

- Riga 545: **si ri-dimensionalizza il tempo** per valutare la legge di guida. E' la conseguenza di quella scelta di design discussa sopra: la legge (0.05 deg/s) e' definita in unita' dimensionali. Funziona, ma significa che `eom_nd` non e' "puramente adimensionale": porta dentro `T_ref`.
- Righe 548-550: qui c'e' la **differenza numerica reale** rispetto a `main.m`. `main.m` (righe 437-439) scrive:

```
Tv = Tmag * cos(gT) * sin(psi)    con psi = deg2rad(90)
Tw = Tmag * cos(gT) * cos(psi)
```

e poiche'  $\cos(\pi/2)$  in floating point vale  $6.1e-17$  e non 0, `main.m` ha una componente Nord di spinta **non esattamente nulla** (dell'ordine di  $1e-12$  N:  $Tmag \sim 7e6$  N per  $\cos(gT) \leq 8.7e-3$  per  $\cos(\psi) = 6.1e-17$ ). `main2.m` scrive direttamente `Tv = Tmag*cos(gT)` e `Tw = 0`, sostituendo a mano  $\sin(90 \text{ deg}) = 1$  e  $\cos(90 \text{ deg}) = 0$ . E' piu' pulito e piu' veloce, e spiega perche' i due script non possono coincidere bit-per-bit. L'effetto e' comunque del tutto trascurabile rispetto alla differenza dovuta alle diverse griglie temporali.

- Nota che `main2.m` **perde la generalita'**: se domani si volesse un azimuth diverso da 90 deg, `main.m` basterebbe cambiare `psi`, mentre `main2.m` andrebbe riscritto. E' un trade-off consapevole (o forse no – il codice non lo dichiara).

## Chain rule (righe 576-577)

```
% Chain rule: dy*/dtau = Delta_k * (dy*/dt*)
dydt = Delta * [dr; dtheta; dphi; du; dv; dw; dm];
```

- Riga 577: l'unica riga che distingue davvero questo EOM da quello dimensionale. Tutto il RHS, **tutte e sette le componenti massa inclusa**, viene moltiplicato per `Delta`.
- E' cruciale che sia *tutto* il vettore: se si dimenticasse di scalare `dm`, la massa evolverebbe con un tempo diverso da quello della dinamica e il veicolo arriverebbe a MECO con la massa sbagliata. Il test `testPropellantBookkeeping` (che verifica  $m(t_b) = m_0 - Qdot*t_b$  a `RelTol 1e-8` **anche per main2.m**) e' precisamente il controllo che smaschererebbe un errore in questa riga: e' un oracolo analitico esatto sulla componente piu' semplice del sistema, e passa solo se la mappa  $\tau \rightarrow t$  e la scalatura `Delta_k` sono entrambe giuste.



## arc\_boundary\_events (righe 582-589)

```
function [value, isterminal, direction] = arc_boundary_events(tau, ~, ~)
    value      = [tau - 1; tau - 2];
    isterminal = [0; 0];
    direction  = [+1; +1];
end
```

- Riga 586: due funzioni evento che si annullano a  $\tau = 1$  e  $\tau = 2$ .
- Riga 587: `isterminal = [0; 0]` – **il solver non si ferma**.
- Riga 588: `direction = [+1; +1]` – evento rilevato solo attraversando in direzione crescente. Corretto ( $\tau$  cresce sempre), ma irrilevante.
- **Come documentato sopra, questa funzione e' inerte:** con `isterminal = 0` ode45 non tocca ne' il passo ne' la griglia di output, e le uscite `te/ye` non vengono nemmeno richieste alla riga 184. Il commento di intestazione (righe 583-585, “so ode45 never straddles an arc boundary”) **descrive un comportamento che non si verifica**.
- La firma ignora sia lo stato sia i parametri (`~, ~`): l'evento dipende solo dalla variabile di integrazione. Questo di per se' e' un indizio: un evento che dipende solo da  $\tau$  e' un evento a *istante noto*, e gli eventi a istante noto non hanno bisogno di essere “trovati” – vanno gestiti spezzando l'integrazione.

---

## Possibili domande d'esame

**D: Qual e' la differenza sostanziale fra `main.m` e `main2.m`?** R: Nessuna, sul piano fisico. Stesso veicolo, stesse forze (gravita'  $1/r^2$ , drag con CD costante, spinta corretta per contropressione), stesse tre fasi, stesse equazioni in coordinate sferiche con velocita' UEN, stessi valori numerici di tutte le costanti. Le differenze sono **due, entrambe di formulazione**: (1) adimensionalizzazione con  $L_{ref} = R_E$ ,  $V_{ref} = 7800$  m/s,  $m_{ref} = m_0$ ; (2) riparametrizzazione del tempo su  $\tau$  in  $[0,3]$  con un arco per unita' di  $\tau$  e il fattore `Delta_k` davanti al RHS. Differenze operative minori: `main2.m` non esporta i PNG, aggiunge una nona figura diagnostica sullo stato adimensionale, e scrive  $T_w = 0$  esattamente in fase 2 dove `main.m` calcola  $\cos(90 \text{ deg})$  in floating point ( $6.1e-17$ ). Il test di cross-validazione conferma che i due convergono agli stessi risultati entro lo 0.5%.

**D: A cosa serve davvero la parametrizzazione in  $\tau$ , visto che in HM0 le durate degli archi sono fisse?** R: Non serve a niente in HM0 – anzi, peggiora il condizionamento del RHS introducendo due discontinuita' (fattori 2 e 14.7) ai confini d'arco. Serve **per gli homework di ottimizzazione**. Quando le durate degli archi diventano variabili di decisione (durata del pitchover, istante di staging, tempo finale libero), la parametrizzazione  $\tau$  garantisce che il **dominio di integrazione resti  $[0, 3]$  qualunque siano i  $\Delta_k$** : la griglia di collocation non si muove, i nodi non si muovono, e i  $\Delta_k$  entrano nel problema come semplici parametri moltiplicativi nel RHS – quindi con derivate banali rispetto alle variabili di decisione. Integrando invece nel tempo fisico con estremi mobili, ogni valutazione dell'obiettivo cambierebbe il dominio, cosa che un NLP gestisce malissimo. E' la forma canonica dei problemi multi-arco a durata libera.

**D: La event function `arc_boundary_events` serve a evitare che ode45 attraversi le discontinuita' ai confini d'arco. Vero o falso?** R: Falso, e i commenti nel codice (riga 178) e in `documentazione.txt` sono sbagliati su questo punto. Con `isterminal = [0; 0]`, ode45 rileva l'evento e lo riporta nelle uscite `te/ye/ie`, ma **non modifica ne' la sequenza dei passi ne' la griglia di output**: verificato sperimentalmente, la griglia con e senza event function e' identica e nessun punto di output cade su  $\tau = 1$  o  $\tau = 2$ . In piu' la riga 184 chiama ode45 con sole **due** uscite, quindi `te/ye/ie` sono scartate: la funzione e' completamente inerte. Cio' che protegge la soluzione ai confini e' semplicemente il controllo di passo adattivo, che rifiuta e riduce i passi finche' l'errore locale rientra in tolleranza. Per ottenere davvero l'effetto dichiarato servirebbe `isterminal = [1; 1]` e **tre chiamate ode45 separate** con restart a ogni confine.

**D:  $\mu_{nd}$  vale 1.027 e  $T_{vac,nd}$  vale 1.516. Sono numeri “casuali” o dicono qualcosa?** R: Dicono molto, ed e' il segno che le scale sono state scelte bene.  $\mu_{nd} \sim 1$  significa che  $V_{ref} = 7800$  m/s e' quasi esattamente la velocita' orbitale circolare rasente (quella vera e'  $\sqrt{\mu/R_E} = 7905$  m/s): l'accelerazione di gravita' adimensionale alla superficie vale quindi  $\sim 1$ . Di conseguenza  $T_{vac,nd} / m_{0,nd} = 1.516 / 1 = 1.516$

**e' direttamente il rapporto spinta/peso in vuoto** del veicolo. Un'adimensionalizzazione ben fatta trasforma i parametri nei numeri di merito del problema: si legge il regime di volo direttamente dai valori dei coefficienti, senza fare conti.

**D: Perche' theta e phi non vengono adimensionalizzati?** R: Perche' sono **gia'** adimensionali: un angolo in radianti e' un rapporto fra lunghezze (arco / raggio). Non esiste una "scala di riferimento per gli angoli" da applicare. Il vettore di stato risulta quindi eterogeneo – 5 componenti scalate e 2 angoli in radianti – ma questo non e' un problema, perche' i radianti sono gia'  $O(1)$  e quindi l'obiettivo dell'adimensionalizzazione (avere tutte le componenti di grandezza confrontabile) e' comunque soddisfatto.

**D: Perche' eom\_nd porta dentro par.T\_ref, che e' un'unita' dimensionale?** R: Perche' la legge di pitchover e' definita in gradi al secondo **dimensionali** (0.05 deg/s), e la riga 546 deve valutarla: la riga 545 ricostruisce percio'  $t_{dim} = t_{nd} * T_{ref}$ . E' una piccola inconsistenza di design: la legge di guida non e' stata adimensionalizzata insieme al resto del modello. Si poteva evitare esprimendo il rateo come  $0.05 * (\pi/180) * T_{ref} = 0.7135$  rad per unita' di  $t*$  e togliendo  $T_{ref}$  dalla struct. Il vantaggio della forma attuale e' che garantisce che `main.m` e `main2.m` usino **la stessa identica legge**, il che e' essenziale perche' il test di cross-validazione abbia senso.



# HM1/ode\_burn.m

## Ruolo del file nel progetto

ode\_burn.m e' il **cuore numerico di tutta HM1**. E' il right-hand side (RHS) dell'arco propulso: la funzione che ode45 chiama a ogni step per propagare in avanti lo stato del lanciatore **con la legge di controllo ottima gia' sostituita dentro**. Non e' un integratore "della dinamica" e basta: e' la dinamica *chiusa sulle condizioni necessarie di Pontryagin*, cioe' il sistema Hamiltoniano ridotto che nasce dal PMP dopo aver eliminato il controllo phi tramite la condizione di massimizzazione.

Il file esiste perche' il metodo indiretto trasforma il problema di controllo ottimo in un **Boundary Value Problem (BVP)**: dati i costati iniziali incogniti, si integra in avanti e si guarda quanto si sbaglia sulle condizioni al contorno finali. ode\_burn e' esattamente il pezzo "si integra in avanti". E' condiviso da **tutti e quattro i task**: lo chiamano main\_task1.m (dentro shooting1 e per la ri-integrazione delle soluzioni convergenti), main\_task2.m (arco propulso dopo la salita verticale), main\_task3.m (arco propulso prima del coast), main\_task4.m e validate\_staging\_corner.m (i due archi separati dallo staging), piu' le suite HM1/tests/odeBurnTest.m e odeBurnPerformanceTest.m.

Implementa le equazioni del moto planare (Introduction.tex, eq. xdot-mdot) piu' le equazioni di Eulero-Lagrange dei costati (eq. lx-lm), con la legge bilinear-tangent (eq. bilinear) inserita al posto del controllo. Il modello e' 2D, Terra piatta, gravita' costante, **niente atmosfera, niente drag, spinta sempre accesa e di modulo costante**  $T = c*Q$ .

Dipende solo dalla struct p che gli passa il chiamante: p.T, p.Q, p.c (in realta' p.c **non viene mai letto** da questa funzione – vedi sotto) e i tre parametri dei costati p.lam\_vx0, p.lam\_vy0, p.lam\_y, che sono esattamente tre delle quattro incognite dello shooting.

---

## ode\_burn – intestazione e contratto (righe 1-14)

```
function dz = ode_burn(t, z, p)
% Powered-flight RHS, linear-tangent steering.
% z - state [x; y; vx; vy; m; lam_m]
% p - struct: T, Q, c, lam_vx0, lam_vy0, lam_y
```

- Riga 1: firma. Tre argomenti nell'ordine imposto da ode45: tempo scalare t, vettore di stato z (6x1), struct di parametri p passata via closure (@(t,z) ode\_burn(t,z,pp)). Restituisce dz, la derivata 6x1.
- Riga 5: il vettore integrato e' [x; y; vx; vy; m; lam\_m]. Attenzione: **non e'** lo stato fisico e basta (che sarebbe 5 componenti, x, y, vx, vy, m), e **non e'** nemmeno stato + tutti i costati (che sarebbero 5+5 = 10). E' un ibrido: 5 stati fisici piu' **un solo costato**, lam\_m. Il motivo e' spiegato sotto (righe 18-21).
- Righe 10-11: il commento dichiara la struttura analitica dei costati: lam\_x = 0, lam\_y = p.lam\_y (costante), lam\_vx = p.lam\_vx0 (costante), lam\_vy = p.lam\_vy0 - p.lam\_y\*t (rampa lineare). Sono le soluzioni in forma chiusa delle equazioni di Eulero-Lagrange – quindi **non serve integrarle numericamente**.
- Riga 11: phi = atan2(lam\_vy, lam\_vx) – la legge di controllo, dichiarata gia' nell'header.

- Righe 13-14: **niente blocco arguments per scelta di design**. Questa non e' pigrezza: la funzione sta nel loop piu' interno di `ode45`, che a sua volta sta nel loop di `fsolve`, che a sua volta sta nel loop di continuazione su `Q` (80 valori) e su `yf` (3 valori). Il numero di chiamate e' dell'ordine di  $1e6+$ . Un blocco **arguments** con `mustBeNumeric/mustBeScalar` costa qualche microsecondo a chiamata e diventerebbe una frazione dominante del tempo totale. La validazione degli input vive **al boundary** (una volta per run), non nel hot loop. E' una convenzione dichiarata nel `CLAUDE.md` della repo e va difesa, non "aggiustata".

**Possibile domanda d'esame** – Perche' il vettore integrato ha 6 componenti e non 10 (5 stati + 5 costati)? *Risposta:* Perche' quattro delle cinque equazioni dei costati si risolvono in forma chiusa. Da  $\lambda_{\dot{x}} = -dH/dx$  si ottiene  $\lambda_{\dot{x}} = 0$  (e la trasversalita' su  $x$  libero da'  $\lambda_x = 0$  identicamente),  $\lambda_{\dot{y}} = 0$  (costante),  $\lambda_{\dot{v}_x} = -\lambda_x = 0$  (costante),  $\lambda_{\dot{v}_y} = -\lambda_y$  (rampa lineare). Solo  $\lambda_{\dot{m}}$  dipende dallo stato ( $m$ ) e va integrato. Quindi si integrano 5 stati fisici +  $\lambda_m = 6$  equazioni, e i tre parametri  $\lambda_{v_x0}$ ,  $\lambda_{v_y0}$ ,  $\lambda_y$  bastano a ricostruire analiticamente tutti gli altri costati a qualunque  $t$ . Si risparmiano 4 ODE e, cosa piu' importante, si elimina l'errore di integrazione su di esse.

## Unpack dello stato (riga 16)

```
vx = z(3); vy = z(4); m = z(5);
```

- Riga 16: si estraggono solo le tre componenti che servono al RHS.  $x = z(1)$  e  $y = z(2)$  non compaiono a destra di nessuna equazione: la dinamica **non dipende dalla posizione** (Terra piatta,  $g$  costante, niente atmosfera). E' proprio questo che rende  $\lambda_x$  e  $\lambda_y$  costanti ( $dH/dx = dH/dy = 0$ ). Se ci fosse il drag,  $dH/dy \sim 0$  (densita' esponenziale in  $y$ ),  $\lambda_y$  non sarebbe piu' costante,  $\lambda_{v_y}$  non sarebbe piu' lineare in  $t$ , e **tutta la struttura linear-tangent cadrebbe**: bisognerebbe integrare anche i costati numericamente.
- Anche  $z(6) = \lambda_m$  non viene letto: la sua derivata (riga 33) dipende da  $m$  e dai costati di velocita', non da se stesso. L'equazione di  $\lambda_m$  e' quindi una **pura quadratura accodata**, accoppiata in una sola direzione.

## Costati: ricostruzione analitica (righe 18-21)

```
lam_vx = p.lam_vx0;
lam_vy = p.lam_vy0 - p.lam_y * t;
lam_v_norm = sqrt(lam_vx^2 + lam_vy^2);
```

**Da dove viene questa forma.** L'Hamiltoniana del problema (Mayer, massimizzare  $m(tf)$ ) e', con  $g = 1$  in nondimensionale:

$$H = \lambda_x v_x + \lambda_y v_y + \lambda_{v_x} (T/m) \cos(\phi) + \lambda_{v_y} ((T/m) \sin(\phi) - 1) - \lambda_m Q$$

Le equazioni di Eulero-Lagrange  $\lambda_{\dot{x}} = -dH/dx$  danno, componente per componente:

$$\begin{aligned} \lambda_{\dot{x}} &= -dH/dx = 0 && \rightarrow \lambda_x = \text{cost} = 0 \\ \lambda_{\dot{y}} &= -dH/dy = 0 && \rightarrow \lambda_y = \text{cost} \\ \lambda_{\dot{v}_x} &= -dH/dv_x = -\lambda_x = 0 && \rightarrow \lambda_{v_x} = \text{cost} = \lambda_{v_x0} \\ \lambda_{\dot{v}_y} &= -dH/dv_y = -\lambda_y && \rightarrow \lambda_{v_y} = \lambda_{v_y0} - \lambda_y t \\ \lambda_{\dot{m}} &= -dH/dm = +(T/m^2) * |\lambda_{v}| \end{aligned}$$

- Riga 19:  $\lambda_{v_x}$  costante. Deriva da  $\lambda_{\dot{v}_x} = -\lambda_x$  e da  $\lambda_x = 0$ . Il fatto che  $\lambda_x = 0$  non e' un'ipotesi: e' la **condizione di trasversalita'** su  $x(tf)$  libero e non presente nel costo ( $d\phi/dx = 0$ ), combinata con  $\lambda_{\dot{x}} = 0$  che la propaga all'indietro su tutto l'arco.

- Riga 20: `lam_vy` rampa lineare. Il segno meno viene da `lam_vy_dot = -lam_y`. **Interpretazione fisica del segno:** con `lam_y > 0` (guess iniziale del Task 1: `lam_y = 14`), `lam_vy` *decresce* nel tempo, quindi `phi` scende da quasi verticale a orizzontale (e persino sotto l'orizzonte). E' il **pitch-over**: la legge lo produce da sola, non e' programmato a mano. Se si mettesse `lam_y < 0` il razzo alzerebbe il muso durante la salita – non ottimo.
- Riga 21: `lam_v_norm = |lam_v|` e' il modulo del **primer vector**. Compare in due posti: nella derivata di `lam_m` (riga 33) e – implicitamente – nella funzione di switching  $S = |lam_v|/m - lam_m/c$  usata dal Task 3 per il cutoff.
- **Nota di onesta':** `t` qui e' il tempo **globale** dell'arco, cioe' lo stesso tempo rispetto a cui `lam_vy0` e' definito. In `main_task4.m` il secondo arco viene integrato su `[ts tf]` (non su `[0 tf-ts]`) proprio perche' la rampa deve restare continua. Se si passasse un `t` locale ripartito da 0 su un arco che non inizia a `t = 0`, la legge di controllo verrebbe silenziosamente sbagliata. Il codice non ha alcun assert su questo: e' un contratto implicito.

**Possibile domanda d'esame** – Cosa significa fisicamente che `lam_vy` e' lineare in `t` mentre `lam_vx` e' costante? *Risposta:* Significa che  $\tan(\phi) = lam\_vy(t)/lam\_vx$  e' una funzione lineare del tempo: e' la **legge linear-tangent** (bilinear-tangent). E' la firma caratteristica dei problemi di ascesa senza atmosfera e con gravita' costante, e vale perche' la dinamica non dipende dalla posizione (`lam_y = cost`) e l'accelerazione di gravita' e' costante. Non appena si aggiunge il drag, o una gravita' che varia con la quota, la linearita' si perde.

## Legge di controllo ottima (righe 23-24)

```
% Optimal thrust angle
phi = atan2(lam_vy, lam_vx);
```

**Derivazione.** Raccogliendo i termini in `phi` nell'Hamiltoniana:

```
H = [lam_x*vx + lam_y*vy - lam_vy - lam_m*Q]
    + (T/m) * [ lam_vx*cos(phi) + lam_vy*sin(phi) ]
```

Il PMP dice: `phi*massimizza` `H` puntualmente. Poiche'  $T/m > 0$  (spinta positiva, massa positiva), massimizzare `H` equivale a massimizzare

```
g(phi) = lam_vx*cos(phi) + lam_vy*sin(phi)
```

che e' il prodotto scalare tra il versore di spinta `[cos(phi), sin(phi)]` e il vettore `lam_v = [lam_vx, lam_vy]`. Un prodotto scalare tra un versore e un vettore fisso e' massimo quando il versore e' **allineato** al vettore:

```
[cos(phi*), sin(phi*)] = lam_v / |lam_v|    ->    g_max = +|lam_v|
```

La condizione di stazionarieta'  $dH/d\phi = 0$  da'  $\tan(\phi^*) = lam\_vy/lam\_vx$ , che ha **due radici per periodo** (differiscono di  $\pi$ ). La radice sbagliata da'  $g = -|lam_v|$ , cioe' il **minimo** di `H`: spinta anti-allineata al primer vector, razzo che frena. `atan2(lam_vy, lam_vx)` – a differenza di `atan(lam_vy/lam_vx)` – usa i **segni separati** dei due argomenti per selezionare il quadrante giusto, quindi sceglie automaticamente il ramo di massimo. Usare `atan` qui sarebbe un bug silenzioso ogni volta che `lam_vx < 0`.

- **Attenzione al segno (punto delicato, spesso confuso).** In molti testi la direzione di spinta ottima si scrive  $-\lambda_v/|\lambda_v|$ . Qui **no**: il codice usa  $+\lambda_v/|\lambda_v|$ . Non e' un errore, e' la convenzione. In HM1 il problema e' posto come **massimizzazione** di  $m(tf)$  con il funzionale aumentato in forma  $(f - x\_dot)$ ; questa scelta da'  $lam\_m(tf) = +1$  e  $lam\_m\_dot = +(T/m^2)*|lam_v| \geq 0$ , e la spinta va **con** `lam_v`. Con la convenzione opposta (minimizzazione del consumo, funzionale in forma  $(x\_dot - f)$ ) tutti i costati cambiano segno e la spinta risulta lungo  $-\lambda_v$ . **Le due formulazioni danno la stessa traiettoria fisica**; quello che conta e' che segno dell'Hamiltoniana, segno di `lam_m_dot` e segno del controllo siano coerenti tra loro – e nel codice lo sono (righe 24 e 33 usano entrambe il +).
- **Vincolo di controllo assente.** `phi` e' libero su tutto il cerchio: nessuna saturazione, nessun limite di rateo di beccheggio, nessun vincolo di angolo d'attacco. E' questo che rende la massimizzazione dell'Hamil-

toniana un problema algebrico banale. Con un vincolo del tipo  $|\phi - \psi| \leq \alpha_{\max}$  la soluzione diventerebbe un problema con archi vincolati (le cose diventano *molto* piu' difficili).

**Possibile domanda d'esame** – Perché `atan2` e non `atan`? *Risposta:* Perché la condizione di stazionarietà  $\tan(\phi) = \lambda_{m\_vy}/\lambda_{m\_vx}$  determina  $\phi$  solo modulo  $\pi$ , e delle due radici una massimizza e l'altra minimizza l'Hamiltoniana. `atan` restituisce sempre un angolo in  $(-\pi/2, \pi/2)$  perché vede solo il rapporto e perde i segni individuali; `atan2` vede i segni di numeratore e denominatore separatamente e restituisce il quadrante corretto, selezionando il ramo che allinea la spinta con il primer vector (il massimo). In Task 1 con  $\lambda_{m\_vx0} > 0$  i due coinciderebbero, ma la funzione è condivisa da tutti i task e non c'è garanzia a priori sul segno di  $\lambda_{m\_vx0}$ .

## Derivate dello stato (righe 26-33)

```
dz = zeros(6,1);
dz(1) = vx;           % dx/dt
dz(2) = vy;           % dy/dt
dz(3) = (p.T / m) * cos(phi); % dvx/dt
dz(4) = (p.T / m) * sin(phi) - 1; % dvy/dt (g = 1 nondim)
dz(5) = -p.Q;         % dm/dt
dz(6) = (p.T / m^2) * lam_v_norm; % dlam_m/dt
```

- Riga 27: preallocazione `zeros(6,1)`. `ode45` pretende un vettore **colonna**; restituire una riga fa fallire l'integrazione.
- Righe 28-29: cinematica. Banali, ma sono loro a rendere  $x$  e  $y$  non presenti nel RHS (vedi sopra).
- Riga 30:  $dv_x/dt = (T/m) \cos(\phi)$ . Nessuna forza orizzontale oltre alla spinta: niente drag, niente Coriolis (Terra non rotante).
- Riga 31:  $dv_y/dt = (T/m) \sin(\phi) - 1$ . **Il -1 è la gravità**. Nello schema di nondimensionalizzazione  $a_{rif} = g$ , quindi  $g/a_{rif} = 1$  esatto. Se si togliesse quel  $-1$  sparirebbe la gravity loss,  $\lambda_{m\_vy}$  resterebbe comunque lineare (perché  $-dH/dv_y$  non contiene  $g$ : la gravità è un termine costante nell'Hamiltoniana, non dipende dallo stato) ma la traiettoria non salirebbe più contro nulla e il problema perderebbe il suo trade-off centrale. **Sottigliezza:** il termine  $-1$  entra in  $H$  come  $-\lambda_{m\_vy}$  (vedi il termine  $\lambda_{m\_vy}*(-1)$ ), ed è esattamente quel  $-\lambda_{m\_vy}$  che compare nella condizione  $H(0) = -\lambda_{m\_vy0} + T*(|\lambda_{m\_v0}| - 1/c)$  usata come quarto residuo dello shooting.
- Riga 32:  $dm/dt = -Q$ , con  $Q$  **costante**. Conseguenza notevole: la massa è **lineare** nel tempo,  $m(t) = m_0 - Q*t = 1 - Q*t$ . Quindi  $m_f = 1 - Q*tf$  in forma chiusa. Il codice invece la legge da `Z(end,5)` dopo l'integrazione – funziona, ed è anche un buon test di consistenza dell'integratore, ma è informazione ridondante. Nota: **non c'è nessun guard su  $m > 0$** . Con  $Q = 7$  (estremo alto dello sweep) basta  $tf > 1/7 \sim 0.143$  per far diventare la massa negativa e  $T/m$  singolare. In pratica non succede perché la soluzione ottima ha  $tf$  ben sotto quella soglia, ma la protezione è assente per costruzione.
- Riga 33:  $d\lambda_{m\_m}/dt = (T/m^2)*|\lambda_{m\_v}|$ . **Derivazione del segno:**

$$\begin{aligned} dH/dm &= \lambda_{m\_vx} T * (-1/m^2) * \cos(\phi) + \lambda_{m\_vy} T * (-1/m^2) * \sin(\phi) \\ &= -(T/m^2) * [\lambda_{m\_vx} \cos(\phi) + \lambda_{m\_vy} \sin(\phi)] \\ &= -(T/m^2) * |\lambda_{m\_v}| \quad (\text{sostituendo } \phi \text{ ottimo}) \end{aligned}$$

e quindi  $\lambda_{m\_m\_dot} = -dH/dm = +(T/m^2)*|\lambda_{m\_v}| \geq 0$ .  $\lambda_{m\_m}$  **cresce sempre** lungo un arco propulso (è costante solo dove  $T = 0$ , cioè in coast). Interpretazione:  $\lambda_{m\_m}$  è la sensibilità del costo (massa finale) alla massa istantanea; più si è leggeri, più ogni chilo in più "vale", perché  $T/m$  è maggiore.

- **Nota di onestà importante:** in `main_task1.m` (e in Task 2 e 4) `z(6)` viene integrato ma il suo valore **non entra mai nel residuo dello shooting**. Il motivo è la normalizzazione  $\lambda_{m\_m0} = 1$ : fissando la scala dei costati all'istante iniziale si *rinuncia* alla condizione di trasversalità  $\lambda_{m\_m}(tf) = 1$ . Quindi `Z(end,6)` a fine integrazione **non vale 1** (vale più di 1, dato che  $\lambda_{m\_m}$  cresce): è semplicemente un multiplo positivo

dei costati “veri”. Questo e’ innocuo perche’ la traiettoria dipende dai costati solo tramite la **direzione phi** (un rapporto, invariante di scala) e perche’  $H$  e’ omogenea di grado 1 in  $\lambda$ , quindi  $H = 0$  e’ anch’essa invariante di scala.  $z(6)$  e’ portato in giro perche’ **serve al Task 3**, dove la normalizzazione  $\lambda_{m0} = 1$  viene abbandonata:  $\lambda_m$  diventa una **quinta incognita vera** dello shooting ( $z(4)$  in `main_task3.m`) e il residuo di trasversalita’  $\lambda_m(tc) = 1$  legge direttamente  $z(6)$ , cioe’ il valore integrato di  $z(6)$  al cutoff (`main_task3.m` riga 302). E’ **questo residuo** – non la funzione di switching – a rendere indispensabile l’integrazione di  $z(6)$ : la switching function teorica e’ effettivamente  $S = |\lambda_v|/m - \lambda_m/c$ , ma nel codice, una volta che il residuo ha fissato  $\lambda_{mc} = 1$ , viene implementata nella forma equivalente  $S = |\lambda_v|/mc - 1/c$  (riga 298), in cui  $\lambda_m$  **non compare**. La scelta e’ deliberata e commentata nel sorgente: la forma letterale  $\lambda_{mc}/c$  fa scivolare `fsolve` sulla radice spuria con  $vyc < 0$ , mentre le due forme coincidono al cutoff.

- **Nota:**  $p.c$  e’ documentato nell’header (riga 6) come campo della struct ma `ode_burn` **non lo usa mai**. La relazione  $T = c*Q$  viene applicata dal chiamante ( $p.T = c * p.Q$  in `main_task1.m`). Il commento e’ quindi leggermente fuorviante:  $p.c$  e’ richiesto dal *contratto complessivo* della pipeline, non da questa funzione.

**Possibile domanda d’esame** – Il sistema e’ autonomo (l’Hamiltoniana e’ costante), eppure `ode_burn` dipende esplicitamente da  $t$ . Contraddizione? *Risposta:* No. Il sistema **stato + costati** e’ autonomo, ed e’ per questo che  $H = \text{cost} = 0$ . La dipendenza esplicita da  $t$  in `ode_burn` e’ un **artefatto dell’aver risolto in forma chiusa l’equazione di  $\lambda_{vy}$** : invece di integrare  $\lambda_{vy\_dot} = -\lambda_y$  come settima equazione, si sostituisce direttamente  $\lambda_{vy}(t) = \lambda_{vy0} - \lambda_y*t$ . E’ una riduzione esatta, non un’approssimazione: l’informazione temporale che vediamo e’ quella del costato eliminato.

## Possibili domande d’esame

**D: Da dove viene esattamente  $dz(6) = (T/m^2)*|\lambda_v|$  e perche’ e’ sempre positiva?** R: E’ l’equazione di Eulero-Lagrange per il costato di massa,  $\lambda_{m\_dot} = -dH/dm$ . Nell’Hamiltoniana la massa compare solo nei termini di spinta  $\lambda_{vx}*(T/m)*\cos(\phi)$  e  $\lambda_{vy}*(T/m)*\sin(\phi)$ ; derivando rispetto a  $m$  esce  $-(T/m^2)*( \lambda_{vx}\cos + \lambda_{vy}\sin )$ , e sostituendo il controllo ottimo la parentesi vale  $|\lambda_v|$ . Il doppio segno meno ( $-dH/dm$  di un termine gia’ negativo) lascia il segno +. E’ sempre  $\geq 0$  perche’  $T \geq 0$ ,  $m > 0$  e  $|\lambda_v| \geq 0$ : lungo un arco propulso  $\lambda_m$  cresce monotonicamente, cioe’ il valore marginale di un chilo di massa aumenta man mano che il veicolo si alleggerisce.

**D: Come si passa da “massimizzare  $H$ ” a  $\phi = \text{atan2}(\lambda_{vy}, \lambda_{vx})$  in due righe?** R: Solo due termini di  $H$  contengono  $\phi$ , e si raccolgono come  $(T/m)*( \lambda_{vx}\cos(\phi) + \lambda_{vy}\sin(\phi) )$ , cioe’  $(T/m)$  per il prodotto scalare tra il versore di spinta e il vettore  $\lambda_v$ . Con  $T/m > 0$  massimizzare  $H$  significa massimizzare quel prodotto scalare, che e’ massimo quando il versore di spinta e’ allineato con  $\lambda_v$ . Quindi  $\cos(\phi) = \lambda_{vx}/|\lambda_v|$ ,  $\sin(\phi) = \lambda_{vy}/|\lambda_v|$ , cioe’  $\phi = \text{atan2}(\lambda_{vy}, \lambda_{vx})$ , e il valore massimo del termine di controllo e’  $(T/m)*|\lambda_v|$ . Il vettore  $\lambda_v$  e’ il **primer vector** di Lawden.

**D: Perche’ questa funzione non ha un blocco arguments di validazione, quando il resto della repo lo usa?** R: Perche’ e’ una hot-loop function. Sta dentro `ode45`, dentro `fsolve`, dentro il doppio loop di continuazione su  $Q$  (80 punti) e  $yf$  (3 valori): l’ordine di grandezza e’  $1e6+$  chiamate per run. Con  $\text{RelTol} = 1e-10$  l’integratore fa migliaia di step per traiettoria, e ogni step richiede piu’ valutazioni del RHS (Runge-Kutta 4/5 ne fa 6). Un blocco `arguments` aggiungerebbe overhead a ogni chiamata senza dare nessuna informazione nuova (gli input arrivano sempre dallo stesso chiamante interno). La validazione e’ collocata al boundary del run, una volta sola. E’ una scelta documentata nel `CLAUDE.md` della repo.

**D: Cosa succede se aggiungi il drag aerodinamico a questo RHS?** R: Si rompe tutta la struttura analitica. Il drag dipende dalla densita’  $\rho(y)$  e dalla velocita’, quindi  $dH/dy \neq 0$  e  $dH/dvx$ ,  $dH/dvy$  acquistano termini nuovi. Conseguenze a cascata:  $\lambda_y$  non e’ piu’ costante, quindi  $\lambda_{vy}$  non e’ piu’ lineare in  $t$ , quindi **la legge linear-tangent non vale piu’**. Bisognerebbe integrare tutti e cinque i costati numericamente (stato aumentato  $10 \times 1$  invece di  $6 \times 1$ ), il controllo resterebbe  $\phi = \text{atan2}(\lambda_{vy}, \lambda_{vx})$  solo se la spinta e’ l’unica forza controllata e il drag non dipende da  $\phi$  – cosa che in generale e’ falsa (il drag dipende dall’angolo d’attacco). Il costo per chiamata almeno raddoppierebbe e il basin di convergenza dello shooting si restringerebbe ulteriormente.

**D:  $z(6)$  (cioe'  $\text{lam\_m}$ ) viene integrato ma in Task 1 non serve. Perche' non toglierlo?** R: Perche' `ode_burn.m` e' condiviso da tutti e quattro i task, e il **Task 3 ne ha bisogno**: li' la normalizzazione  $\text{lam\_m0} = 1$  viene abbandonata,  $\text{lam\_m0}$  diventa una quinta incognita vera dello shooting e il residuo di trasversalita'  $\text{lam\_m}(t_c) = 1$  legge direttamente  $z(6)$  (`main_task3.m` riga 302) – senza integrare  $z(6)$  quel residuo non esisterebbe. Attenzione a non raccontarla male all'orale: la funzione di switching teorica e'  $S = |\text{lam\_v}|/m - \text{lam\_m}/c$ , ma nel codice, dopo che il residuo ha fissato  $\text{lam\_mc} = 1$ , e' implementata nella forma equivalente  $S = |\text{lam\_v}|/mc - 1/c$  (riga 298), dove  $\text{lam\_m}$  non compare – la forma letterale  $\text{lam\_mc}/c$  porterebbe `fsolve` sulla radice spuria  $v_{yc} < 0$ . Portare 6 equazioni invece di 5 in Task 1 costa pochissimo (una quadratura accodata, accoppiata in una sola direzione: nessun altro  $dz(i)$  dipende da  $z(6)$ ) ed evita di duplicare il RHS. Va pero' detto chiaramente che in Task 1 il valore di  $z(6)$  a **tf non soddisfa** la trasversalita'  $\text{lam\_m}(t_f) = 1$ , perche' quella condizione e' stata sostituita dalla normalizzazione  $\text{lam\_m0} = 1$ .

**D: Perche' la spinta e' lungo  $+\text{lambda\_v}$  e non  $-\text{lambda\_v}$  come in molti libri?** R: E' una questione di convenzione, non di fisica. HM1 pone il problema come **massimizzazione** di  $m(t_f)$  e costruisce il funzionale aumentato con  $\text{lambda}^T(f - x_{\text{dot}})$ . Con questa scelta la trasversalita' da'  $\text{lam\_m}(t_f) = +1$ , la PMP richiede di **massimizzare**  $H$ , e la spinta risulta allineata con  $+\text{lam\_v}$ . Con la convenzione duale (minimizzare il consumo, funzionale con  $\text{lambda}^T(x_{\text{dot}} - f)$ ) tutti i costati cambiano segno e la spinta va lungo  $-\text{lambda\_v}$ . Le due traiettorie ottime coincidono. L'unica cosa che conta e' la **coerenza interna**: nel codice il + di riga 24 (controllo) e il + di riga 33 ( $\text{lam\_m\_dot}$ ) appartengono alla stessa convenzione, e la stessa convenzione e' usata in  $H(0) = -\text{lam\_vy0} + T*(|\text{lam\_v0}| - 1/c)$  nel residuo dello shooting.

# HM1/main\_\_task1.m

## Ruolo del file nel progetto

`main_task1.m` e' lo **script di ingresso del Task 1** di HM1: l'ascesa planare a **singolo arco propulso**, dal decollo da fermo fino allo stato di iniezione orbitale, risolta con il metodo indiretto (PMP + single shooting). E' il piu' semplice dei quattro programmi di missione ed e' quello che stabilisce l'infrastruttura riusata da tutti gli altri: la struct di parametri `p`, il residuo di shooting, la strategia di continuazione, le tolleranze.

Lo script risponde a tre domande della traccia, che sono anche le tre figure che produce: - **(a)** come varia la massa finale `mf` al variare della portata `Q`, per tre quote target `yf = 0.04, 0.05, 0.06`; - **(b)** come si decompone la perdita di velocita' (misalignment `Wd` + gravita' `Wg`) in funzione di `Q`, a `yf = 0.04`; - **(c)** che aspetto hanno traiettoria e legge di controllo alla `Q` ottima.

Il punto centrale e' che **`Q` non e' un dato ma un parametro di progetto**: c'e' un massimo interno di `mf(Q)`. A `Q` bassa il razzo e' lento, la spinta e' appena sopra il peso e le gravity loss divorano il delta-V; a `Q` alta il razzo brucia in fretta ma con una spinta enorme e mal allineata alla velocita', e domina la misalignment loss. Il compromesso e' `Q*`.

Lo script contiene, oltre al corpo principale, **tre local function** in fondo: `shooting1` (il residuo del BVP), `set_costates` (utility), `ode_burn_losses` (una copia estesa di `ode_burn` con due integratori di perdita accodati). Dipende da `HM1/ode_burn.m`, che vive in un file a parte perche' e' condiviso con gli altri task.

---

## Header e %% Parameters (righe 1-18)

```
c = 0.6;           % effective exhaust velocity (nondim)
eta = 0.1;         % structural coefficient ms/mp
yf_vec = [0.04, 0.05, 0.06];
Q_vec = linspace(1.8, 7, 80); % Q > 1/c ~ 1.667 for liftoff
```

- Riga 6: `clear; close all; clc` – questo e' uno **script**, non una funzione. Le variabili finiscono nel workspace base. E' il motivo per cui il `close all` conta: l'export delle figure (righe 207-223) rastrella con `findobj(groot, 'Type', 'figure')` **tutte** le figure aperte, quindi partire con la lavagna pulita e' necessario per non esportare roba di sessioni precedenti.
- Riga 9: `c = 0.6`. E' la velocita' di efflusso efficace **nondimensionale**. Lo schema di nondimensionalizzazione (documentato in `HM1/README.md` e nel report) e':

```
a_rif = g = 9.81 m/s^2
V_rif = vxf = 7800 m/s      (velocita' orbitale al target)
m_rif = m0 = 1e6 kg
L_rif = V_rif^2 / g ~ 6200 km
t_rif = V_rif / g ~ 795 s
```

Quindi `c = 0.6` significa `c_dim = 0.6 * 7800 = 4680 m/s`, cioe' `Isp ~ 477 s`. In queste unita' `g = 1`, `m0 = 1`, e la condizione terminale sulla velocita' orizzontale diventa semplicemente `vx(tf) = 1`.

- Riga 10:  $\eta = 0.1$  – coefficiente strutturale  $m_s/m_p$ . **Non entra nella dinamica**: e' usato solo *a posteriori* (riga 104) per convertire la massa finale in payload:  $\mu = m_f(1+\eta) - \eta$ . Deriva da  $m_f = \mu + m_s$  con  $m_s = \eta m_p$  e  $m_p = 1 - m_f$ , da cui  $\mu = m_f - \eta(1-m_f) = m_f(1+\eta) - \eta$ . Conseguenza: il payload puo' venire **negativo** (e in effetti a  $y_f = 0.06$  viene negativo, secondo i risultati riportati nel README) – l'ottimizzatore non lo sa e non se ne cura, perche' massimizza  $m_f$ , non  $\mu$ . Massimizzare  $m_f$  e massimizzare  $\mu$  sono pero' equivalenti, perche'  $\mu$  e' una funzione **affine crescente** di  $m_f$ .
- Riga 13:  $Q\_vec = linspace(1.8, 7, 80)$ . Il limite inferiore non e' arbitrario: a  $t = 0$  si ha  $T/W = c*Q/m_0 = c*Q$ , quindi il decollo richiede  $c*Q > 1$ , cioe'  $Q > 1/c = 1.667$ .  $Q = 1.8$  da'  $T/W = 1.08$ , appena sopra soglia. Il decollo e' il caso vincolante perche'  $T/W = c*Q/m$  **cresce** man mano che la massa scende.
- Riga 15: `opts_ode = odeset('RelTol', 1e-10, 'AbsTol', 1e-12)`. Tolleranze estremamente strette. Il perche' e' il punto piu' importante dell'intero script – vedi il box qui sotto.
- Righe 16-18: opzioni `fsolve` esposte **esplicitamente** (`Display`, `MaxIterations`, `MaxFunctionEvaluations`, `FunctionTolerance`, `StepTolerance`) per riproducibilita'. `FunctionTolerance = StepTolerance = 1e-10`. Nota: **l'algoritmo non e' specificato**, quindi `fsolve` usa il default. Con 4 residui e 4 incognite il sistema e' quadrato, e il default e' `trust-region-dogleg`, con **Jacobiano alle differenze finite** (nessun Jacobiano analitico e' fornito).

**Possibile domanda d'esame** – Perche' `RelTol = 1e-10` e `AbsTol = 1e-12`? Non sono esagerate per una simulazione di ascesa? *Risposta*: Non sono le tolleranze di una *simulazione*, sono le tolleranze di un *residuo dentro un Newton*. `fsolve` deve annullare il residuo a  $1e-10$  (`FunctionTolerance`) e costruisce il Jacobiano per **differenze finite**, perturbando le incognite di circa  $\sqrt{\epsilon} \sim 1.5e-8$  in relativo. Se il residuo fosse affetto da rumore di integrazione dell'ordine di  $1e-6$  (tolleranza `ode45` di default), la derivata alle differenze finite sarebbe **rumore puro**: numeratore  $1e-6$ , denominatore  $1e-8$ , rapporto senza significato. La regola pratica e': la tolleranza dell'integratore deve essere **piu' stretta** della tolleranza sul residuo, non solo confrontabile. In piu' lo shooting sui costati e' intrinsecamente mal condizionato (piccole variazioni di `lambda_0` vengono amplificate esponenzialmente dall'integrazione in avanti nel residuo terminale), quindi il numero di condizionamento del Jacobiano e' grande e amplifica ulteriormente il rumore. Le tolleranze strette sono il prezzo del metodo indiretto.

---

## %% Storage (righe 20-24)

- Righe 23-24: `mf_results = nan(nQ, nyf)` e `sol_results = cell(nQ, nyf)`. L'inizializzazione a **NaN** (non a zero) e' deliberata: i punti in cui `fsolve` non converge restano NaN e vengono poi filtrati nei plot con la maschera `valid = ~isnan(...)` (righe 112, 147). Con zeri, i fallimenti apparirebbero come punti a massa finale nulla – un artefatto grafico indistinguibile da un risultato.
- `sol_results` e' un cell array perche' ogni elemento e' il vettore delle 4 incognite convergenti [`lam_vx0`; `lam_vy0`; `lam_y`; `tf`], che serve poi (i) come warm start per il  $Q$  successivo e (ii) per la ri-integrazione nei plot 1b e 1c.

---

## %% Solve BVP for each $y_f$ , sweeping $Q$ with continuation (righe 26-94)

E' il blocco che fa il lavoro. La struttura e' un **doppio loop con continuazione a due livelli**.

### Punto di partenza (righe 27-63)

```
[~, idx0] = min(abs(Q_vec - 3));      % start near Q = 3
if jj == 1
    z_guess = [0.6; 3.8; 14; 0.30];
```



```

else
    z_guess = sol_results{idx0, jj-1};    % warm start dal yf precedente
end

```

- Righe 32-33: la struct **p** viene (ri)costruita a ogni **jj** con **c** e **yf**. **p** e' una variabile **di script**, non locale a una funzione: viene mutata piu' volte e ricostruita anche nei blocchi dei plot (riga 131, riga 162). Funziona perche' le closure **@(z) shooting1(z, p, opts\_ode)** catturano **p per valore** al momento della creazione dell'handle, ma e' un pattern fragile: basta dimenticare di reimpostare un campo e si risolve silenziosamente il problema sbagliato.
- Riga 36: si parte da **Q** piu' vicina a 3. Sulla griglia **linspace(1.8, 7, 80)** il passo e'  $5.2/79 = 0.0658$ , quindi il nodo piu' vicino a 3 e' **Q ~ 2.985** (indice 19). Perche' proprio 3? Perche'  $T/W = c*Q = 0.6*3 = 1.8$  e' un rapporto spinta-peso "amichevole": abbastanza sopra 1 da non essere marginale, abbastanza basso da non produrre traiettorie estreme. E' il punto in cui il BVP e' meglio condizionato, quindi e' da li' che si parte a fare continuazione.
- Riga 39-40: **il guess iniziale a freddo** [0.6; 3.8; 14; 0.30]. E' l'unico numero "magico" dello script – e va difeso, perche' i costati **non hanno significato fisico diretto** e non si possono indovinare da considerazioni ingegneristiche ovvie. Cosa si puo' dire di questo guess:
  - **phi(0) = atan2(3.8, 0.6) ~ 81 deg**: spinta quasi verticale al decollo. Corretto.
  - **lam\_vy(tf) = 3.8 - 14\*0.30 = -0.4**, quindi **phi(tf) = atan2(-0.4, 0.6) ~ -34 deg**: la spinta finisce **sotto** l'orizzonte. Non e' assurdo: verso fine burn il razzo ha ancora **vy > 0** e deve azzerarla per soddisfare **vy(tf) = 0**, quindi deve spingere leggermente verso il basso.
  - **mf = 1 - Q\*tf = 1 - 2.985\*0.30 ~ 0.105**, dello stesso ordine dei valori finali riportati (**mf\* ~ 0.118** a **yf = 0.04**).
  - Il quarto residuo con questi numeri vale **H0 = -3.8 + T\*(|lam\_v0| - 1/c)** con **T = c\*Q ~ 1.79** e **|lam\_v0| = sqrt(0.6^2 + 3.8^2) = 3.847**, cioe' **H0 ~ -3.8 + 1.79\*(3.847 - 1.667) ~ +0.10** – piccolo rispetto ai termini individuali (~3.9). Il guess e' gia' quasi ammissibile sulla condizione di Hamiltoniana.
- (Questi valori li ho ricavati a mano dai numeri nel codice; lo script non li stampa.)
- Riga 43: per **jj >= 2** (**yf = 0.05, 0.06**) il guess non e' piu' a freddo: si prende la soluzione convergente **allo stesso Q** ma alla **yf** precedente. E' un secondo livello di continuazione, questa volta nella quota target.
- Righe 50-52: si costruisce **p.T = c \* p.Q** (la spinta e' **T = c\*Q**, coerente con **dm/dt = -Q** e **dm/dt = -T/c**) e si lancia il primo **fsolve**.
- Riga 54: **if ef > 0** – **ef** e' l'**exit flag**. **fsolve** restituisce **ef > 0** solo se ha convergito. Se fallisce sul punto di partenza (riga 61) l'intero **yf** viene saltato con **continue**: senza il punto di ancoraggio non c'e' continuazione possibile.
- Righe 55-57: **ri-integrazione**. Una volta convergito, si integra di nuovo con **ode\_burn** per estrarre la massa finale **Z(end,5)**. Nota: dato che **dm/dt = -Q** e' costante, si avrebbe **mf = 1 - Q\*tf** in forma chiusa. La ri-integrazione e' quindi ridondante per la sola massa – ma serve comunque il **Z** completo, ed e' un check implicito di consistenza dell'integratore.
- Riga 56: le condizioni iniziali [0;0;0;0;1;1] – razzo fermo nell'origine, **m0 = 1**, e **lam\_m0 = 1** (la normalizzazione dei costati, sesta componente).

## Sweep forward e backward (righe 65-93)

```

z_prev = z_sol;
for ii = idx0+1:nQ                % Q crescente
    ...
    [z_sol, ~, ef] = fsolve(@(z) shooting1(z,p,opts_ode), z_prev, opts_fs);
    if ef > 0
        ...
        z_prev = z_sol;          % <- aggiornato SOLO se converge
    end
end

```

- Righe 67-78 e 82-93: la **continuazione (omotopia)**, il vero segreto per far convergere questo shooting. Non si risolve ogni **Q** da zero: si risolve **Q\_{k+1}** partendo dalla soluzione convergente di **Q\_k**. Poiche' i due

problemi differiscono di poco ( $\Delta Q \sim 0.066$ ), il guess cade dentro il basin di attrazione di Newton. Un cold start su  $Q = 7$  quasi certamente divergerebbe.

- Si sweepa in **entrambe le direzioni** dal punto di ancoraggio: in avanti verso  $Q = 7$ , all'indietro verso  $Q = 1.8$ . La riga 81 resetta `z_prev` alla soluzione di ancoraggio prima di partire all'indietro – se non lo facesse, ripartirebbe dalla soluzione a  $Q = 7$ , lontanissima.
- Righe 76 e 91: `z_prev = z_sol` sta **dentro** `if ef > 0`. Questo è un dettaglio di robustezza importante: se un  $Q$  fallisce, la catena **non viene avvelenata** – il  $Q$  successivo riparte comunque dall'ultimo guess *buono*, non da un vettore divergente. Costa un buco (NaN) nella curva, non un crollo dell'intero sweep.
- **Nota di onestà**: a differenza del punto di partenza (riga 61), i fallimenti dentro gli sweep sono **completamente silenziosi**: nessun `fprintf`, nessun warning. Se una porzione della curva `mf(Q)` fosse mancante, lo si scoprirebbe solo guardando il grafico. È un limite reale dello script.

**Possibile domanda d'esame** – Cos'è la continuazione e perché è *necessaria*, non solo comoda, nel metodo indiretto? *Risposta*: La continuazione (o omotopia) consiste nell'immergere il problema in una famiglia parametrizzata (qui da  $Q$ ) e risolvere una catena di problemi vicini, usando ogni soluzione come guess iniziale del successivo. È necessaria perché nel metodo indiretto le incognite sono i costati, che non hanno interpretazione fisica: non esiste un modo ragionevole di indovinarli. Inoltre l'integrazione in avanti amplifica esponenzialmente le perturbazioni sui costati iniziali, quindi il basin di convergenza di Newton è piccolissimo. La continuazione garantisce che ogni solve parta già **dentro** il basin. È la ragione per cui lo script risolve prima a  $Q \sim 3$  (dove il problema è ben condizionato) e poi si muove a passi di 0.066.

---

## %% Optimal $Q^*$ per target altitude (righe 96-105)

- Righe 101-102: `[mf_star(jj), idx] = max(mf_results(:, jj))` – l'ottimo su  $Q$  non è trovato con un ottimizzatore, ma per **enumerazione sulla griglia**. La risoluzione su  $Q^*$  è quindi limitata dal passo della griglia,  $\sim 0.066$ . È accettabile perché la curva `mf(Q)` è piatta vicino al massimo (il report lo dice esplicitamente), ma va detto:  $Q^*$  riportato ha 4 cifre decimali (riga 103) che **non sono giustificate** dalla risoluzione della griglia.
- Riga 104: il payload `mf*(1+eta) - eta`, calcolato a posteriori (vedi sopra). Massimizzare `mf` è equivalente a massimizzare il payload perché la mappa è affine crescente in `mf` – quindi lo stesso  $Q^*$  ottimizza entrambi.

---

## %% PLOT 1a: Final mass vs $Q$ (righe 107-118)

- Riga 112: `valid = ~isnan(mf_results(:,jj))` – la maschera che esclude i  $Q$  non convergenti. Buca la curva invece di falsificarla.
- Il risultato atteso (e riportato nel README) è un **massimo interno** in  $Q$  per ciascuna  $y_f$ , che si sposta verso  $Q$  più basse al crescere della quota target.

---

## %% PLOT 1b: Velocity losses vs $Q$ for $y_f = 0.04$ (righe 120-153)

È il blocco più "fisico" dello script: decompone il delta-V perso.

```
ic8 = [0; 0; 0; 0; 1; 1; 0; 0];
[~, Z8] = ode45(@(t,z) ode_burn_losses(t,z,pp), [0 tf], ic8, opts_ode);
Wd_vec(ii) = Z8(end,7);
Wg_vec(ii) = Z8(end,8);
Wt_vec(ii) = c * log(1/mf_ii) - 1;    % total = DV_ideal - V_final
```

**Derivazione dell'identità di decomposizione.** Il delta-V ideale (Tsiolkovsky) è

$DV_{ideal} = c * \ln(m_0/m_f) = c * \ln(1/m_f) = \text{integrale di } (T/m) dt$

(l'ultima uguaglianza segue da  $dm/dt = -T/c$ ). La velocita' effettivamente ottenuta e' pero'  $v_f = 1$  (nondim). La differenza e' la perdita totale. Proiettando le equazioni del moto lungo il versore di velocita', con  $\psi = \text{atan2}(v_y, v_x)$  angolo di traiettoria e  $\phi$  angolo di spinta:

$d|v|/dt = (T/m) * \cos(\phi - \psi) - \sin(\psi)$

Integrando da fermo ( $|v|(0) = 0$ ) fino a  $|v|(t_f) = v_f$  e sottraendo da  $DV_{ideal}$ :

$DV_{ideal} - v_f = \text{integrale di } (T/m) * [1 - \cos(\phi - \psi)] dt \quad (= W_d)$   
 $\quad \quad \quad + \text{integrale di } \sin(\psi) dt \quad \quad \quad (= W_g)$

- $W_d$  (**misalignment loss**, riga 321 in `ode_burn_losses`): la frazione di spinta che non spinge lungo la velocita'. Si annulla **solo se**  $\phi = \psi$  identicamente, cioe' se la spinta e' sempre allineata alla velocita' – cosa che l'ottimo *non* fa, perche' deve anche curvare la traiettoria.
- $W_g$  (**gravity loss**, riga 322): **integrale di  $\sin(\psi)$  dt**. E' la componente di gravita' opposta al moto. Massima quando si vola verticale ( $\psi = 90 \text{ deg}$ ,  $\sin(\psi) = 1$ , si perde 1 g per ogni secondo), nulla quando si vola orizzontale. E' per questo che una Q bassa e' penalizzante: allunga il tempo di volo mentre si e' ancora ripidi.
- Riga 142:  $Wt\_vec(ii) = c * \log(1/mf\_ii) - 1$ . Il -1 e'  $v_f = 1$ , hard-coded. E' legittimo perche'  $v_x(t_f) = 1$ ,  $v_y(t_f) = 0$  sono **condizioni al contorno imposte** dallo shooting; ma se una soluzione fosse mal convergita,  $Wt$  verrebbe calcolata come se  $v_f$  fosse esattamente 1 e la discrepanza si nasconderebbe. **Diagnostico gratis**: nel grafico,  $Wd + Wg$  deve coincidere con  $Wt$ . Se le curve non si sovrappongono, o l'integrazione o la convergenza sono da rifare. Le tre curve sono calcolate per **strade diverse** (le prime due per quadratura lungo la traiettoria, la terza da una formula chiusa sulla massa), quindi il loro accordo e' una verifica non banale.

**Possibile domanda d'esame** – Perche'  $mf(Q)$  ha un massimo interno? Spiegalo con  $Wd$  e  $Wg$ . *Risposta*: La massa finale e'  $mf = \exp(-DV_{ideal}/c)$  e  $DV_{ideal} = v_f + Wd + Wg$ , quindi massimizzare  $mf$  equivale a **minimizzare la perdita totale**  $Wd + Wg$ . A Q bassa la spinta e' appena sopra il peso, il burn dura a lungo e il razzo resta ripido a lungo:  $Wg = \text{integrale di } \sin(\psi) dt$  esplode. A Q alta il burn e' brevissimo,  $Wg$  crolla, ma la spinta enorme deve essere fortemente disallineata dalla velocita' per curvare la traiettoria nel poco tempo disponibile, e  $Wd$  cresce. I due contributi vanno in direzioni opposte, quindi la somma ha un minimo interno:  $Q^*$ .

---

## %% PLOT 1c: Trajectory and angles for optimal Q (righe 155-205)

- Righe 157-158: si ricalcola  $[mf\_opt, idx\_opt] = \max(mf\_results(:, jj\_ref))$ , duplicando quello che le righe 101-102 avevano gia' fatto. Ridondanza innocua ma reale.
- Riga 169: integrazione **densa** su `linspace(0, tf, 500)`. Nota: passare un vettore di tempi a `ode45` **non cambia i passi di integrazione** (che restano adattivi e guidati da `RelTol/AbsTol`): cambia solo dove l'output viene interpolato (interpolazione densa di ordine 4). Quindi l'accuratezza non degrada, si ottengono solo curve lisce da plottare.
- Righe 180-189: ricostruzione degli angoli **a posteriori**, senza reintegrare i costati:  

```
lam_vy_k = z_opt(2) - z_opt(3) * T_sol(kk);  
phi_traj(kk) = atan2(lam_vy_k, z_opt(1));
```

cioe'  $\phi(t) = \text{atan2}(lam\_vy_0 - lam\_y * t, lam\_vx_0)$ . E' esattamente la stessa formula che `ode_burn` usa internamente (righe 20 e 24 di `ode_burn.m`) – e' la legge **linear-tangent**. Il fatto di poterla ricostruire fuori dall'integratore, con tre soli numeri, e' la manifestazione pratica del fatto che i costati di velocita' hanno soluzione in forma chiusa.
- Righe 184-188: guard su  $V_k > 1e-10$ . A  $t = 0$  la velocita' e' **esattamente zero** e  $\psi = \text{atan2}(0,0)$  non e' definito (in MATLAB darebbe 0, che sarebbe fuorviante: suggerirebbe volo orizzontale a un razzo fermo sulla rampa). Il fallback e'  $\psi = \phi$ , che e' la scelta fisicamente sensata: nell'istante in cui il veicolo

si stacca, la velocità nasce nella direzione dell'accelerazione netta, che è dominata dalla spinta. **Un guard analogo esiste anche in `ode_burn_losses`** (righe 307-312, ma con soglia  $1e-12$  invece di  $1e-10$ ), ed è lì che davvero conta: senza, `Wd` avrebbe un integrando indefinito all'istante iniziale.

- Righe 191-203: le due figure (traiettoria  $x-y$  con `axis equal`, e i due angoli  $\phi/\psi$  in gradi vs tempo). Il comportamento atteso di  $\phi$  è la discesa monotona da quasi verticale a quasi orizzontale (ed eventualmente sotto lo zero): è la legge linear-tangent,  $\tan(\phi)$  lineare in  $t$ .

---

## %% EXPORT FIGURES (righe 207-223)

- Riga 208: `fig_dir` costruito con `fileparts(mfilename('fullpath'))` – quindi le figure finiscono in `HM1/figures/` **indipendentemente dalla working directory**.
  - Riga 211: `findobj(groot, 'Type', 'figure')` prende **tutte** le figure aperte. Vedi la nota sul `close all` di riga 6: è quello che garantisce che siano solo le proprie.
  - Righe 215-220: `theme(fig, 'light')` dentro un `try/catch`, con fallback `fig.Color = 'w'` per MATLAB pre-R2025a. Serve a forzare figure su fondo chiaro anche se il desktop MATLAB è in dark mode – altrimenti le PNG esportate finirebbero nel report con sfondo scuro.
  - Riga 210 + 222: `slugify` produce nomi tipo `task1_task_1a_final_mass_vs_q.png` (il “task1\_” del prefisso più il “Task 1a” slugificato dal Name della figura – da cui la ripetizione). Sono i nomi che il report LaTeX si aspetta.
  - **Deviazione dalle convenzioni della repo:** il `CLAUDE.md` chiede SVG (vettoriale) + PNG (preview). Qui si esporta **solo PNG a 200 dpi**. Nel report va bene lo stesso, ma vale la pena saperlo.
- 

## shooting1 – il residuo del BVP (righe 227-274)

È la funzione **più importante dello script**: è la definizione operativa del Boundary Value Problem.

```
function res = shooting1(z0, p, opts_ode)
    lam_vx0 = z0(1); lam_vy0 = z0(2);
    lam_y    = z0(3); tf      = z0(4);
    ...
    ic = [0; 0; 0; 0; 1; 1]; % state + lam_m0 = 1 (normalization)
    [~, Z] = ode45(@ode_burn(t, z, pp), [0 tf], ic, opts_ode);
    zf = Z(end,:);

    Lam0 = sqrt(lam_vx0^2 + lam_vy0^2);
    H0    = -lam_vy0 + p.T * (Lam0 - 1/p.c);

    res = [zf(2) - p.yf; % y(tf) = yf
          zf(3) - 1;    % vx(tf) = 1
          zf(4);        % vy(tf) = 0
          H0];          % H(0) = 0 (free final time)
end
```

- Riga 227: firma. `z0` sono le **4 incognite dello shooting** `[lam_vx0; lam_vy0; lam_y; tf]`; `p` porta `c`, `T`, `Q`, `yf`. Chiamata solo da `fsolve`.
- Riga 238: di nuovo **nessun blocco arguments, per scelta** – `shooting1` gira dentro il loop di `fsolve` (decine di valutazioni per iterazione, per costruire il Jacobiano alle differenze finite, moltiplicato per 240 solve).
- Righe 245-248: **guard sul tempo finale**. Se `tf <= 0` o `tf > 2`, si restituisce un residuo enorme `1e6*ones(4,1)`. **Nota di onestà: è un hack, non una barriera liscia**. Il residuo diventa discontinuo sul bordo della regione ammissibile; se `fsolve` ci finisce dentro con una perturbazione di differenze finite, la

colonna corrispondente del Jacobiano diventa spazzatura ( $\sim 1e6/1e-8$ ). In pratica non dà problemi perché la continuazione tiene **tf** ben all'interno (dell'ordine di 0.1-0.35 nondim), ma è fragile. Nota anche che il limite superiore **tf** > 2 (cioè  $\sim 1590$  s dimensionali) è **arbitrario**: una soluzione con **tf** > 2 verrebbe rifiutata a priori.

- Righe 250-253: si copiano le tre incognite dei costati dentro **pp**, che diventa la struct passata a **ode\_burn**.
- Riga 255: le condizioni iniziali. Le prime cinque sono la fisica ( $x = y = vx = vy = 0, m_0 = 1$ ). La sesta è **lam\_m0 = 1**: **la normalizzazione dei costati**.
- Righe 257-263: **try/catch** attorno a **ode45** con lo stesso residuo-penalità **1e6**. Protegge dai casi in cui l'integratore esplode (per esempio  $m \rightarrow 0$  se  $Q \cdot tf > 1$ ). Stessa critica di prima: è una discontinuità, non una barriera.
- Righe 265-268: **la condizione di Hamiltoniana nulla, imposta a t0**. Derivazione: partendo dall'Hamiltoniana ottima

$$H^* = \text{lam\_y} \cdot v_y + (T/m) \cdot |\text{lam\_v}| - \text{lam\_vy} - \text{lam\_m} \cdot Q$$

e valutandola a  $t = 0$  con  $vx_0 = vy_0 = 0, m_0 = 1, \text{lam\_m0} = 1$  e  $Q = T/c$ :

```
il termine lam_y*v_y -> 0          (perche' vy0 = 0)
il termine (T/m)*|lam_v| -> T*|lam_v0| (perche' m0 = 1)
il termine -lam_vy -> -lam_vy0
il termine -lam_m*Q -> -T/c        (perche' lam_m0 = 1 e Q = T/c)
```

da cui

$$\begin{aligned} H(0) &= -\text{lam\_vy0} + T \cdot |\text{lam\_v0}| - T/c \\ &= -\text{lam\_vy0} + T \cdot (|\text{lam\_v0}| - 1/c) = 0 \end{aligned}$$

che è **esattamente** la riga 268. Notare che la parentesi ( $|\text{lam\_v0}| - 1/c$ ) è il valore iniziale della **funzione di switching**  $S = |\text{lam\_v}|/m - \text{lam\_m}/c$ , valutata con  $m_0 = 1$  e  $\text{lam\_m0} = 1$ .  $S(0) > 0$  significa “motore acceso” – coerente con un arco propulso che parte da terra. ( $S$  diventa una quantità di primo piano nel Task 3, dove il coast inizia quando  $S = 0$ .)

- **Perché  $H = 0$  a  $t_0$  e non a  $tf$ ?** La condizione di trasversalità per tempo finale libero è  $H(tf) = 0$ . Ma il sistema è **autonomo**, quindi  $H$  è costante lungo la traiettoria ottima, e imporre  $H(0) = 0$  è equivalente. Il vantaggio è enorme: a  $t_0$  la formula è **puramente algebrica** e non contiene errore di integrazione, mentre  $H(tf)$  andrebbe valutata sullo stato integrato (e su **lam\_m(tf)**, che ha accumulato errore per tutto l'arco).
- Righe 270-273: **i quattro residui**:
  - **zf(2)** – **p.yf** -> vincolo di quota  $y(tf) = y_f$
  - **zf(3)** – **1** -> vincolo di velocità orizzontale  $vx(tf) = 1$  (velocità orbitale)
  - **zf(4)** -> vincolo  $vy(tf) = 0$  (iniezione orizzontale, apogeo)
  - **H0** -> condizione di tempo finale libero

**Sistema quadrato: 4 incognite, 4 residui.** Notare cosa *non c'è*: non c'è nessun vincolo su  $x(tf)$  (la posizione orizzontale è libera:  $\text{lam\_x} = 0$  per trasversalità), e non c'è nessun residuo su  $\text{lam\_m}(tf) = 1$  (assorbito dalla normalizzazione  $\text{lam\_m0} = 1$ ). Le due “migliorie” numeriche – normalizzazione dei costati e  $H$  imposta a  $t_0$  – tolgono ciascuna **una incognita e una condizione**, portando il BVP da 6x6 a 4x4. E **lam\_m non compare mai nel residuo**: **zf(6)** non è letto.

**Possibile domanda d'esame** – Dove finiscono le condizioni di trasversalità  $\text{lam\_x}(tf) = 0$  e  $\text{lam\_m}(tf) = 1$ ? Non compaiono nel residuo. *Risposta*: Entrambe sono state usate *analiticamente* prima di scrivere il codice, non numericamente.  $\text{lam\_x}(tf) = 0$  (perché  $x(tf)$  è libero e non entra nel costo), combinata con  $\text{lam\_x\_dot} = 0$ , dà  $\text{lam\_x} = 0$  **su tutto l'arco**: sparisce dall'Hamiltoniana e da **ode\_burn**.  $\text{lam\_m}(tf) = 1$  è invece **sostituita** dalla normalizzazione  $\text{lam\_m0} = 1$ : poiché l'Hamiltoniana è omogenea di grado 1 in  $\text{lam\_m}$ , i costati sono determinati solo a meno di un fattore di scala positivo, e la trasversalità su  $\text{lam\_m}$  serviva solo a fissarlo. Fissarlo all'istante iniziale

invece che finale e' equivalente e piu' comodo (rimuove una incognita e una condizione). Conseguenza da sapere: `lam_m(tf)` calcolato dal codice **non vale 1**.

---

## set\_costates (righe 276-287)

- Utility di tre righe: copia `z_sol(1:3)` nei campi `lam_vx0`, `lam_vy0`, `lam_y` della struct. Esiste solo per evitare di ripetere le tre assegnazioni nei cinque punti dello script che ri-integrano una soluzione convergente (righe 55, 72, 87, 133, 164). Nota che **ignora `z_sol(4)` (`tf`)**, che va letto separatamente dal chiamante (righe 134, 165).
- 

## ode\_burn\_losses (righe 289-323)

- E' `ode_burn` con **due integratori di perdita accodati**. Le prime sei componenti (righe 314-320) sono una **copia letterale** di `ode_burn.m` (righe 27-33).
  - Riga 321: `dz(7) = (T/m)*(1 - cos(phi - psi)) -> Wd`, la misalignment loss. Nota che `1 - cos(.) >= 0` sempre, quindi `Wd` e' **monotona crescente**: e' una perdita, non puo' recuperare. E' nulla solo dove `phi = psi`.
  - Riga 322: `dz(8) = sin(psi) -> Wg`, la gravity loss. Non e' monotona: se il razzo scendesse (`psi < 0`), `sin(psi) < 0` e la "perdita" diventerebbe un guadagno. Nell'ascesa `psi > 0` quasi ovunque, quindi in pratica cresce. Notare che **Wg non dipende dalla spinta ne' dalla massa**: e' pura geometria della traiettoria – quanto tempo si passa a volare ripidi.
  - Riga 307-312: il guard `V > 1e-12` con fallback `psi = phi` (stessa struttura del guard di riga 184, ma con soglia `1e-12` invece di `1e-10`: le due soglie sono scelte indipendenti, non una costante condivisa). Qui e' **necessario**, non cosmetico: senza, `psi` sarebbe indefinito a `t = 0` e l'integrando di `Wd` inizierebbe da un valore arbitrario.
  - **Nota di onesta' – duplicazione del codice**. `ode_burn_losses` replica il RHS di `ode_burn` invece di chiamarlo. Se un giorno si cambiasse `ode_burn.m` (per esempio aggiungendo il drag), questa copia **non** si aggiornerebbe e i due divergerebbero silenziosamente. La ragione della scelta e' presumibilmente la performance (evitare una chiamata di funzione e la ricostruzione del vettore nel hot loop), ma il costo e' un rischio di manutenzione reale, e va dichiarato.
- 

## Possibili domande d'esame

**D: Elenca le quattro incognite dello shooting e i quattro residui, e spiega perche' il sistema e' quadrato.** R: Incognite: `lam_vx0`, `lam_vy0`, `lam_y`, `tf`. Residui: `y(tf) - yf`, `vx(tf) - 1`, `vy(tf)`, `H(0)`. Il sistema di partenza avrebbe 5 costati iniziali + `tf = 6` incognite, e 6 condizioni (3 vincoli terminali di stato + `lam_x(tf) = 0` + `lam_m(tf) = 1` + `H(tf) = 0`). Due riduzioni analitiche lo portano a 4x4: (i) `lam_x = 0` identicamente (per trasversalita' + `lam_x_dot = 0`), che elimina un'incognita e la sua condizione; (ii) la normalizzazione `lam_m0 = 1`, lecita perche' `H` e' omogenea di grado 1 in `lambda`, che elimina `lam_m0` come incognita e rende superflua la trasversalita' `lam_m(tf) = 1`. Restano 4 e 4, e `lam_m` non entra mai nel residuo.

**D: Perche' `H = 0` viene imposta a `t = 0` e non a `t = tf`, dove la condizione di trasversalita' la richiede?** R: Perche' il sistema e' autonomo (l'Hamiltoniana non dipende esplicitamente dal tempo), quindi `H` e' un integrale primo: `H(t) = cost`. Se `H(tf) = 0` allora `H(0) = 0`, e viceversa: le due condizioni sono **equivalenti**. Ma a `t = 0` lo stato e' noto esattamente (`vx = vy = 0`, `m = 1`, `lam_m = 1`), quindi `H(0)` si scrive in forma **algebraica chiusa**, `H(0) = -lam_vy0 + T*(|lam_v0| - 1/c)`, senza errore di integrazione. Valutarla a `tf` significherebbe usare lo stato integrato – inquinato dall'errore numerico accumulato – e anche `lam_m(tf)`, che e' il costato con l'errore maggiore. Imporre la condizione dove e' esatta migliora il condizionamento del residuo.

**D: Perché RelTol = 1e-10 / AbsTol = 1e-12 e non le tolleranze di default di ode45?** R: Perché l'output di ode45 è il residuo che fsolve deve annullare a 1e-10, e fsolve costruisce il Jacobiano per differenze finite perturbando le incognite di  $\sim \sqrt{\text{eps}} \sim 1.5e-8$ . Se il residuo avesse rumore di integrazione  $\sim 1e-6$  (default RelTol = 1e-3), la derivata numerica sarebbe  $1e-6/1e-8 = 100$  volte il rumore: il Jacobiano sarebbe puro rumore e Newton non convergerebbe (o convergerebbe a una soluzione fasulla). La regola: la tolleranza dell'integratore deve stare **sotto** la tolleranza sul residuo, con margine. Si aggiunge il fatto che lo shooting sui costati è mal condizionato per natura, quindi il Jacobiano amplifica ulteriormente ogni rumore.

**D: Che cos'è la strategia di continuazione qui, e cosa succederebbe senza?** R: Si risolve prima il BVP a  $Q \sim 3$  ( $T/W = 1.8$ , punto ben condizionato) partendo dal guess a freddo [0.6; 3.8; 14; 0.30], poi si sweepa **Q in avanti e all'indietro** usando ogni soluzione convergente come guess della successiva (passo  $\sim 0.066$ ). Senza continuazione bisognerebbe indovinare i costati a freddo per ognuno degli 80 Q e 3 yf – e i costati non hanno significato fisico, quindi non si indovinano. Il basin di convergenza di Newton per uno shooting indiretto è molto piccolo (l'integrazione in avanti amplifica esponenzialmente le perturbazioni su  $\lambda_0$ ), quindi la maggior parte dei cold start divergerebbe. Il codice fa continuazione anche su yf: la prima soluzione per yf = 0.05 è presa da quella convergente a yf = 0.04 alla stessa Q.

**D: Come si decompone la perdita di velocità e perché Wd e Wg vanno in direzioni opposte con Q?** R:  $DV_{ideal} = c \ln(1/mf) = v_f + Wd + Wg$ , con  $Wd = \int (T/m) * [1 - \cos(\phi - \psi)] dt$  (misalignment) e  $Wg = \int \sin(\psi) dt$  (gravità), ottenute proiettando le equazioni del moto sul versore di velocità. Al crescere di Q il burn si accorcia (perché  $T/W$  cresce), quindi Wg, che è un integrale nel tempo di una quantità limitata da 1, **diminuisce**. Ma per curvare la traiettoria dalla verticale all'orizzontale nel poco tempo disponibile, la spinta deve essere fortemente disallineata dalla velocità, quindi  $1 - \cos(\phi - \psi)$  cresce e Wd **aumenta**. Il minimo della somma è  $Q^*$ , che è anche il massimo di mf (perché  $mf = \exp(-DV_{ideal}/c)$  è decrescente nella perdita totale).

**D: Quali sono i punti deboli / gli hack di questo script?** R: (i) I guard  $tf \leq 0 \ || \ tf > 2$  e il try/catch attorno a ode45 restituiscono un residuo costante 1e6: sono **barriere discontinue**, non lisce, e possono corrompere il Jacobiano alle differenze finite se fsolve ci finisce dentro. In pratica la continuazione le evita. (ii) Il limite  $tf > 2$  è arbitrario. (iii) I fallimenti di fsolve dentro gli sweep sono **silenziosi** (lasciano solo un NaN). (iv)  $Q^*$  è trovato per enumerazione su griglia (passo 0.066), quindi le 4 cifre decimali stampate non sono giustificate. (v) **ode\_burn\_losses duplica** il RHS di ode\_burn invece di chiamarlo: rischio di divergenza in manutenzione. (vi) La massa finale si potrebbe ottenere in forma chiusa ( $mf = 1 - Q*tf$ , perché Q è costante) invece di ri-integrare. (vii) Non c'è nessun guard su  $m > 0$ . (viii) Le figure sono esportate solo in PNG, mentre il CLAUDE.md chiede anche SVG.

# HM1/main\_\_task2.m

## Ruolo del file nel progetto

`main_task2.m` risolve la **seconda variante** della missione di HM1: prima di accendere il gravity turn ottimo, il lanciatore deve compiere una **salita verticale** fino a una quota assegnata  $y1 = 1e-4$  (adimensionale; circa 620 m dimensionali secondo il README di HM1, con  $L_{rif} = V_{rif}^2/g \sim 6200$  km). La missione diventa quindi a due fasi: **salita verticale**  $\rightarrow$  **arco propulso ottimo**. Lo scopo dichiarato nell'intestazione (righe 1-3) e' confrontare il payload con quello del Task 1 (nessuna salita verticale).

La differenza sostanziale rispetto a `main_task1.m` **non e' nella matematica del problema di controllo ottimo**, ma nel **punto di partenza dell'arco ottimizzato**. Nel Task 1 l'arco propulso parte da  $(x, y, vx, vy, m) = (0, 0, 0, 0, 1)$ ; qui parte da  $(0, y1, 0, vy1, m1)$ , dove  $vy1$  e  $m1$  sono il risultato dell'integrazione della salita verticale. La fase 1 **non ha gradi di liberta'** (l'angolo di spinta e' imposto a  $\phi = 90$  deg e la durata e' fissata dall'evento  $y = y1$ ), quindi non c'e' nulla da ottimizzare in fase 1 e i suoi costati sono irrilevanti: non esiste nessuna condizione di continuita' dei costati alla giunzione. Solo lo **stato** viene trasferito da fase 1 a fase 2.

Di conseguenza il vettore delle incognite dello shooting resta **identico** a quello del Task 1 –  $[lam\_vx0, lam\_vy0, lam\_y, t\_burn]$ , 4 incognite e 4 residui – e cambia solo (a) la condizione iniziale passata a `ode45`, e (b) la forma algebrica della condizione  $H(0) = 0$ , che ora contiene i termini  $lam\_y*vy1$  e  $T/m1$  (nel Task 1 valevano zero e  $T$ , perche'  $vy0 = 0$  e  $m0 = 1$ ).

Il file dipende da `ode_burn.m` (RHS condiviso stato+costato dell'arco propulso) e definisce tre funzioni locali: `ode_vertical`, `event_altitude`, `shooting2`. Non e' chiamato da nessun altro script: e' un entry point standalone che, come tutti i `main_task*.m` di HM1, esporta le figure in `HM1/figures/` con prefisso `task2_`.

---

## Richiamo: le condizioni necessarie (PMP) usate dall'arco propulso

Sono le stesse per tutti e quattro i task e sono cablate dentro `ode_burn.m`. Dinamica adimensionale (piana, Terra piatta, senza drag,  $g = 1$ ,  $T = c*Q$  costante):

```
x_dot = vx
y_dot = vy
vx_dot = (T/m)*cos(phi)
vy_dot = (T/m)*sin(phi) - 1
m_dot = -Q
```

Costo: massimizzare  $m(tf)$ . Hamiltoniana (convenzione del **massimo**, quella usata dal codice):

```
H = lam_x*vx + lam_y*vy + lam_vx*(T/m)*cos(phi)
    + lam_vy*((T/m)*sin(phi) - 1) - lam_m*Q
```

Equazioni dei costati ( $lam\_dot = -dH/dstato$ ):

```
lam_x_dot = 0
lam_y_dot = 0
```



```
lam_vx_dot = -lam_x
lam_vy_dot = -lam_y
lam_m_dot = +(T/m^2)*|lam_v|          con |lam_v| = sqrt(lam_vx^2+lam_vy^2)
```

Conseguenze usate dal codice:

- $x(tf)$  e' **libera** (nessuna condizione terminale su  $x$ ) -> trasversalita'  $lam_x(tf) = 0$  -> essendo  $lam_x$  costante,  $lam_x == 0$  ovunque ->  $lam_vx$  e' **costante** =  $lam_{vx0}$ .
- $lam_y$  e' costante (incognita), quindi  $lam_{vy}(t) = lam_{vy0} - lam_y*t$  e' **lineare** nel tempo. Questo e' esattamente cio' che fa `ode_burn.m` (riga 20).
- Massimizzando  $H$  rispetto a  $\phi$ :  $\cos(\phi) = lam_{vx}/|lam_v|$ ,  $\sin(\phi) = lam_{vy}/|lam_v|$ , cioe'  
 $\tan(\phi) = (lam_{vy0} - lam_y*t) / lam_{vx0}$   
cioe' la **legge della tangente lineare** (caso degenerare della bilinear-tangent perche'  $lam_x = 0$ ).
- $m(tf)$  e' libera e coincide col costo ->  $lam_m(tf) = 1$ .
- $tf$  libero + sistema autonomo ->  $H(t) = 0$  **per ogni  $t$**  (non solo in  $tf$ ).

## Intestazione e parametri (righe 1-17)

```
c = 0.6;
eta = 0.1;
y1 = 0.0001; % vertical climb altitude
yf = 0.04; % target orbit altitude
Q = 2;
```

- Righe 8-12: dati fissi.  $c = 0.6$  velocita' efficace di scarico adimensionale,  $\eta = 0.1$  coefficiente strutturale  $ms/mp$ ,  $yf = 0.04$  quota di iniezione.  $Q = 2$  e' il valore nominale della traccia – **non** il  $Q^*$  ottimo trovato nel Task 1 (che per  $yf = 0.04$  vale circa 2.52 secondo il README): il Task 2 non riesgue lo sweep su  $Q$ .
- Riga 10:  $y1 = 1e-4$  e' un **dato di progetto imposto**, non una variabile di ottimizzazione. Se lo si volesse ottimizzare servirebbe una condizione di interior-point sui costati alla giunzione – qui non serve proprio perche'  $y1$  e' fisso.
- Righe 14-17: tolleranze. `ode45` a  $RelTol=1e-10$ ,  $AbsTol=1e-12$  e `fsolve` a  $1e-10$  su funzione e passo: e' la ricetta standard del metodo indiretto, dove i residui dello shooting sono estremamente sensibili ai costati iniziali (il sistema aggiunto e' instabile in avanti) e un'integrazione lasca produce gradienti numerici privi di senso.

## Fase 1 – salita verticale (righe 19-37)

```
T = c * Q;
ic_vert = [0; 0; 1];
opts_vert = odeset('RelTol', 1e-12, 'AbsTol', 1e-14, ...
    'Events', @(t,z) event_altitude(t, z, y1));
[T_vert, Z_vert] = ode45(@(t,z) ode_vertical(t,z,T,Q), ...
    [0 1], ic_vert, opts_vert);
```

- Riga 20:  $T = c*Q = 1.2$ . In unita' adimensionali il peso iniziale e'  $m_0*g = 1$ , quindi  $T$  **coincide numericamente col rapporto  $T/W$  al liftoff**: per questo la riga 22 stampa  $T$  sotto l'etichetta  $T/W$ . E' corretto solo a  $t = 0$ ; man mano che  $m$  cala,  $T/W$  cresce. Condizione di decollo:  $T > 1$  cioe'  $Q > 1/c = 1.667$  (con  $Q = 2$  siamo dentro).

- Righe 24-25: stato ridotto  $z = [y; v_y; m]$  – durante la salita verticale  $x$  e  $v_x$  restano identicamente nulli, quindi si integra solo il sottospazio utile.
- Righe 26-27: tolleranze ancora **piu' strette** ( $1e-12/1e-14$ ) che nell'arco propulso. Motivo:  $v_{y1}$  e  $m1$  sono le condizioni iniziali del BVP a valle; un errore su di esse si propaga in un errore sui residui, ed e' un errore che `fsolve` non puo' compensare (non e' una delle sue incognite).
- Riga 28: `tspan = [0 1]` con evento terminale. **Punto fragile:** il codice non verifica che l'evento sia effettivamente scattato. Se  $T \leq 1$  ( $Q < 1/c$ ) il veicolo non decolla, l'evento non si attiva mai e lo script prosegue silenziosamente con  $t1 = 1$  e uno stato assurdo. Non c'e' guardia.
- Righe 30-33: si estraggono  $t1$ ,  $y_1 = Z\_vert(end,1)$  (la quota **davvero raggiunta**, non  $y1$  nominale),  $v_{y1}$ ,  $m1$ . Nota: `main_task3.m` in questo punto usa invece  $y1$  nominale; e' un'incoerenza tra i due script, di impatto numerico trascurabile (dell'ordine della tolleranza dell'evento).

**Possibile domanda d'esame** – perche' la salita verticale non produce nessuna condizione sui costati alla giunzione  $t1$ ? *Risposta:* perche' in fase 1 non ci sono gradi di liberta':  $\phi$  e' imposto a  $90\ deg$  e la durata  $t1$  e' determinata dal vincolo  $y(t1) = y1$ . I costati servono a esprimere la stazionarieta' rispetto a un controllo libero; se il controllo non e' libero, i moltiplicatori della fase 1 non entrano in nessuna condizione necessaria. La giunzione trasferisce quindi solo lo **stato** ( $0, y1, 0, v_{y1}, m1$ ), e i costati dell'arco propulso ripartono come incognite nuove di zecca. Sarebbero servite condizioni di interior-point solo se  $y1$  (o  $t1$ ) fosse stata una variabile da ottimizzare.

## Fase 2 – impostazione e soluzione del BVP (righe 39-64)

```
p.x0 = 0;    p.y0 = y_1;
p.vx0 = 0;   p.vy0 = vy1;    p.m0 = m1;

z_guess = [0.6; 3.8; 14; 0.30];
[z_sol, ~, ef] = fsolve(@(z) shooting2(z,p,opts_ode), ...
                        z_guess, opts_fs);
```

- Righe 44-52: la struct `p` porta i parametri fisici e lo stato iniziale dell'arco propulso. E' l'unico canale attraverso cui la fase 1 comunica con la fase 2.
- Righe 41-42 (commento): incognite `[lam_vx0, lam_vy0, lam_y, t_burn]`, residui `[y(tf)-yf, vx(tf)-1, vy(tf), H(0)]`. **Identico al Task 1 come cardinalita'**: la normalizzazione  $lam_{m0} = 1$  rimuove una incognita e la condizione  $lam_m(tf) = 1$  viene sostituita da essa (vedi sotto, `shooting2`).
- Riga 55: `z_guess = [0.6; 3.8; 14; 0.30]` e' **letteralmente lo stesso guess cablato in main\_task1.m (riga 40)**. Questo e' l'unico "warm start" presente: un travaso manuale del guess da un task all'altro. **Non c'e' continuazione** in questo script – nessun loop che riparte dalla soluzione precedente. Funziona perche'  $y1 = 1e-4$  e' minuscolo (1/400 della quota finale), quindi la soluzione del Task 2 e' una perturbazione piccola di quella del Task 1. Nota pero' che il guess era tarato su  $Q \sim 3$  (nel Task 1 lo sweep parte dal  $Q$  piu' vicino a 3) mentre qui  $Q = 2$ .
- Righe 60-64: se `fsolve` fallisce ( $ef \leq 0$ ) si riprova **una sola volta** con un secondo guess `[0.4; 3.0; 10; 0.35]`. E' un multi-start rudimentale, non una continuazione: se anche il secondo fallisce, lo script stampa un errore (riga 155) e non produce risultati.
- Lettura fisica del guess: con  $lam_{vy0} = 3.8$  e  $lam_y = 14$ , dopo  $t = 0.30$  si ha  $lam_{vy} = 3.8 - 14 \cdot 0.30 = -0.4$ , quindi  $\phi = atan2(lam_{vy}, 0.6)$  scende da circa  $+81\ deg$  a circa  $-34\ deg$ : e' proprio la pitch-over di un gravity turn. Un guess con  $lam_y < 0$  darebbe una spinta che si alza nel tempo – non convergerebbe mai.

**Possibile domanda d'esame** – se il metodo indiretto e' cosi' sensibile, perche' qui basta un guess cablato e nel Task 1 serviva la continuazione? *Risposta:* nel Task 1 bisogna risolvere il BVP per 80 valori di  $Q$  e 3 valori di  $yf$ : quando  $Q$  si allontana dal valore "amichevole" il bacino di convergenza si restringe e un guess fisso fallisce, quindi si usa la continuazione (warm start dalla soluzione al  $Q$  vicino). Nel Task 2 c'e' **una sola** soluzione da trovare, a  $Q = 2$  fissato, e la salita verticale e' cosi'

corta che la soluzione e' vicinissima a quella del Task 1: il guess del Task 1 cade gia' dentro il bacino di attrazione.

---

## Estrazione della soluzione e risultati (righe 66-87)

- Righe 68-71: i tre costati vengono ricopiati dentro `pp`, che e' la struct che `ode_burn` si aspetta. La riga 72 estrae invece `t_burn` come variabile locale ordinaria: **non** e' un campo di `pp` (`ode_burn` legge solo `T`, `Q`, `c`, `lam_vx0`, `lam_vy0`, `lam_y`), serve come estremo del `tspan` alla riga 75.
- Riga 74: `ic2 = [x0; y0; vx0; vy0; m0; 1]` – il sesto stato e' `lam_m`, inizializzato a **1**: e' la normalizzazione, non una condizione fisica.
- Riga 75: reintegrazione dell'arco su una griglia densa `linspace(0,t_burn,500)` solo per i grafici (la soluzione era gia' nota da `fsolve`).
- Riga 78: la durata totale e' `tf_total = t1 + t_burn` (le righe 80-84 si limitano a stamparla insieme agli altri risultati). `t1` **non** e' un'incognita dello shooting: e' un dato prodotto dalla fase 1.
- Riga 87: payload `mu = mf*(1+eta) - eta`. Derivazione: a fine missione la massa residua e' `mf = ms + mu`, con struttura `ms = eta*mp` e propellente `mp = m0 - mf = 1 - mf` (adimensionale, `m0 = 1`). Sostituendo: `mu = mf - eta*(1 - mf) = mf*(1+eta) - eta`. E' la stessa formula del Task 1, quindi il confronto di payload richiesto dall'intestazione e' un confronto diretto di `mf`.

---

## Plots (righe 89-153)

- Righe 99-106: traiettoria `y(x)`: rosso il tratto verticale (`x == 0`), blu l'arco propulso.
- Righe 108-126: **inset di zoom**. Serve perche' `y1 = 1e-4` e' `1/400` di `yf = 0.04`: a scala piena la salita verticale e' un punto. L'inset mostra il ginocchio di pitch-over. Il rettangolo tratteggiato (righe 117-118) marca la regione ingrandita.
- Righe 129-140: ricostruzione degli angoli lungo l'arco propulso. `lam_vy_k = z_sol(2) - z_sol(3)*T2(kk)` ricalcola il costato **analiticamente** (e' lineare: non serve integrarlo), poi `phi = atan2(lam_vy_k, lam_vx0)` e `psi = atan2(vy, vx)` (angolo del vettore velocita'). La guardia `V_k > 1e-10` evita l'`atan2(0,0)` al primo istante, dove `vx = 0` e `vy = vy1` (in realta' `vy1 > 0`, quindi `V_k` non e' nullo: la guardia e' un residuo del Task 1, dove la velocita' iniziale e' davvero zero).
- Righe 146-147: nella fase verticale si tracciano `phi = 90 deg` e `psi = 90 deg` sovrapposte. Il salto visibile in `t1` tra `90 deg` e `phi(0) = atan2(lam_vy0, lam_vx0)` e' **fisico e atteso**: il controllo e' discontinuo alla giunzione perche' la fase 1 e' imposta, non ottima. Solo se `y1 -> 0` il salto si annulla.

---

## Export figure (righe 158-174)

- Righe 159-160: cartella `HM1/figures/`, creata se manca.
- Righe 166-171: `theme(fig,'light')` forza il tema chiaro (altrimenti in dark mode le PNG escono con sfondo nero e finiscono cosi' nel report LaTeX); fallback su `Color='w'` per MATLAB pre-R2025a.
- Riga 172-173: `exportgraphics` a 200 dpi, nome `task2_<nome-figura-slugificato>.png`.

---

## ode\_vertical (righe 178-187)

```
function dz = ode_vertical(~, z, T, Q)
    vy = z(2); m = z(3);
    dz = [vy; T/m - 1; -Q];
end
```

- Riga 178: firma. Chiamata solo da ode45 alla riga 28.
- Riga 186: e' la dinamica con  $\phi = \pi/2$  **sostituito a mano**:  $v_{x\_dot} = (T/m)*\cos(90 \text{ deg}) = 0$  (quindi  $x$  e  $v_x$  non compaiono),  $v_{y\_dot} = (T/m)*\sin(90 \text{ deg}) - g = T/m - 1$  (con  $g = 1$ ),  $m\_dot = -Q$ .
- Non ci sono costati: la fase non e' ottimizzata.
- Questa funzione e' **duplicata identica** in `main_task3.m` (righe 221-230). Nessuna delle due e' fattorizzata in un file condiviso come `ode_burn.m`.

## event\_altitude (righe 189-199)

```
value = z(1) - y_target;
isterminal = 1;
direction = 1;
```

- Riga 196: la funzione evento si annulla quando  $y = y_1$ .
- Riga 197: `isterminal = 1` -> l'integrazione si ferma.
- Riga 198: `direction = 1` -> si intercetta solo l'attraversamento **in salita**. Se il veicolo non decollasse ( $T < 1$ ),  $y$  non salirebbe mai e l'evento non scatterebbe: da qui la fragilita' segnalata sopra.
- Anche questa e' duplicata in `main_task3.m` (righe 232-242).

## shooting2 (righe 201-247)

E' il cuore del file: la funzione residuo che `fsolve` azzerava.

- Riga 201: firma `res = shooting2(z0, p, opts_ode)`.  $z_0$  sono le 4 incognite,  $p$  porta parametri e stato iniziale dell'arco. Nessun blocco `arguments`: come dichiarato ai commenti (riga 211) e nel `CLAUDE.md` del repo, la funzione gira dentro il loop di `fsolve` (migliaia di chiamate) e la validazione starebbe fuori posto.
- Righe 213-216: unpack delle incognite `[lam_vx0; lam_vy0; lam_y; t_burn]`.
- Righe 218-221: **box constraint via penalita'**. Se  $t\_burn \leq 0$  o  $t\_burn > 2$  si restituisce `1e6*ones(4,1)`. Serve a impedire a `fsolve` (che e' unconstrained) di esplorare durate assurde o negative. Effetto collaterale onesto da dichiarare: il residuo diventa **discontinuo** su quella frontiera, e la Jacobiana per differenze finite che `fsolve` costruisce li' e' priva di senso. Funziona in pratica perche' il guess parte lontano dalla frontiera.
- Riga 228: `ic = [x0; y0; vx0; vy0; m0; 1]` - di nuovo `lam_m0 = 1`.
- Righe 230-236: `try/catch` attorno a `ode45`; un fallimento dell'integratore produce residui `1e6` invece di far crashare lo sweep.
- Riga 240-241: **la condizione di Hamiltoniana nulla**, il pezzo di matematica specifico di questo task.

```
Lam0 = sqrt(lam_vx0^2 + lam_vy0^2);
H0 = lam_y * p.vy0 + (p.T / p.m0) * Lam0 ...
    - lam_vy0 - p.T / p.c;
```

Derivazione. Valutiamo  $H$  all'inizio dell'arco propulso, dove lo stato e' **noto** ( $x=0$ ,  $y=y_1$ ,  $v_x=0$ ,  $v_y=v_{y1}$ ,  $m=m_1$ ) e i costati valgono  $\text{lam}_x = 0$ ,  $\text{lam}_v_x = \text{lam}_v_{x0}$ ,  $\text{lam}_v_y = \text{lam}_v_{y0}$ ,  $\text{lam}_m = 1$ :

$$H(0) = \text{lam}_x v_x + \text{lam}_y v_y + (T/m_0) * |\text{lam}_v| - \text{lam}_v_{y0} g - \text{lam}_m Q$$

con  $\text{lam}_x = 0$ ,  $g = 1$ ,  $\text{lam}_m = 1$  e  $Q = T/c$  (perche'  $T = c*Q$ ):

$$H(0) = \text{lam}_y v_{y1} + (T/m_1) * \sqrt{\text{lam}_v_{x0}^2 + \text{lam}_v_{y0}^2} - \text{lam}_v_{y0} - T/c$$

che e' esattamente la riga 241. Il termine  $(T/m_0) * |\text{lam}_v|$  viene dal fatto che il massimo su  $\phi$  di  $\text{lam}_v_x \cos(\phi) + \text{lam}_v_y \sin(\phi)$  e'  $|\text{lam}_v|$ .

**Perche' in  $t = 0$  e non in  $t_f$ ?** Perche' il sistema e' autonomo ( $H$  non dipende esplicitamente dal tempo), quindi  $H$  e' **costante** lungo l'arco: la condizione di trasversalita'  $H(t_f) = 0$  (tempo finale libero) equivale a  $H(0) = 0$ . Ma in  $t = 0$  lo stato e' esatto, mentre in  $t_f$  sarebbe il risultato dell'integrazione numerica – imporla in 0 la rende **algebrica**, senza errore di integrazione, e migliora sensibilmente il condizionamento dello shooting.

**Confronto col Task 1** (`main_task1.m` riga 268):

```
H(0)_task1 = -lam_vy0 + T*( |lam_v0| - 1/c )
```

E' il caso particolare di questa formula con  $vy_0 = 0$  (il primo termine sparisce) e  $m_0 = 1$  ( $T/m_0 \rightarrow T$ ). Task 2 non introduce una condizione nuova: **generalizza** la stessa condizione a uno stato iniziale non banale.

- Righe 243-246: i 4 residui:

```
res(1) = y(t_burn) - yf      % quota di iniezione
res(2) = vx(t_burn) - 1      % velocita' orbitale (Vrif = vxf)
res(3) = vy(t_burn)          % iniezione orizzontale
res(4) = H(0)                % tempo finale libero
```

Notare che  $x(t_f)$  non compare: e' libera, ed e' precisamente la ragione per cui  $lam_x = 0$  e quindi  $lam_{vx}$  e' costante. Notare anche che  $lam_m(t_f)$  non compare: grazie alla normalizzazione  $lam_{m0} = 1$ ,  $lam_m$  e' un puro quadratura (in `ode_burn.m`, `dz(6)` dipende solo da  $m$  e  $|lam_v|$ , e nessuna altra derivata dipende da  $lam_m$ ), quindi **non entra mai nei residui**.

**Possibile domanda d'esame** – la trasversalita' corretta e'  $lam_m(t_f) = 1$ ; perche' il codice impone invece  $lam_m(0) = 1$ ? *Risposta:* le condizioni necessarie (equazioni dei costati, legge del controllo,  $H = 0$ ) sono **omogenee di grado 1** nel vettore dei costati, e la legge del controllo  $phi = atan2(lam_{vy}, lam_{vx})$  e' addirittura **invariante di scala**. Quindi si puo' riscalarlo l'intero vettore dei costati per una costante positiva senza cambiare la traiettoria: fissare  $lam_m(0) = 1$  invece di  $lam_m(t_f) = 1$  e' solo una scelta di normalizzazione, che pero' **elimina un'incognita** ( $lam_{m0}$ ) e una condizione ( $lam_m(t_f) = 1$ ), portando lo shooting da  $5 \times 5$  a  $4 \times 4$ . Attenzione al fine print: il riscaldamento e' lecito solo con fattore **positivo**, cioe' la normalizzazione assume implicitamente  $lam_m(0) > 0$  alla soluzione. Non e' una proprieta' garantita in generale – infatti nel Task 3  $lam_{m0}$  torna a essere un'incognita, e i guess usati li' sono **negativi**.

## Limiti noti / punti onesti da dichiarare

- **Nessuna continuazione.** Contrariamente al Task 1, qui c'e' solo un guess cablato (identico a quello del Task 1) + un fallback. Se il guess fallisse, non c'e' una strategia di recupero sistematica. Una continuazione naturale sarebbe su  $y_1$ : partire da  $y_1 = 0$  (soluzione del Task 1) e aumentarlo gradualmente fino a  $1e-4$ , warm-startando ogni volta.
- **Nessuna verifica che l'evento sia scattato** (righe 26-30): con  $Q < 1/c$  lo script produrrebbe numeri senza segnalare nulla.
- **$Q = 2$  non e' il  $Q^*$  ottimo** del Task 1: il confronto di payload Task 1 vs Task 2 e' quindi corretto solo se fatto a parita' di  $Q$  (e nel Task 1 a  $Q = 2$  la soluzione esiste, essendo  $Q_{vec}$  che parte da 1.8).
- **Codice duplicato:** `ode_vertical` e `event_altitude` sono copiate identiche in `main_task3.m` invece di stare in un file condiviso.
- **La guardia  $V_k > 1e-10$**  (riga 135) e' inutile in questo task (all'inizio dell'arco  $vy = vy_1 > 0$ ): e' un residuo del Task 1.

## Possibili domande d'esame

**D: Qual e' esattamente la differenza tra Task 1 e Task 2 nel problema di controllo ottimo?** R: Nessuna nella struttura delle condizioni necessarie. Il problema ottimizzato e' sempre e solo l'arco propulso, con le stesse 4 incognite [ $lam_{vx0}$ ,  $lam_{vy0}$ ,  $lam_y$ ,  $t_{burn}$ ] e gli stessi 4 residui [ $y(t_f)-y_f$ ,  $vx(t_f)-1$ ,

$vy(tf)$ ,  $H(0)$ ]. Cambia solo la **condizione iniziale** dell'arco, che nel Task 1 e'  $(0,0,0,0,1)$  e nel Task 2 e'  $(0, y1, 0, vy1, m1)$  prodotta dalla salita verticale. Come conseguenza cambia la forma algebrica di  $H(0)$ , che acquisisce il termine  $lam\_y*vy1$  e sostituisce  $T$  con  $T/m1$ .

**D: Perche' non serve imporre la continuita' dei costati alla giunzione salita-verticale / arco propulso?** R: Perche' la salita verticale non ha gradi di liberta': il controllo e' imposto ( $\phi = 90 \text{ deg}$ ) e la durata e' fissata dal vincolo  $y(t1) = y1$ . Non essendoci un controllo libero, non ci sono condizioni di stazionarieta' da scrivere in fase 1 e i suoi moltiplicatori non compaiono in nessuna condizione necessaria. Se invece  $y1$  (o  $t1$ ) fosse una variabile da ottimizzare, servirebbe una condizione di interior-point: continuita' di  $lam\_x$ ,  $lam\_vx$ ,  $lam\_vy$ ,  $lam\_m$ , salto di  $lam\_y$  pari al moltiplicatore del vincolo interno  $y(t1) - y1 = 0$ , e continuita' di  $H$ .

**D: Perche'  $H = 0$  viene imposta all'inizio dell'arco e non alla fine?** R: Il sistema e' autonomo, quindi  $H$  e' un integrale primo lungo l'arco:  $H(0) = H(tf)$ . Imporla in  $tf$  significherebbe valutarla su uno stato ottenuto per integrazione numerica (con il suo errore); imporla in 0 la rende un'equazione **algebrica esatta** nelle incognite, perche' lo stato iniziale e' noto. Meglio condizionata e piu' economica.

**D: Il codice usa la continuazione (warm start)?** R: No. Il Task 2 usa un guess cablato  $[0.6; 3.8; 14; 0.30]$  – che e' letteralmente il guess iniziale del Task 1 – piu' un unico fallback  $[0.4; 3.0; 10; 0.35]$ . La continuazione vera e propria (warm start a catena su un parametro che varia) e' usata solo nel Task 1, per lo sweep su  $Q$ . Qui non serve perche' c'e' una sola soluzione da trovare e la salita verticale e' cosi' corta ( $y1 = 1e-4$  contro  $yf = 0.04$ ) che la soluzione e' una perturbazione piccola di quella del Task 1.

**D: Perche'  $lam\_vx$  e' costante e  $lam\_vy$  lineare?** R:  $lam\_vx\_dot = -lam\_x$  e  $lam\_vy\_dot = -lam\_y$ , e sia  $lam\_x$  che  $lam\_y$  sono costanti ( $H$  non dipende esplicitamente da  $x$  e  $y$ ). Poiche'  $x(tf)$  e' libera, la trasversalita' da'  $lam\_x(tf) = 0$ , e quindi  $lam\_x == 0$  ovunque:  $lam\_vx = lam\_vx0$  costante.  $lam\_y$  invece resta un'incognita ( $y(tf) = yf$  e' vincolata), quindi  $lam\_vy(t) = lam\_vy0 - lam\_y*t$  e' lineare. Da qui la legge della **tangente lineare**  $\tan(\phi) = (lam\_vy0 - lam\_y*t)/lam\_vx0$ .

**D: Cosa succede fisicamente al payload aggiungendo la salita verticale, e perche'?** R: Peggiora leggermente. La salita verticale e' un tratto in cui la spinta e' allineata contro gravita' e non produce nessuna velocita' orizzontale: e' tutta perdita gravitazionale pura, e nessuna traiettoria non ottima puo' battere l'ottimo del Task 1. La si accetta per realismo (sicurezza della base di lancio, clearance dalla torre, carichi aerodinamici); essendo  $y1$  piccolissima, la penalita' e' contenuta.

**D: Il payload si calcola come  $mf*(1+eta) - eta$ . Da dove viene?** R: A fine missione  $mf = ms + mu$ . Il modello strutturale del corso da'  $ms = eta*mp$  con  $mp = m0 - mf$ . In adimensionale  $m0 = 1$ , quindi  $mp = 1 - mf$  e  $ms = eta*(1-mf)$ . Sostituendo:  $mu = mf - eta*(1-mf) = mf*(1+eta) - eta$ . Massimizzare  $mf$  equivale quindi a massimizzare  $mu$  (la mappa e' affine e crescente), il che giustifica l'uso di  $m(tf)$  come funzionale di costo.

# HM1/main\_\_task3.m

## Ruolo del file nel progetto

`main_task3.m` risolve la **terza variante** della missione di HM1: alla sequenza del Task 2 (**salita verticale** -> **arco propulso**) si aggiunge un **arco balistico di coasting** finale. Il motore viene spento **prima** dell'iniezione, e il veicolo raggiunge la quota target `yf` in volo libero, arrivando all'apogeo esattamente con  $v_y = 0$  e  $v_x = 1$ . Se questo e' possibile, si risparmia propellente: la spinta non deve piu' combattere la gravita' negli ultimi istanti.

La differenza rispetto al Task 2 e' **strutturale**, non solo di condizione iniziale: adesso il problema e' genuinamente **bang-bang** in spinta (motore acceso -> motore spento), e il punto di spegnimento non e' un dato ma un'**incognita determinata dall'ottimalita'**. La condizione che lo determina e' l'annullarsi della **funzione di switching**  $S = |\lambda_m v|/m - \lambda_m m/c$ . Conseguenza diretta:  $\lambda_m m$  **entra nei residui** (compare in  $S$ ), quindi il trucco di normalizzazione del Task 1/2 ( $\lambda_m m_0 = 1$ ) non e' piu' comodo, e  $\lambda_m m_0$  torna a essere un'incognita. Lo shooting passa da **4x4 a 5x5**.

La seconda idea chiave del file e' che **l'arco di coasting non viene mai integrato numericamente dentro lo shooting**. Con  $T = 0$  la dinamica e' un moto parabolico esatto, quindi le tre condizioni terminali del Task 2 ( $y(tf)=y_f$ ,  $v_x(tf)=1$ ,  $v_y(tf)=0$ ) vengono **trasportate all'indietro in forma chiusa** fino all'istante di cutoff, dove diventano due sole condizioni algebriche ( $v_x(tc)=1$  e  $y_c + 0.5*v_{yc}^2 = y_f$ ). La terza ( $v_y(tf)=0$ ) e' soddisfatta per costruzione, perche' il tempo di coasting viene definito come il tempo di salita all'apogeo,  $t_{coast} = v_{yc}$ .

Il file dipende da `ode_burn.m` (RHS condiviso stato +  $\lambda_m m$ ) e definisce tre funzioni locali: `ode_vertical` e `event_altitude` (copiate identiche da `main_task2.m`) e `shooting3`. Esporta le figure in `HM1/figures/` con prefisso `task3_`.

---

## Richiamo: la funzione di switching e il problema bang-bang

Nel Task 1/2 la spinta e' sempre accesa e  $Q$  e' costante. Per far comparire un coast bisogna ammettere una manetta  $u$  in  $[0,1]$ , con  $T = u*c*Q$  e  $m_{dot} = -u*Q$ . L'Hamiltoniana (convenzione del massimo,  $\lambda_m x = 0$ ,  $g = 1$ ) diventa

$$H = \lambda_m y v_y - \lambda_m v_y + u*c*Q*(|\lambda_m v|/m - \lambda_m m/c)$$

perche' il massimo su  $\phi$  di  $\lambda_m v_x \cos(\phi) + \lambda_m v_y \sin(\phi)$  e'  $|\lambda_m v|$ , e i termini in  $u$  si raccolgono. La quantita' tra parentesi e' la **funzione di switching di Lawden**:

$$S = |\lambda_m v|/m - \lambda_m m/c$$

$H$  e' **lineare in  $u$** , quindi il massimo si ottiene agli estremi:

$$\begin{aligned} u &= 1 & \text{se } S > 0 & & (\text{motore acceso}) \\ u &= 0 & \text{se } S < 0 & & (\text{coast}) \end{aligned}$$

e lo **spegnimento avviene dove  $S = 0$** . E' questa l'equazione che il codice impone come quinto residuo ( $S$  e' definita alla riga 298 e inserita nel vettore **res** alla riga 304), ed e' l'unica ragione per cui il cutoff non e' una variabile arbitraria da spazzare: e' fissato dalle condizioni necessarie.

**Possibile domanda d'esame** – perche' la spinta ottima e' bang-bang e non “a manetta intermedia”?

*Risposta:* perche'  $H$  dipende linearmente dalla manetta  $u$ . Il PMP richiede di massimizzare  $H$  rispetto a  $u$  in  $[0,1]$ : il massimo di una funzione lineare su un intervallo sta sempre a un estremo, salvo il caso singolare  $S == 0$  su un intervallo di tempo di misura non nulla (arco singolare), che qui non si presenta. Quindi  $u$  vale 1 o 0 e commuta quando  $S$  cambia segno.

## Intestazione e parametri (righe 1-18)

- Righe 8-13: stessi dati del Task 2 –  $c = 0.6$ ,  $\eta = 0.1$ ,  $y_1 = 1e-4$ ,  $y_f = 0.04$ ,  $Q = 2$ ,  $T = c*Q = 1.2$ .
- Righe 15-18: stesse tolleranze strette (ode45 a  $1e-10/1e-12$ , fsolve a  $1e-10$ ).
- **Attenzione al commento di riga 3:** “*Search for optimal engine cutoff time to maximize payload*”. E' **fuorviante**: nel codice non esiste nessuno sweep o ricerca sul tempo di cutoff. Il cutoff  $t_{burn}$  e' una delle 5 incognite dello shooting ed e' determinato **implicitamente** dalla condizione di ottimalita'  $S(tc) = 0$ . E' esattamente il punto di forza del metodo indiretto: la struttura ottima esce dalle condizioni necessarie, non da un'ottimizzazione esterna.

## Fase 1 – salita verticale (righe 20-31)

```
ic_vert = [0; 0; 1];
opts_vert = odeset('RelTol', 1e-12, 'AbsTol', 1e-14, ...
    'Events', @(t,z) event_altitude(t, z, y1));
[T_vert, Z_vert] = ode45(@(t,z) ode_vertical(t,z,T,Q), ...
    [0 1], ic_vert, opts_vert);
t1 = T_vert(end);
vy1 = Z_vert(end,2);
m1 = Z_vert(end,3);
```

- Identica al Task 2 (righe 19-37 di `main_task2.m`), tolleranze comprese.
- Righe 27-29: si estraggono **solo**  $t1$ ,  $vy1$ ,  $m1$ . La quota raggiunta  $Z\_vert(end,1)$  **non** viene letta: alla riga 48 il codice usa il valore nominale  $p.y0 = y1$ . Nel Task 2 invece si usa  $y_1 = Z\_vert(end,1)$ . Incoerenza reale tra i due script; l'impatto e' dell'ordine della tolleranza dell'evento (trascurabile), ma va dichiarata.
- Stessa fragilita' del Task 2: se  $Q < 1/c = 1.667$  il veicolo non decolla, l'evento non scatta e lo script prosegue silenziosamente con  $t1 = 1$ .

## Fasi 2+3 – impostazione del BVP burn + coast (righe 33-61)

Il commento alle righe 34-41 e' la specifica del BVP, e vale la pena leggerlo come mappa:

```
% BVP unknowns: [lam_vx0, lam_vy0, lam_y, lam_m0, t_burn]
% vx(tc) = 1
% y(tc) + 0.5*vy(tc)^2 = yf
% lam_m(tc) = 1
% lam_vy(tc) = lam_y*vy(tc)
% S(tc) = |lam_v|/m - lam_m/c = 0
```

- Riga 34: **il vettore delle incognite cresce a 5**. Rispetto al Task 2 si aggiunge  $lam\_m0$ . Motivo:  $lam\_m$  compare esplicitamente nella funzione di switching, quindi la sua **scala assoluta rispetto a  $|lam\_v|$  conta**.



Nel Task 1/2 `lam_m` era invece un puro quadratura, disaccoppiato da tutto, e lo si poteva normalizzare a 1 a costo zero.

- Righe 43-51: la struct `p` con parametri e stato iniziale dell'arco propulso – come nel Task 2, la giunzione salita/burn trasferisce **solo lo stato**, senza condizioni sui costati (la fase verticale non ha gradi di libertà).
- Righe 53-61: quattro guess iniziali in una cell array. Tutti hanno `lam_m0 < 0` (-0.5, -0.6, -0.4, -0.5) e `t_burn` corto (0.10 - 0.18). I guess servono a due scopi (vedi sotto): entrare nel bacino della radice **fisica** e non della radice **spuria**.

## La radice spuria (commento righe 53-57)

La condizione `res(2), yc + 0.5*vyc^2 = yf`, e' **quadratica in vyc**: ammette due rami.

- Ramo **fisico**: `vyc > 0`. Il motore si spegne **sotto yf** e il veicolo sale balisticamente fino all'apogeo, che cade esattamente su `yf`. Il tempo di coast e' `t_coast = vyc > 0`.
- Ramo **spurio**: `vyc < 0`. L'equazione algebrica e' soddisfatta lo stesso (c'e' `vyc^2`), ma il "coast" avrebbe durata negativa: `t_coast = vyc < 0`. Il veicolo sarebbe gia' oltre l'apogeo e starebbe scendendo. Non e' una soluzione fisica, ma **fsolve non lo sa**: e' solo un'altra radice del sistema.

Il codice non aggiunge un vincolo `vyc > 0` (`fsolve` e' unconstrained): lo gestisce con un **filtro a posteriori** (righe 65-76). Vedi sotto.

**Possibile domanda d'esame** – perche' la condizione di coast e' `yc + 0.5*vyc^2 = yf` e cosa rappresenta? *Risposta*: durante il coast  $T = 0$ , quindi  $\dot{v}_y = -g = -1$  (adimensionale) e  $v_y(s) = v_{yc} - s$ : l'apogeo ( $v_y = 0$ ) e' raggiunto dopo  $s = v_{yc}$ . La quota guadagnata e'  $v_{yc}s - 0.5s^2$  valutata in  $s = v_{yc}$ , cioe'  $0.5*v_{yc}^2$ . Quindi la quota all'apogeo e' `yc + 0.5*vyc^2` e deve valere `yf`. E' la conservazione dell'energia ( $v^2/2g$  in dimensionale) e sostituisce **due** condizioni terminali del Task 2 ( $y(tf)=yf$  insieme a  $v_y(tf)=0$ ), perche'  $v_y(tf)=0$  e' vera per costruzione all'apogeo.

---

## Loop dei guess e filtro sulla radice fisica (righe 63-77)

```
for gg = 1:numel(guess_list)
    [z_try, ~, ef_try] = fsolve(@(z) shooting3(z,p,opts_ode), ...
                               guess_list{gg}, opts_fs);

    if ef_try > 0
        ... reintegra l'arco con z_try ...
        if Zc(end,4) > 1e-6 % vyc > 0: accept the physical coast
            z_sol = z_try; ef = ef_try; break;
        end
    end
end
end
```

- Righe 65-76: **multi-start, non continuazione**. Si prova ogni guess in sequenza; se `fsolve` converge (`ef_try > 0`) si **reintegra** l'arco propulso (righe 70-71) solo per leggere `vyc = Zc(end,4)` e si accetta la radice **solo se `vyc > 1e-6`** (riga 72). E' il filtro che scarta il ramo spurio.
- Da dichiarare con onesta': **in questo script non c'e' nessuna continuazione** (nessun warm start dalla soluzione di un problema vicino). Il Task 1 usa la continuazione sullo sweep in `Q`; il Task 2 riusa il guess cablato del Task 1; il Task 3 usa una lista di 4 guess tarati a mano. Una continuazione naturale qui sarebbe: partire dalla soluzione del Task 2 (coast di durata nulla) e "aprire" gradualmente il coast, oppure fare continuazione su `yf`.
- Costo: la reintegrazione della riga 70 e' ridondante (l'informazione era gia' dentro `fsolve`), ma e' eseguita al massimo 4 volte, quindi e' irrilevante.
- Riga 77: se nessun guess produce una radice fisica, `ef = -1` e lo script stampa l'errore alla riga 198 senza produrre nulla.

## Post-processing: ricostruzione del coast in forma chiusa (righe 79-119)

```
t_coast = vyc; % time to reach vy = 0
x_coast = xc + vxc * t_c;
y_coast = yc + vyc * t_c - 0.5 * t_c.^2;
vx_coast = vxc * ones(size(t_c));
vy_coast = vyc - t_c;
mf_task3 = mc; % mass doesn't change during coast
```

- Righe 87-88: reintegrazione dell'arco propulso su griglia densa (500 punti) per i grafici, con `ic2(6) = z_sol(4) = lam_m0`.
- Righe 91-95: stato al cutoff (`xc`, `yc`, `vxc`, `vyc`, `mc`).
- Riga 98: `t_coast = vyc`. In adimensionale  $g = 1$ , quindi il tempo per portare  $v_y$  da  $v_{yc}$  a zero e' esattamente  $v_{yc}$ . E' il motivo per cui la condizione  $v_y(tf) = 0$  **non compare tra i residui**: e' imposta per costruzione.
- Righe 102-106: il coast e' **analitico** – moto parabolico con  $v_x$  costante (nessuna forza orizzontale) e  $y$  parabola. Non si chiama `ode45`: cio' elimina ogni errore di integrazione dall'arco balistico e dimezza il costo dello shooting.
- Riga 108: `mf_task3 = mc` – durante il coast la massa non cambia ( $Q = 0$ ). Quindi **massimizzare  $m(tf)$  equivale a massimizzare  $m(tc)$** : e' la massa al cutoff che conta.
- Righe 116-118: i print di verifica confrontano `y_coast(end)`, `vx_coast(end)`, `vy_coast(end)` con i target `yf`, 1, 0. Sono **identita' per costruzione** (discendono direttamente dai residui e dalla definizione di `t_coast`), non un test indipendente: se lo shooting e' convergito, non possono che tornare.
- Riga 119: payload `mf*(1+eta) - eta`, stessa formula del Task 1/2.

---

## Plots (righe 121-195)

- Righe 122-133: traiettoria a tre colori – rosso verticale, blu burn, verde tratteggiato coast. Il coast e' visivamente l'arco che si "appiattisce" raggiungendo l'apogeo con tangente orizzontale.
- Righe 135-153: inset di zoom sulla salita verticale (identico al Task 2, e per la stessa ragione:  $y_1 = 1e-4$  e' invisibile a scala piena).
- Righe 156-167: angoli lungo il burn. `lam_vy_k = z_sol(2) - z_sol(3)*T2(kk)` e' ricalcolato **analiticamente** (il costato e' lineare), poi `phi = atan2(lam_vy_k, lam_vx0)` e `psi = atan2(vy, vx)`.
- Righe 170 e 180: `psi_coast` viene calcolato alla riga 170 e tracciato alla riga 180. Durante il coast si traccia **solo  $\psi$**  – non c'e' spinta, quindi  $\phi$  non e' definito. E' la firma grafica del bang-bang.
- Righe 181-182: `xline` marcano `t1` (fine verticale) e `t1 + t_burn` (engine cutoff).
- Righe 188-195: profilo di massa. Tre tratti: discesa lineare nella verticale (pendenza  $-Q$ ), discesa lineare nel burn (stessa pendenza), **piatto** nel coast. Il plateau finale e' la firma visiva del risparmio di propellente.

---

## Export figure (righe 201-217)

- Identico ai Task 1/2: `theme(fig, 'light')` (evita PNG a sfondo nero se il desktop MATLAB e' in dark mode), `exportgraphics` a 200 dpi, prefisso `task3_`.

---

## ode\_vertical (righe 221-230) e event\_altitude (righe 232-242)

- Copie **identiche** delle omonime in `main_task2.m` (righe 178-199). Nessuna fattorizzazione in file condiviso (a differenza di `ode_burn.m`).
- `ode_vertical`: dinamica con  $\phi = \pi/2$  sostituito a mano,  $dz = [v_y; T/m - 1; -Q]$  (con  $g = 1$ ).

- `event_altitude: value = y - y1, isterminal = 1, direction = 1` (attraversamento in salita).

## shooting3 (righe 244-305)

Il cuore del file. Cinque incognite, cinque residui.

- Riga 244: firma `res = shooting3(z0, p, opts_ode)`. Nessun blocco `arguments` (dichiarato alla riga 257): gira dentro il loop di `fsolve`.
- Righe 259-263: `unpack [lam_vx0; lam_vy0; lam_y; lam_m0; t_burn]`.
- Righe 265-268: stessa box constraint via penalita' del Task 2 (`t_burn <= 0 || t_burn > 2 -> res = 1e6`). Stesso caveat: rende il residuo discontinuo sulla frontiera e la Jacobiana per differenze finite ivi calcolata e' priva di senso.
- Riga 275: `ic = [x0; y0; vx0; vy0; m0; lam_m0]` – il sesto stato ora parte dall'**incognita** `lam_m0`, non piu' da 1.
- Righe 277-283: `try/catch` attorno a `ode45`.
- Righe 285-292: si legge lo stato al cutoff e si ricostruisce **analiticamente** il costato:

```
lam_vy_c = lam_vy0 - lam_y*t_burn
lam_v_norm = sqrt(lam_vx0^2 + lam_vy_c^2)
```

Legittimo: `lam_vy` e' esattamente lineare, quindi non serve integrarlo. Nota che `lam_mc` viene invece **letto dall'integrazione** (`zf(6)`), perche' la sua evoluzione `lam_m_dot = (T/m^2)*|lam_v|` non ha forma chiusa elementare.

## I cinque residui (righe 300-304)

```
res = [vxc - 1;
       yc + 0.5 * vyc^2 - p.yf;
       lam_mc - 1;
       lam_vy_c - lam_y * vyc;
       S];
```

**res(1) – vx(tc) = 1.** Durante il coast non c'e' forza orizzontale ( $T = 0$ , gravita' verticale, niente drag), quindi `vx` e' **costante**: `vx(tf) = vx(tc)`. La condizione terminale `vx(tf) = 1` viene percio' trasportata all'indietro senza approssimazioni fino al cutoff.

**res(2) – yc + 0.5\*vyc^2 = yf.** La condizione balistica derivata sopra. Assorbe in una sola equazione le **due** condizioni terminali `y(tf) = yf` e `vy(tf) = 0` del Task 2.

**res(3) – lam\_m(tc) = 1.** E' la **trasversalita' vera**, `lam_m(tf) = 1` (costo = `m(tf)`), riportata al cutoff. Legittimo perche' durante il coast  $T = 0$  e quindi

$\text{lam\_m\_dot} = (T/m^2)*|\text{lam\_v}| = 0 \rightarrow \text{lam\_m costante nel coast}$

quindi `lam_m(tc) = lam_m(tf) = 1`. Questa e' anche la condizione che **fissa la scala** dei costati (nel Task 1/2 la scala era fissata dalla normalizzazione `lam_m0 = 1`).

**res(4) – lam\_vy(tc) = lam\_y\*vyc.** E' la condizione  $H = 0$  **scritta sull'arco di coast**. Derivazione: con  $T = 0$  e `lam_x = 0`,

$H_{\text{coast}} = \text{lam\_y}*\text{vy} + \text{lam\_vy}*(-g) = \text{lam\_y}*\text{vy} - \text{lam\_vy}$

(gli altri termini contengono  $T$  o  $Q$  e si annullano). Il tempo finale e' libero e il sistema e' autonomo, quindi  $H = 0$ ; valutando in `tc`:

$\text{lam\_y}*\text{vyc} - \text{lam\_vy}(\text{tc}) = 0$

Si verifica facilmente che  $H_{\text{coast}}$  è costante lungo il coast:  $d/dt (\lambda_m y v_y - \lambda_m v_y) = \lambda_m y (-1) - (-\lambda_m y) = 0$ . Coerente.

$\text{res}(5) - S(\tau_c) = 0$ . La condizione di switching. È l'equazione che **determina la durata del burn**  $\tau_{\text{burn}}$ : senza di essa il cutoff sarebbe arbitrario.

## Perché non c'è un residuo $H(0) = 0$ come nei Task 1/2?

Perché è **implicito** da  $\text{res}(4)$  e  $\text{res}(5)$  insieme. Valutiamo  $H$  sull'arco propulso all'istante di cutoff ( $u = 1$ ,  $T = cQ$ ,  $\lambda_m x = 0$ ,  $g = 1$ ):

$$H_{\text{burn}}(\tau_c) = \lambda_m y v_{yc} + (T/mc) * |\lambda_m v_c| - \lambda_m v_{yc} - \lambda_m mc Q$$

Raccogliendo (con  $T = cQ$ ):

$$\begin{aligned} H_{\text{burn}}(\tau_c) &= (\lambda_m y v_{yc} - \lambda_m v_{yc}) \\ &\quad + cQ * (|\lambda_m v_c|/mc - \lambda_m mc/c) \\ &= \text{res}(4) + cQ * S(\tau_c) \end{aligned}$$

Se  $\text{res}(4) = 0$  e  $S(\tau_c) = 0$ , allora  $H_{\text{burn}}(\tau_c) = 0$ ; e poiché il sistema è autonomo,  $H$  è costante sull'arco propulso, quindi  $H = 0$  **su tutto il burn**, inclusi  $t = 0$ . La condizione  $H(0) = 0$  che nei Task 1/2 era un residuo esplicito qui è **automaticamente soddisfatta**. Questo è anche l'enunciato della condizione angolare di Weierstrass-Erdmann:  $H$  è **continua** attraverso il corner, e la continuità è garantita proprio da  $S(\tau_c) = 0$ .

## Le condizioni di corner (Weierstrass-Erdmann) usate implicitamente

Alla giunzione burn  $\rightarrow$  coast **non c'è salto di stato né vincolo interno**, quindi:

- i costati sono **continui**:  $\lambda_m(\tau_c^-) = \lambda_m(\tau_c^+)$ . Il codice lo usa senza dirlo:  $\lambda_m v_{yc}$  calcolato **sul burn** viene inserito nella condizione  $H_{\text{coast}} = 0$  ( $\text{res}(4)$ ), che vive **sul coast**; e  $\lambda_m mc$  calcolato sul burn viene eguagliato al suo valore terminale sul coast ( $\text{res}(3)$ ).
- $H$  è **continua**: garantita da  $S(\tau_c) = 0$ , come mostrato sopra.

Non c'è quindi nessuna condizione aggiuntiva da imporre: la continuità è “cablata” nel modo in cui i residui sono scritti.

## L'hack numerico sulla funzione di switching (righe 294-298)

```
% Using the 1/c form (not the literal lam_mc/c) keeps fsolve in the
% basin of the physical vyc>0 root; the lam_mc/c form drifts to the
% spurious vyc<0 branch. The two coincide at cutoff.
S = lam_v_norm / mc - 1 / p.c;
```

Questo va dichiarato per intero, perché è una **deviazione consapevole dalla forma letterale**:

- La funzione di switching è  $S = |\lambda_m v|/m - \lambda_m m/c$ . Alla soluzione,  $\text{res}(3)$  impone  $\lambda_m mc = 1$ , quindi  $\lambda_m mc/c = 1/c$ : **le due forme coincidono nella radice**, e i due sistemi 5x5 hanno **esattamente lo stesso insieme di soluzioni**.
- Ma **fuori** dalla soluzione (cioè durante le iterazioni di Newton) sono diverse: la forma  $1/c$  **rimuove la dipendenza di  $\text{res}(5)$  da  $\lambda_m m_0$** , cioè azzerava una colonna della Jacobiana. Questo cambia il campo di Newton e, secondo il commento dell'autore, mantiene **fsolve** nel bacino della radice fisica ( $v_{yc} > 0$ ) invece di farlo scivolare su quella spuria.
- Onesta: **è una scelta di condizionamento numerico, non teorica**. La forma teoricamente corretta è  $\lambda_m mc/c$ . Le due sono equivalenti alla convergenza; se all'orale viene chiesta la funzione di switching, la risposta è  $|\lambda_m v|/m - \lambda_m m/c$ .

**Possibile domanda d'esame** – perché nel Task 3 non si può usare la normalizzazione  $\lambda_m m_0 = 1$  del Task 1/2? *Risposta*: nei Task 1/2  $\lambda_m m$  è completamente disaccoppiato: non entra né nella dinamica dello stato, né nella legge del controllo, né nei residui, e può quindi essere scalato liberamente (le condizioni necessarie sono omogenee di grado 1 nei costati). Nel Task 3  $\lambda_m m$  entra

nella funzione di switching  $S = |\lambda_m v|/m - \lambda_m m/c$ , che confronta la sua **scala assoluta** con quella di  $|\lambda_m v|$ : fissarlo arbitrariamente a 1 all'istante iniziale falserebbe il punto di switching. La scala viene perciò fissata dove la teoria la fissa davvero, cioè con la trasversalità  $\lambda_m m(t_f) = 1$ , riportata al cutoff come `res(3)`;  $\lambda_m m_0$  diventa la quinta incognita. Coerentemente, i guess usati nel codice hanno  $\lambda_m m_0 < 0$ :  $\lambda_m m$  cresce lungo il burn ( $\lambda_m m_{dot} = (T/m^2)*|\lambda_m v| > 0$ ) e deve arrivare a 1 al cutoff, quindi parte più in basso. Il codice non stampa mai il  $\lambda_m m_0$  convergito, quindi dal solo sorgente non se ne può confermare il segno alla soluzione.

## Limiti noti / punti onesti da dichiarare

- **Il commento di riga 3 è fuorviante:** non c'è nessuna "ricerca" del tempo di cutoff;  $t_{burn}$  è un'incognita dello shooting fissata da  $S(t_c) = 0$ .
- **S è imposta solo al cutoff**, non verificata sull'arco. Il codice **non controlla** che  $S > 0$  per tutto il burn (cioè che la struttura ottima sia davvero `burn -> coast` e non, per esempio, `coast -> burn -> coast`), né che  $S$  attraversi lo zero **in discesa** in  $t_c$ . E' un controllo di ottimalità della struttura che manca.
- **La forma di S usata ( $1/c$  invece di  $\lambda_m m/c$ )** è un espediente di convergenza (vedi sopra): corretta alla soluzione, non alla lettera durante le iterazioni.
- **Nessuna continuazione:** solo 4 guess cablati + filtro sul segno di  $v_{yc}$ . Se nessuno converge, lo script si arrende.
- **Incoerenza con il Task 2:** qui  $p.y_0 = y_1$  (nominale) invece di  $Z_{vert}(end, 1)$  (raggiunta).
- **Nessuna guardia sull'evento** della salita verticale (come nel Task 2).
- **Codice duplicato:** `ode_vertical` e `event_altitude` copiate dal Task 2.
- Il modello resta senza drag e a Terra piatta: un coast "vero" a 620 m - 240 km di quota adimensionale attraverserebbe l'atmosfera, ma qui non c'è.

## Possibili domande d'esame

**D: Qual è esattamente la variante del Task 3 rispetto al Task 2, letta dal codice?** R: Si aggiunge un arco di coasting finale. In termini di BVP: (1) il vettore delle incognite passa da 4 a 5, aggiungendo  $\lambda_m m_0$ ; (2) i tre residui terminali del Task 2 ( $y(t_f) - y_f$ ,  $v_x(t_f) - 1$ ,  $v_y(t_f)$ ) vengono trasportati analiticamente al cutoff e diventano due ( $v_{xc} - 1$  e  $y_c + 0.5*v_{yc}^2 - y_f$ ), perché  $v_y(t_f) = 0$  è vera per costruzione all'apogeo; (3) il residuo  $H(0) = 0$  sparisce e viene sostituito da due condizioni nuove:  $H_{coast} = 0$  scritta al cutoff ( $\lambda_m v_{y_c} = \lambda_m y * v_{yc}$ ) e la condizione di switching  $S(t_c) = 0$ ; (4) si aggiunge la trasversalità  $\lambda_m m(t_c) = 1$ . Totale: 5 incognite, 5 residui.

**D: Come si ricava la funzione di switching e cosa impone?** R: Si ammette una manetta  $u$  in  $[0, 1]$  con  $T = u*c*Q$  e  $m_{dot} = -u*Q$ . Raccogliendo i termini in  $u$  nell'Hamiltoniana si ottiene  $H = \lambda_m y*v_y - \lambda_m v_y + u*c*Q*S$  con  $S = |\lambda_m v|/m - \lambda_m m/c$ . Poiché  $H$  è lineare in  $u$ , il PMP dà  $u = 1$  se  $S > 0$  e  $u = 0$  se  $S < 0$ ; il passaggio `burn -> coast` avviene dove  $S = 0$ . Imporre  $S(t_c) = 0$  è l'equazione che determina la durata del burn: senza di essa il cutoff sarebbe arbitrario.

**D: Perché l'arco di coasting non viene integrato numericamente?** R: Perché con  $T = 0$  la dinamica è esatta in forma chiusa:  $v_x$  costante,  $v_y(s) = v_{yc} - s$ ,  $y(s) = y_c + v_{yc}*s - 0.5*s^2$ ,  $m$  costante,  $\lambda_m m$  costante,  $\lambda_m v_y$  lineare. Tutte le condizioni terminali si possono quindi riportare all'indietro fino al cutoff in modo esatto. Questo (a) elimina l'errore di integrazione sull'arco balistico, (b) elimina il tempo di coast dal vettore delle incognite ( $t_{coast} = v_{yc}$  per definizione di apogeo), (c) dimezza il costo di ogni valutazione del residuo dentro `fsolve`.

**D: Quali condizioni di giunzione (corner conditions) valgono tra burn e coast, e dove sono nel codice?** R: Non c'è salto di stato né vincolo di interior-point, quindi valgono le condizioni di Weierstrass-Erdmann nella forma "liscia": **costati continui**  $\lambda_m(t_c^-) = \lambda_m(t_c^+)$  e **H continua**. Il codice le usa implicitamente:  $\lambda_m v_{y_c}$  e  $\lambda_m m_c$ , calcolati sull'arco propulso, vengono usati in condizioni che vivono sul **coast** (`res(4)` e `res(3)`) – cioè si sta assumendo la continuità. La continuità di  $H$  è invece garantita esattamente da  $S(t_c) =$

0: si dimostra che  $H_{\text{burn}}(tc) = (lam\_y*vyc - lam\_vy\_c) + c*Q*S(tc)$ , quindi con  $res(4) = 0$  e  $res(5) = 0$  si ha  $H_{\text{burn}}(tc) = H_{\text{coast}}(tc) = 0$ .

**D: Perche' esiste una radice spuria e come la scarta il codice?** R: Perche' il residuo  $yc + 0.5*vyc^2 = yf$  e' quadratico in  $vyc$  e ammette anche  $vyc < 0$  (veicolo gia' in discesa dopo l'apogeo). Formalmente e' una radice del sistema, ma corrisponde a  $t_{\text{coast}} = vyc < 0$ , cioe' un coast di durata negativa: e' non fisica. `fsolve` non ha vincoli e ci puo' cadere. Il codice non impone  $vyc > 0$  come vincolo: prova quattro guess con burn corto ( $t_{\text{burn}} = 0.10-0.18$ , che cadono nel bacino della radice fisica), reintegra l'arco e **accetta la prima radice con  $vyc > 1e-6$**  (riga 72). E' un filtro a posteriori, non un vincolo.

**D: Il Task 3 usa la continuazione?** R: No. Usa un **multi-start** su una lista di 4 guess cablati, con un filtro sul segno di  $vyc$ . La continuazione (warm start a catena da una soluzione vicina) e' usata solo nel Task 1, per lo sweep su  $Q$ . Una continuazione ragionevole per il Task 3 sarebbe partire dalla soluzione del Task 2 (coast di durata nulla) e aumentare gradualmente il parametro che apre il coast, oppure fare continuazione su  $yf$ .

**D: Perche' il coasting fa risparmiare propellente?** R: Perche' la massa non cambia durante il coast ( $Q = 0$ ), quindi  $m(tf) = m(tc)$ : massimizzare la massa finale equivale a massimizzare la massa al cutoff. Negli ultimi istanti di un burn "fino all'iniezione" (Task 2) la spinta deve anche sostenere il veicolo contro gravita' mentre porta  $vy$  a zero: e' una perdita gravitazionale. Se invece si spegne prima, e' la **gravita' stessa** a portare  $vy$  a zero gratuitamente durante la salita balistica, mentre  $vx$  e' gia' al valore richiesto e resta tale. La condizione  $S(tc) = 0$  individua esattamente il punto in cui questo scambio diventa conveniente.

**D: Perche'  $t_{\text{coast}} = vyc$  senza costanti?** R: Perche' il problema e' adimensionalizzato con  $a_{\text{rif}} = g$ , quindi  $g = 1$  in unita' adimensionali. Durante il coast  $vy\_dot = -g = -1$ , cioe'  $vy(s) = vyc - s$ , e l'apogeo ( $vy = 0$ ) e' raggiunto in  $s = vyc$ . In dimensionale sarebbe  $t_{\text{coast}} = vyc/g$ .

# HM1/main\_\_task4.m

## Ruolo del file nel progetto

`main_task4.m` e' lo script del **Task 4 di HM1: staging ottimo**. La variante e' un lanciatore a **due stadi** che parte da fermo all'origine (nessuna salita verticale, a differenza di Task 2 e Task 3), accende il motore e lo tiene acceso per tutto il volo. A un istante `ts` – l'unica novita' rispetto al Task 1 – la struttura del primo stadio viene **buttata via** (jettison): la massa cade di colpo di  $ms1 = \eta * Q * ts$ , e il volo prosegue con lo stesso motore (stesso  $T$ , stesso  $Q$ , stesso  $c$ ) fino all'iniezione.

Il file esiste per rispondere a una domanda quantitativa: **quanto guadagna il payload lo staging, e quando conviene farlo?** La struttura e' a **due livelli**:

- **livello interno (BVP)** – per un `ts` fissato si risolve lo stesso shooting a 4 incognite del Task 1, ma su due archi separati dal salto di massa (`shooting_twostage`, righe 248-298);
- **livello esterno (sweep)** – `ts` viene spazzato su una griglia di 50 punti e per ognuno si legge il payload risultante; il massimo della curva e' lo staging ottimo (righe 38-131).

Cioe': lo script **non impone la condizione di ottimalita' su `ts`**. La cerca numericamente. La condizione analitica corrispondente (la *corner condition* all'istante di giunzione) esiste e viene derivata piu' sotto, ma e' implementata in un file separato, `validate_staging_corner.m`, che serve appunto a verificare che lo sweep di questo script cada dove la teoria dice.

Dipende da `ode_burn.m` (RHS condiviso: dinamica planare + costati + legge di controllo linear-tangent gia' sostituita) e da nient'altro. Non c'e' atmosfera, non c'e' drag, la spinta e' sempre accesa:  $T = c * Q$ ,  $g = 1$  nondimensionale.

---

## %% Parameters (righe 7-17)

```
c = 0.6;
eta = 0.1;
yf = 0.04;
Q = 2;           % same Q for both stages
T = c * Q;
```

- Righe 8-12: i dati della traccia in unita' nondimensionali.  $c = 0.6$  e' la velocita' efficace di scarico ( $Isp * g_0 / V_{rif}$ ),  $\eta = 0.1$  il coefficiente strutturale  $ms/mp$ ,  $yf = 0.04$  la quota target,  $Q = 2$  la portata di massa.  $T = c * Q = 1.2$  e' la spinta, costante e sempre accesa.
- Riga 11: il commento `same Q for both stages` va preso sul serio: e' una **semplificazione di modello**. Un lanciatore reale ha un secondo stadio con spinta e portata molto diverse (e spesso  $Isp$  maggiore, motore da vuoto). Qui i due stadi condividono  $T$ ,  $Q$ ,  $c$ : l'unica cosa che cambia allo staging e' la massa. E' anche il motivo per cui il guadagno di payload che esce (+182% nel README) e' cosi' generoso rispetto ai lanciatori veri.

- Righe 14-17: tolleranze. ode45 a RelTol=1e-10, AbsTol=1e-12, fsolve a 1e-10 su funzione e passo. Serve: i residui dello shooting sono differenze fra numeri O(1) e il Jacobiano di fsolve e' calcolato per differenze finite sul risultato di un'integrazione – se l'integratore e' lasco, il Jacobiano e' rumore.

---

## %% First solve single-stage (Task 1) as reference (righe 19-36)

```
z_guess_1 = [0.6; 3.8; 14; 0.30];
[z_ref, ~, ef] = fsolve(@(z) shooting_single(z, p1, ...
      opts_ode), z_guess_1, opts_fs);
...
payload_single = mf_single*(1+eta) - eta;
```

- Righe 21-24: si risolve prima il **caso monostadio** (identico al Task 1). Serve a due cose: (a) e' il termine di paragone per il guadagno di staging; (b) la sua soluzione **z\_ref** e' il **warm start** dello sweep a due stadi (riga 77).
- Riga 23: il guess [lam\_vx0; lam\_vy0; lam\_y; tf] = [0.6; 3.8; 14; 0.30]. Con lam\_vy0/lam\_vx0 = 3.8/0.6 l'angolo di spinta iniziale e'  $\phi(0) = \text{atan2}(3.8, 0.6) \sim 81$  gradi: quasi verticale, come dev'essere.
- Riga 28: ic = [0;0;0;0;1;1] – stato iniziale [x; y; vx; vy; m; lam\_m], cioe' origine, fermo,  $m_0 = 1$  (nondimensionalizzato su se stesso) e  $\text{lam}_m(0) = 1$ . Quest'ultimo **non e' un dato fisico**: e' la normalizzazione dei costati (vedi la sezione sulla matematica).
- Riga 29: la soluzione convergente viene **ri-integrata** per estrarre mf. E' lavoro duplicato (shooting\_single aveva gia' integrato la stessa traiettoria), ma costa una frazione di secondo ed evita di far restituire lo stato al residuo.
- Riga 32:  $\text{payload\_single} = \text{mf}*(1+\text{eta}) - \text{eta}$ . Derivazione: la massa iniziale si ripartisce in  $m_0 = 1 = \mu + \text{ms} + \text{mp}$  (payload + struttura + propellente). Il propellente bruciato e'  $\text{mp} = Q*\text{tf} = 1 - \text{mf}$ , la struttura e'  $\text{ms} = \text{eta}*\text{mp}$ . Quindi

$$\mu = \text{mf} - \text{ms} = \text{mf} - \text{eta}*(1 - \text{mf}) = \text{mf}*(1 + \text{eta}) - \text{eta}$$

Il payload e' cio' che resta **dopo aver scartato anche la struttura**: e' una *differenza di numeri grandi* (qui  $\text{mf} \sim 0.11$ ,  $\mu \sim 0.024$ ), e questo spiega perche' guadagni percentualmente enormi sul payload corrispondano a variazioni minuscole di tf.

**Possibile domanda d'esame** – perche'  $\text{payload} = \text{mf}*(1+\text{eta}) - \text{eta}$  e non semplicemente mf?

*Risposta:* perche' mf include ancora la struttura (serbatoi, motore) del lanciatore. Il modello a coefficiente strutturale dice che la struttura e' proporzionale al propellente imbarcato,  $\text{ms} = \text{eta}*\text{mp}$ ; siccome  $\text{mp} = 1 - \text{mf}$ , si ricava  $\text{ms} = \text{eta}*(1 - \text{mf})$  e il payload utile e'  $\text{mf} - \text{ms}$ . Se si riportasse mf come figura di merito si premierebbero i lanciatori che bruciano poco propellente ma sono tutti serbatoio.

---

## La matematica dello staging: salto di massa, costati, Hamiltoniana

Questa e' la parte che il codice usa ma non scrive. Vale la pena derivarla per intero, perche' e' il cuore della domanda “i costati sono continui allo staging?”.

### Il problema di controllo ottimo di base

Con lo stato  $\mathbf{x} = [x; y; vx; vy; m]$  e la spinta sempre accesa, la dinamica (in ode\_burn.m) e'



```

xdot = vx
ydot = vy
vxdot = (T/m)*cos(phi)
vydot = (T/m)*sin(phi) - 1      (g = 1 nondim)
mdot = -Q

```

L'Hamiltoniana (convenzione di massimizzazione, quella usata da `ode_burn`) e'

```

H = lam_x*vx + lam_y*vy
    + lam_vx*(T/m)*cos(phi)
    + lam_vy*((T/m)*sin(phi) - 1)
    - lam_m*Q

```

Le equazioni di Eulero-Lagrange ( $\text{lam\_dot} = -dH/dx$ ) danno:

```

lam_x_dot = 0          -> lam_x = const = 0  (x(tf) libero)
lam_y_dot = 0          -> lam_y = const
lam_vx_dot = -lam_x = 0 -> lam_vx = lam_vx0
lam_vy_dot = -lam_y     -> lam_vy = lam_vy0 - lam_y*t
lam_m_dot = (T/m^2)*|lam_v| -> lam_m crescente (unica ODE da integrare)

```

La massimizzazione su  $\phi$  allinea la spinta al **primer vector**  $\text{lam\_v}$ :  $\phi = \text{atan2}(\text{lam\_vy}, \text{lam\_vx})$ , da cui  $\tan(\phi)$  lineare in  $t$  (legge *linear-tangent*). Sostituendo il  $\phi$  ottimo si ottiene la forma compatta

```
H = lam_y*vy - lam_vy + T*( |lam_v|/m - lam_m/c )
```

usando  $Q = T/c$ . A  $t = 0$  (dove  $vy = 0$ ,  $m = 1$ ,  $\text{lam\_m0} = 1$ ) diventa **algebrica**:

```
H(0) = -lam_vy0 + T*( |lam_v(0)| - 1/c )
```

che e' esattamente la riga 243 (e 295) del codice.

## Il salto di massa

Allo staging la mappa di giunzione e'

```
m(ts+) = m(ts-) - eta*Q*ts,      x, y, vx, vy continui
```

Perche'  $\eta Q \tau_s$ ? Il propellente bruciato dal primo stadio e'  $m_{p1} = Q \tau_s$ ; il modello a coefficiente strutturale dice  $m_{s1} = \eta m_{p1} = \eta Q \tau_s$ . **Il salto dipende esplicitamente da  $\tau_s$** : questo e' il punto tecnico chiave, e' cio' che rende la giunzione diversa da un semplice cambio di fase.

Siccome  $\text{mdot} = -Q$  e  $m_0 = 1$ :

```

m_minus = 1 - Q*ts
m_plus  = 1 - (1+eta)*Q*ts

```

(righe 43-44 del codice, e la condizione  $m_{\text{plus}} > 0$  da' il vincolo  $\tau_s < 1/(Q*(1+\eta))$  della riga 45).

## I costati sono continui?

**Si', tutti e cinque.** Ecco perche'. Si aggiunge il vincolo di giunzione  $x(\tau_s+) = x(\tau_s-) + \Delta(\tau_s)$  con un moltiplicatore  $\pi$ :

```
Jaug = ... + pi*[ x(ts-) + Delta(ts) - x(ts+) ] + integrali dei due archi
```

Il coefficiente della variazione  $\delta x(\tau_s-)$  da'  $\text{lam}(\tau_s-) = \pi$ , quello di  $\delta x(\tau_s+)$  da'  $\text{lam}(\tau_s+) = \pi$ . Quindi

```
lam(ts-) = lam(ts+)
```

La ragione strutturale:  $\Delta$  **non dipende dallo stato**  $x(\tau_s-)$ , dipende solo da  $\tau_s$ . In generale la condizione e'  $\text{lam}(\tau_s-)' = \text{lam}(\tau_s+)' * d\Delta/dx-$ , e qui  $d\Delta/dx- = I$  (matrice identita'), quindi i costati passano lisci. Se il modello avesse  $m_{s1} = \eta*(m_0 - m(\tau_s))$  scritto in funzione della *massa* (invece che del tempo), la Jacobiana non sarebbe l'identita' e  $\text{lam\_m}$  **salterebbe**.

Nel codice questa continuità è imposta implicitamente e in modo pulito: `lam_vx`, `lam_vy`, `lam_y` sono formule analitiche nei *parametri* `pp.lam_vx0`, `pp.lam_vy0`, `pp.lam_y`, e la stessa struct `pp` viene passata a `ode_burn` sui due archi (righe 90 e 98) – quindi sono automaticamente continui. `lam_m` è l’unico costato integrato, e la riga 280 tocca **solo** `z_s(5)` (la massa), lasciando `z_s(6) = lam_m` intatto (commento esplicito, riga 281).

## L’Hamiltoniana è continua? NO.

Attraverso `ts` tutto è continuo tranne `m`, che scende. Nella forma compatta  $H = \text{lam}_y \cdot v_y - \text{lam}_{vy} + T \cdot (|\text{lam}_v|/m - \text{lam}_m/c)$ , l’unico termine che cambia è  $T \cdot |\text{lam}_v|/m$ :

$$H(\text{ts}+) - H(\text{ts}-) = T \cdot |\text{lam}_v(\text{ts})| \cdot (1/m_{\text{plus}} - 1/m_{\text{minus}}) > 0$$

**L’Hamiltoniana salta verso l’alto.** Fisicamente è ovvio: buttando massa, l’accelerazione  $T/m$  aumenta di colpo, e  $H$  (il “tasso istantaneo di accumulo di merito”) sale. Questo è il caso classico in cui **H non è continua** perché la mappa di giunzione dipende dal tempo di giunzione.

## La condizione di ottimalità su `ts` (corner condition)

Il coefficiente di `dts` nella variazione da’ la condizione mancante:

$$d\phi/dts + H(\text{ts}-) - H(\text{ts}+) + \text{lam}(\text{ts}) \cdot (d\Delta/dts) = 0$$

dove  $\phi$  è il payoff terminale. Qui il payoff è il **payload**:

$$mu = m(\text{tf}) - \eta \cdot Q \cdot (\text{tf} - \text{ts}) \quad \rightarrow \quad d\phi/dts = +\eta \cdot Q$$

e  $d\Delta/dts = (0, 0, 0, 0, -\eta \cdot Q)'$  da cui  $\text{lam}' \cdot d\Delta/dts = -\eta \cdot Q \cdot \text{lam}_m(\text{ts})$ . Sostituendo il salto di  $H$  calcolato sopra:

$$\eta \cdot Q - T \cdot |\text{lam}_v(\text{ts})| \cdot (1/m_{\text{plus}} - 1/m_{\text{minus}}) - \eta \cdot Q \cdot \text{lam}_m(\text{ts}) = 0$$

e, dividendo per  $Q$  e usando  $T = c \cdot Q$ :

$$\eta \cdot (1 - \text{lam}_m(\text{ts})) = c \cdot |\text{lam}_v(\text{ts})| \cdot (1/m_{\text{plus}} - 1/m_{\text{minus}})$$

Con la trasversalità  $\text{lam}_m(\text{tf}) = 1$  (una massa in più a burnout e una massa in più di payload) si ottiene la **forma usata in `validate_staging_corner.m`**:

$$\eta \cdot [\text{lam}_m(\text{tf}) - \text{lam}_m(\text{ts})] = c \cdot |\text{lam}_v(\text{ts})| \cdot (1/m_{\text{plus}} - 1/m_{\text{minus}})$$

Lettura fisica del bilancio (ritardare lo staging di `dts`): - **guadagno**  $\eta \cdot Q \cdot (1 - \text{lam}_m(\text{ts})) \cdot dts$ : se stadi più tardi, il primo stadio ha bruciato più propellente, quindi la sua struttura è più pesante e viene buttata; di conseguenza la struttura del *secondo* stadio,  $ms2 = \eta \cdot Q \cdot (\text{tf} - \text{ts})$ , è più leggera e resta più payload; - **perdita**  $T \cdot |\text{lam}_v| \cdot (1/m_{\text{plus}} - 1/m_{\text{minus}}) \cdot dts$ : hai volato un `dts` in più con il veicolo *pesante* invece che con quello *leggero*.

L’ottimo è dove i due si pareggiano. `main_task4.m` **non risolve questa equazione**: la cerca facendo `max()` su una griglia. Il file `validate_staging_corner.m` la risolve.

**Nota sulle due formulazioni equivalenti.** L’appendice del report (HM1/report/chapters/AppendixStaging.tex) deriva la stessa identica condizione partendo dal payoff  $\phi = \text{tf}$  (**minimo tempo di combustione**), per cui  $\text{lam}_m(\text{tf}) = d\phi/dm = 0$  e la giunzione non ha termine  $d\phi/dts$ . Il risultato finale coincide, perché le due formulazioni differiscono solo per un **gauge**:  $\text{lam}_m$  entra in  $H$  solo tramite  $-Q \cdot \text{lam}_m$ , e  $\text{lam}_m \cdot \dot{} = (T/m^2) \cdot |\text{lam}_v|$  non dipende da  $\text{lam}_m$  stesso. Le condizioni necessarie sono quindi invarianti sotto

`lam` -> `k*lam` ( $k > 0$ , scala globale)  
`lam_m` -> `lam_m + b` (offset additivo del solo `lam_m`)

Passare dal payoff “payload” ( $\text{lam}_m(\text{tf})=1$ ) a quello “min tf” ( $\text{lam}_m(\text{tf})=0$ ) è esattamente  $k = (1+\eta) \cdot Q$  e  $b = 1$ . La corner condition, scritta come **differenza**  $\text{lam}_m(\text{tf}) - \text{lam}_m(\text{ts})$ , è invariante sotto entrambi: la differenza uccide  $b$ , e i due membri sono omogenei di grado 1 in  $k$ . Ed è proprio per questo che si può valutarla sui costati “sporchi” che il BVP integra con  $\text{lam}_m0 = 1$  e  $H(0) = 0$ .

**Possibile domanda d'esame** – allo staging i costati saltano? *Risposta:* no, sono tutti continui, perche' la massa jettisonata  $ms1 = \eta * Q * ts$  dipende solo dal tempo di giunzione e non dallo stato: la Jacobiana della mappa di salto rispetto a  $x(ts-)$  e' l'identita', e la condizione di corner  $\lambda_m(ts-)' = \lambda_m(ts+)' * d\Delta/dx-$  collassa in  $\lambda_m(ts-) = \lambda_m(ts+)$ . Quello che *salta* e' l'Hamiltoniana, di  $T * |\lambda_m| * (1/m+ - 1/m-) > 0$ , perche' il salto di massa dipende esplicitamente da  $ts$ .

## %% Sweep staging time (righe 38-112)

```
ts_max = min(tf_single*0.95, 1/(Q*(1+eta)) - 0.01);
ts_vec = linspace(0.01, ts_max, 50);
...
[~, idx_mid] = min(abs(ts_vec - tf_single*0.4));
for pass = 1:2
```

- Righe 41-45: il range ammissibile di  $ts$ . Due vincoli:  $ts < tf$  (approssimato con  $0.95 * tf\_single$ , perche'  $tf$  del caso a due stadi non e' ancora noto) e  $m\_plus > 0$ , cioe'  $ts < 1/(Q*(1+\eta)) = 0.4545$  con i dati del problema, con un margine di 0.01. Con  $Q=2$ ,  $\eta=0.1$  vince quasi sempre il primo vincolo.
- Riga 46: **50 punti di griglia**. E' il limite di risoluzione dell'intero Task 4: il passo e' circa 0.008 in  $ts$ , quindi il  $ts\_opt$  riportato e' **agganciato alla griglia** e non e' il vero punto stazionario. Non c'e' nessun raffinamento (fminbnd, interpolazione parabolica del massimo, golden section) – e' un limite reale dello script, non un difetto nascosto.
- Righe 48-51: pre-allocazione a NaN. I punti che non convergono restano NaN e vengono filtrati alla riga 115 con `valid = ~isnan(pay_two)`. E' il pattern giusto: un punto fallito non sporca il massimo.
- Righe 57-64: **continuation a due passate**. Si parte da  $idx\_mid$  (il punto piu' vicino a  $0.4 * tf\_single$ ), si sale fino in fondo (pass 1), poi si torna indietro da  $idx\_mid-1$  a 1 (pass 2). Ogni solve usa come guess la soluzione del  $ts$  precedente ( $z\_prev\_loc$ , righe 74-78 e 109). Questa e' la ricetta standard della repo per far convergere lo shooting su uno sweep: il BVP indiretto ha un bacino di convergenza stretto, e partire dal vicino gia' risolto e' quasi sempre sufficiente.
- Riga 66:  $z\_prev\_loc = []$  **dentro** il loop `pass`. Conseguenza: la seconda passata riparte dal warm start monostadio  $z\_ref$  invece che dalla soluzione gia' convergente a  $idx\_mid$  (che e' in `sol_two{idx_mid}`). Funziona, ma e' un warm start sprecato – la soluzione a  $ts$  adiacente sarebbe stata un guess migliore.
- Riga 109:  $z\_prev\_loc$  viene aggiornato **solo in caso di successo** (`ef > 0`). Corretto: se un punto fallisce, il vicino successivo riparte comunque dall'ultima soluzione buona invece che da spazzatura.
- Righe 88-98: dopo la convergenza la traiettoria viene **ri-integrata a mano**: arco 1 su  $[0, ts]$ , poi il salto  $z\_s(5) = z\_s(5) - ms1$  (riga 95), poi arco 2 su  $[ts, tf]$ . Osservare che  $z\_s(6)$  (cioe'  $\lambda_m$ ) **non viene toccato**: e' la continuita' dei costati, fatta operativamente.
- Righe 103-106: il payload a due stadi.

```
mp2 = Q*(tf - ts)      propellente del secondo stadio
ms2 = eta*mp2          sua struttura
mu  = mf - ms2
```

Nota che qui **non** si usa la formula monostadio  $mf*(1+\eta) - \eta$ : sarebbe sbagliata, perche' la struttura del primo stadio e' gia' stata buttata e non va sottratta una seconda volta.

## L'identita' nascosta: il payload dipende solo da $tf$

Vale la pena esplicitarla, perche' il codice non la scrive e cambia completamente la lettura del Task 4. Sostituendo  $mf = 1 - (1+\eta)*Q*ts - Q*(tf - ts)$  in  $\mu = mf - \eta*Q*(tf - ts)$ :

```
mu = 1 - (1+eta)*Q*ts - (1+eta)*Q*(tf - ts)
```

$$= 1 - (1+\eta)*Q*tf$$

Il payload non dipende esplicitamente da **ts**: dipende solo dal tempo di volo totale **tf**. (La stessa formula vale nel caso monostadio:  $mf*(1+\eta) - \eta$  con  $mf = 1 - Q*tf$  da' anch'essa  $1 - (1+\eta)*Q*tf$ .) Quindi:

- massimizzare il payload equivale a **minimizzare tf**;
- lo staging aiuta solo perche', alleggerendo il veicolo a **ts**, permette di raggiungere le stesse condizioni terminali in **meno tempo di combustione**;
- per **ts** fissato, il BVP interno massimizza **mf**, e siccome  $mf = m(ts+) - Q*(tf - ts)$  con  $m(ts+)$  gia' determinato da **ts**, massimizzare **mf** equivale a minimizzare **tf**, che equivale a massimizzare il payload. **L'obiettivo del livello interno e' quindi coerente con quello esterno** – cosa tutt'altro che scontata in uno schema a due livelli.

Verifica numerica con i numeri del README: monostadio  $\mu = 0.0241$  da'  $tf = (1 - 0.0241)/2.2 = 0.4436$ ; due stadi  $\mu = 0.0680$  da'  $tf = 0.4236$ . Il `validate_staging_corner.m` stampa proprio **tf** di sweep = 0.424 (riga 52). Quindi il famoso “+182% di payload” e' in realta' un taglio del 4.5% sul tempo di combustione: il payload e' una piccola differenza di numeri grandi e amplifica tutto.

**Possibile domanda d'esame** – il livello interno massimizza **mf**, ma l'obiettivo vero e' il payload. Non e' un'incoerenza? *Risposta*: no, e si dimostra. Per **ts** fissato,  $m(ts+) = 1 - (1+\eta)*Q*ts$  e' gia' determinato, quindi **mf** e il payload sono entrambi funzioni affini strettamente decrescenti di **tf**: massimizzare l'uno o l'altro equivale a minimizzare **tf** e produce la stessa traiettoria. L'unica differenza fra le due formulazioni sta nella *normalizzazione dei costati* (in particolare nel valore di  $\lambda_m(tf)$  e in  $H(tf)$ ), non nella soluzione fisica.

## %% Find optimal staging + plot (righe 114-191)

- Righe 115-119: `max()` sul vettore `pay_two` filtrato, poi rimappatura dell'indice sul vettore completo. E' l'ottimizzatore esterno: una **ricerca esaustiva su griglia**, niente di piu'.
- Righe 124-131: `report`, incluso il guadagno percentuale sul payload. Come detto sopra, quel % va letto con cautela: e' l'amplificazione di una differenza.
- Righe 134-144: payload vs **ts**, con la retta orizzontale del monostadio. La riga 144 (`lg.Position(2) = lg.Position(2) + 0.12`) e' un aggiustamento cosmetico per non sovrapporre la legenda alla `yline`.
- Righe 156-188: ri-integrazione e plot della traiettoria ottima e del profilo di massa. La riga 183 disegna il salto verticale della massa a **ts** come segmento tratteggiato – e' l'unica firma visibile dello staging.
- Nota: nessuna di queste righe verifica la corner condition. Lo script si fida del massimo di griglia.

## %% EXPORT FIGURES (righe 195-211)

- Righe 196-199: cartella `figures/` accanto allo script (creata se manca) e `slugify`, che genera i nomi dallo Name della figura; il prefisso `task4_` viene aggiunto alla riga 210, nella chiamata a `exportgraphics`.
- Righe 204-208: `theme(fig, 'light')` con `try/catch` che ricade su `fig.Color = 'w'` per MATLAB pre-R2025a. Serve a non esportare figure con lo sfondo scuro del desktop dentro il report LaTeX.

## shooting\_single (righe 215-246)

```
Lam0 = sqrt(lam_vx0^2 + lam_vy0^2);
H0 = -lam_vy0 + p.T*(Lam0 - 1/p.c);
res = [zf(2)-p.yf; zf(3)-1; zf(4); H0];
```

- Riga 215: firma. Incognite  $z0 = [lam\_vx0; lam\_vy0; lam\_y; tf]$ , 4 residui. Chiamata solo da `fsolve` alla riga 24. **Nessun blocco arguments** (riga 224): scelta di design, sta nel loop interno di `fsolve/ode45`.
- Righe 227-229: guardia  $tf \leq 0 \ || \ tf > 2$  che restituisce `1e6*ones(4,1)`. E' il trucco standard per tenere `fsolve` dentro un bracket fisico, ma va detto: rende il residuo **discontinuo**, e se `fsolve` calcola il Jacobiano per differenze finite a cavallo del bordo ottiene derivate mostruose. Funziona perche' l'ottimo e' lontano dal bordo.
- Riga 234:  $ic = [0;0;0;0;1;1]$ , con  $lam\_m0 = 1$ .
- Righe 242-243: il residuo di trasversalita'. Deriva dalla forma compatta di H valutata a  $t=0$  con  $vy=0$ ,  $m=1$ ,  $lam\_m0=1$ .
- Riga 245: i tre residui terminali.  $zf(2) = y(tf)$ ,  $zf(3) = vx(tf)$ ,  $zf(4) = vy(tf)$ : quota target, velocita' orizzontale unitaria (nondim), velocita' verticale nulla.  $x(tf)$  e' **libero** (nessun vincolo di downrange) – ed e' esattamente da li' che viene  $lam\_x(tf) = 0$ , quindi  $lam\_x = 0$  ovunque, quindi la struttura dei costati che `ode_burn` da' per scontata.

## shooting\_twostage (righe 248-298)

```
[~,Z1] = ode45(@(t,z) ode_burn(t,z,pp), [0 ts], ic, opts_ode);
z_s = Z1(end,:);
ms1 = p.eta * p.Q * ts;
z_s(5) = z_s(5) - ms1;
% lam_m is continuous across staging
[~,Z2] = ode45(@(t,z) ode_burn(t,z,pp), [ts tf], z_s, opts_ode);
```

- Riga 248: firma. Stesse 4 incognite di `shooting_single`; **ts non** e' incognita, arriva come parametro `p.ts` fissato dallo sweep.
- Righe 262-264: guardia, ora anche su  $tf \leq ts$  e  $ts \leq 0$ .
- Righe 266-267: **la stessa pp viene usata su entrambi gli archi**. Questa singola riga e' la continuita' dei costati  $lam\_vx$ ,  $lam\_vy$ ,  $lam\_y$ : `ode_burn` li ricostruisce dai parametri, quindi le loro funzioni del tempo non hanno alcun kink a  $ts$ . La rampa  $lam\_vy(t) = lam\_vy0 - lam\_y*t$  prosegue liscia attraverso lo staging.
- Riga 271-276: arco 1 su  $[0, ts]$ .
- Riga 279-281: **il salto**. Solo  $z\_s(5)$  (massa) e' modificato;  $z\_s(1:4)$  (stato cinematico) e  $z\_s(6)$  ( $lam\_m$ ) passano intatti. Il commento della riga 281 dice esplicitamente cio' che abbiamo derivato sopra.
- Righe 282-284: guardia  $m\_plus > 0$ , correttamente **prima** di integrare l'arco 2 (in `validate_staging_corner.m` la stessa guardia arriva *dopo* la propagazione – vedi quella pagina).
- Riga 287-292: arco 2 su  $[ts, tf]$ , con lo stesso RHS: stesso motore, stesso Q. L'unica cosa cambiata e'  $m$ , quindi l'accelerazione  $T/m$  fa un gradino verso l'alto.
- Riga 294-297: **residui identici a quelli del monostadio**. E qui c'e' il punto onesto da segnalare: il quarto residuo  $H0 = 0$  a  $t=0$  (riga 295) e' ereditato pari pari dal Task 1 – il commento della riga 256 lo enuncia soltanto come  $H=0$  imposed at  $t0$ , senza qualificarlo. Nel monostadio quella riga e' davvero la trasversalita' per  $tf$  libero –  $H$  e' costante lungo l'arco, quindi  $H(0)=0$  e  $H(tf)=0$  sono la stessa cosa. **Nel caso a due stadi non lo e' piu'**, perche'  $H$  salta a  $ts$ . In realta' qui  $H0 = 0$  funziona come **condizione di gauge** (fissa la scala dei costati), non come trasversalita'. E' innocuo, e il motivo e' sottile:
  - i 3 residui terminali dipendono dai costati **solo attraverso i rapporti**  $lam\_vy0/lam\_vx0$  e  $lam\_y/lam\_vx0$  (perche'  $\phi = atan2(...)$  e' invariante per riscaldamento positivo), quindi determinano da soli i due rapporti e  $tf$ , cioe' **tutta la traiettoria**;
  - la quarta incognita e' di fatto la *scala* di  $lam\_v$ , che non tocca la traiettoria;  $H0 = 0$  la fissa e rende il Jacobiano  $4 \times 4$  non singolare.

Conseguenza pratica: traiettoria,  $tf$ ,  $mf$  e payload sono **corretti**, ma i  $lam\_m$  che escono da questo script **non** soddisfano  $lam\_m(tf) = 1$  e non sono i costati "veri" a meno di un riscaldamento e di un offset additivo. Chi vuole usarli in una condizione di corner deve usarne una forma **invariante di gauge** – ed e' precisamente quello che fa `validate_staging_corner.m` con la differenza  $lam\_m(tf) - lam\_m(ts)$ .

**Possibile domanda d'esame** – nel tuo Task 4 imponi  $H(0) = 0$ . Ma se l'Hamiltoniana salta a  $t_s$ , come può essere la condizione di trasversalità per  $t_f$  libero? *Risposta:* non lo è. Nel monostadio  $H$  è costante e  $H(0)=0$  coincide con  $H(t_f)=0$ ; con lo staging  $H$  salta di  $T*|\lambda_v|*(1/m_+ - 1/m_-)$ , quindi  $H(0)=0$  e  $H(t_f)=0$  sono condizioni diverse. Nella mia formulazione  $H(0)=0$  serve a fissare la scala dei costati (che altrimenti resterebbe indeterminata per omogeneità). La traiettoria non se ne accorge, perché dipende solo dalla *direzione* del primer vector e i tre residui terminali la determinano già da soli. Nella normalizzazione “vera” del payoff-payload ( $\lambda_m(t_f)=1$ ) si avrebbe invece  $H(t_f) = \eta*Q$ , perché  $\mu = m(t_f) - \eta*Q*(t_f-t_s)$  dipende esplicitamente da  $t_f$ ; nella formulazione equivalente a minimo  $t_f$  usata nell'appendice del report si ha  $\lambda_m(t_f)=0$  e  $H(t_f)=1$ . Tutte e tre sono lo stesso problema, in tre gauge diversi.

---

## Limiti e onestà sul file

1. **Lo sweep è a 50 punti, senza raffinamento.**  $ts\_opt$  è un nodo della griglia (passo  $\sim 0.008$ ); il vero punto stazionario sta fra due nodi. Nessuna parabola sul massimo, nessun  $fminbnd$ .
  2. **La corner condition non è mai verificata qui.** Lo script trova un massimo numerico e si ferma. E' `validate_staging_corner.m` a chiudere il cerchio.
  3. **Stesso  $Q$ , stesso  $c$ , stesso  $T$  per i due stadi.** Semplificazione forte: niente motore da vuoto, niente  $I_{sp}$  diverso, niente coefficienti strutturali diversi per stadio. Il “+182%” ne è figlio.
  4. **Nessuna salita verticale.** Il veicolo parte da fermo e ruota subito: a  $t=0$  la spinta è già inclinata di  $\sim 81$  gradi ma non 90. È la variante richiesta dal Task 4, non un errore, ma non è un profilo di lancio realistico.
  5. **Nessun coast allo staging.** Jettison istantaneo e riaccensione immediata.
  6.  **$z\_prev\_loc$  azzerato a ogni passata** (riga 66): la seconda passata butta via un warm start migliore già disponibile.
  7. **Le guardie a  $1e6$**  rendono i residui discontinui: se il guess iniziale cadesse fuori bracket, `fsolve` avrebbe un Jacobiano privo di senso.
- 

## Possibili domande d'esame

**D: Allo stage separation la massa salta. I costati saltano?** R: No. La condizione di corner generale per una mappa di giunzione  $x(t_s+) = g(x(t_s-), t_s)$  è  $\lambda_m(t_s-)' = \lambda_m(t_s+)' * dg/dx-$ . Qui  $g(x, t_s) = x + \Delta t$  con  $\Delta t = (0, 0, 0, 0, -\eta*Q*t_s)'$ :  $\Delta t$  dipende solo dal *tempo* di giunzione, non dallo stato, quindi  $dg/dx- = I$  e tutti i costati sono continui. In `main_task4.m` questo si vede alla riga 280: si sottrae la massa  $z_s(5)$  e basta,  $z_s(6) = \lambda_m$  resta com'è. Se invece la massa jettisonata fosse stata scritta come funzione di  $m(t_s)$ ,  $\lambda_m$  avrebbe avuto un salto.

**D: L'Hamiltoniana è continua allo staging?** R: No, salta verso l'alto di  $T*|\lambda_v(t_s)|*(1/m_{plus} - 1/m_{minus}) > 0$ . Tutti i termini di  $H = \lambda_m y v_y - \lambda_m v_y + T*(|\lambda_v|/m - \lambda_m/c)$  sono continui tranne  $1/m$ , che aumenta di colpo. Fisicamente: buttata la struttura, l'accelerazione  $T/m$  cresce e il veicolo “guadagna merito” più in fretta. L'Hamiltoniana si conserva solo *dentro* ciascun arco (il sistema è autonomo), non attraverso la giunzione.

**D: Qual è allora la condizione di ottimalità sull'istante di staging?** R: Il coefficiente di  $dts$  nella variazione:  $d\phi/dts + H(t_s-) - H(t_s+) + \lambda_m(t_s)'*d\Delta t/dts = 0$ . Con  $\phi = m(t_f) - \eta*Q*(t_f-t_s)$  (payload) si ottiene

$$\eta*[ \lambda_m(t_f) - \lambda_m(t_s) ] = c*|\lambda_v(t_s)|*( 1/m_{plus} - 1/m_{minus} )$$

È un bilancio fra due effetti opposti del ritardare lo staging: a sinistra il guadagno (più struttura buttata al primo stadio => meno struttura al secondo, più payload), a destra la perdita (hai volato un  $dt$  in più con il veicolo pesante). `main_task4.m` non impone questa equazione: la sostituisce con uno sweep su  $t_s$  e un `max()`.

**D: Perché non risolvi direttamente il problema con  $t_s$  come quinta incognita?** R: Si può, ed è esattamente quello che fa `validate_staging_corner.m` (5 incognite, 5 residui, il quinto è la corner condition).

Lo sweep di `main_task4.m` ha due vantaggi didattici: (a) produce la **curva** payload-vs-ts, che mostra dove sta l'ottimo e quanto e' piatto attorno; (b) e' robusto – non serve indovinare **ts**, basta il warm start del monostadio e la continuation. Lo svantaggio e' la risoluzione: l'ottimo esce agganciato alla griglia.

**D: Il livello interno massimizza mf, ma il payload e'  $mf - \eta * Q * (tf - ts)$ . Non stai ottimizzando la cosa sbagliata?** R: No. Con  $\dot{m} = -Q$  costante si ha  $mf = 1 - (1+\eta) * Q * ts - Q * (tf - ts)$ , quindi sostituendo nel payload:  $\mu = 1 - (1+\eta) * Q * tf$ . Il payload dipende **solo da tf**. A **ts** fissato, sia **mf** sia **mu** sono affini strettamente decrescenti in **tf**: massimizzare l'uno o l'altro da' la **stessa traiettoria**. Le due formulazioni differiscono solo nella normalizzazione dei costati.

**D: Come mai un guadagno del +182% di payload?** R: Perche'  $\mu = 1 - (1+\eta) * Q * tf$  e' una piccola differenza di numeri grandi. Passando dal monostadio ( $tf \sim 0.444$ ) ai due stadi ( $tf \sim 0.424$ ) il tempo di combustione cala del  $\sim 4.5\%$ , ma **mu** passa da  $\sim 0.024$  a  $\sim 0.068$ . Il fattore amplificatore e'  $(1+\eta) * Q / \mu \sim 32$ . La cifra e' inoltre gonfiata dalle semplificazioni: stesso motore e stesso **eta** per i due stadi, niente atmosfera, niente drag, niente coast di separazione.

**D: Cosa fissa il vincolo superiore su ts alla riga 45?** R: Due cose.  $m_{plus} = 1 - (1+\eta) * Q * ts > 0$  da'  $ts < 1 / (Q * (1+\eta))$ : oltre quel punto il veicolo avrebbe buttato piu' massa di quanta ne abbia (propellente bruciato + struttura corrispondente supererebbero  $m_0$ ). E  $ts < tf$ , approssimato con  $0.95 * tf_{single}$  perche' il **tf** a due stadi non e' ancora noto quando si costruisce la griglia. Con  $Q=2$ ,  $\eta=0.1$  il primo bound vale 0.4545 e il secondo, piu' stringente,  $\sim 0.421$ .

# HM1/validate\_\_rocket\_\_sled.m

## Ruolo del file nel progetto

Questo e' lo **script di validazione della macchina di shooting** di HM1. Non risolve nessun task dell'homework: risolve un problema-giocattolo diverso – il *rocket sled* (slitta a razzo) a energia minima – che ha una **soluzione in forma chiusa nota**, e verifica che la catena `ode45 + fsolve` recuperi esattamente quella soluzione. L'intestazione lo dichiara: "Checks the ode45 + fsolve single-shooting machinery (same tolerances as the ascent BVP) against the analytic optimum before using it on the ascent problem" (righe 1-4).

La logica e' quella dello **strumento tarato prima dell'uso**. Il problema di ascesa indiretta di HM1 non ha soluzione analitica: quando `fsolve` non converge, o converge a qualcosa di strano, non si sa se la colpa e' (a) della derivazione delle condizioni di Pontryagin, (b) di un errore di segno nelle equazioni dei costati, (c) delle tolleranze di `ode45`, (d) del guess iniziale, oppure (e) del fatto che il problema stesso e' mal posto. Il sled toglie di mezzo tutti gli imputati tranne quelli implementativi: se il solver **non** recupera i costati analitici `lam_r0 = lam_v0 = 3/2` su un problema di cui si conosce la risposta, allora il bug e' nel codice di shooting, non nella fisica dell'ascesa.

Il file e' **autonomo**: non chiama `ode_burn.m` ne' nessun altro file di HM1. Contiene due funzioni locali (`sled_ode`, `sled_residual`) che sono repliche in miniatura della struttura usata nei `main_task*.m` – stato + costati integrati insieme, residuo terminale valutato in `tf`, `fsolve` sulle condizioni iniziali incognite dei costati. Riusa **le stesse tolleranze di ode45** del problema vero (`RelTol = 1e-10`, `AbsTol = 1e-12`) e su `fsolve` ne adotta di **piu' strette** (`1e-12` su funzione e passo, contro `1e-10` dei `main_task*.m`), quindi valida anche la scelta delle tolleranze, non solo la logica.

Il commento in riga 1 lo etichetta come "Appendix A": e' il materiale di supporto del report, non parte del flusso di soluzione.

---

## Header e dati del problema (righe 1-22)

```
% Problem: rdot = v,   vdot = u,   min J = int_0^2 u^2 dt
%          r(0)=0, v(0)=0, r(2)=1/2, v(2)=0,   tf = 2 fixed.
% Hamiltonian: H = -u^2 + lam_r*v + lam_v*u
% Costates: lam_r_dot = 0, lam_v_dot = -lam_r ; u = lam_v/2.
% Closed form: lam_r0 = lam_v0 = 3/2 => u*(t) = (3/4)*(1 - t).
```

- **Righe 6-7 – il problema.** Doppio integratore, controllo  $u$  (accelerazione) libero e **non vincolato**, costo quadratico di energia. Si parte da fermo nell'origine e si deve arrivare a  $r = 1/2$  con velocita' nulla in `tf = 2`, che e' **fissato**. E' il problema LQ piu' semplice che esista con una struttura di shooting non banale: due incognite (i due costati iniziali), due condizioni terminali.

- **Riga 8 – l'Hamiltoniana.** Scritta come

$$H = -u^2 + \text{lam}_r*v + \text{lam}_v*u$$



Attenzione al **segno del termine di costo**: e'  $-u^2$ , non  $+u^2$ . Il codice adotta la convenzione di **massimizzazione** di  $H$  (PMP nella forma "maximum principle"): massimizzare l'integrale di  $-u^2$  equivale a minimizzare l'integrale di  $u^2$ . Questa e' la stessa convenzione usata in `ode_burn.m`, dove l'angolo di spinta viene scelto come  $\phi = \text{atan2}(\text{lam\_vy}, \text{lam\_vx})$ , cioe' allineando la spinta al primer vector per **massimizzare**  $\text{lam\_v} \cdot \hat{u}$ . La coerenza fra i due file non e' un dettaglio: e' proprio cio' che il sled sta validando.

- **Riga 9 – legge di controllo e costati.** Da  $dH/du = -2u + \text{lam\_v} = 0$  segue

$$u = \text{lam\_v}/2$$

che e' un controllo **interno** (nessuna saturazione, perche'  $u$  non e' vincolato). Le equazioni dei costati vengono da  $\text{lam\_dot} = -dH/dx$ :

$$\text{lam\_r\_dot} = -dH/dr = 0 \quad (\text{r non compare in } H)$$

$$\text{lam\_v\_dot} = -dH/dv = -\text{lam\_r}$$

Quindi  $\text{lam\_r}$  e' costante e  $\text{lam\_v}$  e' **lineare nel tempo**. Notare l'analogia strutturale con HM1: anche li'  $\text{lam\_vx}$  e' costante e  $\text{lam\_vy}$  e' lineare ( $\text{lam\_vy} = \text{lam\_vy0} - \text{lam\_y} \cdot t$ ), il che produce la *linear tangent law*.

- **Riga 10 – la soluzione chiusa.** Derivazione. Poniamo  $\text{lam\_r} = a$  costante e  $\text{lam\_v} = b - a \cdot t$ . Allora  $u = (b - a \cdot t)/2$  e integrando da fermo:

$$v(t) = (b \cdot t - a \cdot t^2/2)/2$$

$$r(t) = (b \cdot t^2/2 - a \cdot t^3/6)/2$$

Imponendo le due condizioni terminali con  $t_f = 2$ :

$$v(2) = (2b - 2a)/2 = b - a = 0 \quad \rightarrow \quad b = a$$

$$r(2) = (2b - (8/6)a)/2 = b - (2/3)a = 1/2$$

Sostituendo  $b = a$ :  $a/3 = 1/2$ , cioe'  $a = 3/2$ . Da cui  $\text{lam\_r0} = \text{lam\_v0} = 3/2$  e

$$u \cdot(t) = (3/2 - (3/2) \cdot t)/2 = (3/4) \cdot (1 - t)$$

Il controllo ottimo e' una **rampa lineare** che parte da 0.75, si annulla a  $t = 1$  e diventa negativa (frenata) nella seconda meta': accelera, poi frena per arrivare a velocita' nulla. Fisicamente sensato, e questo e' il punto: la risposta e' verificabile *a mano*.

- **Riga 12 – `clear; close all; clc;`.** Questo e' uno **script**, non una funzione e non una classe di test. Conseguenza pratica: non puo' essere raccolto da `runtests`, e non lancia un errore in caso di fallimento (vedi righe 44-48). Va eseguito a mano dalla cartella HM1/.
- **Righe 15-17 – dati.**  $t_f = 2$ ,  $r_f = 1/2$ ,  $v_f = 0$ . Nessuna nondimensionalizzazione: il sled e' gia' adimensionale per costruzione.
- **Righe 19-22 – tolleranze.** Sono deliberatamente **le stesse** (anzi, su `fsolve` piu' strette) di quelle usate nei `main_task*.m`: `ode45` con `RelTol = 1e-10`, `AbsTol = 1e-12`; `fsolve` con `FunctionTolerance = StepTolerance = 1e-12`, `Display` spento, `MaxIterations = 500`, `MaxFunctionEvaluations = 5000`. Se il sled passasse a tolleranze lasche ma non a quelle vere, la validazione non direbbe nulla di utile. Il fatto che siano esplicitate (non lasciate ai default) segue la convenzione di riproducibilita' del repo.

**Possibile domanda d'esame** – Perche' l'Hamiltoniana ha il segno  $-u^2$  e non  $+u^2$ ? *Risposta:* Perche' il codice usa la forma del Principio del Massimo: il controllo ottimo **massimizza**  $H$ . Con  $H = -u^2 + \text{lam} \cdot f$ , massimizzare l'integrale di  $-u^2$  e' identico a minimizzare l'energia. Se scrivessi  $H = +u^2 + \text{lam} \cdot f$  dovrei minimizzare  $H$ , e la condizione  $dH/du = 0$  darebbe  $u = -\text{lam\_v}/2$ , con un segno invertito nei costati. Le due formulazioni sono equivalenti purché si resti coerenti; l'importante e' che il sled usi **la stessa convenzione** di `ode_burn.m`, altrimenti non validerebbe nulla.

## Single shooting: fsolve sui costati iniziali (righe 24-27)

```
lam_guess = [0; 0]; % away from the analytic (3/2, 3/2)
[lam_sol, res, ef] = fsolve(@(L) sled_residual(L, tf, rf, vf, ...
      opts_ode), lam_guess, opts_fs);
```

- **Riga 25 – il guess iniziale.** `[0; 0]`, con il commento esplicito “away from the analytic (3/2, 3/2)”. La scelta e’ voluta: partire dal valore corretto renderebbe il test vuoto (`fsolve` restituirebbe subito il punto di partenza).

**Onesta’ richiesta:** questo test e’ meno severo di quanto sembri. Le dinamiche del sled – stato e costati – sono **lineari**, quindi la mappa di flusso e’ lineare e il residuo e’ una funzione **affine** delle incognite `L`. Un metodo di Newton (e `fsolve` in modalita’ **trust-region-dogleg** lo e’) risolve un sistema affine **in un solo passo**, esattamente, qualunque sia il guess. Quindi il sled **non** mette alla prova la robustezza di `fsolve` sulla non linearita’ (che e’ esattamente cio’ che rende difficile il problema di ascesa e che obbliga alla *continuation*). Cio’ che valida davvero e’: l’accuratezza di `ode45`, l’assemblaggio del residuo, le convenzioni di segno dei costati, il cablaggio della function handle. Sono i bug piu’ comuni, ma non sono tutti i bug possibili.

- **Righe 26-27 – la chiamata.** Le uscite sono `[x, fval, exitflag]` nella firma di `fsolve`, quindi `res` e’ il **vettore residuo valutato nella soluzione** (non il “residuo” nel senso di norma) e `ef` e’ l’exitflag (`> 0` = convergenza). Le uniche incognite sono i due costati iniziali: lo stato iniziale e’ noto e fissato dentro `sled_residual`.

---

## Report e cross-check sul controllo (righe 29-48)

```
[tt, S] = ode45(@(t,s) sled_ode(t,s), [0 tf], ...
      [0;0;lam_r0;lam_v0], opts_ode);
u_num = S(:,4)/2; % u = lam_v/2
u_ana = 0.75*(1 - tt);
```

- **Righe 31-36 – cosa viene stampato.** Quattro diagnostiche: l’exitflag, i due costati recuperati confrontati con 1.5, la norma del residuo terminale, e l’errore sui costati `||lam - [1.5; 1.5]||`. Notare la riga 36: `norm(lam_sol - 1.5)` sfrutta l’espansione scalare di MATLAB (sottrae 1.5 a entrambe le componenti) – corretto, ma implicito.
- **Righe 38-42 – il secondo livello di verifica.** Non basta che i costati iniziali siano giusti: si **reintegra** la traiettoria con i costati trovati e si confronta il **profilo di controllo intero** `u_num(t) = lam_v(t)/2` contro la rampa analitica `u*(t) = 0.75*(1 - t)`, stampando `max |u_num - u*|`.

Questo e’ un controllo piu’ forte del solo confronto sui costati iniziali: verifica che la **propagazione temporale** dei costati sia corretta lungo tutto l’arco, non solo il punto di partenza. Un errore di segno in `lam_v_dot`, per esempio, potrebbe (in linea di principio) essere compensato da un valore iniziale sbagliato in modo da soddisfare comunque le condizioni terminali; il confronto sul profilo `u(t)` lo scoprirebbe.

- **Righe 44-48 – il criterio di PASS.** Due condizioni **in AND**: `ef > 0` (`fsolve` dichiara convergenza) e `norm(lam_sol - 1.5) < 1e-6`.

La tolleranza di accettazione e’ quindi **1e-6 sui costati**, molto piu’ larga delle tolleranze numeriche usate (`1e-10 / 1e-12`): e’ una soglia di *correttezza strutturale*, non di precisione numerica. In pratica il codice, con quelle tolleranze, recupera i costati a molte cifre in piu’.

**Limite da dichiarare:** il fallimento produce solo una `fprintf` con la scritta **FAIL**. Lo script **non lancia un errore**, non ritorna un exit code, non e’ una `matlab.unittest.TestCase`. Quindi non entra in `runtests('HM1/tests')` e non puo’ rompere una pipeline di CI: se qualcuno rompesse il solver, questo file lo direbbe soltanto a chi lo guarda. Trasformarlo in una classe di test con una `verifyEqual(lam_sol,`

[1.5;1.5], 'AbsTol', 1e-6) sarebbe il naturale passo successivo – il resto della repo (HM1/tests/) segue già quella convenzione.

**Nota:** il max |u\_num - u\*| stampato alla riga 42 **non entra** nel criterio di PASS/FAIL. E' solo informativo.

**Possibile domanda d'esame** – Se il solver recupera i costati iniziali corretti, perché serve anche il confronto sul profilo  $u(t)$ ? *Risposta:* Perché le condizioni terminali sono solo due numeri: più errori possono in linea di principio cancellarsi e produrre comunque un residuo nullo in `tf`. Confrontare l'intera storia del controllo verifica anche la dinamica dei costati **lungo** l'arco (`lam_v_dot = -lam_r`), cioè la propagazione, non solo il punto iniziale. E' un test sulla funzione, non su due scalari.

---

## sled\_ode (righe 51-61)

```
function ds = sled_ode(~, s)
    ds = [ s(2);          % rdot = v
           s(4)/2;        % vdot = u = lam_v/2
           0;             % lam_r_dot = 0
          -s(3) ];        % lam_v_dot = -lam_r
end
```

- **Riga 51 – firma.** Il tempo è scartato con `~`: il sistema aumentato (stato + costati) è **autonomo**, il tempo non compare esplicitamente. E' esattamente la stessa struttura di `ode_burn.m`, con una differenza importante: lì i costati `lam_vx`, `lam_vy` **non** sono integrati, sono ricostruiti in forma chiusa da `p` e `t` (per questo `ode_burn` usa `t` esplicitamente e integra solo 6 stati). Qui invece i due costati sono **stati veri** dell'integrazione, quindi il vettore è `[r; v; lam_r; lam_v]`, 4 componenti.
- **Righe 57-60 – il sistema aumentato.** La sostituzione della legge di controllo  $u = \text{lam\_v}/2$  dentro la dinamica (riga 58) è il cuore del metodo indiretto: il controllo sparisce come variabile indipendente e sopravvive solo come funzione dei costati. Quel che resta è un sistema di ODE puro, integrabile con `ode45`, la cui unica indeterminazione sono le **condizioni iniziali dei costati** – che è esattamente ciò su cui si fa shooting.
- Notare che il sistema è **lineare** in `s` (matrice costante 4x4). Da qui la conseguenza detta sopra: il residuo di shooting è affine.

---

## sled\_residual (righe 63-74)

```
function res = sled_residual(L, tf, rf, vf, opts_ode)
    [~, S] = ode45(@(t,s) sled_ode(t,s), [0 tf], ...
                   [0; 0; L(1); L(2)], opts_ode);
    res = [S(end,1) - rf; S(end,2) - vf];
end
```

- **Riga 63 – firma.** Prende le due incognite `L = [lam_r0; lam_v0]` e tutti i dati del problema passati per valore (nessuna variabile globale, nessuna dipendenza dal workspace dello script): la funzione è pura, e quindi il residuo è riproducibile.
- **Riga 72 – integrazione forward.** Lo stato iniziale è `[0; 0; L(1); L(2)]`: posizione e velocità **note** (partenza da fermo nell'origine), costati **incogniti**. Questa è la definizione stessa di *single shooting*: si trasforma un problema ai limiti (BVP) in una sequenza di problemi ai valori iniziali (IVP), e si cerca il valore iniziale che azzecca le condizioni al bordo opposto.

- **Riga 73 – il residuo.** Due componenti,  $r(tf) - rf$  e  $v(tf) - vf$ : tante quante le incognite. Sistema quadrato  $2 \times 2$ , come deve essere perché `fsolve` abbia senso.

**Il conto delle incognite e' il punto pedagogico.** Nel sled: 2 incognite (costati iniziali), 2 residui (condizioni terminali), **tf fissato**. Nel problema vero di HM1 (Task 1): 4 incognite ( $lam\_vx0$ ,  $lam\_vy0$ ,  $lam\_y$ ,  $tf$ ) e 4 residui ( $y(tf) - yf$ ,  $vx(tf) - 1$ ,  $vy(tf)$ , e  $H(0) = 0$ ), perché il tempo finale e' **libero** e questo aggiunge una condizione di trasversalita' ( $H = 0$ ) e un'incognita.

**Limite di copertura da dichiarare apertamente:** il sled **non** valida ne' la condizione  $H = 0$  a tempo libero, ne' la normalizzazione  $lam\_m0 = 1$ , ne' la gestione dell'arco di coast di Task 3 (funzione di switching), ne' lo staging di Task 4. Valida il **nucleo** – integrazione + residuo + Newton – non gli strati sopra. Va detto: e' una validazione della *plumbing*, non della formulazione completa.

**Possibile domanda d'esame** – Perché il sled ha 2 incognite e HM1 ne ha 4, se in entrambi lo stato e' completamente noto all'istante iniziale? *Risposta:* Perché in HM1 il tempo finale e' **libero**. Nel sled  $tf = 2$  e' dato, quindi le incognite sono solo i costati iniziali che non hanno condizione iniziale ( $lam\_r0$ ,  $lam\_v0$ ). In HM1  $tf$  e' esso stesso un'incognita, e la corrispondente condizione di trasversalita' e' l'annullamento dell'Hamiltoniana; inoltre HM1 ha una condizione terminale in piu' ( $y(tf)$  oltre alle due velocita'). Il conteggio torna: 3 costati incogniti +  $tf = 4$  incognite, 3 condizioni terminali +  $H = 0 = 4$  residui.

## Possibili domande d'esame

**D: Perché validare un solver indiretto contro un problema con soluzione in forma chiusa, invece che contro un metodo diretto o un altro codice?** R: Perché una soluzione chiusa e' un **oracolo esatto**: non ha errore numerico, non ha ipotesi implicite, e non puo' essere sbagliata "allo stesso modo" del codice che sto testando. Confrontare due solver numerici mi dice solo che sono d'accordo, non che hanno ragione (potrebbero condividere lo stesso errore di segno nei costati). Il sled invece mi da' il valore vero  $lam\_r0 = lam\_v0 = 3/2$  derivabile a mano in cinque righe: se il codice non lo riproduce, il codice ha torto, punto. In piu', un oracolo esatto consente di misurare l'**errore assoluto** ( $\max |u\_num - u^*|$ ), non solo la coerenza interna.

**D: Che cos'e' esattamente il "single shooting" e cosa lo distingue dal collocation usato in HM2?** R: Lo shooting converte il BVP in un problema di ricerca di radici sulle sole condizioni iniziali incognite: si integra **in avanti** l'intero arco con `ode45` e si annulla il residuo al bordo finale con `fsolve`. Le incognite sono poche (2 nel sled, 4 in HM1) ma il residuo e' **estremamente non lineare** nelle incognite, perché passa attraverso l'intera propagazione – piccole variazioni dei costati iniziali possono far esplodere lo stato finale. Il collocation (HM2) discretizza invece l'intera traiettoria e tratta *tutti* i nodi come incognite: molte piu' variabili, ma ciascun vincolo e' locale e il problema e' molto meglio condizionato. Lo shooting compra accuratezza (integratore adattivo,  $RelTol = 1e-10$ ) al prezzo del condizionamento; il collocation fa il contrario.

**D: Il guess iniziale e' [0; 0], lontano dalla soluzione. Vuol dire che il solver e' robusto?** R: No, e va detto onestamente. Il sistema aumentato del sled e' lineare, quindi il residuo e' una funzione affine delle incognite e Newton lo risolve esattamente in un passo qualunque sia il punto di partenza. La convergenza da [0;0] non dimostra robustezza rispetto alla non linearita'. E' proprio perché il problema di ascesa **non** ha questa proprieta' che HM1 deve ricorrere alla *continuation* (partire da Q con  $T/W \sim 1.8$  e fare warm start sui vicini). Il sled valida la correttezza, non la robustezza.

**D: Quali parti della macchina di HM1 il sled NON valida?** R: Diverse, e sono le piu' delicate: (1) la condizione di trasversalita'  $H(tf) = 0$  per tempo finale libero (nel sled  $tf$  e' fissato); (2) la normalizzazione  $lam\_m0 = 1$  che sfrutta l'omogeneita' di grado 1 di  $H$  nei costati; (3) la valutazione di  $H = 0$  all'istante **iniziale** invece che finale, dove diventa algebrica ( $H(0) = -lam\_vy0 + T*(|lam\_v0| - 1/c)$ ); (4) la funzione di switching e la giunzione dell'arco di coast di Task 3; (5) lo staging di Task 4. Il sled valida il nucleo `ode45` + residuo + `fsolve` e le convenzioni di segno di Pontryagin. Gli strati sopra restano coperti solo dai test in `HM1/tests/` e dalla plausibilita' fisica dei risultati.

**D: La tolleranza di PASS e'  $1e-6$  sui costati mentre l'ODE gira a  $1e-10$ . Non e' incoerente?** R: No, sono due cose diverse.  $1e-10 / 1e-12$  sono le tolleranze **numeriche** dell'integratore e del root finder: definiscono

quanto accuratamente il solver insegue la propria soluzione.  $1e-6$  e' invece la soglia di **accettazione del test**: distingue “il codice implementa il problema giusto” da “il codice implementa un problema diverso”. Un errore di segno o una formula sbagliata produrrebbe un errore di ordine unitario, non  $1e-7$ . La soglia larga rende il test robusto rispetto a differenze di piattaforma/versione MATLAB senza perdere potere diagnostico.

**D: Come miglioreresti questo file?** R: Tre cose concrete. (1) Convertirlo in una `matlab.unittest.TestCase` in `HM1/tests/`, così' che `runtests` lo esegua e un fallimento diventi un errore vero invece che una `fprintf`. (2) Includere `max |u_num - u*|` nel criterio di PASS, oggi solo stampato. (3) Aggiungere un secondo benchmark **a tempo libero** (per esempio il classico min-time double integrator con controllo limitato, che ha soluzione bang-bang nota) per validare anche la condizione  $H = 0$  e la struttura di switching, che il sled a energia minima e tempo fissato non tocca.

# HM1/validate\_\_staging\_\_corner.m

## Ruolo del file nel progetto

Diciamolo subito e senza giri di parole: **questo non e' uno script di produzione, e' uno script di sanity-check**. Non calcola nessun risultato nuovo per il report, non genera figure, non entra in nessuna suite `matlab.unittest` (non e' una classe di test, e' uno script `clear; close all; clc` con `fprintf` finale). Il suo unico scopo e' **verificare che l'ottimo trovato dallo sweep di `main_task4.m` sia davvero il punto stazionario che la teoria prevede**, e non un artefatto della griglia o di un errore di bookkeeping delle masse.

Il meccanismo e' una **cross-validation fra due strade indipendenti** verso lo stesso numero:

- **strada A (`main_task4.m`)** – ottimizzazione di *ordine zero*: fissa `ts`, risolvi il BVP a 4 incognite, leggi il payload; ripeti su 50 valori di `ts`; prendi il `max()`;
- **strada B (questo file)** – condizione necessaria di *primo ordine*: promuovi `ts` a **quinta incognita** dello shooting e aggiungi la **corner condition** (Weierstrass-Erdmann all'interior junction) come **quinto residuo**; una sola chiamata a `fsolve` restituisce direttamente `ts*`.

Se A e B danno lo stesso `ts`, allora: (i) la corner condition e' stata derivata giusta; (ii) lo sweep non e' finito su un massimo spurio; (iii) le formule di payload e di massa jettisonata sono coerenti fra i due file. Se divergessero, uno dei due sarebbe sbagliato – e non si saprebbe quale, ma si saprebbe che c'e' un problema. Questo e' l'**invariante giusto** da controllare, perche' e' l'unico che mette alla prova la *derivazione analitica* (la parte in cui e' facile sbagliare segni e normalizzazioni) contro un risultato ottenuto senza usarla.

Il file corrisponde allo **Step 7 dell'Appendice C del report** (HM1/report/chapters/AppendixStaging.tex), come dichiara la riga 1. Riusa `ode_burn.m` – stesso RHS, stessa fisica, stessi parametri di `main_task4.m` (righe 16-19): se cambiassero i parametri, il confronto non sarebbe piu' valido – anche perche' i valori di riferimento dello sweep sono hardcoded (righe 51-53).

C'e' pero' un **secondo controllo, piu' sottile e piu' interessante**, nascosto alle righe 62-72: una *cautionary check* che dimostra che la forma della corner condition deve essere **invariante di gauge**. Ne parliamo alla fine – e' il pezzo piu' istruttivo del file.

---

## La condizione che stiamo validando

Riportata verbatim nell'header (righe 7-9):

```
eta*[lam_m(tf) - lam_m(ts)] = c*|lam_v(ts)|*(1/m_plus - 1/m_minus)
con m_minus = 1 - Q*ts, m_plus = 1 - (1+eta)*Q*ts
```

Da dove viene (derivazione completa nella pagina di `main_task4.m`, qui il succo). Allo staging la mappa di giunzione e'  $m(ts+) = m(ts-) - \eta * Q * ts$ , con tutto il resto continuo. Siccome il salto **non dipende dallo stato** ma solo dal *tempo* di giunzione:

- **tutti i costati sono continui** ( $d\Delta/dx = 0 \Rightarrow$  la Jacobiana della mappa di salto e' l'identita');;
- l'**Hamiltoniana salta**, di

$$H(ts+) - H(ts-) = T*|\lambda_v(ts)|*(1/m_{plus} - 1/m_{minus}) > 0$$

(l'unico termine discontinuo in  $H = \lambda_y v_y - \lambda_m v_y + T*(|\lambda_v|/m - \lambda_m/c)$  e'  $T*|\lambda_v|/m$ , perche'  $m$  scende);

- il coefficiente di  $dt$  nella variazione fissa **quanto** deve saltare  $H$ :

$$H(ts+) - H(ts-) = \eta * Q * [\lambda_m(tf) - \lambda_m(ts)]$$

Uguagliando le due espressioni e dividendo per  $Q$  (con  $T = c*Q$ ) si ottiene la corner condition. **Entrambi i membri sono positivi** ( $\lambda_m$  e' strettamente crescente perche'  $\lambda_{m\_dot} = (T/m^2)*|\lambda_v| > 0$ , e  $1/m_{plus} > 1/m_{minus}$  perche'  $m_{plus} < m_{minus}$ ): e' un buon controllo di segno gratuito.

Lettura fisica: a destra il **beneficio propulsivo** dello staging (il salto di accelerazione  $T/m$ , prezzato dal primer vector  $|\lambda_v|$  = quanto vale un incremento di  $\Delta v$  in quell'istante); a sinistra la **penalita' strutturale** (stadiare piu' tardi significa portarsi dietro  $\eta*Q$  di massa inerte in piu' per unita' di tempo). All'ottimo si pareggiano, che e' esattamente  $dtf/dts = 0$ .

## %% Parameters (righe 15-27)

```
c = 0.6; eta = 0.1; yf = 0.04; Q = 2; T = c * Q;
opts_ode = odeset('RelTol', 1e-10, 'AbsTol', 1e-12);
opts_fs = optimoptions('fsolve', 'Display', 'off', ...
    'MaxIterations', 500, 'MaxFunctionEvaluations', 5000, ...
    'FunctionTolerance', 1e-12, 'StepTolerance', 1e-12);
```

- Righe 16-20: **identici a main\_task4.m** (il commento della riga 15 lo dichiara). E' condizione necessaria per il confronto: se i parametri divergessero, i due  $ts$  non avrebbero motivo di coincidere.
- Riga 22: tolleranze dell'integratore (RelTol 1e-10, AbsTol 1e-12), **identiche** a quelle di main\_task4.m (riga 14).
- Righe 23-25: tolleranze di fsolve **piu' strette** di main\_task4.m (riga 17): 1e-12 invece di 1e-10 su funzione e passo (i cap MaxIterations = 500 e MaxFunctionEvaluations = 5000 sono invece gli stessi). Sensato: qui la corner condition e' un residuo *aggiuntivo* e piu' delicato dei tre terminali, e vogliamo che  $ts$  esca con piu' cifre significative dello sweep con cui lo confrontiamo.

## %% Step 1: warm-start from a fixed-ts inner BVP (righe 29-36)

```
ts0 = 0.33;
p_inner = p; p_inner.ts = ts0;
z4_guess = [0.6; 3.8; 14; 0.42];
[z4, ~, ef1] = fsolve(@z shooting_inner(z, p_inner, ...
    opts_ode), z4_guess, opts_fs);
```

- Riga 30:  $ts0 = 0.33$ , cioe' **gia' vicinissimo** alla risposta attesa (0.336). Va detto con onesta': il warm start e' *informato dalla risposta*. Non e' una circolarita' logica (il solve a 5 incognite deve comunque far annullare la corner condition, che a  $ts=0.33$  **non** e' soddisfatta), ma e' una scorciatoia: se si partisse da  $ts0 = 0.10$  il bacino di convergenza dello shooting indiretto potrebbe non bastare. Lo script non lo dice.
- Riga 32: guess  $[\lambda_{vx0}; \lambda_{vy0}; \lambda_y; tf] = [0.6; 3.8; 14; 0.42]$ . I primi tre sono gli stessi di main\_task4.m;  $tf = 0.42$  e' pero' gia' il valore del caso *a due stadi* (in main\_task4.m il guess monostadio era 0.30).
- Righe 33-36: si risolve prima il BVP interno a  $ts$  **fissato** (4 incognite, 4 residui – esattamente la shooting\_twostage di main\_task4.m). Serve solo a produrre un punto di partenza consistente per il solve aumentato. Se non converge, error() immediato.

---

## %% Step 2: augmented solve (righe 38-41)

```
w_guess = [z4; ts0];  
[w, res, ef] = fsolve(@(w) shooting_corner(w, p, opts_ode), ...  
    w_guess, opts_fs);
```

- Riga 40: si appende `ts0` al vettore delle incognite: da 4 a **5 incognite** [`lam_vx0`; `lam_vy0`; `lam_y`; `tf`; `ts`] (la riga 39 e' il commento che elenca i cinque residui).
- Riga 41: un solo `fsolve` su 5 residui. **Struttura del sistema 5x5** (vale la pena capirla, perche' spiega perche' e' ben posto):
  - i **3 residui terminali** ( $y(tf)=y_f$ ,  $v_x(tf)=1$ ,  $v_y(tf)=0$ ) dipendono dai costati **solo attraverso i rapporti** `lam_vy0/lam_vx0` e `lam_y/lam_vx0` (l'angolo  $\phi = \text{atan2}(\text{lam\_vy}, \text{lam\_vx})$  e' invariante per riscalamento positivo). Insieme a `tf`, quei tre residui determinano **la traiettoria**, dato `ts`;
  - il residuo **corner** determina `ts` – ed e' l'unico che porta informazione di ottimalita' sulla giunzione;
  - il residuo **H0 = 0** fissa la **scala** dei costati (che altrimenti resterebbe libera per omogeneita': le condizioni necessarie sono omogenee di grado 1 in `lambda`). Non e' – nel caso a due stadi – la condizione di trasversalita' per `tf` libero, perche'  $H$  salta a `ts` e quindi  $H(0)=0$  non implica  $H(tf)=0$ . E' un **gauge fixing**, e la sua innocuita' e' garantita dal fatto che la corner condition e' scritta in forma invariante di gauge (vedi sotto).

---

## %% Extract and report (righe 43-60)

```
payload = mf - eta*Q*(tf - ts);  
...  
if ef > 0 && abs(ts - 0.336) < 5e-3  
    fprintf('\nPASS: corner solve matches the swept optimum.\n');
```

- Riga 47: stessa formula di payload di `main_task4.m` (riga 106): `mf` meno la struttura del **secondo** stadio, `ms2 = eta*Q*(tf-ts)`. Coerente per costruzione.
- Righe 51-53: i valori di riferimento dello sweep sono **hardcoded nelle stringhe di formato**: (`sweep: 0.336`), (`sweep: 0.424`), (`sweep: 0.068`).
- Riga 56: il **criterio di PASS**: `ef > 0 && abs(ts - 0.336) < 5e-3`.

Qui vanno fatte tre osservazioni oneste.

1. **Lo script non ri-esegue `main_task4.m`**. Confronta contro tre *costanti letterali* trascritte a mano da una run precedente. Se cambiassero `c`, `eta`, `yf` o `Q`, quelle costanti diventerebbero stantie e il test darebbe FAIL pur essendo il codice corretto. E' un **oracolo registrato**, non un oracolo ricalcolato. E' una scelta legittima per uno script di verifica una-tantum, ma e' una fragilita' da conoscere.
2. **La tolleranza `5e-3` non e' arbitraria, ed e' la scelta giusta**. Lo sweep di `main_task4.m` usa 50 punti su un intervallo di circa `[0.01, 0.42]`, quindi ha un passo di griglia di circa 0.008: il suo `ts_opt` e' **agganciato a un nodo** e non puo' essere piu' preciso di ~mezzo passo. Una tolleranza di `5e-3` e' esattamente dell'ordine di mezzo passo di griglia. Chiedere `1e-6` sarebbe stato scorretto: si starebbe pretendendo dallo sweep una precisione che non ha. (Corollario onesto: e' il **solve di questo file** ad essere accurato, non lo sweep. Il numero "buono" di `ts*` e' quello che esce da qui.)
3. **Il `norm(res)` viene stampato ma non usato come gate** (riga 54). Un test piu' severo avrebbe messo anche `norm(res) < 1e-8` nella condizione di PASS: `fsolve` puo' restituire `ef > 0` con criteri di arresto (es. `StepTolerance`) che non garantiscono un residuo piccolo. Nella pratica il residuo esce a `1e-10` o meglio (cosi' riporta il report), ma il gate non lo verifica.

Da segnalare anche una piccola incoerenza documentale sul valore attribuito allo **sweep**: il README di HM1 e la tabella dei risultati di Task 4 nel report (`Task4.tex`, `ts* = 0.337`) danno 0.337, mentre le stringhe hardcoded



di questo script (righe 51-53, etichettate proprio `sweep:`), il testo di `Task4.tex` e lo Step 7 dell'Appendice C danno 0.336, che è il numero del **corner solve**. È una differenza nel terzo decimale, del tutto compatibile con l'aggancio alla griglia dello sweep (passo  $\sim 0.008$ ), ma la provenienza dei due numeri va tenuta distinta: 0.337 = nodo della griglia, 0.336 = zero della corner condition.

## %% Check: the burnout reference lam\_m(tf) is essential (righe 62-72)

```
[w_bad, ~, ef_bad] = fsolve(@(w) shooting_corner_wrong(w, p, ...
    opts_ode), [z4; ts0], opts_fs);
fprintf(' eta*lam_m(ts) alone => ts = %.6f (spurious)\n', ...
    w_bad(5));
```

**Questa è la parte più importante del file**, e non è un semplice “controesempio decorativo”: è una dimostrazione numerica di **invarianza di gauge**.

Il punto. Il BVP integra  $\text{lam}_m$  a partire da  $\text{lam}_m(0) = 1$  (riga 87). Quel 1 **non è fisica, è una convenzione**. Le condizioni necessarie hanno una libertà a **due parametri**:

```
lam -> k*lam      (k > 0: scala globale di tutti i costati)
lam_m -> lam_m + b (offset additivo del solo lam_m)
```

L'offset  $b$  è ammissibile perché  $\text{lam}_m$  entra in  $H$  solo tramite  $-Q*\text{lam}_m$  (un termine costante nel tempo... a meno del salto, che però è preservato) e perché  $\text{lam}_m\dot{=} (T/m^2)*|\text{lam}_v|$  **non dipende da  $\text{lam}_m$  stesso**: la sua ODE è un'integrazione pura, quindi cambiare la costante d'integrazione trasla l'intera curva  $\text{lam}_m(t)$  rigidamente. Nel report, la formulazione a minimo  $t_f$  porta a  $\text{lam}_m(t_f) = 0$ ; nella formulazione a massimo payload porta a  $\text{lam}_m(t_f) = 1$ ; il codice usa  $\text{lam}_m(0) = 1$ . **Sono tre gauge dello stesso problema**, e la traiettoria è la stessa in tutti e tre.

Conseguenza: qualunque condizione di ottimalità scritta in termini di  $\text{lam}_m$  deve essere **invariante** sotto  $(k, b)$ , altrimenti il suo zero dipende da una convenzione arbitraria. La forma

$$\text{eta} * [\text{lam}_m(t_f) - \text{lam}_m(t_s)] = c * |\text{lam}_v(t_s)| * (1/m_{\text{plus}} - 1/m_{\text{minus}})$$

lo è: - la **differenza**  $\text{lam}_m(t_f) - \text{lam}_m(t_s)$  **cancella l'offset  $b$** ; - entrambi i membri sono **omogenei di grado 1 in  $k$**  ( $\text{lam}_m(t_f) - \text{lam}_m(t_s)$  è l'integrale di  $(T/m^2)*|\text{lam}_v|$ , che scala con  $k$ , e a destra  $|\text{lam}_v(t_s)|$  scala con  $k$ ).

La forma

$$\text{eta} * \text{lam}_m(t_s) = c * |\text{lam}_v(t_s)| * (1/m_{\text{plus}} - 1/m_{\text{minus}})$$

(riga 185, `shooting_corner_wrong`) **non lo è**:  $\text{lam}_m(t_s)$  da solo si porta dietro l'offset arbitrario  $b = 1$  ereditato da  $\text{lam}_m(0) = 1$ . Il suo zero è quindi **una funzione della convenzione**, e infatti cade altrove. L'appendice del report riporta il valore spurio:  $t_s \sim 0.225$  invece di 0.336.

- Righe 65-72: il solve “sbagliato” viene lanciato con lo **stesso warm start** `[z4; ts0]` del solve corretto, e converge (`ef_bad > 0`) – ecco perché l'errore è insidioso: **non esplode**, ti dà un numero perfettamente plausibile.
- Riga 71: se non converge, lo script lo dice e basta. Non c'è assertion su `w_bad(5)`: il check è **informativo**, stampa e non fallisce mai. Onesta': non è un test, è una demo.

**Possibile domanda d'esame** – perché nella corner condition compare la *differenza*  $\text{lam}_m(t_f) - \text{lam}_m(t_s)$  e non semplicemente  $\text{lam}_m(t_s)$ ? *Risposta*: perché la condizione deve essere indipendente dalla normalizzazione dei costati.  $\text{lam}_m$  obbedisce a  $\text{lam}_m\dot{=} (T/m^2)*|\text{lam}_v|$ , che non contiene  $\text{lam}_m$ : cambiare la sua costante d'integrazione trasla rigidamente tutta la curva. Il mio shooting parte da  $\text{lam}_m(0) = 1$ , la teoria a minimo tempo vuole  $\text{lam}_m(t_f) = 0$ , quella a massimo payload vuole  $\text{lam}_m(t_f) = 1$ . Scrivendo la condizione come differenza, l'offset si cancella e posso valutarla

direttamente sui costati che il BVP integra. `shooting_corner_wrong` dimostra numericamente cosa succede se lo si dimentica: converge lo stesso, ma su un `ts` spurio (~0.225 invece di 0.336).

---

## propagate (righe 75-94)

```
ic = [0;0;0;0;1;1]; % lam_m0 = 1
[~, Z1] = ode45(@(t,z) ode_burn(t,z,pp), [0 ts], ic, opts_ode);
zs_minus = Z1(end,:);
z_plus = zs_minus;
z_plus(5) = z_plus(5) - p.eta*p.Q*ts; % jettison
[~, Z2] = ode45(@(t,z) ode_burn(t,z,pp), [ts tf], z_plus, opts_ode);
zf = Z2(end,:);
```

- Riga 75: firma. Restituisce **due** cose: lo stato finale `zf` e lo stato a `ts^-` (`zs_minus`), *prima* del jettison. Serve entrambi: `zf(6) = lam_m(tf)` e `zs(6) = lam_m(ts)` per la corner condition, `zs(5) = m_minus` per il salto di massa. E' il **refactoring** che `main_task4.m` non ha fatto (la' la propagazione e' duplicata a mano dentro `shooting_twostage` e di nuovo nel corpo dello script).
- Righe 84-85 (dove `pp` viene costruita) + righe 88 e 92 (le due chiamate a `ode45`): la **stessa struct** `pp` viene passata a `ode_burn` su entrambi gli archi. E' cosi' che i costati `lam_vx`, `lam_vy`, `lam_y` restano continui: sono formule analitiche nei parametri, non stati integrati, quindi non hanno alcun kink a `ts`.
- Riga 87: `lam_m(0) = 1`. Il gauge.
- Riga 91: **il salto**. Solo la componente 5 (massa). `z_plus(6) = lam_m` **non viene toccato**: e' la continuita' del costato di massa, che vale perche' la massa jettisonata `eta*Q*ts` dipende solo da `ts` e non da `m(ts)`.
- Riga 92: arco 2 con lo **stesso** RHS: stesso `T`, stesso `Q`, stesso `c`. L'unica cosa che cambia e' `m`, quindi l'accelerazione `T/m` fa un gradino verso l'alto.

**Possibile domanda d'esame** – se avessi modellato il jettison come “butto una frazione `k` della massa corrente”, cosa sarebbe cambiato? *Risposta*: tutto. Con `m(ts+) = (1-k)*m(ts-)` la mappa di salto dipende dallo stato, la sua Jacobiana rispetto a `m(ts-)` vale `1-k` e non `1`, e la condizione di corner `lam(ts-)' = lam(ts+)' * dg/dx-` **non** collassa piu' in continuita': si otterrebbe `lam_m(ts-) = (1-k)*lam_m(ts+)`, *cioe' il costato di massa salterebbe*. E' proprio il fatto che la massa buttata sia *assoluta* (funzione del solo `ts`) a lasciare passare `lam_m` intatto.

---

## shooting\_inner (righe 96-121)

- Riga 96: 4 incognite, 4 residui, `ts` fisso in `p.ts`. E' la **stessa formulazione** di `shooting_twostage` in `main_task4.m`, riscritta sopra `propagate`.
  - Righe 106-108: guardia di bracket (`tf <= ts || tf > 2 || ts <= 0`) che restituisce `1e6*ones(4,1)`.
  - Righe 110-117: **qui c'e' un difetto di ordinamento**. La guardia su `m_plus > 0` (riga 115) e' eseguita **dopo** `propagate` (riga 111), *cioe' dopo che l'arco 2 e' gia' stato integrato* – potenzialmente con `m <= 0`, che in `ode_burn` finisce nei denominatori `T/m` e `T/m^2`. In `main_task4.m` (`shooting_twostage`, righe 282-284) la stessa guardia e' correttamente **prima** dell'integrazione dell'arco 2. In pratica non morde, perche' il `try/catch` cattura gli errori e i guess restano lontani dal bordo, ma se `ode45` restituisse `NaN` senza sollevare eccezione il `catch` non scatterebbe e il residuo sarebbe `NaN`.
  - Righe 118-120: residui. `H0` calcolato dalla forma algebrica a `t=0` (`vx=vy=0`, `m=1`, `lam_m0=1`), e i tre terminali `y(tf)-yf`, `vx(tf)-1`, `vy(tf)`.
-

## shooting\_corner (righe 123-154)

```
lam_vy_ts = lam_vy0 - lam_y*ts;
lam_v_ts = sqrt(lam_vx0^2 + lam_vy_ts^2);
corner = p.eta*(zf(6) - zs(6)) ...
        - p.c*lam_v_ts*(1/m_plus - 1/m_minus);
res = [zf(2)-p.yf; zf(3)-1; zf(4); H0; corner];
```

- Riga 123: **il cuore del file**: 5 incognite [lam\_vx0; lam\_vy0; lam\_y; tf; ts], 5 residui.
- Riga 133: guardia estesa. Oltre a  $tf > ts > 0$  e  $tf < 2$ , c'è  $ts > 1/(Q*(1+eta)) - 1e-3 \rightarrow reject$ . E' il vincolo fisico  $m_{plus} > 0$ :  $m_{plus} = 1 - (1+eta)*Q*ts$  si annulla a  $ts = 1/(Q*(1+eta)) = 0.4545$ . Lo stesso bound compare in `main_task4.m` alla riga 45.
- Righe 141-142:  $m_{minus} = zs(5)$  (letto dall'integrazione) e  $m_{plus} = m_{minus} - eta*Q*ts$ . Nota:  $m_{minus}$  **potrebbe** essere calcolato in forma chiusa ( $1 - Q*ts$ , perche'  $\dot{m} = -Q$  costante e  $m_0 = 1$ ) – infatti l'header lo fa (riga 9). Leggerlo dall'integrazione e' piu' robusto e serve anche come controllo incrociato implicito: se l'integratore fosse fuori tolleranza,  $zs(5)$  e  $1 - Q*ts$  divergerebbero.

- Righe 150-151: **lam\_v(ts) viene ricostruito analiticamente**, non letto dallo stato. E' obbligatorio: `ode_burn` **non integra**  $lam_vx$  e  $lam_vy$  (li ricava dai parametri), quindi non esistono come componenti di  $z$ . La formula e' la soluzione chiusa delle equazioni di Eulero-Lagrange:

```
lam_vx(t) = lam_vx0          (costante)
lam_vy(t) = lam_vy0 - lam_y*t (rampa lineare)
|lam_v(ts)| = sqrt(lam_vx0^2 + (lam_vy0 - lam_y*ts)^2)
```

- Riga 152: il residuo di corner, nella forma **burnout-referenced**  $eta*(lam_m(tf) - lam_m(ts))$ , con  $zf(6) = lam_m(tf)$  e  $zs(6) = lam_m(ts)$ . Osservare che  $zs$  e' lo stato a  $ts^-$ , ma  $lam_m$  e' continuo, quindi  $lam_m(ts^-) = lam_m(ts^+)$  e la scelta e' irrilevante – cosa che, dovendo scriverlo in un report, va detta esplicitamente.
- Riga 153: l'ordinamento dei residui. I primi 4 sono identici a quelli del BVP interno; il quinto e' la novita'.

**Possibile domanda d'esame** – nel solve a 5 incognite, chi determina cosa? *Risposta*: i 3 residui terminali determinano la traiettoria (i due rapporti dei costati e  $tf$ ), dato  $ts$ ; il residuo di corner determina  $ts$ ; il residuo  $H_0 = 0$  fissa la scala globale dei costati, che i residui terminali non vedono (dipendono solo dalla *direzione* del primer vector). Il sistema e' ben posto proprio perche' i tre gruppi sono indipendenti. Il residuo di corner puo' essere valutato con i costati "sporchi" ( $lam_m0 = 1, H(0) = 0$ ) perche' e' scritto in forma invariante di gauge.

---

## shooting\_corner\_wrong (righe 156-187)

- Riga 156: **copia deliberatamente corrotta** di `shooting_corner`. Unica differenza: la riga 185, dove  $eta*(zf(6) - zs(6))$  diventa  $eta*zs(6)$ , cioe'  $eta*lam_m(ts)$  senza il riferimento a burnout.
  - Onesta' massima: **non e' un "quasi-errore" che qualcuno commetterebbe per distrazione derivando la teoria** – e' un residuo costruito apposta come *esperimento di controllo*. Serve a rispondere alla domanda "e se avessi dimenticato il termine  $lam_m(tf)$ ?", e la risposta e' "converge lo stesso, e ti da' un  $ts$  sbagliato". Il valore didattico e' esattamente questo: **l'errore non si manifesta come un fallimento numerico**, si manifesta come un risultato plausibile ma falso. Senza il confronto con lo sweep non te ne accorgereesti.
  - Il resto della funzione (righe 167-184) e' identico a `shooting_corner`, duplicazione di codice inclusa. Accettabile per uno script di validazione, ma e' copia-incolla.
-

## Cosa questo script valida davvero, e cosa no

**Valida:** - che la corner condition derivata nel report abbia lo **stesso zero** del massimo numerico trovato dallo sweep di `main_task4.m` (con `ts` a  $5e-3$ , cioè entro la risoluzione dello sweep); - che segni, normalizzazioni e bookkeeping delle masse (`m_minus`, `m_plus`, `ms1 = eta*Q*ts`) siano coerenti fra derivazione analitica e codice; - che la forma **burnout-referenced** sia necessaria (righe 62-72).

**Non valida:** - che l'ottimo sia un **massimo** e non un minimo o un flesso – la corner condition è una condizione **necessaria del primo ordine**, non sufficiente. È lo sweep di `main_task4.m` a mostrare che la curva payload-vs-`ts` ha un massimo lì. (I due file quindi si sostengono a vicenda: lo sweep dà l'ordine zero e la natura del punto, il corner solve la precisione. Nessuno dei due basta da solo.) - che la soluzione sia **globale**: `fsolve` è un metodo locale, e il warm start è a `ts0 = 0.33`. - niente **al di fuori del singolo set di parametri** `c=0.6`, `eta=0.1`, `yf=0.04`, `Q=2`. Non c'è nessuno sweep su `Q` o `yf` che verifichi la robustezza della condizione. - il **residuo finale**, che viene stampato ma non incluso nel criterio di PASS.

---

## Possibili domande d'esame

**D: A cosa serve `validate_staging_corner.m`? Non è ridondante rispetto a `main_task4.m`? R:** No, è proprio il contrario: è l'unico controllo indipendente che ho. `main_task4.m` trova `ts*` con una ricerca esaustiva su griglia – un metodo di ordine zero che non usa mai la teoria di Pontryagin sulla giunzione. Questo script la usa e basta: promuove `ts` a quinta incognita e impone la corner condition come quinto residuo. Il fatto che le due strade cadano sullo stesso `ts` (0.336 dal corner solve contro 0.336-0.337 dello sweep, cioè entro la risoluzione della griglia) è l'invariante che verifico. Se avessi sbagliato un segno o una normalizzazione nella derivazione, i due numeri divergerebbero.

**D: Perché la tolleranza del PASS è  $5e-3$  e non  $1e-8$ ? R:** Perché il termine di paragone non è accurato. Lo sweep di `main_task4.m` usa 50 punti su circa  $[0.01, 0.42]$ , quindi un passo di  $\sim 0.008$ : il suo `ts_opt` è un nodo della griglia e non può essere più preciso di mezzo passo.  $5e-3$  è proprio quell'ordine di grandezza. Chiedere  $1e-8$  significherebbe pretendere dallo sweep una precisione che per costruzione non ha. Nota il rovescio: è il corner solve ad essere il numero *buono*; lo sweep è il numero *approssimato*.

**D: I costati sono continui allo staging? E l'Hamiltoniana? R:** I costati sono **tutti continui**, perché la massa jettisonata `ms1 = eta*Q*ts` dipende solo dal tempo di giunzione e non dallo stato: la Jacobiana della mappa di salto rispetto a `x(ts-)` è l'identità e la condizione `lam(ts-)' = lam(ts+)' * dg/dx-` diventa `lam(ts-) = lam(ts+)`. Nel codice si vede alla riga 91: si tocca **solo** `z_plus(5)`, la massa; `z_plus(6) = lam_m` passa intatto. L'Hamiltoniana invece **salta**, di `T*|lam_v(ts)|*(1/m_plus - 1/m_minus) > 0`, perché la massa a denominatore in `T*|lam_v|/m` scende di colpo. Ed è proprio l'entità di quel salto che la corner condition prescrive.

**D: Perché `lam_v(ts)` è ricalcolato a mano (righe 150-151) invece di leggerlo dallo stato integrato? R:** Perché non è nello stato. `ode_burn` integra `[x; y; vx; vy; m; lam_m]`: gli altri costati hanno **soluzione in forma chiusa** (`lam_vx = lam_vx0` costante, `lam_vy = lam_vy0 - lam_y*t` rampa lineare, `lam_x = 0`, `lam_y` costante) e non vengono propagati numericamente. `lam_m` è l'unico che richiede un'integrazione, perché la sua ODE `lam_m_dot = (T/m^2)*|lam_v|` dipende dalla massa. Quindi `|lam_v(ts)|` si valuta direttamente dalla formula, il che è anche più accurato che leggerla da un integratore.

**D: Cosa dimostra il “cautionary check” delle righe 62-72? R:** Che la corner condition deve essere scritta in forma **invariante di gauge**. Il mio BVP integra `lam_m` da `lam_m0 = 1`, che è una convenzione arbitraria: siccome `lam_m_dot` non dipende da `lam_m`, cambiare la costante d'integrazione trasla rigidamente tutta la curva. Se scrivo la condizione con `lam_m(ts)` da solo, il suo zero dipende da quella convenzione ed è privo di significato; scrivendola con la **differenza** `lam_m(tf) - lam_m(ts)` l'offset si cancella. Il check lo mostra numericamente: la versione senza riferimento converge senza errori, ma su un `ts` spurio ( $\sim 0.225$  secondo l'appendice del report) invece di 0.336. È un errore che non fa rumore, ed è per questo che vale la pena averlo messo nero su bianco.

**D: Il solve a 5 incognite garantisce che quello sia un massimo del payload? R:** No. La corner condition è una condizione **necessaria del primo ordine** (`dtf/dts = 0`): identifica un punto stazionario, non ne certifica la natura. È lo sweep di `main_task4.m` a mostrare che la curva payload-vs-`ts` ha lì un massimo interno e non

un minimo. I due file sono complementari: lo sweep da' la *forma* della curva, il corner solve la *posizione precisa* dell'estremo.

# HM2\_\_powered\_\_descent/ode\_\_descent.m

## Ruolo del file nel progetto

Questo file contiene la **dinamica continua** del problema di powered descent di HM2, scritta in forma adimensionale. E' il "cuore fisico" di tutta l'homework: ogni trascrizione diretta (trapezoidale in Task 1, ZOH+RK4, LTV+SCvx in Task 2) alla fine chiama questa funzione per sapere come si muove il razzo. Il modello e' un **punto materiale 2D su Terra piatta, senza aerodinamica**: niente drag, niente portanza, gravita' costante. La massa e' uno stato, non un parametro, perche' il razzo brucia propellente e la sua accelerazione dipende da  $m(t)$ .

La parametrizzazione del controllo e' il **vettore di spinta  $T = [T_x; T_y]$** : le componenti di  $T$  sono le variabili di decisione dell'NLP, e sull'intervallo ZOH e'  $T$  (non l'accelerazione) a essere tenuto costante. Questo e' il punto che distingue questo file dal gemello `ode_descent_uacc.m`, che invece tiene costante l'accelerazione  $u = T/m$ .

Chi lo chiama: - `main_task1.m` riga 314, attraverso il wrapper `dyn_rhs`, per costruire i difetti trapezoidali; - `main_task2.m` riga 773 (`trap_nonlcon`, difetti trapezoidali del baseline); - `main_task2.m` riga 295 (`ltv_aug_rhs`), dove serve a costruire l'offset affine  $c_{off} = f - A*x_{ref} - B*u_{ref}$  della linearizzazione di Appendice A; - `main_task2.m` riga 852 (`fwd_integrate`), come RHS del replay `ode45` che misura la fedelta' della trascrizione; - `rk4_zoh.m` righe 17-20, come RHS dei quattro stadi RK4; - i test `tests/odeDescentTest.m`, `tests/rk4ZohTest.m`, `tests/descentDynamicsPerformanceTest.m`.

Il file e' lungo 16 righe di cui 2 di codice vero: la densita' sta tutta nella derivazione delle due righe.

---

## ode\_descent (righe 1-16)

```
function dx = ode_descent(x, u, Vc)
...
Tmag = sqrt(u(1)^2 + u(2)^2);
dx = [ x(3); x(4); u(1)/x(5); u(2)/x(5) - 1; -Vc * Tmag ];
```

- **Riga 1:** firma. Tre soli argomenti: lo stato  $x$  (5x1), il controllo  $u$  (2x1) e lo scalare  $Vc$ . Notare l'**assenza dell'argomento tempo  $t$** : la dinamica e' *autonoma*. Non lo e' per caso: con il controllo tenuto costante sull'intervallo (ZOH) il tempo non compare mai esplicitamente nel RHS, e questo e' esattamente cio' che permette a `rk4_zoh.m` di passare lo stesso  $u$  a tutti e quattro gli stadi senza interpolare nulla. Dove serve un RHS con il tempo (il replay `ode45` in `fwd_integrate`) e' il chiamante a costruire il wrapper `@(tt, x) ode_descent(x, u_fcn(tt), d.Vc)`.
- **Righe 2-11:** la docstring. Dichiaro il layout dello stato  $x = [x; y; vx; vy; m]$ , il controllo  $u = [T_x; T_y]$ , e la convenzione chiave: "**Gravity = -1**". Adimensionalizzato, il modulo della gravita' vale esattamente 1. Righe 10-11 dichiarano esplicitamente la scelta di design di non mettere il blocco `arguments`.
- **Riga 13:**  $Tmag = \sqrt{u(1)^2 + u(2)^2}$  – il modulo del vettore spinta,  $\|T\|$ . Scritto a mano invece che con `norm(u)` per velocita' (siamo in un hot loop; `norm` fa dispatch e controlli). E' l'unica non-linearita' *nel controllo* dell'intero RHS, ed e' anche il punto dolente numerico: `sqrt` di una somma di quadrati e' **non differenziabile in  $T = 0$** , cioe' esattamente dove si trova l'arco di coast della soluzione bang-off-bang.

Il README di HM2 attribuisce a questo kink lo stallo del criterio di ottimalità di `fmincon` a  $1e-3..1e-4$  sui run di sensitività. Da notare che **qui la norma non è regolarizzata**: è la funzione `jacobians` in `main_task2.m` (riga 266) ad aggiungere  $+1e-6$  sotto radice per tenere finita  $df/du$  in  $T = 0$ . Le due funzioni quindi non sono perfettamente coerenti fra loro – il RHS è il vero modello, lo Jacobiano è una versione ammorbidita.

- **Riga 14**: le cinque righe della dinamica, in ordine.

## Derivazione della dinamica

Il modello dimensionale (punto materiale, Terra piatta, no drag) è:

```
x_dot = vx
y_dot = vy
vx_dot = Tx/m
vy_dot = Ty/m - g
m_dot = -||T|| / c          con c = Isp * g0   (velocità efficace di scarico)
```

La quinta riga è la legge di consumo del razzo: la spinta è  $||T|| = m\_dot\_propellente * c$ , quindi il flusso di massa vale  $||T||/c$ . Nel codice non compaiono né `Isp` né `g0`: sono già collasati dentro `Vc`. In `main_task2.m` riga 31 si ha `data.c = data.Isp * data.g0` e riga 226 `dnd.Vc = ref.V / d.c`.

**Adimensionalizzazione** (funzione `nondim`, `main_task2.m` righe 213-227):

```
L_ref = y0 = 3000 m
g_ref = g = 9.81 m/s^2
t_ref = sqrt(L_ref/g)      -> 17.49 s
V_ref = sqrt(g*L_ref)      -> 171.55 m/s
m_ref = m0 = 2000 kg
T_ref = m_ref*g = 19620 N
```

Sostituendo  $x = L\_ref x'$ ,  $t = t\_ref t'$ ,  $v = V\_ref v'$ ,  $m = m\_ref m'$ ,  $T = T\_ref T'$  e usando le identità  $t\_ref V\_ref = L\_ref$  e  $t\_ref/V\_ref = 1/g$ :

- posizione:  $dx'/dt' = (t\_ref V\_ref/L\_ref) vx' = \mathbf{v}x'$  (coefficiente 1);
- velocità:  $dvx'/dt' = (t\_ref/V\_ref)(T\_ref/m\_ref) Tx'/m' = (1/g)(g) Tx'/m' = \mathbf{T}x'/m'$ ;
- gravità:  $(t\_ref/V\_ref)*g = 1$ , da cui il **-1** costante nella quarta riga;
- massa:  $dm'/dt' = -(t\_ref T\_ref)/(m\_ref c) * ||T'|| = -(t\_ref g/c)||T'|| = -(V\_ref/c)*||T'|| = -\mathbf{V}c * ||\mathbf{T}'||$ .

Quindi  $Vc = V\_ref/c$  (il “numero di Tsiolkovsky”) è l’**unico** parametro fisico rimasto nel RHS: tutta la fisica del propulsore è compressa in uno scalare. Con i dati di HM2 vale  $Vc = 171.55/2206.5 = 0.0777$  (il valore usato nei test come costante). Il messaggio implicito è potente: due razzi con `Isp` e scale diverse ma stesso `Vc` hanno **la stessa traiettoria ottima adimensionale**.

## Cosa NON c’è nella dinamica

- **Nessun drag**. Il RHS non ha termini in  $\rho$ ,  $S$ ,  $Cd$ , né  $V^2$ . Il modello è valido solo perché HM2 lo dichiara esplicitamente (2D point-mass, no aerodynamics). In una discesa vera da 3 km il drag non è trascurabile; è un’ipotesi dell’assegnazione, non una verità fisica.
- **Gravità costante**, non inverso del quadrato: su 3 km di quota la variazione di  $g$  è  $\sim 1e-3$  relativo, quindi l’approssimazione è innocua.
- **Nessun vincolo di puntamento della spinta** e nessuna dinamica di assetto:  $T$  può ruotare istantaneamente. La direzione della spinta è un controllo libero.
- **Nessuna guardia su  $m > 0$** . Se un’iterata di `fmincon` porta  $x(5)$  a zero o sotto, le righe 3 e 4 esplodono o cambiano segno silenziosamente. La protezione è delegata al chiamante: `box_bounds` in `main_task2.m` riga 691 impone `lb = 1e-3` sulla massa.

**Possibile domanda d’esame** – La dinamica è affine nel controllo? *Risposta*: No, non del tutto. Le righe di posizione non dipendono da  $u$ ; le righe di velocità sono *lineari* in  $u$  a stato fissato ( $u1/m$ ,  $u2/m$ ), ma il coefficiente  $1/m$  dipende dallo stato, quindi il prodotto è **bilineare** in  $(u, 1/m)$  e non

convesso. La riga della massa contiene  $\|u\|$ , che e' convessa ma non affine. Percio' il vincolo di difetto  $x_{k+1} = F(x_k, u_k)$  resta un'uguaglianza non convessa. E' proprio questo che costringe le varianti (b) e (c) a linearizzare e iterare (SCvx), e che la variante (d) rimuove con il cambio di variabili  $z = \ln m$ ,  $u = T/m$ .

## Perche' due RHS diversi (ode\_descent vs ode\_descent\_uacc)

Il confronto vale la pena farlo qui perche' e' la domanda piu' probabile all'orale. Le due funzioni descrivono la stessa fisica, ma differiscono per *quale grandezza viene tenuta costante sull'intervallo ZOH*:

|                           | ode_descent                                            | ode_descent_uacc                       |
|---------------------------|--------------------------------------------------------|----------------------------------------|
| Controllo tenuto costante | spinta $T = [T_x; T_y]$                                | accelerazione $u = T/m$                |
| $vx\_dot$                 | $T_x/m$ (varia: $m$ cala)                              | $u_x$ (costante)                       |
| $m\_dot$                  | $-V_c \cdot \text{norm}(T)$ (costante sull'intervallo) | $-V_c m \text{norm}(u)$                |
| $d(\ln m)/dt$             | $-V_c \cdot \text{norm}(T)/m$ (varia)                  | $-V_c \cdot \text{norm}(u)$ (costante) |

Con  $T$  tenuta costante, l'**accelerazione cresce** durante l'intervallo perche' la massa cala:  $T/m(t)$  e' crescente. Con  $u$  tenuta costante, e' la **spinta a crescere/calare** seguendo la massa,  $T(t) = m(t) \cdot u$ . Sono due ZOH diversi, quindi due plant discreti diversi: non e' lecito confrontare i due set di controlli nodo per nodo pretendendo che descrivano la stessa traiettoria. Infatti `main_task2.m` usa due funzioni di replay distinte (`fwd_integrate` riga 852 con `ode_descent`, `fwd_integrate_uacc` riga 1298 con `ode_descent_uacc`) proprio per non barare nella validazione.

Osservazione (mia, non nel codice): con  $u$  costante la traiettoria dell'intervallo ha **soluzione analitica banale** (doppio integratore + log-massa lineare), mentre con  $T$  costante la velocita' segue una legge logaritmica di tipo Tsiolkovsky,  $vx(t) = vx_k - (T_x/(V_c \|T\|)) \ln(m(t)/m_k)$ , che degenera in 0/0 quando  $\|T\| \rightarrow 0$  (arco di coast). Questo e' un buon motivo pratico per integrare `ode_descent` con RK4 invece che con la sua forma chiusa: RK4 e' uniformemente valido anche a spinta nulla.

## Nessun blocco arguments: e' voluto

Righe 10-11 lo dichiarano: *"No arguments validation by design: hot-loop RHS inside ode45/fmincon; validate at the call site."*

Il conto: nella variante (a) `zoh_nonlcon` chiama `rk4_zoh` per ciascuno dei  $N-1 = 49$  intervalli, ognuno con `n_sub = 2` sotto-passi da 4 stadi  $\rightarrow 49 \cdot 2 = 98$  valutazioni di `ode_descent` per **una sola** valutazione del vincolo. Con `fmincon` in modalita' SQP e gradienti alle differenze finite su  $7N = 350$  variabili, ogni iterazione maggiore ricostruisce lo Jacobiano dei vincoli con  $\sim 351$  valutazioni di `nonlcon`, cioe' circa **1.4e5 chiamate a ode\_descent per iterazione**. Un blocco `arguments` (che fa validazione di classe e dimensione a ogni chiamata) costerebbe piu' del RHS stesso – il benchmark `descentDynamicsPerformanceTest` (riga 39) stima una singola valutazione intorno ai **60 ns**, tanto bassa da dover essere misurata a batch di 1000.

La validazione vive invece nelle funzioni di frontiera chiamate una volta per run: `nondim`, `dim_sol`, `compute_ltv_zoh`, `fwd_integrate`, `lti_zoh`. E' la convenzione dichiarata anche in `CLAUDE.md` ("Hot-loop functions carry no arguments validation by design").

**Possibile domanda d'esame** – Se il RHS non valida gli input, come si evita di passargli uno stato con la massa negativa? *Risposta*: Non lo si evita nel RHS, lo si evita a monte. I bound di scatola dell'NLP (`box_bounds`, `main_task2.m` riga 691) impongono  $m$  in  $[1e-3, m_0]$  su ogni nodo, quindi `fmincon` non valuta mai il vincolo fuori da quella scatola (con l'algoritmo SQP i bound sono



rispettati a ogni iterata). Il rischio residuo e' negli *stadi interni* di RK4, dove uno stato intermedio potrebbe in teoria uscire dai bound: e' un'ipotesi implicita che il codice non verifica.

---

## Possibili domande d'esame

**D: Perche' la gravita' compare come un “-1” secco e non come 9.81?** R: Perche' il problema e' adimensionalizzato scegliendo  $a_{\text{ref}} = g$ . Con le scale  $L_{\text{ref}} = y_0$ ,  $t_{\text{ref}} = \sqrt{L_{\text{ref}}/g}$ ,  $V_{\text{ref}} = \sqrt{gL_{\text{ref}}}$  si ottiene  $(t_{\text{ref}}/V_{\text{ref}})g = 1$ , quindi il termine gravitazionale nella quarta riga vale esattamente -1. Il vantaggio e' che tutti gli stati sono  $O(1)$ : posizioni e velocita' iniziali diventano (0.33, 1) e (1.75, -1.17), la massa iniziale 1. Con uno scaling cosi' l'NLP e' molto meglio condizionato e le tolleranze di `fmincon` (1e-5 su ottimalita', 1e-6 sui vincoli) hanno lo stesso significato su tutte le componenti.

**D: Cosa rappresenta fisicamente  $V_c$  e perche' e' l'unico parametro rimasto?** R:  $V_c = V_{\text{ref}}/c = \sqrt{gL_{\text{ref}}}/(Ispg_0)$ . E' il rapporto fra la velocita' caratteristica del problema e la velocita' efficace di scarico del motore – un “numero di Tsiolkovsky”. Regola quanto costa in massa una data manovra: dalla riga 5 non-dimensionale, la massa consumata su un intervallo e'  $V_c$  volte l'impulso adimensionale. Con i dati di HM2,  $V_c = 0.0777$ . Se  $V_c \rightarrow 0$  (motore infinitamente efficiente) il razzo non consuma e il problema di minimo consumo degenera.

**D: Perche'  $m_{\text{dot}}$  dipende solo dal modulo della spinta e non dalla sua direzione?** R: Perche' la portata di propellente e' fissata dal regime del motore, e la spinta prodotta e'  $\|T\| = m_{\text{dot}} \text{prop} * c$  indipendentemente da dove la si punta. Ruotare il vettore di spinta (nel modello, gimbal ideale istantaneo) non costa massa. Il test `testMassFlowDependsOnlyOnThrustMagnitude` (`odeDescentTest.m` riga 39) verifica proprio questa invarianza confrontando  $T = [1;0]$  e  $T = [0;-1]$ .

**D: Il modulo  $\|T\|$  e' non differenziabile in  $T = 0$ . Che problema crea e come lo si aggira?** R: La soluzione ottima e' bang-off-bang (max-coast-max): l'arco di coast si piazza *esattamente* sul kink  $T = 0$ . `fmincon` con gradienti alle differenze finite vede una derivata mal definita li' e il criterio di ottimalita' di prim'ordine si blocca a 1e-3..1e-4 invece di scendere a 1e-6 (documentato nel README di HM2). Il codice mitiga solo lo Jacobiano (`jacobians`, riga 266, regolarizza con  $\sqrt{T_x^2 + T_y^2 + 1e-6}$ ), non il RHS. Le cure vere sono: (i) il rilassamento lossless con slack  $\sigma \geq \|T\|$ , che toglie la norma dalla dinamica e la trasforma in un vincolo conico (la strada della variante GFOLD); (ii) gradienti analitici con norma smussata; (iii) continuation.

**D: In `ltv_aug_rhs` (riga 295-296) `ode_descent` e' usata per calcolare  $c_{\text{off}} = f - A*x_{\text{ref}} - B*u_{\text{ref}}$ . Perche' serve questo termine?** R: Perche' la linearizzazione di una dinamica non lineare attorno a una traiettoria di riferimento non e' lineare ma **affine**:  $f(x,u) \sim f(x_r, u_r) + A(x-x_r) + B(u-u_r) = Ax + Bu + [f(x_r, u_r) - Ax_r - Bu_r]$ . Il termine in parentesi e' l'offset  $c_{\text{off}}$ , che non e' zero perche' il riferimento non e' un punto di equilibrio (il razzo sta accelerando). Se lo si dimenticasse, il sistema linearizzato “perderebbe” gravita' e la parte di dinamica non catturata dai Jacobiani, e `SCvx` non convergerebbe alla soluzione non lineare.

**D: Questa dinamica e' convessa? Se no, cosa la rende non convessa?** R: No. Due sorgenti di non convessita': (i) il termine  $u/m$ , bilineare fra controllo e reciproco della massa (il vincolo di difetto e' un'uguaglianza non lineare, quindi non convessa a prescindere dal segno); (ii) il vincolo di **lower bound** sulla spinta  $\|T\| \geq T_{\text{min}}$ , che in HM2 e' inattivo perche'  $T_{\text{min}} = 0$  (`main_task2.m` riga 32) ma che in un caso realistico e' un vincolo di tipo “esterno a una palla”, palesemente non convesso. La riga della massa  $-V_c*\|T\|$  e' invece convessa in  $u$ , ma comparando dentro un'uguaglianza non aiuta. Il cambio di variabili di Acikmese-Blackmore ( $z = \ln m$ ,  $u = T/m$ ,  $\sigma \geq \|u\|$ ) elimina la prima e convessifica la seconda.

# HM2\_\_powered\_\_descent/ode\_\_descent\_\_uacc.m

## Ruolo del file nel progetto

Questo file e' la stessa fisica di `ode_descent.m` riscritta con un'altra parametrizzazione del controllo: invece della spinta  $T = [T_x; T_y]$ , il controllo e' l'**accelerazione comandata**  $u = T/m$ . Non e' un dettaglio cosmetico: e' il passo che apre la strada alla convessificazione. Con  $u$  come controllo, le righe di velocita' perdono il fattore  $1/m$  (che era la sorgente della bilinearita' fra controllo e stato) e diventano un doppio integratore puro; e la riga della massa diventa **omogenea in  $m$** , il che significa che nella variabile  $z = \ln m$  diventa lineare.

E' il RHS "nativo" della **variante (d) di Task 2 – GFOLD log-mass** (Acikmese & Blackmore). La trascrizione GFOLD non lavora in coordinate  $(m, T)$  ma in coordinate  $(z = \ln m, u = T/m, \sigma \geq \|u\|)$ , dove la dinamica e' **esattamente LTI** e viene discretizzata con un unico esponenziale di matrice in `lti_zoh.m`. Questo file esiste per due ragioni precise:

1. **Definire il plant fisico corrispondente al ZOH sull'accelerazione.** La convenzione ZOH di GFOLD tiene costante  $u$ , non  $T$ : sull'intervallo la spinta fisica  $T(t) = m(t) * u$  fluttua seguendo la massa che cala. Se si replayasse una soluzione GFOLD con `ode_descent` (T-hold) si otterrebbe una traiettoria diversa e la validazione sarebbe scorretta.
2. **Fare da ground-truth non lineare** per la trascrizione LTI. Il replay `fwd_integrate_uacc` (`main_task2.m` righe 1285-1303) integra questo RHS con `ode45` a tolleranze strette *nelle coordinate fisiche originali*  $[x; y; vx; vy; m]$ , e confronta il risultato con la predizione lineare. Il confronto avviene su due metriche distinte, e vale la pena tenerle separate: `node_err` (`main_task2.m` righe 859-868) misura **solo posizione e velocita'** ( $X(:,1:4)$  – la docstring dice esplicitamente "*mass excluded*"), ed e' li' che il README riporta  $7.3e-12$  nondim, cioe' il pavimento dell'integratore; il canale di massa e' invece controllato dal drift di `m_f` calcolato da `land` (righe 142-144, che legge `X(end,5)`). La distinzione non e' pedanteria: le righe di posizione e velocita' non contengono  $\sigma$ , quindi uno slack non aderente ( $\sigma > \|u\|$ ) sarebbe **invisibile** all'errore di nodo e comparirebbe solo nel drift di massa.

Chi lo chiama: `main_task2.m` riga 1298 (dentro `fwd_integrate_uacc`, usato sia per la validazione finale sia – riga 1251 – dentro il **ratio test** della SCvx GFOLD, dove  $J_{act}$  e' misurato sul modello non lineare), `proto_gfold_logmass.m` riga 249, e i test `tests/gfoldLogMassTest.m` (righe 20-52 e 92).

---

## ode\_descent\_uacc (righe 1-21)

```
function dx = ode_descent_uacc(x, uacc, Vc)
...
umag = sqrt(uacc(1)^2 + uacc(2)^2);
dx = [ x(3); x(4); uacc(1); uacc(2) - 1; -Vc * x(5) * umag ];
```

- **Riga 1:** firma identica a `ode_descent` per forma (stato, controllo,  $V_c$ ) ma con `uacc` al posto di  $u$ , a rimarcare che il secondo argomento e' un'**accelerazione**, non una forza. Lo stato resta  $x = [x; y; vx; vy; m]$  – **in massa, non in log-massa**. Questa e' una scelta di design importante: il RHS non lavora in  $z$ . Il motivo e' che il replay deve produrre una traiettoria confrontabile con quelle delle altre tre varianti:

land (main\_task2.m righe 142-144) legge  $X(\text{end}, 5)$  come **massa fisica** per il drift di  $m_f$ , e così fanno il campo sol.m e i plot; node\_err (righe 859-868) si limita invece alle colonne 1:4 (posizione e velocità).

- **Righe 2-16:** la docstring, che è insolitamente ricca e contiene già la derivazione. Riga 11-13: *“With  $u$  held constant the thrust  $T = m(t)u$  floats with the (depleting) mass, so  $v_{x\_dot} = u_x$  and  $v_{y\_dot} = u_y - 1$  are exact, while the mass row reads  $m\_dot = -Vc//T// = -Vc//u//$ ”. Riga 13-14 mettono nero su bianco il contrasto con ode\_descent.m. Riga 16 dichiara l’assenza del blocco arguments (hot loop dentro ode45).*
- **Riga 18:**  $umag = \sqrt{uacc(1)^2 + uacc(2)^2}$  – il modulo dell’**accelerazione** comandata, non della spinta. Adimensionalmente, poiché  $T_{ref} = m_{ref}g$  e  $a_{ref} = g$ , la relazione  $u = T/m$  in variabili adimensionali resta letteralmente  $u' = T'/m'$  senza fattori di conversione: è uno dei regali della scelta  $T_{ref} = m_{ref}g_{ref}$ .
- **Riga 19:** le cinque righe della dinamica.

## Derivazione riga per riga

Partiamo dalla dinamica dimensionale (identica a quella di ode\_descent):

$$v_{x\_dot} = T_x/m, \quad v_{y\_dot} = T_y/m - g, \quad m\_dot = -||T||/c$$

e sostituiamo la definizione del nuovo controllo,  $T = m*u$ :

- $v_{x\_dot} = u_x, v_{y\_dot} = u_y - 1$  (adimensionale). La massa **scompare completamente** dalle righe di velocità: erano bilineari in  $(T, 1/m)$ , ora sono lineari in  $u$  e indipendenti dallo stato. Questo è il guadagno strutturale principale. Il test testAccelerationIsDirect (gfoldLogMassTest.m riga 29) verifica proprio questo: con la stessa uacc e masse 0.9 e 0.3, dx(3:4) è identico.
- **$m\_dot = -Vc//T// = -Vc||mu// = -Vc*||u||$  (perché  $m > 0$ , quindi il modulo si porta fuori). È l’unica riga che ancora accoppia stato e controllo, ed è bilineare\*\*** – apparentemente siamo tornati da capo. Il colpo di scena è che il termine è *omogeneo di grado 1 in m*, quindi:

$$d(\ln m)/dt = m\_dot / m = -Vc * ||u||$$

cioè **con  $z = \ln m$  la riga della massa non dipende più dallo stato affatto** ed è funzione del solo controllo. La non linearità residua è la norma  $||u||$ , che si elimina introducendo lo slack  $\sigma \geq ||u||$  (vincolo di **cono di secondo ordine**, convesso) e scrivendo  $z\_dot = -Vc*\sigma$ , ora **lineare**. Questo è l’intero trucco di Acikmese-Blackmore: lo si vede istanziato in lti\_zoh.m riga 24, dove  $B(5,3) = -Vc$  e il coefficiente di  $\sigma$  nella riga di  $z$ .

Il test testMassFlowScalesWithMass (riga 39) verifica l’omogeneità: con  $||u|| = 1$ ,  $m\_dot$  vale  $-Vc1.0$  a massa 1.0 e  $-Vc0.5$  a massa 0.5, cioè  $d(\ln m)/dt$  è la stessa.

- **Terzo/quarto stato:** la gravità resta un -1 costante, come in ode\_descent, e finisce nel vettore di offset  $c = [0;0;0;-1;0]$  della forma LTI (lti\_zoh.m riga 25). Non è un controllo: è un termine affine noto, ed è per questo che la discretizzazione ZOH ha bisogno del terzo blocco cbar oltre ad Abar e Bbar.

## Il punto sottile: $\sigma$ è davvero uguale a $||u||$ ?

La dinamica LTI usa  $\sigma$ , il RHS non lineare usa  $||u||$ . **Coincidono solo se il vincolo conico è attivo all’ottimo.** Se il solver restituisse  $\sigma > ||u||$ , la predizione LTI consumerebbe **più** massa di quanta ne consumi il razzo vero, e il replay con ode\_descent\_uacc darebbe una  $m$  finale più alta della predizione.

Perché allora è attivo? Perché il costo è  $-XI(5,N)$  (main\_task2.m riga 1183), cioè si **massimizza  $z_N$** , e integrando  $z\_dot = -Vc\sigma$  si ha  $z_N = z_0 - Vcsum(\sigma\_kdt)$ . *Massimizzare  $z_N$  equivale a minimizzare la somma degli  $\sigma$ , quindi l’ottimizzatore li spinge sul loro lower bound, che è esattamente  $||u_k||$ . Il cono si chiude **da solo**, non serve imporlo. È l’argomento di lossless convexification\*, ed è esattamente ciò che il test testMassRowConsistency (gfoldLogMassTest.m riga 85) verifica numericamente: integrando questo RHS con ode45 e ponendo  $\sigma = ||u||$ , la variazione di  $\ln(m)$  non lineare e la predizione LTI coincidono a  $1e-9$ .*

**Possibile domanda d’esame** – Se la dinamica in coordinate  $(z, u, \sigma)$  è esattamente LTI, perché la variante (d) usa comunque un ciclo SCvx e non un singolo SOCP? *Risposta:* Perché

e' **la dinamica** a essere esatta, non tutti i vincoli. Il bound superiore di spinta  $\|T\| \leq T_{\max}$  diventa, in coordinate GFOLD,  $\sigma \leq T_{\max} \exp(-z)$ : *un vincolo non convesso\** (esponenziale come maggiorante di una variabile). Il codice lo linearizza attorno a  $z_{\text{ref}}$  con lo sviluppo di Taylor  $\sigma \leq T_{\max} \exp(-z_{\text{ref}}) * (1 - (z - z_{\text{ref}}))$  (`main_task2.m` righe 1158-1159) e itera nella SCvx per far convergere  $z_{\text{ref}}$ . Il vero single-shot SOCP (lossless convexification completa, con la maggiorazione quadratica del lower bound e il bound superiore trattato una volta sola su un profilo di massa a priori) e' ancora aperto nel backlog come ticket T006.

---

## Perche' esistono entrambi i RHS

La domanda "perche' non un solo file?" ha una risposta precisa: **descrivono due plant discreti diversi**, non due implementazioni della stessa cosa. Il ZOH e' parte della definizione del problema discretizzato, non un dettaglio di integrazione. Su un intervallo  $[t_k, t_{k+1}]$ :

- con `ode_descent` (T costante) l'accelerazione  $a(t) = T/m(t)$  **cresce** perche' m cala. La massa e' esattamente affine in t ( $m_{\text{dot}} = -V_c \|T\|$  e' costante).
- con `ode_descent_uacc` (u costante) l'accelerazione e' piatta e la **spinta fisica cala**,  $T(t) = m(t)u$ . *Il log-massa e' esattamente affine in t* ( $z_{\text{dot}} = -V_c \|u\|$  e' costante), non la massa.

Sono due modelli di attuatore diversi. Non c'e' un "giusto" in assoluto: il computer di bordo puo' benissimo comandare una spinta costante (T-hold) o un'accelerazione costante compensando la massa (u-hold). Cio' che conta e' che la trascrizione e la validazione usino **lo stesso ZOH** – ed e' esattamente per questo che il codice tiene due replay separati (`fwd_integrate` con `ode_descent` per le varianti a/b/c, `fwd_integrate_uacc` con questo file per la variante d, scelta esplicitamente alle righe 124-127 di `main_task2.m`).

Osservazione onesta: nel **limite  $dt \rightarrow 0$**  le due convenzioni convergono alla stessa traiettoria continua, quindi la differenza e' un effetto di discretizzazione, non di fisica. Con  $N = 50$  e  $dt_{\text{nd}} \sim 0.044$  la differenza fra i due plant e' piccola ma non nulla, ed e' sufficiente a rovinare una validazione fatta con il RHS sbagliato.

---

## Nessun blocco arguments: e' voluto

Riga 16: *"No arguments validation by design: hot-loop RHS inside ode45."* Qui il carico e' meno estremo che per `ode_descent` (non c'e' `fmincon` con differenze finite sopra), ma questo RHS viene comunque chiamato dentro `ode45` **a ogni iterazione SCvx**, su  $N-1 = 49$  intervalli, per calcolare  $J_{\text{act}}$  nel ratio test (riga 1251). Con  $\text{RelTol } 1e-10$  /  $\text{AbsTol } 1e-12$  `ode45` fa decine di passi per intervallo, ciascuno con 6 valutazioni del RHS. La validazione sta nel chiamante: `fwd_integrate_uacc` **ha** il suo blocco `arguments` (righe 1289-1292).

---

## Possibili domande d'esame

**D: Qual e' esattamente la differenza fra `ode_descent` e `ode_descent_uacc`?** R: La parametrizzazione del controllo. Il primo prende come controllo la **spinta**  $T = [T_x; T_y]$  (forza, adimensionalizzata con  $T_{\text{ref}} = m_0 g$ ) e produce  $v_{x_{\text{dot}}} = T_x/m$ ,  $m_{\text{dot}} = -V_c \|T\|$ . Il secondo prende come controllo l'**accelerazione comandata**  $u = T/m$  e produce  $v_{x_{\text{dot}}} = u_x$  (nessun  $1/m$ ),  $m_{\text{dot}} = -V_c m \|u\|$ . Nel ZOH questo si traduce in due plant discreti diversi: nel primo caso e' la spinta a essere piecewise-constant e l'accelerazione a crescere mentre il razzo si alleggerisce; nel secondo e' l'accelerazione a essere piatta e la spinta a calare con la massa.

**D: E' la versione "uacc" quella che rende la dinamica affine nel controllo?** R: Quasi, ma va detto con precisione. Le righe di velocita' diventano **esattamente affini** in u ( $v_{x_{\text{dot}}} = u_x$ ,  $v_{y_{\text{dot}}} = u_y - 1$ : e' un doppio integratore con offset costante), e questo elimina la bilinearita'  $T/m$ . Ma la riga della massa, **scritta cosi' come e' nel file** (in variabile m), e' ancora  $-V_c m \|u\|$ : bilineare in m e non lineare in u. La convessificazione richiede due passi ulteriori, che *non* stanno in questo file ma in `l1i_zoh.m` e nel SOCP: (i)  $z = \ln m$ , che rende la riga

indipendente dallo stato:  $\dot{z} = -V_c/||u||$ ; (ii) lo slack  $\sigma \geq ||u||$ , che sostituisce la norma con una variabile lineare:  $\dot{z} = -V_c\sigma$ , e sposta la non linearità in un vincolo conico convesso. Quindi: `ode_descent_uacc` è il ponte verso la forma convessa, ma di per sé è ancora un modello non lineare – ed è voluto, perché è il ground truth con cui si verifica la trascrizione lineare.

**D: Da dove viene esattamente  $\dot{m} = -V_c m ||u||$ ?** R: Dal fatto che il consumo dipende dalla **spinta fisica**, non dall'accelerazione. La legge del razzo è  $\dot{m} = -||T||/c$ . Sostituendo la definizione del controllo,  $T = mu$ , e usando  $m > 0$  per portare la massa fuori dal modulo:  $\dot{m} = -||mu||/c = -m||u||/c$ . Adimensionalizzando (vedi la pagina di `ode_descent.m`) il fattore  $1/c$  diventa  $V_c = V_{ref}/c$ , da cui  $\dot{m} = -V_c m ||u||$ . Il fattore  $m$  è ciò che dice “un razzo pesante deve bruciare più propellente per ottenere la stessa accelerazione”.

**D: Cosa succederebbe se si validasse la soluzione GFOLD con `fwd_integrate` (cioè con il RHS T-hold) invece che con `fwd_integrate_uacc`?** R: Si misurerebbe un errore di nodo spurio, perché si starebbe integrando un plant diverso da quello che la trascrizione ha imposto. Il solver GFOLD ha prodotto ( $u_x, u_y$ ) piecewise-constant; ricostruendo  $T_k = m_{k,u_k}$  e tenendola costante sull'intervallo, l'accelerazione crescerebbe ( $m$  cala) mentre la trascrizione la assumeva piatta. L'errore sarebbe di ordine  $dt$  volte il tasso di variazione della massa, cioè  $O(V_c ||u|| * dt)$  relativo: piccolo ma ordini di grandezza sopra il  $7.3e-12$  riportato, e verrebbe scambiato per un difetto della trascrizione LTI che invece è esatta.

**D: La dinamica in  $z$  è lineare. Vuol dire che il problema di minimo consumo è convesso?** R: La dinamica sì, ma il problema completo no, per due vincoli: il bound superiore di spinta  $\sigma \leq T_{max} \exp(-z)$  (non convesso, va linearizzato o maggiorato) e – in un caso realistico con  $T_{min} > 0$  – il bound inferiore  $\sigma \geq T_{min} \exp(-z)$ , che nella forma originale  $||T|| \geq T_{min}$  è un vincolo “fuori da una palla”, palesemente non convesso. In HM2  $T_{min} = 0$  (`main_task2.m` riga 32), quindi il secondo problema non si pone e resta solo il primo, che il codice affronta con SCvx. Il vincolo di glide-slope e il cono  $\sigma \geq ||u||$  sono invece già convessi.

**D: Perché il file integra la massa  $m$  e non direttamente  $z = \ln m$ , visto che in  $z$  tutto è lineare?** R: Perché il suo compito è fare da **verifica indipendente**. Se integrasse  $z$  con la stessa legge lineare usata dalla trascrizione, il confronto sarebbe circolare e non proverebbe nulla. Integrando invece  $m$  con la legge non lineare  $\dot{m} = -V_c m ||u||$  e confrontando  $\exp(z_k)$  con  $m_k$ , si testa davvero che il cambio di variabili e la ricostruzione  $T = m*u$  siano corretti. In più, il layout  $[x; y; vx; vy; m]$  è quello atteso dal post-processing: `land` e i plot leggono la quinta colonna come massa fisica, `node_err` legge le colonne 1:4 (posizione e velocità, massa esclusa).

# HM2\_\_powered\_\_descent/rk4\_zoh.m

## Ruolo del file nel progetto

`rk4_zoh` e' il **propagatore discreto** della variante (a) di Task 2 ("Nonlinear ZOH + RK4"). Prende lo stato al nodo  $k$ , il controllo tenuto costante sull'intervallo, e restituisce lo stato al nodo  $k+1$  integrando `ode_descent` con un Runge-Kutta classico a 4 stadi e passo fisso.

Serve a costruire il vincolo di difetto della trascrizione ZOH. Mentre Task 1 usa la regola trapezoidale

$$x_{k+1} - x_k - (dt/2)*(f_k + f_{k+1}) = 0$$

la variante ZOH usa un **multiple-shooting** su un solo intervallo:

$$x_{k+1} - RK4(x_k, u_k, dt) = 0$$

(vedi `zoh_nonlcon`, `main_task2.m` righe 797-800). La differenza concettuale e' profonda: nel trapezoidale il difetto e' una *approssimazione* dell'integrale esatto (errore  $O(dt^2)$ ), mentre nel ZOH il difetto e' una *simulazione* del plant sotto un controllo che e' esattamente quello che verra' comandato. Il residuo di errore non e' piu' un errore di modello, ma solo l'errore di integrazione di RK4, che si puo' spingere a piacere aumentando `n_sub`. Il risultato si vede nel README di HM2: il replay `ode45` della soluzione trapezoidale atterra a 4.3 m dal pad, quello della soluzione ZOH+RK4 riproduce i nodi a  $1.4e-8$  nondim.

Chiamato da: `main_task2.m` riga 798 (`zoh_nonlcon`, dentro `fmincon`), `tests/rk4ZohTest.m`, `tests/descentDynamicsPerform` riga 57. Chiama: `ode_descent.m` (quattro volte per sotto-passo).

---

## rk4\_zoh (righe 1-25)

```
function x_next = rk4_zoh(x, u, dt, Vc, n_sub)
...
h = dt / n_sub;
for ii = 1:n_sub
    k1 = ode_descent(x, u, Vc);
    k2 = ode_descent(x + 0.5*h*k1, u, Vc);
    k3 = ode_descent(x + 0.5*h*k2, u, Vc);
    k4 = ode_descent(x + h*k3, u, Vc);
    x = x + (h/6)*(k1 + 2*k2 + 2*k3 + k4);
end
x_next = x;
```

- **Riga 1:** firma. `x` (5x1), `u` (2x1, la spinta tenuta costante), `dt` (lunghezza dell'intervallo ZOH), `Vc`, `n_sub` (numero di sotto-passi RK4 dentro l'intervallo). Il valore usato in produzione e' `n_sub = 2` (`main_task2.m` riga 37).
- **Righe 2-13:** docstring. Riga 12-13: *"No arguments validation by design: hot-loop propagator inside fmincon; validate at the call site."*

- **Riga 15:**  $h = dt / n\_sub$ . Il passo di integrazione **non** e' l'intervallo ZOH: l'intervallo viene suddiviso in  $n\_sub$  sotto-passi. Il controllo pero' resta lo stesso in tutti i sotto-passi – il ZOH e' sull'intervallo  $dt$ , il raffinamento e' solo numerico. Con  $tf\_nd = 2.173$  e  $N = 50$  si ha  $dt = 0.0443$ , quindi  $h = 0.0222$  con  $n\_sub = 2$ .
- **Righe 16-22:** il loop di sotto-passi. Notare che  $x$  viene **riassegnato in place** (riga 21): la variabile d'ingresso viene sovrascritta a ogni giro. In MATLAB e' innocuo (semantica per valore, copy-on-write), ed evita di allocare un secondo buffer nel hot loop.
- **Righe 17-20:** i quattro stadi. **Il punto centrale del file e' che u viene passato identico a tutti e quattro.** In un RK4 non autonomo standard gli stadi andrebbero valutati a tempi diversi ( $t_k$ ,  $t_k+h/2$ ,  $t_k+h/2$ ,  $t_k+h$ ) e il controllo andrebbe interpolato a quei tempi. Qui il controllo e' costante sull'intervallo, quindi non c'e' niente da interpolare e la dinamica e' **autonoma**: e' esattamente per questo che `ode_descent` non ha l'argomento  $t$ . Lo ZOH non e' un dettaglio dell'integratore, e' cio' che rende l'integrando autonomo.
- **Riga 21:** la combinazione con i pesi di Butcher classici  $(1, 2, 2, 1)/6$ .
- **Riga 23:**  $x\_next = x$ , lo stato a fine intervallo.

## Derivazione dei quattro stadi

Per un problema autonomo  $\dot{x} = f(x)$ , il RK4 classico (tableau di Kutta 1901) e':

```
k1 = f(x_k)
k2 = f(x_k + (h/2)*k1)
k3 = f(x_k + (h/2)*k2)
k4 = f(x_k + h*k3)
x_{k+1} = x_k + (h/6)*(k1 + 2*k2 + 2*k3 + k4)
```

L'idea e' quella di una **quadratura di Simpson sulla derivata**: se  $f$  dipendesse solo dal tempo, l'aggiornamento si ridurrebbe a  $x + (h/6)(f(t) + 4f(t+h/2) + f(t+h))$ , che e' esattamente Simpson –  $k2$  e  $k3$  sono due stime distinte della derivata a meta' intervallo, mediate con peso 2 ciascuna. Il fatto che  $k2$  usi  $k1$  e  $k3$  usi  $k2$  (invece di ripartire da  $x_k$ ) e' cio' che rende il metodo capace di catturare anche la dipendenza da  $x$ , portando la consistenza al quarto ordine.

**Ordine di accuratezza:** RK4 soddisfa le condizioni di ordine fino al quarto, quindi l'errore locale di troncamento e'  $O(h^5)$  e l'errore **globale** sull'intervallo e'  $O(h^4)$ . Con  $n\_sub$  sotto-passi l'errore sull'intero intervallo ZOH scala come  $(dt/n\_sub)^4$ : raddoppiare  $n\_sub$  divide l'errore per 16. Il test `testFourthOrderConvergence` (`rk4ZohTest.m` riga 34) misura l'ordine empirico confrontando  $n\_sub = 1, 2, 4, 8$  contro un riferimento `ode45` a  $RelTol$   $1e-12$  e verifica che l'ordine osservato sia  $> 3.5$ .

## La riga della massa e' integrata esattamente

Osservazione non banale, ed e' un test a se' (`testMassRowIsExact`, riga 26). Con  $u$  costante, la quinta componente di `ode_descent` vale

```
m_dot = -Vc * ||u|| = costante sull'intervallo
```

Applicare RK4 a un'equazione  $\dot{x} = \text{const}$  da'  $k1 = k2 = k3 = k4 = \text{const}$ , e l'aggiornamento diventa  $x + (h/6)(1+2+2+1)\text{const} = x + h*\text{const}$ : **esatto**, per qualunque numero di sotto-passi, anche  $n\_sub = 1$ . Quindi tutto l'errore di RK4 in questo problema vive nelle righe di posizione e velocita', mai nella massa. Il test lo verifica a  $1e-14$ : `m_end = m0 - Vc*||u||*dt`.

Questo e' un altro regalo del ZOH: **se il controllo fosse piecewise-lineare** (FOH), la spinta  $T(t)$  sarebbe affine in  $t$  ma il suo **modulo**  $||T(t)||$  non lo sarebbe (la norma di una funzione affine e' una iperbole, non una retta), e la riga della massa introdurrebbe errore di integrazione. Con lo ZOH questo non succede mai.

**Possibile domanda d'esame** – Perche' usare un RK4 a passo fisso dentro il vincolo, invece di chiamare direttamente `ode45` che e' piu' accurato? *Risposta:* Perche' `fmincon` calcola i gradienti dei vincoli alle differenze finite, e ha bisogno che `nonlcon` sia una funzione **liscia e deterministica**

delle variabili di decisione. Un integratore adattivo cambia il numero e la lunghezza dei passi in modo discontinuo quando gli input variano di poco: il vincolo risultante ha un “rumore numerico” dell’ordine della tolleranza, che distrugge la stima alle differenze finite (il rapporto incrementale con  $\delta \sim 1e-7$  misura il rumore, non la derivata). RK4 a passo fisso e’ invece una composizione finita di operazioni lisce: la mappa  $(x_k, u_k) \rightarrow x_{k+1}$  e’ C-infinito (a parte il kink di  $\|u\|$  in  $u = 0$ ) e i gradienti FD sono affidabili. In piu’ e’ molto piu’ veloce: `ode45` a `RelTol 1e-10` su un intervallo costa decine di valutazioni del RHS, RK4 con `n_sub = 2` ne costa 8.

---

## Perche’ ZOH e non un’interpolazione di ordine superiore

Cinque ragioni, in ordine di forza:

1. **Realismo di implementazione.** Un computer di bordo aggiorna il comando di spinta a istanti discreti e lo *tiene* fino all’aggiornamento successivo. Lo ZOH e’ letteralmente cio’ che l’attuatore fara’. Una soluzione trascritta con controllo *piecewise-linear* non e’ direttamente eseguibile: bisognerebbe ricampionarla, reintroducendo un errore. Il “replay gap” del trapezoidale (4.3 m dal pad, README) e’ esattamente questo fenomeno.
2. **La dinamica diventa autonoma.** Con  $u$  costante non serve interpolare il controllo agli stadi intermedi di RK4, e `ode_descent` non ha nemmeno bisogno dell’argomento tempo.
3. **La discretizzazione lineare ha forma chiusa.** Per la variante LTV (Appendice A) le matrici discrete si ottengono da  $B_k = \int \Phi(t_{k+1}, s) B(s) ds$  su un solo hold. Con un **first-order hold** servirebbero **due** matrici di ingresso per intervallo (una pesata sul valore a inizio intervallo, una su quello a fine intervallo), raddoppiando il sistema aumentato di `ltv_aug_rhs` e la dimensione delle matrici da propagare. Il costo del guadagno di accuratezza non e’ gratis.
4. **La riga della massa resta esatta** (vedi sopra): con FOH la norma della spinta non sarebbe piu’ costante e il canale di massa introdurrebbe errore.
5. **La soluzione ottima e’ bang-off-bang.** Il profilo di spinta ottimo e’ max-coast-max (README: switch a  $t \sim 14.0$  s e  $t \sim 33.1$  s): a meno degli istanti di switch e’ **gia’ piecewise-constant in modulo**. Un’interpolazione di ordine superiore non aggiungerebbe niente di utile, mentre soffrirebbe di ringing attorno alle discontinuita’.

**Il prezzo da pagare (onesta’).** Lo ZOH e’ una rappresentazione di ordine 0 del controllo: rispetto all’ottimo *continuo* la soluzione ZOH e’ sub-ottima di  $O(dt)$ , e i tempi di switch sono risolti solo con la granularita’ della griglia. La trascrizione riproduce con altissima fedelta’ ( $1.4e-8$ ) **il proprio** plant discreto – cioe’ e’ *consistente*, non necessariamente piu’ *ottima*. Infatti le masse finali delle quattro varianti in HM2 sono molto simili: nessuna e’ “piu’ ottima”, cambiano il costo computazionale e la fedelta’ del replay.

---

## Costo computazionale e assenza del blocco arguments

Righe 12-13 dichiarano l’assenza della validazione. Il conto e’ brutale:

- Una valutazione di `zoh_nonlcon = (N-1) = 49` chiamate a `rk4_zoh`, ciascuna con `n_sub = 2` sotto-passi da 4 stadi = **392 valutazioni di `ode_descent`**.
- `fmincon` in SQP con gradienti FD su  $7N = 350$  variabili ricostruisce lo Jacobiano dei vincoli con  $\sim 351$  valutazioni di `nonlcon` per iterazione maggiore =  **$\sim 1.4e5$  chiamate a `ode_descent` per iterazione**.
- Con `MaxIterations = 1000` (`fmincon_opts`, riga 733) si arriva nell’ordine di  $1e8$  valutazioni del RHS per una singola risoluzione.

Un blocco `arguments` con `mustBeFinite` / controlli di dimensione aggiungerebbe un overhead confrontabile con il costo dell’aritmetica stessa (il benchmark stima `ode_descent` intorno ai 60 ns). Da qui la scelta, dichiarata anche in `CLAUDE.md`. La validazione sta nelle funzioni di frontiera: `solve_zoh`, `nondim`, `fwd_integrate`.



**Possibile domanda d'esame** – Come si sceglie  $n_{\text{sub}}$ ? *Risposta:* E' un compromesso fra fedeltà e costo, entrambi lineari-ish in  $n_{\text{sub}}$  (il costo cresce come  $n_{\text{sub}}$ , l'errore cala come  $n_{\text{sub}}^{-4}$ ). Con  $n_{\text{sub}} = 2$  e  $h = 0.022$  nondim l'errore di nodo osservato e'  $1.4\text{e-}8$ , gia' molto sotto la **ConstraintTolerance** di  $1\text{e-}6$  di **fmincon**: raffinare oltre sarebbe sprecato, perche' l'errore dominante diventerebbe quello del solver, non quello dell'integratore. La regola pratica e': si sceglie  $n_{\text{sub}}$  minimo tale che l'errore di integrazione sia almeno un ordine di grandezza sotto la tolleranza sui vincoli.

---

## Possibili domande d'esame

**D: Che differenza c'è fra il difetto trapezoidale e il difetto ZOH+RK4?** R: Il trapezoidale impone  $x_{k+1} - x_k = (dt/2)(f_k + f_{k+1})$ : e' un'approssimazione dell'integrale, accurata a  $O(dt^2)$ , e presuppone un controllo *piecewise-lineare* (i due  $f$  coinvolgono  $u_k$  e  $u_{k+1}$ ). Il ZOH impone  $x_{k+1} = RK4(x_k, u_k, dt)$ : non approssima l'integrale, lo *simula* sotto un controllo che e' esattamente quello che sara' comandato, con errore  $O(h^4)$  riducibile a piacere. Il primo e' *collocation\** (implicito, tutti i nodi accoppiati due a due), il secondo e' *multiple shooting* (esplicito, ogni intervallo propagato in avanti). Entrambi producono un NLP con le stesse variabili di decisione, ma il secondo ha un errore di replay molto piu' piccolo.

**D: Perche' il controllo  $u$  viene passato identico ai quattro stadi di RK4?** R: Perche' e' tenuto costante sull'intervallo (zero-order hold). In un RK4 non autonomo gli stadi campionerebbero il controllo a  $t_k$ ,  $t_k+h/2$ ,  $t_k+h/2$  e  $t_k+h$ ; essendo  $u$  costante quei quattro valori coincidono. Corollario: la dinamica e' autonoma sull'intervallo, ed e' per questo che **ode\_descent** non prende l'argomento tempo. Se il controllo fosse *piecewise-lineare* bisognerebbe valutarlo a tempi diversi e il RHS dovrebbe accettare  $t$ .

**D: Qual e' l'ordine di accuratezza e come si verifica?** R: RK4 classico ha errore locale  $O(h^5)$  e globale  $O(h^4)$ . Il test **testFourthOrderConvergence** lo misura empiricamente: calcola l'errore rispetto a un riferimento **ode45** (RelTol  $1\text{e-}12$ ) per  $n_{\text{sub}} = 1, 2, 4, 8$ , prende  $\log_2(\text{err}_i / \text{err}_{i+1})$  e verifica che sia  $> 3.5$  (cioe' che dimezzando il passo l'errore cali di circa un fattore 16). Attenzione: la riga della massa e' integrata *esattamente* per costruzione, quindi l'ordine misurato riguarda solo le componenti di posizione e velocita'.

**D: Perche' RK4 e non un metodo implicito o simplettico?** R: Il problema non e' rigido (non ci sono scale temporali molto separate: massa, posizione e velocita' evolvono tutte sulla scala di  $t_{\text{ref}} = 17.5$  s) quindi non serve un metodo implicito, che costerebbe una risoluzione non lineare dentro ogni sotto-passo, dentro ogni valutazione del vincolo. Non e' nemmeno un problema hamiltoniano conservativo (c'e' dissipazione di massa e un controllo esterno), quindi un simplettico non porta vantaggi. RK4 esplicito e' il punto ottimo: alto ordine, zero risoluzioni implicite, mappa liscia per i gradienti FD.

**D: Il file non ha il blocco arguments. Non e' un rischio?** R: E' una scelta di design dichiarata (righe 12-13) e allineata a **CLAUDE.md**. Il propagatore vive dentro il loop di **fmincon**, dove viene chiamato  $\sim 1\text{e}5$  volte per iterazione: la validazione costerebbe piu' del calcolo. Il contratto ( $x$   $5 \times 1$ ,  $u$   $2 \times 1$ ,  $dt > 0$ ,  $n_{\text{sub}}$  intero positivo) e' garantito dal chiamante **zoh\_nonlcon**, che a sua volta e' costruito da **solve\_zoh**, che ha la validazione. Il costo di questa scelta e' che un bug di dimensioni si manifesterebbe come un errore criptico di MATLAB dentro **ode\_descent**, non come un messaggio chiaro: e' il prezzo che si paga.

# HM2\_\_powered\_\_descent/lti\_zoh.m

## Ruolo del file nel progetto

Questo file discretizza **esattamente** il sistema LTI che si ottiene dopo il cambio di variabili GFOLD (Acikmese & Blackmore):  $z = \ln m$ ,  $u = T/m$ , con lo slack  $\sigma \geq \|u\|$ . E' il cuore numerico della **variante (d) di Task 2**, ed e' il file dove si vede materializzato tutto il guadagno del cambio di variabili: la dinamica di powered descent, che in coordinate  $(m, T)$  e' non lineare e non convessa, diventa qui un sistema **lineare, tempo-invariante e con offset costante**, discretizzabile con un unico esponenziale di matrice.

Il contrasto con le altre varianti e' netto:

| Variante         | Come si ottengono le matrici discrete                                                                                                                                                    | Costo                                         |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| (a) ZOH+RK4      | non si linearizza affatto: <code>rk4_zoh</code> propaga la dinamica non lineare                                                                                                          | 392 valutazioni RHS per vincolo               |
| (b)/(c) LTV+SCvx | <code>compute_ltv_zoh</code> integra con <code>ode45</code> un sistema aumentato a 70 stati ( $\Phi, \Psi, \beta, \gamma$ ) su ciascuno dei 49 intervalli, <b>a ogni iterazione SCvx</b> | 49 chiamate <code>ode45</code> per iterazione |
| (d) GFOLD        | <code>lti_zoh</code> : un <code>expm</code> di una matrice $9 \times 9$ , <b>una volta sola</b>                                                                                          | 1 <code>expm</code> , punto                   |

`main_task2.m` riga 1218: `[Abar, Bbar, cbar] = lti_zoh(tf/(N-1), d.Vc);` con il commento “*exact LTI ZOH, once*” – fuori dal ciclo SCvx, perche' le matrici non dipendono ne' dal nodo (tempo-invarianza) ne' dalla traiettoria di riferimento (non c'e' nulla da linearizzare nella dinamica). Le tre matrici finiscono poi nel vincolo di dinamica del SOCP (`solve_gfold_socp`, riga 1156):

```
XI(:,k+1) == Abar*XI(:,k) + Bbar*W(:,k) + cbar
```

che e' un'**uguaglianza lineare** nelle variabili di decisione – ed e' esattamente questo che permette al sotto-problema di essere un SOCP genuino, risolvibile da ECOS in frazioni di secondo con certificato di ottimalita' globale.

Testato da `tests/gfoldLogMassTest.m` righe 55-83 (forma chiusa analitica + confronto con `ode45`).

## lti\_zoh (righe 1-28)

```
function [Abar, Bbar, cbar] = lti_zoh(dt, Vc)
...
A = zeros(5);    A(1,3) = 1;  A(2,4) = 1;
B = zeros(5,3);  B(3,1) = 1;  B(4,2) = 1;  B(5,3) = -Vc;
```

```

c = [0; 0; 0; -1; 0];
E = expm([A, B, c; zeros(4,9)] * dt);
Abar = E(1:5, 1:5); Bbar = E(1:5, 6:8); cbar = E(1:5, 9);

```

- **Riga 1:** firma. Restituisce **tre** oggetti: la matrice di transizione **Abar** (5x5), la matrice di ingresso **Bbar** (5x3) e l'**offset affine** **cbar** (5x1). Il terzo non e' un dettaglio: la gravita' e' un termine costante, non un controllo, quindi il sistema non e'  $\dot{x} = Ax + Bw$  ma  $\dot{x} = Ax + Bw + c$ , e la sua discretizzazione richiede un terzo pezzo.
- **Righe 2-18:** la docstring, che dichiara esplicitamente lo stato  $\mathbf{x} = [x; y; vx; vy; z]$  con  $z = \ln m$ , il controllo  $\mathbf{w} = [ux; uy; \sigma]$  e la dinamica continua. Righe 11-13 sottolineano che le matrici sono **CONSTANT across the grid** perche' il sistema e' tempo-invariante. Righe 15-18 anticipano il trucco della matrice a blocchi.
- **Righe 19-22: il blocco arguments.** *Attenzione: contrariamente a `ode_descent.m`, `ode_descent_uacc.m` e `rk4_zoh.m`, questo file ha la validazione degli input* (**dt** positivo e finito, **Vc** finito). Non e' una dimenticanza ne' un'incoerenza: **lti\_zoh non e' una funzione di hot loop**. Viene chiamata una volta sola per risoluzione (riga 1218 di `main_task2.m`, fuori dal ciclo SCvx), quindi ricade nella categoria "boundary helper" della convenzione di `CLAUDE.md`, dove la validazione e' dovuta. Questa e' proprio la dimostrazione che la regola "niente **arguments** nel hot loop" e' applicata con criterio e non a tappeto. Nota minore: **Vc** e' validato solo come finito, non come positivo. **Vc = 0** (Isp infinito, nessun consumo) e' un caso limite fisicamente sensato e resta ammesso; **Vc < 0** sarebbe non fisico ma non viene intercettato.
- **Riga 23:**  $A(1,3) = 1$ ;  $A(2,4) = 1$  – le due sole entrate non nulle:  $\dot{x} = vx$  e  $\dot{y} = vy$ . E' un **doppio integratore** in 2D. Il resto di **A** e' zero: nessun accoppiamento con la massa, nessun drag, nessun gradiente di gravita'.
- **Riga 24:**  $B(3,1) = 1$ ;  $B(4,2) = 1$ ;  $B(5,3) = -Vc$  – l'accelerazione comandata entra direttamente nelle righe di velocita' (coefficiente 1, non  $1/m$ : e' il guadagno del cambio di variabili  $u = T/m$ ), e lo slack  $\sigma$  entra nella riga di log-massa con coefficiente  $-Vc$ . Cioe'  $\dot{z} = -Vc \sigma$ : la riga della massa non dipende piu' dallo stato.
- **Riga 25:**  $\mathbf{c} = [0; 0; 0; -1; 0]$  – la gravita' adimensionale, che agisce solo su  $\dot{y}$ . E' un **termine affine**, non un controllo: e' noto e costante, e va discretizzato come tale.
- **Riga 26:** l'esponenziale della matrice aumentata (vedi sotto).
- **Riga 27:** estrazione dei tre blocchi dal risultato 9x9.

## Derivazione della discretizzazione ZOH esatta

Il sistema continuo e'

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{w} + \mathbf{c} \quad \text{con } \mathbf{w} \text{ costante su } [t_k, t_k + dt] \quad (\text{ZOH})$$

La soluzione esatta di un sistema lineare con ingresso costante e' la formula di variazione delle costanti:

$$\mathbf{x}(t_k + dt) = \expm(\mathbf{A}dt)\mathbf{x}(t_k) + \int_0^{dt} \expm(\mathbf{A}s) \mathbf{B}\mathbf{w} + \mathbf{c} \, ds$$

da cui, per confronto con  $\mathbf{x}_{k+1} = \mathbf{Abar}\mathbf{x}_k + \mathbf{Bbar}\mathbf{w}_k + \mathbf{cbar}$ :

$$\mathbf{Abar} = \expm(\mathbf{A}dt)$$

$$\mathbf{Bbar} = \left[ \int_0^{dt} \expm(\mathbf{A}s) \mathbf{B} \, ds \right] * \mathbf{B}$$

$$\mathbf{cbar} = \left[ \int_0^{dt} \expm(\mathbf{A}s) \mathbf{c} \, ds \right] * \mathbf{c}$$

Il fatto che l'ingresso sia **costante** e' cio' che permette di portarlo fuori dall'integrale. Se  $\mathbf{w}(t)$  variasse (per esempio FOH), l'integrale sarebbe  $\int_0^{dt} \expm(\mathbf{A}(dt-s))\mathbf{B}\mathbf{w}(s) \, ds$  e servirebbero due matrici di ingresso distinte.

## Il trucco della matrice a blocchi (van Loan)

Calcolare l'integrale di un esponenziale di matrice a mano e' scomodo (e diventa singolare se A non e' invertibile – qui A **non lo e'**, e' addirittura nilpotente, quindi la scorciatoia  $A^{-1} * (\expm(A*dt) - I) * B$  **non e' utilizzabile**). Il trucco di van Loan lo evita: si costruisce la matrice aumentata (9x9)

```
M = [ A   B   c ] * dt
     [ 0   0   0 ]      <- 4 righe di zeri (3 per w, 1 per la costante)
```

e si dimostra che

```
expm(M) = [ Abar   Bbar   cbar ]
           [ 0      I3     0   ]
           [ 0      0      1   ]
```

**Dimostrazione** (serve saperla): sia  $M = [[F, G], [0, 0]]$  con  $F = A dt$  e  $G = [B c] dt$ . Per induzione,  $M^n = [[F^n, F^{(n-1)}G], [0, 0]]$  per ogni  $n \geq 1$  (il blocco in basso e' nullo, quindi le potenze non lo "risvegliano" mai). Sommando la serie:

```
expm(M) = I + somma_{n>=1} M^n/n!
         = [ somma F^n/n!      (somma_{n>=1} F^{(n-1)}/n!) * G ]
           [ 0                  I                               ]
```

Il blocco in alto a sinistra e'  $\expm(F) = \expm(A dt) = \mathbf{Abar}$ . Il blocco in alto a destra:  $\text{somma}_{\{n \geq 1\}} F^{(n-1)}/n! = \int_0^1 \expm(Fs) ds$  (basta integrare la serie termine a termine). Quindi il blocco vale  $\int_0^1 \expm(Adts) * [B c] dt ds$ , e con la sostituzione  $\tau = dts$  ( $d\tau = dtds$ ) diventa

```
integrale_0^dt expm(A*tau) * [B c] dtau = [ Bbar   cbar ]
```

che e' esattamente cio' che cercavamo. Il codice lo scrive nella docstring alla riga 17 e lo implementa alla riga 26. Un solo `expm` di una 9x9 restituisce **tutte e tre** le matrici in un colpo, senza integrare nulla e senza invertire A.

## Verifica in forma chiusa

Qui la struttura di A e' cosi' semplice che la forma chiusa si scrive a mano – ed e' quello che fa il test `testZohClosedForm` (`gfoldLogMassTest.m` riga 55).

**A e' nilpotente di indice 2:**  $(A^2)_{(1,j)} = \text{riga 3 di } A = 0$ ,  $(A^2)_{(2,j)} = \text{riga 4 di } A = 0$ , quindi  $A^2 = 0$ . La serie esponenziale si tronca dopo due termini:

```
expm(A*s) = I + A*s
integrale_0^dt (I + A*s) ds = dt*I + (dt^2/2)*A
```

Da cui:

```
Abar = I + A*dt      ->  Abar(1,3) = dt,  Abar(2,4) = dt, resto identita'
Bbar = dt*B + (dt^2/2)*A*B
cbar = dt*c + (dt^2/2)*A*c
```

Svolgendo ( $A*B$  ha riga 1 = riga 3 di B e riga 2 = riga 4 di B):

```
Bbar = [ dt^2/2   0       0       ]      cbar = [ 0           ]
        [ 0       dt^2/2   0       ]      [ -dt^2/2 ]
        [ dt      0       0       ]      [ 0         ]
        [ 0       dt      0       ]      [ -dt        ]
        [ 0       0       -Vc*dt  ]      [ 0          ]
```

Che sono **letteralmente** `B_exp` e `c_exp` del test (righe 60-65). La lettura fisica e' immediata:

- righe 1-2 di Bbar: lo spostamento dovuto a un'accelerazione costante,  $(1/2)adt^2$ ;
- righe 3-4: la variazione di velocita',  $a*dt$ ;
- riga 2 e 4 di cbar: la **caduta libera** su un intervallo,  $-(1/2)gdt^2$  di quota e  $-g*dt$  di velocita' verticale (con  $g = 1$  adimensionale);
- riga 5 di Bbar:  $z_{k+1} = z_k - Vc_{\sigma} kdt$ , la depletion di log-massa, perfettamente lineare.

Quest'ultima riga chiude anche il cerchio sulla funzione obiettivo: poiché  $z_N = z_0 - \int \sum_k \sigma_k$ , massimizzare  $z_N$  (il costo del SOCP è  $-XI(5,N)$ , riga 1183) equivale a **minimizzare la somma degli  $\sigma_k$** , cioè a minimizzare il consumo. L'obiettivo è lineare nelle variabili di decisione.

**Possibile domanda d'esame** – Perché non calcolare  $\bar{B}$  con la formula  $A^{-1} * (\expm(A dt) - I) \bar{B}$ , che si trova su tutti i libri? *Risposta:* Perché qui **A è singolare** (è nilpotente:  $A^2 = 0$ , quindi tutti gli autovalori sono nulli e  $\det(A) = 0$ ), e quella formula richiede A invertibile. È un caso tutt'altro che raro: qualunque doppio integratore ha A nilpotente. Il trucco della matrice a blocchi funziona **sempre**, invertibile o no, ed è anche numericamente più stabile (non moltiplica per l'inversa di una matrice mal condizionata).

## “Esatta”: in che senso, e quando conviene davvero rispetto a RK4

**In che senso è esatta.** La mappa  $xi_{k+1} = \bar{A} x_k + \bar{B} w_k + \bar{c}$  riproduce la soluzione della ODE continua sotto ZOH **senza alcun errore di troncamento**, per qualunque  $dt$ : non c'è un  $O(h^p)$  da controllare, non c'è un  $n_{sub}$  da scegliere. L'unico errore è quello di macchina di  $\expm$  ( $\sim 1e-16$ ). Il test `testZohMatchesOde45` (riga 71) lo conferma confrontando un passo discreto con un'integrazione `ode45` a  $RelTol$   $1e-12$  (tolleranza  $1e-9$ ), e il replay non lineare completo raggiunge un errore di nodo di  $7.3e-12$  nondim (README).

**Onesta' – in questo caso specifico anche RK4 sarebbe esatto.** Vale la pena dirlo perché è una domanda che un professore può fare. Poiché  $A^2 = 0$ , la soluzione esatta su un intervallo è un **polinomio di grado 2 in  $dt$** , e RK4 è esatto sui polinomi fino al grado 4. Facendo il conto (con  $b = B*w + c$  e  $A^2 = 0$ ):

```
k1 = A*xi + b
k2 = k3 = k1 + (h/2)*A*b
k4 = k1 + h*A*b
xi + (h/6)*(k1 + 2*k2 + 2*k3 + k4) = xi + h*(A*xi + b) + (h^2/2)*A*b
```

che coincide con  $(I + Ah)xi + (hI + (h^2/2)A)*b$ , cioè con la formula esatta. Quindi **su questo particolare sistema RK4 anche con  $n_{sub} = 1$  darebbe lo stesso risultato**. Il vantaggio dell'`expm` non è l'accuratezza in questo caso: è strutturale.

**I veri motivi per cui `lti_zoh` è preferibile:**

1. **Costo.** Le matrici sono costanti: un `expm` 9x9, una volta sola per l'intera risoluzione. La variante LTV (`b/c`) deve integrare con `ode45` un sistema aumentato a 70 stati su 49 intervalli **a ogni iterazione SCvx**; la variante
  - (a) ricalcola 392 valutazioni del RHS a ogni valutazione di vincolo. Il GFOLD converge in 3 iterazioni SCvx in  $\sim 5$  s di wall time (README).
2. **Convessita'.** Con  $\bar{A}$ ,  $\bar{B}$ ,  $\bar{c}$  **costanti e note**, il vincolo di dinamica è un'uguaglianza **lineare** nelle variabili di decisione: il sotto-problema è un SOCP genuino, che ECOS risolve a ottimalità globale. Con RK4 sulla dinamica non lineare in  $(m, T)$  il difetto è un'uguaglianza non convessa e serve `fmincon`, con tutte le sue patologie (minimi locali, gradienti FD, stallo sul kink di  $\|T\|$ ).
3. **Niente riga di massa singolare** (dichiarato alla riga 18 della docstring). Nelle coordinate originali la linearizzazione ha  $A(3,5) = -T_x/m^2$  e  $A(4,5) = -T_y/m^2$  (`jacobians`, `main_task2.m` righe 270-271): termini che esplodono quando  $m \rightarrow 0$  e che rendono la linearizzazione fragile a fine traiettoria, dove il razzo è leggero. In log-massa questi termini **non esistono**: le righe di velocità non dipendono affatto dallo stato.
4. **Generalita'.** Se il modello venisse arricchito (drag lineare, frame rotante con Coriolis, gradiente di gravità), **A non sarebbe più nilpotente** e la soluzione esatta diventerebbe genuinamente trascendente: lì RK4 avrebbe un errore  $O(h^4)$  reale, mentre `expm` resterebbe esatto. La formulazione a blocchi si adatta senza modifiche.
5. **Nessun parametro da sintonizzare.** Nessun  $n_{sub}$ , nessuna analisi di convergenza di griglia sull'integratore.

---

## Cosa resta non convesso (e perche' c'e' ancora una SCvx)

E' il limite piu' importante da esplicitare. La **dinamica** e' esatta e lineare, ma **un vincolo non lo e'**: il bound superiore di spinta  $\|T\| \leq T_{\max}$ , riscritto in coordinate GFOLD, diventa

```
sigma <= Tmax * exp(-z)
```

Il membro destro e' **convesso in z**, quindi "variabile  $\leq$  funzione convessa" e' un vincolo **non convesso**. Il codice lo linearizza con Taylor al prim'ordine attorno al riferimento  $z_{\text{ref}}$  (`main_task2.m` righe 1158-1159):

```
sigma_k <= Tmax * exp(-z_ref_k) * (1 - (z_k - z_ref_k))
```

e itera su  $z_{\text{ref}}$  nel ciclo SCvx, con trust region adattiva e ratio test. **Quindi la variante (d) non e' un singolo SOCP**: e' una successione di SOCP. Il vero single-shot (lossless convexification completa) e' ancora nel backlog come ticket T006, come dichiara il README.

Nota: l'altro vincolo tipicamente non convesso – il **lower bound**  $\|T\| \geq T_{\min}$  (vincolo "fuori da una palla") – in HM2 non si pone, perche' `data.Tmin = 0` (`main_task2.m` riga 32). Con  $T_{\min} > 0$  servirebbe la maggiorazione quadratica di Acikmese-Blackmore, ed e' proprio li' che il teorema di *lossless convexification* diventa non banale.

---

## Possibili domande d'esame

**D: Deriva Abar e Bbar per un sistema LTI con ingresso ZOH.** R: Dalla variazione delle costanti,  $x_i(t_k + dt) = \expm(Adt)x_{i,k} + \int_0^{dt} \expm(A(dt - \tau))Bw(\tau) d\tau$ . Con  $w$  costante (ZOH) l'ingresso esce dall'integrale  $e$ , con il cambio di variabile  $s = dt - \tau$ , si ottiene  $Bbar = [\int_0^{dt} \expm(A*s) ds] B$ , e  $Abar = \expm(Adt)$ . L'offset costante  $c$  (la gravita') si tratta identicamente:  $cbar = [\int_0^{dt} \expm(A*s) ds] c$ . Sono **esatti**, non approssimati: nessun errore di troncamento, per qualunque  $dt$ .

**D: Spiega il trucco della matrice a blocchi. Perche' funziona?** R: Si costruisce  $M = [[A, B, c], [0, 0, 0]] * dt$  (9x9, con 4 righe di zeri). Poiche'  $M$  e' triangolare a blocchi con blocco diagonale inferiore nullo, si ha  $M^n = [[F^n, F^{(n-1)}G], [0, 0]]$  con  $F = A dt$ ,  $G = [B c] dt$ . Sommando la serie esponenziale, il blocco in alto a sinistra da'  $\expm(Adt) = Abar$  e quello in alto a destra da'  $(\sum_{n=1}^{\infty} F^{(n-1)}G/n!) = \int_0^{dt} \expm(Fs) ds G$ , che con la sostituzione  $\tau = dts$  e' esattamente  $[Bbar cbar]$ . Un solo **expm** restituisce tutte e tre le matrici. Il vantaggio pratico: funziona **anche se A e' singolare** (qui  $A$  e' nilpotente, quindi la formula  $A^{-1}(\expm(Adt) - I)B$  non sarebbe applicabile).

**D: Perche' la dinamica GFOLD e' esattamente LTI, mentre quella originale non lo e'?** R: Per il doppio cambio di variabili di Acikmese & Blackmore. (i) Prendendo come controllo l'accelerazione  $u = T/m$  invece della spinta, le righe di velocita' perdono il fattore  $1/m$  e diventano un doppio integratore:  $\dot{v}_x = u_x$ ,  $\dot{v}_y = u_y - 1$ . (ii) Prendendo come stato  $z = \ln m$  invece di  $m$ , la riga della massa  $\dot{m} = -Vc m \|u\|$  diventa  $\dot{z} = -Vc \|u\|$ , indipendente dallo stato. (iii) Introducendo lo slack  $\sigma \geq \|u\|$  (vincolo conico convesso) si ha  $\dot{z} = -Vc \sigma$ , lineare. Tutte le non linearita' della dinamica sono state spostate in un vincolo **convesso**. Nel codice, il punto (iii) e'  $B(5,3) = -Vc$  (riga 24) e il cono e'  $\text{norm}(W(1:2,k)) \leq W(3,k)$  (riga 1157 di `main_task2.m`).

**D: Il costo del SOCP e' -XI(5,N). Perche' e' equivalente a minimizzare il consumo, e come si sa che  $\sigma = \|u\|$  all'ottimo?** R:  $XI(5,N) = z_N = \ln(m_f)$ , quindi massimizzare  $z_N$  e' massimizzare  $m_f$ , cioe' minimizzare il propellente ( $m_0$  e' fissata). Dalla riga 5 di `Bbar`,  $z_N = z_0 - Vc dt \sum \sigma_k$ : massimizzare  $z_N$  significa minimizzare la somma degli  $\sigma_k$ , e l'ottimizzatore li spinge quindi contro il loro **unico lower bound**, che e' il cono  $\sigma_k \geq \|u_k\|$ . Il cono si chiude da solo, senza bisogno di imporlo: e' il nucleo dell'argomento di *lossless convexification*. Il test `testMassRowConsistency` (`gfoldLogMassTest.m` riga 85) verifica numericamente che, con  $\sigma = \|u\|$ , l'aggiornamento LTI di  $z$  coincide con la depletion di massa non lineare integrata da `ode45`.

**D: Se la discretizzazione e' esatta e la dinamica lineare, perche' serve ancora un ciclo SCvx?** R: Perche' e' la **dinamica** a essere esatta, non tutti i vincoli. Il bound superiore  $\|T\| \leq T_{\max}$  diventa  $\sigma \leq T_{\max} \exp(-z)$ , che e' non convesso (maggiorante convesso). Il codice lo linearizza attorno a  $z_{\text{ref}}$  e itera. Con la lossless convexification completa (ticket T006) si potrebbe fissare il punto di espansione su un profilo di massa a priori e risolvere **un solo** SOCP; qui si e' scelta la strada SCvx, che comunque converge in 3 iterazioni perche' la dinamica esatta non introduce errore di linearizzazione.

**D: Questo file ha il blocco arguments mentre gli altri tre no. Incoerenza?** R: No, e' esattamente la regola applicata correttamente. `ode_descent`, `ode_descent_uacc` e `rk4_zoh` sono chiamati  $\sim 1e5$  volte per iterazione dentro `fmincon/ode45`, e per loro la validazione costerebbe piu' del calcolo (una valutazione di `ode_descent` sta sui 60 ns). `lti_zoh` invece e' chiamata **una volta per risoluzione** (riga 1218, fuori dal ciclo SCvx), quindi e' un "boundary helper" e la validazione e' dovuta e gratuita. La convenzione di `CLAUDE.md` dice proprio questo: niente `arguments` nel hot loop, validazione al confine.

**D: Cosa succederebbe se dimenticassi cbar?** R: Il razzo cadrebbe in assenza di gravita'. `cbar = [0; -dt^2/2; 0; -dt; 0]` e' la discretizzazione esatta del termine affine  $c = [0; 0; 0; -1; 0]$ , cioe' la caduta libera:  $-(1/2)gdt^2$  di quota e  $-gdt$  di *velocita' verticale per intervallo* (con  $g = 1$  adimensionale). Non e' assorbibile in  $Bbar$  perche' la gravita' **non e' un controllo**: e' un ingresso noto e costante. E' la ragione per cui il sistema si scrive  $\dot{x} = Ax + Bw + c$  e non semplicemente  $\dot{x} = Ax + Bw$ , e per cui la matrice aumentata ha una colonna in piu' (la nona).

# HM2\_\_powered\_\_descent/main\_\_task1.m

## Ruolo del file nel progetto

Questo e' lo script "Task 1" di HM2: risolve il problema di **powered descent and landing** a combustibile minimo con un **metodo diretto**, cioe' trascrivendo il problema di controllo ottimo continuo in un **NLP a dimensione finita** e passandolo a `fmincon`. E' il file di riferimento dell'homework: le quattro varianti del Task 2 (ZOH+RK4 nonlineare, SCvx LTV con `fmincon`, SCvx LTV con YALMIP/ECOS, GFOLD log-massa) sono tutte confronti contro questa trascrizione, e due di esse partono da una sua soluzione come warm start.

La trascrizione scelta e' la **collocazione diretta trapezoidale** su una griglia uniforme di  $N = 50$  nodi, con tempo finale **fissato** (`tf = 38 s`, dato dalla traccia). Il vettore delle incognite impila stato e controllo a ogni nodo; i vincoli di collocazione ("defect") impongono che la traiettoria discreta soddisfi le equazioni del moto; il costo massimizza la massa finale, che e' esattamente "minimizza il carburante" perche' la massa iniziale e' fissata.

Il file e' **autocontenuto tranne per un'unica dipendenza esterna**: `ode_descent.m`, il right-hand side della dinamica adimensionale (riga 314, tramite il wrapper `dyn_rhs`). Tutto il resto – non dimensionalizzazione, guess iniziale, bounds, condizioni al contorno, vincoli di percorso, diagnostiche KKT, studio di convergenza sulla griglia, plot, export figure – vive dentro `main_task1.m` come local function. Nessuna dipendenza da toolbox esterni (YALMIP/ECOS servono solo alle varianti (c)/(d) del Task 2).

Il flusso e': dati SI -> non dimensionalizzazione -> sweep di sensitivita' su `tf` (tre solve indipendenti a 0.95/1.00/1.05 volte il nominale) -> plot ed export -> studio di convergenza di griglia ( $N = 25/50/100$ ) che misura la fedelta' rigiocando il controllo ottimo attraverso `ode45`.

---

## Header e setup (righe 1-12)

```
% Solved non-dim: L_ref = y0, a_ref = g,  
% t_ref = sqrt(L_ref/g), V_ref = sqrt(g*L_ref),  
% m_ref = m0, T_ref = m0*g. Only residual  
% parameter is V_c = V_ref/c.
```

- Righe 1-10: il blocco di commento dichiara tutte le scelte di progetto – trapezoidale, durata fissa, minimo carburante, punto materiale 2D senza aerodinamica su Terra piatta, solver `fmincon` con algoritmo `sqp`.
- Riga 12: `clear; close all; clc;`. Il `close all` e' funzionale: il blocco di export figure (righe 62-78) fa `findobj(groot, 'Type', 'figure')` e salva *tutte* le figure aperte, quindi lo script deve partire da un desktop pulito.

**Il problema di controllo ottimo continuo.** Il modello e' un punto materiale 2D in un campo di gravita' uniforme, senza aerodinamica. Stato  $x = [x, y, vx, vy, m]$ , controllo  $u = [Tx, Ty]$  (vettore spinta cartesiano). In forma dimensionale:

```
x_dot = vx  
y_dot = vy  
vx_dot = Tx / m
```



$vy\_dot = Ty / m - g$   
 $m\_dot = -|T| / c, \quad \text{con } |T| = \sqrt{Tx^2 + Ty^2}, \quad c = Isp * g0$

Il problema e':

$\min \int_0^{tf} |T(t)| dt \quad (\text{equivalente: } \max m(tf))$   
 s.t. dinamica di sopra  
 $x(0) = (1000, 3000) \text{ m}, \quad v(0) = (300, -200) \text{ m/s}, \quad m(0) = 2000 \text{ kg}$   
 $x(tf) = 0, y(tf) = 0, vx(tf) = 0, vy(tf) = 0 \quad (\text{pinpoint + soft})$   
 $Tmin \leq |T(t)| \leq Tmax$   
 $|x(t)| \leq \tan(\theta_{max}) * y(t) \quad (\text{glide slope})$   
 $tf = 38 \text{ s fissato}$

Non c'e' **nessun angolo di pitch** nel modello: essendo un punto materiale, l'assetto non e' uno stato e non esiste un vincolo di puntamento della spinta (del tipo  $T_y \geq |T| \cos(\theta_{pointing})$ ). L'unico vincolo angolare e' il glide slope, che e' un vincolo **geometrico sulla posizione**, non sull'assetto. Questo e' fedele alla traccia dell'homework, non una semplificazione arbitraria.

**Possibile domanda d'esame** – Perche'  $\min \int |T| dt$  e' equivalente a  $\max m(tf)$ ? *Risposta:* perche'  $m\_dot = -|T|/c$  con  $c$  costante, quindi integrando  $m(tf) = m0 - (1/c) * \int_0^{tf} |T| dt$ . Con  $m0$  fissata, massimizzare  $m(tf)$  e' identico a minimizzare l'integrale del modulo della spinta. Il vantaggio pratico e' che la massa e' gia' uno stato del sistema: il costo diventa di tipo **Mayer** (una funzione del solo stato finale), quindi nel NLP e' semplicemente  $-z(i\_massa\_ultimo\_nodo)$  – lineare nelle variabili di decisione, con gradiente costante. Non serve nessuna quadratura del costo.

## Problem data (Table 1, dimensional) (righe 14-29)

- Righe 15-26: i dati della Tabella 1 della traccia, in SI. Stato iniziale (1000, 3000) m, (300, -200) m/s,  $m0 = 2000 \text{ kg}$ ;  $Isp = 225 \text{ s}$ ;  $Tmin = 0$ ,  $Tmax = 70 \text{ kN}$ ;  $\theta_{mx} = 60 \text{ deg}$  (semiapertura del cono di glide slope).
- Riga 20 vs riga 22: **due gravita' diverse, e non e' un errore**.  $data.g = 9.81$  e' la gravita' locale che entra nelle equazioni del moto;  $data.g0 = 9.80665$  e' la gravita' standard, usata **solo** alla riga 23 per convertire l'impulso specifico in velocita' di efflusso,  $c = Isp * g0 = 2206.5 \text{ m/s}$ . La definizione di  $Isp$  e' per convenzione legata a  $g0$  standard, non alla gravita' del sito.
- Riga 24:  $Tmin = 0$ . E' il valore della traccia, e ha una conseguenza pesante sulla struttura del problema (vedi `trap_nonlcon`): con  $Tmin = 0$  l'insieme ammissibile della spinta e' il **disco**  $|T| \leq Tmax$ , che e' convesso. Se fosse  $Tmin > 0$  sarebbe una **corona circolare**, non convessa – il caso classico che motiva la lossless convexification (ticket T006, ancora aperto).
- Riga 28:  $tf\_nom = 38 \text{ s}$ , tempo di volo **fissato dalla traccia**, non ottimizzato.
- Riga 29:  $N = 50$  nodi di collocazione.

## Non-dimensionalisation (righe 31-35)

- Riga 32: chiama `nondim(data)` (definita a riga 109) e stampa le scale.
- Righe 33-35: `fprintf` diagnostici. Il numero che conta e'  $V\_c = V\_ref/c$ , perche' e' l'**unico parametro residuo** del problema adimensionale.

**Perche' e' l'unico parametro residuo.** Con  $L\_ref = y0$ ,  $a\_ref = g$ ,  $t\_ref = \sqrt{L\_ref/g}$ ,  $V\_ref = \sqrt{g*L\_ref}$ ,  $m\_ref = m0$ ,  $T\_ref = m0*g$ , le equazioni si riscrivono cosi' (tilde = adimensionale):

$dx~/dt~ = vx~ \quad (\text{coefficiente } 1 \text{ per costruzione})$   
 $dvx~/dt~ = (Tx/m) / a\_ref$   
 $= (Tx~ * T\_ref) / (m~ * m\_ref) / g$

```

      = Tx~ / m~                                perche' T_ref = m_ref * g
dvy~/dt~ = Ty~/m~ - 1                          perche' g/a_ref = 1
dm~/dt~  = -(|T|/c) * t_ref/m_ref
          = -|T~| * (m_ref*g*t_ref) / (c*m_ref)
          = -|T~| * (g*t_ref)/c
          = -|T~| * V_ref/c                    perche' g*t_ref = sqrt(g*L_ref) = V_ref

```

Cioe': tutte le equazioni hanno coefficienti unitari **tranne** quella della massa, che porta il gruppo  $V_c = V_{ref}/c$  (un numero di Tsiolkovsky). Questa e' esattamente la riga 14 di `ode_descent.m`:

```
dx = [ x(3); x(4); u(1)/x(5); u(2)/x(5) - 1; -Vc*Tmag ];
```

Coi numeri della traccia:  $t_{ref} = 17.49$  s,  $V_{ref} = 171.6$  m/s,  $T_{ref} = 19.62$  kN,  $V_c = 0.0778$ . Le condizioni iniziali adimensionali diventano  $x0~ = 1/3$ ,  $y0~ = 1$ ,  $vx0~ = 1.75$ ,  $vy0~ = -1.17$ ,  $m0~ = 1$ , e  $Tmax~ = 70000/19620 = 3.57$  – che si legge subito come **rapporto spinta/peso massimo 3.57**. Il tempo di volo nominale diventa  $tf~ = 2.17$ .

Il beneficio per il solver e' il **condizionamento**: in SI le variabili di decisione andrebbero da  $1e-1$  (velocita' finali) a  $7e4$  (spinta), cinque ordini di grandezza. `fmincon/sqp` usa differenze finite con passo relativo fisso e un test di ottimalita' su una norma mista di gradiente e moltiplicatori: con variabili tutte  $O(1)$  il passo di differenza finita e' significativo per tutte, e la tolleranza `OptimalityTolerance = 1e-5` ha lo stesso peso su ogni componente.

**Possibile domanda d'esame** – La `ConstraintTolerance = 1e-6` e' adimensionale: quanto vale in metri? *Risposta*: i defect di posizione sono normalizzati su  $L_{ref} = 3000$  m, quindi  $1e-6$  non-dim = 3 mm di errore di posizione per intervallo; i defect di velocita' sono su  $V_{ref} = 171.6$  m/s, quindi  $1e-6$  non-dim = 0.17 mm/s. La tolleranza e' fisicamente molto stretta. Attenzione a non confondere questo con l'errore di **trascrizione**, che e' molto piu' grande (vedi la sezione sullo studio di griglia): il defect misura quanto bene si soddisfa la dinamica *discreta*, non quanto la dinamica discreta approssima quella *vera*.

## Sensitivity sweep on flight time (righe 37-57)

- Riga 38: `tf_list = tf_nom * [0.95, 1.00, 1.05] -> [36.10, 38.00, 39.90]` s.
- Righe 41-57: tre solve **completamente indipendenti**. Non c'e' continuazione / warm start fra un `tf` e il successivo: ogni run riparte dalla guess lineare costruita dentro `solve_trapcol`. E' una scelta discutibile (la README lo ammette: la continuazione e' elencata fra i possibili rimedi allo stallo di convergenza), ma rende i tre risultati indipendenti fra loro.
- Riga 43: `tf_nd = tf_list(k) / ref.t` – il solve avviene tutto in adimensionale.
- Riga 45: `dim_sol` riporta la soluzione in SI **solo per stampa e plot**.
- Righe 48-54: diagnostiche post-solve (tempi di switch, coast, margine di glide slope, attivita' KKT).
- Riga 55-56: stampa `iters`, `firstorderopt`, `exitflag` di `fmincon`.

**Attenzione: questo NON e' un'ottimizzazione a tf libero.** `tf` non e' una variabile di decisione del NLP: e' un **parametro** che entra solo tramite `dt = tf/(N-1)`. Lo sweep e' un'analisi di sensitivita' a tre punti, non una ricerca dell'ottimo. Dai risultati riportati nella README (593.67 / 596.80 / 601.18 kg di carburante per `tf` crescente) il consumo **cresce monotonicamente** con `tf` nella finestra esplorata: significa che, dentro quella finestra, `tf = 38` s non e' il tempo di volo a minimo carburante, e che l'ottimo (se esiste) sta a sinistra del bordo inferiore, sotto i 36.10 s. Lo script non lo cerca, e non lo puo' dire.

L'interpretazione fisica del trend e' la gravity loss: piu' a lungo si sta in volo, piu' impulso serve semplicemente per sostenere il peso durante il coast. Ma il trade-off ha un altro lato che questa finestra non vede: accorciando troppo `tf` la decelerazione richiesta supererebbe `Tmax` e il problema diventerebbe infeasible. Lo sweep esplora solo il ramo monotono.

**Possibile domanda d'esame** – Se volessi rendere `tf` libero, cosa cambieresti? *Risposta*: due strade.

- (1) Aggiungere `tf` come  $7N+1$ -esima variabile di decisione, con `dt = tf/(N-1)` che compare dentro i defect: i defect diventano **nonlineari anche in tf** e la Jacobiana acquista una colonna densa. Il

costo `-m_N` resta lineare. (2) Tenere `tf` come parametro e mettere una ricerca 1D esterna (bisezione / golden section) sul valore ottimo, sfruttando il fatto che ogni solve interno resta piu' semplice. Nel piano del ticket T006 la seconda strada e' quella scelta per la variante SOCP, proprio perche' con la lossless convexification il problema interno e' convesso e la ricerca su `tf` conserva un certificato di ottimalita' globale per ogni `tf` provato.

---

## Plots / Export figures / Summary table (righe 59-85)

- Riga 60: chiama `plot_results` (definita a riga 389).
- Righe 63-78: export. `slugify` (riga 65) converte il `Name` della figura in un nome file; `theme(fig, 'light')` (riga 71) forza il tema chiaro anche se il desktop MATLAB e' in dark mode, col `catch` alla riga 73 che ricade su `Color = 'w'` per MATLAB pre-R2025a. Export in PNG a 200 dpi in `HM2_powered_descent/figures/`.
- Righe 81-85: la tabella di sintesi `tf | m_f | fuel`.

Nota di onesta': l'export salva **tutte** le figure attualmente aperte, non solo quelle create da `plot_results`. Funziona perche' la riga 12 ha fatto `close all`, ma e' fragile se lo script venisse eseguito per sezioni.

---

## Grid-convergence study (righe 87-101)

- Riga 90: `N_list = [25, 50, 100]`, tutti al `tf` nominale.
- Riga 95: risolve di nuovo il NLP (nota: il caso `N = 50` viene ricalcolato da zero, era gia' stato risolto nello sweep – spreco innocuo).
- Riga 97: `fwd_integrate_pwl` rigioca il controllo ottimo attraverso `ode45`.
- Riga 98: `err = max(node_err(...))` – massimo errore nodale su posizione e velocita', in adimensionale.
- Riga 99-100: stampa `N`, massa finale in kg, errore, wall time.

**Che cosa misura davvero questa metrica.** Il NLP e' (quasi) esattamente ammissibile per la dinamica **discreta**: i defect sono azzerati a  $1e-6$ . Ma la dinamica discreta e' un'approssimazione  $O(dt^2)$  di quella vera. Il replay con `ode45` (tolleranze  $1e-10$  /  $1e-12$ , quindi praticamente esatto) misura la distanza fra le due: e' l'**errore di trascrizione**, non una violazione di vincolo. La README riporta che l'errore cala di un fattore  $\sim 4$  a ogni raddoppio di `N` – cioe' il fattore atteso  $(dt/2)^2 / dt^2 = 1/4$ . E' la conferma numerica dell'ordine 2 della regola trapezoidale.

E' anche il motivo per cui, pur avendo  $y(tf) = 0$  imposto come vincolo di uguaglianza **esatto**, il replay open-loop atterra a 4.3 m dal pad a 0.11 m/s (numeri dalla README). Il vincolo e' soddisfatto – ma sulla traiettoria *discreta*.

**Possibile domanda d'esame** – Il tuo NLP dice  $y_N = 0$  esattamente, ma rigiocando il controllo con `ode45` atterri 4 metri fuori. La tua soluzione e' sbagliata? *Risposta:* no, e' *ammissibile per il problema che ho scritto*. Il NLP impone la dinamica **trapezoidale**, non quella continua; il gap di 4.3 m e' l'errore di quadratura  $O(dt^2)$  con `dt = 0.78 s`. Si chiude in tre modi: infittendo la griglia (`N = 100` lo riduce di  $\sim 4x$ ), passando a uno schema di ordine superiore (Hermite-Simpson,  $O(dt^4)$ ), oppure – che e' quello che si fa in volo – non usando mai il piano open-loop: si ri-risolve il problema a ogni ciclo di guida (MPC) o si mette un tracker in retroazione sopra la traiettoria nominale.

---

## nondim (righe 109-134)

- Riga 109: firma `[ref, dnd] = nondim(d)`. Chiamata una volta sola (riga 32), quindi ha un blocco `arguments` (righe 116-118): la validazione sta sui **boundary helper**, non sulle funzioni hot-loop – convenzione della repo.

- Righe 119-124: costruisce le scale. `ref.L = y0` (quota iniziale), `ref.g = g`, `ref.t = sqrt(L/g)`, `ref.V = sqrt(g*L)`, `ref.m = m0`, `ref.T = m0*g`. La scelta `T_ref = m_ref * g` e' quella che fa collassare i coefficienti nelle equazioni di velocita' (vedi derivazione sopra).
- Righe 125-133: applica le scale. `dnd.m0 == 1` per costruzione (riga 129, il commento lo dice). `dnd.Vc = ref.V / d.c` (riga 132) e' il gruppo residuo.
- Riga 133: `theta_mx` **non viene scalato** – e' gia' un angolo, adimensionale per natura.

## dim\_sol (righe 136-164)

- Righe 147-158: rimoltiplica ogni campo per la sua scala. `sol.Tmag` (riga 155) viene ricalcolata in SI da `Tx`, `Ty` invece di essere scalata: equivalente, perche' la norma e' omogenea di grado 1.
- Riga 158: `fuel = (m0~ - m_f~) * m_ref` – il carburante consumato.
- Righe 159-163: **i moltiplicatori di Lagrange restano adimensionali** e il commento lo dichiara esplicitamente. E' corretto non scalarli con una scala di stato (avrebbero dimensioni miste:  $d(\text{costo})/d(\text{vincolo})$ ), ma significa che i numeri stampati da `diagnostics` (riga 348, `max_gs_mult`) vanno letti solo in senso relativo (zero / non zero), mai come grandezze fisiche.

## solve\_trapcol (righe 166-267) – il cuore della trascrizione

E' la funzione che costruisce e risolve il NLP. Vale la pena smontarla pezzo per pezzo.

### Il vettore delle variabili di decisione (righe 181-183)

```
dt = tf / (N - 1);
nz = 7 * N;
idx = @(i) (i-1)*7 + (1:7);
```

- Riga 181: griglia **uniforme**, `dt = tf/(N-1)`. Con `tf = 38 s` e `N = 50`: `dt = 0.776 s` (`dt~ = 0.0443`).
- Riga 182: `nz = 7*N = 350` incognite.
- Riga 183: l'ordinamento. `idx(i)` restituisce le 7 posizioni del nodo `i`:

```
z = [ x_1 y_1 vx_1 vy_1 m_1 Tx_1 Ty_1 |
      x_2 y_2 vx_2 vy_2 m_2 Tx_2 Ty_2 | ... | ...nodo N ]
```

Cioe' **stacking node-major**: prima tutto il nodo 1, poi tutto il nodo 2, ecc. Il controllo e' discretizzato **ai nodi**, non agli intervalli – coerente con la trapezoidale, che implica un controllo **piecewise-linear**.

**Perche' proprio quest'ordine?** Perche' rende la Jacobiana dei vincoli **a banda**. Il defect dell'intervallo `k` coinvolge solo i nodi `k` e `k+1`, cioe' 14 componenti **contigue** di `z` (le posizioni `7(k-1)+1 ... 7(k+1)`). Con l'ordinamento alternativo – tutte le `x`, poi tutte le `y`, poi tutte le `vx`, ecc. – le stesse 14 componenti sarebbero sparse per tutto il vettore, e la Jacobiana perderebbe la struttura a banda. Nota pero' l'onesta' del caso: **qui il codice non sfrutta questa proprieta'**, perche' non dichiara nessuno sparsity pattern e non fornisce Jacobiane analitiche (vedi sotto). L'ordinamento e' comunque quello giusto, e sarebbe l'unico da usare se si passasse a **interior-point** con **ConstraintGradient** sparso.

Un altro vantaggio pratico: `reshape(z, 7, N)` (riga 280, dentro `trap_nonlcon`) recupera la matrice nodi-per-colonna **senza copie ne' permutazioni**, perche' MATLAB e' column-major. Con l'ordinamento alternativo servirebbe una trasposizione. (Attenzione a non confondere con l'unpack finale a riga 251, `Z = reshape(z_opt, 7, N)';` li' la trasposizione c'e', ed e' voluta, perche' serve la matrice nodi-per-riga da cui estrarre le colonne `sol.x`, `sol.y`, ecc. Ma e' fatta una volta sola a fine solve, non nel hot loop.)

## La guess iniziale (righe 185-197)

```
a = (i-1)/(N-1);
z0(s(1)) = (1-a) * d.x0;      % ... idem y, vx, vy
z0(s(5)) = d.m0 * (1 - 0.3*a);
z0(s(6)) = 0;
z0(s(7)) = d.m0;              % hover
```

- Righe 188-193: **interpolazione lineare** dallo stato iniziale a **zero** (perché  $(1-a)$  va da 1 a 0). Non è una propagazione: è la corda che unisce i due estremi. Siccome i target finali di  $x$ ,  $y$ ,  $vx$ ,  $vy$  sono tutti nulli, la corda arriva esattamente sulle condizioni finali.
- Riga 194: massa da  $m_0$  a  $0.7*m_0$ : si indovina il 30% di frazione di carburante. (La soluzione vera consuma ~597 kg su 2000, cioè 29.8% – la guess è sospettosamente centrata, verosimilmente tarata a posteriori.)
- Righe 195-196: controllo costante  $T = (0, m_0)$  adimensionale, cioè spinta verticale pari al **peso iniziale** (perché  $T_{ref} = m_0*g$ ): la guess di hover. Il commento “gravity = 1 nondim” è corretto.

**Proprietà notevole della guess** (che vale la pena saper difendere): questa guess soddisfa **tutti i vincoli tranne i defect**. Le BC lineari sono esatte per costruzione. Il vincolo di spinta è rispettato ( $|T| = 1 < T_{max} \approx 3.57$ ). E il glide slope pure: lungo la corda  $x/y = x_0/y_0$  è **costante** (entrambi sono scalati dallo stesso  $(1-a)$ ), quindi l'angolo resta  $\text{atan}(1000/3000) = 18.4 \text{ deg} < 60 \text{ deg}$  su tutti i nodi. Quindi l'unica infeasibility che `fmincon` deve sanare è quella dinamica. E' esattamente la situazione in cui SQP lavora bene.

## Bounds (righe 199-211)

- Riga 204:  $lb(y) = 0$  – niente volo sottoterra.
- Righe 205-206:  $1e-3 \leq m \leq m_0$ . Il lower bound è **essenziale**, non cosmetico: il RHS ha  $T_x/m$  e  $T_y/m$ , quindi  $m \rightarrow 0$  fa esplodere la dinamica. Il bound superiore  $m \leq m_0$  è ridondante fisicamente (la massa non può crescere: dal defect di massa,  $m_{k+1} - m_k = -0.5*dt*Vc*(|T_k| + |T_{k+1}|) \leq 0$  sempre) ma dà a SQP una scatola finita su cui lavorare.
- Righe 207-210:  $-T_{max} \leq T_x, T_y \leq T_{max}$ . E' una **scatola**, non un disco: da sola permetterebbe  $|T|$  fino a  $\sqrt{2}*T_{max}$ . E' il vincolo nonlineare in `trap_nonlcon` a imporre il disco. La scatola serve solo a dare bounds finiti a tutte le variabili.

Nota di onestà:  $lb(T_y) = -T_{max}$  **permette spinta verso il basso**. Fisicamente un lander col motore sotto non può spingere all'ingiu'. La traccia non impone un vincolo di puntamento, e all'ottimo la spinta verso il basso non viene mai usata (sprecerebbe carburante e violerebbe il glide slope), ma il modello lo consentirebbe. E' una libertà non fisica lasciata aperta.

## Condizioni al contorno (righe 213-225)

```
Aeq = sparse(9, nz);
Aeq(1, s1(1)) = 1; beq(1) = d.x0;      % ... 5 iniziali
Aeq(6, sN(1)) = 1; beq(6) = 0;         % ... 4 finali
```

- Righe 217-221: **5 BC iniziali** ( $x$ ,  $y$ ,  $vx$ ,  $vy$ ,  $m$  al nodo 1).
- Righe 222-225: **4 BC finali** ( $x$ ,  $y$ ,  $vx$ ,  $vy$  al nodo  $N$  a zero: pinpoint + soft landing). **La massa finale NON è vincolata** – è l'obiettivo.
- Sono messe in `Aeq / beq`, cioè fra le **equalità lineari**, non dentro `nonlcon`. E' la scelta giusta: `fmincon` le tratta esattamente (le usa nel QP senza linearizzarle) e soprattutto **non le differenzia numericamente**.

Piccola stonatura: `Aeq` viene costruita con `sparse` (riga 215) e poi passata a `fmincon` con `full(Aeq)` (riga 243), il che annulla il beneficio. Con una  $9 \times 350$  non cambia nulla in pratica, ma la `sparse` è inutile.

## Il costo (righe 227-229)

```
iN_m = (N-1)*7 + 5;
f_obj = @(z) -z(iN_m);
```

- Riga 228: `iN_m` e' l'indice della **massa all'ultimo nodo**. Con  $N = 50$ :  $49*7 + 5 = 348$ .
- Riga 229: minimizza `-m_N`, cioe' massimizza la massa finale.

Il costo e' **lineare** nelle variabili di decisione. Il suo gradiente e' il vettore `-e_348`, esatto e costante. Ne segue una conseguenza importante: nel NLP **tutta la nonlinearita' vive nei vincoli** (i defect e il modulo della spinta), non nel costo.

## Le opzioni di `fmincon` (righe 234-244)

```
opts = optimoptions('fmincon', ...
    'Algorithm', 'sqp', ...
    'Display', 'final', ...
    'MaxIterations', 1000, ...
    'MaxFunctionEvaluations', 1e6, ...
    'OptimalityTolerance', 1e-5, ...
    'ConstraintTolerance', 1e-6, ...
    'StepTolerance', 1e-10);
```

- `Algorithm = 'sqp'`: Sequential Quadratic Programming. A ogni iterazione costruisce un QP locale (Hessiano del Lagrangiano approssimato BFGS, vincoli linearizzati) e ne prende la soluzione come direzione di ricerca. Gestisce bene i punti di partenza **infeasible** – che e' esattamente il caso della guess (vedi sopra). Lo script non motiva la scelta.
- `ConstraintTolerance = 1e-6`, `OptimalityTolerance = 1e-5`, `StepTolerance = 1e-10`, `MaxIterations = 1000`. Anche `Display = 'final'` (riga 236) e' esposta esplicitamente: convenzione della repo, tutte le opzioni che determinano il comportamento del solver stanno scritte nello script, cosi' il run e' riproducibile.
- Righe 246-248: se `exitflag <= 0` emette una **warning**. **Questo warning scatta davvero**: la README documenta che i run a `tf = 36.10 s` e `tf = 38.00 s` finiscono col cap di `MaxIterations` (`exitflag = 0`) e first-order optimality bloccata a `1e-3-1e-4` non-dim. La feasibility resta invece sotto `1e-6`.

**Il buco nero: nessun gradiente analitico.** Le opzioni **non** contengono `SpecifyObjectiveGradient` ne' `SpecifyConstraintGradient`, e nessuno sparsity pattern viene dichiarato. Quindi `fmincon` calcola **tutte** le derivate per differenze finite in avanti. Il conto: 350 variabili -> 351 valutazioni di `trap_nonlcon` per ogni Jacobiana, ognuna delle quali fa un loop di 50 chiamate a `ode_descent`. E' il motivo per cui il singolo solve costa ~1-1.5 minuti. Sono tutte derivate che si scriverebbero a mano in poche righe:

- gradiente del costo: `-e_{iN_m}`, esatto e costante;
- Jacobiana dei defect:  $d(\text{zeta}_k)/d(\mathbf{z}_k) = -\mathbf{I} - (\text{dt}/2)*\text{df}/\text{dx}|_k$  e  $d(\text{zeta}_k)/d(\mathbf{z}_{k+1}) = +\mathbf{I} - (\text{dt}/2)*\text{df}/\text{dx}|_{k+1}$ , con  $\text{df}/\text{dx}$  un blocco  $5 \times 7$  analitico (le uniche derivate non banali sono  $d(T/m)/dm = -T/m^2$  e  $d|T|/dT = T/|T|$ );
- Jacobiana dei vincoli di percorso: quelli di glide slope sono **lineari**, quelli di spinta hanno gradiente  $\pm T/|T|$ .

Sarebbe una vittoria di uno-due ordini di grandezza in tempo di calcolo. Non e' stato fatto: e' un limite reale dello script, non una scelta.

---

## `trap_nonlcon` (righe 269-304) – defect e vincoli di percorso

```
Z = reshape(z, 7, N);
for i = 1:N
    f(:,i) = dyn_rhs(Z(:,i), d.Vc);
end
for k = 1:N-1
    defs(:,k) = Z(1:5,k+1) - Z(1:5,k) ...
        - 0.5*dt*(f(:,k) + f(:,k+1));
end
```

(indentazione e spaziatura ricompattate rispetto al sorgente, righe 280-292.)

- Riga 279: il commento dichiara che **non c'è** blocco **arguments**, per scelta: è una funzione hot-loop dentro **fmincon**. Convenzione della repo, non una dimenticanza.
- Riga 280: **reshape(z, 7, N)** – nodi in colonna. Gratis, per l'ordinamento node-major.
- Righe 283-286: valuta il RHS a **tutti** i nodi. Nota che **f** è 5xN mentre **Z** è 7xN: la dinamica ha 5 stati, le altre 2 righe sono il controllo.
- Righe 289-292: **i defect**.

## Derivazione della regola trapezoidale

Su un intervallo  $[t_k, t_{k+1}]$ , integrando esattamente l'equazione di stato:

$$x_{k+1} - x_k = \int_{t_k}^{t_{k+1}} f(x(t), u(t)) dt$$

L'integrale non è calcolabile (non conosciamo  $x(t)$ ). Lo si approssima con una quadratura. Se si approssima l'integrando **f** con la sua interpolante **lineare** fra i due estremi noti  $f_k = f(x_k, u_k)$  e  $f_{k+1}$ :

$$f(t) \sim f_k + (f_{k+1} - f_k) * (t - t_k)/dt$$

e si integra questa retta su  $[t_k, t_{k+1}]$ , si ottiene l'area del trapezio:

$$\int f dt \sim (dt/2) * (f_k + f_{k+1})$$

Da cui il **defect** (il “difetto”, cioè il residuo dell'equazione discretizzata):

$$zeta_k = x_{k+1} - x_k - (dt/2)*(f_k + f_{k+1}) = 0, \quad k = 1 \dots N-1$$

Sono **vincoli di uguaglianza** che il solver deve azzerare: quando  $zeta_k = 0$  per ogni  $k$ , la sequenza di nodi  $\{x_k\}$  è una traiettoria valida del sistema **discreto**. Se il solver li lasciasse diversi da zero, i nodi sarebbero semplicemente numeri che non stanno su nessuna traiettoria fisica. Da qui il nome: il “difetto” è quanto manca alla traiettoria per essere ammissibile.

Osservazioni chiave:

- **Il metodo è implicito.**  $f_{k+1}$  dipende da  $x_{k+1}$ , che è incognito. In uno schema di integrazione classico questo richiederebbe di risolvere un'equazione nonlineare a ogni passo. Nella collocazione diretta **non serve**: tutte le  $x_k$  sono variabili di decisione, e l'implicitezza è semplicemente parte del sistema di vincoli che il NLP risolve *simultaneamente*. È il vantaggio strutturale della trascrizione (a volte detta “simultaneous approach”, in opposizione al “sequential”/shooting).
- **Ordine di accuratezza.** L'errore della quadratura trapezoidale su un intervallo è  $-(dt^3/12) * f''(xi)$ : locale  $O(dt^3)$ , quindi globale  $O(dt^2)$ . Il metodo coincide col trapezio implicito (Crank-Nicolson): A-stabile, simmetrico, secondo ordine. Lo studio di griglia (righe 87-101) lo conferma empiricamente col fattore 4 per raddoppio di  $N$ .
- **Modello implicito del controllo.** Poiché l'integrando è approssimato linearmente, anche il controllo è implicitamente **piecewise-linear** fra i nodi. Per questo **fwd\_integrate\_pw1** (riga 351) rigioca il controllo con interpolazione **lineare** e non ZOH: rigiocarlo con uno ZOH misurerebbe la fedeltà della trascrizione sbagliata.

**Confronto con la trascrizione ZOH/RK4 del Task 2.** Le due sono concettualmente diverse:

|                        | trapezoidale (Task 1)                       | ZOH + RK4 (Task 2a)                      |
|------------------------|---------------------------------------------|------------------------------------------|
| controllo              | piecewise-linear ai nodi                    | costante a tratti sull'intervallo        |
| vincolo di uguaglianza | $x_{k+1} - x_k - (dt/2)(f_k + f_{k+1}) = 0$ | $x_{k+1} - RK4(x_k, u_k, dt) = 0$        |
| $x_{k+1}$ compare      | implicitamente (dentro $f_{k+1}$ )          | solo a sinistra                          |
| ordine                 | 2                                           | 4 (con substep, <code>rk4_zoh.m</code> ) |
| fedeltà al replay      | ~4.3 m (README)                             | ~1.4e-8 non-dim (README)                 |

La colonna di destra e' quello che fa `rk4_zoh.m`: propaga `ode_descent` con Runge-Kutta 4 tenendo `u` **costante** sull'intervallo. E' anche un vincolo di uguaglianza, ma di tipo **multiple shooting** (il difetto e' fra lo stato propagato e il nodo successivo), non di collocazione. Attenzione al terzo file della famiglia: `ode_descent_uacc.m` tiene costante l'**accelerazione**  $u = T/m$  invece del vettore spinta – e' la convenzione ZOH nativa della formulazione GFOLD log-massa (Task 2d), e non e' intercambiabile con `ode_descent.m`.

## I vincoli di percorso (righe 295-303)

```
Tmag      = sqrt(Z(6,:).^2 + Z(7,:).^2).';
g_thr_lo  = d.Tmin - Tmag;           % <= 0
g_thr_hi  = Tmag - d.Tmax;           % <= 0
tt        = tan(d.theta_mx);
g_gs_pos  = ( Z(1,:).' - tt*Z(2,:).');
g_gs_neg  = (-Z(1,:).' - tt*Z(2,:).');
c_ineq    = [g_thr_lo; g_thr_hi; g_gs_pos; g_gs_neg];
```

**Vincolo di spinta** (righe 296-298). Scritto come due disuguaglianze scalari per nodo:

$$T_{\min} - |T_k| \leq 0 \quad \text{e} \quad |T_k| - T_{\max} \leq 0$$

E' la trascrizione letterale di  $T_{\min} \leq |T| \leq T_{\max}$ . La domanda cruciale e': **e' convesso?**

- L'insieme  $\{T : |T| \leq T_{\max}\}$  e' un **disco**: convesso. Il vincolo  $|T| - T_{\max} \leq 0$  e' una funzione convessa  $\leq 0$ , quindi definisce un insieme convesso. OK.
- L'insieme  $\{T : |T| \geq T_{\min}\}$  con  $T_{\min} > 0$  e' il **complemento** di un disco: **non convesso**. Il vincolo  $T_{\min} - |T| \leq 0$  e'  $-|T| + T_{\min} \leq 0$ , cioe' una funzione **concava**  $\leq 0$ . La coppia definisce una **corona circolare**, che non e' convessa.
- **Ma qui  $T_{\min} = 0$**  (riga 24). Il vincolo inferiore diventa  $-|T| \leq 0$ , che e' sempre vero: **e' vacuo**. L'insieme ammissibile della spinta e' quindi il solo disco, ed e' convesso.

Quindi: il vincolo di spinta *come scritto* e' convesso, per il valore di  $T_{\min}$  della traccia. **Ma il NLP nel suo complesso resta non convesso**, perche' i defect contengono  $T_x/m$  e  $T_y/m$  (bilineari nelle incognite) e  $|T|$  dentro la dinamica di massa. `fmincon` restituisce un **minimo locale**, senza alcun certificato di ottimalita' globale. Questo e' il punto che motiva l'intera linea del Task 2 e il ticket T006 (lossless convexification): col cambio di variabili log-massa  $z = \ln m$ ,  $u = T/m$ , piu' lo slack  $\Gamma \geq |u|$ , l'intero problema diventa **un solo SOCP** convesso, risolto in meno di un secondo con certificato globale – e la convexification e' *lossless* nel senso che l'ottimo del problema rilassato e' dimostrabilmente anche ottimo del problema originale.

**Il vero problema numerico:  $|T|$  non e' differenziabile in  $T = 0$ .** La norma euclidea ha un cono (una punta) nell'origine; il suo gradiente  $T/|T|$  li' non esiste. E la soluzione ottima **ci passa in mezzo**: durante il coast arc la spinta e' esattamente zero. `fmincon`, che sta usando differenze finite in avanti con passo  $\sim 1.5e-8$ , in quel punto calcola una pendenza unilatera e sistematicamente di modulo  $\sim 1$ , indipendentemente dalla direzione – un gradiente **sbagliato**.  $|T|$  compare sia nel vincolo `g_thr_lo/hi` sia (via `ode_descent`) nel defect di massa, quindi la contaminazione e' doppia. **Questa e' la causa radice dello stallo di convergenza** documentato nella README (first-order optimality bloccata a  $1e-3$ - $1e-4$ , run che finiscono a `MaxIterations`). Non e' un bug: e' una patologia strutturale della formulazione min-fuel con norma nella dinamica.

**Vincolo di glide slope** (righe 299-301). Il vincolo fisico e'

$$|x| \leq \tan(\theta_{\max}) * y, \quad \theta_{\max} = 60 \text{ deg}$$

cioe': il veicolo deve stare dentro un cono di semiapertura 60 gradi **misurata dalla verticale**, con vertice sul pad. Il valore assoluto lo rende non differenziabile in  $x = 0$ , ma si spacca esattamente nella **coppia di disuguaglianze lineari**:

```
+x - tan(theta_max)*y <= 0
-x - tan(theta_max)*y <= 0
```

Le due insieme sono equivalenti a  $|x| \leq \tan(\theta_{\max}) * y$  – e sono **lineari**, quindi convesse e differenziabili ovunque. E' lo stesso trucco che si usa per linearizzare una norma-1. (Il fatto che  $y \geq 0$  sia garantito dal bound



alla riga 204 e' quello che rende l'equivalenza corretta: con  $y < 0$  le due disuguaglianze sarebbero infeasible, il che e' comunque il comportamento voluto.)

Nota di onesta': essendo **lineari**, questi vincoli dovrebbero stare negli slot A, b di **fmincon** (che alla riga 243 sono passati vuoti, []), non dentro **nonlcon**. Così' come sono, **fmincon** li differenzia numericamente insieme a tutto il resto -  $2N = 100$  righe di Jacobiana calcolate a differenze finite quando sarebbero note esattamente e costanti. Innocuo in accuratezza, sprecato in tempo.

**Un limite strutturale della collocazione, valido per entrambi i vincoli.** I vincoli di percorso sono imposti **solo ai nodi**. Fra un nodo e l'altro lo stato e' una spline (quadratica, per la trapezoidale) e nulla vieta che sconfini dal corridoio o che  $|T|$  superi  $T_{max}$ . La **gs\_margin** calcolata in **diagnostics** e' un margine **nodale**, non un certificato sull'intera traiettoria. Con  $N = 50$  e un margine di 1-2 gradi (README), il rischio e' basso ma non nullo.

**Ordine di impilamento di c\_ineq** (riga 303): [thr\_lo; thr\_hi; gs\_pos; gs\_neg], N elementi ciascuno,  $4N = 200$  in totale. **Quest'ordine e' un contratto:** **diagnostics** (righe 347-348) slica **lambda.ineqnonlin** esattamente su questi blocchi ( $N+1:2N$  per il bound superiore di spinta,  $2N+1:4N$  per il glide slope). Riordinare qui e non li' rompe silenziosamente le diagnostiche KKT.

**Possibile domanda d'esame** - Quanti vincoli e quante incognite ha il tuo NLP, e quanti gradi di liberta' restano? *Risposta:* con  $N = 50$ : **350** variabili ( $7 \times 50$ ); **245** uguaglianze nonlineari ( $5 \text{ defect} \times 49 \text{ intervalli}$ ) piu' **9** uguaglianze lineari ( $5 \text{ BC iniziali} + 4 \text{ finali}$ ) =  $254$  uguaglianze; **200** disuguaglianze ( $4 \times 50$ ), piu' i bound di scatola. Restano  $350 - 254 = 96$  gradi di liberta' sulla varieta' ammissibile, sui quali si minimizza  $-m_N$  rispettando le 200 disuguaglianze.

---

## dyn\_rhs (righe 306-315)

```
dx = ode_descent(s(1:5), s(6:7), Vc);
```

- Riga 314: e' solo un **adattatore**. **ode\_descent** vuole stato e controllo separati; il NLP li tiene impilati in un unico vettore di 7 elementi per nodo. **dyn\_rhs** spacca **s(1:5)** (stato) da **s(6:7)** (controllo).
- Riga 313: nessuna validazione **arguments**, di nuovo per scelta dichiarata (hot loop dentro **fmincon** e dentro **ode45**, riga 372).

Il fatto che l'unica dipendenza esterna dello script passi da qui e' importante: **ode\_descent.m** e' l'**unica** definizione della dinamica in tutto HM2 (la usano anche **rk4\_zoh.m** e i replay di Task 2), quindi non c'e' rischio che Task 1 e Task 2 stiano confrontando modelli diversi.

---

## diagnostics (righe 317-349)

### Tempi di switch (righe 333-340)

```
thr = 0.5 * d.Tmax;  
i_dn = find(Tm(1:end-1) >= thr & Tm(2:end) < thr, 1, 'first');  
i_up = find(Tm(1:end-1) < thr & Tm(2:end) >= thr, 1, 'last');
```

- Riga 333: soglia **euristica** a meta' della spinta massima.
- Riga 335: **i\_dn** = **primo** attraversamento in discesa -> fine della prima bruciata.
- Riga 336: **i\_up** = **ultimo** attraversamento in salita -> inizio della seconda.
- Riga 337: **cross(i)** interpola **linearmente**  $|T|$  fra i due nodi per stimare l'istante esatto. E' coerente col modello: la trapezoidale *dice* che il controllo e' piecewise-linear.
- Riga 340: **coast** =  $t_{sw2} - t_{sw1}$ .

**Perche' esiste un coast arc – la teoria dietro il numero.** Applicando il principio del minimo di Pontryagin al problema continuo, l'Hamiltoniano e'

$$H = \lambda_x v_x + \lambda_y v_y + \lambda_{v_x} (T_x/m) + \lambda_{v_y} (T_y/m - 1) - \lambda_m V_c |T|$$

Scrivendo  $T = |T| * \hat{u}$  con  $\hat{u}$  versore, la parte che dipende dal controllo diventa

$$H_u = |T| * [ (\lambda_v \cdot \hat{u})/m - V_c \lambda_m ]$$

Due conseguenze:

1. La **direzione** ottima allinea  $\hat{u}$  con  $-\lambda_v$  (il *primer vector* di Lawden): la spinta punta lungo il costato di velocita', cambiato di segno.
2. H e' **affine in |T|**: il coefficiente e' la **switching function**  $S = -|\lambda_v|/m - V_c \lambda_m$ . Una funzione affine su un intervallo  $[T_{min}, T_{max}]$  ha il minimo **su un estremo**. Quindi:

$$\begin{aligned} |T| &= T_{max} & \text{se } S < 0 \\ |T| &= T_{min} & \text{se } S > 0 \\ \text{arco singolare} && \text{se } S == 0 \text{ su un intervallo} \end{aligned}$$

Da qui la struttura **bang-off-bang** (max - coast - max), esattamente quella osservata numericamente:  $t_{sw1} \sim 14.0$  s,  $t_{sw2} \sim 33.1$  s per il caso nominale (README). **Lo script non calcola mai i costati** – sono le condizioni KKT del NLP a fare da controparte discreta al PMP, e i moltiplicatori di `fmincon` sono, a meno della scalatura per `dt`, i costati discretizzati. E' il ponte fra HM2 (metodi diretti) e HM1 (metodi indiretti), e vale la pena saperlo dire.

Nota di onesta' sui limiti del codice: `find(..., 1, 'first')` e `find(..., 1, 'last')` **assumono** proprio la struttura max-coast-max. Se il profilo avesse piu' archi, `i_dn` e `i_up` catturerebbero il coast piu' esterno, mascherando la struttura interna; se  $|T|$  non attraversasse mai  $0.5 * T_{max}$  con quel pattern, sarebbero **vuoti** e le diagnostiche degenererebbero silenziosamente (nessun guard, nessun controllo `isempty`).

## Margine di glide slope (righe 342-344)

- Riga 342: `ok = sol.y > 1` – maschera i nodi sotto 1 metro di quota. Il motivo e' scritto nel commento alle righe 325-326: al pad `atan2(|x|, y)` e' 0/0, e i residui numerici entro tolleranza (millimetri) producono angoli **arbitrari** fra 0 e 90 gradi. Senza la maschera il grafico e il margine sarebbero rumore.
- Riga 343-344: `gs_margin = theta_max_deg - max(theta_deg)`. Positivo = corridoio mai violato **sui nodi mascherati**.

## Attivita' KKT (righe 346-348)

```
lam = sol.lambda.ineqnonlin;
dg.n_thr_active = sum(lam(N+1:2*N) > 1e-6);
dg.max_gs_mult = max(lam(2*N+1:4*N));
```

- Riga 347: conta i nodi in cui il **moltiplicatore** del bound superiore di spinta e' strettamente positivo. La lettura e' la **complementarita'**: nel KKT vale  $\lambda_i * g_i(z) = 0$  con  $\lambda_i \geq 0$ , quindi  $\lambda_i > 0$  implica  $g_i = 0$ , cioe' vincolo **attivo**. Il codice deduce l'attivita' dal moltiplicatore, non dal residuo – che e' la lettura corretta (piu' robusta numericamente, e assume complementarita' stretta).
- Riga 348: il massimo moltiplicatore fra tutti quelli di glide slope. La README riporta che e' **numericamente zero**: il corridoio non e' mai attivo, la soluzione ci sta dentro **spontaneamente**. Cioe': la traiettoria min-fuel non e' vincolata dal glide slope, in questa geometria. Il solo vincolo di percorso attivo e'  $|T| \leq T_{max}$ .
- Il valore `1e-6` come soglia di positivita' e' arbitrario e non commentato.
- Ricordare (dalla riga 159) che questi `lambda` sono **adimensionali** e non vanno letti come grandezze fisiche.

## fwd\_integrate\_pwl (righe 351-377) e node\_err (righe 379-387)

```
u_fcn = @(tt) [  
    sol.Tx(k) + (sol.Tx(k+1)-sol.Tx(k))*(tt-t_k)/(t_kp-t_k);  
    sol.Ty(k) + (sol.Ty(k+1)-sol.Ty(k))*(tt-t_k)/(t_kp-t_k)];  
[~, Y] = ode45(rhs_t, [t_k, t_kp], X(k,:).', opts);  
X(k+1,:) = Y(end,:);
```

- Riga 365: parte dalle **condizioni iniziali esatte**, non dal nodo 1 del NLP (identiche comunque, essendo la BC un'uguaglianza).
- Riga 366: tolleranze `RelTol = 1e-10`, `AbsTol = 1e-12` – `ode45` e' qui il “verita' di riferimento”, quindi deve essere praticamente esatto (convenzione della repo per il lavoro shooting/indiretto).
- Righe 369-371: ricostruisce il controllo **piecewise-linear** fra i nodi `k` e `k+1`. Questa e' la scelta consistente con la trapezoidale (vedi sopra).
- Righe 373-374: **la propagazione riparte da  $X(k,:)$ , non dal nodo NLP  $sol.x(k)$**  (la condizione iniziale passata a `ode45` alla riga 373 e'  $X(k,:)$ , cioè il risultato del passo precedente, memorizzato alla riga 374). Cioe' e' un replay **open-loop** genuino: gli errori si **accumulano** intervallo dopo intervallo. Non e' un errore locale per intervallo – e' la domanda “se accendessi i motori esattamente cosi', dove finirei davvero?”. La risposta (README): a 4.3 m dal pad, a 0.11 m/s.
- Riga 386 (**node\_err**): norma euclidea della differenza su **posizione e velocita' soltanto** ( $X(:,1:4)$ ), la massa e' esclusa. Il codice non dice perche'; ragionevolmente perche' la massa e' l'obiettivo e non un errore di tracking, e perche' la sua scala e' diversa. Ma va detto: la metrica **non cattura** un eventuale errore sulla massa consumata.

---

## plot\_results (righe 389-460)

Quattro figure, con le opzioni utili da saper spiegare:

- Righe 404-418, **traiettoria**: `axis equal` (obbligatorio, altrimenti il cono di glide slope apparirebbe deformato), le due rette tratteggiate del corridoio costruite come `xx = tan(theta_max)*yy` (riga 407), il marker triangolare nero sul pad.
- Righe 421-430, **modulo della spinta**: `ylime(Tmax)` per rendere visibile che la soluzione **satura** il bound. E' il grafico che mostra il bang-off-bang.
- Righe 433-441, **massa**: monotona decrescente con un plateau durante il coast (dove `m_dot = -Vc*|T| = 0`).
- Righe 446-459, **angolo di glide slope**: stessa maschera `y > 1` di **diagnostics** (righe 444-445 lo commentano), e alla riga 456 un `ylim` forzato per tenere la linea di `theta_max` dentro l'inquadratura anche quando la curva le sta molto sotto – altrimenti la scala automatica la butterebbe fuori e non si vedrebbe il margine.

---

## Possibili domande d'esame

**D: Che cos'e' esattamente un “defect constraint” e perche' e' un vincolo di uguaglianza?** R: E' il residuo dell'equazione di stato discretizzata:  $zeta_k = x_{k+1} - x_k - (dt/2)*(f_k + f_{k+1})$ . Nella collocazione diretta i valori dello stato ai nodi sono **variabili di decisione indipendenti**: nulla, di per se', le lega alla dinamica. Il defect e' proprio il vincolo che le lega. Se  $zeta_k = 0$  per ogni  $k$ , i nodi giacciono su una traiettoria del sistema discreto; se non lo fosse, sarebbero numeri senza significato fisico. Deve essere un'**uguaglianza** (non una disuguaglianza) perche' la dinamica non e' negoziabile: non esiste un “quasi soddisfatta in un verso”. Sono 5  $x$   $(N-1) = 245$  uguaglianze nonlineari con  $N = 50$ .

**D: Perche' collocazione diretta e non shooting (come in HM1)?** R: Nel single shooting le uniche incognite sono i parametri iniziali, e lo stato si ottiene propagando: il problema e' piccolo ma **estremamente mal condizionato** (la sensitivita' dello stato finale rispetto alle condizioni iniziali cresce esponenzialmente col

tempo di volo, ed e' la ragione per cui HM1 ha bisogno di continuazione). Nella collocazione diretta lo stato e' **tutto** incognito, quindi il problema e' grande (350 variabili) ma **sparso e ben condizionato**: ogni vincolo tocca solo due nodi adiacenti, quindi un errore locale non si propaga lungo tutta la traiettoria. In piu' i vincoli di percorso (glide slope, bound di spinta) si impongono **direttamente** ai nodi, mentre in un metodo indiretto richiederebbero moltiplicatori di percorso e una struttura di archi da indovinare a priori. Il prezzo e' l'accuratezza:  $O(dt^2)$  invece della precisione dell'integratore.

**D: Il tuo problema di ottimizzazione e' convesso?** R: **No**, anche se molti dei suoi pezzi lo sono. Il costo e' lineare ( $-m_N$ ); il glide slope e' scritto come coppia di disuguaglianze **lineari**; il bound di spinta e'  $|T| \leq T_{max}$ , che con  $T_{min} = 0$  e' un **disco** convesso (con  $T_{min} > 0$  sarebbe una corona, non convessa – ed e' il caso interessante). La non convessita' viene tutta dai **defect**, che contengono  $T_x/m$  e  $T_y/m$  (bilineari) e  $|T|$  nell'equazione di massa. Quindi `fmincon/sqp` restituisce un **minimo locale** senza certificato globale, e la risposta dipende (in linea di principio) dalla guess. E' proprio questo che motiva la variante GFOLD del Task 2: col cambio di variabili log-massa  $z = \ln m$ ,  $u = T/m$ , la dinamica diventa **esattamente LTI** e con lo slack  $\Gamma \geq |u|$  l'intero problema collassa in un **solo SOCP** convesso, con ottimo globale certificato.

**D: Il vincolo  $T_{min} \leq |T| \leq T_{max}$  non e' non convesso? Come lo hai gestito?** R: Nel caso generale si', e' una **corona circolare** (l'insieme  $\{|T| \geq T_{min}\}$  e' il complemento di un disco). Nel mio problema pero' la traccia da'  $T_{min} = 0$ , quindi il vincolo inferiore  $-|T| \leq 0$  e' **vacuo** e l'insieme si riduce al disco convesso. Il codice lo scrive comunque in forma generica (riga 297), quindi con  $T_{min} > 0$  girerebbe ancora – ma `fmincon` lo tratterebbe come un vincolo nonlineare qualsiasi, cioe' cercherebbe un minimo locale su un insieme non convesso, senza garanzie. La cura corretta e' la **lossless convexification**: si rilassa a  $T_{min} \leq \Gamma \leq T_{max}$  con  $|T| \leq \Gamma$ , e si dimostra che all'ottimo il rilassamento e' **attivo** ( $\Gamma = |T|$ ), quindi la soluzione del problema convesso e' anche soluzione dell'originale.

**D: `fmincon` ti dice `exitflag = 0` su due run su tre. La tua soluzione vale qualcosa?** R: `exitflag = 0` significa **MaxIterations raggiunto**, non “fallito”. La **feasibility** e' comunque sotto  $1e-6$  (i vincoli sono soddisfatti); a stallare e' la **first-order optimality**, ferma a  $1e-3$ – $1e-4$  non-dim. Quindi ho una traiettoria valida, ma non ho la prova che sia stazionaria. La causa radice e' identificata: la norma  $|T| = \sqrt{T_x^2 + T_y^2}$  **non e' differenziabile in  $T = 0$** , e la soluzione ottima ci passa attraverso per tutto il coast arc. Con gradienti a differenze finite (che e' quello che `fmincon` sta usando, perche' lo script non fornisce Jacobiane analitiche) il gradiente li' e' spazzatura. Rimedi, in ordine di eleganza: (1) lo slack  $\Gamma \geq |T|$  della lossless convexification, che **toglie la norma dalla dinamica**; (2) gradienti analitici con norma regolarizzata  $\sqrt{|T|^2 + \epsilon}$ ; (3) continuazione, warm-startando ogni run dello sweep dalla soluzione nominale.

**D: Perche' il vettore delle incognite e' impilato nodo per nodo e non variabile per variabile?** R: Perche' con lo stacking node-major (`[stato_1; u_1; stato_2; u_2; ...]`) il defect dell'intervallo  $k$  tocca **14 componenti contigue** di  $z$  – quelle dei nodi  $k$  e  $k+1$ . La Jacobiana dei vincoli risulta a **banda** (block-bidiagonale), ed e' esattamente la struttura che un solver sparso (interior-point con `ConstraintGradient`, o IPOPT) sfrutta. Con l'ordinamento alternativo (tutte le  $x$ , poi tutte le  $y$ , ...) le stesse 14 componenti sarebbero disperse per tutto il vettore. In piu', essendo MATLAB column-major, `reshape(z, 7, N)` recupera i nodi in colonna **senza copie**. Onesta': lo script **non** sfrutta questa sparsita' – non dichiara sparsity pattern ne' Jacobiane analitiche, e `fmincon` calcola tutto per differenze finite (351 valutazioni di `nonlcon` per Jacobiana). E' il principale limite di performance del Task 1.

**D: Come costruisci la guess iniziale, e perche' funziona?** R: **Interpolazione lineare** (una corda, non una propagazione) fra lo stato iniziale e lo stato finale per  $x$ ,  $y$ ,  $v_x$ ,  $v_y$ ; massa lineare da  $m_0$  a  $0.7*m_0$  (30% di frazione di carburante indovinata); controllo costante di **hover**,  $T = (0, m_0*g)$ , cioe'  $(0, 1)$  in adimensionale. La proprieta' che la rende buona e' che **soddisfa gia' tutti i vincoli tranne i defect**: le BC per costruzione, il bound di spinta ( $|T| = 1 < 3.57$ ), e anche il glide slope, perche' lungo la corda il rapporto  $x/y$  resta **costante** a  $\text{atan}(1000/3000) = 18.4 \text{ deg}$ , ben dentro il cono di 60 gradi. Quindi l'unica infeasibility che SQP deve sanare e' quella dinamica – che e' proprio il caso in cui SQP e' forte (gestisce bene punti di partenza infeasible).

**D: Cosa ti aspetteresti di guadagnare passando a Hermite-Simpson?** R: Hermite-Simpson e'  $O(dt^4)$  invece di  $O(dt^2)$ , a costo di aggiungere un punto di collocazione a meta' intervallo (lo stato al midpoint e' ricostruito con un'interpolante cubica di Hermite, il controllo al midpoint diventa una nuova variabile). A parita' di  $N$  l'errore di replay crollerebbe di ordini di grandezza; equivalentemente, a parita' di accuratezza servirebbero molti meno nodi, quindi un NLP piu' piccolo. Nel mio caso specifico pero' il collo di bottiglia **non e' l'ordine**

**dello schema:** e' la non differenziabilita' di  $|T|$  in  $T = 0$  combinata con le differenze finite, che nessun aumento di ordine risolve. Prima si sistema il gradiente, poi ha senso alzare l'ordine.

# HM2\_\_powered\_\_descent/main\_\_task2.m

## Ruolo del file nel progetto

Questo e' lo script piu' lungo e piu' denso della repo (1420 righe). Implementa il **Task 2 (opzionale)** dell'Homework 2: la traccia chiede una trascrizione alternativa del problema di powered descent basata su **Zero-Order Hold (ZOH)**, seguendo l'Appendice A del PDF, e di validarla ri-propagando i controlli ottimizzati con un integratore ad alta fedelta'.

Lo script non si limita a una trascrizione: ne implementa **quattro**, tutte sullo stesso problema fisico (stessi dati, stessa griglia, stesso costo), piu' la baseline trapezoidale del Task 1 come termine di paragone. Le quattro varianti sono, in ordine di "quanta struttura convessa espongono al solver":

- (a) **ZOH nonlineare + RK4** (`solve_zoh`, righe 626-650): la dinamica resta nonlineare, il difetto di collocazione e'  $x_{k+1} - \text{RK4}(x_k, u_k, dt) = 0$ . E' un multiple shooting. Risolto con `fmincon`/SQP.
- (b) **ZOH LTV + SCvx con fmincon** (`solve_scvx`, righe 429-537): si linearizza la dinamica attorno a una traiettoria di riferimento, si costruiscono le matrici discrete  $\bar{A}_k$ ,  $\bar{B}_k$ ,  $\bar{c}_k$  con l'ODE aumentata dell'Appendice A, si risolve il sottoproblema LTV, si aggiorna il riferimento, si itera.
- (c) **ZOH LTV + SCvx con YALMIP/ECOS** (`solve_scvx_yalmip`, righe 1042-1121): stesso loop esterno di (b), ma il sottoproblema interno e' scritto come **SOCP** (Second-Order Cone Program) e dato a un solver conico invece che a un NLP generico.
- (d) **GFOLD log-mass** (`solve_gfold_scvx`, righe 1198-1283): cambio di variabili  $z = \ln(m)$ ,  $u = T/m$ , slack  $\sigma \geq |u|$ . La dinamica diventa **esattamente LTI**, quindi la ZOH dell'Appendice A si riduce a un singolo esponenziale di matrice; l'unica cosa che resta da linearizzare e' il bound superiore di spinta.

Il file dipende da quattro mattoni esterni condivisi con `main_task1.m` e con la test suite: `ode_descent.m` (RHS nonlineare, controllo = spinta), `ode_descent_uacc.m` (RHS con controllo = accelerazione, convenzione GFOLD), `rk4_zoh.m` (propagatore RK4 a passo fisso) e `lti_zoh.m` (ZOH esatta via `expm` per il sistema log-mass). Tutto il resto – Jacobiane, ODE aumentata, assemblaggio degli NLP, loop SCvx, replay di validazione, plotting – vive in **local functions** dentro questo file (dalla riga 203 in poi).

Il flusso e' lineare: dati SI -> non-dimensionalizzazione -> risolvi le 5 trascrizioni -> ri-propaga i controlli con `ode45` -> misura l'errore ai nodi e la dispersione al touchdown -> tabelle e figure.

---

## Il problema di partenza e perche' NON e' convesso

La dinamica continua (non-dim, gravita' = 1, stato  $x = [x; y; vx; vy; m]$ , controllo  $T = [Tx; Ty]$ ) e' quella di `ode_descent.m`:

```
x_dot = vx
y_dot = vy
vx_dot = Tx / m
vy_dot = Ty / m - 1
m_dot = -Vc * |T|,      Vc = V_ref / c
```

con  $c = I_{sp} * g_0$  velocità efficace di scarico. Costo: massimizzare  $m(tf)$  (equivalente a minimizzare il carburante, perché  $m_0$  è fissa).

Ci sono **due** sorgenti di non convessità, ed è importante non confonderle:

1. **La dinamica è bilineare / nonlineare.** I termini  $T_x/m$  e  $T_y/m$  sono un prodotto fra controllo e (inverso della) massa: bilineari. Il termine  $-V_c * |T|$  è una norma dentro un vincolo di **uguaglianza** (l'equazione di stato). Anche se  $|T|$  è una funzione convessa, un vincolo  $m_{dot} = -V_c * |T|$  con la funzione convessa **al secondo membro di un'uguaglianza** definisce un insieme non convesso. Quindi il set di traiettorie ammissibili non è convesso.
2. **Il vincolo di spinta minima.** In generale il motore ha  $T_{min} \leq |T| \leq T_{max}$ : è un **guscio anulare** (in 2D una corona circolare), il classico insieme non convesso. In *questo* homework  $T_{min} = 0$  (riga 32), quindi l'insieme è un disco pieno, già convesso – ma la macchinaria per gestire  $T_{min} > 0$  (lossless convexification) va comunque capita, perché è quello che la traccia e la letteratura GFOLD hanno in mente e perché lo slack  $\sigma$  serve comunque, come vedremo, per un'altra ragione.

Nota bene: i **vincoli di percorso** dell'homework sono invece già convessi. Il glide-slope  $|x| \leq \tan(\theta_{max}) * y$  è una coppia di semipiani (righe 817-818), il bound  $|T| \leq T_{max}$  è un cono di Lorentz, i box su massa e quota sono lineari. Quindi tutta la non convessità è concentrata **nella dinamica** (più  $T_{min}$  se fosse  $> 0$ ). Questo è esattamente il motivo per cui la ricetta SCvx funziona bene qui: basta convessificare la dinamica.

## Teoria: la trascrizione ZOH per sistemi LTV (Appendice A)

Sia il sistema **lineare tempo-variante**

$$\dot{x}(t) = A(t) x(t) + B(t) u(t) + c(t)$$

su una griglia uniforme  $t_k$ , con passo  $dt$ , e controllo tenuto costante sull'intervallo (**zero-order hold**):  $u(t) = u_k$  per  $t$  in  $[t_k, t_{k+1})$ .

La formula di variazione delle costanti dà

$$x(t) = \Phi(t, t_k) x_k + \int_{t_k}^t \Phi(t, s) [B(s) u_k + c(s)] ds$$

dove  $\Phi(t, s)$  è la matrice di transizione di stato del sistema omogeneo. Valutando in  $t = t_{k+1}$  si ottiene la ricorsione discreta

$$x_{k+1} = Abar_k x_k + Bbar_k u_k + cbar_k$$

$$Abar_k = \Phi(t_{k+1}, t_k)$$

$$Bbar_k = \int_{t_k}^{t_{k+1}} \Phi(t_{k+1}, s) B(s) ds$$

$$cbar_k = \int_{t_k}^{t_{k+1}} \Phi(t_{k+1}, s) c(s) ds$$

Il punto è che  $u_k$  esce dall'integrale **solo perché è costante**: è tutta qui la ragion d'essere della ZOH. Con un first-order hold (controllo lineare a tratti) servirebbe un secondo integrale ausiliario.

**Come si calcolano gli integrali.** L'Appendice A usa l'identità di semigrupp

$$\Phi(t_{k+1}, s) = \Phi(t_{k+1}, t_k) * \Phi(t_k, s)$$

che permette di portare fuori dall'integrale il fattore fisso  $\Phi(t_{k+1}, t_k)$ . Definendo la transizione **all'indietro**  $\Psi(s) := \Phi(t_k, s) = \Phi(s, t_k)^{-1}$  e i due integrali (riferiti a  $t_k$ )

$$\beta(t) = \int_{t_k}^t \Psi(s) B(s) ds$$

$$\gamma(t) = \int_{t_k}^t \Psi(s) c(s) ds$$

si ha

$$Bbar_k = \Phi(t_{k+1}, t_k) * \beta(t_{k+1})$$

$$cbar_k = \Phi(t_{k+1}, t_k) * \gamma(t_{k+1})$$

e le quantita' (Phi, Psi, beta, gamma) obbediscono a un sistema di **ODE lineari** che si integrano tutte insieme su un intervallo:

```
Phi_dot   = A Phi,           Phi(t_k)   = I
Psi_dot   = -Psi A,          Psi(t_k)   = I
beta_dot  = Psi B,           beta(t_k)  = 0
gamma_dot = Psi c,           gamma(t_k) = 0
```

L'equazione per Psi si ricava derivando l'identita'  $\Phi^{-1} \Phi = I$ :

```
d(Phi^{-1})/dt = -Phi^{-1} * Phi_dot * Phi^{-1} = -Phi^{-1} A Phi Phi^{-1}
               = -Psi A
```

Le equazioni per beta e gamma sono **quadrature pure**: il loro integrando dipende solo da s, non da beta/gamma stessi. Nessun feedback – ecco perche' nel codice (riga 292-293) si dice esplicitamente che le loro derivate non dipendono da loro.

Questo e' letteralmente cio' che fanno `ltv_aug_rhs` (righe 279-303) e `compute_ltv_zoh` (righe 305-339).

**Possibile domanda d'esame** – Perche' portarsi dietro  $\Psi = \Phi^{-1}$  come stato aumentato, invece di calcolare  $\Phi(t_{k+1}, s)$  direttamente? *Risposta*: perche'  $\Phi(t_{k+1}, s)$  ha l'**estremo finale fisso e quello iniziale variabile**: integrandolo in avanti da  $t_k$  non lo si ottiene mai in un colpo solo. Riferendo tutto a  $t_k$  con l'identita' di semigrupp, tutte e quattro le quantita' partono da condizioni iniziali note in  $t_k$  e si integrano in **una sola passata in avanti**; si paga un prodotto matriciale finale (righe 336-337) per ricostruire  $\bar{B}_k, \bar{c}_k$ . E' il prezzo della convenienza numerica.

## Teoria: Successive Convexification (SCvx)

### Linearizzazione

Il problema di partenza e'  $\dot{x} = f(x, u)$  con  $f$  nonlineare. SCvx lo affronta cosi': si prende una **traiettoria di riferimento**  $(x_{ref}(t), u_{ref}(t))$  e si espande  $f$  al primo ordine attorno ad essa:

```
f(x, u) ≈ f(x_ref, u_ref) + A (x - x_ref) + B (u - u_ref)
```

con

```
A(t) = df/dx |_ref      (5x5)
B(t) = df/du |_ref      (5x2)
```

Raccogliendo, si ottiene la forma LTV in **variabili assolute** (non in deviazioni):

```
x_dot ≈ A(t) x + B(t) u + c(t)
```

```
c(t) = f(x_ref, u_ref) - A x_ref - B u_ref      <- termine affine / residuo
```

Il termine  $c(t)$  e' il pezzo che spesso si dimentica: e' quello che fa passare la retta tangente **per il punto di riferimento**. Se lo toglie, la linearizzazione passa per l'origine e il modello e' completamente sbagliato. Nel codice e' la riga 296 ( $c_{off} = f_{val} - A_{jac}x_{ref} - B_{jac}u_k$ ).

Per il nostro  $f$ , le Jacobiane si calcolano a mano (funzione `jacobians`, righe 255-277):

```
A = [ 0 0 1 0      0
      0 0 0 1      0
      0 0 0 0  -Tx/m^2
      0 0 0 0  -Ty/m^2
      0 0 0 0      0  ]
```

```
B = [ 0      0
      0      0
```



$$\begin{bmatrix} 1/m & 0 \\ 0 & 1/m \\ -Vc \cdot T_x / |T| & -Vc \cdot T_y / |T| \end{bmatrix}$$

Le righe 3 e 4 di A vengono da  $d/dm (T_x/m) = -T_x/m^2$ . La riga 5 di A e' nulla perche'  $m\_dot = -Vc \cdot |T|$  non dipende dallo stato. La riga 5 di B e'  $d/dT (-Vc \cdot |T|) = -Vc \cdot T/|T|$ , cioe'  $-Vc$  per il **versore** di spinta: e' singolare in  $T = 0$  (il gradiente della norma non esiste nell'origine). Il codice regolarizza con  $\sqrt{T_x^2 + T_y^2 + 1e-6}$  (riga 266). Torneremo su questo hack.

Con la dinamica linearizzata, tutti i vincoli del problema diventano convessi (uguaglianze lineari + semipiani + un cono per nodo), quindi il **sottoproblema e' un SOCP**: ha un ottimo globale unico raggiungibile in tempo polinomiale.

## Il ciclo

1. parti da un riferimento ( $x\_ref, u\_ref$ )
2. calcola A, B, c lungo il riferimento; discretizza (ZOH) -> Abar, Bbar, cbar
3. risolvi il sottoproblema convesso dentro una trust region
4. la soluzione diventa il nuovo riferimento
5. torna al punto 2 finche'  $\|x^{j+1} - x^j\| < tol$

Il fatto che il sottoproblema sia risolto **globalmente** non significa che SCvx trovi l'ottimo globale del problema originale: converge a un punto stazionario (KKT) del problema nonlineare. La convessita' serve a rendere ogni passo affidabile e veloce, non a certificare l'ottimalita' globale.

## La trust region: perche' serve e com'e' implementata qui

La linearizzazione e' accurata **solo vicino al riferimento**. Se il sottoproblema convesso e' lasciato libero, succede il fenomeno dell'**artificial unboundedness**: il modello lineare "promette" un guadagno che la dinamica vera non consegna. Nel nostro caso il colpevole tipico e' proprio la riga della massa: con  $|T|$  piccolo,  $df5/dT$  e' quasi singolare, il modello lineare crede di poter guadagnare massa e il solver ci si butta dentro (nel report questo e' descritto come "mass creation",  $m\_dot > 0$ ).

Nel codice la trust region e':

- **Hard**, non una penalita' soft. E' un box in norma infinito, per-variabile e per-nodo, **intersecato con i box bounds** gia' esistenti (apply\_trust, righe 571-600):  $lb = \max(lb, ref - raggio)$ ,  $ub = \min(ub, ref + raggio)$ .
- **Anisotropia**: raggi diversi per posizione, velocita', massa, spinta (riga 49: pos 0.17, vel 0.6, mass 0.1, thrust 1.0 in unita' non-dim - circa 510 m, 103 m/s, 200 kg, 19620 N).
- **Adattiva**, scalata da un fattore rho in [1e-3, 1] (righe 464-468).

L'aggiornamento e' il **ratio test** classico (righe 487-530). Definendo:

```
J_pred = m_f(candidato secondo il modello LTV) - m_f(riferimento)
J_act   = m_f(candidato ri-propagato con ode45) - m_f(riferimento)
eta     = J_act / J_pred
```

cioe' **guadagno reale / guadagno predetto**, si ha:

```
eta < eta_l = 0.25    -> il modello lineare mente: RIFIUTA, rho <- rho/2
0.25 <= eta < eta_h=0.7 -> accetta, rho invariato
eta >= eta_h = 0.7    -> il modello e' ottimo: accetta, rho <- min(1, 2*rho)
```

Se rho scende sotto  $\rho\_min = 1e-3$ , il loop si ferma ("trust region collapsed", riga 527). Notare che  $\rho\_max = 1.0$ : la trust region non puo' mai crescere oltre i raggi base.

**Un dettaglio importante e specifico di questo codice**: il "guadagno reale" e' misurato ri-propagando i controlli candidati attraverso la **dinamica nonlineare vera** con ode45 (riga 489). Non e' un semplice ricalcolo del costo: e' un test di fedelta' dinamica. Questo e' un punto forte, ed e' anche il motivo per cui il loop sopravvive senza virtual control (vedi sotto).

## Virtual control / slack: perche' servono e perche' QUI NON CI SONO

Il secondo modo di fallire di SCvx e' l'**artificial infeasibility**: il sottoproblema linearizzato puo' risultare **infeasible anche se il problema nonlineare e' feasible**. Succede quando la dinamica linearizzata, combinata con le condizioni al contorno (qui: stato iniziale completamente assegnato e posizione/velocita' finali nulle), non ammette nessuna soluzione: il modello lineare semplicemente non riesce a "chiudere" il problema ai due estremi.

La cura canonica e' rilassare i vincoli che possono diventare vuoti:

- **virtual control** `nu_k`: si aggiunge un termine libero alla dinamica discreta

$$x_{k+1} = A_{bar,k} x_k + B_{bar,k} u_k + c_{bar,k} + nu_k$$

Con `nu` libero, la dinamica e' **sempre** soddisfacibile: il sottoproblema non puo' piu' essere infeasible per colpa della dinamica.

- **buffer / slack** `w`  $\geq 0$  sui vincoli di percorso linearizzati:

$$g(x_{ref}) + dg/dx (x - x_{ref}) + w \leq 0$$

- entrambi vengono **penalizzati in costo** con una penalita' esatta L1:

$$J_{nu} = \lambda_{nu} * ||nu||_1 + \lambda_w * ||w||_1$$

I pesi vengono alzati finche' `nu`  $\rightarrow 0$  e `w`  $\rightarrow 0$  alla convergenza. La penalita' L1 e' *esatta*: per `lambda` abbastanza grande, se esiste una soluzione con `nu` = 0 il minimo la trova, senza dover mandare `lambda`  $\rightarrow$  infinito.

Nel codice di `main_task2.m` **NON ci sono ne' virtual control ne' buffer**. E' la deviazione piu' importante rispetto alla ricetta canonica. Il loop se la cava per due motivi: (i) il warm start dalla soluzione trapezoidale mette il riferimento gia' vicino all'ottimo; (ii) quando il sottoproblema *diventa* infeasible, il solver interno restituisce comunque il suo iterato meno infeasible, e il ratio test con `ode45` lo vaglia prima di accettarlo. Il codice si limita a stampare un warning (riga 410 per `fmincon`, riga 1028 per `YALMIP`). E' un comportamento fragile: un run cold-started, o una trust region piu' larga, potrebbero rompersi.

**Possibile domanda d'esame** – Trust region e virtual control curano due patologie diverse. Quali?

*Risposta:* la **trust region** cura l'*artificial unboundedness*: impedisce al sottoproblema linearizzato di allontanarsi dal riferimento fin dove il modello lineare promette guadagni che la dinamica vera non conferma. I **virtual control** curano l'*artificial infeasibility*: rendono il sottoproblema sempre risolvibile, aggiungendo alla dinamica un termine libero penalizzato in costo, che si annulla alla convergenza. Sono complementari: la prima restringe, i secondi rilassano.

---

## Teoria: lossless convexification e il cambio di variabili GFOLD

Questo e' il cuore teorico dell'homework, ed e' la variante (d).

### Il cambio di variabili

Seguendo Acikmese & Ploen (2007) e Acikmese & Blackmore (GFOLD – *Guidance for Fuel-Optimal Large Diverts*), si introducono:

```
z      = ln(m)           <- log-massa (nuovo stato)
u      = T / m           <- accelerazione di spinta (nuovo controllo)
sigma >= |u|             <- slack scalare (nuovo controllo ausiliario)
```

Sostituiamo nella dinamica:

- `vx_dot` = `Tx/m` = `ux`  $\rightarrow$  **lineare** (il prodotto bilineare e' sparito per costruzione)
- `vy_dot` = `Ty/m` - 1 = `uy` - 1  $\rightarrow$  **lineare** (piu' un termine affine costante)
- massa: da `m_dot` = `-Vc*|T|` e `z` = `ln(m)`,

$$\mathbf{z\_dot} = \mathbf{m\_dot} / m = -V_c * |T| / m = -V_c * |u|$$

e sostituendo  $|u|$  con lo slack  $\sigma$ :

$$\mathbf{z\_dot} = -V_c * \sigma \quad \rightarrow \text{**lineare**}$$

Risultato: il sistema in  $\mathbf{x} = [x; y; vx; vy; z]$  con controllo  $\mathbf{w} = [ux; uy; \sigma]$  e' esattamente LTI:

$$\dot{\mathbf{x}} = \mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{w} + \mathbf{c}$$

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0; & 0 & 0 & 0 & 1 & 0; & 0 & 0 & 0 & 0 & 0; & 0 & 0 & 0 & 0 & 0; & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\mathbf{B} = \begin{bmatrix} 0 & 0 & 0; & 0 & 0 & 0; & 1 & 0 & 0; & 0 & 1 & 0; & 0 & 0 & -V_c \end{bmatrix}$$

$$\mathbf{c} = \begin{bmatrix} 0; & 0; & 0; & -1; & 0 \end{bmatrix} \quad \leftarrow \text{gravita'}$$

Questo e' esattamente `lti_zoh.m` (righe 23-25 di quel file).

**Attenzione al ruolo di  $\sigma$ .** In questo homework  $T_{\min} = 0$ , quindi il vincolo di spinta e' gia' convesso e – a rigore – lo slack “non servirebbe” per convessificare il bound. Ma  $\sigma$  serve **comunque**, per un motivo diverso: senza di lui la riga della massa sarebbe  $\mathbf{z\_dot} = -V_c * |u|$ , cioe' una **uguaglianza con una norma dentro**: non lineare, non convessa. Introdurre  $\sigma$  con la **disuguaglianza**  $|u| \leq \sigma$  (che e' un cono di Lorentz, convesso) e mettere  $\sigma$  – non  $|u|$  – nella dinamica e' quello che rende il sistema LTI. E' questo il punto che spesso si perde.

## Perche' il rilassamento e' LOSSLESS

Il rilassamento sostituisce l'insieme  $\{ (u, \sigma) : \sigma = |u| \}$  (non convesso) con  $\{ (u, \sigma) : \sigma \geq |u| \}$  (convesso). E' un rilassamento: si e' allargato l'insieme ammissibile. **Lossless** significa che all'ottimo il vincolo e' **attivo**, cioe'  $\sigma^* = |u^*|$ , quindi la soluzione del problema rilassato e' anche soluzione del problema originale – niente e' stato perso.

Nel caso di questo codice la dimostrazione e' quasi banale ed e' istruttivo farla, perche' la struttura ZOH la rende esplicita. Poiche' la riga 5 di  $\mathbf{A}$  e' nulla, l'esponenziale di matrice ha  $\exp(\mathbf{A} * s)$  con quinta riga  $[0 \ 0 \ 0 \ 0 \ 1]$ , e quindi la discretizzazione esatta della riga della massa e':

$$\mathbf{z}_{k+1} = \mathbf{z}_k - V_c * dt * \sigma_k$$

cioe'  $\mathbf{Bbar}(5,3) = -V_c * dt$  e  $\mathbf{cbar}(5) = 0$ . Iterando:

$$\mathbf{z}_N = \mathbf{z}_0 - V_c * dt * \sum_{k=1}^{N-1} \sigma_k$$

Il **costo** e'  $\max \mathbf{z}_N$  (righe 1183: `optimize(cstr, -XI(5,N))`). Ma  $\mathbf{z}_0$  e' fissato dalla condizione iniziale, quindi

$$\max \mathbf{z}_N \quad \Leftrightarrow \quad \min \sum_k \sigma_k$$

Ogni  $\sigma_k$  compare quindi nel costo con **coefficiente positivo** (lo si vuole il piu' piccolo possibile), e negli altri vincoli solo come: -  $\sigma_k \geq |u_k|$  (limite inferiore), -  $\sigma_k \leq T_{\max} * e^{-z}$  (limite superiore, che abbassare non viola).

Quindi all'ottimo ogni  $\sigma_k$  viene **schiacciato sul suo limite inferiore**:  $\sigma_k = |u_k|$ . Il cono e' attivo. Il rilassamento e' lossless. Nessun  $\sigma$  “spreco” darebbe vantaggio: brucerebbe carburante senza produrre accelerazione.

Nota che questo argomento **non e' quello generale**. Il risultato forte di Acikmese & Ploen e' che il rilassamento resta lossless anche con  $T_{\min} > 0$ , dove il vincolo  $T_{\min} \leq \sigma \leq T_{\max}$  **non** ha piu'  $|u|$  come lower bound naturale – li' la dimostrazione passa per il Principio del Massimo di Pontryagin (un cono inattivo forzerebbe  $\lambda_m(tf) = 0$ , in contraddizione con la condizione di trasversalita'). Nel codice  $T_{\min} = 0$ , quindi la parte “difficile” del teorema non viene esercitata: siamo nel caso facile.

## L'unica non convessita' superstite: il bound superiore di spinta

Il vincolo fisico e'  $|T| \leq T_{\max}$ . Con  $T = m * u = e^{-z} * u$ :

$$|u| \leq T_{\max} * e^{-z} \quad \text{cioe'} \quad \sigma \leq T_{\max} * e^{-z}$$

$e^{-z}$  e' **convessa**, quindi  $\sigma \leq (\text{funzione convessa di } z)$  definisce un insieme **non convesso** (il sopra-grafo di una funzione convessa non e' convesso). Ecco perche' serve ancora un loop SCvx, sia pure per un solo vincolo scalare.

Si linearizza attorno a  $z_{\text{ref}}$ :

$$e^{-z} \approx e^{-z_{\text{ref}}} * (1 - (z - z_{\text{ref}}))$$

da cui il vincolo del codice (righe 1158-1159):

$$\sigma_k \leq T_{\text{max}} * e^{-z_{\text{ref},k}} * (1 - (z_k - z_{\text{ref},k}))$$

Poiche'  $e^{-z}$  e' convessa, la sua **tangente sta sempre sotto** la funzione: la retta linearizzata **sottostima** il bound vero. Quindi il vincolo linearizzato e' **conservativo** (approssimazione interna): ogni iterato accettato e' **feasible per il problema originale**. E' una proprieta' molto comoda – il conservatorismo si riduce man mano che  $z_{\text{ref}}$  insegue la soluzione, ma non si viola mai il bound fisico.

**Possibile domanda d'esame** – Il rilassamento  $\sigma \geq |u|$  allarga l'insieme ammissibile: perche' non "bara"? *Risposta:* perche'  $\sigma$  entra nella dinamica della massa con segno negativo ( $\dot{z} = -V_c \sigma$ ) e il costo e'  $\max z_N$ , quindi il costo e' una funzione **monotona decrescente** di ogni  $\sigma_k$ . L'ottimizzatore ha percio' un incentivo stretto a comprimere ogni  $\sigma_k$  sul suo limite inferiore  $|u_k|$ : il cono e' attivo all'ottimo e la soluzione rilassata coincide con quella originale. Con  $T_{\text{min}} > 0$  l'argomento e' meno immediato e richiede il PMP (Acikmese & Ploen 2007).

## %% Problem data (Table 1, dimensional) (righe 25-39)

- Righe 26-33: i dati della Tabella 1 della traccia, in SI. Stato iniziale (1000, 3000) m, velocita' (300, -200) m/s,  $m_0 = 2000$  kg,  $I_{\text{sp}} = 225$  s,  $T_{\text{max}} = 70$  kN, glide-slope  $\theta_{\text{max}} = 60$  deg.
- Riga 31:  $c = I_{\text{sp}} * g_0$  con  $g_0 = 9.80665$  (gravita' standard, non  $g = 9.81$  del pianeta). E' la distinzione classica:  $g_0$  e' una **costante di conversione** per passare da  $I_{\text{sp}}$  a velocita' efficace di scarico,  $g$  e' l'accelerazione di gravita' del corpo su cui si atterra. Confonderle e' un errore comune.
- Riga 32: **data.Tmin = 0**. Come discusso, questo elimina la non convessita' anulare del vincolo di spinta. Tutta la macchinaria GFOLD implementata qui e' quindi esercitata nel suo caso "facile".
- Riga 35: **tf = 38 s fissato** (non e' una variabile di decisione). Questo e' cio' che tiene le trascrizioni lineari: con **tf** libero il problema diventa nonlineare anche nel caso GFOLD (bisognerebbe fare una ricerca 1-D su **tf**).
- Riga 36:  $N = 50$  nodi  $\rightarrow dt = 38/49 = 0.7755$  s.
- Riga 37:  $n_{\text{sub}} = 2$  sotto-passi RK4 per intervallo ZOH (variante a).
- Righe 38-39: **scvx\_max = 15** iterazioni esterne, **scvx\_tol = 1e-3** (non-dim). Il commento dice "(variant b)" ma i valori sono passati a **tutti e tre** i loop SCvx (righe 79, 90, 104): il commento e' fuorviante.

## %% Non-dimensionalisation (righe 41-49)

- Riga 42: chiama **nondim** (righe 203-228). Lo schema di riferimento e':

```
L_ref = y0 = 3000 m           (quota iniziale)
g_ref = g = 9.81 m/s^2
t_ref = sqrt(L/g) = 17.487 s
V_ref = sqrt(g*L) = 171.55 m/s
m_ref = m0 = 2000 kg
T_ref = m0*g = 19620 N        (peso iniziale)
Vc = V_ref / c = 0.0777      (numero di Tsiolkovsky)
```

Con questa scelta la gravita' non-dim vale **esattamente 1** (ecco perche' in `ode_descent.m` la riga di  $\dot{v}_y$  e'  $u(2)/x(5) - 1$ ), e  $m_{\text{nd}} = 1$ ,  $T_{\text{max\_nd}} = 70000/19620 = 3.568$  (cioe' il motore puo' dare ~3.57 g a massa piena).  $t_{\text{f\_nd}} = 38/17.487 = 2.173$ .

- Riga 46: `tf_nd = tf / ref.t.`
- Riga 49: i raggi base della **trust region**, in unita' non-dim:

```
trust = struct('pos', 0.17, 'vel', 0.6, ...
              'mass', 0.1, 'thrust', 1.0);
```

In SI:  $0.17 \cdot 3000 = 510$  m,  $0.6 \cdot 171.55 = 103$  m/s,  $0.1 \cdot 2000 = 200$  kg,  $1.0 \cdot 19620 = 19620$  N. Sono raggi **larghi**: il warm start e' gia' vicino all'ottimo, il ratio test si occupa di stringerli. Questi raggi valgono solo per le varianti (b) e (c): la (d) ha i suoi, hard-coded alla riga 1231.

## %% YALMIP + ECOS availability check (righe 51-55)

- Riga 52: `yalmip_ok = exist('yalmip','file') && exist('ecos','file')`. Se manca uno dei due, le varianti (c) e (d) vengono **saltate con grazia** e lo script cade su `plot_compare3`. E' la ragione per cui la repo dichiara "no external dependency" per le varianti (a)/(b): sono pure `fmincon`.

## %% Solve all three transcriptions (righe 57-110)

Il banner dice "three" ma le trascrizioni risolte sono cinque (trapezoidale + quattro varianti ZOH): il commento e' rimasto indietro rispetto al codice.

- Righe 58-64: baseline **trapezoidale** (Task 1), `solve_trap`. Serve a due cose: (i) termine di paragone, (ii) **warm start** per le varianti (b) e (c).
- Riga 66-72: variante (a), `solve_zoh` con `n_sub = 2`.
- Righe 74-85: variante (b), `solve_scvx`, **warm-startata da sol\_trap\_nd** (riga 80). Il commento alle righe 76-77 e' esplicito: la linearizzazione LTV e' accurata solo localmente, quindi serve un riferimento gia' quasi ottimo piu' una trust region hard.
- Riga 87-96: variante (c), `solve_scvx_yalmip`, stesso warm start, stessi raggi.
- Righe 98-109: variante (d), `solve_gfold_scvx`. Notare la firma: **non riceve ne' sol\_trap\_nd ne' trust** (riga 104). E' **self-starting** – si costruisce da sola il riferimento (riga 1223). E' la conseguenza diretta del fatto che la sua dinamica e' esatta: non c'e' errore di modello da cui difendersi con un buon warm start.

Ogni solve e' cronometrato con `tic/toc` e la soluzione riportata in SI con `dim_sol. sol.iter` (righe 83, 94, 107) raccoglie il numero di iterazioni SCvx.

## %% Forward-integration validation (righe 112-137)

E' il cuore della **validazione** richiesta dalla traccia: si prendono i controlli ottimizzati e li si "rivola" attraverso la dinamica continua vera con `ode45` (`RelTol = 1e-10`, `AbsTol = 1e-12`, riga 839), campionando ai nodi della griglia.

- Righe 114-116: `fwd_integrate` con la convenzione di interpolazione **coerente con la trascrizione**: 'pwl' (piecewise-linear) per la trapezoidale, 'zoh' (piecewise-constant) per le ZOH. Questo e' fondamentale: replayare i controlli trapezoidali come ZOH darebbe un errore artificialmente enorme, e viceversa.
- Righe 124-127: la variante GFOLD richiede un replay **diverso** (`fwd_integrate_uacc`, righe 1285-1303), perche' tiene costante l'**accelerazione**  $u = T/m$ , non la spinta  $T$ . Con  $u$  costante la spinta vera  $T(t) = m(t) * u$  **fluttua** con la massa che si consuma. E' la convenzione ZOH nativa di GFOLD, e sbagliarla vanificherebbe tutta la fedelta' guadagnata.
- Righe 118-120, 123, 127: `node_err` calcola la norma dell'errore ai nodi.

**Possibile domanda d'esame** – Perche' GFOLD ha bisogno di un replay diverso dalle altre varianti ZOH? *Risposta*: perche' il suo controllo non e' la spinta  $T$  ma l'accelerazione  $u = T/m$ . Tenere  $u$  costante su un intervallo significa che  $T = m(t) * u$  varia con la massa che si esaurisce. Se lo si replayasse tenendo  $T$  costante si simulerebbe una legge di controllo diversa da quella ottimizzata, e l'errore di fedelta' sarebbe dominato da questo mismatch di convenzione, non dalla qualita' della trascrizione.

## %% Replay landing accuracy + wall-time summary (righe 139-165)

- Righe 142-144: la funzione anonima `land` estrae tre numeri dal replay:

```
land = @(s_nd, X) [norm(X(end,1:2)) * ref.L, ...  
                  norm(X(end,3:4)) * ref.V, ...  
                  (s_nd.m_f - X(end,5)) * ref.m];
```

cioe' **dispersione di posizione** al touchdown (norma della posizione finale replayata rispetto al bersaglio (0,0)), **dispersione di velocita'** (norma della velocita' finale, bersaglio (0,0)) e **drift della massa finale** (differenza fra la `m_f` che la trascrizione crede di avere e quella che il replay consegna).

- Righe 149-165: tabella formattata. E' la figura di merito piu' operativa dell'homework: *se volassi questa legge di controllo open-loop, dove atterreresti?*

## %% Plots e %% Export figures (righe 167-193)

- Righe 168-175: `plot_compare5` se YALMIP c'e', `plot_compare3` altrimenti.
- Righe 178-193: esporta **tutte** le figure aperte in `figures/`, forzando il tema chiaro (riga 186, `theme(fig,'light')`), con fallback `Color = 'w'` per MATLAB pre-R2025a). I nomi file sono slugificati dal `'Name'` della figura (riga 180).

---

## nondim (righe 203-228) e dim\_sol (righe 230-253)

Boundary helper: qui l'`arguments` validation c'e' (righe 210-212, 237-240), a differenza delle funzioni hot-loop. E' una scelta esplicita documentata nel banner alle righe 195-201: validare una volta per run al confine, mai dentro i loop di `fmincon` / `ode45` (che chiamano quelle funzioni ~1e6 volte).

`dim_sol` (righe 241-252) riporta la soluzione in SI e calcola `sol.fuel = (m0 - m_f) * m_ref`. Nota che **non** copia `sol.iter`: per questo lo script deve ricopiarlo a mano alle righe 83, 94, 107. Piccola scomodita' del design.

## jacobians (righe 255-277)

```
Tmag_reg = sqrt(Tx^2 + Ty^2 + 1e-6);  
...  
A_jac(3,5) = -Tx / m^2;  
A_jac(4,5) = -Ty / m^2;  
B_jac(3,1) = 1/m;  
B_jac(4,2) = 1/m;  
B_jac(5,1) = -Vc * Tx / Tmag_reg;  
B_jac(5,2) = -Vc * Ty / Tmag_reg;
```

- Riga 265: legge massa e spinta dallo stato/controllo.
- Riga 266: **la regolarizzazione**.  $d/dT (-Vc*|T|) = -Vc * T/|T|$  non esiste in  $T = 0$ . Sostituendo  $|T| \rightarrow \sqrt{|T|^2 + \text{eps}^2}$  con  $\text{eps}^2 = 1e-6$  (quindi  $\text{eps} = 1e-3$  non-dim, circa 20 N) la derivata resta finita e tende a zero linearmente vicino all'origine. **Il costo:** una distorsione  $O(\text{eps})$  proprio dove  $|T| \sim \text{eps}$  - cioe' **sull'arco di coast**, che e' esattamente dove il modello lineare fatica di piu'. E' l'hack piu' rilevante del file, ed e' proprio il motivo per cui la variante (d) (che elimina quella riga singolare per costruzione) e' cosi' piu' pulita.
- Righe 267-271:  $A = df/dx$ . La quinta riga e' **identicamente nulla**: la portata di massa non dipende dallo stato. Le righe 3-4 catturano l'unico accoppiamento stato-stato non banale: la sensitivita' dell'accelerazione alla massa.
- Righe 272-276:  $B = df/du$ . Le righe 3-4 sono  $1/m$ ; la riga 5 e'  $-Vc$  per il versore di spinta.

Nessuna `arguments` validation: sta dentro il loop di `ode45` chiamato da `compute_ltv_zoh`, quindi viene invocata migliaia di volte per intervallo.

## ltv\_aug\_rhs (righe 279-303)

E' l'ODE aumentata dell'Appendice A, nella forma **beta-gamma** (riferita a `t_k`). Lo stato aumentato `z` e' un vettore 70x1:

```
z(1:5)    = x_ref      (5)    traiettoria di riferimento
z(6:30)   = vec(Phi)   (25)   matrice di transizione
z(31:55)  = vec(Psi)   (25)   transizione inversa, Psi = Phi^{-1}
z(56:65)  = vec(Beta) (10)   integrale di Psi*B      (5x2)
z(66:70)  = Gamma      (5)   integrale di Psi*c
```

```
[A_jac, B_jac] = jacobians(x_ref, u_k, Vc);
f_val = ode_descent(x_ref, u_k, Vc);
c_off = f_val - A_jac * x_ref - B_jac * u_k;
dx_ref = f_val;
dPhi    = A_jac * Phi;
dPsi    = -Psi * A_jac;
dBeta   = Psi * B_jac;
dGamma  = Psi * c_off;
```

- Righe 289-291: spacchetta lo stato aumentato.
- Riga 294: **le Jacobiane sono rivalutate a ogni passo interno di `ode45`**, sulla `x_ref` che sta evolvendo dentro l'intervallo (riga 297: `dx_ref = f_val`). Quindi `A(t)` e `B(t)` sono genuinamente **tempo-varianti dentro l'intervallo**, non congelate al valore nodale. E' la lettura "letterale" dell'Appendice A – alcune implementazioni si accontentano di congelare `A`, `B` a `t_k` (e allora la ZOH diventa un **expm** per intervallo), qui no.
- Riga 296: il **termine affine** `c = f(x_ref, u_ref) - A x_ref - B u_ref` derivato sopra. E' cio' che rende la linearizzazione esatta nel punto di riferimento.
- Riga 299: `dPsi = -Psi * A`, con la derivazione fatta sopra.
- Righe 300-301: quadrature pure, nessun feedback.

**Sottigliezza da capire (e da dire all'orale):** `x_ref` dentro l'intervallo e' la **propagazione nonlineare** dello stato nodale di riferimento sotto `u_k`. Se il riferimento non e' dinamicamente consistente con la ZOH (e la soluzione trapezoidale del warm start **non lo e'**), allora `x_ref(t_{k+1})` calcolato qui **non coincide** con `ref.x(k+1)`. La linearizzazione e' quindi costruita attorno a una traiettoria leggermente diversa da quella nodale. E' una delle ragioni per cui i primi passi SCvx vengono rifiutati.

## compute\_ltv\_zoh (righe 305-339)

- Riga 322: `dt = tf/(N-1)`.
- Riga 326: **tolleranze `RelTol 1e-8`, `AbsTol 1e-10`** – piu' lasche di quelle del replay (`1e-10 / 1e-12`, riga 839). Ragionevole: qui si sta costruendo un modello che verra' comunque rifatto alla prossima iterazione SCvx, li' si sta misurando la verita'.
- Riga 331: condizioni iniziali dell'ODE aumentata: `Phi(t_k) = Psi(t_k) = I`, `Beta = 0`, `Gamma = 0`.
- Riga 332: **una `ode45` per intervallo** – quindi 49 integrazioni di un sistema 70-D per **ogni** iterazione SCvx. E' il collo di bottiglia computazionale delle varianti
  - (b) e (c) (e la ragione per cui la (d), che fa un solo **expm**, e' ~10x piu' veloce).
- Righe 334-337: lettura dei valori terminali e ricostruzione

```
Abar_k = Phi(t_{k+1})
Bbar_k = Phi(t_{k+1}) * Beta(t_{k+1})
```

```
cbar_k = Phi(t_{k+1}) * Gamma(t_{k+1})
```

E' il prodotto finale imposto dall'identita' di semigrupp.

## **solve\_ltv\_nlp (righe 341-412) e ltv\_nonlcon (righe 414-427)**

E' il **sottoproblema interno** della variante (b), risolto con **fmincon**.

- Righe 362-363: **nz** = 7N, con il layout per nodo [x; y; vx; vy; m; Tx; Ty]. Stesso layout di tutte le altre varianti **fmincon** (e di **main\_task1.m**).
- Righe 364-368: **warm start** = il riferimento corrente (**ref\_to\_z**), oppure la guess lineare se non c'e' riferimento.
- Riga 369: box bounds con **zero\_uN** = **true** (il controllo del nodo N e' inchiodato a 0: sotto ZOH non ha nessun intervallo su cui agire).
- Righe 370-372: intersezione con la trust region.
- Righe 375-405: **assemblaggio della matrice di uguaglianza sparsa via triplette** (**rows**, **cols**, **vals**). Le righe di **Aeq** sono  $9 + 5*(N-1)$ :
  - righe 1-5: stato iniziale completo (**x0**, **y0**, **vx0**, **vy0**, **m0**);
  - righe 6-9: posizione e velocita' finali nulle (**la massa finale e' libera**: e' proprio la variabile che si massimizza);
  - righe 10 in poi, a blocchi di 5: la **dinamica LTV**  $x_{k+1} - Abar_k x_k - Bbar_k u_k = cbar_k$ . Ogni riga ha 8 entrate non nulle: 1 (per  $x_{k+1}$ ) + 5 (per A) + 2 (per B). Da qui **nnz\_dyn** =  $8*5*(N-1)$  (riga 376).
- Riga 405: **Aeq** = **sparse(...)**, poi riga 408 la converte in **full(Aeq)** per **fmincon**. Costruire in sparso e passare in denso e' un po' contorto (il commento a riga 374 dice che serviva a evitare un warning di indicizzazione sparsa); con N = 50 la matrice e' 254 x 350, quindi non e' un problema.
- Righe 407-409: obiettivo **-z(iN\_m)** con **iN\_m** =  $(N-1)*7 + 5$ , cioe' **massimizza la massa al nodo N**. E' un obiettivo **lineare**. Vincoli: uguaglianze lineari (dinamica + BC), box (bounds + trust), e **ltv\_nonlcon** per i vincoli di percorso.
- **ltv\_nonlcon** (righe 414-427): **solo path\_ineq**, **c\_eq** = []. La dinamica non e' piu' qui, e' gia' dentro **Aeq**. Questo e' il salto qualitativo rispetto a **solve\_zoh**: la dinamica e' passata da vincolo **nonlineare** a vincolo **lineare**.

**Possibile domanda d'esame** – Il sottoproblema di **solve\_ltv\_nlp** e' convesso. Perche' allora e' dato a **fmincon**, che e' un solver per NLP non convessi? *Risposta*: funziona (**fmincon** trova lo stesso ottimo globale, il problema e' convesso), ma e' uno spreco: SQP approssima il cono di spinta  $|T| \leq T_{max}$  con successive linearizzazioni quadratiche invece di trattarlo come primitiva, e paga una Jacobiana alle differenze finite. La variante (c) e' proprio la stessa cosa fatta bene: stesso loop esterno, sottoproblema riscritto come SOCP e dato a ECOS – circa 3x piu' veloce end-to-end e con fedelta' per-passo un ordine di grandezza migliore.

## **solve\_scvx (righe 429-537)**

Il **loop esterno SCvx** con **fmincon**. Il docstring alle righe 442-447 riassume esattamente la logica del ratio test.

- Righe 458-462: riferimento iniziale = **init\_ref** (il trapezoidale) o, se assente, la guess lineare. Nella pratica e' sempre il trapezoidale (riga 80).
- Righe 464-468: parametri del ratio test.

```
rho      = 1.0;      rho_min = 1e-3;      rho_max = 1.0;
eta_l     = 0.25;     eta_h   = 0.7;
```

Nota **rho\_max** = 1.0: la trust region puo' solo **restringersi** rispetto ai raggi base, mai allargarsi oltre. E' una scelta conservativa coerente col fatto che i raggi base sono gia' generosi.

- Righe 470-474: **conv\_hist** traccia per-iterazione **m\_f**, **delta\_x**, **rho**, **eta**, e se il passo e' stato accettato.



- Righe 484-485: ricalcola le matrici LTV **attorno al riferimento corrente** e risolve il sottoproblema. Questo e' il "successive" di successive convexification.
- Righe 487-496: il **ratio test**.

```
J_pred = sol_cand.m_f - ref.m_f;           % guadagno predetto (LTV)
[~, X_act] = fwd_integrate(sol_cand, d, 'zoh');
m_f_actual = X_act(end, 5);
J_act      = m_f_actual - ref.m_f;         % guadagno reale (ode45)
eta = J_act / J_pred;                     % (con guardia se J_pred ~ 0)
```

Il costo e'  $-m_f$ , quindi "riduzione del costo" = "aumento di  $m_f$ ": il commento a riga 487 lo dice.

**Onesta'**: numeratore e denominatore usano **la stessa baseline** `ref.m_f`, che e' la massa finale *secondo l'NLP* del riferimento – non la sua massa finale *replayata*. Poiche' il riferimento e' a sua volta un candidato accettato la cui  $m_f$  NLP differisce (di poco) dal suo replay, **eta** porta dentro l'errore di trascrizione del riferimento. Un ratio test rigoroso userebbe `m_f_replay(ref)` come baseline in entrambi. L'effetto e' piccolo ma non nullo, e spiega parte del comportamento a scacchiera di **eta** (accettati/rifiutati alternati) riportato nel report.

- Righe 498-501: `delta_x` = norma della variazione dell'iterato di **stato** (posizione, velocita', massa). **I controlli non entrano** nel criterio di convergenza: due soluzioni con lo stesso stato ma spinte diverse (possibile solo in coast) sarebbero considerate convergenti. In pratica non succede.
- Riga 503: `accepted = (eta >= eta_1)`. Notare: **eta** puo' essere **negativo** (il modello predice un guadagno e la realta' consegna una perdita) – in quel caso e' giustamente rifiutato.
- Righe 513-530: l'aggiornamento.
  - accettato: `sol_best = sol_cand; ref = sol_cand`; e, se `eta > eta_h`, `rho` raddoppia (cappato a `rho_max`). Convergenza se `delta_x < tol`.
  - rifiutato: **il riferimento NON viene aggiornato** (questo e' essenziale: si ri-risolve lo stesso sottoproblema con una trust region piu' stretta), `rho` dimezza. Se `rho < rho_min` si esce.
- Riga 535: `sol = sol_best`. **Attenzione al nome**: `sol_best` e' l'**ultimo passo accettato**, non quello con la  $m_f$  migliore. Se un passo accettato peggiorasse  $m_f$  (possibile: `eta >= 0.25` non implica `J_act > 0` se `J_pred < 0`), verrebbe comunque tenuto. Il nome e' fuorviante.

## ternary (righe 539-548)

```
function out = ternary(cond, a, b)
    if cond, out = a; else, out = b; end
end
```

Innocuo di per se', ma **attenzione a come e' usato**. Alla riga 1091 (e 1253):

```
eta = ternary(abs(J_pred) < 1e-10, 1, J_act / J_pred);
```

MATLAB valuta **entrambi** gli argomenti prima di chiamare la funzione: `J_act/J_pred` viene calcolato **anche** quando `J_pred ~ 0`, producendo `Inf` o `NaN` (che poi viene scartato). Non e' un bug – MATLAB non solleva errore sulla divisione per zero – ma la guardia **non guarda' un bel niente**. `solve_scvx` (righe 492-496) usa invece un `if/else` vero, che e' corretto. Incoerenza fra le tre implementazioni dello stesso loop.

## ref\_to\_z (righe 550-569) e apply\_trust (righe 571-600)

- `ref_to_z`: impacchetta la struct di riferimento nel vettore di decisione  $7N \times 1$ . **Piccola verruca**: copia anche `ref.Tx(N)`, `ref.Ty(N)` (righe 566-567). Ma il riferimento e' la soluzione **trapezoidale**, che NON inchioda il controllo del nodo  $N$  a zero. Poiche' `box_bounds(..., zero_uN=true)` impone `lb = ub = 0` su quelle due variabili, il punto iniziale passato a `fmincon` e' **fuori dai bound**, e `fmincon` lo proietta silenziosamente dentro. Innocuo ma sporco.

- `apply_trust` (righe 581-599): intersezione **in norma infinito** dei bound esistenti con `[ref - raggio, ref + raggio]`. La trust region e' quindi un **box**, non una palla euclidea, e non e' una penalita' soft. E' la variante "adaptive hard trust region" della tassonomia SCvx. I controlli sono vincolati solo per  $i < N$  (riga 593), coerentemente con la convenzione ZOH.

## `solve_trap` (righe 602-624) e `solve_zoh` (righe 626-650)

Stessa struttura, differiscono solo nel `nonlcon` e nella convenzione sul controllo terminale.

- `solve_trap` (baseline Task 1): difetti trapezoidali, controllo piecewise-linear (tutti gli  $N$  controlli sono significativi -> `zero_uN = false`).
- `solve_zoh` (variante a): difetti ZOH via RK4, controllo piecewise-constant -> `zero_uN = true`, `u_N` inchiodato a zero. Il vettore di decisione ha la stessa dimensione  $7N$ : si spreca un controllo, ma il layout resta identico e riusabile.
- Entrambi: obiettivo `-z(iN_m)` (massimizza `m_N`), `Aeq/beq` dalle bcs, opzioni da `fmincon_opts()` con `Display = 'final'`. `solve_ltv_nlp` invece usa `fmincon_opts('off')` (riga 409), per non stampare 15 volte dentro il loop SCvx.

## `init_guess` (righe 652-676) e `box_bounds` (righe 678-700)

- `init_guess`: interpolazione **lineare** fra stato iniziale e origine per posizione e velocita'; massa che scende linearmente del 30% (riga 668); controllo `Tx = 0`, `Ty = m0` - cioe' **spinta di hover** (riga 673), che in non-dim vale esattamente `m*g = 1*1 = 1` (ecco perche' il codice scrive `d.m0`).
- `box_bounds`: `y >= 0` (non si scava), massa in `[1e-3, m0]` (**strettamente positiva**: `m` compare a denominatore in `ode_descent`, `m = 0` farebbe esplodere tutto), componenti di spinta in `[-Tmax, Tmax]`. Nota che questo box e' un **quadrato** che contiene il disco `|T| <= Tmax`: il vincolo circolare vero e' imposto separatamente in `path_ineq`. Il box e' ridondante ma aiuta il solver a restare in una regione limitata.

## bcs (righe 702-720)

9 uguaglianze lineari: 5 per lo stato iniziale completo, 4 per posizione e velocita' finali nulle. **La massa finale non e' vincolata** - e' l'obiettivo.

## `fmincon_opts` (righe 722-736)

```
opts = optimoptions('fmincon', ...
    'Algorithm', 'sqp', 'Display', display_mode, ...
    'MaxIterations', 1000, 'MaxFunctionEvaluations', 1e6, ...
    'OptimalityTolerance', 1e-5, 'ConstraintTolerance', 1e-6, ...
    'StepTolerance', 1e-10);
```

- `sqp`: buono per NLP con vincoli densi e problemi di dimensione media come questo.
- `MaxFunctionEvaluations = 1e6`: serve, perche' **le Jacobiane sono alle differenze finite**. Con 350 variabili, ogni gradiente costa ~350 valutazioni del `nonlcon`, e ogni valutazione del `nonlcon` della variante (a) costa 49 propagazioni RK4. E' il costo dominante e la ragione per cui la variante (b) e' lenta pur avendo dinamica lineare (la nonlinearity residua in `path_ineq` obbliga comunque `fmincon` a fare differenze finite sull'intero vettore).
- Il README ammette che le run del Task 1 a `tf` corto arrivano al cap di 1000 iterazioni con ottimalita' del primo ordine ferma a `1e-3-1e-4`: la causa e' proprio il punto **non differenziabile** `T = 0` della norma `|T|` in `path_ineq`, che cade esattamente sull'arco di coast.

## unpack (righe 738-757)

Da vettore di decisione a struct. Calcola `sol.Tmag`, `sol.m_f = sol.m(end)`.

## trap\_nonlcon (righe 759-781) e zoh\_nonlcon (righe 783-803)

Le due trascrizioni “a difetti”:

- **trapezoidale** (riga 777):

```
def_k = x_{k+1} - x_k - (dt/2) * (f_k + f_{k+1})
```

Regola del trapezio: ordine  $O(dt^2)$  globale. Il controllo e' implicitamente piecewise-linear (entra sia in `f_k` che in `f_{k+1}`).

- **ZOH/RK4** (riga 798):

```
def_k = x_{k+1} - RK4(x_k, u_k, dt, Vc, n_sub)
```

Qui il “difetto” e' la differenza rispetto al **flusso esatto** (approssimato a ordine  $O(h^4)$  con  $h = dt/n_{sub}$ ). E' un **multiple shooting**, non collocazione. Con `n_sub = 2`,  $h = dt/2 = 0.0222$  non-dim, l'errore globale e'  $\sim 1e-8$  – e infatti il replay `ode45` della variante (a) misura  $1.4e-8$  (README, riga 110). I due numeri **coincidono per costruzione**: il difetto ZOH e' la mappa di flusso RK4, quindi l'errore di trascrizione e' esattamente l'errore RK4-vs-ode45.

## path\_ineq (righe 805-820)

```
Tmag      = sqrt(Z(6,:).^2 + Z(7,:).^2).';
g_thr_lo  = d.Tmin - Tmag;           % Tmin - |T| <= 0
g_thr_hi  = Tmag - d.Tmax;           % |T| - Tmax <= 0
tt        = tan(d.theta_mx);
g_gs_pos  = ( Z(1,:).' - tt*Z(2,:).' ); % x - tan(th)*y <= 0
g_gs_neg  = (-Z(1,:).' - tt*Z(2,:).' ); % -x - tan(th)*y <= 0
```

- Il glide-slope: la coppia di semipiani equivale a  $|x| \leq \tan(\theta_{max}) * y$ , cioe' il veicolo deve restare **dentro un cono** con vertice sulla piazzola e semiapertura `theta_max = 60 deg` misurata **dalla verticale**. E' un vincolo **convesso** (un cono di Lorentz in 2D degenerato in due semipiani). Il punto iniziale ha `x0/y0 = 1000/3000 = 0.33`, ben dentro  $\tan(60) = 1.73$ .
- `g_thr_lo` con `Tmin = 0` diventa  $-|T| \leq 0$ : **sempre soddisfatto**. E' un vincolo ridondante – ma non gratuito: e' **non differenziabile in  $T = 0$** , e siccome l'arco di coast sta esattamente li', `fmincon` ci sbatte contro con le sue differenze finite. Il README lo identifica come la causa radice della convergenza lenta.

## fwd\_integrate (righe 822-857)

Il replay di validazione. Per ogni intervallo, integra la dinamica **nonlineare vera** con `ode45` a tolleranze strette (riga 839:  $1e-10 / 1e-12$ ), partendo **sempre dallo stato replayato precedente** `X(k,:)` – non dallo stato NLP. E' quindi un replay **open-loop puro**: gli errori si accumulano, ed e' esattamente cio' che si vuole misurare (dove atterrei davvero?).

Due modalita' (righe 842-851): `'zoh'` (controllo costante) e `'pwl'` (interpolazione lineare fra `u_k` e `u_{k+1}`). Scegliere la modalita' **coerente con la trascrizione** e' obbligatorio, altrimenti si misura il mismatch di convenzione, non la fedelta'.

## node\_err (righe 859-868)

```
e = vecnorm([sol.x sol.y sol.vx sol.vy] - X(:,1:4), 2, 2);
```

Norma euclidea dell'errore su **posizione e velocita'**, per nodo, in unita' non-dim. **La massa e' esclusa** (il docstring alla riga 860 lo dice). Ma il messaggio stampato alla riga 130 dice "max grid-node nondim **state error**": e' una piccola imprecisione, lo stato completo avrebbe 5 componenti. Il canale della massa e' monitorato separatamente, tramite il drift di **m\_f** (riga 144) e tramite il ratio test.

## plot\_compare3 (righe 870-963)

Percorso di fallback quando YALMIP/ECOS non ci sono. Quattro figure (traiettoria con corridoio glide-slope, |T| con **stairs** per le ZOH e **plot** per la trapezoidale – scelta corretta, la ZOH e' davvero una scala –, massa, fedelta' in scala semilog) piu' il pannello di convergenza SCvx (righe 928-962: **delta\_x**, **rho** e **m\_f** sopra, barre di **eta** con le soglie 0.25 / 0.7 sotto, e i passi rifiutati colorati in arancione).

---

## solve\_ltv\_nlp\_yalmip (righe 965-1040)

Il sottoproblema interno della variante (c): lo stesso problema di **solve\_ltv\_nlp**, ma scritto come **SOCP** e dato a ECOS.

- Righe 988-989: le variabili di decisione **modellate correttamente**:

```
X = sdpvar(5, N, 'full'); % stato per nodo
U = sdpvar(2, N-1, 'full'); % controllo per INTERVALLO
```

Notare U ha N-1 colonne, non N: sotto ZOH esistono solo N-1 controlli. E' piu' pulito della versione fmincon, che porta un **u\_N** fantasma inchiodato a zero. Il padding a N avviene solo in uscita (riga 1032).

- Righe 993-994: BC iniziali (5) e finali (4, massa libera).
- Righe 996-998: **dinamica LTV come uguaglianze lineari**.
- Righe 1000-1002: **il vincolo conico**:

```
cstr = [cstr, norm(U(:,k)) <= d.Tmax];
```

Questo e' un **cono di Lorentz** (SOC). YALMIP lo riconosce come tale e lo passa a ECOS in forma nativa. Il commento (riga 1000) nota giustamente che con **Tmin = 0** il bound inferiore e' banalmente soddisfatto e non serve modellarlo.

- Righe 1004-1007: glide-slope, lineare.
- Righe 1009-1010: quota  $\geq 0$ , massa in  $[1e-3, m0]$ .
- Righe 1012-1024: trust region, **lineare** (box in norma infinito). Anche la trust region resta dentro la classe SOCP.
- Riga 1026:

```
res = optimize(cstr, -X(5,N), ...
              sdpsettings('solver','ecos','verbose',0));
```

Obiettivo **lineare** ( $-m_N$ ), vincoli lineari + conici: e' un SOCP puro. ECOS e' un solver interior-point per conici (lineari, di secondo ordine, esponenziali): risolve in tempo polinomiale con **certificato di ottimalita' globale del sottoproblema**.

- Righe 1027-1029: se **res.problem**  $\approx 0$  stampa un warning ma **prosegue comunque**, e **value(X)** restituisce l'ultimo iterato. E' quello che salva il loop nelle iterazioni in cui il sottoproblema linearizzato risulta infeasible (perche' mancano i virtual control). Fragile, ma funzionante.

- Righe 1031-1039: spacchetta e ricostruisce la struct nel formato standard.

**Nota di performance:** i vincoli sono accumulati con `cstr = [cstr, ...]` dentro dei loop (crescita quadratica) e non c'è nessun `yalmip('clear')` fra una chiamata e l'altra, quindi il contatore interno di `sdpvar` cresce per tutte le 15 iterazioni. Con  $N = 50$  non è un problema (29 s end-to-end contro 82 s della variante `fmincon`), ma non scalerebbe.

## solve\_scvx\_yalmip (righe 1042-1121)

**Copia carbone** di `solve_scvx`, con la sola sostituzione della riga 485 (`solve_ltv_nlp`) con la riga 1086 (`solve_ltv_nlp_yalmip`). Stesso ratio test, stessi `eta_l = 0.25 / eta_h = 0.7`, stesso warm start trapezoidale, stessi raggi di trust region. È proprio questo che rende il confronto (b) vs (c) **pulito**: l'unica variabile che cambia è il solver interno.

Duplicazione di codice notevole (~80 righe quasi identiche). Un refactor con un handle al solver interno passato come argomento sarebbe stato più elegante; il codice ha scelto la chiarezza espositiva.

Unica differenza sostanziale: riga 1091 usa `ternary` (con il problema di valutazione eager discusso sopra) invece dell'`if/else` di `solve_scvx`.

## solve\_gfold\_socp (righe 1123-1196)

Il sottoproblema interno della variante (d). È il pezzo teoricamente più denso.

- Righe 1147-1148: le nuove variabili.

```
XI = sdpvar(5, N, 'full'); % [x; y; vx; vy; z], z = ln(m)
W  = sdpvar(3, N-1, 'full'); % [ux; uy; sigma]
```

Il controllo ha **tre** componenti: le due dell'accelerazione più lo **slack sigma**. Lo slack è una variabile di decisione a tutti gli effetti.

- Riga 1150:  $z_0 = \log(d.m_0)$  che vale **esattamente 0**, perché  $m_{0\_nd} = 1$ .
- Righe 1152-1153: BC. Notare il commento: **z\_N è libero** – è l'obiettivo.
- Riga 1156: **dinamica LTI**:

```
cstr = [cstr, XI(:,k+1) == Abar*XI(:,k) + Bbar*W(:,k) + cbar];
```

`Abar`, `Bbar`, `cbar` sono **le stesse per tutti i k** (senza indice!): il sistema è tempo-invariante. Calcolate una volta sola alla riga 1218 con `lti_zoh`.

Vale la pena esplicitare cosa sono davvero. In `lti_zoh.m`, `A` è **nilpotente** ( $A^2 = 0$ ), quindi  $\expm(A*s) = I + A*s$  **esattamente** e gli integrali si fanno a mano:

```
Abar = I + A*dt          -> doppio integratore ZOH
Bbar = (I*dt + A*dt^2/2) * B
cbar = (I*dt + A*dt^2/2) * c
```

che, componente per componente, danno

```
x_{k+1} = x_k + dt*vx_k + (dt^2/2)*ux_k
y_{k+1} = y_k + dt*vy_k + (dt^2/2)*(uy_k - 1)
vx_{k+1} = vx_k + dt*ux_k
vy_{k+1} = vy_k + dt*(uy_k - 1)
z_{k+1} = z_k - Vc*dt*sigma_k
```

Cioè: **le formule del moto uniformemente accelerato**, esatte, più una massa che decade esponenzialmente in modo esatto. Non c'è **nessun errore di discretizzazione**. Ecco perché la fedeltà del replay è  $7e-12$  (il floor dell'integratore) invece di  $1e-4$ – $1e-8$ .

- Riga 1157: il cono lossless:

```
cstr = [cstr, norm(W(1:2,k)) <= W(3,k)]; % |u| <= sigma
```

- Righe 1158-1159: il bound superiore linearizzato – l’unica non convessità residua:

```
ezr = exp(-z_ref(k));
cstr = [cstr, W(3,k) <= d.Tmax*ezr*(1 - (XI(5,k) - z_ref(k)))];
```

cioè  $\sigma_k \leq T_{\max} * e^{-z_{\text{ref},k}} * (1 - (z_k - z_{\text{ref},k}))$ , la tangente di  $T_{\max} * e^{-z}$  in  $z_{\text{ref}}$ . Sotto-stima il bound vero -> **conservativa** -> ogni iterato accettato è feasible per il problema originale. È l’**unico** motivo per cui serve ancora un loop SCvx.

- Righe 1162-1166: glide-slope (lineare) e bound sulla massa **in log**:  $\log(1e-3) \leq z \leq 0$ . **Fragile**: l’upper bound 0 è hard-coded assumendo `m0_nd = 1`; sarebbe più robusto scrivere  $\leq z_0$ .
- Righe 1168-1181: trust region, con raggi **su variabili diverse**: `pos`, `vel`, `lz` (in log-massa!), `u` (accelerazione), `sig`. Non sono i raggi della riga 49 – sono quelli della riga 1231.
- Riga 1183: `optimize(cstr, -XI(5,N))` -> **massimizza  $z_N$** , che per monotonia di `exp` equivale a massimizzare `m_N`.
- Righe 1184-1186: `res.problem ~= 0 && res.problem ~= 1`. Il flag 1 di YALMIP è “infeasible problem”: il codice lo **sopprime deliberatamente** e prosegue con l’iterato che ECOS restituisce. È l’ammissione implicita che manca il virtual control. Da dichiarare all’orale: è l’hack più discutibile del file.
- Righe 1188-1195: **back-transform** alle variabili originali:

```
m = exp(z), T = m * u
```

con l’ultimo nodo paddato a zero.

**Possibile domanda d’esame** – Se la dinamica GFOLD è esatta, perché serve ancora un loop SCvx? *Risposta*: perché resta una non convessità: il bound superiore di spinta  $|u| \leq T_{\max} * e^{-z}$ . La funzione  $e^{-z}$  è convessa, quindi vincolare  $\sigma$  sotto di essa definisce un insieme non convesso. Lo si linearizza attorno a  $z_{\text{ref}}$  e si itera. Ma è **un solo vincolo scalare per nodo**, non l’intera dinamica: per questo il loop converge in 3 iterazioni invece delle 15 delle varianti (b)/(c), e ogni passo è accettato con  $\epsilon \approx 1$ .

## solve\_gfold\_scvx (righe 1198-1283)

Il loop esterno della variante (d). Struttura identica alle altre due, con **tre differenze sostanziali**:

1. **Riga 1218**: `[Abar, Bbar, cbar] = lti_zoh(tf/(N-1), d.Vc)` – **una volta sola**, fuori dal loop. Un solo expm 9x9 contro le 15 iterazioni x 49 intervalli = 735 integrazioni ode45 70-D delle varianti (b)/(c). È da qui che viene il fattore ~10 sul tempo di calcolo (5 s contro 82 s / 29 s).
2. **Righe 1221-1229**: **e’ self-starting**. Nessun warm start trapezoidale. Il riferimento iniziale è:

```
m_apri = max(d.m0 - d.Vc*d.Tmax*t_grid, 1e-2);
ref.z = log(m_apri);
```

cioè il **profilo di massa analitico a spinta massima costante**:  $m(t) = m_0 - V_c * T_{\max} * t$  (integrale banale di  $\dot{m} = -V_c * T_{\max}$ ). È il profilo di consumo **più pessimistico possibile** – serve solo come punto attorno a cui linearizzare il bound di spinta, e siccome quel bound è l’unica cosa linearizzata, un riferimento grossolano basta. Posizione e velocità sono interpolate linearmente e `ref.ux/uy/sig` sono valori segnastati (0, 1, 1) che di fatto non vengono mai usati (all’iterazione 1 non c’è trust region; dalla 2 in poi `ref` è già il candidato accettato).

3. **Righe 1240-1248**: la prima iterazione è risolta **SENZA** trust region.

```
if iter == 1
    cand = solve_gfold_socp(tf,N,d,Abar,Bbar,cbar,ref.z,[],[]);
```

```
else
    ...
end
```

Il commento (righe 1241-1242) lo motiva: la dinamica e' esatta, quindi lasciare il SOCP libero di trovare una traiettoria dinamicamente feasible e' sicuro. E' l'opposto di quello che si farebbe con dinamica linearizzata (dove il primo passo senza trust region e' esattamente quello che esplode).

- Riga 1231: i raggi base **specifici di GFOLD**: pos 0.5, vel 1.0, lz 0.4, u 4.0, sig 4.0 – molto piu' larghi di quelli delle varianti (b)/(c), coerentemente col fatto che qui non c'e' errore di modello dinamico da cui difendersi.  $lz = 0.4$  in log-massa corrisponde a un fattore  $e^{0.4} = 1.49$ .
- Riga 1251: il ratio test usa **fwd\_integrate\_uacc** (replay ad accelerazione costante), non **fwd\_integrate**.
- **Onesta' sull'iterazione 1**:  $ref.m\_f = \exp(ref.z(N))$  con il profilo a spinta massima da' circa 0.40 non-dim, mentre l'ottimo vero e'  $\sim 0.70$ . Quindi al primo passo sia  $J\_pred$  sia  $J\_act$  sono grandi e positivi e  $eta \sim 1$  **quasi per costruzione**: il ratio test della prima iterazione non sta davvero verificando nulla. Diventa informativo dalla seconda in poi.

## fwd\_integrate\_uacc (righe 1285-1303)

Gemello di **fwd\_integrate**, ma il RHS e' **ode\_descent\_uacc** e il controllo tenuto costante e'  $u = T/m$ , non  $T$ . La dinamica replayata e':

```
vx_dot = ux
vy_dot = uy - 1
m_dot = -Vc * m * |u|      <- NOTA: dipende da m!
```

Con  $u$  costante, le righe di velocita' e posizione sono **integrabili esattamente** in forma polinomiale (moto uniformemente accelerato), e la massa decade **esponenzialmente**:  $m(t) = m_k * \exp(-Vc*|u|*(t - t_k))$ , cioe'  $z(t) = z_k - Vc*|u|*(t - t_k)$  – **lineare in  $z$** . Che e' esattamente cio' che la riga discreta  $z_{\{k+1\}} = z_k - Vc*dt*sigma_k$  prevede, a patto che  **$sigma_k = |u_k|$** .

Questo chiude il cerchio: **la fedelta'  $7e-12$  del replay GFOLD e' anche una verifica numerica indipendente che il cono lossless e' attivo**. Se  $sigma_k > |u_k|$ , il SOCP farebbe decadere  $z$  piu' in fretta di quanto il replay consegna, e l'errore sarebbe visibile. Non lo e'. Bel risultato da citare all'orale.

## plot\_compare5 (righe 1305-1420)

Come **plot\_compare3** piu' le varianti YALMIP (viola) e GFOLD (verde acqua). Le figure di convergenza SCvx sono **tre**, generate in un ciclo (righe 1377-1419), una per ciascun loop SCvx, cosi' da poterle confrontare a colpo d'occhio.

---

## Cosa misura davvero il confronto, e cosa emerge

Lo script produce **quattro** metriche per ogni trascrizione:

1. **Massa finale  $m\_f$**  (quindi carburante) – e' l'ottimalita'.
2. **Errore massimo ai nodi ( $node\_err$ )** – e' la **fedelta' di trascrizione**: quanto la traiettoria discretizzata si discosta da un'integrazione **ode45** degli stessi controlli.
3. **Dispersione al touchdown** (posizione, velocita', drift di massa) dal replay open-loop – e' la metrica operativa.
4. **Wall time e numero di iterazioni SCvx**.

I numeri (dal report, Tabelle **tab:hm2\_compare4** e **tab:hm2\_replay**,  $tf = 38$  s,  $N = 50$ ):



| Trascrizione                | m_f [kg] | Err. max nodi | Pos. touchdown | Wall [s] | Iter |
|-----------------------------|----------|---------------|----------------|----------|------|
| Trapezoidale (PWL)          | 1403.20  | 1.55e-3       | 4.3 m          | 90       | –    |
| ZOH nonlineare + RK4        | 1403.37  | 1.40e-8       | 6.1e-6 m       | 100      | –    |
| LTV + SCvx (fmincon)        | 1399.75  | 4.01e-4       | 0.18 m         | 82       | 15   |
| LTV + SCvx<br>(YALMIP/ECOS) | 1400.84  | 4.09e-5       | 0.019 m        | 29       | 15   |
| GFOLD log-mass              | 1402.77  | 7.30e-12      | 2.1e-8 m       | 5        | 3    |

Cosa emerge, in ordine di importanza:

- **Tutte e cinque trovano la stessa soluzione fisica.** m\_f varia di ~4 kg su 1400, cioè meno dello 0.3%. Le differenze sono sotto la precisione ingegneristica del modello (niente aerodinamica, niente assetto). Tutte esibiscono la struttura **max-coast-max** (bang-off-bang) prevista dal PMP.
- **Fedelta' e ordine di convergenza sono cose diverse dall'ottimalita'.** La trapezoidale ha m\_f fra le migliori ma la fedelta' peggiore (1e-3): il suo "ottimo" e' calcolato su una dinamica che vale solo a  $O(dt^2)$ , quindi e' leggermente **ottimistico**. La ZOH-RK4 e'  $O(h^4)$  e va a 1e-8.
- **Esporre la struttura conica paga.** Varianti (b) e (c) sono **matematicamente lo stesso sottoproblema**: (c) e' 3x piu' veloce e ha fedelta' un ordine di grandezza migliore, solo perche' il cono  $|T| \leq T_{max}$  e' passato a ECOS come primitiva invece di essere approssimato da SQP con Jacobiane alle differenze finite.
- **Il cambio di variabili batte l'iterazione numerica.** GFOLD e' l'unica variante che attacca la **causa** della non convessita' invece di gestirne gli **effetti**: dinamica esatta -> nessun errore di modello -> ogni passo accettato con **eta** ~ 1 -> 3 iterazioni contro 15 -> 5 s contro 82 s -> fedelta' al floor dell'integratore.
- **Entrambi i loop LTV+SCvx (b)/(c) NON convergono:** arrivano al cap di 15 iterazioni con  $||\delta x|| \sim 1e-2$ , sopra la tolleranza 1e-3. Il report lo dice apertamente. Non e' un disastro (ogni passo accettato ha superato il ratio test contro ode45, quindi l'iterato restituito e' realizzabile), ma non e' convergenza.

## Limiti, hack e deviazioni dalla teoria – da dichiarare

Elenco onesto, in ordine di gravita':

1. **Nessun virtual control, nessun buffer di slack sui vincoli di percorso.** E' la deviazione piu' seria dalla ricetta SCvx canonica. Il codice riconosce l'infeasibility del sottoproblema (righe 410, 1028) ma si limita a un warning; la variante GFOLD la **sopprime esplicitamente** (riga 1184: `res.problem ~= 1`). Il loop sopravvive solo grazie al warm start e al fatto che i solver restituiscono comunque il loro iterato meno infeasible.
2. **La regolarizzazione  $\sqrt{|T|^2 + 1e-6}$  nella riga della massa della Jacobiana** (riga 266). Introduce un bias  $O(\epsilon)$  proprio dove  $|T| \sim \epsilon$ , cioè sull'arco di coast, che e' anche dove il modello lineare fatica di piu'. La variante (d) elimina il problema alla radice.
3. **Il vincolo ridondante e non differenziabile  $T_{min} - |T| \leq 0$**  con  $T_{min} = 0$  (riga 814). Non vincola nulla, ma introduce un punto di non differenziabilita' in  $T = 0$  che fmincon deve attraversare con differenze finite. E' la causa radice della convergenza lenta documentata nel README.
4. **La baseline del ratio test e' ref.m\_f (valore NLP) sia al numeratore che al denominatore.** Rigorosamente, il numeratore dovrebbe usare la m\_f **replayata** del riferimento. L'errore di trascrizione del riferimento si infila in eta.
5. **ternary valuta eagerly entrambi i rami** (righe 1091, 1253): la divisione  $J_{act}/J_{pred}$  viene calcolata anche quando  $J_{pred} \sim 0$ . Innocuo in MATLAB, ma la guardia non guardia niente. `solve_scvx` usa un if/else corretto: le tre implementazioni dello stesso loop non sono coerenti.
6. **sol\_best non e' il migliore**, e' l'ultimo accettato (riga 535).
7. **Duplicazione massiccia:** `solve_scvx`, `solve_scvx_yalmip` e `solve_gfold_scvx` sono ~80 righe quasi identiche ciascuna. Idem `plot_compare3` / `plot_compare5`.
8. **Warm start fuori dai bound:** `ref_to_z` copia `ref.Tx(N)`, `ref.Ty(N)` dal riferimento trapezoidale, ma `box_bounds` li inchioda a  $[0, 0]$ . fmincon proietta silenziosamente.



9.  **$z \leq 0$  hard-coded** in `solve_gfold_socp` (riga 1166): assume `m0_nd = 1`. Fragile se si cambiasse lo schema di non-dimensionalizzazione.
10. **Commenti disallineati**: il banner alla riga 57 dice “three transcriptions” (sono cinque); il commento alla riga 38 attribuisce `scvx_max` alla sola variante (b) (vale per tutte e tre); il messaggio alla riga 130 dice “state error” ma `node_err` esclude la massa.
11. **tf e’ fissato**. Con `tf` libero il problema tornerebbe nonlineare anche in GFOLD; la strada canonica e’ una ricerca 1-D su `tf` risolvendo un SOCP per ogni valore. Non implementata.
12. **Il ticket T006 (single-SOCP lossless) resta aperto**. L’obiettivo finale – eliminare del tutto il loop SCvx e risolvere **un solo SOCP** con certificato di ottimalita’ globale – non e’ stato raggiunto: la variante (d) resta un loop SCvx perche’ il bound superiore di spinta e’ ancora linearizzato. Per chiuderlo servirebbe la sostituzione canonica (bound superiore approssimato con un’espansione al secondo ordine, o riformulazione che rende il vincolo convesso senza iterare).

## Possibili domande d’esame

**D: Il problema di powered descent e’ non convesso. Da dove viene esattamente la non convessita’, e come la affronti?** R: Da due sorgenti distinte. (1) **La dinamica**: i termini  $T_x/m$ ,  $T_y/m$  sono bilineari e il termine di massa  $\dot{m} = -V_c \cdot |T|$  mette una norma dentro un’uguaglianza, il che rende non convesso l’insieme delle traiettorie ammissibili. (2) **Il vincolo di spinta minima**  $T_{\min} \leq |T| \leq T_{\max}$ , che e’ un guscio anulare (nel mio caso  $T_{\min} = 0$ , quindi questa seconda sorgente non c’e’). I vincoli di percorso – glide-slope, bound superiore di spinta, box – sono invece **gia’ convessi**. Quindi il bersaglio e’ la dinamica: o la si linearizza e si itera (SCvx, varianti b/c), o la si rende lineare per **cambio di variabili** (GFOLD, variante d).

**D: Deriva la linearizzazione LTV e spiega perche’ il termine affine  $c(t)$  e’ indispensabile.** R: Espando  $f(x, u)$  al primo ordine attorno a  $(x_{\text{ref}}, u_{\text{ref}})$ :  $f \approx f(x_{\text{ref}}, u_{\text{ref}}) + A(x - x_{\text{ref}}) + B(u - u_{\text{ref}})$  con  $A = df/dx$ ,  $B = df/du$  valutate sul riferimento. Raccogliendo in variabili **assolute** ottengo  $\dot{x} \approx A x + B u + c$  con  $c = f(x_{\text{ref}}, u_{\text{ref}}) - A x_{\text{ref}} - B u_{\text{ref}}$ . Il termine  $c$  e’ la costante che fa passare l’iperpiano tangente **per il punto di riferimento**: se lo ometto, la retta tangente passa per l’origine e il modello e’ completamente sbagliato (ad esempio la gravita’, che e’ un termine costante di  $f$ , sparirebbe). Nel codice e’ la riga 296, e viene propagato nell’integrale **gamma** dell’ODE aumentata.

**D: A cosa serve la trust region, come e’ implementata nel tuo codice, e come viene aggiornata?** R: Serve contro l’**artificial unboundedness**: la linearizzazione e’ accurata solo localmente, e se lascio il sottoproblema convesso libero, questo “promette” un guadagno di massa che la dinamica vera non consegna (tipicamente sfruttando la riga singolare della massa, dove  $df/dT$  e’ quasi indefinita a spinta nulla). Nel mio codice e’ una trust region **hard** (non una penalita’ soft): un box in norma infinito, per-variabile e per-nodo, intersecato con i bound gia’ esistenti (`apply_trust`, righe 571-600), con raggi anisotropi (0.17, 0.6, 0.1, 1.0) non-dim e un fattore di scala adattivo  $\rho$ . L’aggiornamento e’ il **ratio test**: calcolo  $\eta = (\text{guadagno reale in } m_f) / (\text{guadagno predetto dal modello LTV})$ , dove il guadagno reale viene da un replay `ode45` della dinamica **nonlineare** con i controlli candidati. Se  $\eta < 0.25$  rifiuto e dimezzo  $\rho$ ; se  $0.25 \leq \eta < 0.7$  accetto e tengo  $\rho$ ; se  $\eta \geq 0.7$  accetto e raddoppio  $\rho$  (cappato a 1).

**D: Cosa sono i virtual control, perche’ servono, e li hai implementati?** R: Servono contro l’**artificial infeasibility**: il sottoproblema linearizzato puo’ risultare infeasible anche se il problema nonlineare e’ feasible, perche’ la dinamica lineare non riesce a “chiudere” fra stato iniziale assegnato e stato finale assegnato. La cura canonica e’ aggiungere alla dinamica discreta un termine libero  $u_k$  ( $x_{k+1} = A_{\text{bar}} x_k + B_{\text{bar}} u_k + \bar{c} + \nu_k$ ) e dei buffer  $w \geq 0$  sui vincoli di percorso, penalizzandoli in costo con una penalita’ esatta L1 ( $\lambda_{\text{nu}} \cdot |\nu|_1 + \lambda_{\text{w}} \cdot |w|_1$ ), i cui pesi si alzano finche’  $\nu$  e  $w$  si annullano alla convergenza. **Nel mio codice non ci sono**. Il loop sopravvive per due motivi: il warm start dalla soluzione trapezoidale mette il riferimento gia’ vicino all’ottimo, e quando il sottoproblema diventa infeasible il solver restituisce comunque il suo iterato meno infeasible, che poi il ratio test contro `ode45` vaglia prima di accettarlo. E’ un comportamento fragile e una versione di produzione dovrebbe portarli.

**D: Deriva il cambio di variabili GFOLD e spiega perche’ rende la dinamica esattamente LTI.** R: Pongo  $z = \ln(m)$ ,  $u = T/m$  (accelerazione di spinta) e introduco lo slack  $\sigma \geq |u|$ . Allora:  $\dot{v}_x = T_x/m$

$= u_x$  e  $v_{y\_dot} = T_y/m - 1 = u_y - 1$  diventano lineari **per costruzione** (il prodotto bilineare  $T/m$  e' la nuova variabile). Per la massa:  $z\_dot = m\_dot/m = -Vc*|T|/m = -Vc*|u|$ , e **sostituendo**  $|u|$  con  $\sigma$  ottengo  $z\_dot = -Vc*\sigma$ , lineare. Il sistema in  $[x, y, vx, vy, z]$  con controllo  $[u_x, u_y, \sigma]$  ha  $A, B, c$  **costanti**, quindi la ZOH dell'Appendice A si riduce a un singolo esponenziale di matrice (van Loan), calcolato una volta per tutta la griglia (`lti_zoh.m`). E siccome  $A$  e' nilpotente,  $\expm(A*dt) = I + A*dt$  esattamente: le equazioni discrete sono le formule del moto uniformemente accelerato, **senza nessun errore di discretizzazione**. E' per questo che il replay dei controlli GFOLD ha errore  $7e-12$ , cioe' il floor di `ode45`, invece dei  $1e-3$ - $1e-8$  delle altre trascrizioni.

**D: Perche' il rilassamento  $\sigma \geq |u|$  e' "lossless"?** R: Perche' all'ottimo il vincolo e' **attivo**,  $\sigma = |u|$ , quindi la soluzione del problema rilassato (convesso) e' anche soluzione del problema originale. Nel mio setup lo si vede in modo diretto: la riga discreta della massa e'  $z_{k+1} = z_k - Vc*dt*\sigma_k$  (esatta, perche' la quinta riga di  $A$  e' nulla), quindi  $z_N = z_0 - Vc*dt*\sum(\sigma_k)$ . Il costo e'  $\max z_N$ , cioe'  $\min \sum(\sigma_k)$ : ogni  $\sigma_k$  entra nel costo con coefficiente **positivo**, e negli altri vincoli compare solo come  $\sigma_k \geq |u_k|$  (limite inferiore) e  $\sigma_k \leq T_{max}*e^{-z}$  (limite superiore, che abbassare non viola). L'ottimizzatore lo schiaccia quindi sul suo limite inferiore  $|u_k|$ . Uno  $\sigma$  "in eccesso" brucerebbe carburante senza produrre accelerazione: e' strettamente sub-ottimo. **Attenzione:** questo argomento e' il caso facile ( $T_{min} = 0$ ). Il risultato forte di Acikmese & Ploen (2007) e' che il rilassamento resta lossless anche con  $T_{min} > 0$ , e li' la dimostrazione passa per il Principio del Massimo (un cono inattivo forzerebbe  $\lambda_m(tf) = 0$ , in contraddizione con la condizione di trasversalita').

**D: Se la dinamica GFOLD e' esatta e il cono e' lossless, perche' il tuo codice fa ancora un loop SCvx? Cosa servirebbe per arrivare a un singolo SOCP?** R: Resta **una** non convessita': il bound superiore di spinta.  $|T| \leq T_{max}$  con  $T = e^{-z} * u$  diventa  $\sigma \leq T_{max} * e^{-z}$ , e  $e^{-z}$  e' **convessa**: vincolare  $\sigma$  sotto una funzione convessa da' un insieme non convesso. Lo linearizzo attorno a  $z_{ref}$  (riga 1159), il che e' **conservativo** (la tangente di una funzione convessa sta sempre sotto la funzione, quindi ogni iterato accettato e' feasible per il problema vero), e itero finche'  $z_{ref}$  insegue la soluzione. Ma e' **un solo vincolo scalare per nodo**, non l'intera dinamica: infatti il loop converge in 3 iterazioni con  $\eta \approx 1$  a ogni passo, contro le 15 iterazioni non convergenti delle varianti (b)/(c). Per arrivare a un **singolo SOCP** (ottimo globale certificato, nessun warm start, nessuna iterazione) servirebbe la formulazione canonica GFOLD completa - e' il ticket T006, ancora aperto.

**D: Le varianti (b) e (c) risolvono lo stesso identico sottoproblema. Perche' allora danno risultati diversi?** R: Perche' lo risolvono in modo diverso. Il sottoproblema e' un SOCP: obiettivo lineare ( $\max m_N$ ), uguaglianze lineari (dinamica LTV discreta + condizioni al contorno), disuguaglianze lineari (glide-slope, box, trust region) e **un cono di Lorentz per nodo** ( $|T_k| \leq T_{max}$ ). La variante (c) lo passa a ECOS, che tratta il cono come **primitiva** e lo risolve con un metodo interior-point in tempo polinomiale. La variante (b) lo passa a `fmincon`/SQP, che **butta via** la struttura convessa: approssima il cono con successive linearizzazioni quadratiche e calcola le Jacobiane alle **differenze finite** (350 variabili -> ~350 valutazioni del `nonlcon` per gradiente). Il risultato: (c) e' ~3x piu' veloce end-to-end (29 s vs 82 s) e la fedelta' per-passo e' un ordine di grandezza migliore ( $4e-5$  vs  $4e-4$ ). Non e' un difetto di `fmincon` - e' che lo si sta usando fuori dalla sua classe.

**D: Le tue due varianti LTV+SCvx non convergono. E' un problema?** R: E' un limite dichiarato. Entrambe arrivano al cap di 15 iterazioni con  $||\delta x|| \sim 1e-2$ , sopra la tolleranza di  $1e-3$ . Le cause sono tre: (i) manca il virtual control, quindi quando il sottoproblema diventa infeasible il loop procede su un iterato non del tutto pulito; (ii) la regolarizzazione  $\sqrt{|T|^2 + 1e-6}$  della riga di massa introduce un bias  $O(\epsilon)$  proprio sull'arco di coast; (iii) il ratio test usa come baseline il valore NLP di  $m_f$  del riferimento invece del suo replay, quindi  $\eta$  porta dentro l'errore di trascrizione del riferimento. Detto questo, **ogni passo accettato ha superato il ratio test contro un'integrazione ode45 della dinamica nonlineare vera**, quindi l'iterato restituito e' fisicamente realizzabile: replayando i controlli si atterra a 0.18 m (`fmincon`) e 0.019 m (ECOS) dalla piazzola. La  $m_f$  finale differisce di 2-4 kg su 1400 dalle altre trascrizioni, cioe' lo 0.2%: e' un ottimo locale vicino, non un risultato sbagliato. La variante GFOLD, che rimuove le cause (ii) e in gran parte (i), converge in 3 iterazioni e atterra a  $2e-8$  m.

# HM2\_\_powered\_\_descent/proto\_gfold\_logma

## Ruolo del file nel progetto

Questo file e' il **prototipo standalone della trascrizione GFOLD log-mass**, cioe' la quarta (e ultima) trascrizione del Task 2 di HM2. E' uno script: si lancia da solo, risolve l'intero problema di powered descent in pochi secondi e stampa a schermo un blocco di validazione. L'header (righe 1-9) dichiara esplicitamente che il codice e' stato poi promosso dentro `main_task2.m` come "variant (d)" (`solve_gfold_scvx / solve_gfold_socp`), e che i due kernel numerici sono stati estratti in file autonomi – `lti_zoh.m` e `ode_descent_uacc.m` – condivisi fra il prototipo e `main_task2.m`. Il prototipo e' quindi **codice vivo ma ridondante**: serve da demo minimale, e l'header stesso dice "Safe to delete if the integrated version is enough". Onesta': nessun altro file lo chiama, e i test (`tests/gfoldLogMassTest.m`) testano i due kernel condivisi, *non* questo script.

Il problema risolto e' sempre quello della traccia HM2: punto materiale 2D, massa variabile, atterraggio pinpoint e soft da (1000, 3000) m con velocita' iniziale (300, -200) m/s, `tf = 38 s` fisso,  $0 \leq |T| \leq 70$  kN, corridoio glide-slope a 60 gradi, `Isp = 225 s`, costo = massimizzare la massa finale. Le altre tre trascrizioni (trapezoidale, ZOH+RK4 nonlineare, LTV+SCvx) attaccano il problema come un **NLP non convesso**. Questa lo attacca cambiando variabili in modo che il problema diventi (quasi) un **programma conico del second'ordine**.

L'idea, dovuta ad Acikmese/Ploen/Blackmore, e' in due mosse. Prima si sostituisce il controllo: invece del vettore spinta  $T$  si usa l'**accelerazione comandata**  $u = T/m$ , e invece della massa si usa il suo logaritmo  $z = \ln(m)$ . Con questa sostituzione la dinamica diventa **esattamente lineare tempo-invariante** in  $(r, v, z)$  con ingresso  $(u, \sigma)$  – non linearizzata: *esattamente* lineare. Seconda mossa: si introduce uno **slack scalare**  $\sigma$  con  $|u| \leq \sigma$  (vincolo conico, convesso) che rimpiazza la norma  $|u|$  dentro l'equazione della massa. Il rilassamento e' **lossless**: all'ottimo  $\sigma = |u|$  e quindi la soluzione del problema convesso rilassato risolve anche l'originale.

Il file usa questi due mattoni condivisi:

- `lti_zoh.m` – costruisce le matrici ZOH discrete ( $A_{bar}$ ,  $B_{bar}$ ,  $c_{bar}$ ) con un singolo `expm` (trucco di van Loan). Poiche' il sistema e' LTI, le matrici sono **costanti lungo tutta la griglia** e si calcolano **una volta sola** (riga 181).
- `ode_descent_uacc.m` – il right-hand side nonlineare "vero" con l'accelerazione  $u$  tenuta costante sull'intervallo; serve solo come *ground truth* per il replay `ode45` (righe 241-254), che misura la fedelta' della trascrizione.

Cio' che il codice **non** fa, e va detto subito: non risolve un unico SOCP. Il paper GFOLD linearizza il bound di spinta superiore attorno a un profilo di massa a-priori *fissato* e risolve **un solo** problema convesso, con certificato di ottimalita' globale. Qui invece la linearizzazione viene ri-centrata a ogni iterazione dentro un loop SCvx (righe 179-239). Il README di HM2 lo riconosce: il ticket "Lossless convexification -> single SOCP" e' ancora aperto.

---

## Header e cambio di variabile (righe 1-27)

```
% Change of variables (GFOLD, Acikmese/Blackmore):
% state   xi = [x; y; vx; vy; z],   z = ln(m)
% control w = [ux; uy; sigma],      u = T/m, sigma >= ||u||
```

- Righe 11-23: il commento dichiara il cuore teorico del file. Vale la pena derivarlo per intero, perche' e' il punto d'orale.

**Punto di partenza.** La dinamica non-dim in variabili originali (quella delle altre tre trascrizioni, `ode_descent.m`) e':

```
x' = vx
y' = vy
vx' = Tx/m
vy' = Ty/m - 1
m' = -Vc * |T|
```

dove  $|T| = \sqrt{Tx^2 + Ty^2}$ , la gravita' vale 1 in unita' non-dim (vedi la scelta delle scale, righe 103-113) e  $Vc = V_{rif}/c$  con  $c = Isp * g_0$  la velocita' efficace di scarico. Due nonlinearita': la **divisione per m** nelle righe di velocita' (bilineare in  $(T, 1/m)$ ) e la **norma** nella riga di massa. Sono entrambe *algebriche*, non dinamiche.

**Mossa 1 – cambio di controllo.** Si definisce l'accelerazione comandata

```
u = T/m      ->      T = m*u
```

La sostituzione e' invertibile finche'  $m > 0$ , quindi non si perde nulla. Le righe di velocita' diventano

```
vx' = ux
vy' = uy - 1
```

cioe' **non contengono piu' la massa**. Questa e' esattamente la proprieta' che il test `testAccelerationIsDirect` verifica.

**Mossa 2 – cambio di stato.** Si definisce  $z = \ln(m)$  (diffeomorfismo fra  $m > 0$  e  $z$  reale). Allora

```
z' = m'/m = -Vc*|T|/m = -Vc*m*u/m = -Vc*|u|
```

La massa **sparisce anche dalla riga di massa**. In variabili dimensionali lo stesso conto da'

```
d(ln m)/dt = -|u| / (Isp*g0) = -alpha*|u|,      alpha = 1/(Isp*g0)
```

e  $Vc$  non e' altro che  $\alpha$  reso non-dim ( $Vc = \alpha * V_{rif}$ , vedi riga 111: `dnd.Vc = ref.V/d.c`).

**Mossa 3 – slack.** Resta la norma  $|u|$ , che e' convessa ma **nonlineare**: una *uguaglianza* che contiene una funzione convessa nonlineare e' un vincolo non convesso. Si introduce lo scalare **sigma** e si scrive

```
z' = -Vc*sigma      (lineare)
|u| <= sigma        (cono del second'ordine, convesso)
```

Il sistema risultante e' affine (LTI + termine costante di gravita'):

```
xi' = A*xi + B*w + c
A: x<-vx, y<-vy   B: vx<-ux, vy<-uy, z<--Vc*sigma   c = [0;0;0;-1;0]
```

**Perche' e' LTI e non "linearizzato".** Non c'e' nessuno sviluppo di Taylor, nessun termine trascurato, nessun punto di lavoro.  $A, B, c$  sono matrici costanti che valgono per **qualsunque** stato e qualunque controllo, non solo vicino a una traiettoria di riferimento. Le variabili  $(u, \sigma)$  sono controlli *genuini* (l'inversa  $T = m*u$  e' esplicita), quindi la riscrittura e' un cambio di coordinate esatto sullo spazio stato-controllo. Contrasto con la variante (c), dove la linearizzazione LTV e' un'approssimazione al prim'ordine attorno alla traiettoria di riferimento, valida solo dentro la trust region: li' l'errore di linearizzazione esiste e il ratio test SCvx deve misurarlo.

- Righe 20-23: il commento e' preciso nel dire che **l'unica non convessita' residua** e' il bound superiore di spinta  $|u| <= T_{max} * \exp(-z)$ . Su questo torno alla sezione `solve_gfold_socp`.

**Possibile domanda d'esame** – Se il cambio di variabile e' esatto, dove e' finita la non convessita' del problema originale? *Risposta:* Si e' spostata dalla **dinamica** ai **vincoli**. Nel problema originale la non convessita' sta nelle equazioni di stato ( $T/m$  bilineare,  $|T|$  dentro un'uguaglianza). Dopo la sostituzione la dinamica e' convessa (affine), il costo e' lineare, glide-slope e quota sono lineari: resta solo  $|u| \leq T_{max} \exp(-z)$ , cioe' la regione **sotto** il grafico di una funzione convessa di  $z$ , che e' non convessa. Il guadagno e' netto: una singola disuguaglianza scalare non convessa al posto di 5N vincoli di uguaglianza non convessi.

---

## Dati del problema e non-dimensionalizzazione (righe 31-48)

- Righe 32-40: dati di Tabella 1 della traccia in SI.  $T_{min} = 0$  (riga 38): la traccia non impone una spinta minima. Questo ha una conseguenza importante che il codice non commenta – vedi sotto.
- Riga 42: `[ref, d] = nondim(d_si)` – scale di riferimento e struct non-dim.
- Riga 43-44: `tf = 38/ref.t`, `dt = tf/(N-1)` con  $N = 50$ . Griglia uniforme,  $N-1 = 49$  intervalli ZOH.
- Righe 46-48: stampa diagnostica delle scale.

Con i numeri della riga 32-40 le scale valgono  $L = 3000$  m,  $V = \sqrt{g \cdot L} \sim 171.6$  m/s,  $t = \sqrt{L/g} \sim 17.49$  s, quindi  $V_c \sim 0.0777$  e  $dt \sim 0.0444$  – sono esattamente i due valori hard-coded come costanti nel test (`gfoldLogMassTest.m`, righe 7-8).

**Possibile domanda d'esame** –  $T_{min} = 0$ : il rilassamento lossless serve ancora a qualcosa? *Risposta:* Si', ma per un motivo diverso da quello del paper. Nel paper lo slack serve soprattutto a convessificare il bound **inferiore**  $|T| \geq T_{min}$ , che e' la non convessita' piu' cattiva (un lower bound su una norma non e' mai convesso). Qui  $T_{min} = 0$  e quel vincolo non c'e'. Lo slack serve comunque perche' l'*uguaglianza* della massa  $z' = -V_c \cdot |u|$  contiene una norma: rilassarla in  $z' = -V_c \cdot \sigma$  con  $|u| \leq \sigma$  e' cio' che rende il vincolo dinamico affine. Onesta': questo codice **non esercita la meta' piu' spettacolare** della lossless convexification.

---

## Controllo disponibilita' solver (righe 50-53)

- Righe 51-53: se YALMIP (`sdpvar`) o ECOS non sono sul path, lo script muore con un **error**. Nessun fallback: a differenza di `main_task2.m` (che secondo il README salta con grazia le varianti coniche se mancano i pacchetti), qui la dipendenza e' dura. E' accettabile per un prototipo.
- 

## Chiamata al solver SCvx (righe 55-57)

- Riga 56: `max_iter = 20`, `tol = 1e-3`.
  - Riga 57: `sol = solve_gfold_scvx(...)`. Il secondo output `hist` della funzione (righe 197-198) esiste ma **viene scartato**: nessuno lo usa.
- 

## Validazione in variabili originali (righe 59-86)

```
[~, X_rep] = fwd_integrate_uacc(sol, d);    % replay nonlinear u-ZOH
pos_err = norm(X_rep(end,1:2)) * ref.L;
cone_gap = max(abs(sol.sig(1:N-1) - ...
    vecnorm([sol.ux(1:N-1) sol.uy(1:N-1)],2,2)));
```

- Riga 60: `dim_sol` riporta la soluzione in SI.

- Riga 61: **replay**. La soluzione ottima (le accelerazioni  $u_k$ ) viene rigiocata con `ode45` attraverso la dinamica **nonlineare** in  $(x, y, vx, vy, m)$ . E' il test di fedelta' della trascrizione: se il modello discreto mente, il replay atterra altrove.
- Righe 62-65: errori di touchdown in posizione [m] e velocita' [m/s], drift di massa finale modello-vs-replay [kg], errore nodo per nodo. Attenzione alla riga 65: `node_e` confronta **solo le colonne 1:4** (posizione e velocita'), la massa e' esclusa – esattamente come `node_err` in `main_task2.m` (righe 859-868). Il canale di massa e' coperto da `dmf` (riga 64), non dall'errore di nodo.
- Riga 69: margine glide-slope `tan(theta_mx)*y - |x| >= 0`.
- Riga 70: monotonia della massa `(diff(m) <= 1e-9)`.
- **Riga 71: cone\_gap** – la riga piu' importante del file. Misura `max_k |sigma_k - |u_k||`, cioe' **quanto e' attivo il rilassamento lossless**. Se questo numero e'  $\sim 1e-9$  o meno, allora  $\sigma = |u|$  a tutti i nodi e la soluzione del SOCP rilassato e' **anche soluzione del problema originale non convesso**. Nota che l'indice si ferma a  $N-1$ : il nodo  $N$  non ha controllo (i controlli sono  $N-1$ , uno per intervallo) ed e' stato riempito con zeri alla riga 173, quindi includerlo falserebbe il massimo.
- Righe 73-86: stampa. Riga 76: valori di riferimento delle altre trascrizioni, **hard-coded a mano** nel `fprintf` (non ricalcolati) – vanno letti come commento, non come output.
- Righe 79-80: check spinta con tolleranza  $+1 \cdot N$ . Riga 81-84: glide-slope e quota con tolleranza  $-1e-6$  non-dim.

**Perche' il replay deve tornare a macchina.** Nelle varianti (a)/(b)/(c) il replay `ode45` ha un errore residuo (la trascrizione approssima). Qui no: la mappa discreta e' la soluzione esatta della dinamica LTI con  $w$  costante, quindi *se* la dinamica vera vista dal replay e' quella con  $u$  costante (ed e' proprio cio' che fa `ode_descent_uacc`), l'unica discrepanza possibile fra modello e replay e'  $\sigma - |u|$ . In altre parole: `dmf` (riga 64) e `cone_gap` (riga 71) misurano **la stessa cosa** da due angoli diversi. Il README riporta per questa variante un errore di nodo di  $7.3e-12$  non-dim, cioe' il floor dell'integratore.

**Possibile domanda d'esame** – Come fai a sapere che il rilassamento e' lossless e non ti sei semplicemente risolto un problema *diverso* (piu' facile)? *Risposta:* Perche' `cone_gap` e' zero a precisione numerica:  $\sigma_k = |u_k|$  per ogni  $k$ . Se la disuguaglianza  $|u| \leq \sigma$  e' attiva ovunque, il vincolo  $z' = -Vc \cdot \sigma$  coincide con  $z' = -Vc \cdot |u|$ , cioe' la vera equazione della massa, e il bound  $\sigma \leq T_{max} \cdot \exp(-z)$  coincide con  $|u| \leq T_{max} \cdot \exp(-z)$ , cioe' il vero bound di spinta. Ogni vincolo del problema rilassato collassa sul vincolo originale, quindi la soluzione e' ammissibile per l'originale – ed essendo l'ottimo di un rilassamento (feasible set piu' grande), il suo costo e' un lower bound: coincidendo con un punto ammissibile dell'originale, e' l'ottimo globale dell'originale.

## nondim (righe 103-113)

- Riga 105-107: scale –  $L = y_0 = 3000$  m,  $g = 9.81$ ,  $t = \sqrt{L/g}$ ,  $V = \sqrt{g \cdot L}$ ,  $m = m_0$ ,  $T = m_0 \cdot g$ . La scala di accelerazione implicita e'  $L/t^2 = g$ , quindi  $u$  non-dim e' **letteralmente in “g”**.
- Riga 111: `dnd.Tmax = Tmax/(m0*g) = 70000/19620  $\sim 3.57$` , cioe' 3.57 g di accelerazione massima **alla massa iniziale**. E `dnd.Vc = ref.V/c`.
- Riga 110: `dnd.Tmin = 0/ref.T = 0`. **Variabile morta:** `d.Tmin` non compare mai nel SOCP (righe 125-177). Con `Tmin = 0` il vincolo sarebbe comunque vacuo ( $\sigma \geq 0$  e' gia' implicato da  $\sigma \geq |u| \geq 0$ ), ma il lettore va avvisato: se domani si volesse `Tmin > 0`, non basta cambiare `d_si.Tmin` – bisogna *aggiungere* il vincolo  $\sigma \geq T_{min} \cdot \exp(-z)$  al SOCP.
- Con  $m_0$  non-dim = 1, si ha  $z_0 = \ln(1) = 0$  (usato alla riga 132).

## dim\_sol (righe 115-123)

- Righe 117-122: ri-dimensionalizzazione. Riga 119-120:  $T_x = m \cdot u_x$ ,  $T_y = m \cdot u_y$  ricostruiti a monte (riga 174) e qui riscalati;  $T_{mag} = \sqrt{T_x^2 + T_y^2}$ .
- Attenzione al significato di `Tmag`:  $T = m_k \cdot u_k$  usa la massa al nodo **sinistro** dell'intervallo. Dentro l'intervallo  $u$  e' costante ma  $m(t)$  cala, quindi la spinta vera  $T(t) = m(t) \cdot u$  **decresce**. Il valore riportato

e' dunque il **massimo** della spinta sull'intervallo. Conseguenza elegante: se il bound e' rispettato ai nodi, e' rispettato **con continuita'** su tutto l'intervallo – non serve nessun vincolo inter-nodo per il bound superiore. (Il contrario varrebbe per un eventuale bound *inferiore*  $T_{min} > 0$ : li' il punto critico sarebbe la fine dell'intervallo, e il check ai nodi non basterebbe.)

## solve\_gfold\_socp (righe 125-177)

E' il sotto-problema convesso: un **SOCP** costruito con YALMIP e risolto da ECOS.

```
XI = sdpvar(5, N, 'full'); % [x; y; vx; vy; z]
W = sdpvar(3, N-1, 'full'); % [ux; uy; sigma] per intervallo ZOH
```

- Righe 127-128: variabili di decisione.  $5N + 3(N-1)$  scalari:  $250 + 147 = 397$ . Nota il **disallineamento voluto** stato/controllo:  $N$  nodi di stato,  $N-1$  controlli (uno per intervallo), com'e' giusto per un ZOH.
- Riga 130:  $z_0 = \log(d.m_0) = 0$ .
- Righe 132-133: condizione iniziale su tutte e 5 le componenti; condizione finale **solo su**  $XI(1:4,N) = 0$  (pinpoint + soft landing).  $z_N$  resta libera: e' la quantita' che si massimizza.

## Dinamica (riga 136)

```
XI(:,k+1) == Abar*XI(:,k) + Bbar*W(:,k) + cbar
```

Vincolo di **uguaglianza lineare**: e' cio' che rende il problema un SOCP e non un NLP.  $Abar$ ,  $Bbar$ ,  $cbar$  arrivano da `lti_zoh` e sono **le stesse per ogni k** (LTI). In un problema LTV avresti  $Abar_k$ , e dovresti ricalcolarle a ogni iterazione integrando l'ODE aumentata dell'Appendice A.

## Cono lossless (riga 137)

```
norm(W(1:2,k)) <= W(3,k) -> |u_k| <= sigma_k
```

E' un vincolo conico del second'ordine (SOC) in forma standard. Questo e' il rilassamento: al posto dell'uguaglianza non convessa  $z' = -Vc*|u|$  abbiamo l'uguaglianza lineare  $z' = -Vc*sigma$  (dentro  $Bbar$ ) piu' questa disuguaglianza convessa.

**Perche' e' lossless (l'argomento serio).** Il costo (riga 164) e'  $-z_N$ , e sommando la riga di massa lungo la griglia

```
z_N = z_0 - Vc*dt*sum_k sigma_k
```

quindi massimizzare  $z_N$  significa **minimizzare sum sigma\_k**. Lo slack ha dunque un costo strettamente positivo: l'ottimizzatore lo vuole il piu' piccolo possibile, e il suo unico lower bound e'  $|u_k|$ . L'intuizione "quindi  $sigma = |u|$  all'ottimo" e' giusta, ma **attenzione**: non e' un argomento di scambio banale, perche' abbassare  $sigma_k$  alza  $z_j$  per  $j > k$ , il che *restringe* il bound superiore ai nodi successivi (piu' massa = meno accelerazione disponibile). L'argomento rigoroso e' via PMP sul problema rilassato continuo: l'Hamiltoniana e'

```
H = lambda_r . v + lambda_v . (u + g) - alpha*lambda_z*sigma
```

e il controllo  $u$  compare **solo** nel termine  $lambda_v . u$ . Minimizzando  $H$  rispetto a  $u$  a  $sigma$  fissato, con  $|u| <= sigma$ :

```
u* = -sigma * lambda_v / |lambda_v| -> |u*| = sigma
```

ogni volta che il primer vector  $lambda_v$  e' diverso da zero. Il cono e' quindi attivo quasi ovunque, **indipendentemente** da come viene scelto  $sigma$ . Il caso patologico  $lambda_v = 0$  su un insieme di misura positiva viene escluso nel paper con un argomento di controllabilita'. Nota che questo argomento **non e' nel codice**: il codice si limita a *verificare a posteriori* la losslessness stampando `cone_gap` (riga 86).

Interpretazione fisica bonus:  $\sum sigma_k * dt \sim \text{integrale di } |u| dt = \text{integrale di } |T|/m dt$ , che per Tsiolkovsky vale  $\ln(m_0/m_f)/alpha$ , cioe' **il delta-V ideale**. Minimizzare il carburante e' *esattamente* minimizzare il delta-V – non un'analogia, un'identita'.

## Bound superiore di spinta linearizzato (righe 138-140)

```
ezr = exp(-z_ref(k));  
cstr = [cstr, W(3,k) <= d.Tmax*ezr*(1 - (XI(5,k) - z_ref(k)))];
```

Il vincolo **vero** è  $|T| \leq T_{\max}$ , cioè, diviso per  $m = \exp(z)$ :

```
sigma <= Tmax * exp(-z)
```

L'insieme  $\{(z, \sigma) : \sigma \leq T_{\max} \exp(-z)\}$  è la regione **sotto** il grafico di una funzione convessa: **non convesso** (prova: presi due punti sul grafico con  $z_1 \neq z_2$ , il loro punto medio ha  $\sigma = T_{\max} * (e^{-z_1} + e^{-z_2})/2 > T_{\max} * e^{-(z_1+z_2)/2}$  per stretta convessità, quindi cade fuori).

Il codice lo sostituisce con la **retta tangente** a  $\exp(-z)$  in  $z_{\text{ref}}$ :

```
sigma <= Tmax * e^{-z_ref} * (1 - (z - z_ref))
```

che è il Taylor al **prim'ordine**. Due osservazioni cruciali:

1. **E' un'approssimazione conservativa (inner), non una violazione.** Una funzione convessa sta sempre **sopra** la sua tangente:  $\exp(-z) \geq e^{-z_{\text{ref}}} * (1 - (z - z_{\text{ref}}))$  per ogni  $z$ . Quindi ogni punto che soddisfa il vincolo linearizzato soddisfa **anche** quello vero: la spinta  $|T| = m * |u| \leq m * \sigma \leq m * T_{\max} * e^{-z_{\text{ref}}} * (1 - (z - z_{\text{ref}})) \leq m * T_{\max} * e^{-z} = T_{\max}$ . **Il bound di 70 kN non può essere violato**, per costruzione, a qualunque iterazione. Ciò che si può perdere è ottimalità (si sta risolvendo un problema più stretto del vero).
2. **La conservatività è nulla in  $z = z_{\text{ref}}$  ed è tanto peggiore quanto più  $z$  si allontana da  $z_{\text{ref}}$ .** Da qui la necessità di ri-centrare la tangente, che è esattamente il compito del loop SCvx.

**Differenza dal paper.** Acikmese/Blackmore usano  $z_{\text{ref}}(t) = \ln(m_0 - \alpha * T_{\max} * t)$  (profilo di svuotamento a spinta massima) **fissato**, usano lo stesso Taylor al prim'ordine per il bound superiore, e un Taylor al **second'ordine** ( $\mu_1 * [1 - dz + dz^2/2]$ , un vincolo quadratico convesso, quindi SOC-rappresentabile) per il bound **inferiore**  $\sigma \geq T_{\min} \exp(-z)$ ; poi risolvono **un solo SOCP**. Questo codice: (a) non ha il bound inferiore ( $T_{\min} = 0$ , quindi il Taylor al second'ordine **non compare da nessuna parte**); (b) parte da quel  $z_{\text{ref}}$  del paper (riga 186) ma poi lo **aggiorna** iterativamente. Se qualcuno all'orale cita "Taylor al second'ordine come nel paper", la risposta onesta è: nel paper c'è, **in questo codice no**, perché serve solo per  $T_{\min} > 0$ .

## Vincoli di percorso (righe 143-147)

- Righe 143-145: glide-slope.  $|x| \leq \tan(\theta_{\text{mx}}) * y$  scritto come **coppia di disuguaglianze lineari**  $x \leq \tan * y$  e  $-x \leq \tan * y$ . Con  $\theta_{\text{mx}} = 60^\circ$ ,  $\tan = 1.732$ . Convesso e lineare – nessun costo aggiuntivo per il SOCP.
- Riga 146:  $y \geq 0$  (quota).
- Riga 147:  $\log(1e-3) \leq z \leq 0$ , cioè  $1e-3 * m_0 \leq m \leq m_0$ . Il bound superiore è fisico (non si guadagna massa). Il bound inferiore è un **floor numerico arbitrario** (2 kg): impedisce a  $z$  di divergere verso  $-\infty$  ma **non è una massa a secco**. Il problema, così com'è scritto, **non ha un vincolo di massa minima strutturale**: nulla garantisce che il propellente consumato sia fisicamente disponibile. Il paper usa invece un lower bound *tempo-variante*,  $z \geq \ln(m_0 - \alpha * T_{\max} * t)$ , che serve anche a limitare l'errore dell'approssimazione di Taylor.
- **Limite comune a tutte le trascrizioni per collocazione:** glide-slope e  $y \geq 0$  sono imposti **solo ai nodi**. Fra due nodi la traiettoria è una parabola e in linea di principio può uscire dal corridoio. Il codice non lo controlla.

## Obiettivo e risoluzione (righe 164-167)

- Riga 164: `optimize(cstr, -XI(5,N), ...)` – minimizza  $-z_N$ , cioè massimizza  $z_N$ . **Perché  $z_N$  e non  $m_N$ ?** Perché  $m_N = \exp(z_N)$  sarebbe una funzione nonlineare della variabile di decisione, mentre il SOCP vuole un obiettivo lineare (o al più convesso).  $\exp$  è monotona crescente, quindi  **$\arg\max z_N = \arg\max m_N$** : massimizzare il log-massa è *equivalente* a massimizzare la massa. Il cambio di variabile regala anche questo.



- Righe 165-167: se ECOS restituisce un flag  $\neq 0$  il codice **si limita a un warning e prosegue**. Su un problema infeasible `value(XI)` restituisce NaN, che poi si propaga silenziosamente nel replay e nei check. Fragilità nota: dovrebbe essere un **error** o almeno un rifiuto dell'iterazione.

## Estrazione della soluzione (righe 169-176)

- Riga 173: i controlli vengono **paddati con uno zero** al nodo N (`sol.ux = [Wv(1,:).' ; 0]`), perché ci sono N-1 controlli e N nodi. Conseguenze: (i) il `cone_gap` alla riga 71 deve escludere il nodo N (e lo fa); (ii) nel grafico `stairs` della spinta (riga 92) l'ultimo gradino **crolla a zero** – artefatto cosmetico, non un vero coast finale.
- Riga 174: `T = m .* u` con `m = exp(z)` nodo per nodo.

## `solve_gfold_scvx` (righe 179-239)

Loop esterno di successive convexification attorno al SOCP.

- **Riga 181:** `[Abar, Bbar, cbar] = lti_zoh(tf/(N-1), d.Vc)` – **una volta sola, fuori dal loop**. E' la firma della trascrizione LTI: nelle varianti LTV le matrici discrete vanno ricostruite a ogni iterazione, integrando l'ODE aumentata dell'Appendice A per ognuno dei 49 intervalli.
- Righe 185-192: **warm start**. `m_apri = max(m0 - Vc*Tmax*t, 1e-2)` e `ref.z = log(m_apri)`: e' proprio il profilo  $z_0(t) = \ln(m_0 - \alpha \cdot \rho_0 \cdot t)$  del paper, cioè il consumo che si avrebbe **spingendo al massimo per tutto il volo**. E' quindi una *sottostima* della massa vera (la soluzione ottima e' un bang-off-bang con un arco di coast – vedi README: max-coast-max), il che rende la prima tangente parecchio conservativa. Stato di riferimento: interpolazione lineare fra le condizioni al contorno; controllo di riferimento: `u = [0; 1]` (hover-ish, un g verso l'alto).
- Riga 194: dimensioni base della trust region (non-dim): `pos 0.5` (~1500 m), `vel 1.0` (~172 m/s), `lz 0.4`, `u 4.0` (4 g), `sig 4.0`. Sono **box molto larghi**.
- Riga 195: `rho = 1`, `rho_max = 1`, `rho_min = 1e-3`, `eta_l = 0.25`, `eta_h = 0.7`. Nota: **rho parte già al massimo**, quindi la riga 228 (`rho = min(rho_max, 2*rho)`) non può mai farla crescere. La trust region può solo **restringersi**.
- Righe 202-212: alla **prima iterazione** il SOCP viene risolto **senza trust region** (`ref_sol = []`, `trust = []`). Il commento (righe 203-206) spiega il perché ed e' corretto: la dinamica e' esatta, quindi non serve restare vicini al riferimento per fidarsi del modello; e il warm start e' cinematicamente incoerente (interpolazione lineare), quindi un box stretto attorno ad esso renderebbe il problema infeasible.
- Righe 214-217: **ratio test**. `J_pred = m_f(modello) - m_f(riferimento)`, `J_act = m_f(replay ode45) - m_f(riferimento)`, `eta = J_act/J_pred`.
- Riga 219: `delta` = norma della variazione di stato rispetto al riferimento.
- Righe 221-235: accetta se `eta >= 0.25`; se accettato aggiorna il riferimento, eventualmente allarga `rho` (di fatto mai, vedi sopra) e dichiara convergenza se `delta < tol = 1e-3`. Se rifiutato dimezza `rho` e si ferma se `rho < 1e-3`.

**Il ratio test qui e' quasi vuoto, e va detto.** L'unica sorgente di discrepanza fra modello discreto e replay nonlineare e' il gap del cono (`sigma - |u|`): le righe di posizione e velocita' sono riprodotte **esattamente** dal replay (perché `vx' = ux` e' vera anche nel modello nonlineare), e la riga di massa coincide non appena `sigma = |u|`. Ma `sigma = |u|` e' garantito **a ogni ottimo del SOCP**, per la struttura del costo. Quindi `eta ~ 1` già dalla prima iterazione, ogni passo viene accettato, `rho` non si restringe mai e la trust region non morde. In pratica **quello che fa convergere il loop non e' l'SCvx ma la ri-linearizzazione del bound di spinta**, e il criterio che conta e' `delta < tol` (punto fisso della tangente), non `eta`. Peggio: `eta` e' **strutturalmente cieco** all'errore che il loop sta davvero correggendo, perché l'errore di linearizzazione del bound `sigma <= Tmax*exp(-z)` non influenza la *fedeltà dinamica* (il replay non conosce i vincoli), solo la *feasibility/ottimalità*. In sostanza questo e' un **successive linear programming con safeguard**, non un SCvx nel senso pieno. Il README riporta convergenza in 3 iterazioni (~5 s), coerente con questa lettura.

**Possibile domanda d'esame** – Se la dinamica e' esatta, perché c'e' una trust region? *Risposta:* Non serve per la dinamica (nessun errore di linearizzazione lì), ma in linea di principio per l'unico

vincolo linearizzato,  $\sigma \leq T_{\max} \exp(-z)$ : la tangente e' affidabile solo vicino a  $z_{\text{ref}}$ , quindi limitare  $|z - z_{\text{ref}}|$  (`trust.lz`) limita la conservativita'/il salto fra iterazioni. In questo codice pero' il box e' largo e il ratio test non misura quell'errore, quindi la trust region e' di fatto inerte. Nel GFOLD "puro" non c'e' ne' trust region ne' loop: si accetta la conservativita' della tangente attorno al profilo a-priori e si risolve un SOCP solo.

---

## fwd\_integrate\_uacc (righe 241-254)

- Righe 244-245: parte dallo stato iniziale **esatto** (non da quello del SOCP).
  - Riga 246: `opts = odeset('RelTol',1e-10,'AbsTol',1e-12)` – la tolleranza stretta prevista dalle convenzioni della repo.
  - Righe 247-252: per ogni intervallo tiene l'**accelerazione**  $u_k$  costante e integra `ode_descent_uacc` con `ode45` usando quelle tolleranze.
  - La convenzione ZOH e'  $u$  costante, **non**  $T$  costante: dentro l'intervallo  $T(t) = m(t) \cdot u$  "galleggia" al calare della massa. E' la convenzione nativa del GFOLD ed e' esattamente quella che rende la mappa discreta esatta. Se si rigiocasse la soluzione tenendo  $T$  costante (`ode_descent.m`), il replay **non** tornerebbe: sarebbe un errore di consistenza, non un errore numerico.
- 

## ternary (righe 256-258)

- Helper cosmetico per gli `fprintf`. Valuta entrambi i rami (qui sono stringhe o scalari gia' calcolati, quindi innocuo). Usato anche alla riga 217 per proteggere la divisione  $J_{\text{act}}/J_{\text{pred}}$  quando  $|J_{\text{pred}}| < 1e-10$ .
- 

## Limiti noti, in chiaro

1. **Non e' il GFOLD del paper.** Il paper risolve un **SOCP** con certificato di ottimalita' globale; qui c'e' un loop SCvx di ri-linearizzazione. Il ticket "single SOCP" e' aperto nel README.
  2. **Tmin = 0:** il bound inferiore di spinta (la non convessita' piu' interessante, quella per cui la lossless convexification e' famosa) **non e' implementato**. `d.Tmin` e' calcolato e mai usato.
  3. **Nessuna massa a secco:** il floor  $z \geq \log(1e-3)$  e' numerico, non fisico.
  4. **Vincoli solo ai nodi:** glide-slope e  $y \geq 0$  non sono garantiti fra i nodi.
  5. **Errore di solver ignorato:** flag `ECOS != 0` produce solo un warning; i NaN si propagano.
  6. **rho non puo' crescere** (parte a `rho_max`); `hist` e' calcolato e scartato; il grafico `stairs` della spinta mostra un finto azzeramento all'ultimo nodo per via del padding.
  7. **Il ratio test non misura l'errore che il loop corregge** (vedi sopra).
- 

## Possibili domande d'esame

**D:** Deriva il cambio di variabile GFOLD e spiega perche' la dinamica risultante e' LTI e non semplicemente linearizzata. **R:** Si pone  $u = T/m$  (accelerazione comandata) e  $z = \ln(m)$ . Le righe di velocita'  $v' = T/m + g$  diventano  $v' = u + g$ : la massa scompare. La riga di massa  $m' = -\alpha \cdot |T|$  diventa  $z' = m'/m = -\alpha \cdot |T|/m = -\alpha \cdot |u|$ : la massa scompare di nuovo. Aggiungendo lo slack  $\sigma \geq |u|$  si ha  $z' = -\alpha \cdot \sigma$ , e il sistema in  $(r, v, z)$  con ingresso  $(u, \sigma)$  e' **affine a coefficienti costanti**:  $\dot{x} = A \cdot x + B \cdot w + c$ , con  $A, B, c$  indipendenti da stato e controllo. Non c'e' Taylor, non c'e' punto di lavoro, non c'e' resto trascurato: e' un cambio di coordinate esatto (invertibile finche'  $m > 0$ ), valido globalmente. Per questo la discretizzazione ZOH e' esatta e le matrici discrete sono **le stesse per tutti gli intervalli**, calcolate una volta con un `expm`.

**D: Il rilassamento con lo slack sigma: perche' e' lossless?** R: Perche' all'ottimo il cono e' attivo,  $|u| = \sigma$ , e quindi tutti i vincoli rilassati collasano su quelli originali. La dimostrazione pulita e' via PMP: nell'Hamiltoniana il controllo  $u$  compare solo nel termine  $\lambda_v \cdot u$ , quindi a  $\sigma$  fissato il minimo su  $\{|u| \leq \sigma\}$  si ottiene sul **bordo**,  $u^* = -\sigma \lambda_v / |\lambda_v|$ , cioe'  $|u^*| = \sigma$ , ogni volta che il primer vector  $\lambda_v$  non e' nullo (escluso su insiemi di misura positiva da un argomento di controllabilita'). Intuizione complementare:  $\sigma$  costa carburante (il costo e'  $-z_N = -z_0 + \alpha dt \sum \sigma_k$ ), quindi l'ottimizzatore lo schiaccia verso il basso finche' non sbatte contro  $|u|$ . Conseguenza pratica: la soluzione del problema **convesso** rilassato e' ammissibile per il problema **non convesso** originale e, in quanto ottimo di un rilassamento, ne e' l'ottimo **globale**. Nel codice questo si verifica alla riga 71/86: `cone_gap = max|sigma_k - |u_k||` deve essere zero a precisione numerica.

**D: Che fine ha fatto il vincolo non convesso  $T_{min} \leq |T| \leq T_{max}$ ? Come lo tratta il codice?** R: Diviso per la massa diventa  $T_{min} \exp(-z) \leq |u| \leq T_{max} \exp(-z)$ . Il lower bound su una norma non e' mai convesso: e' li' che lo slack fa il vero lavoro (si scrive  $\sigma \geq T_{min} \exp(-z)$ , convesso in  $(z, \sigma)$  perche' epigrafo di funzione convessa, e la losslessness restituisce  $|u| = \sigma$ ). **Ma questo codice ha  $T_{min} = 0$ , quindi quel pezzo non e' implementato:** `d.Tmin` e' una variabile morta. Resta l'upper bound  $\sigma \leq T_{max} \exp(-z)$ , non convesso perche' e' la regione sotto il grafico di una funzione convessa. Il codice (riga 140) lo sostituisce con la **tangente** in `z_ref`:  $\sigma \leq T_{max} e^{-z_{ref}} (1 - (z - z_{ref}))$ . Essendo  $\exp(-z)$  convessa, la tangente le sta sempre **sotto**: il vincolo linearizzato e' piu' stretto del vero, quindi **conservativo ma mai violato** (i 70 kN sono garantiti per costruzione). Il loop SCvx (righe 179-239) ri-centra `z_ref` sulla soluzione corrente per azzerare la conservativita' al punto fisso.

**D: Il paper usa un Taylor al second'ordine. Dov'e' nel codice?** R: Non c'e'. Il Taylor al second'ordine del paper serve per il bound **inferiore**  $\sigma \geq T_{min} \exp(-z)$ : si approssima  $\exp(-z)$  dall'alto con  $e^{-z_0} (1 - dz + dz^2/2)$ , che e' un vincolo quadratico convesso (SOC-rappresentabile). Con  $T_{min} = 0$  quel vincolo non esiste e il codice usa **solo** la tangente (prim'ordine) sul bound superiore. Dire il contrario all'orale sarebbe raccontare il paper, non il codice.

**D: Perche' lti\_zoh (discretizzazione esatta con expm) e non un RK4 come nella variante (b)?** R: Perche' qui il sistema e' davvero LTI e il controllo e' ZOH: la soluzione dell'intervallo e' nota in forma chiusa,  $x_{k+1} = e^{A \Delta t} x_k + (\int_0^{\Delta t} e^{A s} ds) (B w_k + c)$ , ottenibile con un solo `expm` del sistema aumentato (van Loan). Un RK4 sarebbe un'approssimazione di qualcosa che si sa risolvere **esattamente**: errore locale  $O(\Delta t^5)$  invece di zero, piu' costo, piu' Jacobiani. In piu'  $A$  e' nilpotente ( $A^2 = 0$ ), quindi la serie si tronca e le matrici discrete sono le formule elementari del moto uniformemente accelerato:  $\bar{A} = I + A \Delta t$ ,  $\bar{B} = \Delta t B + (\Delta t^2/2) A B$ ,  $\bar{c} = \Delta t c + (\Delta t^2/2) A c$ . Nella variante (b), invece, con la **spinta** (non l'accelerazione) tenuta costante,  $v = T/m(t)$  varia dentro l'intervallo: la mappa ZOH non e' piu' affine nelle variabili di decisione, serve integrazione numerica o una linearizzazione LTV, e le Jacobiane si portano dietro la massa. Guardando `main_task2.m:jacobians` (righe 255-277): le **righe di velocita'** di  $df/dx$  vanno come  $1/m^2$  ( $A_{jac}(3,5) = -T_x/m^2$ ,  $A_{jac}(4,5) = -T_y/m^2$ , righe 270-271) e come  $1/m$  in  $df/du$ ; la **riga di massa**  $m' = -Vc/|T|$  non contiene affatto  $m$  (Jacobiano rispetto a  $m$  identicamente nullo), ma e' **non differenziabile in  $T = 0$** , ed e' quella la singolarita' che il codice tampona con  $T_{mag\_reg} = \sqrt{T_x^2 + T_y^2 + 1e-6}$  (riga 266) prima di usarla in `B_jac(5,1:2)`. E' l'epsilon della "epsilon-regularised singular mass row" citata nell'header (righe 17-19), e la trascrizione GFOLD la elimina alla radice: con  $z' = -Vc \sigma$  la riga di massa e' lineare e non c'e' nulla da regolarizzare.

**D: Perche' l'obiettivo e'  $-z_N$  e non  $-m_N$ ?** R: Perche'  $m_N = \exp(z_N)$  e' una funzione nonlineare della variabile di decisione, e il SOCP richiede un obiettivo lineare/convesso. Essendo `exp` strettamente monotona,  $\argmax z_N = \argmax m_N$ , quindi massimizzare il log-massa e' *equivalente* a massimizzare la massa finale. In piu'  $z_N = z_0 - Vc dt \sum \sigma_k$  rende esplicito che minimizzare il carburante equivale a minimizzare  $\sum \sigma_k$ , cioe' (per Tsiolkovsky) il **delta-V ideale**.

**D: Come verifichi che la trascrizione sia fedele, e cosa ti aspetti di vedere?** R: Con un replay open-loop: si prendono le accelerazioni ottime `u_k`, si integrano con `ode45` (tolleranze  $1e-10/1e-12$ ) attraverso la dinamica **nonlineare** in massa (`ode_descent_uacc`, riga 249) e si confrontano gli stati nodo per nodo. Poiche' la mappa discreta e' esatta, ci si aspetta un errore al **floor dell'integratore** (il README riporta  $7.3e-12$  non-dim), contro l' $O(\Delta t^2)$  della trascrizione trapezoidale. Attenzione pero' a *cosa* misura ciascuna metrica. L'errore di nodo (riga 65) guarda solo posizione e velocita', e quelle righe non contengono  $\sigma$ : sono riprodotte esattamente dal replay **comunque**, quindi sono cieche a un cono non attivo. La patologia  $\sigma > |u|$  (rilassamento non lossless) si

manifesta nell'unico canale che la vede, la massa: `dmf` (riga 64) e `cone_gap` (riga 71) misurano quella, da due angoli diversi.

# HM3/load\_\_hw3\_\_params.m

## Ruolo del file nel progetto

E' il **punto unico di verita' numerica** di tutto HM3. Restituisce una struct `p` con i coefficienti del modello pitch-plane del lanciatore fittizio *Greensite* congelati all'istante di massima pressione dinamica ( $t = 72$  s, il punto strutturalmente critico dell'ascesa). Ogni altro file di HM3 parte da qui: `build_plant_rigid(p)`, `build_plant_full(p)`, `build_tvc(p)`, `build_notch_filter(p)`, `load_wind_profile(p)`, i tre `main_task*.m`, il Monte Carlo e l'estensione LPV in `LTV_FULL_ASCENT/`.

Il file fa tre cose in cascata:

1. scrive i **letterali della Table 1** della traccia (geometria, masse, forze, coefficienti aero/controllo, modo di bending, sensore INS, attuatore TVC);
2. se il dataset di riferimento `General/hw3-v3/GreensiteLPV_DATA.mat` e' presente, **sovrascrive i coefficienti tempo-varianti** interpolandoli a `t_ref` – la Table 1 resta come fallback e come cross-check;
3. applica gli **scaling di incertezza** `mu_alpha_scale` / `mu_c_scale` usati nelle quattro corner di Task 3 (+/-30 % su `mu_alpha` e `mu_c`).

Il parametro `t_ref` (default 72) e' quello che trasforma questa funzione da “caricatore di una tabella” a **griglia LPV**: `LTV_FULL_ASCENT/main_full_ascent.m` la chiama in ciclo su `t_ref = 5...140` s (griglia `tm = t0:2.5:Tstop` con `t0 = 5` s, `Tstop = 140` s, default di `init_simulink_lpv.m`) per costruire il modello tempo-variante di tutta l'ascesa. La stessa funzione serve quindi sia il design frozen-time (HM3 propriamente detto) sia l'estensione fuori-traccia.

I valori interpolati dal dataset LPV a  $t = 72$  s (verificati eseguendo il codice) sono:

```
A6 = 3.381828    K1 = 4.564724    a1 = -0.015423
a3 = 20.609026   a4 = -27.270977   V  = 937.7087 m/s
Tc = 1.5213e6 N  wBM = 18.9 rad/s  sigma_ins = 0.178   phi_ins = 0.8
phi_tvc = 4.309544e-05 1/kg      rho = 0.18453    qbar = 81129 Pa
```

cioe' coincidono con la Table 1 fino alla 4a-5a cifra: e' esattamente il cross-check che il test `tests/hm3PlantTest.m` (righe 23-31) pinna, con tolleranze diverse per grandezza – `AbsTol = 5e-3` su `A6` e `K1` (righe 26-27), `0.5` su `V`, `0.05` su `wBM`, `5e-2` su `a4` (righe 28-30).

---

## load\_hw3\_params – firma e blocco arguments (righe 1-23)

```
function p = load_hw3_params(opt)
arguments
    opt.mu_alpha_scale (1,1) {mustBeNumeric, mustBeReal} = 1.0
    opt.mu_c_scale     (1,1) {mustBeNumeric, mustBeReal} = 1.0
    opt.t_ref          (1,1) {mustBeNumeric, mustBeReal} = 72
end
```

- Riga 1: firma con **solli argomenti name-value** (opt). Chiamata nominale: `p = load_hw3_params()`. Chiamata di corner: `load_hw3_params('mu_alpha_scale',1.3,'mu_c_scale',0.7)`.
- Righe 19-23: la validazione qui c'è (è un helper di boundary, chiamato una volta per run e non dentro un loop di ottimizzazione), coerente con la convenzione del repo.
- Righe 3-16: l'header documenta i campi. Attenzione: la riga 9 dichiara **A6,K1,a1,a3,a4** tutti in  $[1/s^2]$ . **Non è vero dimensionalmente**: vedi il blocco successivo. È un'imprecisione del commento, non del codice.

## Letterali Table 1: geometria, masse, forze (righe 25-35)

```
p.m      = 7.38e4;    % kg
p.l_alpha = 10.39;    % m
p.l_c     = 9.84;     % m
p.Iyy     = 3.28e6;   % kg m^2
p.Alt     = 15143;    % m
p.Tt_minus_D = 1.71e6; % N
p.N_alpha = 1.07e6;   % N/rad
```

- Righe 30-31: **l\_alpha** è la **distanza fra centro di pressione e baricentro** (braccio della forza normale aerodinamica), **l\_c** la distanza fra il **punto di gimbal** del motore e il baricentro (braccio del controllo). Sono i due bracci che generano i due momenti in competizione.
- Riga 35: **N\_alpha** è la derivata della **forza normale** rispetto all'angolo d'attacco,  $dN/d\alpha$  [N/rad]. È il tipico  $\bar{q} \cdot S \cdot C_{N_\alpha}$ .

Da questi numeri si ricostruiscono i coefficienti della Table 1 – ed è la derivazione da saper rifare a mente:

```
A6 = mu_alpha = N_alpha * l_alpha / Iyy
    = 1.07e6 * 10.39 / 3.28e6 = 3.39 1/s^2      (tab: 3.3818)

K1 = mu_c      = Tc * l_c / Iyy
    = 1.52e6 * 9.84 / 3.28e6 = 4.56 1/s^2      (tab: 4.5647)

a1 = -N_alpha / (m*V)
    = -1.07e6 / (7.38e4 * 937.7) = -0.01546 1/s  (tab: -0.0154)

a3 = Tc / m = 1.52e6 / 7.38e4 = 20.60 m/s^2 per rad  (tab: 20.6090)
```

Il significato: **A6** (=  $\mu_{\alpha}$ ) è l'**accelerazione angolare per unità di angolo d'attacco** prodotta dalla forza normale aerodinamica applicata **davanti** al baricentro; **K1** (=  $\mu_c$ ) è l'**efficacia di controllo** del TVC, cioè l'accelerazione angolare per unità di deflessione dell'ugello. Il rapporto  $A6/K1 = 0.74$  è la misura di quanto controllo serve per contrastare l'instabilità aerodinamica: sono le due sole quantità che `design_controller.m` legge dal plant (righe 44-45 di quel file) per calcolare i guadagni di partenza  $K_p = 2 \cdot A6/K1$ ,  $K_d = \sqrt{A6}/K1$ .

- **Nota di unità (il commento della riga 9 sbaglia)**. Solo **A6** e **K1** sono in  $1/s^2$  (per radiante). **a1** =  $-N_{\alpha}/(m \cdot V)$  ha unità  $1/s$  (moltiplica  $\dot{z}$  [m/s] e deve dare  $m/s^2$ ); **a3** e **a4** sono in  $m/s^2$  per radiante (moltiplicano  $\delta$  e  $\theta$ ). Il fatto che il codice funzioni comunque dipende dal fatto che i numeri entrano nelle matrici A/B alle righe giuste, non dal commento.

**Possibile domanda d'esame** – Perché  $A6 > 0$  implica instabilità statica? *Risposta*:  $A6 = N_{\alpha} \cdot l_{\alpha} / I_{yy}$  è positivo solo se la forza normale aerodinamica agisce **davanti** al baricentro (centro di pressione a monte del CG,  $l_{\alpha} > 0$  col segno della traccia). In quella configurazione un angolo d'attacco positivo genera un momento che **aumenta** l'angolo d'attacco: feedback positivo. La dinamica rotazionale isolata diventa  $\ddot{\theta} = A6 \cdot \theta$ , cioè  $s^2 - A6 = 0$ , con un polo reale a  $+\sqrt{A6} = +1.84$  rad/s. Nessun lanciatore snello è staticamente stabile in volo atmosferico: la stabilità è un prodotto del controllo, non della configurazione.

---

## L'incongruenza su a4 (righe 36-38, 44)

```
% a4 inconsistency: -(Tt-D)/m = -23.17 1/s^2 from the values above, but the
% table lists a4 = -27.2710. The LPV set agrees with -27.2710, so that is
% what enters the dynamics.
p.a4 = -27.2710;
```

- Righe 36-38: il commento e' onesto ed e' **corretto**. La formula strutturale del coefficiente e'  $a4 = -(T - D)/m$ ; con i valori di tabella ( $Tt\_minus\_D = 1.71e6$  N,  $m = 7.38e4$  kg) si ottiene -23.17, mentre la Table 1 dichiara -27.2710. Per ottenere -27.2710 servirebbe  $T - D = 27.271 * 7.38e4 = 2.01e6$  N, cioe' ~18 % in piu' della spinta netta dichiarata.
- Il codice **non** risolve l'incongruenza: usa -27.2710 perche' e' quello che il dataset LPV conferma ( $a4 = -27.270977$  a  $t = 72$  s). E' la scelta giusta (il dataset e' la fonte del professore), ma va detto all'orale: la Table 1 non e' internamente consistente sulla riga di  $Tt - D$ , oppure  $Tt\_minus\_D$  e' tabulato al netto di qualcos'altro (per es. la sola spinta al vuoto contro la spinta reale, o una D valutata diversamente).
- La combinazione che entra davvero nella dinamica laterale e'  $a1*V + a4 = -14.46 - 27.27 = -41.73$  m/s<sup>2</sup> per rad, che con la formula strutturale varrebbe  $-[(T-D) + N\_alpha]/m = -(1.71e6 + 1.07e6)/7.38e4 = -37.7$ . La differenza (~10 %) sposta di poco i poli ma **non** cambia la fisica.

---

## Bending, sensore INS, attuatore TVC (righe 47-57)

```
p.wBM      = 18.9;      % rad/s
p.zBM      = 0.005;     % -
p.phi_ins  = 0.8;      % -
p.sigma_ins= 0.178;     % rad/m
p.phi_tvc  = 4.31e-5;   % 1/kg
p.wTVC     = 70;      p.zTVC = 0.7;  p.tau = 0.020;
```

- Righe 48-49: primo modo di **flessione** del corpo.  $zBM = 0.005$  e' uno smorzamento strutturale bassissimo (0.5 %): il fattore di qualita' e'  $Q = 1/(2*zBM) = 100$ , cioe' **+40 dB** di amplificazione alla risonanza rispetto al guadagno statico. E' la ragione per cui il modo di bending non si puo' ignorare anche se il suo guadagno DC e' piccolo.
- Righe 51-52: coefficienti della **Eq. (2)** della traccia, l'osservazione INS.  $\sigma\_ins$  [rad/m] e' la **pendenza** della forma modale alla stazione dell'INS (contamina  $\theta\_m$ ),  $\phi\_ins$  [-] e' lo **spostamento** modale alla stessa stazione (contamina  $z\_m$ ). Sono i due numeri che rendono il modo di bending **visibile in retroazione**: senza di loro il loop non lo vedrebbe affatto (vedi la pagina di `build_plant_full.m`).
- Riga 54:  $\phi\_tvc$  [1/kg] e' il **forcing** del bending da parte del TVC: moltiplicato per la spinta di controllo  $Tc$  da' l'accelerazione modale per radiante di deflessione,  $\phi\_tvc*Tc = 65.56$  m/s<sup>2</sup> per rad.
- Righe 55-57: attuatore TVC di **Eq. (3)** – secondo ordine con  $wTVC = 70$  rad/s (ben oltre la banda di controllo ~2-3 rad/s e anche **sopra** il triplo di  $wBM$ :  $3*18.9 = 56.7$  rad/s < 70, quindi l'attuatore da solo non attenua la risonanza strutturale – il roll-off utile arriva dal notch),  $zTVC = 0.7$ , piu' un ritardo puro  $\tau = 20$  ms che in `build_tvc.m` viene approssimato con un Pade.

---

## Override dal dataset LPV (righe 59-80)

```
datafile = fullfile(fileparts(mfilename('fullpath')), ...
                    'General', 'hw3-v3', 'GreensiteLPV_DATA.mat');
if isfile(datafile)
```

```

S = load(datafile); L = S.GreensiteLPV;
at = @(ts) interp1(ts.Time, squeeze(ts.Data), opt.t_ref);
p.A6 = at(L.A6); ... p.wBM = at(L.omega);
p.src = 'GreensiteLPV_DATA.mat @ t=72 s';
else
p.src = 'Table 1 literals (data file not found)';
end

```

- Righe 60-61: il path e' costruito con `fileparts(mfilename('fullpath'))`, quindi **relativo al file, non alla cwd**: la funzione si puo' chiamare da qualunque directory (i main di LTV\_FULL\_ASCENT/ lo fanno).
- Riga 65: `at` e' la closure di interpolazione. `squeeze(ts.Data)` e' **difensivo**: nel dataset i campi `timeseries` di `GreensiteLPV` hanno `Data` gia' come vettore colonna `Nt x 1`, quindi qui `squeeze` e' un no-op – servirebbe solo se `Data` arrivasse nel layout `1 x 1 x Nt` (quello tipico dei log Simulink, dove infatti la stessa chiamata ricorre in `run_full_ascent_simulink.m`). `interp1` di default e' **lineare e senza extrapolation**: se `t_ref` cade fuori dalla griglia temporale del dataset, i coefficienti tornano `NaN` silenziosamente (nessun errore). E' un limite reale della funzione quando la si usa come griglia LPV.
- Righe 66-76: cosa viene **sovrascritto** – `A6`, `K1`, `a1`, `a3`, `a4`, `V`, `Tc`, `sigma_ins`, `phi_ins`, `phi_tvc`, `wBM` (da `L.omega`). Cosa **resta letterale** – `zBM`, `wTVC`, `zTVC`, `tau`, `m`, `Iyy`, `Alt`, `l_alpha`, `l_c`, `N_alpha`, `Tt_minus_D`. Quindi in un sweep su `t_ref` la massa, l'inerzia e soprattutto l'**altitudine restano congelate a quelle di t = 72 s**, mentre `V` varia: la `qbar` calcolata sotto e' corretta solo al punto di progetto.
- Righe 77-79: `p.src` e' un flag di provenienza stampato dai main – utile all'orale per dire “questi numeri vengono dal dataset del professore, non li ho ricopiati a mano”.

**Possibile domanda d'esame** – Se hai gia' la Table 1, perche' leggere il `.mat`? *Risposta*: per tre motivi. (1) Cross-check: se i due insiemi coincidono a 4-5 cifre (e coincidono, il test lo pinna), la trascrizione della tabella e' giusta. (2) Precisione: la tabella e' arrotondata a 4 cifre, il dataset no. (3) Riutilizzabilita': la stessa funzione con `t_ref` variabile diventa la griglia LPV di tutta l'ascesa, che e' quello che serve all'estensione LTV\_FULL\_ASCENT/. La Table 1 rimane come fallback se il file non c'e'.

## Pressione dinamica derivata (righe 82-84)

```

p.rho = 1.225 * exp(-p.Alt/8000); % kg/m^3
p.qbar = 0.5 * p.rho * p.V^2; % Pa

```

- Riga 83: **atmosfera esponenziale**, la stessa convenzione usata in HM0 (`rho0 = 1.225 kg/m^3`, `Hscale = 8000 m`). A `Alt = 15143 m` da' `rho = 0.1845 kg/m^3`.
- Riga 84: `qbar = 0.5*0.1845*937.71^2 = 81.1 kPa`. E' il numero con cui i main convertono l'angolo d'attacco nell'indicatore di carico `qbar*alpha [kPa*deg]`, la quantita' dimensionante al max-q.
- **Caveat**: `qbar` e' calcolato con `p.Alt` letterale (mai aggiornato dal dataset LPV) e `p.V` interpolato. Per `t_ref = 72` e' esatto; per qualunque altro `t_ref` la densita' e' quella sbagliata. Lo stesso vale per la dispersione del vento in `load_wind_profile.m` (riga 43), che cerca `sigma` nel dataset `drywind` all'altitudine `p.Alt/1000 = 15.1 km` sempre.

## Scaling di incertezza Task 3 (righe 86-92)

```

p.A6 = p.A6 * opt.mu_alpha_scale; % mu_alpha
p.K1 = p.K1 * opt.mu_c_scale; % mu_c

```

- Righe 89-90: le uniche due quantita' scalate. **`a1`, `a3`, `a4`, `V`, `Tc` non vengono toccati** – il test `testUncertaintyScalingAppliesToA6K1` (righe 40-47) verifica esplicitamente che `a3` resti invariato.



- Le corner di Task 3 (+/-30 % su ciascuno, 4 vertici: V1-V4 in `main_task3.m`) agiscono quindi solo sulla **equazione dei momenti**: si perturba il momento aerodinamico destabilizzante e l'efficacia del controllo, non le forze. Fisicamente equivale a un'incertezza sul **braccio** (spostamento del centro di pressione, errore sulla posizione del gimbal / sull'allineamento dell'ugello), non sul modulo di `N_alpha` o della spinta. Una perturbazione "vera" del +30 % su `N_alpha` cambierebbe anche `a1` (che ne dipende linearmente) e quindi il termine di drift.
- E' comunque **quello che chiede la traccia**, che definisce l'incertezza su `mu_alpha` e `mu_c`; ma conviene saperlo dire, perche' e' esattamente la domanda "e se ti muovo anche `a1`?".
- **Ordine delle operazioni**: lo scaling avviene **dopo** il calcolo di `qbar` (righe 82-84), quindi `p.qbar` resta il valore nominale anche nelle corner – il che e' corretto (la pressione dinamica non dipende da `mu_alpha`) e rende confrontabili i carichi `qbar*alpha` fra i vertici.
- Righe 87-88: i due fattori vengono anche **memorizzati** nella struct, cosi' i main possono etichettare le figure con la corner corrispondente.

## Possibili domande d'esame

**D: Cosa sono fisicamente `mu_alpha` e `mu_c`, e perche' HM3 li tratta come le due uniche grandezze incerte?** R: `mu_alpha` =  $A6 = N\_alpha * l\_alpha / I_{yy}$  e' il momento aerodinamico destabilizzante per unita' di angolo d'attacco; `mu_c` =  $K1 = Tc * l\_c / I_{yy}$  e' il momento di controllo per unita' di deflessione dell'ugello. Sono i due coefficienti dell'equazione di beccheggio, cioe' i due che decidono da soli la stabilita' del loop di assetto: il polo instabile e'  $\pm \sqrt{\mu\_alpha}$  e l'autorita' per contrastarlo e' `mu_c`. Sono anche i due meno noti a priori in un progetto reale: `mu_alpha` dipende dalla posizione del centro di pressione (fortemente dipendente dal Mach e mal predetta dalla CFD in transonico), `mu_c` dalla spinta effettiva e dagli allineamenti. Perturbarli del +/-30 % e' la corner analysis della traccia.

**D: Il file dichiara `Tt_minus_D = 1.71e6 N` ma usa `a4 = -27.271`, che corrisponde a  $2.01e6 N$ . Come lo giustifichi?** R: E' un'incongruenza della Table 1, e il codice la documenta nel commento alle righe 36-38 invece di nascondersela. Ho scelto di usare il valore della tabella / del dataset LPV (che concordano su -27.2710) perche' il dataset e' la fonte primaria del professore e perche' i coefficienti `a*` sono quelli effettivamente tabulati; `Tt_minus_D` e' un dato accessorio che nel codice non entra in nessuna matrice. L'effetto sulla dinamica e' comunque piccolo: `a4` compare solo nella riga di `z_ddot` e sposta il polo lento di drift da  $(T-D)/(m*V) = 0.0247$  a  $0.0291$  rad/s.

**D: Perche' `zBM = 0.005` e' cosi' critico?** R: Perche' fissa il picco di risonanza. Un oscillatore del secondo ordine ha amplificazione  $1/(2*\zeta)$  alla risonanza: con  $\zeta = 0.005$  sono 100, cioe' +40 dB sopra il guadagno statico. E' quello che porta il guadagno di anello a +29 dB a 18.9 rad/s nel modello completo (vedi `build_plant_full.m`), cioe' ben oltre il punto critico: senza filtro il loop e' instabile. Uno smorzamento strutturale del 5 % invece che dello 0.5 % renderebbe il problema quasi irrilevante.

**D: Come diventa una griglia LPV questa funzione?** R: Tramite `t_ref`. `LTV_FULL_ASCENT/main_full_ascent.m` chiama `build_plant_rigid(load_hw3_params('t_ref', tm(i)))` su una griglia `tm = 5:2.5:140 s`, ottenendo un plant congelato per ciascun istante e quindi un modello LTV per tutta l'ascesa. Il limite noto e' che `Alt`, `m` e `Iyy` non sono interpolati dal dataset (restano ai valori di 72 s), quindi la `qbar` derivata e' valida solo al punto di progetto; l'estensione LPV infatti si ricalcola la propria `qbar` per lo scheduling.

**D: `interp1` senza extrapolation: e' un bug?** R: Non nel caso d'uso nominale (`t_ref = 72` e' ben dentro la griglia), ma e' una fragilita': se qualcuno chiedesse `t_ref = 200 s` la funzione restituirebbe una struct piena di NaN senza sollevare alcun errore, e il primo sintomo sarebbe un plant `ss` con matrici NaN. Un `mustBeInRange` sulla griglia del dataset (o un 'extrap' esplicito con warning) sarebbe la correzione onesta.

# HM3/build\_\_plant\_\_rigid.m

## Ruolo del file nel progetto

Costruisce il **modello di corpo rigido** del lanciatore nel piano di beccheggio al max-q: 4 stati [*z*, *zdot*, *theta*, *thetadot*], 2 ingressi [*delta*, *alpha\_w*], 7 uscite. E' la **Eq. (1) della traccia privata delle due righe di bending**, ed e' il plant su cui gira tutto Task 1 (progetto PD, carta di Nichols, risposta alla raffica) e su cui *design\_controller.m* ricava i guadagni iniziali.

Il file e' volutamente banale come codice – tre matrici e un **ss** – ma **e' il punto in cui tutta la fisica del problema viene messa per iscritto**. All'orale e' qui che si gioca la partita: ogni elemento delle matrici va saputo derivare.

Chi lo chiama: *main\_task1.m* (riga 18), *main\_task2.m* (riga 23, come termine di paragone rigido), *main\_montecarlo.m*, *make\_pz\_figures.m*, *init\_simulink\_hm3.m*, *run\_simulink\_closed\_loop.m*, e in ciclo su *t\_ref* LTV\_FULL\_ASCENT/main\_full\_ascent.m. Dipende solo dalla struct *p* di *load\_hw3\_params*. Il suo output finisce in *assemble\_loop*(*G*, *K*, *Wact*), che chiude il loop PD e restituisce *L* (per Nichols) e *T* (per le simulazioni).

Il risultato chiave, verificabile in una riga (*pole(build\_plant\_rigid(p))*):

```
poli = { 0 , +0.0291 , +1.8165 , -1.8610 } [rad/s]
```

cioe' **due poli a parte reale positiva**: il lanciatore rigido e' instabile in open loop, e la retroazione non e' un'opzione ma una condizione di esistenza.

---

## Firma e contratto (righe 1-9)

```
function G = build_plant_rigid(p)
% G - ss, 4 states [z zdot theta thetadot], in [delta alpha_w],
% out [theta_m thetadot_m z_m zdot_m theta z zdot]
```

- Righe 1: unico argomento, la struct *p*. **Nessun blocco arguments**: coerente con lo stile del repo, che tiene la validazione al confine di ingresso – qui sta nel blocco **arguments** di *load\_hw3\_params*, che e' l'unica sorgente di *p*. (Gli unici usi in ciclo di questa funzione sono gli sweep su *t\_ref* di LTV\_FULL\_ASCENT/main\_full\_ascent.m riga 43 e *main\_q\_scheduling.m* riga 56; *main\_montecarlo.m* la chiama **una sola volta**, riga 41, fuori dal **parfor** – dentro al ciclo Monte Carlo gira *build\_plant\_full*.)
- Righe 6-9: il contratto delle uscite. Le **prime 4** sono le *misure* che alimentano il controllore (*theta\_m*, *thetadot\_m*, *z\_m*, *zdot\_m*); le **ultime 3** sono segnali di plotting (*theta*, *z*, *zdot* veri). Nel modello rigido misure e stati veri coincidono – la distinzione esiste solo perche' in *build\_plant\_full.m* **non** coincideranno piu' (contaminazione INS del bending), e le due funzioni devono esporre la **stessa interfaccia** per essere intercambiabili in *assemble\_loop*.

## La matrice A: le equazioni del moto (righe 11-14)

```
A = [0    1    0    0;
      0  p.a1  p.a1*p.V+p.a4  0;
      0    0    0    1;
      0  p.A6/p.V p.A6    0];
```

Le due equazioni fisiche dietro queste quattro righe sono l'equazione dei **momenti** (beccheggio) e quella delle **forze laterali** (drift), entrambe linearizzate attorno alla traiettoria nominale e congelate a  $t = 72$  s.

### Equazione dei momenti (riga 4 di A)

Il momento attorno al baricentro ha due contributi: quello aerodinamico ( $N_{\alpha} * \alpha$  applicata a distanza  $l_{\alpha}$  davanti al CG) e quello di controllo ( $T_c * \delta$  applicata a distanza  $l_c$  dietro il CG):

$$I_{yy} \ddot{\theta} = N_{\alpha} l_{\alpha} \alpha + T_c l_c \delta$$

$$\ddot{\theta} = A_6 \alpha + K_1 \delta, \quad \begin{aligned} A_6 &= N_{\alpha} l_{\alpha} / I_{yy} \\ K_1 &= T_c l_c / I_{yy} \end{aligned}$$

L'angolo d'attacco **non e' uno stato**: e' una combinazione degli stati piu' il disturbo. Con  $\dot{z}$  la velocita' laterale del veicolo e  $V$  la velocita' assiale,

$$\alpha = \theta + \dot{z} / V - \alpha_w$$

(sul segno di  $\alpha_w$  vedi la sezione dedicata piu' avanti). Sostituendo:

$$\ddot{\theta} = A_6 \theta + (A_6 / V) \dot{z} - A_6 \alpha_w + K_1 \delta$$

che e' **esattamente** la riga 14 del codice ( $[0, A_6/V, A_6, 0]$ ) piu' le colonne di ingresso  $K_1$  e  $-A_6$ . Il termine  $(A_6/V) \dot{z}$  e' il **coupling drift -> assetto**: e' quello che rende il problema un vero sistema 4x4 e non due sottosistemi separati, ed e' la ragione per cui i margini vanno letti sul loop accoppiato e non sulla sola dinamica rotazionale.

- Riga 13 ( $[0 \ 0 \ 0 \ 1]$ ): semplicemente  $d(\theta)/dt = \dot{\theta}$ .

### Equazione delle forze laterali (riga 2 di A)

$$m \ddot{z} = -N_{\alpha} \alpha - (T - D) \theta + T_c \delta$$

Dividendo per  $m$  e usando le definizioni  $a_1 = -N_{\alpha}/(mV)$ ,  $a_4 = -(T-D)/m$ ,  $a_3 = T_c/m$ :

$$\begin{aligned} \ddot{z} &= a_1 V \alpha + a_4 \theta + a_3 \delta \\ &= a_1 V (\theta + \dot{z} / V - \alpha_w) + a_4 \theta + a_3 \delta \\ &= a_1 \dot{z} + (a_1 V + a_4) \theta + a_3 \delta - a_1 V \alpha_w \end{aligned}$$

che riproduce **carattere per carattere** la riga 12 del codice ( $[0, a_1, a_1 V + a_4, 0]$ ) e le colonne  $a_3$  e  $-a_1 V$ . Il significato dei tre coefficienti:

| coeff. | formula            | segno           | significato fisico                                                                                                        |
|--------|--------------------|-----------------|---------------------------------------------------------------------------------------------------------------------------|
| $a_1$  | $-N_{\alpha}/(mV)$ | $-0.0154 (< 0)$ | smorzamento aerodinamico del drift: la forza normale si oppone alla velocita' laterale                                    |
| $a_3$  | $T_c/m$            | $+20.61 (> 0)$  | componente <b>laterale della spinta</b> quando l'ugello e' deflesso: e' il canale con cui il TVC muove il veicolo di lato |

| coeff.            | formula                 | segno          | significato fisico                                                                                                                              |
|-------------------|-------------------------|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| $a1 \cdot V + a4$ | $-(T-D) + N_{\alpha}/m$ | $-41.73 (< 0)$ | accelerazione laterale per unita' di assetto: la spinta assiale <b>inclinata</b> di <b>theta</b> piu' la forza normale generata da <b>theta</b> |

**Le formule sono definizioni, i numeri vengono dalla tabella.** Attenzione a non farsi prendere in castagna su **a4**: la definizione e'  $a4 = -(T-D)/m$ , ma con i valori di Tabella 1 ( $T-D = 1.71e6$  N,  $m = 7.38e4$  kg) darebbe  $-23.17$ , mentre la tabella – e il set LPV, che concorda – elenca **a4** =  $-27.2710$ . L'incoerenza e' nota e documentata in `load_hw3_params.m` righe 36-38: nella dinamica entra  $-27.2710$ , ed e' quel valore (non il rapporto ricalcolato) a produrre tutti i numeri di questa pagina.

- Riga 11 (`[0 1 0 0]`):  $d(z)/dt = \dot{z}$ .
- **La prima colonna di A e' identicamente nulla:**  $z$  non retroagisce su nulla. Il drift e' un puro integratore della velocita' laterale, e infatti uno dei quattro poli e' esattamente 0. Questo ha una conseguenza pratica: la retroazione su  $z_m$  (guadagni  $Kp_z = Kd_z = -1e-3$ , fissati in `design_controller.m`) non puo' *stabilizzare* nulla – serve a **contenere la deriva accumulata**. Attenzione a non chiamarla “load relief”: disattivandola il picco di incidenza cambia di appena l'1% ( $0.584$  contro  $0.577$  deg). Compra *drift control*, non alleggerimento del carico.

**Possibile domanda d'esame** – Perche'  $a3 (= Tc/m)$  e' positivo mentre  $K1 (= Tc \cdot l_c / I_{yy})$  genera un momento che si oppone? Non e' contraddittorio? *Risposta:* no, e' la caratteristica **non-minimum-phase** del TVC. Deflettere l'ugello di  $\delta > 0$  produce simultaneamente (i) una forza laterale istantanea sul veicolo nella direzione della componente di spinta ( $a3 \cdot \delta$ ) e (ii) un momento che ruota il veicolo nel verso opposto ( $K1 \cdot \delta$ ). Nel breve termine il veicolo si sposta “dalla parte sbagliata”, nel lungo termine ruota e la spinta assiale lo riporta. E' l'analogo del “wrong-way” dell'elevatore aeronautico e si manifesta come uno zero a destra nella funzione di trasferimento  $\delta \rightarrow z$ .

## Le colonne di ingresso (righe 16-17)

```
Bd = [0; p.a3; 0; p.K1];           % delta column
Bw = [0; -p.a1*p.V; 0; -p.A6];     % alpha_w column
```

- Riga 16: il TVC entra in **due** righe: forza laterale **a3** e momento **K1**. Sono i due effetti simultanei discussi sopra.
- Riga 17: il vento entra con  $-a1 \cdot V$  e  $-A6$ , cioe' **con il segno opposto** a come **theta** entra tramite **alpha**. Ed e' l'unica informazione che serve per ricostruire la convenzione di segno adottata: se il disturbo entrasse come  $\alpha = \dots + \alpha_w$ , la colonna sarebbe `[0; +a1*V; 0; +A6]`.

## Il segno di **alpha\_w**: perche' il meno e' l'unico coerente con l'Eq. (1)

Oggi tutta la catena usa lo **stesso** segno, il meno:

- il **plant** (`build_plant_rigid.m` riga 17, e identicamente `build_plant_full.m` riga 28) implementa  
 $\alpha = \theta + \dot{z}/V - \alpha_w \quad \rightarrow \quad Bw = [0; -a1 \cdot V; 0; -A6]$
- il **post-processing** (`simulate_gust_response.m` riga 29, usato da `main_task1.m` e da `main_task2.m` riga 188) ricostruisce  
 $r.\alpha = r.\theta + r.\dot{z}/w.V - r.\alpha_w \quad \% \text{ total angle of attack}$

**Perche' il meno non e' una convenzione ma un vincolo.**  $\alpha_w = v_w/V$  con  $v_w$  la velocita' laterale dell'aria (definizione di `load_wind_profile.m`, riga 2). L'incidenza e' l'angolo fra l'asse del veicolo e il vento relativo, quindi dipende dalla velocita' laterale del veicolo **rispetto all'aria**:

$$\alpha = \theta + (\dot{z} - v_w)/V = \theta + \dot{z}/V - \alpha_w$$

Il plant **lo conferma da solo**, e basta guardare  $B_w$ : le due componenti non nulle sono  $-a_1*V$  e  $-A_6$ , cioe' **esattamente** i coefficienti con cui  $\theta$  entra nella dinamica attraverso  $\alpha$ , **cambiati di segno**. Se l'incidenza fosse  $\theta + \dot{z}/V + \alpha_w$ , la colonna del vento dovrebbe essere  $[0; +a_1*V; 0; +A_6]$ . Il segno di  $B_w$  e' la definizione di  $\alpha$ : non sono due informazioni indipendenti. E' anche la convenzione degli appunti del corso (Lez. 17,  $-\alpha_w$ ) e quella derivata nel report (`Introduction.tex`, righe 61-64).

**Nota storica – il tipo di domanda che un esaminatore adora.** Fino a poco fa `simulate_gust_response.m` ricostruiva l'incidenza con il segno **piu'**, mentre il plant aveva gia' il meno. **Il modello non e' mai stato sbagliato**: il difetto era solo nella formula a valle, cioe' nella ricostruzione di  $\alpha$  usata per l'indicatore di carico. Ma le conseguenze non erano cosmetiche:

| formula                          | picco $ \alpha $ (Task 1) | $\bar{q}\alpha$         |
|----------------------------------|---------------------------|-------------------------|
| $+\alpha_w$ (vecchio, sbagliato) | 0.255 deg                 | 20.7 kPa*deg            |
| $-\alpha_w$ (attuale, corretto)  | <b>0.577 deg</b>          | <b>**46.8 kPa*deg**</b> |

Piu' del doppio – e con la **conclusione fisica rovesciata**. Con il +, il picco di incidenza cadeva *sotto* il contributo del solo vento (0.390 deg) e sembrava che l'anello facesse **load relief**. Con il segno giusto il picco lo **supera**: per tenere l'assetto il loop becca il muso **dentro** il vento relativo e il suo contributo si **somma** a quello del vento. Un puro attitude-hold e' **load-aggravating**. Coerentemente, con  $A_6 > 0$  il centro di pressione sta davanti al baricentro: il momento aerodinamico e' divergente e **non c'e' nessuna stabilita' a banderuola** che allevi il carico da sola.

Cosa la correzione **non** ha toccato (ed e' importante saperlo dire):

- **Nulla** su stabilita', margini, Nichols, progetto del notch, corner di Task 3, Monte Carlo. Motivo:  $\alpha_w$  e' un **ingresso di disturbo** e non entra nel loop di retroazione.  $L(s)$  e' calcolata da `assemble_loop` come `getLoopTransfer(T,'delta',-1)`, cioe' sul percorso `delta -> misure -> delta`: la colonna  $B_w$  **non compare**. Tutti i numeri di margine sono immuni.
- **Nulla** sui picchi di  $\theta$ ,  $z$  e  $\delta$ : quelle sono uscite del plant, che era gia' giusto.
- **Solo** la quantita' ricostruita a valle – il picco di incidenza totale e quindi  $\bar{q}\alpha$  – e con essa la lettura del trade attitude-hold / load-relief.

Da dire all'orale cosi' com'e': il modello e' sempre stato corretto, il segno lo detta la colonna  $B_w$ , e la metrica di carico ora la rispetta.

## Le uscite: misure e segnali di plot (righe 19-28)

```
C = [0 0 1 0;    % theta_m    = theta
     0 0 0 1;    % thetadot_m= thetadot
     1 0 0 0;    % z_m       = z
     0 1 0 0;    % zdot_m    = zdot
     0 0 1 0;    % theta
     1 0 0 0;    % z
     0 1 0 0];   % zdot
D = zeros(7, 2);
```

- Righe 21-27: le prime 4 righe (misure) sono **identiche** alle ultime 3 (verita') a meno della permutazione. E' esattamente cio' che il test `testRigidMeasurementsEqualTrueStates` (`hm3PlantTest.m` righe 65-70)

verifica:  $G.C(1,:) == G.C(5,:)$  e  $G.C(3,:) == G.C(6,:)$ . La ridondanza e' **voluta**: serve a mantenere l'interfaccia identica a quella del plant flessibile, dove le due famiglie divergono.

- Riga 28:  $D = 0$ , il plant e' **strettamente proprio**: nessun percorso diretto  $\delta \rightarrow$  misure. E' questo che permette a `getLoopTransfer` di rompere il loop su  $\delta$  senza cappi algebrici.

---

## L'oggetto `ss` e i suoi nomi (righe 30-34)

```
G = ss(A, [Bd Bw], C, D);
G.StateName = {'z','zdot','theta','thetadot'};
G.InputName = {'delta','alpha_w'};
G.OutputName = {'theta_m','thetadot_m','z_m','zdot_m','theta','z','zdot'};
```

- Righe 31-33: i nomi non sono cosmetici, sono **load-bearing**. Due usi concreti:
  - (i) `assemble_loop.m` (riga 35) usa `connect(G, Kc, Wa, {'alpha_w','theta_ref'}, {'theta','z','zdot','delta','delta'})`, che collega i blocchi **per nome**;
  - (ii) `design_controller.m` (righe 41-45) ri-estrae i coefficienti dal plant per nome:  
`iTh = strcmp(G.StateName,'theta');` `iTd = strcmp(G.StateName,'thetadot');`  
`A6 = G.A(iTd, iTh);` `K1 = G.B(iTd, iDe);`  
cioe' legge  $A6$  e  $K1$  **dalla matrice**, non dalla struct `p`. Rinominare uno stato rompe silenziosamente il seed dei guadagni.

---

## Il polo instabile: da dove viene esattamente

La domanda "perche' il polo e'  $+\sqrt{A6}$ " ha una risposta esatta e una piu' onesta.

**Risposta esatta (dinamica rotazionale disaccoppiata).** Se si ignora il drift ( $zdot \rightarrow 0$ ), resta

$$\theta_{ddot} = A6\theta + K1\delta \quad \rightarrow \quad s^2 - A6 = 0 \quad \rightarrow \quad s = \pm \sqrt{A6}$$

con  $A6 = 3.3818 \rightarrow \sqrt{A6} = 1.839$  rad/s. E' una **coppia** di poli reali simmetrici, uno stabile e uno instabile: la firma della classica instabilita' statica a sella (nessuna oscillazione, divergenza esponenziale con costante di tempo  $1/1.84 = 0.54$  s). E' il modello su cui D'Antuono ricava i guadagni canonici  $Kp = 2*A6/K1$ ,  $Kd = \sqrt{A6}/K1$  usati come punto di partenza in `design_controller.m` (righe 55).

**Risposta onesta (plant 4x4 completo).** La prima colonna di  $A$  e' nulla, quindi un polo e' in 0 (l'integratore su  $z$ ). I restanti tre autovalori sono quelli del blocco `[zdot, theta, thetadot]`, il cui polinomio caratteristico e'

$$s^3 - a1s^2 - A6s - A6a4/V = 0$$

Sostituendo i valori a  $t = 72$  s:

$$s^3 + 0.01542s^2 - 3.3818s + 0.09835 = 0$$
$$\rightarrow s = -1.8610, \quad +1.8165, \quad +0.02908$$

- il **polo veloce instabile**  $+1.8165$  e' lo  $\sqrt{A6} = 1.839$  spostato di  $\sim 1.2\%$  dal coupling del drift (e' proprio la tolleranza del  $2\%$  del test `testRigidAirframeUnstablePole`, riga 61-62 di `hm3PlantTest.m`);
- il **polo lento instabile**  $+0.0291$  **non esiste nel modello rotazionale puro**: nasce dal termine noto  $-A6a4/V$ , e vale approssimativamente

$$s_{lento} \approx -a4/V = 27.271/937.71 = 0.02908 \text{ rad/s}$$

(coincide a 4 cifre). E' la **divergenza di drift**: con l'assetto tenuto fermo, la componente laterale della spinta accelera lentamente il veicolo di lato e il moto laterale a sua volta modifica l'angolo d'attacco. Costante di tempo  $\sim 34$  s, quindi non e' un problema di stabilita' pratica in una finestra di volo di 10 s, ma e' la **ragione per cui il loop e' condizionalmente stabile** e per cui un singolo numero di

`margin()` non ha senso: con due poli instabili la curva di Nichols deve tenere il punto critico (-180 deg, 0 dB) **incastrato** fra i suoi attraversamenti della fase critica, e servono margini classificati per banda di frequenza (`classify_margins.m`).

**Possibile domanda d'esame** – Quanti poli instabili ha il lanciatore rigido, e cosa implica per la lettura dei margini? *Risposta:* due (+1.82 e +0.029 rad/s), piu' un integratore in 0. Per il criterio di Nyquist con  $P = 2$  poli instabili in anello aperto, la stabilita' richiede  **$N = 2$  encerchiamenti antiorari** del punto critico: il loop e' *conditionally stable*, cioe' diventa instabile sia aumentando sia **riducendo** il guadagno. Per questo il codice non si fida di `margin()` e classifica i margini per banda: il **gain margin aerodinamico** all'attraversamento della fase critica -180 deg a bassa frequenza (~0.6 rad/s, protegge dal *ridurre* il guadagno), i margini di fase/guadagno **rigidi** al crossover di controllo (~2.5 rad/s) e il delay margin. (Sulla carta di Nichols il punto critico sta a (-180 deg, 0 dB), convenzione degli appunti del corso da  $1 + L = 0$ ; D'Antuono etichetta lo stesso punto +180 – fase mod 360 – ed e' la rietichettatura che il codice usava fino a poco fa.)

---

## Possibili domande d'esame

**D: Scrivi le equazioni del moto del pitch-plane e mostra come diventano la matrice  $A$  del codice.**  
R: Momenti:  $\ddot{\theta} = A_6 \alpha + K_1 \delta$  con  $A_6 = N_{\alpha} l_{\alpha} / I_{yy}$ ,  $K_1 = T c l_c / I_{yy}$ . Forze:  $\ddot{z} = a_1 V \alpha + a_4 \theta + a_3 \delta$  con  $a_1 = -N_{\alpha} / (m V)$ ,  $a_4 = -(T - D) / m$ ,  $a_3 = T c / m$ . Il coupling e'  $\alpha = \theta + \dot{z} / V - \alpha_w$ : sostituendolo,  $\ddot{\theta}$  acquista il termine  $(A_6 / V) \dot{z}$  (riga 4 di  $A$ ) e  $\ddot{z}$  acquista  $a_1 \dot{z}$  e il termine combinato  $(a_1 V + a_4) \theta$  (riga 2 di  $A$ ). Le colonne di ingresso escono da  $\delta$  ( $a_3$  sulla forza,  $K_1$  sul momento) e da  $\alpha_w$  ( $-a_1 V$ ,  $-A_6$ ).

**D:  $A_6$  e' un momento aerodinamico. Perche' e' destabilizzante e non stabilizzante come su un aeroplano?** R: Perche' su un lanciatore snello il centro di pressione sta **davanti** al baricentro (il corpo e' quasi un cilindro con un'ogiva, non ci sono superfici di coda a spostare il cp all'indietro). Con  $l_{\alpha} > 0$  misurato dal CG verso prua,  $A_6 = N_{\alpha} l_{\alpha} / I_{yy} > 0$  e la dinamica di beccheggio isolata e'  $\ddot{\theta} = +A_6 \theta$ : un angolo d'attacco positivo genera un momento che lo fa crescere. Su un aeroplano il cp e' dietro il CG, il segno si inverte e si ottiene una coppia di poli immaginari (short period stabile).

**D: Perche'  $z$  compare come stato se la prima colonna di  $A$  e' nulla?** R: Perche' e' la **variabile di uscita** che interessa (il drift laterale accumulato, che entra nel budget di traiettoria) e perche' serve la sua misura  $z_m$  per chiudere il debole loop di deriva. Dinamicamente e' un puro integratore: non retroaziona su nulla, e infatti contribuisce l'autovalore 0. La retroazione su  $z$  con guadagni  $-1e-3$  non stabilizza (non puo'): riduce la deriva accumulata. Non e' un dispositivo di load relief – togliendola il picco di incidenza si muove dell'1% – ed e' bene dirlo prima che lo chiedano.

**D: Cosa succede se togli il termine  $(A_6 / V) \dot{z}$  dalla matrice  $A$ ?** R: Si ottiene il modello rotazionale disaccoppiato: i poli diventano  $\{0, a_1, +\sqrt{A_6}, -\sqrt{A_6}\} = \{0, -0.0154, +1.839, -1.839\}$ . Il blocco  $[\dot{z}, \theta, \dot{\theta}]$  si triangolarizza – il drift non retroaziona piu' sul beccheggio – quindi  $\dot{z}$  resta con il proprio autovalore  $a_1$  e il beccheggio con la coppia simmetrica  $\pm \sqrt{A_6}$ . Sparisce cosi' il polo lento **instabile** a +0.029 (al suo posto compare lo smorzamento aerodinamico **stabile**  $a_1$ ) e con esso la parte a bassa frequenza del problema. Con i guadagni canonici il gain margin aerodinamico “sulla carta” sarebbe ~6 dB; sul loop accoppiato reale scende a ~4 dB, ed e' proprio la ragione per cui `design_controller.m` **ri-tuna** i guadagni sul loop completo invece di fermarsi alla formula chiusa (lo dice esplicitamente nel suo header, righe 6-10).

**D: Il modello e' valido solo a  $t = 72$  s. Perche' progettare li'?** R: Perche' e' il punto di **massima pressione dinamica** ( $q_{bar} = 81$  kPa), quindi quello di massimo momento aerodinamico destabilizzante ( $A_6$  scala con  $q_{bar}$ ) e di massimo carico strutturale ( $q_{bar} \alpha$ ). E' il caso peggiore del volo atmosferico: un controllore che chiude i margini richiesti li' e' tipicamente adeguato altrove. La verifica di questa affermazione e' l'estensione LPV in `LTV_FULL_ASCENT/`, che confronta il design frozen-time con uno gain-scheduled sull'intera ascesa.

# HM3/build\_\_plant\_\_full.m

## Ruolo del file nel progetto

E' il plant di **Task 2** e **Task 3**: il modello rigido di `build_plant_rigid.m` esteso con il **primo modo di flessione** del corpo (2 stati in piu', `eta` e `etadot`) e con l'**osservazione INS contaminata** dalla deformazione elastica (Eq. 2 della traccia). Sei stati [`z`, `zdot`, `theta`, `thetadot`, `eta`, `etadot`], due ingressi [`delta`, `alpha_w`], le stesse 7 uscite del plant rigido – interfaccia identica, cosi' che `assemble_loop`, `design_controller`, `classify_margins` e `simulate_gust_response` funzionino senza modifiche su entrambi.

E' il file che **crea il problema che Task 2 deve risolvere**. Con il plant rigido il loop e' pulito; sostituendo questo plant, il guadagno di anello alla frequenza del modo di bending sale a **+29 dB** e il loop diventa instabile. Da li' nascono il trade fra quattro filtri, il notch profondo scelto e il ri-tuning del PD.

Chi lo chiama: `main_task2.m` (riga 29), `main_task3.m` (righe 21 e 50, per le quattro corner), `main_montecarlo.m`, `make_pz_figures.m`, `init_simulink_hm3.m`, `run_simulink_closed_loop.m` (riga 40), `LTV_FULL_ASCENT/main_flex.m`, e i test. Il ramo `meas = 'true'` **non e' usato da nessun main**: e' esercitato solo da `tests/hm3PlantTest.m` (riga 92) e serve come ablazione didattica – ed e' proprio quell'ablazione a dimostrare la tesi centrale di Task 2 (vedi sotto).

---

## Firma, arguments, contratto (righe 1-16)

```
function G = build_plant_full(p, meas)
arguments
    p (1,1) struct
    meas {mustBeTextScalar} = 'ins'    % 'ins' / 'true', case-insensitive
end
```

- Riga 1: due argomenti posizionali. `meas` di default `'ins'`, cioe' **il caso realistico** (la piattaforma inerziale sente la deformazione). Chi vuole il confronto pulito chiede esplicitamente `'true'`.
- Righe 10-11 (commento): l'header dichiara la tesi del file – “*INS bending contamination ( $\sigma_{ins}$ ,  $\phi_{ins}$ ) is what destabilises the loop at  $w_{BM}$  and motivates the Task-2 notch*”. E' vero, e il codice lo dimostra in modo falsificabile: vedi la sezione finale.
- Riga 18: `w = p.wBM`; `z = p.zBM`; – attenzione, `z` qui e' lo **smorzamento modale**, non lo stato di drift. Shadowing di nome locale, innocuo ma potenzialmente confondente in lettura.

---

## La matrice A: le due righe nuove (righe 20-25)

```
A = [0    1    0    0    0    0;
      0    p.a1  p.a1*p.V+p.a4 0    0    0;
      0    0    0    1    0    0;
```



```

0    p.A6/p.V    p.A6    0    0    0;
0    0           0       0    0    1;
0    0           0       0    -w^2  -2*z*w];

```

- Righe 20-23: il blocco 4x4 in alto a sinistra e' **identico** a quello di `build_plant_rigid.m` (stesse equazioni dei momenti e delle forze laterali, stessa convenzione  $\alpha = \theta + \dot{z}/V - \alpha_w$ ).
- Righe 24-25: l'**oscillatore modale**. In forma di equazione differenziale:

$$\ddot{\eta} + 2\zeta_{BM}\omega_{BM}\dot{\eta} + \omega_{BM}^2\eta = f(t)$$

cioe' la classica equazione modale disaccoppiata che si ottiene proiettando le equazioni della trave elastica sul primo modo.  $\eta$  e' la **coordinata modale generalizzata**; qui ha unita' di metri (lo si deduce dalle unita' di  $\sigma_{ins}$  [rad/m] e  $\phi_{tvc}$  [1/kg], vedi sotto). Con  $\omega_{BM} = 18.9$  rad/s e  $\zeta_{BM} = 0.005$  i due autovalori sono

$$-\zeta\omega \pm j\omega\sqrt{1-\zeta^2} = -0.0945 \pm j 18.8998j$$

(verificato con `pole()`), pinnati dal test `testFullPlantBendingMode` (`hm3PlantTest.m` righe 72-80) con tolleranza  $1e-9$ .

## Il punto chiave: il coupling nella matrice A e' a senso unico

Le colonne 5-6 delle righe rigide (2 e 4) sono **nulle**, e le righe 5-6 hanno colonne 1-4 **nulle**. Significa:

- la deformazione elastica **non retroagisce** sulla dinamica di corpo rigido (si trascurano gli effetti inerziali/aerodinamici del modo sul moto rigido);
- il moto rigido **non eccita** il modo di bending (nessun forcing da  $\theta$ ).

L'unico accoppiamento nella dinamica e' l'ingresso **delta** (riga 27), e l'unico accoppiamento nell'uscita e' la matrice  $C_m$  (righe 33-36). Quindi **il modo di bending si chiude in un anello solo passando per il controllore**:

```

delta -> eta (forcing TVC) -> theta_m, zdot_m (contaminazione INS)
      -> controllore -> delta

```

Non e' un fenomeno strutturale: e' un **loop di controllo indesiderato** creato dal sensore. Questo e' esattamente il motivo per cui la soluzione e' un filtro (notch) e non un irrigidimento del corpo.

## Le colonne di ingresso (righe 27-28)

```

Bd = [0; p.a3; 0; p.K1; 0; -p.phi_tvc*p.Tc]; % delta column
Bw = [0; -p.a1*p.V; 0; -p.A6; 0; 0];         % alpha_w column

```

- Riga 27: il **forcing del bending da parte del TVC**. La forza laterale  $T_c \cdot \delta$  applicata al gimbal eccita il modo con un guadagno pari alla pendenza/ampiezza della forma modale in quel punto normalizzata sulla massa generalizzata – che e' il significato di  $\phi_{tvc}$  [1/kg]. Numericamente:

$$\phi_{tvc} * T_c = 4.3095e-5 * 1.5213e6 = 65.56 \text{ m/s}^2 \text{ per rad}$$

Il **segno negativo** dice che una deflessione positiva flette il corpo nel verso di  $\eta$  negativo (coerente con il fatto che la forza al gimbal e' opposta al momento generato). Coerenza dimensionale:  $[1/kg] * [N] * [rad] = [m/s^2]$ , quindi  $\eta$  e' in metri e  $\ddot{\eta}$  in  $m/s^2$ . Torna.

- Riga 28: la colonna del vento ha **zeri sulle righe 5-6**: la raffica **non eccita direttamente il modo di flessione**. E' un'approssimazione del modello della traccia (in un lanciatore vero il carico distribuito di raffica eccita eccome i modi elastici). Conseguenza pratica: nelle simulazioni di raffica il bending si vede solo *indirettamente*, attraverso il **delta** che il controllore comanda. Va detto all'orale se si viene interrogati sui carichi.

**Possibile domanda d'esame** – Se  $\alpha_w$  non eccita il bending e il bending non retroagisce sul corpo rigido, perché è un problema? *Risposta:* perché il problema non è dinamico ma di **retroazione**. Il TVC eccita il modo (Bd(6)), l'INS lo misura come se fosse assetto ( $C_m(1,5)$ ), il controllore ci reagisce e comanda altro TVC. Se il guadagno complessivo di questo giro supera 0 dB con la fase sbagliata alla risonanza, il modo va in autoeccitazione: il loop è instabile anche se la struttura, da sola, è stabile (smorzamento positivo, seppur 0.5 %).

## L'osservazione INS: Eq. (2) (righe 30-44)

```
case 'ins' % Eq. (2): bending leaks into the measurements
    Cm = [0 0 1 0 p.sigma_ins 0; % theta_m = theta + sigma*eta
          0 0 0 1 0 p.sigma_ins; % thetadot_m = thetadot + sigma*etadot
          1 0 0 0 -p.phi_ins 0; % z_m = z - phi*eta
          0 1 0 0 0 -p.phi_ins]; % zdot_m = zdot - phi*etadot
```

- Righe 33-36: è la **Eq. (2)** della traccia. L'INS è montato in una stazione precisa del corpo e non può distinguere fra rotazione rigida e rotazione locale dovuta alla flessione:
  - il **giroscopio** misura l'inclinazione locale della struttura, cioè  $\theta + \sigma_{\text{ins}} \eta$ , dove  $\sigma_{\text{ins}}$  [rad/m] è la **pendenza della forma modale** alla stazione dell'INS. Con  $\sigma_{\text{ins}} = 0.178$  rad/m: un metro di ampiezza modale generalizzata si presenta al sensore come 0.178 rad = 10.2 deg di assetto **falso**;
  - l'**accelerometro/integratore di posizione** misura lo spostamento locale, cioè  $z - \phi_{\text{ins}} \eta$ , con  $\phi_{\text{ins}} = 0.8$  [-] l'**ampiezza** (non la pendenza) della forma modale nello stesso punto. I segni opposti (+ $\sigma$ , - $\phi$ ) sono quelli della traccia e riflettono la geometria della forma modale in quella stazione; il test `testInsMeasurementsContaminatedByBending` (righe 82-88) li pinna.
- Righe 37-41: il ramo 'true' azzerà le colonne 5-6 delle misure: retroazione sugli stati **veri**, come se esistesse un filtro/osservatore ideale che separa rigido ed elastico. Non è realizzabile, ma è l'esperimento di controllo che isola la causa.
- Righe 42-43: `error('build_plant_full:meas', ...)` con identificatore esplicito, testato (`testFullPlantRejectsUnknownInputs`).
- Righe 45-48: le 3 righe di plotting ( $\theta$ ,  $z$ ,  $\dot{z}$  **veri**) sono aggiunte sotto in entrambi i rami. Sono i segnali che i main mandano nei grafici: quello che il veicolo fa davvero, non quello che il sensore crede.

## Da dove vengono i +29 dB (la derivazione che serve all'orale)

Il percorso destabilizzante è  $\delta \rightarrow \eta \rightarrow \theta_m$ . La sua funzione di trasferimento si legge direttamente dalle matrici:

$$\theta_m / \delta |_{\text{bending}} = \sigma_{\text{ins}} * (-\phi_{\text{tvc}} * T_c) / (s^2 + 2 * z_{\text{BM}} * w_{\text{BM}} * s + w_{\text{BM}}^2)$$

Alla risonanza ( $s = j * w_{\text{BM}}$ ) il denominatore vale  $2 * j * z_{\text{BM}} * w_{\text{BM}}^2$ , quindi il modulo del **residuo di bending** è:

$$\begin{aligned} |\theta_m / \delta| (w_{\text{BM}}) &= \sigma_{\text{ins}} * \phi_{\text{tvc}} * T_c / (2 * z_{\text{BM}} * w_{\text{BM}}^2) \\ &= 0.178 * 65.56 / (2 * 0.005 * 18.9^2) \\ &= 11.67 / 3.572 = 3.267 \rightarrow +10.3 \text{ dB} \end{aligned}$$

A questo si moltiplica il **guadagno del controllore alla stessa frequenza**. Il PD di Task 1 ( $K_p = 1.78$ ,  $K_d = 0.44$ ) a 18.9 rad/s vale

$$|K_p + j * w_{\text{BM}} * K_d| = |1.78 + j * 8.32| = 8.50 \rightarrow +18.6 \text{ dB}$$

perché a quella frequenza il termine derivativo domina di un fattore ~5. Somma:

$$+10.3 \text{ dB} + 18.6 \text{ dB} = +28.9 \text{ dB}$$

e infatti, valutando il vero guadagno di anello con `assemble_loop` sul plant completo con attuatore ideale, si ottiene  $|L(j\omega_{BM})| \sim 29.0 \text{ dB}$  – i +29 dB del README. La derivazione chiude entro ~0.1 dB, ed e' il modo di rispondere alla domanda “da dove viene quel numero” senza dire “me lo dice MATLAB”.

Da qui segue tutto il resto di Task 2: per **gain-stabilizzare** il modo servono almeno ~41 dB di attenuazione (per portare +29 dB sotto i -12 dB tipicamente richiesti a un modo gain-stabilizzato); il notch profondo scelto (-51 dB) porta il lobo a **-18 dB**, cioè 18 dB di bending gain margin.

## L'ablazione che dimostra la tesi

Valutando lo stesso guadagno di anello con `meas = 'true'`:

```
|L(j*omega_BM)| con 'ins' = ~+29.0 dB      (instabile senza filtro)
|L(j*omega_BM)| con 'true' = ~-19.4 dB
|L(j*omega_BM)| plant rigido = ~-19.4 dB   (identico!)
```

Con la retroazione sugli stati veri il modo di bending, pur essendo **eccitato** dal TVC ed **esistente** nel plant, e' **completamente invisibile al loop**:  $L(s)$  coincide con quella del corpo rigido. Il che prova che il colpevole non e' la flessibilita' della struttura ne' l'eccitazione da parte dell'attuatore, ma il **sensor coupling**: e' l'INS che chiude l'anello elastico. Da questa osservazione seguono due famiglie di rimedi – filtrare la misura (il notch, scelta di HM3) o spostare/combinare i sensori in modo che `sigma_ins` si annulli (blending di giroscopi in stazioni diverse, soluzione industriale).

**Possibile domanda d'esame** – Perche' non basta abbassare  $K_d$  per gain-stabilizzare il bending, visto che i +18.6 dB vengono quasi tutti dal termine derivativo? *Risposta*: perche'  $K_d$  e' anche cio' che da' il margine di fase al crossover rigido (~2.5 rad/s): ridurlo abbastanza da tagliare 29 dB a 18.9 rad/s (servirebbe un fattore ~30) distruggerebbe lo smorzamento del modo rigido e il PM. Serve un filtro **selettivo in frequenza** che attenui a  $\omega_{BM}$  senza toccare la banda di controllo: e' esattamente la definizione di un notch. Il prezzo lo si paga comunque in fase (il notch + TVC + ritardo fanno crollare il PM da 30 a 14.6 deg, e infatti il PD va ri-tunato sul loop completo).

---

## Possibili domande d'esame

**D: Quali sono i sei stati e cosa rappresentano?** R:  $z$  (drift laterale, m),  $\dot{z}$  (velocita' laterale, m/s),  $\theta$  (assetto di beccheggio, rad),  $\dot{\theta}$  (velocita' angolare, rad/s),  $\eta$  (coordinata modale generalizzata del primo modo di flessione, m),  $\dot{\eta}$  (sua derivata). I primi quattro sono il corpo rigido, gli ultimi due un oscillatore del secondo ordine con  $\omega_{BM} = 18.9 \text{ rad/s}$  e  $\zeta_{BM} = 0.005$ .

**D: Perche' il modo di bending sta a 18.9 rad/s e non a 2 rad/s, e perche' e' comunque un problema se la banda di controllo e' ~2.5 rad/s?** R: 18.9 rad/s (3 Hz) e' la prima frequenza propria della struttura, circa un ordine di grandezza sopra il crossover del controllo – in teoria “fuori banda”. Il problema e' che con  $\zeta = 0.005$  il picco di risonanza vale  $1/(2\zeta) = 100$  (+40 dB) e che il controllore PD ha guadagno **crescente** con la frequenza (termine derivativo): il roll-off naturale non c'e'. Risultato: nonostante la separazione in frequenza, il guadagno di anello alla risonanza e' +29 dB invece che ben sotto 0 dB.

**D: Differenza fra gain stabilization e phase stabilization del modo elastico, e cosa hai scelto?** R: *Gain stabilization* = attenuare il modo sotto 0 dB (tipicamente  $\leq -12 \text{ dB}$ ) cosi' che la fase alla risonanza sia irrilevante; *phase stabilization* = lasciare passare il guadagno ma garantire che la fase alla risonanza mantenga il margine (si usa quando il modo e' troppo vicino in frequenza al crossover per poterlo attenuare). Qui il modo e' ben separato in frequenza, quindi ho gain-stabilizzato con un notch profondo, arrivando a  $|L(\omega_{BM})| = -18 \text{ dB}$ . Il prezzo e' la sensibilita' alla conoscenza esatta di  $\omega_{BM}$  (il notch tollera -10 % di detuning ma diventa instabile a +5 %), che e' il tipico trade-off del notch profondo.

**D: Il modello dice che il bending non retroagisce sul corpo rigido e che il vento non lo eccita. Sono ipotesi accettabili?** R: Sono le ipotesi della traccia e vanno dichiarate. Non retroazionare l'elastico sul rigido e' ragionevole per il **primo** modo di un lanciatore, dove la massa generalizzata coinvolta e' una frazione piccola della massa totale e l'interesse e' la stabilita' del loop, non il calcolo dei carichi. Trascurare l'eccitazione

del bending da parte della raffica e' piu' discutibile: significa che le simulazioni di raffica sottostimano l'ampiezza modale reale e quindi i carichi strutturali. Per il **progetto del controllore** non cambia nulla ( $\alpha_w$  non entra in  $L(s)$ ), per un'analisi di carichi cambierebbe.

**D: Perche' build\_plant\_full espone 7 uscite quando il controllore ne usa 4?** R: Perche' le prime 4 sono le **misure** (contaminate dal bending in modalita' 'ins') e le ultime 3 sono i **valori veri** di **theta**, **z**, **zdot**, che servono per i grafici e per le metriche di prestazione. La distinzione e' la ragione d'essere del file: nel plant rigido le due famiglie coincidono, qui no, e mostrare in figura la differenza fra **theta** e **theta\_m** e' il modo piu' diretto di visualizzare la contaminazione INS. Mantenere le stesse 7 uscite nei due plant permette poi di passarli indifferente a **assemble\_loop**.

# HM3/build\_\_tvc.m

## Ruolo del file nel progetto

`build_tvc.m` costruisce il blocco **attuatore** della catena di controllo: la funzione di trasferimento che sta fra il comando di deflessione calcolato dal PD (`u_pd`, o `delta_cmd`) e la deflessione fisica dell'ugello (`delta`) che entra nel piano dell'`build_plant_rigid` / `build_plant_full`. Implementa l'Eq. 3 della traccia: un **servo di secondo ordine** ( $\omega_{TVC} = 70$  rad/s,  $\zeta_{TVC} = 0.7$ ) in serie con un **ritardo puro di trasporto**  $\tau = 20$  ms.

Nel flusso dell'homework il file non esiste in Task 1: `main_task1.m` chiama `design_controller(G, [])` (riga 22) e legge il loop da `m.L` / `m.T` (righe 30-31); l'alias `[]` diventa `Wact = tf(1)` dentro `assemble_loop` (`assemble_loop.m`, righe 18 e 21), cioè **attuatore ideale**. Il file entra in scena in `main_task2.m` (riga 30, `Wtvc = build_tvc(p, 3)`) e poi in `main_task3.m` / `main_montecarlo.m`. E' esattamente il passaggio in cui il progetto smette di essere accademico: l'attuatore e il ritardo aggiungono **fase negativa senza aggiungere attenuazione utile**, e insieme al modo di bending sono cio' che fa crollare il phase margin rigido da 30 gradi a 14.6 gradi (vedi `main_task2.m`, "Step C decision", righe 131-139 – il commento nel codice arrotonda a ~15), obbligando a ri-tarare il PD sul loop completo.

Il punto di sostanza – ed e' il punto che va difeso all'orale – e' che il ritardo puro **non e' una funzione di trasferimento razionale**:  $\exp(-\tau s)$  e' trascendente. Tutta la catena a valle (`allmargin`, `pole`, `isstable`, `minreal`, `fminsearch` nel tuner) lavora su modelli razionali. La scelta di approssimare con **Pade** non e' cosmetica: e' cio' che rende il resto del codice eseguibile, e ha una motivazione fisica precisa (l'approssimante di Pade e' **all-pass**: modulo esattamente unitario, solo fase – come il ritardo vero).

Dipendenze: solo `p` (struct da `load_hw3_params`, campi `wTVC`, `zTVC`, `tau`) e la Control System Toolbox (`tf`, `pade`). Nessuno stato, nessun file su disco.

---

## build\_tvc (righe 1-23)

```
function Wtvc = build_tvc(p, padeOrder)
s = tf('s');
Wact = p.wTVC^2 / (s^2 + 2*p.zTVC*p.wTVC*s + p.wTVC^2);
[nd, dd] = pade(p.tau, padeOrder);
Wdelay = tf(nd, dd);
Wtvc = Wact * Wdelay;
```

- **Riga 1:** firma `Wtvc = build_tvc(p, padeOrder)`. Restituisce una singola `tf` SISO da `u_pd` a `delta`. Il chiamante la passa come terzo argomento di `assemble_loop`, dove viene inserita **in cascata nel ramo diretto**, fra il guadagno statico del controllore e l'ingresso `delta` del plant (`assemble_loop.m`, righe 30-32). Chiamanti reali: `main_task2.m` (riga 30), `main_task3.m`, `main_montecarlo.m`, `init_simulink_hm3.m`.
- **Righe 10-13:** blocco `arguments`. `padeOrder` ha default **3** ed e' vincolato intero positivo. Nota: questa e' una funzione di *boundary* (viene chiamata una volta per run, non dentro un loop di `fsolve`/`fmincon`),

quindi la validazione qui e' coerente con la convenzione della repo (le funzioni hot-loop non ce l'hanno per costo).

## Riga 16 – il servo di secondo ordine

$$Wact(s) = \omega_{TVC}^2 / (s^2 + 2\zeta_{TVC}\omega_{TVC}s + \omega_{TVC}^2)$$

Forma canonica di un secondo ordine con **guadagno statico unitario** ( $Wact(0) = \omega^2/\omega^2 = 1$ ): a regime l'ugello raggiunge esattamente la deflessione comandata, come deve essere per un servo posizionale. I valori vengono da `load_hw3_params.m` (righe 55-56):  $\omega_{TVC} = 70$  rad/s,  $\zeta_{TVC} = 0.7$ .

Perche' questi numeri contano: la banda dell'attuatore (70 rad/s) sta **quasi un ordine di grandezza sopra** il crossover rigido di progetto (~2.5-3.2 rad/s) ma solo un fattore ~3.7 sopra il modo di bending ( $\omega_{BM} = 18.9$  rad/s). Le conseguenze sono due, e sono opposte a quello che l'intuizione suggerisce:

- **Al crossover rigido** ( $w = 2.445$  rad/s, il crossover di Task 1) l'attuatore costa pochissima fase: sfasamento  $= -\tan^{-1}(2\zeta\omega/\omega^2 - w^2) = -2.80$  gradi. Praticamente trasparente.
- **A  $\omega_{BM} = 18.9$  rad/s** costa -22.18 gradi di fase, ma il suo modulo e'  $|Wact(j*18.9)| = 0.9988$ , cioe' **-0.01 dB**. Zero attenuazione.

Questo secondo punto e' il piu' importante e va detto esplicitamente: **non si puo' contare sull'attuatore per attenuare il bending**. Il rolloff del servo comincia a 70 rad/s, il modo flessionale sta a 18.9 rad/s: l'attuatore lo lascia passare integro (+29 dB di picco nel loop, verificato:  $|L(\omega_{BM})| = 28.95$  dB senza filtro) e in compenso gli aggiunge 22 gradi di ritardo di fase. E' esattamente il peggiore dei due mondi, ed e' la ragione per cui serve `build_notch_filter.m`.

## Righe 18-19 – l'approssimante di Pade

```
[nd, dd] = pade(p.tau, padeOrder);    % tau = 0.020 s, ordine 3
Wdelay  = tf(nd, dd);
```

**Da dove viene.** Il ritardo puro e'  $\exp(-\tau s)$ . L'approssimante di Pade diagonale di ordine (n,n) e'

$$\exp(-\tau s) \sim N(s)/D(s), \quad \text{con} \quad N(s) = D(-s)$$

$$D(s) = \sum_{k=0}^n c_k (\tau s)^k, \\ c_k = (2n-k)! * n! / [(2n)! * k! * (n-k)!]$$

Per  $n = 3$  i coefficienti valgono  $c_0 = 1$ ,  $c_1 = 1/2$ ,  $c_2 = 1/10$ ,  $c_3 = 1/120$ , quindi

$$D(x) = 1 + x/2 + x^2/10 + x^3/120, \quad N(x) = D(-x), \quad x = \tau s$$

**Perche' e' la forma giusta per un ritardo.** La proprieta' chiave e'  $N(s) = D(-s)$ . Valutata sull'asse immaginario:

$$N(j\omega) = D(-j\omega) = \text{coniugato di } D(j\omega) \quad (\text{coefficienti reali})$$

$$\Rightarrow |N(j\omega)| = |D(j\omega)| \quad \text{per OGNI } \omega \\ \Rightarrow |Wdelay(j\omega)| = 1 \quad \text{esattamente, a tutte le frequenze}$$

L'approssimante e' **all-pass**: modulo unitario ovunque, esattamente come il ritardo vero ( $|\exp(-j\omega\tau)| = 1$ ). Tutto quello che fa e' fase:

$$\arg(Wdelay(j\omega)) = -2\arg(D(j\omega))$$

che approssima  $-\omega\tau$ . **Questo e' il motivo per cui Pade e' il modo corretto di modellare un ritardo, e non un semplice "filtro passa-basso equivalente":** un lag del primo ordine  $1/(1+\tau s)$  darebbe fase negativa *ma* anche attenuazione, e l'attenuazione fittizia farebbe sembrare il loop piu' robusto di quanto sia (nasconderebbe il picco di bending). Pade toglie fase e **non regala guadagno**.

**Fase non minima.** Poiche'  $N(s) = D(-s)$ , gli zeri di  $N$  sono l'immagine speculare dei poli di  $D$  rispetto all'asse immaginario. I poli di  $D$  sono stabili (semipiano sinistro), quindi **gli zeri sono tutti nel semipiano destro**: il blocco e' a **fase non minima**. Verificato numericamente per  $\tau = 0.02$  s, ordine 3:

zeri (RHP): +232.2 , +183.9 +/- 175.4j [rad/s]  
 poli (LHP): -232.2 , -183.9 +/- 175.4j [rad/s]

Per l'ordine 1 il conto e' a mano:  $W_{delay} = (1 - \tau/2)/(1 + \tau/2)$ , zero in  $s = +2/\tau = +100$  rad/s, polo in  $-100$  rad/s. E' la firma classica del ritardo: uno zero RHP costa fase senza dare attenuazione, ed e' l'unico modo di ottenere il comportamento "fase che scende linearmente e senza limite" con una funzione razionale.

**Accuratezza e scelta dell'ordine.** La regola pratica e' che Pade di ordine  $n$  e' fedele fino a circa  $w\tau \sim n$ , cioe'  $w \sim n/\tau$ . Con  $\tau = 20$  ms: ordine 1 -> buono fino a  $\sim 50$  rad/s, ordine 3 -> fino a  $\sim 150$  rad/s. Verifica numerica della fase (Pade-3 vs ritardo esatto  $-w\tau$ ):

| w [rad/s]                | Pade-3       | esatto $-w\tau$ |
|--------------------------|--------------|-----------------|
| 2.445 (crossover rigido) | -2.80 gradi  | -2.80 gradi     |
| 18.9 (omega_BM)          | -21.66 gradi | -21.66 gradi    |

**Nota di onesta':** il commento alle righe 5-6 dice "higher = better phase lag near wBM". E' vero in generale ma sovrastimato nel caso concreto: gia' Pade-1 sbaglia solo  $\sim 0.24$  gradi a omega\_BM (-21.42 contro -21.66). Il vero guadagno dell'ordine 3 sta **sopra  $\sim 50$  rad/s** - dove Pade-1 sbaglia di 10 gradi al corner dell'attuatore (70 rad/s: -69.98 contro -80.21 esatti) - e nel calcolo del **delay margin**, che **allmargin** legge su tutti i crossover, compresi quelli ad alta frequenza. Cioe': l'ordine 3 serve, ma per una ragione diversa da quella scritta nel commento.

## Riga 21 – la cascata

$W_{tvc} = W_{act} * W_{delay}$ ;

Prodotto in serie: 2 stati (servo) + 3 stati (Pade-3) = **5 stati**. Verificato: il loop aperto di Task 2 senza notch ha ordine 11 = 6 (plant full) + 2 + 3, senza cancellazioni.

Il **bilancio di fase totale** dell'attuatore alle due frequenze che contano:

| w [rad/s]                | servo  | Pade-3 | totale TVC          |
|--------------------------|--------|--------|---------------------|
| 2.445 (crossover rigido) | -2.80  | -2.80  | <b>-5.60 gradi</b>  |
| 18.9 (omega_BM)          | -22.18 | -21.66 | <b>-43.84 gradi</b> |

Da leggere cosi': al crossover rigido il TVC costa poco (5.6 gradi su 30 di phase margin). A omega\_BM ruota il lobo di bending di **44 gradi** sulla carta di Nichols - e un lobo che sta a +29 dB sopra lo 0 dB, ruotato di 44 gradi, e' cio' che decide se avvolgi o no il punto critico.

**Possibile domanda d'esame** - Perche' il ritardo di 20 ms e' cosi' velenoso per questo loop, che e' *condizionalmente stabile*? *Risposta:* Perche' un loop condizionalmente stabile deve **infilarsi** fra due punti critici sulla Nichols: la fase deve restare in una fascia sia alle basse frequenze (dove il polo aerodinamico instabile a +1.84 rad/s impone un guadagno minimo, l'aero gain margin) sia alle alte (dove il rolloff e il bending impongono un guadagno massimo). Il ritardo aggiunge una fase  $-w\tau$  che cresce **linearmente e senza limite** con la frequenza, e lo fa **a modulo invariato**. Non e' quindi "compensabile" abbassando il guadagno: attenuare non restituisce fase. Alle alte frequenze il ritardo trascina l'intera curva verso sinistra proprio dove il margine e' piu' stretto, e per questo il **delay margin** (classify\_margins.m, riga 64: `min(am.DelayMargin)` su TUTTI i crossover) e' il numero di robustezza piu' stringente del progetto.

## Possibili domande d'esame

**D:** Perche' approssimare il ritardo con Pade invece di usare InputDelay di MATLAB, che sarebbe esatto? **R:** Perche' con un ritardo esatto il modello ad anello chiuso non e' a dimensione finita: `pole(T)`

e `isstable(T)` non sono definite su un sistema con ritardo interno, e `main_task2.m` le usa continuamente (riga 34, righe 60-61 e riga 93, dentro i loop dello Step C per scartare i candidati instabili). Anche il tuner (`design_controller.m`, riga 89) chiama `isstable` a ogni valutazione di costo. Pade rende il loop **razionale** e quindi rende eseguibile tutta la catena a valle. Il prezzo e' che i margini sono accurati solo dove l'approssimante lo e': con ordine 3 e  $\tau = 20$  ms, fino a  $\sim 150$  rad/s – ampiamente sopra il crossover ( $\sim 3$  rad/s) e sopra il bending (18.9 rad/s), quindi il prezzo qui e' trascurabile.

**D: Cosa vuol dire che l'approssimante di Pade e' "all-pass", e perche' e' esattamente la proprieta' che vogliamo?** R: All-pass = modulo unitario a tutte le frequenze,  $|W_{\text{delay}}(j\omega)| = 1$ . Segue da  $N(s) = D(-s)$ : sull'asse immaginario  $N(j\omega)$  e' il coniugato di  $D(j\omega)$ , quindi i moduli si cancellano identicamente. E' la proprieta' del ritardo vero ( $|exp(-j\omega\tau)| = 1$ ): un ritardo **non attenua nulla, sposta solo la fase**. Un modello che invece attenuasse (per esempio un lag  $1/(1+\tau s)$ ) falserebbe il progetto verso l'ottimismo: farebbe sembrare il picco di bending piu' basso di quanto sia, e quindi il notch meno necessario.

**D: Perche' l'approssimante di Pade e' a fase non minima, e che effetto ha sull'anello?** R: Perche' i suoi zeri sono l'immagine speculare dei suoi poli ( $N(s) = D(-s)$ ): se i poli sono stabili, gli zeri sono nel semipiano destro. Per  $\tau = 20$  ms e ordine 3 stanno a  $+232$  rad/s e  $+184 \pm 175j$  rad/s. Uno zero RHP contribuisce fase **negativa** con modulo crescente, cioe' fa esattamente cio' che fa un ritardo. La conseguenza pratica e' un limite fondamentale di banda: non si puo' spingere il crossover verso lo zero RHP senza perdere fase in modo irrecuperabile (regola pratica: crossover ben sotto il modulo dello zero RHP). Qui il crossover e'  $\sim 3$  rad/s contro zeri a  $\sim 230$  rad/s, quindi il vincolo non morde – il ritardo ci fa male attraverso il **bending** e il **delay margin**, non attraverso il crossover rigido.

**D: L'attuatore TVC ha banda 70 rad/s. Non basta il suo rolloff a gain-stabilizzare il bending a 18.9 rad/s?** R: No, ed e' un errore comune. A 18.9 rad/s il servo del secondo ordine ha modulo  $|W_{\text{act}}| = 0.9988$ , cioe' **-0.01 dB**: non attenua niente, perche' il suo corner e' a 70 rad/s, un fattore 3.7 piu' in alto. In compenso gli regala -22 gradi di fase, e il Pade altri -22. Quindi l'attuatore, sul bending, fa **solo danni**: lascia passare i +29 dB del picco e ci aggiunge 44 gradi di ritardo. Il modo va attenuato con un filtro dedicato (`build_notch_filter.m`) oppure fase-stabilizzato – l'attuatore non e' un'opzione.

**D: Il ritardo di 20 ms rappresenta solo il trasporto dell'attuatore?** R: Nella traccia e' dato come ritardo puro dell'Eq. 3, e il codice lo prende cosi' (`p.tau = 0.020` in `load_hw3_params.m`, riga 57). Fisicamente, in un progetto di volo, quel numero e' un aggregato: latenza di calcolo del flight computer, filtraggio dei sensori, campionamento (uno ZOH a periodo T aggiunge un ritardo efficace  $\sim T/2$ ), e il trasporto vero dell'attuatore. Il codice non modella nulla di tutto questo separatamente – e' un singolo  $\tau$  a valle del controllore.

**D: Cosa succede se metto `padeOrder = 1`?** R: La funzione accetta (il validatore chiede solo intero positivo) e il progetto non esplode: a  $\omega_{\text{BM}}$  l'errore di fase e' solo 0.24 gradi. Cambiano invece i numeri ad alta frequenza – al corner dell'attuatore l'errore e' 10 gradi – e quindi cambiano il **delay margin** (che **allmargin** legge anche sui crossover alti) e la posizione dei crossover di guadagno sopra i 100 rad/s. In pratica: i margini pubblicati nel report sono legati alla scelta `padeOrder = 3` fatta a `main_task2.m` riga 30, ed e' onesto dichiararlo.



# HM3/build\_\_notch\_\_filter.m

## Ruolo del file nel progetto

build\_notch\_filter.m costruisce la **sezione di filtro per la stabilizzazione del modo di bending**: una biquadratica con numeratore e denominatore centrati sulla stessa frequenza  $\omega_x$  ma con smorzamenti diversi. E' l'Eq. 4 della traccia, e in assemble\_loop viene messa in cascata con il TVC nel ramo diretto (main\_task2.m riga 140:  $W_{full} = W_{tvc} * H_n$ ).

Serve perche' il modo flessionale del lanciatore ( $\omega_{BM} = 18.9$  rad/s,  $\zeta_{BM} = 0.005$  – praticamente non smorzato) **entra nelle misure** attraverso l'INS (build\_plant\_full.m, righe 33-36:  $\theta_m = \theta + \sigma_{ins} * \eta$ ) e viene **eccitato dal TVC** (riga 27: la colonna delta forza  $\dot{\eta}$  con  $-\phi_{tvc} * T_c$ ). Chiudere il PD su una misura contaminata da un modo a smorzamento 0.005 significa avere nel loop aperto un picco di risonanza di **+29 dB** (il codice calcola  $|L(\omega_{BM})| = 29.01$  dB, che stampa come 29.0 dB) che destabilizza l'anello: main\_task2.m Step B lo mostra con  $isstable(T_b) = 0$ . L'attuatore non aiuta (a 18.9 rad/s attenua -0.01 dB, vedi hm3\_build\_tvc.md), quindi serve un filtro dedicato.

Il file e' **una sola funzione parametrica** che copre due usi opposti, decisi dal flag numSign:

- numSign = +1 -> **notch a fase minima**: zeri complessi nel semipiano sinistro, buco profondo a  $\omega_x$ , guadagno unitario lontano. Serve a **gain-stabilizzare** il modo (abbassarlo sotto 0 dB).
- numSign = -1 (default, "Eq. 4 come stampata") -> **zeri nel semipiano destro**, filtro a **fase non minima**: stesso modulo del precedente, ma fase completamente diversa. Serve a **fase-stabilizzare** il modo (ruotarlo sulla Nichols invece di attenuarlo).

main\_task2.m usa entrambi nel trade study dello Step C (righe 51-113) e alla fine **ritiene il notch profondo a fase minima** (righe 74-78:  $z_N = 0.002$ ,  $z_D = 0.7$ ,  $sgn = +1$ ).

---

## build\_notch\_filter (righe 1-25)

```
function Hx = build_notch_filter(wx, zN, zD, numSign)
num = [1, numSign*2*zN*wx, wx^2];
den = [1, 2*zD*wx, wx^2];
Hx = tf(num, den);
```

- **Riga 1**: firma Hx = build\_notch\_filter(wx, zN, zD, numSign). Quattro scalari -> una tf SISO. Chiamanti: main\_task2.m (righe 58, 71, 78, 81-83, 91, 106), main\_task3.m, main\_montecarlo.m, init\_simulink\_hm3.m, tests/hm3FilterTest.m.
- **Righe 13-18**: blocco arguments. Da notare i vincoli **effettivi**: zN e' mustBeNonnegative (quindi  $z_N = 0$  e' ammesso), zD e' mustBePositive, numSign e' vincolato a {-1, +1}. **Non c'e' nessun limite superiore** su  $z_N/z_D$ : i "guideline 0.1-0.3 / 0.4-0.6" della docstring (righe 6-7) sono un commento, non un vincolo. E infatti il design ritenuto ( $z_N = 0.002$ ) sta **due ordini di grandezza sotto** la guideline. Questa e' una scelta consapevole, non un bug, ma va detta all'orale: il filtro ritenuto **non e' il filtro dell'Eq. 4 con i valori suggeriti**, e' un notch profondo custom.

## Righe 20-21 – la forma della biquadratica

$$H_x(s) = \frac{(s^2 + \text{sgn} \cdot 2 \cdot \text{zeta}_N \cdot \omega_x \cdot s + \omega_x^2)}{(s^2 + 2 \cdot \text{zeta}_D \cdot \omega_x \cdot s + \omega_x^2)}$$

Perche' questa forma e non un'altra. Numeratore e denominatore hanno lo **stesso termine costante**  $\omega_x^2$  e lo **stesso coefficiente di  $s^2$**  (= 1). Le conseguenze sono immediate e sono esattamente quello che si vuole da un filtro di notch:

- **Guadagno unitario in continua:**  $H_x(0) = \omega_x^2 / \omega_x^2 = 1$ . Il filtro non tocca il guadagno di anello a bassa frequenza, quindi **non erode l'aero gain margin** (che vive a  $\sim 0.6$ - $0.8$  rad/s).
- **Guadagno unitario all'infinito:** per  $s \rightarrow \infty$ ,  $H_x \rightarrow s^2/s^2 = 1$ . Il filtro non ha rolloff: **non e' un passa-basso**, non aiuta contro i modi flessionali superiori. E' un buco chirurgico e nient'altro.
- **Relativo di grado 0** (proprio ma non strettamente proprio): non aggiunge ritardo asintotico.

L'unica differenza fra num e den e' il **coefficiente del termine in s**. Quello e' l'unico grado di liberta' che il filtro usa.

**La profondita' del buco.** Valutando in  $s = j \cdot \omega_x$ , i termini  $s^2$  e  $\omega_x^2$  si cancellano esattamente ( $-\omega_x^2 + \omega_x^2 = 0$ ) e resta solo il termine in s:

$$H_x(j \cdot \omega_x) = (\text{sgn} \cdot 2 \cdot \text{zeta}_N \cdot \omega_x \cdot j \cdot \omega_x) / (2 \cdot \text{zeta}_D \cdot \omega_x \cdot j \cdot \omega_x) \\ = \text{sgn} \cdot \text{zeta}_N / \text{zeta}_D$$

$$\Rightarrow |H_x(j \cdot \omega_x)| = \text{zeta}_N / \text{zeta}_D \quad (\text{profondita' del notch})$$

Questa e' l'equazione di progetto. Con  $z_N = 0.002$  e  $z_D = 0.7$ :

$$|H_x(j \cdot \omega_{BM})| = 0.002/0.7 = 0.002857 \rightarrow 20 \cdot \log_{10}(0.002857) = -50.88 \text{ dB}$$

Verificato numericamente: la funzione restituisce esattamente -50.88 dB a 18.9 rad/s. Con  $z_N \ll z_D$  si ottiene il **buco profondo**, con  $z_N = z_D$  il filtro degenera in  $H_x = 1$  (nessun effetto), con  $z_N > z_D$  diventa un **amplificatore risonante**. Il caso limite  $z_N = 0$  (ammesso dal validatore!) mette gli zeri **esattamente sull'asse immaginario** in  $\pm j \cdot \omega_x$  e da' profondita' **infinita** ( $|H_x(j \cdot \omega_x)| = 0$ ), un notch ideale – irrealizzabile in pratica e numericamente fragile.

**Verifica del bilancio di guadagno sul bending** (il conto che chiude il progetto di Task 2):

$$\begin{aligned} |L(\omega_{BM})| \text{ senza filtro} &= +29.01 \text{ dB} \\ \text{profondita' del notch} &= -50.88 \text{ dB} \\ \hline |L(\omega_{BM})| \text{ con il notch} &= -21.87 \text{ dB} \quad (\text{classify\_margins: } L_{\omega_{BM}} \text{ dB} = -21.87, \\ &\quad \text{stampato -21.9 nella tabella}) \end{aligned}$$

L'attenuazione e' **esattamente additiva in dB**, perche' il filtro e' in serie e il suo modulo a  $\omega_x$  non dipende dal resto del loop.

**Nota di onesta' sui commenti del sorgente.** L'header di `main_task2.m` (righe 7-8) dichiara "the +39 dB bending resonance", con il numero scritto alla **riga 7**. Il valore vero, che lo script stampa a runtime alla riga 33 (formato `%.1f`), e' **+29.0 dB** (29.01 dB il valore interno). Il README riporta correttamente +29 dB, ed e' consistente con l'aritmetica qui sopra ( $29.0 - 50.9 = -21.9$  dB, che e' esattamente il numero nella tabella del README). Il **commento a riga 7 di `main_task2.m` e' quindi stale**.

## Righe 20 – il flag numSign e i due modi di stabilizzare

Il numeratore e':

$$\begin{aligned} \text{numSign} = +1: \quad &s^2 + 2 \cdot \text{zeta}_N \cdot \omega_x \cdot s + \omega_x^2 \\ &\text{zeri in } -\text{zeta}_N \cdot \omega_x \pm j \cdot \omega_x \cdot \sqrt{1 - \text{zeta}_N^2} \quad \rightarrow \text{LHP} \\ &\Rightarrow \text{FASE MINIMA} \end{aligned}$$

```
numSign = -1:  s^2 - 2*zeta_N*wx*s + wx^2
               zeri in  +zeta_N*wx +/- j*wx*sqrt(1 - zeta_N^2)  -> RHP
               => FASE NON MINIMA
```

**Il modulo e' identico nei due casi.** Infatti

```
num(j*w) con sgn=-1 =  coniugato di [ num(j*w) con sgn=+1 ]
```

perche' cambia solo il segno della parte immaginaria. Quindi  $|\text{num}|$  e' lo stesso, e **le due varianti hanno lo stesso diagramma di Bode del modulo**. Cambia solo la fase:

```
arg(num_NMP) = -arg(num_MP)
```

```
=> fase(Hx_NMP) - fase(Hx_MP) = -2*arg(num_MP)
```

Sopra  $w_x$ ,  $\arg(\text{num\_MP})$  tende a  $+180$  gradi, quindi la variante a fase non minima **perde ~360 gradi di fase attraversando la risonanza**. E' una rotazione enorme e completamente controllata: e' lo strumento con cui si **ruota il lobo di bending sulla carta di Nichols** attorno al punto critico – che nella convenzione degli appunti del corso sta a **(-180 deg, 0 dB)**, da  $1 + L = 0$  (D'Antuono lo etichetta  $+180$ : stesso punto, fase mod 360, ed e' la rietichettatura che il codice usava fino a poco fa).

**Gain stabilization vs phase stabilization** – la distinzione centrale dell'homework:

|                       | Gain stabilization                                                | Phase stabilization                                  |
|-----------------------|-------------------------------------------------------------------|------------------------------------------------------|
| Obiettivo             | portare il lobo <b>sotto 0 dB</b>                                 | lasciarlo sopra 0 dB ma con la <b>fase giusta</b>    |
| Requisito tipico      | $  L(w_{BM})   \leq -12$ dB (spesso -20)                          | il lobo passa dalla parte “sicura” del punto critico |
| Strumento             | notch profondo, $\text{numSign} = +1$                             | sezione NMP / lead-lag, $\text{numSign} = -1$        |
| Cosa devo sapere bene | <b>omega_BM</b> (la frequenza)                                    | <b>fase e segno della forma modale</b> al sensore    |
| Fragilita'            | detuning di $\omega_{BM}$                                         | errore di fase/segno del mode shape                  |
| Nel codice            | $H_n$ (righe 74-78 di <code>main_task2</code> ) – <b>ritenuto</b> | $H_{l1}$ (riga 71) – <b>scartato</b>                 |

Il progetto di Task 2 sceglie la **gain stabilization**: il notch profondo porta il bending a -21.9 dB (poi -18 dB dopo il ri-tuning del PD), cioe' un **bending gain margin di 18 dB**, sopra i 12 dB tipicamente richiesti. La ragione e' pragmatica: la gain stabilization richiede di conoscere solo *dove* sta il modo, non *con che fase* arriva al sensore – e la fase al sensore dipende dalla forma modale, da  $\sigma_{ins}$ , da  $\phi_{ins}$ , cioe' da parametri strutturali molto meno affidabili della frequenza.

## Il prezzo del notch profondo (due prezzi, entrambi pagati nel codice)

**Prezzo 1 – il detuning.** Il buco e' profondo *ma strettissimo*. Vicino a  $w_x$  si puo' linearizzare: posto  $w = w_x + dw$  con  $dw \ll w_x$ ,

```
wx^2 - w^2 ~ -2*wx*dw
|num(j*w)| ~ 2*wx*sqrt( dw^2 + (zeta_N*wx)^2 )
```

Il fondo del buco ( $dw = 0$ ) vale  $2*wx*zeta_N$ . Il modulo **raddoppia (+6 dB)** gia' quando  $dw = \sqrt{3}*zeta_N*wx$ . Con  $z_N = 0.002$  e  $w_x = 18.9$ :

```
zeta_N * wx = 0.0378 rad/s      -> il "fondo" del notch e' largo ~0.08 rad/s
```

Cioe' **0.4% di omega\_BM**. Un errore dell'1% sulla conoscenza di  $\omega_{BM}$  (0.19 rad/s) e' gia' 5 volte la mezza-larghezza del null. Verifica numerica della profondita' effettiva al variare del disaccordo:

| omega_BM vero    | \ Hx\  [dB]   | \ L(omega_BM)\  risultante      |
|------------------|---------------|---------------------------------|
| 0.90 * wx        | -16.53        | 29.01 - 16.53 = <b>+12.5 dB</b> |
| 0.95 * wx        | -22.71        | <b>+6.3 dB</b>                  |
| <b>1.00 * wx</b> | <b>-50.88</b> | <b>-21.9 dB</b>                 |
| 1.05 * wx        | -23.15        | <b>+5.9 dB</b>                  |
| 1.10 * wx        | -17.38        | <b>+11.6 dB</b>                 |

Un disaccordo del 5% **butta via 28 dB di attenuazione** e riporta il bending sopra 0 dB. E' esattamente lo Step D di `main_task2.m` (righe 163-179), che verifica la stabilita' con i filtri fissi e omega\_BM vero perturbato di +/-10%, e il README riporta il risultato: il notch profondo regge -10% ma **va instabile a +5%**. Da notare l'onestà del dettaglio: la tabella qui sopra e' quasi **simmetrica**, quindi l'asimmetria -10% ok / +5% instabile **non viene dalla profondita'** ma dalla **fase** del loop alla nuova frequenza del modo – cioè da quale lato del punto critico (-180 deg, 0 dB) passa il lobo. Il codice non chiarisce questo punto; la spiegazione va cercata sulla Nichols.

**Prezzo 2 – il ritardo di fase sul crossover rigido.** Sotto wx il notch e' in **ritardo di fase**. Il conto: per  $w < wx$  entrambe le parti reali sono positive, quindi

$$\text{fase}(H_x) = \text{atan}(2 * zeta\_N * wx * w / (wx^2 - w^2)) - \text{atan}(2 * zeta\_D * wx * w / (wx^2 - w^2))$$

e con  $zN \ll zD$  il primo termine e' trascurabile: **resta solo il ritardo del denominatore**, che e' governato da `zeta_D`. Al crossover rigido di Task 1 ( $w = 2.4547$  rad/s, `mR.rigidPM_w`; il README arrotonda a 2.45) il notch ( $zN=0.002$ ,  $zD=0.7$ ,  $wx=18.9$ ) costa **-10.45 gradi**. Confronto con gli altri contributi alla stessa frequenza:

| contributo             | fase a 2.4547 rad/s |
|------------------------|---------------------|
| servo TVC (70 rad/s)   | -2.81 gradi         |
| ritardo 20 ms (Pade-3) | -2.81 gradi         |
| <b>notch profondo</b>  | <b>-10.45 gradi</b> |
| <b>totale</b>          | <b>-16.07 gradi</b> |

**Il notch e' il contributore dominante di ritardo di fase**, piu' dell'attuatore e del ritardo messi insieme. Ed e' precisamente il conto che spiega lo Step C decision di `main_task2.m` (righe 131-140): il phase margin rigido di Task 1 (30.0 gradi – il target su cui `design_controller` converge) meno ~16 gradi da' ~14 gradi – e infatti il codice misura **14.6 gradi** con i guadagni di Task 1 sul loop completo (righe 144-149, `mB.rigidPM_deg`; e' lo stesso numero della tabella del README). Da qui l'obbligo di **ri-tarare il PD sul loop pieno** (righe 152-154), alzando `Kd_th` da 0.44 a 0.69 per recuperare la fase persa.

Nota: `zeta_D` e' il parametro che governa questo prezzo.  $zD = 0.7$  e' generoso (largo) e costa i 10 gradi. Un  $zD$  piu' piccolo restringerebbe il denominatore e ridurrebbe il ritardo lontano da wx, ma renderebbe il filtro piu' risonante e piu' fragile. Il codice non esplora questo trade-off:  $zD = 0.7$  e' fissato a mano alla riga 76 di `main_task2.m`.

## Riga 24 – `Hx.Name = 'Notch_Hx'`

Il nome e' hard-coded a 'Notch\_Hx' **anche quando la funzione viene usata come lead-lag a fase non minima** (`numSign = -1`). E' cosmetico (`Name` non entra in `connect`, che lavora su `InputName/OutputName`), ma e' fuorviante leggendo un H11 che si chiama "Notch".

**Possibile domanda d'esame** – Perche' hai scelto la gain stabilization e non la phase stabilization, visto che l'Eq. 4 della traccia e' stampata proprio nella forma a fase non minima? *Risposta:* Perche' `main_task2.m` (Step C, righe 51-70) fa il trade study e lo misura: sui 75 candidati lead-lag della guideline ( $wx = w_{BM} \pm 4$ ,  $zN$  in 0.1-0.3,  $zD$  in 0.4-0.6) **nessuno** stabilizza da solo il loop in modo soddisfacente – il migliore lascia il bending a +23 dB e un phase margin rigido di 11 gradi. Il motivo strutturale e' che la guideline da' profondita'  $zN/zD$  fra  $0.1/0.6 = -15.6$  dB e  $0.3/0.4 = -2.5$  dB: **troppo poco per un picco di +29 dB**. La sezione NMP puo' solo *ruotare* il lobo, e per

farlo passare dalla parte giusta del punto critico servirebbe conoscere con precisione la fase del modo al sensore ( $\sigma_{ins}$ ,  $\phi_{ins}$ , la forma modale) – informazione molto meno affidabile della sola frequenza. La gain stabilization compra robustezza con l'unico parametro di cui mi fido:  $\omega_{BM}$ .

---

## Possibili domande d'esame

**D: Ricava la profondita' del notch e spiega perche' dipende solo dal rapporto degli smorzamenti.** R: In  $s = j\omega$  i termini  $s^2$  e  $\omega^2$  si cancellano identicamente in numeratore e denominatore ( $-\omega^2 + \omega^2 = 0$ ), perche' i due polinomi hanno lo stesso termine costante e lo stesso coefficiente di  $s^2$ . Resta solo il termine lineare:  $Hx(j\omega) = (2\zeta_N \omega \cdot j\omega) / (2\zeta_D \omega \cdot j\omega) = \zeta_N / \zeta_D$ . Tutto il resto si semplifica, quindi la profondita' e' **puramente il rapporto degli smorzamenti**, indipendente da  $\omega$ . Con 0.002/0.7 si ottengono -50.9 dB. Corollario di progetto: la profondita' e' governata da  $z_N$ , la larghezza (e il ritardo di fase lontano dal buco) da  $z_D$ . Sono due manopole quasi ortogonali.

**D: Perche' il filtro ha guadagno unitario sia in continua sia all'infinito, e perche' e' importante?** R: Perche' num e den condividono il coefficiente di  $s^2$  ( $= 1$ ) e il termine costante ( $\omega^2$ ). In DC:  $\omega^2 / \omega^2 = 1$ ; all'infinito:  $s^2 / s^2 = 1$ . E' importante per due motivi. Primo: **non tocca l'aero gain margin**, che vive a  $\sim 0.6-0.8$  rad/s, dove il filtro e' trasparente in modulo (il loop e' condizionalmente stabile e un'attenuazione a bassa frequenza sarebbe destabilizzante, non conservativa). Secondo: **non e' un passa-basso**. Non da' alcun rolloff, quindi non fa nulla contro modi flessionali di ordine superiore o rumore ad alta frequenza – per quelli servirebbe un filtro diverso.

**D: Gain stabilization e phase stabilization – cosa cambia operativamente?** R: Gain stabilization = porto  $|L(\omega_{BM})|$  ben sotto 0 dB (requisito tipico -12 dB o meglio), cosi' il lobo di bending sulla Nichols non puo' avvolgere il punto critico perche' e' semplicemente troppo in basso. Serve conoscere bene  **$\omega_{BM}$** . Phase stabilization = lascio il lobo sopra 0 dB ma ne aggiusto la fase perche' passi dalla parte sicura del punto critico (mantenendo il numero corretto di avvolgimenti richiesto da Nyquist). Serve conoscere bene la **fase** del modo al sensore, cioe' segno e ampiezza della forma modale – un'informazione strutturale, tipicamente incerta. Qui la scelta e' gain stabilization ( $\text{numSign} = +1$ ,  $z_N = 0.002$ ), e la sezione NMP dell'Eq. 4 ( $\text{numSign} = -1$ ), che e' lo strumento per la phase stabilization, viene testata e scartata.

**D: Il notch profondo costa 10 gradi di fase al crossover rigido. Come li recuperi?** R: Ri-tarando il PD sul loop completo, non su quello rigido. `main_task2.m` riga 153 richiama `design_controller` passando `Wfull = Wtvc*Hn` come catena attuatore: il tuner (`fminsearch` sui log dei guadagni) ottimizza `Kp_th`, `Kd_th` contro gli stessi target (aero  $|GM|$  6 dB, rigid PM 30 gradi) ma **vedendo** il ritardo di fase di attuatore + delay + notch. Il risultato e' `Kd_th` che sale da 0.44 a 0.69: piu' azione derivativa = piu' anticipo di fase al crossover, che ricompra i gradi persi. Il concetto e' quello di D'Antuono: le dinamiche del TVC (e qui anche del filtro) **devono entrare nel piazzamento del PD**, non essere trattate come una perturbazione a posteriori.

**D: Il tuo notch e' fuori dalla guideline della traccia ( $z_N$  0.1-0.3,  $z_D$  0.4-0.6). Come lo giustifichi?** R: Con l'aritmetica. La guideline da' profondita'  $z_N/z_D$  fra -2.5 dB e -15.6 dB. Il picco di bending nel loop e' **+29 dB**. Nessuna combinazione della guideline puo' portarlo sotto 0 dB, e infatti lo sweep sui 75 candidati (`main_task2.m`, righe 54-67) lo conferma. La guideline dell'Eq. 4 descrive una sezione **lead-lag / phase-shaper** (con il numeratore NMP), non un notch di gain-stabilization: e' lo strumento per l'altra strategia. Per gain-stabilizzare serve  $z_N/z_D$  dell'ordine di  $1e-3$ , che e' cio' che il codice usa. Il validatore lo permette (nessun limite superiore su  $z_N/z_D$ ,  $z_N$  solo `mustBeNonnegative`), ma va dichiarato: il filtro ritenuto **non e' l'Eq. 4 con i valori suggeriti**.

**D: Cosa succede se il notch e' disaccordato del 5%?** R: L'attenuazione crolla da -50.9 dB a circa -23 dB, e il bending nel loop torna a +5.9 dB, cioe' **sopra 0 dB**: il modo non e' piu' gain-stabilizzato. Il motivo e' che la mezza-larghezza del fondo del buco vale  $\sim \zeta_N \omega = 0.038$  rad/s, cioe' lo **0.2% di  $\omega_{BM}$** : e' un buco largo 0.08 rad/s su una frequenza di 18.9 rad/s. Lo Step D di `main_task2.m` verifica questo e trova che il notch profondo va instabile a +5% di errore su  $\omega_{BM}$ . La contromisura di corso (il "triplet" di notch a 0.9/1.0/1.1 $\omega_{BM}$ , righe 81-83) copre la banda ma costa  $\sim 30$  gradi di ritardo al crossover rigido e destabilizza il loop rigido (PM = -7.3 gradi nella tabella del README) – il codice lo prova e lo scarta.

# HM3/assemble\_loop.m

## Ruolo del file nel progetto

assemble\_loop.m è il **nodo centrale** di HM3: prende il plant (rigido o completo), la struct dei guadagni e la catena attuatore/filtro, e restituisce i due oggetti su cui vive tutto il resto dell'homework:

- **L** – il **loop aperto SISO**, rotto in corrispondenza di **delta**, nella convenzione  $1 + L$ . E' l'oggetto su cui si legge la carta di Nichols e su cui girano `allmargin` / `classify_margins` / `margin`.
- **T** – il **loop chiuso MIMO**, da  $\{\alpha_w, \theta_{ref}\}$  a  $\{\theta, z, \dot{z}, \delta\}$ . E' l'oggetto che si simula (`lsim` in `simulate_gust_response`) e su cui si testa la stabilità (`isstable`, `pole`).

E' chiamato praticamente da ovunque: `main_task1.m` (indirettamente via `design_controller`), `main_task2.m` (righe 26, 31, 59, 92, 123, 144, 155, 175), `main_task3.m`, `main_montecarlo.m`, e – criticamente – **dentro la funzione di costo del tuner** (`design_controller.m`, riga 82), quindi viene rieseguito centinaia di volte per ogni run di `fminsearch`.

La legge di controllo che implementa è la Eq. dell'assignment:

$$\begin{aligned} \text{delta\_cmd} = & K_p\text{-th} * (\theta_{ref} - \theta_{m}) - K_d\text{-th} * \dot{\theta}_{m} \\ & - K_p\text{-z} * z_m - K_d\text{-z} * \dot{z}_m \end{aligned}$$

cioè un **PD di assetto** più un **feedback debole di deriva laterale** ( $K_p\text{-z} = K_d\text{-z} = -1e-3$ , fissati in `design_controller.m` righe 28-29).

Il punto più delicato del file – e probabilmente di tutto HM3 – è che quel **feedback di posizione laterale**  $K_p\text{-z}$  porta nel loop un **integratore libero**. Il plant ha  $z' = \dot{z}$  con la prima colonna della matrice  $A$  tutta nulla (`build_plant_rigid.m`, righe 11-14):  $z$  non retroagisce su nulla, quindi  $s = 0$  è un **autovalore del plant** e la funzione `delta` →  $z$  ha un polo nell'origine. Comunque piccolo sia  $K_p\text{-z}$ , il ramo  $K_p\text{-z} * z_m$  inietta quel  $1/s$  in  $L$ :  $|L| \rightarrow \infty$  per  $\omega \rightarrow 0$ . E' questo che fa “venire la curva dall'alto” sulla Nichols, che produce **crossing multipli** a bassa frequenza, e che rende un singolo numero di `margin()` privo di significato. E' la ragione per cui esiste `classify_margins.m`.

---

## Firma e contratto (righe 1-21)

```
function [L, T, info] = assemble_loop(G, K, Wact)
arguments
    G {mustBeA(G, 'lti')}
    K (1,1) struct
    Wact = tf(1)
end
if isempty(Wact), Wact = tf(1); end    % [] => ideal actuator
```

- **Riga 1:** tre output. `info` (riga 41) è una struct con i due blocchi IO costruiti, nei campi `Kc` e `Wact`. **Nessun chiamante lo richiede:** in tutto il repo `assemble_loop` è sempre invocato con due output al massimo (`[L, T]`, `[~, T]` o solo `L`), e `init_simulink_hm3.m` non la chiama affatto – ricostruisce i blocchi

da se' (build\_plant\_\*, build\_tvc, build\_notch\_filter). E' quindi un **dead output**: utile solo per debug interattivo.

- **Righe 15-19:** arguments. G deve essere un lti (accetta ss, tf, zpk, genss). K una struct scalare – **nessun controllo sui campi**: se manca Kd\_z la funzione crasha alla riga 25 con un errore poco parlante. E' una fragilita' nota e accettabile (chiamata sempre con struct costruite in casa).
- **Riga 21:** if isempty(Wact), Wact = tf(1); end. Alias documentato: [] e argomento assente significano **attuatore ideale**. E' cosi' che main\_task1.m ottiene il caso "Task 1": design\_controller(G, []). Con Wact = 1 il comando del PD e' **istantaneamente** la deflessione fisica (delta == u\_pd), che e' l'idealizzazione di Task 1.

## Il controllore come guadagno statico (righe 23-27)

```
Kc = ss([K.Kp_th, -K.Kp_th, -K.Kd_th, -K.Kp_z, -K.Kd_z]);
Kc.InputName = {'theta_ref', 'theta_m', 'thetadot_m', 'z_m', 'zdot_m'};
Kc.OutputName = {'u_pd'};
```

- **Riga 25:** il controllore e' una **riga 1x5** promossa a ss (sistema con zero stati, solo matrice D). Applicata al vettore di ingressi:

```
u_pd = [ Kp_th, -Kp_th, -Kd_th, -Kp_z, -Kd_z ] *
        [ theta_ref; theta_m; thetadot_m; z_m; zdot_m ]

      = Kp_th*(theta_ref - theta_m) - Kd_th*thetadot_m
        - Kp_z*z_m - Kd_z*zdot_m
```

che e' esattamente la docstring (riga 3) e la legge della traccia. Il primo e il secondo coefficiente sono +Kp\_th e -Kp\_th: l'errore di assetto.

**Attenzione al segno effettivo del ramo di deriva.** I guadagni sono Kp\_z = Kd\_z = -1e-3 (negativi), quindi le entrate della riga valgono -Kp\_z = +1e-3 e -Kd\_z = +1e-3: sul canale z\_m/zdot\_m il feedback entra in u\_pd con **segno positivo**. Il doppio segno negativo (gain negativo x segno negativo nella legge) e' esattamente cio' che produce l'azione di **load relief** descritta nel README: il veicolo, invece di tenere l'assetto rigidamente, si lascia beccheggiare **verso** il vento e riduce l'incidenza aerodinamica, pagando in deriva laterale. E' la contrapposizione classica attitude-hold vs load-relief/drift-minimum.

**Nessuna azione integrale.** Il PD e' puro. Non e' una dimenticanza: un integratore nel controllore aggiungerebbe -90 gradi di fase a bassa frequenza, proprio dove l'anello ha bisogno di **guadagno minimo** per stabilizzare il polo aerodinamico RHP (l'aero gain margin). Sommato all'integratore gia' presente nel plant (z), darebbe un doppio integratore in DC. E non c'e' errore stazionario da azzerare: la raffica e' un transitorio.

- **Righe 26-27:** nomi di IO. Sono la **chiave del cablaggio**: connect (riga 35) non usa indici, usa i **nomi dei segnali**. theta\_m, thetadot\_m, z\_m, zdot\_m sono esattamente i primi quattro OutputName del plant (build\_plant\_rigid.m riga 33 / build\_plant\_full.m riga 54), quindi connect li collega da solo.

## La catena attuatore (righe 29-32)

```
Wa = ss(Wact);
Wa.InputName = {'u_pd'};
Wa.OutputName = {'delta'};
```

- **Righe 30-32:** la catena Wact (TVC \* Pade \* notch, oppure 1) viene inserita **nel ramo diretto**, fra l'uscita del controllore e l'ingresso delta del plant. Non e' in retroazione, non e' sul sensore: e' un blocco in serie.

Conseguenza: **tutta la fase che TVC + ritardo + notch tolgono finisce dentro L**, e quindi dentro i margini.

Nota: il ramo `u_pd` -> `delta` chiude il cerchio dei nomi. `Wa.OutputName = 'delta'` coincide con `G.InputName{1} = 'delta'`, quindi `connect` chiude l'anello automaticamente.

## Chiusura dell'anello (righe 34-35)

```
T = connect(G, Kc, Wa, {'alpha_w','theta_ref'}, ...  
            {'theta','z','zdot','delta'}, {'delta'});
```

- **Riga 35:** `connect` con quattro gruppi di argomenti.
  1. I **blocchi** (`G`, `Kc`, `Wa`), cablati per nome: `G.theta_m` -> `Kc.theta_m`, ..., `Kc.u_pd` -> `Wa.u_pd`, `Wa.delta` -> `G.delta`. L'anello e' chiuso.
  2. Gli **ingressi esterni** mantenuti: `{'alpha_w','theta_ref'}`. `alpha_w` e' il **disturbo** (entra nel plant sulla seconda colonna di `B`), `theta_ref` il riferimento di assetto.
  3. Le **uscite** mantenute: `{'theta','z','zdot','delta'}`. Nota bene: `theta`, `z`, `zdot` sono le uscite di **plotting** del plant (righe 45-47 di `build_plant_full.m`), cioè' gli stati **veri**, NON le misure INS contaminate `theta_m/z_m`. Scelta corretta e importante: **l'anello si chiude sulla misura sporca, ma si riporta cio' che il veicolo fa davvero**. `delta` e' l'uscita di `Wa`, quindi la deflessione **post-attuatore** (in Task 1, con `Wact = 1`, coincide con `u_pd`).
  4. Il **sesto** argomento della chiamata (l'ultimo), `{'delta'}`: sono gli **analysis point** (APs). `connect` inserisce un punto di rottura *virtuale* sul segnale `delta`: `T` non e' piu' un **ss** ma un **genss**, con dentro l'informazione di dove l'anello puo' essere aperto. **Senza questo, la riga 38 non funzionerebbe**. L'AP e' trasparente in simulazione (chiuso al valore nominale), quindi `T` resta il vero loop chiuso.

## Loop aperto e riduzione (righe 37-39)

```
L = getLoopTransfer(T, 'delta', -1);  
L = minreal(tf(L), 1e-6);
```

- **Riga 38:** `getLoopTransfer(T, 'delta', -1)` apre l'anello all'AP `delta` e restituisce la funzione di trasferimento di anello. Il terzo argomento `-1` e' il **segno del loop**: chiede la `L` nella convenzione **negativa**, cioè' quella per cui il loop chiuso e'  $T = L/(1+L)$ . E' la convenzione che **margin**, **allmargin**, **nichols** e il criterio di Nyquist si aspettano (punto critico in `-1`, ovvero `-180` gradi / `0` dB). Senza il `-1` si otterrebbe la `L` col segno fisico dell'anello (che, essendo in retroazione negativa, vale `-L`) e **tutti i margini uscirebbero ruotati di 180 gradi**.

Nota tecnica: per un anello SISO singolo la `L` **non dipende** da dove si rompe (il prodotto scalare `Kc*Wa*G` e' invariante per rotazione ciclica). Rompere in `delta` e' una scelta **fisica**: e' il punto in cui, su un banco, si inietterebbe uno stimolo nell'attuatore, ed e' il punto rispetto al quale il **delay margin** ha il significato di "quanto ritardo extra sopporta la catena di comando".

- **Riga 39:** `minreal(tf(L), 1e-6)`. Due operazioni. `tf(L)` converte il **genss** in funzione di trasferimento razionale; `minreal` con tolleranza `1e-6` cancella coppie polo-zero quasi coincidenti.

**Nota di onesta':** nei casi nominali `minreal` **non cancella nulla**. Verificato: Task 1 -> `order(L) = 4` = i 4 stati del plant rigido (`Kc` e `Wa` sono statici); Task 2 senza notch -> `order(L) = 11 = 6` (plant full) + 2 (servo TVC) + 3 (Pade-3); Task 2 con notch -> `order(L) = 13`. Nessuna riduzione. La chiamata e' quindi **difensiva**, non necessaria – utile perche' il tuner la ripete centinaia di volte e i modi non-minimi possono comparire con guadagni esotici, ma va detto che nel flusso nominale e' un no-op. Va anche detto il rischio: `tf()` su un sistema di ordine 13 con poli che spaziano da 0 a ~254 rad/s (il polo piu' veloce e' quello del Pade-3 sul ritardo di 20 ms) e' la rappresentazione **numericamente meno condizionata** (i coefficienti del polinomio spaziano molti ordini di grandezza); **ss** o **zpk** sarebbero piu' sicuri. Con ordine 13 il problema non morde, ma e' un punto molle noto.



## PUNTO CRITICO – perche' margin() da solo non significa niente

Questo e' il cuore concettuale del file, e la domanda su cui l'orale si gioca.

### 1. Il plant ha un integratore libero sulla deriva

Nella matrice A del plant (build\_plant\_rigid.m, righe 11-14) la **prima colonna e' identicamente nulla**: lo stato **z** non compare in nessuna equazione. E' un puro integratore di **zdot**. Quindi **s = 0** e' un autovalore. Autovalori del plant rigido, verificati:

```
eig(A) = [ 0 , -1.861 , +0.0291 , +1.8165 ] [rad/s]
```

Il polo in **0** e' l'integratore di posizione laterale. Il polo in **+1.8165** e' il polo aerodinamico instabile (la coppia disaccoppiata sarebbe  $\pm \sqrt{A6} = \pm 1.839$ ; l'accoppiamento con la deriva la sposta leggermente e produce anche il polo lento **+0.0291**). Quindi il loop aperto ha **due poli nel semipiano destro**, non uno.

### 2. Quell'integratore entra in L attraverso Kp\_z\*z\_m

Il ramo **z\_m** del controllore (-Kp\_z nella riga 25) moltiplica una funzione delta -> z che ha un 1/s. Di conseguenza:

```
|L(j*w)| -> infinito per w -> 0 (pendenza -20 dB/dec)
```

Sulla carta di Nichols la curva **entra dall'alto**. E' esattamente cio' che dice il commento di classify\_margins.m (righe 13-14): *"the drift position integrator makes the Nichols come from the top"*.

**E succede comunque, per qualunque Kp\_z diverso da zero.** Kp\_z = -1e-3 e' minuscolo, ma non cambia il fatto: cambia solo **a che frequenza** il lobo di deriva attraversa lo 0 dB, non **se** lo attraversa. Non e' un effetto che si puo' rendere trascurabile abbassando il guadagno – si puo' solo spostare.

### 3. Il risultato: crossing multipli, e allmargin restituisce vettori

allmargin(L) sul loop rigido di Task 1 (guadagni Kp = 1.78, Kd = 0.44):

```
GainMargin : -6.00 dB @ 0.593 rad/s
PhaseMargin : -133.12 , -40.53 , +30.00 gradi
              @ 0.161 , 0.222 , 2.455 rad/s
```

**Tre** crossing di fase. I primi due (0.161 e 0.222 rad/s) sono **artefatti del lobo di deriva**: classify\_margins.m li scarta con la maschera pf > opts.w\_drift (riga 49), con w\_drift = 0.3\*sqrt(A6) = 0.55 rad/s, e li riporta separatamente come mm.drift\_w (riga 62) marcandoli esplicitamente come "NOT rigid-body margins". Il crossing utile e' il terzo: **PM = 30.00 gradi a 2.455 rad/s**, il crossover rigido. (I due numeri cadono esattamente sui target di progetto – 6 dB e 30 gradi – perche' il fminsearch di design\_controller.m minimizza proprio  $(|aeroGM|-6)^2 + (rigidPM-30)^2$ , riga 88, con due gradi di liberta' per due target.)

Sul loop completo di Task 2 (con TVC + Pade + notch, ordine 13, PD ri-tarato – il design effettivamente consegnato, main\_task2.m righe 153-155) la situazione e' peggiore: **5 crossing di guadagno e 3 di fase**.

```
GM dB : -283.43 -6.00 +7.56 +39.18 +87.96
@ w : 0.000 0.542 11.110 129.280 846.680
```

La prima riga, a **w = 0 esatta**, e' l'artefatto numerico dell'integratore – ed e' precisamente il motivo della maschera gf > 0 in classify\_margins.m (maschere alle righe 44-45, commento a riga 43: *"exclude the DC / integrator entry at gf == 0"*).

### 4. E c'e' anche la stabilita' condizionale

Con **due poli RHP** in anello aperto (P = 2), il criterio di Nyquist richiede esattamente **2 avvolgimenti antiorari** del punto critico. Conseguenza: il loop e' **condizionalmente stabile**, cioe' stabile solo per guadagni in una **fascia [K\_min, K\_max]**, non in (0, K\_max]. Se il guadagno **scende** troppo il polo aerodinamico non viene piu' stabilizzato; se **sale** troppo si destabilizza dal lato alto.

Questo e' visibile nei segni dei gain margin:

- **Aero GM** = margine di **riduzione** di guadagno, con `gmdb < 0` (`classify_margins.m`, riga 44: maschera `gmdb < 0`). Qui **-6.00 dB**: il loop tollera al massimo 6 dB **in meno** di guadagno.
- **Rigid GM** = margine di **aumento**, con `gmdb > 0` (riga 45). Nel loop rigido di Task 1 **non esiste** (attuatore ideale, nessun rolloff -> nessun crossing di guadagno in aumento). Compare in Task 2, a +7.56 dB @ 11.1 rad/s.

Un "gain margin" letto ingenuamente come "quanto guadagno posso aggiungere" e' quindi **letteralmente al contrario** del numero che conta di piu' in questo progetto.

## 5. Cosa restituisce `margin()` davvero

Verificato sul loop di Task 1: `margin(L)` restituisce `GM = -6.00 dB @ 0.593`, `PM = 30.00 deg @ 2.455`. Per **onesta'**: in questo caso specifico il PM che `margin` sceglie e' proprio quello rigido, quindi non e' "sbagliato" - ma:

- **nasconde** gli altri due crossing di fase (-133 e -40 gradi), che sono la firma del lobo di deriva e della stabilita' condizionale;
- restituisce un GM **negativo** (-6.00 dB), che un lettore distratto interpreterebbe come "instabile", mentre e' il margine di **riduzione** ed e' esattamente il target di progetto (6 dB);
- sul loop completo di Task 2 **nasconde il Rigid GM** (+7.56 dB @ 11.1 rad/s), che e' il margine che il ri-tuning del PD deve preservare.

Cioe': il problema non e' che `margin` restituisce un numero *falso*, e' che restituisce **un** numero dove ne servono **cinque**, tutti con un significato fisico diverso, e senza dire a quale banda appartiene. Da qui `classify_margins.m`, che li **bina per frequenza** (deriva / rigido / flex) e li riporta con il nome del fenomeno che rappresentano.

**Possibile domanda d'esame** - Perche' il loop include il feedback di posizione laterale, visto che complica tutta la lettura dei margini? Non potevi mettere `Kp_z = 0`? *Risposta:* Perche' senza feedback di deriva il veicolo tiene l'assetto e incassa **tutta** l'incidenza del vento: a max-q e' la condizione di carico peggiore. Il termine `Kp_z*z` (con `Kp_z` negativo) e' l'azione di **load relief / drift-minimum**: lascia beccheggiare il veicolo verso il vento riducendo `q_bar*alpha`, al prezzo di deriva laterale. Il costo di progetto e' quello descritto sopra - e c'e' anche un costo quantitativo, dichiarato dalla docstring di `design_controller.m` (righe 8-10): *"the lateral-drift feedback erodes the aerodynamic gain margin (the canonical decoupled 6 dB drops to ~4 dB on the full loop)"*. E' proprio per questo che il tuner ri-tara i guadagni **sul loop pieno** invece di fidarsi della formula di piazzamento disaccoppiata  $K_p = 2*A6/K1$ ,  $K_d = \text{sqrt}(A6)/K1$ .

---

## Possibili domande d'esame

**D: Perche' la funzione restituisce sia L sia T? Non basta uno dei due?** R: No, servono entrambi e servono a cose diverse. L e' **SISO** e vive nel dominio della frequenza: e' l'oggetto su cui si legge Nichols, si calcolano `allmargin/classify_margins`, si progettano guadagno e fase. T e' **MIMO** (2 ingressi, 4 uscite) e vive nel tempo: e' cio' che si simula con `lsim` e su cui si chiama `isstable/pole`. Il punto decisivo e' che **alpha\_w non compare in L**: la raffica e' un disturbo che entra nel plant in un punto diverso dalla rottura dell'anello, quindi da L non si puo' ricavare la risposta alla raffica. Viceversa da T non si leggono i margini. E siccome i due escono dallo **stesso connect**, non possono divergere fra loro (nessun rischio di simulare un modello diverso da quello su cui si sono letti i margini).

**D: Cos'e' l'analysis point {'delta'} nel connect, e cosa succede se lo tolgo?** R: E' un punto di rottura virtuale che `connect` inserisce sul segnale `delta`. Trasforma T da `ss` a `genss` e memorizza **dove** l'anello puo' essere aperto. Senza di esso `getLoopTransfer(T, 'delta', -1)` (riga 38) non avrebbe un punto a cui riferirsi e fallirebbe: dopo la chiusura, `delta` sarebbe solo un filo interno. L'AP e' trasparente in simulazione (viene chiuso al valore nominale), quindi T resta il vero anello chiuso e `lsim` da' la risposta corretta.

**D: Perché getLoopTransfer con il segno -1?** R: Perché margin, allmargin, nichols e il criterio di Nyquist lavorano nella convenzione  $1 + L$ , con punto critico in -1 (equivalentemente -180 gradi / 0 dB sulla Nichols). L'anello fisico è in retroazione negativa, quindi il guadagno d'anello "come cablato" vale  $-L$ . Il flag -1 chiede a MATLAB di restituire la  $L$  **già nella convenzione positiva**, quella per cui il loop chiuso è  $L/(1+L)$ . Senza il flag tutti i margini uscirebbero sfasati di 180 gradi e i grafici di Nichols sarebbero letti sul punto critico sbagliato.

**D: L'integratore libero della deriva – da dove viene esattamente, e perché non lo puoi eliminare abbassando Kp\_z?** R: Viene dal plant, non dal controllore: nella matrice A la prima colonna è nulla (build\_plant\_rigid.m righe 11-14), cioè lo stato  $z$  non retroaziona su nessuna equazione – è un puro integratore di  $zdot$ . Quindi  $s = 0$  è un autovalore del plant e la funzione delta  $\rightarrow z$  ha un polo nell'origine. Il ramo  $Kp\_z * z\_m$  lo porta dentro  $L$ . Abbassare  $Kp\_z$  **non lo elimina**: il termine  $Kp\_z/s$  domina comunque per  $w \rightarrow 0$ , qualunque sia  $Kp\_z \neq 0$ . Cambia solo la frequenza a cui il lobo di deriva attraversa lo 0 dB (più  $Kp\_z$  è piccolo, più bassa). L'unico modo di eliminarlo è  $Kp\_z = 0$  esatto – cioè rinunciare al load relief.

**D: Il loop è condizionalmente stabile. Cosa significa operativamente sui margini?** R: Che il guadagno d'anello ammissibile è una **fascia** e non una semiretta. Il plant ha due poli RHP (verificato: +1.8165 e +0.0291 rad/s), quindi Nyquist richiede 2 avvolgimenti antiorari del punto critico: se il guadagno **scende** sotto una soglia perdo gli avvolgimenti e il polo aerodinamico non è più stabilizzato; se **sale** troppo destabilizzo dal lato alto. Operativamente: esistono **due** gain margin, uno di riduzione (l'**Aero GM**, negativo in dB, il target da 6 dB dell'assignment) e uno di aumento (il **Rigid GM**, positivo, che compare solo quando c'è rolloff, cioè da Task 2 in poi). Sulla Nichols la curva si **infiltra fra i due punti critici** invece di stare tutta da una parte – è proprio la firma grafica della stabilità condizionale, visibile nelle figure di Task 1 e Task 2.

**D: Le uscite di T sono theta, z, zdot – perché non theta\_m, z\_m?** R: Perché l'anello si chiude sulle **misure** (contaminate dal bending via INS, build\_plant\_full.m righe 33-36:  $\theta_m = \theta + \sigma_{ins} * \eta$ ) ma si vuole riportare ciò che il **veicolo fa davvero**. Il plant espone entrambe le famiglie di uscite apposta: le prime quattro ( $\theta_m, \dots, zdot\_m$ ) vanno al controllore, le ultime tre ( $\theta, z, zdot$ , righe 45-47) sono gli stati veri per il plotting. connect prende le prime per cablare e le seconde come uscite esterne. È la distinzione fra "cosa vede il sensore" e "cosa succede al razzo", ed è esattamente il motivo per cui il bending è pericoloso: **lo vedi anche se non c'è**, e il controllore reagisce a un fantasma.

**D: minreal(tf(L), 1e-6) – serve davvero?** R: Nei casi nominali no. Verificato: order(L) vale 4 in Task 1 (= i 4 stati del plant, dato che  $K_c$  e  $W_a$  sono statici), 11 in Task 2 senza notch (6+2+3) e 13 con il notch. **Nessuna cancellazione avviene**. La chiamata è difensiva: assemble\_loop gira dentro fminsearch (design\_controller.m riga 82) centinaia di volte con guadagni arbitrari, e lì una quasi-cancellazione può comparire. Il rovescio della medaglia è che tf() su un sistema di ordine 13 con poli da 0 a ~254 rad/s è la rappresentazione numericamente peggio condizionata; zpk o ss sarebbero più robusti. È un punto molle noto, non un bug attivo.

# HM3/load\_\_wind\_\_profile.m

## Ruolo del file nel progetto

E' il **generatore del disturbo** di tutto HM3. Produce una storia temporale di vento e la converte in un **angolo d'attacco di disturbo**  $\alpha_w(t)$ , che e' l'unico ingresso esogeno del plant (`build_plant_rigid / build_plant_full` hanno ingressi `[delta, alpha_w]`). Restituisce una struct `w` che `simulate_gust_response` passa direttamente a `lsim`.

Chi lo chiama (verificato con `grep`): `main_task1.m` (riga 44), `main_task2.m` (riga 182), `main_task3.m` (riga 43), `main_montecarlo.m` (riga 45), `init_simulink_hm3.m` (riga 60) e `LTV_FULL_ASCENT/main_full_ascent.m` (riga 52). Tutti i `main_task*` passano `Tend = 80` invece del default 12, per lasciar decadere il modo lento di deriva ( $\tau \sim 18$  s ad anello chiuso).

Il file fa due cose ben distinte:

1. **profili analitici** (`gust`, `step`, `doublet`) – deterministici, calcolati a mano, ampiezza presa dalla dispersione di `drywind.mat`;
2. **profilo strongwind** – simula il modello Simulink del professore (`strong_wind.slx`: vento medio + turbolenza di Dryden schedulata in quota), e ne ritaglia una finestra intorno al max-qbar.

Il profilo di default (`gust`, severita' `severe`) e' quello della traccia: raffica 1-coseno con picco  $V_g = 6.38$  m/s ->  $\alpha_w = 0.39$  gradi.

---

## Firma, docstring e blocco arguments (righe 1-28)

```
function w = load_wind_profile(p, o)
% Wind angle-of-attack disturbance for the gust simulation.
%   alpha_w = v_w/V.
```

- Riga 1: `p` e' la struct dei parametri (`load_hw3_params`); di essa usa solo tre campi: `p.V` (velocita' relativa), `p.Alt` (quota, per pescare la dispersione del vento) e `p.t_ref` (solo nel ramo `strongwind`).
- Righe 19-28: `arguments` con name-value. Default: `profile = 'gust'`, `severity = 'severe'`, `Vg = []` (= "calcolamela tu"), `Tg = 3 s`, `Tend = 12 s`, `dt = 0.005 s`, `t0 = 1 s`.
- Riga 22: `severity` e' validata con `mustBeMember` -> errore immediato se si scrive male. `profile` invece e' solo `mustBeTextScalar`: il controllo vero avviene nel `switch` (riga 63-64), che lancia `load_wind_profile:profile`. E' testato in `tests/hm3LoopTest.m` (righe 62-64).
- Riga 23: `Vg` e' validato `mustBeScalarOrEmpty` ma **non** `mustBePositive`: un `Vg` negativo passerebbe silenziosamente (produce semplicemente una raffica di segno opposto).
- `dt = 0.005 s = 200 Hz`. La frequenza di Nyquist e'  $\pi/dt = 628$  rad/s, ampiamente sopra il bending ( $\omega_{BM} = 18.9$ ) e sopra l'attuatore TVC (70 rad/s): il campionamento non introduce aliasing nella `lsim`.

## Il ramo strongwind: uscita anticipata (righe 30-34)

```
if strcmpi(o.profile, 'strongwind')
    w = local_strong_wind(p, o);
    return;
end
```

Dispatch precoce: il ramo Simulink non condivide nulla con i profili analitici (nemmeno il calcolo di  $V_g$ ). Vedi sotto, righe 71-112.

## Ampiezza della raffica dalla dispersione di drywind.mat (righe 36-49)

```
Vg = o.Vg;
if isempty(Vg)
    dwfile = fullfile(fileparts(mfilename('fullpath')), ...
        'General','hw3-v3','drywind.mat');
    if.isfile(dwfile)
        S = load(dwfile); dw = S.drywind;
        alt_km = p.Alt/1000; % drywind.alt is in km
        sig = dw.sigma.(o.severity);
        Vg = interp1(dw.alt, sig, alt_km, 'linear', 'extrap');
    else
        Vg = 8.0; % fallback [m/s]
    end
end
```

- Righe 39-40: il dataset del professore, HM3/General/hw3-v3/drywind.mat. Ispezionandolo (verificato) contiene:
  - dw.alt = [1 2 4 6 8 10 12 14 18 20 25 30] – quote in km;
  - dw.sigma.light / .moderate / .severe – intensita' di turbolenza (deviazione standard, m/s) alle stesse quote;
  - dw.Lh – scale di lunghezza della turbolenza (usate dal modello Dryden, non da questa funzione).
- Riga 43: alt\_km = p.Alt/1000 – la conversione e' **necessaria** perche' la griglia e' in km e p.Alt in m. Il commento nel codice lo dice esplicitamente. E' l'errore classico che qui e' stato evitato.
- Riga 45: interpolazione lineare a Alt = 15143 m -> 15.143 km. Il punto cade fra 14 e 18 km, quindi l'extrap non entra mai in gioco al max-qbar (ma proteggerebbe una chiamata a quota fuori griglia).

Valori ottenuti eseguendo il codice a  $t = 72$  s (Alt = 15143 m,  $V = 937.709$  m/s):

| severity         | $V_g$ [m/s]   | alpha_w di picco    |
|------------------|---------------|---------------------|
| light            | 0.2543        | 0.0155 gradi        |
| moderate         | 2.5686        | 0.1569 gradi        |
| severe (default) | <b>6.3785</b> | <b>0.3897 gradi</b> |

Il  $V_g = 6.4$  m/s e il picco  $\alpha_w = 0.39$  gradi citati nel README e nel report vengono **da qui**, non da un numero scritto a mano.

- Righe 46-48: se il .mat non c'e', fallback a  $V_g = 8.0$  m/s. E' un valore arbitrario, piu' severo del dato reale (6.38): non rompe nulla ma **cambia silenziosamente il risultato**. Vale la pena saperlo se una simulazione restituisce numeri che non tornano con il report.

**Approssimazione da dichiarare all'orale.** dw.sigma.severe e' una **deviazione standard di turbolenza** (l'intensita' RMS che il modello di Dryden usa per generare un processo stocastico). Qui viene riutilizzata come **ampiezza di picco** di una raffica discreta deterministica. Non e' la definizione

standard di una discrete gust (nelle norme aeronautiche l'ampiezza di progetto e' legata alla scala della raffica e a un fattore di attenuazione, non a sigma). E' una scorciatoia ingegneristica legittima – “prendo l'intensita' severa a questa quota e la uso come picco” – ma va chiamata con il suo nome:  $V_g$  non e' un picco a 1 sigma di un processo, e' un'ampiezza scelta.

---

## I tre profili analitici (righe 51-65)

```
t = 0:o.dt:o.Tend;
vw = zeros(size(t));
switch lower(o.profile)
case 'gust' % 1-cosine discrete gust
    idx = t >= o.t0 & t <= o.t0 + o.Tg;
    vw(idx) = 0.5*Vg*(1 - cos(2*pi*(t(idx)-o.t0)/o.Tg));
```

### gust – la raffica 1-coseno (righe 54-56)

$$v_w(t) = 0.5 \cdot V_g \cdot [1 - \cos(2\pi \cdot (t - t_0) / T_g)] \quad \text{per } t_0 \leq t \leq t_0 + T_g$$
$$v_w(t) = 0 \quad \text{altrove}$$

E' la forma “1-coseno a periodo pieno”: l'argomento del coseno percorre 0 ->  $2\pi$  sull'intervallo  $[t_0, t_0 + T_g]$ , quindi la raffica **parte da zero, sale al picco e torna a zero**. Con i default ( $t_0 = 1$ ,  $T_g = 3$ ):

- $v_w(1) = 0$ ,  $v_w(2.5) = V_g = 6.38$  m/s (picco),  $v_w(4) = 0$ ;
- anche la **derivata** e' nulla agli estremi ( $\sin(0) = \sin(2\pi) = 0$ ), quindi il profilo e'  $C^1$ : non ci sono gradini ne' spigoli.

La continuita'  $C^1$  non e' un dettaglio estetico. Lo spettro di un impulso 1-coseno (finestra di Hann) decade asintoticamente come  $1/w^3$ : alla frequenza del bending ( $w_{BM} = 18.9$  rad/s, cioe' circa 9 volte  $2\pi/T_g = 2.09$  rad/s) l'energia della raffica e' ormai decine di dB sotto il picco. Conseguenza pratica e **limite onesto**: la risposta al gusto **non e' un test del modo flessibile**. La raffica eccita la banda rigida (crossover 2.45-3.2 rad/s), che e' esattamente dove serve, ma la stabilizzazione del bending e' giustificata in frequenza (Nichols,  $|L(w_{BM})| = -18$  dB), non da questa simulazione temporale.

### step (righe 57-58) e doublet (righe 59-62)

- **step**:  $v_w = V_g$  per  $t \geq t_0$ . Discontinuo -> spettro  $1/w$ , contenuto a banda larga. Serve a leggere il comportamento **a regime**.
- **doublet**:  $+V_g$  sulla prima meta' di  $T_g$ ,  $-V_g$  sulla seconda. Due gradini -> eccita bene, ma e' un'onda quadra: contiene energia a tutte le frequenze, bending compreso. E' il profilo giusto se si vuole *provocare* il modo flessibile, non quello della traccia.

Nessuno dei due e' usato dai `main_task*`; esistono per i test (`tests/hm3LoopTest.m`, riga 50) e per esplorazione.

---

## Dal vento all'angolo d'attacco: $\alpha_{haw} = v_w/p.V$ (righe 67-69)

```
w = struct('t',t,'vw',vw,'alphaw',vw/p.V, 'V',p.V, ...
          'Vg',Vg,'Tg',o.Tg,'profile',o.profile,'severity',o.severity);
```

E' una riga sola, ed e' il cuore fisico del file.

## La derivazione

Il veicolo vola con velocità relativa  $V$  lungo il suo asse. Un vento **laterale**  $v_w$  non cambia (al primo ordine) il modulo della velocità relativa, ma ne **ruota la direzione**: il vettore velocità dell'aria rispetto al veicolo acquista una componente trasversale  $-v_w$ . L'incidenza indotta è quindi l'angolo fra l'asse e il nuovo vento relativo:

$$\alpha_w = \arctan(v_w / V) \sim v_w / V \quad (\text{piccoli angoli})$$

Numeri:  $\alpha_w = 6.3785/937.709 = 0.006802 \text{ rad} = 0.3897 \text{ gradi}$ . L'errore dell'approssimazione ai piccoli angoli è  $\alpha^2/3 \sim 1.5e-5$  in relativo: del tutto trascurabile. Ha senso ricordarlo:  **$V$  è 150 volte più grande di  $v_w$** , quindi anche una raffica “severa” di 6.4 m/s produce meno di mezzo grado di incidenza. Non è la raffica a essere grande, è il  **$q$  a essere enorme**.

## Perché il vento entra come disturbo su $\alpha$ e non come forza

Perché nel modello linearizzato **tutta** la forza e il momento aerodinamici sono proporzionali all'incidenza. La riga di  $\ddot{\theta}$  del plant è

$$\begin{aligned}\ddot{\theta} &= A_6\theta + (A_6/V)\dot{z} + K_1\delta - A_6\alpha_w \\ &= A_6(\theta + \dot{z}/V - \alpha_w) + K_1\delta \\ &= A_6\alpha + K_1\delta\end{aligned}$$

e la riga di  $\ddot{z}$ , raccogliendo  $a_1$ :

$$\begin{aligned}\ddot{z} &= a_1\dot{z} + (a_1V + a_4)\theta + a_3\delta - a_1V\alpha_w \\ &= a_1V(\theta + \dot{z}/V - \alpha_w) + a_4\theta + a_3\delta \\ &= a_1V\alpha + a_4\theta + a_3\delta\end{aligned}$$

**Entrambe** le righe dipendono dal vento **solo attraverso la combinazione**  $\alpha = \theta + \dot{z}/V - \alpha_w$ . Di qui il fatto – verificabile leggendo `build_plant_rigid` – che la colonna del vento è

$$B_w = [0; -a_1V; 0; -A_6]$$

cioè la colonna di  $\theta$  privata dei termini non aerodinamici e cambiata di segno:  $A_6$  (momento) e  $a_1V$  (forza normale) rispondono all'incidenza,  $a_4$  no (è la componente di spinta/resistenza, che dipende dall'assetto, non dall'incidenza). Questo è il motivo strutturale per cui **non serve un ingresso di forza separato**: modellare il vento come  $\alpha_w$  è *esatto* dentro l'ipotesi di linearizzazione, non è una comodità.

Ordine di grandezza della forza corrispondente:  $N_\alpha * \alpha_w = 1.07e6 * 0.006802 = 7.3 \text{ kN}$ , che su una massa di 73.8 t vale  $7278/73800 = 0.099 \text{ m/s}^2$ , cioè appena **0.01 g** di accelerazione laterale; e momento  $N_\alpha * l_\alpha * \alpha_w = 75.6 \text{ kNm}$ . Il TVC deve compensarlo con  $\delta \sim (A_6/K_1) * \alpha_w = 0.741 * 0.39 = 0.29 \text{ gradi}$  a regime – coerente con il picco simulato di 0.53 gradi (che include il transitorio).

**IL SEGNO DI  $\alpha_w$  – il punto d'orale più redditizio di questa pagina.** Codice e report usano oggi, coerentemente, il segno **meno**:

$$\alpha = \theta + \dot{z}/V - \alpha_w$$

Lo usa il **plant** (colonna  $B_w = [0; -a_1V; 0; -A_6]$ , `build_plant_rigid.m` riga 17) e lo usa il **post-processing** (`simulate_gust_response.m`, riga 29). Il report lo deriva in `Introduction.tex` (righe 61-64) e lo riusa nell'indicatore di carico (riga 149, e `Task1.tex` riga 240).

**Perché il meno è l'unico segno possibile.**  $\alpha_w = v_w/V$  con  $v_w$  la velocità *dell'aria*. L'incidenza è l'angolo fra l'asse del veicolo e il vento **relativo**, quindi dipende dalla velocità laterale del veicolo **rispetto all'aria**,  $\dot{z} - v_w$ , e non da  $\dot{z}$  in assoluto:

$$\alpha = \theta + (\dot{z} - v_w)/V = \theta + \dot{z}/V - \alpha_w$$

Il segno non è una convenzione libera: è **imposto dall'Eq. (1)**. La colonna del vento ha  $-a_1V$  e  $-A_6$ , cioè esattamente i coefficienti con cui  $\theta$  entra tramite  $\alpha$ , **cambiati di segno**. Se l'incidenza fosse  $\theta + \dot{z}/V + \alpha_w$ , la colonna dovrebbe essere  $[0; +a_1V; 0; +A_6]$ . Un + nel post-processing contraddirebbe il plant che si sta simulando.

**Prova indipendente (la piu' bella da citare all'orale).** Con la retroazione di deriva disattivata ( $Kp_z = Kd_z = 0$ ) e una raffica a gradino, la simulazione converge a  $z\_dot = +6.379$  m/s =  $+V*\alpha_w$  – esattamente il valore che annulla  $\alpha = \theta + z\_dot/V - \alpha_w$  con  $\theta = 0$ . Cioe': il veicolo si lascia trascinare sottovento finche' la sua velocita' laterale eguaglia quella del vento, e l'incidenza relativa sparisce. Se il segno fosse +, l'incidenza a regime raddoppierebbe invece di annullarsi – fisicamente assurdo. (Coerentemente, Task1.tex riga 261 riporta  $z\_dot\_ss = +V*\alpha_w$ .)

**Nota storica, ed e' esattamente la domanda che un esaminatore ama.** Fino a poco fa `simulate_gust_response.m` calcolava l'indicatore di carico con il segno **piu'**, mentre il plant aveva (correttamente) il meno. Il plant non e' mai stato sbagliato: il bug era **solo** nel post-processing, cioe' nella ricostruzione a valle di  $\alpha$ . Conseguenze numeriche, sul Task 1:

| formula                          | picco $ \alpha $   | $qbar*\alpha$             |
|----------------------------------|--------------------|---------------------------|
| $+\alpha_w$ (vecchio, sbagliato) | 0.255 gradi        | 20.7 kPa*grado            |
| $-\alpha_w$ (attuale, corretto)  | <b>0.577 gradi</b> | <b>**46.8 kPa*grado**</b> |

Cioe' **piu' del doppio**, e – soprattutto – si **ribalta la conclusione fisica**. Con il +, il picco di incidenza (0.255 gradi) risultava *sotto* il contributo del solo vento (0.390 gradi), e sembrava che l'anello facesse **load relief**. Con il segno giusto il picco (0.577 gradi) lo **supera**: per tenere l'assetto il loop becca il muso **dentro** il vento relativo, e quel contributo si **somma** a quello del vento. Una legge di puro attitude-hold e' quindi **load-aggravating**, non load-relieving.

**Cosa non e' cambiato:** stabilita', margini, Nichols, notch, Task 3, Monte Carlo.  $\alpha_w$  e' un ingresso di **disturbo** e non entra in  $L(s)$ , quindi il segno non tocca la dinamica di anello chiuso. Non cambiano nemmeno i picchi di  $\theta$ ,  $z$  e  $\delta$ : cambia **solo** la metrica di carico, cioe' l'unica quantita' ricostruita a valle dall' $\alpha$ .

## Perche' il max-qbar e' la condizione critica

$qbar = 0.5*\rho*V^2$ . Salendo,  $V$  cresce e  $\rho$  cala esponenzialmente ( $\rho = 1.225*\exp(-h/8000)$  in `load_hw3_params`, riga 83): il prodotto ha un **massimo**, che per questo lanciatore cade a  $t = 72$  s, quota 15143 m, con  $\rho = 0.1845$  kg/m<sup>3</sup>,  $V = 937.7$  m/s ->  $qbar = 81.1$  kPa.

Tre cose diventano critiche **contemporaneamente** in quel punto:

1. **Carico strutturale.** Il momento flettente su un lanciatore snello e' proporzionale alla forza normale distribuita, cioe' a  $qbar * \alpha$ . Il prodotto  $qbar*\alpha$  (in *Pagrado*) e' **l'indicatore di carico standard**, ed e' massimo dove  $qbar$  e' massimo – tanto piu' che le quote 10-15 km sono la fascia del jet stream, cioe' dove i venti sono\* piu' forti. Massimo  $qbar$  e massimo vento nello stesso punto: e' li' che si dimensiona la struttura.
2. **Instabilita' aerodinamica massima.**  $A_6 = \mu_\alpha$  e' proporzionale a  $qbar$  ( $A_6 = N_\alpha*l_\alpha/I_{yy}$  con  $N_\alpha \sim qbar*S*C_{N_\alpha}$ ): il polo instabile  $+sqrt(A_6) = +1.84$  rad/s e' **piu' a destra che mai** proprio a max-qbar. Il punto piu' difficile da controllare e il punto piu' caricato coincidono.
3. **Conflitto di progetto, e con esso un mito da smontare.** Tenere l'assetto ( $\theta \rightarrow 0$ ) contro una raffica significa **mantenere l'incidenza**, cioe' **caricare** la struttura. La contromossa istintiva sarebbe “lascia che il veicolo si allinei al vento” (*weathervaning*): ma **su questo lanciatore la banderuola non esiste**. Con  $A_6 > 0$  il centro di pressione sta **davanti** al baricentro, quindi il momento aerodinamico e' **divergente**, non raddrizzante: abbandonato a se' stesso il veicolo non si allinea al vento relativo, ci **diverge contro**. Non c'e' nessuna stabilita' a banderuola da cui estrarre load relief gratis. E' il motivo per cui il load relief, quando serve davvero, va **costruito** (guadagni piccoli e negativi sulla deriva,  $Kp_z = Kd_z = -1e-3$ , o – in un progetto di volo vero – una retroazione esplicita su un accelerometro o su una stima di  $\alpha$ ).

Ecco perche' HM3 progetta a tempo congelato **proprio** a  $t = 72$  s: e' il caso peggiore su tutti e tre i fronti. Con la convenzione corretta il picco vale  $qbar*\alpha = 46.8$  kPa\*grado (README e report riportano ~47): resta



un ordine di grandezza sotto i limiti strutturali tipici, ma il valore di soglia di riferimento **non e' nel codice** (non e' calcolato, e' un'affermazione del testo).

---

## local\_strong\_wind (righe 71-112)

Il ramo “professore”: invece di una raffica analitica, simula il modello `strong_wind.slx` (vento medio + turbolenza di Dryden schedulata in quota) e ne ritaglia una finestra intorno al `max-qbar`.

```
load_system(fullfile(gdir,'strong_wind.slx'));
cleanup = onCleanup(@() close_system('strong_wind', 0)); % discard edits
```

- Righe 77-80: carica `drywind.mat` e `GreensiteLPV_DATA.mat` (il modello ne ha bisogno come variabili di workspace).
- Righe 82-83: `load_system` (non `open_system`: nessuna finestra) e `onCleanup` con `close_system(...,0)` -> **il .slx non viene mai modificato su disco**, anche se la funzione esce per errore. Il secondo argomento 0 significa “chiudi senza salvare”. E' una precauzione corretta e deliberata (il file e' materiale del corso, non va toccato).
- Righe 85-89: il logging viene attivato **sulle porte di uscita del sottosistema**, non sulle linee di segnale – il commento di riga 87 lo dice. E' la ragione per cui si passa da `get_param(..., 'PortHandles')` e si fa `set_param(ph.Outport(k), 'Name', ..., 'DataLogging', 'on')`. Questo e' un edit al modello in memoria: motivo in piu' per l'`onCleanup`.
- Riga 91: `t1 = p.t_ref - o.t0 = 72 - 1 = 71 s`. E' l'**allineamento temporale**: la finestra estratta parte 1 s prima del `max-qbar`, cosi' che l'istante locale `t = 1 s` (che nei profili analitici e' l'inizio della raffica) corrisponda esattamente a `t = 72 s` di volo. Elegante: le due famiglie di profili sono confrontabili nello stesso grafico.
- Righe 92-97: `Simulink.SimulationInput`, con le due variabili iniettate e `StopTime = t1 + Tend = 83 s`.
- Righe 99-107: si estraggono i due segnali (`sw_vwp` = vento medio, `sw_turb` = turbolenza), si eliminano i tempi duplicati con `unique` (i log a passo variabile possono ripetere lo stesso istante – ed `interp1` con ascisse ripetute fallisce), e si **ricampiona a passo fisso `dt`** sulla finestra `[t1, t1+Tend]`.
- Riga 106-107: `vw` = vento\_medio + turbolenza. Il totale, non solo la turbolenza.
- Righe 109-111: stessa struct dei profili analitici, cosi' che `simulate_gust_response` non debba sapere da dove viene il vento. Ma: `Vg = max(abs(vw))` (non e' piu' un'ampiezza di progetto, e' un **massimo a posteriori**) e `Tg = o.Tend` (l'intera finestra: non c'e' una durata di raffica).

**Limiti onesti di questo ramo.** - La docstring (riga 76) dice “*Seeds fixed -> reproducible*”, ma **la funzione non fissa nessun seme**: i semi sono dentro `strong_wind.slx`. La riproducibilita' e' quindi una proprieta' del modello, non di questo codice. - Le opzioni `Vg`, `Tg` e **severity** sono **ignorate** in questo ramo (la severita' e' scelta dentro il `.slx`), eppure la struct restituita dichiara '`severity`', '`severe`' (riga 111) qualunque cosa l'utente abbia passato. Se si chiama `load_wind_profile(p, 'profile', 'strongwind', 'severity', 'light')` non succede nulla e la struct mente. - `alphaw = vw/p.V` usa il **V congelato a `t = 72 s`** anche se la finestra copre 71-83 s, durante i quali `V` cambia di parecchio. E' coerente con l'ipotesi frozen-time di tutto HM3, ma e' un'approssimazione: nel modello LTV completo (`LTV_FULL_ASCENT/`) il vento va diviso per `V(t)`. - `interp1(..., 'linear')` senza '`extrap`': se il log Simulink non copre tutta la finestra richiesta escono NaN silenziosi, che poi propagano dentro `lsim`.

---

## Possibili domande d'esame

**D: Perche' il vento entra nel modello come angolo d'attacco e non come una forza laterale applicata?**

R: Perche' nel modello linearizzato la forza normale e il momento aerodinamico sono *entrambi* proporzionali alla sola incidenza `alpha`, e un vento laterale non fa altro che modificare `alpha` (ruota il vento relativo). Raccogliendo, sia la riga di `z_ddot` sia quella di `theta_ddot` dipendono dal vento solo tramite `alpha = theta + z_dot/V` -

**alpha\_w.** La colonna del disturbo del plant e' infatti  $B_w = [0; -a_1*V; 0; -A_6]$ , cioe' la colonna di **theta** privata del termine di spinta **a\_4** (che non e' aerodinamico) e cambiata di segno. Aggiungere un ingresso di forza separato sarebbe ridondante: **alpha\_w** e' il canale del vento, esattamente, dentro l'ipotesi di linearita'.

**D: Da dove vengono i 6.4 m/s e i 0.39 gradi?** R:  $V_g$  non e' un numero scritto a mano: e' **dw.sigma.severe** interpolato linearmente in quota nel dataset **drywind.mat** del corso, alla quota di  $\max-qbar$  (15143 m = 15.143 km, fra i nodi di 14 e 18 km)  $\rightarrow$  6.3785 m/s. L'angolo e' la conversione ai piccoli angoli  $\alpha_w = V_g/V = 6.3785/937.709 = 0.006802 \text{ rad} = 0.3897 \text{ gradi}$ . Va detto che **sigma** e' formalmente una **RMS di turbolenza** del modello di Dryden, riusata qui come ampiezza di picco di una raffica discreta: e' una scelta, non una definizione da norma.

**D: Perche' una raffica 1-coseno e non un gradino?** R: Perche' un gradino ha una discontinuita' e quindi contenuto spettrale a tutte le frequenze: eccita l'attuatore e il modo flessibile per ragioni puramente matematiche, non fisiche. La 1-coseno parte e finisce con valore e derivata nulli ( $C^{-1}$ ), quindi il suo spettro decade come  $1/w^3$  e l'energia resta concentrata attorno a  $2\pi/T_g = 2.1 \text{ rad/s}$ , cioe' proprio nella banda del controllo rigido (crossover 2.4-3.2 rad/s). E' un carico realistico: nell'aria vera una raffica ha una scala spaziale finita, e attraversarla a  $V$  produce esattamente una salita e una discesa. Il rovescio della medaglia (limite da dichiarare): a  $w_{BM} = 18.9 \text{ rad/s}$  la raffica non ha praticamente energia, quindi **la risposta al gusto non prova nulla sulla stabilita' del bending** – quella si dimostra solo in frequenza.

**D: Perche' progettare a max-qbar e non, che so, a decollo o a fine primo stadio?** R: Perche' a  $\max-qbar$  tre criticita' coincidono. (1)  $qbar = 81 \text{ kPa}$  e' massimo, quindi il carico strutturale  $qbar*\alpha$  e' massimo; e la fascia 10-15 km e' anche quella del jet stream, dove il vento e' piu' forte. (2)  $\mu_{\alpha} = A_6$  e' proporzionale a  $qbar$ , quindi l'instabilita' aerodinamica e' massima: il polo a  $+1.84 \text{ rad/s}$  (raddoppio dell'errore in 0.38 s) e' il piu' a destra dell'intero volo. (3) Il conflitto attitude-hold / load-relief e' massimo. Progettare li' significa progettare nel caso peggiore; se il progetto tiene a  $\max-qbar$ , per il resto dell'ascesa e' conservativo (cosa che l'estensione LPV in LTV\_FULL\_ASCENT/ verifica esplicitamente).

**D: Tend di default e' 12 s, ma tutti i main\_task\* passano 80 s. Perche'?** R: Perche' 12 s bastano a vedere la raffica (1-4 s) e il transitorio d'assetto (la coppia dominante di anello chiuso ha  $\tau = 1.05 \text{ s}$ ), ma **non** a vedere esaurirsi il modo lento di deriva laterale, che ad anello chiuso ha  $w_n = 0.24 \text{ rad/s}$ ,  $\zeta = 0.23$ , cioe'  $\tau = 17.9 \text{ s}$ . Servono circa  $5*\tau$  per vedere  $z$  e  $\theta$  tornare a zero: da qui gli 80 s (il commento di riga 44 di **main\_task1.m** lo dice esattamente).

**D: Con che segno entra alpha\_w nell'incidenza, e come lo giustifichi?** R: Con il meno:  $\alpha = \theta + z_{\dot{}}/V - \alpha_w$ . Non e' una convenzione scelta, e' imposta dall'Eq. (1): la colonna del vento e'  $B_w = [0; -a_1*V; 0; -A_6]$ , cioe' i coefficienti con cui **theta** entra tramite **alpha**, cambiati di segno. Fisicamente, l'incidenza dipende dalla velocita' laterale **relativa all'aria**,  $(z_{\dot{}} - v_w)/V$ . Prova indipendente: togliendo la retroazione di deriva, sotto raffica a gradino la simulazione converge a  $z_{\dot{}} = +V*\alpha_w$ , cioe' esattamente al valore che **annulla alpha** – il veicolo si fa trascinare finche' il vento relativo sparisce. Con il + l'incidenza a regime raddoppierebbe: assurdo.

**D: Quindi il loop d'assetto allevia o aggrava il carico?** R: Lo **aggrava**. Il picco di incidenza del Task 1 e' **0.577 gradi**, cioe' **sopra** il contributo del solo vento (**0.390 gradi**): per tenere **theta**  $\rightarrow$  0 il controllore becca il muso **dentro** il vento relativo, e il suo contributo si **somma** a quello del vento invece di elidersi. Una legge di puro attitude-hold e' quindi **load-aggravating**. Vale la pena aggiungere il perche' strutturale: con  $A_6 > 0$  il centro di pressione e' davanti al baricentro, il momento aerodinamico e' divergente e **non esiste stabilita' a banderuola** da cui ricavare load relief gratuito. Con  $qbar = 81 \text{ kPa}$  il picco dell'indicatore di carico vale **46.8 kPa\*grado**.

*Nota storica (spendibile all'orale).* Fino a poco fa il post-processing (**simulate\_gust\_response.m**) ricostruiva **alpha** con il segno **piu'**, mentre il plant aveva gia' – correttamente – il meno. Il difetto dava un picco di **0.255 gradi** (**20.7 kPa\*grado**), **sotto** il vento da solo, e suggeriva una falsa azione di load relief. Il modello non e' mai stato sbagliato: lo era la formula a valle. La correzione **non tocca margini, stabilita', Nichols o Task 3** (**alpha\_w** e' un disturbo, non entra in  $L(s)$ ) e nemmeno i picchi di **theta**, **z**, **delta**: cambia solo la metrica di carico, di un fattore 2.3.

# HM3/simulate\_gust\_response.m

## Ruolo del file nel progetto

`simulate_gust_response.m` è il **verificatore nel dominio del tempo** di HM3. Il progetto è fatto tutto in frequenza (carta di Nichols, `classify_margins`), ma un margine di 6 dB / 30 gradi non dice quanto beccheggia il razzo o quanto ugello serve. Questa funzione prende il **loop chiuso T** costruito da `assemble_loop`, gli passa il profilo di raffica costruito da `load_wind_profile`, e restituisce le storie temporali più i **picchi** che diventano i numeri del report.

È chiamata da `main_task1.m` (riga 45), `main_task2.m` (righe 183-184, sia sul modello completo sia su quello rigido per il confronto), `main_task3.m` (per i quattro vertici della scatola di incertezza) e `main_montecarlo.m`.

Il file è brevissimo – 35 righe – ma condensa tre cose che vanno capite:

1. **Come il disturbo entra nell’anello.** `alpha_w` NON è nel loop aperto L: è un ingresso di disturbo che entra nel plant (seconda colonna di B) in un punto diverso da dove si rompe l’anello. È per questo che serve T e non basta L (vedi `hm3_assemble_loop.md`).
2. **Cosa si monitora:** assetto `theta`, deflessione `delta` (il budget di attuatore), deriva `z`, e soprattutto il **bilancio di angolo d’attacco** `alpha`, che a max-q è la grandezza dimensionante.
3. **L’indicatore di carico** `q_bar*alpha`, calcolato dai chiamanti a partire da `r.peak_alpha`.

Il punto della pagina è la **riga 29**, dove `alpha` viene assemblata con un **segno MENO** su `alpha_w`:

```
r.alpha = r.theta + r.zdot/w.V - r.alphaw;
```

Il meno non è una scelta di gusto: è l’unico segno compatibile con l’Eq. (1) della traccia e con il plant che la funzione sta simulando. La sezione “Il segno di `alpha_w`” lo dimostra per algebra e ne tira le conseguenze fisiche, che sono controintuitive e valgono l’orale.

---

## `simulate_gust_response` (righe 1-35)

```
function r = simulate_gust_response(T, w)
t = w.t(:);
u = [w.alphaw(:), zeros(numel(t),1)]; % [alpha_w, theta_ref]
y = lsim(T, u, t);
```

- **Riga 1:** firma `r = simulate_gust_response(T, w)`. Un solo output, una struct con tutto dentro.
- **Righe 9-12 (docstring):** la funzione **documenta esplicitamente il segno**. Dice che il meno è “the plant’s own convention” e cita la colonna di disturbo `Bw = [0; -a1*V; 0; -A6]`. Non è un dettaglio cosmetico: è l’unico posto della repo dove la convenzione di `alpha` è scritta a parole, ed è lì apposta perché è il punto in cui è facile sbagliare (vedi la nota storica più sotto).
- **Righe 14-17: arguments.** T deve essere lti (il `genss` restituito da `assemble_loop` lo è), w una struct scalare. **Nessun controllo sui campi di w:** la funzione assume `w.t`, `w.alphaw`, `w.V`, cioè assume che w venga da `load_wind_profile`.

## Righe 19-21 – costruzione dell’ingresso e simulazione

- **Riga 19:** `t = w.t(:)` – forza colonna. Il vettore tempi arriva da `load_wind_profile` (riga 51: `t = 0:o.dt:o.Tend`), quindi e’ **uniforme** con `dt = 0.005 s` di default. I chiamanti passano `Tend = 80` (`main_task1` riga 44, `main_task2` riga 182) perche’ il modo lento dell’anello chiuso ha  $\tau \sim 18\text{--}20\text{ s}$  e serve un orizzonte lungo per vedere l’assetto tornare a zero.

- **Riga 20:** la matrice degli ingressi.

```
u = [ w.alphaw(:) , zeros(N,1) ]
      |           |
      alpha_w      theta_ref = 0
```

L’ordine delle colonne e’ **posizionale** e deve combaciare con l’ordine degli `InputName` di `T`, che `assemble_loop` fissa alla riga 35: `{'alpha_w','theta_ref'}`. Se qualcuno invertisse quell’ordine in `assemble_loop`, questa funzione **non darebbe errore**: darebbe risultati sbagliati in silenzio. E’ una fragilita’ reale (accoppiamento implicito fra due file), da segnalare.

`theta_ref = 0` per tutta la simulazione: la prova e’ un **test di reiezione del disturbo** (regolatore), non di inseguimento. Il ramo `Kp_th*theta_ref` del controllore (`assemble_loop.m` riga 25) non viene mai eccitato qui.

- **Riga 21:** `y = lsim(T, u, t)`. `T` e’ un `genss` con l’analysis point su `delta`: `lsim` lo valuta **con l’AP chiuso al valore nominale**, cioe’ simula il vero anello chiuso. Le colonne di `y` seguono l’ordine degli `OutputName` di `T`, di nuovo fissato in `assemble_loop` riga 35: `{'theta','z','zdot','delta'}` – stesso accoppiamento posizionale implicito.

Nota sulla discretizzazione: `lsim` su un sistema continuo con griglia uniforme discretizza internamente a `dt = 5 ms` (frequenza di Nyquist  $\sim 628\text{ rad/s}$ ). I poli piu’ veloci del modello sono gli zeri/poli di Pade-3 a  $\sim 230\text{ rad/s}$  e il servo TVC a  $70\text{ rad/s}$ : la griglia e’ **adeguata ma non abbondante**. Il notch, con un null largo solo  $\sim 0.08\text{ rad/s}$  a  $18.9\text{ rad/s}$ , e’ invece risolto benissimo.

## Righe 23-29 – lo spaccettamento e il bilancio di alpha

```
r.theta = y(:,1);
r.z      = y(:,2);
r.zdot   = y(:,3);
r.delta  = y(:,4);
r.alpha  = r.theta + r.zdot/w.V - r.alphaw; % total angle of attack
```

- **Righe 25-28:** unita’ di misura. `theta` e `delta` sono in **radianti** (il plant e’ in rad; `alpha_w = v_w/V` e’ un rapporto adimensionale, quindi rad), `z` in metri, `zdot` in m/s. Tutti i chiamanti convertono con `*180/pi` in stampa. `r.delta` e’ la deflessione **post-attuatore** (uscita di `Wa` in `assemble_loop`), non il comando `u_pd`: in Task 1, con attuatore ideale, coincidono; in Task 2 no.

- **Riga 29 – il bilancio di angolo d’attacco.** Questa e’ la riga concettuale del file:

```
alpha = theta + zdot/V - alpha_w
```

Tre contributi, ognuno con un significato fisico distinto:

| termine              | cosa rappresenta                                                                                                                                | chi lo controlla                           |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| <code>+theta</code>  | l’assetto del corpo rispetto all’inerziale                                                                                                      | il <b>controllore</b>                      |
| <code>+zdot/V</code> | l’incidenza generata dalla <b>deriva laterale propria</b> del veicolo (angolo di traiettoria relativa: velocita’ laterale su velocita’ assiale) | l’anello, solo indirettamente e lentamente |

| termine  | cosa rappresenta                                                                                                   | chi lo controlla        |
|----------|--------------------------------------------------------------------------------------------------------------------|-------------------------|
| -alpha_w | l'incidenza indotta dal <b>vento</b> (= $v_w/V$ , <code>load_wind_profile.m</code> riga 67), <b>col segno meno</b> | nessuno: e' il disturbo |

**Perche' il vento entra col meno.** alpha non e' un angolo rispetto al suolo: e' l'incidenza rispetto alla velocita' **relativa all'aria**. Se l'aria stessa si muove lateralmente con  $v_w$  nello stesso verso in cui il veicolo deriva, la velocita' laterale *relativa* e'  $\dot{z} - v_w$ , e dividendo per  $V$ :

$$\alpha = \theta + (\dot{z} - v_w)/V = \theta + \dot{z}/V - \alpha_w$$

Il vento **sottrae** perche' un veicolo che vola gia' alla velocita' dell'aria non vede nessuna incidenza. Il caso limite lo conferma: con un gradino di vento sostenuto e il feedback di deriva spento, la simulazione converge a  $\dot{z}_{ss} = +V*\alpha_w$  (rapporto verificato 1.0000) con  $\theta_{ss} = 0$ , e quindi  $\alpha_{ss} = 0$ . Il veicolo viene spinto sottovento finche' non **vola insieme all'aria**, e a quel punto l'incidenza relativa e' esattamente zero. Con il segno opposto (+alpha\_w) lo stato stazionario darebbe  $\alpha = 2*\alpha_w$ , che e' fisicamente assurdo.

A max-q questo alpha e' la grandezza **dimensionante**: il momento aerodinamico e il carico di flessione sul corpo scalano con  $q_{bar}*\alpha$ .

## Righe 31-34 – le metriche di picco

```
r.peak_theta = max(abs(r.theta));
r.peak_z      = max(abs(r.z));
r.peak_delta = max(abs(r.delta));
r.peak_alpha  = max(abs(r.alpha));
```

Sono i quattro numeri che finiscono nel report e nelle tabelle di Task 3. `max(abs(...))` su **tutto** l'orizzonte, quindi il **segno e' perso**: sapere che `peak_theta = 0.261` gradi non dice se il razzo si e' beccheggiato **verso** o **contro** il vento – distinzione che e' proprio quella fra load relief e load aggravation. I chiamanti recuperano il segno solo guardando i grafici.

`r.peak_alpha` e' poi usato dai chiamanti per l'indicatore di carico (`main_task1.m` riga 51):

$$q_{bar}*\alpha = p.qbar/1000 * peak\_alpha*180/pi \quad [kPa * gradi]$$

Nota: e' un'unita' **mista** (kPa moltiplicati per gradi), non SI. Va bene come indicatore relativo e per il confronto fra i casi di Task 3, ma per confrontarlo con un limite di letteratura bisogna verificare che il limite sia espresso nelle stesse unita' (spesso e' in psf-deg o Pa-rad).

## Il segno di alpha\_w (riga 29) – perche' e' un MENO

E' il punto di questa pagina, ed e' una domanda d'orale quasi garantita perche' si risolve con **pura algebra sul codice**, senza interpretazioni.

### Cosa fa il plant

Dalle matrici di `build_plant_rigid.m` (righe 11-17), l'equazione del momento e':

$$\begin{aligned} \ddot{\theta} &= A6*\theta + (A6/V)*\dot{z} + K1*\delta + (-A6)*\alpha_w \\ &= A6 * ( \theta + \dot{z}/V - \alpha_w ) + K1*\delta \\ &\quad \underbrace{\hspace{1.5cm}}_{\text{alpha "vista" dal plant}} \end{aligned}$$

La colonna del disturbo e'  $B_w = [0; -a_1 V; 0; -A_6]$  (riga 17 di `build_plant_rigid.m`, riga 28 di `build_plant_full.m`): il **segno e' negativo** su  $\alpha_w$ , ed e' esattamente cio' che scrive l'Eq. (1) della traccia. Lo stesso vale nell'equazione della forza normale (seconda riga di `A` e di `B_w`):  $\ddot{z} = a_1(\dot{z} + V\theta - V\alpha_w) + a_4\theta + a_3\delta$ , che di nuovo si raggruppa attorno a  $\theta + \dot{z}/V - \alpha_w$ .

Entrambe le righe aerodinamiche del plant – laterale e beccheggio – portano i termini  $a_1 V \alpha$  e  $A_6 \alpha$  con **QUESTO**  $\alpha$ . Il plant e' quindi internamente coerente, e la riga 29 riporta la **stessa quantita'** che il plant integra davvero. Un + contraddirebbe il plant che la funzione sta simulando.

## Nota storica (utile all'orale)

Fino a poco fa la riga 29 aveva un + **r.alphaw**. Era un **bug di post-processing**, non di modello: il plant e' sempre stato giusto (la colonna  $B_w$  non e' mai cambiata), sbagliava solo la formula con cui si *ri-costruiva*  $\alpha$  a valle di `lsim` per graficarla e per calcolare  $\bar{q} \alpha$ . Il bug e' stato corretto e la docstring ora fissa la convenzione a parole.

Vale la pena saperlo raccontare, per due ragioni:

- **Cosa NON toccava:** margini, Nichols, stabilita', tutte le time history di  $\theta$ ,  $z$ ,  $\dot{z}$ ,  $\delta$ , e l'intero Task 3.  $\alpha_w$  non entra in  $L(s)$ , e la riga 29 e' pura post-elaborazione: non retroagisce sulla simulazione.
- **Cosa toccava:** i due numeri di carico ( $\text{peak}_\alpha$  e  $\bar{q} \alpha$ ), la figura del budget di  $\alpha$ ... e la **narrazione fisica**, che con il segno giusto si ribalta (sotto).

## Quanto pesa il segno (numeri della run reale)

Modello rigido di Task 1, raffica **severe**,  $V_g$  dalla dispersione `drywind`,  $T_{end} = 80$ :

| grandezza                                                                                        | valore                                                                  |
|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| picco $\alpha_w$ (vento da solo)                                                                 | 0.390 gradi                                                             |
| picco $\backslash \theta \backslash$                                                             | 0.261 gradi (negativo al picco di raffica: -0.178 gradi)                |
| picco $\backslash z \backslash / \backslash \delta \backslash$                                   | 2.27 m / 0.528 gradi                                                    |
| <b>picco <math>\backslash \alpha \backslash</math> – codice attuale (<math>-\alpha_w</math>)</b> | <b>**0.577 gradi -&gt; <math>\bar{q} \alpha = 46.8</math> kPa deg**</b> |
| picco $\backslash \alpha \backslash$ – vecchio codice buggato ( $+\alpha_w$ )                    | 0.255 gradi -> $\bar{q} \alpha = 20.7$ kPa deg                          |

Sul modello completo di Task 2 i valori corretti sono **0.565 gradi** e **45.8 kPa deg**: praticamente identici, come deve essere (attuatore, ritardo e notch cambiano l'anello, non il bilancio cinematico di  $\alpha$ ).

## La fisica: un puro attitude hold e' LOAD-AGGRAVATING

Ecco il punto controintuitivo, ed e' il vero contenuto d'orale.

Il picco di incidenza totale (**0.577 gradi**) **supera** il contributo del solo vento (**0.390 gradi**). Non lo attenua: lo **amplifica** di circa il 50%.

Il perche' e' semplice una volta scritto il segno giusto. Il vento genera un momento  $-A_6 \alpha_w$  che tende a spingere il muso in negativo; l'anello di assetto, che ha come unico obiettivo  $\theta \rightarrow 0$ , reagisce riportando l'assetto verso zero – e cosi' facendo **beccheggia il muso DENTRO il vento relativo**. Il  $\theta$  che ne risulta (-0.18 gradi al picco) entra in  $\alpha = \theta + \dot{z}/V - \alpha_w$  con lo **stesso segno** del termine  $-\alpha_w$ : si **somma** al vento invece di cancellarlo.

Una legge di **puro attitude-hold e' quindi load-aggravating**, non load-relieving. Ed e' esattamente il motivo per cui i lanciatori reali aggiungono un **termine esplicito di load relief** (retroazione di accelerometro laterale o di  $\alpha$  stimata) invece di limitarsi al PD di assetto: senza quel termine, il controllo di assetto lavora *contro* il carico strutturale.

Il velivolo, da parte sua, non offre nessun aiuto: con  $A_6 > 0$  il centro di pressione sta **davanti** al baricentro, il momento aerodinamico e' **divergente**, e non esiste nessuna stabilita' a banderuola che riallinei spontaneamente il muso col vento relativo.

## E allora a cosa servono i guadagni di deriva $K_{p_z} = K_{d_z} = -1e-3$ ?

Non sono un dispositivo di load relief – o meglio, non in questa misura. Sono anzitutto un **requisito di stabilità**: chiudono l'**integratore libero della posizione laterale** (il plant ha un polo esatto in  $s = 0$ , perché  $z$  e' l'integrale di  $\dot{z}$ ).

Verifica diretta, spegnendo i due guadagni ( $K_{p_z} = K_{d_z} = 0$ ):

|                                   | poli di anello chiuso                | picco $\backslash  \alpha $ | picco $\backslash  z $ |
|-----------------------------------|--------------------------------------|-----------------------------|------------------------|
| con drift feedback                | -0.056 +/- 0.233i, -0.953 +/- 1.905i | 0.577 gradi                 | 2.27 m                 |
| senza ( $K_{p_z} = K_{d_z} = 0$ ) | 0, -0.076, -0.98 +/- 1.93i           | 0.584 gradi                 | <b>9.54 m</b>          |

Senza i guadagni di deriva resta un polo **esattamente nell'origine**: il sistema e' solo **marginalmente stabile** e  $z$  non torna mai a zero. Sull'incidenza quei guadagni incidono per l'**1%** (0.584 -> 0.577 gradi: irrilevante), ma tagliano la deriva **da 9.5 m a 2.3 m**. La loro funzione e' contenere la deriva e chiudere l'integratore, non alleggerire il carico.

**Possibile domanda d'esame** – Mi ricavi il bilancio di angolo d'attacco e mi dici quale dei tre termini il controllore puo' effettivamente usare? *Risposta:*  $\alpha = \theta + \dot{z}/V - \alpha_w$ . Il meno sul vento perché  $\alpha$  e' misurata rispetto alla velocità **relativa all'aria**: se il veicolo deriva insieme al vento, l'incidenza relativa si annulla.  $\alpha_w = v_w/V$  e' il disturbo puro, non controllabile.  $\dot{z}/V$  e' l'incidenza dovuta alla deriva propria: il controllore la influenza solo indirettamente e lentamente (la deriva e' l'integrale dell'accelerazione laterale). L'unico termine con autorità diretta e'  **$\theta$** : il TVC produce un momento ( $K_1 \cdot \delta$ ) che ruota il corpo in ~1 secondo. **Ma qui sta il punto**: un PD che porta  $\theta \rightarrow 0$  mette il muso dentro il vento relativo, e il suo contributo si **somma** a  $-\alpha_w$  invece di cancellarlo. Il picco di  $\alpha$  (0.577 gradi) risulta **maggiore** del solo vento (0.390 gradi): l'attitude hold **aggrava** il carico. Un vero load relief richiede un termine dedicato (accelerometro laterale /  $\alpha$  stimata), che questo controllore **non ha**. I guadagni  $K_{p_z} = K_{d_z} = -1e-3$  non lo sostituiscono: servono a chiudere l'integratore di posizione laterale (senza, un polo resta in  $s = 0$ ) e tagliano la deriva da 9.5 m a 2.3 m, ma sull'incidenza pesano l'1%.

## Possibili domande d'esame

**D: Perché la simulazione della raffica ha bisogno di T e non si puo' fare con L?** R: Perché  $\alpha_w$  **non compare in L**. Il loop aperto e' la catena controllore -> attuatore -> plant -> sensore rotta in  $\delta$ : e' un oggetto SISO che descrive la stabilità e i margini. La raffica invece entra nel plant sulla **seconda colonna di B**, in un punto diverso dalla rottura. T, costruito con `connect` in `assemble_loop`, mantiene  $\alpha_w$  come ingresso esterno proprio per questo. La regola generale: L risponde a "e' stabile e con che margine", T risponde a "quanto beccheggia e quanto ugello serve". Corollario utile: siccome  $\alpha_w$  non entra in L, **nessun margine dipende dal segno della riga 29**.

**D: Che raffica e', e da dove viene l'ampiezza?** R: Di default e' una **raffica 1-cosine** (`load_wind_profile.m` riga 56):  $v_w(t) = 0.5 \cdot V_g \cdot (1 - \cos(2\pi \cdot (t-t_0)/T_g))$ , durata  $T_g = 3$  s, onset  $t_0 = 1$  s. E' il profilo di raffica discreta standard (nessun contenuto ad alta frequenza, transizione dolce). L'ampiezza  $V_g$  non e' inventata: viene interpolata dalla dispersione  $\sigma$  del file `drywind.mat` all'altitudine di max-q (15143 m) per la severita' richiesta (**severe** di default) – righe 39-48 di `load_wind_profile.m`. Ne risulta un picco di  $\alpha_w$  di 0.390 gradi. Esiste anche il profilo **strongwind**, che gira il modello Simulink del professore (vento medio + Dryden schedulato in quota) e finestra il risultato intorno a max-q.

**D: Che unita' hanno le uscite, e dove si converte?** R: Tutto **SI e radianti** dentro la funzione:  $\theta$ ,  $\delta$ ,  $\alpha$ ,  $\alpha_w$  in rad;  $z$  in m;  $\dot{z}$  in m/s. La conversione in gradi la fanno i chiamanti ( $\cdot 180/\pi$  nelle `fprintf` e nei plot). L'indicatore di carico  $q_{bar} \cdot \alpha$  viene calcolato **fuori** da questa funzione, in `main_task*.m`, come  $p \cdot q_{bar}/1000 \cdot \text{peak\_alpha} \cdot 180/\pi$ , quindi in kPa\*gradi – unita' mista, non SI, ottima per confrontare i casi di Task 3 fra loro ma da maneggiare con cura se la si confronta con un limite di letteratura.

**D:** `max(abs(...))` – che informazione perdi? **R:** Il **segno**, e con esso la lettura fisica del transitorio. `peak_theta = 0.261` gradi non dice che al picco di raffica `theta = -0.178` gradi, cioè che il veicolo ruota **nel verso opposto** al segno del disturbo. E siccome `alpha = theta + zdot/V - alpha_w`, quel `theta` negativo si **somma** a `-alpha_w`: e' proprio il meccanismo di load aggravation. Il picco scalare non lo mostra – bisogna guardare `task1_alpha_load.png`, dove i tre contributi sono disegnati separatamente e si vede che `theta` e `-alpha_w` vanno nella stessa direzione.

**D:** `theta_ref` e' sempre zero. Non dovresti provare anche l'inseguimento? **R:** Nella finestra di max-q il compito e' **regolazione**, non tracking: il programma di beccheggio nominale e' gia' stato eseguito e in quei 10-20 secondi il riferimento e' congelato. La prova che conta e' la **reiezione del disturbo** (raffica), che e' quello che questa funzione fa. Il ramo `Kp_th*theta_ref` esiste in `assemble_loop` (riga 25) e T espone `theta_ref` come secondo ingresso, quindi un test di step di assetto **sarebbe possibile senza toccare nulla** – basta mettere una colonna non nulla alla riga 20. Il codice, semplicemente, non lo fa.

**D:** Il modello di Task 2 ha un notch largo 0.08 rad/s a 18.9 rad/s. La griglia di `lsim` a 5 ms basta? **R:** Si', ampiamente. `dt = 5 ms` da' una frequenza di Nyquist di ~628 rad/s, contro i poli piu' veloci del modello (Pade-3 a ~230 rad/s, servo TVC a 70 rad/s). Il null del notch, essendo **stretto in frequenza ma centrato in basso** (18.9 rad/s), e' risolto con enorme margine. Il vincolo vero su `dt` viene dai poli di Pade, non dal notch. Nota comunque che l'orizzonte lungo (`Tend = 80 s`, imposto dal modo lento di deriva con `tau ~ 20 s`) rende il vettore lungo 16001 punti: la simulazione e' dominata dalla dinamica lenta, non da quella veloce.



# HM3/classify\_\_margins.m

## Ruolo del file nel progetto

Questo file e' la **risposta di HM3 alla domanda "quanto margine ho?"** su un lanciatore che e' open-loop instabile. E' chiamato da `design_controller` (riga 66 e riga 84, dentro la funzione di costo), da `main_task2.m` (righe 125 e 145) e da `main_task3.m` (riga 56). Prende il loop aperto pieno  $L = K_{FCS} * F * C * G$  (PD + attuatore TVC + ritardo + notch + impianto con deriva e bending), chiama `allmargin` e **smista ogni attraversamento nella banda di frequenza a cui appartiene fisicamente**, restituendo una struct con margini etichettati: Aero GM, Rigid GM, Rigid PM, Flex GM, Flex PM,  $|L(\omega_{BM})|$  e delay margin.

Il file esiste perche' `margin()` – il comando che uno userebbe d'istinto – **su questo loop non ha significato**, per due motivi distinti che vanno tenuti separati.

**Primo motivo: la stabilita' condizionale.** L'aeroshell a max-qbar e' aerodinamicamente instabile: il polo del corpo rigido sta a  $+\sqrt{A_6} = +1.84$  rad/s (nel loop assemblato, con l'accoppiamento di deriva, si misura  $+1.8165$  rad/s, piu' un secondo polo instabile lento a  $+0.0291$  rad/s e l'integratore di posizione laterale in 0). Con poli instabili in catena aperta, il criterio di Nyquist non e' "non circondare -1" ma "circondare -1 esattamente P volte in senso antiorario". La conseguenza pratica e' che il loop e' **condizionalmente stabile**: esiste una **banda** di guadagni ammissibili, e si esce dalla stabilita' sia **abbassando** sia **alzando** il guadagno. Verificato eseguendo il codice sul loop completo del Task 2 (TVC + ritardo + notch): il ciclo chiuso e' stabile per k in **[0.50, 2.39]**, instabile fuori. I due estremi sono esattamente i due margini che il classificatore etichetta come **Aero GM** (-6.0 dB, cioe'  $k = 0.50$ ) e **Rigid GM** (+7.56 dB, cioe'  $k = 2.39$ ). Un solo numero – quello che restituisce `margin()` – non puo' descrivere una banda a due estremi.

**Secondo motivo: il feedback di deriva laterale.** Il canale  $z/\delta$  contiene un integratore libero (posizione laterale), quindi  $|L| \rightarrow \infty$  a DC e la curva di Nichols "viene dall'alto": eseguendo il codice, a  $\omega = 0.01$  rad/s il loop e' a +48 dB. Il lobo di bassa frequenza cosi' generato **attraversa gli 0 dB due volte** (nel Task 1 a 0.161 e 0.222 rad/s) con fasi che nulla hanno a che vedere col corpo rigido – e a queste frequenze `allmargin` diligentemente riporta phase margin di **-133 deg e -40 deg**. Sono artefatti del canale di deriva, non margini. La prova che questo non e' un timore teorico: nel corner **V3** del Task 3 ( $\mu_{\alpha} 1.3$ ,  $\mu_c 0.7$ ), `margin(L)` restituisce **PM = -8.72 deg @ 0.416 rad/s** su un sistema che `isstable` dichiara **stabile**. Il classificatore, sullo stesso loop, restituisce il phase margin vero: **18.0 deg @ 1.50 rad/s**.

Onesta' doverosa: nei casi **nominali** (Task 1 e Task 2 dopo il re-tune) `margin()` per pura coincidenza numerica restituisce gli stessi valori del classificatore (-6.00 dB e 30.0 deg). L'errore di `margin()` non e' sistematico: morde quando i parametri si spostano, cioe' esattamente nei corner e nel Monte Carlo, cioe' esattamente dove il numero serve. Questo e' il motivo per cui il classificatore non e' una raffinatezza estetica ma un requisito.

La struttura per bande segue D'Antuono (Fig. 3.2, Tab. 3.1) e Trotta (Tab. 4.1), come dichiarato nella docstring righe 4-12.

---

## Docstring e contratto (righe 1-25)

- Righe 4-12: la mappa delle bande. Vale la pena rileggerla perche' e' il glossario di tutto l'homework:

- **Aero GM**: gain-reduction margin di bassa frequenza. Il commento a riga 7 lo dice esplicitamente: `gmdb < 0`. E' il **bordo inferiore** della banda di guadagni condizionalmente stabili.
- **Rigid PM**: phase margin al crossover del corpo rigido ( $\sim \sqrt{A_6}$ ).
- **Rigid GM**: gain-increase margin di media frequenza, prodotta dal ritardo di fase di attuatore + delay. **Assente con attuatore ideale** (Task 1) – il codice lo dichiara a riga 11 e il calcolo lo conferma: `rigidGM_dB = NaN` nel Task 1.
- **Flex GM/PM**: margini nella banda del bending.
- Righe 13-15: gli attraversamenti sotto `w_drift` sono **artefatti di deriva** e **non** vengono riportati come margini rigidi; il commento dice testualmente che “*taking margin()’s default instead would pick one of these*”.

## Blocco arguments e setup (righe 27-37)

- Riga 29: `w_drift` default **0.5** rad/s. Nota: e' un default fisso, ma **tutti i chiamanti lo sovrascrivono** con `0.3*sqrt(A6)` (0.5517 al nominale), che ha il pregio di **scalare con l'instabilita'** dell'aeroshell. Nel corner V3 (`mu_alpha = 1.3`) diventa 0.629 rad/s: senza questo scaling, la crossing di deriva a 0.4164 rad/s resterebbe comunque sotto 0.5, ma il margine e' sottile.
- Riga 30: `w_flex` default **Inf** = “nessun modo flessibile”. E' il caso Task 1, dove tutte le maschere `gf < w_flex` diventano “tutte le frequenze” e quelle `gf >= w_flex` diventano vuote -> Flex GM/PM = NaN. Elegante: la stessa funzione serve il modello rigido e quello flessibile senza rami `if`.
- Riga 32: `w_bending` default NaN, risolto a riga 34 in `w_flex`.
- Righe 36-37: stesso `onCleanup` di `design_controller` per silenziare `Control:analysis:MarginUnstable` e ripristinarlo all'uscita.

## allmargin e l'estrazione degli attraversamenti (righe 39-41)

```
am = allmargin(L);
gmdb = 20*log10(am.GainMargin(:)); gf = am.GMFrequency(:);
pm = am.PhaseMargin(:); pf = am.PMFrequency(:);
```

- Riga 39: **allmargin invece di margin**. E' la scelta che rende possibile tutto il resto: **allmargin** restituisce **tutti** gli attraversamenti (vettori), **margin** ne sceglie **uno** con una regola che ignora la fisica del problema. Sul loop pieno del Task 2, **allmargin** trova **5** gain margins e **3** phase margins.
- Riga 40: la conversione in dB. `am.GainMargin` e' un **fattore moltiplicativo**, non un dB: per l'Aero GM vale **0.5012**, cioe' “moltiplica il guadagno per 0.50 e vai instabile”. In dB e' -6.0. **Il segno negativo e' il modo in cui il codice riconosce una gain-reduction margin** (riga 44: `gmdb < 0`). E' la convenzione di Barrows & Orr (cap. 9).
- I margini di fase (`pm`) sono in gradi, le frequenze in rad/s.

**Possibile domanda d'esame** – perche' il gain margin aerodinamico e' *negativo* in dB? Un margine negativo non vuol dire instabile? *Risposta*: no. Il gain margin di MATLAB e' il fattore `k` per cui il loop diventa marginalmente stabile. Se il loop e' condizionalmente stabile con banda `k` in `[0.5, 2.39]`, allora **due** `k` critici esistono: `k = 0.5` (`< 1`, cioe' `< 0` dB -> gain **reduction** margin) e `k = 2.39` (`> 1`, `> 0` dB -> gain **increase** margin). Il segno codifica la **direzione** in cui il guadagno destabilizza, non lo stato di stabilita' presente. Il verdetto di stabilita' e' un'altra cosa (`isstable`).

## Gain margins: aero, rigid, flex (righe 43-46)

```
[mm.aeroGM_dB, mm.aeroGM_w] = pick(gmdb, gf, ...
    gf > 0 & gf < opts.w_flex & gmdb < 0, 'minf');
[mm.rigidGM_dB, mm.rigidGM_w] = pick(gmdb, gf, ...
    gf > 0 & gf < opts.w_flex & gmdb > 0, 'minf');
[mm.flexGM_dB, mm.flexGM_w] = pick(gmdb, gf, ...
    gf >= opts.w_flex & gf <= opts.w_flex_hi, 'near', opts.w_bending);
```

- Riga 43 (commento) e la maschera **gf > 0**: esclude l'attraversamento a **frequenza zero**. Non e' teorico: sul loop pieno del Task 2, **allmargin** restituisce un'entrata a **gf = 0** con **gmdb = -283 dB**. Da dove viene? Dal fatto che il canale di deriva ha un integratore, quindi  $|L(0)| = \text{Inf}$  e il "fattore per andare instabile" e' numericamente zero -> -283 dB (limite di precisione macchina). Senza la maschera, **aeroGM\_dB** varrebbe -283 dB e tutta la sintonizzazione sarebbe assurda. **L'integratore di deriva e' il colpevole, e questa maschera e' la sua neutralizzazione.**
- Riga 44 – **Aero GM**: fra tutti gli attraversamenti con **gmdb < 0** sotto **w\_flex**, prende quello a **frequenza minima** ('minf'). Traduzione fisica: il bordo inferiore della banda di stabilita' condizionale e' quello piu' a bassa frequenza, dove la curva incrocia la fase critica scendendo dal lobo di deriva. Task 1: **-6.000 dB @ 0.593 rad/s**. Task 2: **-6.000 dB @ 0.542 rad/s**. Corner V3: **-0.91 dB @ 0.600 rad/s** – il margine aerodinamico si e' quasi annullato, ed e' il risultato quantitativo piu' importante del Task 3.
- Riga 45 – **Rigid GM**: stessa banda, ma **gmdb > 0**: il bordo **superiore** della banda. Task 1: **NaN** (attuatore ideale, la fase non riattraversa mai il punto critico – verificato: il loop resta stabile fino a  $k = 1e6$ ). Task 2: **+7.561 dB @ 11.11 rad/s**, che corrisponde a  $k = 2.39$ , esattamente il bordo superiore misurato per forza bruta.
- Riga 46 – **Flex GM**: nella finestra **[w\_flex, w\_flex\_hi]**, si prende l'attraversamento **piu' vicino a w\_bending** (modo 'near'). Nel Task 2 con il notch profondo il risultato e' **NaN**: il modo e' *gain-stabilizzato*, la curva non arriva mai a 0 dB nella banda del bending, quindi **non esiste** un flex crossover. Ecco perche' serve **LwBM\_dB** (righe 52-58). Senza notch, invece, il classificatore trova **flexGM = -24.17 dB @ 18.76 rad/s** e **flexPM = -148 deg @ 21.5 rad/s** – numeri di un loop palesemente instabile (**stable\_am = 0**).

**Possibile domanda d'esame** – nel Task 2 **w\_flex = 0.6\*omega\_BM = 11.34 rad/s** e il Rigid GM cade a 11.11 rad/s. Non e' pericolosamente vicino al confine? *Risposta*: si', ed e' un punto debole onesto del setup. L'attraversamento di guadagno rigido sta appena **2 % sotto** il confine rigido/flex. Se si fosse scelto **w\_flex = 0.5\*omega\_BM = 9.45 rad/s** quello stesso attraversamento verrebbe **classificato come Flex GM** (+7.56 dB) e il Rigid GM diventerebbe NaN. Attenzione pero' a cosa *non* succederebbe: il tuner **non se ne accorgerebbe**. La guardia della funzione di costo (**design\_controller** riga 85) testa solo **isnan(mt.aeroGM\_dB) || isnan(mt.rigidPM\_deg)** – **non** il Rigid GM – e con **w\_flex = 0.5\*omega\_BM** l'Aero GM resta -6.00 dB e il Rigid PM resta 30.0 deg: la penalita'  $1e6$  non scatta e l'ottimizzazione converge come se nulla fosse. (Prova indipendente: nel Task 1 **rigidGM\_dB** e' NaN e il tuner gira senza problemi.) Il danno sarebbe quindi peggiore di un fallimento rumoroso: una **mis-classificazione silenziosa**, un gain margin del corpo rigido riportato in tabella come margine del bending. La scelta 0.6 funziona, ma il confine non e' generoso: e' un parametro che andrebbe verificato ogni volta che l'attuatore o il notch cambiano.

## Phase margins (righe 48-50)

```
[mm.rigidPM_deg, mm.rigidPM_w] = pick(pm, pf, ...
    pf > opts.w_drift & pf < opts.w_flex, 'maxv');
```

- Riga 49 – **Rigid PM**: la maschera esclude tutto cio' che sta **sotto w\_drift**. E' la riga che salva il progetto. Nel Task 1 **allmargin** riporta tre phase margins: **-133.1 deg @ 0.161**, **-40.5 deg @ 0.222** e **+30.0 deg @ 2.455**. I primi due sono i due attraversamenti a 0 dB del lobo di deriva; solo il terzo e' il crossover del corpo rigido. Con **w\_drift = 0.5517** la maschera li elimina e resta il 30.0 deg vero.
- Il modo di selezione e' '**maxv**', cioe' il **valore massimo**, non il minimo. Questa e' una scelta di robustezza contro gli artefatti: se un attraversamento di deriva "sfuggisse" appena sopra **w\_drift**, porterebbe un PM molto negativo e verrebbe scartato dal **max**. **Il prezzo e' che la regola e' ottimistica**: se esistessero *due* crossover genuinamente rigidi, **classify\_margins** riporterebbe quello col margine **migliore** invece del peggiore. Nei casi effettivamente eseguiti (Task 1, Task 2, tutti i corner del Task 3) in banda rigida c'e' **un solo** attraversamento, quindi l'ambiguita' non si presenta – ma la regola in se' non e' conservativa e va saputo.
- Riga 50 – **Flex PM**: analogo, in banda bending, con selezione '**near**' a **w\_bending**. NaN nel design ritenuto (notch profondo).

## Attenuazione del bending: LwBM\_dB (righe 52-58)

```
if ~isnan(opts.w_bending) && isfinite(opts.w_bending)
    mm.LwBM_dB = 20*log10(abs(freqresp(L, opts.w_bending)));
else
    mm.LwBM_dB = NaN;
end
```

- Righe 52-53 (commento) spiegano il perché: un modo **gain-stabilizzato** non ha crossover, quindi non ha né GM né PM in senso classico. Il suo “margine” è semplicemente **quanto sta sotto gli 0 dB** alla frequenza del modo. Questo è il numero che nel report vale come *bending gain margin*.
- Riga 55: `freqresp(L, w_bending)` valuta  $L(j\omega_{\text{BM}})$  direttamente – niente griglia, niente interpolazione, valore esatto.
- Valori misurati eseguendo il codice: **+29.0 dB senza filtro** (il modo domina il loop e lo destabilizza: è il “+29 dB bending resonance” del README), **-18.2 dB con il notch profondo dopo il re-tune** (il modo è attenuato ben oltre i 12 dB tipicamente richiesti per una gain-stabilisation). Il salto di ~47 dB è tutto il lavoro del notch.
- Righe 56-57 (else): NaN nel Task 1 (nessun bending). La guardia `isfinite` è necessaria perché quando `w_bending` non è dato, riga 34 lo pone uguale a `w_flex = Inf`, e `freqresp(L, Inf)` non avrebbe senso.

## Attraversamenti di deriva (righe 60-62)

```
mm.drift_w = pf(pf > 0 & pf <= opts.w_drift);
```

- Riga 62: **non li butta via, li conserva**. Vengono passati a `plot_nichols_lv` che li disegna con una croce nera e la label esplicita “*drift 0 dB crossing (not a margin)*”. È una scelta didattica corretta: nascondere gli attraversamenti spuri farebbe sembrare la curva di Nichols più semplice di quello che è; marcarli come “non margini” **documenta** la scelta di classificazione invece di occultarla. Task 1: [0.161, 0.222] rad/s. Task 2: [0.158, 0.227] rad/s.

## Delay margin e flag di stabilità (righe 64-65)

```
mm.DM_s = min(am.DelayMargin);
mm.stable_am = am.Stable;
```

- Riga 64: **il delay margin è il minimo su TUTTI gli attraversamenti**, artefatti di deriva compresi. In pratica non fa danno, perché  $DM = PM/\omega$  e gli attraversamenti di deriva stanno a  $\omega \sim 0.2$  rad/s, quindi danno delay margin **enormi** (24.6 s e 25.2 s nel Task 1) e non vincono mai il **min**: vince il crossover rigido (0.2133 s nel Task 1, 0.1652 s nel Task 2, 0.209 s in V3, verificati come  $PM_{\text{rad}}/\omega$ ). **Ma la regola non è filtrata per banda come le altre**, quindi in un caso patologico (crossover di deriva a frequenza relativamente alta con PM piccolo) potrebbe riportare un delay margin che non è quello rigido. È l'unica incoerenza metodologica del file e va dichiarata.
- Riga 65: `am.Stable` è il flag di stabilità che `allmargin` deriva **dal loop aperto L**. Attenzione: nel resto della repo il verdetto usato è `isstable(T)` sul **ciclo chiuso** (`design_controller` riga 67). I due coincidono nei casi esaminati, ma quello autorevole è `isstable(T)`.

## Funzione annidata pick (righe 67-79)

```
function [v, w] = pick(vals, freqs, mask, mode, target)
    idx = find(mask);
    if isempty(idx), v = NaN; w = NaN; return; end
    switch mode
        case 'minf' % lowest crossover frequency
```

```

        [w, j] = min(freqs(idx));
    case 'near'      % nearest a target frequency
        [~, j] = min(abs(freqs(idx) - target)); w = freqs(idx(j));
    case 'maxv'      % largest value (rigid PM)
        [~, j] = max(vals(idx)); w = freqs(idx(j));
    end
    v = vals(idx(j));
end

```

- Riga 67: firma. E' il **motore di selezione** condiviso da tutti e cinque i margini: una maschera logica (la banda) + una regola di scelta (quale attraversamento nella banda).
- Riga 69: **banda vuota** -> NaN, non errore. E' il contratto su cui si regge la penalita' 1e6 della funzione di costo di **design\_controller** (riga 85-87): un NaN significa "questa banda non ha piu' un attraversamento", ed e' un'informazione fisica, non un fallimento numerico.
- Righe 71-77: i tre modi. 'minf' per i bordi della banda condizionale (il primo attraversamento incontrato salendo in frequenza), 'near' per i modi flessibili (dove si sa gia' *dove* aspettarsi il fenomeno), 'maxv' per il PM rigido (difesa dagli artefatti di deriva, vedi la nota sopra).
- Nota su 'minf': [w, j] = min(freqs(idx)) restituisce direttamente la frequenza in w; negli altri due rami w va ricostruita come freqs(idx(j)). Asimmetria stilistica, non un bug.

## Possibili domande d'esame

**D: Perche' margin() non basta su questo loop? Fai l'esempio concreto.** R: Per due ragioni. (1) Il loop e' **condizionalmente stabile** (l'aeroshell ha poli a destra: +1.82 e +0.029 rad/s), quindi la stabilita' vive in una **banda** di guadagni - misurata sul loop pieno del Task 2: k in [0.50, 2.39] - e un singolo gain margin non puo' descriverla. (2) L'integratore di deriva laterale fa "venire la curva dall'alto" e genera due attraversamenti a 0 dB a bassa frequenza (0.16 e 0.22 rad/s) con phase margin di -133 deg e -40 deg, che **allmargin** riporta assieme al margine vero. Esempio concreto: nel corner V3 del Task 3, **margin(L)** restituisce **PM = -8.7 deg @ 0.42 rad/s** su un sistema che **isstable(T)** dichiara **stabile**; il classificatore restituisce il valore vero, **18.0 deg @ 1.50 rad/s**. Nei casi nominali **margin()** per coincidenza azzecca il numero, ma non e' affidabile dove serve.

**D: Cosa sono, fisicamente, i cinque margini classificati?** R: **Aero GM** = bordo *inferiore* della banda di guadagni stabili: quanto puoi *ridurre* il guadagno prima che l'instabilita' aerodinamica prenda il sopravvento. E' negativo in dB. **Rigid GM** = bordo *superiore*: quanto puoi *aumentare* il guadagno prima che il ritardo di fase di attuatore + delay porti il crossover di media frequenza sul punto critico (assente con attuatore ideale). **Rigid PM** = margine di fase al crossover del corpo rigido ( $\sim \sqrt{A_6}$ ). **Flex GM/PM** = margini nella banda del bending, se il modo e' *phase*-stabilizzato.  $|L(\omega_{BM})|$  = attenuazione del modo, se invece e' *gain*-stabilizzato (il nostro caso: -18.2 dB col notch, +29.0 dB senza).

**D: Perche' la maschera gf > 0 sui gain margins?** R: Perche' **allmargin** sul loop pieno restituisce un'entrata spuria a frequenza **zero** con -283 dB, prodotta dall'integratore di posizione laterale ( $|L(0)| = \text{Inf}$ ). Senza quella maschera il "margine aerodinamico" verrebbe letto come -283 dB e il tuner inseguirebbe un fantasma. E' la traccia numerica piu' diretta del fatto che il canale di deriva contamina la lettura dei margini.

**D: 'maxv' per il Rigid PM: perche' il massimo e non il minimo? Non e' ottimistico?** R: Si', e' ottimistico, ed e' una scelta deliberata di difesa contro gli artefatti: se un attraversamento del lobo di deriva sfuggisse appena sopra **w\_drift**, avrebbe un PM molto negativo e il **max** lo scarterebbe automaticamente. Il rovescio della medaglia e' che, se in banda rigida esistessero *due* attraversamenti genuini, verrebbe riportato il piu' favorevole invece del piu' critico. Nei casi effettivamente calcolati (Task 1, Task 2, tutti i corner del Task 3) in banda rigida c'e' sempre **un solo** attraversamento, quindi il punto resta teorico - ma la regola in se' non e' conservativa.

**D: Come e' definito il delay margin qui, ed e' classificato per banda?** R: **DM\_s** = min(am.DelayMargin), cioe' il **minimo su tutti** gli attraversamenti, **non** filtrato per banda - unica eccezione alla filosofia del file, e vale segnalarla. In pratica il minimo cade sempre sul crossover rigido perche' **DM** = PM\_rad/omega e gli

attraversamenti di deriva stanno a frequenze ~10 volte piu' basse (delay margin di 24-25 s contro 0.17-0.21 s). Valori: 213 ms nel Task 1, 165 ms nel Task 2 dopo il re-tune, 209 ms nel corner V3 – tutti sopra la soglia tipica di 100 ms per un lanciatore.

**D: Perche' il Rigid GM e' NaN nel Task 1?** R: Perche' l'attuatore e' ideale e non aggiunge ritardo di fase: la fase del loop tende a -90 deg alle alte frequenze e **non riattraversa mai** la fase critica. Non esiste quindi un bordo superiore alla banda di guadagni: verificato numericamente, il ciclo chiuso resta stabile fino a  $k = 1e6$ . La stabilita' condizionale nel Task 1 e' quindi **a un solo lato** (solo la riduzione di guadagno destabilizza). Il bordo superiore compare nel Task 2, quando i 20 ms di ritardo di trasporto e il TVC del 2o ordine introducono il lag che crea l'attraversamento a 11.1 rad/s (+7.56 dB).

# HM3/design\_controller.m

## Ruolo del file nel progetto

Questa funzione è l'**auto-tuner** del PD d'assetto di HM3. Riceve l'impianto (`build_plant_rigid` per il Task 1, `build_plant_full` per i Task 2/3), la catena attuatore/filtro `Wact` (vuota per il Task 1, `TVC * notch` per il Task 2), e restituisce la coppia di guadagni (`Kp_th`, `Kd_th`) che porta i **margini classificati** ai target d'assegnazione:  $|GM| \sim 6$  dB e  $|PM| \sim 30$  deg. È chiamata da `main_task1.m` riga 22, da `main_task2.m` riga 25 (Step A, rigido) e riga 153 (re-tune sul loop pieno), da `main_task3.m` riga 23 e da `main_montecarlo.m` riga 42; la usano anche `make_pz_figures.m` (righe 40 e 55), `init_simulink_hm3.m` (riga 34), `LTV_FULL_ASCENT/init_simulink_lpv.m` (righe 72 e 82) e i test.

Il file esiste per risolvere un problema molto concreto. La formula analitica di libro (D'Antuono, Trotta) per un lanciatore aerodinamicamente instabile dà in forma chiusa la coppia

$$Kp0 = 2*A_6/K_1 \quad Kd0 = \sqrt{A_6}/K_1$$

ma quelle formule sono ricavate sulla **dinamica rotazionale disaccoppiata**, cioè su  $\ddot{\theta} = A_6\theta + K_1\delta$ . Il loop vero di HM3 non è quello: c'è anche il feedback di deriva laterale ( $Kp_{zz\_m} + Kd_{zzdot\_m}$ ) e c'è l'accoppiamento ( $A_6/V$ ) *dentro l'impianto*. Il risultato è che i guadagni canonici, applicati al loop pieno, **non centrano più i target**: eseguendo il codice, il seed canonico ( $Kp0 = 1.4817$ ,  $Kd0 = 0.4029$ ) sul loop rigido pieno dà  $|GM_{aero}| = 4.20$  dB e  $PM_{rigid} = 27.33$  deg – e non 6 dB / 30 deg. La docstring (righe 7-9) dichiara esattamente questo: "the lateral-drift feedback erodes the aerodynamic gain margin (the canonical decoupled 6 dB drops to ~4 dB on the full loop)". Verificato.

La strategia adottata è quindi in due tempi: **(1)** parti dalla coppia canonica come punto iniziale (riga 55), **(2)** ri-sintonizza con un `fminsearch` che minimizza lo scarto quadratico dai target, dove i margini sono letti sul **loop pieno e classificati per banda di frequenza** da `classify_margins`. La classificazione è obbligatoria, non un vezzo: il loop è condizionalmente stabile e `margin()` da solo restituisce un numero senza significato (vedi la pagina di `classify_margins`).

Dipendenze: `assemble_loop` (costruisce L e T), `classify_margins` (legge i margini per banda), `isstable` per il verdetto finale. Nessun toolbox oltre al Control System Toolbox; il tuner usa `fminsearch` di base MATLAB (nessun Optimization Toolbox richiesto).

---

## Firma, docstring e contratto (righe 1-23)

```
function [K, m] = design_controller(G, Wact, o)
```

- Riga 1: firma. `G` = impianto LTI (4 stati nel Task 1, 6 nel Task 2/3), `Wact` = blocco in serie controllore->impianto (attuatore TVC + ritardo + notch), `o` = name-value. Restituisce `K` (struct dei 4 guadagni) e `m` (i margini classificati, arricchiti con `stable`, `L`, `T`).
- Righe 2-11: la docstring dichiara il metodo – seed canonico D'Antuono Eq. 3.6-3.7, poi re-tune sul loop pieno, margini classificati per banda, e il verdetto di stabilità preso da `isstable(T)` e **non** dal segno di un margine. Questa distinzione è il cuore dell'homework.



- Riga 15: `Wact = []` o `tf(1)` significa **attuatore ideale** (Task 1).

## Blocco arguments (righe 25-37)

- Righe 28-29: `Kp_z`, `Kd_z` di default **-1e-3** entrambi. Sono i guadagni di deriva laterale: piccoli e **negativi**. Non sono ottimizzati dal tuner – restano fissi. Il segno negativo e' il classico *load relief / drift-minimum* (il lanciatore lascia un po' di deriva pur di scaricare l'incidenza aerodinamica a  $\max-qbar$ ). Il canale `z/delta` ha uno zero a destra (eseguendo il codice: `z_m/delta = 20.609*(s+3.553)*(s-3.553) / [s*(s+1.861)*(s-1.816)*(s-0.02909)]`), quindi e' **non-minimo di fase**: un feedback di posizione laterale forte destabilizzerebbe. Da qui l'ordine di grandezza  $1e-3$ .
  - **Il segno e' l'intenzione, non il risultato.** A  $1e-3$  l'effetto di load relief e' trascurabile e il bilancio di incidenza resta dominato dal PD d'assetto: misurando la risposta alla raffica nominale, il picco di  $|\alpha| = |\theta + \dot{z}/V - \alpha_w|$  vale **0.577 deg** contro i **0.390 deg** che il vento produrrebbe da solo. Cioe' l'anello **aggrava** il carico invece di alleggerirlo – vedi la domanda d'esame in fondo alla pagina. Va detto cosi', senza spacciare l'intento per un risultato.
- Righe 30-31: `i_target GM = 6 dB` e `PM = 30 deg` – i numeri della traccia.
- Riga 32: `K0` e' **accettato ma non usato** (il commento lo dice: “*accepted, unused*”). E' un residuo di una versione precedente in cui il tuner poteva essere warm-startato. Conseguenza reale e onesta: **il re-tune del Task 2 non riparte dai guadagni del Task 1**, ma sempre dalla coppia canonica. Va detto all'orale se qualcuno chiede “e il warm start?”.
- Righe 33-35: le tre frequenze di banda passate al classificatore. Default `Inf/Inf/NaN` = “nessun modo flessibile”, che e' il caso Task 1.
- Riga 38: `[] -> tf(1)`, l'alias per attuatore ideale.

## Estrazione di `A_6`, `K_1` e definizione delle bande (righe 40-48)

```
iTh = strcmp(G.StateName, 'theta');
iTd = strcmp(G.StateName, 'thetadot');
iDe = strcmp(G.InputName, 'delta');
A6 = G.A(iTd, iTh);
K1 = G.B(iTd, iDe);
w_drift = 0.3*sqrt(A6);
```

- Righe 41-45: i coefficienti **non** vengono passati come argomento, ma **letti per nome** dalle matrici dell'impianto. `A6 = A(thetadot, theta)` e `K1 = B(thetadot, delta)` sono per costruzione (vedi `build_plant_rigid` righe 11-16) esattamente i coefficienti della riga `theta_ddot`. Il vantaggio: la stessa funzione lavora sull'impianto a 4 stati e su quello a 6 stati senza modifiche, e nei corner del Task 3 (dove `load_hw3_params` scala `A6` e `K1`) legge automaticamente i valori **scalati**. Nessuna possibilita' di disallineamento fra impianto e guadagni.
- Riga 46: `w_drift = 0.3*sqrt(A6)`. E' il confine fra la **banda di deriva** e la **banda rigida** usato dal classificatore. La scala naturale del corpo rigido e'  $\sqrt{A_6} = 1.839 \text{ rad/s}$  (il polo instabile aerodinamico); il lobo di deriva sta ben sotto.  $0.3*\sqrt{A_6} = \mathbf{0.5517 \text{ rad/s}}$  al nominale. Nota che il default interno di `classify_margins` e' 0.5 fisso: qui viene **sovrascritto** con un valore che **scala con l'instabilita' dell'aeroshell**, il che e' essenziale nei corner del Task 3 (con  $\mu_\alpha = 1.3$ , `w_drift` diventa  $0.629 \text{ rad/s}$ ).
- Righe 47-48: le bande in formato cell array, poi espanse con `bands{:}` nelle chiamate a `classify_margins`. Task 1: default (nessun bending). Task 2/3: il chiamante passa `w_flex = 0.6*wBM = 11.34`, `w_flex_hi = 1.5*wBM = 28.35`, `w_bending = wBM = 18.9`.

**Possibile domanda d'esame** – perche' `A_6` e `K_1` sono letti dalle matrici dell'impianto invece di essere passati come parametri? *Risposta:* perche' cosi' i guadagni canonici di partenza sono garantiti coerenti con l'impianto che si sta effettivamente chiudendo. Nei corner del Task 3 `load_hw3_params` scala  $\mu_\alpha$  e  $\mu_c$  del  $\pm 30\%$ : se `A_6` e `K_1` venissero passati a mano si rischierebbe di seminare il tuner con i valori nominali su un impianto perturbato. Leggendoli per nome di stato non c'e' modo di sbagliare.



## Silenziamento del warning MarginUnstable (righe 50-52)

```
warnState = warning('off', 'Control:analysis:MarginUnstable');
restoreWarn = onCleanup(@() warning(warnState));
```

- Riga 51: MATLAB emette `Control:analysis:MarginUnstable` a ogni chiamata di `allmargin/margin` su un loop open-loop instabile – ed e' il nostro caso (l'impianto ha il polo a  $+1.82$  rad/s). Con `fminsearch` che valuta il costo centinaia di volte, il command window verrebbe sommerso.
- Riga 52: `onCleanup` **ripristina** lo stato del warning all'uscita, anche in caso di errore. E' testato esplicitamente: `tests/hm3LoopTest.m` righe 88-96 (`testDesignControllerRestoresWarningState`). Non e' pedanteria: silenziare un warning globalmente e non ripristinarlo e' un effetto collaterale che inquinerebbe tutte le sessioni successive dell'utente.

## Il re-tune: seed canonico + fminsearch (righe 54-57)

```
x0 = log([2*A6/K1, sqrt(A6)/K1]); % D'Antuono Eq. 3.6-3.7
x0 = fminsearch(@cost, x0, ...
    optimset('Display','off','TolX',1e-4, ...
    'TolFun',1e-3,'MaxFunEvals',400));
```

Qui c'e' tutta la teoria dell'homework compressa in due righe. **Derivo le due formule canoniche**, perche' e' la domanda d'orale piu' probabile su questo file.

**Dinamica rotazionale disaccoppiata.** Si trascura l'accoppiamento con la deriva laterale (il termine  $(A_6/V)*\dot{z}$ , che pesa  $1/V \sim 1e-3$ ) e resta

$\theta_{ddot} = A_6\theta + K_1\delta$

Con la legge PD ( $\theta_{ref} = 0$ ):  $\delta = -K_p\theta - K_d\dot{\theta}$ , il loop aperto e'

$L_{\theta}(s) = K_1(K_p + K_d s) / (s^2 - A_6)$

e la caratteristica a ciclo chiuso e'

$s^2 + K_1K_d s + (K_1K_p - A_6) = 0$

da cui, per confronto con  $s^2 + 2\zeta\omega_n s + \omega_n^2$ ,

$\omega_n = \sqrt{K_1K_p - A_6}$        $\zeta = K_1K_d / (2\omega_n)$

**\*\* (a)  $K_{p0} = 2A_6/K_1$  viene dal requisito di 6 dB di gain margin. \*\*** Si moltiplica il loop per un guadagno  $k$  e si riapplica Routh alla caratteristica  $s^2 + kK_1K_d s + (kK_1K_p - A_6)$ : la stabilita' richiede il termine noto positivo, cioe'

$k > k_{min} = A_6 / (K_1K_p)$

Attenzione: e' un **limite inferiore** su  $k$ . Il margine e' una **riduzione** di guadagno, non un aumento – ed e' la ragione per cui in dB il margine aerodinamico e' **negativo** (cfr. Barrows & Orr, cap. 9: *gain-reduction margin*). In dB:

$GM_{aero} = 20\log_{10}(A_6/(K_1K_p))$

Imporre  $|GM_{aero}| = 6$  dB significa  $k_{min} = 1/2$ , cioe'  $K_1K_p = 2A_6$ :

$K_{p0} = 2A_6/K_1$

Lettura equivalente e piu' geometrica:  $|L_{\theta}(0)| = K_1K_p/A_6 = 2 = +6.02$  dB, e la fase di  $L_{\theta}$  a DC e' **esattamente la fase critica** (il denominatore  $s^2 - A_6$  vale  $-A_6 < 0$  in continua, e un segno meno e' -180 deg di fase – ossia +180: stesso punto, fase mod 360). Quindi la curva di Nichols **parte esattamente sulla fase critica, 6 dB sopra il punto critico** (-180 deg, 0 dB nella convenzione del corso, da  $1 + L = 0$ ; D'Antuono etichetta lo stesso punto +180 sulla carta rietichettata di +360 – e' la convenzione che il codice usava fino a poco

fa): abbassa il guadagno di 6 dB e ci finisci dentro. Verifica numerica eseguendo il codice: `allmargin(L_theta)` restituisce **GM = -6.02 dB alla frequenza 0**.

(b)  $Kd0 = \sqrt{A\_6}/K\_1$  viene dai **30 deg di phase margin**. Con  $Kp = Kp0$  si ha  $K\_1Kp = 2A\_6$  e  $K\_1Kd = \sqrt{A\_6}$ . La frequenza di attraversamento a 0 dB risolve  $|L\_theta(jw)| = 1$ , cioè  $K\_1*\sqrt{Kp^2 + Kd^2*w^2} = w^2 + A\_6$ . Elevando al quadrato:

$$w\_c^4 + A\_6*w\_c^2 - 3*A\_6^2 = 0 \quad \rightarrow \quad w\_c^2 = A\_6*(\sqrt{13}-1)/2 = 1.3028*A\_6$$

La fase di  $L\_theta$  è  $-180 \text{ deg} + \text{atan}(Kd*w/Kp)$  (il numeratore ha fase positiva, il denominatore reale negativo contribuisce la fase critica  $-180 \text{ deg} - +180$  se si preferisce l'altro ramo, è lo stesso mod 360), quindi il phase margin rispetto al punto critico a  $-180 \text{ deg}$  è

$$PM = \text{atan}(Kd*w\_c/Kp) = \text{atan}(w\_c / (2*\sqrt{A\_6})) = \text{atan}(\sqrt{1.3028}/2) = 29.71 \text{ deg}$$

**che non dipende ne' da  $A\_6$  ne' da  $K\_1$** . Ecco perché la coppia canonica è la coppia canonica: da' 6 dB e ~30 deg su qualunque lanciatore instabile, quali che siano i suoi coefficienti. Verifica numerica: sul loop disaccoppiato con  $Kp0 = 1.4817$ ,  $Kd0 = 0.4029$  si ottiene **GM = -6.02 dB @  $w = 0$  e PM = 29.71 deg @ 2.099 rad/s**, con la coppia a ciclo chiuso  $\omega_n = \sqrt{A\_6} = 1.839 \text{ rad/s}$  e **zeta = 0.5 esatto**.

**Perché allora il re-tune?** Perché il loop vero non è quello disaccoppiato. Eseguendo il codice sul loop rigido pieno (4 stati + feedback di deriva) con gli stessi guadagni canonici si ottiene  $|GM\_aero| = 4.20 \text{ dB}$  e  $PM\_rigid = 27.33 \text{ deg}$ . Spegnendo i soli guadagni di deriva ( $Kp\_z = Kd\_z = 0$ ) si risale a 5.56 dB / 28.74 deg: quindi l'erosione è **in parte** dovuta all'accoppiamento già presente nell'impianto (i termini  $a1, a4, A\_6/V$ ) e **in parte maggiore** al feedback di deriva. Il tuner recupera la differenza spostando i guadagni a  **$Kp\_th = 1.7845$ ,  $Kd\_th = 0.4433$** , che sul loop pieno danno esattamente  $|GM\_aero| = 6.000 \text{ dB}$  @ 0.593 rad/s e  $PM\_rigid = 30.00 \text{ deg}$  @ 2.455 rad/s.

- Riga 55: `x0 = log([...])`. La **parametrizzazione logaritmica** è il modo di imporre  $Kp > 0$  e  $Kd > 0$  con un ottimizzatore **non vincolato** come `fminsearch`: si ottimizza su  $x = \log(K)$ , e  $K = \exp(x)$  è positivo per costruzione, qualunque cosa faccia il semplice. Guadagni negativi invertirebbero il segno del feedback e renderebbero il problema privo di senso fisico.
- Righe 56-57: `fminsearch = Nelder-Mead, derivative-free`. È la scelta giusta qui, non un ripiego: la funzione di costo contiene una **classificazione** (quale attraversamento appartiene a quale banda) e due penalità a gradino (righe 86 e 89). Il costo è quindi discontinuo a tratti e il suo gradiente non esiste dove la classificazione cambia ramo: `fsolve` o `fmincon` con gradienti alle differenze finite sarebbero fragili. Tolleranze: `TolX = 1e-4` sui log-guadagni, `TolFun = 1e-3` sul costo (che ha unità  $\text{dB}^2 + \text{deg}^2$ ), `MaxFunEvals = 400`.

**Possibile domanda d'esame** – il problema ha 2 incognite e 2 target: perché minimizzare uno scarto quadratico invece di risolvere il sistema con `fsolve`? *Risposta:* il problema è effettivamente quadrato, e infatti il costo viene annullato (i margini finali sono 6.000 dB e 30.00 deg, non “vicini a”). Ma la mappa  $(Kp, Kd) \rightarrow (GM\_aero, PM\_rigid)$  passa attraverso `allmargin` + classificazione per banda: è liscia solo finché gli attraversamenti restano nelle rispettive bande. Formulare come minimizzazione permette di **penalizzare in modo continuo** il fallimento (`NaN`  $\rightarrow 1e6$ , instabile  $\rightarrow +1e4$ ) invece di far divergere un root-finder; e Nelder-Mead non ha bisogno di derivate. È la scelta pragmatica corretta.

## Applicazione dei guadagni e verdetto di stabilità (righe 59-69)

```
K.Kp_th = exp(xo(1));
K.Kd_th = exp(xo(2));
...
[L, T] = assemble_loop(G, K, Wact);
L = minreal(L, 1e-6);
m = classify_margins(L, bands{:});
m.stable = isstable(T);
```

- Righe 59-62: si esce dai log e si riattaccano i guadagni di deriva **fissi** (non ottimizzati).
- Riga 64: `assemble_loop` ricostruisce il loop una volta finale, con i guadagni vincitori.
- Riga 65: `minreal(L, 1e-6)` **cancella i modi non minimi** introdotti da `connect/getLoopTransfer`. Serve: senza cancellazione, `allmargin` lavora su una realizzazione con poli/zeri quasi coincidenti e i margini si sporcano.
- Riga 67: **il verdetto di stabilit  e' `isstable(T)`, non un segno di margine**. Questa e' la riga concettualmente piu' importante del file. Su un sistema condizionalmente stabile il segno del gain margin non dice se il loop chiuso e' stabile (il criterio di Nyquist richiede il **conteggio degli avvolgimenti** rispetto ai poli a destra dell'impianto, e qui i poli instabili sono **due**: +1.8165 e +0.0291 rad/s, oltre all'integratore in 0). L'unico test onesto e' guardare i poli del ciclo chiuso.

## Stampa diagnostica (righe 71-76)

- Riga 75: stampa `abs(m.aeroGM_dB)`. **La `abs()` non e' cosmetica**: il margine aerodinamico e' *negativo* in dB per costruzione (e' una gain-reduction margin), e stamparlo come  $|GM| = 6.0$  dB e' cio' che rende confrontabile il numero con il requisito "6 dB" della traccia. Il segno resta pero' nella struct: chi legge `m.aeroGM_dB` trova -6.000.

## Funzione annidata `cost` (righe 78-90)

```
function c = cost(x)
    Kt.Kp_th = exp(x(1)); Kt.Kd_th = exp(x(2));
    Kt.Kp_z = o.Kp_z; Kt.Kd_z = o.Kd_z;
    [Lt, Tt] = assemble_loop(G, Kt, Wact);
    Lt = minreal(Lt, 1e-6);
    mt = classify_margins(Lt, bands{:});
    if isnan(mt.aeroGM_dB) || isnan(mt.rigidPM_deg)
        c = 1e6; return;
    end
    c = (abs(mt.aeroGM_dB) - o.GM)^2 + (mt.rigidPM_deg - o.PM)^2;
    if ~isstable(Tt), c = c + 1e4; end
end
```

- Riga 78: funzione **annidata** (non sotto-funzione): vede `G`, `Wact`, `o` e `bands` per chiusura lessicale. E' il motivo per cui `fminsearch(@cost, x0)` puo' essere chiamata con un solo argomento.
- Righe 80-84: a ogni valutazione si **riassembla l'intero loop** e si **riclassificano** i margini. E' costoso (un `connect` + un `minreal` + un `allmargin` per iterazione, fino a 400 volte) ma e' l'unico modo per ottimizzare sui margini *veri* del loop pieno e non su una loro approssimazione.
- Righe 85-87: **penalita' dura 1e6** se una delle due bande **non ha piu' un attraversamento** (`classify_margins` restituisce NaN). Traduzione fisica: se il guadagno e' tale che la curva non attraversa piu' la fase critica in banda aerodinamica, il "margine aerodinamico" non esiste come numero – non e' che sia infinito, e' che il concetto e' perso. Il `return` anticipato evita di calcolare un costo su NaN (che si propagherebbe e farebbe uscire `fminsearch` con un semplice degenerare).
- Riga 88: **la funzione di costo vera e propria**.

$$c = (|GM_{aero}| - 6)^2 + (PM_{rigid} - 30)^2$$

Tre osservazioni oneste:

1. **Le unita' sono mescolate**:  $dB^2$  sommati a  $deg^2$ , con peso implicito 1:1. Un errore di 1 dB "costa" quanto un errore di 1 deg. E' arbitrario, ma essendo il problema quadrato (2 incognite, 2 target) il minimo e' comunque lo zero e la pesatura non cambia la soluzione – influenza solo il cammino del semplice.

2. Su `aeroGM_dB` c'è `abs()` (perché è negativo), su `rigidPM_deg` **no**. Voluto: un PM negativo (curva dalla parte sbagliata) verrebbe così penalizzato pesantemente,  $((-8) - 30)^2$ , e spinto verso +30.
  3. Il tuner insegue il **target**, non lo massimizza: cerca  $|GM| = 6$  dB, non  $|GM| \geq 6$  dB. Se un guadagno desse 9 dB di margine aerodinamico, il costo salirebbe. È coerente con la traccia (che chiede quei valori) ma va detto: non è un'ottimizzazione di robustezza, è un *matching* di specifica.
- Riga 89: **penalita' additiva 1e4** se il ciclo chiuso è instabile. Additiva e non sostitutiva: il costo dei margini resta dentro, quindi anche nella regione instabile il simplesso ha ancora un gradiente numerico che lo guida verso il target. Se fosse `c = 1e4` secco, la regione instabile sarebbe un plateau piatto e Nelder-Mead ci si perderebbe.

**Possibile domanda d'esame** – perché la penalita' di instabilità è *additiva* (`c = c + 1e4`) mentre quella dei NaN è *sostitutiva* (`c = 1e6`)? *Risposta:* nel caso instabile i margini classificati esistono ancora e portano informazione utile (“quanto sei lontano dai target”), quindi si somma la penalita' e si conserva il gradiente. Nel caso NaN i margini **non esistono**: non c'è nulla da sommare, e l'unica cosa sensata è un valore piatto e molto alto che dice al simplesso “torna indietro”. Le due situazioni sono qualitativamente diverse e sono trattate diversamente.

## Possibili domande d'esame

**\*\*D:** Da dove escono  $K_p = 2A_6/K_1$  e  $K_d = \sqrt{A_6}/K_1$ ? Ricavale. **R:** Dalla dinamica rotazionale disaccoppiata  $\ddot{\theta} = A_6\theta + K_1\delta$  con PD. La caratteristica a ciclo chiuso è  $s^2 + K_1K_d s + (K_1K_p - A_6)$ . *Scalando il loop di k, Routh richiede  $k > A_6/(K_1K_p)$ : è un limite inferiore\*\**, cioè una *gain-reduction* margin. Imporre 6 dB ( $k_{\min} = 1/2$ ) da'  $K_1K_p = 2A_6$ . Con quel  $K_p$ , la frequenza di crossover risolve  $\omega_c^4 + A_6\omega_c^2 - 3A_6^2 = 0$  cioè  $\omega_c = \sqrt{1.3028A_6}$ , e la fase di L è  $-180 \text{ deg} + \text{atan}(K_d\omega_c/K_p)$  (la fase critica viene dal denominatore reale negativo), da cui il margine rispetto al punto critico a  $-180 \text{ deg}$  è  $PM = \text{atan}(\omega_c/(2\sqrt{A_6})) = 29.71 \text{ deg}$  **indipendentemente da  $A_6$  e  $K_1$**  se  $K_d = \sqrt{A_6}/K_1$ . Quindi 6 dB e 30 deg escono automaticamente dalla coppia canonica. La coppia dà anche  $\omega_n = \sqrt{A_6}$  e  $\zeta = 0.5$  esatti a ciclo chiuso.

**D: Se le formule canoniche danno già' 6 dB e 30 deg, perché' il codice fa comunque un `fminsearch`?**

**R:** Perché' quelle formule valgono sul loop **disaccoppiato**. Il loop vero include il feedback di deriva laterale ( $K_{p_z}, K_{d_z} = -1e-3$ ) e l'accoppiamento  $(A_6/V)*\dot{z}$  dentro l'impianto. Eseguendo il codice, la coppia canonica sul loop pieno dà' 4.20 dB / 27.33 deg invece di 6 / 30: il margine aerodinamico è eroso di quasi 2 dB. Il `fminsearch` ricompra quei 2 dB muovendo i guadagni a  $K_p = 1.7845$ ,  $K_d = 0.4433$  (Task 1). Nel Task 2, con TVC + ritardo + notch in serie, il re-tune sposta soprattutto **Kd** (0.44 → 0.69) per rimpiazzare la fase mangiata dall'attuatore e dal ritardo.

**D: Perché' `fminsearch` e non `fmincon` o `fsolve`?** **R:** (a) La funzione di costo contiene una classificazione per banda e due penalita' a gradino, quindi è discontinua a tratti e senza gradiente definito dove un attraversamento passa da una banda all'altra; Nelder-Mead è derivative-free e non si spezza. (b) La positività dei guadagni è già' imposta dalla parametrizzazione  $x = \log(K)$ , quindi non serve un ottimizzatore vincolato. (c) Non serve alcun toolbox aggiuntivo: `fminsearch` è base MATLAB, e HM3 dichiara di dipendere dal solo Control System Toolbox.

**D: Perché' il verdetto finale di stabilità è `isstable(T)` e non il segno del gain margin?** **R:** Perché' il lanciatore è open-loop instabile: eseguendo il codice, il loop rigido ha **due** poli a parte reale positiva (+1.8165 e +0.0291 rad/s) più l'integratore di deriva in 0. Il criterio di Nyquist su un sistema con  $P > 0$  non si legge dal segno di un margine ma dal conteggio degli avvolgimenti attorno al punto critico ( $-180 \text{ deg}$ , 0 dB sulla carta di Nichols). Il loop è **condizionalmente stabile**: nel Task 2 (loop pieno con TVC + ritardo + notch) la banda di guadagni stabili verificata numericamente è  $k$  in  $[0.50, 2.39]$  – sia ridurre sia aumentare il guadagno destabilizza, e i due estremi sono esattamente l'Aero GM (-6.0 dB) e il Rigid GM (+7.56 dB): il punto critico resta incastrato fra i due attraversamenti della fase critica. L'unico test che non mente è guardare i poli di T.

**D: Nel Task 1 la banda di guadagni stabili è davvero a due estremi?** **R: No, e questo va detto onestamente.** Con l'attuatore *ideale* (Task 1) il loop è stabile per  $k$  in  $[0.50, +\infty)$ : verificato fino a  $k = 1e6$ .

La fase del loop tende a -90 deg alle alte frequenze e **non riattraversa** mai la fase critica, quindi non c'è un estremo superiore. Coerentemente, `classify_margins` restituisce `rigidGM_dB = NaN` nel Task 1 (e il commento nel codice, righe 10-11 di `classify_margins`, lo dice: “*absent for an ideal actuator*”). La conditional stability a **due** lati compare solo nel Task 2, quando il ritardo di 20 ms e la dinamica TVC del 2o ordine aggiungono il ritardo di fase che crea l'attraversamento superiore a 11.1 rad/s.

**D: I guadagni di deriva  $K_{p_z}$ ,  $K_{d_z}$ : perché negativi, perché piccoli, e perché il tuner non li tocca?** R: Negativi perché *intendono* realizzare il *load relief* (drift-minimum): il veicolo accetta un po' di deriva laterale pur di ridurre l'incidenza aerodinamica

$\alpha = \theta + \dot{z}/V - \alpha_w$

e quindi il carico  $q\bar{\alpha}$  a  $\max q\bar{\alpha}$ . (Il **meno** su  $\alpha_w$  lo impone il plant: l'Eq. (1) ha la colonna di vento  $B_w = [0; -a_1 V; 0; -A_6]$ , quindi  $\alpha$  è l'incidenza rispetto alla velocità **relativa all'aria**.) Piccoli ( $1e-3$ ) perché il canale  $z/\delta$  è non-minimo di fase (zero a  $+3.553$  rad/s): un feedback di posizione laterale forte destabilizzerebbe. Il tuner non li tocca perché sono una **scelta di missione** fissata dalle guidelines della traccia, non un grado di libertà di stabilizzazione: si sintonizza il PD d'assetto dato\* il canale di load relief. Il prezzo lo si paga in margine aerodinamico (i  $\sim 2$  dB persi) e lo si recupera con il re-tune.

**Ma va detta la verità sui numeri:** a  $1e-3$  quel load relief **non funziona**. Misurando la risposta alla raffica nominale, il picco di  $|\alpha|$  vale **0.577 deg** mentre il vento da solo produrrebbe solo **0.390 deg**: l'assetto non elide il vento, **ci si somma**, e  $q\bar{\alpha}$  sale a 46.8 kPa\*deg. La ragione è che per tenere  $\theta_{ref} = 0$  il loop deve opporsi al momento aerodinamico, e così facendo **becca il muso dentro il vento relativo**. **Un puro attitude-hold è load-aggravating, non load-relieving**. E non c'è effetto banderuola a salvarlo: con  $A_6 > 0$  il centro di pressione sta davanti al baricentro, quindi il momento aerodinamico è **divergente** – il veicolo non si allinea al vento, ci diverge contro, ed è esattamente per questo che serve il PD. Un load relief vero richiederebbe guadagni di deriva molto più aggressivi (o un accelerometro), pagati in deriva laterale e in margine.

*Onesta storica, se l'esaminatore chiede della versione precedente:* fino a poco fa il **post-processing** di `simulate_gust_response` scriveva  $\alpha = \theta + \dot{z}/V + \alpha_w$  (con il +), e dava 0.255 deg / 20.7 kPa\*deg – facendo sembrare il progetto load-relieving. Era un bug **di post-processing**, non di modello: il plant integrato (`build_plant_rigid`, `ode_lpv_ascent`) ha sempre avuto il meno, quindi  $\theta$ ,  $z$ ,  $\delta$  e **tutti i margini** non ne erano toccati; a sbagliare era solo il canale diagnostico  $\alpha$ . Corretto in tutti i siti.

# HM3/plot\_\_nichols\_\_lv.m

## Ruolo del file nel progetto

Questa funzione disegna la **carta di Nichols del loop pieno con il punto critico a (-180 deg, 0 dB)** – la convenzione degli appunti del corso, da  $1 + L = 0 \Leftrightarrow L = -1 = e^{-j*180 \text{ deg}}$  – curva che “viene dall’alto”, e i margini **classificati** da `classify_margins` marcati ciascuno alla sua frequenza di attraversamento. E’ chiamata da `main_task1.m` riga 63 e da `main_task2.m` riga 197; il Task 3 (`main_task3.m` riga 81) e la figura di trade dei filtri (`main_task2.m` righe 222-243) rifanno lo shift a mano per poter sovrapporre piu’ loop con **un unico** riferimento di fase comune.

Perche’ non basta `nichols(L)` e via? Perche’ la figura deve **raccontare la storia** della stabilita’ condizionale, e la figura di default la nasconde:

1. **La convenzione di fase.** Il punto critico di Nyquist (-1, 0) si traduce sulla carta di Nichols in **(-180 deg, 0 dB)**: e’ dove gli appunti del corso leggono il GM (single-sheet chart con fase in [-360, 0]). La fisica del lanciatore rende la lettura immediata: la dinamica rotazionale instabile ha denominatore  $s^2 - A_6$ , che in continua vale  $-A_6 < 0$  – e quel segno meno e’ la fase critica (-180 deg, ossia +180: stesso punto, fase mod 360) – quindi la curva **parte** esattamente sulla fase critica e il margine aerodinamico e’ l’altezza della curva sopra il punto critico all’attraversamento di bassa frequenza. Il **PhaseMatching** serve a garantire che quell’attraversamento cada davvero a -180 e non su una copia a  $-180 + 360k$ . (Nota storica: D’Antuono Fig. 3.2 e Trotta mostrano la **stessa** carta rietichettata di +360, punto critico a +180; il codice usava quella rietichettatura fino a poco fa ed e’ stato riallineato alla convenzione del corso. E’ una pura rietichettatura dell’asse: **nessun margine cambia.**)
2. **La curva viene dall’alto.** Il canale di deriva laterale ha un integratore libero, quindi  $|L| \rightarrow \text{Inf}$  a DC: eseguendo il codice, a  $\omega = 0.01 \text{ rad/s}$  il loop del Task 2 sta a **+47.9 dB**. Una Nichols di libro parte da destra e scende; questa parte in cima.
3. **I margini vanno etichettati.** Su questa curva ci sono *cinque* possibili attraversamenti e due di essi (gli 0 dB del lobo di deriva) **non sono margini**. La funzione li marca esplicitamente come artefatti.

---

## Docstring e contratto (righe 1-22)

- Riga 1: firma `ax = plot_nichols_lv(L, mm, opts)`. Prende il loop `L` e la struct dei **margini gia’ classificati** `mm`. La funzione **non ricalcola niente**: e’ un puro renderer, e questo e’ corretto – la classificazione e’ responsabilita’ di `classify_margins`, il disegno di questa.
- Righe 4-6: la scelta di usare **nicholsplot nativo** e non un `plot` a mano. Il motivo dichiarato e’ la **griglia M/N standard** con le etichette in dB (0, +/-0.25, +/-0.5, +/-1, +/-3, +/-6, +/-12, +/-20 dB). Quelle non sono linee di guadagno del loop aperto: sono i **contorni a modulo costante del ciclo chiuso**  $|T| = |L/(1+L)|$ , cioe’ le M-circles di Hall trasportate sul piano di Nichols. Il contorno a 6 dB e’ quello che nel report si dice “sfiorato” o “evitato”: significa picco di risonanza del ciclo chiuso  $\leq 6 \text{ dB}$ .
- Righe 6-8: il punto critico rigido a **-180 deg** – “the course convention, from  $1 + L = 0 \Leftrightarrow L = -1$ ” – con le copie “wrapate” dei punti critici flex a -540 / -900. Con l’`xlim` di default [-720, 0] ne restano visibili **due**: -180 e -540.

- Righe 9-10: la nota di raccordo con la letteratura: D’Antuono Fig. 3.2 mostra la **stessa carta rietichettata di +360 deg** (la fase e’ definita mod 360, quindi i due display sono la stessa curva). E’ la risposta pronta se all’orale chiedono perche’ nella tesi il punto critico sta a +180.

## Blocco arguments (righe 24-30)

- Riga 27: `wrange` default `[1e-3 1e3]`; entrambi i main passano `[1e-2 1e2]`.
- Riga 28: `xlim` default `[-720 0]` – la finestra che contiene il punto critico a -180 e la sua copia wrappata a -540, e regge le escursioni di fase misurate (la fase visualizzata del loop Task 1 copre `[-448, -90]` deg, quella del Task 2 `[-914, -149]` deg).
- Righe 32-33: al solito, `Control:analysis:MarginUnstable` silenziato con ripristino via `onCleanup` (il loop e’ open-loop instabile, `nicholsplot` e `bode` lo urlerebbero a ogni chiamata).

## Scelta della frequenza di riferimento (riga 35)

```
wref = mm.rigidPM_w; if isnan(wref), wref = mm.aeroGM_w; end
```

- Riga 35: **tutto lo shift di fase e’ ancorato al crossover del corpo rigido**. E’ la scelta giusta: e’ il punto della curva di cui si vuole leggere il phase margin, quindi e’ quello che deve cadere dove ci si aspetta. Il fallback su `aeroGM_w` copre il caso patologico in cui il PM rigido non esiste (banda vuota -> NaN). **Non c’e’ un terzo fallback**: se anche `aeroGM_w` fosse NaN, `wref` resterebbe NaN e lo shift a riga 49 produrrebbe NaN su tutta la curva (la Nichols nativa verrebbe comunque disegnata, ma i marker sparirebbero). Non succede nei casi dell’homework, ma il codice non lo protegge.

## Nichols nativa e phase matching a -180 (righe 37-44)

```
L.InputName = ''; L.OutputName = '';
h = nicholsplot(L, {opts.wrange(1), opts.wrange(2)});
setoptions(h, 'PhaseMatching','on', 'PhaseMatchingFreq', wref, ...
            'PhaseMatchingValue', -180, ...
            'Grid','on', 'XLimMode','manual', 'YLimMode','manual', ...
            'XLim', {opts.xlim}, 'YLim', {[ -40 40]});
```

- Riga 38: azzerare `InputName/OutputName` serve solo a **togliere il sottotitolo “From: delta To: delta”** che MATLAB stampa sui modelli con IO nominati (il loop esce da `assemble_loop`, che nomina tutto). MATLAB passa gli argomenti per valore, quindi **la L del chiamante non viene toccata**.
- Riga 39: `nicholsplot` (non `nichols`) perche’ restituisce un **handle di plot** su cui si puo’ chiamare `setoptions`.
- Riga 40: **il cuore della convenzione** (nel sorgente le tre opzioni `PhaseMatching / PhaseMatchingFreq / PhaseMatchingValue` stanno tutte sulla stessa riga; il blocco qui sopra le manda a capo per leggibilita’). `PhaseMatching` istruisce MATLAB a sommare a tutta la curva un multiplo di 360 deg tale che, alla frequenza `PhaseMatchingFreq = wref`, la fase risulti il piu’ vicino possibile a `PhaseMatchingValue = -180`. Non e’ una deformazione: **sommare 360 deg alla fase e’ un’identita’** (la risposta in frequenza e’ la stessa), e’ solo una scelta di ramo. Nel Task 1 il ramo naturale di MATLAB mette gia’ il crossover rigido a **-150 deg**, quindi lo shift risulta **0**: la convenzione del corso coincide con il ramo naturale. Nel Task 2, invece, la fase grezza (unwrapped) al crossover rigido esce a **+570 deg** e lo shift e’ **-720**, che la riporta a **-150**. In entrambi i casi il crossover cade **30 deg a destra del punto critico -180** – e il phase margin di 30 deg si legge dal grafico invece di doverlo dedurre.
- Riga 42: `YLim [-40 40]` dB. Nota: la curva a bassissima frequenza arriva a +48 dB, quindi **il ramo iniziale esce dall’inquadratura**. E’ voluto: quel tratto non porta informazione (e’ l’asintoto dell’integratore) e comprimerlo schiaccerebbe la regione interessante.

**Che aspetto ha la curva risultante** (numeri misurati eseguendo il codice sul loop pieno del Task 2, gia’ shiftati nella convenzione del corso, punto critico a -180):

| omega [rad/s] | fase [deg]     |              | L                                                                         |
|---------------|----------------|--------------|---------------------------------------------------------------------------|
| 0.01          | -430.0         | +47.9        | la curva entra dall'alto (integratore di deriva)                          |
| 0.158         | -320.5         | 0.00         | 1o attraversamento 0 dB del lobo di deriva – <b>non un margine</b>        |
| 0.227         | -214.4         | 0.00         | 2o attraversamento 0 dB del lobo di deriva – <b>non un margine</b>        |
| 0.542         | <b>-180.00</b> | <b>+6.00</b> | <b>Aero GM:</b> attraversa la fase critica <b>sopra</b> il punto critico  |
| 3.170         | -150.0         | 0.00         | <b>Rigid PM:</b> 0 dB a 30 deg dal punto critico                          |
| 11.11         | <b>-179.99</b> | <b>-7.56</b> | <b>Rigid GM:</b> rientra sulla fase critica <b>sotto</b> il punto critico |
| 18.90         | -230.0         | -18.2        | omega_BM: il notch tiene il modo ben sotto 0 dB (gain-stabilizzato)       |
| 100           | -488.0         | -33.0        | coda ad alta frequenza                                                    |

Ed ecco disegnata la **stabilita' condizionale**: la curva attraversa la retta fase = -180 deg **due volte**, una a +6.00 dB e una a -7.56 dB. Il punto critico (-180 deg, 0 dB) resta **incastrato fra i due attraversamenti**. Abbassare il guadagno di 6 dB fa scendere il primo attraversamento sul punto critico; alzarlo di 7.56 dB fa salire il secondo. Entrambe le direzioni destabilizzano, ed e' esattamente la banda k in [0.50, 2.39] verificata per forza bruta con `isstable(feedback(k*L,1))`. **La curva deve tenere il punto critico incastrato fra i due passaggi sulla fase critica** – e su questa figura lo si vede.

**Possibile domanda d'esame** – negli appunti del corso il punto critico sta a (-180 deg, 0 dB), ma nella tesi di D'Antuono (Fig. 3.2) sta a +180. Chi ha ragione? E il codice quale convenzione usa? *Risposta:* hanno ragione entrambi, perche' +180 e -180 sono **lo stesso punto** (la fase e' definita modulo 360:  $e^{+j180} = e^{-j180} = -1$ ). Il -180 discende direttamente da  $1 + L = 0 \Leftrightarrow L = -1$  ed e' il ramo su cui il corso legge il GM (single-sheet chart con fase in [-360, 0]); D'Antuono mostra la stessa carta **rietichettata di +360**, una scelta diffusa nella letteratura lanciatori perche' il denominatore  $s^2 - A_6$  a DC vale  $-A_6 < 0$ , che si puo' leggere come "+180 deg di fase". La fisica non cambia con l'etichetta: in entrambe le convenzioni la curva **parte** dalla fase critica e il gain margin aerodinamico e' "quanti dB sopra il punto critico sono a bassa frequenza". Nota onesta: il codice usava la rietichettatura a +180 fino a poco fa ed e' stato allineato alla convenzione del corso; e' una pura rietichettatura dell'asse e **nessun margine cambia** (`allmargin` non guarda il display – verificato: 6.00 dB / 30.0 deg identici prima e dopo, suite di test 35/35). Bonus pratico: la griglia M/N nativa di `ngrid` vive in [-360, 0], quindi le figure disegnate a mano (trade del Task 2, corner del Task 3) ora cascano dentro la griglia.

## Replica manuale dello shift per gli overlay (righe 46-49)

```
wv = logspace(log10(opts.wrange(1)), log10(opts.wrange(2)), 4000);
[mag, ph] = bode(L, wv); mag = squeeze(mag); ph = squeeze(ph);
gdb = 20*log10(mag);
sh = ph + 360*round((-180 - interp1(wv, ph, wref))/360);
```

- Righe 47-48: si ricampiona L su 4000 punti log-spaziati **nella stessa wrange** della Nichols nativa. Serve perche' i marker vanno messi sopra la curva gia' disegnata, e per farlo bisogna sapere dove si trova – ma il `PhaseMatching` di MATLAB e' interno all'oggetto `plot` e non e' interrogabile.



- Riga 49: **si replica a mano lo stesso shift**. `interp1(wv, ph, wref)` da' la fase grezza al riferimento;  $(-180 - \text{quella})/360$  e' di quanti giri va spostata; `round` sceglie il multiplo intero **piu' vicino**, che e' esattamente la regola di **PhaseMatching**. Verificato numericamente: Task 1 -> shift **0** (fase grezza -150, gia' sul ramo giusto), Task 2 -> shift **-720** (fase grezza +570 -> -150). Se questa riga divergesse dalla regola interna di MATLAB, i marker cadrebbero **fuori** dalla curva – ed e' l'unico modo in cui questa funzione puo' mentire.
- Nota sull'unico difetto reale: `interp1` fuori dal range restituisce **NaN**, e `plot` con NaN non disegna nulla **senza errore**. Se un margine classificato cadesse fuori da `wrange` (per esempio gli attraversamenti ad alta frequenza a 129 e 847 rad/s che `allmargin` trova sul loop del Task 2, ben oltre il `wrange = [1e-2 1e2]` usato dai main) il suo marker **sparirebbe in silenzio**. Nei casi dell'homework nessun margine *classificato* cade fuori, ma e' una fragilita' da conoscere.

## Marker dei margini classificati (righe 51-56)

```
hleg = addmark(hleg, mm.aeroGM_w, 'Aero |GM|', 'rs');
hleg = addmark(hleg, mm.rigidPM_w, 'Rigid PM', 'rd');
hleg = addmark(hleg, mm.rigidGM_w, 'Rigid GM', 'r^');
hleg = addmark(hleg, mm.flexGM_w, 'Flex GM', 'ro');
hleg = addmark(hleg, mm.flexPM_w, 'Flex PM', 'rv');
```

- Righe 52-56: un marker per **banda**, con forma diversa: quadrato (Aero GM), rombo (Rigid PM), triangolo su (Rigid GM), cerchio (Flex GM), triangolo giu' (Flex PM). Tutti rossi. I margini **assenti** (NaN) sono saltati da `addmark` (riga 77) e non compaiono in legenda: nel Task 1 restano solo Aero GM e Rigid PM (niente Rigid GM perche' l'attuatore e' ideale, niente Flex perche' non c'e' bending); nel Task 2 compaiono Aero GM, Rigid PM e Rigid GM, ma **non** Flex GM/PM, perche' il notch **gain-stabilizza** il modo e non esiste alcun attraversamento in banda flessibile. **L'assenza dei marker Flex e' quindi informazione, non una dimenticanza**: e' la firma grafica della gain stabilisation.

## Attraversamenti di deriva marcati come non-margini (righe 58-67)

```
for wd = mm.drift_w(:)
    hd = plot(ax, interp1(wv, sh, wd), interp1(wv, gdb, wd), 'kx', ...
             'MarkerSize', 10, 'LineWidth', 1.8, 'HandleVisibility', 'off');
end
set(hd, 'HandleVisibility', 'on', ...
     'DisplayName', 'drift 0 dB crossing (not a margin)');
```

- Righe 59-67: gli attraversamenti a 0 dB del lobo di deriva vengono disegnati con una **croce nera** e una label che dice a chiare lettere *“not a margin”*. E' la scelta editoriale piu' importante della figura: quei punti **ci sono** sulla curva, **allmargin li vede**, `margin()` potrebbe **sceglierli**; segnalarli e dichiararli non-margini e' piu' onesto che nasconderli.
- **Sottigliezza da conoscere** (righe 60-64): dentro il `for`, `hd` viene **sovrascritto** a ogni iterazione (`hd = plot(...)`, non un `append`). Tutti i marker sono creati con `HandleVisibility = 'off'`, e alla fine **solo l'ultimo** viene riacceso e messo in legenda. Il risultato e' quello voluto (tutte le croci disegnate, **una sola** voce di legenda), ma per effetto della sovrascrittura, non per costruzione esplicita: `hleg(end+1) = hd` funziona proprio perche' `hd` e' scalare. Se qualcuno “correggesse” il loop accumulando gli handle, quella riga darebbe errore. Il `hd = gobjects(0)` di riga 60 e' di fatto inutile (il ciclo e' gia' protetto dall'`isempty` di riga 59).

## Legenda (righe 68-72)

```
% Call the built-in legend on the marker handles (avoid the Nichols chart's
% own overloaded legend method, which would treat handles as labels).
legend(hleg, 'Location', 'southwest', 'FontSize', 8, 'Box', 'on');
```

- Righe 69-71: **legend** viene chiamata sugli **handle dei marker**, non sugli assi. Il commento spiega perché: gli assi di un chart Nichols hanno un metodo **legend sovraccaricato** (quello dei **resppack** del Control System Toolbox) che interpreta il primo argomento come **elenco di etichette**, non di handle. Passando gli handle si forza il **legend** built-in di MATLAB. E' un bug scoperto sul campo, ed e' documentato nel codice – bene così.

## Funzione annidata addmark (righe 74-82)

```
function hl = addmark(hl, w, name, style)
    if isnan(w), return; end
    h2 = plot(ax, interp1(wv, sh, w), interp1(wv, gdb, w), style, ...
        'MarkerSize', 10, 'LineWidth', 1.8, ...
        'DisplayName', sprintf('%s (%.2g rad/s)', name, w));
    hl(end+1) = h2;
end
```

- Riga 77: **if isnan(w), return; end** – la guardia che rende inutili tutti i rami **if** nel corpo principale. Un margine NaN (banda senza attraversamento) semplicemente **non viene disegnato**, e per contratto non finisce in legenda.
- Righe 78-80: la posizione del marker viene **interpolata dalla curva ricampionata**, non ricalcolata: così il marker cade **sulla curva disegnata** per costruzione, anche se la griglia **wv** non contiene esattamente la frequenza di attraversamento. La label include la frequenza (**%.2g rad/s**), che e' l'altra meta' dell'informazione: dire “PM = 30 deg” senza dire “a 2.45 rad/s” non identifica il crossover.
- La funzione e' **annidata**, quindi vede **ax, wv, sh, gdb** per chiusura – ecco perché la firma e' così corta.

## Possibili domande d'esame

**D: Cosa fa esattamente il PhaseMatching e perché non e' un imbroglio?** R: Somma a tutta la curva un multiplo intero di 360 deg scelto in modo che, alla frequenza di riferimento (il crossover rigido), la fase risulti il più vicino possibile a -180 deg. Non e' un imbroglio perché la fase di una risposta in frequenza e' definita **modulo 360**: sommare un giro intero non cambia il sistema, cambia solo il ramo su cui lo si legge. Il codice replica lo stesso shift a mano a riga 49 (**sh = ph + 360\*round((-180 - ph(wref))/360)**) per poter piazzare i marker sulla curva già disegnata. Task 1: shift 0 (fase grezza -150 deg, già sul ramo del corso). Task 2: shift -720 (fase grezza +570 -> -150). In entrambi i casi il PM di 30 deg si legge come distanza dal punto critico a -180.

**D: Come si legge la stabilita' condizionale su questa figura?** R: Si guarda **quante volte** la curva attraversa la retta fase = -180 deg e **a che quota**. Sul loop pieno del Task 2 la attraversa **due volte**: a 0.542 rad/s con +6.00 dB (sopra il punto critico) e a 11.11 rad/s con -7.56 dB (sotto). Il punto critico (-180 deg, 0 dB) e' quindi **bracketed** fra i due attraversamenti: ridurre il guadagno di 6 dB fa collassare il primo sul punto critico, aumentarlo di 7.56 dB fa salire il secondo. La banda di guadagni stabili e' **k** in [0.50, 2.39], verificata per forza bruta. Nel Task 1 (attuatore ideale) il secondo attraversamento **non esiste** – la fase tende a -90 deg e non torna mai sulla critica – quindi la conditional stability e' a un solo lato.

**D: Perché la curva “viene dall'alto”?** R: Perché il canale di deriva laterale  $z/\delta$  ha un **integratore libero** (posizione = doppio integrale dell'accelerazione, meno un polo cancellato):  $|L(j\omega)| \rightarrow \infty$  per  $\omega \rightarrow 0$ . Misurato: +47.9 dB a 0.01 rad/s, oltre il limite superiore dell'asse (**YLim [-40 40]**). E' anche il motivo per cui esistono i due attraversamenti a 0 dB a bassa frequenza (0.158 e 0.227 rad/s), che sono **artefatti** del lobo di deriva e vengono marcati con una croce nera e la label “not a margin”.

**D: Nel Task 2 non compaiono i marker Flex GM e Flex PM. E' un bug?** R: No, e' il risultato. Il notch profondo ( $\zeta_N = 0.002$ ,  $\zeta_D = 0.7$ ) **gain-stabilizza** il primo modo di bending: la curva a  $\omega_{BM} = 18.9$  rad/s sta a **-18.2 dB** dopo il re-tune (e' il LwBM\_dB che `classify_margins` calcola con `freqresp(L, w_bending)` alla riga 55, cioe' esattamente il punto della curva), quindi **non c'e' alcun attraversamento** in banda flessibile e `classify_margins` restituisce NaN per entrambi. `addmark` (riga 77) salta i NaN. L'assenza dei marker Flex e' la firma grafica della gain stabilisation: il "margine" del modo non e' un GM/PM ma l'attenuazione, riportata a parte come LwBM\_dB e messa nel titolo della figura da `main_task2.m` riga 199.

**D: Che cos'e' la griglia di sfondo e cosa vuol dire "sfiorare il contorno a 6 dB"?** R: La griglia M/N di Nichols non riguarda il loop aperto: sono i **contorni a modulo costante del ciclo chiuso**  $|T| = |L/(1+L)|$  (le M-circles di Hall trasformate nel piano guadagno-fase) e i contorni a fase costante di T. Il contorno etichettato "6 dB" e' il luogo dei punti in cui il ciclo chiuso ha un picco di 6 dB. Dire che la curva "resta fuori dal contorno a 6 dB" significa che il picco di risonanza del ciclo chiuso e' sotto 6 dB, cioe' un requisito di smorzamento – **un'informazione diversa** dal gain margin, che invece e' una distanza orizzontale/verticale dal punto critico. Confonderli e' l'errore piu' comune sulla carta di Nichols.

# HM3/make\_\_pz\_figures.m

## Ruolo del file nel progetto

E' uno **script di sola visualizzazione**: non progetta niente, non cambia nessun guadagno, non produce numeri che finiscono nelle tabelle del report. Prende gli stessi oggetti costruiti dai `main_task*.m` (plant, attuatore TVC, notch, PD) e li guarda **nel piano s** invece che sulla carta di Nichols. Serve a rispondere a una domanda che la Nichols non risponde in modo diretto: *dove stanno fisicamente i poli, e dove li porta la retroazione?*

La scelta di progetto di HM3 e' interamente in frequenza (Nichols, margini classificati per banda). La mappa poli-zeri e' il **controllo incrociato**: se la Nichols dice "stabile con 6 dB e 30 gradi", la mappa deve mostrare tutti i poli di anello chiuso nel semipiano sinistro. Le quattro figure prodotte sono, nell'ordine dei banner %:

1. tre **luoghi delle radici** (`intro_rootlocus`, `task2_rootlocus`, `task2_rootlocus_full`) – la stabilita' condizionata;
2. `task1_polemap` – poli ad anello aperto vs. anello chiuso del loop rigido;
3. `task2_notch_poles` – la coppia del bending con e senza notch;
4. `task3_pole_migration` – migrazione dei poli sui vertici del box +/-30%.

Dipende da: `load_hw3_params`, `build_plant_rigid`, `build_plant_full`, `build_tvc`, `build_notch_filter`, `design_controller`, `assemble_loop`. Scrive PNG in `HM3/figures/` (che poi il report LaTeX pesca via `\graphicspath`). Non e' chiamato da nessun altro script: si lancia a mano.

**Attenzione, e' il punto piu' importante di questa pagina.** Le figure 2, 3 e 4 **non usano gli stessi guadagni**. La figura 2 usa il PD del Task 1 ( $K_p = 1.7845$ ,  $K_d = 0.4433$ , riga 40). Le figure 3 e 4 usano il PD **ri-sintonizzato** del Task 2 ( $K_p = 1.7318$ ,  $K_d = 0.6867$ , righe 55-56). Verificato eseguendo il codice. Vedi sotto per le conseguenze.

---

## Intestazione, stile grafico, cartella di uscita (righe 1-35)

- Righe 1-16: commento di testa che dichiara le quattro figure. E' onesto e corretto. Le righe 6-7 promettono *"the +sqrt(A6) open-loop pole pulled into the LHP by the rigid PD"*, e la figura 2 **lo mostra davvero** – ma solo dopo l'allargamento del `ylin` di riga 73. Fino a poco fa la finestra era troppo stretta e la promessa restava sulla carta: la storia e' raccontata per esteso nella sezione righe 67-79, e vale la pena saperla.
- Riga 18: `clear; close all; clc;` – script, non funzione: gira in base workspace.
- Riga 19: `warning('off','Control:analysis:MarginUnstable')`. Necessario perche' l'anello e' aperto-instabile: ogni chiamata a `margin/allmargin` dentro `design_controller` emette un warning. Qui viene solo silenziato, non aggirato.
- Righe 21-23: `here = fileparts(mfilename('fullpath'))` – lo script e' eseguibile da qualunque cartella corrente, le figure vanno sempre in `HM3/figures/`.
- Righe 26-35: palette. `cUnst` (rosso) = instabile / anello aperto / senza notch; `cStab` (blu) = stabile / anello chiuso / con notch; `cRHP` = ombreggiatura del semipiano destro; `cCorn` = 5 colori per Nominale + V1..V4.

## Punto di progetto condiviso: quali guadagni entrano nelle figure (righe 37-41)

```
p0      = load_hw3_params();
Grigid  = build_plant_rigid(p0);
K       = design_controller(Grigid, [], 'verbose', false);
zN = 0.002; zD = 0.7; sgn = +1; % deep min-phase notch
```

- Riga 38: `load_hw3_params()` senza argomenti -> caso nominale ( $\mu_{\alpha\_scale} = \mu_{c\_scale} = 1$ ), istante di riferimento  $t_{ref} = 72$  s (max-qbar). I coefficienti arrivano dal dataset LPV, non dai letterali della Tabella 1: eseguendo il codice si ottiene  $A6 = 3.381828$ ,  $K1 = 4.564724$ ,  $a1 = -0.015423$ ,  $a3 = 20.6090$ ,  $a4 = -27.2710$ ,  $V = 937.709$  m/s,  $w_{BM} = 18.9$  rad/s,  $z_{BM} = 0.005$ ,  $q_{bar} = 81129$  Pa,  $Alt = 15143$  m.
- Riga 40:  $K$  e' il PD del **Task 1**, ritarato da `design_controller` sul loop rigido con attuatore ideale. Eseguito:  $K_{p\_th} = 1.7845$ ,  $K_{d\_th} = 0.4433$ ,  $K_{p\_z} = K_{d\_z} = -1e-3$ .
- Riga 41: parametri del notch.  $zN = 0.002$  (numeratore, quasi senza smorzamento -> notch profondo),  $zD = 0.7$  (denominatore, ben smorzato),  $sgn = +1$  -> **notch a fase minima** (zeri nel semipiano sinistro). Il default di `build_notch_filter` sarebbe  $-1$  (Eq. 4 come stampata nella traccia, zeri nel semipiano destro): qui viene **deliberatamente scartato**. Profondita' analitica del notch alla frequenza centrale:

$$|H_x(j\omega_{BM})| = zN/zD = 0.002/0.7 = 0.002857 \rightarrow -50.9 \text{ dB}$$

(verificato numericamente: -50.88 dB).

---

## Anatomia del piano s: da dove viene ogni polo e ogni zero (righe 43-58)

Queste righe costruiscono tutto cio' che poi si vede nelle figure: TVC, plant flessibile, notch, PD ri-tarato, e i due anelli aperti  $L_{rig}$  e  $L_{ful}$ .

```
Wtvc    = build_tvc(p0,3);
Gfull   = build_plant_full(p0,'ins');
notch    = build_notch_filter(p0.wBM, zN, zD, sgn);
K2 = design_controller(Gfull, Wtvc*notch, 'w_flex',0.6*p0.wBM, ...
                       'w_flex_hi',1.5*p0.wBM, 'w_bending',p0.wBM, ...
                       'verbose',false);
[Lrig,~] = assemble_loop(Grigid, K, []);
[Lful,~] = assemble_loop(Gfull, K2, Wtvc*notch);
```

- Riga 51: `build_tvc(p0,3)` – attuatore del secondo ordine ( $w_{TVC} = 70$  rad/s,  $z_{TVC} = 0.7$ ) **in serie** con l'approssimante di Pade di ordine 3 del ritardo puro  $\tau = 20$  ms.
- Riga 52: `build_plant_full(p0,'ins')` – plant a 6 stati con la contaminazione INS della misura (Eq. 2):  $\theta_m = \theta + \sigma_{ins}\eta$ ,  $z_m = z - \phi_{ins}\eta$ . E' proprio questo canale che destabilizza il modo.
- Righe 55-56:  $K2$  e' il PD **ri-sintonizzato sull'anello completo** (plant flessibile + TVC + ritardo + notch). Eseguito:  $K_{p\_th} = 1.7318$ ,  $K_{d\_th} = 0.6867$ . Le opzioni  $w_{flex} = 0.6*w_{BM} = 11.34$ ,  $w_{flex\_hi} = 1.5*w_{BM} = 28.35$ ,  $w_{bending} = w_{BM}$  servono a `classify_margins` per capire quali crossing sono “rigidi” e quali “flessibili”.
- Riga 57-58: i due anelli aperti (convenzione  $1 + L$ , rottura sul segnale  $\delta$ ).

## Tabella dei poli/zeri (tutti verificati eseguendo il codice)

| oggetto                      | poli                         | zeri                      | origine fisica                         |
|------------------------------|------------------------------|---------------------------|----------------------------------------|
| Grigid (4 stati)             | 0, +0.0291, +1.8165, -1.8610 | (verso theta) -0.0317     | rigido + deriva laterale               |
| bending in Gfull             | -0.0945 +/- 18.8998i         | -                         | wBM=18.9, zBM=0.005                    |
| attuatore TVC                | -49.00 +/- 49.99i            | -                         | wTVC=70, zTVC=0.7                      |
| Pade(tau=0.02, ord. 3)       | -232.2, -183.9 +/- 175.4i    | +232.2, +183.9 +/- 175.4i | ritardo puro -> <b>fase non minima</b> |
| notch                        | -13.23 +/- 13.50i            | -0.0378 +/- 18.9i         | zD=0.7 / zN=0.002                      |
| coppia INS/bending (in Lful) | -                            | +15.26, -15.08            | sigma_ins * phi_tvc * Tc               |

## 1. Il polo instabile: da dove esce +sqrt(A6)

La dinamica rotazionale disaccoppiata (traccia, Eq. 1, riga 4 della matrice di stato) e'

$$\theta_{ddot} = A_6 \theta + (A_6/V) \dot{z} + K_1 \delta - A_6 \alpha_w$$

Se si ignora l'accoppiamento con la deriva ( $\dot{z} \sim 0$ ) e il vento, resta

$$\theta_{ddot} = A_6 \theta + K_1 \delta \quad \rightarrow \quad G(s) = K_1 / (s^2 - A_6)$$

cioe' **due poli reali** in  $s = \pm \sqrt{A_6} = \pm 1.83897 \text{ rad/s}$ . Il segno + di  $A_6$  e' il cuore del problema:  $A_6 = N_{\alpha} l_{\alpha} / I_{yy}$  con  $l_{\alpha} > 0$  = distanza fra centro di pressione e centro di massa, con il **CP davanti al CM**. Un'incidenza positiva genera un momento che *aumenta* l'incidenza -> divergenza aerodinamica. Verifica dei numeri di Tabella 1:  $1.07e6 * 10.39 / 3.28e6 = 3.389 \sim A_6 = 3.3818$ .

Il tempo di raddoppio dell'errore d'assetto e'

$$t_2 = \ln(2) / \sqrt{A_6} = 0.693 / 1.839 = 0.377 \text{ s}$$

Ecco perche' il controllo e' obbligatorio: senza retroazione il lanciatore raddoppia l'errore d'assetto ogni 0.38 s.

## 2. Ma nella mappa il polo NON e' a +1.839: e' a +1.8165 (e ce n'e' un secondo)

E' un dettaglio che vale un punto all'orale. Il plant rigido **completo** include la deriva laterale, e la sua equazione caratteristica non e'  $s^2 - A_6$ . Sviluppando il determinante della A di `build_plant_rigid` (lo stato  $z$  non retroagisce su nessuno, quindi da' un polo in 0; restano  $\dot{z}$ ,  $\theta$ ,  $\dot{\theta}$ ):

$$s[(s - a_1)(s^2 - A_6) - (a_1 V + a_4) A_6 / V] = 0$$

Con i valori nominali  $a_1 V + a_4 = -14.46 - 27.27 = -41.73$ , il termine di accoppiamento vale +0.1505 e il polinomio cubico diventa

$$s^3 + 0.0154 s^2 - 3.3818 s + 0.0984 = 0$$

le cui radici sono +1.8165, -1.8610, +0.0291. Quindi:

- il polo aerodinamico **si sposta** da +1.839 a +1.8165 (la deriva lo addolcisce di circa l'1%);
- compare un **secondo polo instabile lentissimo** in +0.0291 rad/s (costante di tempo 34 s), il modo di **deriva**: il veicolo, se non controllato, oltre a capottare scivola lateralmente sottovento;
- nel canale  $\delta \rightarrow \theta$  questo polo e' quasi cancellato da uno zero in -0.0317 (dipolo lento). *Quasi*, non del tutto: il polo e' a destra, lo zero a sinistra. E' un dipolo di quelli che non si possono ignorare, perche' contiene il segno.

Quindi il plant rigido ha **due poli RHP**, non uno. Il commento del codice e il README parlano solo di **+sqrt(A6)**: e' la lettura disaccoppiata, comoda per il progetto, ma la mappa disegnata dallo script mostra la verita' completa.

**Possibile domanda d'esame** – Nella mappa vedo due croci a destra dell'asse immaginario. La seconda e' un errore numerico? *Risposta*: No. La prima e' il polo aerodinamico (+1.82, ~+sqrt(A6) perturbato dall'accoppiamento con la deriva). La seconda, in +0.029, e' il modo di deriva laterale:

nasce dai termini  $a_1$  e  $a_4$  della riga di  $z_{ddot}$  combinati con  $A_6/V$ . E' instabile ma lentissimo ( $\tau = 34$  s), e nel solo canale  $\delta$   $\rightarrow \theta$  e' quasi cancellato da uno zero in  $-0.032$ . La retroazione debole sulla deriva ( $K_{p_z} = K_{d_z} = -1e-3$ ) serve proprio a gestirlo.

### 3. La coppia del bending: perche' e' praticamente sull'asse

Il primo modo flessionale entra come oscillatore del secondo ordine  $s^2 + 2*z_{BM}*w_{BM}*s + w_{BM}^2$ , con  $w_{BM} = 18.9$  rad/s e  $z_{BM} = 0.005$ . I poli:

$$s = -z_{BM}*w_{BM} \pm i*w_{BM}*sqrt(1 - z_{BM}^2) = -0.0945 \pm i*18.8998i$$

La parte reale e' **-0.09**, contro una parte immaginaria di **18.9**: nella scala della figura sono di fatto *sull'asse immaginario*. Fisicamente: lo smorzamento strutturale di un serbatoio in alluminio riempito di propellente e' dello 0.5%, quindi il modo, se eccitato, suona per  $\tau = 1/(z_{BM}*w_{BM}) = 10.6$  s. Una struttura di questo tipo non si "smorza da sola" nei tempi del volo: **o la si evita, o la si spegne attivamente**.

### 4. Gli zeri a fase non minima: due famiglie diverse (attenzione!)

(a) **Pade**. Il ritardo  $\exp(-\tau*s)$  non e' razionale;  $pade(0.02, 3)$  lo approssima con una funzione **tutta-passa**: gli zeri sono l'immagine speculare dei poli rispetto all'asse immaginario. Con l'approssimante  $[3/3]$  il polinomio (in  $x = \tau*s$ ) e'  $x^3 - 12x^2 + 60x - 120$ , con radici  $x = 4.644$  e  $x = 3.678 \pm i*3.509i$ ; dividendo per  $\tau = 0.02$ :

```
zeri : +232.2 , +183.9 +/- 175.4i (RHP, fase non minima)
poli : -232.2 , -183.9 +/- 175.4i
```

E' esattamente cio' che dichiara il commento di riga 65 ("+232 and +184+/-175i"), verificato. Il modulo e' unitario a ogni frequenza (all-pass): il ritardo **non toglie guadagno, toglie solo fase** - circa  $-\tau*w = -0.02*3.2 = -0.064$  rad =  $-3.7$  gradi al crossover rigido, ma  $-0.02*18.9 = -21.6$  gradi alla frequenza del bending. E' il motivo per cui la fase precipita ad alta frequenza e per cui il *delay margin* (213 ms nel Task 1, 165 ms nel Task 2) e' una metrica riportata separatamente.

(b) **La coppia +/-15 dalla contaminazione INS**. Questa il commento del codice la *non* menziona correttamente (riga 63 parla solo di "the -2.5/-15 real zeros"), ma eseguendo `zero(Lful)` compare uno **zero reale nel semipiano destro a +15.26** (e il suo compagno a -15.08). Da dove viene? La misura e'  $\theta_m = \theta + \sigma_{ins}*\eta$ , e il TVC forza il bending con  $-\phi_{tvc}*T_c$ . Trascurando la deriva:

$$\begin{aligned} \theta/\delta &\sim K_1/(s^2 - A_6) \\ \eta/\delta &= -c/(s^2 + 2*z_{BM}*w_{BM}*s + w_{BM}^2), \quad c = \phi_{tvc}*T_c = 65.6 \end{aligned}$$

quindi il numeratore di  $\theta_m/\delta$  e'

$$(K_1 - \sigma_{ins}*c)*s^2 + K_1*2*z_{BM}*w_{BM}*s + (K_1*w_{BM}^2 + \sigma_{ins}*c*A_6)$$

Il coefficiente di  $s^2$  vale  $4.5647 - 0.178*65.6 = 4.565 - 11.67 = -7.11$ : **e' negativo**. Il segno del termine dominante si ribalta, e il numeratore ha due radici reali quasi simmetriche rispetto all'asse:

$$s \sim \pm \sqrt{1670/7.11} = \pm 15.3 \text{ rad/s}$$

(numericamente +15.26 e -15.08). Significato fisico: la deflessione strutturale vista dall'INS e' **in controfase** rispetto alla risposta rigida, e in modo cosi' forte da dominarla. E la conseguenza per il progetto e' decisiva: gli zeri stanno **sotto**  $w_{BM}$  ( $15.3 < 18.9$ ). Quando gli zeri precedono il polo del modo, il lobo di bending sulla Nichols entra con la fase "sbagliata" e **non e' stabilizzabile in fase**: l'unica strada e' **stabilizzarlo in guadagno**, cioe' schiacciarlo sotto 0 dB con il notch. Questo e' il motivo strutturale, non estetico, per cui il Task 2 usa un notch e non un lead-lag.

**Possibile domanda d'esame** - *Perche' non stabilizzi il bending in fase (phase stabilization) invece di usare un notch profondo?* Risposta: Perche' la contaminazione INS produce una coppia di zeri reali a  $\pm 15.3$  rad/s, cioe' **sotto**  $w_{BM} = 18.9$  e uno dei due nel semipiano destro. Con zeri sotto il polo, il lobo di bending attraversa la Nichols dalla parte sbagliata e nessuna rotazione di fase realizzabile lo porta fuori dal punto critico. La stabilizzazione in fase e' possibile solo quando il sensore vede il modo con la fase "giusta" (zeri sopra il polo); qui non e' il caso, quindi si va per forza in gain stabilization.

---

## (1) I tre luoghi delle radici (righe 60-65)

- Righe 60-61: root locus del **loop rigido del Task 1** (attuatore ideale, guadagni K). Finestra [-10, 2.5] x [-4, 4].
- Righe 62-63: root locus del **loop completo del Task 2**, zoom vicino all'origine, [-16, 3] x [-3, 3].
- Righe 64-65: lo stesso loop, vista completa [-250, 250] x [-200, 200], per far vedere i rami che corrono verso gli zeri RHP di Pade.

rlocusplot(L) traccia le radici di  $1 + kL(s) = 0$  al variare di k da 0 a infinito. **Il punto di progetto e' k = 1**: il luogo dice quanto sono "lontani" da instabilita' i poli se il guadagno d'anello viene scalato.

Lrig in forma zeri-poli-guadagno (verificato):

```
L_rig(s) = 2.003*(s + 4.032)*(s^2 + 0.0562*s + 0.03222)
-----
          s*(s + 1.861)*(s - 1.816)*(s - 0.02909)
```

Note utili all'orale: - il guadagno di alta frequenza 2.003 e' essenzialmente  $K_d * K_1 = 0.4433 * 4.5647 = 2.024$ : a frequenza alta il ramo derivativo domina il PD; - lo zero in -4.032 e' il rapporto  $K_p/K_d = 1.7845/0.4433 = 4.03$  - e' lo zero del PD, non del plant. Il commento di riga 61 lo chiama giustamente "the -4.03 zero"; - l'anello ha **4 poli e 3 zeri** -> un solo asintoto (a 180 gradi, verso -inf); - il polo in 0 c'e' perche' la deriva e' retroazionata ( $K_{p_z * z_m}$ ): senza la retroazione di deriva quel polo non comparirebbe in L.

**Stabilita' condizionata.** Eseguendo `classify_margins(Lrig)` si ottiene `aeroGM = -6.00 dB @ 0.593 rad/s` (margine di **riduzione** di guadagno) e nessun margine di aumento (`rigidGM = NaN`, l'attuatore e' ideale). Tradotto:

$k_{min} = 10^{(-6/20)} = 0.501$  (nessun  $k_{max}$  nel Task 1)

Il sistema e' stabile **solo se il guadagno d'anello supera il 50% del nominale**. E' l'opposto dell'intuizione da corso base ("abbassa il guadagno e sei al sicuro"): con un plant aperto-instabile serve *guadagno minimo*, perche' sotto quella soglia il ramo che parte dal polo +1.8165 non ha ancora attraversato l'asse immaginario. E' proprio quello che si vede nel luogo: per k piccolo i poli chiusi coincidono con quelli aperti (due a destra), e solo crescendo k migrano a sinistra.

Nel loop completo (Lful) si aggiunge il **limite superiore**: il ritardo e l'attuatore fanno tornare la fase a -180 gradi ad alta frequenza, quindi k troppo grande destabilizza. Da qui il "rigid GM" (7.56 dB nel Task 2) - la banda di guadagno ammissibile e' *chiusa*, ed e' la firma della stabilita' condizionata sul lanciatore. Lful ha 13 poli e 10 zeri -> 3 asintoti, e i rami "lontani" corrono verso gli zeri RHP di Pade (+232, +184 +/- 175i): per questo la figura di riga 64-65 esiste, altrimenti si vedrebbero solo rami che escono dal quadro.

**Possibile domanda d'esame** - *Che differenza c'e' fra il margine di guadagno "aerodinamico" e quello "rigido"? Risposta:* Sono due attraversamenti diversi della fase -180 gradi dello stesso anello. Quello aerodinamico e' a bassa frequenza (0.59 rad/s) ed e' un margine di **riduzione**: se il guadagno cala di 6 dB il ramo del polo instabile torna a destra. Quello rigido e' al crossover di controllo ed e' un margine di **aumento**: se il guadagno cresce, il ritardo TVC porta la fase a -180 e si destabilizza dall'altra parte. Un singolo numero di `margin()` non ha senso qui: il codice li separa in `classify_margins.m`.

---

## (2) Task 1: mappa dei poli in anello chiuso (righe 67-79)

```
[~,T1] = assemble_loop(Grigid, K, []);
pOL = pole(Grigid); pCL = pole(T1);
...
x1 = [-6 2]; y1 = [-2.5 2.5]; % tall enough for the PD-placed pair at -0.95 +/- 1.90i
```



- Riga 68: `assemble_loop(Grigid, K, []) - []` = attuatore ideale ( $w_{act} = 1$ ), cioè il Task 1 puro.
- Riga 69: si estraggono i poli aperti (croci rosse) e chiusi (cerchi blu).
- Righe 74-76: `shade_rhp` ombreggia il semipiano destro, poi le due serie.
- Riga 73: **i limiti degli assi.** `y1 = [-2.5, 2.5]` e' abbastanza alto da contenere la coppia dominante di anello chiuso a  $-0.95 \pm 1.90i$ , che e' **il soggetto stesso della figura**. Su questa riga c'era un difetto, corretto di recente: vedi il riquadro in fondo alla sezione.

I poli, calcolati eseguendo il codice:

|                 | poli                                       |
|-----------------|--------------------------------------------|
| aperti (Grigid) | 0, +0.0291, +1.8165, -1.8610 (tutti reali) |
| chiusi (T1)     | -0.9533 +/- 1.9047i e -0.0559 +/- 0.2329i  |

La coppia dominante  $-0.9533 \pm 1.9047i$  ha  $w_n = 2.13$  rad/s,  $\zeta = 0.448$ ; la coppia lenta  $-0.0559 \pm 0.2329i$  ha  $w_n = 0.239$  rad/s,  $\zeta = 0.234$ ,  $\tau = 17.9$  s (e' il modo di deriva stabilizzato – lo stesso che giustifica l'orizzonte di 80 s scelto in `main_task1.m`).

**Il legame con il progetto analitico.** Sulla dinamica rotazionale disaccoppiata, chiudere con  $\delta = -(K_p + K_d s) \theta$  da'

$$s^2 + K_d K_1 s + (K_p K_1 - A_6) = 0$$

da cui, immediatamente,

$$\begin{aligned} w_n &= \sqrt{K_p K_1 - A_6} = \sqrt{1.7845 \cdot 4.5647 - 3.3818} = 2.183 \text{ rad/s} \\ \zeta &= K_d K_1 / (2 w_n) = 0.4433 \cdot 4.5647 / (2 \cdot 2.183) = 0.464 \end{aligned}$$

che sono esattamente i valori ( $w_c = 2.18$ ,  $\zeta = 0.46$ ) riportati nel README. I poli **veri** dell'anello chiuso completo sono  $2.13 / 0.448$ : la differenza (~2%) e' l'effetto della retroazione di deriva, che il calcolo analitico ignora. Vale la pena saperlo: se all'orale si chiede "coincidono?", la risposta corretta e' "quasi, e la discrepanza e' il prezzo della retroazione di deriva".

Nota sui **semi** usati da `design_controller` (D'Antuono Eq. 3.6-3.7):  $K_{p0} = 2 A_6 / K_1 = 1.4817$ ,  $K_{d0} = \sqrt{A_6} / K_1 = 0.4029$ . Sostituendo nelle formule sopra:  $w_n = \sqrt{2 A_6 - A_6} = \sqrt{A_6} = 1.839$  e  $\zeta = \sqrt{A_6} / (2 \sqrt{A_6}) = 0.5$  esatti. Cioe' i semi analitici sono la scelta "specchia il polo instabile a sinistra con  $\zeta = 0.5$ ". Poi `fminsearch` li sposta a  $1.7845 / 0.4433$  per centrare i target 6 dB / 30 gradi *sull'anello completo*.

**Difetto corretto, ma da saper raccontare.** Oggi la riga 73 impone `y1im = [-2.5, 2.5]` e la figura mostra **tutti e quattro** i poli di anello chiuso – la coppia veloce piazzata dal PD a  $-0.95 \pm 1.90i$  e la coppia lenta di deriva a  $-0.06 \pm 0.23i$  – insieme al polo instabile ad anello aperto a  $+1.82$  dentro la fascia RHP ombreggiata. E' esattamente cio' che il titolo promette.

**Fino a poco fa non era cosi'.** La riga 73 imponeva `y1im = [-0.28, 0.28]`, cioe' una finestra alta poco piu' di un quarto di rad/s: dentro ci stava **solo** la coppia lenta a  $\pm 0.233i$ . La coppia dominante, a  $\pm 1.905i$ , cadeva **fuori quadro**. La figura intitolata "*closed-loop pole placement*" tagliava fuori proprio i poli che il PD piazza: si vedeva il problema (il polo a  $+1.82$ ) ma non la soluzione (la sua immagine stabilizzata). Il difetto era tanto piu' insidioso perche' la figura *sembrava* corretta – croci rosse, cerchi blu, tutto al suo posto – e solo confrontando i numeri con `pole(T1)` si scopriva che meta' dei cerchi mancava.

**Perche' [-0.28, 0.28] era la scelta sbagliata anche in linea di principio.** Una mappa di *pole placement* deve essere scalata sulla coppia **piazzata**, non su quella residua. La coppia veloce ha  $w_n = 2.13$  rad/s: il riquadro deve contenere almeno  $\pm w_n$  in immaginario, altrimenti si sta inquadrando la dinamica che il progetto **non** controlla (il modo di deriva) e si nasconde quella che controlla. Il difetto e' il tipo di errore che passa inosservato in una figura che "gira" senza errori – e per questo e' una bella domanda d'orale: *come fai a sapere che la tua figura mostra cio' che dici che mostra?* Risposta: la si valida contro i numeri, non contro l'occhio.

### (3) Task 2: la coppia del bending con e senza notch (righe 81-96)

```
[~,Tno] = assemble_loop(Gfull, K2, Wtvc);      % senza notch
[~,Tnf] = assemble_loop(Gfull, K2, Wtvc*notch); % con notch
```

- Riga 82: anello chiuso **senza** notch (solo TVC + ritardo).
- Riga 83: anello chiuso **con** il notch profondo.
- Righe 88: finestra [-30, 25] x [-45, 45].
- Righe 90-91: yline a wBM per ancorare visivamente la frequenza del modo.

Risultati (verificati eseguendo il codice):

|             | poli in zona bending              | stabile?           |
|-------------|-----------------------------------|--------------------|
| senza notch | +2.7582 +/- 16.3135i – <b>RHP</b> | no (isstable = 0)  |
| con notch   | -0.1023 +/- 18.9191i              | si' (isstable = 1) |

Lettura fisica: - **Senza notch**, la coppia flessibile (aperta a  $-0.0945 \pm 18.90i$ , cioè a 0.09 dalla stabilità) viene **spinta a destra dall'anello**: il guadagno d'anello a wBM e' positivo (+29 dB con i guadagni del Task 1), la fase e' quella sbagliata per via degli zeri INS a  $\pm 15$ , e la retroazione **pompa energia nel modo** invece di sottrarla. E' la classica instabilità di interazione controllo-struttura: il TVC eccita la flessione, l'INS la legge come assetto, il controllore reagisce alla flessione e la rinforza. - **Con notch**, la coppia torna a  $-0.1023 \pm 18.9191i$ , cioè **praticamente dove stava ad anello aperto** ( $-0.0945 \pm 18.90i$ ). E' *esattamente* il significato di *gain stabilization*: il notch non smorza il modo, gli toglie il guadagno d'anello ( $|L(wBM)| = -18$  dB) così che la retroazione **non lo veda**. Lo smorzamento di anello chiuso resta  $\zeta = 0.1023/18.92 = 0.0054$ , contro lo 0.005 strutturale. - Compare inoltre una coppia nuova,  $-7.7342 \pm 10.8265i$ : sono i **poli del notch stesso** ( $-13.23 \pm 13.50i$  ad anello aperto), trascinati dalla retroazione. Ben smorzati, innocui.

**Limite onesto della figura.** La comparazione “senza notch” e' fatta con K2, cioè i guadagni già **ri-sintonizzati sul loop col notch** (il commento di riga 82 lo ammette). Nel README/report, invece, la riga di tabella “no filter,  $|L(wBM)| = +29$  dB” e' valutata con i guadagni del **Task 1**. Non e' quindi la stessa configurazione: la figura risponde alla domanda “il PD ri-tarato da solo basterebbe a domare il bending?” (no), non alla domanda “cosa succede al progetto del Task 1 se aggiungo il modo flessibile?”. La conclusione qualitativa (instabile senza notch) non cambia, ma le due cifre non sono confrontabili.

**Limite onesto della finestra.** Con xlim = [-30, 25], i poli dell'attuatore TVC ( $-49 \pm 50i$ ) e quelli di Pade ( $-232, -184 \pm 175i$ ) sono **fuori quadro**. La figura mostra il cluster rigido/bending/notch, non l'insieme completo dei 13 poli. Il titolo dice “bending poles”, quindi e' legittimo, ma non si puo' dire “la figura mostra che tutti i poli sono stabili”.

**Possibile domanda d'esame** – *Se il notch riporta i poli del bending praticamente dove stavano ad anello aperto, a che serve? Il modo resta smorzato allo 0.5%. Risposta:* E' proprio l'obiettivo. Il modo flessibile e' una proprieta' della struttura: il controllo d'assetto non ha ne' l'autorita' ne' la banda per smorzarlo (servirebbe un attuatore molto piu' veloce e un sensore pulito). Cio' che si deve garantire e' che il loop **non lo destabilizzi**. Senza notch il loop lo porta a  $+2.76 \pm 16.3i$ , instabile; con il notch lo lascia a  $-0.10 \pm 18.9i$ , cioè esattamente com'e' in natura. La riserva di sicurezza si legge in guadagno ( $|L(wBM)| = -18$  dB, sopra i 12 dB tipicamente richiesti), non in smorzamento.

### (4) Task 3: migrazione dei poli sul box +/-30% (righe 98-117)

```
cases = {'Nominal',1.00,1.00; 'V1',0.70,0.70; 'V2',0.70,1.30; ...
        'V3',1.30,0.70; 'V4',1.30,1.30};
```

```
...
p = load_hw3_params('mu_alpha_scale',cases{i,2},'mu_c_scale',cases{i,3});
Gf = build_plant_full(p,'ins');
Wf = build_tvc(p,3) * build_notch_filter(p0.wBM, zN, zD, sgn);
[~,T] = assemble_loop(Gf, K2, Wf);
```

- Righe 99-100: i 4 vertici del box di incertezza su  $\mu_{\alpha} = A_6$  (piu' o meno instabile) e  $\mu_c = K_1$  (piu' o meno autorita' di controllo), piu' il nominale.
- Riga 109: **solo**  $A_6$  e  $K_1$  vengono scalati (lo fa `load_hw3_params` alle righe 89-90).
- Riga 111: il notch e' costruito su `p0.wBM` – “frozen @ nominal”. Il commento e' corretto in spirito, ma **in Task 3 e' ridondante**: `wBM`, `wTVC`, `zTVC`, `tau` non sono fra i parametri scalati, quindi `build_tvc(p,3)` e `build_notch_filter(p.wBM,...)` darebbero comunque gli stessi identici sistemi. La distinzione conta davvero solo in `main_montecarlo.m`, dove `wBM` viene disperso.
- Riga 112: il controllore **non cambia mai** ( $K_2$  fisso). E' la definizione stessa dello studio di robustezza: guadagni congelati, plant che si muove.

Massima parte reale dei poli di anello chiuso (verificata eseguendo il codice):

| caso      | $\mu_{\alpha}$ | $\mu_c$     | max Re(polo) | stabile |
|-----------|----------------|-------------|--------------|---------|
| Nominale  | 1.00           | 1.00        | -0.0497      | si'     |
| V1        | 0.70           | 0.70        | -0.0502      | si'     |
| V2        | 0.70           | 1.30        | -0.0297      | si'     |
| <b>V3</b> | <b>1.30</b>    | <b>0.70</b> | -0.1023      | si'     |
| V4        | 1.30           | 1.30        | -0.0494      | si'     |

Tutti nel semipiano sinistro -> la conclusione del commento di riga 12 (“all poles stay in the LHP -> robust”) e' **corretta**.

**Trappola concettuale, da preparare.** Il polo piu' vicino all'asse **non e' mai** quello aerodinamico ne' quello del bending: e' il **modo lento di deriva** ( $\text{Re} \sim -0.03 \dots -0.10$ ). Di conseguenza la mappa **non ordina i casi come li ordinano i margini**: V3, che il Task 3 identifica come il caso peggiore (aero GM = 0.91 dB, PM = 18 gradi), ha in realta' la parte reale **piu' negativa** di tutte (-0.1023), mentre V2 – un caso comodo – e' quello con il polo piu' vicino all'asse (-0.0297). Non e' una contraddizione: la parte reale misura la *velocita' di decadimento*, il margine di guadagno misura *quanto si puo' sbagliare il guadagno prima di perdere la stabilita'*. Sono robustezze diverse. La mappa poli-zeri **non e' in grado di vedere** che V3 e' a 0.91 dB dal disastro. Per quello serve la Nichols. Se all'orale si presenta questa figura come “prova di robustezza”, bisogna sapere che e' una prova di *stabilita' nominale ai vertici*, non di *robustezza dei margini*.

## Export (righe 119-128)

- Righe 120-127: per ogni figura si forza il tema chiaro (`theme(f,'light')`, in `try/catch` perche' `theme` esiste solo da R2025a in poi) e si esporta a 220 DPI con `exportgraphics`.
- Riga 126: il nome del PNG e' `get(f,'Name')` – quindi i nomi dei file (`task1_polemap.png`, `task2_notch_poles.png`, `task3_pole_migration.png`) sono decisi dalla proprieta' `Name` delle figure alle righe 71, 86, 103. Le tre figure di root locus vengono invece esportate dentro `rl_fig`.

## Helper (righe 131-168)

- `shade_rhp` (righe 131-137): disegna una `patch` sul semipiano destro con `FaceAlpha = 0.08` e `HandleVisibility='off'` (non entra in legenda). C'e' la guardia `if x1(2) > 0`: se la finestra e' tutta a sinistra dell'asse non disegna nulla.

- **finish\_ax** (righe 139-149): stile comune (font, griglia, assi, etichette in  $s^{-1}$ ). Nota che l'unità è corretta: le parti reale e immaginaria di un polo sono frequenze, in  $\text{rad/s} = s^{-1}$ .
- **rl\_fig** (righe 151-168): wrapper attorno a **rlocusplot** – **non ridisegna nulla a mano**, usa il rendering nativo del Control System Toolbox e ne cambia solo assi, titolo e font via **setoptions**. Scelta deliberata (dichiarata nel commento): le convenzioni di croce/cerchio/direzione dei rami restano quelle standard MATLAB, così la figura è leggibile da chiunque conosca il tool. Le tre figure di root locus **non** ricevono l'ombreggiatura RHP.

## Possibili domande d'esame

**D: Cosa aggiunge la mappa poli-zeri rispetto alla carta di Nichols, se il progetto è fatto tutto in frequenza?** R: Tre cose che la Nichols non dice esplicitamente. (1) *Dove* stanno i poli instabili e quanto sono veloci:  $+1.84 \text{ rad/s}$  significa raddoppio in 0.38 s, e questo fissa la banda minima del controllo. (2) *Dove* finisce ogni polo dopo la chiusura: la Nichols garantisce la stabilità ma non dice se il modo di deriva si è fermato a  $\tau = 18 \text{ s}$  (informazione che serve per scegliere l'orizzonte di simulazione). (3) La distinzione fra gain e phase stabilization del bending: sulla mappa è evidente che il notch **riporta** i poli flessibili dove stavano, mentre non li smorza. Di contro, la mappa non vede i margini di guadagno: è il motivo per cui le due viste sono complementari e non alternative.

**D: Con quali guadagni sono state generate queste figure?** R: Dipende dalla figura, ed è importante saperlo. La figura 2 (**task1\_polemap**) e il primo root locus usano il PD del Task 1,  $K_p = 1.7845$ ,  $K_d = 0.4433$ , progettato su plant rigido con attuatore ideale. Le figure 3 (**task2\_notch\_poles**), 4 (**task3\_pole\_migration**) e i due root locus del loop completo usano il PD **ri-sintonizzato** del Task 2 sul loop con TVC + ritardo + notch,  $K_p = 1.7318$ ,  $K_d = 0.6867$ . In tutti i casi i guadagni di deriva sono fissi a  $K_{p_z} = K_{d_z} = -1e-3$ .

**D: Perché il PD ri-tarato ha un  $K_d$  molto più alto (0.69 contro 0.44) ma un  $K_p$  quasi uguale?** R: Perché ciò che si è perso, aggiungendo attuatore + ritardo + notch, è **fase** al crossover, non guadagno. La catena TVC (2o ordine a 70 rad/s) + Pade (20 ms) + notch (denominatore a  $z_D = 0.7$ ) introduce insieme una ventina di gradi di ritardo di fase intorno a 3 rad/s, e il PM del Task 1 crolla da 30 a 14.6 gradi con i guadagni originali. Il termine derivativo è l'unico che *aggiunge* fase ( $+\text{atan}(w \cdot K_d/K_p)$ ): alzarlo sposta lo zero del PD da  $K_p/K_d = 4.03$  a  $2.52 \text{ rad/s}$ , cioè più vicino al crossover, recuperando anticipo di fase.  $K_p$  invece fissa il margine aerodinamico a bassa frequenza (6 dB), che non è cambiato: quindi resta dov'era.

**D: Nel root locus del loop completo alcuni rami vanno verso destra, verso +232 e +184  $\pm 175i$ . Il sistema è instabile ad alto guadagno?** R: Sì, ed è fisicamente corretto. Quegli zeri nel semipiano destro sono l'approssimante di Pade del ritardo di 20 ms: un ritardo puro è un all-pass che a frequenza crescente accumula fase senza limite, quindi qualunque anello con ritardo diventa instabile se si alza abbastanza il guadagno. È esattamente la lettura in piano-s del *delay margin* (165 ms nel Task 2). Va detto però che gli zeri sono **artefatti dell'approssimazione**: il ritardo vero non ha zeri, ha solo fase. L'approssimante di Pade di ordine 3 è fedele fino a circa  $3/\tau = 150 \text{ rad/s}$ ; i rami che corrono verso +232 sono ben oltre quella banda e non vanno interpretati quantitativamente.

**D: Il modo di bending, ad anello chiuso, ha smorzamento 0.0054. Non è pochissimo?** R: Sì, ed è voluto. Il modo è *gain-stabilized*, non *phase-stabilized*: il notch lo mette a -18 dB di guadagno d'anello, quindi il controllore non lo eccita e non lo smorza. Resta lo smorzamento strutturale (0.5%). La conseguenza pratica: se qualcosa lo eccita comunque (separazione, transitorio del motore, turbolenza a banda larga) la struttura suona per  $1/(\zeta \cdot w_{BM}) \sim 10 \text{ s}$ . Il requisito che si soddisfa non è "il modo è smorzato" ma "il modo non è destabilizzato dal controllo, con almeno 12 dB di margine". Il prezzo, dichiarato nel README, è che il notch profondo richiede di conoscere  $w_{BM}$  con precisione: tollera -10% di detuning ma va instabile a +5%.

**D: Qual è il limite più importante di queste figure?** R: La figura del Task 3 dimostra la stabilità ai quattro vertici, ma il polo più vicino all'asse è **sempre** il modo di deriva, quindi la figura **non ordina** i casi per criticità: V3, il caso peggiore per margini (0.91 dB), risulta il più "lontano" dall'asse. Non è un errore di calcolo, è un limite intrinseco della lettura in piano-s: la robustezza in guadagno non si legge dalla posizione dei poli nominali. Serve la Nichols.

C'era anche un **difetto vero**, ora corretto, che vale la pena citare spontaneamente: `task1_polemap` aveva `ylim = [-0.28, 0.28]`, mentre la coppia di anello chiuso piazzata dal PD sta a  $-0.95 \pm 1.90i$  – **fuori dal riquadro**. La figura che doveva mostrare il pole placement mostrava solo il modo lento di deriva. Oggi la riga 73 usa `ylim = [-2.5, 2.5]` e tutti e quattro i poli chiusi sono in quadro, insieme al polo aperto instabile a  $+1.82$ .

# HM3/main\_\_task1.m

## Ruolo del file nel progetto

E' il **primo dei tre entry point** di HM3, quello che il professore fa girare per il Task 1 della traccia ("Rigid LV"). Lo script prende il lanciatore fittizio di Greensite congelato all'istante di **max-q\_bar** ( $t = 72$  s), lo modella come **corpo rigido con attuatore ideale** (niente dinamica TVC, niente ritardo, niente bending), progetta un controllore **PD di assetto** con una debole retroazione negativa di deriva laterale, ne verifica la stabilit  sul **piano di Nichols** e la valida con una **risposta temporale a raffica di vento**. Sono esattamente i due deliverable che la traccia chiede: "verify closed-loop stability using classical control tools in the frequency domain (e.g., the Nichols plot)" e "simulate the system's time-domain response to a wind gust. Plot the time histories of theta, z, zdot, delta".

Lo script e' un **orchestratore**: non contiene matematica propria se non due righe di cross-check analitico (righe 25-26). Tutto il lavoro sta nei moduli, che chiama in cascata:

```
load_hw3_params -> build_plant_rigid -> design_controller
                                   |
                                   (assemble_loop + classify_margins)
                                   |
load_wind_profile -> simulate_gust_response
                                   |
                                   plot_nichols_lv
```

Il punto concettuale che governa tutto il file e' che il velivolo a max-q\_bar e' **aerodinamicamente instabile**: il centro di pressione sta davanti al centro di massa, quindi il momento aerodinamico e' *divergente* ( $A_6 > 0$ ). Il polo di corpo rigido sta a  $+\sqrt{A_6} = +1.84$  rad/s. La retroazione non e' un miglioramento opzionale, e' l'unica cosa che tiene insieme il lanciatore. La conseguenza tecnica, che ricorre in tutti e tre i task, e' che l'anello e' **condizionalmente stabile**: si destabilizza sia *alzando* troppo il guadagno sia *abbassandolo* troppo. Per questo lo script non legge mai i margini con un `margin()` secco, ma li fa **classificare per banda di frequenza** da `classify_margins`.

I gain di deriva  $K_{p,z} = K_{d,z} = -1e-3$  non vengono toccati dallo script: sono i default di `design_controller`, e vengono dalla guideline della traccia ("The values of  $K_{P,z}$  and  $K_{D,z}$  must be negative in the order of  $10^{-3}$ ").

---

## %% Model and parameters (Table 1 @ $t = 72$ s) (righe 12-18)

```
p = load_hw3_params();
fprintf('Parameters source: %s\n', p.src);
G = build_plant_rigid(p);
```

- Riga 13: `load_hw3_params()` senza argomenti -> scaling di incertezza a 1 (nominale). La funzione prova a leggere `General/hw3-v3/GreensiteLPV_DATA.mat` e a interpolare i coefficienti tempo-varianti a  $t = 72$  s; se il file manca, ricade sui letterali di Table 1. Il campo `p.src` dice quale delle due strade e' stata presa,

ed e' stampato apposta a riga 14: e' l'unico modo, a run-time, per sapere se i numeri vengono dal dataset o dai letterali.

- Righe 15-16: stampa i tre coefficienti che contano e, soprattutto, `sqrt(p.A6)`, etichettato come “unstable airframe pole”. Nella run reale:

`A6 = 3.3818 1/s^2, K1 = 4.5647 1/s^2, V = 937.7 m/s`  
`polo instabile a +sqrt(A6) = +1.839 rad/s`

Da dove viene: nella dinamica rotazionale disaccoppiata (trascurando lo `zdot-coupling`) si ha  $\ddot{\theta} = A_6 \theta$ , cioè  $s^2 - A_6 = 0$ , quindi i due poli  $\pm \sqrt{A_6}$ .  $A_6 = N_{\alpha} / I_{yy} > 0$  perché il braccio  $l_{\alpha}$  del centro di pressione e' misurato davanti al CG: momento destabilizzante.

- Riga 18: `build_plant_rigid(p)` costruisce lo state-space a 4 stati  $[z, \dot{z}, \theta, \dot{\theta}]$ , 2 ingressi  $[\delta, \alpha_w]$ , 7 uscite (4 misurate + 3 di plotting). E' Eq. (1) della traccia privata delle righe di bending.

**Possibile domanda d'esame** – I poli in anello aperto del modello rigido sono davvero solo  $\pm \sqrt{A_6}$ ? *Risposta:* No. Quelli sono i poli della **dinamica rotazionale disaccoppiata**. Il plant a 4 stati completo, che include l'accoppiamento con la deriva laterale (termine  $A_6/V$  su  $\dot{z}$  e il termine  $a_1 V + a_4$  su  $\theta$ ), ha poli  $[0, -1.861, +0.0291, +1.8165]$ . Ci sono quindi **due poli a parte reale positiva** ( $+1.8165$ , il modo aerodinamico, e  $+0.0291$ , un modo lento di deriva) piu' un **integratore in zero** (la posizione laterale  $z$  e' l'integrale di  $\dot{z}$ ). Lo script stampa  $+1.839$  perché e' il valore analitico di riferimento, non l'autovalore esatto del plant accoppiato.

## %% Controller design (PD pitch + weak negative drift feedback) (righe 20-31)

```
[K, m] = design_controller(G, []);    % [] => ideal actuator

% Pole-placement cross-check: CL is s^2 + K1*Kd*s + (K1*Kp - A6).
wc_eq = sqrt(p.K1*K.Kp_th - p.A6);
ze_eq = p.K1*K.Kd_th/(2*wc_eq);
```

- Riga 22: il secondo argomento `[]` e' l'alias di “attuatore ideale” (`assemble_loop` lo converte in `tf(1)`). E' questo che rende il Task 1 il caso rigido puro della traccia.

## La coppia canonica di partenza (dentro `design_controller`, riga 55)

`design_controller` non parte da guadagni a caso: parte dalla **coppia in forma chiusa** ottenuta col pole placement sulla dinamica rotazionale disaccoppiata. Vale la pena rifare la derivazione, perché e' la domanda d'orale piu' probabile su questo task.

Sulla sola rotazione, con  $\theta_{ref} = 0$  e  $\delta = K_p(0 - \theta) - K_d \dot{\theta}$ :

$$\begin{aligned} \ddot{\theta} &= A_6 \theta + K_1 \delta \\ &= A_6 \theta - K_1 K_p \theta - K_1 K_d \dot{\theta} \end{aligned}$$

cioe' il polinomio caratteristico in anello chiuso

$$s^2 + K_1 K_d s + (K_1 K_p - A_6) = 0$$

Confrontandolo con la forma standard  $s^2 + 2\zeta \omega_n s + \omega_n^2$ :

$$\begin{aligned} \omega_n &= \sqrt{K_1 K_p - A_6} \\ \zeta &= K_1 K_d / (2 \omega_n) \end{aligned}$$

Ora si scelgono  $K_p$  e  $K_d$  per ottenere una coppia “bella”. La scelta canonica (D'Antuono Eq. 3.6-3.7) e':

```
Kp0 = 2*A_6/K_1      -> wn = sqrt(2*A_6 - A_6) = sqrt(A_6)
Kd0 = sqrt(A_6)/K_1  -> 2*zeta*wn = sqrt(A_6)  -> zeta = 0.5
```

**Interpretazione:** Kp0 e' il guadagno che **specchia il polo instabile +sqrt(A\_6) nel semipiano sinistro alla stessa frequenza naturale**, e Kd0 e' quello che gli da' smorzamento 0.5. E' la scelta minimale: non si chiede al controllo di essere piu' veloce dell'instabilita' che deve domare, solo di invertirne il segno. Numericamente:

```
Kp0 = 2*3.3818/4.5647 = 1.4817
Kd0 = sqrt(3.3818)/4.5647 = 0.4029
```

- Righe 25-26: **cross-check a posteriori**. Dopo che il tuner ha finito, lo script ricalcola wn e zeta *con le stesse due formule di sopra* ma usando i guadagni tunati. Serve a controllare che l'auto-tuner non sia scappato in una regione assurda. Risultato reale:

```
Kp = 1.7845, Kd = 0.4433
wc_eq = sqrt(4.5647*1.7845 - 3.3818) = 2.18 rad/s
ze_eq = 4.5647*0.4433/(2*2.18) = 0.46
```

Il commento a riga 27 dice "course band 1-4": la banda di crossover tipica di un lanciatore sta fra 1 e 4 rad/s (sopra si eccitano i modi flessibili, sotto non si domina l'instabilita' aerodinamica). 2.18 rad/s ci sta comodamente.

**Attenzione al limite:** wc\_eq e ze\_eq sono grandezze della **dinamica disaccoppiata**. Non sono i poli veri dell'anello chiuso a 4 stati (che includono la deriva). Sono un sanity check, non un risultato. Lo script non lo dice esplicitamente, ma il commento a riga 24 ("Pole-placement cross-check") lo lascia intendere.

Per confronto, i **poli veri** dell'anello chiuso a 4 stati sono due coppie:

```
-0.9533 +/- 1.9047i    (wn = 2.13 rad/s, zeta = 0.45)  <- la coppia piazzata dal PD
-0.0559 +/- 0.2329i    (wn = 0.24 rad/s, zeta = 0.23)  <- il modo lento di deriva
```

La coppia veloce e' ben approssimata dal cross-check disaccoppiato (2.18 / 0.46 contro 2.13 / 0.45): il seed analitico regge. La coppia lenta, invece, il modello disaccoppiato **non la vede proprio** – e' il modo che la retroazione di deriva crea chiudendo l'integratore di posizione, ed e' quello che detta l'orizzonte di simulazione (tau ~ 18 s, da cui Tend = 80 s).

## Perche' il tuner sposta i guadagni dai valori canonici

Da 1.4817/0.4029 (canonici) a 1.7845/0.4433 (tunati) c'e' un +20% su Kp e un +10% su Kd. Il motivo e' spiegato nell'header di **design\_controller**: la coppia canonica da' 6 dB di margine sull'anello **disaccoppiato**, ma sull'anello **completo** la retroazione di deriva erode il margine aerodinamico (scende a circa 4 dB). Il tuner **fminsearch** rialza il guadagno finche' il margine *classificato sull'anello vero* torna a 6 dB.

Il costo minimizzato (design\_controller, riga 88) e':

```
c = (|AeroGM_dB| - 6)^2 + (RigidPM_deg - 30)^2      [+ 1e4 se instabile]
```

- La ricerca avviene in **coordinate logaritmiche**,  $x = \log([Kp \ Kd])$  (riga 55), cosi' i guadagni restano positivi per costruzione senza vincoli espliciti.
- **Limite onesto:** il costo somma dB<sup>2</sup> e deg<sup>2</sup>, cioe' pesa implicitamente 1 dB = 1 grado. E' una scelta arbitraria (per quanto convenzionale). Il codice non la giustifica e non espone un peso relativo. In pratica funziona perche' entrambi i target vengono centrati esattamente (6.00 dB / 30.0 deg), quindi il minimo e' praticamente zero e la pesatura non discrimina.
- Righe 30-31: L = m.L (anello aperto SISO, rotto sul segnale **delta**) e T = m.T (anello chiuso completo a 4 stati). Da qui in poi lo script usa solo questi due oggetti: L per la frequenza, T per il tempo.

**Possibile domanda d'esame** – Perche' non usare **place()** o **lqr()** invece di un **fminsearch** su due margini? *Risposta:* Perche' la specifica della traccia **non e' sui poli, e' sui margini** ("values of KP,theta and KD,theta that yield a GM about 6 dB and PM about 30 deg"). Un pole placement ti da' i poli che vuoi ma non ti garantisce i margini, e viceversa: sull'anello *completo* (con deriva e attuatore) la relazione fra poli e margini non e' piu' quella della coppia disaccoppiata. La struttura



sceita e' la piu' onesta: usa il pole placement come **seed analitico** e poi lascia che l'ottimizzatore centri la specifica vera sul loop vero.

---

## %% Rigid-body stability margins (righe 33-41)

- Righe 34-41: solo stampe, i margini sono gia' dentro `m` (calcolati da `classify_margins` dentro `design_controller`). Output reale:

```
Aero gain margin : |GM| = 6.00 dB @ 0.59 rad/s (low-freq gain-reduction)
Rigid phase margin: PM  = 30.0 deg @ 2.45 rad/s
Delay margin      : DM   = 213 ms
Rigid GM / Flex margins: none (ideal actuator, no bending)
Full 4-state closed loop stable (isstable): 1
```

## Perche' i margini vanno classificati e non letti con `margin()`

E' il cuore concettuale di tutto HM3. Il plant ha **due poli instabili e un integratore**. Per il criterio di Nyquist, con  $P$  poli instabili in anello aperto servono  $P$  *encirclement* antiorari del punto critico per essere stabili in anello chiuso. Questo significa che il diagramma **deve** passare "dalla parte giusta" del punto critico, e quindi:

- **abbassando** il guadagno il diagramma si abbassa e perde l'encirclement -> instabile;
- **alzandolo** troppo lo si perde dall'altra parte -> instabile.

L'anello e' **condizionalmente stabile**: esiste una *banda* di guadagni ammessi, non una semiretta. Ne segue che il margine di guadagno a bassa frequenza e' un margine di **riduzione** ( $gmdb < 0$ ): e' quanto guadagno puoi *togliere* prima di cadere. `margin()` di default restituisce un solo numero (il crossover piu' "vicino") e su un anello del genere quel numero e' privo di significato fisico: tipicamente becca uno degli attraversamenti di deriva a bassissima frequenza, che non sono margini ma artefatti dell'integratore di posizione.

`classify_margins` risolve questo prendendo **tutti** gli attraversamenti da `allmargin()` e assegnandoli a bande fisiche:

- **Aero GM** (riga 44): fra gli attraversamenti della fase critica (-180 deg, mod 360) con  $gmdb < 0$  prende quello a **frequenza piu' bassa** (mode 'minf'). La maschera e'  $gf > 0 \ \& \ gf < w\_flex \ \& \ gmdb < 0$ : **esclude solo la voce di DC** ( $gf == 0$ , l'integratore), **non** gli attraversamenti sotto  $w\_drift$ . Qui: -6.00 dB @ 0.59 rad/s. (Che 0.59 cada *sopra*  $w\_drift = 0.55$  e' un fatto di questa run, non una regola imposta dal codice – comodo saperlo se un giorno il tuner spostasse il crossover.)
- **Rigid PM** (riga 49): margine di fase al crossover di corpo rigido. Qui si', la maschera filtra esplicitamente  $pf > w\_drift$  (e prende il *massimo* valore, mode 'maxv'). Qui: 30.0 deg @ 2.45 rad/s.
- **Rigid GM** (riga 45): margine di *aumento* di guadagno ( $gmdb > 0$ ). Qui **assente** (NaN).
- **drift\_w** (riga 62): gli attraversamenti di fase sotto  $w\_drift = 0.3 * \sqrt{A6} = 0.55$  rad/s, raccolti a parte e marcati esplicitamente come "non margini".

## Perche' il Rigid GM e' assente nel Task 1

Con attuatore ideale l'anello aperto e'  $L = K_c * G$ , dove il PD contribuisce uno zero ( $K_p + K_d * s$ ) e il plant rigido ha 4 poli. La fase ad alta frequenza tende all'asintoto di -90 deg (relativo grado 1 sulla catena di assetto: il derivativo cancella un ordine). **Non riesce mai a riavvolgersi fino a un secondo attraversamento della fase critica -180 deg**, quindi non esiste un margine di aumento di guadagno: puoi alzare il guadagno quanto vuoi e l'anello resta stabile. E' la riga 40 che lo dichiara. Nel Task 2 l'attuatore TVC (2 poli), il ritardo e il notch aggiungono ritardo di fase sufficiente a creare quel crossover, e infatti li' spunta un Rigid GM = 7.56 dB @ 11.11 rad/s.

**Possibile domanda d'esame** – Il delay margin e' 213 ms. Ma nel Task 1 l'attuatore e' ideale e non c'e' nessun ritardo. A cosa serve quel numero? *Risposta*: E' una **previsione**: dice quanto ritardo puro l'anello rigido tollerebbe prima di perdere stabilita'. Serve come budget da spendere nel Task 2, dove entrano un ritardo reale di 20 ms e la dinamica TVC (che a bassa frequenza si comporta come

ritardo equivalente). 213 ms e' molto sopra il requisito tipico di 100 ms citato a riga 39, quindi il Task 1 lascia margine abbondante. Nel Task 2, dopo il notch, scende a 165 ms; e con i guadagni del Task 1 non ri-tunati scenderebbe a 98 ms, cioe' sotto il requisito.

## %% Wind-gust time response (righe 43-55)

```
w = load_wind_profile(p, Tend=80);
r = simulate_gust_response(T, w);
```

- Riga 44: la raffica di default e' un **1-cosine discrete gust** di durata  $T_g = 3$  s con onset a  $t_0 = 1$  s. L'ampiezza  $V_g$  non e' hard-coded: viene interpolata dalla dispersione `drywind.mat` (severita' **severe**, il default) alla quota di volo  $Alt = 15143$  m. Risultato reale:  **$V_g = 6.38$  m/s**, quindi  

$$\alpha_w\text{_{peak}} = V_g/V = 6.38/937.7 = 6.80e-3 \text{ rad} = 0.39 \text{ deg}$$
Il profilo e'  $v_w(t) = 0.5 V_g(1 - \cos(2\pi(t - t_0)/T_g))$  sulla finestra della raffica, zero altrove: parte da zero con derivata nulla e ci torna, quindi non inietta discontinuita' nell'anello.
- Il commento a riga 44 giustifica `Tend=80`: il modo dominante in anello chiuso ha  $\tau \sim 18$  s ( $\omega_n = 0.24$ ,  $\zeta = 0.23$ ), e servono ~5 costanti di tempo per vedere l'assetto tornare a zero. E' il **modo di deriva lento**, non la coppia di beccheggio a 2.18 rad/s: la deriva laterale, chiusa con guadagni da  $1e-3$ , e' lentissima. Se si simulasse a 12 s (il default di `load_wind_profile`) si vedrebbe solo il transitorio di assetto e si perderebbe il ritorno a zero.
- Riga 45: `simulate_gust_response` fa un `lsim` di `T` con ingresso `[alpha_w(t), theta_ref = 0]`: e' una simulazione di **pura reiezione di disturbo**, senza comando di assetto.

Risultati reali:

```
peak |theta| = 0.261 deg
peak |z|     = 2.27 m
peak |delta| = 0.528 deg
peak |zdot|  = 0.68 m/s
```

- Righe 51-52: il **budget di incidenza** e il carico aerodinamico. A `max-q_bar` il parametro dimensionante non e' `theta` ne' `delta`, e' il prodotto **`q_bar * alpha`** (pressione dinamica per incidenza), che e' proporzionale al momento flettente alla base del lanciatore. Con `q_bar = 81.1` kPa e un picco di incidenza di 0.577 deg, lo script riporta **`**46.8 kPa*deg**`**.
- Righe 53-55: dichiara la deriva laterale come "canale di load relief" e non come margine di Nichols, e la confronta con due soglie (500 m, 15 m/s) che pero' **non sono nel codice come costanti verificate**: sono numeri scritti a mano nella stringa di formato. Non c'e' nessun `assert`, nessun confronto: sono puramente decorativi. **E l'etichetta "load relief" e' un residuo storico fuorviante**: la retroazione di deriva contiene la deriva e stabilizza l'integratore di posizione, ma sull'incidenza incide per l'1% (vedi sotto).

## Il segno di alpha: perche' e' un MENO (e perche' ribalta la narrazione)

`simulate_gust_response.m` (riga 29) calcola:

```
r.alpha = r.theta + r.zdot/w.V - r.alphaw      (segno MENO su alpha_w)
```

ed e' l'**unico segno coerente col plant**. La Eq. (1) della traccia ha la colonna del disturbo pari a `[0; -a1*V; 0; -A_6]`, e il codice la riproduce identica (`build_plant_rigid.m` riga 17). Sviluppando la riga di `theta_ddot`:

```
theta_ddot = A_6*theta + (A_6/V)*zdot + K_1*delta - A_6*alpha_w
            = A_6*(theta + zdot/V - alpha_w) + K_1*delta
```

cioe' l'incidenza aerodinamica che il **momento vede davvero** e'

```
alpha = theta + zdot/V - alpha_w
```

Idem per la riga di `zdot_dot`, che si raggruppa in  $\alpha V(\theta + \dot{z}/V - \alpha_w)$ . Il meno ha una lettura fisica immediata:  $\alpha$  e' l'incidenza rispetto alla velocità **relativa all'aria**, quindi un vento laterale si **sottrae** allo  $\dot{z}$  del veicolo. Il caso limite lo conferma: con un gradino di vento sostenuto e il feedback di deriva spento, la simulazione converge a  $\dot{z}_{ss} = +V\alpha_w$  (rapporto verificato 1.0000) con  $\theta_{ss} = 0$  e dunque  $\alpha_{ss} = 0$ : il veicolo viene spinto sottovento finché non vola *insieme* all'aria, e l'incidenza relativa si annulla.

**Nota storica (buona da sapere all'orale).** Fino a poco fa questa riga aveva un `+ r.alphaw`. Era un **bug di post-processing**, non di modello: il plant e' sempre stato giusto, sbagliava solo la formula con cui si ri-costruiva  $\alpha$  a valle di `lsim`. Corretto. Cosa cambia e cosa no:

```
alpha col vecchio + (buggato):  picco 0.255 deg  ->  q_bar*alpha = 20.7 kPa*deg
alpha col meno (corretto):      picco 0.577 deg  ->  q_bar*alpha = 46.8 kPa*deg
```

Un **fattore 2.3** sul carico. **Non cambia nulla** di margini, Nichols, stabilità, time history di  $\theta/z/\dot{z}/\delta$  e Task 3:  $\alpha_w$  non entra in  $L(s)$ , e la riga e' pura post-elaborazione. Cambiano solo i due numeri di carico (righe 51-52) e la figura f3 – e con essi la **narrazione fisica**.

## La conseguenza: un puro attitude hold e' LOAD-AGGRAVATING

Il picco di incidenza totale (**0.577 deg**) supera il contributo del solo vento (**0.390 deg**). Il vento genera un momento  $-A_6\alpha_w$  che spinge il muso in negativo; il PD, che ha come unico obiettivo  $\theta \rightarrow 0$ , reagisce **beccheggiando il muso dentro il vento relativo**, e quel  $\theta$  (-0.178 deg al picco di raffica) entra in  $\alpha = \theta + \dot{z}/V - \alpha_w$  con lo **stesso segno** di  $-\alpha_w$ : si **somma** al vento invece di cancellarlo.

Una legge di puro attitude-hold e' quindi **load-aggravating**, non load-relieving – ed e' esattamente il motivo per cui i lanciatori reali aggiungono un **termine esplicito di load relief** (accelerometro laterale o  $\alpha$  stimata). Il velivolo non offre aiuto: con  $A_6 > 0$  il centro di pressione sta davanti al baricentro, il momento aerodinamico e' divergente, non c'e' nessuna stabilità a banderuola che riallinei spontaneamente il muso col vento.

**Possibile domanda d'esame** – Il picco di  $q_{bar}\alpha$  e' 46.8 kPa\*deg. E' tanto o poco? *Risposta:* Poco: circa un ordine di grandezza sotto i limiti tipici di un lanciatore snello (alcune centinaia di kPa\*deg). Il motivo pero' non e' merito del controllore – che anzi **aggrava** il carico rispetto al solo vento (0.577 vs 0.390 deg di incidenza) – ma della raffica, che e' modesta:  $V_g = 6.4$  m/s, la sigma della dispersione drywind a 15 km, non un caso di vento estremo. Il caso di carico vero si ottiene con `load_wind_profile(..., 'profile', 'strongwind')`, che usa il generatore di vento del professore.

## %% Figures (righe 57-99)

Tre figure, tutte con `'Color','w'` esplicito.

### f1 – Nichols (righe 62-65)

Delegata a `plot_nichols_lv(L, m, ...)`, con `wrange = [1e-2 1e2]` e `xlim = [-720 0]`. La convenzione e' quella degli appunti del corso (lo dichiara il commento alle righe 58-61): il punto critico sta a **(-180 deg, 0 dB)**, derivato da  $1 + L = 0 \Leftrightarrow L = -1$ . La fase e' **phase-matched** in modo che il crossover rigido cada sul ramo che contiene -180: lo shift e' un **multiplo esatto di 360 deg** (`sh = ph + 360*round(...)`, riga 49 di `plot_nichols_lv`), quindi e' una pura **rietichettatura del ramo di fase**, non una rotazione: la curva non viene distorta (nel Task 1 lo shift risulta addirittura 0: il ramo naturale di MATLAB e' gia' quello del corso). Verificato sulla figura: l'attraversamento aerodinamico casca esattamente a -180 deg, 6 dB sopra il punto critico, e il crossover rigido a -150 deg, cioè 30 deg a destra del punto critico – il PM si legge direttamente dal grafico.

(D'Antuono Fig. 3.2 mostra la **stessa carta rietichettata di +360 deg**, punto critico a +180: +180 e -180 sono lo stesso punto, fase mod 360. Il codice usava quella rietichettatura fino a poco fa ed e' stato riallineato alla convenzione del corso; nessun margine cambia, e' solo l'etichetta dell'asse.)

Il senso della figura: l'anello **viene dall'alto** (a DC il guadagno tende a infinito per via dell'integratore di posizione laterale  $z$ ) e passa **sopra** il punto critico con 6 dB di scarto. E' la firma grafica della stabilit  condizionale (a un solo lato, nel Task 1: manca l'attraversamento superiore). I marker sovrapposti sono i margini classificati (quadrato = Aero GM, rombo = Rigid PM) e le croci nere sono gli attraversamenti di deriva, etichettati letteralmente "not a margin".

## f2 – Gust response (righe 68-82)

Tiled 2x2:  $\theta$ ,  $z$ ,  $\dot{z}$ ,  $\delta$ . Sono **esattamente le quattro variabili che la traccia elenca**. Nessuna elaborazione, plot diretti di  $r$ .

## f3 – Alpha budget e carico (righe 86-99)

Il primo pannello scompone  $\alpha$  nei suoi tre contributi ( $\theta$ ,  $\dot{z}/V$ ,  $\alpha_w$ ), il secondo mostra  $q_{bar} \cdot \alpha$  (picco  $46.8 \text{ kPa} \cdot \text{deg}$ ). Il commento a riga 84 e il titolo del pannello a riga 96 riportano entrambi la formula col segno giusto,  $\alpha = \theta + \dot{z}/V - \alpha_w$ .

E' la **figura piu' interessante di tutto il Task 1**, ed e' quella da mostrare all'orale: si vede a occhio che la curva di  $\theta$  e quella di  $\alpha_w$  vanno nella **stessa direzione** nel bilancio ( $\theta$  negativo,  $-\alpha_w$  negativo), e che la  $\alpha$  risultante e' **piu' grande in modulo** del solo contributo del vento. E' la prova grafica del load aggravation discusso sopra.

## Export (righe 101-113)

- Riga 102: `fig_dir` risolto via `mfilename('fullpath')`, quindi lo script scrive sempre in `HM3/figures/` **indipendentemente dalla working directory**. Buona pratica, e la ragione per cui il professore puo' lanciarlo da qualunque cartella.
- Righe 105-109: `theme(f, 'light')` dentro un `try/catch` con fallback su `f.Color = 'w'`. Serve a forzare figure chiare anche se il desktop MATLAB e' in dark mode; `theme()` esiste solo da R2025a, da cui il catch.
- Righe 110-111: `exportgraphics` a 200 dpi, nomi `task1_<Name>.png` dove `<Name>` e' la proprieta' 'Name' della figura.

## Possibili domande d'esame

**D: Cosa vuol dire che l'anello e' "condizionalmente stabile" e come lo si vede sul Nichols?** R: Vuol dire che la stabilit  richiede il guadagno dentro una **banda**, non sopra una soglia: sia riducendolo sia aumentandolo troppo si perde. Nasce dal fatto che il plant ha poli instabili in anello aperto ( $+1.82$  e  $+0.03 \text{ rad/s}$ , piu' un integratore), quindi per Nyquist il diagramma deve produrre un numero preciso di encirclement del punto critico. Sul Nichols si riconosce perche' la curva **arriva dall'alto** (guadagno infinito a DC) e attraversa la fase critica  $-180 \text{ deg}$  **sopra** il punto critico ( $-180, 0 \text{ dB}$ ), senza toccarlo: c'e' margine di guadagno verso il basso e margine di fase al crossover. Il margine aerodinamico e' quello di **riduzione** ( $gmdb < 0$ ), qui  $6.00 \text{ dB}$  a  $0.59 \text{ rad/s}$ .

**D: Perche' i guadagni di deriva  $Kp_z$  e  $Kd_z$  sono negativi e cosi' piccoli, e cosa fanno davvero?** R: Il segno negativo lo impone la traccia ("must be negative in the order of  $10^{-3}$ "). La loro funzione primaria e' un **requisito di stabilit **, non di load relief: chiudono l'**integratore libero della posizione laterale** (il plant ha un polo esatto in  $s = 0$ , perche'  $z$  e' l'integrale di  $\dot{z}$ ). Verifica diretta, spegnendoli ( $Kp_z = Kd_z = 0$ ):

|                             | poli di anello chiuso                  | picco $\alpha$      | picco $z$                          |
|-----------------------------|----------------------------------------|---------------------|------------------------------------|
| con drift feedback          | $-0.056 \pm 0.233i, -0.953 \pm 1.905i$ | $0.577 \text{ deg}$ | $2.27 \text{ m}$                   |
| senza ( $Kp_z = Kd_z = 0$ ) | $0, -0.076, -0.98 \pm 1.93i$           | $0.584 \text{ deg}$ | <b><math>9.54 \text{ m}</math></b> |

Senza di loro resta un polo **esattamente nell'origine**: il sistema e' solo **marginalmente stabile** e  $z$  non torna mai a zero. Sull'incidenza incidono per l'1% (0.584 -> 0.577 deg: irrilevante), ma tagliano la deriva **da 9.5 m a 2.3 m**. Chiamarli "load relief" (come fanno i commenti del codice e il README) e' fuorviante: un vero load relief richiederebbe un termine dedicato in  $\alpha$  o accelerazione laterale. Il valore piccolo ( $1e-3$ ) serve perche' il canale di deriva non interferisca con la banda di assetto.

**D: L'auto-tuner centra 6.00 dB e 30.0 deg esattamente. Non e' sospetto?** R: No, e' atteso: il costo e' una somma di due quadrati con due gradi di liberta' ( $K_p$ ,  $K_d$ ) e due target. Il sistema e' quadrato, quindi esiste generalmente una soluzione a costo (quasi) nullo e `fminsearch` la trova. La conseguenza pero' e' che **la pesatura relativa  $dB^2$  vs  $deg^2$  non conta**: se i target fossero incompatibili (piu' target che gradi di liberta') quella pesatura arbitraria diventerebbe determinante e andrebbe giustificata.

**D: Che differenza c'e' fra i guadagni canonici (1.48, 0.40) e quelli tunati (1.78, 0.44), e perche' il tuner li alza?** R: I canonici  $K_{p0} = 2*A_6/K_1$  e  $K_{d0} = \sqrt{A_6}/K_1$  sono esatti sulla dinamica rotazionale **disaccoppiata** e li' danno 6 dB di margine con  $\zeta = 0.5$ . Ma sull'anello **completo** l'aggiunta della retroazione di deriva (e del suo integratore) abbassa il margine aerodinamico a circa 4 dB. Il tuner alza  $K_p$  del 20% per recuperare i 6 dB sul loop vero. La lezione e': la formula chiusa e' un ottimo **seed**, non il progetto finale.

**D: Il codice riporta un picco di  $\alpha$  di 0.577 deg, ma il vento da solo ne fa 0.390. Come fa l'incidenza totale a essere PIU' GRANDE del disturbo?** R: Perche' il controllore la peggiora. Il bilancio e'  $\alpha = \theta + \dot{z}/V - \alpha_w$  (il meno viene dalla colonna di disturbo  $[0; -a_1*V; 0; -A_6]$  di Eq. (1):  $\alpha$  e' l'incidenza rispetto all'aria, quindi il vento si sottrae allo  $\dot{z}$  del veicolo). Il momento  $-A_6\alpha_w$  spinge il muso in negativo; il PD di assetto, che vuole solo  $\theta \rightarrow 0$ , reagisce ruotando il muso **dentro** il vento relativo, e quel  $\theta$  (-0.178 deg al picco) entra nel bilancio con lo **stesso segno** di  $-\alpha_w$ . I due contributi si **sommano**. Morale: un puro attitude hold e' **load-aggravating**; il load relief vero richiede un termine esplicito (accelerometro laterale /  $\alpha$  stimata) che questo controllore non ha. Nota storica: fino a poco fa `simulate_gust_response.m` aveva un  $+$  su  $\alpha_w$  e riportava 0.255 deg / 20.7 kPadeg, facendo sembrare l'anello load-relieving. Era un bug di **post-processing** – il plant e' sempre stato giusto – e infatti la correzione non tocca ne' i margini ne' le time history di  $\theta$ ,  $z$ ,  $\dot{z}$ ,  $\delta$ .

**D: Perche' simulare 80 s se la raffica dura 3 s?** R: Perche' il modo dominante in anello chiuso non e' quello di beccheggio (2.18 rad/s,  $\tau < 1$  s) ma il **modo lento di deriva laterale**, con  $\omega_n = 0.24$  rad/s e  $\zeta = 0.23$ , cioe'  $\tau \sim 18$  s. Servono ~5 costanti di tempo per vedere l'assetto e la deriva rientrare. Con il default di 12 s si vedrebbe solo il transitorio veloce e si perderebbe la parte piu' interessante (il ritorno a zero e il picco di deriva).

# HM3/main\_\_task2.m

## Ruolo del file nel progetto

E' il **secondo entry point** di HM3 e il piu' ricco dei tre. Implementa il Task 2 della traccia ("Full LV Model"): estendere il progetto del Task 1 al modello completo, aggiungendo (a) la **dinamica dell'attuatore TVC** di Eq. (3), (b) il **ritardo puro di 20 ms**, (c) il **primo modo di bending** con la sua contaminazione della misura inerziale (Eq. 2), e infine inserire il **filtro notch** e ri-verificare la stabilita'.

Lo script e' organizzato in quattro step espliciti (i banner %% sono la scomposizione logica voluta dall'autore):

```
Step A : ricostruisce il PD del Task 1 (attuatore ideale)
Step B : monta TVC + ritardo + bending SENZA filtro -> l'anello esplode
Step C : trade study su QUATTRO filtri di bending -> sceglie il notch profondo
         + RI-SINTONIZZA il PD sull'anello completo
Step D : detuning di omega_BM (+/-10%) a filtri fissi
```

Il fatto tecnico che genera tutto il resto e' questo: il modo di bending ha **zeta\_BM = 0.005**, cioe' un fattore di merito  $Q = 1/(2*zeta\_BM) = 100$ , una risonanza da +40 dB. Il modo entra nell'anello per due strade contemporaneamente:

- **in avanti**: il TVC forza il bending, riga eta\_ddot del plant, con il termine -phi\_tvc Tcdelta;
- **all'indietro**: la piattaforma inerziale non misura theta ma theta\_m = theta + sigma\_ins\*eta (Eq. 2, sigma\_ins = 0.178 rad/m), quindi il bending **rientra nella retroazione**.

Il prodotto di questi due percorsi chiude un anello parassita attorno alla risonanza. Il codice misura che a omega\_BM il guadagno d'anello vale **+29 dB**: ventinove decibel *sopra* la soglia di instabilita', con fase arbitraria. Senza filtro l'anello chiuso e' instabile (polo a Re = +1.61).

Nota: il bending contamina anche z\_m e zdot\_m (con -phi\_ins = -0.8), ma quei canali sono chiusi con guadagni da 1e-3, quindi il percorso che conta davvero e' sigma\_ins sul canale di assetto.

**Refuso nel sorgente**: il commento di header alle righe 7-8 parla della "+39 dB bending resonance". Il valore reale, stampato dal codice stesso alla riga 33, e' **+29.0 dB**. Il README riporta correttamente 29 dB.

---

## %% Model and parameters + %% Step A (righe 19-26)

```
p = load_hw3_params();
Grigid = build_plant_rigid(p);
[K, mR] = design_controller(Grigid, []);
[~, Trigid] = assemble_loop(Grigid, K);
```

- Riga 17: warning('off','Control:analysis:MarginUnstable'). margin/allmargin emettono un warning ogni volta che valutano un anello instabile in anello aperto – che qui e' **sempre**, per costruzione. Senza questo silenziamento la console verrebbe sommersa. E' legittimo, non e' nascondere un problema.

- Righe 23-25: **Step A non riusa un file di risultati: ri-esegue il tuner del Task 1.** `design_controller(Grigid, [])` e' deterministico (`fminsearch` da un seed fisso), quindi riproduce esattamente  $K_p = 1.7845$ ,  $K_d = 0.4433$ . Il commento dice "reused from Task 1" ma tecnicamente e' **ricalcolato**. Costa un tuner run in piu' ma rende lo script autosufficiente.
- Riga 26: `Trigid` serve solo alla fine, per il confronto rigido-vs-completo (figura f3).

## %% Step B - full plant + TVC + delay, NO bending filter (righe 28-34)

```
Gfull = build_plant_full(p, 'ins');
Wtvc = build_tvc(p, 3);           % 2nd-order TVC + Pade(20 ms, order 3)
[Lb, Tb] = assemble_loop(Gfull, K, Wtvc);
```

- Riga 29: 'ins' seleziona la matrice di uscita di **Eq. (2)**, quella contaminata dal bending. L'alternativa 'true' (misure pulite) esiste nel modulo ma non e' usata qui: usarla farebbe sparire il problema, che e' proprio cio' che si vuole studiare.
- Riga 30: `build_tvc(p, 3)` costruisce

$$W_{TVC}(s) = w_{TVC}^2 / (s^2 + 2*z_{TVC}*w_{TVC}*s + w_{TVC}^2) * \text{Pade}(\tau, 3)$$

con  $w_{TVC} = 70$  rad/s,  $z_{TVC} = 0.7$ ,  $\tau = 0.020$  s. **Perche' Pade ordine 3:** il ritardo puro  $\exp(-\tau*s)$  non e' razionale e non si puo' mettere in un `ss/tf`. L'approssimante di Pade di ordine 3 riproduce fedelmente la fase fino a circa  $3/\tau \sim 150$  rad/s, quindi copre bene sia il crossover rigido ( $\sim 3$  rad/s) sia la zona di bending (18.9 rad/s). Un ordine 1 sarebbe troppo grossolano a  $\omega_{BM}$ . **Costo:** Pade introduce zeri a destra (non-minimum phase), che e' esattamente il comportamento di un ritardo, quindi e' fedele.

- Righe 32-34: output reale

$$|L(\omega_{BM})| = 29.0 \text{ dB} \rightarrow \text{closed-loop stable: 0 (max Re pole} = 1.61)$$

Il modo di bending non e' ne' gain-stabilizzato (sarebbe  $|L| \ll 0$  dB) ne' phase-stabilizzato (la fase a  $\omega_{BM}$  e' quella che capita): l'anello e' semplicemente instabile.

**Possibile domanda d'esame** – I margini di bassa frequenza (Aero GM, Rigid PM) sopravvivono all'aggiunta di TVC e ritardo? *Risposta:* Si', quasi intatti. Nella tabella di Step C la riga "no filter" da' Aero GM = 6.25 dB e Rigid PM = 26.1 deg: partivano da 6.00 e 30.0. Il motivo e' che il TVC ha  $w_{TVC} = 70$  rad/s, ben oltre un decennio sopra il crossover rigido ( $\sim 2.5$  rad/s), e 20 ms di ritardo a 2.5 rad/s valgono solo  $0.020 \cdot 2.5 = 0.05 \text{ rad} = 3 \text{ deg}$ . **Il problema del Task 2 non e' l'attuatore in banda: e' il bending.** L'attuatore diventa un problema solo dopo\*, in combinazione con il ritardo di fase del notch (vedi Step C).

## %% Step C - bending filter trade study (righe 36-129)

Quattro candidati, valutati a guadagni PD fissi (quelli del Task 1). La griglia comune (righe 47-49):

```
wx_grid = wBM + (-4:2:4) = [14.9  16.9  18.9  20.9  22.9] rad/s
zN_grid = 0.10 : 0.05 : 0.30          (5 valori)
zD_grid = 0.40 : 0.10 : 0.60          (3 valori)
-> 5 * 5 * 3 = 75 combinazioni
```

Le tre griglie sono **esattamente i range suggeriti dalla traccia** ("zeta\_x,N ~ 0.1-0.3, zeta\_x,D ~ 0.4-0.6,  $\omega_{x}$  around the first bending mode frequency +/- 4 rad/s").

## La matematica di Eq. (4)

`build_notch_filter(wx, zN, zD, sgn)` costruisce

$$H_x(s) = (s^2 + \text{sgn} \cdot 2 \cdot zN \cdot wx \cdot s + wx^2) / (s^2 + 2 \cdot zD \cdot wx \cdot s + wx^2)$$

Due osservazioni importanti:

1. **La traccia ha un refuso:** Eq. (4) e' stampata come  $(s^2 - 2 \cdot \zeta_{xN} \cdot \omega_x + \omega_x^2) / (\dots)$ , cioe' senza la  $s$  sul termine centrale del numeratore – dimensionalmente impossibile. Il codice ripristina la  $s$  mancante, che e' l'unica lettura sensata.
2. **Il segno del numeratore e' il vero interruttore di progetto.**
  - $\text{sgn} = -1$  (Eq. 4 come stampata): il numeratore ha zeri a **parte reale positiva** -> filtro **non-minimum-phase**. Il modulo e' identico a quello del corrispondente minimum-phase, ma la fase e' specchiata: il filtro **ruota** la curva senza abbassarla. E' un **phase shaper**: e' esattamente il "targeted phase shift at the bending mode frequency in the Nichols plot (left-right shift)" di cui parla la traccia.
  - $\text{sgn} = +1$ : notch **minimum-phase** simmetrico, che **abbassa** il modulo. E' un **gain stabiliser**.

Attenuazione al centro (per entrambi i segni), valutando  $|H_x(j \cdot wx)|$ : i termini in  $s^2$  e  $wx^2$  si cancellano e resta

$$|H_x(j \cdot wx)| = (2 \cdot zN \cdot wx \cdot wx) / (2 \cdot zD \cdot wx \cdot wx) = zN / zD$$

-> profondita' in dB =  $20 \cdot \log_{10}(zN/zD)$ . Con i range della traccia il massimo che si ottiene e'  $20 \cdot \log_{10}(0.1/0.6) = -15.6$  dB: molto poco contro una risonanza a +29 dB. **Da qui nasce tutta la logica del trade:** i valori suggeriti dalla traccia non possono gain-stabilizzare il modo, quindi o si phase-stabilizza (usando  $\text{sgn} = -1$ ) o si esce dai range.

## C-LL: lead-lag di Eq. 4 da sola (righe 51-71)

- Righe 54-67: triplo ciclo sulle 75 combinazioni,  $\text{sgn} = -1$ . Per ognuna chiude l'anello e registra  $\max(\text{real}(\text{pole}(\text{Tc}_i)))$ . Il criterio di selezione (riga 61) e' **il minimo del massimo polo reale**, cioe' "il meno instabile" / il piu' smorzato. **Non e' una selezione sui margini.**
- Output reale:

C-LL alone: 45/75 guideline candidates stabilise the loop; least unstable  
( $wx=18.9$ ,  $zN=0.30$ ,  $zD=0.60$ ) has  $\max \text{Re}(\text{pole}) = -0.06$

**Contraddizione fra commento e codice, da segnalare:** il commento alla riga 39 dice "Alone it never stabilises". E' **falso**: 45 candidati su 75 stabilizzano l'anello, e il migliore ha  $\max \text{Re}(\text{pole}) = -0.06$ , cioe' e' **stabile** (appena). Anche la legenda della figura (riga 229) lo etichetta "marginal", che e' piu' corretto. Il commento va letto come "non stabilizza *bene*", non "non stabilizza".

Riga di tabella corrispondente:

|      |               |  |      |      |      |      |    |         |
|------|---------------|--|------|------|------|------|----|---------|
| C-LL | lead-lag only |  | 5.96 | 11.4 | 8.02 | 23.0 | 15 | stabile |
|------|---------------|--|------|------|------|------|----|---------|

Il dato chiave e'  $|L(\omega_{BM})| = +23.0$  dB: il modo e' ancora **sopra** 0 dB. La lead-lag lo ha attenuato solo di 6 dB ( $= 20 \cdot \log_{10}(0.30/0.60)$ ), esattamente come predice la formula. Sopravvive quindi **solo per fase**: il lobo di bending passa dalla parte giusta del punto critico. E' phase stabilisation pura. Ma il delay margin e' **15 ms**, cioe' **inferiore ai 20 ms di ritardo gia' modellati**: il progetto e' sul filo del rasoio e non e' difendibile.

## C-N: notch profondo minimum-phase (righe 73-78) – il progetto ritenuto

```
notch.wx = p.wBM;    notch.zN = 0.002;  
notch.zD = 0.7;      notch.sgn = +1;
```

- Profondita':  $20 \cdot \log_{10}(0.002/0.7) = -50.9$  dB (verificato). Contro i +29 dB della risonanza ->  $|L(\omega_{BM})| = 29 - 51 = -21.9$  dB: il modo e' **gain-stabilizzato**, cioe' semplicemente non ha piu' abbastanza guadagno per destabilizzare, qualunque sia la sua fase.



- **Deviazione dichiarata dalla traccia:**  $zN = 0.002$  e  $zD = 0.7$  sono **fuori dai range suggeriti** ( $zN$  0.1-0.3,  $zD$  0.4-0.6). E' una scelta consapevole – come mostrato sopra, dentro quei range la profondita' massima e' -15.6 dB, che non basta. Il progetto ritenuto **cambia dispositivo**: non e' piu' la lead-lag di Eq. (4), e' un notch stretto e profondo. All'orale va difeso cosi', non nascosto.

### C-T: tripletta di notch (righe 80-83)

Tre notch identici a 0.9, 1.0 e 1.1 volte  $\omega_{BM}$ . E' la ricetta classica per coprire l'incertezza sulla frequenza del modo. Risultato:

C-T notch triplet | NaN -7.3 NaN -55.8 2469 | INSTABILE

- $|L(\omega_{BM})| = -55.8$  dB: il bending e' sepolto.
- Ma **Rigid PM = -7.3 deg**: l'anello e' instabile. **Ogni notch costa fase alle frequenze sotto il suo centro**, e tre notch impilati costano circa il triplo: al crossover rigido ( $\sim 2.5$  rad/s) mangiano  $\sim 30$  deg di margine di fase, che e' tutto quello che c'era. **E' la lezione piu' istruttiva del trade**: i notch non sono gratis, e sovra-filtrare uccide il corpo rigido.
- Il **DM = 2469 ms** in quella riga e' un numero **privo di senso**: `allmargin` restituisce margini anche su anelli instabili, dove non significano nulla. Vale lo stesso per la riga "no filter" (DM = 41 ms). **L'unica colonna che decide e' stable**. Il codice stampa i margini comunque, il che e' onesto ma va letto con questa avvertenza.

### C-NLL: notch + lead-lag (righe 85-106)

- Secondo sweep sulle stesse 75 combinazioni, questa volta in **serie al notch profondo** (`Wtvc*Hn*Hc`). Criterio di selezione (riga 97): **massimo delay margin**, valutato solo sui candidati stabili.
- Output: 67/75 stabilizzano; il migliore e'  $w_x = 22.9$ ,  $zN = 0.10$ ,  $zD = 0.40$ .
- Riga di tabella: Aero 5.72 / PM 8.0 /  $|L(w_{BM})|$  -28.1 dB / DM 54 ms. Margini nominali **piu' magri** del solo notch, ma vedremo in Step D che e' molto piu' robusto al detuning.

### Tabella comparativa (righe 108-129)

- Riga 119: le bande passate a `classify_margins`:

```
w_drift   = 0.3*sqrt(A6) = 0.55 rad/s   (sotto: artefatti di deriva)
w_flex    = 0.6*wBM      = 11.34 rad/s  (confine rigido/flessibile)
w_flex_hi = 1.5*wBM      = 28.35 rad/s
w_bending = wBM          = 18.90 rad/s
```

**Limite da segnalare:** il Rigid GM del progetto finale cade a **11.11 rad/s**, contro un confine `w_flex` a **11.34 rad/s**. Sta *dentro* la banda rigida per soli 0.23 rad/s. E' fragile: una piccola variazione di guadagno o di  $\omega_{BM}$  lo riclassificherebbe come margine flessibile e i numeri stampati cambierebbero categoria (non valore). La scelta di `w_flex = 0.6*wBM` e' euristica e il codice non la giustifica.

Tabella completa dalla run reale:

| candidate |               | AeroGM | RigidPM | RigidGM | $ L(w_{BM}) $ | DM[ms] | stable |
|-----------|---------------|--------|---------|---------|---------------|--------|--------|
| B         | no filter     | 6.25   | 26.1    | NaN     | 29.0          | 41     | 0      |
| C-LL      | lead-lag only | 5.96   | 11.4    | 8.02    | 23.0          | 15     | 1      |
| C-N       | deep notch    | 6.08   | 14.6    | 10.43   | -21.9         | 98     | 1      |
| C-T       | notch triplet | NaN    | -7.3    | NaN     | -55.8         | 2469   | 0      |
| C-NLL     | notch+leadlag | 5.72   | 8.0     | 7.41    | -28.1         | 54     | 1      |

Si legge cosi': **tutti** i candidati che sopravvivono pagano il bending con il margine di fase rigido. Nessuno mantiene i 30 deg del Task 1. Ecco perche' serve lo step successivo.

**Possibile domanda d'esame** – Gain stabilisation o phase stabilisation: qual e' la differenza e perche' qui si sceglie la prima? *Risposta:* **Gain stabilisation** = si abbassa  $|L|$  sotto 0 dB alla frequenza del modo, cosi' la fase li' non conta piu': il lobo e' troppo piccolo per circondare il punto critico. **Phase stabilisation** = si lascia  $|L| > 0$  dB ma si ruota la fase in modo che il lobo passi dalla parte

sicura del punto critico. La seconda e' piu' efficiente (non serve un filtro profondo, quindi si paga meno fase in banda) ma e' **intrinsecamente fragile**: dipende dal conoscere la fase del modo, che dipende da  $\omega_{BM}$ , da  $\zeta_{BM}$ , dai coefficienti modali. La prima e' robusta in fase ma costosa in fase-in-banda. Qui si sceglie il **gain stabilisation** perche' il margine risultante (-18 dB finale) e' ampiamente sopra i 12 dB tipicamente richiesti e perche' la lead-lag phase-stabilizzante da sola lascia un delay margin di 15 ms, inaccettabile.

## %% Step C decision - deep notch, then RE-TUNE the PD (righe 131-161)

```
Wfull = Wtvc * Hn;
[Lb1, Tb1] = assemble_loop(Gfull, Ktask1, Wfull); % PRIMA
mB = classify_margins(Lb1, bands{:});
...
[K, mF] = design_controller(Gfull, Wfull, 'w_flex', 0.6*p.wBM, ...); % DOPO
```

### Prima del re-tune (righe 142-149)

Kp=1.784 Kd=0.443 | Aero |GM|=6.08 dB Rigid PM=14.6 deg @2.59 DM=98 ms

Il margine di fase rigido **crolla da 30.0 a 14.6 gradi** e il delay margin scende a 98 ms, cioe' **sotto** il requisito tipico di 100 ms citato nel Task 1.

### Da dove viene esattamente la fase persa

Il commento (righe 132-139) attribuisce il crollo a "the actuator + transport delay + notch phase lag". Ho misurato i tre contributi separatamente alla frequenza di crossover di quel caso ( $\omega_c = 2.59$  rad/s):

|                                                         |              |
|---------------------------------------------------------|--------------|
| TVC 2nd-order ( $\omega_{TVC}=70$ , $\zeta_{TVC}=0.7$ ) | : -2.97 deg  |
| Pade(20 ms, ordine 3)                                   | : -2.97 deg  |
| notch profondo ( $\zeta_D=0.7$ @ 18.9)                  | : -11.03 deg |
| -----                                                   |              |
| totale                                                  | : -16.97 deg |

**Il colpevole principale e' il notch, non l'attuatore.** Da solo vale 11 dei 17 gradi persi, cioe' i due terzi. Attuatore e ritardo insieme valgono appena 6 gradi. Il commento del sorgente elenca i tre in ordine, il che suggerisce che l'attuatore sia il problema; **quantitativamente non lo e'.**

Il motivo e' geometrico: il denominatore del notch e' un secondo ordine a  $\zeta_D = 0.7$  centrato a 18.9 rad/s, e un secondo ordine *inizia a ruotare la fase gia' una decade sotto* il suo centro. A 2.6 rad/s (una decade sotto) ha gia' speso 11 gradi. Un notch **piu' stretto in denominatore** ( $\zeta_D$  piu' basso) costerebbe meno fase in banda - ma renderebbe il null ancora piu' fragile al detuning.

### Il re-tune (righe 151-161)

PD design (full loop): Kp\_th=1.7318 Kd\_th=0.6867  
 Aero |GM|=6.00 dB @0.54 Rigid PM=30.0 deg @3.17 DM=165 ms | stable: 1  
 |L( $\omega_{BM}$ )| = -18.2 dB  
 Rigid GM = 7.56 dB @ 11.11 rad/s

- **Kp scende leggermente** (1.784 -> 1.732), **Kd sale del 55%** (0.443 -> 0.687). E' la mossa attesa: il termine derivativo  $Kd*s$  contribuisce fase **positiva** (+90 deg asintotici), quindi alzarlo e' il modo diretto di recuperare il ritardo di fase introdotto dal notch. Il crossover si sposta in alto (2.45 -> 3.17 rad/s) perche' l'anello ha piu' guadagno ad alta frequenza.

- **Il prezzo, che il codice non commenta:**  $|L(\omega_{BM})|$  peggiora da **-21.9 dB a -18.2 dB**. Alzando Kd si alza il guadagno d'anello *anche* a  $\omega_{BM}$  (il derivativo cresce con la frequenza), quindi il re-tune **baratta 3.7 dB di margine di bending per 15 gradi di margine di fase**. Resta comunque un margine di bending di 18 dB, sopra i 12 dB usualmente richiesti per un modo gain-stabilizzato.

## Deviazione dalla guideline della traccia

La traccia, al punto 2 delle guidelines, mette una nota a pie' di pagina esplicita: "*In most cases, retuning is not necessary.*" **Questo script ri-sintonizza comunque.** La deviazione e' giustificata dai numeri (PM 14.6 -> 30.0, DM 98 -> 165 ms), ma va dichiarata all'orale come **scelta deliberata contro il suggerimento**, non come applicazione della traccia. Nota inoltre che la nota della traccia si riferisce al punto 2 (TVC + ritardo, **senza notch**): e li' infatti il retune *non* serve (Aero 6.25 / PM 26.1). E' il **notch** ad aver reso il retune necessario, e il notch e' il punto 3.

## % Step D - sensitivity to the bending-frequency knowledge (righe 163-179)

```
scales = 0.90:0.05:1.10;
for i = [3 5]                                % C-N (retained) e C-NLL
    for sc = scales
        ps = load_hw3_params(); ps.wBM = sc*ps.wBM;
        Gs = build_plant_full(ps, 'ins');
        [~, Ts] = assemble_loop(Gs, K, Wtvc*cand{i,2});
        fprintf(' %5d', isstable(Ts));
```

I **filtri restano fissi** (progettati per  $\omega_{BM} = 18.9$ ), mentre il **modo vero** si sposta. E' il test di robustezza giusto: nella realta'  $\omega_{BM}$  e' noto con qualche percento di incertezza (dipende da riempimento serbatoi, temperatura, modellazione FEM).

Risultato reale:

| candidate |               | x0.90 | x0.95 | x1.00 | x1.05 | x1.10 |
|-----------|---------------|-------|-------|-------|-------|-------|
| C-N       | deep notch    | 1     | 1     | 1     | 0     | 0     |
| C-NLL     | notch+leadlag | 1     | 1     | 1     | 1     | 1     |

Il notch profondo **tollera -10% ma va instabile gia' a +5%**. La combinazione notch+lead-lag tiene tutta la banda.

## La spiegazione del README e' sbagliata – ecco quella corretta

Il README attribuisce l'asimmetria a "the asymmetric edge of its 0.08 rad/s null", cioe' alla forma del null di attenuazione. **Non regge**, e si verifica in due righe. L'attenuazione del notch al modo spostato:

```
|Hn| a 0.90*wBM = -16.5 dB
|Hn| a 1.05*wBM = -23.1 dB    <-- attenua DI PIU' a +5% che a -10%
|Hn| a 1.10*wBM = -17.4 dB
```

Il notch attenua **di piu'** a +5% che a -10%, eppure e' -10% a sopravvivere. L'attenuazione non puo' essere la causa. Misurando il guadagno d'anello al modo vero:

| sc   | wBM_true | Hn(wBM_true) | L(wBM_true) | stabile |
|------|----------|--------------|-------------|---------|
| 0.90 | 17.01    | -16.5 dB     | +17.1 dB    | si      |
| 0.95 | 17.95    | -22.7 dB     | +10.4 dB    | si      |
| 1.00 | 18.90    | -50.9 dB     | -18.2 dB    | si      |
| 1.05 | 19.84    | -23.1 dB     | +9.1 dB     | NO      |
| 1.10 | 20.79    | -17.4 dB     | +14.4 dB    | NO      |

Il fatto decisivo: **appena il modo si sposta, il lobo di bending torna sopra 0 dB in tutti i casi** (da +9 a +17 dB). La gain stabilisation **evapora completamente** con qualunque detuning: il null e' largo pochi centesimi di rad/s, il modo esce e la risonanza da +29 dB riemerge quasi intera. Da quel momento in poi la sopravvivenza dipende **solo dalla fase** del lobo, cioe' da quale lato del punto critico passa.

E la fase e' asimmetrica attorno al notch: **sotto** il centro il notch contribuisce una rotazione favorevole, **sopra** una sfavorevole. Ecco perche' a +5% l'anello cade pur avendo *meno* guadagno al modo (9.1 dB) di quanto ne abbia a -5%... anzi, di quanto ne abbia a 0.95 (10.4 dB), che invece e' stabile. E' una dimostrazione pulita che qui **conta la fase, non il modulo**.

**Conclusione onesta:** il progetto ritenuto e' gain-stabilizzato **solo al punto nominale**. Fuori dal nominale sopravvive per fortuna (phase stabilisation accidentale), e da un lato solo. E' il vero punto debole del Task 2, ed e' anche il motivo per cui la variante C-NLL, pur avendo margini nominali peggiori, e' l'unica robusta.

## Due incoerenze minori in Step D

1. **I guadagni usati sono quelli RI-SINTONIZZATI.** A riga 153 la variabile K viene **sovrascritta** con il design del Task 2. Step D (riga 175) usa quindi il K nuovo. Ma il partner lead-lag di C-NLL (**bestC**, righe 88-102) era stato **selezionato con i guadagni del Task 1**. Step D confronta quindi un filtro ottimizzato per un set di guadagni con un set di guadagni diverso.
2. **La tabella del trade (righe 122-129) usa i guadagni del Task 1**, mentre il progetto finale usa quelli ri-sintonizzati. La riga "C-N deep notch" della tabella (PM 14.6) **non e' il progetto finale** (PM 30.0). Lo script lo dice ("BEFORE re-tuning"), quindi non e' fuorviante – ma le righe C-LL, C-T e C-NLL non vengono mai ri-valutate con i guadagni finali. Se all'orale si mostra la tabella, va detto che e' una **fotografia a guadagni Task-1**.

---

## % Time response to a wind gust (righe 181-189)

Stessa raffica del Task 1 (Tend=80, 1-cosine severa, Vg = 6.38 m/s). Risultati:

```
peak |theta| = 0.231 deg, |z| = 2.27 m, |delta| = 0.513 deg
peak |alpha| = 0.565 deg -> peak qbar*alpha = 45.8 kPa deg
```

Praticamente **identici al Task 1** (theta 0.261 / z 2.27 / delta 0.528, e alpha 0.577 deg -> 46.8 kPa\*deg). Ed e' il risultato giusto: bending, attuatore e ritardo vivono tutti a frequenze molto sopra la banda della raffica (la raffica dura 3 s, cioe' contenuto spettrale sotto ~2 rad/s). La risposta temporale a bassa frequenza **non li vede**. E' esattamente per questo che il bending e' un problema di **stabilita'** e non di **prestazione**: non lo trovi guardando le time history, lo trovi solo sul Nichols.

**Il segno di alpha\_w (righe 188-189).** L'incidenza ricostruita da `simulate_gust_response.m` (riga 29) e'

```
alpha = theta + zdot/V - alpha_w
```

con il **meno**, che e' l'unico segno coerente con l'Eq. (1): la colonna del vento del plant e'  $B_w = [0; -a1*V; 0; -A6; 0; 0]$  (`build_plant_full.m` riga 28), cioe' i coefficienti di **theta** cambiati di segno – il che e' la definizione  $\alpha = \theta + \dot{z}/V - \alpha_w$ . Fisicamente: l'incidenza si misura rispetto alla velocita' **relativa all'aria**.

La conseguenza e' un punto d'orale: il picco di incidenza (**0.565 deg**) **supera** quello del vento da solo (0.390 deg). Per tenere l'assetto il loop becca il muso **dentro** il vento relativo, e quel contributo si **somma** a quello del vento. Un puro attitude-hold e' quindi **load-aggravating**, non load-relieving – e con  $A6 > 0$  (centro di pressione davanti al baricentro, momento aerodinamico divergente) non esiste nessuna stabilita' a banderuola che allevi il carico da sola.

**Nota storica.** Fino a poco fa `simulate_gust_response.m` usava il segno **piu'**, dando un picco di 0.253 deg (20.5 kPa\*deg) – sotto\* il vento da solo, e quindi un'apparente azione di load relief. Il plant e' sempre stato corretto: il difetto stava solo nella ricostruzione a valle di **alpha**. La correzione **non**

tocca margini, Nichols, notch, Step C/D o stabilita' ( $\alpha_w$  e' un disturbo, non entra in  $L(s)$ ), e nemmeno i picchi di  $\theta$ ,  $z$ ,  $\delta$ : cambia solo la metrica di carico.

---

## %% Figures (righe 191-267)

Cinque figure.

- **f1 nichols** (righe 196-200): il progetto ritenuto, via `plot_nichols_lv`. Convenzione degli appunti del corso: punto critico a **(-180 deg, 0 dB)**, da  $1 + L = 0$ , finestra `xlim [-720 0]`. (D'Antuono Fig. 3.2 mostra la stessa carta rietichettata di +360; il codice usava quella rietichettatura fino a poco fa ed e' stato riallineato al corso – pura rietichettatura dell'asse, margini invariati.)
- **f5 retune** (righe 205-214): **la figura piu' importante del task**. Sovrappone l'anello con i guadagni Task-1 ( $L_{b1}$ ) e quello ri-sintonizzato ( $L_c$ ), con `PhaseMatching` agganciato alla frequenza di crossover del design finale e `PhaseMatchingValue = -180` (riga 208) – stessa convenzione del corso. Mostra visivamente il recupero del margine di fase.
- **f4 nichols\_trade** (righe 222-243): il trade a tre (nessun filtro / lead-lag / notch profondo), disegnato **a mano** invece che con `nicholsplot`, perche' serve applicare a tutte e tre le curve **lo stesso** shift di fase. La riga 226:

```
sh0 = 360*round((-180 - interp1(wv, ph0, mB.rigidPM_w))/360);
```

Lo shift e' **un multiplo esatto di 360 deg** -> e' una rietichettatura del ramo di fase, non una rotazione. Le tre curve restano confrontabili: le regioni rigide si sovrappongono attorno a -180 deg (e' quanto dichiara il commento riscritto alle righe 217-221) e i lobi di bending sono direttamente confrontabili sullo stesso foglio. Il punto critico e' marcato con una croce rossa a -180 (riga 239); il gemello a +180 di riga 238 cade fuori dalla finestra `xlim [-720 0]` di riga 240. Bonus della convenzione: la griglia M/N di `ngrid` (riga 223) vive nativamente in  $[-360, 0]$ , quindi ora la regione rigida della figura casca **dentro** la griglia (con la vecchia rietichettatura a +180 finiva fuori). Stesso trucco in `plot_nichols_lv` (riga 49) e in `main_task3.m` (riga 81).

- **f2 gust\_response** (righe 246-256):  $\theta$ ,  $z$ ,  $\dot{z}$ ,  $\delta$  – le quattro variabili chieste dalla traccia.
  - **f3 comparison\_rigid\_vs\_full** (righe 259-267): rigido vs completo, che e' il “compare the results” richiesto dalla traccia.
  - Export (righe 269-280): identico al Task 1, PNG a 200 dpi in `HM3/figures/`, prefisso `task2_`.
- 

## Possibili domande d'esame

**D: Perche' il modo di bending destabilizza l'anello? Descrivi il percorso.** R: E' un anello parassita chiuso attorno alla risonanza. In avanti, il TVC forza il modo: nella riga `eta_ddot` del plant compare il termine  $-\phi_{tvc}T\delta$ . All'indietro, la piattaforma inerziale non misura l'assetto vero ma  $\theta_m = \theta + \sigma_{ins}\eta$  (Eq. 2,  $\sigma_{ins} = 0.178$  rad/m): il bending **rientra nel controllore**, che lo riampifica e lo rimanda al TVC. Con  $\zeta_{BM} = 0.005$  il Q vale 100 (+40 dB di risonanza), e il guadagno d'anello a  $\omega_{BM}$  risulta **+29 dB**: molto sopra 0 dB, con fase non controllata -> encirclement sbagliato -> instabilita' (polo a  $\text{Re} = +1.61$ ). Se si usasse la misura pulita (`build_plant_full(p, 'true')`) il problema sparirebbe: e' la **contaminazione della misura**, non il modo in se', a chiudere l'anello.

**D: Perche' non basta la lead-lag di Eq. (4), che pure e' quella suggerita dalla traccia?** R: Perche' la sua attenuazione al centro vale  $zN/zD$ , cioe' al massimo  $20*\log_{10}(0.1/0.6) = -15.6$  dB dentro i range suggeriti – contro una risonanza di +29 dB. Non puo' gain-stabilizzare. Puo' solo **phase-stabilizzare**: con  $\text{sgn} = -1$  gli zeri sono a destra, il filtro ruota la fase senza abbassare il modulo, ed e' il “left-right shift” di cui parla la traccia. Nel run 45/75 candidati effettivamente stabilizzano cosi'. Ma il migliore lascia  $|L(\omega_{BM})| = +23$  dB e un **delay**

**margin di 15 ms**, cioè inferiore ai 20 ms di ritardo già presenti nel modello. Non è un progetto difendibile. (Nota: il commento nel sorgente dice che la lead-lag “non stabilizza mai” – è sbagliato, 45 su 75 stabilizzano.)

**D: La tripletta di notch attenua il bending di -56 dB. Perché è il candidato peggiore?** R: Perché è **instabile**: Rigid PM = -7.3 deg. Ogni notch spende fase alle frequenze *sotto* il proprio centro (il denominatore è un secondo ordine che inizia a ruotare già una decade prima), e tre notch impilati spendono circa il triplo. Al crossover rigido (~2.5 rad/s) si mangiano ~30 gradi, cioè tutto il margine di fase disponibile. È la dimostrazione che **i notch non sono gratis**: si compra margine di bending pagando in margine di fase rigido, e la tripletta spende più di quanto si possa permettere. Attenzione anche al DM = 2469 ms stampato su quella riga: è un numero senza senso, perché **allmargin** calcola margini anche su anelli instabili. L'unica colonna che decide è **stable**.

**D: Il re-tune del PD contraddice la traccia, che dice “in most cases retuning is not necessary”. Come lo difendi?** R: Distinguendo i due punti della guideline. Al **punto 2** (TVC + ritardo, senza notch) il retune davvero **non serve**: i margini restano 6.25 dB / 26.1 deg, praticamente quelli del Task 1, perché wTVC = 70 rad/s è oltre una decade sopra il crossover e 20 ms di ritardo a 2.5 rad/s costano solo 3 gradi. È il **punto 3**, cioè l'inserimento del **notch**, a rendere necessario il retune: il notch profondo costa da solo **11 gradi** di fase al crossover (misurati), e insieme a attuatore e ritardo porta il margine di fase da 30 a 14.6 gradi, con delay margin sotto i 100 ms. Alzando Kd del 55% (0.443 → 0.687) si recuperano i 30 gradi. La nota della traccia si riferisce all'attuatore, non al filtro.

**D: Il notch profondo tollera -10% di detuning su omega\_BM ma cade a +5%. Perché questa asimmetria?** R: Non è una questione di profondità del null, come si potrebbe pensare (e come sostiene il README): il notch attenua **di più** a +5% (-23.1 dB) che a -10% (-16.5 dB). Il punto è che **appena il modo si sposta, la gain stabilisation evapora del tutto**: il lobo di bending torna sopra 0 dB in *tutti* i casi detunati (da +9 a +17 dB di guadagno d'anello al modo). Il null è largo pochi centesimi di rad/s e il modo ne esce subito. Da lì in poi la stabilità dipende **solo dalla fase** del lobo, cioè da che lato del punto critico passa – e la fase del notch è asimmetrica attorno al proprio centro (favorevole sotto, sfavorevole sopra). Prova decisiva: a  $0.95\omega_{BM}$  il guadagno al modo è **10.4 dB** e l'anello è **stabile**; a  $1.05\omega_{BM}$  è **9.1 dB** (cioè **meno**) e l'anello è **instabile**. Il modulo non spiega nulla, la fase spiega tutto.

**D: Perché la risposta alla raffica del Task 2 è praticamente identica a quella del Task 1?** R: Perché bending (18.9 rad/s), attuatore (70 rad/s) e ritardo (20 ms) vivono tutti **molto sopra** la banda del disturbo: la raffica 1-cosine dura 3 s, quindi ha contenuto spettrale sotto ~2 rad/s. La risposta temporale a bassa frequenza semplicemente non li vede. È il motivo per cui il bending è un problema di **stabilità** e non di **prestazione**: non lo si scopre guardando le time history, lo si scopre solo sul Nichols. Chiunque validasse il Task 2 solo con **lsim** non si accorgerebbe di nulla.

**D: Perché Pade di ordine 3 e non 1?** R: Perché l'approssimante deve essere fedele **fino a omega\_BM = 18.9 rad/s**, non solo al crossover rigido. Pade di ordine n riproduce bene la fase di  $\exp(-\tau s)$  fino a circa  $n/\tau$ : con  $\tau = 20$  ms, l'ordine 1 è affidabile fino a ~50 rad/s in modulo ma sbaglia già la fase molto prima, mentre l'ordine 3 copre bene fino a ~150 rad/s. Dato che tutta la decisione sul filtro si gioca sulla **fase a omega\_BM**, un Pade grossolano falserebbe proprio il numero che conta.

# HM3/main\_\_task3.m

## Ruolo del file nel progetto

E' il **terzo entry point** di HM3 e copre il Task 3 della traccia ("Robust Control Design", dichiarato *optional*). La domanda e' precisa: il coefficiente di momento aerodinamico **`mu_alpha = A_6`** e l'efficacia di controllo **`mu_c = K_1`** sono incerti; si valuti la robustezza del **controllore gia' progettato** (quindi **senza ri-sintonizzarlo**) sui **quattro casi d'angolo** ottenuti variando indipendentemente i due parametri di  $\pm 30\%$ .

Lo script e' concettualmente il piu' semplice dei tre – e' un ciclo su nove casi – ma e' quello che produce il messaggio ingegneristico piu' forte dell'homework: **il vincolo che stringe non e' il margine di fase, e' il margine aerodinamico**, e il vertice peggiore V3 lo porta a **0.91 dB**, cioe' a un capello dalla perdita di stabilita'.

Struttura:

1. Ricostruisce il controllore FISSO del Task 2 (PD ri-sintonizzato + notch)
2. Cicla su 9 casi: Nominale + 4 vertici V1-V4 + 4 sensibilita' S1-S4
3. Per ognuno: margini classificati per banda + risposta alla stessa raffica
4. Due figure: overlay Nichols sui vertici, overlay risposta alla raffica

La distinzione **V** vs **S** e' il punto metodologico:

- I **V** sono i **vertici della scatola di incertezza** – le combinazioni  $(\mu\_alpha, \mu\_c) = (0.7, 0.7), (0.7, 1.3), (1.3, 0.7), (1.3, 1.3)$ . Sono i quattro corner cases che la traccia chiede letteralmente.
- Gli **S** sono **sensibilita' one-at-a-time**: si muove un parametro alla volta tenendo l'altro al nominale. Sono i **punti medi degli spigoli**, non i vertici. Il commento di header (righe 5-7) dice che "miss the worst combo (V3)" – vero nel senso che nessuna singola riga S e' cattiva quanto V3. Ma come mostro sotto, le perdite in dB degli S **si sommano quasi esattamente** a quella di V3, quindi gli S sono in realta' un ottimo predittore lineare dei vertici.

---

## %% Fixed controller (Task 2 retained design) (righe 16-26)

```
p0      = load_hw3_params();
notch   = struct('wx',p0.wBM,'zN',0.002,'zD',0.7,'sgn',+1);
Gfull0  = build_plant_full(p0,'ins');
Wact0   = build_tvc(p0,3) * build_notch_filter(notch.wx,notch.zN,notch.zD,notch.sgn);
K = design_controller(Gfull0, Wact0, 'w_flex',0.6*p0.wBM, 'w_flex_hi',1.5*p0.wBM, ...
    'w_bending',p0.wBM, 'verbose',false);
```

- Righe 19-24: lo script **non hard-codifica**  $K_p = 1.732$  /  $K_d = 0.687$ : li **ri-deriva** rilanciando `design_controller` sull'anello completo nominale, esattamente come fa `main_task2.m` alla riga 153. Essendo `fminsearch` deterministico da un seed fisso, riproduce gli stessi valori (verificato:  $K_{p\_th} = 1.732$ ,  $K_{d\_th} = 0.687$ ). Costa un run del tuner ma rende lo script **autosufficiente**: puo' essere lanciato da solo, senza aver prima eseguito `main_task2`. E' la scelta giusta per uno script che il professore fara' girare isolatamente.

- Riga 20: il notch e' **ricopiato a mano** come struct letterale ( $zN = 0.002$ ,  $zD = 0.7$ ,  $sgn = +1$ ) invece di essere importato da un file di configurazione condiviso. **Limite di manutenibilita' da segnalare:** se si cambiasse il notch in `main_task2.m`, il Task 3 continuerebbe a usare quello vecchio senza avvisare. Non c'e' una singola sorgente di verita' per il progetto del filtro.
- Riga 25: la stampa conferma che il controllore congelato e' quello del **Task 2 ri-sintonizzato**, non quello del Task 1. E' importante: usare i guadagni del Task 1 ( $Kd = 0.443$ ) darebbe un margine di fase nominale di 14.6 gradi, e la campagna di robustezza partirebbe gia' in deficit.

## %% Cases: box vertices + one-at-a-time sensitivities (righe 28-67)

```
cases = {
    'Nominal', 1.00, 1.00
    'V1',      0.70, 0.70      % vertici della scatola
    'V2',      0.70, 1.30
    'V3',      1.30, 0.70      % peggiore: max instabilita', min autorita'
    'V4',      1.30, 1.30
    'S1',      0.70, 1.00      % sensibilita' one-at-a-time (extra)
    'S2',      1.30, 1.00
    'S3',      1.00, 0.70
    'S4',      1.00, 1.30 };
```

- Riga 41: `nPlot = 5` -> solo **Nominale + V1..V4** vengono disegnati. Gli S sono calcolati e tabulati ma non compaiono in nessuna figura.
- Riga 43: **la stessa raffica per tutti i casi** (`load_wind_profile(p0, ...)`, costruita su `p0` nominale). Corretto: il disturbo e' un dato d'ambiente, non deve cambiare con l'incertezza del veicolo. `Vg` dipende solo da quota e severita'.

## Il ciclo (righe 48-65)

```
p = load_hw3_params('mu_alpha_scale',cases{i,2}, 'mu_c_scale',cases{i,3});
Gf = build_plant_full(p,'ins');
Wf = build_tvc(p,3) * build_notch_filter(notch.wx,notch.zN,notch.zD,notch.sgn);
[L{i}, T] = assemble_loop(Gf, K, Wf);
mm{i} = classify_margins(L{i}, 'w_drift',0.3*sqrt(p.A6), ...);
r = simulate_gust_response(T, w);
```

- Riga 49: lo scaling e' applicato **dentro** `load_hw3_params` (righe 89-90 di quel file):  $p.A6 = p.A6 * \mu\_alpha\_scale$ ,  $p.K1 = p.K1 * \mu\_c\_scale$ . **Nient' altro viene scalato:**  $a1$ ,  $a3$ ,  $a4$ ,  $V$ ,  $\omega_{BM}$ ,  $\sigma_{ins}$ ,  $\phi_{tvc}$  restano nominali.

**Osservazione fisica onesta:**  $A_6 = N\_alpha \cdot l\_alpha / I\_yy$  e  $a1 = -N\_alpha / (mV)$  **condividono**  $N\_alpha$ . Una dispersione fisica reale del +30% sulla derivata di portanza muoverebbe **entrambi**. Il codice li disaccoppia. Non e' un errore: e' letteralmente cio' che la traccia chiede ("mu\_alpha = A6 and mu\_c = K1 are uncertain"), quindi l'incertezza e' definita **sui coefficienti**, non sulle grandezze fisiche a monte. Ma va detto: e' un'astrazione parametrica, non una dispersione fisica.

- Riga 51: `Wf` viene **ricostruito ad ogni iterazione** ma e' **identico in tutti i casi**: `build_tvc` dipende solo da  $w_{TVC}/z_{TVC}/\tau$  (non scalati) e il notch e' centrato su  $notch.wx = p0.w_{BM}$  (nominale). E' una **ridondanza innocua** ma merita di essere capita: significa che **il notch resta centrato sul nominale**, che e' proprio il senso di "controllore fisso".
- Righe 56-57: `classify_margins` riceve  $w\_drift = 0.3 * \sqrt{p.A6}$  **calcolato sul corner corrente**. Sensato: il confine fra artefatti di deriva e corpo rigido si sposta con l'instabilita'. Le altre bande (`w_flex`, `w_flex_hi`, `w_bending`) usano  $p.w_{BM}$ , che **non e' scalato**, quindi sono le stesse in tutti i casi.



## Risultati reali (run completo)

| Case    | mu_a | mu_c | AeroGM | RigidPM | RigidGM | DM[ms] | peakTh | peakZ | stab |
|---------|------|------|--------|---------|---------|--------|--------|-------|------|
| Nominal | 1.00 | 1.00 | 6.00   | 30.0    | 7.56    | 165    | 0.231  | 2.27  | 1    |
| V1      | 0.70 | 0.70 | 5.70   | 26.6    | 8.70    | 196    | 0.241  | 2.28  | 1    |
| V2      | 0.70 | 1.30 | 11.06  | 30.8    | 6.50    | 121    | 0.097  | 1.57  | 1    |
| V3      | 1.30 | 0.70 | 0.91   | 18.0    | 8.75    | 209    | 0.878  | 4.92  | 1    |
| V4      | 1.30 | 1.30 | 6.17   | 30.9    | 6.57    | 135    | 0.222  | 2.26  | 1    |
| S1      | 0.70 | 1.00 | 8.79   | 30.6    | 7.53    | 156    | 0.142  | 1.80  | 1    |
| S2      | 1.30 | 1.00 | 3.93   | 28.9    | 7.59    | 176    | 0.346  | 2.83  | 1    |
| S3      | 1.00 | 0.70 | 2.95   | 23.6    | 8.72    | 206    | 0.417  | 3.19  | 1    |
| S4      | 1.00 | 1.30 | 8.26   | 30.9    | 6.54    | 128    | 0.154  | 1.90  | 1    |

**Tutti e nove i casi restano stabili.** Il controllore fisso del Task 2 sopravvive alla scatola +/-30%. Ma i margini si muovono moltissimo.

## La matematica dietro i numeri (derivazione, non nel codice)

### Perche' mu\_c muove il margine di guadagno di esattamente $20 \cdot \log_{10}(\mu_c)$

K\_1 compare **solo nella matrice B** (colonna di delta, riga di thetadot\_dot). Non tocca la matrice A, quindi **non sposta i poli del plant**. Scalare K\_1 per mu\_c scala quasi esattamente il guadagno dell'intero anello aperto L. E un margine di guadagno, misurato in dB, trasla di conseguenza:

$$\Delta(\text{AeroGM}_{\text{dB}}) = 20 \cdot \log_{10}(\mu_c)$$

Verifica sui dati:

S4 (mu\_c = 1.30): previsto  $20 \cdot \log_{10}(1.30) = +2.28$  dB | misurato +2.26 dB  
 S3 (mu\_c = 0.70): previsto  $20 \cdot \log_{10}(0.70) = -3.10$  dB | misurato -3.05 dB

Accordo quasi perfetto. (Non e' esatto al 100% perche' la colonna di delta (build\_plant\_full.m, riga 27: Bd = [0; a3; 0; K1; 0; -phi\_tvc\*Tc]) contiene anche a3 sulla riga di zdot e -phi\_tvc\*Tc sulla riga di etadot - la sesta, cioe' l'equazione del modo flessibile - e **nessuno dei due viene scalato**: solo il canale di assetto scala. Ma quel canale domina.)

### Perche' mu\_alpha e' diverso

A\_6 compare nella **matrice A** (riga di thetadot\_dot) e nella colonna del disturbo. Sclarlo **sposta i poli**: il polo instabile passa da  $+\sqrt{A_6} = 1.84$  a  $+\sqrt{1.3A_6} = 2.10$  rad/s. Non e' quindi una pura traslazione di guadagno, e infatti l'accordo con  $-20 \log_{10}(\mu_\alpha)$  e' solo approssimativo:

S2 (mu\_alpha = 1.30):  $-20 \cdot \log_{10}(1.30) = -2.28$  dB previsti | -2.07 dB misurati  
 S1 (mu\_alpha = 0.70):  $-20 \cdot \log_{10}(0.70) = +3.10$  dB previsti | +2.79 dB misurati

## La scoperta: le perdite in dB si sommano

Se i due effetti fossero indipendenti e additivi in dB, il margine di un vertice sarebbe prevedibile dagli spigoli:

$$\text{AeroGM}(V) \sim \text{AeroGM}(\text{nom}) + \Delta_S(\mu_\alpha) + \Delta_S(\mu_c)$$

Verifica su tutti e quattro i vertici (Delta\_S presi dalle righe S misurate):

V1 (0.7, 0.7):  $6.00 + 2.79 - 3.05 = 5.74$  | misurato 5.70  
 V2 (0.7, 1.3):  $6.00 + 2.79 + 2.26 = 11.05$  | misurato 11.06  
 V3 (1.3, 0.7):  $6.00 - 2.07 - 3.05 = 0.88$  | misurato 0.91  
 V4 (1.3, 1.3):  $6.00 - 2.07 + 2.26 = 6.19$  | misurato 6.17

**Accordo entro 0.05 dB su tutti e quattro.** Il margine aerodinamico e' essenzialmente **separabile e additivo in dB** nei due scaling logaritmici. Sono tre conseguenze importanti:

1. **Il vertice peggiore e' prevedibile:** V3 e' il corner in cui entrambe le perdite sono negative. Non c'e' bisogno di cercarlo, si sa in anticipo che sara' ( $\mu_{\alpha}$  alto,  $\mu_c$  basso).
2. **Gli S non "mancano" V3** come dice il commento di header: lo **predicono** con precisione, semplicemente vanno **sommati**. Il commento e' vero solo in senso letterale (nessuna singola riga S vale 0.91 dB), ma fuorviante sul metodo.
3. **La ricerca sui vertici e' giustificata a posteriori:** se la dipendenza e' monotona in entrambi i parametri, il peggior caso su una scatola sta necessariamente su un vertice. **Il codice pero' non lo dimostra** – campiona 4 vertici e 4 spigoli, e da questo non si puo' escludere formalmente un caso interno peggiore. La quasi-additivita' osservata lo rende molto improbabile, ma e' un argomento empirico, non una prova. Una vera prova richiederebbe un'analisi mu (structured singular value), che non e' nel codice.

## Perche' $\mu_c$ alto peggiora invece Rigid GM e DM

Guardando le colonne RigidGM e DM:  $\mu_c = 1.30$  (V2, V4, S4) le **abbassa** (6.50-6.57 dB e 121-135 ms contro i 7.56 dB / 165 ms nominali), mentre  $\mu_c = 0.70$  (V1, V3, S3) le **alza** (8.70-8.75 dB, 196-209 ms). E' il rovescio esatto della medaglia: il Rigid GM e' un margine di **aumento** di guadagno, quindi piu' guadagno d'anello ( $\mu_c$  alto) lo **erode**; l'Aero GM e' un margine di **riduzione**, quindi piu' guadagno lo **migliora**.

**Ecco il vero trade-off di HM3:** i due margini si muovono in **direzioni opposte** rispetto a  $\mu_c$ . Non esiste un valore di guadagno che li massimizzi entrambi – e' esattamente la firma della **stabilita' condizionale**. Il progetto sta in mezzo alla banda ammessa, e l'incertezza lo spinge verso un bordo o verso l'altro.

## Perche' V3 amplifica anche la risposta alla raffica

|             |                 |    |                |   |      |
|-------------|-----------------|----|----------------|---|------|
| peak theta: | 0.231 deg (nom) | -> | 0.878 deg (V3) | = | 3.8x |
| peak z:     | 2.27 m          | -> | 4.92 m         | = | 2.2x |

L'ampiezza dell'eccitazione da vento e' proporzionale ad **A\_6** (il termine  $-A_6 \alpha_w$  nella colonna del disturbo), mentre la capacita' di reagire e' proporzionale a **K\_1**. V3 alza il primo del 30% e abbassa il secondo del 30%: il rapporto  $A_6/K_1$  passa da 0.74 a **1.37**, cioe' **+85%**. La risposta cresce di conseguenza. Il caso opposto V2 ( $\mu_{\alpha}$  basso,  $\mu_c$  alto) porta il picco a 0.097 deg, cioe' a meno della meta' del nominale.

## %% Figures (righe 69-105)

### f1 – Nichols overlay sui corner (righe 77-92)

- Riga 79: `wv = logspace(-2, log10(30), 3000)` – la griglia arriva **solo a 30 rad/s**.
- Riga 81: `sh0 = 360*round((-180 - interp1(wv, ph1, mm{1}.rigidPM_w))/360)` – lo **stesso** shift di fase (derivato dal caso nominale) e' applicato a **tutte** le curve, e ancora il crossover nominale sul ramo che contiene il punto critico a **-180 deg** (convenzione del corso, da  $1 + L = 0$ ; il commento riscritto alle righe 71-76 la dichiara). Essendo un multiplo esatto di 360 deg e' una rietichettatura del ramo, non una rotazione: la dispersione fra i corner resta confrontabile. Stesso trucco di `plot_nichols_lv` (riga 49) e di `main_task2.m` (riga 226). (Nota storica: fino a poco fa lo shift era riferito a +180, la rietichettatura D'Antuono della stessa carta spostata di +360; riallineato al corso, margini invariati.)
- Riga 88: `plot(ax, -180, 0, 'r+')` – il punto critico e' marcato esplicitamente a (-180, 0 dB).
- Riga 89: `xlim [-270 -90], ylim [-15 20]` – **zoom sulla sola regione rigida**, centrato sul punto critico a -180 e tutto dentro il foglio nativo [-360, 0] della griglia di `ngrid` (riga 78).

**Conseguenza da conoscere:** con `ylim` a [-15 20], il **lobo di bending non e' visibile** (sta a -18 dB). La figura non dice nulla sulla robustezza del modo flessibile. E' pero' una scelta **difendibile**, e vale la pena saper spiegare perche': lo scaling di  $\mu_{\alpha}$  e  $\mu_c$  **quasi non tocca il bending**. Il modo e' forzato da  $-\phi_{tvc} T_c$  (non scalato) e rientra via  $\sigma_{ins}$  (non scalato);  $\omega_{BM}$  e  $\zeta_{BM}$  non sono scalati.  $A_6$  e  $K_1$  muovono solo il **canale rigido**, che a 18.9 rad/s e' gia' rotolato via e conta poco contro la risonanza. Quindi  $|L(\omega_{BM})|$  e' praticamente lo stesso in tutti i corner, e non c'e' niente da

vedere. Il posto dove il bending **viene** perturbato e' `main_montecarlo.m`, che disperde anche `omega_BM`, `zeta_BM` e il ritardo.

- La curva di **V3** e' quella che riga piu' vicino al punto critico a -180 deg: 0.91 dB di margine sono, sul grafico, un capello.

## f2 – Gust response overlay (righe 95-105)

Due pannelli (theta e z) con Nominale + V1..V4 sovrapposti. Mostra visivamente l'escursione di V3 (0.88 deg / 4.9 m) contro il resto del gruppo.

## Export (righe 107-118)

Identico agli altri due script: `theme(f, 'light')` in `try/catch`, PNG a 200 dpi in `HM3/figures/`, prefisso `task3_`.

---

## Cosa il Task 3 NON copre (limiti dichiarati)

- **Nessuna incertezza su `omega_BM`, `zeta_BM` o sul ritardo.** Eppure lo Step D del Task 2 mostra che il notch profondo va **instabile a +5% su `omega_BM`**. Il Task 3 potrebbe quindi dare un falso senso di sicurezza: dice che il controllore e' robusto a +/-30% sui due coefficienti aerodinamici, **ma il vero punto fragile del progetto e' altrove**. La combinazione (incertezza parametrica) x (detuning del bending) e' esplorata solo in `main_montecarlo.m`, che non e' un deliverable della traccia.
  - **Nessuna prova formale che il caso peggiore stia su un vertice.** Si campionano 4 vertici e 4 spigoli. La quasi-additivita' in dB (verificata sopra a 0.05 dB) e' un forte indizio di monotonia, ma non e' una dimostrazione.
  - **Nessun gain scheduling.** Il messaggio implicito del task (V3 a 0.91 dB e' troppo poco per un progetto di volo) e' che servirebbe schedare i guadagni; lo script non lo fa. Lo fa l'estensione `LTV_FULL_ASCENT/`, fuori traccia.
- 

## Possibili domande d'esame

**D: Perche' il vertice V3 e' il peggiore, e si poteva prevederlo senza simulare?** R: Si', si poteva. V3 e' ( $\mu_{\alpha} = 1.3$ ,  $\mu_c = 0.7$ ): **velivolo piu' instabile** ( $A_6$  alto -> polo aerodinamico da 1.84 a 2.10 rad/s) e **meno autorita' di controllo** ( $K_1$  basso -> meno guadagno d'anello). Il margine aerodinamico e' un margine di **riduzione** di guadagno: misura quanto guadagno puoi togliere prima di non dominare piu' l'instabilita'. V3 aumenta cio' che va dominato e diminuisce cio' con cui lo domini: i due effetti si **sommano**, e in dB lo fanno quasi esattamente. Numericamente: la riga S2 (solo  $\mu_{\alpha}$  alto) costa 2.07 dB, la riga S3 (solo  $\mu_c$  basso) costa 3.05 dB; la somma e' 5.12 dB, e V3 perde 5.09 dB (6.00 -> 0.91). Coincidenza entro 0.03 dB.

**D: Il commento nel codice dice che le sensibilita' S1-S4 "mancano" il caso peggiore V3. E' vero?** R: Letteralmente si', ma metodologicamente e' fuorviante. E' vero che **nessuna singola riga S** e' cattiva quanto V3 (la peggiore, S3, si ferma a 2.95 dB contro 0.91). Ma le **perdite in dB delle righe S si sommano** a quella di V3 con un errore di 0.03 dB, e lo stesso vale per tutti e quattro i vertici (accordo entro 0.05 dB). Quindi gli S non mancano affatto il vertice: lo **predicono** in modo quasi esatto, purché si sommino invece di leggerli isolati. La ragione e' che l'Aero GM in dB e' **separabile** nei due scaling logaritmici – perche'  $K_1$  e' un puro guadagno d'anello ( $\Delta_{dB} = 20 \cdot \log_{10}(\mu_c)$ , verificato: -3.05 dB misurati contro -3.10 previsti per  $\mu_c = 0.7$ ).

**D: Perche'  $\mu_c$  alto migliora il margine aerodinamico ma peggiora quello rigido?** R: Perche' sono margini di **segno opposto**. L'anello e' **condizionalmente stabile**: c'e' una banda di guadagni ammessi, con un bordo sotto e uno sopra. L'Aero GM misura la distanza dal **bordo inferiore** (quanto guadagno puoi togliere), il Rigid GM dal **bordo superiore** (quanto puoi aggiungerne). Alzare  $\mu_c$  alza il guadagno d'anello: ti allontani dal bordo inferiore (Aero GM: 6.00 -> 8.26 dB) e ti avvicini a quello superiore (Rigid GM: 7.56 -> 6.54 dB; DM:

165 -> 128 ms). E' il trade-off strutturale di tutto HM3: **non esiste un guadagno che massimizzi entrambi**, si sta in mezzo.

**D: Perche' la figura di Nichols del Task 3 non mostra il modo di bending?** R: Perche' e' zoomata su  $y_{lim} = [-15 \ 20]$  dB e il lobo di bending sta a -18 dB. Ma la scelta e' difendibile: **lo scaling di  $\mu_{\alpha}$  e  $\mu_c$  non tocca praticamente il bending**. Il modo e' forzato dal termine  $-\phi_{tvc} T_{\delta}$  e rientra nella retroazione via  $\sigma_{ins}$  – **nessuno dei due e' scalato**, e nemmeno  $\omega_{BM}$  o  $\zeta_{BM}$  lo sono.  $A_6$  e  $K_1$  muovono solo il canale rigido, che a 18.9 rad/s e' gia' rotolato via e non compete con la risonanza. Quindi  $|L(\omega_{BM})|$  resta essenzialmente lo stesso in tutti i corner: non c'e' dispersione da mostrare. La robustezza del bending si studia altrove (Step D del Task 2 per  $\omega_{BM}$ , `main_montecarlo.m` per tutto il resto).

**D: Il controllore sopravvive a tutti e nove i casi. Il progetto e' allora robusto?** R: **No, e questa e' la risposta corretta**. Sopravvive, ma V3 lascia **0.91 dB** di margine aerodinamico e **18.0 gradi** di margine di fase: nessuno dei due e' accettabile per un progetto di volo (le soglie tipiche sono 6 dB e 30 gradi, che sono i target su cui il controllore e' stato tunato al nominale). Inoltre V3 triplica l'escursione di beccheggio (0.88 gradi contro 0.23) e raddoppia la deriva (4.9 m contro 2.3). Il messaggio ingegneristico del Task 3 e' proprio questo: il progetto a punto fisso **non ha abbastanza margine** per coprire una dispersione +/-30%, ed e' l'argomento quantitativo a favore del **gain scheduling**. E c'e' un secondo motivo, ancora piu' serio: il Task 3 **non disperde  $\omega_{BM}$** , e lo Step D del Task 2 ha gia' mostrato che bastano **+5%** su  $\omega_{BM}$  per far cadere l'anello. La fragilita' vera del progetto non e' nemmeno in questa tabella.

**D: Perche' lo script ri-esegue il tuner invece di leggere i guadagni del Task 2?** R: Per autosufficienza: cosi' `main_task3` puo' essere lanciato da solo, senza aver prima eseguito `main_task2`. `fminsearch` parte da un seed analitico fisso ( $K_{p0} = 2 \cdot A_6 / K_1$ ,  $K_{d0} = \sqrt{A_6} / K_1$ ) ed e' deterministico, quindi riproduce esattamente  $K_p = 1.732$  /  $K_d = 0.687$ . Il prezzo e' un run del tuner in piu' e, come limite di manutenibilita', il fatto che il **notch e' ricopiato a mano** come struct letterale alla riga 20: non c'e' una sorgente unica di verita' per il progetto del filtro, quindi una modifica in `main_task2.m` non si propagherebbe qui.

# HM3/main\_\_montecarlo.m

## Ruolo del file nel progetto

`main_montecarlo.m` e' un **extra** rispetto alla traccia di HM3: la traccia si ferma al Task 3, cioè alla robustezza valutata sui **quattro vertici** della scatola di incertezza  $\pm 30\%$  su  $\mu_{\alpha}$  (= A6) e  $\mu_c$  (= K1). Questo script sostituisce i vertici con **variabili aleatorie**: estrae  $N = 1500$  campioni su **cinque** fattori incerti ( $\mu_{\alpha}$ ,  $\mu_c$ ,  $\omega_{BM}$ ,  $\zeta_{BM}$ , ritardo TVC), riassume il loop aperto L e il loop chiuso T ad ogni estrazione, e produce distribuzioni di margini, una probabilit  di stabilit , una nuvola di Nichols e due scatter di sensibilit .

La catena di dipendenze e' la stessa dei Task 1-3: `load_hw3_params` per i parametri Greensite a  $t = 72$  s, `build_plant_full(p, 'ins')` per il modello a 6 stati con la contaminazione INS del modo flessionale, `build_tvc(p, 3)` per l'attuatore TVC piu' il ritardo approssimato con Pade di ordine 3, `build_notch_filter` per il notch profondo, `assemble_loop` per chiudere il PD e restituire (L, T), `simulate_gust_response` per la risposta alla raffica. Le uscite sono `HM3/figures/mc_margins_hist.png`, `HM3/figures/mc_nichols_cloud.png`, `HM3/figures/mc_sensitivity.png` e `HM3/mc_results.mat`.

Il file **non e' citato dal report LaTeX** (`HM3/report/`): compare solo nel README di HM3 e in `HM3/docs/flowcharts.md`. E' materiale di contorno, non consegnato.

---

## RIQUADRO DI ONESTA': lo script E' STALE rispetto ai Task 1-3

Ho verificato entrambi i punti sul codice. **Entrambe le incoerenze ci sono**, piu' una terza.

### (a) I guadagni NON sono quelli del Task 2

L'intestazione (riga 2) dichiara: *"Task-2 controller (rigid PD + bending notch fixed at nominal wBM) held fixed"*, e il banner di riga 39 dice *"Fixed controller (Task-2 design)"*. Ma il codice reale e':

```
Grigid = build_plant_rigid(p0);
K      = design_controller(Grigid, [], 'verbose', false);
```

(righe 41-42)

Questa e' **esattamente** la chiamata del Task 1 (`main_task1.m` riga 22: `[K, m] = design_controller(G, []);`). Il secondo argomento `[]` viene convertito in `tf(1)` da `design_controller` (riga 38), cioè **attuatore ideale**, e il plant e' quello **rigido a 4 stati**. Nessuna banda `w_flex` / `w_bending` viene passata, quindi il classificatore interno gira con i default "nessun modo flessionale".

Il Task 2 e il Task 3 fanno tutt'altro. `main_task3.m` righe 23-24:

```
K = design_controller(Gfull0, Wact0, 'w_flex', 0.6*p0.wBM, ...
    'w_flex_hi', 1.5*p0.wBM, 'w_bending', p0.wBM, ...
    'verbose', false);
```

cioe' ri-sintonizzano il PD **sul loop pieno** (plant a 6 stati + TVC + ritardo + notch). Secondo il README di HM3 i due set di guadagni sono:

| origine                                                                        | Kp_theta | Kd_theta |
|--------------------------------------------------------------------------------|----------|----------|
| Task 1 (plant rigido, attuatore ideale) – <b>quello che usa il Monte Carlo</b> | 1.78     | 0.44     |
| Task 2 ri-sintonizzato sul loop pieno – quello dei Task 2 e 3                  | 1.73     | 0.69     |

Il **notch** invece e' giusto: riga 43 costruisce  $z_N = 0.002$ ,  $z_D = 0.7$ ,  $\text{sgn} = +1$  a  $w_x = p0.w_{BM}$ , identico al notch profondo ritenuto nel Task 2 (`main_task2.m` righe 74-77). Quindi il Monte Carlo monta il **notch del Task 2 sopra i guadagni del Task 1**: e' precisamente la configurazione che `main_task2.m` chiama "BEFORE re-tuning" (righe 142-149) e che **scarta**, perche' il ritardo piu' l'attuatore piu' il lag del notch fanno collassare il margine di fase rigido da 30 gradi a 14.6 gradi, con delay margin a 98 ms (numeri dal README, tabella del trade e testo del Task 2).

**Conseguenza operativa:** il "nominale" attorno a cui il Monte Carlo disperde NON e' il progetto finale di HM3. E' il progetto intermedio bocciato. Le distribuzioni di margine, la  $P(\text{stabile})$  e le percentuali stampate a video **non sono confrontabili** con la tabella del Task 3 del README (Nominale: Aero  $|GM| = 6.00$  dB, Rigid PM = 30.0 gradi, DM = 165 ms).

## (b) I margini NON passano da `classify_margins`

Nel corpo del `parfor` (righe 90-111) i margini sono letti cosi':

```
am = allmargin(L);
gmv = 20*log10(am.GainMargin);
gf = am.GMFrequency;
...
minGM(i) = min(abs(gmv));
idx = find(gf>0.2 & gf<1, 1);
if ~isempty(idx), rigidGM(i) = abs(gmv(idx)); end
...
[~,Pm] = margin(L);
```

Nessuna chiamata a `classify_margins`. I problemi, in ordine di gravita':

1. **margin(L) restituisce il margine di fase PIU' PICCOLO** fra tutti gli attraversamenti a 0 dB. Ma il loop ha un lobo di drift laterale a bassa frequenza che genera attraversamenti spuri: e' scritto nero su bianco in `classify_margins.m` (righe 13-15) – "Crossings below `w_drift` are lateral-drift artifacts ... Taking `margin()`'s default instead would pick one of these." Il Monte Carlo fa esattamente quello che il resto di HM3 ha smesso di fare. Il vettore PM quindi non e' il "Rigid PM" delle tabelle dei Task 1-3.
2. **minGM = min(abs(gmv)) mescola bande e butta via il segno.** Nel loop di un lanciatore instabile il segno del gain margin e' fisica, non convenzione:  $gmdb < 0$  e' il margine di **riduzione** di guadagno (bordo inferiore della banda condizionalmente stabile, il margine *aerodinamico*),  $gmdb > 0$  e' il margine di **aumento** di guadagno (bordo superiore, il margine *rigido*). `classify_margins` li separa proprio con i test  $gmdb < 0$  e  $gmdb > 0$  (righe 44-45). Prendendo il valore assoluto e il minimo su tutto, `minGM` e' un numero senza banda di appartenenza.
3. **rigidGM e' mal nominato.** La finestra hard-coded  $gf > 0.2 \ \& \ gf < 1$  e il commento di riga 97 ("*low-freq aerodynamic crossover*") dicono che il numero e' il margine **aerodinamico**, non quello rigido. Ma la variabile si chiama `rigidGM` e l'istogramma la etichetta '`rigid |GM| [dB]`' (riga 153) con la linea di riferimento a 6 dB, che e' il target *aerodinamico*. Il numero e' plausibilmente giusto, il nome e l'etichetta sono sbagliati. In piu' la finestra  $[0.2, 1]$  rad/s e' fissa mentre A6 disperde di  $\pm 30\%$ : `find(..., 1)` prende la prima voce in finestra senza controllare il segno di `gmv`, cosa che `classify_margins` invece fa.
4.  $PM(i) = 180 * \text{double}(\text{isStab})$  (riga 108) quando non c'e' attraversamento di fase: e' un tappo arbitrario a 180 gradi. Va bene come segnapposto, ma inquina l'istogramma di PM e la statistica  $P(|PM| \geq 30)$ .

### (c) Terzo scostamento: orizzonte della raffica

Riga 45: `w = load_wind_profile(p0);` – senza `Tend`. Il default di `load_wind_profile` è `Tend = 12 s` (riga 25 di quel file). Task 1, Task 2 e Task 3 chiamano tutti `load_wind_profile(p, Tend=80)`, con un commento esplicito (main\_task1 riga 44) che 80 s servono perché il modo lento di drift ha  $\tau \sim 18\text{--}20$  s. Quindi **peakTh** e **peakZ** del Monte Carlo sono presi su **12 s di simulazione**: il picco di  $\theta$  (che avviene presto) è probabilmente catturato, ma il picco di  $z$  (drift, che cresce lentamente) può essere **sottostimato**. Anche questi due numeri non sono confrontabili con le colonne **peak th** / **peak z** della tabella del Task 3.

### Cosa è comunque valido

La **struttura** dell'esperimento (campionamento, propagazione, statistiche, nuvola di Nichols, scatter di sensibilità) è corretta e riusabile. Le **tendenze qualitative** –  $\omega_{BM}$  come driver dominante, instabilità che compare all'interno della scatola quando si disperdono anche flessione e ritardo – restano indicative. Sono i **valori numerici** a non essere allineati. Per riallinearlo servirebbero tre modifiche: (1) costruire `K` con `design_controller(build_plant_full(p0,'ins'), build_tvc(p0,3)*Wnotch, ...)` passando le bande, (2) sostituire il blocco `allmargin/margin` con `classify_margins(L, 'w_drift', 0.3*sqrt(p.A6), 'w_flex', 0.6*p.wBM, ...)`, (3) mettere `Tend=80` nella raffica.

## Intestazione e configurazione (righe 1-37)

- Righe 1-21: `docstring`. Elenca i cinque fattori incerti e le loro leggi, e dichiara la dipendenza dal solo Control System Toolbox. La riga 15 contiene l'osservazione progettuale più importante: *“Notch NOT returned when wBM disperses (deep notch needs near-exact wBM), so wBM is the dominant driver.”* Cioè: il notch è **congelato** a  $\omega_{BM}$  nominale, quindi ogni disaccordo fra il notch e la vera frequenza di flessione è pura perdita.
- Riga 24: `warning('off','Control:analysis:MarginUnstable')`. Serve perché `margin/allmargin` avvisano ad ogni valutazione di un loop condizionalmente stabile. Nota: è spento sul **client**; i worker di **parfor** non ereditano automaticamente lo stato dei warning del client, quindi in esecuzione parallela il messaggio può ricomparire (le funzioni di libreria `classify_margins` e `design_controller` se lo spengono localmente con `onCleanup`, qui invece no).
- Riga 27: `N = 1500` estrazioni. Riga 28: `Nsub = 150` sono le sole curve disegnate nella nuvola di Nichols (le altre 1350 non vengono ridisegnate). Riga 29: `wgrid = logspace(-2,2,600)` rad/s, la griglia su cui si valuta Nichols. Riga 30: `seed = 2026`, seme fisso.

## Specifiche delle incertezze (righe 32-37)

```
unc.mu_alpha = struct('dist','gauss','sigma',0.10,'trunc',0.30);
unc.mu_c      = struct('dist','gauss','sigma',0.10,'trunc',0.30);
unc.wBM       = struct('dist','gauss','sigma',0.02,'trunc',0.06);
unc.zBM       = struct('dist','lognorm','sigma',0.40);
unc.tau       = struct('dist','uniform','half', 0.25);
```

Tutti i fattori sono **moltiplicativi** sul valore nominale:  $p.A6 = p0.A6 \cdot f_a$ , e così via. Le scelte:

- **mu\_alpha**, **mu\_c**: gaussiane con  $\sigma = 10\%$  **troncate** a  $\pm 30\%$ . La troncatura è esattamente la scatola del Task 3, quindi  $\pm 30\% = 3$  **sigma**. Questa è una scelta di modellazione, non un dato: la traccia da un intervallo ( $\pm 30\%$ ), non una distribuzione. Interpretare l'intervallo come 3 sigma di una gaussiana è molto più ottimista che interpretarlo come supporto di una uniforme, e cambia le probabilità che escono. Va detto all'orale.
- **omega\_BM**: gaussiana  $\sigma = 2\%$  troncata a  $\pm 6\%$  (di nuovo 3 sigma). È molto più stretta perché è il parametro a cui il notch profondo è ipersensibile: il README documenta che con i filtri fissi il notch tollera  $-10\%$  ma va instabile a  $+5\%$ .
- **zeta\_BM**: lognormale,  $f = \exp(0.40 \cdot \text{randn})$ . Mediana 1, **media**  $\exp(0.5 \cdot 0.40^2) = \exp(0.08) = 1.083$  (una lognormale non è centrata sulla mediana). A 2 sigma il fattore sta in  $[\exp(-0.8), \exp(+0.8)] = [0.45,$

2.23], che e' quanto dice il commento di riga 12: il commento e' corretto. La scelta lognormale e' fisicamente sensata perche' lo smorzamento e' **positivo per costruzione** ( $\text{zeta\_BM} = 0.005$  nominale) e non puo' diventare negativo: una gaussiana lo permetterebbe.

- **tau**: uniforme su  $\pm 25\%$  (15-25 ms). Uniforme = “non so nulla dentro l'intervallo”, la scelta agnostica per un ritardo di implementazione.

**Non c'e' Latin Hypercube, non c'e' Sobol, non c'e' correlazione fra i fattori.** E' un Monte Carlo i.i.d. puro (`randn/rand`), con i cinque fattori **indipendenti**. E' la scelta piu' semplice; costa un fattore  $\sim 1/\sqrt{N}$  in convergenza rispetto a un LHS a parita' di  $N$ .

**Possibile domanda d'esame** – perche' troncare la gaussiana invece di usare direttamente una uniforme sulla scatola  $\pm 30\%$ ? *Risposta*: Sono due modelli di ignoranza diversi. La gaussiana troncata a 3 sigma dice “il valore vero e' quasi certamente vicino al nominale, i  $\pm 30\%$  sono un caso estremo”; la uniforme dice “qualunque valore nella scatola e' ugualmente probabile, vertici inclusi”. Con la gaussiana i vertici del Task 3 hanno probabilita' praticamente nulla di essere campionati, quindi il Monte Carlo **non riproduce** il caso peggiore del Task 3 e le due analisi restano complementari. Con la uniforme le due si avvicinerebbero, al prezzo di una ipotesi molto piu' pessimistica.

---

## Controller fisso e modello nominale (righe 39-54)

- Righe 40-45: costruisce  $p_0$  (nominale), il PD  $K$ , il notch  $W_{\text{notch}}$  e il profilo di vento  $w$ . Vedi il riquadro di onesta' sopra per i due difetti (guadagni Task 1 anziche' Task 2;  $T_{\text{end}}$  di default a 12 s).
- Riga 48-50: loop nominale di riferimento.

```
Gn      = build_plant_full(p0, 'ins');
Ln      = assemble_loop(Gn, K, build_tvc(p0, 3)*Wnotch);
[~, mn] = nichols_branch(Ln, wgrid, []);
```

$mn$  e' il **vettore di fase nominale**: serve solo come riferimento per allineare tutte le curve della nuvola sullo stesso ramo di 360 gradi (vedi `nichols_branch` piu' avanti).  $G_n$  usa la misura 'ins', cioe' Eq. (2):  $\theta_m = \theta + \sigma_{\text{inseta}}$ ,  $z_m = z - \phi_{\text{inseta}}$ . E' proprio questa contaminazione del modo flessionale nelle misure a rendere necessario il notch.

---

## Pre-campionamento (righe 56-63)

```
rng(seed);
fa = sample_factor(unc.mu_alpha, N);
fc = sample_factor(unc.mu_c,     N);
fw = sample_factor(unc.wBM,      N);
fz = sample_factor(unc.zBM,      N);
ft = sample_factor(unc.tau,      N);
```

Il commento di riga 57 spiega il perche': “*Sample OUTSIDE the parfor: deterministic given the seed, loop body RNG-free.*” E' il punto tecnico importante del parallelismo. Se si chiamasse `randn` dentro il `parfor`, ogni worker avrebbe il proprio stream e l'ordine di assegnazione delle iterazioni ai worker non e' deterministico: il risultato non sarebbe riproducibile fra un run seriale e uno parallelo, ne' fra pool di dimensioni diverse. Estrahendo tutto **prima**, con un `rng(seed)` singolo sul client, i cinque vettori di fattori sono fissati una volta per tutte e il corpo del loop diventa una funzione **deterministica** dell'indice  $i$ . Stesso seme, stessi numeri, con o senza Parallel Computing Toolbox.



## Propagazione Monte Carlo (righe 65-118)

- Righe 66-72: pre-allocazione di sette vettori  $N \times 1$  (`nan` per gli scalari, `false(N,1)` per il flag di stabilit ).
- Riga 75: `parfor i = 1:N`. **Senza Parallel Computing Toolbox MATLAB esegue `parfor` come un normale `for`**: nessun errore, solo esecuzione seriale. E' il "degrado seriale" citato nel README. Le variabili `p0`, `K`, `Wnotch`, `w`, `fa`, `fc`, `fw`, `fz`, `ft` sono *broadcast* (lette da tutti i worker), i sette vettori di uscita sono *sliced* (ogni iterazione tocca solo l'elemento `i`) – e' la condizione che rende il loop parallelizzabile senza dipendenze fra iterazioni.
- Righe 77-82: perturbazione. `p = p0`; poi si scalano **cinque** campi: `A6`, `K1`, `wBM`, `zBM`, `tau`. Nota che si scala il **plant vero**, non il modello usato per il progetto: il controller `K` e il notch `Wnotch` restano quelli nominali. E' il senso stesso di un'analisi di robustezza: filtro progettato sul nominale, veicolo vero diverso.
- Righe 85-87: riassettaggio.

```
Gf      = build_plant_full(p,'ins');
Wf      = build_tvc(p,3) * Wnotch;
[L, T]  = assemble_loop(Gf, K, Wf);
```

`build_tvc(p,3)` viene **ricostruito** con il `tau` perturbato (Pade di ordine 3 sul nuovo ritardo), mentre `Wnotch` e' il blocco nominale, congelato. Il disaccordo notch-flessione nasce qui: `Gf` ha il modo a `p0.wBM*fw(i)`, il notch ha lo zero a `p0.wBM`. - Righe 90-111: lettura dei margini. **E' il blocco problematico**, vedi il riquadro di onesta'. In sintesi: `allmargin + margin` grezzi, niente `classify_margins`. - Riga 94: se `gmv` e' vuoto, `minGM = Inf` – nessun attraversamento di guadagno significa margine di guadagno infinito, che e' formalmente giusto ma va gestito nelle statistiche (infatti `local_quantile` filtra con `isfinite`). - Righe 103-104: `isStab = isstable(T)`. **Questo e' il vero verdetto di stabilit **, ed e' l'unico che si puo' credere ciecamente. Il codice lo dice a voce alta nelle righe 132-134 dello stampato: con un veicolo instabile ad anello aperto il loop e' condizionalmente stabile, quindi il **segno** del gain margin non e' un indicatore di stabilit  e la sola cosa binaria affidabile e' la posizione dei poli del loop chiuso. - Righe 114-116: risposta alla raffica, sui 12 s di default (vedi punto (c) del riquadro). `peak_theta` viene convertito in gradi, `peak_z` resta in metri.

**Possibile domanda d'esame** – perche' `isstable(T)` e non il segno del gain margin? *Risposta*: Il velivolo e' aerodinamicamente instabile: ha un polo di corpo rigido in  $+\sqrt{A6} = +1.84$  rad/s. Un sistema a ciclo aperto instabile e' stabile in ciclo chiuso solo dentro una **banda** di guadagni: esistono un margine di riduzione (sotto il quale il polo instabile non viene piu' vinto) e un margine di aumento (sopra il quale si eccita l'attraversamento ad alta frequenza). La curva di Nichols passa **fra** due punti critici. In questa situazione il criterio "GM positivo = stabile" non vale, perche' dipende da quanti giri fa la curva attorno al punto critico (Nyquist generalizzato). Il conteggio dei poli a parte reale positiva del loop chiuso, cioe' `isstable`, e' l'unico verdetto non ambiguo.

---

## Statistiche (righe 120-145)

- Riga 121: `Pstab = mean(stab)`. E' la stima Monte Carlo della probabilit  di stabilit : la media di una variabile di Bernoulli su  $N$  estrazioni indipendenti.
- Righe 122-123: `Pgm3 = mean(minGM >= 3)` e `Ppm30 = mean(PM >= 30)`. Sono probabilit  di **soddisfare un requisito**, non margini. Attenzione: entrambe ereditano i difetti di lettura del blocco 90-111, quindi vanno lette come indicative.
- Riga 126: `[~,iWorst] = min(minGM)` – l'estrazione con il margine piu' stretto viene poi stampata con i suoi cinque fattori (righe 142-144). E' il modo corretto di riportare un Monte Carlo: non solo statistiche aggregate, ma anche il **campione peggiore**, con i valori dei parametri che l'hanno prodotto, in modo da poterlo riprodurre e analizzare a mano.
- Righe 135-141: percentili `p05` / `p50` / `p95` per sei metriche via `print_pct`. L'uso di `p05` e `p95` anziche' media  $\pm$  sigma e' la scelta giusta: le distribuzioni dei margini non sono gaussiane (sono troncate a sinistra dalla perdita di stabilit , e `minGM` ha un atomo a  $+\text{Inf}$ ).

Il commento stampato alle righe 132-134 e' importante e corretto:  $|GM|$  e  $|PM|$  sono **magnitudini**, e l'indicatore vincolante e' il flag di `isstable()`, non il segno del gain margin.

---

## Figura 1 – istogrammi (righe 147-157)

Sei riquadri, uno per metrica, prodotti da `hist_metric`: `minGM`, `rigidGM` (con linea di riferimento a 6 dB), `PM` (linea a 30 gradi), `DM`, `peakTh`, `peakZ`. Ogni riquadro riporta la **mediana** come linea nera etichettata (riga 310).

Come si legge un istogramma di margine:

- la **massa a sinistra della linea di riferimento** e' la frazione di estrazioni che violano il requisito: e' esattamente la probabilita' di violazione stimata dal Monte Carlo;
- la **coda sinistra** conta piu' del picco. Un progetto con mediana 8 dB e coda a 0.5 dB e' peggiore di uno con mediana 6 dB e coda a 4 dB;
- l'istogramma di `minGM` e' **troncato** dalla presenza di campioni instabili: quelli non hanno un margine sensato, quindi il fatto che l'istogramma sembri "sano" non dice nulla sulla  $P(\text{instabile})$ , che va letta separatamente nel titolo (riga 150).

**Nota di coerenza:** l'etichetta '`rigid |GM| [dB]`' con la linea a 6 dB e' sbagliata, vedi punto (b)(3) del riquadro di onesta': quel numero e' il margine aerodinamico.

---

## Figura 2 – nuvola di Nichols (righe 159-186)

- Righe 164-167: allineamento globale.

```
[gn, phn] = nichols_branch(Ln, wgrid, mn);
sc = find(gn(1:end-1).*gn(2:end) <= 0, 1, 'last');
if isempty(sc), shift = 360*round(median(phn)/360);
else, shift = 360*round((phn(sc)+180)/360); end
```

`find(gn(1:end-1).*gn(2:end) <= 0, 1, 'last')` cerca il **cambio di segno del guadagno in dB**, cioe' l'attraversamento a 0 dB, e prende l'**ultimo** (il piu' alto in frequenza). Poi calcola uno spostamento di fase multiplo di 360 gradi che porta quell'attraversamento vicino a -180 gradi – lo stesso ruolo del `PhaseMatchingValue = -180` di `plot_nichols_lv` (riga 40 di quel file). E' cosmetica: sposta la curva sul ramo di fase "giusto" del piano di Nichols, senza cambiare la fisica (aggiungere 360 gradi alla fase non cambia il sistema). - **Convenzione di fase:** il punto critico e' a **(-180 gradi, 0 dB)** (righe 182-184 marcano le copie  $-180 + 360k$ ), cioe' la convenzione degli appunti del corso, da  $1 + L = 0 \Leftrightarrow L = -1$ . Oggi e' la convenzione di **tutto** HM3: `plot_nichols_lv` e i tre main sono stati riallineati a -180, quindi la nuvola Monte Carlo e' direttamente confrontabile a occhio con le altre Nichols dell'homework. (Nota storica: fino a poco fa i Task 1-3 usavano la rietichettatura D'Antuono con il punto critico a +180 – stessa curva, fase mod 360, spostata di un giro – e questo script era l'**eccezione**; dopo il riallineamento non lo e' piu', e infatti **non e' stato toccato**. Il flip e' una pura rietichettatura dell'asse: nessun margine cambia.) - Righe 169-178: si ridisegnano `Nsub = 150` estrazioni prese con `round(linspace(1,N,150))`, cioe' un sottoinsieme **regolarmente spaziato negli indici**, non un campione casuale del campione – va bene perche' gli indici non sono ordinati per gravita', quindi un passo regolare e' equivalente a un'estrazione casuale. - Riga 176: colore. Grigio trasparente (alpha 0.18) se stabile, rosso (alpha 0.35) se instabile. Riga 179: la curva nominale in blu spesso sopra tutte. - **Costo:** il ciclo delle righe 170-178 **riassembla il loop da zero** per ognuna delle 150 estrazioni (`build_plant_full`, `build_tvc`, `assemble_loop`, `nichols`), duplicando lavoro gia' fatto dentro il `parfor`. E' inefficiente ma innocuo (150 assemblaggi contro i 1500 gia' fatti).

**Come si legge la nuvola:** la dispersione dei parametri produce un **fascio** di curve. Se una parte del fascio circonda diversamente il punto critico rispetto al nominale, li' c'e' un cambio di stabilita'. Le curve rosse mostrano *dove* la curva va a sbattere: con il notch congelato e  $\omega_{BM}$  disperso, ci si aspetta che il lobo flessionale – che nel nominale sta a -18 dB, ben sotto lo 0 dB – si alzi e risalga verso il punto critico quando la vera  $\omega_{BM}$  esce dal null del notch.

---

## Figura 3 – sensibilita' (righe 188-216)

- Righe 196-205, pannello (a): piano ( $\mu_{\alpha}$ ,  $\mu_c$ ), estrazioni stabili in verde e instabili con croce rossa, con sovrapposti il **rettangolo  $\pm 30\%$  del Task 3** e i suoi quattro vertici V1..V4.

Il punto sottile: dato che  $\mu_{\alpha}$  e  $\mu_c$  sono gaussiane troncate con  $\sigma = 10\%$  e troncatura a 3 sigma, la nuvola di punti **si concentra al centro** del rettangolo e i vertici marcati V1..V4 restano praticamente **vuoti**. La probabilita' di finire simultaneamente a 3 sigma su entrambi i fattori e' dell'ordine di  $(1.35e-3)^2 \sim 2e-6$ : con  $N = 1500$  non capita mai. La figura mostra quindi due popolazioni diverse: i vertici (deterministici, Task 3) e la nuvola (probabilistica, interno della scatola). Il commento delle righe 193-195 lo dice a modo suo: “*Task 3 found all four vertices stable in ( $\mu_{\alpha}$ ,  $\mu_c$ ) alone; dispersing bending/delay too makes instability appear inside the box.*”

- Righe 208-216, pannello (b): **minGM** contro la dispersione percentuale di  $\omega_{BM}$ . E' la verifica della tesi dichiarata a riga 15: il **detuning del notch e' il driver dominante**. Se l'ipotesi e' vera, i punti rossi (instabili) si addensano ad un estremo dell'asse x e il margine crolla in modo monotono al crescere del disaccordo.

**Possibile domanda d'esame** – se il notch e' cosi' sensibile a  $\omega_{BM}$ , perche' non lo si ri-sintonizza ad ogni estrazione? *Risposta:* Perche' sarebbe barare. In volo il controllore e' congelato: la vera  $\omega_{BM}$  del veicolo non e' misurabile in tempo reale, e' proprio l'incertezza che si vuole quantificare. Ri-sintonizzare il notch su ogni estrazione risponderebbe alla domanda “quanto e' buono il metodo di progetto”, non “quanto e' robusto **questo** controllore”. La riga 15 del sorgente lo dichiara esplicitamente. Il prezzo e' che il notch profondo ( $\zeta_N = 0.002$ , null strettissimo, largo  $\sim 2\zeta_{\omega_{BM}} \sim 0.08$  rad/s) e' fragile: e' l'argomento a favore dell'alternativa notch + lead-lag, che tiene tutta la banda  $\pm 10\%$  con margini nominali piu' magri.

---

## Export (righe 218-237)

- Righe 221-228: forza il tema chiaro (`theme(f,'light')` in `try`, con `f.Color = 'w'` come fallback per MATLAB pre-R2025a) ed esporta i PNG a 200 dpi con il prefisso `mc_`.
- Righe 231-236: salva **tutti i dati grezzi** in `mc_results.mat`: i cinque vettori di fattori, le sei metriche per estrazione, il flag di stabilita' e le tre probabilita'. E' la cosa giusta da fare: le figure sono ricostruibili dai dati, i dati no dalle figure. Con `fa.ft` e `stab` salvati, si puo' rifare qualunque post-processing (per esempio ri-leggere i margini con `classify_margins`, se non fosse che `L` non e' salvato).

---

## Funzioni locali (righe 239-315)

### `sample_factor` (righe 240-263)

```
case 'gauss'
    f = 1 + spec.sigma*randn(n,1);
    if isfield(spec,'trunc') && ~isempty(spec.trunc)
        lo = 1-spec.trunc; hi = 1+spec.trunc;
        bad = f<lo | f>hi;
        while any(bad)
            f(bad) = 1 + spec.sigma*randn(nnz(bad),1);
            bad = f<lo | f>hi;
        end
    end
end
```

- Righe 246-255, 'gauss': gaussiana con **troncatura per rifiuto** (*rejection sampling*): i campioni fuori dalla scatola vengono ri-estratti finche' cadono dentro. Questo produce la **gaussiana troncata** corretta

(densita' rinormalizzata sul supporto), a differenza del **clipping** ( $f = \min(\max(f, lo), hi)$ ) che accumulerebbe massa di probabilita' sui bordi – creerebbe due atomi in  $lo$  e  $hi$ , cioe' una distribuzione fisicamente assurda. La differenza conta: con clipping la scatola avrebbe i bordi “pesanti” e la  $P$ (violazione) sarebbe sovrastimata.

- Il **while** termina quasi certamente: con  $\sigma = 0.10$  e troncatura a 3  $\sigma$  la probabilita' di rifiuto e'  $\sim 0.27\%$  per campione, quindi il ciclo si chiude praticamente al primo giro. Ma **non c'e' guardia sul numero di iterazioni**: se qualcuno impostasse **trunc** molto minore di **sigma** il ciclo diventerebbe lentissimo (probabilita' di accettazione minuscola). E' un rischio latente, non un bug attivo con questi numeri.
- Righe 256-257, 'uniform':  $f = 1 + \text{half} \cdot (2 \cdot \text{rand} - 1)$ , supporto  $[1-\text{half}, 1+\text{half}]$ .
- Righe 258-259, 'lognorm':  $f = \exp(\sigma \cdot \text{randn})$ . Mediana 1 esatta, media  $\exp(\sigma^2/2) > 1$ , supporto  $(0, +\infty)$  – **strettamente positivo**, che e' il motivo per cui e' la legge giusta per uno smorzamento.

### nichols\_branch (righe 265-278)

```
[mag, phase] = nichols(L, w);
g = 20*log10(squeeze(mag));
ph = squeeze(phase);
if ~isempty(ref)
    ph = ph - 360*round((ph(1)-ref(1))/360);
end
```

Il problema che risolve: **nichols** restituisce la fase **srotolata** (unwrapped), ma il ramo su cui finisce dipende dal sistema. Due loop quasi identici possono uscire su rami distanti 360 gradi, e nella nuvola apparirebbero come due popolazioni separate anche se fisicamente coincidono. La correzione confronta la fase **al primo punto della griglia** ( $w = 1e-2$  rad/s, cioe' molto sotto la dinamica) con quella nominale e sottrae il multiplo di 360 piu' vicino. E' lecito perche' una fase definita modulo 360 gradi non cambia il sistema.

**Limite**: l'allineamento e' fatto su **un solo punto** ( $ph(1)$ ). Se una estrazione avesse una fase a bassa frequenza gia' spostata di piu' di 180 gradi rispetto al nominale, l'arrotondamento potrebbe agganciare il ramo sbagliato. Con questi livelli di dispersione e' improbabile, ma e' un allineamento fragile.

### print\_pct e local\_quantile (righe 280-298)

```
x = sort(x(isfinite(x)));
n = numel(x);
pos = min(max(p/100*n + 0.5, 1), n);
qv = interp1(1:n, x, pos, 'linear');
```

Percentili **senza lo Statistics Toolbox**. La regola implementata e' quella del “midpoint”: il  $k$ -esimo campione ordinato rappresenta il percentile  $(k - 0.5)/n * 100$ . Invertendo, il percentile  $p$  corrisponde alla posizione  $pos = p/100 * n + 0.5$ , con interpolazione lineare fra i due campioni adiacenti e saturazione agli estremi ( $[1, n]$ ). E' la stessa convenzione di **quantile** di MATLAB con il metodo di default. Il filtro **isfinite** (riga 293) e' necessario perche' **minGM** e **DM** possono valere  $+\infty$  e **rigidGM** puo' essere NaN quando l'attraversamento non cade nella finestra hard-coded  $[0.2, 1]$  rad/s.

**Effetto collaterale da tenere a mente**: scartando i  $+\infty$  e i NaN, i percentili sono calcolati su un **sottoinsieme** dei campioni, la cui dimensione non viene stampata. Un  $p05$  di **rigidGM** calcolato su 900 campioni su 1500 e' una statistica diversa da un  $p05$  su 1500, e il codice non lo segnala.

### hist\_metric (righe 300-315)

30 bin, mediana come linea nera etichettata (**xline**), linea di riferimento opzionale (6 dB oppure 30 gradi) tratteggiata. Usa **median(v, 'omitnan')** dopo aver gia' filtrato con **isfinite**, quindi l'**omitnan** e' ridondante – innocuo.

## Senso statistico: vertici (Task 3) contro Monte Carlo

Questo è il punto teorico da difendere all'orale.

**Cosa fa il Task 3.** Prende la scatola di incertezza  $[0.7, 1.3] \times [0.7, 1.3]$  su  $(\mu\_alpha, \mu\_c)$  e valuta i **quattro vertici**. La logica implicita è: *se la stabilità (o il margine) è peggiore ai vertici, allora la stabilità ai vertici implica la stabilità ovunque nella scatola.*

**Quando questa logica è valida.** Solo se la dipendenza della metrica dai parametri è **monotona** in ciascun parametro (o più in generale se il minimo del margine sulla scatola è assunto sulla frontiera, e sulla frontiera in un vertice). Per una funzione monotona in ogni argomento, il minimo su un ipercubo cade sempre in un vertice. Intuitivamente qui è plausibile:

$\mu\_alpha$  su  $\rightarrow$  velivolo più instabile  $\rightarrow$  margine aerodinamico peggiore  
 $\mu\_c$  giù  $\rightarrow$  meno autorità di controllo  $\rightarrow$  margine peggiore

quindi il vertice V3 = (1.3, 0.7) è “il peggiore dei quattro” e infatti il README lo conferma (0.91 dB / 18.0 gradi).

**Ma la monotonia è una congettura, non un teorema:** nessuno la dimostra. Con quattro valutazioni non si può escludere che il minimo cada **dentro** la scatola. Basta un'interazione non monotona – per esempio un attraversamento a 0 dB che cambia banda al variare di un parametro, o una risonanza che entra o esce da un filtro – e i vertici non sono più il caso peggiore. Con **cinque** parametri incerti la faccenda peggiora: i vertici diventano  $2^5 = 32$ , e comunque nessuno garantisce la monotonia in  $\omega\_BM$ : lì il notch ha un null strettissimo, quindi l'attenuazione ha un massimo al centro del null e degrada da **entrambi** i lati – una dipendenza palesemente **non monotona**, per cui i vertici in  $\omega\_BM$  sarebbero anzi i punti migliori, non i peggiori.

**Cosa aggiunge il Monte Carlo.**

1. **Campiona l'interno**, non solo i vertici. Se il minimo sta dentro, un campionamento denso lo trova (approssimativamente).
2. **Restituisce una probabilità, non un verdetto binario.** Il Task 3 dice “tutti e quattro i vertici sono stabili”. Il Monte Carlo dice “ $P(\text{stabile}) = x\%$ ”, che è l'unica cosa con cui si può fare un budget di affidabilità di missione. Il caso peggiore assoluto è spesso inutilizzabile: se il progetto è instabile in un vertice a probabilità  $1e-6$ , è un problema o no? Il ragionamento sui vertici non sa rispondere, quello probabilistico sì.
3. **Permette di aggiungere fattori** che la traccia non disperde ( $\omega\_BM$ ,  $\zeta\_BM$ ,  $\tau$ ). Ed è proprio l'aggiunta di questi che fa comparire instabilità *dentro* la scatola ( $\mu\_alpha$ ,  $\mu\_c$ ) dove il Task 3 non ne trovava: il Task 3 non è sbagliato, sta semplicemente rispondendo a una domanda su uno spazio di incertezza **più piccolo**.

**Cosa il Monte Carlo NON aggiunge.**

1. **Non è un certificato.** Non copre il caso peggiore: con code gaussiane troncate, i vertici hanno probabilità  $\sim 2e-6$  e con  $N = 1500$  non vengono mai estratti. Quindi il Monte Carlo **non sostituisce** il Task 3: sono complementari. Un certificato vero richiederebbe  $\mu$ -analisi o LMI su un politopo di incertezza.
2. **La risposta dipende dalle distribuzioni assunte**, che sono un'ipotesi di modellazione dell'ingegnere, non un dato della traccia. Cambiando le gaussiane troncate in uniformi, la  $P(\text{violazione})$  cambia di ordini di grandezza.
3. **La precisione va come  $1/\sqrt{N}$ .** Per una probabilità  $p$  stimata su  $N$  campioni indipendenti, l'errore standard è

$$SE = \sqrt{p(1-p) / N}$$

Con  $N = 1500$  e  $p \sim 0.01$ ,  $SE \sim 0.26\%$ , cioè un errore **relativo** del  $\sim 26\%$  sulla stima di una probabilità dell'1%. Le probabilità piccole sono proprio quelle mal stimate. Caso limite: se su 1500 estrazioni non se ne osservano di instabili, la “regola del tre” dà come limite superiore al 95% di confidenza  $p \leq 3/N = 0.2\%$ . **Non**  $p = 0$ .

**Possibile domanda d'esame** – il Monte Carlo trova un'estrazione instabile e i quattro vertici del Task 3 sono tutti stabili. Contraddizione? *Risposta:* No. Sono spazi di incertezza diversi. Il Task 3 disperde solo  $\mu\_alpha$  e  $\mu\_c$ ; il Monte Carlo disperde anche  $\omega\_BM$ ,  $\zeta\_BM$  e  $\tau$ , e

il notch profondo e' congelato alla  $\omega_{BM}$  nominale. L'instabilita' che compare e' guidata dal detuning del notch, un meccanismo che il Task 3 non esplora proprio. Se si fissassero  $\omega_{BM}$ ,  $\zeta_{BM}$  e  $\tau$  al nominale, la nuvola Monte Carlo tornerebbe tutta stabile dentro la scatola.

---

## Possibili domande d'esame

**D: Perche' in questo homework un `margin(L)` secco non va bene, e cosa fa invece `classify_margins`?**

R: Il loop aperto ha piu' di un attraversamento a 0 dB: uno a bassa frequenza generato dal canale di drift laterale (che contiene un integratore di posizione), uno di corpo rigido attorno a  $\sqrt{A_6}$ , e potenzialmente uno flessionale a  $\omega_{BM}$ . `margin()` restituisce il **piu' piccolo** dei margini fra tutti gli attraversamenti, e nel caso del lanciatore pesca l'artefatto di drift, che non ha alcun significato progettuale. `classify_margins` legge invece `allmargin` e smista gli attraversamenti per **banda di frequenza**: sotto  $\omega_{drift}$  = artefatti (scartati), fra  $\omega_{drift}$  e  $\omega_{flex}$  = corpo rigido (Rigid PM, Rigid GM per  $g_{mdb} > 0$ , Aero GM per  $g_{mdb} < 0$ ), sopra  $\omega_{flex}$  = modo flessionale. E' l'unico modo di ottenere numeri confrontabili con i target della traccia (6 dB / 30 gradi). **Nota critica: `main_montecarlo.m` NON usa `classify_margins`** ed e' rimasto alla lettura grezza – e' il difetto principale dello script.

**D: I guadagni usati dal Monte Carlo sono quelli finali del progetto HM3?** R: **No.** La riga 42 chiama `design_controller(Grigid, [], ...)`, cioe' progetta sul plant **rigido** con attuatore **ideale**: sono i guadagni del Task 1 ( $K_p = 1.78$ ,  $K_d = 0.44$ ). Il progetto finale di HM3 e' quello ri-sintonizzato sul loop pieno nel Task 2 ( $K_p = 1.73$ ,  $K_d = 0.69$ ), che e' quello che usano `main_task2.m` e `main_task3.m`. Il notch invece e' quello giusto (Task 2). Il commento in testa allo script, che parla di "Task-2 controller held fixed", e' falso. Il Monte Carlo sta quindi valutando la robustezza della configurazione che il Task 2 chiama esplicitamente "BEFORE re-tuning" e **scarta** (Rigid PM 14.6 gradi contro i 30 richiesti). Le sue statistiche non sono confrontabili con la tabella del Task 3.

**D: Perche' i campioni casuali vengono estratti PRIMA del `parfor` e non dentro?** R: Per la riproducibilita'. Dentro un `parfor` ogni worker ha il proprio stream di numeri casuali e l'assegnazione delle iterazioni ai worker non e' deterministica: lo stesso seme darebbe risultati diversi al variare della dimensione del pool, o fra esecuzione parallela e seriale. Estrahendo i cinque vettori di fattori sul client dopo un singolo `rng(seed)`, il corpo del loop diventa una funzione deterministica dell'indice e il risultato e' identico con o senza Parallel Computing Toolbox. Questo e' anche il motivo per cui `parfor` degrada senza problemi a `for` seriale se il toolbox non c'e'.

**D: Perche' lo smorzamento del modo flessionale e' modellato lognormale e non gaussiano?** R: Perche' lo smorzamento e' una quantita' **strettamente positiva** ( $\zeta_{BM} = 0.005$  nominale, gia' vicino a zero). Una gaussiana con dispersione ampia assegnerebbe probabilita' non nulla a valori negativi, cioe' a un modo flessionale **auto-eccitato**: fisicamente assurdo e numericamente catastrofico. La lognormale `exp(sigma*randn)` ha supporto (0, +Inf), mediana esatta 1 e la giusta asimmetria (piu' facile che lo smorzamento reale sia 2x del nominale che 0.5x). Per  $\mu_{\alpha}$  e  $\mu_c$ , che sono coefficienti con segno noto e dispersione del 10 %, la gaussiana troncata va benissimo.

**D: Perche' la troncatura per rifiuto invece del clipping?** R: Il clipping (`min(max(f,lo),hi)`) schiaccerebbe tutta la coda esterna sui due estremi, creando due **atomi** di probabilita' in `lo` e `hi`. Il risultato non e' una gaussiana troncata: e' una gaussiana troncata piu' due masse concentrate sui bordi della scatola, che nel nostro caso sono proprio i punti piu' critici. La  $P(\text{violazione})$  risulterebbe sovrastimata e l'istogramma avrebbe due picchi artificiali agli estremi. Il rejection sampling ri-estrae fino a cadere dentro il supporto e produce la densita' gaussiana correttamente rinormalizzata.

**D: Che cosa aggiunge il Monte Carlo rispetto ai quattro vertici del Task 3?** R: I vertici sono il caso peggiore **solo se** la metrica dipende in modo monotono da ciascun parametro: per una funzione monotona su un ipercubo il minimo cade in un vertice. Qui la monotonia e' plausibile per  $\mu_{\alpha}$  e  $\mu_c$  (piu' instabile, meno autorita' = peggio) ma non e' dimostrata, e per  $\omega_{BM}$  e' palesemente **falsa** (il notch ha un null stretto: l'attenuazione degrada da entrambi i lati della frequenza nominale). Il Monte Carlo campiona l'**interno** della scatola, aggiunge tre fattori che il Task 3 non disperde, e soprattutto restituisce una **probabilita' di violazione** invece di un verdetto binario – che e' l'unica cosa integrabile in un budget di affidabilita'. Non e' pero' un

certificato: con code troncate, i vertici hanno probabilita'  $\sim 2e-6$  e con  $N = 1500$  non vengono mai estratti, quindi il Monte Carlo **non sostituisce** l'analisi ai vertici.

**D: Con  $N = 1500$  quanto e' precisa la  $P(\text{stabile})$  stimata?** R: La stima e' una media di Bernoulli, quindi ha errore standard  $SE = \sqrt{p^*(1-p)/N}$ . Per  $p \sim 0.5$  (caso peggiore)  $SE \sim 1.3 \%$ ; per  $p \sim 0.01$ ,  $SE \sim 0.26 \%$ , cioe' un errore relativo del 26 % sulla probabilita' piccola – e le probabilita' piccole sono esattamente quelle che interessano in affidabilita'. Se si osservano **zero** fallimenti su 1500, non si puo' concludere  $p = 0$ : la regola del tre da'  $p \leq 3/1500 = 0.2 \%$  al 95 % di confidenza. Per stimare probabilita' dell'ordine di  $1e-4$  servono  $N \sim 1e6$ , oppure tecniche di riduzione della varianza (importance sampling, subset simulation) o un metodo deterministico (mu-analisi/LMI) che copra il caso peggiore per costruzione.

**D: Perche' il notch non viene ri-sintonizzato quando  $\omega_{BM}$  disperde?** R: Perche' il controllore in volo e' congelato: la vera  $\omega_{BM}$  non e' misurabile in tempo reale, e' esattamente l'incertezza che si sta quantificando. Ri-sintonizzare a ogni estrazione risponderebbe a una domanda diversa ("quanto e' buono il metodo di progetto") anziche' a quella giusta ("quanto e' robusto questo controllore"). La riga 15 del sorgente lo dichiara e ne trae la conseguenza: proprio perche' il notch e' profondo ( $\zeta_N = 0.002$ ) e strettissimo,  $\omega_{BM}$  diventa il driver dominante dell'instabilita' nel Monte Carlo.

# HM3/init\_simulink\_hm3.m

## Ruolo del file nel progetto

`init_simulink_hm3.m` e' il **ponte fra il track script e il track Simulink** di HM3. Non contiene ne' dinamica ne' progetto di controllo: ricalcola, chiamando gli stessi moduli usati dai `main_task*.m` (`load_hw3_params`, `build_plant_rigid`, `build_plant_full`, `build_tvc`, `build_notch_filter`, `design_controller`, `load_wind_profile`), **ogni singolo parametro che i blocchi di `models/hm3_closed_loop.slx` devono leggere**, e li spara nel base workspace con `assignin`. Il modello Simulink non contiene nemmeno un numero scritto a mano: nelle maschere dei blocchi ci sono solo *nomi di variabile* (`A_full`, `tvc_num`, `Kp_th`, `wind_ts`, `Tstop`, `task`), che Simulink risolve nel base workspace alla compilazione. Questo e' verificabile aprendo lo `.slx` come zip: nel `system_root.xml` il blocco Gain Controller\_PD ha `Gain = [Kp_th, -Kp_th, -Kd_th, -Kp_z, -Kd_z]` e il blocco From Workspace ha `VariableName = wind_ts`; i blocchi State-Space stanno invece nei file dei due rami del variant – `system_22.xml` (Plant\_Full: `A = A_full`, `B = [Bdelta_full, Bwind_full]`, `C = [C_meas_full; C_plot_full]`, `D = zeros(7,2)`) e `system_7.xml` (Plant\_Rigid, con gli omologhi `*_rigid`).

**Perche' esiste il track Simulink**, se gli script bastano gia' a produrre Nichols, margini e risposte al gust? Due ragioni, entrambe difendibili all'orale:

1. **Validazione incrociata.** Il track script vive interamente nel dominio delle funzioni di trasferimento: `assemble_loop` costruisce  $T(s)$  con `connect` e `simulate_gust_response` la integra con `lsim`, cioe' con una discretizzazione esatta del sistema LTI. Il track Simulink e' invece una integrazione numerica a passo variabile (`ode45`) di un diagramma a blocchi assemblato in modo indipendente: realizzazioni di stato diverse, ordine dei blocchi diverso, aritmetica diversa. Se le due risposte coincidono, gli errori possibili si riducono drasticamente: un segno sbagliato in una C, una riga della B scambiata, un guadagno con il segno invertito nel PD *romperebbero* la sovrapposizione. E' una prova indipendente, non una ripetizione.
2. **Formato di consegna industriale.** Il controllo d'assetto, nell'industria, si consegna come modello (Simulink -> autocodifica -> HIL), non come script di analisi. Il track Simulink e' la traduzione del progetto classico nel formato in cui verrebbe effettivamente integrato, con il vantaggio che il modello e' *scriptato dai dati* e non ri-digitato.

La filosofia dichiarata (README HM3 e `models/SIMULINK_GUIDE.md`) e' **script-first**: gli script sono la fonte di verita', `init_simulink_hm3` e' il compilatore che li traduce in parametri di blocco, `run_simulink_closed_loop` e' il test di regressione. Il file e' chiamato da `run_simulink_closed_loop.m` (riga 33) e a mano dall'utente prima di aprire il modello.

**Attenzione – c'e' una discrepanza reale fra questo file e i `main_task2/3`, verificata numericamente e documentata in fondo alla pagina** (sezione "Discrepanze verificate"): i guadagni PD che questo file esporta *non sono* quelli del progetto finale di Task 2/3.

---

## Firma e arguments block (righe 1-26)

```
function S = init_simulink_hm3(task, o)
arguments
    task (1,1) {mustBeMember(task, [1 2 3])} = 2
```



```

o.mu_alpha_scale (1,1) {mustBeNumeric, mustBeReal} = 1.0
o.mu_c_scale      (1,1) {mustBeNumeric, mustBeReal} = 1.0
o.severity ... = 'severe'
o.profile {mustBeTextScalar} = 'gust'
o.push (1,1) logical = true
end

```

- Riga 1: la funzione restituisce una **struct S** con tutto cio' che ha calcolato. Il valore di ritorno esiste perche' `run_simulink_closed_loop` ha bisogno degli stessi dati per costruire la baseline analitica (non li rilegge dal base workspace, se li fa restituire): e' il modo pulito per garantire che script e modello vedano *esattamente* gli stessi numeri.
- Riga 20: `task` seleziona il **variant** del modello, non cosa viene calcolato. Il file calcola comunque **sia** le matrici rigide **sia** quelle flessibili, ed esporta entrambe. Il default e' `task = 2`.
- Righe 21-22: `mu_alpha_scale` e `mu_c_scale` sono i fattori moltiplicativi sui coefficienti  $\mu_{\alpha} = A_6$  (momento aerodinamico destabilizzante) e  $\mu_c = K_1$  (efficacia del controllo) per i vertici +/- 30 % di Task 3. Vengono solo inoltrati a `load_hw3_params`.
- Righe 23-24: `severity` e `profile` scelgono il vento; 'strongwind' fa girare il generatore Simulink del professore (General/hw3-v3/strong\_wind.slx) invece del gust analitico 1-cosine.
- Riga 25: `push` (default true) e' l'unica opzione che ha un effetto collaterale: senza di essa il modello Simulink non troverebbe le variabili.

**Possibile domanda d'esame** – Perche' `task` non cambia cosa viene calcolato? *Risposta:* perche' il modello e' un unico .slx con un **Variant Subsystem** (Plant\_and\_Actuator) a due scelte (Rigid\_Ideal con espressione di controllo `task == 1`, Full\_TVC\_Notch con `task ~= 1`). Il blocco non imposta `VariantActivationTime`, quindi vale il default *update diagram*: Simulink valuta l'espressione e **pota** la scelta inattiva prima della compilazione – le variabili del ramo non attivo non vengono nemmeno risolte, e in linea di principio potrebbero mancare. Esportare comunque **entrambi** i set (22 variabili, righe 67-71) e' quindi una scelta di comodita', non un obbligo del compilatore: rende il base workspace valido per tutti e tre i task, cosi' che cambiare `task` e ri-simulare non richieda di rilanciare l'init.

## Parametri e controllore (righe 28-38)

```

p = load_hw3_params('mu_alpha_scale',o.mu_alpha_scale, ...
                  'mu_c_scale',o.mu_c_scale);
p0 = load_hw3_params(); % nominal design point
K = design_controller(build_plant_rigid(p0), [], 'verbose', false);

```

- Riga 32: `p` = parametri **perturbati** (con gli scale factor del corner). Da qui escono le matrici del plant che il modello simulerà.
- Riga 33: `p0` = parametri **nominali**. Servono per il controllore e per il notch, che devono restare *ignari* della perturbazione – questo e' il cuore concettuale di uno studio di robustezza: il controllore vola con la sua taratura nominale, e' il veicolo a essere diverso da come lo si era immaginato.
- Riga 34: qui si progetta il PD. **Ed e' qui il problema.** La chiamata e' `design_controller(build_plant_rigid(p0), [], ...)`: plant **rigido** (4 stati, niente bending) e `Wact = []`, che dentro `design_controller` (riga 38 di quel file) diventa `tf(1)`, cioe' **attuatore ideale**. Questa e' esattamente la configurazione di **Task 1**. I `main_task2.m` (riga 153) e `main_task3.m` (righe 23-24) chiamano invece `design_controller(Gfull, Wtvc*Hn, ...)`, cioe' ri-tarano il PD **sul loop completo** (plant flessibile + TVC + ritardo + notch). I due tuner convergono a guadagni diversi:

```

init_simulink_hm3 : Kp_th = 1.7845   Kd_th = 0.4433   (Task 1)
main_task2 / 3    : Kp_th = 1.7318   Kd_th = 0.6867   (Task 2 re-tuned)

```

Numeri ottenuti eseguendo i due tuner in MATLAB, non stimati.

- Righe 36-38: la struct **S** viene inizializzata e i quattro guadagni (**Kp\_th**, **Kd\_th**, **Kp\_z**, **Kd\_z**) copiati dentro. **Kp\_z = Kd\_z = -1e-3** sono i guadagni di drift, fissi per default in **design\_controller** (riga 28-29): il segno negativo e' il *load relief* – il lanciatore si lascia derivare un po' sottovento invece di inseguire  $z = 0$ , riducendo l'incidenza aerodinamica.

Il commento alle righe 29-31 dice: “*Controller and notch FROZEN at the nominal point, as in main\_task3: robustness = fixed nominal gains on the perturbed plant*”. Lo **spirito** e' giusto (non si ri-tara sul corner). Il **valore congelato**, pero', e' il controllore di Task 1, non quello di Task 2. Il commento della testata (righe 16-17) – “*Gains/filters are exactly those designed by the scripts; the model only mirrors them*” – e' quindi **falso per task = 2 e task = 3**, ed e' precisamente il tipo di contraddizione commento/codice che va segnalata.

**Possibile domanda d'esame** – Cosa cambia, in pratica, far volare il loop completo con i guadagni di Task 1 invece di quelli ri-tarati? *Risposta:* il loop resta **stabile**, ma con margini molto peggiori. Valutato sul loop di Task 2 (plant flessibile INS + TVC + Pade + notch profondo): con **Kp = 1.784 / Kd = 0.443** -> Aero |GM| = 6.08 dB, **Rigid PM = 14.6 deg**, delay margin 98 ms; con **Kp = 1.732 / Kd = 0.687** -> Aero |GM| = 6.00 dB, **Rigid PM = 30.0 deg**, delay margin 165 ms. E' esattamente il confronto “BEFORE / AFTER re-tuning” che **main\_task2.m** stampa alle righe 146-161. Il derivativo piu' alto e' cio' che ricompra la fase mangiata da servo + ritardo + notch.

## Matrici del plant (righe 40-51)

```
Gr = build_plant_rigid(p);
S.A_rigid = Gr.A;
S.Bdelta_rigid = Gr.B(:,1); S.Bwind_rigid = Gr.B(:,2);
S.C_meas_rigid = Gr.C(1:4,:); % [theta_m thetadot_m z_m zdot_m]
S.C_plot_rigid = Gr.C(5:7,:); % [theta z zdot]
```

- Righe 41-45: plant rigido, 4 stati [**z**, **zdot**, **theta**, **thetadot**], 2 ingressi [**delta**, **alpha\_w**], 7 uscite.
- Riga 43: **la B viene spaccata in due colonne**. Questo e' l'unico vero “adattamento” a Simulink presente nel file. In MATLAB **Gr.B** e' 4x2 e **lsim** accetta un ingresso vettoriale a 2 componenti. In Simulink i due ingressi arrivano da posti fisici diversi – **u\_pd** (o l'uscita del TVC) da un lato, **alpha\_w** dal blocco **From Workspace** dall'altro – e vanno ricomposti in un segnale 2-wide con un **Mux** prima del blocco **State-Space**. Esportare le due colonne separatamente permette di scrivere nella maschera **B = [Bdelta\_rigid, Bwind\_rigid]**, che documenta l'ordine (delta = porta 1 del Mux, alpha\_w = porta 2) invece di nascondere.
- Righe 44-45: la **C** viene spaccata in **righe di misura** (1-4, cio' che il controllore legge) e **righe di plot** (5-7, gli stati veri). La distinzione e' fisica, non estetica: nel plant flessibile le righe 1-4 sono contaminate dal bending (Eq. 2, coefficienti **sigma\_ins**, **phi\_ins**), le righe 5-7 no. Nel modello questa separazione sopravvive nel **Demux** a 7 uscite: le prime 4 tornano al Mux del controllore (chiudono l'anello), le ultime 3 vanno ai blocchi **To Workspace**.
- Righe 47-51: idem per il plant **completo**, 6 stati (aggiunge **eta**, **etadot** del primo modo di bending), con **meas = 'ins'**, cioe' la variante in cui il bending **entra nelle misure**. E' questa contaminazione – non il modo di bending in se' – che destabilizza il loop e motiva il notch.

**Possibile domanda d'esame** – Perche' non si esporta direttamente l'oggetto **ss** e si usa un blocco LTI System? *Risposta:* si potrebbe (esiste il blocco **LTI System** della Control System Toolbox). Il codice sceglie il blocco **State-Space** base, che usa solo Simulink core: rende il modello leggibile senza toolbox aggiuntive e, cosa piu' importante didatticamente, **espone le matrici** nel diagramma, dove si vede che **B** ha due colonne e **C** sette righe. Con un blocco LTI il modello sarebbe una scatola nera con dentro un oggetto MATLAB.

## Attuatore TVC + notch (righe 53-57)

```
Wtvc = build_tvc(p,3);
[S.tvc_num, S.tvc_den] = tfdata(tf(Wtvc),'v');
Hx = build_notch_filter(p0.wBM, 0.002, 0.7, +1);
[S.notch_num, S.notch_den] = tfdata(Hx,'v');
```

- Riga 54: `build_tvc(p, 3)` = servo TVC di 2o ordine ( $\omega_{TVC} = 70$  rad/s,  $\zeta_{TVC} = 0.7$ ) **in serie con l'approssimante di Pade di ordine 3** del ritardo puro  $\tau = 20$  ms. Nota: `p` e' il set perturbato, ma `wTVC`, `zTVC`, `tau` non sono toccati dagli scale factor, quindi `build_tvc(p,3)` e `build_tvc(p0,3)` danno la stessa `tf` – la scelta di `p` qui e' innocua.
- Riga 55: `tfdata(tf(Wtvc),'v')` estrae numeratore e denominatore come **vettori riga** ('v' = vector, non cell array). Il risultato: denominatore di grado 5 (2 poli del servo + 3 poli di Pade), numeratore di grado 3 (gli zeri di Pade), restituito con **due zeri iniziali di padding** per avere la stessa lunghezza del denominatore. Il blocco **Transfer Fcn** di Simulink accetta esattamente questo formato. Poli e zeri di Pade(3) per  $\tau = 0.02$  s (calcolati):

poli: -183.9 +/- 175.4i, -232.2 (semipiano sinistro)  
zeri: +183.9 +/- 175.4i, +232.2 (semipiano DESTRO, speculari)

La simmetria e' il punto: modulo unitario a tutte le frequenze (all-pass), solo fase. Pade **non e'** il ritardo vero, e' la sua approssimazione razionale – l'unica cosa che `margin`, `pole`, `isstable`, `minreal` e il tuner `fminsearch` sanno maneggiare.

- Riga 56: `build_notch_filter(p0.wBM, 0.002, 0.7, +1)`, cioe'

$$H_x(s) = (s^2 + 2*\zeta_N * w_x * s + w_x^2) / (s^2 + 2*\zeta_D * w_x * s + w_x^2)$$

con  $w_x = \omega_{BM} = 18.9$  rad/s,  $\zeta_N = 0.002$ ,  $\zeta_D = 0.7$ , `numSign` = +1 (numeratore a fase minima -> notch simmetrico, non lead-lag). I coefficienti esportati sono (calcolati):

`notch_num` = [1, 0.0756, 357.2] ( $2*\zeta_N * w_x = 0.0756$ )  
`notch_den` = [1, 26.46, 357.2] ( $2*\zeta_D * w_x = 26.46$ ,  $w_x^2 = 357.2$ )

Profondita' del notch =  $20\log_{10}(\zeta_N/\zeta_D) = -50.9$  dB alla frequenza centrale. Da segnalare onestamente: *build\_notch\_filter* documenta nel suo proprio header la guideline\* `zN` in 0.1-0.3 – qui si usa 0.002, cioe' **due ordini di grandezza fuori dalla linea guida della traccia**. E' voluto (e' il "deep notch" scelto dal trade di `main_task2.m`), ma va saputo difendere: il prezzo e' che il notch e' **strettissimo** – la coppia di zeri e' larga  $2\zeta_N w_x = 0.076$  rad/s su una centrale di 18.9 rad/s – ed e' proprio questa strettezza che obbliga a conoscere  $\omega_{BM}$  con precisione (Step D di `main_task2` mostra che +5 % di errore su  $\omega_{BM}$  destabilizza).

- Riga 56, dettaglio importante: il notch e' centrato su `p0.wBM` (nominale) e non su `p.wBM`. Concettualmente e' la scelta giusta (il filtro non deve conoscere la perturbazione). Numericamente qui non cambia nulla, perche' gli scale factor toccano solo `A6` e `K1`, mai `wBM`.

**Possibile domanda d'esame** – Perche' precalcolare `tvc_num/tvc_den` in MATLAB invece di scrivere l'espressione nella maschera del blocco? *Risposta:* perche' i coefficienti non sono scrivibili a mano. Sono il risultato del prodotto polinomiale servo x Pade(3): sei coefficienti a denominatore che nessuno vuole ricalcolare a mano ogni volta che cambia  $\tau$  o l'ordine di Pade. Metterli in maschera come espressione MATLAB (`pade(0.02,3)` dentro il blocco) sarebbe possibile ma **duplicherebbe la definizione**: da quel momento in poi il modello e lo script potrebbero divergere senza che nessuno se ne accorga. Con il precalcolo esiste **una sola** definizione (`build_tvc.m`) e il modello e' un consumatore passivo.

## Vento come timeseries (righe 59-64)

```
w = load_wind_profile(p,'severity',o.severity,'profile',o.profile);
S.wind_ts = timeseries(w.alphaw(:), w.t(:), 'Name','alpha_w');
S.Tstop = w.t(end);
S.task = task;
```

- Riga 60: il vento e' generato **una volta sola**, dallo stesso `load_wind_profile` che usano gli script. Il gust di default e' un 1-cosine di 3 s con `V_g` preso dalla dispersione `drywind.mat` alla quota di volo (`V_g = 6.38 m/s` -> picco `alpha_w = 0.39 deg`).
- Riga 61: viene impacchettato come oggetto `timeseries` perche' e' il formato che il blocco **From Workspace** legge nativamente (`VariableName = wind_ts` nel `.slx`; `OutputAfterFinalValue = 'Holding final value'`). Dentro il blocco Simulink **interpola linearmente** fra i campioni: la griglia del vento ha `dt = 0.005 s`, quella di `ode45` e' variabile, quindi l'interpolazione c'e' ed e' una delle sorgenti (piccole) di scostamento fra i due track.
- Riga 62: `Tstop` = fine del profilo di vento = **12 s** con i default (`load_wind_profile` ha `Tend = 12` di default). Attenzione: `main_task2.m` e `main_task3.m` chiamano invece `load_wind_profile(p, Tend=80)` – 80 s, per vedere il modo lento di drift ( $\tau \sim 20$  s). **Le figure Simulink coprono quindi un orizzonte diverso (12 s) da quelle degli script (80 s)**. Per il corner peggiore V3 i picchi cadono comunque entro i primi 12 s (verificato: stessi picchi a 12 e a 80 s), quindi non e' un errore, ma e' una differenza da conoscere se si confrontano le figure.
- Riga 64: `S.task = task` – questa e' la variabile che l'espressione di controllo del Variant Subsystem (`task == 1 / task ~= 1`) legge.

---

## Push nel base workspace (righe 66-75)

```
if o.push
    fn = fieldnames(S);
    for i = 1:numel(fn)
        assignin('base', fn{i}, S.(fn{i}));
    end
    fprintf('init_simulink_hm3: pushed %d variables to base ...');
end
```

- Righe 67-71: **22 variabili** finiscono nel base workspace (`p`, i 4 guadagni, 5 matrici rigide, 5 matrici full, 4 vettori di coefficienti, `wind_ts`, `Tstop`, `task`). Il loop su `fieldnames` fa si' che aggiungere un campo a `S` lo esporti automaticamente: nessuna lista da tenere aggiornata a mano.
- **Perche' il base workspace e non il model workspace?** Simulink risolve i nomi nelle maschere risalendo la gerarchia: `mask workspace` -> `model workspace` -> `data dictionary` -> **base workspace**. Il modello HM3 non ha `model workspace` ne' `data dictionary`, quindi il base workspace e' l'unico posto dove le variabili possono stare. Il prezzo e' che il base workspace e' **globale**: se qualcuno ha una variabile `task` o `p` gia' definita, viene sovrascritta. Una versione piu' industriale userebbe `Simulink.SimulationInput.setVariable` (che infatti `load_wind_profile` usa alle righe 92-96 per il modello del professore) – qui non lo si fa, e vale la pena saperlo dire.
- Riga 72: il `fprintf` di conferma stampa quante variabili e con quali scale factor. E' l'unica traccia visibile che il push e' avvenuto.

---

## Discrepanze verificate fra i due track

Tutto quanto segue e' stato **eseguito**, non dedotto:

1. **I guadagni PD del modello Simulink, per task = 2 e task = 3, non sono quelli del progetto consegnato.** `init_simulink_hm3` (riga 34) tara il PD sul plant rigido con attuatore ideale ( $K_p = 1.7845$ ,  $K_d = 0.4433$ , i guadagni di Task 1), mentre `main_task2.m` (riga 153) e `main_task3.m` (riga 23) lo ri-tarano sul loop completo ( $K_p = 1.7318$ ,  $K_d = 0.6867$ ). Il loop risultante e' stabile ma con Rigid PM = 14.6 deg invece di 30 deg e delay margin 98 ms invece di 165 ms. Il commento in testata del file ("Gains/filters are exactly those designed by the scripts") **contraddice il codice.**
2. **Conseguenza sulla validazione:** la sovrapposizione script-vs-Simulink *continua a funzionare*, perche' `run_simulink_closed_loop` costruisce la sua baseline analitica con la **stessa K** restituita da `init_simulink_hm3` (riga 43 di quel file). I due track coincidono – ma coincidono su un controllore che non e' quello del report. La validazione e' internamente coerente e **esternamente sfasata** rispetto a Task 2/3.
3. **I numeri di Task 3 in models/SIMULINK\_GUIDE.md (righe 222-224) non tornano.** La guida dice: corner V3, picco theta  $\sim 0.35$  deg e picco z  $\sim 4.2$  m, "*matching main\_task3.m*". In realta' `main_task3.m` (guadagni ri-tarati) da' **0.878 deg / 4.92 m** (come riporta il README), e il modello Simulink cosi' com'e' inizializzato (guadagni di Task 1) da' **1.402 deg / 4.88 m**. La guida non coincide con nessuno dei due: e' testo stantio.
4. **Nessun conflitto sul Pade.** Verificando lo `.slx` (`system_22.xml`), il ramo `Full_TVC_Notch` contiene solo due blocchi `Transfer Fcn` (`notch_num/notch_den` e `tvc_num/tvc_den`): **non c'e' nessun blocco Transport Delay**. Il ritardo e' approssimato con Pade in *entrambi* i track, ed e' esattamente la stessa `tf`. Su questo punto i due track sono coerenti.

Come si aggiusterebbe la (1), se lo si volesse: sostituire la riga 34 con una progettazione condizionata a `task` – per `task == 1` la chiamata attuale, per `task >= 2` `design_controller(build_plant_full(p0,'ins'), build_tvc(p0,3)*Hx, 'w_flex', 0.6*p0.wBM, 'w_flex_hi', 1.5*p0.wBM, 'w_bending', p0.wBM)`. Non lo faccio qui perche' la consegna e' documentare, non modificare.

---

## Possibili domande d'esame

**D: Perche' costruire un modello Simulink se gli script producono gia' tutti i risultati richiesti dalla traccia?** R: Per due motivi. Il primo e' la **validazione incrociata**: due implementazioni indipendenti dello stesso anello chiuso (funzioni di trasferimento + `lsim` da un lato, diagramma a blocchi + `ode45` dall'altro) che devono dare la stessa risposta temporale. Un errore di segno, una C sbagliata, un guadagno invertito farebbero divergere le due curve: la coincidenza e' quindi una prova, non una ripetizione. Il secondo e' che il **controllo d'assetto si consegna come modello**, non come script: Simulink e' il formato da cui si fa autocodifica e HIL, e il modello e' anche il documento che si mostra a chi integra.

**D: Perche' precalcolare ogni parametro in MATLAB invece di mettere espressioni nelle maschere dei blocchi?** R: Perche' i parametri non sono costanti: i guadagni PD escono da un `fminsearch` su margini classificati per banda (`design_controller`), i coefficienti del TVC dal prodotto servo x Pade(3), le matrici del plant da `build_plant_full`. Scrivere queste espressioni in maschera significherebbe **duplicare la definizione** e permettere a modello e script di divergere in silenzio; inoltre le espressioni in maschera vengono rivalutate a ogni compile del modello, il che vorrebbe dire rieseguire il tuner ogni volta. Con il precalcolo esiste una sola fonte di verita' (gli `.m`) e il modello e' un consumatore passivo di nomi di variabile.

**D: Come fa un unico .slx a coprire tre task con plant e attuatori diversi?** R: Con un **Variant Subsystem** (`Plant_and_Actuator`) a due scelte: `Rigid_Ideal` (espressione di controllo `task == 1`: plant rigido a 4 stati, attuatore ideale,  $\delta = u_{pd}$ ) e `Full_TVC_Notch` (`task ~= 1`: notch -> TVC con Pade -> plant flessibile a 6 stati). La variabile `task`, esportata da `init_simulink_hm3` (riga 64), decide quale ramo e' attivo a inizio simulazione. Task 2 e Task 3 usano lo *stesso* ramo: differiscono solo per i valori dentro `A_full/Bdelta_full`, che cambiano quando si passano `mu_alpha_scale` e `mu_c_scale`. Questa e' proprio la definizione operativa di studio di robustezza: **cambia il veicolo, non il controllore.**

**D: Perche' il notch e' centrato su `p0.wBM` e i guadagni sono calcolati su `p0`, mentre le matrici del plant usano `p`?** R: Perche' e' la differenza fra quello che il controllore sa e quello che il veicolo e'. Il controllore vola con la sua taratura nominale: non conosce il vero `A_6`, non conosce il vero `K_1`. Il plant, invece,

e' perturbato. Se ri-tarassimo il PD su ogni corner, non staremmo piu' misurando la robustezza del progetto ma la bravura del tuner: e infatti il commento alle righe 29-31 avverte che il re-tuning sul corner destabilizza il loop con il bending. Nel caso specifico gli scale factor toccano solo  $A_6$  e  $K_1$ , quindi  $p.wBM == p0.wBM$  e il notch e' identico nei due casi – ma la struttura del codice e' quella giusta e resta corretta se domani si perturbasse anche  $\omega_{BM}$  (cosa che `main_montecarlo.m` fa).

**D: Il modello Simulink implementa davvero il controllore che hai consegnato nel report?** R: Per **Task 1** si', per **Task 2 e 3** no – ed e' onesto dirlo. La riga 34 di `init_simulink_hm3.m` tara il PD sul plant *rigido* con attuatore *ideale* ( $K_p = 1.784$ ,  $K_d = 0.443$ ), che e' il progetto di Task 1; i `main_task2/3` ri-tarano invece sul loop completo ( $K_p = 1.732$ ,  $K_d = 0.687$ ). Il loop simulato resta stabile, ma con margine di fase rigido 14.6 deg invece di 30 deg. La validazione script-vs-Simulink resta valida (entrambi i track usano la stessa  $K$ ), ma valida la configurazione “before re-tuning”. La correzione e' di una riga: condizionare il progetto del controllore al valore di `task`.

**D: Cosa succede se si dimentica di rilanciare `init_simulink_hm3` dopo aver cambiato un guadagno negli script?** R: Il modello simula tranquillamente **con i vecchi valori**, perche' i blocchi leggono le variabili gia' presenti nel base workspace e non hanno modo di sapere che l'.m e' cambiato. Non c'e' nessun meccanismo di invalidazione: questa e' la fragilita' strutturale della strategia “push nel base workspace”, e il motivo per cui l'header del file (riga 17) raccomanda “*Re-run whenever a gain changes*”. Una versione robusta userebbe `Simulink.SimulationInput` con `setVariable`, cosi' che i parametri viaggino con la singola simulazione invece di vivere in uno stato globale.

# HM3/run\_simulink\_closed\_loop.m

## Ruolo del file nel progetto

`run_simulink_closed_loop.m` e' il **driver di validazione** del track Simulink. Fa tre cose in sequenza: (1) chiama `init_simulink_hm3` per popolare il base workspace con guadagni, matrici, filtri e vento; (2) ricostruisce con gli stessi dati la **baseline analitica** degli script (`assemble_loop` + `simulate_gust_response`); (3) simula `models/hm3_closed_loop.slx` con `sim()`, estrae i segnali loggati e **sovrappone** le due risposte in un'unica figura, `figures/task<N>_simulink_vs_script.png`.

E' l'unico punto della repo in cui i due track si toccano. La logica e' quella di un **test di equivalenza**: due implementazioni indipendenti dello stesso anello chiuso – una in funzioni di trasferimento integrata con `lsim`, l'altra come diagramma a blocchi integrato con `ode45` – devono produrre la stessa storia temporale a fronte dello **stesso identico vento**. La riga 46 e' la chiave del rigore: il vento non viene rigenerato, viene **rimesso in gioco** leggendolo dal `timeseries` che il modello stesso usa (`S.wind_ts`), cosi' che l'unica differenza fra i due track sia numerica e non di ingresso.

Ordine di chiamata: `run_simulink_closed_loop(1|2|3, ...)` a mano dall'utente. Dipende da `init_simulink_hm3`, `build_plant_rigid`, `build_plant_full`, `build_tvc`, `build_notch_filter`, `assemble_loop`, `simulate_gust_response`, e dal file `models/hm3_closed_loop.slx`, che **non genera** (se manca, stampa un messaggio e ritorna []).

Due cose vanno dette subito, perche' sono il punto debole del file: **non calcola nessuna metrica di scostamento** (il confronto e' puramente visivo: produce una figura, non un numero) e **non esiste nessun test** in `HM3/tests/` che copra il track Simulink (verificato con `grep`: nessun match su `simulink` o `hm3_closed_loop`). I numeri di accordo citati nel report sono stati calcolati fuori da questa funzione.

---

## Firma, arguments e guardia sul modello (righe 1-29)

```
function out = run_simulink_closed_loop(task, o)
here = fileparts(mfilename('fullpath'));
model = 'hm3_closed_loop';
mdlfile = fullfile(here,'models',[model '.slx']);
if ~isfile(mdlfile)
    fprintf(['[run_simulink_closed_loop] Model not found:\n %s\n' ...
            'Build it first by following models/SIMULINK_GUIDE.md ...']);
    out = []; return;
end
```

- Righe 13-19: l'`arguments` block replica quello di `init_simulink_hm3` meno l'opzione `push` (che qui deve restare `true`, altrimenti il modello non troverebbe le variabili). Le opzioni vengono poi inoltrate in blocco alla riga 32 con `namedargs2cell(o)`, che trasforma la struct name-value in una cell array `{'mu_alpha_scale',1.3,'mu_c_scale',0.7,...}` da splattare nella chiamata: e' il modo canonico per fare *pass-through* di name-value senza ricopiarli uno a uno.



- Righe 21-23: `mfilename('fullpath')` rende la funzione **indipendente dalla current directory**: tutti i percorsi (modello, figure) sono ancorati alla cartella HM3/, non a dove l'utente si trova.
- Righe 24-29: **guardia**. Lo `.slx` è un binario che gli agenti/script non generano: se manca, la funzione non esplode ma stampa l'istruzione ("costruisci il modello seguendo `models/SIMULINK_GUIDE.md`") e ritorna []. Il commento in testata (righe 9-10) dice *"The .slx is built by hand ..., not auto-generated"* – ed è coerente con lo stato della repo, dove il `.slx` esiste ed è committato.

## Init del workspace e baseline degli script (righe 31-47)

```
optArgs = namedargs2cell(o);
S = init_simulink_hm3(task, optArgs{:});
p = S.p;
if task == 1
    G = build_plant_rigid(p); Wact = [];
else
    G = build_plant_full(p, 'ins');
    Wact = build_tvc(p, 3) * build_notch_filter(p.wBM, 0.002, 0.7, +1);
end
K.Kp_th=S.Kp_th; ...
[~,T] = assemble_loop(G,K,Wact);
w = struct('t', S.wind_ts.Time(:), 'alphaw', squeeze(S.wind_ts.Data), 'V', p.V);
rs = simulate_gust_response(T,w);
```

- Riga 33: `init_simulink_hm3` viene chiamato **prima** di tutto. È lui che spara le 22 variabili nel base workspace e restituisce `S`. Il fatto che la funzione poi legga i guadagni **da S** (riga 43) e non li ricalcoli è proprio ciò che garantisce che baseline e modello condividano lo stesso `K`.
- Righe 37-42: la baseline analitica viene ricostruita **replicando a mano** ciò che il modello contiene. Per `task == 1`: plant rigido, `Wact = []` che `assemble_loop` interpreta come `tf(1)` (attuatore ideale) – nel modello è il ramo `Rigid_Ideal`, dove `delta` è semplicemente `u_pd` derivato dalla linea. Per `task >= 2`: plant flessibile 'ins' (bending nelle misure) e `Wact = TVC * notch`.
- Riga 41, **dettaglio sull'ordine**: nello script la catena è `build_tvc(...) * build_notch_filter(...)`, cioè `TVC poi notch`; nel modello (verificato nell'XML dello `.slx`, `system_22.xml`) le linee sono `u_pd -> Notch_Hx -> TVC -> plant`, cioè `notch poi TVC`. **Non è un bug**: sono due blocchi SISO LTI in serie e il prodotto è commutativo, la funzione di trasferimento risultante è identica. Cambiano solo le realizzazioni di stato interne (e quindi, marginalmente, gli errori di arrotondamento) – che è esattamente il tipo di indipendenza che si vuole in una validazione incrociata.
- Riga 41, **insidia latente**: qui il notch è centrato su `p.wBM` (parametri **perturbati**), mentre `init_simulink_hm3` (riga 56) lo centra su `p0.wBM` (**nominale**). Oggi non fa differenza, perché `mu_alpha_scale` e `mu_c_scale` toccano solo `A_6` e `K_1` e mai `omega_BM`. Ma se domani si perturbasse `omega_BM` (cosa che `main_montecarlo.m` fa), i due track userebbero **notch diversi** e la sovrapposizione fallirebbe – non per un errore di Simulink, ma per questa asimmetria. Vale la pena saperlo.
- Riga 44: `assemble_loop(G,K,Wact)` restituisce `T`, il closed loop `{alpha_w, theta_ref} -> {theta, z, zdot, delta}`.
- Righe 45-46: **il punto più importante del file**. Il vento non viene ri-generato con `load_wind_profile`: viene ricostruito **dal timeseries che il modello legge**, con `S.wind_ts.Time` e `S.wind_ts.Data`. Se lo si rigenerasse, per il gust analitico si otterrebbe lo stesso risultato – ma per `profile = 'strongwind'` (che gira il generatore stocastico del professore) si potrebbe ottenere una realizzazione diversa, e la sovrapposizione perderebbe ogni significato. Così com'è, **è garantito per costruzione** che i due track vedano lo stesso `alpha_w(t)` campione per campione.
- Riga 47: `simulate_gust_response(T,w)` fa `lsim(T, [alphaw, 0], t)` sulla griglia uniforme del vento (`dt = 0.005 s`).

**Possibile domanda d'esame** – Se `lsim` è esatto per un sistema LTI, perché le due curve non coincidono a machine precision? *Risposta*: `lsim` è esatto **dato un modello di ricostruzione**



**dell'ingresso** (mantiene l'ingresso costante o lineare fra due campioni della griglia) e discretizza esattamente la dinamica su quel passo. Simulink invece integra con `ode45` a passo variabile e ricostruisce l'ingresso interpolando linearmente il `timeseries` del blocco `From Workspace` sui suoi istanti. Le due sorgenti di differenza sono quindi: (a) la **ricostruzione dell'ingresso** su griglie diverse, e (b) l'**errore di integrazione** di `ode45` (controllato da `RelTol = 1e-6`). Entrambe sono  $O(1e-6)$  relative, e infatti e' quello che si misura.

## Simulazione del modello (righe 49-57)

```
addpath(fullfile(here,'models'));
so = sim(model, 'StopTime', num2str(S.Tstop));
get_ts = @(nm) get_logged_signal(so, nm);
[sl.t, sl.theta] = get_ts('theta_sl');
[~, sl.z] = get_ts('z_sl');
[~, sl.zdot] = get_ts('zdot_sl');
[~, sl.delta] = get_ts('delta_sl');
```

- Riga 50: `addpath` sulla cartella `models/` perche' `sim('hm3_closed_loop')` cerca il modello **sul path**, non nella cartella corrente. Effetto collaterale: il path resta sporcato dopo l'uscita (nessun `onCleanup` che lo rimuova). Difetto minore ma reale.
- Riga 51: `sim(model, 'StopTime', num2str(S.Tstop))` – forma comando classica, con override esplicito dello stop time. Nota che il modello ha gia' `StopTime = 'Tstop'` nella sua config (letto dallo `.slx`), quindi l'override e' **ridondante ma esplicito**: rende il codice leggibile senza dover aprire il modello. `Tstop` vale 12 s con il vento di default.
- Riga 51, cosa restituisce: nel modello `ReturnWorkspaceOutputs = 'on'` (letto in `configSet0.xml`), quindi `so` e' un oggetto `Simulink.SimulationOutput` e le variabili dei blocchi `To Workspace` (`theta_sl`, `z_sl`, `zdot_sl`, `delta_sl`, salvate in formato `Timeseries`) ne diventano **proprietà**, invece di essere sparate nel base workspace.
- Righe 53-57: estrazione dei quattro segnali con un handle anonimo. **Il contratto e' sui nomi delle variabili, non su quelli dei blocchi**: nel root del modello i quattro `To Workspace` si chiamano `log_theta`, `log_z`, `log_zdot`, `log_delta` (verificato nell'XML), ma cio' che questa funzione cerca sono i loro `VariableName`: `theta_sl`, `z_sl`, `zdot_sl`, `delta_sl`. Rinominare un blocco e' quindi innocuo; cambiare un `VariableName` rompe la validazione – e, come si vede sotto, la rompe *in silenzio*.

## Configurazione del solver (dalla config del modello, non dallo script)

Estratta direttamente da `hm3_closed_loop.slx` (`simulink/configSet0.xml`):

```
SolverName = ode45          (Dormand-Prince, esplicito, PASSO VARIABILE)
RelTol      = 1e-6
AbsTol      = auto
MaxStep     = auto,        FixedStep = auto
StopTime    = Tstop       (variabile di workspace)
```

Vale la pena **derivare** perche' questa e' la scelta giusta, perche' l'istinto (“c'e' un modo poco smorzato -> sistema stiff -> usa `ode15s`”) porta fuori strada.

Poli del loop chiuso di Task 2 (calcolati): il piu' lento ha  $|\lambda| = 0.233$  rad/s (il modo di drift laterale), il piu' veloce  $|\lambda| = 256$  rad/s (i poli di Pade a  $-183.9 \pm 175.4i$  e  $-232.2$ ). **Rapporto di stiffness**  $\sim 1.1e3$ : modesto. La condizione di stabilita' di `ode45` impone  $h \sim 2.8/256 \sim 11$  ms; con `RelTol = 1e-6` l'accuratezza impone comunque passi molto piu' piccoli, quindi la stabilita' non e' il vincolo. Su un orizzonte di 12 s il costo e' irrisorio.

Il modo di bending non e' “stiff”: **e' un problema di accuratezza, non di stabilita'**. Con  $\omega_{BM} = 18.9$  rad/s e  $\zeta_{BM} = 0.005$  si ha  $Q = 1/(2\zeta) = 100$ , periodo  $0.332$  s, tempo di decadimento  $1/(\zeta\omega_{BM})$

= 10.6 s. Su 12 s di simulazione il modo compie ~36 periodi e **non fa in tempo a smorzarsi**. Un solver che introduce smorzamento numerico (le formule BDF di `ode15s`, o `ode23tb`) attenuerebbe artificialmente proprio il modo che il notch deve tenere sotto controllo, e la sovrapposizione con la baseline `lsim` peggiorerebbe di ordini di grandezza. `ode45` esplicito con tolleranza stretta e' la scelta conservativa: **paga passi piccoli, ma non falsifica l'ampiezza della risonanza**.

## Algebraic loop: non c'e', e si puo' dimostrare

Percorso di retroazione nel modello: `Controller_PD` (Gain -> feedthrough diretto) -> `Plant_and_Actuator` -> `Demux` -> `Meas_Mux` -> `Controller_PD`. Il loop si spezza dentro il variant:

- ramo `Full_TVC_Notch`: `Notch_Hx` e' un **Transfer Fcn biproprio** (num e den entrambi di grado 2) e quindi **ha feedthrough**; ma subito dopo c'e' il blocco TVC, con numeratore di grado 3 e denominatore di grado 5, cioe' **strettamente proprio** (grado relativo 2, nessun feedthrough). Il plant State-Space ha `D = zeros(7,2)`. Il feedthrough e' interrotto **due volte**.
- ramo `Rigid_Ideal`: `u_pd` -> `Mux` -> State-Space con `D = zeros(7,2)`: nessun feedthrough. (Il ramo `delta = u_pd` e' feedthrough, ma va solo a un `To Workspace`, non chiude nessun anello.)

Quindi Simulink non ha nulla da risolvere iterativamente: nessun warning di algebraic loop, nessun solutore algebrico. **Ma e' fragile**: il notch e' biproprio, e basterebbe che il plant avesse un `D` non nullo (per esempio se la misura di `theta` contenesse un termine proporzionale a `delta`) perche' l'anello diventasse algebrico e Simulink dovesse risolvere il punto fisso a ogni passo. Il fatto che `D = 0` non e' un caso: e' la struttura fisica del problema (la deflessione dell'ugello genera un momento, quindi accelerazione, quindi arriva sulle uscite solo dopo due integrazioni).

## Overlay e figura (righe 59-83)

```
nexttile; plot(rs.t,rs.theta*180/pi,'b-', ...
              sl.t,sl.theta*180/pi,'r--','LineWidth',1.3);
```

- Righe 60-68: tre riquadri – `theta` [deg], `z` [m], `delta` [deg] – con lo script in **blu continuo** e Simulink in **rosso tratteggiato**. Convenzione grafica volutamente esplicita: se le due curve sono distinguibili a occhio, qualcosa non va.
- Righe 72-76: `theme(f,'light')` con fallback su `f.Color = 'w'` – forza il tema chiaro anche se il desktop MATLAB e' in dark mode (altrimenti le PNG esportate uscirebbero con sfondo scuro nel report).
- Righe 77-81: la figura di default (`profile = 'gust'`) si chiama `task<N>_simulink_vs_script.png`; qualunque altro profilo (es. `strongwind`) aggiunge un suffisso, cosi' che la figura del `gust` nominale non venga sovrascritta. Nella repo esistono infatti `task1_/task2_/task3_simulink_vs_script.png` e `task2_simulink_vs_script_strongwind.png`.
- Riga 83: `out = struct('script',rs,'simulink',sl)` – le due strutture vengono restituite, cosi' che il chiamante *possa* calcolarsi lo scostamento. **Ma la funzione non lo calcola**.

## Che tolleranza si accetta, e cosa significherebbe uno scostamento

Il codice **non impone nessuna soglia**. La quantificazione esiste solo nel report (`HM3/report/chapters/SimulinkReproduction` tabella alle righe 106-120), calcolata fuori da questa funzione:

| caso                 | max d theta  [rad] | max d z  [m] | max d delta  [rad] |
|----------------------|--------------------|--------------|--------------------|
| Task 1 - gust        | 3.2e-6             | 8.8e-4       | 5.4e-6             |
| Task 2 - gust        | 1.7e-7             | 1.0e-5       | 6.2e-7             |
| Task 2 - strong wind | 8.7e-7             | 7.7e-5       | 1.8e-5             |
| Task 3 - corner V3   | 3.7e-7             | 2.3e-5       | 9.4e-7             |

Da leggere in **termini relativi**, con il normalizzatore giusto: il picco a cui rapportare l'errore e' quello del **run Simulink**, che gira con i guadagni pre-retune (punto 2 piu' sotto), cioe' ~0.29 deg = 5e-3 rad – **non** gli 0.231 deg

del progetto ri-tarato di `main_task2`. Quindi  $1.7e-7$  rad e' un errore relativo  $\sim 3e-5$ , coerente con `RelTol = 1e-6` accumulato su 12 s piu' la differente ricostruzione dell'ingresso. Su  $z$  (picco  $\sim 2.2$  m) l'errore sub-millimetrico e' dello stesso ordine relativo.

Cosa significherebbe **uno scostamento vero**:

- **1e-3 relativo o piu', ma con la stessa forma d'onda** -> problema numerico: tolleranza troppo lasca, `MaxStep` troppo grande, o interpolazione del vento su una griglia troppo rada. Si stringe `RelTol` e si riconrolla.
- **Forme d'onda diverse, o segno opposto, o ampiezza doppia** -> errore **strutturale**: una riga di `C` scambiata, la `B` con le colonne invertite (delta e `alpha_w` scambiati nel Mux!), un segno sbagliato nel vettore dei guadagni, il demux ricablato nell'ordine sbagliato. E' *esattamente* la classe di errori che questa validazione esiste per pescare, e la ragione per cui il track Simulink non e' un esercizio di stile.
- **Divergenza a fine transitorio, con oscillazione a  $\sim 19$  rad/s** -> il modo di bending sta venendo smorzato numericamente da una parte e non dall'altra: solver o tolleranza sbagliati.

**Possibile domanda d'esame** – La sovrapposizione perfetta dimostra che il progetto e' corretto?

*Risposta*: no. Dimostra che le **due implementazioni** dello stesso progetto coincidono. Se il modello matematico e' sbagliato (segno di `A_6` invertito, coefficiente della traccia letto male), entrambi i track sbagliano allo stesso modo e la sovrapposizione e' perfetta lo stesso. La validazione incrociata pesca gli errori di *trascrizione*, non quelli di *modellazione*: per quelli servono altri controlli (il polo instabile a  $+\sqrt{A_6} = +1.84$  rad/s che corrisponde alla fisica, i margini che tornano con i valori attesi, la risposta al gust dell'ordine di grandezza giusto).

---

## get\_logged\_signal (righe 86-102)

```
function [t,y] = get_logged_signal(so, name)
t = []; y = [];
try
    if isprop(so,'logsout') && ~isempty(so.logsout) && ...
        any(strcmp(so.logsout.getElementNames, name))
        e = so.logsout.getElement(name);
        t = e.Values.Time; y = e.Values.Data;
    elseif isprop(so,name) || isfield(so,name)
        v = so.(name); t = v.Time; y = v.Data;
    end
catch
    warning('run_simulink_closed_loop:signal', ...
        'Could not read signal "%s".',name);
end
```

- Riga 86: helper locale. Serve perche' in Simulink **ci sono due modi** di tirar fuori un segnale da una simulazione, e la funzione li prova entrambi.
- Righe 92-95: prima strada – `logsout`, il *signal logging dataset*. Si popola quando una **linea** del modello e' marcata con `DataLogging = 'on'` (e' quello che fa `load_wind_profile` alle righe 87-89 per il modello del professore). Nel modello HM3 la config ha `SignalLogging = 'on'`, ma **nessuna linea e' marcata**, quindi `logsout` e' vuoto e questo ramo non scatta mai.
- Righe 96-97: seconda strada – i blocchi `To Workspace`. Con `ReturnWorkspaceOutputs = 'on'`, le loro variabili diventano proprieta' dell'oggetto `SimulationOutput`, quindi `so.theta_sl` e' un `timeseries` con `.Time` e `.Data`. **E' il ramo effettivamente usato**. (Nota: `isfield` su un oggetto `SimulationOutput` ritorna sempre `false`; `isprop` e' cio' che regge.)
- Righe 90 e 99-101: **degradazione silenziosa**. Se il segnale non esiste sotto nessuna delle due forme, la funzione non solleva errore: ritorna `t = [], y = []`. Il plot a valle disegnera' semplicemente la curva blu e nessuna curva rossa, e la figura verra' salvata come se nulla fosse. Il `catch` avvisa solo se l'accesso *lancia* un

errore, non se il nome semplicemente non c'è. Questo è un difetto vero: **il fallimento della validazione può passare inosservato**. Un `assert(~isempty(sl.theta), ...)` dopo la riga 57 costerebbe una riga.

---

## Discrepanze e limiti verificati

Tutto quanto segue è stato ispezionato sul codice e sullo `.slx` (aperto come archivio zip), non ipotizzato:

1. **Il ritardo puro e' Pade in ENTRAMBI i track.** Il sospetto ricorrente ("script con Pade, Simulink con blocco `Transport Delay`") **non si verifica**: nel ramo `Full_TVC_Notch` del modello ci sono solo due blocchi `Transfer Fcn`, `notch_num/notch_den` e `tvc_num/tvc_den`, e quest'ultimo *contiene già* il Pade di ordine 3 calcolato da `build_tvc`. Nessun blocco `Transport Delay` esiste nel modello. I due track condividono perciò la stessa approssimazione. Se così non fosse, la differenza sarebbe sostanziale: `exp(-tau*s)` è trascendente (fase esattamente  $-\omega\tau$  a ogni  $\omega$ , modulo 1); *Pade(3)* è una razionale all-pass che riproduce quella fase solo in banda (a  $\omega_{BM}$ :  $\omega\tau = 0.378$  rad = 21.7 deg, dove *Pade(3)* è praticamente esatto; l'errore cresce a decine di rad/s più in alto). Inoltre il blocco `Transport Delay` in Simulink è implementato con un **buffer di stati passati interpolati**, non con un'equazione di stato: introduce un errore di interpolazione proprio, forza `ode45` a passi ridotti vicino alle discontinuità, e – cosa decisiva – **non è linearizzabile**, quindi il progetto in frequenza (`margin`, `Nichols`, `allmargin`, il tuner `fminsearch` di `design_controller`) non potrebbe girarci sopra. Il Pade non è una scorciatoia: è il prezzo per poter fare controllo classico. Usare il ritardo esatto in Simulink e il Pade nello script farebbe divergere i due track **per costruzione**, e la validazione perderebbe di significato.
2. **I guadagni PD del track Simulink non sono quelli del progetto di Task 2/3.** `init_simulink_hm3` (riga 34) tara il PD sul plant *rigido* con attuatore *ideale* ( $K_p = 1.7845$ ,  $K_d = 0.4433$  = progetto Task 1); `main_task2.m` (riga 153) e `main_task3.m` (riga 23) lo ri-tarano sul loop completo ( $K_p = 1.7318$ ,  $K_d = 0.6867$ ). Sul loop completo la differenza è seria: **Rigid PM 14.6 deg contro 30.0 deg**, delay margin 98 ms contro 165 ms (entrambi stabili). La sovrapposizione script-vs-Simulink *funziona lo stesso*, perché la riga 43 di questo file prende  $K$  da  $S$  – quindi anche la baseline analitica è costruita con i guadagni "sbagliati". La validazione è **internamente coerente ma valida il loop pre-retune**, non quello consegnato.
3. **Orizzonte diverso.** Qui si simula fino a  $T_{stop} = 12$  s (default di `load_wind_profile`), mentre `main_task2/3` usano  $T_{end} = 80$  s per catturare il modo di drift lento ( $\tau \sim 20$  s). Confrontando le figure Simulink con quelle del report, gli assi temporali non coincidono. (Per i picchi non cambia nulla: verificato che sul corner V3 i massimi cadono comunque entro i primi 12 s.)
4. **Nessuna asserzione, nessun test.** La funzione produce una figura, non un verdetto. Non c'è un `max(abs(rs.theta - interp1(sl.t,sl.theta,rs.t)))` confrontato con una soglia, e in `HM3/tests/` non c'è nessun test che tocchi il track Simulink. La tolleranza "accettata" è quella riportata a mano nel report.
5. **Numeri stantii nella guida.** `models/SIMULINK_GUIDE.md` (righe 11-13) dichiara accordo " $\sim 1e-7$  rad per Task 1 e  $\sim 1e-8$  rad per Task 2", mentre la tabella del report misura  $3.2e-6$  (Task 1) e  $1.7e-7$  (Task 2). E le righe 222-224 della guida danno per il corner V3 picchi di 0.35 deg / 4.2 m "matching `main_task3.m`": `main_task3.m` dà 0.878 deg / 4.92 m, e il modello così inizializzato dà 1.402 deg / 4.88 m. La guida non coincide con nessuno dei due.

---

## Possibili domande d'esame

**D: A cosa serve davvero il track Simulink, se gli script producono già tutti i risultati?** R: A due cose. **Validazione incrociata**: due implementazioni indipendenti dello stesso anello (trasferimento + `lsim` da un lato; blocchi + `ode45` dall'altro, con realizzazioni di stato e ordine dei blocchi diversi) che devono dare la stessa risposta temporale al medesimo vento. Un segno invertito nel PD, le colonne di  $B$  scambiate, il demux ricablato male romperebbero la sovrapposizione. È un test di *trascrizione*. Seconda ragione: il controllo d'assetto **si consegna come modello** (Simulink  $\rightarrow$  autocodifica  $\rightarrow$  HIL), non come script di analisi; il diagramma è il formato con cui il progetto entra in un flusso industriale.

**D: Quale solver usa il modello e perché non uno stiff, visto che c'è un modo a  $\zeta = 0.005$ ?** R: ode45 a passo variabile, RelTol =  $1e-6$  (letto dalla configurazione del modello). Un modo poco smorzato **non è un modo stiff**: la stiffness nasce da scale temporali *decadenti* molto separate, e qui il rapporto fra il polo più veloce (Pade,  $|\lambda| = 256$  rad/s) e il più lento (drift, 0.233 rad/s) è solo  $\sim 1.1e3$  – modesto, e su 12 s di simulazione un solver esplicito lo digerisce senza problemi. Il bending pone invece un problema di **accuratezza**: con  $Q = 100$  e tempo di decadimento 10.6 s, il modo compie  $\sim 36$  periodi senza smorzarsi, e un solver implicito con smorzamento numerico (BDF di ode15s, ode23tb) lo attenuerebbe artificialmente – falsificando proprio la grandezza che il notch deve controllare. ode45 con tolleranza stretta paga passi piccoli ma preserva l'ampiezza della risonanza.

**D: Nel modello c'è un algebraic loop?** R: No, e si dimostra guardando i feedthrough. Il Gain del PD ha feedthrough; il Transfer Fcn del notch è biproprio (grado 2 su grado 2) e quindi anche lui ha feedthrough; ma il Transfer Fcn del TVC è strettamente proprio (numeratore di grado 3, denominatore di grado 5) e il blocco State-Space ha  $D = \text{zeros}(7,2)$ . La catena di feedthrough è interrotta due volte, quindi il ciclo di retroazione non è algebrico. È fragile però: se il plant avesse un  $D$  non nullo, il notch biproprio chiuderebbe l'anello algebrico e Simulink dovrebbe risolvere un punto fisso a ogni passo. Che  $D = 0$  non è un caso: la deflessione dell'ugello agisce sull'accelerazione, e arriva alle uscite solo dopo due integrazioni.

**D: Che tolleranza di accordo si accetta fra i due track, e cosa vorrebbe dire uno scostamento?** R: Il codice non ne impone nessuna – produce una figura, non un numero (ed è un limite). Le misure riportate nel report stanno a  $\max|d\theta| = 1.7e-7$  rad (Task 2) e  $3.2e-6$  rad (Task 1), con drift sub-millimetrico: rapportate al picco del run Simulink ( $\sim 5e-3$  rad) sono errori relativi  $\sim 3e-5$  per il Task 2, cioè **errore numerico puro** (RelTol =  $1e-6$  + diversa ricostruzione dell'ingresso: lsim mantiene l'ingresso fra i campioni, From Workspace lo interpola linearmente sugli istanti di ode45). Uno scostamento di ordine  $1e-3$  relativo, ma con la stessa forma d'onda, indicherebbe un problema di tolleranza/passi; forme d'onda diverse, segno opposto o ampiezza raddoppiata indicherebbero un errore **strutturale** di cablaggio – ed è quella la classe di errori che questa validazione esiste per catturare.

**D: Come fai a garantire che i due track vedano lo stesso vento?** R: Non lo rigenero. La riga 46 ricostruisce il vento dal timeseries che il modello stesso legge (S.wind\_ts.Time, S.wind\_ts.Data), quindi lo stesso oggetto alimenta il blocco From Workspace di Simulink e il lsim della baseline. È essenziale con profile = 'strongwind', dove il vento viene dal generatore stocastico del professore: rigenerarlo darebbe una realizzazione diversa (anche a seme fisso, con orizzonti diversi) e la sovrapposizione perderebbe significato.

**D: Se domani cambi il VariableName di un blocco To Workspace, cosa succede?** R: Niente di visibile, ed è un problema. (Rinominare il blocco – oggi log\_theta, log\_z, log\_zdot, log\_delta – non fa invece nulla: il contratto è sul VariableName, non sul nome del blocco.) get\_logged\_signal (righe 86-102) cerca il nome prima in logsout, poi fra le proprietà del SimulationOutput; se non lo trova ritorna  $t = []$ ,  $y = []$  senza errore né warning (il catch scatta solo se l'accesso lancia). La figura viene disegnata con la sola curva blu dello script e salvata come se tutto fosse a posto. Il contratto sui nomi (theta\_sl, z\_sl, zdot\_sl, delta\_sl) è documentato ma non verificato: basterebbe un assert(~isempty(sl.theta)) dopo la riga 57.

**D: Il modello Simulink è il progetto che hai consegnato in Task 2?** R: Onestamente no. La baseline e il modello girano entrambi con i guadagni tarati da init\_simulink\_hm3 sul plant rigido con attuatore ideale ( $K_p = 1.784$ ,  $K_d = 0.443$ , cioè il progetto di Task 1), non con quelli ri-tarati sul loop completo dai main\_task2/3 ( $K_p = 1.732$ ,  $K_d = 0.687$ ). Il loop resta stabile ma con margine di fase rigido 14.6 deg invece di 30 deg – cioè la configurazione che main\_task2.m stampa come “BEFORE re-tuning”. La validazione resta valida come test di equivalenza (i due track condividono la stessa  $K$ ), ma valida il loop pre-retune. È un bug da una riga in init\_simulink\_hm3.m (riga 34), non un problema del modello.

# HM3/LTV\_FULL\_ASCENT/ode\_lpv\_ascent

## Ruolo del file nel progetto

Questo file e' il **cuore numerico** dell'estensione LPV di HM3: e' la right-hand side (RHS) che `ode45` valuta per propagare il modello rigido di beccheggio **lungo tutta l'ascesa (0-140 s)**, non piu' congelato a max-q. La traccia dell'homework chiede solo il punto di progetto a  $t = 72$  s (pressione dinamica massima), dove il sistema e' LTI e si puo' fare tutta l'analisi classica in frequenza (Nichols, margini, notch). Qui invece i coefficienti del plant diventano funzioni del tempo e il sistema e' un LTV (o LPV, se si legge il tempo come parametro di scheduling): non esiste piu' una funzione di trasferimento, e l'unico modo onesto di sapere che cosa fa il veicolo e' **integrare davvero le equazioni**.

Il file e' il gemello dinamico di `HM3/build_plant_rigid.m`. Quello costruisce la quadrupla (A, B, C, D) con i coefficienti valutati a un istante fissato; questo prende le stesse identiche equazioni e sostituisce ogni coefficiente con un interpolante nel tempo, richiamato **a ogni valutazione della RHS**. La struttura M che riceve in ingresso e' preparata da `HM3/LTV_FULL_ASCENT/init_simulink_lpv.m` (griglie + `griddedInterpolant`) e impacchettata dalla helper locale `make_model` in `HM3/LTV_FULL_ASCENT/main_full_ascent.m` (righe 123-134).

Dentro la RHS e' chiuso **anche l'anello di controllo**: non c'e' un blocco "controllore" separato, la legge PD e' scritta in linea (riga 22). Questo rende il file l'unico punto in cui coesistono plant, vento e controllore – per questo `main_full_ascent.m` lo dichiara "source of truth" e il modello Simulink `hm3_full_ascent.slx` (costruito da `build_hm3_full_ascent.m`) viene validato **contro** di esso, non viceversa.

Chi lo chiama – sono tre i punti di ingresso, non uno: `main_full_ascent.m` (righe 31-32, due integrazioni: guadagni congelati e guadagni schedulati; piu' la verifica di consistenza a  $t = 72$  s, righe 54-61), `run_full_ascent_simulink.m` (riga 49: il replay `ode45` sullo stesso vento prodotto dal modello – e' esattamente la validazione del `.slx` di cui sopra) e `main_q_scheduling.m` (riga 48: lo studio T008 sullo scheduling in  $q(t)$  invece che in  $t$ , che lavora sul plant rigido e quindi riusa questa stessa RHS). Il file **non** e' usato dallo studio flessibile: quello ha la sua RHS, `ode_lpv_flex.m`.

---

## Firma e contratto (righe 1-13)

```
function dx = ode_lpv_ascent(t, x, M)
% LTV rigid pitch-plane RHS, full-ascent LPV baseline.
% ...
% No arguments validation by design: ode45 inner loop.
```

- Riga 1: firma (t, x, M). Lo stato e'  $x = [z; \dot{z}; \theta; \dot{\theta}]$  (4 stati):  $z$  [m] deriva laterale rispetto alla traiettoria di riferimento,  $\theta$  [rad] assetto di beccheggio. L'uscita  $dx$  e'  $4 \times 1$ .
- Righe 2-9: la docstring dichiara esattamente cosa e' dentro M: `fc1..fc7` (interpolanti dei coefficienti), `windfun`, `fKp/fKd` (schedule dei guadagni), i guadagni congelati e il flag `sched`.
- Righe 11-12: **nessun blocco arguments, per scelta**. Questa funzione sta nel loop interno di `ode45` e viene chiamata dell'ordine di  $1e5$  volte per simulazione; un blocco di validazione la' dentro costerebbe piu' della matematica che esegue. La validazione vive al confine del run, in `init_simulink_lpv` (che ha il suo

arguments, righe 32-37). E' la stessa convenzione dichiarata nel CLAUDE.md del repo per ODE RHS, residui di shooting e callback di fmincon.

---

## Perche' il sistema e' LPV (e non solo "non lineare")

Le equazioni sono **lineari nello stato**, ma i coefficienti dipendono dal tempo. Il modello rigido di HM3 (Eq. 1 della traccia, senza bending) e':

$$\begin{aligned} \ddot{z} &= a1 \cdot \dot{z} + (a1 \cdot V + a4) \cdot \theta + a3 \cdot \delta - a1 \cdot V \cdot \alpha_w \\ \ddot{\theta} &= (A6/V) \cdot \dot{z} + A6 \cdot \theta + K1 \cdot \delta - A6 \cdot \alpha_w \end{aligned}$$

Chi varia lungo l'ascesa, e perche':

- **A6 =  $\mu_\alpha$**  [ $1/s^2$  per rad] – momento aerodinamico. E' il termine **destabilizzante**: il sottosistema di solo assetto e'  $\ddot{\theta} = A6 \cdot \theta$ , che ha poli in  $\pm \sqrt{A6}$ . Con  $A6 > 0$  (centro di pressione davanti al baricentro) c'e' sempre un polo reale positivo: il razzo e' staticamente instabile in aria e **non puo' volare senza controllo attivo**.  $A6$  e' proporzionale alla forza normale aerodinamica, quindi cresce con la pressione dinamica: il suo massimo cade nella regione di max-q, ed e' esattamente per questo che HM3 progetta li'.
- **K1 =  $\mu_c$**  [ $1/s^2$  per rad] – efficacia del TVC (spinta per braccio diviso inerzia). Varia perche' variano spinta, massa e inerzia mentre il primo stadio consuma propellente.
- **V(t)** [m/s] – velocita' relativa. Entra tre volte: in  $a1 \cdot V$ , in  $A6/V$  e nel termine di vento. Parte da 0 al lift-off (vedi la guardia **Vsafe**, piu' sotto) e cresce fino a valori supersonici.
- **a1, a3, a4** – coefficienti della dinamica traslazionale ( $a1 \sim -N_\alpha/(mV)$ ,  $a1V + a4 \sim -(N_\alpha + T - D)/m$ ). Anche loro sono tabulati nel tempo nel dataset.
- **q\_bar(t)** (campo Q del dataset) non entra nella RHS, ma serve a valle come indicatore di carico (`main_full_ascent.m`, riga 154).

Conseguenza concreta: **il plant congelato e' una comodita' di progetto, non il sistema vero**. Congelare significa scrivere  $A(t)$  e fare finta che  $A(t) = A(t_0)$  per sempre. Propagare la dinamica vera vuol dire chiamare  $A(t)$  a ogni step del solver – ed e' quello che fanno le righe 25-28.

**Possibile domanda d'esame** – perche' il segno di  $A6$  e' cosi' importante? *Risposta:*  $A6 > 0$  significa che il momento aerodinamico amplifica la perturbazione di assetto (cp davanti al cg): il polo  $+\sqrt{A6}$  e' in semipiano destro. La costante di tempo di divergenza e'  $1/\sqrt{A6}$ , che a max-q e' dell'ordine del mezzo secondo: il controllo deve avere banda nettamente superiore. Se  $A6$  fosse negativo il veicolo sarebbe aerodinamicamente stabile (come una freccia) e il problema di controllo sarebbe un altro problema.

---

## Vento: $\alpha_w = M.\text{windfun}(t)$ (riga 14)

```
aw = M.windfun(t); % wind angle of attack alpha_w(t)
```

- Riga 14: il vento **non e' un forzamento tabulato a parte, e' un ingresso del plant valutato dentro la RHS**. `M.windfun` e' un `griddedInterpolant` costruito in `init_simulink_lpv.m` alla riga 107 su `wg.t`, la griglia temporale prodotta dal generatore del professore (`General/hw3-v3/strong_wind.slx`, eseguito una volta sola alle righe 87-90, con la sua media `v_wp` piu' la turbolenza di Dryden).
- Il segnale interpolato **non e' la velocita' di vento**, e'  $\alpha_w = v_w / V$  gia' adimensionalizzato (righe 89-90 di `init_simulink_lpv`: la riga 89 costruisce `Vwg = max(interp1(V), 1)` a proteggere la divisione, la riga 90 forma `alpha_w = wg.vw ./ Vwg`). Quindi  $\alpha_w$  e' un **angolo [rad]**, e la RHS lo tratta come tale.
- Convenzione di segno: nella RHS il vento entra con il **meno** in entrambi i canali (`- M.fc4(t)*aw`, `- M.fc6(t)*aw`). Non e' arbitrario: raccogliendo il canale di beccheggio si legge



```
thetaddot = A6*(theta + zdot/V - alpha_w) + K1*delta
```

cioe' l'angolo d'attacco aerodinamico **effettivo** e'  $\alpha_{aero} = \theta + \dot{z}/V - \alpha_w$ . E' fisicamente corretto: conta la velocita' **relativa** all'aria, e la componente laterale relativa e'  $(\dot{z} - v_w)$ ; dividendo per  $V$  si ottiene proprio  $\dot{z}/V - \alpha_w$ . Lo stesso raccoglimento funziona sul canale traslazionale, perche'  $c2 = a1*V + a4$  e  $c4 = a1*V$  condividono il fattore  $a1*V$ .

**Incoerenza da segnalare.** L'indicatore di carico in `main_full_ascent.m`, riga 153, calcola  $\alpha = x(:,3) + x(:,2)/V + S.\text{windfun}(tt)$ , cioe' con il **piu'** davanti a  $\alpha_w$ , e il README della cartella scrive la stessa formula ( $\alpha_{total} = \theta + \dot{z}/V + \alpha_w$ ). Questa non e' la stessa  $\alpha$  che guida il plant: le due convenzioni non possono essere entrambe giuste, e la RHS (che e' la dinamica) usa il meno. Il grafico  $q_{bar}*\alpha$  va quindi letto con cautela – il modulo del picco cambia a seconda del segno scelto.

## Il controllore chiuso dentro la RHS (righe 17-22)

```
if M.sched
    Kp = M.fKp(t); Kd = M.fKd(t);
else
    Kp = M.Kp_th0; Kd = M.Kd_th0;
end
delta = -(Kp*x(3) + Kd*x(4) + M.Kp_z*x(1) + M.Kd_z*x(2));
```

- Righe 17-21: l'unico "if" della funzione. Il flag `M.sched` sceglie fra i **guadagni congelati** (il PD progettato una volta a max-q, `init_simulink_lpv.m` righe 71-72) e la **schedule**  $K_p(t)$ ,  $K_d(t)$  (righe 74-85: un PD ridisegnato per ogni punto della griglia `tsched`).
- **Attenzione: il warm-start della schedule non esiste davvero.** I commenti di `init_simulink_lpv.m` promettono una continuation (riga 74, "warm-started (continuation)"; riga 84, `Kprev = [Kk.Kp_th Kk.Kd_th]; % continuation`) e la riga 82 passa effettivamente 'K0', `Kprev` a `design_controller`. Ma `design_controller.m` dichiara `o.K0 (1,2) ... = [0 0] % accepted, unused` (riga 32) e la docstring lo dice esplicitamente: "K0 ignored (kept for call compatibility)" (riga 19). Il tuner riparte **sempre** dallo stesso punto iniziale in forma chiusa di D'Antuono,  $x0 = \log([2*A6/K1, \sqrt{A6}/K1])$  (riga 55): ogni nodo della griglia e' un `fminsearch` indipendente. Il commento nel sorgente e' stale – da sapere, se all'orale il professore chiede "e la continuation dove la vedo?".
- Riga 22: legge di controllo.  $\theta_{ref} = 0$  non compare esplicitamente: e' assorbito nel segno meno (regolazione a zero). La struttura coincide esattamente con quella di `HM3/assemble_loop.m` (riga 25), quindi la RHS e l'analisi in frequenza chiudono **lo stesso anello**: e' questo che rende legittimo confrontare margini congelati e simulazione LTV.
- **Attuatore ideale:**  $\delta$  calcolato alla riga 22 va *direttamente* nel plant. Niente TVC, niente ritardo di 20 ms, niente notch, nessuna saturazione ne' limite di rate. E' la stessa ipotesi del Task 1 di HM3 (`assemble_loop(G, K, [])`). Tutta la parte "vera" dell'attuazione compare solo in `ode_lpv_flex.m`.
- **I guadagni di drift non sono mai schedulati:**  $M.Kp_z$  e  $M.Kd_z$  sono costanti anche quando `sched = 1` (vedi `make_model`, riga 133 di `main_full_ascent.m`, che passa sempre `S.K0.Kp_z`). Inoltre non sono nemmeno *tuned*: sono i default di `design_controller.m` (righe 28-29,  $Kp_z = Kd_z = -1e-3$ ). Il progetto ottimizza solo la coppia  $(Kp_{th}, Kd_{th})$ .
- Segno: essendo  $Kp_z$  negativo, il termine  $-(M.Kp_z*x(1))$  e' **positivo** in  $z$ . E' una scelta del progetto originale, non un errore di questo file.

## Il plant tempo-variante (righe 25-28)

```
dx = [ x(2);
       M.fc1(t)*x(2) + M.fc2(t)*x(3) + M.fc3(t)*delta ...
       - M.fc4(t)*aw;
```



```
x(4);
M.fc5(t)*x(2) + M.fc6(t)*x(3) + M.fc7(t)*delta ...
- M.fc6(t)*aw ];
```

- Righe 25-28: e' riga per riga la matrice A di `build_plant_rigid.m` (righe 11-14) con `p.a1 -> fc1(t)`, `p.a1*p.V + p.a4 -> fc2(t)`, `p.A6/p.V -> fc5(t)`, `p.A6 -> fc6(t)`, e le colonne di ingresso `Bd = [0; a3; 0; K1] -> [fc3; fc7]`, `Bw = [0; -a1*V; 0; -A6] -> [-fc4; -fc6]` (righe 16-17 di `build_plant_rigid.m`). **La corrispondenza e' esatta**: e' questo che garantisce che, congelando gli interpolanti a 72 s, si ritrovi il modello di HM3 – ed e' proprio il test di consistenza di `main_full_ascent.m` (righe 49-61, `err_consistency` dell'ordine di  $1e-10$  rad secondo il README).
- Nota di economia: `fc6` e' usato **due volte** (coefficiente di `theta` e del vento), coerentemente con il fatto che entrambi i termini valgono `A6`. Una sola tabella, due prodotti – la stessa scelta e' replicata in Simulink (`build_hm3_full_ascent.m`, riga 86: `W('c6/1','P6/1')`; `W('c6/1','P8/1')`).
- Ogni valutazione della RHS costa **8 valutazioni di interpolante** – le 7 tabelle `fc1..fc7`, ma `fc6(t)` compare due volte nella riga 28 – **piu' 1 chiamata al vento** (e altre 2 se `sched = 1`). Con `ode45` a `RelTol = 1e-8` (`main_full_ascent.m`, riga 29) su 135 s di volo sono parecchie centinaia di migliaia di lookup: da qui la scelta di `griddedInterpolant` (oggetto precompilato) invece di `interp1` dentro il loop.

## Che cosa viene interpolato davvero (`init_simulink_lpv.m`, righe 49-107)

Questo e' il punto sottile che vale la pena saper difendere.

```
c1 = a1; % * zdot
c2 = a1.*V + a4; % * theta
c5 = A6./Vsafe; % * zdot
gi = @(y) griddedInterpolant(tg, y, 'linear', 'nearest');
```

- Righe 50-52: la griglia `tg` e' quella **del dataset** (L.V.Time, troncata a `Tstop`); tutti i segnali vengono riportati su `tg` con `interp1` (riga 52), interpolazione lineare di default. Il codice non specifica il passo della griglia: e' quello del `.mat` del professore.
- Righe 61-68: si costruiscono i **coefficienti effettivi gia' combinati**. `c2` non e' `a1`, `V`, `a4` separati: e' il **prodotto calcolato sulla griglia** e poi tabulato come una singola funzione del tempo. Idem `c5 = A6/Vsafe` e `c4 = a1*Vsafe`.
- Righe 101-107: gli interpolanti sono `griddedInterpolant(tg, y, 'linear', 'nearest')` – **lineare dentro** la griglia, **nearest fuori**. La “nearest” come metodo di *extrapolation* significa: fuori dai breakpoint il valore viene **agganciato all'estremo** (clamp). Non c'e' errore, non c'e' NaN, non c'e' warning: se per sbaglio si integrasse a  $t = 500$  s si otterrebbero i coefficienti di  $t = 140$  s, silenziosamente. E' una guardia implicita, non una guardia esplicita – va detto.
- Riga 59: `Vsafe = max(V, 1)` protegge `A6/V` e `1/V` al lift-off, dove  $V(0) = 0$ . Il commento e' esplicito. Nota pero' che la protezione e' **asimmetrica**: `c4 = a1.*Vsafe` (riga 64) usa `Vsafe`, mentre nella formulazione teorica il termine e' `a1*V`. La giustificazione nel commento e' che il fattore si semplifica con `alpha_w = v_w/Vsafe` – ma la semplificazione e' solo **approssimata**, perche' il `Vsafe` del coefficiente vive sulla griglia `tg` mentre quello dentro `alpha_w` e' stato valutato sulla griglia del vento `wg.t` (riga 89). Ai fini pratici (si parte da  $t_0 = 5$  s) e' irrilevante; concettualmente e' un piccolo hack.

## Interpolare i coefficienti grezzi o quelli combinati non e' la stessa cosa

L'interpolazione lineare **non commuta con la moltiplicazione**. Dati due breakpoint consecutivi  $t_k, t_{k+1}$  e  $s$  in  $[0,1]$ :

```
interp(a1*V)(s) = (1-s)*a1_k*V_k + s*a1_{k+1}*V_{k+1}
interp(a1)(s) * interp(V)(s)
= [(1-s)*a1_k + s*a1_{k+1}] * [(1-s)*V_k + s*V_{k+1}]
```

I due risultati coincidono solo nei nodi ( $s = 0$  e  $s = 1$ ); in mezzo differiscono di un termine  $s*(1-s)*(a1_{k+1}-a1_k)*(V_{k+1}-V_k)$  cioè di un  $O(dt^2)$  proporzionale alle variazioni dei due fattori. Conseguenza operativa, ed è un fatto verificabile nel repo:

- `ode_lpv_ascent.m` (questo file) integra le **combinazioni tabulate** ( $fc2 =$  interpolante di  $a1*V + a4$ );
- `ode_lpv_flex.m` interpola i **coefficienti grezzi** ( $fa1, fV, fa4$ , righe 19-20) e forma  $a1*V + a4$  **dentro** la RHS (riga 38).

Quindi i due file **non integrano esattamente lo stesso sistema LTV rigido**, pur partendo dallo stesso dataset. La differenza è piccola (i coefficienti sono lisci sulla griglia del dataset) e nessuno dei due è “sbagliato” – sono due discretizzazioni diverse dello stesso LPV continuo. Ma va saputo: se domani si confrontassero numero per numero le uscite dei due file, un disaccordo dell’ordine di  $O(dt^2)$  è atteso, non è un bug.

Una terza variante esiste in Simulink: `build_hm3_full_ascent.m` (righe 56-64) costruisce  $\alpha_w = (v_{wp} + \text{turb}) * \text{invV}(t)$  moltiplicando il vento *istantaneo* del generatore per la **lookup** `invV`, mentre la RHS interpola  $\alpha_w$  già diviso. Anche qui: stessa fisica, discretizzazioni diverse. Il fatto che i due si sovrappongano a  $\sim 1e-7$  rad (README) è un risultato numerico, non un’identità algebrica.

**Possibile domanda d’esame** – è meglio tabulare  $a1*V + a4$  o interpolare  $a1, V, a4$  separatamente?

*Risposta:* Dipende da che cosa è liscio. Tabulare il coefficiente effettivo garantisce che il plant congelato ricostruito a un nodo sia esattamente quello del dataset e toglie dal loop caldo l’aritmetica di ricombinazione (un prodotto per termine, niente  $a1*V + a4$  da riformare a ogni step). Non fa però risparmiare lookup: qui servono 7 tabelle (`fc1..fc7`), mentre `ode_lpv_flex` se la cava con 6 grezze ( $fa1, fa3, fa4, fa6, fK1, fV$ , riga 19). Interpolare i grezzi è più flessibile (posso ricostruire qualunque combinazione, per esempio  $A6/V$  e  $a1*V$  con lo stesso  $V$ ) e mantiene il significato fisico dei singoli termini. Le due scelte differiscono solo a  $O(dt^2)$  fra i nodi; l’importante è non mescolarle inconsapevolmente.

## La fallacia del frozen-time (il punto centrale)

Tutto l’impianto di HM3 – Nichols, margine di guadagno 6 dB, margine di fase 30 gradi, notch – vive su plant **congelati**. La tentazione è concludere: “ho verificato che ogni LTI congelato  $A(t_i)$  è stabile per ogni  $t_i$ , dunque il sistema tempo-variante è stabile”. **E’ falso**, ed è un classico controesempio della teoria dei sistemi LTV.

Il fatto matematico: per un sistema  $\dot{x} = A(t)*x$ , gli autovalori di  $A(t)$  **non governano** la stabilità. Esistono matrici  $A(t)$  i cui autovalori congelati hanno parte reale negativa **costante** per ogni  $t$ , e la cui soluzione **diverge** esponenzialmente. L’esempio standard nei testi (Khalil, *Nonlinear Systems*) è una  $A(t)$  periodica di rotazione+scala i cui autovalori sono costanti in  $(-1 \pm i\sqrt{7})/4$  (parte reale -0.25 per ogni  $t$ ), ma la cui matrice di transizione contiene un termine  $\exp(+0.5*t)$ . Intuizione: la stabilità LTV dipende dalla **matrice di transizione**  $\Phi(t, t_0)$ , non dallo spettro istantaneo; una  $A(t)$  che ruota velocemente i propri autovettori può “pompare” energia da una direzione contraente a una espansiva prima che la contrazione abbia il tempo di agire. Vale anche il viceversa: un sistema i cui LTI congelati sono **tutti instabili** può essere LTV-stabile.

Cosa **salva** l’ingegneria: il teorema di variazione lenta. Se  $A(t)$  è stabile per ogni  $t$  con margini uniformi e  $\| \dot{A}(t) \|$  è sufficientemente piccola, allora l’LTV è esponenzialmente stabile. E’ esattamente l’ipotesi implicita del gain scheduling: “il parametro varia lentamente rispetto alla dinamica dell’anello”. Qui la dinamica di anello ha banda dell’ordine di qualche rad/s (costante di tempo  $\sim 1$  s) mentre  $A6(t), K1(t), V(t)$  cambiano su scale di decine di secondi: la separazione c’è, ma è **assunta, non dimostrata**. Il codice non calcola da nessuna parte una stima di  $\| \dot{A}(t) \|$  né una funzione di Lyapunov parameter-dependent, e non risolve nessuna LMI.

**Come il codice affronta la cosa.** In modo pragmatico e onesto:

1. `main_full_ascent.m` righe 40-47 fa lo **sweep dei margini congelati** lungo la traiettoria – che è *necessario* ma non *sufficiente*;
2. righe 31-32 **propaga davvero l’LTV** con `ode45` sullo stesso anello, con il vento vero del generatore, e guarda a posteriori se  $\theta$  e  $z$  restano limitati;

3. le figure mostrano entrambe le cose, e il README dichiara che “la integrazione LTV ode45 e’ la source of truth”.

Il limite residuo, da dire all’orale prima che lo dica l’esaminatore: **anche il punto 2 non e’ una dimostrazione di stabilita’**. E’ *una* traiettoria, con *una* realizzazione di vento, da *una* condizione iniziale ( $x_0 = \text{zeros}(4,1)$ , riga 28 di `main_full_ascent.m`). Un sistema LTV lineare non ha nemmeno bisogno di ingressi per divergere: se si volesse una prova, si dovrebbe integrare la **matrice di transizione** (4 run con le colonne dell’identita’ come condizione iniziale e ingressi nulli) e verificare che  $\|\Phi(t, t_0)\|$  decada, oppure cercare una  $P(t) > 0$  con  $\dot{P} + A'P + PA < 0$ . Nessuna delle due cose e’ nel repo.

**Possibile domanda d’esame** – se i margini congelati non bastano, perche’ li calcoliamo? *Risposta:* Perche’ sono una condizione necessaria (un LTI congelato instabile e’ quasi sempre una bandiera rossa) e perche’ sono la sola cosa su cui si sa *progettare* con metodi classici. Sono uno strumento di sintesi; la verifica va fatta sul sistema vero. Il codice rispetta questa divisione del lavoro: progetta congelato, verifica integrando l’LTV.

---

## Limiti e punti fragili (riepilogo onesto)

- **Attuatore ideale.** Niente TVC, ritardo, notch, saturazione: **delta** e’ applicato istantaneamente. Confrontare i margini di questo anello con quelli del Task 2 di HM3 e’ quindi un confronto fra cose diverse.
  - **Nessuna guardia esplicita fuori griglia.** Gli interpolanti fanno clamp silenzioso (‘nearest’). Idem  $f_{Kp}/f_{Kd}$ , definiti da  $t_0 = 5$  s: sotto i 5 s si usano i guadagni del primo nodo (la lookup Simulink usa `ExtrapMethod = 'Clip'`, righe 106-109 di `build_hm3_full_ascent.m`, proprio per replicare questo comportamento).
  - **Coerenza del segno di  $\alpha_w$ :** la RHS usa  $-\alpha_w$ , l’indicatore di carico  $q_{bar} \cdot \alpha$  in `main_full_ascent.m` (riga 153) usa  $+\alpha_w$ .
  - **I margini vengono letti da `assemble_loop`** (HM3/`assemble_loop.m`, riga 38: `getLoopTransfer(T, 'delta', -1)`), cioe’ su un anello SISO rotto su **delta** che include **anche** la retroazione di drift  $z$ ,  $z_{dot}$ . E’ la stessa convenzione di tutto HM3: qualunque critica a quella scelta si eredita qui identica.
  - **La linearizzazione su 140 s.**  $z$ ,  $z_{dot}$  sono perturbazioni normali alla traiettoria di riferimento: su tutto l’arco dell’ascesa l’ipotesi si indebolisce (lo dice il README stesso).  $z$  va letto come indicatore relativo, non come cross-range fisico.
  - Il vento non forza la deriva in modo indipendente: entra solo attraverso  $\alpha_w$ , cioe’ come angolo, non come forza laterale addizionale.
- 

## Possibili domande d’esame

**D: Che cosa rende questo sistema LPV, e perche’ non basta il modello congelato di `build_plant_rigid`?** R: I coefficienti  $A_6$  (momento aerodinamico destabilizzante),  $K_1$  (efficienza TVC),  $V$ ,  $a_1$ ,  $a_3$ ,  $a_4$  cambiano lungo l’ascesa perche’ cambiano pressione dinamica, massa, inerzia e spinta. Il modello congelato prende una loro istantanea a  $t = 72$  s e produce un LTI su cui si puo’ progettare con metodi in frequenza. Ma la dinamica vera e’  $\dot{x} = A(t)x + B(t)u$ : per propagarla bisogna interpolare i coefficienti **dentro** la RHS a ogni valutazione, ed e’ quello che fanno le righe 25-28. Il progetto rimane congelato, la verifica no.

**D: La stabilita’ di tutti i plant congelati garantisce la stabilita’ del sistema tempo-variante?** R: No, ed e’ l’errore concettuale piu’ comune. Per  $\dot{x} = A(t)x$  la stabilita’ dipende dalla matrice di transizione, non dallo spettro istantaneo: esistono controesempi classici (Khalil) con autovalori congelati costanti a parte reale  $-0.25$  e soluzioni che divergono come  $\exp(+0.5t)$ . Vale un teorema di variazione lenta – se  $\|\dot{A}\|$  e’ abbastanza piccola la conclusione diventa valida – ma “abbastanza piccola” e’ un’ipotesi che qui viene assunta, non verificata. Il codice mitiga il rischio propagando davvero l’LTV con `ode45` (righe 31-32 di `main_full_ascent.m`) invece di fidarsi dei soli margini congelati.

**D: E allora la simulazione ode45 dimostra la stabilita'?** R: Neanche. Dimostra che *quella* traiettoria, con *quel* vento e *quella* condizione iniziale, resta limitata su 140 s. Una prova richiederebbe di propagare la matrice di transizione (o esibire una funzione di Lyapunov tempo-variante / risolvere una LMI parameter-dependent). Il repo non lo fa e non lo pretende: e' evidenza numerica forte, non un certificato.

**\*\*D: Perche' l'interpolazione dei coefficienti "effettivi" ( $c2 = a1*V + a4$ ) non e' equivalente a interpolare  $a1$ ,  $V$ ,  $a4$  e moltiplicare dentro la RHS?** R: Perche' l'interpolazione lineare non commuta con il prodotto: fra due breakpoint la differenza vale  $s*(1-s)*(delta\_a1)*(delta\_V)$ , che si annulla solo nei nodi. `ode_lpv_ascent` interpola i coefficienti combinati (fc1..fc7), `ode_lpv_flex` interpola i grezzi e li combina in linea: i due file integrano quindi due sistemi LTV leggermente diversi, entrambi consistenti con il dataset ai nodi. E' una differenza  $O(dt^2)$ , non un bug, ma va dichiarata se si confrontano i risultati.

**D: Perche' non c'e' un blocco arguments in questa funzione?** R: E' un hot loop dentro `ode45`, chiamato ~1e5 volte per simulazione. La validazione degli input vive nel confine del run (`init_simulink_lpv`, che ha il suo `arguments`) e nei main. E' una scelta esplicita, dichiarata nelle righe 11-12 e nella convenzione del repo, non una dimenticanza.

**D: Come entra il vento e come e' chiuso l'anello dentro la RHS?** R: Il vento e' un `griddedInterpolant` (`M.windfun`) costruito una volta sola sull'uscita del generatore Simulink del professore, gia' convertito in angolo  $\alpha_w = v_w/V$ ; viene valutato all'istante corrente alla riga 14 ed entra con segno meno in entrambi i canali, perche' l'incidenza aerodinamica vera e'  $\theta + \dot{z}/V - \alpha_w$  (conta la velocita' relativa all'aria). L'anello e' chiuso in linea alla riga 22:  $\delta = -(K_p*\theta + K_d*\dot{\theta} + K_p*z + K_d*\dot{z})$ , attuatore ideale,  $\theta_{ref} = 0$ . Non c'e' un blocco controllore separato: plant, vento e controllore stanno tutti dentro la stessa RHS, ed e' per questo che il file e' la reference contro cui si valida il modello Simulink.

# HM3/LTV\_FULL\_ASCENT/ode\_lpv\_flex.

## Ruolo del file nel progetto

E' la RHS della **versione flessibile** dell'estensione LPV: stesso plant rigido di `ode_lpv_ascent.m`, ma con il **primo modo di bending riattivato**, la contaminazione INS delle misure, la catena di attuazione reale (TVC di secondo ordine + ritardo di trasporto approssimato con Pade) e – il punto dello studio – un **notch che puo' inseguire  $\omega_{BM}(t)$** .

Il problema che risolve. In HM3 il notch profondo del Task 2 e' centrato su  $\omega_{BM}(72) = 18.9 \text{ rad/s}$ , la frequenza del primo modo **al punto di progetto**. Ma  $\omega_{BM}$  non e' una costante: dipende da massa e rigidita generalizzate, che cambiano mentre il primo stadio brucia propellente. Sul dataset la frequenza sale (il README riporta uno sweep 16.5 -> 31.8 rad/s): un notch fisso a 18.9 rad/s **si detuna**, la risonanza esce dalla buca, il guadagno d'anello alla frequenza di bending torna sopra 0 dB e l'anello va instabile. Questo file permette di simulare i due casi – notch fisso e notch variabile – **sullo stesso identico plant LTV**, cambiando una sola cosa: il handle `M.fwn`.

Dove sta nel flusso: e' il gemello dinamico di `HM3/build_plant_full.m` (che costruisce lo stesso modello a 6 stati ma congelato), viene chiamato da `HM3/LTV_FULL_ASCENT/main_flex.m` (righe 40-41, due `ode45`: `S.fomega` per il notch variabile, `@(t) w72` per quello fisso) e riceve la struttura `M` dalla helper locale `make_flex` (righe 93-106 dello stesso file), riempita con gli interpolanti creati da `init_simulink_lpv.m` (righe 104-117). Il modello Simulink `hm3_full_ascent_flex.slx` (`build_hm3_full_ascent_flex.m`) e' il suo mirror a blocchi elementari e viene validato contro di lui, non viceversa.

Nota di scoping: qui i **guadagni PD restano congelati** a `max-q` (`make_flex` passa `'sched'`, `false`, riga 105 di `main_flex.m`). Lo showcase e' il notch, non lo scheduling. Il codice *supporta* `sched = true` (riga 26) ma nessuno script lo esercita in questa configurazione.

---

## Firma, stato e contratto (righe 1-16)

```
function dx = ode_lpv_flex(t, x, M)
% x = [z zdot theta thetadot eta etadot | xn1 xn2 | x_tvc(1:nt)]
% No arguments validation by design: ode45 inner loop.
```

- Riga 1: firma `(t, x, M)`, stessa convenzione di `ode_lpv_ascent`.
- Righe 5-9: lo stato e' **concatenato per blocchi**, e questa e' la chiave di lettura di tutto il file:
  - `x(1:6)` = plant flessibile `[z zdot theta thetadot eta etadot]`;
  - `x(7:8)` = stati del **notch** (realizzazione in forma canonica);
  - `x(9:end)` = stati del **TVC** (attuatore + ritardo di Pade).
- La docstring dichiara 13 stati:  $6 + 2 + nt$ , con  $nt = 5$  perche' il TVC e' `build_tvc(p0, 3)` = attuatore di 2 ordine per un Pade di ordine 3 (`HM3/build_tvc.m`, righe 16-21). `main_flex.m` non lo hard-codea: legge `nt = size(S.tvc.At, 1)` (riga 37) e dimensiona `x0` di conseguenza (riga 38).
- Riga 13: di nuovo **nessun blocco arguments, per design** – hot loop di `ode45`. Vale identico a `ode_lpv_ascent`.

- Riga 15: lo split (xp, xn, xt). Riga 16: destrutturazione in nomi parlanti. Costa qualche copia in piu' per chiamata, ma rende leggibili le righe 38-40; e' un compromesso deliberato fra velocita' e chiarezza.

## Coefficienti tempo-varianti: qui si interpola il grezzo (righe 19-21)

```
a1 = M.fa1(t); a3 = M.fa3(t); a4 = M.fa4(t);
A6 = M.fA6(t); K1 = M.fK1(t); V = M.fV(t);
w = M.fomega(t); aqk = M.faqk(t);
sig = M.fsig(t); phi = M.fphi(t);
aw = M.windfun(t);
```

- Righe 19-20: **dieci** lookup per valutazione (contro le sette di `ode_lpv_ascent`). Sono tutti `griddedInterpolant(tg, y, 'linear', 'nearest')` – la stessa closure `gi`, riga 101 di `init_simulink_lpv.m` – costruiti alle righe 110-111 (`fV` fa eccezione: nasce alla riga 104, fuori dal blocco `flex`, insieme a `fQ`) sulla griglia del dataset `GreensiteLPV_DATA.mat`. Interpolazione **lineare** dentro la griglia, **clamp all'estremo** fuori ('nearest' come metodo di estrapolazione): nessun errore, nessun NaN, nessun warning se si esce.
- **Differenza sostanziale rispetto a `ode_lpv_ascent`**: li' venivano interpolati i coefficienti **gia' combinati** (`fc2` = tabella di  $a1 \cdot V + a4$ , `fc5` = tabella di  $A6/V_{safe}$ ), qui si interpolano i **coefficienti grezzi** e le combinazioni si formano **dentro** la RHS (righe 38-39). Poiche' l'interpolazione lineare non commuta con la moltiplicazione, fra due breakpoint i due file integrano sistemi LTV **leggermente diversi** (differiscono di un  $O(dt^2)$  proporzionale alle variazioni dei fattori; coincidono esattamente sui nodi). Nessuno dei due e' sbagliato, ma non ci si deve aspettare che le uscite coincidano bit a bit.
- Riga 19:  $V = M.fV(t)$  e' l'interpolante della **V grezza**, senza la guardia  $V_{safe} = \max(V, 1)$  che `init_simulink_lpv.m` applica al ramo rigido (riga 59). Alla riga 39 si divide per  $V$ . Al lift-off  $V(0) = 0$ , quindi **questa RHS non e' integrabile da  $t = 0$** : la divisione esploderebbe. Funziona solo perche' `main_flex.m` parte da  $t_0 = S.t_0$  (riga 19), e quel valore vale 5 s (default dell'`arguments` block di `init_simulink_lpv.m`, riga 34). E' una fragilita' reale, non protetta da nessun controllo; nel ramo rigido la guardia c'e', qui no.
- Riga 21: il vento,  $\alpha_w(t)$  [rad], stesso `griddedInterpolant` del ramo rigido (`init_simulink_lpv.m`, riga 107), gia' diviso per  $V$  a monte (riga 90). Il generatore del professore gira **una volta sola**, quindi le due corse (notch fisso e variabile) vedono **la stessa realizzazione di vento**: il confronto e' pulito.

## Misure INS contaminate dal bending (righe 24-25)

```
theta_m = theta + sig*eta;
thetadot_m = thetadot + sig*etadot;
z_m = z - phi*eta;
zdot_m = zdot - phi*etadot;
```

- Righe 24-25: e' letteralmente l'Eq. (2) della traccia, la stessa matrice  $C_m$  di `HM3/build_plant_full.m` (righe 33-36):  $\sigma_{ins}$  [rad/m] pesa la **pendenza** della forma modale nella posizione della IMU (contamina l'assetto misurato),  $\phi_{ins}$  [-] pesa lo **spostamento** modale (contamina la posizione misurata). I segni sono opposti (+sig, -phi) e replicano esattamente il modello congelato.
- **Qui e' dove nasce tutto il problema del bending**. Il modo elastico non entra nella dinamica rigida (righe 38-39: nessun eta); entra **solo** attraverso la misura. Il PD retroaziona `theta_m`, che contiene `sig*eta`; il comando `delta` che ne risulta eccita il modo elastico (riga 40, `aqk*delta`), che a sua volta contamina di piu' la misura: e' un anello chiuso attorno a un modo con smorzamento  $z_{BM} = 0.005$ , cioe' praticamente nullo. Senza notch il picco di risonanza in  $L(j\omega_{BM})$  sta sopra 0 dB e l'anello e' instabile. Da qui il notch.

- Attenzione:  $\text{sig}$  e  $\text{phi}$  sono **anch'essi tempo-varianti** (interpolati alla riga 20 dal dataset, campi  $\text{sigma\_ins}$  e  $\text{phi\_ins}$ ). Non e' solo la frequenza a muoversi lungo l'ascisa: anche il *guadagno* di contaminazione. Il codice ne tiene conto senza dirlo esplicitamente.

## Il notch tempo-variante (righe 30-33)

```
wn = M.fwn(t);
v = 2*(M.zN - M.zD)*wn*xn(2) + u_pd;
xn1d = xn(2);
xn2d = -wn^2*xn(1) - 2*M.zD*wn*xn(2) + u_pd;
```

**Derivazione.** Il notch di HM3 (Eq. 4, variante a fase minima usata come design ritenuto – `main_task2.m`, righe 73-78, con  $zN = 0.002$ ,  $zD = 0.7$ ,  $\text{sgn} = +1$ ) e':

$$N(s) = (s^2 + 2*zN*wn*s + wn^2) / (s^2 + 2*zD*wn*s + wn^2)$$

Realizzazione in **forma canonica di controllabilita'**, con denominatore  $s^2 + \alpha_1*s + \alpha_0$  ( $\alpha_1 = 2zDwn$ ,  $\alpha_0 = wn^2$ ) e numeratore  $b_2*s^2 + b_1*s + b_0$  ( $b_2 = 1$ ,  $b_1 = 2zNwn$ ,  $b_0 = wn^2$ ):

```
xn1' = xn2
xn2' = -alpha_0*xn1 - alpha_1*xn2 + u
y = (b_0 - alpha_0*b_2)*xn1
    + (b_1 - alpha_1*b_2)*xn2 + b_2*u
```

Sostituendo:  $b_0 - \alpha_0 = wn^2 - wn^2 = 0$  (il coefficiente di  $xn1$  sparisce, ed e' il motivo per cui alla riga 31  $xn(1)$  non compare) e  $b_1 - \alpha_1 = 2*(zN - zD)*wn$ . Il feedthrough e'  $b_2 = 1$ , quindi il guadagno in continua e ad alta frequenza vale 1: il notch **non tocca** la banda utile, agisce solo attorno a  $wn$ . Il codice alle righe 31-33 e' esattamente questo, senza semplificazioni nascoste.

- Con  $zN = 0.002$  e  $zD = 0.7$ , il coefficiente d'uscita e' circa  $-1.4*wn$ : buca profondissima e strettissima (attenuazione  $\sim zN/zD$ , circa -51 dB al centro). E' un notch **molto** piu' aggressivo delle linee guida della traccia (che nella docstring di `build_notch_filter.m`, righe 6-7, suggerisce  $zN$  in 0.1-0.3,  $zD$  in 0.4-0.6): scelta deliberata di HM3 ("deep, narrow null"), ereditata qui identica. Un notch cosi' stretto e' precisamente quello che **si detuna di piu'**: il fatto che l'esperimento T008 mostri la divergenza e' in parte una conseguenza di questa scelta di progetto, non solo della fisica.
- Riga 30:  $wn = M.fwn(t)$ . **E' l'unico bottone dell'esperimento.** `main_flex.m` passa `S.fomega` (il `griddedInterpolant` di `omega(t)`) per il notch variabile e `@(t) w72` (una anonymous function costante) per quello fisso: la RHS non sa e non deve sapere quale delle due riceve – duck typing, entrambe si chiamano con `f(t)`. Elegante, e rende il confronto rigoroso perche' **nient'altro cambia**.
- Solo la variante a **fase minima** e' realizzabile con queste righe: il numeratore ha  $+2*zN*wn*s$ . La variante di `build_notch_filter` con `numSign = -1` (Eq. 4 come stampata sulla traccia, zeri in semipiano destro) **non e' rappresentabile** senza cambiare il segno alla riga 31. Limite del file, da dichiarare.

## Il caveat teorico che vale un punto all'orale

Sostituire  $wn \rightarrow wn(t)$  **dentro una realizzazione** non e' un'operazione innocua. Una funzione di trasferimento ha infinite realizzazioni equivalenti; sono equivalenti **solo per parametri costanti**. Se i coefficienti variano nel tempo, realizzazioni diverse della *stessa*  $N(s)$  congelata producono sistemi LTV **diversi** (i cambi di base diventano  $T(t)$ , e la trasformazione di similarita' genera termini in  $Tdot$ ). Inoltre la derivazione "corretta" dell'equazione di stato di un filtro con parametro variabile fa comparire termini in  $wn\_dot$ , che qui **non ci sono**: le righe 31-33 sono la forma "frozen-form" (congela la struttura, sostituisci il parametro), che e' la prassi ingegneristica ma non un'identita' matematica. Vale finche'  $wn\_dot$  e' piccola rispetto a  $wn^2$  – ipotesi di variazione lenta, di nuovo assunta e non verificata dal codice. Lo stesso identico caveat vale per la realizzazione a blocchi elementari del notch in Simulink (`build_hm3_full_ascent_flex.m`, righe 125-141), che ricalca queste righe.

---

## TVC e ritardo: unico blocco LTI (righe 34-35)

```
delta = M.Ct*xt + M.Dt*v;  
xtd   = M.At*xt + M.Bt*v;
```

- Righe 34-35: la catena di attuazione e' una **quadrupla costante** (At, Bt, Ct, Dt) (uscita alla 34, derivata di stato alla 35), ottenuta in `init_simulink_lpv.m` (righe 112-115) da `ssdata(ss(build_tvc(p0, 3)))`, con `p0 = load_hw3_params()`, cioe' **congelata a t = 72 s**. E' legittimo:  $w_{TVC} = 70 \text{ rad/s}$ ,  $z_{TVC} = 0.7$ ,  $\tau = 0.020 \text{ s}$  sono costanti di tabella, non funzioni del tempo (`load_hw3_params.m`, righe 55-57). Quindi il TVC e' l'**unico** pezzo dell'anello che non e' LPV – e non lo e' per motivi fisici, non per approssimazione.
- Il ritardo di trasporto di 20 ms e' un **Pade di ordine 3** (`build_tvc.m`, righe 18-19): un'approssimazione razionale, che introduce zeri in semipiano destro. E' fedele a fase bassa e sbaglia sempre di piu' ad alta frequenza. Poiche'  $\omega_{BM}$  arriva fino a  $\sim 32 \text{ rad/s}$  lungo l'ascesa (contro i  $18.9 \text{ rad/s}$  a cui e' stato scelto l'ordine del Pade), **la qualita' del ritardo simulato degrada proprio dove serve**: l'analisi di detuning e' quantitativamente ottimistica o pessimistica a seconda del segno dell'errore di fase, e il codice non lo quantifica.
- **Ordine dei blocchi**: `u_pd` -> `notch` -> `TVC` -> `delta`. Il notch filtra il comando *prima* dell'attuatore, e questo replica l'ordine di `assemble_loop` in HM3 ( $W_{act} = W_{tvc} * \text{notch}$ , vedi `main_flex.m` righe 27-28). Coerente.
- Non ci sono **saturazioni** ne' limiti di rate sul TVC: `delta` puo' assumere qualunque valore. Nel caso "notch fisso" la simulazione diverge, e i valori di `delta` a fine corsa sono privi di senso fisico (un attuatore vero saturerebbe). La divergenza rimane il risultato corretto sul piano della stabilita' lineare, ma le ampiezze non vanno lette come numeri fisici.

---

## Il plant flessibile (righe 38-40)

```
zdd   = a1*zddot + (a1*V + a4)*theta + a3*delta - a1*V*aw;  
thdd  = (A6/V)*zddot + A6*theta + K1*delta - A6*aw;  
etadd = -w^2*eta - 2*M.zBM*w*etadot + aqk*delta;
```

- Righe 38-39: **identiche** alle prime quattro righe della A di `build_plant_full.m` (righe 20-28), con i coefficienti sostituiti dai loro interpolanti. Confermano la lettura del segno del vento: raccogliendo, `thdd = A6*(theta + zddot/V - alpha_w) + K1*delta`, quindi l'incidenza aerodinamica **effettiva** e'  $\theta + zddot/V - \alpha_w$  (conta la velocita' relativa all'aria: la componente laterale relativa e'  $zddot - v_w$ ).
- **Il bending non retroagisce sulla dinamica rigida**: nelle righe 38-39 non compaiono `eta` ne' `etadot`. E' la stessa struttura triangolare a blocchi di `build_plant_full.m` (le prime 4 righe della A hanno zeri nelle colonne 5-6). Fisicamente e' un'approssimazione del modello di corso: la deformazione elastica non perturba il moto del corpo rigido, ma sporca le misure ed e' eccitata dal TVC.
- Riga 40: oscillatore del primo modo.  $w = \omega(t)$  **varia** (e' questo che detuna il notch),  $M.z_{BM} = 0.005$  e' **costante** – viene da  $S.\text{notch}.z_{BM} = p0.z_{BM}$  (`init_simulink_lpv.m`, riga 116), cioe' dal letterale di tabella (`load_hw3_params.m`, riga 49), non dal dataset. Il dataset fornisce  $\omega$  ma il codice non cerca uno  $\zeta$  tempo-variante.
- $aqk = -\phi_{tvc} * T_c$  [interpolato dal dataset, riga 20] e' il forzamento del bending da parte del TVC: la spinta deviata agisce sulla forma modale nel punto di applicazione. Segno negativo, quindi  $\delta > 0$  eccita  $\eta < 0$ .
- **Il vento non eccita il bending**: nella riga 40 non c'e' `aw`. Coerente con `build_plant_full.m` (riga 28:  $B_w$  ha 0 nella riga di `etadot`), ma fisicamente e' una semplificazione forte – una raffica reale carica la struttura direttamente. Il modo elastico qui puo' essere eccitato **solo** attraverso il comando di TVC.



- Nota sull'omega variabile: come per il notch, `etadd` con  $\mathbf{w} = \mathbf{w}(t)$  e' la forma "frozen-form" dell'oscillatore. L'equazione modale rigorosa di una struttura a massa/rigidezza variabili contiene termini aggiuntivi (derivate della massa generalizzata e della forma modale). Il codice li ignora – scelta standard, ma va detta.

## Assemblaggio (riga 42)

```
dx = [zdot; zdd; thetadot; thdd; etadot; etadd;
      xn1d; xn2d; xtd];
```

- Riga 42: concatenazione a colonna nell'**esatto ordine** dello split di riga 15. Se qualcuno cambiasse l'ordine di `x0` in `main_flex.m` (riga 38) senza cambiare qui, il sistema integrato sarebbe silenziosamente un altro: il contratto e' posizionale, non nominale. E' il costo dell'avere una RHS veloce senza validazione.
- `xtd` e' un vettore  $n_t \times 1$ : la concatenazione funziona solo perche' `At*xt` restituisce una colonna. Nessun `(:)` difensivo – corretto e voluto in un hot loop.

## Rigido vs flessibile: cosa cambia davvero

|              | <code>ode_lpv_ascent.m</code>         | <code>ode_lpv_flex.m</code>                                  |
|--------------|---------------------------------------|--------------------------------------------------------------|
| stati        | 4                                     | 13 (6 + 2 notch + 5 TVC)                                     |
| coefficienti | combinati ( <code>fc1..fc7</code> )   | grezzi, combinati in linea                                   |
| guardia su V | <code>Vsafe = max(V,1)</code> a monte | nessuna: divide per <code>fV(t)</code>                       |
| attuatore    | ideale ( <code>delta = u_pd</code> )  | TVC 2 ordine + Pade(3), LTI                                  |
| filtro       | nessuno                               | notch a $\mathbf{w}(t)$ (o fisso)                            |
| misura       | stato vero                            | contaminata dall'INS (Eq. 2)                                 |
| bending      | assente                               | modo 1, $\boldsymbol{\omega}(t)$ , <code>zBM</code> costante |
| guadagni PD  | congelati o schedulati                | congelati (schedule supportata, mai usata)                   |

La **variabile aggiuntiva chiave** e' `omega_BM(t)`: la frequenza del primo modo cambia lungo l'ascesa perche' cambiano massa e rigidezza generalizzate del veicolo mentre brucia propellente. Nel modello rigido questo non ha alcun effetto (il modo non c'e'); nel modello flessibile e' la cosa che rompe il progetto congelato di HM3, perche' il notch e' un filtro **puntato** su una frequenza e quella frequenza si sposta.

## La fallacia del frozen-time, vista da qui

Il ragionamento "ho verificato i margini su ogni plant congelato, dunque il sistema tempo-variante e' stabile" **non e' valido**. Per `xdot = A(t)*x` la stabilita' dipende dalla matrice di transizione  $\Phi(t, t_0)$ , non dagli autovalori istantanei: esiste il classico controesempio (Khalil, *Nonlinear Systems*) di una  $A(t)$  i cui autovalori congelati stanno **sempre** in  $(-1 \pm i\sqrt{7})/4$ , parte reale -0.25 per ogni  $t$ , e la cui soluzione diverge come  $\exp(+0.5*t)$ . L'implicazione diventa vera solo sotto **ipotesi di variazione lenta** ( $\| \dot{A} \|$  sufficientemente piccola), che qui e' assunta e mai quantificata.

Va sottolineato che **anche la direzione opposta e' invalida**: dal fatto che gli LTI congelati siano instabili non segue che l'LTV lo sia. Questo tocca direttamente lo studio T008, perche' `main_flex.m` (righe 24-33) conclude "il notch fisso va instabile da  $t \sim 75$  s" leggendo `isstable(Tf)` **sui plant congelati**. Preso da solo sarebbe un argomento formalmente insufficiente.

Come il codice si difende: non si ferma li'. Alle righe 40-41 di `main_flex.m` **propaga davvero l'LTV** con `ode45` (`RelTol = 1e-8`, `AbsTol = 1e-10`) sulla stessa realizzazione di vento, e osserva che il coordinato di bending `eta`

diverge davvero nel caso fisso e resta dell'ordine di  $1e-3$  nel caso variabile. I due argomenti – sweep congelato + integrazione LTV – si sostengono a vicenda: nessuno dei due, da solo, sarebbe una prova. E nemmeno insieme lo sono in senso stretto: la simulazione e' **una** traiettoria, con **un** vento, da **una** condizione iniziale ( $x0 = \text{zeros}(6 + 2 + nt, 1)$ , cioè  $\text{zeros}(13,1)$ , riga 38). Una prova vera richiederebbe di propagare  $\Phi(t,t_0)$  a ingresso nullo, oppure una funzione di Lyapunov tempo-variante / una LMI parameter-dependent. Il repo non le fa e non le rivendica.

**Possibile domanda d'esame** – il notch variabile e' un controllore LPV legittimo o un trucco?

*Risposta:* E' gain scheduling nel senso classico: si progetta una famiglia di filtri sui plant congelati e si interpola il parametro. E' legittimo sotto variazione lenta, ma non porta con se' alcuna garanzia di stabilita' LTV: serve la verifica a posteriori (l'integrazione LTV), oppure una sintesi LPV vera (LMI con funzione di Lyapunov dipendente dal parametro, che vincola anche  $wn\_dot$ ). Il codice fa la prima strada e lo dichiara.

## Limiti e punti fragili (riepilogo onesto)

- **Nessuna guardia su V:** la riga 39 divide per  $fV(t)$ ;  $V(0) = 0$ . La RHS non puo' partire da  $t = 0$ . Il ramo rigido ha  $V_{safe}$ , questo no.
- **Solo notch a fase minima** realizzabile (riga 31 hard-codea il segno +).
- **zBM costante** (0.005, letterale di tabella), mentre  $\omega$  e' tabulato.
- **Il vento non forza il bending e il bending non forza il corpo rigido:** entrambi ereditati da `build_plant_full.m`, entrambi semplificazioni.
- **Nessuna saturazione su delta:** nel caso divergente le ampiezze non hanno significato fisico.
- **Pade(3) tarato sul punto a 18.9 rad/s** ma usato fino a  $\sim 32$  rad/s.
- **Forma “frozen-form”** per notch e bending: mancano i termini in  $wn\_dot$  e  $\omega\_dot$ ; valida solo sotto variazione lenta.
- **Contratto posizionale sullo stato** (riga 15): fragile a modifiche di  $x0$ .
- **M.sched** esiste ma nessuno script lo attiva su questa RHS: e' codice supportato ma non esercitato (quindi non testato).

## Possibili domande d'esame

**D: Che cosa aggiunge questa RHS rispetto a `ode_lpv_ascent`, e perche' e' proprio quello a rompere il progetto congelato?** R: Aggiunge il primo modo di bending (riga 40), la contaminazione INS delle misure (righe 24-25) e la catena TVC + ritardo con il notch (righe 30-35). Il punto critico e' che  $\omega_{BM}$  **varia** lungo l'ascesa, perche' massa e rigidezza generalizzate cambiano mentre brucia il primo stadio. Il notch profondo di HM3 e' centrato su  $\omega_{BM}(72) = 18.9$  rad/s: quando la frequenza si sposta, la risonanza esce dalla buca, il guadagno d'anello alla frequenza di bending risale sopra 0 dB e l'anello – chiuso su una misura contaminata da un modo con smorzamento 0.005 – diverge.

**D: Perche' l'instabilita' passa dalle misure e non dalla dinamica?** R: Perche' nel modello del corso il bending **non retroagisce** sul corpo rigido: nelle righe 38-39 non compaiono  $\eta$  ne'  $\eta_{dot}$ . Il modo entra solo in  $\theta_{m} = \theta + \sigma \eta$  e  $z_m = z - \phi \eta$  (Eq. 2). Il PD retroaziona quella misura sporca, il comando  $\delta$  che ne esce eccita il modo ( $aqk \cdot \delta$ , riga 40), e il ciclo si chiude. Senza contaminazione INS il bending sarebbe uno stato non osservato e innocuo.

**D: La stabilita' di ogni plant congelato garantisce la stabilita' del sistema tempo-variante?** R: No. Per un LTV la stabilita' dipende dalla matrice di transizione, non dagli autovalori istantanei; esistono controesempi standard con autovalori congelati costanti a parte reale negativa e soluzioni divergenti (Khalil). Serve l'ipotesi di variazione lenta. E vale anche il contrario: da “tutti gli LTI congelati sono instabili” non segue che l'LTV lo sia – il che indebolisce, se preso da solo, lo sweep `isstable` di `main_flex.m` (righe 24-33). Per questo il codice propaga davvero l'LTV con `ode45` (righe 40-41) e usa i due argomenti insieme.

**D: Come e' realizzato il notch e perche'  $x_n(1)$  non compare nell'uscita?** R: E' la forma canonica di controllabilita' di  $N(s) = (s^2 + 2 z_N \omega_n s + \omega_n^2)/(s^2 + 2 z_D \omega_n s + \omega_n^2)$ . Il coefficiente d'uscita su  $x_{n1}$  vale  $b_0 - \alpha_0 d = \omega_n^2 - \omega_n^2 = 0$ , quindi sparisce; su  $x_{n2}$  vale  $b_1 - \alpha_1 d = 2*(z_N - z_D)*\omega_n$  (riga 31) e il feedthrough vale 1 (guadagno unitario fuori dalla buca). Con  $z_N = 0.002$  e  $z_D = 0.7$  il notch e' profondissimo e strettissimo – che e' anche il motivo per cui si detuna cosi' male.

**D: Sostituire  $\omega_n(t)$  in una realizzazione congelata e' matematicamente corretto?** R: Non esattamente. Due realizzazioni della stessa funzione di trasferimento sono equivalenti solo a parametri costanti; con parametri variabili realizzazioni diverse danno sistemi LTV diversi, e la derivazione rigorosa fa comparire termini in  $\omega_n$  che qui non ci sono. E' la prassi standard (“frozen-form”), valida sotto variazione lenta, ma e' un'approssimazione, non un'identita'. Lo stesso vale per l'oscillatore di bending con  $\omega(t)$ .

**D: Perche' il TVC e' l'unico blocco LTI in un modello LPV?** R: Perche' i suoi parametri sono davvero costanti:  $\omega_{TVC} = 70$  rad/s,  $z_{TVC} = 0.7$ ,  $\tau = 20$  ms sono proprieta' dell'attuatore, non del volo. La quadrupla ( $A_t$ ,  $B_t$ ,  $C_t$ ,  $D_t$ ) viene estratta una volta da `build_tvc(p0, 3)` e riusata. L'unica approssimazione e' il Pade di ordine 3 per il ritardo, tarato implicitamente attorno alla frequenza di bending del punto di progetto (18.9 rad/s) ma usato fino a ~32 rad/s.

# HM3/LTV\_FULL\_ASCENT/build\_hm3\_fu

## Ruolo del file nel progetto

Questo file **e' il modello Simulink**. Non e' uno script che carica un `.slx` gia' esistente e nemmeno un generatore di griglie di plant o di array di `ss`: e' un *authoring script* che, usando le API `new_system` / `add_block` / `add_line` / `set_param` / `save_system`, costruisce da zero il file `HM3/LTV_FULL_ASCENT/hm3_full_ascent.slx` blocco per blocco e filo per filo. Il `.slx` e' un artefatto **derivato**; la sorgente versionabile e' questo `.m`.

Il modello che costruisce e' la controparte Simulink della baseline MATLAB `ode_lpv_ascent.m` + `main_full_ascent.m`: il plant rigido di beccheggio di HM3 (`build_plant_rigid`), che nella traccia e' congelato a max-q ( $t = 72$  s), qui diventa **tempo-variante (LPV)** sull'intera ascesa 0-140 s, e il generatore di vento del professore (`General/hw3-v3/strong_wind.slx`) viene messo **dentro l'anello** invece che pre-campionato su una finestra di 12 s attorno a max-q. Il punto della dimostrazione e' confrontare, sulla stessa tela, i guadagni **frozen** (l'unico design a max-q tenuto per tutto il volo) contro i guadagni **gain-scheduled** in tempo.

Dipendenze: nel modello non c'e' un solo numero di **dati o di design**. Tabelle, breakpoint, guadagni e perfino `StopTime` sono **nomi di variabili** del base workspace (`lpv_t`, `lpv_c1...lpv_c7`, `lpv_invV`, `Kp_th0`, `tsched`, `Kp_sched`, `sched`, `Tstop`, ...) che `init_simulink_lpv.m` deve avere gia' pushato. Gli unici numeri letterali cablati nel `.slx` sono costanti *strutturali*, non parametri del problema: `StartTime = 0` e `MaxStep = 0.02` del solver (riga 36), `Threshold = 0.5` degli Switch (righe 111-112), `InitialCondition = 0` degli integratori (righe 73-74, 81-82), `Inputs = 2` dei Product e `NumberOfTableDimensions = 1` delle lookup. Chi lo chiama: `run_full_ascent_simulink.m` (con `o.rebuild=true`, oppure automaticamente se il `.slx` non esiste). Chi consuma il suo output: sempre `run_full_ascent_simulink.m`, che simula il modello e sovrappone il risultato alla replica `ode45`.

Le equazioni realizzate sono quelle scritte nel README della cartella e nella RHS `ode_lpv_ascent.m`:

```
zddot      = a1*zdot + (a1*V + a4)*theta + a3*delta - a1*V*alpha_w
thetaddot = (A6/V)*zdot + A6*theta      + K1*delta - A6*alpha_w
```

con tutti i coefficienti funzione del tempo di volo.

---

## Firma, header e arguments (righe 1-18)

```
function build_hm3_full_ascent(o)
arguments
    o.open (1,1) logical = false
end
```

- Righe 1 e 16-18: la firma prende il solo struct name-value `o`; il blocco `arguments` dichiara l'unica opzione, `o.open (1,1) logical = false` (riga 17), che decide se lasciare il modello aperto a video dopo la build. Nessun argomento di dati: **tutti i dati stanno nel base workspace**, non passano da qui.
- Righe 2-14: l'header dichiara il contratto: "Simulink mirror of ODE\_LPV\_ASCENT / MAIN\_FULL\_ASCENT (source of truth). Run INIT\_SIMULINK\_LPV first so the referenced base variables exist." La gerarchia

e' esplicita: la verita' e' la RHS MATLAB, il .slx la specchia. Questo va detto all'orale cosi' com'e': il modello Simulink **non** e' usato per validare la teoria, e' usato per mostrare che la stessa matematica si puo' realizzare a blocchi.

---

## Percorsi, generatore del professore, modello nuovo e solver (righe 20-37)

```
load_system(fullfile(gdir, 'strong_wind.slx'));
cleanupSW = onCleanup(@() close_system('strong_wind', 0));
```

- Righe 20-24: mdl = 'hm3\_full\_ascent'; gdir punta a HM3/General/hw3-v3, la cartella del materiale del professore. addpath(here) e addpath(hm3) sono effetti collaterali sul path MATLAB **mai ripristinati**: igiene discutibile, ma innocua in una sessione interattiva.
- Righe 27-28: carica strong\_wind.slx (il generatore di vento del professore) solo per poterne **copiare** il sottosistema. L'onCleanup lo chiude con il flag 0 = *discard changes*: qualunque cosa succeda, anche in caso di errore, il file del professore non viene mai salvato. E' la traduzione in codice della regola "non si tocca il materiale del corso".
- Riga 31: close\_system(mdl, 0) se il modello era gia' caricato, poi riga 32 new\_system(mdl). La build e' **distruttiva e idempotente**: si riparte sempre da zero, non si fa merge con un modello esistente. Questa e' la ragione per cui non ha senso modificare a mano il .slx: la modifica sparisce alla prossima esecuzione.
- Righe 35-37: solver ode45 a passo variabile, StartTime = 0, StopTime = 'Tstop' (una **stringa**: e' il nome di una variabile base, non un numero), MaxStep = 0.02.
- Righe 33-34, il commento: "MaxStep bounded to resolve the generator's 0.1 s noise innovations". Il generatore di turbolenza Dryden aggiorna il rumore ogni 0.1 s; con MaxStep = 0.02 si hanno 5 passi per innovazione, altrimenti il solver potrebbe scavalcare le discontinuita' del rumore e la replica ode45 non riuscirebbe piu' a ricostruire lo stesso ingresso.

**Possibile domanda d'esame** – il modello raggiunge  $\sim 1e-7$  rad di accordo con ode45: e' merito delle tolleranze? *Risposta*: No, ed e' un punto onesto da ammettere. Il builder **non imposta mai RelTol / AbsTol** sul modello, quindi Simulink gira con il suo default (RelTol =  $1e-3$ ), mentre la replica in run\_full\_ascent\_simulink.m (riga 29) usa odeset('RelTol',  $1e-9$ , 'AbsTol',  $1e-11$ ). Le due integrazioni hanno tolleranze diversissime; l'accordo si regge su MaxStep = 0.02, che inchioda il passo di Simulink a un valore cosi' piccolo da rendere l'errore locale molto piu' fine di quanto la tolleranza richiederebbe. Se si rilassa MaxStep, l'accordo degrada.

---

## Le scorciatoie di authoring: A, W, LK (righe 39-42)

```
A = @(name, src, pos, varargin) add_block(src, ...
    [mdl '/' name], 'Position', pos, varargin{:});
W = @(s, d) add_line(mdl, s, d, 'autorouting', 'on');
LK = 'simulink/Lookup Tables/n-D Lookup Table';
```

- Riga 40: A = "aggiungi blocco". src e' il path di libreria (built-in/Product, built-in/Integrator, ...), pos il rettangolo [x1 y1 x2 y2] sul canvas, varargin le coppie parametro/valore.
- Riga 41: W = "collega". La sintassi 'blocco/porta' (es. 'c1/1') e' la forma testuale di add\_line; autorouting fa instradare i fili a Simulink.
- Riga 42: la libreria del blocco lookup. Si usa la **n-D Lookup Table** forzandone la dimensione a 1 (NumberOfTableDimensions = '1') invece del vecchio blocco Lookup Table 1-D, che e' deprecato.

Nota onesta: buona parte del file e' **layout** (le coordinate [430 60 460 90] e simili sono numeri magici scelti a occhio). Non hanno alcun effetto sulla matematica, ma occupano molte righe e rendono il diff rumoroso.

---

## Clock e le lookup dei coefficienti effettivi (righe 44-54)

```
cs = {'c1','c2','c3','c4','c5','c6','c7','invV'};
tb = {'lpv_c1','lpv_c2', ... , 'lpv_c7','lpv_invV'};
for k = 1:numel(cs)
    A(cs{k}, LK, [...], ...
      'NumberOfTableDimensions','1', ...
      'BreakpointsForDimension1','lpv_t', 'Table',tb{k});
    W('Clock/1', [cs{k} '/1']);
end
```

- Riga 45: un unico blocco **Clock**. E' il parametro di scheduling: il modello e' un LPV schedulato sul tempo di volo, e il Clock e' l'unica sorgente di quel parametro.
- Righe 48-54: otto lookup 1-D, tutte con gli **stessi breakpoint** `lpv_t`, ognuna con la propria tabella. Il Clock alimenta tutte e otto. Interpolazione: il parametro `InterpMethod` non viene toccato, quindi resta il default **Linear**.

## Il meccanismo LPV: perche' non un blocco State-Space

Un blocco **State-Space** di Simulink valuta **A**, **B**, **C**, **D** **una volta**, in fase di compilazione: sono costanti. Non esiste modo di farle variare nel tempo. Per un plant LPV le strade sono tre:

1. **Blocchi elementari** (la scelta di questo file): una lookup per coefficiente, **Product** per moltiplicarla per il segnale, **Sum** per sommare i termini, **Integrator** per integrare. Ogni coefficiente resta un segnale visibile e sondabile a video.
2. **MATLAB Function block**: si riscriverebbe `ode_lpv_ascent` dentro un blocco. Funziona, ma il modello diventa una scatola nera: dal canvas non si vede piu' nessun coefficiente, e non c'e' piu' niente da "mostrare".
3. **LPV System block** (Control System Toolbox): richiede un array di **ss** costruito su una griglia di scheduling. Va detto onestamente che, a parita' di griglia e di interpolazione lineare, sarebbe **matematicamente equivalente** alla soluzione 1 (interpolare linearmente ogni entrata della matrice **A** = interpolare linearmente ogni coefficiente). Le ragioni per cui e' scartato sono pratiche: dipendenza da un toolbox, opacita', e un array di 141 modelli **ss** 4x4 solo per rappresentare 7 scalari.

Il codice non motiva mai esplicitamente la scelta; il README della cartella dice solo "so every coefficient stays inspectable, mirroring the style of the professor's own generator". Il resto della motivazione va ricostruito, ed e' corretto dirlo all'orale come ricostruzione.

## Il trucco dei coefficienti effettivi (il cuore della pagina)

`init_simulink_lpv.m` (righe 61-68) **non** tabula i fattori grezzi **a1**, **V**, **a4**, **A6**, **K1**. Tabula i **coefficienti gia' combinati**, uno per termine dell'equazione:

```
c1 = a1          (moltiplica zdot)
c2 = a1*V + a4    (moltiplica theta)
c3 = a3          (moltiplica delta)
c4 = a1*V         (moltiplica alpha_w)
c5 = A6/V         (moltiplica zdot)
c6 = A6          (moltiplica theta e -alpha_w)
c7 = K1          (moltiplica delta)
invV = 1/V       (alpha_w = v_w * invV)
```

Perche' e' essenziale e non solo elegante: **l'interpolazione lineare non commuta con la moltiplicazione**. Sia  $L\{\cdot\}$  l'interpolatore lineare su una cella di ampiezza  $h$ , con  $u = s/h$  in  $[0,1]$ ,  $Df = f1 - f0$ ,  $Dg = g1 - g0$ . Allora

$$L\{f*g\}(u) - L\{f\}(u)*L\{g\}(u) = Df*Dg * u*(1-u)$$

che vale zero **solo** sui breakpoint e raggiunge il massimo  $Df \cdot Dg/4$  a metà cella. E' un errore di ordine  $h^2$ , ma e' un errore di **modello**, non di solver: non si riduce stringendo il passo di integrazione.

Conseguenza operativa: il `.slx` e la RHS `ode_lpv_ascent.m` integrano davvero la **stessa** funzione **soltanto se entrambi interpolano le stesse tabelle combinate**. Ed e' esattamente quello che accade nel track rigido:

- il `.slx` legge `lpv_c1...lpv_c7` (righe 48-49 di questo file);
- `init_simulink_lpv.m` (righe 102-103) costruisce `S.fc1 = gi(c1) ... S.fc7 = gi(c7)` con `griddedInterpolant(tg, y, 'linear', 'nearest')` sulle **stesse identiche** array;
- `ode_lpv_ascent.m` (righe 26-28) usa `M.fc1(t) ... M.fc7(t)` e non ricombina mai nulla.

Le due catene interpolano gli stessi numeri con la stessa regola. Questo, e non la fortuna, e' cio' che rende possibile l'accordo a  $\sim 1e-7$  rad. (Nel modello **flessibile** questa disciplina viene rotta: vedi la pagina `lpv_build_full_ascent_flex.md`.)

## Numeri reali della griglia

Verificato leggendo `GreensiteLPV_DATA.mat`: il tempo del dataset e' uniforme,  $t = 0:1:150$  s (151 punti), quindi `lpv_t = 0:1:140 -> 141 breakpoint`, passo  $h = 1$  s. E' una griglia grossolana: l'errore di non-commutazione, se ci si cadesse dentro, sarebbe tutt'altro che trascurabile.

**Possibile domanda d'esame** – se avessi tabulato `a1`, `V`, `a4` separatamente e ricostruito  $c2 = a1 \cdot V + a4$  a valle con un `Product` e un `Sum`, il modello sarebbe stato sbagliato? *Risposta:* No, sarebbe stato un modello LPV perfettamente legittimo – ma un modello **diverso**, che integra una funzione diversa fra i breakpoint. Il confronto con la baseline `ode45` avrebbe avuto un pavimento di errore fissato dal termine  $Df \cdot Dg \cdot u \cdot (1-u)$ , indipendente dalle tolleranze. Su questa griglia a 1 s,  $L\{a1 \cdot V + a4\} - (L\{a1\} \cdot L\{V\} + L\{a4\})$  vale fino a  $2.2e-3$  in valore assoluto (circa 0.003% del coefficiente): abbastanza da mascherare completamente il livello  $1e-7$  che si vuole dimostrare.

## Estrapolazione: una asimmetria che vale la pena conoscere

Le otto lookup dei coefficienti (righe 51-52) **non** impostano `ExtrapMethod`, quindi restano al default `Linear`. La `griddedInterpolant` della baseline usa invece `'nearest'` come metodo di estrapolazione (= clip, tiene il valore d'estremo). Le due regole **non coincidono**. In pratica non morde, perche' `lpv_t` copre  $[0, 140]$  e la simulazione gira su  $[0, 140]$ : non si estrapola mai. Ma e' una incoerenza latente. Nota che l'autore la conosce, perche' per le lookup dei guadagni (righe 108-109) mette `ExtrapMethod = 'Clip'` esplicitamente, e nel modello flessibile lo mette su **tutte** le lookup.

---

## Il generatore di vento e `alpha_w` (righe 56-64)

```
add_block('strong_wind/Subsystem', [mdl '/WindGen'], ...);
A('vw_sum','built-in/Sum',[...],'Inputs','++', ...);
A('aw_prod','built-in/Product',[...],'Inputs','2');
```

- Riga 57: il sottosistema del professore viene **copiato** dentro il nuovo modello. E' una copia, non un link a libreria: se il professore cambiasse `strong_wind.slx`, il `.slx` generato non se ne accorgerebbe finche' non si rilancia il builder. Il sorgente resta immutato grazie all'`onCleanup` di riga 28.
- Riga 60: il Clock alimenta l'ingresso del generatore. Vento e veicolo condividono **lo stesso orologio**: e' proprio questa la differenza rispetto a HM3, dove `load_wind_profile` ritagliava 12 s di vento attorno a `max-q`.
- Riga 58 + righe 61-62: `vw_sum` (`Inputs = '++'`) somma le due uscite del generatore (profilo medio `v_wp` e turbolenza Dryden), coerentemente con `init_simulink_lpv/run_wind_generator`, che legge `Outputport(1) = sw_vwp` e `Outputport(2) = sw_turb`.

- Riga 59 + righe 63-64:  $aw\_prod = vw\_sum * invV$ , cioè

$$\alpha_w(t) = (v_{wp}(t) + turbolenza(t)) / V(t)$$

L'angolo d'attacco indotto dal vento è il rapporto fra velocità laterale del vento e velocità del veicolo: è la linearizzazione  $\alpha_w \sim \tan(\alpha_w)$ , valida perché  $v_w \ll V$  ( $V$  va da 410 a 3116 m/s nel dataset).

Nota su `invV`: `init_simulink_lpv` calcola  $V_{safe} = \max(V, 1)$  con il commento “ $V(0)=0$ : guard  $A6/V$  and  $1/V$  at lift-off”. **Il commento è falso per questo dataset**:  $\min(V) = 410.4$  m/s, quindi  $\max(V, 1)$  è bit-per-bit uguale a  $V$  e la guardia non si attiva mai. È codice morto più un commento che contraddice i dati. Innocuo, ma da non raccontare come se fosse una protezione attiva.

## Il plant: prodotti -> somme -> integratori (righe 66-100)

```
A('zdd','built-in/Sum',[...],'Inputs','+++-', ...);
A('int_zd','built-in/Integrator',[...],'InitialCondition','0');
A('int_z','built-in/Integrator',[...],'InitialCondition','0');
```

- Righe 68-71: P1..P4, quattro Product a due ingressi. Ognuno realizza un termine:  $c1 \cdot zdot$ ,  $c2 \cdot \theta$ ,  $c3 \cdot \delta$ ,  $c4 \cdot \alpha_w$ .
- Riga 72: `zdd` è un Sum con `Inputs = '+++-'`. **L'ordine delle porte è semantico**:  $+P1 +P2 +P3 -P4$ , cioè

$$zddot = c1 \cdot zdot + c2 \cdot \theta + c3 \cdot \delta - c4 \cdot \alpha_w$$

Il segno meno su  $\alpha_w$  è fisico: il vento riduce l'angolo d'attacco totale visto dal corpo rispetto a quello dovuto all'assetto.

- Righe 73-74: catena `int_zd -> int_z`. Il primo integratore trasforma `zddot` in `zdot`, il secondo `zdot` in `z`. Condizioni iniziali entrambe a zero, coerenti con `zeros(4,1)` usato dalla replica `ode45` (`run_full_ascent_simulink.m` riga 49).
- Righe 76-82: identico per il canale di beccheggio, con  $\theta_{ddot} = c5 \cdot zdot + c6 \cdot \theta + c7 \cdot \delta - c6 \cdot \alpha_w$ .
- Riga 86: `W('c6/1','P6/1');` `W('c6/1','P8/1');` – **una sola lookup** `c6` alimenta due prodotti. È il motivo per cui `A6` compare due volte nell'equazione di  $\theta_{ddot}$  (moltiplica  $\theta$  e  $-\alpha_w$ ): il termine aerodinamico dipende dall'angolo d'attacco totale  $\theta - \alpha_w$ , e la scrittura a due termini con lo stesso coefficiente lo rende esplicito.
- Righe 95-100: retroazione degli stati sulla **porta 2** dei prodotti. I segnali arrivano dalle uscite degli integratori, quindi **non c'è loop algebrico**: ogni anello di feedback attraversa almeno un integratore.

**Possibile domanda d'esame** – da dove viene  $c5 = A6/V$  e non  $A6$ ? *Risposta*: Il momento aerodinamico è proporzionale all'angolo d'attacco totale  $\alpha = \theta + zdot/V - \alpha_w$  (deriva laterale + assetto - vento). Sostituendo in  $\theta_{ddot} = A6 \cdot \alpha + K1 \cdot \delta$  si ottiene  $\theta_{ddot} = (A6/V) \cdot zdot + A6 \cdot \theta + K1 \cdot \delta - A6 \cdot \alpha_w$ : il  $1/V$  nasce dalla conversione della velocità laterale `zdot` in angolo. Lo stesso  $V$  compare in  $c2 = a1 \cdot V + a4$  per la forza normale.

## Il controllore: frozen contro scheduled (righe 102-133)

```
A('sched','built-in/Constant',[...],'Value','sched');
A('Kp_f','built-in/Constant',[...],'Value','Kp_th0');
A('Kp_s',LK,[...],'BreakpointsForDimension1','tsched', ...
  'Table','Kp_sched','ExtrapMethod','Clip');
```



```
A('Kp_sw','built-in/Switch',[...], ...
  'Criteria','u2 >= Threshold','Threshold','0.5');
```

- Riga 103: `sched` e' una `Constant` che legge la variabile base omonima. `run_full_ascent_simulink.m` (riga 41) fa `assignin('base','sched',sc)` prima di ogni `sim`, quindi lo stesso `.slx` serve entrambi gli esperimenti.
- Righe 104-105: `Kp_f`, `Kd_f` sono i guadagni **congelati**, cioe' il design HM3 Task-1 a max-q (`Kp_th0`, `Kd_th0` prodotti da `design_controller` su `build_plant_rigid(load_hw3_params())`, `t_ref = 72 s`).
- Righe 108-109: `Kp_s`, `Kd_s` sono lookup **1-D** su `tsched`, con tabelle `Kp_sched` / `Kd_sched`. Interpolazione lineare (default). I punti di design sono `tsched = 5:5:140 -> 28 punti`, uno ogni 5 s: `init_simulink_lpv` (righe 75-85) risolve 28 volte `design_controller` su altrettanti plant congelati, in **continuazione** (warm start `Kprev` dal punto precedente), esattamente la ricetta di continuazione usata altrove nella repo.
- Righe 106-107 e `ExtrapMethod='Clip'`: qui la clip **serve davvero**. La griglia dei guadagni parte da `t = 5 s` mentre la simulazione parte da `t = 0`, quindi fra 0 e 5 s si estrapola per forza. `Clip` tiene il valore d'estremo, che e' esattamente cio' che fa `griddedInterpolant(...,'nearest')` nella baseline. Il commento del codice lo dice ed e' corretto.
- Righe 111-112: `Switch` con `Criteria = 'u2 >= Threshold'`, `Threshold = 0.5`. Semantica: se `sched >= 0.5` passa la **porta 1** (schedulato), altrimenti la **porta 3** (congelato). Il ramo non selezionato **non** viene eseguito: il builder non tocca il parametro di configurazione `ConditionallyExecuteInputs`, che di default e' `'on'`, e la *conditional input branch execution* di Simulink calcola solo il ramo effettivamente selezionato quando quel ramo alimenta in esclusiva lo `Switch` – ed e' il caso di `Kp_s/Kp_f` verso `Kp_sw` e di `Kd_s/Kd_f` verso `Kd_sw`. Quindi non si paga nemmeno il costo di calcolo del ramo inattivo.
- Righe 117-120: qui c'e' una asimmetria da dichiarare. `Kp` e `Kd` di beccheggio passano da `Product` (guadagno = segnale variabile). I guadagni di **deriva** `Kp_z0` e `Kd_z0` sono invece blocchi `Gain` con valore costante: **non sono mai schedulati**, nemmeno con `sched = 1`. Il “gain scheduling” di questo showcase riguarda solo la coppia PD di assetto. Coerente con `ode_lpv_ascent.m` (riga 22, usa sempre `M.Kp_z`, `M.Kd_z`) e con la tabella del README, che infatti dice “pitch gains”.
- Riga 121: `delta_sum` con `Inputs = '----'`, cioe'
 
$$\text{delta} = -(\text{Kp} \cdot \text{theta} + \text{Kd} \cdot \text{thetadot} + \text{Kp}_z \cdot z + \text{Kd}_z \cdot \text{zdot})$$
 Legge di controllo PD puro con riferimento `theta_ref = 0`, identica alla riga 22 di `ode_lpv_ascent.m`.
- Righe 132-133: `delta` rientra nel plant sulle porte 2 di P3 (`c3*delta`) e P7 (`c7*delta`).

**Limite strutturale importante:** in questo modello rigido `delta` va dal sommatore direttamente al plant. Non c'e' **nessun attuatore TVC**, **nessun ritardo di trasporto**, **nessuna saturazione**, **nessun rate limit**, **nessun modo flessibile**, **nessun notch**. E' l'ipotesi di “attuatore ideale” dichiarata nell'header di `ode_lpv_ascent.m`. Qualunque affermazione sui margini letta su questo anello va qualificata: e' l'anello rigido idealizzato, non quello di HM3 Task 2/3.

## Logging (righe 135-146)

```
tw = {'-1','MaxDataPoints','inf'};
A('log_theta','built-in/ToWorkspace',[...], ...
  'VariableName','theta_sl','SaveFormat','Timeseries', ...
  'SampleTime',tw{:});
```

- Riga 136: `SampleTime = '-1'` (ereditato: su un segnale continuo diventa “ogni passo maggiore del solver”) e `MaxDataPoints = 'inf'` per togliere il tetto di default a 1000 punti. Senza quest'ultimo, il confronto con `ode45` sarebbe fatto su una manciata di campioni.

- Righe 137-146: cinque To Workspace in formato Timeseries: `theta_sl`, `z_sl`, `zdot_sl`, `delta_sl`, `alpha_w_sl`. E' il contratto con `run_full_ascent_simulink.m`, che li legge come `so.theta_sl` ecc.

Osservazioni oneste sul logging:

- `zdot_sl` viene loggato ma `run_full_ascent_simulink.m` non lo legge mai (riga 44 preleva solo `theta_sl`, `z_sl`, `delta_sl`, `alpha_w_sl`). Log inutilizzato.
- Riga 37: il modello abilita `SignalLogging` con `SignalLoggingName = 'logsout'`, ma **nessun segnale viene marcato per il logging** in tutta la build. `logsout` resta vuoto e non viene mai letto. E' configurazione morta, probabilmente trascinata da `run_wind_generator` (dove `logsout` serve davvero).

## Perche' `alpha_w_sl` e' loggato: il punto piu' delicato dell'intera validazione

`run_full_ascent_simulink.m` (riga 92) costruisce la windfun della replica ode45 come `griddedInterpolant` sull'`alpha_w` che il modello Simulink ha appena prodotto. Il vento **non viene rigenerato** in MATLAB.

Va detto chiaramente: questa non e' una validazione indipendente, e' un **controllo di autoconsistenza** con il ramo del vento cortocircuitato per costruzione. Cio' che viene davvero confrontato e': tabelle dei coefficienti, cablaggio del plant, legge di controllo, integrazione. Il generatore di vento e' comune ai due lati e quindi non e' sotto test. L'header di `run_full_ascent_simulink.m` (righe 3-5) lo ammette: "the residual is just wind interpolation between solver steps".

---

## Salvataggio (righe 148-153)

- Righe 149-150: `save_system` scrive `hm3_full_ascent.slx` **accanto allo script**. Il `.slx` e' quindi un artefatto derivato che vive nella stessa cartella del suo generatore.
- Riga 152: apre o chiude senza salvare a seconda di `o.open`.

## Perche' costruire il modello da script

1. **Riproducibilita'**. Il `.slx` e' binario: non si diffonde, non si revisiona, non si merge. Il `.m` si'. Chi legge la repo vede *tutto* il modello – blocchi, parametri, fili – in 153 righe di testo.
2. **La sorgente resta unica**. Se cambia il design (un guadagno, una tabella, il passo della schedula), si modifica `init_simulink_lpv.m` o questo file e si rilancia: il modello si rigenera. Non esiste il rischio classico del "modello Simulink che ha divergito dallo script".
3. **Contrasto voluto con HM3**. Il README della cartella lo dice esplicitamente (righe 54-57): `HM3/models/hm3_closed_loop.slx` e' costruito a mano seguendo una guida; qui il modello **e'** il codice.

Il prezzo: il modello **non e' modificabile a mano** (ogni edit si perde alla prossima build) e **non e' autosufficiente** (tutti i parametri di dati sono nomi di variabili base; senza `init_simulink_lpv` il `.slx` non compila nemmeno, perche' persino `StopTime` e' la stringa 'Tstop'). Una scelta piu' robusta sarebbe stata mettere i dati nel Model Workspace o in un data dictionary.

---

## Limiti, hack e codice stale (riepilogo onesto)

- **Nessuna tolleranza impostata sul modello**: Simulink gira a `RelTol` di default (1e-3) contro 1e-9 della replica. L'accordo dipende da `MaxStep`.
- **SignalLogging/logsout attivi ma vuoti** (riga 37): configurazione morta.
- `zdot_sl` loggato e mai letto.
- **ExtrapMethod non impostato sulle lookup c1..c7** (default `Linear`), mentre la baseline usa estrapolazione `nearest`. Innocuo solo perche' la griglia copre l'orizzonte; incoerente con il modello flessibile, che mette `Clip` ovunque.
- `Vsafe = max(V,1)` in `init_simulink_lpv` e' **inerte** e il commento "V(0)=0 ... at lift-off" e' smentito dai dati (`min(V) = 410.4 m/s`).

- **Attuatore ideale:** niente TVC, ritardo, saturazione, rate limit, bending.
- **Guadagni di deriva mai schedulati,** nemmeno con `sched = 1`.
- **addpath non ripristinato.**
- **La validazione e' autoconsistente,** non indipendente: il vento e' preso dal modello stesso.
- Le variabili base `lpv_Q`, `lpv_V`, `lpv_h`, `Tstart` sono pushate da `init_simulink_lpv` ma **nessun blocco di questo modello le usa.**
- Il modello copia il sottosistema del vento: se il file del professore cambiasse, serve un rebuild manuale, senza alcun avviso automatico.

## Possibili domande d'esame

**D: Come si realizza un plant LPV in Simulink, visto che il blocco State-Space prende matrici costanti?** R: Si scende al livello dei blocchi elementari. Il Clock genera il parametro di scheduling (qui il tempo di volo), una lookup table 1-D per coefficiente restituisce  $c_i(t)$ , un Product moltiplica il coefficiente per lo stato o l'ingresso corrispondente, un Sum compone la derivata e una catena di Integrator chiude lo stato. Il blocco State-Space e' escluso perche' valuta A,B,C,D una sola volta a compile time. Le alternative sono un MATLAB Function block (che pero' rende il modello una scatola nera) o l'LPV System block (matematicamente equivalente a parita' di griglia, ma dipendente da un toolbox e molto piu' pesante: servirebbe un array di 141 modelli ss per rappresentare 7 scalari).

**D: Cos'e' il "trucco dei coefficienti effettivi" e perche' e' indispensabile?** R: Invece di tabulare i fattori grezzi  $a_1$ ,  $V$ ,  $a_4$ , ... e ricombinarli con blocchi, si tabulano i coefficienti **gia' combinati** ( $c_2 = a_1 \cdot V + a_4$ ,  $c_5 = A_6/V$ , ...) e sia il .slx sia la RHS ode45 interpolano le stesse tabelle. Serve perche' l'interpolazione lineare non commuta con il prodotto: su una cella,  $L\{f \cdot g\} - L\{f\} \cdot L\{g\} = Df \cdot Dg \cdot u \cdot (1-u)$ , che si annulla solo sui breakpoint e vale fino a  $Df \cdot Dg / 4$  a meta' cella. Se i due lati combinassero i fattori in momenti diversi, integrerebbero **funzioni diverse** e il residuo avrebbe un pavimento di modello indipendente dalle tolleranze. E' proprio la condivisione delle tabelle combinate che permette l'accordo a  $\sim 1e-7$  rad.

**D: Perche' MaxStep = 0.02 e non un valore piu' comodo?** R: Il generatore di turbolenza del professore aggiorna il rumore ogni 0.1 s. Con `MaxStep = 0.02` si hanno 5 passi per innovazione: il solver non scavalca le discontinuita' e i campioni di `alpha_w` loggati bastano a ricostruire lo stesso ingresso nella replica ode45. E' anche cio' che, di fatto, garantisce l'accuratezza del modello, dato che le tolleranze di Simulink restano ai valori di default.

**D: Il confronto Simulink vs ode45 a 1e-7 rad dimostra che il modello e' corretto?** R: Dimostra che le due implementazioni della **stessa** matematica coincidono. Non e' una validazione indipendente: la replica ode45 viene alimentata con l'`alpha_w` che il modello Simulink ha appena generato e loggato, quindi il ramo del vento e' comune ai due lati ed esce dal confronto. Cio' che il test copre e' il cablaggio del plant, le tabelle dei coefficienti e la legge di controllo; cio' che non copre e' il generatore di vento e la correttezza fisica del modello LPV.

**D: In cosa consiste esattamente il gain scheduling di questo modello?** R: `init_simulink_lpv` risolve `design_controller` su 28 plant congelati (`tsched = 5:5:140`, uno ogni 5 s), in continuazione con warm start dal punto precedente, e ne salva `Kp_sched`, `Kd_sched`. Nel modello, due lookup 1-D su `tsched` con estrapolazione Clip (necessaria perche' la simulazione parte a  $t=0$  mentre la griglia parte a  $t=5$  s) restituiscono  $Kp(t)$ ,  $Kd(t)$ , e due Switch pilotati dalla costante `sched` scelgono fra questi e la coppia congelata a max-q. **Solo** i guadagni di beccheggio sono schedulati: quelli di deriva restano bloccati al design di max-q in entrambe le configurazioni.

**D: Perche' costruire il modello da script invece che disegnarlo?** R: Perche' cosi' il modello e' testo: diffabile, revisionabile, riproducibile. La build e' distruttiva e idempotente (`new_system` da zero), quindi il .m resta l'unica fonte di verita' e il .slx un artefatto derivato che si rigenera quando il design cambia; sparisce il rischio che il modello Simulink diverga silenziosamente dallo script. Il costo e' che il modello non si puo' piu' ritoccare a mano (ogni edit si perde al rebuild) e che non e' autosufficiente: ogni parametro di dati dei blocchi e' un nome di variabile del base workspace, quindi senza `init_simulink_lpv` il .slx non compila.

# HM3/LTV\_FULL\_ASCENT/build\_hm3\_fu

## Ruolo del file nel progetto

Come il suo gemello rigido, questo file **e'** il modello Simulink: un authoring script che con `new_system` / `add_block` / `add_line` / `set_param` / `save_system` costruisce da zero HM3/LTV\_FULL\_ASCENT/hm3\_full\_ascent\_flex.slx. Non produce array di `ss`, non tocca alcun modello preesistente: cancella e ricrea. Il `.slx` e' un artefatto derivato.

Rispetto a `build_hm3_full_ascent.m`, aggiunge tutta l'**architettura flessibile** che in HM3 vive solo a max-q congelato (`build_plant_full`):

- primo modo di **bending** con frequenza `omega(t)` tempo-variante;
- accoppiamento **INS**: i sensori misurano `theta_m = theta + sigma*eta`, `z_m = z - phi*eta` (e le rispettive derivate), cioe' la piattaforma inerziale legge anche la deformazione strutturale;
- **notch** realizzato da blocchi elementari, il cui centro **insegue** `omega(t)`;
- **TVC** (servoattuatore + ritardo di trasporto approssimato con Pade) come funzione di trasferimento LTI.

E toglie una cosa: il **gain scheduling**. I guadagni PD sono congelati a max-q (header, riga 5: "PD gains FROZEN at max-q; the showcase is notch tracking, not scheduling"). Il messaggio del track e' un altro: la frequenza di bending spazia su `omega(t)` in `[16.48, 30.12]` rad/s sull'orizzonte effettivamente simulato (0-140 s; il dataset grezzo arriva a 31.77 rad/s, ma a `t = 150` s, fuori orizzonte – `init_simulink_lpv` riga 51 tronca la griglia a `Tstop`), quindi un notch fisso centrato su `omega(72)` si **detuna** e l'anello perde la gain-stabilization del modo strutturale.

La controparte MATLAB, dichiarata sorgente di verita', e' `ode_lpv_flex.m` (13 stati). Chi lo chiama: `run_flex_simulink.m`. Le variabili base necessarie (`lpv_omega`, `lpv_omega2`, `lpv_2zBMw`, `lpv_aqk`, `lpv_sig`, `lpv_phi`, `notch_zN`, `notch_zD`, `tvc_num`, `tvc_den`, oltre a tutte quelle del modello rigido) sono pushate da `init_simulink_lpv.m`.

**Nota fondamentale, dichiarata subito:** a differenza del track rigido, qui il `.slx` e la RHS `ode45` **non interpolano le stesse tabelle**. Vedi la sezione "La disciplina dei coefficienti effettivi, rotta".

---

## Firma, header e setup (righe 1-41)

```
lk = @(name, tbl, bp, pos) A(name, LK, pos, ...  
    'NumberOfTableDimensions','1', ...  
    'BreakpointsForDimension1', bp, 'Table', tbl, ...  
    'ExtrapMethod','Clip');
```

- Righe 18-20: unica opzione `o.open`, come nel rigido.
- Righe 28-29: carica `strong_wind.slx` e installa l'onCleanup che lo chiude con flag 0 (discard). Il file del professore non viene mai salvato.
- Righe 31-35: `new_system` da zero; solver `ode45` variable-step, `StopTime = 'Tstop'` (nome di variabile base), `MaxStep = 0.02` per risolvere le innovazioni a 0.1 s del generatore di rumore. Come nel rigido, `RelTol`

e **AbsTol** non vengono mai impostati: restano ai default Simulink (1e-3), mentre `run_flex_simulink.m` (riga 26) fa la replica con `odeset('RelTol',1e-9,'AbsTol',1e-11)`.

- Righe 40-41: qui c'è una differenza sostanziale rispetto al rigido. L'helper `lk` mette **ExtrapMethod** = 'Clip' su tutte le lookup, non solo su quelle dei guadagni. Clip (tieni il valore d'estremo) e' cio' che corrisponde all'extrapolazione 'nearest' usata dalle `griddedInterpolant` della baseline. Nel modello rigido questo non era stato fatto per `c1..c7`. Buona correzione, anche se su [0, 140] con `lpv_t = 0:1:140` non si estrapola comunque mai.

## Clock e le 14 lookup (righe 43-51)

```
names = {'c1','c2','c3','c4','c5','c6','c7','invV', ...
         'omega2','w2zBM','aqk','sig','phi','omega'};
tbls = {'lpv_c1', ... , 'lpv_invV', ...
        'lpv_omega2','lpv_2zBMw','lpv_aqk','lpv_sig', ...
        'lpv_phi','lpv_omega'};
for k = 1:numel(names)
    lk(names{k}, tbls{k}, 'lpv_t', [...]);
    W('Clock/1', [names{k} '/1']);
end
```

- Riga 44: un unico Clock: come nel rigido, il parametro di scheduling e' il tempo di volo e nient'altro.
- Righe 45-51: **quattordici** lookup 1-D, tutte sugli stessi breakpoint `lpv_t` (griglia verificata sul dataset: 0:1:140, **141 punti, passo 1 s**), interpolazione lineare (default `InterpMethod`). Otto sono quelle del plant rigido (`c1..c7, invV`); le sei nuove sono:

| blocco | tabella    | contenuto (da <code>init_simulink_lpv</code> ) |
|--------|------------|------------------------------------------------|
| omega2 | lpv_omega2 | $\omega.^2$ – <b>gia' elevato al quadrato</b>  |
| w2zBM  | lpv_2zBMw  | $2*p0.zBM*\omega$ – <b>gia' moltiplicato</b>   |
| aqk    | lpv_aqk    | forzante di bending del TVC (dal dataset)      |
| sig    | lpv_sig    | <code>sigma_ins</code> , leakage INS su theta  |
| phi    | lpv_phi    | <code>phi_ins</code> , leakage INS su z        |
| omega  | lpv_omega  | $\omega(t)$ grezzo (serve al notch)            |

- `lpv_2zBMw = 2*p0.zBM*omega`: nota che `zBM`, lo smorzamento del modo, e' preso **congelato** dai parametri a `t_ref = 72 s` (`p0 = load_hw3_params()`). Solo  $\omega$  e' tempo-variante. E' un'approssimazione mai dichiarata nel codice: la frequenza di bending varia dell'83% (16.48 -> 30.12 rad/s sull'orizzonte 0-140 s) mentre lo smorzamento e' tenuto costante.
- `lpv_aqk` viene dal campo `aqk` del dataset; il commento in `init_simulink_lpv` (riga 57) lo descrive come `-phi_tvc*Tc`, cioe' la spinta di controllo proiettata sulla forma modale nel punto di applicazione del TVC.

## Il generatore di vento e alpha\_w (righe 53-58)

Identico al modello rigido: il sottosistema del professore viene **copiato** (riga 54), il Clock lo alimenta, un `Sum` '++' somma profilo medio e turbolenza, un `Product` moltiplica per `invV`:

```
alpha_w(t) = ( v_wp(t) + turbolenza(t) ) * (1/V(t))
```

Vento e veicolo condividono lo stesso orologio: e' la differenza chiave rispetto a HM3, che ritagliava 12 s di vento attorno a `max-q`.

## Plant z/theta (righe 60-83)

Topologia **identica** al modello rigido: P1..P4 -> zdd('+++-' ) -> int\_zd -> int\_z e P5..P8 -> thdd('+++-' ) -> int\_thd -> int\_th, con c6 che alimenta sia P6 (c6\*theta) sia P8 (c6\*alpha\_w, con segno meno nel Sum).

```
zddot      = c1*zdot + c2*theta + c3*delta - c4*alpha_w
thetaddot = c5*zdot + c6*theta + c7*delta - c6*alpha_w
```

- Righe 81-83: la retroazione degli stati arriva dalle uscite degli integratori, quindi nessun loop algebrico.
- Il commento di riga 60 (“identical to the rigid model”) e’ vero, e l’header (riga 11) aggiunge la cosa importante: **“bending does not feed it”**. Il modo flessibile **eta non** entra nell’equazione del corpo rigido: entra solo attraverso i **sensori** (accoppiamento INS) e viene eccitato dal TVC. E’ l’ipotesi standard di modo strutturale disaccoppiato dalla dinamica di corpo rigido, e chiude l’anello attraverso il controllore, non attraverso la fisica.

---

## Bending (righe 85-96)

```
A('etadd','built-in/Sum',[...],'Inputs','--+',...);
A('int_etad','built-in/Integrator',[...],'InitialCondition','0');
A('int_eta','built-in/Integrator',[...],'InitialCondition','0');
W('omega2/1','Pe1/1'); W('int_eta/1','Pe1/2');
W('w2zBM/1','Pe2/1'); W('int_etad/1','Pe2/2');
```

- Righe 86-88: tre prodotti:  $Pe1 = \omega^2 * \eta$ ,  $Pe2 = (2*z_{BM}*\omega) * \dot{\eta}$ ,  $Pe3 = a_qk * \delta$ .
- Riga 89: Sum con Inputs = '--+', quindi

```
etaddot = -omega(t)^2 * eta - 2*zBM*omega(t) * etadot + aqk(t) * delta
```

E’ l’oscillatore del secondo ordine del primo modo di bending, forzato dalla deflessione dell’ugello. E’ esattamente la riga 40 di `ode_lpv_flex.m`.

- Righe 90-91: catena `int_etad -> int_eta`, condizioni iniziali nulle, coerenti con `zeros(6+2*nt,1)` della replica (`run_flex_simulink.m`, riga 41).
- Riga 94: `aqk` entra su `Pe3/1`; l’altro fattore, `delta`, arriva molto piu’ tardi (riga 147) dall’uscita del TVC. E’ il **solo** canale con cui il controllo eccita la struttura: se `aqk = 0` il modo resta a zero per sempre.

**Possibile domanda d’esame** – perche’ il bending e’ forzato da `delta` e non dal vento? *Risposta:* Nel modello del corso il modo flessibile e’ eccitato dalla spinta laterale dell’ugello applicata in coda ( $a_{qk} = -\phi_{tvc} * T_c$ , cioe’ la forma modale valutata nel punto di applicazione della spinta). Il vento entra come `alpha_w` nelle equazioni aerodinamiche di corpo rigido. E’ una scelta di modellazione: si trascura l’eccitazione aeroelastica distribuita e si tiene il solo percorso che chiude l’anello di controllo, che e’ quello che puo’ rendere instabile il sistema ( $\delta \rightarrow \eta \rightarrow INS \rightarrow \text{controllore} \rightarrow \delta$ ).

---

## Accoppiamento INS (righe 98-114)

```
A('theta_m','built-in/Sum',[...],'Inputs','++',...); % theta + sig*eta
A('z_m','built-in/Sum',[...],'Inputs','+-',...); % z - phi*eta
```

- Righe 99-106: quattro prodotti, `sig*eta`, `sig*etadot`, `phi*eta`, `phi*etadot`. Sia `sig` sia `phi` sono lookup su `t`: il leakage varia lungo l’ascesa.
- Righe 107-114: i quattro sommatori delle **misure**:

```

theta_m    = theta    + sigma(t)*eta
thetadot_m = thetadot + sigma(t)*etadot
z_m        = z        - phi(t)*eta
zdot_m     = zdot     - phi(t)*etadot

```

I segni ('++' per l'assetto, '+-' per la deriva) riproducono le righe 24-25 di `ode_lpv_flex.m`. Fisicamente: il giroscopio, montato in un punto della struttura, misura la rotazione locale, che e' quella di corpo rigido **piu'** la pendenza locale della deformata; l'accelerometro/posizione vede lo spostamento di corpo rigido **meno** lo spostamento modale nel suo punto. Segni opposti perche' **sigma** e **phi** sono, rispettivamente, la pendenza e lo spostamento della forma modale nel punto di misura.

Questo e' il meccanismo che rende il problema pericoloso: il controllore **non puo' distinguere** l'assetto vero dalla vibrazione. Se il guadagno d'anello alla frequenza di bending non e' attenuato, il PD reagisce a **sigma\*eta**, comanda **delta**, che tramite **aqk** rieccita **eta**: e' il flutter di controllo. Da qui il notch.

## Controllore a guadagni congelati (righe 116-123)

```

A('Gth', 'built-in/Gain', [...], 'Gain', 'Kp_th0');
A('Gthd', 'built-in/Gain', [...], 'Gain', 'Kd_th0');
A('u_pd', 'built-in/Sum', [...], 'Inputs', '----', ...);

```

- Righe 117-120: **quattro Gain costanti**. Non ci sono Switch, non ci sono lookup sui guadagni, non c'e' sched. Il PD e' il design HM3 Task-1 a max-q.
- Righe 121-123: `u_pd` con `Inputs = '----'`:

```
u_pd = -( Kp*theta_m + Kd*thetadot_m + Kp_z*z_m + Kd_z*zdot_m )
```

Corrisponde alla riga 27 di `ode_lpv_flex.m`. Attenzione: il controllore chiude sulle **misure** `theta_m`, `z_m`, ..., non sugli stati veri. E' proprio questo che porta **eta** dentro l'anello.

**Asimmetria fra i due track, da dichiarare:** `ode_lpv_flex.m` (riga 26) conserva il ramo `if M.sched`, `Kp = M.fKp(t)`; ... e riceve `fKp/fKd` da `run_flex_simulink.m` (righe 39), che poi imposta `'sched', false`. Il `.slx` flessibile **non ha proprio l'hardware** per la schedula: non e' selezionabile, va ri-autorata la build. Quindi il ramo `sched` della RHS flessibile e' capacita' morta rispetto al modello Simulink. Coerentemente, il README elenca fra i follow-up rimandati proprio "co-scheduled gains **and** notch in one flexible run".

## Il notch tempo-variante (righe 125-141)

```

A('G2zDw', 'built-in/Gain', [...], 'Gain', '2*notch_zD');
A('Gcout', 'built-in/Gain', [...], 'Gain', '2*(notch_zN-notch_zD)');
W('omega/1', 'G2zDw/1'); W('omega/1', 'Gcout/1');
...
A('xn2d', 'built-in/Sum', [...], 'Inputs', '--+', ...);
A('v_sum', 'built-in/Sum', [...], 'Inputs', '++', ...);

```

E' il pezzo piu' interessante del file. Il notch **non** e' un blocco di libreria: e' realizzato a mano in forma canonica di controllabilita', con i coefficienti presi dalle lookup su `omega(t)`.

- Righe 126-128: due **Gain** costanti applicati al segnale `omega(t)`, che producono i coefficienti tempo-varianti  $2*zD*omega(t)$  e  $2*(zN - zD)*omega(t)$ .
- Righe 129-133:  $Pn1 = omega^2 * xn1$  (usa la lookup `omega2`),  $Pn2 = 2*zD*omega * xn2$ , Sum '--+' con `u_pd`, poi la catena `int_xn2 -> int_xn1`. Quindi

```

xn1_dot = xn2
xn2_dot = -omega^2 * xn1 - 2*zD*omega * xn2 + u_pd

```

- Righe 134-141:  $Pn3 = 2*(zN - zD)*omega * xn2$ , e  $v\_sum$  '++':

```

v = u_pd + 2*(zN - zD)*omega * xn2

```

Corrisponde riga per riga alle righe 31-33 di `ode_lpv_flex.m`.

## Derivazione: perche' questa realizzazione E' il notch

Congelando  $\omega = \omega_n$ , la realizzazione e'

```

A = [ 0      1      ]   B = [0]
    [-wn^2  -2*zD*wn]   [1]
C = [ 0      2*(zN - zD)*wn ]   D = 1

```

Il denominatore e'  $\text{den}(s) = s^2 + 2*zD*\omega_n*s + \omega_n^2$  e si ha  $(sI - A)^{-1} * B = [1; s] / \text{den}(s)$ . Quindi

```

v/u = C*(sI-A)^-1*B + D
     = 2*(zN - zD)*wn*s / den(s) + 1
     = ( s^2 + 2*zD*wn*s + wn^2 + 2*zN*wn*s - 2*zD*wn*s ) / den(s)
     = ( s^2 + 2*zN*wn*s + wn^2 ) / ( s^2 + 2*zD*wn*s + wn^2 )

```

che e' esattamente il notch classico: stessa  $\omega_n$  a numeratore e denominatore (guadagno unitario in DC e ad alta frequenza),  $zN$  piccolo per scavare,  $zD$  grande per allargare. La profondita' e'

$$|N(j\omega_n)| = (2*j*zN*\omega_n^2) / (2*j*zD*\omega_n^2) = zN/zD$$

Con  $\text{notch\_zN} = 0.002$  e  $\text{notch\_zD} = 0.7$  (`init_simulink_lpv`, riga 116):  $zN/zD = 2.857e-3$ , cioe' **-50.9 dB** al centro. E' il "deep notch" di HM3.

## Il punto delicato: notch LTV, non "notch con parametro che cambia"

Va detto con onesta': quando  $\omega_n = \omega_n(t)$ , la funzione di trasferimento **non esiste** piu'. Cio' che il modello realizza e' una **realizzazione di stato tempo-variante** che, congelata a ogni istante, ha come TF il notch centrato su  $\omega_n(t)$ . Due conseguenze:

1. **Il risultato dipende dalla realizzazione.** Forme canoniche diverse danno la stessa TF congelata ma sistemi LTV **diversi**. Il `.slx` e `ode_lpv_flex.m` usano la stessa realizzazione, quindi coincidono fra loro - ma "il notch variabile" e' definito *da questa scelta*, non da un oggetto matematico unico.
2. **Mancano i termini in  $\omega_n\dot{\omega}_n$ .** Una conversione rigorosa da forma congelata a LTV genererebbe termini proporzionali a  $d(\omega_n)/dt$ . Il codice li ignora silenziosamente. La giustificazione (mai scritta nel codice) e' quasi-stazionaria:  $\omega_n$  passa da 16.5 a 30.1 rad/s in 140 s, cioe'  $\omega_n\dot{\omega}_n \sim 0.1 \text{ rad/s}^2$ , del tutto trascurabile rispetto a  $\omega_n^2 \sim 270\text{-}910 \text{ rad}^2/\text{s}^2$ .

## Notch a conoscenza perfetta

Il modello alimenta la lookup  $\omega_n^2$  sia al bending (Pe1, riga 92) sia al notch (Pn1, riga 136): centro del notch e frequenza vera del modo sono letteralmente **la stessa tabella**. E' un notch a conoscenza perfetta: nessun errore di stima di  $\omega_n$ , nessun disallineamento fra modello e realta'. E' il caso migliore possibile e va presentato come tale.

Inoltre: il caso **notch fisso** (quello che nel README diverge dopo  $t \sim 85 \text{ s}$ ) **non e' realizzato in questo .slx**. Il modello costruisce solo la versione variabile. Il confronto fisso-vs-variabile esiste unicamente nel track script (`main_flex.m` / `ode_lpv_flex.m`, che accetta un `M.fwn` qualsiasi). Il `.slx` non ha uno switch per riprodurlo.



## TVC (righe 143-147)

```
A('TVC','built-in/TransferFcn',[...], ...
    'Numerator','tvc_num','Denominator','tvc_den');
W('v_sum/1','TVC/1');
W('TVC/1','P3/2'); W('TVC/1','P7/2'); W('TVC/1','Pe3/2');
```

- Riga 144: il TVC e' un blocco **Transfer Fcn LTI**, i cui coefficienti arrivano da `init_simulink_lpv` (righe 112-114): `Wtvc = build_tvc(p0, 3)`, cioe' servo + **approssimante di Pade di ordine 3** del ritardo di trasporto (20 ms), poi `tfddata(tf(Wtvc),'v')`.
- Riga 147: l'uscita **delta** (deflessione fisica) va nel plant rigido (P3, P7) e nel bending (Pe3). Il terzo consumatore, il log (`log_delta`), e' cablato solo piu' avanti, alla riga 158 – il commento di riga 146 li elenca tutti e tre, il cablaggio della riga 147 ne fa due su tre.

Onesta' obbligatoria su questo blocco:

- **Il TVC e' congelato a max-q**: `p0 = load_hw3_params()` e' valutato a `t_ref = 72 s`. In un modello dove *tutto il resto* e' tempo-variante, l'attuatore e il ritardo restano LTI. E' una limitazione reale, non dichiarata nell'header.
- **Realizzazioni diverse fra i due track**. Il `.slx` usa un **Transfer Fcn** (che internamente costruisce la forma canonica di controllabilita' dai coefficienti `tvc_num/tvc_den`), mentre `ode_lpv_flex.m` (righe 34-35) usa la quadrupla `At, Bt, Ct, Dt` presa da `ssdata(ss(Wtvc))`. Sono equivalenti in ingresso-uscita ma hanno **stati diversi**: si possono confrontare `delta` e le uscite, non gli stati interni del TVC. Il passaggio `ss -> tf -> canonica` su un sistema di ordine 5 (servo 2 + Pade 3) e' anche numericamente il punto piu' fragile della catena.
- `run_flex_simulink.m` (riga 42) ricostruisce `delta_ode = Ct*x_tvc + Dt*0` con il commento "(Dt=0)": e' un **no-op aritmetico** che documenta l'assunzione che il TVC sia strettamente proprio. L'assunzione e' plausibile (servo di ordine 2 a grado relativo 2 in cascata a un Pade a grado relativo 0 -> grado relativo 2), ma **non e' verificata da nessuna asserzione**: se `Dt` fosse diverso da zero, quel confronto sarebbe semplicemente sbagliato.
- Nessuna **saturazione** e nessun **rate limit** sulla deflessione, ne' qui ne' nel modello rigido.

---

## La disciplina dei coefficienti effettivi, rotta (discrepanza reale)

Questa e' la cosa piu' importante della pagina.

Nel **track rigido** il `.slx` legge `lpv_c1..lpv_c7` e la RHS `ode_lpv_ascent.m` (righe 26-28) legge `M.fc1(t)..M.fc7(t)`, cioe' le `griddedInterpolant` costruite da `init_simulink_lpv` (righe 102-103) **sulle stesse identiche array**. Le due catene interpolano gli stessi numeri: integrano la stessa funzione.

Nel **track flessibile** questo **non accade**:

| grandezza                                             | cosa interpola il <code>.slx</code>                        | cosa interpola <code>ode_lpv_flex.m</code>    |
|-------------------------------------------------------|------------------------------------------------------------|-----------------------------------------------|
| coeff. di <code>theta</code> in <code>zddot</code>    | $L\{a_1 V + a_4\}$ (tabella <code>lpv_c2</code> , riga 46) | $L\{a_1\} * L\{V\} + L\{a_4\}$ (righe 19, 38) |
| coeff. di <code>alpha_w</code> in <code>zddot</code>  | $L\{a_1 V\}$ ( <code>lpv_c4</code> )                       | $L\{a_1\} * L\{V\}$ (riga 38)                 |
| coeff. di <code>zdot</code> in <code>thetaddot</code> | $L\{A_6/V\}$ ( <code>lpv_c5</code> )                       | $L\{A_6\}/L\{V\}$ (riga 39)                   |
| rigidezza di bending / notch                          | $L\{\omega^2\}$ ( <code>lpv_omega2</code> )                | $L\{\omega\}^2$ (righe 20, 33, 40)            |

ode\_lpv\_flex.m riceve fa1, fa3, fa4, fa6, fK1, fV, fomega – gli interpolanti dei **fattori grezzi** – e li ricombina *dentro* la RHS. Il .slx interpola invece le tabelle **gia' combinate**.

Poiche' l'interpolazione lineare non commuta con il prodotto, i due lati integrano **funzioni diverse fra i breakpoint**. Su una cella di ampiezza h, con  $u = s/h$ ,  $Df = f1-f0$ ,  $Dg = g1-g0$ :

$$L\{f*g\}(u) - L\{f\}(u)*L\{g\}(u) = Df*Dg * u*(1-u)$$

nulla solo sui nodi, massimo  $Df*Dg/4$  a meta' cella. E' un errore di **modello**: non si riduce stringendo la tolleranza del solver.

Quantificato sul dataset vero (griglia lpv\_t = 0:1:140, passo 1 s):

| coefficiente   | max     | scarto    |
|----------------|---------|-----------|
| c2 = a1*V + a4 | 2.24e-3 | 0.0034 %  |
| c4 = a1*V      | 2.24e-3 | 0.016 %   |
| c5 = A6/V      | 4.89e-7 | 0.014 %   |
| omega^2        | 6.81e-3 | 0.00075 % |

I coefficienti c1 = a1, c3 = a3, c6 = A6, c7 = K1 sono **grezzi** e quindi commutano esattamente: per quelli non c'e' discrepanza. Anche  $2*z_{BM}*omega$  e' lineare in omega, quindi  $L\{2*z_{BM}*omega\} = 2*z_{BM}*L\{omega\}$ : commuta anch'esso. La discrepanza riguarda solo le quattro righe della tabella sopra.

**Cosa si puo' e non si puo' dire.** Si puo' dire, con certezza dal codice, che i due lati del track flessibile **non integrano la stessa funzione** e che quindi il residuo del confronto ha un **pavimento di modello** che nessuna tolleranza elimina. Si puo' osservare che il residuo dichiarato nel README per il caso flessibile (**5e-7 rad su theta**) e' 2-5 volte quello del caso rigido (1.1e-7 frozen, 2.2e-7 scheduled), dove le tabelle invece coincidono – il che e' **coerente** con questa spiegazione. Non si puo' affermare che sia *provato* che la causa sia questa: non ho eseguito la simulazione, e il modello flessibile ha anche piu' stati, il notch e il TVC.

La correzione sarebbe banale: far consumare a ode\_lpv\_flex.m gli stessi S.fc1..S.fc7 e un S.fomega2 = gi(omega.^2) invece di ricombinare a1,V,a4 e omega^2 a mano.

**Possibile domanda d'esame** – il modello flessibile Simulink e il suo ode45 di riferimento sono lo stesso sistema? *Risposta:* No, non esattamente. Coincidono sui breakpoint della griglia (passo 1 s) ma differiscono in mezzo, perche' il .slx interpola i coefficienti **gia' combinati** (a1\*V+a4, A6/V, omega^2) mentre la RHS interpola i fattori grezzi e li ricombina. L'interpolazione lineare non commuta con il prodotto, quindi le due integrano funzioni diverse. Lo scarto e' piccolo (fino a ~0.02 % sui coefficienti) ma e' un errore di modello, non di solver: e' il motivo strutturale per cui il residuo flessibile non puo' scendere al livello del residuo rigido, dove invece la disciplina delle tabelle combinate e' rispettata.

## Logging (righe 149-158)

```
tw = {'-1','MaxDataPoints','inf'};
A('log_theta', ... , 'VariableName','theta_sl', ...);
A('log_eta', ... , 'VariableName','eta_sl', ...);
W('int_th/1','log_theta/1'); W('int_eta/1','log_eta/1');
W('TVC/1','log_delta/1');
```

- Riga 150: SampleTime = '-1' (ogni passo maggiore del solver) e MaxDataPoints = 'inf' (via il tetto di 1000 punti).
- Righe 151-158: sei To Workspace in formato Timeseries: theta\_sl, z\_sl, zdot\_sl, eta\_sl, delta\_sl, alpha\_w\_sl.
- **Cosa viene loggato come theta\_sl:** int\_th/1, cioe' l'assetto vero, non la misura theta\_m. E' la scelta corretta, perche' la replica confronta con x(:,3) di ode\_lpv\_flex, che e' anch'esso lo stato fisico.

- `delta_sl` viene preso dall'**uscita del TVC** (riga 158), quindi e' la deflessione fisica, non il comando `v` a valle del notch.

Osservazioni oneste:

- `z_sl` e `zdot_sl` sono loggati ma `run_flex_simulink.m` non li legge mai (riga 30 prende solo `theta_sl`, `eta_sl`, `delta_sl`, `alpha_w_sl`).
- Riga 35: `SignalLogging` e' abilitato con `SignalLoggingName = 'logout'`, ma **nessun segnale e' marcato per il logging** in tutta la build. `logout` resta vuoto e non viene letto. Configurazione morta, identica al modello rigido.
- Come nel rigido, `alpha_w_sl` serve a una cosa sola: `run_flex_simulink.m` (riga 36) costruisce la `windfun` della replica `ode45` come `griddedInterpolant` sull'`alpha_w` **generato dal modello stesso**. Il vento non viene rigenerato: e' quindi un **controllo di autoconsistenza** fra due implementazioni della stessa matematica, non una validazione indipendente. Il ramo del vento e' comune ai due lati e non e' sotto test.

# Salvataggio (righe 160-165)

`save_system` scrive `hm3_full_ascent_flex.slx` accanto allo script. Il modello e' un artefatto derivato: si rigenera lanciando il builder, e ogni modifica fatta a mano nel canvas viene distrutta dal `new_system` della build successiva (riga 32). Vale qui la stessa argomentazione del modello rigido: il `.m` e' l'unica fonte di verita', il modello e' testo diffabile, e il design non puo' divergere silenziosamente dal codice. Prezzo pagato: il `.slx` **non e' autosufficiente** (ogni parametro dei blocchi e' un nome di variabile del base workspace, persino `StopTime = 'Tstop'`), quindi senza `init_simulink_lpv` non compila nemmeno.

# Rigido contro flessibile: il riassunto

|                                          | <code>build_hm3_full_ascent</code>                                                    | <code>build_hm3_full_ascent_flex</code>                                                 |
|------------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| stati                                    | 4 ( <code>z</code> , <code>zdot</code> , <code>theta</code> , <code>thetadot</code> ) | 6 + 2 (notch) + <code>nt</code> (TVC) = 13                                              |
| lookup                                   | 8                                                                                     | 14                                                                                      |
| <code>ExtrapMethod</code>                | default Linear su <code>c1..c7</code> , Clip sui guadagni                             | Clip ovunque                                                                            |
| attuatore                                | <b>nessuno</b> (delta ideale)                                                         | Transfer Fcn LTI: servo + Pade(3), <b>congelato a max-q</b>                             |
| bending                                  | assente                                                                               | oscillatore 2o ordine, <code>omega(t)</code> da lookup, <b>zBM costante</b>             |
| sensori                                  | stati veri                                                                            | misure INS contaminate: <code>theta + sigma*eta</code> , <code>z - phi*eta</code>       |
| notch                                    | assente                                                                               | canonica di controllabilita', centro = <code>omega(t)</code> (stessa tabella del plant) |
| guadagni                                 | Switch frozen / scheduled (28 punti su <code>tsched = 5:5:140</code> )                | <b>solo frozen</b> a max-q                                                              |
| tabelle condivise con <code>ode45</code> | <b>si'</b> ( <code>c1..c7</code> )                                                    | <b>no</b> (la RHS ricombina i fattori grezzi)                                           |
| residuo dichiarato vs <code>ode45</code> | 1.1e-7 / 2.2e-7 rad                                                                   | 5e-7 rad                                                                                |

## Limiti, hack e codice stale (riepilogo onesto)

- **Discrepanza tabelle combinate vs fattori grezzi** fra `.slx` e `ode_lpv_flex.m` (`c2`, `c4`, `c5`,  $\omega^2$ ): i due lati non integrano la stessa funzione fra i breakpoint. E' il difetto piu' sostanziale.
  - **TVC LTI congelato a max-q** in un modello per il resto tutto tempo-variante.
  - **zBM costante** (preso a 72 s) mentre  $\omega$  varia dell'83 %.
  - **Notch a conoscenza perfetta**: centro del notch e frequenza vera del modo sono la stessa lookup. Nessun errore di stima, nessun disallineamento.
  - **Il caso "notch fisso" non e' realizzato nel .slx**: esiste solo nel track script. Il confronto fisso-vs-variabile del README non e' riproducibile con questo modello.
  - **Nessuno scheduling dei guadagni** nel `.slx` flessibile, mentre `ode_lpv_flex.m` conserva il ramo `sched` (capacita' morta lato Simulink).
  - **Termini in omega\_dot ignorati** nella conversione del notch a LTV; giustificabile come quasi-stazionario, ma mai dichiarato.
  - **Tolleranze del solver mai impostate**: Simulink a `RelTol` di default ( $1e-3$ ) contro  $1e-9$  della replica; l'accordo si regge su `MaxStep` = 0.02.
  - **Dt = 0 assunto e non verificato** (`run_flex_simulink.m`, riga 42: `+ M.Dt*0`, aritmetica morta usata come commento).
  - **Nessuna saturazione, nessun rate limit** su delta.
  - **SignalLogging/logsout attivi ma vuoti**; `z_sl`, `zdot_sl` loggati e mai letti.
  - **Validazione autoconsistente**, non indipendente: il vento e' preso dal modello stesso.
  - **Copia (non link) del sottosistema del professore**: se `strong_wind.slx` cambia, serve un rebuild manuale, senza avvisi.
- 

## Possibili domande d'esame

**D: Come si realizza un notch il cui centro varia nel tempo, senza blocchi di libreria?** R: Si scrive il notch in forma canonica di controllabilita' e si alimentano i coefficienti con lookup sul parametro di scheduling. Con  $A = \begin{bmatrix} 0 & 1 \\ -\omega_n^2 & -2\zeta\omega_n \end{bmatrix}$ ,  $B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ ,  $C = \begin{bmatrix} 0 & 2(\zeta\omega_n - \omega_n^2) \end{bmatrix}$ ,  $D = 1$  si ottiene  $C(sI-A)^{-1}B + D = \frac{(s^2 + 2\zeta\omega_n s + \omega_n^2)}{(s^2 + 2\zeta\omega_n s + \omega_n^2)}$ , cioe' il notch classico. In Simulink diventano due integratori, due Product (con  $\omega^2$  e  $2\zeta\omega$  dalle lookup), un Sum '---+' per  $\dot{x}_2$  e un Sum '++' per l'uscita. Rendendo  $\omega_n$  un segnale invece che una costante, si ha un notch che insegue  $\omega(t)$ .

**D: Che profondita' ha il notch e da cosa dipende?** R:  $|N(j\omega_n)| = \zeta\omega_n/\omega_n^2$ . Con `notch_zN` = 0.002 e `notch_zD` = 0.7 si ottiene 2.857e-3, cioe' **-50.9 dB** al centro.  $\zeta\omega_n$  controlla la profondita',  $\omega_n^2$  la larghezza di banda dell'attenuazione. Numeratore e denominatore condividono la stessa  $\omega_n$ , quindi il guadagno e' unitario sia in DC sia ad alta frequenza: il notch non tocca la banda di controllo ne' l'alta frequenza, scava solo attorno a  $\omega$ .

**D: Perche' serve un notch variabile e non basta quello di HM3?** R: Perche' HM3 congela tutto a max-q e centra il notch su  $\omega(72) = 18.9$  rad/s, ma sull'orizzonte simulato (0-140 s) la frequenza del primo modo spazia in [16.48, 30.12] rad/s (fino a 31.77 rad/s a  $t = 150$  s, sul dataset completo, che il modello non simula). Salendo,  $\omega(t)$  esce dalla campana del notch e il guadagno d'anello alla risonanza torna sopra 0 dB: il modo non e' piu' gain-stabilized e l'anello (che passa da  $\delta \rightarrow \eta \rightarrow INS \rightarrow PD \rightarrow \delta$ ) diventa instabile. Il README riporta divergenza da  $t \sim 75-85$  s con notch fisso.

**D: Ma un notch che conosce esattamente  $\omega(t)$  e' realistico?** R: No, ed e' un limite da dichiarare. Nel modello la lookup `omega2` alimenta sia il bending sia il notch: sono letteralmente la stessa tabella. E' il caso ideale di conoscenza perfetta. In volo  $\omega(t)$  andrebbe stimata (da modello strutturale, da massa residua, da identificazione online) e l'errore di stima ridurrebbe l'attenuazione effettiva. Il codice non modella nessun errore di stima.

**D: Il modello flessibile Simulink e il suo ode45 sono davvero lo stesso sistema?** R: No, non fra i breakpoint. Il `.slx` interpola i coefficienti **gia' combinati** (`lpv_c2` =  $a_1V+a_4$ , `lpv_c5` =  $A_6/V$ , `lpv_omega2` =  $\omega^2$ ), mentre `ode_lpv_flex.m` interpola i fattori grezzi e li ricombina nella RHS. Poiche' l'interpolazione lineare non commuta con il prodotto ( $L\{f*g\} \neq L\{f\} * L\{g\} = Df * Dg * u * (1-u)$ ), i due integrano funzioni diverse.

Lo scarto sui coefficienti arriva a  $\sim 0.02\%$  su questa griglia a 1 s; e' un errore di modello, non di solver, quindi mette un pavimento al residuo indipendente dalle tolleranze. Nel track **rigido** questa disciplina e' invece rispettata (entrambi leggono `c1..c7`), ed e' proprio questo che rende possibile l'accordo a  $1e-7$  rad.

**D: Perche' il bending non compare nelle equazioni di  $\ddot{z}$  e  $\ddot{\theta}$ ?** R: Per ipotesi di modellazione: il modo strutturale e' considerato disaccoppiato dalla dinamica di corpo rigido, e chiude l'anello **attraverso il controllore**, non attraverso la fisica. Il percorso e': il TVC eccita il modo ( $\mathbf{a} \mathbf{q} \mathbf{k} * \mathbf{\delta}$ ), il modo contamina le misure INS ( $\mathbf{\theta}_m = \mathbf{\theta} + \mathbf{\sigma} * \mathbf{\eta}$ ), il PD reagisce alla contaminazione e comanda  $\mathbf{\delta}$ . E' esattamente questo anello che il notch deve tagliare. Se il bending retroagisse anche sul corpo rigido, servirebbero i termini di accoppiamento inerziale nelle prime due equazioni, che qui sono trascurati.

# HM3/LTV\_FULL\_ASCENT/main\_full\_asc

## Ruolo del file nel progetto

E' lo script principale dello studio "oltre la traccia" (ticket T007). HM3 congela tutto all'istante di massima pressione dinamica ( $t_{ref} = 72$  s): matrici del plant, guadagni PD, notch e TVC sono valutati li' e l'anello chiuso e' LTI. Questo script solleva quel design all'**intera ascesa (0-140 s)**: il plant rigido di piano di beccheggio diventa **tempo-variante (LTV / LPV)**, i coefficienti sono letti istante per istante da `GreensiteLPV_DATA.mat`, e il generatore di vento del professore (`strong_wind.slx`, che copre tutta l'ascesa) viene messo **direttamente nell'anello** invece di essere finestrato attorno a max-q come fa `load_wind_profile`.

Lo scopo dichiarato e' rispondere a una domanda che la traccia non pone: *quanto lontano arriva un design a punto singolo?* Per farlo confronta, sullo stesso plant LTV e con lo stesso vento, due controllori: quello **frozen** (l'unica coppia PD progettata a 72 s, tenuta per tutto il volo) e quello **gain-scheduled** (una coppia PD per ogni punto di una griglia temporale, interpolata in volo). Le metriche sono il picco di assetto **theta**, il picco di deriva laterale **z**, il picco di comando TVC **delta** e l'indicatore di carico strutturale  $\bar{q} * \alpha$ .

Dipendenze: `init_simulink_lpv` (setup dati + schedule + vento, cartella LTV\_FULL\_ASCENT/), `ode_lpv_ascent` (RHS LTV, 4 stati), e i moduli HM3 riusati `load_hw3_params`, `build_plant_rigid`, `design_controller`, `assemble_loop`, `load_wind_profile`, `simulate_gust_response`. Lo script e' la **sorgente di verita'** numerica: `hm3_full_ascent.slx` (autorato da `build_hm3_full_ascent.m`) deve riprodurre la stessa risposta, e `run_full_ascent_simulink.m` sovrappone le due.

Nota importante da tenere presente leggendo tutta la pagina: **i numeri stampati oggi dal codice non coincidono con quelli scritti nel README della cartella** (che promette picco di deriva 33 m -> 27 m a favore dello scheduling). Ho eseguito lo script e la conclusione si e' *ribaltata*. La sezione "Cosa produce oggi il codice" spiega esattamente perche', con la causa individuata nel sorgente. Vale la pena saperlo prima dell'orale.

---

## Intestazione e preambolo (righe 1-20)

- Righe 1-15: banner di documentazione. Dichiaro onestamente lo scopo ("Portfolio showcase, NOT part of the HM3 deliverable"), i due controllori confrontati e il fatto che `ode45` e' la sorgente di verita' e Simulink il replay.
- Riga 18: `warning('off', 'Control:analysis:MarginUnstable')`. E' necessario perche' l'anello del lanciatore e' **condizionalmente stabile**: `margin/allmargin` emettono un warning ad ogni valutazione. Spegnerlo qui e' innocuo, ma va ricordato che il warning che si sta zittendo e' proprio quello che segnala la natura condizionale dell'anello.
- Righe 19-20: aggiunge al path la cartella padre HM3/ per riusare gli helper.

## %% Setup – coefficienti LPV, gain schedule, vento (righe 22-24)

```
S = init_simulink_lpv();  
t0 = S.t0; Tend = S.Tstop;
```

Una riga sola, ma e' il 90% del lavoro. `init_simulink_lpv` (default `t0 = 5 s`, `Tstop = 140 s`, `tsched_step = 5 s`) fa quattro cose:

**1. Costruisce i coefficienti efficaci del plant LTV.** Il modello rigido di HM3 (`build_plant_rigid`) e'

```
zddot      = a1*zdot + (a1*V + a4)*theta + a3*delta - a1*V*alpha_w  
thetaddot = (A6/V)*zdot + A6*theta      + K1*delta - A6*alpha_w
```

con  $A6$  = coefficiente di momento aerodinamico ( $\mu_{\alpha}$ , l'instabilita': il polo rigido e'  $+\sqrt{A6}$ ) e  $K1$  = efficacia di controllo ( $\mu_c$ ). `init_simulink_lpv` (righe 61-68) raggruppa i prodotti in **sette coefficienti efficaci**, uno per termine:

```
c1 = a1      (* zdot)      c5 = A6/V    (* zdot)  
c2 = a1*V + a4 (* theta)   c6 = A6      (* theta e * (-alpha_w))  
c3 = a3      (* delta)     c7 = K1      (* delta)  
c4 = a1*V     (* alpha_w)  invV = 1/V    (alpha_w = v_w * invV)
```

Il motivo del raggruppamento e' Simulink: cosi' ogni termine dell'equazione e' una lookup 1-D per un segnale, e il modello resta ispezionabile blocco per blocco. Ogni `c_i` diventa un `griddedInterpolant` lineare in `t` (righe 101-104 di `init_simulink_lpv`), che e' esattamente cio' che `ode_lpv_ascent` valuta ad ogni passo.

**2. Progetta i guadagni frozen S.K0** chiamando `design_controller` sul plant congelato a 72 s con attuatore ideale (Task 1 di HM3). Verificato eseguendo: `Kp_th = 1.7845`, `Kd_th = 0.4433`, `Kp_z = Kd_z = -1e-3`. I due guadagni di deriva `Kp_z`, `Kd_z` sono **costanti e mai schedulati** in tutto lo studio.

**3. Costruisce la gain schedule:** 28 punti su `tsched = 5:5:140 s`, una chiamata a `design_controller(build_plant_rigid(tsched(i)))` per punto.

**4. Simula il vento** una volta sola su tutto l'orizzonte (funzione locale `run_wind_generator`, righe 142-178) e ne ricava  $\alpha_w(t) = v_w(t)/V(t)$  come interpolante `S.windfun`. Verificato: 1408 campioni su  $[0, 140]$  s, picco  $|v_w| = 22.60$  m/s, picco  $|\alpha_w| = 2.42$  deg.

**Possibile domanda d'esame** – perche' i coefficienti sono raggruppati in `c1..c7` invece di tenere separati `a1`, `V`, `a4`? *Risposta:* per il solo `ode45` sarebbe indifferente. Serve al gemello Simulink: con `c2 = a1*V + a4` il termine in `theta` e' un prodotto (una lookup per il segnale `theta`), mentre tenendo `a1`, `V`, `a4` separati servirebbero due prodotti e una somma in piu' per lo stesso termine. Il prezzo e' che i coefficienti fisici non sono piu' leggibili singolarmente nel modello: si vede `c2(t)`, non `a1(t)` e `V(t)`.

---

## %% LTV closed-loop integration – frozen vs scheduled (righe 26-35)

```
tt = (t0:0.02:Tend).';  
x0 = zeros(4, 1); % [z, zdot, theta, thetadot]  
odeo = odeset('RelTol', 1e-8, 'AbsTol', 1e-10);  
[~, xF] = ode45(@(t,x) ode_lpv_ascent(t, x, make_model(S,0)), tt, x0, odeo);  
[~, xS] = ode45(@(t,x) ode_lpv_ascent(t, x, make_model(S,1)), tt, x0, odeo);
```

- Riga 27: griglia di uscita a 20 ms. Non e' il passo del solver (`ode45` e' a passo variabile): e' **solo** la griglia su cui `ode45` interpola l'uscita densa, e non influenza ne' i passi interni ne' la frequenza con cui la RHS campiona il vento (`ode_lpv_ascent`, riga 14: `aw = M.windfun(t)` viene valutata ai passi che il solver sceglie in base a `RelTol/AbsTol`, riga 29). Serve al confronto e alla sovrapposizione con Simulink. Il vincolo di risolvere le innovazioni di turbolenza del generatore (rumore a 0.1 s) vive invece nel gemello Simulink, dove e' imposto da `MaxStep = 0.02` (`build_hm3_full_ascent.m`, righe 33-36).

- Riga 28: **stato iniziale nullo a  $t_0 = 5$  s.** Approssimazione onesta da segnalare: il generatore di vento gira da  $t = 0$ , quindi fra 0 e 5 s il veicolo sarebbe già stato perturbato; qui si riparte da zero. L'errore è piccolo (il vento è debole a bassa quota) ma esiste.
- Riga 29: tolleranze strette. Non è pedanteria: la differenza fra i due controllori si legge su picchi di deriva di qualche metro, e il confronto con Simulink dichiara accordi a  $\sim 1e-7$  rad, quindi il baseline deve essere molto più accurato di così.
- Righe 31-32: **stesso plant, stesso vento, stesso stato iniziale;** cambia solo il flag `sched` dentro `make_model`. È il confronto controllato che rende il risultato interpretabile.
- Righe 34-35: **unpack** ricostruisce `delta`, `alpha` e `q_bar*alpha` a posteriori, con i guadagni *effettivamente usati* in quella corsa (vedi sezione `unpack`).

Il modello di controllo dentro `ode_lpv_ascent` (riga 22 di quel file) è

$$\delta = -(\mathbf{K}_p \theta + \mathbf{K}_d \dot{\theta} + \mathbf{K}_p z + \mathbf{K}_d \dot{z})$$

cioè **attuatore ideale** (nessun TVC, nessun ritardo, nessun notch) e `theta_ref = 0`: è un problema di *regolazione* dell'assetto attorno alla traiettoria nominale, non di inseguimento di un pitch program. Coerente con il Task 1 di HM3.

---

## %% Frozen-time margin sweep (righe 37-47)

```
tm = (t0:2.5:Tend).';
for i = 1:numel(tm)
    Gi = build_plant_rigid(load_hw3_params('t_ref', tm(i)));
    [gmF(i), pmF(i)] = loop_margin(Gi, S.K0);           % guadagni frozen
    Ksi = S.K0; Ksi.Kp_th = S.fKp(tm(i));
    Ksi.Kd_th = S.fKd(tm(i));
    [gmS(i), pmS(i)] = loop_margin(Gi, Ksi);           % guadagni schedulati
end
```

È il cuore concettuale dello script. Ad ogni istante `tm(i)` **congela** il plant (ricostruisce un **ss** LTI con i coefficienti a quell'istante) e ne legge i margini di guadagno e fase con (a) i guadagni fissi di `max-q` e (b) i guadagni schedulati.

La lettura “da manuale” del risultato è: il design frozen raggiunge il **minimo** dei suoi margini **esattamente a `max-q`** e ha più margine ovunque altrove, perché `max-q` è l'istante più instabile (**A6** massimo  $\rightarrow$  polo  $+\sqrt{A6}$  massimo). Questo pezzo **si riproduce**: eseguendo,  $|GM|$  frozen varia in [6.10, 37.76] dB e  $|PM|$  in [30.2, 52.0] deg, con **entrambi i minimi a  $t = 72.5$  s** (il punto della griglia adiacente a 72 s). È la giustificazione quantitativa della scelta della traccia.

### Due difetti reali di questo sweep, da conoscere.

1. `loop_margin` (righe 165-176) usa `margin(L)`, cioè il margine **di default** di MATLAB. Ma HM3 ha una funzione apposta, `classify_margins`, il cui commento dice testualmente che le attraversate sotto `w_drift` sono artefatti della deriva laterale e che “*Taking margin()'s default instead would pick one of these*”. Lo sweep qui usa proprio quel default. Al punto di design i due coincidono (verificato: a 72 s `margin()` dà 6.00 dB / 30.0 deg, identico all'aero-GM e al rigid-PM classificati), ma lontano da `max-q` **non coincidono più**.
2. Le figure plottano `abs(gm)` e `abs(pm)` (righe 104 e 108). Il valore assoluto **nasconde il segno**: un margine di fase negativo (anello instabile) appare come un margine grande e positivo. Verificato: a  $t = 25$  s con i guadagni schedulati l'anello chiuso è **instabile** (`isstable(T) = 0`, aero-GM classificato = NaN, PM = -163.8 deg) ma `margin()` restituisce GM = +10.22 dB e PM = -163.8 deg, che dopo `abs()` diventano “10.2 dB / 163.8 deg”, cioè hanno l'aria di margini eccellenti.

**Possibile domanda d'esame** – perché non basta `margin()` su questo anello? *Risposta*: perché l'anello è condizionalmente stabile e ha un integratore di deriva (la posizione  $z$  è l'integrale di  $\dot{z}$ ): il diagramma di Nichols entra “dall'alto” e produce attraversate a bassissima frequenza che non sono margini di corpo rigido. `margin()` restituisce il primo/minimo margine che trova, che può essere uno



di quegli artefatti. HM3 risolve con `classify_margins`, che separa le bande (drift / rigido / flessibile) e legge un margine per banda; questo script però non la usa.

---

## %% Consistency check at t\_ref (righe 49-61)

```
p72 = load_hw3_params(); % t_ref = 72 s
[~, T72] = assemble_loop(build_plant_rigid(p72), S.K0, []);
wg72 = load_wind_profile(p72); % raffica 1-cos, 12 s
rHM3 = simulate_gust_response(T72, wg72);
Mf = make_model(S, 0);
cc = @(v) griddedInterpolant([0 200], [v v], 'linear', 'nearest');
Mf.fc1 = cc(S.fc1(72)); ... % coefficienti congelati
Mf.windfun = griddedInterpolant(wg72.t(:), wg72.alphaw(:), ...);
[~, xf72] = ode45(@(t,x) ode_lpv_ascent(t, x, Mf), wg72.t, x0, odeo);
err_consistency = max(abs(xf72(:,3) - rHM3.theta));
```

E' il test di non-regressione più importante della cartella: **se congelo il modello LPV a 72 s, deve tornare esattamente il modello di HM3**. Il trucco della riga 55 è elegante: `cc(v)` costruisce un `griddedInterpolant` su due soli nodi con lo stesso valore, cioè una funzione costante; sostituendo `Mf.fc1..fc7` con queste costanti, `ode_lpv_ascent` gira "in modalità LTI" senza modificare una riga di `ode_lpv_ascent`.

Le due strade sono radicalmente diverse dal punto di vista numerico – da un lato `lsim` su un `ss` LTI costruito da `connect/getLoopTransfer`, dall'altro `ode45` sul RHS LTV – quindi l'accordo prova che il *plant*, la *convenzione dei segni* e il *cablaggio del controllore* sono gli stessi. Il vento usato è la **raffica 1-cosine** di default di `load_wind_profile` (`profile = 'gust'`, ampiezza da `drywind.mat` alla quota di max-q: verificato  $V_g = 6.38$  m/s,  $T_g = 3$  s, orizzonte 12 s), non la finestra di strong wind.

Verificato eseguendo: `err_consistency = 1.68e-9` rad (il README dichiara  $\sim 7e-10$ ; stesso ordine, la differenza dipende dal fatto che `S.K0` non è più lo stesso di quando il README è stato scritto – vedi sotto). In entrambi i casi è rumore di tolleranza.

---

## %% Summary (righe 63-73)

Stampa la tabella dei picchi per i due controllori, l'errore di consistenza e – riga 71 – `[qmax, iqm] = max(S.Q)`, cioè il **massimo della pressione dinamica del dataset**, che viene stampato accanto al punto di design.

E' un dettaglio che vale oro all'orale: `max(S.Q)` cade a **t = 67 s** (43.9 kPa) mentre il punto di design della traccia è **t = 72 s**. Non è un errore: il punto di design non è il massimo di `q_bar`, è il massimo di **A6** (verificato sui dati: A6 ha il suo massimo esattamente a  $t = 72$  s, valore  $3.382 \text{ 1/s}^2$ ). Cioè il "max-q" del titolo è in realtà il **massimo dell'instabilità aerodinamica**, che per questo veicolo è 5 s dopo il massimo di pressione dinamica – perché  $A6 = N_{\alpha} * l_{\alpha} / I_{yy}$  dipende anche da Mach (tramite `C_N_alpha`) e dall'inerzia, che cala per consumo di propellente. Tutta la sezione sullo scheduling in `q` (`main_q_scheduling.m`) nasce da questo sfasamento.

---

## %% Figures (righe 75-111) e %% Export (righe 113-120)

- f1 (righe 77-86): tre riquadri  $\theta$  /  $z$  /  $\delta$ , frozen (blu) vs scheduled (rosso), con `xline(72)` a marcare il punto di design.
- f2 (righe 89-98): l'indicatore di carico `q_bar * alpha` con la regione di max-q evidenziata da un `patch`. E' il *proxy* del carico strutturale (il momento flettente è proporzionale a `q_bar * alpha` a meno di fattori geometrici).

- f3 (righe 101-111): il margin sweep, con le linee di target 6 dB e 30 deg. Attenzione agli `abs()` di cui sopra.
- Righe 116-119: `theme(f,'light')` in `try/catch` (la funzione `theme` esiste solo su MATLAB recenti) e export PNG a 200 dpi in `figures/`.

## make\_model (righe 123-134)

Impacchetta `S` nella struct piatta che `ode_lpv_ascent` si aspetta. L'unico parametro e' `sched` (0/1), che diventa `M.sched` logico. Nota che passa **sempre** `fKp/fKd` e i guadagni frozen `Kp_th0/Kd_th0`: e' `ode_lpv_ascent` (righe 17-21) a scegliere quale usare. I guadagni di deriva `Kp_z`, `Kd_z` sono sempre quelli di `S.K0`: **la deriva non e' mai schedulata**.

## unpack (righe 136-163)

- Righe 146-150: ricostruisce i guadagni usati nella corsa (schedulati o frozen).
- Riga 151: **ricostruisce delta a posteriori** dalla stessa legge di controllo del RHS. E' corretto solo perche' l'attuatore e' ideale: `delta` e' una funzione algebrica dello stato, non ha dinamica propria. Nel caso flessibile (`ode_lpv_flex`) non si potrebbe fare, e infatti li' `delta` e' uscita di uno stato TVC.
- Riga 153: `alpha = theta + zdot/V - alpha_w - angolo di attacco totale`, con il **segno meno** sul vento (le righe 154-155 sono il commento che lo giustifica in sorgente). La derivazione: `theta` e' l'assetto perturbato, `zdot/V` e' l'angolo di traiettoria perturbato (per piccole perturbazioni normali alla traiettoria di riferimento, con `V` la velocita' nominale), e `alpha_w = v_w/V` e' la rotazione del **vettore velocita' relativa** prodotta dal vento laterale. `alpha` e' l'incidenza rispetto alla velocita' **relativa all'aria**, non rispetto al suolo: un vento che spinge nella direzione in cui il muso e' gia' inclinato **riduce** l'incidenza vista dal veicolo, da cui la sottrazione.
  - **E' la stessa convenzione del RHS che si sta integrando**, e questa e' la ragione decisiva: `ode_lpv_ascent.m` (righe 26 e 28) forza con `- M.fc4(t)*aw` e `- M.fc6(t)*aw`, cioe' su `alpha = theta + zdot/V - alpha_w`. Un `+` nel post-processing descriverebbe un'incidenza **diversa da quella che guida la dinamica appena integrata**.
  - *Nota onesta*: fino a poco fa questa riga aveva un `+`. Era un bug **di post-processing** – il plant LPV ha sempre avuto il meno – quindi `theta`, `z` e `delta` non ne erano toccati (e infatti non cambiano); a sbagliare era **solo** il canale diagnostico `alpha`, e con esso `qbar*alpha`. Ora e' allineato. L'effetto sul numero e' visibile: il picco di `qbar*alpha` frozen passa da 73.3 a **\*\*81.3 kPa\*deg\*\***.
- Riga 156: `qa = (Q/1000) * alpha_deg` in kPa\*deg. Usa `S.fQ`, cioe' la **Q del dataset** (picco 43.9 kPa), non la `p.qbar` di HM3 (circa 81 kPa, da atmosfera esponenziale a 15.1 km). Le due non coincidono: verificato `Q(72) = 42.4 kPa` dal dataset contro `0.5*1.225*exp(-15143/8000)*937.7^2 = 81.1 kPa` dalla formula di `load_hw3_params`. Il README lo dichiara. Onestamente: le due grandezze **non sono la stessa cosa** e il dataset non e' internamente coerente con un'atmosfera standard (anche il Mach del dataset a 72 s, 2.07, con `V = 937.7 m/s` implicherebbe una velocita' del suono di 453 m/s, che non e' quella a 15 km). Quindi `q_bar*alpha` qui va letto come **indicatore relativo**, buono per confrontare due controllori sullo stesso dataset, non come carico in unita' assolute.

## loop\_margin (righe 165-176)

Tre righe: assembla l'anello aperto con `assemble_loop(G, K, [])` (attuatore ideale) e chiama `margin`. `assemble_loop` rompe l'anello sul segnale `delta` con `getLoopTransfer(T, 'delta', -1)`, convenzione `1 + L`. Vedi i due difetti discussi nella sezione sul margin sweep: e' qui che vivono.

# Cosa produce oggi il codice (verifica numerica) e divergenze dal README

Ho eseguito il percorso di calcolo dello script (stessa `init_simulink_lpv`, stessi `ode45`, stesso `sweep`) sul commit corrente. Risultati **reali**:

| grandezza                      | frozen                               | scheduled                          |
|--------------------------------|--------------------------------------|------------------------------------|
| picco <code>abs(theta)</code>  | 0.971 deg                            | <b>25.36 deg</b>                   |
| picco <code>abs(z)</code>      | 29.53 m                              | <b>36.19 m</b>                     |
| picco <code>abs(delta)</code>  | 1.056 deg                            | 3.44 deg                           |
| picco <code>q_bar*alpha</code> | 81.3 kPa*deg                         | 263.9 kPa*deg                      |
| min GM (sweep)                 | 6.10 dB (a <code>t</code> = 72.5 s)  | 0.80 dB (a <code>t</code> = 120 s) |
| min PM (sweep)                 | 30.2 deg (a <code>t</code> = 72.5 s) | 4.5 deg (a <code>t</code> = 35 s)  |

I valori di `q_bar*alpha` sono quelli **dopo** la correzione del segno di `alpha` (vedi la sezione su `unpack`): col vecchio + erano 73.3 e 263.8. `theta`, `z` e `delta` **non cambiano** – il bug era solo nel post-processing, il plant integrato aveva già il segno giusto – quindi l’analisi qui sotto sullo scheduling non è toccata.

Il README della cartella dichiara invece “entrambi limitati, picco `theta` ~0.95 deg” e “la deriva cala da 33 m a 27 m grazie allo scheduling”, e “lo schedule tiene 6 dB / 30 deg piatti su tutto il volo”. **Oggi non è così: lo schedule è peggiore del design frozen.**

**Causa, individuata nel sorgente.** `init_simulink_lpv` (righe 74-85) costruisce lo schedule con una *continuation*: passa il risultato precedente come warm start, `design_controller(..., 'K0', Kprev, ...)`. Ma il `design_controller` attuale (HM3/`design_controller.m`) documenta esplicitamente `K0 ignored (kept for call compatibility)` – così alla riga 19 dell’help, ribadito dall’arguments alla riga 32 (`o.K0 (1,2) ... = [0 0] % accepted, unused`) – e parte **sempre** (riga 55) dal punto canonico di D’Antuono

`Kp = 2*A6/K1`                      `Kd = sqrt(A6)/K1`

**Quindi la continuation è un no-op:** il warm start viene calcolato, passato, e buttato. Verificato con git: al commit in cui il README è stato scritto (036cef6), `design_controller` usava davvero `x0 = log(o.K0)` e una cost basata su `margin()`; il refactor successivo (b43c5e9) lo ha riscritto nella forma D’Antuono + `classify_margins`, con `cost = 1e6` quando una banda di margine non ha attraversamento. Il README e le figure in `figures/` sono **antecedenti** a quel refactor.

Conseguenza fisica: agli istanti in cui l’instabilità aerodinamica è piccola (inizio ascesa, `A6(5 s) = 0.010`; e fine ascesa, `A6(140) = 0.558`) il punto di partenza canonico `2*A6/K1` **tende a zero**, e la cost restituisce `1e6` costante (nessun attraversamento aero-GM da agganciare), quindi `fminsearch` non si muove e restituisce il punto iniziale. Verificato: per `t <= 35 s` e `t >= 120 s` i guadagni schedulati coincidono **esattamente** con `2*A6/K1` e `sqrt(A6)/K1`; solo nella finestra `t = 40..115 s` la ritaratura riesce e centra 6.00 dB / 30.0 deg. Con `Kp_th ~ 0.006` a `t = 5 s` l’anello di assetto è di fatto aperto e i guadagni di deriva fissi (`-1e-3`) lo destabilizzano: l’LTI congelato a `t = 25 s` con i guadagni schedulati è **instabile** (`isstable(T) = 0`). Da lì i 25 deg di escursione di `theta`.

Come si presenta all’orale: non nascondere il fatto. La lettura corretta è che lo studio mette in luce un limite del **tuner**, non dello scheduling in sé: un gain schedule che insegue guadagni proporzionali ad `A6` collassa dove `A6 -> 0`, e serve un vincolo di banda minima (o mantenere i guadagni dell’ultimo punto valido) fuori dalla regione aerodinamicamente significativa. La parte che *si riproduce* – e che è il punto più importante – è che **il design frozen a max-q ha il suo minimo di margine esattamente a max-q** (6.10 dB / 30.2 deg a `t = 72.5 s`) e resta adeguato ovunque: la scelta di progetto della traccia è quantitativamente giustificata.

---

## Il generatore di vento del professore: come è cablato e perché usarlo

Cablaggio nel percorso `ode45` (funzione locale `run_wind_generator` in `init_simulink_lpv`, righe 142-178):

1. `load_system` di `General/hw3-v3/strong_wind.slx`, con `onCleanup` che chiude il modello **senza salvare** (riga 154): il file del professore non viene mai modificato.
2. Il logging viene attivato sulle **porte di uscita** del Subsystem (righe 157-160), non sulle linee – e' l'unico modo che funziona quando le linee non hanno nome.
3. `Simulink.SimulationInput` inietta le variabili che il generatore si aspetta (`drywind`, `GreensiteLPV`) e fissa lo `StopTime` (righe 162-167).
4. Le due uscite sono il **profilo medio di vento** `v_wp` (schedulato in quota) e la **turbolenza di Dryden** (con `sigma` schedulata in quota, letta da `drywind.mat`). Vengono sommate (righe 175-176) su una griglia unione, con `unique` per proteggersi dai tempi ripetuti dei log a passo variabile.
5. `alpha_w(t) = v_w(t) / V(t)` con `V` dal dataset (riga 90), e diventa `S.windfun`, un `griddedInterpolant` che `ode_lpv_ascent` valuta ad ogni passo.

Nel `.slx` (`build_hm3_full_ascent.m`, righe 57-64) il generatore e' **copiato una volta sola** come blocco (`add_block('strong_wind/Subsystem', ...)`) e alimentato dal Clock; `alpha_w` e' formato in-linea come `(v_wp + turb) * invV(t)`. Stessa formula, stessa quota, stesso seme.

**Perche' il generatore del professore e non un vento sintetico.** Tre motivi solidi: (a) il profilo medio e' funzione della **quota**, quindi ha senso solo se il veicolo vola per davvero da 0 a 140 s – una raffica 1-cosine di 3 s non puo' rappresentare il **plateau** di vento sostenuto nella regione di max-q, che e' proprio la sollecitazione dimensionante; (b) i semi sono fissi, quindi il confronto frozen/scheduled e' **riproducibile** e le differenze sono attribuibili al controllore e non al vento; (c) e' lo stesso disturbo su cui e' costruito il caso "strong wind" di HM3, quindi i due studi sono confrontabili (il picco di theta frozen, 0.97 deg, e' in effetti coerente con il caso strong-wind finestrato di HM3).

Limite onesto: e' **una sola realizzazione** di un processo stocastico. Nessuna affermazione statistica e' possibile da questa corsa; per quello serve il Monte Carlo (che HM3 ha, in `main_montecarlo.m`, ma sul modello congelato).

## La fallacia del frozen-time e come questo script la mette alla prova

La progettazione a punti congelati poggia su un'ipotesi che **non e' un teorema**: che se ogni sistema LTI congelato  $A(t_i)$  e' stabile, allora il sistema tempo-variante  $\dot{x} = A(t)x$  e' stabile. **E' falso in generale**: esistono matrici  $A(t)$  i cui autovalori stanno sempre in  $\text{Re} = -1$  e le cui soluzioni divergono (il controesempio classico e'  $A(t)$  con autovettori che ruotano abbastanza in fretta). Vale invece un risultato di **variazione lenta**: se  $\|dA/dt\|$  e' sufficientemente piccola rispetto al margine di stabilita' dei congelati, la stabilita' congelata implica quella LTV.

Questo script mette alla prova l'ipotesi nel modo empirico giusto: **calcola i margini congelati** (righe 37-47) e **integra il vero LTV** (righe 26-35), poi confronta. Ed e' proprio la separazione di scale a salvare il caso rigido: la banda dell'anello e' dell'ordine di  $\sqrt{A_6} \sim 1.8 \text{ rad/s}$  a max-q (periodo  $\sim 3.4 \text{ s}$ ), mentre i coefficienti variano su decine di secondi – circa un ordine di grandezza di separazione. Coerentemente, il controllore frozen, che e' congelato-stabile ovunque con margine  $\geq 6 \text{ dB}$ , produce una risposta LTV limitata (picco theta 0.97 deg).

Il caso in cui la fallacia si vede davvero e' nel gemello flessibile (`main_flex.m`): li' il notch fisso rende l'anello **congelato-instabile da  $t = 75 \text{ s}$** , eppure la risposta LTV resta piccola ancora per una decina di secondi (a  $t = 85 \text{ s}$  la coordinata di bending vale  $3.9e-4$ , come nel caso stabile) e diverge solo dopo  $\sim 90\text{-}100 \text{ s}$ . Instabilita' congelata non significa divergenza immediata: significa **crescita esponenziale a partire da quel punto**, e su un orizzonte finito la crescita puo' non fare in tempo a manifestarsi.

## Possibili domande d'esame

**D: Perche' progettare a max-q e non, per esempio, a meta' volo?** R: Perche' max-q e' l'istante **dimensionante**. Il polo instabile del corpo rigido e'  $+\sqrt{A_6}$  e  $A_6$  (il momento aerodinamico destabilizzante) ha il suo massimo li': il plant e' il piu' difficile da stabilizzare e il vento produce il momento maggiore. Lo sweep di margini congelati di questo script lo dimostra: con i guadagni fissi di max-q, i margini toccano il **minimo**

**esattamente a  $t = 72.5$  s** (6.10 dB, 30.2 deg – cioè i target di progetto) e sono piu' ampi ovunque altrove. Un design fatto altrove non garantirebbe nulla a max-q; un design fatto a max-q e' automaticamente conservativo altrove.

**D: Nel dataset la pressione dinamica ha il massimo a 67 s, ma il punto di design e' 72 s. Contraddizione?** R: No, e' un punto sottile. Il design point e' il massimo di  $A6$ , non di  $q_{bar}$ .  $A6 = N_{alpha} * l_{alpha} / I_{yy}$  e  $N_{alpha} \sim q_{bar} * S * C_{N_{alpha}}(Mach)$ : oltre a  $q_{bar}$  ci sono la dipendenza da Mach di  $C_{N_{alpha}}$  e la caduta dell'inerzia  $I_{yy}$  per consumo di propellente. Verificato sui dati:  $A6/q_{bar}$  **non e' costante** (0.044 a  $t = 40$  s, 0.080 a  $t = 72$  s, 0.145 a  $t = 105$  s). Il massimo di  $A6$  cade quindi 5 s dopo il massimo di  $q_{bar}$ . Questo sfasamento e' esattamente cio' che rompe lo scheduling in  $q$ .

**D: Che cosa dimostra il consistency check a 72 s e perche' non e' banale?** R: Dimostra che il modello LPV, congelato al punto di design, **coincide con il modello di HM3**. Non e' banale perche' le due strade sono numericamente indipendenti: da un lato `lsim` su una realizzazione `ss` ottenuta da `connect + getLoopTransfer`, dall'altro `ode45` sul RHS scritto a mano con interpolanti. Un errore di segno su  $\alpha_w$  **dentro il plant**, uno scambio di stati, un  $A6/V$  scritto come  $A6 * V$  – si vedrebbero subito. L'errore misurato e'  $\sim 1.7e-9$  rad, cioe' rumore di tolleranza dell'integratore.

**Ma va detto cosa il check NON copre**, perche' e' istruttivo: confronta solo  $\theta$ . Un errore di segno nel **post-processing** di  $\alpha$  (che non entra nella dinamica, ma solo nel calcolo diagnostico a valle) passerebbe indisturbato – ed e' esattamente quello che e' successo: `unpack` ha avuto per un po'  $\alpha = \theta + \dot{z}/V + \alpha_w$  invece del meno, e il consistency check restava verde perche'  $\theta$  era ed e' corretto in entrambe le strade. La lezione: un check di consistenza **verifica cio' che confronta**, e un canale diagnostico che non retroagisce sulla dinamica non e' coperto da nessun confronto di stato. Per coprirlo servirebbe confrontare anche  $\alpha$  fra le due strade, o – meglio – ricavare  $\alpha$  dalla dinamica stessa ( $\theta_{ddot} - K1 * \delta = A6 * \alpha$ ) invece di riscriverne la formula a mano.

**D: Lo scheduling nel tuo codice peggiora la risposta. Perche'?** R: E' un difetto del *tuner*, non del principio. `design_controller` riparte, ad ogni punto della griglia, dal valore canonico  $K_p = 2 * A6 / K1$ ,  $K_d = \sqrt{A6} / K1$ , e la sua cost vale  $1e6$  (costante) quando l'anello congelato non ha l'attraversamento aero-GM da agganciare. Dove  $A6 \rightarrow 0$  ( $t \leq 35$  s e  $t \geq 120$  s) succedono entrambe le cose: i guadagni canonici tendono a zero e `fminsearch` non ha gradiente da seguire, quindi resta li'. Con  $K_{p\_th} \sim 0.006$  l'anello di assetto e' praticamente aperto mentre i guadagni di deriva restano fissi a  $-1e-3$ , e l'LTI congelato risulta **instabile** (verificato a  $t = 25$  s). La correzione naturale e' saturare lo schedule ai guadagni dell'ultimo punto in cui il tuner converge, oppure imporre una banda minima. Da segnalare anche che la *continuation* dichiarata (warm start K0) e' oggi un **no-op**: `design_controller` accetta ma ignora K0.

**D: Che cos'e'  $q_{bar} * \alpha$  e perche' lo si guarda?** R: E' l'indicatore di **carico strutturale**: il momento flettente sul corpo del lanciatore e' proporzionale alla forza normale, che scala con  $q_{bar} * \alpha$ , dove  $\alpha$  e' l'angolo di attacco totale

$\alpha = \theta + \dot{z}/V - \alpha_w$

cioe' l'incidenza rispetto alla velocita' **relativa all'aria** (il meno lo impone il plant:  $B_w = [0; -a1 * V; 0; -A6]$ ). Un controllore che tiene l'assetto rigidamente (attitude hold) **non** si limita a subire  $\alpha \sim -\alpha_w$ : per opporsi al momento aerodinamico becca il muso **dentro** il vento relativo, e il termine d'assetto si **somma** a quello di vento. Misurato su HM3 a max-q: picco  $|\alpha| = 0.577$  deg contro i 0.390 deg del vento da solo – **l'attitude hold e' load-aggravating**. Un controllore di **load relief** lascerebbe invece che il veicolo si orienti nel vento riducendo  $\alpha$ , a costo di deriva; ma i guadagni di deriva della traccia ( $-1e-3$ ) sono troppo piccoli per farlo davvero. E' un trade-off, e questo script lo mostra apertamente invece di nascondere.

**D: Perche' il vento entra come  $\alpha_w = v_w / V$  e non come forza?** R: Perche' il modello e' **linearizzato**: la forza normale e'  $N = N_{alpha} * \alpha$  e un vento laterale  $v_w$  produce, al primo ordine, una rotazione del vettore velocita' relativa pari a  $\alpha_w = v_w / V$  (per piccoli angoli, con  $V$  la velocita' relativa nominale). Da qui i termini  $-a1 * V * \alpha_w$  e  $-A6 * \alpha_w$  nelle due equazioni: sono le stesse colonne di  $\alpha_w$  di `build_plant_rigid` ( $B_w = [0; -a1 * V; 0; -A6]$ ), e le stesse dei  $-M.fc4(t) * a_w / -M.fc6(t) * a_w$  di `ode_lpv_ascent` (righe 26 e 28). Il segno meno viene dal fatto che l'incidenza **aerodinamica** e'  $\alpha = \theta + \dot{z}/V - \alpha_w$ : raccogliendo, le due righe forzano su  $a1 * V * \alpha$  e  $A6 * \alpha$ , e la colonna di

vento eredita il meno. In altre parole: il vento non aggiunge incidenza al veicolo, **ruota l'aria sotto di lui**. Un limite del modello:  $V$  e' quella **nominale**, non la velocita' perturbata.

# HM3/LTV\_FULL\_ASCENT/main\_flex.m

## Ruolo del file nel progetto

E' il gemello flessibile di `main_full_ascent.m` (ticket T008, Goal 1). Il Task 2 di HM3 stabilizza il primo modo di bending **in guadagno** (gain stabilisation) mettendo un notch profondo centrato sulla frequenza modale a max-q,  $\omega_{BM}(72) = 18.9$  rad/s. Ma quella frequenza **non e' una costante**: il primo modo flessionale di un lanciatore e' essenzialmente  $\omega \sim \sqrt{k/m}$ , e mentre il propellente brucia la massa crolla, quindi  $\omega$  **sale** lungo l'ascesa. Sul dataset del corso  $\omega(t)$  va da 16.6 a 30.1 rad/s sull'orizzonte simulato (5-140 s): **quasi raddoppia**.

Un notch fisso, quindi, si **detuna**: dopo un po' non copre piu' la risonanza. Questo script lo dimostra in due modi complementari e li confronta:

- **frequenza** (righe 21-33): congela il plant flessibile completo ad ogni istante e legge il guadagno d'anello **alla frequenza di bending istantanea**,  $|L(j\omega(t))|$ , con il notch fisso e con il notch che insegue  $\omega(t)$ ; registra anche la stabilita' dell'anello chiuso congelato.
- **tempo** (righe 35-41): integra il vero sistema LTV flessibile a 13 stati (`ode_lpv_flex`) con il vento del generatore nell'anello, una volta col notch fisso e una col notch variabile.

La risposta alla domanda del brief – *il notch va anch'esso schedulato?* – e' **si'**, e il codice lo fa: il centro del notch e' un handle `fwn(t)` passato al RHS (riga 40: `make_flex(S, S.fomega)` per il caso variabile, riga 41: `make_flex(S, @t) w72` per il fisso), e dentro `ode_lpv_flex` (riga 30) `wn = M.fwn(t)` entra nei coefficienti del filtro ad ogni passo. **Solo il centro** e' schedulato: gli smorzamenti  $z_N = 0.002$  e  $z_D = 0.7$  restano costanti, e cosi' pure il TVC e i guadagni PD.

Dipende da `init_simulink_lpv`, `ode_lpv_flex`, e dai moduli HM3 `build_plant_full`, `build_tvc`, `build_notch_filter`, `assemble_loop`, `load_hw3_params`. E' la sorgente di verita' di `hm3_full_ascent_flex.slx` (autorato da `build_hm3_full_ascent_flex.m`, con il notch realizzato da blocchi elementari perche' nessun blocco di libreria ha i coefficienti tempo-varianti).

**Buona notizia rispetto agli altri due script della cartella:** `main_flex` usa i guadagni PD **frozen** (`sched = false`, riga 105), quindi **non e' toccato** dal problema della gain schedule descritto in `lpv_main_full_ascent.md`. I suoi risultati si riproducono.

---

## Intestazione e preambolo (righe 1-19)

- Righe 1-11: header. Dichiara  $\omega(t)$  in 16.5 -> 31.8 rad/s. Attenzione: **31.8 rad/s e' il valore a  $t = 150$  s**, cioe' fuori dall'orizzonte simulato. Verificato sui dati: sul dataset completo [0, 150] s  $\omega$  va da 16.48 a 31.77 rad/s, ma sull'orizzonte effettivo dello script – [5, 140], perche' `init_simulink_lpv` ha `t0 = 5` di default e la griglia di sweep e' 5:5:140 – va da **16.65 a 30.12 rad/s**. Il numero che lo script stampa a runtime (riga 45-46) e' quest'ultimo; l'header e il README riportano l'altro.
- Riga 14: si spegne il warning di margine instabile (l'anello e' condizionalmente stabile).
- Riga 18: `S = init_simulink_lpv()` – di nuovo il setup completo (dati LPV, 28 design PD, simulazione del generatore di vento).

- Riga 19:  $w72 = S.\text{notch}.wn72$ , cioè  $\omega$  interpolata a  $t = 72$  s. Verificato: **18.900 rad/s**, esattamente il valore di Tabella 1 della traccia (il dataset e la tabella coincidono al punto di design).

## %% Frozen-time detuning sweep (righe 21-33)

```
tm = (t0:5:Tend).';
for i = 1:numel(tm)
    p = load_hw3_params('t_ref', tm(i));
    G = build_plant_full(p, 'ins'); Wt = build_tvc(p, 3);
    [Lf, Tf] = assemble_loop(G, S.K0, ...
        Wt*build_notch_filter(w72, 0.002, 0.7, +1));
    [Lv, Tv] = assemble_loop(G, S.K0, ...
        Wt*build_notch_filter(p.wBM, 0.002, 0.7, +1));
    Lfix(i) = 20*log10(abs(squeeze(freqresp(Lf, p.wBM))));
    Lvar(i) = 20*log10(abs(squeeze(freqresp(Lv, p.wBM))));
    stbF(i) = isstable(Tf); stbV(i) = isstable(Tv);
end
t_unstable = tm(find(~stbF, 1));
```

- Riga 26: `build_plant_full(p, 'ins')` – plant a **6 stati** [ $z$   $\dot{z}$   $\theta$   $\dot{\theta}$   $\eta$   $\dot{\eta}$ ] con la **contaminazione INS**: la piattaforma inerziale non misura  $\theta$  pura ma  $\theta_m = \theta + \sigma_{\text{ins}} \eta$  e  $z_m = z - \phi_{\text{ins}} \eta$  (Eq. 2 della traccia). E' proprio questa contaminazione a chiudere l'anello sul modo flessibile e a renderlo destabilizzabile: senza di essa il bending non tornerebbe nel controllore.
- Riga 26: `build_tvc(p, 3)` – servo del 2o ordine ( $w_{\text{TVC}} = 70$  rad/s,  $z_{\text{TVC}} = 0.7$ ) in serie con l'approssimante di **Pade di ordine 3** del ritardo puro  $\tau = 0.020$  s. Nota:  $w_{\text{TVC}}$ ,  $z_{\text{TVC}}$ ,  $\tau$  sono **letterali di Tabella 1**, non presenti in `GreensiteLPV_DATA.mat` – quindi  $W_t$  e' **identico ad ogni iterazione**: l'attuatore non e' tempo-variante e non ha senso schedarlo. (Ricostruirlo dentro il loop e' solo lavoro sprecato, non un errore.)
- Righe 27-28: i due anelli. La catena serie e'  $W_{\text{act}} = \text{TVC} * \text{notch}$  (in LTI l'ordine non conta). Il notch e' `build_notch_filter(wx, 0.002, 0.7, +1)`, cioè **esattamente** il “deep notch” che HM3 sceglie in `main_task2.m` (righe 74-78:  $z_N = 0.002$ ,  $z_D = 0.7$ ,  $\text{sgn} = +1$ , min-phase). L'unica differenza fra  $L_f$  e  $L_v$  e' il **centro**:  $w72$  fisso contro  $p.w_{\text{BM}} = \omega(t)$  variabile.
- Righe 29-30: la metrica e'  $|L(j\omega(t))|$  in dB, cioè il guadagno d'anello **alla frequenza di risonanza istantanea**. E' la grandezza giusta da guardare per una stabilizzazione in guadagno: il criterio di progetto e' “tieni  $|L|$  ben sotto 0 dB dove il modo risuona”.
- Riga 31: `isstable(Tf)` sull'anello **chiuso** congelato – il verdetto vero, che non dipende da come leggi i margini.
- Riga 33: `t_unstable` = primo istante in cui l'anello col notch fisso e' instabile.

### Risultati verificati eseguendo il codice:

| t [s]                                          | 5     | 40    | 55    | 65    | 70           | 75    | 90    | 140   |
|------------------------------------------------|-------|-------|-------|-------|--------------|-------|-------|-------|
| $\omega(t)$<br>[rad/s]                         | 16.65 | 17.83 | 18.33 | 18.67 | 18.83        | 19.39 | 21.87 | 30.12 |
| $\backslash  L $<br>notch<br>fisso [dB]        | +7.8  | +4.8  | +0.5  | -6.4  | <b>-15.8</b> | +0.3  | +14.9 | +23.2 |
| $\backslash  L $<br>notch<br>variabile<br>[dB] | -28.1 | -24.6 | -23.2 | -22.4 | -22.0        | -21.9 | -22.2 | -22.7 |



| t [s]                            | 5       | 40      | 55      | 65      | 70      | 75               | 90        | 140       |
|----------------------------------|---------|---------|---------|---------|---------|------------------|-----------|-----------|
| anello chiuso,<br>notch<br>fisso | stabile | stabile | stabile | stabile | stabile | <b>INSTABILE</b> | INSTABILE | INSTABILE |

- `t_unstable = 75 s`. Il notch fisso perde l'anello **3 secondi dopo il punto di design**.
- Il notch variabile tiene  $|L(j\omega(t))|$  fra **-28.1 e -21.9 dB** su tutta l'ascesa, sempre stabile. (Il README dichiara "-12...-18 dB": **non e' quello che esce oggi**, il margine reale e' migliore di quanto il README prometta.)

**Quanto e' stretto quel notch?** Il valore al centro esatto e'  $|H(j\omega_n)| = z_N/z_D = 0.002/0.7 = 2.9e-3$ , cioe' **-51 dB**: una tacca profondissima e sottilissima. Il prezzo si legge nella tabella: a  $t = 70$  s il notch e' disallineato dello **0.4 %** e attenua ancora -15.8 dB; a  $t = 65$  s il disallineamento e' dell'**1.2 %** e restano solo -6.4 dB; a  $t = 75$  s siamo al **2.6 %** e l'attenuazione e' **sparita** (+0.3 dB). *Circa il 2-3 % di errore in frequenza consuma tutto il notch*. E' proprio la ragione per cui HM3, nel Task 2, propone anche una **tripletta di notch** a {0.9, 1.0, 1.1} \* `omega_BM` (main\_task2.m, righe 81-83): una tacca larga e' la difesa contro l'incertezza su `omega_BM` **quando non la si puo' schedulare**. Qui, avendo `omega(t)` dal dataset, si sceglie invece di inseguirla.

**Possibile domanda d'esame** – nella tabella, a  $t = 5-55$  s il notch fisso ha  $|L(j\omega)| > 0$  dB eppure l'anello chiuso e' **stabile**. Come e' possibile? *Risposta*: perche'  $|L| > 0$  dB alla frequenza di risonanza **non e' un criterio di instabilita'**. La stabilizzazione di un modo flessibile puo' avvenire in due modi: **in guadagno** (si tiene  $|L|$  sotto 0 dB attorno alla risonanza, ed e' quello che fa il notch) oppure **in fase** (si lascia passare il guadagno ma si garantisce che il lobo di bending sul Nichols passi dalla parte giusta del punto critico). Nell'ascesa iniziale il lobo, pur sopra 0 dB, ha una fase che non produce accerchiamento: il modo e' *phase stabilised per caso*. Il criterio vero e' quello della riga 31, **isstable** sull'anello chiuso. La yline alla riga 65, etichettata "0 dB (resonance uncovered)", **semplifica troppo** ed e' l'unico punto della figura da difendere con cautela.

## %% Time-domain flexible response (righe 35-41)

```
tt = (t0:0.02:Tend).';
nt = size(S.tvc.At, 1);
x0 = zeros(6 + 2 + nt, 1);
odeo = odeset('RelTol', 1e-8, 'AbsTol', 1e-10);
[~, xv] = ode45(@(t,x) ode_lpv_flex(t,x,make_flex(S,S.fomega)), tt, x0, odeo);
[~, xf] = ode45(@(t,x) ode_lpv_flex(t,x,make_flex(S,@(t) w72)), tt, x0, odeo);
```

- Riga 37: `nt = 5` (verificato: servo di 2o ordine + Pade 3 = 5 stati), quindi lo stato totale e' **6 + 2 + 5 = 13**: 6 di plant flessibile, **2 di notch**, 5 di TVC.
- Righe 40-41: **unica differenza fra le due corse**: l'`handle fwn`. `S.fomega` e' il `griddedInterpolant` di `omega(t)`; `@(t) w72` e' la costante. Tutto il resto (plant, vento, guadagni, TVC, `zN`, `zD`, `zBM`) e' identico. Confronto controllato pulito.

**Cosa fa `ode_lpv_flex` (il file chiamato, per capire cosa si sta integrando):**

- Righe 24-27: **misura INS contaminata** e legge di controllo.

```
theta_m = theta + sigma*eta      z_m      = z - phi*eta
u_pd     = -(Kp*theta_m + Kd*thetadot_m + Kp_z*z_m + Kd_z*zdot_m)
```

Il PD **non vede `theta`**, vede `theta_m`: e' l'`eta`' del bending che rientra nell'anello.

- Righe 30-33: **il notch tempo-variante**, realizzato in forma canonica di controllo:

```
wn      = fwn(t)
```

```

xn1' = xn2
xn2' = -wn^2*xn1 - 2*zD*wn*xn2 + u_pd
v     = 2*(zN - zD)*wn*xn2 + u_pd

```

Derivazione (vale la pena saperla): per  $H(s) = (s^2 + b_1 s + b_0)/(s^2 + a_1 s + a_0)$ , la forma canonica di controllo da'  $y = (b_0 - a_0)*x_1 + (b_1 - a_1)*x_2 + u$ . Nel notch  $b_0 = a_0 = \omega_n^2$  (numeratore e denominatore hanno **lo stesso termine noto**: e' cio' che rende la tacca simmetrica attorno a  $\omega_n$ ), quindi il termine in  $x_1$  **si cancella**; restano  $b_1 - a_1 = 2*zN*\omega_n - 2*zD*\omega_n = 2*(zN - zD)*\omega_n$  e il passante diretto  $+u$ . Da qui la riga 31 esattamente com'e' scritta. Con  $zN \ll zD$  il coefficiente e' negativo e grande, ed e' quello che scava la tacca.

- Righe 34-35:  $v$  (uscita notch) entra nel TVC (At, Bt, Ct, Dt, LTI) che produce la **deflessione fisica delta**. Ordine della catena: PD -> notch -> TVC -> plant.
- Righe 38-40: le equazioni del plant flessibile LTV,

```

zddot   = a1*zdot + (a1*V + a4)*theta + a3*delta - a1*V*alpha_w
thddot  = (A6/V)*zdot + A6*theta + K1*delta - A6*alpha_w
etaddot = -omega^2*eta - 2*zBM*omega*etadot + aqk*delta

```

con  $aqk = -\phi_{tvc}*T_c$  (verificato:  $aqk(72) = -65.56$ , coerente con  $-4.31e-5 * 1.52e6$  di Tabella 1): e' il **momento che il TVC inietta nel modo flessibile** deflettendo l'ugello. Il vento **non** forza direttamente il bending (nessun  $\alpha_w$  nella terza equazione), coerentemente con `build_plant_full` ( $B_w(6) = 0$ ).

**Limite tecnico da dichiarare (importante).** Realizzare un filtro tempo-variante congelando la sua forma canonica **non e' esatto**: per un sistema LTV non esiste la "funzione di trasferimento", e realizzazioni diverse della stessa  $H(s, \omega_n)$  congelata danno **sistemi LTV diversi** (i termini  $d(\omega_n)/dt$  che comparirebbero derivando la trasformazione di stato sono qui semplicemente **omessi**). L'approssimazione e' difendibile perche'  $\omega$  varia lentamente: circa 0.1 rad/s al secondo su un  $\omega \sim 20$  rad/s, cioe' **~0.5 % per secondo**, mentre la dinamica del filtro ha costanti di tempo dell'ordine di  $1/(zD*\omega_n) \sim 0.07$  s. Ma va detto, non nascosto.

## %% Summary (righe 43-51)

Stampa il range di  $\omega$  sulla griglia di sweep, l'istante di prima instabilita', il range di  $|L|$  col notch variabile, e i picchi di  $\theta$ . Verificato eseguendo:

```

omega(t) sweeps 16.6 -> 30.1 rad/s ; HM3 notch fixed at 18.9 rad/s
fixed notch: loop goes UNSTABLE at t = 75 s
varying notch: stable all ascent, |L| in [-28.1, -21.9] dB
peak |theta|: varying = 1.014 deg (bounded) ; fixed = 1.04e+20 deg (diverges)

```

Sul picco  $|\eta|$ : **variabile 3.5e-3, fisso 1.2e20**. Il 1e20 e' ovviamente privo di senso fisico: il modello e' **lineare, senza saturazione dell'attuatore e senza limiti strutturali** – un lanciatore vero si romperebbe molto prima. Va letto come "diverge", non come un numero.

## %% Figures (righe 53-81) e %% Export (righe 83-90)

- f1 (righe 55-69): lo sweep di detuning, con un patch rosso che marca la regione in cui il notch fisso rende l'anello instabile (da `t_unstable` in poi). E' la figura che racconta la storia.
- f2 (righe 72-81): dominio del tempo. Riquadro sinistro:  $\theta$  con `ylim([-2 2])` (riga 76) – la curva del notch fisso **esce dal grafico**, e il titolo lo dichiara onestamente ("theta off-scale = divergence"). Riquadro destro:  $|\eta|$  in scala **semilogaritmica** (riga 79), l'unico modo per mostrare sullo stesso asse 1e-3 e 1e20; il `+1e-12` evita  $\log(0)$ .
- Righe 86-89: export PNG con tema chiaro.

## make\_flex (righe 93-106)

Costruisce la struct per ode\_lpv\_flex. Punti da notare:

- Riga 102: 'fwn', fwn – e' l'unico parametro che distingue le due corse.
- Riga 105: 'sched', false – i guadagni PD restano quelli frozen di max-q (S.K0). I campi fKp/fKd sono passati ma **mai usati** (codice morto innocuo). Come scrive il README, il Goal 1 tiene i guadagni fermi e muove solo il notch; combinare le due cose e' un follow-up dichiarato aperto.
- Righe 102-103: zN, zD, zBM sono **costanti**. zBM = 0.005 viene da Tabella 1 (il dataset non fornisce uno smorzamento modale variabile), quindi non c'e' nulla da schedulare li'. Il TVC (At..Dt) e' l'LTI costruito una volta a  $t = 72$  s.

**Divergenza da HM3 da dichiarare all'orale.** S.K0 sono i guadagni progettati sul plant **rigido con attuatore ideale** (Task 1: design\_controller(build\_plant\_rigid(p0), []), riga 72 di init\_simulink\_lpv). Ma HM3, nel Task 2, dopo aver messo il notch **ri-tara il PD sull'anello completo** (main\_task2.m, righe 131-161), perche' – parole del sorgente – “the actuator + transport delay + notch lag collapses the rigid phase margin”. **main\_flex salta quella ritaratura:** usa i guadagni Task-1 con la catena Task-2. Ho verificato la conseguenza congelando l'anello flessibile a  $t = 72$  s con S.K0: aero GM = -6.08 dB (ok),  $|L(\omega_{BM})| = -21.9$  dB (ok, il notch fa il suo lavoro), anello chiuso **stabile**, ma **rigid PM = 14.6 deg** contro i 30 deg di target. Cioe' main\_flex gira nella configurazione che HM3 chiama “BEFORE re-tuning”. Questo **non invalida** il confronto fisso-vs-variabile (entrambe le corse usano lo stesso PD, quindi la differenza e' attribuibile al solo notch), ma i **margini assoluti** di questo studio non sono quelli del design finale di HM3.

---

## La fallacia del frozen-time, vista qui

Questo script e' il posto giusto per capire perche' “ogni LTI congelato e' stabile” **non e'** un teorema di stabilita' per il sistema tempo-variante – e vale anche il viceversa.

Il fatto: l'anello col notch fisso e' **congelato-instabile da  $t = 75$  s**. Se la stabilita' congelata implicasse quella LTV punto per punto, ci si aspetterebbe divergenza immediata. Invece, verificato sulla simulazione:

| t [s]                    | 85     | 100    | 140    |
|--------------------------|--------|--------|--------|
| $ \eta $ notch fisso     | 3.9e-4 | 6.6e-2 | 1.9e18 |
| $ \eta $ notch variabile | 3.9e-4 | 5.2e-5 | 1.3e-5 |

A  $t = 85$  s, dieci secondi *dopo* che l'anello congelato e' diventato instabile, la coordinata di bending del caso fisso e' **indistinguibile** da quella del caso stabile (3.9e-4 contro 3.9e-4). La divergenza diventa visibile solo intorno a **90-100 s** e poi esplode. Motivo: l'instabilita' congelata dice che esiste un **autovalore a parte reale positiva** in quel punto, ma la crescita e' **esponenziale con una costante di tempo finita**; su un orizzonte finito il modo instabile ha bisogno di tempo per emergere dal rumore. Un'analisi puramente congelata avrebbe dichiarato il veicolo perso a 75 s; il veicolo vero (LTV) sopravvive ancora un po'. E, simmetricamente, esistono sistemi in cui **tutti** i congelati sono stabili e l'LTV diverge lo stesso (il controesempio classico: matrici i cui autovalori stanno sempre a sinistra ma i cui autovettori ruotano rapidamente).

La regola operativa, ed e' quella che questi tre script applicano: **il design a punti congelati e' un'euristica di sintesi, non una prova**. La prova e' l'integrazione del vero LTV (o, se si vuole una garanzia, una funzione di Lyapunov parameter-dependent con un vincolo sul tasso di variazione dei parametri). Qui la separazione di scale e' generosa – i coefficienti variano su decine di secondi, la dinamica di bending ha periodo  $2\pi/20 \sim 0.3$  s – quindi il congelamento e' ben giustificato; ma la giustificazione va **esibita**, non presuppota.

## Possibili domande d'esame

**D: Perché la frequenza del primo modo di bending cresce lungo l'ascesa?** R: Perché è essenzialmente  $\omega \sim \sqrt{k/m}$  (rigidezza modale su massa modale). La rigidezza strutturale del corpo cambia poco, ma la **massa crolla** con il consumo di propellente (il lanciatore brucia gran parte della sua massa nel primo stadio), quindi  **$\omega$  sale**. Sul dataset del corso passa da 16.5 a ~30 rad/s fra il decollo e  $t = 140$  s: quasi il doppio. Un notch progettato a max-q e lasciato lì è fuori bersaglio dopo pochi secondi.

**D: Il notch va schedato o basta allargarlo?** R: Entrambe le strade sono legittime e HM3 le esplora tutte e due. Allargarlo (o metterne tre, a  $0.9/1.0/1.1 * \omega_{BM}$ , come in `main_task2.m`) è la difesa quando  $\omega_{BM}$  è **incerta ma non nota**: si copre un intervallo, pagando più ritardo di fase alla frequenza di crossover rigido e quindi margine di fase. Schedarlo è la scelta quando  $\omega(t)$  è **nota o stimabile** (qui viene dal dataset): si mantiene la tacca stretta e profondissima – -51 dB al centro, con  $zN/zD = 0.002/0.7$  – senza pagarne la larghezza. Il codice dimostra il costo del non farlo: **2-3 % di errore in frequenza consuma tutta l'attenuazione**.

**D: Cosa succede esattamente quando il notch si detuna? Perché l'anello si perde?** R: Il modo flessibile rientra nel controllore attraverso la **contaminazione INS** ( $\theta_m = \theta + \sigma \cdot \eta$ ,  $z_m = z - \phi \cdot \eta$ ): il sensore misura anche la vibrazione della struttura. Se il guadagno d'anello a quella frequenza non è attenuato, il PD reagisce alla vibrazione comandando il TVC, il TVC **eccita** il modo (termine  $aqk \cdot \delta$  nell'equazione di  $\eta$ , con  $aqk = -\phi_{tvc} \cdot T_c$ ), e si chiude un anello positivo strutturale: è il classico **accoppiamento aeroservoelastico** (pogo/tail-wags-dog). Il notch spezza l'anello attenuando  $|L|$  proprio lì.

**D: Con il notch fisso l'anello congelato è instabile da  $t = 75$  s, ma la simulazione diverge solo dopo ~90-100 s. Contraddizione?** R: No, è proprio la **fallacia del frozen-time** vista dal verso benigno. Instabilità congelata significa che a quell'istante esiste un autovalore a parte reale positiva: la crescita è esponenziale, ma con una costante di tempo finita, e parte da un'ampiezza piccolissima (a  $t = 85$  s  $|\eta|$  vale ancora  $3.9e-4$ , come nel caso stabile). Su un orizzonte finito il modo ha bisogno di secondi per emergere. Il corollario è che **la stabilità dei congelati non è né necessaria né sufficiente** per la stabilità dell'LTV: è un'euristica valida sotto ipotesi di variazione lenta, che qui è soddisfatta e che comunque va verificata integrando il sistema vero – che è esattamente ciò che fa questo script.

**D: Il tuo notch tempo-variante è esatto?** R: No, ed è bene dirlo. Realizzo il notch in forma canonica di controllo e poi rendo tempo-varianti i coefficienti ( $w_n = w_n(t)$  nelle righe 30-33 di `ode_lpv_flex`). Per un sistema LTV il concetto di funzione di trasferimento non esiste, e realizzazioni diverse dello stesso filtro congelato producono **dinamiche LTV diverse**: i termini in  $d(w_n)/dt$  sono omessi. L'approssimazione è difendibile perché  $\omega$  varia dello ~0.5 % al secondo mentre il filtro ha costanti di tempo di ~0.07 s – separazione di scale di oltre due ordini di grandezza – ma è un'approssimazione, non un'identità.

**D: Perché main\_flex non schedula anche i guadagni PD?** R: Per isolare l'effetto. Se schedulassi contemporaneamente notch e guadagni non saprei a quale dei due attribuire la differenza. Il Goal 1 (questo file) tiene i guadagni frozen e muove solo il notch; il Goal 2 (`main_q_scheduling.m`) muove i guadagni sul plant rigido. Il README dichiara apertamente che la combinazione dei due è un follow-up ancora aperto. Va però segnalato che i guadagni usati qui sono quelli **Task-1 (attuatore ideale)** e non quelli ritirati sull'anello completo come fa HM3 nel Task 2: con la catena TVC + ritardo + notch, il margine di fase rigido scende a **14.6 deg** contro i 30 deg di target.

# HM3/LTV\_FULL\_ASCENT/main\_q\_sched

## Ruolo del file nel progetto

`main_full_ascent.m` schedula i guadagni sul **tempo di volo**. E' comodo ma e' barare: un computer di bordo non "misura" il tempo dall'accensione come parametro fisico – o meglio, lo conosce, ma se la missione devia dal profilo nominale (ritardo di accensione, spinta fuori tolleranza, vento che rallenta la salita) il tempo non e' piu' una buona etichetta per lo stato del veicolo. La scelta canonica in letteratura LPV per un lanciatore e' schedulare su una grandezza **misurabile e fisicamente legata al plant**, e il candidato naturale e' la **pressione dinamica  $q_{bar}$**  (che il veicolo stima da pressione statica e velocita' relativa).

Questo script (ticket T008, Goal 2) prende **gli stessi identici guadagni** progettati da `design_controller` sulla griglia temporale e li **ri-indicizza su  $q$** , poi chiede:  $q$  e' una buona variabile di scheduling per questo veicolo? La risposta del codice – scritta in chiaro nell'header, righe 7-11 – e' **no, non pulitamente**, e lo script quantifica il perche':  $A6(t)$  (l'instabilita' aerodinamica) ha il massimo a  $t \sim 72$  s mentre  $Q(t)$  ha il massimo prima ( $t = 65-67$  s), quindi la mappa guadagno-vs- $q$  e' **isteretica**: il ramo crescente e quello calante di  $q$  chiedono guadagni diversi allo stesso  $q$ .

Struttura: costruisce la lookup  $Kp(q)$  dal solo **ramo ascendente** (dove  $q$  e' monotona), misura l'isteresi sul ramo discendente, integra tre risposte LTV (frozen, t-scheduled, q-scheduled) sullo stesso plant e sullo stesso vento, e fa uno sweep di margini congelati confrontando t-schedule e q-schedule. Dipende da `init_simulink_lpv`, `ode_lpv_ascent`, `build_plant_rigid`, `load_hw3_params`, `assemble_loop`. Il plant e' quello **rigido**: lo scheduling dei guadagni e' indipendente dal bending (che e' il Goal 1, in `main_flex.m`).

Anche qui vale l'avvertenza generale della cartella: **i numeri del README non si riproducono con il codice attuale** (l'isteresi dichiarata e' 41%, oggi esce 125%; il crollo di margine dichiarato "0.5 dB / 2 deg attorno a  $t = 105$  s" oggi non si legge li'). La causa e' la stessa spiegata in `lpv_main_full_ascent.md` (il refactor di `design_controller`). La **conclusione qualitativa dello studio, invece, regge** – e la regge in modo anche piu' netto.

---

## Intestazione e preambolo (righe 1-21)

- Righe 1-13: l'header dichiara gia' il risultato e la raccomandazione ("Mach (monotonic) would be the better measurable"). E' un buon esempio di come si scrive un'analisi negativa: si dice cosa non funziona e si propone l'alternativa.
  - Riga 16: `warning('off', ...MarginUnstable)` – come in `main_full_ascent`, l'anello e' condizionalmente stabile.
  - Righe 20-21: `S = init_simulink_lpv()` rifa tutto il setup (dati LPV, 28 design PD sulla griglia 5:5:140, simulazione del generatore di vento). Nota: lo script **ri-esegue** il generatore e i 28 tuning, non li riusa da `main_full_ascent` – ogni script e' autoconsistente, al costo di ripetere il lavoro.
-

## %% Build a q-keyed gain lookup (righe 23-37)

```
Qs = S.fQ(S.tsched)/1000;           % q ai punti di schedule [kPa]
[~, ipk] = max(Qs);                 % il picco separa i due rami
qa = Qs(1:ipk); Kpa = S.Kp_sched(1:ipk); Kda = S.Kd_sched(1:ipk);
Kp_of_q = griddedInterpolant(qa, Kpa, 'linear', 'nearest');
Kd_of_q = griddedInterpolant(qa, Kda, 'linear', 'nearest');
fKp_q = @(t) Kp_of_q(S.fQ(t)/1000);
fKd_q = @(t) Kd_of_q(S.fQ(t)/1000);
```

- Riga 24: valuta **q ai punti di design** (28 valori,  $t = 5:5:140$ ).
- Riga 25: trova l'indice del picco. Verificato:  $ipk = 13$ , cioè  $t = 65$  s,  $q = 43.29$  kPa (il vero massimo del dataset è a  $t = 67$  s con 43.9 kPa, ma sulla griglia a 5 s il massimo cade a 65 s). Le etichette delle legende alle righe 82-83 dicono infatti “ $t \leq 65$  s” / “ $t \geq 65$  s”: sono **hardcoded** e coerenti solo perché il picco cade lì con questi parametri di default; se si cambia **tsched\_step** diventano sbagliate.
- Riga 26: taglia il **ramo ascendente** (da  $t = 5$  a  $t = 65$  s). Verificato: su quel ramo **q** è strettamente monotona (da 0.39 a 43.29 kPa), che è la condizione necessaria perché **griddedInterpolant** accetti **qa** come griglia (richiede nodi crescenti). Fuori da lì la mappa  $t \rightarrow q$  **non è invertibile**, ed è esattamente questo il problema.
- Righe 27-28: la lookup è costruita **solo** sul ramo ascendente, con estrapolazione **'nearest'** (clip agli estremi).
- Righe 30-31: la composizione  $fKp\_q(t) = Kp\_of\_q(q(t))$  è la simulazione onesta di ciò che farebbe il computer di bordo: **misura q, cerca il guadagno in tabella**. Il fatto che qui **q** sia riottenuta da **t** è solo un artificio di simulazione (il veicolo la misurerebbe da sensori); la logica di controllo, però, vede solo **q**.

### L'isteresi (righe 33-37).

```
Kp_cmd_dn = fKp_q(S.tsched(ipk+1:end)); % cio' che la lookup comanda
Kp_need_dn = S.Kp_sched(ipk+1:end);      % cio' che quel ramo richiede
hyst = max(abs(Kp_cmd_dn - Kp_need_dn) ./ Kp_need_dn) * 100;
```

La metrica: sul ramo discendente, confronta il guadagno che la lookup-in-q comanda con il guadagno che il design a quel *tempo* aveva prodotto. Se **q** fosse una buona variabile di scheduling i due coinciderebbero. Verificato eseguendo: **hyst = 125.4 %**, massimo raggiunto a  $t = 140$  s.

---

## Perché q non basta – la derivazione (il cuore dello studio)

Vale la pena farla per bene, perché è *la* domanda d'orale su questo file.

L'argomento **a favore** di **q** è quello da manuale: il coefficiente destabilizzante è

$$A6 = \mu_{\alpha} = N_{\alpha} * l_{\alpha} / I_{yy}, \quad N_{\alpha} \sim q_{\text{bar}} * S_{\text{ref}} * C_{N_{\alpha}}$$

quindi il termine aerodinamico “scala con la pressione dinamica” e **q** sembra il parametro di scheduling naturale. E' inoltre **misurabile a bordo** (pressione statica + velocità relativa), a differenza del tempo, che è legato a un timeline nominale specifico.

L'argomento è però **incompleto su due punti**, ed entrambi si verificano sul dataset.

**Punto 1: A6 non è proporzionale a q.** In  $N_{\alpha} \sim q_{\text{bar}} * S * C_{N_{\alpha}}(\text{Mach})$  c'è la dipendenza da Mach del coefficiente aerodinamico, e nel denominatore c'è  $I_{yy}$ , che **cala** con il consumo di propellente. Verificato sul dataset (rapporto  $A6 / q_{\text{bar}}$ ,  $1/(s^2 \text{ kPa})$ ):

| t [s]      | 20     | 40     | 72     | 95     | 105    |
|------------|--------|--------|--------|--------|--------|
| A6 / q_bar | 0.0438 | 0.0438 | 0.0797 | 0.1045 | 0.1453 |

Il rapporto **triplica**: A6 e q non sono la stessa informazione. E' precisamente per questo che il massimo di A6 (t = 72 s) e' 5 s dopo il massimo di q (t = 67 s) – e da qui l'isteresi.

**Punto 2 (il piu' forte, e il piu' spesso dimenticato): K1 non c'entra nulla con q.** L'efficacia di controllo e'

$$K1 = \mu_c = T_c * l_c / I_{yy}$$

cioe' **spinta per braccio diviso inerzia**: e' una grandezza **propulsivo-inerziale**, non aerodinamica. Non dipende da q\_bar in alcun modo. Verificato: K1 cresce **monotonicamente** da 3.21 a 9.64 1/s<sup>2</sup> sull'orizzonte usato (le tabelle sono troncate a Tstop = 140 s; il dataset prosegue fino a t = 150 s, dove K1 arriva a 10.84, ma quel punto e' fuori dalla simulazione), mentre q sale e poi scende. E il guadagno canonico **da cui il tuner parte** e'

$$Kp = 2*A6/K1 \quad Kd = \text{sqrt}(A6)/K1$$

(design\_controller, riga 55: x0 = log([2\*A6/K1, sqrt(A6)/K1])); l'obiettivo effettivo della cost, riga 88, e' il matching dei margini sull'anello completo – ma dove la ritaratura non trova l'attraversamento da agganciare i guadagni **restano** il seed, ed e' il caso di buona parte della griglia, vedi lpv\_main\_full\_ascent.md). Quel guadagno dipende da **entrambi** i coefficienti. Una lookup in q e' cieca su K1.

La prova numerica piu' pulita, letta direttamente sui dati:

| t [s]       | q_bar [kPa] | A6    | K1   | Kp = 2A6/K1 |
|-------------|-------------|-------|------|-------------|
| 30 (salita) | 10.57       | 0.479 | 3.68 | 0.260       |
| 140 (fine)  | 10.59       | 0.558 | 9.64 | 0.116       |

**Stessa identica pressione dinamica (10.6 kPa), A6 quasi uguale, ma il guadagno richiesto e' 2.25 volte diverso** – interamente per colpa di K1, che q non puo' vedere. Ed e' esattamente il punto dove lo script misura il massimo di isteresi: la lookup comanda Kp = 0.2607 mentre servirebbe Kp = 0.1157 (+125.4 %, sovra-guadagno).

Il difetto duale, il **sotto-guadagno**, si legge a t = 105 s: li' q = 13.53 kPa, che sul ramo ascendente corrisponde a t ~ 33 s, dove pero' A6 valeva 0.589 contro 1.965 a t = 105 s (3.3 volte meno instabile). La lookup comanda Kp = 0.3221 quando servirebbe Kp = 0.8772: **63 % di sotto-guadagno** proprio nella discesa ad alto A6 – che e' la frase del README, e questa parte si riproduce.

**Conclusione corretta.** Serve un parametro di scheduling rho(t) che sia (a) misurabile, (b) **monotono** lungo la traiettoria (altrimenti la mappa rho -> plant non e' una funzione), e (c) sufficientemente informativo sul plant. q fallisce (b) e in parte (c). Il **Mach** e' monotono (verificato: da 0.03 a 8.34, all(diff(Mach)) > 0) ed e' misurabile: e' il candidato corretto, ed e' quello che il README indica come follow-up. In alternativa si schedula su **due** parametri (q e Mach, o q e massa), che e' cio' che si fa nei lanciatori veri.

**Possibile domanda d'esame** – “ma allora perche' i libri dicono di schedulare su q?” *Risposta:* perche' nella maggior parte dei casi si schedula la **parte aerodinamica** del problema (per esempio un load-relief o il guadagno di un anello che deve contrastare N\_alpha) e su un intervallo di volo in cui q e' monotona. Qui invece si schedula il PD completo, il cui guadagno dipende anche da K1 = Tc\*l\_c/I\_yy, e su un orizzonte che attraversa il massimo di q. Le due ipotesi implicite del “manuale” saltano entrambe.

## %% Closed-loop responses (righe 39-50)

```
runs = struct('name', {'frozen', 't-sched', 'q-sched'});
mdls = {make_model(S, 'frozen', fKp_q, fKd_q), ...
        make_model(S, 't',      fKp_q, fKd_q), ...
        make_model(S, 'q',      fKp_q, fKd_q)};
```

```

for k = 1:3
    [~, x] = ode45(@(t,x) ode_lpv_ascent(t,x,mdls{k}), tt, zeros(4,1), odeo);
    runs(k).r = unpack(tt, x, S, mdls{k});
end

```

Tre integrazioni sullo **stesso** plant LTV, **stesso** vento (S.windfun, il generatore del professore), **stesso** stato iniziale nullo a  $t_0 = 5$  s, tolleranze  $1e-8/1e-10$ . Cambia solo la sorgente dei guadagni di beccheggio. I guadagni di deriva  $K_{p_z} = K_{d_z} = -1e-3$  sono sempre quelli di S.K0, in tutti e tre i casi (righe 137-138): **la deriva non e' mai schedulata**.

Verificato eseguendo (picchi su tutta l'ascesa):

| controllore | picco $\backslash \theta\backslash $ | picco $\backslash z\backslash $ | picco $\backslash \delta\backslash $ |
|-------------|--------------------------------------|---------------------------------|--------------------------------------|
| frozen      | 0.971 deg                            | 29.5 m                          | 1.06 deg                             |
| t-sched     | 25.36 deg                            | 36.2 m                          | 3.44 deg                             |
| q-sched     | <b>530.9 deg</b>                     | <b>1517 m</b>                   | 59.4 deg                             |

La corsa **q-sched** **diverge**. Attenzione a come si legge: una parte della divergenza e' ereditata dal difetto del tuner descritto in `lpv_main_full_ascent.md` (lo schedule di partenza e' gia' cattivo per  $t \leq 35$  s), e la lookup in **q** lo **amplifica**, perche' a **q** bassa (inizio volo) restituisce guadagni quasi nulli e li ri-applica anche in coda al volo. La conclusione qualitativa (**q** non e' una buona variabile di scheduling) resta valida e anzi e' rafforzata; ma la magnitudine (531 deg) **non e' un risultato pulito** e non va presentata come tale.

---

## %% Frozen-time margin sweep (righe 52-61)

Stesso schema di `main_full_ascent` (e stesso `loop_margin` locale con `margin()` invece di `classify_margins`, e stesso `abs()` nei plot – **stessi due difetti**, vedi `lpv_main_full_ascent.md`): congela il plant ogni 2.5 s e legge GM/PM con i guadagni **t-sched** e con quelli **q-sched**.

Verificato, valori  $|GM|$  in dB (t-sched vs q-sched):

| t [s]   | 45   | 65   | 75          | 85          | 95          | 105      |
|---------|------|------|-------------|-------------|-------------|----------|
| t-sched | 6.00 | 6.00 | 6.00        | 6.00        | 6.00        | 6.00     |
| q-sched | 6.00 | 6.00 | <b>3.72</b> | <b>2.75</b> | <b>1.71</b> | 8.03 (*) |

(\*) da  $t \sim 100$  s in poi la lettura `margin()` cade su un'attraversata di deriva e il numero non e' un margine di corpo rigido: e' l'artefatto discusso sopra, che l'`abs()` fa sembrare ottimo. **Il vero segnale e' la banda 75-95 s**: mentre il t-schedule tiene i 6 dB di progetto, il q-schedule li perde progressivamente (3.7 -> 2.7 -> 1.7 dB) proprio nella discesa ad alto **A6**, cioe' dove la lookup sotto-guadagna. Questo e' il grafico che dimostra la tesi dello studio.

Il README dichiara invece “il margine crolla da 6 dB / 30 deg a 0.5 dB / 2 deg attorno a  $t = 105$  s”. Oggi non si legge cosi': il minimo del q-schedule esce **0.83 dB a  $t = 35$  s** (dove pero' anche il t-schedule e' rotto, 0.80 dB a  $t = 120$  s), quindi quel minimo **non e' attribuibile a q**. La degradazione attribuibile a **q** in modo pulito e' quella di 75-95 s.

---

## %% Summary (righe 63-74)

Stampa il tempo del picco di **Q**, l'isteresi in %, e per ciascuna delle tre corse i picchi piu' il minimo di  $|GM|/|PM|$  (solo per t-sched e q-sched: alla riga 69 **g** e **p** sono inizializzati a NaN e riempiti solo per **k = 2, 3**, quindi la



riga “frozen” mostra NaN nelle colonne dei margini – **non e’ un bug**, e’ voluto: lo sweep dei margini frozen sta nell’altro script).

---

## %% Figures (righe 76-115) e %% Export (righe 117-124)

- f1 (righe 78-90): **il grafico chiave**. Plotta Kp e Kd **contro q**, separando in blu il ramo q crescente e in rosso quello calante. Se q fosse una buona variabile di scheduling, le due curve si sovrapporrebbero; invece formano un **ciclo di isteresi**. E’ la rappresentazione visiva di “la mappa  $t \rightarrow q$  non e’ invertibile”.
- f2 (righe 93-102): risposte t-sched vs q-sched (theta, z, delta). Nota che la cell array **sig** (riga 96) usa i nomi dei campi di **r** per generare i tre riquadri in loop: elegante, ma implica che **unpack deve** produrre campi con quei nomi esatti.
- f3 (righe 105-115): sweep dei margini con i target 6 dB / 30 deg.

---

## make\_model (righe 127-144)

```
switch mode
    case 'frozen', M.sched = false; M.fKp = S.fKp; M.fKd = S.fKd;
    case 't',      M.fKp = S.fKp; M.fKd = S.fKd;
    case 'q',      M.fKp = fKp_q; M.fKd = fKd_q;
end
```

Il campo **sched** e’ inizializzato a **true** alla riga 138 e messo a **false** solo nel caso ‘frozen’. `ode_lpv_ascent` guarda **M.sched**: se falso usa **Kp\_th0/Kd\_th0** (i frozen di max-q), se vero valuta **M.fKp(t)**. Quindi la **stessa** funzione RHS serve i tre casi: nel caso ‘q’ la “funzione del tempo” **fKp\_q** e’ in realta’ una composizione **Kp\_of\_q(q(t))** – il RHS non se ne accorge. E’ un’astrazione pulita: il modello di scheduling e’ completamente incapsulato in un handle **t -> guadagno**.

---

## unpack (righe 146-161)

Come in `main_full_ascent`, ma piu’ snello (niente **alpha** ne’ **q\_bar\*alpha**). Ricostruisce **delta** a posteriori con i guadagni della corsa (riga 157) – lecito perche’ l’attuatore e’ ideale. Nota alla riga 155: legge **M.sched**, **M.fKp**, **M.fKd dal modello**, non da un flag esterno: cosi’ i guadagni usati per ricostruire **delta** sono garantiti identici a quelli usati dentro l’ODE. E’ un dettaglio di igiene che evita l’errore classico di plottare un **delta** incoerente con la simulazione.

---

## loop\_margin (righe 163-174)

Identica a quella di `main_full_ascent` (codice duplicato fra i due script – si sarebbe potuta estrarre in una funzione condivisa). Vedi li’ per la discussione su **margin()** vs **classify\_margins**.

---

## Possibili domande d’esame

**D: Perche’ si vorrebbe schedulare su  $q_{bar}$  invece che sul tempo?** R: Perche’ il tempo di volo non e’ un parametro *fisico* del sistema: e’ l’etichetta di un timeline nominale. Se la missione si discosta dal nominale (spinta fuori tolleranza, vento che rallenta la salita, accensione ritardata) il veicolo a  $t = 72$  s non e’ piu’ nello

stato per cui i guadagni erano stati progettati. Un parametro misurato a bordo –  $q_{\text{bar}}$ , Mach, massa stimata – segue lo stato reale del veicolo e rende lo scheduling robusto alle dispersioni di profilo. E' l'idea base del gain scheduling / LPV: schedulare su  $\rho(t)$  misurato, non sull'indice temporale.

**D: E allora perche' il tuo studio conclude che  $q_{\text{bar}}$  qui e' una scelta cattiva?** R: Per due motivi verificati sui dati. (1)  $q(t)$  non e' monotona: ha il massimo a  $t = 67$  s, quindi la mappa  $t \rightarrow q$  non e' invertibile e a uno stesso  $q$  corrispondono due istanti con plant diversi – la tabella dei guadagni diventa isteretica. (2) Il guadagno richiesto e'  $K_p = 2 \cdot A_6 / K_1$ , e mentre  $A_6$  e' aerodinamico (legato a  $q$ , ma non proporzionale:  $A_6/q$  triplica lungo il volo per effetto del Mach e della caduta di  $I_{yy}$ ),  $K_1 = T_c \cdot l_c / I_{yy}$  e' propulsivo-inerziale e cresce monotonamente da 3.2 a 9.6 sull'orizzonte simulato (10.8 all'ultimo punto del dataset,  $t = 150$  s, fuori dall'orizzonte) –  $q$  e' completamente cieca su di esso. Esempio numerico: a  $t = 30$  s e a  $t = 140$  s il veicolo ha la stessa  $q_{\text{bar}}$  (10.6 kPa) ma richiede guadagni 2.25 volte diversi.

**D: Come misuri l'isteresi nel codice, e quanto vale?** R: Costruisco la lookup  $K_p(q)$  sul solo ramo ascendente (dove  $q$  e' monotona,  $t = 5-65$  s), poi sul ramo discendente confronto il guadagno che quella lookup comanda con quello che il design a quel tempo aveva prodotto:  $\text{hyst} = \max(|K_{p\_cmd} - K_{p\_need}| / K_{p\_need})$ . Verificato oggi: 125.4 %, con il massimo a  $t = 140$  s. (Il README riporta 41 %: e' un numero antecedente al refactor di `design_controller`.)

**D: Qual e' la variabile di scheduling corretta, allora?** R: Il Mach: e' misurabile a bordo, e' monotono lungo l'ascesa (verificato sul dataset: da 0.03 a 8.34, derivata sempre positiva) e cattura la fisica che manca a  $q$  (la dipendenza da Mach di  $C_{N\_alpha}$ ). Non risolve pero' da solo il problema di  $K_1$ : la soluzione industriale e' uno scheduling a due parametri (per esempio  $q_{\text{bar}}$  e Mach, o  $q_{\text{bar}}$  e massa), oppure schedulare su una variabile derivata come  $A_6/K_1$  stessa se e' stimabile.

**D: Che differenza c'e' fra gain scheduling e vero controllo LPV?** R: Il gain scheduling classico e' quello che fa questo codice: si progetta un controllore LTI su ogni punto congelato e si interpola fra i guadagni; la stabilita' del sistema tempo-variante risultante non e' garantita da nessun teorema, va verificata a posteriori (ed e' esattamente cio' che fa l'integrazione ode45 dell'LTV). Il controllo LPV vero e proprio progetta un unico controllore parametrizzato in  $\rho$  con garanzie di stabilita' sull'intera famiglia (via LMI / funzioni di Lyapunov parameter-dependent), tipicamente imponendo un limite sulla velocita' di variazione  $\rho_{\text{dot}}$ . Il primo e' quello che si usa sui lanciatori reali; il secondo e' la copertura teorica.

**D: Se la corsa q-sched diverge, come fai a dire che il problema e' l'isteresi e non il tuner rotto?** R: Non lo dico dalla risposta temporale, che e' contaminata (lo schedule di base e' gia' degradato per  $t \leq 35$  s). Lo dico dallo sweep di margini congelati nella banda 75-95 s, dove il t-schedule tiene esattamente i 6 dB di progetto mentre il q-schedule scende a 3.7 -> 2.7 -> 1.7 dB. In quella banda entrambi gli schedule sono ben tarati, quindi la differenza e' interamente attribuibile alla ri-indicizzazione in  $q$ : e' li' che si misura il costo dell'isteresi, non nella divergenza temporale.

# HM3/LTV\_FULL\_ASCENT/init\_simulink\_

## Ruolo del file nel progetto

Questo file e' il **setup unico** dello showcase LPV “full ascent” (la parte di HM3 che va oltre la traccia: il design a max-q sollevato a tutta l'ascesa, 0-140 s). Fa una cosa sola ma la fa per due consumatori diversi:

1. **carica e pre-elabora tutti i dati che dipendono dal tempo** – i coefficienti del plant LPV, il guadagno schedulato, il vento generato dal modello del professore, i dati del modo flessibile, il notch e il TVC;
2. li **restituisce due volte**, in due formati:
  - come **struct S di griddedInterpolant** -> e' quello che consumano i RHS `ode_lpv_ascent / ode_lpv_flex` dentro `ode45` (la *baseline*, la “fonte di verita” dichiarata dal README);
  - come **variabili nel base workspace** (righe 119-138) -> e' quello che consumano i due modelli Simulink `hm3_full_ascent.slx` e `hm3_full_ascent_flex.slx`, che referenziano quelle variabili per nome dentro i blocchi Lookup Table, Gain, Constant e Transfer Fcn.

Il punto chiave, e la ragione per cui questo file esiste come modulo separato, e' proprio il **numero 2**: script e modello Simulink devono partire *dagli stessi identici numeri*, altrimenti il confronto Simulink-vs-ode45 non misurerebbe l'errore di integrazione ma una differenza di dati. `init_simulink_lpv` e' il collo di bottiglia comune, il punto in cui i due binari si toccano.

E' il corrispettivo LPV di `HM3/init_simulink_hm3.m` (quello congelato a `t_ref = 72 s`): la' le matrici di `build_plant_rigid` sono **costanti**, qui diventano **storie temporali** campionate da `General/hw3-v3/GreensiteLPV_DATA.mat` (151 campioni, passo 1 s, t da 0 a 150 s – verificato caricando il `.mat`).

Chiamato da: `main_full_ascent.m`, `main_flex.m`, `main_q_scheduling.m`, `run_full_ascent_simulink.m`, `run_flex_simulink.m`. Dipende da (cartella padre HM3/): `load_hw3_params`, `build_plant_rigid`, `design_controller`, `build_tvc`.

---

## Firma, contratto e opzioni (righe 1-37)

```
function S = init_simulink_lpv(o)
arguments
    o.tsched_step (1,1) ... = 5      % passo griglia schedule [s]
    o.t0          (1,1) ... = 5      % inizio schedule / sim [s]
    o.Tstop       (1,1) ... = 140    % orizzonte [s]
    o.push        (1,1) logical = true
end
```

- Riga 1: unica uscita **S**, tutto il resto (le variabili base) e' un **side effect** controllato da `o.push`. Chi vuole solo la baseline ode45 puo' chiamare `init_simulink_lpv('push', false)`.
- Righe 2-30: l'header documenta il contratto piu' importante del file, quello dei **coefficienti efficaci** (righe 7-12):

```
c1 = a1      (* zdot)      c5 = A6/V      (* zdot)
```

```

c2 = a1*V + a4 (* theta)      c6 = A6      (* theta e * (-alpha_w))
c3 = a3          (* delta)    c7 = K1      (* delta)
c4 = a1*V        (* alpha_w)  invV = 1/V    (alpha_w = v_w * invV)

```

cioe' ogni termine dell'equazione e' UN lookup moltiplicato per UN segnale. Non e' un dettaglio estetico: e' la scelta che rende possibile l'accordo a  $1e-7$  rad (vedi la sezione sui coefficienti, sotto).

- Righe 39-42: `addpath` della cartella padre HM3/ – il file riusa gli helper dell'homework “vero” invece di duplicarli.

## Caricamento dati di riferimento (righe 44-47)

```

D = load(fullfile(gdir,'GreensiteLPV_DATA.mat')); L = D.GreensiteLPV;
W = load(fullfile(gdir,'drywind.mat'));          drywind = W.drywind;
GreensiteLPV = L;                               % lo vuole anche il generatore

```

- Riga 45: `GreensiteLPV` e' una struct di timeseries (campi `A6`, `K1`, `a1`, `a3`, `a4`, `sigma_ins`, `phi_tvc`, `phi_ins`, `omega`, `Q`, `V`, `Tc`, `h`, `aqk`, `Mach`).
- Riga 47: e' un **alias di comodo** (`GreensiteLPV` e `L` sono lo stesso dataset), non un vincolo funzionale. Il generatore riceve il dataset come **argomento** (riga 88: `run_wind_generator(gdir, o.Tstop, drywind, L)`), e il nome viene imposto come **stringa letterale** sia da `in.setVariable('GreensiteLPV', ...)` (riga 164) sia dal `fieldname` del push in base workspace (riga 127): il nome della variabile *chiamante* e' quindi irrilevante. Cio' che conta davvero e' che nel **base workspace** esista una variabile di nome `GreensiteLPV`, perche' la copia del generatore dentro i `.slx` la risolve per nome.

## Coefficienti efficaci e breakpoints (righe 49-68)

```

tg = L.V.Time(:); tg = tg(tg <= o.Tstop + eps);
at = @(f) interp1(L.(f).Time, squeeze(L.(f).Data), tg);
...
Vsafe = max(V, 1);           % V(0)=0: guard A6/V e 1/V al lift-off
c1 = a1; c2 = a1.*V + a4; c3 = a3; c4 = a1.*Vsafe;
c5 = A6./Vsafe; c6 = A6; c7 = K1; invV = 1./Vsafe;

```

- Righe 50-51: i **breakpoints** sono i tempi del dataset stesso, tagliati a `Tstop`. Con i default: 141 punti,  $t = 0, 1, \dots, 140$  s. Non c'e' raffinamento: la griglia della lookup e' la griglia dei dati.
- Riga 52: `at` ricampiona ogni serie su `tg` con `interp1` lineare. Poiche' le serie condividono gia' la stessa base tempi, qui e' di fatto un'identita'.
- Righe 53-58: campionamento di `V`, `A6`, `K1`, `a1`, `a3`, `a4`, `Q`, `h` e – per il modello flessibile (T008) – `omega` (frequenza del primo modo di bending), `sigma_ins`, `phi_ins` (accoppiamento INS-bending) e `aqk` (forzante del TVC sul bending).
- Righe 61-68: le equazioni del plant rigido sono

```

zddot      = a1*zdot + (a1*V + a4)*theta + a3*delta - a1*V*alpha_w
thetaddot = (A6/V)*zdot + A6*theta      + K1*delta - A6*alpha_w

```

e i coefficienti `c1..c7` sono esattamente i **fattori davanti a ciascun segnale**, gia' combinati. Nota che `c6` compare *due volte* nella seconda equazione (davanti a `theta` e davanti a `-alpha_w`): infatti nel modello Simulink un solo lookup `c6` alimenta due Product (P6 e P8).

Perche' non tabulare `a1`, `V`, `a4` separatamente e moltiplicare a runtime? Perche' l'interpolazione non commuta con il prodotto:

`interp(a1*V)(t) != interp(a1)(t) * interp(V)(t)` per t fra due breakpoint

Su questo dataset (breakpoint a 1 s) ho misurato lo scarto:

| quantita'        | max scarto assoluto | scarto relativo |
|------------------|---------------------|-----------------|
| $c2 = a1*V + a4$ | 2.24e-3             | 5.5e-5          |
| $invV = 1/V$     | 1.23e-7             | 1.1e-4          |
| $c5 = A6/V$      | 4.9e-7              | ~4e-4           |
| $\omega^2$       | 6.8e-3              | 1.9e-5          |

Se il modello Simulink usasse  $c2$  (tabella del prodotto) e l'ODE usasse  $fa1(t)*fV(t) + fa4(t)$  (prodotto delle tabelle), i due starebbero integrando **due sistemi LTV diversi**, e il residuo del confronto non potrebbe mai scendere sotto  $\sim 5e-5$  in relativo. Con i coefficienti efficaci, invece, `ode_lpv_ascent` (righe 26-28) chiama `M.fc1..M.fc7`, cioè i **griddedInterpolant** costruiti sulle **stesse identiche tabelle** che finiscono nei blocchi Lookup – e i due sistemi coincidono punto per punto.

**Possibile domanda d'esame** – Perché hai tabulato  $a1*V + a4$  come un unico coefficiente invece di tabulare  $a1$ ,  $V$ ,  $a4$  e fare il prodotto nel modello? *Risposta:* Perché l'interpolazione lineare non commuta con la moltiplicazione: la tabella del prodotto e il prodotto delle tabelle coincidono solo sui breakpoint, e in mezzo differiscono (qui  $\sim 5e-5$  in relativo). Tabulando il coefficiente *efficace* il modello Simulink e il RHS `ode45` vedono la stessa funzione del tempo, quindi integrano lo stesso sistema LTV: il residuo del confronto misura solo l'errore numerico, non una differenza di modello.

**Attenzione – il guard  $V_{safe}$  e' codice morto e il commento e' sbagliato.** La riga 59 dice  $V(0)=0$ : `guard A6/V and 1/V at lift-off`. Ho caricato il `.mat` in repo:  $\min(V) = 410.4$  m/s (a  $t = 7$  s),  $V(0) = 413.0$  m/s. Quindi  $\max(V,1) == V$  identicamente e il guard **non entra mai in funzione**. E' innocuo, ma il commento descrive un dataset diverso da quello presente. (Nota collaterale: i primi campioni del dataset sono comunque strani –  $Q(0) = -16.4$  Pa, negativa, e  $Mach(0) = 0.03$  mentre  $V(0) = 413$  m/s: incoerenze del dato di partenza, non del codice.)

## Controllore congelato di riferimento (righe 70-72)

```
p0 = load_hw3_params(); % t_ref = 72 s
K0 = design_controller(build_plant_rigid(p0), [], 'verbose', false);
```

- Riga 71: `load_hw3_params` senza argomenti ->  $t_{ref} = 72$  s, il punto di max-q su cui e' costruito tutto HM3.
- Riga 72: `Wact = []` -> attuatore ideale (Task 1 di HM3). `K0` e' la coppia PD di pitch *piu'* i guadagni di drift  $Kp_z = Kd_z = -1e-3$  (default di `design_controller`, righe 28-29 di quel file). Questi ultimi **non vengono mai schedati** in nessuno dei due modelli: restano blocchi Gain costanti.

## Gain schedule (righe 74-85)

```
tsched = (o.t0:o.tsched_step:o.Tstop).'; % 5:5:140 -> 28 punti
Kprev = [2.0 1.4]; % first warm start
for i = 1:numel(tsched)
    pk = load_hw3_params('t_ref', tsched(i));
    Kk = design_controller(build_plant_rigid(pk), [], 'K0', Kprev, ...);
    Kp_sched(i) = Kk.Kp_th; Kd_sched(i) = Kk.Kd_th;
    Kprev = [Kk.Kp_th Kk.Kd_th]; % continuation
end
```

- Riga 75: griglia dello schedule, 5 s di passo (default), da  $t_0 = 5$  a 140 s -> 28 punti. Molto piu' rada dei breakpoint del plant (1 s): i guadagni variano lentamente, i coefficienti aerodinamici no.
- Righe 77-78: `margin()` emette un warning su ogni loop condizionatamente stabile; viene silenziato e ripristinato con `onCleanup` (pattern corretto: il warning torna al suo stato anche se la funzione esce per errore).
- Righe 80-85: la logica e' "**design at frozen plants**", la ricetta classica del gain scheduling: si congela il plant a ogni istante `tsched(i)`, si ri-progetta il PD con lo stesso tuner di HM3 (`design_controller`, che ritocca `Kp_th`, `Kd_th` con `fminsearch` finche' GM e PM classificati non colpiscono 6 dB / 30 gradi), e si tabula il risultato contro il tempo.

**Attenzione** – la “**continuation**” (**warm start**) **non esiste**. Le righe 79 e 84 costruiscono `Kprev`, che la riga 82 passa come 'K0', e il commento (e il README della cartella) lo chiamano *continuation*. Ma `design_controller.m` dichiara `o.K0(1,2) ... = [0 0] % accepted, unused` (riga 32) e nell'header `K0 ignored (kept for call compatibility)` (riga 19): **il valore viene ignorato**. Il `fminsearch` riparte ogni volta dal punto analitico di D'Antuono  $x_0 = \log([2 \cdot A_6 / K_1, \sqrt{A_6} / K_1])$  (riga 55 di `design_controller`). Ogni punto dello schedule e' quindi progettato in modo **indipendente**, non a continuazione. Funziona lo stesso (il punto di partenza analitico e' gia' un'ottima stima), ma commento e README sono fuorvianti e vanno corretti.

## Vento: il generatore del professore, una volta sola (righe 87-90 e 141-178)

```
wg      = run_wind_generator(gdir, o.Tstop, drywind, L);
Vwg     = max(interp1(L.V.Time, squeeze(L.V.Data), wg.t), 1);
alphaw  = wg.vw ./ Vwg;                                % angolo d'attacco da vento
```

- Riga 88: simula `strong_wind.slx` sull'intero orizzonte. In HM3 “vero”, invece, `load_wind_profile` ritaglia solo 12 s di vento attorno a `max-q`: qui il vento e' full-ascent perche' anche il plant lo e'.
- Righe 89-90:  $\alpha_w = v_w / V(t)$  – piccolo angolo: la componente di vento normale alla traiettoria divisa per la velocita' relativa produce un angolo d'attacco apparente.

### `run_wind_generator` (righe 141-178)

- Righe 153-154: `load_system + onCleanup(@() close_system('strong_wind', 0))`. Lo 0 significa **chiudi senza salvare**: il modello del professore viene modificato in memoria (si accende il logging sulle porte) ma **mai riscritto su disco**. E' il modo pulito di rispettare il vincolo “non toccare il materiale del corso”.
- Righe 156-160: il logging viene messo sulle **porte sorgente** del sottosistema (`get_param(..., 'PortHandles') -> set_param(ph.Outport(k), 'DataLogging', 'on')`), non su blocchi To Workspace aggiunti. Motivo pratico: aggiungere blocchi al modello altrui e' invasivo, mentre marcare una porta e' una proprieta' del segnale che si perde alla chiusura senza salvataggio.
- Righe 162-167: `Simulink.SimulationInput` con `setVariable` per iniettare `drywind` e `GreensiteLPV` (isolamento: non si sporca il base workspace del chiamante) e `setModelParameter` per `StopTime` e `SignalLogging`.
- Righe 169-177: le due uscite sono la **raffica media**  $v_{wp}$  (profilo di vento medio) e la **turbolenza** (Dryden, schedulata in quota). I log a passo variabile possono contenere **tempi ripetuti** (riga 171: `unique`), quindi si costruisce la griglia unione e si somma:

$$v_w(t) = v_{wp}(t) + \text{turbolenza}(t)$$

Il commento (riga 145) parla di *fixed seeds*: i seed stanno dentro i blocchi del generatore del professore, il codice qui **non li imposta** – la riproducibilita' e' ereditata, non garantita da questo file.

**Possibile domanda d'esame** – Il vento e' generato una volta qui e anche, di nuovo, dentro il modello Simulink (che contiene una copia del generatore). Le due realizzazioni sono identiche? *Risposta:* No, non necessariamente. I seed sono fissi, quindi la *sequenza di rumore* e' la stessa, ma i filtri di Dryden sono a stati continui e vengono integrati con solver e passi diversi nei due modelli (**strong\_wind.slx** standalone contro la copia dentro **hm3\_full\_ascent.slx**, che gira con **MaxStep = 0.02**). Le uscite differiscono quindi di poco. E' esattamente per questo che **run\_full\_ascent\_simulink** **non** usa **S.windfun**, ma ripiega l'**alpha\_w** *loggato dal modello stesso*.

## Struct di ritorno e interpolanti (righe 92-117)

```
gi = @(y) griddedInterpolant(tg, y, 'linear', 'nearest');
S.fc1 = gi(c1); ... S.fc7 = gi(c7);
S.fKp = griddedInterpolant(tsched, Kp_sched, 'linear', 'nearest');
S.windfun = griddedInterpolant(wg.t, alphaw, 'linear', 'nearest');
```

- Riga 101: 'linear' = metodo di **interpolazione**, 'nearest' = metodo di **estrapolazione** (tiene il valore dell'estremo fuori dal supporto). Questa seconda scelta e' quella che permette allo schedule (che parte da **t0 = 5 s**) di essere valutato anche per **t < 5 s**: tiene **Kp\_sched(1)**. Nel modello Simulink lo stesso comportamento e' ottenuto con 'ExtrapMethod', 'Clip' sui lookup dei guadagni (**build\_hm3\_full\_ascent.m**, righe 108-109) – e il commento la' lo dichiara esplicitamente: e' cio' che rende *esatto* l'overlay del caso scheduled anche nei primi 5 s.
- Righe 110-111: per il modello flessibile vengono esposti anche i coefficienti **grezzi** **fa1**, **fa3**, **fa4**, **fa6**, **fK1**, **fomega**, **faqk**, **fsig**, **fphi**. **Qui si rompe la simmetria**: **ode\_lpv\_flex** (righe 19, 38-39) usa questi grezzi e ricostruisce (**a1\*V + a4**) e **A6/V** a runtime, mentre **hm3\_full\_ascent\_flex.slx** continua a usare i lookup **c1..c7** gia' combinati. Fra i breakpoint i due **non sono lo stesso sistema** (vedi la tabella degli scarti sopra). E' la spiegazione piu' probabile del perche' il residuo flessibile dichiarato dal README (5e-7 rad) sia ~5 volte quello rigido (1.1e-7 / 2.2e-7): non e' solo tolleranza del solver, e' anche una piccola incoerenza di modello.
- Righe 112-115: il TVC (**build\_tvc(p0, 3)** = attuatore del 2o ordine per un Pade del 3o ordine sul ritardo di 20 ms -> **5 stati**) viene esportato in **due forme**: **ssdata** (per il RHS ode45, che integra **x\_tvc**) e **tfddata** (per il blocco Transfer Fcn del modello). E' **LTI**, quindi le due realizzazioni hanno lo stesso comportamento ingresso-uscita e la duplicazione e' sicura (vedi la domanda d'esame in fondo).
- Riga 116-117: le costanti del notch profondo di HM3: **zN = 0.002** (numeratore poco smorzato -> notch stretto e profondo), **zD = 0.7** (denominatore ben smorzato), piu' **wn72 = omega(72) = 18.9 rad/s**, la frequenza di bending al punto di design – il riferimento contro cui si misura il detuning.

## Push al base workspace (righe 119-138)

```
base = struct('lpv_t', tg, 'lpv_c1', c1, ..., 'sched', 0, ...
             'drywind', drywind, 'GreensiteLPV', GreensiteLPV, ...);
fn = fieldnames(base);
for i = 1:numel(fn), assignin('base', fn{i}, base.(fn{i})); end
```

- Righe 121-133: **il contratto con i due .slx**. Ogni nome qui e' referenziato letteralmente in un blocco:
  - **lpv\_t** -> BreakpointsForDimension1 di tutte le lookup;
  - **lpv\_c1..lpv\_c7**, **lpv\_invV** -> Table delle lookup dei coefficienti;
  - **Kp\_th0**, **Kd\_th0**, **Kp\_z0**, **Kd\_z0** -> blocchi Constant / Gain (guadagni congelati);
  - **tsched**, **Kp\_sched**, **Kd\_sched** -> breakpoints e tabelle dei lookup dei guadagni;
  - **sched** -> il blocco Constant che pilota i due Switch (0 = frozen, 1 = scheduled);
  - **drywind**, **GreensiteLPV** -> li richiede la copia del generatore;
  - **Tstart**, **Tstop** -> StopTime del modello e' letteralmente la stringa '**Tstop**';

- `lpv_omega`, `lpv_omega2`, `lpv_2zBMw`, `lpv_aqk`, `lpv_sig`, `lpv_phi`, `notch_zN`, `notch_zD`, `tv_num`, `tv_den` -> solo modello flessibile.
- Riga 130: nota che vengono spinte **sia** `lpv_omega` **sia** `lpv_omega2 = omega.^2`. Il modello flessibile usa la tabella `omega2` per il termine di rigidità `-omega^2*eta` e per il notch; `ode_lpv_flex` invece calcola `w = M.fomega(t)` e poi `w^2` (riga 40). Di nuovo: `interp(omega^2) != interp(omega).^2` (scarto relativo misurato  $1.9e-5$ ). `lpv_2zBMw = 2*zBM*omega` e' invece **lineare** in `omega`, quindi tabella e prodotto coincidono esattamente: la discrepanza c'e' solo sui termini quadratici.
- Riga 135: `assignin('base', ...)` in un ciclo. E' l'accoppiamento script-modello piu' fragile che c'e' (variabili globali per nome, nessun controllo a compile-time), ma e' anche il modo standard di parametrizzare un modello Simulink senza Data Dictionary. Conseguenza pratica: **nessuno dei due .slx puo' essere simulato se prima non si e' chiamato `init_simulink_lpv`**.

**Possibile domanda d'esame** – Come si rappresenta un plant *LPV* in Simulink, visto che il blocco State-Space accetta solo matrici costanti? *Risposta:* Qui si scompone la dinamica in blocchi elementari: **Clock -> Lookup Table 1-D (breakpoints = tempo di volo, tabella = coefficiente) -> Product -> Sum -> Integrator**. Ogni coefficiente variabile diventa un segnale generato da una lookup e moltiplicato per lo stato corrispondente da un blocco Product; gli integratori chiudono la catena. Le alternative (il blocco *LPV System*, che interpola un array di `ss`, oppure un *MATLAB Function block* che valuta il RHS) funzionerebbero ma nascondono i coefficienti dentro un oggetto o dentro del codice: la scelta fatta li lascia tutti ispezionabili sul canvas – ed e' anche lo stile del generatore di vento del professore. Il codice non discute esplicitamente le alternative: questa e' la lettura della scelta fatta.

## Possibili domande d'esame

**D: Che cos'e' un plant LPV e perche' HM3 “vero” non ne aveva bisogno?** R: LPV = *Linear Parameter-Varying*: la struttura del modello resta lineare ma le matrici dipendono da un parametro che varia (qui il tempo di volo, che fa da proxy per Mach, quota e pressione dinamica). HM3 congela tutto a `t = 72 s` (max-q) e ottiene un LTI, perche' la traccia chiede un design di punto: e' legittimo perche' max-q e' l'istante peggiore (A6, cioe' l'instabilita' aerodinamica, e' massima li'). Questo showcase toglie il congelamento per vedere quanto lontano si estende quel design.

**D: Su cosa si schedulano i guadagni, e come e' realizzata la schedulazione nel modello?** R: Su **tempo di volo**, con una Lookup Table 1-D per `Kp_theta` e una per `Kd_theta`: breakpoints `tsched = 5:5:140` (28 punti), tabelle `Kp_sched / Kd_sched`, interpolazione **lineare** ed estrapolazione **Clip** sotto i 5 s. Un blocco Switch pilotato dalla costante `sched` sceglie fra il ramo schedulato e la coppia congelata di max-q (`Kp_th0`, `Kd_th0`). I guadagni di drift `Kp_z`, `Kd_z` restano fissi in entrambi i casi. Il file spinge anche `lpv_Q`, per cui una lookup indicizzata sulla pressione dinamica sarebbe possibile – ma i due modelli documentati qui non la usano (lo studio su `q` sta in `main_q_scheduling.m`, e conclude che `q` e' una cattiva variabile di scheduling perche' non e' monotona e genera isteresi).

**D: Come vengono calcolati i guadagni schedulati?** R: Con la ricetta “design at frozen plants”: per ogni `t` della griglia si ricostruisce il plant congelato (`build_plant_rigid(load_hw3_params('t_ref', t))`) e si riesegue il tuner di HM3 (`design_controller`), che cerca con `fminsearch` la coppia (`Kp_th`, `Kd_th`) che porta il gain margin aerodinamico a 6 dB e il phase margin rigido a 30 gradi. Il risultato viene tabulato contro il tempo. Il codice *dice* di usare una continuation (warm start dalla soluzione precedente), ma `design_controller` ignora l'opzione K0: ogni punto riparte dalla formula chiusa di D'Antuono  $Kp = 2 \cdot A6 / K1$ ,  $Kd = \sqrt{A6} / K1$ .

**D: Perche' il TVC viene esportato sia come `ss` sia come `tf`? Non e' una duplicazione pericolosa?** R: E' una duplicazione, ma sicura, **perche' il TVC e' LTI**: due realizzazioni diverse (stato-spazio per `ode45`, forma canonica interna del blocco Transfer Fcn) della stessa funzione di trasferimento hanno lo stesso comportamento ingresso-uscita, quindi lo stato interno puo' benissimo essere diverso. La stessa liberta' **non** vale per il notch variabile: se i coefficienti dipendono dal tempo, realizzazioni diverse della stessa funzione di trasferimento “congelata” danno sistemi LTV *diversi*. Ed e' infatti per questo che sia `ode_lpv_flex` sia il modello Simulink usano la **stessa** forma canonica di controllabilita' per il notch.



**D: Se dovessi fidarti di un solo numero prodotto da questo file, quale guarderesti con sospetto?** R:  $V_{\text{safe}} = \max(V, 1)$  e il commento associato: dichiara di proteggere da  $V(0) = 0$ , ma nel dataset in repo  $V$  non scende mai sotto 410 m/s, quindi il guard non entra mai in funzione e il commento e' semplicemente sbagliato. Non cambia i risultati, ma e' il tipo di commento che porta un lettore a credere che i primi secondi siano trattati in modo speciale, quando non lo sono. Da segnalare anche il warm start della schedule, che il codice descrive ma non esegue.

# HM3/LTV\_FULL\_ASCENT/run\_full\_asce

## Ruolo del file nel progetto

Questo e' il file di **validazione incrociata** del track LPV rigido: simula il modello `hm3_full_ascent.slx` (nelle due varianti *frozen* e *scheduled*) e sovrappone il risultato alla **baseline MATLAB** integrata con `ode45` sul RHS `ode_lpv_ascent`. Produce la figura `figures/fullascent_simulink_vs_script.png` e stampa i residui massimi su `theta`, `z` e `delta`.

La domanda a cui risponde e' semplice ma non banale: *il modello Simulink, costruito da script con lookup table e blocchi elementari, e' davvero lo stesso sistema che integro in MATLAB?* La risposta dichiarata dal README della cartella e' "si", a  $\sim 1e-7$  rad su `theta` ( $1.1e-7$  frozen,  $2.2e-7$  scheduled). Questa pagina spiega **come** si ottiene un numero cosi' stretto e **cosa** lo romperebbe.

Il rapporto gerarchico fra i due track e' esplicito nel codice e nel README: **ode45 e' la fonte di verita'**, Simulink e' la replica. Non e' una scelta scontata (in molti progetti industriali e' il contrario); qui ha senso perche' il design – plant, PD, margini – e' tutto scritto in MATLAB, e il `.slx` serve a dimostrare che la stessa legge di controllo gira in un ambiente di simulazione “di volo”, con il generatore di vento del professore **dentro l'anello**.

Chiamato a mano dall'utente (vedi il *How to run* del README). Dipende da: `init_simulink_lpv` (dati + variabili base), `build_hm3_full_ascent` (autore del `.slx`), `ode_lpv_ascent` (RHS della baseline).

---

## Firma e opzioni (righe 1-26)

```
function out = run_full_ascent_simulink(o)
arguments
    o.rebuild (1,1) logical = false
end
...
if o.rebuild || ~isfile mdlfile
    build_hm3_full_ascent();
end
```

- Riga 1: ritorna `out`, una struct con un campo per variante (`frozen`, `scheduled`), ciascuno con `t` e `err`.
- Righe 24-26: **il `.slx` e' un artefatto derivato**. Se manca, viene ri-generato da `build_hm3_full_ascent` (che *e'* la definizione del modello:  $\sim 150$  righe di `add_block` / `add_line`). Con 'rebuild', `true` lo si rigenera sempre. Nota pero' che il `.slx` e' **comunque committato in repo** – quindi in pratica coesistono la sorgente (lo script) e il binario (il modello); il meccanismo garantisce che il binario sia riproducibile, non che sia sempre aggiornato. Se si modifica `build_hm3_full_ascent.m` e non si ri-esegue con 'rebuild', `true`, si simula un modello vecchio senza accorgersene.

**Possibile domanda d'esame** – Perche' costruire il modello Simulink da script invece che disegnarlo a mano? *Risposta:* Tre motivi concreti. (1) **Riproducibilita'**: il `.slx` e' un binario che il version control non sa diffare; lo script `.m` invece si legge, si diffa e si rivede in code review – lo script *e'* il

modello. (2) **La sorgente resta unica**: i parametri dei blocchi non sono numeri incollati nel canvas ma nomi (`lpv_c1`, `Kp_th0`, `tsched`) risolti dal workspace, quindi il modello eredita automaticamente qualunque cambio di design fatto in MATLAB. (3) **Rigenerabilita'**: se cambia il design (nuovo passo dello schedule, nuovo orizzonte, nuovo notch) basta ri-eseguire il build. Il contrasto e' con il modello a mano di HM3 (`hm3_closed_loop.slx`, costruito seguendo `models/SIMULINK_GUIDE.md`), che va rifatto a mano a ogni cambio.

---

## Setup: dati, tolleranze, modello (righe 28-37)

```
S = init_simulink_lpv();
odeo = odeset('RelTol', 1e-9, 'AbsTol', 1e-11);
load_system mdlfile;
variants = {'frozen', 0; 'scheduled', 1};
```

- Riga 28: `init_simulink_lpv()` con i default  $\rightarrow$  `push = true`: spinge nel base workspace tutte le variabili che il modello risolve per nome, e restituisce `S` (gli interpolanti) per la baseline. Le due strade partono quindi *dagli stessi numeri*: e' la preconditione numero uno dell'accordo a  $1e-7$ . **Costo nascosto**: questa chiamata ri-simula il generatore di vento (`strong_wind.slx`) e **ri-progetta i 28 punti dello schedule** con `fminsearch` – decine di secondi – e il vento cosi' generato (`S.wind`, `S.windfun`) **non viene poi usato** (vedi `pack_model`). Lavoro sprecato: non esiste un'opzione per saltare il generatore.
  - Riga 29: tolleranze della baseline: `RelTol = 1e-9`, `AbsTol = 1e-11`. Molto strette, coerenti con la convenzione della repo per il lavoro di precisione. **Nota importante**: il *modello Simulink* NON usa queste tolleranze – la `set_param` di `build_hm3_full_ascent` (righe 35-37) fissa `Solver = ode45`, `SolverType = Variable-step`, `MaxStep = 0.02`, piu' `StartTime/StopTime` e il logging, ma **non** tocca ne' `RelTol` ne' `AbsTol`: il modello gira quindi con le tolleranze **di default** di Simulink (`RelTol = 1e-3`). I due integratori hanno tolleranze diverse di sei ordini di grandezza: l'accordo a  $1e-7$  **non** viene da tolleranze appaiate (vedi sotto).
  - Riga 32: le due varianti sono solo due valori dello scalare `sched`: lo stesso canvas, un blocco Switch. Non ci sono due modelli.
- 

## Ciclo sulle due varianti (righe 39-70)

### Simulazione e lettura dei log (righe 40-45)

```
assignin('base', 'sched', sc);
so = sim(mdl, 'StopTime', num2str(S.Tstop));
th = so.theta_sl; z = so.z_sl; de = so.delta_sl; aw = so.alpha_w_sl;
tt = th.Time;
```

- Riga 41: la variante si seleziona **scrivendo nel base workspace**. Il blocco Constant `sched` legge quella variabile, il blocco Switch confronta con la soglia 0.5 e passa il ramo schedulato (`u2 >= 0.5`) o quello congelato. Accoppiamento fragile (nessun controllo statico), ma e' il pattern Simulink standard senza Data Dictionary.
- Riga 42: `so` e' un `Simulink.SimulationOutput`; i cinque segnali arrivano dai blocchi To Workspace del modello, salvati in formato `Timeseries` con `SampleTime = -1` e `MaxDataPoints = inf` (`build_hm3_full_ascent`, riga 136): **ogni major step del solver, nessun tetto a 1000 punti**. Il tetto di default avrebbe decimato il log e reso impossibile un confronto a  $1e-7$ .
- Riga 45: `tt` sono i tempi dei passi Simulink. Con `MaxStep = 0.02` su 140 s sono circa 7000 punti, di fatto uniformi. **Fragilita'**: `tt` (cioe' `th.Time`) viene passato tale e quale come `tspan` a `ode45` (riga 49), mentre la griglia del `griddedInterpolant` del vento e' `aw.Time` (`pack_model`, riga 92) – **non** `tt`: sono due log distinti prodotti dagli stessi passi del solver, quindi in pratica gli stessi istanti, ma sono due vettori diversi.

Entrambi gli usi pretendono tempi **strettamente crescenti**, e i log a passo variabile possono ripetere un istante (e infatti `init_simulink_lpv`, righe 171-172, si difende con `unique`) – qui quella difesa **non c’è**, ne’ su `tt` ne’ su `aw.Time`. In pratica non capita, ma è una fragilità reale.

## Replay ode45 sullo stesso vento (righe 47-52)

```
M = pack_model(S, sc, aw);
[~, x] = ode45(@(t, x) ode_lpv_ascent(t, x, M), tt, zeros(4,1), odeo);
if sc, Kp = S.fKp(tt); Kd = S.fKd(tt);
else, Kp = S.K0.Kp_th*ones(size(tt)); Kd = S.K0.Kd_th*ones(size(tt)); end
del_ode = -(Kp.*x(:,3) + Kd.*x(:,4) + S.K0.Kp_z*x(:,1) + S.K0.Kd_z*x(:,2));
```

- Riga 48: `pack_model` (righe 80-93) è il cuore metodologico del file: costruisce la struct per `ode_lpv_ascent` usando **il vento che il modello ha appena prodotto**, non `S.windfun`:  
`'windfun', griddedInterpolant(aw.Time, squeeze(aw.Data), ...)`  
 Questo elimina d’un colpo la piu’ grande sorgente di scostamento. Se si usasse `S.windfun`, si confronterebbero **due realizzazioni diverse dello stesso processo stocastico**: il generatore standalone (simulato dentro `init_simulink_lpv`) e la sua copia dentro `hm3_full_ascent.slx` girano con solver e passi diversi, e i filtri di Dryden sono a stati continui – stesso seed, ma uscite leggermente diverse. In piu’ il modello calcola  $\alpha_w = v_w * \text{interp}(1/V)$  mentre `init_simulink_lpv` (riga 90) calcola  $\alpha_w = v_w / \text{interp}(V)$ : **due funzioni diverse fra i breakpoint** (scarto relativo misurato  $\sim 1.1e-4$ ). Nessuno dei due errori sarebbe compatibile con un residuo di  $1e-7$  rad.
- Riga 49: `ode45` con `tspan = tt` (vettore lungo): **non** forza i passi interni a coincidere con quelli di Simulink – restituisce solo l’uscita *densa* valutata su `tt`. I due integratori seguono percorsi diversi; il fatto che arrivino allo stesso posto è proprio la verifica.
- Righe 50-52: **delta non è uno stato** (attuatore ideale nel modello rigido): va ricostruita algebricamente dagli stati con gli stessi guadagni. Nel caso `scheduled` si usa `S.fKp(tt)`, lo stesso interpolante che alimenta il lookup del modello; sotto `tsched(1) = 5 s` l’interpolante estrapola **‘nearest’** (tiene l’estremo) e il lookup del modello ha `ExtrapMethod = ‘Clip’` (tiene l’estremo): **stessa funzione**. E’ esattamente cio’ che dichiara il commento in `build_hm3_full_ascent`, righe 106-107. Se il lookup avesse l’extrapolazione lineare di default, l’overlay `scheduled` avrebbe un errore visibile nei primi 5 s.
- Nota di coerenza: nel ramo `else` i guadagni di drift restano `S.K0.Kp_z` e `S.K0.Kd_z` **in entrambe le varianti** – lo `schedule` tocca solo il PD di pitch, e cosi’ fa il modello (i blocchi `Gain_Kpz / Gain_Kdz` sono costanti).

## Residui e figura (righe 54-69)

```
err = struct('theta', max(abs(x(:,3) - squeeze(th.Data))), ...
            'z',      max(abs(x(:,1) - squeeze(z.Data))), ...
            'delta', max(abs(del_ode - squeeze(de.Data))));
```

- Righe 54-56: **norma infinito** dell’errore, non RMS: la metrica piu’ severa, giusta per una validazione.
- Righe 62-69: overlay con baseline `ode45` in linea continua e Simulink tratteggiato – la convenzione classica: se il tratteggiato copre il continuo, i due coincidono. Solo il tile di **theta** riporta il residuo nel titolo (riga 64, `\Delta = 1e rad`); i tile **z** e **delta** (righe 67 e 69) hanno il solo nome della variante – **err.z** e **err.delta** finiscono unicamente nella `fprintf` (righe 58-59).

## Perche’ l’accordo è 1e-7 rad – cosa deve essere identico

Un accordo di **1e-7 rad su theta**, con picco di  $\theta \sim 0.95$  gradi =  $1.7e-2$  rad, è un errore **relativo di  $\sim 1e-5$** . Perche’ scenda cosi’ in basso devono valere *tutte* queste condizioni, e ognuna è verificabile sul codice:

1. **Stessi coefficienti fra i breakpoint.** Il modello interpola linearmente le tabelle `lpv_c1..lpv_c7; ode_lpv_ascent` (righe 26-28) chiama `M.fc1..M.fc7`, cioè `griddedInterpolant('linear')` costruiti sulle stesse tabelle e sugli stessi breakpoint (`init_simulink_lpv`, righe 101-103). Interpolazione lineare in entrambi i casi -> **stesso sistema LTV**, non solo sui nodi ma ovunque. E' possibile solo perché `init_simulink_lpv` tabula i *coefficienti efficaci* ( $c2 = a1 \cdot V + a4$  in un'unica tabella): se il modello tabulasse `a1` e `V` separatamente e li moltiplicasse, sarebbe un sistema diverso (scarto relativo  $\sim 5e-5$ , misurato).
2. **Stesso vento.** Garantito per costruzione: la baseline riceve l'`alpha_w` loggato dal modello. Resta un residuo, perché il replay interpola linearmente fra i campioni loggati mentre il generatore, dentro Simulink, è un filtro continuo eccitato da rumore a passo 0.1 s: fra due campioni le due funzioni non coincidono. Con `MaxStep = 0.02` i campioni sono fitti e questo errore è piccolo – l'header del file (righe 3-5) lo dichiara come la sorgente residua.
3. **Stessa famiglia di solver:** `ode45` (Dormand-Prince) da entrambe le parti.
4. **Stesse condizioni iniziali:** tutti gli integratori del modello hanno `InitialCondition = 0`, la baseline parte da `zeros(4,1)`.
5. **Stessa interpolazione dei guadagni**, estrapolazione inclusa (Clip contro 'nearest'): vedi sopra.

**Cosa NON è identico, e va detto:**

- **Le tolleranze del solver.** Il modello gira con i default di Simulink (`RelTol = 1e-3`), la baseline con `1e-9`. L'accordo non viene dalle tolleranze appaiate: viene da **`MaxStep = 0.02`**, imposto nel build (riga 36). Con passo forzato a 0.02 s su una dinamica di scala  $\sim 1$  s, l'errore locale di `ode45` (ordine 5) è dell'ordine di  $h^5 \sim 3e-9$  per passo: il controllo di tolleranza a `1e-3` non morde mai, e l'accuratezza effettiva è fissata dal passo. Il commento nel build (righe 33-34) motiva **`MaxStep`** con la necessità di risolvere le innovazioni di rumore a 0.1 s del generatore – l'accuratezza è un effetto collaterale (fortunato). Se si alzasse **`MaxStep`**, il residuo peggiorerebbe, e non per colpa del modello.
- **I passi.** Simulink cammina a 0.02 s; `ode45` in MATLAB sceglie i suoi passi (grandi, vista la tolleranza) e produce l'uscita su `tt` per **dense output** (interpolante di ordine 4 interno a `ode45`). Le due traiettorie di integrazione sono diverse: coincidono perché entrambe convergono.

**Cosa lo romperebbe (in ordine di gravità):**

- usare `S.windfun` invece dell'`alpha_w` loggato (due realizzazioni diverse del vento + asimmetria `1/interp(V)` contro `interp(1/V)`) -> errore di ordini di grandezza superiore;
- tabulare i coefficienti in modo diverso fra i due track (è *esattamente* ciò che succede nel modello flessibile, vedi `lpv_run_flex_simulink.md`);
- cambiare il metodo di interpolazione di un lookup (es. `Flat / Nearest` invece di `Linear`), o l'`ExtrapMethod` dei guadagni;
- alzare **`MaxStep`** o togliere `MaxDataPoints = inf` dai To Workspace (il log verrebbe decimato e il confronto perderebbe risoluzione);
- introdurre nel modello uno stato che la baseline non ha (per esempio un attuatore reale al posto di quello ideale) senza aggiornare `ode_lpv_ascent`.

**Possibile domanda d'esame** – Il residuo è `1e-7` rad: è "tolleranza del solver" come dice il README? *Risposta:* In parte. Le due integrazioni **non** hanno la stessa tolleranza (Simulink gira ai default `1e-3`, la baseline a `1e-9`): a rendere accurato il lato Simulink è il vincolo **`MaxStep = 0.02`**, non la tolleranza. Il residuo è quindi la somma di due contributi: l'errore di troncamento accumulato del lato Simulink a passo 0.02 s, e l'errore di interpolazione lineare del vento fra i campioni loggati. Dire "tolleranza del solver" è un'abbreviazione accettabile ma non è letteralmente ciò che il codice fa.

## pack\_model (righe 80-93)

```
M = struct('fc1', S.fc1, ..., 'fc7', S.fc7, 'fKp', S.fKp, 'fKd', S.fKd, ...
          'Kp_th0', S.K0.Kp_th, 'Kd_th0', S.K0.Kd_th, ...
          'Kp_z', S.K0.Kp_z, 'Kd_z', S.K0.Kd_z, ...
```

```
'sched', logical(sched), ...
'windfun', griddedInterpolant(aw.Time, squeeze(aw.Data), ...));
```

- Righe 88-92: e' la stessa struct che costruisce `make_model` dentro `main_full_ascent.m` (righe 130-133), con **una sola differenza**: `windfun`. La' e' `S.windfun` (il generatore standalone); qui e' il vento loggato dal modello. La duplicazione della struct in due file e' una piccola violazione DRY, ma e' voluta: separa il "run scientifico" (main) dal "run di validazione" (questo).
- Riga 91: `logical(sched) - ode_lpv_ascent` fa `if M.sched` (riga 17), quindi gli basterebbe lo 0/1 numerico; il cast e' difensivo.

## Possibili domande d'esame

**D: Perche' la baseline ode45 e' la "fonte di verita'" e non il modello Simulink?** R: Perche' tutto il design (`plant build_plant_rigid`, `tuner design_controller`, `margini`, `notch`) e' scritto in MATLAB e il RHS `ode_lpv_ascent` e' la trascrizione diretta delle equazioni; il `.slx` e' invece una *ricostruzione* delle stesse equazioni con blocchi. La direzione della verifica va dalla forma piu' vicina alla matematica a quella piu' vicina all'implementazione. Il valore del modello Simulink non e' l'accuratezza (che e' identica) ma il fatto di poter mettere il **generatore di vento del professore dentro l'anello**, senza ricampionarlo o ritagiarlo.

**D: Il vento della baseline e' quello generato dal modello. Non e' un modo di "barare" sul confronto?** R: No: e' l'unico modo di isolare cio' che si vuole misurare. L'ingresso esogeno (il vento) e' un processo stocastico generato da un modello Simulink a stati continui; due simulazioni con solver diversi danno realizzazioni leggermente diverse anche con lo stesso seed. Se si lasciassero divergere gli ingressi, il residuo misurerebbe la differenza fra i venti, non fra i due modelli di veicolo. Fissando l'ingresso, il residuo misura esattamente cio' che interessa: la fedelta' della trascrizione della dinamica e del controllore. Il prezzo e' che la validazione **non** copre il generatore stesso (quello resta non verificato, ma e' materiale del professore preso as-is).

**D: Il modello ha un solo canvas per due controllori. Come fa?** R: Un blocco Constant legge lo scalare `sched` dal base workspace; due blocchi Switch (`Kp_sw`, `Kd_sw`, criterio `u2 >= 0.5`) selezionano fra i lookup dello schedule (`Kp_s`, `Kd_s`, tabelle `Kp_sched/Kd_sched` su breakpoint `tsched`) e le costanti congelate (`Kp_th0`, `Kd_th0`). Il resto del canvas – `plant`, generatore, drift feedback – e' condiviso. Così le due varianti vedono **esattamente lo stesso vento e lo stesso plant**, e la differenza fra le due risposte e' attribuibile solo al controllore.

**D: Quali sono i limiti onesti di questa validazione?** R: (1) Valida solo il **plant rigido con attuatore ideale**: nessun TVC, nessun bending. (2) Non valida il generatore di vento (usato come ingresso comune). (3) Copre un solo scenario di vento – una sola realizzazione, nessun Monte Carlo. (4) **delta** non e' confrontata come segnale del modello contro segnale del modello, ma contro una ricostruzione algebrica dagli stati della baseline: se il modello sbagliasse *la stessa* formula in modo consistente, il confronto non lo vedrebbe (in realta' non e' così, perche' nel modello **delta** viene dalla somma dei quattro rami di feedback e non da una formula riscritta). (5) Il residuo dipende da `MaxStep = 0.02`, non da tolleranze dichiarate: non c'e' alcuna garanzia formale, e' un accordo empirico.

**D: `run_full_ascent_simulink` chiama `init_simulink_lpv()` a ogni esecuzione. Costo?** R: Alto e in parte inutile: ogni chiamata ri-simula `strong_wind.slx` sull'intero orizzonte e ri-progetta i 28 punti dello schedule con `fminsearch` (ognuno con decine di valutazioni di `margin`). Il vento così generato viene poi **scartato** (il replay usa quello loggato dal modello). Sarebbe naturale un `o.skip_wind` o una cache su disco; il codice non ce l'ha.

# HM3/LTV\_FULL\_ASCENT/run\_flex\_simulink

## Ruolo del file nel progetto

Gemello di `run_full_ascent_simulink.m`, ma per il **veicolo flessibile**: simula `hm3_full_ascent_flex.slx` e sovrappone il risultato alla baseline `ode45` sul RHS `ode_lpv_flex`. Produce `figures/flex_simulink_vs_script.png` e stampa i residui su `theta`, `eta` (coordinata di bending) e `delta`.

Il modello flessibile e' l'architettura completa di HM3 Task 2 sollevata al caso LPV: plant rigido + **primo modo di bending** + **accoppiamento INS** (l'inerziale legge un `theta` contaminato dalla flessione) + **TVC con ritardo di trasporto** (Pade del 3o ordine) + **notch che insegue  $\omega(t)$** . Sono 13 stati:

```
x = [ z   zdot   theta   thetadot   eta   etadot | xn1   xn2   | x_tvc(1:5) ]  
      ----- plant rigido ----- -bending-   -notch-   ---- TVC ----
```

Lo showcase di T008 Goal 1 e': la frequenza del primo modo spazza  $\omega(t)$  in  $[16.5, 31.8]$  rad/s durante l'ascesa, mentre il notch di HM3 e' centrato una volta per tutte su  $\omega(72) = 18.9$  rad/s; il notch fisso quindi si **detuna** e il loop diverge (secondo `main_flex`, instabile da  $t \sim 75$  s), mentre il notch che insegue  $\omega(t)$  regge tutta la salita.

Questo file **non** dimostra quel risultato – lo dimostra `main_flex.m`. Questo file dimostra una cosa piu' modesta e piu' ingegneristica: che il modello Simulink e lo script integrano *lo stesso sistema*. Il README dichiara un accordo di **5e-7 rad su  $\theta$**  (contro  $1.1e-7$  /  $2.2e-7$  del modello rigido); sotto spiego perche' e' cinque volte peggiore, e la ragione **non e' solo numerica**.

Chiamato a mano. Dipende da: `init_simulink_lpv`, `build_hm3_full_ascent_flex`, `ode_lpv_flex`.

---

## Firma, rebuild, setup (righe 1-27)

```
function out = run_flex_simulink(o)  
arguments  
    o.rebuild (1,1) logical = false  
end  
...  
S = init_simulink_lpv();  
nt = size(S.tvc.At, 1);           % ordine del TVC = 5  
odeo = odeset('RelTol', 1e-9, 'AbsTol', 1e-11);
```

- Righe 20-22: stessa politica del modello rigido – il `.slx` e' un artefatto derivato, rigenerabile con `build_hm3_full_ascent_flex()`. Vale la stessa cautela: il `.slx` e' comunque committato, quindi se si modifica il build senza 'rebuild', `true` si simula un modello vecchio.
- Riga 24: `init_simulink_lpv()` con `push = true` -> spinge nel base workspace anche i dati flessibili (`lpv_omega`, `lpv_omega2`, `lpv_2zBMW`, `lpv_aqk`, `lpv_sig`, `lpv_phi`, `notch_zN`, `notch_zD`, `tvc_num`, `tvc_den`), che i blocchi del modello risolvono per nome.



- Riga 25: `nt = size(S.tvc.At, 1)`. Il TVC e' `build_tvc(p0, 3)` = attuatore del 2o ordine ( $w_{TVC} = 70$  rad/s,  $z_{TVC} = 0.7$ ) moltiplicato per un **Pade del 3o ordine** del ritardo  $\tau = 20$  ms -> **5 stati**. Il codice non lo scrive: lo deduce a runtime dalla dimensione di `At`. Buona pratica – cambiare `padeOrder` in `init_simulink_lpv` non rompe questo file.
- Riga 26: tolleranze strette per la baseline. Come nel caso rigido, il **modello Simulink NON le usa**: la `set_param` di `build_hm3_full_ascent_flex` (righe 33-35) fissa `ode45`, `Variable-step`, `MaxStep = 0.02`, piu' `StartTime/StopTime` e il logging, ma **non** tocca `RelTol` ne' `AbsTol` -> il modello gira con le tolleranze di default (`RelTol = 1e-3`). L'accuratezza del lato Simulink e' fissata dal `MaxStep`, non dalla tolleranza.

## Simulazione e replay ode45 (righe 29-43)

```
so = sim mdl, 'StopTime', num2str(S.Tstop));
th = so.theta_sl; et = so.eta_sl; de = so.delta_sl; aw = so.alpha_w_sl;
tt = th.Time;

M = struct('fa1', S.fa1, 'fa3', S.fa3, 'fa4', S.fa4, 'fA6', S.fA6, ...
          'fK1', S.fK1, 'fV', S.fV, 'fomega', S.fomega, ...
          'faqk', S.faqk, 'fsig', S.fsig, 'fphi', S.fphi, ...
          'windfun', griddedInterpolant(aw.Time, squeeze(aw.Data), ...), ...
          'fwn', S.fomega, 'zN', S.notch.zN, 'zD', S.notch.zD, ...
          'zBM', S.notch.zBM, 'At', S.tvc.At, ..., 'sched', false);
[~, x] = ode45(@(t,x) ode_lpv_flex(t,x,M), tt, zeros(6+2*nt,1), odeo);
delta_ode = M.Ct*x(:, 9:end) + M.Dt*0; % delta = Ct*x_tvc (Dt = 0)
```

- Riga 29: nessun `assignin('base','sched', ...)` – il modello flessibile **non ha lo Switch**: i guadagni sono blocchi Gain fissi (`Kp_th0`, `Kd_th0`, `Kp_z0`, `Kd_z0`). Coerente con `'sched', false` alla riga 40. Il messaggio del build e' esplicito: *“PD gains FROZEN at max-q; the showcase is notch tracking, not scheduling”*. **Limite**: il modello flessibile non puo' esercitare lo schedule dei guadagni, e il rigido non puo' esercitare il notch: i due contributi (Goal 1 e Goal 2 del ticket T008) non sono mai combinati – il README lo ammette nei “follow-ups deferred”.
- Riga 36: **stessa mossa chiave del modello rigido**: la baseline riceve l'`alpha_w` loggato dal modello, non `S.windfun`. Senza questo, si confronterebbero due realizzazioni diverse del vento.
- Riga 37: `'fwn', S.fomega` -> il centro del notch e'  $\omega(t)$ , cioe' il **notch variabile**. `main_flex` usa lo stesso RHS passando `@(t) w72` per il caso fisso; qui il caso fisso **non e' validabile**, perche' il `.slx` costruito da `build_hm3_full_ascent_flex` cabla il notch sui lookup di  $\omega(t)$  (righe 125-141 di quel file) e non ha un interruttore. Onestamente: la validazione copre **solo** la configurazione variabile.
- Riga 41: `zeros(6+2*nt, 1)` = 13 stati a zero, coerenti con gli `InitialCondition = 0` di tutti gli integratori del modello (e con la Transfer Fcn del TVC, che parte da stato nullo).
- Riga 42: `M.Dt*0` – `delta` esce dal TVC come  $Ct*x_{tvc} + Dt*v$ , ma qui il termine di feedthrough viene moltiplicato per **zero** invece che per l'ingresso  $v$ . E' corretto *solo perche'* la funzione di trasferimento del TVC e' **strettamente propria**: attuatore del 2o ordine per `Pade(3)` -> numeratore di grado 3, denominatore di grado 5, quindi  $D = 0$ . Il commento lo dichiara ( $Dt=0$ ). Ma e' una scorciatoia: se un giorno il TVC diventasse bi-proprio – per esempio aggiungendo in serie una rete di anticipo, o un feedthrough nell'attuatore – questa riga produrrebbe silenziosamente un `delta` sbagliato, mentre `ode_lpv_flex` (riga 34, che scrive correttamente `M.Ct*x + M.Dt*v`) resterebbe giusta. Meglio sarebbe ricostruire  $v$  e usarlo. Attenzione pero' a non citare `padeOrder` come esempio: `build_tvc` lo vincola a essere positivo (riga 12, `mustBePositive`) e comunque nessun ordine di Pade puo' rendere bi-proprio il TVC – per un Pade di ordine  $n$  il numeratore ha grado  $n$  e il denominatore grado  $n+2$  (i due poli dell'attuatore), quindi  $D = 0$  sempre.



## Il notch variabile: perche' e' realizzato "a mano" (contesto, ode\_lpv\_flex righe 30-33 e build\_hm3\_full\_ascent\_flex righe 125-141)

Entrambi i track realizzano il notch in **forma canonica di controllabilita'**:

```
xn1_dot = xn2
xn2_dot = -omega^2 * xn1 - 2*zD*omega * xn2 + u_pd
v        = u_pd + 2*(zN - zD)*omega * xn2
```

Verifica che sia davvero un notch: da  $xn1\_ddot + 2*zD*omega*xn1\_dot + omega^2*xn1 = u\_pd$  segue  $Xn1(s) = U(s)/(s^2 + 2*zD*omega*s + omega^2)$  e  $xn2 = s*Xn1$ , quindi

$$\begin{aligned} V(s)/U(s) &= 1 + 2*(zN - zD)*omega*s / (s^2 + 2*zD*omega*s + omega^2) \\ &= (s^2 + 2*zN*omega*s + omega^2) / (s^2 + 2*zD*omega*s + omega^2) \end{aligned}$$

che e' esattamente il notch standard: con  $zN = 0.002$  (numeratore quasi non smorzato) si scava un buco profondo a  $omega$ , con  $zD = 0.7$  (denominatore ben smorzato) si evita di introdurre risonanza. Questa e' la **stessa** forma usata nei due track – e questo e' importante:

**Possibile domanda d'esame** – Perche' il notch e' costruito con integratori e prodotti invece di usare un blocco Transfer Fcn con coefficienti variabili? *Risposta:* Perche' il blocco Transfer Fcn di Simulink accetta coefficienti **costanti**: non si puo' far variare  $omega$  a runtime. Serve quindi una realizzazione a stati, con i coefficienti presi da lookup su  $omega(t)$  e moltiplicati per gli stati con blocchi Product. Ma c'e' un punto teorico piu' profondo: quando i coefficienti variano nel tempo, **la funzione di trasferimento non definisce piu' univocamente il sistema** – realizzazioni diverse della stessa TF "congelata" danno sistemi LTV diversi (compaiono i termini in  $d\omega/dt$ ). Sia `ode_lpv_flex` sia il modello usano la *stessa* forma canonica di controllabilita', quindi coincidono fra loro; ma nessuno dei due modella i termini derivativi di  $omega(t)$ : e' una **sostituzione a coefficienti congelati**, un'approssimazione valida perche'  $omega(t)$  varia lentamente rispetto alla banda del notch. Va detto: e' una approssimazione, non un'identita'.

Il TVC, invece, e' **LTI**, e per questo puo' essere un blocco Transfer Fcn nel modello (`tvc_num/tvc_den`) e uno **ss** nella baseline (`At`, `Bt`, `Ct`, `Dt`): due realizzazioni diverse dello stesso sistema tempo-invariante hanno lo stesso comportamento ingresso-uscita, quindi lo stato interno puo' essere diverso senza conseguenze. La liberta' che si ha con il TVC e' esattamente quella che **non** si ha con il notch.

---

## Residui e figura (righe 45-67)

```
err = struct('theta', max(abs(x(:,3) - squeeze(th.Data))), ...
            'eta',   max(abs(x(:,5) - squeeze(et.Data))), ...
            'delta', max(abs(delta_ode - squeeze(de.Data))));
```

- Righe 45-47: si confrontano `theta` (stato 3), `eta` (stato 5, la coordinata modale del bending) e `delta`. **z non viene confrontata**, pur essendo loggata dal modello (`z_sl`): una lacuna gratuita, il dato c'e' gia'. Confrontare `eta` e' la scelta giusta: e' la variabile che diverge quando il notch e' scontrato, quindi e' il test piu' sensibile.
- Righe 56-61: overlay `ode45` continuo / Simulink tratteggiato, tre tile.

## Perche' 5e-7 e non 1e-7: la discrepanza vera fra i due track

Il modello rigido raggiunge  $\sim 1e-7$  rad, il flessibile si ferma a  $\sim 5e-7$ . Il codice non lo spiega; leggendolo, la ragione principale e' **strutturale, non numerica**:

- `hm3_full_ascent_flex.slx` usa gli **stessi lookup di coefficienti efficaci** del modello rigido: `c1..c7`, `invV` (`build_hm3_full_ascent_flex`, righe 45-51 e 75-83). Cioe' tabelle di  $a1*V + a4$ ,  $a1*V$ ,  $A6/V$ ,  $1/V$ .

- `ode_lpv_flex` invece prende i coefficienti **grezzi** (righe 19-20: `fa1`, `fa3`, `fa4`, `fa6`, `fK1`, `fV`) e ricompone i prodotti a **runtime** (righe 38-39):

```
zdd = a1*zdot + (a1*V + a4)*theta + a3*delta - a1*V*aw
thdd = (A6/V)*zdot + A6*theta + K1*delta - A6*aw
```

Ma l'**interpolazione lineare non commuta con prodotto e divisione**: fra due breakpoint (qui distanti 1 s) `interp(a1*V) != interp(a1)*interp(V)`. Misurando sul dataset in repo (massimo scarto relativo, valutato ai punti medi fra i breakpoint sull'orizzonte 0-140 s):

| termine                                               | modello (.slx)                 | baseline ( <code>ode_lpv_flex</code> )                         | scarto relativo |
|-------------------------------------------------------|--------------------------------|----------------------------------------------------------------|-----------------|
| coeff. di <code>theta</code> in <code>zddot</code>    | <code>interp(a1*V + a4)</code> | <code>interp(a1)*interp(V)</code><br>+ <code>interp(a4)</code> | ~3.9e-5         |
| coeff. di <code>zdot</code> in <code>thetaddot</code> | <code>interp(A6/Vsafe)</code>  | <code>interp(A6)/interp(V)</code>                              | ~6.8e-4         |
| rigidezza bending / notch                             | <code>interp(omega^2)</code>   | <code>interp(omega)^2</code>                                   | ~1.9e-5         |

(Nella riga di mezzo c'è anche una seconda differenza, di second'ordine: le tabelle del `.slx` proteggono la divisione con `Vsafe = max(V, 1) - init_simulink_lpv`, righe 59 e 64-68, perché  $V(0) = 0$  – mentre `ode_lpv_flex`, righe 38-39, usa la  $V$  grezza. La discrepanza vive solo nel primo secondo di volo, dove  $V < 1$  m/s e il veicolo non è ancora eccitato.)

I due lati stanno quindi integrando **sistemi LTV leggermente diversi**, e il residuo non può scendere sotto quell'ordine di grandezza. Con `theta` di picco ~1.7e-2 rad, lo scarto relativo di ~4e-5 sul coefficiente di `theta` vale ~7e-7 rad – proprio la scala del residuo osservato. Il modello rigido non ha questo problema perché `ode_lpv_ascent` (righe 26-28) usa `fc1..fc7`, cioè **gli interpolanti delle stesse tabelle** che finiscono nei lookup.

Non è un bug che invalida i risultati (gli scarti relativi stanno fra ~4e-5 e ~7e-4, molto sotto qualunque incertezza fisica del modello), ma **è una incoerenza fra i due track**: il residuo del caso flessibile **non** è spiegabile come “solver tolerance”. Attenzione a citare bene il README: quella formula la usa (a ragione) solo per l'overlay del modello **rigido** (riga 64, “frozen 1.1e-7, scheduled 2.2e-7 – solver tolerance”), mentre per il flessibile (riga 139) si limita a riportare i 5e-7 rad senza attribuirli. La spiegazione strutturale qui sopra è quella che manca. La correzione sarebbe di una riga: far leggere a `ode_lpv_flex` gli stessi `fc1..fc7` (e `fomega2`) invece dei coefficienti grezzi. Nota che  $2*z_{BM}*\omega$  è **lineare** in  $\omega$ , quindi la tabella `lpv_2zBMw` e il prodotto a runtime coincidono esattamente: la discrepanza vive solo nei termini non lineari nei coefficienti.

- Termine di smorzamento del bending:  $-2*z_{BM}*\omega*\dot{\epsilon}$  con  $z_{BM} = 0.005$  (dalla tabella di HM3): il modo è **quasi non smorzato** – ecco perché un notch scentrato lo lascia crescere senza limite invece di vederlo estinguersi.

## Cosa deve essere identico perché l'overlay funzioni (riepilogo)

1. **Stesso vento** -> garantito per costruzione (l'`alpha_w` loggato dal modello entra come `windfun` nella baseline, riga 36). Residuo: interpolazione lineare fra i campioni loggati.
2. **Stessa realizzazione del notch** -> sì, entrambi in forma canonica di controllabilità con centro  $\omega(t)$ . Se uno dei due usasse una realizzazione diversa (es. forma modale, o `tf` congelata ricostruita a ogni passo), le due traiettorie **divergerebbero** anche a solver perfetto.
3. **Stessa realizzazione del TVC** -> non serve: è LTI, basta la stessa funzione di trasferimento.
4. **Stesse condizioni iniziali** -> 13 zeri contro tutti gli integratori a zero.
5. **Stessi coefficienti fra i breakpoint** -> **NO**: è l'unico punto in cui i due track divergono (vedi tabella sopra). È il limite principale di questa validazione.
6. **Stesso solver** -> stessa famiglia (`ode45`), **tolleranze diverse**; l'accuratezza del lato Simulink viene da `MaxStep = 0.02`.

---

## Possibili domande d'esame

**D: Come si realizza un filtro (il notch) con parametri variabili in Simulink?** R: Non con un blocco Transfer Fcn (accetta solo coefficienti costanti), ma con una realizzazione a stati: due integratori in cascata, i coefficienti  $\omega(t)^2$  e  $2*zD*\omega(t)$  presi da lookup table 1-D sul tempo di volo e moltiplicati per gli stati con blocchi Product, e l'uscita  $v = u + 2*(zN - zD)*\omega(t)*x_2$ . Congelando  $\omega$ , la TF ingresso-uscita è  $(s^2 + 2*zN*\omega*s + \omega^2)/(s^2 + 2*zD*\omega*s + \omega^2)$ . È una sostituzione a coefficienti congelati: i termini in  $d\omega/dt$  non sono modellati.

**D: Perché il notch fisso fa divergere il veicolo, mentre il modello rigido con i guadagni congelati resta stabile per tutta l'ascesa?** R: Perché sono due meccanismi diversi. Il PD congelato di max-q resta adeguato perché max-q è l'istante peggiore (l'instabilità aerodinamica A6 è lì massima), quindi il design dimensionato lì è conservativo altrove. Il notch, invece, non è un margine: è una **cancellazione in frequenza**. Se  $\omega(t)$  esce dalla banda stretta del notch ( $zN = 0.002 \rightarrow$  notch molto stretto), il modo di bending non è più attenuato e, essendo quasi non smorzato ( $zBM = 0.005$ ), va instabile per feedback attraverso l'INS (che legge  $\theta_m = \theta + \sigma*\eta$ ) e il TVC (che eccita il modo con  $aq*\delta$ ). Un design "robusto" in ampiezza non protegge da un errore di sintonia in frequenza.

**D: L'accordo del caso flessibile è 5e-7 rad, cinque volte peggiore del rigido. Colpa del solver?** R: Solo in parte. La causa dominante è che i due track **non usano le stesse tabelle di coefficienti**: il .slx usa i coefficienti efficaci già combinati ( $\text{interp}(a1*V + a4)$ ), mentre ode\_lpv\_flex interpola i coefficienti grezzi e li moltiplica a runtime ( $\text{interp}(a1)*\text{interp}(V) + \text{interp}(a4)$ ). Fra un breakpoint e l'altro sono funzioni diverse (scarto relativo ~4e-5 sul coefficiente di  $\theta$ , fino a ~7e-4 su quello di  $\dot{z}$  in  $\dot{\theta}$ ), quindi i due lati integrano sistemi LTV leggermente diversi. Il modello rigido non ha il problema perché lì anche il RHS usa  $fc1..fc7$ .

**D: Cosa NON è validato da questo file?** R: (1) La configurazione a **notch fisso** – il .slx implementa solo il notch variabile, quindi il risultato più spettacolare (la divergenza) è dimostrato solo dallo script, non replicato in Simulink. (2) I **guadagni schedulati** sul plant flessibile: nel modello flessibile il PD è congelato per costruzione. (3) La coordinata  $z$ , loggata ma non confrontata. (4) Il generatore di vento (usato come ingresso comune ai due lati). (5) Un solo scenario di vento, nessun Monte Carlo.

**D: A cosa serve  $nt = \text{size}(S.tvc.At,1)$ ?** R: A dimensionare il vettore di stato della baseline ( $6 + 2 + nt = 13$ ) senza cablare a mano l'ordine del TVC. L'ordine dipende da `padeOrder` (3 in `init_simulink_lpv`, riga 112) e dall'attuatore del 2o ordine: se si cambia l'ordine dell'approssimazione di Pade, questo file continua a funzionare. È anche il motivo per cui `delta_ode` estrae  $x(:, 9:\text{end})$ : gli stati del TVC sono in coda.

# HM0\_\_falcon9\_\_ascent/tests/falcon9AscentTest

## Ruolo del file nel progetto

E' la suite di test di HM0: una classe `matlab.unittest.TestCase` che esegue **entrambi** gli script (`main.m` dimensionale e `main2.m` adimensionale) una sola volta, ne raccoglie i risultati e verifica cinque proprieta'. Si lancia con `runtests('HM0_falcon9_ascent/tests')`.

La sua strategia e' interessante e vale la pena capirla bene, perche' e' condizionata da un vincolo che il codice sotto test le impone: **gli script HM0 non sono funzioni**. Non hanno una firma, non restituiscono nulla, non accettano parametri, e cominciano con `clear` (riga 6 di `main.m`, riga 35 di `main2.m`). Non c'e' quindi nessun modo "pulito" di chiamarli e leggerne l'output: bisogna eseguirli e poi **raccogliere le variabili dal workspace**. Tutto il design della classe discende da qui.

La suite e' quindi un **test di integrazione end-to-end**, non un insieme di unit test. Nessuna delle funzioni interne (`eom`, `eom_nd`, `arc_boundary_events`) e' testabile singolarmente: sono *local function* dentro gli script, quindi non raggiungibili dall'esterno. Il test puo' solo osservare il *risultato finale* della propagazione e chiedersi se e' plausibile.

Il fulcro concettuale e' il **test di cross-validazione** (righe 54-61): due implementazioni indipendenti, con scalature e parametrizzazioni temporali diverse, devono produrre gli stessi numeri. E' l'oracolo piu' forte disponibile in assenza di una soluzione analitica.

---

## Dichiarazione della classe e docstring (righe 1-8)

```
classdef falcon9AscentTest < matlab.unittest.TestCase
    % Note: running the scripts regenerates the PNGs in figures/ (the
    % repository pipeline owns those files, so this is intended).
```

- Riga 1: classe di test class-based (non function-based), coerente con la convenzione della repo (CLAUDE.md).
- Righe 7-8: la docstring avvisa di un **effetto collaterale**: eseguire i test **riscrive i PNG in figures/**. E' un'onestà apprezzabile, ma va precisata: solo `main.m` esporta i PNG (righe 352-369 di quel file); `main2.m` **non ha il blocco di export**. Quindi l'avviso e' corretto solo a meta'. Il fatto che una suite di test *scriva su disco nel repository* e' comunque una scelta discutibile: idealmente un test dovrebbe essere osservativo. Qui e' stato accettato consapevolmente perche' i PNG sono comunque il prodotto della pipeline.

---

## properties (righe 10-13)

```
properties
    dim    % results harvested from main.m
    nd     % results harvested from main2.m
end
```

- Due proprietà che ospitano le due struct di risultati. Sono popolate una volta sola nel `TestClassSetup` e poi lette (mai scritte) dai test.
- I nomi `dim` / `nd` **non sono cosmetici**: coincidono con i valori del `TestParameter impl` alla riga 16, e questo è ciò che permette l'indicizzazione dinamica `testCase.(impl)` alla riga 31. Se si rinominasse una proprietà senza rinominare il parametro, i test parametrizzati fallirebbero con un errore oscuro di “campo inesistente”.

---

## properties (TestParameter) (righe 15-17)

```
properties (TestParameter)
    impl = {'dim', 'nd'}
end
```

- Questo è il meccanismo di **parametrizzazione** di `matlab.unittest`. Ogni metodo di test che dichiara `impl` fra i suoi argomenti viene **eseguito una volta per ciascun valore** del cell array.
- Effetto concreto sul conteggio: quattro metodi parametrizzati (righe 29, 36, 42, 47) x due implementazioni = **8 punti di test**, più il test di cross-validazione non parametrizzato (riga 54) = **9 test totali**. Ciascuno appare separatamente nel report, con nomi tipo `testPropellantBookkeeping(impl=nd)`.
- Il guadagno è che **le stesse quattro proprietà fisiche sono verificate su entrambe le implementazioni senza duplicare una riga di codice**. Se domani arrivasse un `main3.m`, basterebbe aggiungere 'x' al cell array e una proprietà 'x'.

---

## TestClassSetup – runScripts (righe 19-26)

```
methods (TestClassSetup)
    function runScripts(testCase)
        hm0 = fileparts(fileparts(mfilename('fullpath')));
        testCase.dim = runAscentScript(fullfile(hm0, 'main.m'));
        testCase.nd = runAscentScript(fullfile(hm0, 'main2.m'));
        testCase.addTeardown(@() close('all'));
    end
end
```

- Riga 19: `TestClassSetup` (non `TestMethodSetup`) significa **una sola esecuzione per l'intera classe**, non una per test. È una scelta obbligata dal costo: ogni script fa un'integrazione `ode45` con `RelTol = 1e-10` e genera 8-9 figure. Con `TestMethodSetup` il setup – che esegue **entrambi** gli script – girerebbe una volta per ciascuno dei 9 punti di test: ogni script girerebbe **9 volte** invece di 1, cioè **18 esecuzioni** di script invece di 2. Il prezzo di questa scelta è che i test **condividono lo stato**: se un test modificasse `testCase.dim`, contaminerebbe i successivi. Qui non succede (i test sono tutti in sola lettura), ma è una fragilità latente.
- Riga 21: `fileparts(fileparts(mfilename('fullpath')))` risale di **due livelli**: da `.../HMO_falcon9_ascent/tests/falcon9` al file, poi a `tests/`, poi a `HMO_falcon9_ascent/`. Il doppio `fileparts` è il modo idiomatico di ottenere la cartella genitore. Così il test **non dipende dal working directory**: si può lanciare da qualunque parte della repo.
- Righe 22-23: le due chiamate alla funzione locale `runAscentScript` – il cuore del pattern, discusso sotto.
- Riga 24: `addTeardown(@() close('all'))` chiude le figure **dopo che tutti i test della classe sono finiti**. Senza questo, una runtests lascerebbe aperte le **9 finestre di main2.m** (non  $8 + 9 = 17$ : `main2.m` comincia alla riga 35 con `clear; close all; clc;`, quindi il suo `close all` chiude le 8 figure appena create da `main.m` prima di aprire le proprie 9 – lo stesso `close all` di `main.m` è ciò che garantisce che il

suo export loop, basato su `findobj(groot,'Type','figure')` alla riga 359, esporti esattamente 8 PNG). Registrato in `TestClassSetup`, il `teardown` ha durata di classe, coerentemente con il `setup`.

## runAscentScript – il pattern “run + harvest” (righe 65-74)

E’ la funzione piu’ importante del file, ed e’ l’unico modo per testare degli script MATLAB che cominciano con `clear`.

```
function S = runAscentScript(scriptPath)
% The script's leading `clear` wipes this workspace first, then
% the script repopulates it; results are collected after the run.
    run(scriptPath);
    S = struct('t', t, 'h', h, 'mass', mass, 'Vmag', Vmag, ...
              'qdyn', qdyn, 'Mach', Mach, 'qmax', qmax, ...
              'm0', m0, 'Qdot1', Qdot1, 'tb1', tb1, ...
              'im1', im1, 'imQ', imQ);
end
```

## Perche’ non si puo’ chiamare `run()` direttamente in un metodo di test

Questo e’ la domanda d’esame su questo file, e la risposta va capita a fondo.

`run(scriptPath)` non e’ una funzione “normale”: esegue lo script **nel workspace del chiamante**. E’ proprio cio’ che serve – altrimenti le variabili prodotte dallo script sarebbero irraggiungibili – ma ha una conseguenza brutale.

Immaginiamo di scrivere:

```
function testQualcosa(testCase) % NON funziona
    run('main.m'); % <- la riga 6 di main.m fa `clear`
    testCase.verifyGreaterThan(h(end), 50e3); % <- ERRORE
end
```

La prima istruzione eseguita da `main.m` e’ `clear` (riga 6). Quel `clear` viene eseguito **nel workspace del metodo di test**, e cancella **tutto** cio’ che c’e’ dentro – **testCase compreso**. Alla riga successiva, `testCase` non esiste piu’: si ottiene un errore di variabile non definita. Non c’e’ modo di aggirarlo dal metodo: qualunque cosa si salvi in una variabile locale prima della `run` viene spazzata via.

**La soluzione e’ il sacrificio di un workspace.** `runAscentScript` e’ una funzione la cui *unica* variabile locale e’ `scriptPath` – che serve solo a `run` e che, dopo, non serve piu’ a nessuno. Il `clear` dello script distrugge quel workspace usa-e-getta (`scriptPath` incluso!), lo script lo ripopola con le sue variabili, e la riga 70 le raccoglie in una struct che viene **restituita per valore** al chiamante. Il workspace del metodo di test, con dentro `testCase`, non e’ mai stato esposto.

In sintesi: **la funzione locale fa da “camera di decontaminazione”** fra il `clear` dello script e lo stato del test framework. E’ esattamente il pattern che CLAUDE.md documenta come intenzionale per HM0 (“the scripts’ leading `clear` makes assertions on `testCase` impossible inside the same workspace, hence the helper pattern”).

## Il prezzo del pattern

- **Accoppiamento per nome, non per contratto.** La riga 70 pesca 12 variabili (`t`, `h`, `mass`, `Vmag`, `qdyn`, `Mach`, `qmax`, `m0`, `Qdot1`, `tb1`, `im1`, `imQ`) **per nome** dal workspace dello script. Non c’e’ nessuna interfaccia, nessun contratto, nessun controllo statico. Se domani si rinominasse `qmax` in `q_max` dentro `main.m`, il test esploderebbe con un errore di variabile non definita, non con un fallimento leggibile. Gli script sono di fatto **un’API implicita** definita dai nomi delle loro variabili interne.
- **Fragilita’ rispetto ai due script.** Le stesse 12 variabili devono esistere con gli **stessi nomi** in `main.m` e in `main2.m`. Cosa che al momento e’ vera (verificato: `main2.m` produce `t` alla riga 200, `h` alla 212, `mass`

alla 209, Vmag alla 215, qdyn alla 224, Mach alla 225, qmax/imQ alla 250, im1 alla 249, m0 alla 100, Qdot1 alla 67, tb1 alla 64), ma e' un invariante non dichiarato e non verificato.

- **Lentezza ed effetti collaterali.** Ogni chiamata esegue l'integrazione completa, disegna 8-9 figure e (per main.m) scrive 8 PNG su disco. Nessuna di queste cose serve ai test.
- MException / warning degli script si propagano al test come errori di setup.

**L'alternativa vera**, se un giorno si volesse una suite pulita: rifattorizzare la fisica in una funzione ([t, Y] = simulate\_ascent(par)), lasciare agli script solo il ruolo di driver + plotting, e testare la funzione. In quel caso eom diventerebbe testabile isolatamente (per esempio verificando che a velocita' nulla il RHS dia esattamente  $du = -\mu/r^2 + T_{SL}/m0$ ). Il pattern attuale e' una **risposta corretta al vincolo dato**, non un design ideale.

**Possibile domanda d'esame** – Perche' non basta salvare testCase in una variabile temporanea prima di run() per proteggerla dal clear? *Risposta:* Perche' clear senza argomenti cancella **l'intero workspace**, non una lista selettiva: qualunque copia, con qualunque nome, viene distrutta. L'unica difesa e' che il clear non venga mai eseguito **nello stesso workspace** in cui vive testCase. Da qui la funzione locale: si sacrifica un workspace vuoto e monouso, e si passano i risultati indietro **per valore** tramite il return della funzione – che il clear non puo' toccare, perche' avviene prima.

## testPropellantBookkeeping (righe 29-34)

```
function testPropellantBookkeeping(testCase, impl)
    % dm/dt = -Qdot is constant: m(tb) = m0 - Qdot*tb exactly
    S = testCase.(impl);
    testCase.verifyEqual(S.mass(end), S.m0 - S.Qdot1*S.tb1, 'RelTol', 1e-8);
end
```

- Riga 29: la firma dichiara impl come secondo argomento – e' cio' che attiva la parametrizzazione. Riga 31: testCase.(impl) e' indicizzazione dinamica di proprieta' (testCase.dim oppure testCase.nd).
- Righe 32-33: **questo e' l'unico test con un oracolo analitico esatto**, ed e' il piu' potente della suite nonostante l'apparenza banale. L'equazione di massa e'  $dm/dt = -Qdot$  con **Qdot costante** (riga 463 di main.m, riga 574 di main2.m): la portata non dipende ne' dalla quota ne' dallo stato, perche' l'ugello e' in condizioni critiche. La soluzione esatta e' quindi

$$m(t) = m_0 - Qdot * t \quad \Rightarrow \quad m(t_b) = m_0 - Qdot * t_b$$

con  $t_b = 162$  s. Non c'e' nessuna approssimazione: e' la soluzione vera, non una stima.

- Cosa verifica davvero questo test, componente per componente:
  - **Per main.m:** che ode45 propaghi correttamente per 162 s. La tolleranza RelTol 1e-8 e' un controllo diretto sul fatto che le tolleranze dell'integratore (RelTol 1e-10) stiano facendo il loro lavoro. Se qualcuno allentasse RelTol a 1e-4, questo test lo prenderebbe.
  - **Per main2.m:** molto di piu'. Questo test e' l'unico controllo automatico sulla **correttezza dell'intera riparametrizzazione in tau**. Perche'  $m*(t*)$  arrivi al valore giusto devono essere corrette **tutte** queste cose insieme:
    - (a)  $Qdot\_nd = Qdot*T\_ref/m\_ref$  (riga 121); (b) la scalatura chain-rule  $dydt = Delta * [...]$  applicata **anche alla componente di massa** (riga 577);
    - (b) le tre Delta\_k (righe 145-147); (d) la mappa  $tau \rightarrow t*$  dentro eom\_nd (righe 480-496); (e) la mappa inversa in post-processing (righe 195-197); (f) la ri-dimensionalizzazione  $mass = Y\_nd(:,7)*m\_ref$  (riga 209). Sbagliare **una qualunque** di queste fa fallire il test. E' un ottimo "canarino" per la parametrizzazione a tre archi, e vale molto piu' di quanto la sua semplicita' suggerisca.

**Possibile domanda d'esame** – Verificare che  $m(t_b) = m_0 - Qdot*t_b$  sembra tautologico: e' un'equazione lineare, cosa c'e' da testare? *Risposta:* Per main.m e' effettivamente poco piu' di un controllo sull'accuratezza dell'integratore. Per main2.m invece e' il test **piu' informativo** della suite:

e' l'unico oracolo **esatto** disponibile, e per passare richiede che siano simultaneamente corretti l'adimensionalizzazione della portata, la scalatura chain-rule `Delta_k` applicata anche alla componente di massa, le tre durate d'arco, la mappa  $\tau \rightarrow t$  dentro l'EOM, la mappa inversa in post-processing e la ri-dimensionalizzazione finale. Un errore in una sola di queste sei cose lo fa fallire. Testare la componente **piu' semplice** del sistema e' spesso il modo migliore per smascherare errori nella struttura.

---

## testMachOneBeforeMaxQ (righe 36-40)

```
S = testCase.(impl);
testCase.verifyNotEmpty(S.im1);
testCase.verifyLessThan(S.t(S.im1), S.t(S.imQ));
```

- Riga 38: `verifyNotEmpty(S.im1)` non e' pignoleria. `im1` e' prodotto da `find(Mach >= 1, 1, 'first')`, che restituisce `[]` se il veicolo non supera Mach 1. Senza questo controllo, la riga 39 farebbe `S.t([]) ->` array vuoto, e `verifyLessThan([], ...)` **passerebbe silenziosamente** (un confronto su array vuoto e' vacuamente vero). Il test darebbe verde su una simulazione in cui il razzo non decolla. La riga 38 chiude questo buco: e' esattamente il tipo di dettaglio che distingue un test scritto bene da uno scritto in fretta.
  - Riga 39: verifica l'**ordinamento** degli eventi di missione, Mach 1 prima di max-Q. E' un controllo **qualitativo di fisica**, non un confronto numerico: non fissa i valori (61.8 s e 74.9 s nella run nominale), fissa la *relazione*. Questo lo rende **robusto ai cambiamenti di modello**: se domani si introducesse un CD dipendente da Mach, i due istanti cambierebbero ma l'ordine no, e il test continuerebbe a essere valido.
  - **Limite**: e' un test debole. Un bug che spostasse max-Q da 74.9 s a 150 s lo supererebbe tranquillamente. Verifica una condizione necessaria, non sufficiente.
- 

## testAltitudeStaysAboveGround (righe 42-45)

```
testCase.verifyGreaterThanOrEqual(min(S.h), -1e-6);
```

- Riga 44: la quota non deve mai essere negativa (il razzo non scava). La soglia  $-1e-6$  m (un micrometro) e' una **tolleranza di floating point**, non una concessione fisica: a  $t = 0$  vale  $h = r_0 - RE = RE - RE = 0$  esattamente, ma i passi successivi dell'integratore possono produrre, per errore di arrotondamento, valori dell'ordine di  $-1e-9$  m. Senza la tolleranza, il test fallirebbe su un artefatto numerico.
  - E' un **sanity check strutturale**: cattura un errore di segno nella gravita', nella spinta, o nella componente  $du$  – cioe' proprio le classi di bug piu' probabili in un file di EOM. Se  $g_u$  avesse segno sbagliato, o se il termine centrifugo fosse  $-(v^2+w^2)/r$ , il razzo sprofonderebbe e il test lo direbbe subito.
  - **Copre anche un caso limite reale**: il rapporto spinta/peso al decollo e' 1.275, non enorme. Se qualcuno sbagliasse la correzione di contropressione (per esempio **sommando** invece di sottrarre  $p \cdot A_{ex}$ , o dimenticandola del tutto e usando  $T_{vac}$  al livello del mare), il T/W cambierebbe e questo test in generale **non** lo prenderebbe (il razzo decollerebbe comunque, anzi meglio). E' un buco: la correzione di contropressione, che e' il pezzo di propulsione piu' interessante del codice, **non e' testata da nessuno dei cinque test**.
- 

## testFinalAltitudeInPlausibleBand (righe 47-52)

```
% Loose sanity band for the Falcon 9 first-stage MECO altitude
testCase.verifyGreaterThan(S.h(end), 50e3);
testCase.verifyLessThan(S.h(end), 120e3);
```



- Righe 50-51: banda 50-120 km sulla quota a MECO. Il valore nominale (README) e' **82.84 km**, quindi il test ha ~33 km di margine sotto e ~37 km sopra.
- E' un **test di regressione volutamente lasco**. Il commento alla riga 48 lo ammette apertamente ("Loose sanity band"). La banda e' larga abbastanza da sopravvivere a un cambio di modello ragionevole (un CD dipendente da Mach, un profilo di atmosfera diverso) ma stretta abbastanza da catturare un disastro (un errore di unita', una spinta sbagliata di un fattore, un Qdot errato).
- **Limite dichiarato**: non e' un confronto con un valore di riferimento della traccia, e non stringe abbastanza da catturare regressioni sottili. Un bug che spostasse la quota MECO da 82.8 a 95 km passerebbe indisturbato. La banda e' onesta su cio' che e': una rete di sicurezza, non un test di accuratezza.

## testDimensionalVsNonDimensionalAgreement (righe 54-61)

```
% The two scripts integrate the same physics with different
% scalings: results must agree to well below 1%
testCase.verifyEqual(testCase.nd.h(end), testCase.dim.h(end), 'RelTol', 5e-3);
testCase.verifyEqual(testCase.nd.Vmag(end), testCase.dim.Vmag(end), 'RelTol', 5e-3);
testCase.verifyEqual(testCase.nd.mass(end), testCase.dim.mass(end), 'RelTol', 1e-6);
testCase.verifyEqual(testCase.nd.qmax, testCase.dim.qmax, 'RelTol', 1e-2);
```

- Riga 54: l'unico test non parametrizzato – per costruzione, perche' li confronta entrambi.
- E' il test **concettualmente piu' forte** della suite. In assenza di soluzione analitica, l'oracolo migliore e' una **seconda implementazione indipendente**. main.m e main2.m non sono banalmente lo stesso codice: differiscono per scalatura di tutte le variabili, parametrizzazione del tempo (t contro tau), griglia dell'integratore, e persino per come e' scritta la spinta in fase 2 (cos(deg2rad(90)) contro 0 esatto). Che convergano agli stessi numeri e' una verifica sostanziale.

La struttura delle quattro tolleranze e' il punto piu' interessante del file – non sono scelte a caso, e ognuna dice qualcosa:

- **mass(end) a RelTol 1e-6** (riga 59) – la piu' **stretta**. Perche'? Perche' la massa e' l'unica componente con **soluzione esatta** (lineare in t). Entrambi gli script devono arrivare allo stesso  $m_0 - Qdot \cdot t_b$  indipendentemente da come hanno parametrizzato il tempo. Non c'e' errore di modello, solo errore di integrazione, che con RelTol 1e-10 e' minuscolo. Una tolleranza lasca qui sarebbe uno spreco: si perderebbe potere diagnostico.
- **h(end) e Vmag(end) a RelTol 5e-3** (righe 57-58) – 0.5%. Queste sono **quantita' integrate**: dipendono da tutta la storia della traiettoria. I due script hanno **griglie temporali diverse** (main.m ha MaxStep = 1 s e integra in t; main2.m non ha MaxStep e integra in tau con due discontinuita' del RHS ai confini d'arco), quindi accumulano errori di troncamento locale diversi. Inoltre c'e' la differenza reale della componente Tw in fase 2. Lo 0.5% e' la banda che assorbe tutto questo. Su 82.84 km significa ~400 m di tolleranza.
- **qmax a RelTol 1e-2** (riga 60) – la **piu' lasca**, 1%. E la ragione e' importante e sottile: qmax non e' una quantita' integrata, e' un **massimo campionato**. Entrambi gli script lo calcolano con [qmax, imQ] = max(qdyn), cioe' prendendo il valore piu' alto fra i **campioni della griglia di output**, senza interpolare. Il vero massimo cade fra due campioni; il valore riportato e' quindi sistematicamente **sottostimato**, e di quanto dipende **da dove capitano i campioni**. Poiche' i due script hanno griglie diverse (in t contro in tau), i loro campioni cadono in punti diversi attorno al picco e "tagliano" il massimo in modo diverso. La tolleranza dell'1% non e' un errore di fisica: e' l'errore di **quantizzazione della griglia**. Questa e' esattamente la ragione per cui main.m ha MaxStep = 1 s.

**Possibile domanda d'esame** – Perche' la tolleranza su qmax (1%) e' dieci volte piu' lasca di quella su h(end) (0.5%) e diecimila volte piu' lasca di quella su mass(end) (1e-6)? *Risposta*: Perche' le tre quantita' hanno **fonti di errore di natura diversa**. mass(end) ha soluzione analitica esatta: l'unico errore e' quello dell'integratore, minuscolo con RelTol 1e-10. h(end) e Vmag(end) sono integrate su tutta la traiettoria e risentono delle diverse griglie e discontinuita' dei due script: 0.5% e' il margine per l'errore accumulato. qmax invece e' un **massimo campionato senza interpolazione**: il suo valore dipende da *dove cadono i campioni* rispetto al picco, e i due script hanno griglie diverse.

L'1% assorbe l'errore di quantizzazione della griglia, non un errore fisico. Se si volesse stringere, bisognerebbe interpolare il picco (per esempio con una parabola sui tre campioni attorno al massimo) invece di prendere il campione più alto.

---

## Cosa la suite NON verifica (limiti dichiarati)

Vale la pena avere in tasca anche i buchi, perché all'orale la domanda "e cosa non hai testato?" è sempre in agguato.

1. **La correzione di contropressione**  $T = T_{vac} - p \cdot A_{ex}$  non è testata da nessuna parte. E' il pezzo di modellazione propulsiva più interessante del codice e passerebbe indenne anche se avesse il segno sbagliato (il razzo volerebbe *meglio* e la banda 50-120 km probabilmente reggerebbe).
2. **La legge di pitchover** non è testata: nessun controllo che  $\gamma T$  scenda da 90 a 89.5 deg fra  $t = 5$  e  $t = 15$  s.
3. **Il gravity turn** non è testato: nessun controllo che la spinta sia effettivamente allineata a  $V_{rel}$  in fase 3. (E il grafico che sembrerebbe mostrarlo, Figura 7, è una tautologia – vedi `hm0_main.md`.)
4. **eom / eom\_nd non sono unit-testate** e non lo *possono* essere: sono *local function* dentro gli script, non raggiungibili dall'esterno. Non si può verificare, per esempio, che a stato iniziale il RHS restituisca esattamente  $du = 2.719 \text{ m/s}^2$ .
5. **Nessun valore di riferimento della traccia** è verificato:  $\max-Q = 29.5 \text{ kPa}$ , Mach 1 a 61.8 s, quota MECO 82.84 km compaiono nel README ma nessun test li fissa. La suite verifica **relazioni** e **bande**, mai numeri.
6. **La event function inerte di main2.m** (vedi `hm0_main2.md`) non viene smascherata da nessun test – e non potrebbe esserlo, dato che rimuoverla non cambia i risultati.
7. **Nessun matlab.perftest**: `CLAUDE.md` prevede benchmark `<feature>PerformanceTest.m` per ogni homework; HM0 non ne ha.

Questo non significa che la suite sia mal fatta. Fa esattamente ciò che si può fare **dato il vincolo di testare degli script**: verifica invarianti globali, ordinamenti fisici, bande di plausibilità e la cross-validazione fra due implementazioni. Per andare oltre bisognerebbe rifattorizzare la fisica in funzioni pure – che è un cambio di design, non un test in più.

---

## Possibili domande d'esame

**D: Perché i test non chiamano `run('main.m')` direttamente in un metodo, ma passano per una funzione locale?** R: Perché `run()` esegue lo script **nel workspace del chiamante**, e la prima istruzione di `main.m` (riga 6) è `clear`. Chiamato dentro un metodo di test, quel `clear` cancellerebbe l'intero workspace del metodo, **testCase compreso**: la riga successiva darebbe errore di variabile non definita. Nessuna copia difensiva salva, perché `clear` senza argomenti cancella tutto. La soluzione è isolare la `run()` in una funzione locale il cui workspace è **sacrificabile** (contiene solo `scriptPath`): il `clear` distrugge quel workspace, lo script lo ripopola, la funzione raccoglie le variabili in una struct e la **restituisce per valore**. Il workspace del test, con `testCase`, non è mai stato esposto.

**D: Il TestClassSetup esegue gli script una volta sola per tutta la classe. Quali sono i pro e i contro?** R: Pro: costo. Ogni script fa un'integrazione con  $RelTol = 1e-10$  e disegna 8-9 figure; con `TestMethodSetup` il setup girerebbe una volta per ciascuno dei 9 punti di test, quindi ogni script girerebbe **9 volte** invece di 1 (**18 esecuzioni** totali invece di 2), e la suite diventerebbe intollerabilmente lenta. Contro: i test **condividono lo stato**. Se un test modificasse `testCase.dim`, contaminerebbe tutti quelli successivi, e l'ordine di esecuzione diventerebbe significativo – cosa che un test framework non garantisce. Qui il rischio non si concretizza perché tutti i test sono in sola lettura, ma è una fragilità latente. È il classico trade-off velocità contro isolamento, risolto a favore della velocità con una scelta consapevole.

**D:** Il test di accordo fra `main.m` e `main2.m` usa quattro tolleranze diverse (1e-6, 5e-3, 5e-3, 1e-2). Come si giustificano? **R:** Rispecchiano tre meccanismi d'errore diversi. `mass(end)` a 1e-6: la massa ha soluzione analitica esatta (lineare,  $dm/dt = -Qdot$  costante), quindi l'unico errore possibile e' quello dell'integratore, minimo con `RelTol` 1e-10; stringere qui massimizza il potere diagnostico. `h(end)` e `Vmag(end)` a 5e-3: sono quantita' **integrate** su 162 s, e i due script hanno griglie temporali diverse (`main.m` integra in `t` con `MaxStep` = 1; `main2.m` integra in `tau`, senza `MaxStep`, con due discontinuita' del RHS ai confini d'arco), quindi accumulano errori di troncamento diversi. `qmax` a 1e-2: e' un **massimo campionato senza interpolazione**, quindi il suo valore dipende da dove cadono i campioni rispetto al picco – un errore di quantizzazione della griglia, non di fisica. Ogni tolleranza e' calibrata sul meccanismo d'errore che le compete.

**D:** `verifyNotEmpty(S.im1)` alla riga 38 sembra ridondante, visto che alla riga successiva `im1` viene comunque usato. Serve davvero? **R:** Si', ed e' un dettaglio importante. `im1 = find(Mach >= 1, 1, 'first')` restituisce [] se il veicolo non supera mai Mach 1. Senza la riga 38, la riga 39 valuterebbe `S.t([])`, che e' un **array vuoto**, e `verifyLessThan([], [])` **passa** – perche' un'asserzione su un array vuoto e' vacuamente vera. Il test darebbe quindi verde su una simulazione in cui il razzo non decolla nemmeno. La riga 38 trasforma un falso negativo silenzioso in un fallimento esplicito.

**D:** Qual e' il test piu' informativo della suite, e perche'?? **R:** Non e' quello che sembra. `testPropellantBookkeeping` sembra banale – verifica un'equazione lineare – ma applicato a `main2.m` e' il **canarino della riparametrizzazione**: per far tornare  $m(t_b) = m_0 - Qdot * t_b$  devono essere simultaneamente corretti l'adimensionalizzazione della portata, le tre durate d'arco, la scalatura chain-rule `Delta_k` applicata **anche** alla componente di massa, la mappa  $\tau \rightarrow t$  dentro l'EOM, la mappa inversa in post-processing e la ri-dimensionalizzazione finale. Un errore in una sola di queste sei cose lo fa fallire. A pari merito c'e' `testDimensionalVsNonDimensionalAgreement`, che e' l'unico oracolo **indipendente** disponibile in assenza di soluzione analitica: due implementazioni con scalature e parametrizzazioni diverse che convergono agli stessi numeri.

# HM1/tests/odeBurnTest.m

## Ruolo del file nel progetto

E' la **suite di unit test** di HM1/ode\_burn.m, cioe' del cuore numerico di tutto l'homework: la RHS che integra insieme stato e legge di controllo ottima (*linear tangent*) per l'arco propulso. Tutti e quattro i `main_task*.m` chiamano `ode_burn` dentro `ode45`, dentro il residuo di shooting, dentro `fsolve`. Se `ode_burn` ha un errore di segno o una formula sbagliata, **tutti** i risultati di HM1 sono sbagliati, e lo sono in modo silenzioso: `fsolve` converge lo stesso, semplicemente al problema sbagliato.

La classe eredita da `matlab.unittest.TestCase` e contiene **sei test**. Si lancia con `runtests('HM1/tests')` secondo la convenzione del repo. Il commento di intestazione dichiara la copertura: "Covers the linear-tangent steering law, the costate equations, and two analytic limits (ballistic flight, vertical Tsiolkovsky burn). Non-dimensional ( $g = 1$ )" (righe 2-5).

La strategia dei test e' su **due livelli**, ed e' la parte interessante da difendere all'orale:

1. **Test puntuali sulla RHS** (test 1-4): una sola valutazione di `ode_burn`, con parametri scelti in modo che il valore atteso sia calcolabile a mano. Verificano *forma* del vettore, cinematica, legge di steering, equazione del costato di massa.
2. **Test di limite analitico** (test 5-6): si **integra davvero** con `ode45` e si confronta la traiettoria con una soluzione in forma chiusa ottenuta degenerando il problema (spinta nulla -> caduta balistica; spinta verticale -> Tsiolkovsky con perdita gravitazionale). Sono i test con vero potere diagnostico, perche' l'oracolo e' indipendente dal codice.

Per riferimento, la RHS sotto test (`ode_burn.m`, righe 16-33) e':

```
lam_vx = p.lam_vx0
lam_vy = p.lam_vy0 - p.lam_y * t
phi     = atan2(lam_vy, lam_vx)

dz = [ vx ;
       vy ;
       (T/m)*cos(phi) ;
       (T/m)*sin(phi) - 1 ;      (g = 1 nondim)
       -Q ;
       (T/m^2)*|lam_v| ]        (dlam_m/dt)
```

---

## addHm1ToPath – TestClassSetup (righe 7-12)

```
methods (TestClassSetup)
function addHm1ToPath(testCase)
    hm1 = fileparts(fileparts(mfilename('fullpath')));
    testCase.applyFixture( ...
        matlab.unittest.fixtures.PathFixture(hm1));
```

```
end
end
```

- **Riga 9 – risoluzione del percorso.** `mfilename('fullpath')` da' `.../HM1/tests/odeBurnTest`; il primo `fileparts` toglie il nome file (`.../HM1/tests`), il secondo sale di un livello (`.../HM1`). E' un modo **relativo al file, non alla working directory**: la suite funziona ovunque sia lanciata, il che e' precisamente cio' che serve per `runtests('HM1/tests')` eseguito dalla root della repo.
- **Riga 10 – PathFixture.** Aggiunge `HM1/` al path MATLAB *per la durata della classe di test* e lo **rimuove automaticamente** alla fine (teardown garantito, anche se un test lancia un'eccezione). Senza questo, `ode_burn` non sarebbe visibile dalla cartella `tests/`. Usare la fixture invece di un `addpath` nudo evita di inquinare il path dell'utente: e' il motivo per cui il framework fornisce le fixture.
- `TestClassSetup` (non `TestMethodSetup`): gira **una volta sola** per l'intera classe. Corretto – il path non cambia fra un test e l'altro.

## testKinematicsAndMassFlow (righe 15-25)

```
p = struct('T', 1.2, 'Q', 2, 'c', 0.6, ...
          'lam_vx0', 1, 'lam_vy0', 0.5, 'lam_y', 0.2);
z = [0.1; 0.2; 0.3; 0.4; 0.9; 1.1];
dz = ode_burn(0.5, z, p);
testCase.verifySize(dz, [6 1]);
testCase.verifyEqual(dz(1), z(3), 'AbsTol', 1e-15);
testCase.verifyEqual(dz(2), z(4), 'AbsTol', 1e-15);
testCase.verifyEqual(dz(5), -p.Q, 'AbsTol', 1e-15);
```

- **Riga 21 – `verifySize(dz, [6 1])`.** Il test piu' banale e uno dei piu' utili. `ode45` richiede un **vettore colonna**: se `ode_burn` restituisse un `1x6` (riga), `ode45` fallirebbe con un errore criptico sulle dimensioni. Ma soprattutto: le 6 componenti sono `[x; y; vx; vy; m; lam_m]` – se qualcuno aggiungesse o togliesse uno stato, tutti i `main_task*.m` (che costruiscono a mano un vettore iniziale a 6 componenti: `ic = [0;0;0;0;1;1]` in Task 1 e 4, `ic = [p.x0; p.y0; p.vx0; p.vy0; p.m0; 1]` in Task 2, con l'ultima componente lasciata incognita in Task 3) si romperebbero. Il test blocca il **contratto d'interfaccia**.
- **Righe 22-23 – cinematica.**  $dx/dt = vx$ ,  $dy/dt = vy$ : le prime due righe della RHS devono essere **copie esatte** di due componenti dello stato. La tolleranza `1e-15` (non 0) e' formalmente corretta, ma di fatto qui il confronto e' bit-esatto. E' un test *anti-refuso*: protegge contro un `dz(1) = z(4)` da copia-incolla.
- **Riga 24 – portata di massa.**  $dm/dt = -Q$ , **costante**. E' un fatto modellistico non ovvio e vale la pena difenderlo: nel modello di HM1 la portata e' costante (motore a spinta fissa), quindi la massa decresce **linearmente** in `t`, e vale la relazione  $Q = T/c$  (infatti nei dati del test  $T/c = 1.2/0.6 = 2 = Q$ , coerente). La conseguenza numerica e' importante: l'equazione di massa e' esattamente integrabile, e infatti la forma integrata  $m(t) = m0 - Q*t$  viene usata come **check di conservazione** nei test 5-6 di questa suite (righe 73 e 88) e nel performance test (`odeBurnPerformanceTest.m`, riga 51). Qui, invece, si asserisce solo la derivata  $dz(5) = -Q$ , senza integrare.
- **Il punto della scelta dei parametri:** `lam_vx0`, `lam_vy0`, `lam_y` sono tutti **non nulli** (1, 0.5, 0.2) e `t = 0.5` e' non nullo. Quindi il test afferma che queste tre componenti della RHS sono **indipendenti dai costati e dal tempo**: qualunque cosa faccia la legge di steering,  $dx/dt$  resta `vx`. E' un'affermazione strutturale, non un caso particolare.

**Possibile domanda d'esame** – Perche' testare `verifySize` invece di lasciare che sia `ode45` a lamentarsi? *Risposta:* Perche' l'errore di `ode45` arriva in fondo a una catena `fsolve -> residual -> ode45` e dice qualcosa come “the size of the derivative vector must be equal to the size of the initial condition” – non dice *dove* e' il bug. Il test lo localizza in `ode_burn` in un millisecondo e senza dipendere dal solver. Piu' in generale, la dimensione dello stato e' un contratto pubblico di `ode_burn`: tutti

i main\_task\*.m lo assumono quando costruiscono a mano un ic a 6 componenti ([0;0;0;0;1;1] in Task 1 e 4, [p.x0; p.y0; p.vx0; p.vy0; p.m0; 1] in Task 2).

---

## testThrustAlongConstantCostate (righe 27-36)

```
% lam = (1, 0) constant -> phi = 0: horizontal thrust, dvy = -g
p = struct('T', 1.2, 'Q', 2, 'c', 0.6, ...
          'lam_vx0', 1, 'lam_vy0', 0, 'lam_y', 0);
m = 0.8;
z = [0; 0; 0.5; 0.1; m; 1];
dz = ode_burn(0.3, z, p);
testCase.verifyEqual(dz(3), p.T/m, 'AbsTol', 1e-14);
testCase.verifyEqual(dz(4), -1, 'AbsTol', 1e-14);
```

- **Righe 29-30** – la costruzione.  $\text{lam\_vy0} = 0$  e  $\text{lam\_y} = 0$ , quindi  $\text{lam\_vy}(t) = 0 - 0 \cdot t = 0$  per ogni  $t$ . Con  $\text{lam\_vx0} = 1 > 0$ , l'angolo di spinta è

$$\phi = \text{atan2}(0, 1) = 0$$

cioè spinta perfettamente orizzontale.

- **Righe 34-35** – cosa si verifica. Con  $\phi = 0$ :  $\cos(\phi) = 1$ ,  $\sin(\phi) = 0$ , quindi

$$\begin{aligned} dvx/dt &= T/m &&= 1.2/0.8 = 1.5 \\ dvy/dt &= 0 - 1 &&= -1 \quad (\text{sola gravita'}) \end{aligned}$$

Il primo assert isola la **normalizzazione dell'accelerazione di spinta** ( $T/m$ , non  $T$ , non  $T \cdot m$ ): verifica che la massa sia a denominatore. Il secondo isola il **termine gravitazionale**: nel modello nondimensionale  $g = 1$ , hard-coded come  $-1$  in `ode_burn.m` riga 31. Se qualcuno rimettesse  $9.81$  (dimensionale), o cambiasse segno, questo test si accorgerebbe subito.

**Perché è l'invariante giusto:** è il caso in cui la legge di controllo degenera a un valore banale ( $\phi = 0$ ) noto senza calcoli, il che permette di testare *separatamente* i due termini di  $dvy/dt$  – il contributo di spinta (che si annulla) e quello di gravità (che resta). Con un  $\phi$  generico i due termini sarebbero mescolati e il test perderebbe potere diagnostico.

- Il tempo di valutazione è  $t = 0.3$  (non zero) ma con  $\text{lam\_y} = 0$  non ha effetto: è proprio il test successivo a occuparsi della dipendenza dal tempo.
- 

## testLinearTangentSwitch (righe 38-49)

```
% lam_vy(t) = lam_vy0 - lam_y*t vanishes at t* = lam_vy0/lam_y,
% where the thrust must be exactly horizontal (phi = 0)
p = struct('T', 1.2, 'Q', 2, 'c', 0.6, ...
          'lam_vx0', 1, 'lam_vy0', 2, 'lam_y', 4);
tStar = p.lam_vy0 / p.lam_y;
...
dz = ode_burn(tStar, z, p);
testCase.verifyEqual(dz(3), p.T/m, 'AbsTol', 1e-14);
testCase.verifyEqual(dz(4), -1, 'AbsTol', 1e-14);
```

- **Questo è il test più intelligente della suite** e va capito bene, perché isola l'unica formula davvero “teorica” della RHS.

- **Il contesto teorico.** Le equazioni dei costati per il problema planare di HM1 si ottengono da  $\text{lam\_dot} = -dH/dx$ . Siccome  $x$  non compare nella dinamica e  $x(t_f)$  e' libero,  $\text{lam}_x = 0$  identicamente. Siccome  $y$  non compare nella dinamica (Terra piatta,  $g$  costante, no drag),  $\text{lam}_y$  e' **costante**. Da  $\text{lam\_vx\_dot} = -\text{lam}_x = 0$  segue che  $\text{lam\_vx}$  e' **costante**; da  $\text{lam\_vy\_dot} = -\text{lam}_y$  segue che  $\text{lam\_vy}$  e' **lineare nel tempo**:

$$\text{lam\_vy}(t) = \text{lam\_vy0} - \text{lam}_y * t$$

L'angolo ottimo (che massimizza  $\text{lam}_v \cdot \hat{u}$ ) e' allineato al primer vector:

$$\tan(\phi(t)) = \text{lam\_vy}(t)/\text{lam\_vx} = (\text{lam\_vy0} - \text{lam}_y * t)/\text{lam\_vx0}$$

cioe' la **tangente dell'angolo di spinta e' lineare nel tempo**: e' la *linear tangent law*, il risultato classico per ascesa a Terra piatta senza drag. (E' il caso particolare della *bilinear tangent law* quando  $\text{lam}_x = 0$ .)

- **Righe 42-43 – la costruzione del test.**  $\text{lam\_vy0} = 2$ ,  $\text{lam}_y = 4$ , quindi  $\text{lam\_vy}(t)$  si annulla in

$$t^* = \text{lam\_vy0}/\text{lam}_y = 2/4 = 0.5$$

Il test valuta `ode_burn` esattamente in  $t^*$  e verifica che li' la spinta sia orizzontale ( $dv_x = T/m$ ,  $dv_y = -1$ ), esattamente le stesse asserzioni del test precedente.

- **Perche' questo e' l'invariante giusto.** L'assert e' identico a quello di `testThrustAlongConstantCostate`, ma il *modo* di ottenerlo e' opposto: qui  $\text{lam\_vy0} = 2 \neq 0$  e sono la **dipendenza dal tempo** e il valore di  $\text{lam}_y$  a produrre  $\text{lam\_vy}(t^*) = 0$ . Quindi il test fallisce **se e solo se** il termine  $-p.\text{lam}_y * t$  e' sbagliato: se lo si cancellasse, in  $t^*$  si avrebbe  $\text{lam\_vy} = 2$ ,  $\phi = \text{atan2}(2, 1) \approx 63.4$  gradi, e  $dv_x$  scenderebbe da 1.2 a circa 0.54. Se il segno fosse +,  $\text{lam\_vy}(t^*) = 4$  e sarebbe ancora peggio. E' un test **chirurgico** su un singolo termine, costruito sfruttando uno **zero noto** della funzione. Questa e' la tecnica generale: cercare i punti dove il modello degenera a un valore calcolabile a mano.
- Il "switch" nel nome e' un po' fuorviante: **non** e' lo switch di spegnimento motore di Task 3 (funzione di switching  $S = |\text{lam}_v|/m - \text{lam}_m/c$ ). E' semplicemente il passaggio a zero di  $\text{lam\_vy}$ , cioe' l'istante in cui il vettore spinta attraversa l'orizzontale ruotando verso il basso – il tipico *pitch-over* dell'ascesa ottima.

**Possibile domanda d'esame** – Da dove viene la *linear tangent law* e cosa la distrugge? *Risposta:* Viene dal fatto che, con Terra piatta, gravita' costante e nessun drag, la dinamica non dipende ne' da  $x$  ne' da  $y$ . Quindi  $\text{lam}_x = 0$  e  $\text{lam}_y$  costante, da cui  $\text{lam\_vx}$  costante e  $\text{lam\_vy}$  lineare, e  $\tan(\phi) = \text{lam\_vy}/\text{lam\_vx}$  lineare in  $t$ . Basta introdurre il drag (che dipende da  $y$  tramite la densita', e da  $v$ ) o la gravita' inversa-quadrato (che dipende da  $r$ ) perche'  $\text{lam}_y$  smetta di essere costante: le equazioni dei costati si accoppiano allo stato e vanno integrate numericamente insieme ad esso. La legge lineare-tangente e' quindi una **proprieta' del modello semplificato**, non una legge generale – ed e' esattamente il motivo per cui `ode_burn` puo' permettersi di **non** integrare  $\text{lam\_vx}$  e  $\text{lam\_vy}$  (li ricostruisce in forma chiusa da  $p$  e  $t$ ), integrando solo 6 stati invece di 9.

## testCostateMassEquation (righe 51-59)

```
% dlam_m/dt = (T/m^2) * |lam_v|, with |lam_v| = 5 for lam = (3,4)
p = struct('T', 1.2, 'Q', 2, 'c', 0.6, ...
          'lam_vx0', 3, 'lam_vy0', 4, 'lam_y', 0);
m = 0.5;
z = [0; 0; 0; 0; m; 1];
dz = ode_burn(0, z, p);
testCase.verifyEqual(dz(6), p.T/m^2 * 5, 'AbsTol', 1e-12);
```

- **Righe 53-54 – la terna pitagorica.**  $\text{lam\_vx0} = 3$ ,  $\text{lam\_vy0} = 4$  danno  $|\text{lam}_v| = \text{sqrt}(9 + 16) = 5$  **esatto**, senza errori di arrotondamento nel valore atteso. Trucco deliberato: il valore di riferimento ( $1.2/0.25 * 5 = 24$ ) e' rappresentabile esattamente, quindi la tolleranza **1e-12** misura solo l'errore del codice, non quello dell'oracolo.

- **La derivazione dell'equazione.** L'Hamiltoniana del problema di ascesa e'

$$H = \text{lam\_x} \cdot v_x + \text{lam\_y} \cdot v_y + \text{lam\_vx} \cdot (T/m) \cdot \cos(\phi) + \text{lam\_vy} \cdot ((T/m) \cdot \sin(\phi) - 1) - \text{lam\_m} \cdot Q$$

Con  $\phi$  scelto ottimamente, il termine di spinta vale  $(T/m) \cdot (\text{lam\_vx} \cdot \cos + \text{lam\_vy} \cdot \sin) = (T/m) \cdot |\text{lam\_v}|$ . Allora

$$dH/dm = -(T/m^2) \cdot |\text{lam\_v}|$$

e per l'equazione dei costati  $\text{lam\_m\_dot} = -dH/dm$ :

$$\text{lam\_m\_dot} = +(T/m^2) \cdot |\text{lam\_v}|$$

che e' esattamente la riga 33 di `ode_burn.m`. **Segno positivo:**  $\text{lam\_m}$  cresce durante il burn. Interpretazione:  $\text{lam\_m}$  e' il valore marginale di un chilo di massa; man mano che la massa cala,  $T/m$  cresce e la stessa massa residua "vale di piu'". A `tf`, per trasversalita' con costo  $-m(\text{tf})$ , si ha  $\text{lam\_m}(\text{tf}) = 1$ . Nei Task 1, 2 e 4 la normalizzazione del repo pone invece  $\text{lam\_m0} = 1$ , sfruttando l'omogeneita' di grado 1 di  $H$  nei costati; in Task 3 (arco di coast)  $\text{lam\_m0}$  resta un'incognita del BVP e la normalizzazione e' imposta al cutoff,  $\text{lam\_m}(\text{tc}) = 1$  (`main_task3.m`, righe 34 e 39).

- **Onesta' sul potere del test.** Questo assert e' un **mirror della formula**: ricopia in MATLAB la stessa espressione che il codice implementa. Protegge da regressioni (se domani qualcuno scrive  $m^3$  invece di  $m^2$ , il test cade), ma **non verifica la derivazione**: se il segno fosse sbagliato *nella teoria*, test e codice sarebbero sbagliati insieme e il test passerebbe. E' quello che si chiama un *change detector*. I test 5 e 6, che confrontano contro soluzioni analitiche indipendenti, sono gli unici a poter smentire il codice.
- **Nota:**  $\text{lam\_y} = 0$  fa si' che  $|\text{lam\_v}|$  sia costante in  $t$ , e il test valuta comunque in  $t = 0$ . Quindi il test **non** copre l'interazione fra la variazione temporale di  $\text{lam\_vy}$  e  $|\text{lam\_v}|$  – cioe' non verifica che  $|\text{lam\_v}(t)|$  sia ricalcolata ad ogni passo. Un test aggiuntivo con  $\text{lam\_y} \neq 0$  e  $t \neq 0$  chiuderebbe questa lacuna.

## testBallisticLimitMatchesAnalytic (righe 61-74)

```
% T = Q = 0: pure ballistic flight under unit gravity
p = struct('T', 0, 'Q', 0, 'c', 0.6, ...
          'lam_vx0', 1, 'lam_vy0', 1, 'lam_y', 0);
z0 = [0; 0; 0.3; 0.5; 1; 1]; tf = 0.8;
opts = odeset('RelTol', 1e-10, 'AbsTol', 1e-12);
[~, Z] = ode45(@(t,z) ode_burn(t, z, p), [0 tf], z0, opts);
testCase.verifyEqual(Z(end,1), z0(3)*tf, 'AbsTol', 1e-9);
testCase.verifyEqual(Z(end,2), z0(4)*tf - 0.5*tf^2, 'AbsTol', 1e-9);
testCase.verifyEqual(Z(end,3), z0(3), 'AbsTol', 1e-10);
testCase.verifyEqual(Z(end,4), z0(4) - tf, 'AbsTol', 1e-9);
testCase.verifyEqual(Z(end,5), z0(5), 'AbsTol', 1e-12);
```

- **Il primo test che integra davvero.** Fin qui si e' valutata la RHS in un punto; qui si chiama `ode45` sull'intervallo  $[0, 0.8]$  con le tolleranze vere del repo (`RelTol = 1e-10`, `AbsTol = 1e-12`, riga 67). Quindi si sta testando anche il cablaggio `@(t,z) ode_burn(t,z,p)`, cioe' esattamente il modo in cui i `main_task*.m` lo usano.
- **Righe 63-64 – la degenerazione.** Ponendo  $T = 0$  e  $Q = 0$  la RHS collassa a  
 $dx/dt = vx, \quad dy/dt = vy, \quad dvx/dt = 0, \quad dvy/dt = -1, \quad dm/dt = 0$   
cioe' **moto parabolico sotto gravita' unitaria**. Nota: con  $T = 0$  anche  $d\text{lam\_m}/dt = 0$ , ma  $\text{lam\_m}$  non viene asserito.
- **Righe 69-73 – le cinque asserzioni** sono la soluzione esatta del moto balistico, verificabile a mano:



|          |                       |                |                  |
|----------|-----------------------|----------------|------------------|
| $x(tf)$  | $= vx0*tf$            | $= 0.3 * 0.8$  | $= 0.24$         |
| $y(tf)$  | $= vy0*tf - 0.5*tf^2$ | $= 0.4 - 0.32$ | $= 0.08$         |
| $vx(tf)$ | $= vx0$               | $= 0.3$        | (costante)       |
| $vy(tf)$ | $= vy0 - tf$          | $= 0.5 - 0.8$  | $= -0.3$         |
| $m(tf)$  | $= m0$                | $= 1$          | (nessun consumo) |

Notare che  $vy(tf) < 0$ : il proiettile e' gia' nella fase discendente. Non e' un problema, e' solo aritmetica – ma dimostra che il test non e' stato scelto per far “sembrare” fisica la traiettoria, e' scelto per avere numeri esatti.

- **Perche' e' l'invariante giusto.** E' il **check di consistenza gravitazionale e cinematica** in un caso dove la spinta non c'e': isola il termine  $-1$  nella riga di  $dvy/dt$  e la doppia integrazione  $y \rightarrow vy \rightarrow -g$ . Un errore di fattore 2 sulla gravita', o un  $+1$  invece di  $-1$ , produce un errore di ordine 0.1 su  $y(tf)$ , enormemente sopra la tolleranza  $1e-9$ .
- **La scelta delle tolleranze di asserzione:**  $1e-9$  sugli stati integrati,  $1e-10$  su  $vx$  (che non e' integrato, e' costante),  $1e-12$  su  $m$  (idem). Sono coerenti col fatto che `ode45` a  $RelTol = 1e-10$  accumula un errore di troncamento di quell'ordine. Le tolleranze **non** sono uniformi: chi le ha scritte ha ragionato su quanto errore ciascuna quantita' puo' accumulare. E' esattamente cio' che va fatto – mettere  $1e-12$  su  $y(tf)$  renderebbe il test fragile senza guadagno diagnostico.

**Possibile domanda d'esame** – Se  $T = 0$  la traiettoria non e' piu' ottima: che senso ha testare un caso non ottimo? *Risposta:* Il test non verifica l'ottimalita', verifica la **dinamica**. `ode_burn` implementa due cose insieme: la dinamica del veicolo e la legge di controllo ottima. Spegnendo la spinta si mette a nudo la sola dinamica, che ha soluzione chiusa e puo' quindi essere validata contro un oracolo esatto. La legge di controllo e' validata separatamente dai test 2-3. E' la strategia corretta: **isolare un sottosistema alla volta**, invece di testare tutto insieme dove nessun oracolo e' disponibile.

## testVerticalBurnTsiolkovskyWithGravity (righe 76-89)

```
% lam = (0, 1) constant -> phi = 90 deg: vertical burn.
% Analytic: vy(t) = (T/Q)*ln(m0/(m0 - Q*t)) - t with m0 = 1
p = struct('T', 1.2, 'Q', 2, 'c', 0.6, ...
    'lam_vx0', 0, 'lam_vy0', 1, 'lam_y', 0);
z0 = [0; 0; 0; 0; 1; 1]; tf = 0.3;
opts = odeset('RelTol', 1e-12, 'AbsTol', 1e-14);
[~, Z] = ode45(@(t,z) ode_burn(t, z, p), [0 tf], z0, opts);
vyAnalytic = (p.T/p.Q) * log(1/(1 - p.Q*tf)) - tf;
testCase.verifyEqual(Z(end,4), vyAnalytic, 'AbsTol', 1e-9);
testCase.verifyEqual(Z(end,3), 0, 'AbsTol', 1e-12);
testCase.verifyEqual(Z(end,5), 1 - p.Q*tf, 'AbsTol', 1e-12);
```

- **Il test piu' forte della suite.** E' l'unico che valida la RHS **completa** (spinta accesa + consumo di massa + gravita') contro un oracolo analitico indipendente.
- **Righe 79-80 – la degenerazione.**  $lam\_vx0 = 0, lam\_vy0 = 1, lam\_y = 0$  danno  $phi = atan2(1, 0) = pi/2$ : **spinta puramente verticale e costante**. La RHS collassa a

$$\begin{aligned}
 dvx/dt &= (T/m)*\cos(pi/2) = 0 \\
 dvy/dt &= (T/m)*\sin(pi/2) - 1 = T/m - 1 \\
 dm/dt &= -Q \quad \rightarrow \quad m(t) = m0 - Q*t = 1 - Q*t
 \end{aligned}$$

- **La derivazione dell'equazione di Tsiolkovsky con gravita'.** Sostituendo  $m(t) = 1 - Q*t$  in  $dvy/dt$ :

$$dvy/dt = T/(1 - Q*t) - 1$$

Integrando da 0 a  $t$  con  $vy(0) = 0$ :

$$vy(t) = (T/Q) * \ln(1/(1 - Q*t)) - t$$

Il primo termine e' il **delta-v ideale di Tsiolkovsky** – infatti  $T/Q = c$  (velocita' efficace di scarico; nei dati del test  $1.2/2 = 0.6 = p.c.$ , coerente) e  $1/(1 - Q*t) = m_0/m(t)$ , quindi il termine e'  $c * \ln(m_0/m)$ . Il secondo termine,  $-t$ , e' la **perdita gravitazionale** ( $g*t$  con  $g = 1$ ): e' letteralmente quanto delta-v la gravita' si mangia durante un burn verticale di durata  $t$ .

Riga 85:  $(p.T/p.Q) * \log(1/(1 - p.Q*t_f)) - t_f$  – corrisponde esattamente.

- **Riga 88 – la conservazione della massa.**  $m(t_f) = 1 - Q*t_f = 1 - 0.6 = 0.4$ . Positivo, quindi il burn e' fisicamente ammissibile (non si esaurisce la massa prima di  $t_f$ ). Con  $Q = 2$ ,  $t_f$  non puo' superare 0.5 – un vincolo implicito del test che vale la pena conoscere:  $t_f = 0.3$  e' scelto con margine.
- **Riga 83 – tolleranze piu' strette.** Qui  $RelTol = 1e-12$ ,  $AbsTol = 1e-14$ , contro  $1e-10 / 1e-12$  del test balistico. Motivo:  $1/(1 - Q*t)$  diverge quando  $Q*t \rightarrow 1$ , quindi la RHS e' molto piu' “ripida” e l'errore locale dell'integratore cresce; per poter asserire  $1e-9$  su  $vy(t_f)$  serve un margine numerico maggiore. E' una scelta consapevole, non un vezzo.
- **Perche' e' l'invariante giusto:** verifica **contemporaneamente** che (a)  $\phi$  sia calcolato con  $\text{atan2}$  nel verso giusto (un  $\text{atan2}(lam\_vx, lam\_vy)$  invertito darebbe  $\phi = 0$  e spinta orizzontale, e il test esploderebbe), (b) la massa entri a denominatore come  $T/m$  e non altrimenti, (c)  $dm/dt = -Q$ , (d)  $g = 1$  sia sottratta. Se **una qualsiasi** di queste e' sbagliata,  $vy(t_f)$  non torna. E' l'unico test che accoppia tutti i pezzi.

**Possibile domanda d'esame** – Il test si chiama “Tsiolkovsky with gravity”: perche' non basta l'equazione classica  $dv = c * \ln(m_0/m_f)$ ? *Risposta:* Perche' Tsiolkovsky vale in assenza di forze esterne. In salita verticale la gravita' agisce per tutta la durata del burn e sottrae  $g*t_b$  al delta-v ottenuto, con  $t_b$  durata del burn: questa e' la *gravity loss*. Il test ne fa un oracolo esatto perche' in salita verticale (e senza drag) la perdita e' integrabile in forma chiusa:  $-g*t$ , cioe'  $-t$  in nondimensionale. E' anche il motivo fisico per cui l'ascesa ottima **non** e' verticale se non per un breve tratto iniziale: minimizzare la gravity loss spinge a ruotare presto verso l'orizzontale, ed e' esattamente cio' che la linear-tangent law fa.

## Lacune di copertura (onesta' richiesta)

Cose che la suite **non** verifica, e che vale la pena saper elencare all'orale:

- **Conservazione dell'Hamiltoniana.** Il sistema aumentato e' **autonomo** (la dipendenza esplicita da  $t$  in  $ode\_burn$  e' solo la ricostruzione in forma chiusa di  $lam\_vy(t)$ , non una vera non autonomia), quindi lungo la traiettoria ottima  $H$  deve essere **costante**, e per il caso a tempo finale libero deve valere  $H = 0$ . Ricostruendo i costati da  $p$  e  $t$  si potrebbe calcolare  $H$  a ogni punto della soluzione di  $ode45$  e asserire  $\max|H(t) - H(0)| < tol$ . **Questo test non esiste nella suite.** Sarebbe l'invariante piu' potente disponibile, e coprirebbe simultaneamente steering law, costati e dinamica.
- **Confronto derivata analitica vs differenze finite.** Nessun test confronta  $ode\_burn$  con una derivata numerica dello stato  $((z(t+h) - z(t))/h)$ . Sarebbe ridondante rispetto ai test 5-6, ma non e' presente.
- **Casi limite / robustezza numerica.** Non c'e' nessun test su  $m \rightarrow 0$  (dove  $T/m$  e  $T/m^2$  divergono), ne' su  $lam\_vx = lam\_vy = 0$  (dove  $\text{atan2}(0,0) = 0$  per convenzione MATLAB, ma il primer vector e' indefinito e la legge di controllo perde senso), ne' su input di dimensione sbagliata (che con la scelta di **non** avere un blocco `arguments` produrrebbero errori oscuri).
- **Test 1-4 sono change detector:** ricopiano le formule del codice. Solo i test 5-6 hanno un oracolo indipendente.

## Possibili domande d'esame

**D: Che cosa rende un test “buono” per una RHS di ODE, e come si vede in questa suite?** R: Un test buono confronta il codice con un **oracolo indipendente**, non con se stesso. Nella suite ci sono due famiglie: i test 1-4 valutano la RHS in un punto e confrontano con la formula ricopiata a mano (proteggono da regressioni, ma se la teoria fosse sbagliata passerebbero comunque); i test 5-6 **degenerano** il problema a un caso con soluzione chiusa (balistica, Tsiolkovsky) e integrano davvero con `ode45`. Solo questi ultimi possono smentire il codice. La tecnica generale è proprio quella: **spegnere termini** ( $T = 0$ ,  $Q = 0$ ) o **allineare i costati** ( $\lambda_m = (1,0)$ ,  $\lambda_m = (0,1)$ ) per far collassare il modello su qualcosa che si sa risolvere a mano.

**D: Perché `testLinearTangentSwitch` valuta la RHS proprio in  $t^* = \lambda_{m\_vy0}/\lambda_{m\_y}$ ?** R: Perché lì  $\lambda_{m\_vy}(t^*) = 0$  per costruzione, quindi  $\phi = \text{atan2}(0, \lambda_{m\_vx0}) = 0$  e la spinta è esattamente orizzontale: l'atteso è calcolabile a mano ( $dv_x = T/m$ ,  $dv_y = -1$ ). È il modo per testare **chirurgicamente** il termine  $-\lambda_{m\_y} * t$  – l'unica parte della RHS che dipende dal tempo. Se quel termine sparisse o cambiasse segno, in  $t^*$  la spinta non sarebbe più orizzontale e il test cadrebbe con un errore grosso. Testare in un  $t$  generico non permetterebbe di scrivere l'atteso senza rifare gli stessi conti del codice.

**D: Perché  $\lambda_{m\_vx0} = 3$ ,  $\lambda_{m\_vy0} = 4$  e non due numeri qualsiasi?** R: Terna pitagorica:  $|\lambda_{m\_v}| = 5$  esatto. Il valore atteso del test ( $T/m^2 * 5$ ) è quindi rappresentabile esattamente in floating point, e la tolleranza  $1e-12$  misura solo l'errore del codice sotto test, non quello dell'oracolo. È una piccola accortezza che rende il test più pulito da leggere e più stretto da asserire.

**D: L'equazione  $d\lambda_{m\_m}/dt = +(T/m^2)*|\lambda_{m\_v}|$  ha segno positivo. È corretto? Da dove viene?** R: Sì. Viene da  $\lambda_{m\_m\_dot} = -dH/dm$  con  $H = \dots + (T/m)*|\lambda_{m\_v}| - \lambda_{m\_m}*Q$  (avendo già sostituito il  $\phi$  ottimo, che rende il prodotto  $\lambda_{m\_v} \cdot \hat{u}$  pari a  $|\lambda_{m\_v}|$ ). Derivando,  $dH/dm = -(T/m^2)*|\lambda_{m\_v}|$ , e il segno meno dell'equazione dei costati lo ribalta in +. Quindi  $\lambda_{m\_m}$  cresce durante il burn. Interpretazione fisica:  $\lambda_{m\_m}$  è il valore marginale della massa; man mano che il veicolo si alleggerisce l'accelerazione  $T/m$  cresce, e ogni chilo residuo diventa più prezioso. La condizione di trasversalità del problema a massa finale massima da  $\lambda_{m\_m}(tf) = 1$ , e nei Task 1, 2 e 4 HM1 sfrutta l'omogeneità di grado 1 di  $H$  nei costati per normalizzare invece  $\lambda_{m\_m0} = 1$ . In Task 3  $\lambda_{m\_m0}$  è una delle incognite del BVP e la normalizzazione  $\lambda_{m\_m} = 1$  è imposta al cutoff ( $\lambda_{m\_m}(tc) = 1$ , dove  $\lambda_{m\_m}$  è costante durante il coast).

**D: Come testeresti che la traiettoria trovata è davvero ottima, non solo che l'integrazione è corretta?** R: Con l'**invariante Hamiltoniano**, che oggi manca nella suite. Il sistema aumentato è autonomo, quindi lungo una traiettoria che soddisfa le condizioni di Pontryagin  $H$  deve essere costante; e siccome in HM1 il tempo finale è libero, la condizione di trasversalità impone  $H = 0$  per tutto l'arco. Ricostruendo  $\lambda_{m\_vx}$ ,  $\lambda_{m\_vy}(t)$  e  $\lambda_{m\_y}$  da  $p$ , e prendendo  $\lambda_{m\_m}$  dalla sesta componente dello stato integrato, si può valutare  $H$  su tutta la griglia di `ode45` e asserire  $\max |H(t)| < 1e-8$  sulla soluzione convergente di `fsolve`. Sarebbe l'unico test che verifica l'**ottimalità**, non solo la dinamica – tutti gli altri passerebbero anche con costati sbagliati, purché la RHS li propaghi coerentemente.

**D: Perché `PathFixture` invece di un semplice `addpath`?** R: Perché la fixture garantisce il **teardown**: MATLAB rimuove HM1/ dal path alla fine della classe di test anche se un test lancia un'eccezione. Un `addpath` nudo lascerebbe il path dell'utente sporco, e in una suite più grande potrebbe far “funzionare” test che invece dovrebbero fallire per file non trovati (contaminazione fra test). È la stessa filosofia del RAI: l'acquisizione della risorsa è legata a uno scope.

# HM1/tests/odeBurnPerformanceTest.m

## Ruolo del file nel progetto

E' il **benchmark** di HM1/ode\_burn.m. Non verifica la correttezza (quello lo fa odeBurnTest.m): **misura il tempo**. Esiste per una ragione precisa e molto concreta, dichiarata nel commento di intestazione: ode\_burn e' l'*hot loop* di HM1, "called ~1e6+ times by the fsolve sweeps of main\_task1..4" (righe 3-4).

Il file e' l'evidenza sperimentale a supporto di una **decisione di design** controintuitiva presa in ode\_burn.m e codificata nel CLAUDE.md del repo: la funzione **non ha un blocco arguments di validazione**. Il commento in ode\_burn.m (righe 13-14) lo dice esplicitamente:

```
% No arguments block by design: called ~1e6+ times inside ode45/fsolve;  
% validate at the call site.
```

La convenzione del repo e' che la validazione stia nei **boundary helper** (le funzioni chiamate una volta per run: nondim, solve\_\*, plotting), non nelle funzioni chiamate un milione di volte. Questo benchmark serve a poter **difendere quella scelta con dei numeri** invece che con un'opinione.

La classe eredita da matlab.perftest.TestCase – che a sua volta eredita da matlab.unittest.TestCase. Conseguenza pratica: i suoi metodi sono **sia** benchmark (con runperf) **sia** test veri (con runtests), perche' contengono delle **verify\***. Si lancia con:

```
results = runperf('odeBurnPerformanceTest')
```

come indicato alla riga 6.

Misura **due unita' diverse**: 1. testRhsEvaluation – il costo di **una singola valutazione** della RHS: la grandezza microscopica, quella che viene moltiplicata per ~1e6. 2. testBurnArcIntegration – il costo di **un'intera integrazione** dell'arco propulso con ode45, parametrizzato su tolleranza lasca vs stretta: la grandezza macroscopica, quella che l'utente percepisce come "quanto ci mette main\_task1".

---

## Perche' ode\_burn e' un hot loop – il conto degli ordini di grandezza

Vale la pena ricostruire la stima 1e6+ invece di ripeterla a memoria, perche' e' esattamente la domanda che ti fanno all'orale. (Sono stime di ordine di grandezza, non misure: il codice non le verifica.)

La catena di chiamata di main\_task1.m e':

```
sweep su Q e yf (continuation)  
-> fsolve (per ogni punto dello sweep)  
    -> residual (per ogni iterazione + per ogni colonna del Jacobiano)  
        -> ode45 (una integrazione completa dell'arco)  
            -> ode_burn (per ogni stage di ogni passo)
```

Moltiplicando:

- **ode45 -> ode\_burn**: Dormand-Prince e' un Runge-Kutta a 7 stadi con proprieta' FSAL, quindi ~6 valutazioni della RHS per passo **accettato**, piu' quelle dei passi rifiutati. A RelTol = 1e-10 (la tolleranza

del repo) i passi sono molti: ordine delle centinaia/migliaia per arco. Diciamo  $\sim 1e3$ - $1e4$  valutazioni per integrazione.

- **residual -> ode45**: una integrazione completa per ogni valutazione del residuo.
- **fsolve -> residual**: fsolve non ha il Jacobiano analitico, quindi lo approssima per **differenze finite**: con 4 incognite servono  $4 + 1 = 5$  valutazioni del residuo per ogni Jacobiano, piu' quelle della line search / trust region. Con  $\sim 10$ - $30$  iterazioni per solve, siamo sull'ordine di  $1e2$  integrazioni per solve.
- **sweep -> fsolve**: main\_task1 fa continuation su Q (in avanti e all'indietro dal punto di partenza) per **tre** valori di yf: la griglia e' linspace(1.8, 7, 80), quindi fino a  $80 \times 3 = 240$  solve – centinaia, non decine.

Il prodotto –  $1e3$  valutazioni/integrazione  $\times 1e2$  integrazioni/solve  $\times$  centinaia di solve – arriva comodamente a  **$1e7$  chiamate di ode\_burn e oltre** in un singolo run di main\_task1 (la stima  $\sim 1e6+$  del commento e' quindi conservativa). Ecco perche' il **costo unitario** di quella funzione conta davvero, ed ecco perche' esiste questo file.

**Possibile domanda d'esame** – Perche' non fornire il Jacobiano analitico a fsolve invece di ottimizzare la RHS? *Risposta*: Sarebbe la mossa piu' efficace in assoluto: eliminerebbe il fattore  $n+1 = 5$  delle differenze finite e migliorerebbe anche la robustezza della convergenza. Ma il Jacobiano del residuo di shooting richiede di propagare le **equazioni variazionali** (la matrice di transizione di stato) insieme alla traiettoria – cioe' integrare  $6 + 6 \times 4 = 30$  stati invece di 6. HM1 non lo fa: e' una semplificazione consapevole, che scarica il costo sul numero di integrazioni. Il benchmark documenta il prezzo pagato.

## Properties e fixture (righe 8-31)

```
properties
    p      % costate/thrust parameter struct
    z0      % initial state [x; y; vx; vy; m; lam_m]
end

properties (TestParameter)
    RelTol = struct('Loose', 1e-6, 'Tight', 1e-10)
end
```

- **Righe 8-11 – properties normali.** p e z0 sono i dati del benchmark, riempiti dal TestMethodSetup. Sono property e non variabili locali perche' devono essere costruite **fuori** dalla regione misurata (vedi sotto): il setup non entra nel tempo.
- **Righe 13-15 – TestParameter.** Questo e' il meccanismo di **parametrizzazione** di matlab.unittest: una struct i cui campi diventano altrettanti casi di test. Qui RelTol ha due valori – Loose =  $1e-6$  e Tight =  $1e-10$  – quindi il metodo testBurnArcIntegration(testCase, RelTol) viene eseguito **due volte**, e i risultati compaiono come due voci separate (testBurnArcIntegration(RelTol=Loose) e (RelTol=Tight)). I nomi dei campi (Loose, Tight) diventano le etichette nel report: e' il motivo per cui si usa una struct e non un cell array.

Il **valore diagnostico** e' proprio nel rapporto fra i due tempi: quantifica quanto costa la tolleranza stretta che la convenzione del repo impone (RelTol =  $1e-10$ , AbsTol =  $1e-12$  per il lavoro indiretto/shooting). Non e' una tolleranza gratuita, e questo benchmark ne mette il prezzo su un numero.

- **Righe 17-22 – TestClassSetup.** Identico a quello di odeBurnTest.m: risale di due livelli da mfilename('fullpath') per trovare HM1/ e lo aggiunge al path con una PathFixture (rimozione automatica alla fine). Gira **una volta sola** per la classe.
- **Righe 24-31 – TestMethodSetup.** Gira **prima di ogni metodo di test**.

```
% Representative Task 1 solution neighbourhood (Q = 2.5, yf = 0.04)
testCase.p = struct('T', 1.5, 'Q', 2.5, 'c', 0.6, ...
    'lam_vx0', 0.6, 'lam_vy0', 3.8, 'lam_y', 14);
testCase.z0 = [0; 0; 0; 0; 1; 1];
```

I numeri **non sono arbitrari**: sono presi dall'intorno della soluzione vera di Task 1. Il README di HM1 riporta  $Q^* = 2.52$  come portata ottima per  $y_f = 0.04$ , e il commento alla riga 26 lo dichiara. La coerenza interna torna:  $T/c = 1.5/0.6 = 2.5 = Q$ , come deve essere visto che  $dm/dt = -T/c = -Q$ .

**Perche' e' importante**: un benchmark ha senso solo se misura il **regime operativo reale**. Con costati piccoli e  $\text{lam\_y}$  nullo, l'integratore farebbe pochi passi e il numero misurato sarebbe ottimistico e inutile. Con  $\text{lam\_y} = 14$  l'angolo di spinta ruota rapidamente e `ode45` deve lavorare come lavora davvero dentro `fsolve`.  $\text{z0} = [0;0;0;0;1;1]$  e' letteralmente la condizione iniziale usata da `main_task1` (`ic = [0;0;0;0;1;1]`, con  $\text{lam\_m0} = 1$  per la normalizzazione).

---

## testRhsEvaluation (righe 34-41)

```
function testRhsEvaluation(testCase)
    pp = testCase.p;
    z = [0.05; 0.01; 0.4; 0.2; 0.7; 1.2];
    while testCase.keepMeasuring
        dz = ode_burn(0.15, z, pp);
    end
    testCase.verifySize(dz, [6 1]);
end
```

- **Riga 35** – `pp = testCase.p`. Sembra pedanteria, non lo e'. L'accesso a una **property** di un oggetto MATLAB passa dal meccanismo delle classi ed e' sensibilmente piu' lento dell'accesso a una variabile locale. Copiando `p` in `pp` fuori dal ciclo, la misura riguarda `ode_burn` e non l'overhead di `testCase.p`. E' la regola generale del micro-benchmarking: tutto cio' che non vuoi misurare va fuori dalla regione misurata.
- **Righe 37-39** – `while testCase.keepMeasuring`. E' il costrutto centrale di `matlab.perftest`. Delimita la **regione cronometrata**: tutto cio' che sta prima (`pp = ...`, `z = ...`) e dopo (`verifySize`) e' **escluso** dal tempo. Il framework si occupa di ripetere il corpo quanto serve per superare la granularita' del timer (una singola chiamata a `ode_burn` dura frazioni di microsecondo, ben sotto la risoluzione di `tic/toc`) e di riportare il tempo **per iterazione**.

Senza `keepMeasuring` (cioe' misurando l'intero metodo) il tempo sarebbe dominato dall'overhead del framework e la misura sarebbe priva di significato.

- **Riga 40** – `verifySize` **FUORI dal ciclo**. Due motivi: (a) l'asserzione non deve entrare nel tempo misurato; (b) mettendola comunque, il metodo resta un **test valido** eseguibile con `runtests`, non solo un benchmark. Questo e' il vantaggio di `matlab.perftest.TestCase < matlab.unittest.TestCase`.

**Nota di stile**: `dz` viene usata dopo il ciclo, quindi il codice assume che il corpo del `while` giri **almeno una volta**. In pratica `keepMeasuring` garantisce sempre almeno un'iterazione, quindi funziona – ma e' una dipendenza implicita dal comportamento del framework, non un invariante scritto.

- **Cosa misura davvero**. Il costo di: 7 accessi a campi della struct (`lam_vx0`, `lam_vy0`, `lam_y`, `Q`, piu' `T` letto tre volte) + 1 `sqrt` + 1 `atan2` + 1 `sin` + 1 `cos` + 1 `zeros(6,1)` + poche moltiplicazioni. Le funzioni trascendenti (`atan2`, `sin`, `cos`) sono la parte dominante. E' il numero da moltiplicare per  $\sim 1e6$  per capire quanto pesa `ode_burn` nel tempo totale di `main_task1`.

**Possibile domanda d'esame** – Come useresti questo benchmark per giustificare l'assenza del blocco `arguments` in `ode_burn.m`? *Risposta*: Misurerei `testRhsEvaluation` cosi' com'e' (baseline), poi aggiungerei a `ode_burn` un blocco `arguments` con i validatori tipici (`mustBeNumeric`, `mustBeReal`, controllo di dimensione su `z`, `mustBeField` su `p`) e rimisurerei. Il corpo della funzione costa poche centinaia di nanosecondi – dominato da `atan2`, `sin`, `cos` – mentre un blocco `arguments` con validatori costa tipicamente qualche microsecondo per chiamata. Il rapporto puo' quindi essere di un ordine di grandezza, e moltiplicato per  $1e6+$  chiamate diventa la differenza fra uno sweep che gira in decine di secondi e uno che gira in minuti. **Onesta': questo A/B il file non lo fa** – fornisce solo il termine di paragone. La misura del ramo “con `arguments`” andrebbe fatta apposta.

## testBurnArcIntegration (righe 43-52)

```
function testBurnArcIntegration(testCase, RelTol)
    pp = testCase.p;
    ic = testCase.z0;
    opts = odeset('RelTol', RelTol, 'AbsTol', RelTol*1e-2);
    while testCase.keepMeasuring
        [~, Z] = ode45(@(t,z) ode_burn(t, z, pp), [0 0.3], ic, opts);
    end
    % Mass equation is linear: exact propellant bookkeeping
    testCase.verifyEqual(Z(end,5), 1 - pp.Q*0.3, 'AbsTol', 1e-8);
end
```

- **Riga 43 – la firma parametrizzata.** Il secondo argomento `RelTol` viene iniettato dal framework, che chiama il metodo una volta per ogni campo della `TestParameter`. Non e' un argomento che si passa a mano.

- **Riga 46 – le tolleranze.** `AbsTol = RelTol * 1e-2`, quindi:

```
Loose: RelTol = 1e-6,   AbsTol = 1e-8
Tight: RelTol = 1e-10,  AbsTol = 1e-12
```

Il caso `Tight` riproduce **esattamente** le tolleranze usate dai `main_task*.m` (`RelTol = 1e-10`, `AbsTol = 1e-12`, la convenzione del repo per il lavoro indiretto/shooting). Il caso `Loose` e' il termine di paragone.

La regola `AbsTol = RelTol/100` e' un'euristica sensata: `AbsTol` deve dominare vicino allo zero (dove `RelTol` perde senso perche' moltiplica una quantita' nulla), e le componenti dello stato di HM1 sono  $O(0.1-1)$  in nondimensionale, quindi due ordini sotto `RelTol` e' un buffer ragionevole.

- **Righe 47-49 – la regione misurata.** Una **intera integrazione** dell'arco propulso su `[0, 0.3]`. E' l'unita' di lavoro che `fsolve` chiama a ogni valutazione del residuo, quindi il numero misurato e' quello che, moltiplicato per il numero di valutazioni del residuo, da' il tempo di un solve.

Anche qui `pp` e `ic` sono copiati fuori dal ciclo, e `opts` e' costruita fuori (`odeset` non e' gratis).

- **Riga 51 – l'asserzione di sanita'.** `m(tf) = 1 - Q * 0.3 = 1 - 0.75 = 0.25`, esatto. Il commento (riga 50) spiega perche' funziona: **l'equazione di massa e' lineare** ( $dm/dt = -Q$  costante), quindi ogni metodo Runge-Kutta la integra in modo **esatto** a meno dell'arrotondamento macchina – indipendentemente da `RelTol`. Ecco perche' la stessa `AbsTol = 1e-8` sull'asserzione passa sia nel caso `Loose` che nel caso `Tight`.

E' un'asserzione ben scelta: e' l'**unica** quantita' del problema che si puo' asserire con la stessa tolleranza in entrambi i rami parametrizzati. Asserire `vy(tf)` con `1e-8` fallirebbe nel caso `Loose`. E' anche il motivo per cui il commento la chiama "exact propellant bookkeeping": non e' un test di accuratezza, e' un test che l'integrazione sia **avvenuta** e sull'arco giusto.

- **Il vero output del test non e' l'assert, e' il tempo.** Confrontando i due campioni si ottiene il **fattore di costo della tolleranza stretta**. E' un numero che vale la pena conoscere prima di dire "tanto le tolleranze si mettono strette e basta": in un metodo indiretto `RelTol = 1e-10` non e' un vezzo – i costati sono estremamente sensibili alle condizioni iniziali e un'integrazione imprecisa produce un residuo di shooting rumoroso, che manda in confusione il Jacobiano alle differenze finite di `fsolve` e uccide la convergenza. Il benchmark dice **quanto** si paga quella necessita'.

**Possibile domanda d'esame** – Perche' un metodo indiretto ha bisogno di `RelTol = 1e-10` mentre un metodo diretto (HM2) sopravvive con tolleranze molto piu' larghe? *Risposta:* Perche' nello shooting il residuo e' una funzione dell'**intera propagazione**: l'errore dell'integratore entra direttamente nel residuo, e `fsolve` costruisce il Jacobiano per **differenze finite** perturbando le incognite di quantita' piccole. Se il rumore numerico dell'integratore e' dello stesso ordine della perturbazione, il Jacobiano stimato e' spazzatura e Newton non converge. Inoltre le dinamiche dei costati sono

spesso instabili in avanti: piccoli errori si amplificano lungo l'arco. Nel collocation invece la traiettoria e' una variabile di decisione discretizzata e i vincoli sono **locali** (un residuo per intervallo): non c'e' propagazione da amplificare, e l'accuratezza si controlla infittendo la griglia, non stringendo un integratore.

---

## Limiti del file (onesta' richiesta)

- **Non e' un regression gate.** Non c'e' nessuna asserzione sul **tempo** (non c'e' nessun `verifyLessThan(t, soglia)`). `runperf` produce dei numeri, ma nessuno li confronta automaticamente con un baseline salvato: se domani una modifica rendesse `ode_burn` 10 volte piu' lenta, il test **passerebbe comunque**. Per farne un gate servirebbe salvare un baseline e confrontarlo (per esempio con `matlab.perftest.TimeExperiment` e una soglia esplicita).
  - **Non misura l'alternativa.** Come detto sopra, non c'e' un ramo "con blocco `arguments`" da confrontare: il file fornisce la baseline che *renderebbe possibile* quel confronto, ma il confronto non e' fatto. La tesi "niente `arguments` perche' hot loop" resta quindi **argomentata ma non misurata** dentro il repo.
  - **Non misura l'intero `main_task1`.** L'unita' massima e' una singola integrazione, non un solve di `fsolve` ne' uno sweep di continuation. La stima `1e6+` chiamate resta una stima analitica, non un dato raccolto (per contarle davvero basterebbe un contatore `persistent` in `ode_burn`, ma sarebbe intrusivo proprio nell'hot loop).
  - **c e' nel parameter struct ma `ode_burn` non lo usa.** `ode_burn.m` legge solo `T`, `Q`, `lam_vx0`, `lam_vy0`, `lam_y`: il campo `c` viene passato ma mai letto (serve altrove, per esempio nel calcolo di `H(0)` dentro il residuo). Non e' un bug, ma e' un pelo di grasso nella struct che viene copiata a ogni chiamata.
- 

## Possibili domande d'esame

**D: Che cos'e' `matlab.perftest` e cosa fa in piu' rispetto a `tic/toc`?** R: E' il framework di performance testing di MATLAB, costruito sopra `matlab.unittest`. Rispetto a un `tic/toc` a mano aggiunge tre cose. (1) **Statistica:** `runperf` non fa una misura sola, ne fa molte – di default esegue alcune run di warm-up (per assorbire l'effetto della JIT compilation e delle cache fredde) e poi raccoglie campioni finche' il margine di errore relativo non scende sotto una soglia (default 5% al 95% di confidenza), fino a un massimo di campioni. (2) **Isolamento della regione misurata:** `while testCase.keepMeasuring` esclude setup, teardown e asserzioni dal cronometro, e ripete il corpo quanto basta a superare la granularita' del timer. (3) **Integrazione con il framework di test:** parametrizzazione (`TestParameter`), fixture, e il fatto che gli stessi metodi restano test veri eseguibili con `runtests`.

**D: Perche' `ode_burn.m` non ha un blocco `arguments`, se il resto del repo lo raccomanda?** R: Perche' e' l'hot loop. `ode_burn` viene chiamata circa `1e6+` volte in un run di `main_task1` (6 valutazioni per passo di `ode45`, x centinaia/migliaia di passi per integrazione a `RelTol = 1e-10`, x ~5 integrazioni per Jacobiano alle differenze finite su 4 incognite, x decine di iterazioni di `fsolve`, x centinaia di solve nello sweep di continuation – 80 nodi in `Q x 3` valori di `yf`). Il corpo della funzione costa poche centinaia di nanosecondi; un blocco `arguments` con validatori costa tipicamente qualche microsecondo – potenzialmente **piu' della funzione stessa**. La convenzione del repo (scritta in `CLAUDE.md`) e' quindi: la validazione vive nei **boundary helper** chiamati una volta per run (`nondim`, `solve_*`, `plotting`), non nelle RHS di ODE, nei residui di shooting o nelle callback di `nonlcon`. E' un **trade-off consapevole**: si accetta un messaggio di errore piu' criptico in cambio di un ordine di grandezza sul tempo di esecuzione.

**D: Perche' parametrizzare su `RelTol` invece di misurare solo la tolleranza vera (`1e-10`)?** R: Perche' un numero isolato non dice niente. Sapere che un'integrazione costa X millisecondi e' inutile senza un termine di paragone; sapere che a `1e-10` costa k volte quello che costa a `1e-6` **quantifica il prezzo della scelta di tolleranza**, che e' una decisione di progetto documentata nel `CLAUDE.md`. Il benchmark trasforma una



convenzione (“tolleranze strette per il lavoro indiretto”) in un costo misurabile, e permette di rispondere a “e se le allentassi?” con un numero invece che con un’opinione.

**D: L’asserzione sulla massa ( $m(t_f) = 1 - Q \cdot t_f$ ) passa con la stessa tolleranza sia a  $RelTol = 1e-6$  che a  $1e-10$ . Come e’ possibile?** R: Perche’  $dm/dt = -Q$  e’ costante: la soluzione e’ un polinomio di grado 1 in  $t$ , e ogni metodo Runge-Kutta di ordine  $\geq 1$  integra esattamente i polinomi di grado 1, a meno dell’arrotondamento macchina. Il controllo di passo adattivo non c’entra: l’errore locale su quella componente e’ zero per costruzione. Ecco perche’ e’ l’asserzione giusta da mettere in un test **parametrizzato sulla tolleranza** – e’ l’unica quantita’ che si puo’ asserire con la stessa  $AbsTol$  in entrambi i rami. Se avessi asserito  $vy(t_f)$  con  $1e-8$ , il ramo *Loose* sarebbe fallito.

**D: Il benchmark protegge da regressioni di performance?** R: **No, non da solo**, e va detto. Non c’e’ nessuna asserzione sul tempo: *runperf* stampa i tempi ma non li confronta con un baseline salvato. Se *ode\_burn* diventasse 10 volte piu’ lenta, la suite passerebbe. Per farne un vero gate servirebbe salvare i risultati di *runperf* come baseline (per esempio serializzando i *MeasurementResult*) e aggiungere un confronto esplicito, oppure inserire una soglia assoluta. Oggi il file e’ uno **strumento di misura**, non un guardrail.

# HM2\_\_powered\_\_descent/tests/odeDescentTest

## Ruolo del file nel progetto

Questa e' la suite di unit test (class-based `matlab.unittest`) del cuore dinamico di HM2: `ode_descent.m`, il right-hand side (RHS) adimensionale del problema di powered descent. La funzione sotto test e' minuscola – due righe di codice eseguibile – ma e' il pezzo su cui poggia TUTTO il resto dell'homework: la collocazione trapezoidale la usa per costruire i defect, il propagatore `rk4_zoh.m` la chiama quattro volte per substep, la linearizzazione LTV di SCvx ne calcola le Jacobiane analitiche, e la replay open-loop con `ode45` la integra per misurare la fedelta' della soluzione. Un errore di segno o di scala qui non farebbe fallire nessun solver: produrrebbe semplicemente una traiettoria ottimale... del problema sbagliato. Da qui l'esigenza di un test che inchiodi la definizione della derivata prima ancora di parlare di ottimizzazione.

La funzione testata (`HM2_powered_descent/ode_descent.m`, righe 13-14) e':

```
Tmag = sqrt(u(1)^2 + u(2)^2)
dx    = [ x(3); x(4); u(1)/x(5); u(2)/x(5) - 1; -Vc*Tmag ]
```

cioe', in forma matematica, il punto materiale 2D a Terra piatta, senza aerodinamica, in variabili adimensionali:

```
x_dot = vx
y_dot = vy
vx_dot = Tx/m
vy_dot = Ty/m - 1
m_dot = -Vc * ||T||
```

Il "-1" nella riga di `vy` e' la gravita' adimensionalizzata ( $g = 1$  per costruzione dello scaling: l'accelerazione di riferimento e'  $g$  stessa).  $Vc = V_{ref}/c$  e' il numero di Tsiolkovsky, con  $c = Isp \cdot g_0$  velocita' efficace di scarico; nel codice di produzione vale 0.0777 (vedi `main_task2.m`).

Attenzione a una convenzione che ritorna in tutta HM2: qui il **controllo e' la spinta  $T$** , non l'accelerazione. Il file gemello `ode_descent_uacc.m` tiene costante  $u = T/m$  ed e' quello usato dalla variante GFOLD log-massa. Le due RHS NON sono la stessa cosa sotto ZOH (ci torniamo nella pagina di `rk4_zoh`), e questa suite documenta solo la prima.

I quattro test scelgono quattro stati/controlli in cui la risposta esatta e' scrivibile a mano. Non c'e' nessun confronto con differenze finite, e vedremo perche' e' la scelta giusta per una RHS (mentre sarebbe la scelta giusta – e manca – per le Jacobiane analitiche di SCvx).

---

## TestClassSetup – addHm2ToPath (righe 5-10)

```
hm2 = fileparts(fileparts(mfilename('fullpath')));
testCase.applyFixture( ...
    matlab.unittest.fixtures.PathFixture(hm2));
```

- Righe 6-9: `mfilename('fullpath')` restituisce il path assoluto di QUESTO file di test (`.../HM2_powered_descent/tests`). Due `filepath`s annidati risalgono di due livelli e danno la cartella `HM2_powered_descent/`, dove vivono `ode_descent.m` e `rk4_zoh.m`.
- Riga 8: `PathFixture` aggiunge quella cartella al MATLAB path e la **rimuove automaticamente al teardown** della classe. E' la differenza fra un test pulito e un `addpath` che sporca l'ambiente dell'utente: dopo `runtests` il path e' esattamente quello di prima.
- Il blocco e' `TestClassSetup` e non `TestMethodSetup`: la fixture serve una volta sola per l'intera classe (i quattro test condividono lo stesso path). Metterla per-metodo la applicherebbe e rimuoverebbe quattro volte, senza alcun guadagno.
- Effetto pratico: si puo' lanciare `runtests('HM2_powered_descent/tests')` dalla root della repo senza `cd` preventivo. E' anche la ragione per cui i test girano identici in CI headless (`matlab -batch`).

**Possibile domanda d'esame** – Perche' usare `PathFixture` invece di un semplice `addpath` in cima al test? *Risposta:* Perche' la fixture e' transazionale: il framework garantisce il ripristino del path anche se un test fallisce o solleva un'eccezione. Un `addpath` "nudo" lascerebbe la cartella nel path del MATLAB dell'utente, creando dipendenze invisibili fra test successivi (un test potrebbe passare solo perche' un test precedente aveva aggiunto una cartella). L'isolamento fra test e' una proprieta' che si progetta, non che si spera.

## testDerivativeDefinition (righe 13-21)

```
x = [1; 2; 0.3; -0.4; 0.8];
u = [0.6; 0.8];           % ||u|| = 1
Vc = 0.0777;
dx = ode_descent(x, u, Vc);
expected = [0.3; -0.4; 0.6/0.8; 0.8/0.8 - 1; -Vc];
testCase.verifyEqual(dx, expected, 'AbsTol', 1e-15);
```

- Righe 15-17: lo stato e' `[x; y; vx; vy; m] = [1; 2; 0.3; -0.4; 0.8]` e il controllo `u = [0.6; 0.8]`. La scelta della terna pitagorica 3-4-5 riscalata non e' casuale:  $||u|| = \sqrt{0.36 + 0.64} = 1$  esattamente, il che permette di scrivere la riga di massa attesa come `-Vc` puro, senza dover ricalcolare una radice a mano nel test. Un test il cui valore atteso richiede una formula complicata e' un test che puo' sbagliare quanto il codice.
- Riga 19: `expected` e' la derivata calcolata **a mano** dalla definizione: `[vx; vy; Tx/m; Ty/m - 1; -Vc*||T||] = [0.3; -0.4; 0.75; 0; -0.0777]`. Nota che `0.6/0.8` e `0.8/0.8` sono lasciati come espressioni: il test dichiara la formula, non il numero, cosi' un lettore vede immediatamente quale riga della dinamica sta verificando.
- Riga 20: `AbsTol 1e-15` invece di `verifyEqual` esatto. Serve:  $0.6^2 = 0.36$  non e' rappresentabile esattamente in binario, quindi  $\sqrt{0.36 + 0.64}$  puo' differire da 1.0 di un ulp (circa  $2.2e-16$ ), e la riga di massa `-Vc*Tmag` eredita quell'errore. La tolleranza e' scelta appena sopra la macchina epsilon: e' ancora un test di identita' esatta, non un test "circa uguale".
- Questo e' il test di **correttezza di definizione**: se qualcuno invertisse `x(3)` e `x(4)`, dimenticasse la divisione per la massa, o mettesse il segno sbagliato alla gravita', qui si vede subito.

Un dettaglio non intenzionale ma vero: con `Ty = 0.8` e `m = 0.8`, `dx(4)` viene esattamente 0, cioe' questo stato e' anche in equilibrio verticale. Il test funziona lo stesso, ma un lettore distratto potrebbe non accorgersi che la riga di `vy` sta verificando  $1 - 1 = 0$  e non un numero "vivo".

**Possibile domanda d'esame** – Perche' non confrontare la RHS con una derivata numerica per differenze finite? Non e' quello il test canonico? *Risposta:* Le differenze finite sono il test canonico per una **Jacobiana** (una derivata di una funzione gia' implementata), non per una RHS. `ode_descent` non e' la derivata di qualcosa che il codice conosce: e' il modello fisico stesso, un'espressione algebrica chiusa. Confrontarla con differenze finite di cosa? Bisognerebbe integrare la traiettoria e derivarla: si introdurrebbero errori di troncamento ( $O(h)$  o  $O(h^2)$ ) e si perderebbe la capacita' di distinguere un bug da un errore di discretizzazione. Il confronto con il valore esatto calcolato a mano e' piu' forte:

tolleranza 1e-15 invece di 1e-6. Il test a differenze finite serve invece per la funzione `jacobians` in `main_task2.m` (righe 255-277), che calcola A e B analitiche per SCvx – e quel test, ad oggi, NON esiste nella suite: e’ un buco di copertura reale.

---

## testBallisticCoast (righe 23-28)

```
x = [0.5; 1; 0.2; -0.1; 0.9];
dx = ode_descent(x, [0; 0], 0.0777);
testCase.verifyEqual(dx, [0.2; -0.1; 0; -1; 0], 'AbsTol', 1e-15);
```

- Riga 26: motore spento,  $u = [0; 0]$ . La derivata attesa (riga 27) e’ caduta libera pura: le velocita’ si propagano nelle posizioni,  $vx\_dot = 0$ ,  $vy\_dot = -1$  (solo gravita’),  $m\_dot = 0$ .
  - **Perche’ e’ l’invariante giusto:** l’arco di coast e’ un pezzo *reale* della soluzione ottima. La traccia produce un profilo bang-off-bang (max-coast-max): nella soluzione nominale il motore e’ spento fra circa  $t = 14.0$  s e  $t = 33.1$  s (vedi il README di HM2). Con  $tf = 38$  s sono 19.1 s di coast: il solver passera’ **circa meta’** del tempo di volo a valutare la RHS esattamente in  $u = 0$ . Se la RHS fosse mal definita li’, l’ottimizzazione lavorerebbe su spazzatura proprio nel tratto piu’ lungo.
  - Il punto delicato che questo test tocca senza dirlo:  $u = 0$  e’ il **punto di non differenziabilita’** della norma  $||u|| = \sqrt{u_1^2 + u_2^2}$ . Il *valore* della RHS li’ e’ perfettamente definito (e il test lo prova: nessun NaN,  $m\_dot = 0$ ), ma il *gradiente* rispetto a  $u$  non lo e’: la norma euclidea ha un cono in zero. Il test verifica la continuita’ del valore, non la differenziabilita’ – ed e’ bene sapere che quest’ultima *manca*. E’ esattamente la causa radice dello stallo di convergenza documentato nel README (fmincon con gradienti a differenze finite si blocca a ottimalita’ del primo ordine  $1e-3 / 1e-4$  sull’arco di coast). La convessificazione lossless con la slack  $\Gamma \geq ||T||$  esiste proprio per togliere la norma dalla dinamica.
  - Nota di onesta’:  $u = [0; 0]$  produce  $0/x(5) = 0$  solo perche’  $x(5) = 0.9 > 0$ . La funzione **non ha alcuna guardia** su massa nulla:  $x(5) = 0$  darebbe  $0/0 = \text{NaN}$  o  $\text{Inf}$ . Nessun test copre quel caso, e la funzione non ha `arguments` block. Il contratto e’ delegato al call site: in `main_task2.m` i box bounds impongono  $m \geq 1e-3$ .
- 

## testHoverEquilibrium (righe 30-37)

```
m = 0.73;
x = [0; 1; 0; 0; m];
dx = ode_descent(x, [0; m], 0.0777);
testCase.verifyEqual(dx(3), 0, 'AbsTol', 1e-15);
testCase.verifyEqual(dx(4), 0, 'AbsTol', 1e-15);
```

- Righe 32-34: spinta puramente verticale di modulo pari alla massa,  $u = [0; m]$ . Allora  $vy\_dot = m/m - 1 = 0$  e  $vx\_dot = 0/m = 0$ .
- **Perche’ e’ l’invariante giusto:** e’ l’unico test che verifica la *scala* della gravita’, non solo il suo segno. In variabili dimensionali la condizione di hover e’  $T = m \cdot g$ ; nel codice adimensionalizzato diventa  $T_y = m$  perche’  $g = 1$ . Se lo scaling fosse sbagliato (per esempio se qualcuno avesse lasciato  $-9.80665$  o un  $-g_0/a\_ref$  diverso da 1 nella riga 14 di `ode_descent.m`), `testDerivativeDefinition` fallirebbe ma in modo poco parlante; qui il fallimento dice esattamente “la tua condizione di hover non e’ hover”, cioe’ punta al bug fisico.
- C’e’ un legame diretto col solver: `init_guess` in `main_task2.m` (riga 673) inizializza il controllo proprio a  $T = m_0$  (hover, gravita’ = 1 nondim). Questo test e’ quindi anche la validazione della *guess iniziale* usata da tutte le varianti.
- Sottigliezza importante: il test asserisce solo `dx(3)` e `dx(4)`, **non** l’intero vettore. Corretto: l’hover NON e’ un punto di equilibrio del sistema a 5 stati, perche’  $m\_dot = -Vc \cdot m < 0$ . Il veicolo tiene la quota

ma consuma propellente, quindi la spinta di hover richiesta cambia istante per istante. Assertire `dx == 0` sarebbe stato un errore concettuale.

**Possibile domanda d'esame** – L'hover e' un equilibrio del sistema? *Risposta:* No. E' un equilibrio del *sottosistema traslazionale* (le derivate di  $v_x$  e  $v_y$  si annullano istantaneamente), ma non del sistema completo: la riga di massa resta  $m\_dot = -Vc * ||T|| < 0$ . Il sistema non ha punti fissi con motore acceso; l'unico equilibrio genuino sarebbe motore spento e veicolo a terra. Per questo il test verifica solo le componenti 3 e 4 del vettore derivata.

---

## testMassFlowDependsOnlyOnThrustMagnitude (righe 39-47)

```
x = [0; 1; 0; 0; 1];
dx1 = ode_descent(x, [1; 0], Vc);
dx2 = ode_descent(x, [0; -1], Vc);
testCase.verifyEqual(dx1(5), -Vc, 'AbsTol', 1e-15);
testCase.verifyEqual(dx2(5), dx1(5), 'AbsTol', 1e-15);
```

- Righe 43-44: due controlli di modulo unitario ma direzione completamente diversa: uno orizzontale (`[1; 0]`), l'altro verticale **verso il basso** (`[0; -1]`).
  - Righe 45-46: entrambi devono dare  $m\_dot = -Vc$ . E' l'**isotropia del consumo**: il razzo brucia propellente in proporzione al modulo della spinta, non alla sua direzione. La legge e'  $m\_dot = -||T||/c$  che, adimensionalizzata, diventa  $m\_dot = -Vc * ||T||$ .
  - **Perche' e' l'invariante giusto**: e' un test di *falsificazione mirata*. La scelta di `[0; -1]` e' astuta: se qualcuno implementasse la riga di massa con una componente con segno invece che con la norma (per esempio  $-Vc * u(2)$ , errore plausibile in un modello 1D verticale poi esteso a 2D), quel controllo darebbe  $m\_dot = +Vc$ , cioe' massa **crescente**: il razzo si riempirebbe di propellente spingendo verso il basso, e il solver di minimo consumo sfrutterebbe immediatamente quel bug per "guadagnare" massa finale. La funzione obiettivo di HM2 e'  $\max m(tf)$ : qualsiasi errore sulla riga 5 e' un errore *sull'obiettivo*, non solo sulla dinamica. Da qui la dignita' di test dedicato.
  - Nota fisica:  $u = [0; -1]$  e' assurdo per un lander (spinta verso il suolo), ma la RHS non deve saperlo – i vincoli fisici stanno nei bound e nei path constraint, non nel modello. Il test sfrutta correttamente questa separazione di responsabilita'.
- 

## Cosa questa suite NON copre (onesta' sulla copertura)

- **Nessun test sulla singolarita' di massa**.  $x(5) \rightarrow 0$  fa esplodere  $u/x(5)$ . La protezione vive nei box bounds del NLP ( $m \geq 1e-3$ ), non nella funzione: e' una scelta deliberata (nessun `arguments` block nelle hot loop), ma va detta.
  - **Nessun test sulle Jacobiane analitiche**. `jacobians` in `main_task2.m` (righe 255-277) calcola A e B a mano, con una regolarizzazione `Tmag_reg` per evitare la divisione per zero della derivata della norma. Quella si' che meriterebbe un confronto con differenze finite – ed e' assente.
  - **Nessun test sulla forma degli input**. La funzione non ha `arguments` block; il contratto (colonne 5x1 e 2x1) e' solo nel commento di header.
  - **Nessun test di conservazione lungo una traiettoria**. Qui si testa solo la RHS puntuale; la monotonia di  $m(t)$  e la positivita' della quota sono verificate altrove (o non verificate affatto).
- 

## Possibili domande d'esame

**D: Quali invarianti verifica questa suite, e perche' proprio quelli?** R: Quattro. (1) La *definizione* della derivata in un punto generico, contro un valore calcolato a mano – inchioda ordine delle righe, divisione per la

massa, segno della gravita'. (2) Il *coast balistico* ( $u = 0$ ), perche' e' un arco reale della soluzione bang-off-bang ed e' anche il punto di non differenziabilita' della norma. (3) L'*hover* ( $T_y = m$ ), che e' l'unico test che verifica la scala dell'adimensionalizzazione ( $g = 1$ ). (4) L'*isotropia del consumo* ( $\dot{m} = -V_c/|T|$ ), che protegge la funzione obiettivo  $\max m(tf)$  da un errore di segno. Insieme coprono le cinque righe della RHS con test che falliscono in modo diagnostico, cioe' dicono quale\* pezzo di fisica e' rotto.

**D: Perche' la tolleranza e' 1e-15 e non zero?** R: Perche'  $0.6^2 = 0.36$  non e' rappresentabile esattamente in virgola mobile binaria, quindi  $\text{sqrt}(0.6^2 + 0.8^2)$  puo' differire da 1 di circa un ulp ( $2.2e-16$ ). Con `verifyEqual` esatto il test sarebbe fragile rispetto a dettagli di arrotondamento della libreria matematica. 1e-15 e' appena sopra la macchina epsilon: resta un test di identita' esatta, non un test di "vicinanza numerica".

**D: L'RHS divide per la massa. Cosa succede se la massa va a zero, e perche' non c'e' un controllo?**

R: Si otterrebbe Inf o NaN e il solver divergerebbe. Non c'e' controllo perche' `ode_descent` e' una hot loop: viene chiamata dell'ordine di  $1e8$  volte dentro `fmincon` (49 intervalli x 4 valutazioni RK4 x  $n_{\text{sub}}$  substep x  $\sim 350$  valutazioni per gradiente a differenze finite x fino a 1000 iterazioni). Un `arguments` block costerebbe circa un microsecondo per chiamata contro i  $\sim 60$  ns della funzione stessa: renderebbe la validazione piu' costosa della fisica. La protezione e' spostata al call site (box bound  $m \geq 1e-3$  nel NLP). E' la stessa filosofia documentata nell'header del file e nel CLAUDE.md della repo.

**D: Perche' l'arco di coast e' il punto piu' delicato di tutta HM2?** R: Perche' e' il punto in cui la dinamica e' perfettamente sana ma il suo *gradiente* non esiste.  $||T||$  ha un cono in  $T = 0$ ; `fmincon` con differenze finite vi calcola un gradiente arbitrario, e la convergenza al primo ordine si ferma a  $1e-3/1e-4$  (documentato nel README). Il test `testBallisticCoast` prova che il valore e' corretto, non che il problema sia liscio. La cura strutturale e' la convessificazione lossless: si introduce la slack  $\Gamma \geq ||T||$  e si sostituisce la norma nella dinamica con  $\Gamma$ , che e' lineare – e si dimostra che all'ottimo il vincolo e' attivo ( $\Gamma = ||T||$ ), quindi non si perde nulla.

**D: Perche' esistono due RHS diverse (`ode_descent` e `ode_descent_uacc`) e questa suite ne testa una sola?** R: Perche' realizzano due convenzioni ZOH diverse. `ode_descent` tiene costante la **spinta**  $T$  sull'intervallo (convenzione della trascrizione nonlineare + RK4); `ode_descent_uacc` tiene costante l'**accelerazione**  $u = T/m$  (convenzione nativa della trascrizione GFOLD log-massa, dove il cambio di variabile  $z = \ln m$  rende la dinamica esattamente LTI). Sotto ZOH le due producono traiettorie *diverse* sullo stesso intervallo (coincidono solo nel limite  $dt \rightarrow 0$ ). `odeDescentTest` copre la prima; la seconda, insieme a `lti_zoh.m`, e' coperta da `gfoldLogMassTest.m`.

# HM2\_\_powered\_\_descent/tests/rk4ZohTest.m

## Ruolo del file nel progetto

Questa suite (matlab.unittest, class-based) testa `rk4_zoh.m`, il propagatore Runge-Kutta a 4 stadi che HM2 usa per la trascrizione ZOH (Task 2, variante a: “Nonlinear ZOH + RK4”, multiple shooting). E’ il pezzo che trasforma la dinamica continua in una mappa discreta:

```
x_{k+1} = RK4(x_k, u_k, dt)
```

e i vincoli di uguaglianza del NLP (i *defect*) sono esattamente  $Z(1:5, k+1) - \text{rk4\_zoh}(\dots)$  (vedi `zoh_nonlcon` in `main_task2.m`, righe 797-800). Quindi la mappa discreta e’ il **modello** su cui `fmincon` ottimizza: se il propagatore ha un bug, il solver converge alla soluzione esatta di una dinamica che non esiste, e la replay open-loop con `ode45` la smaschera con un errore di atterraggio. La suite serve a evitare che si arrivi a quel punto.

`rk4_zoh.m` (righe 15-23) e’ il classico RK4 con `n_sub` substep interni:

```
h = dt / n_sub
k1 = f(x), k2 = f(x + h/2*k1), k3 = f(x + h/2*k2), k4 = f(x + h*k3)
x = x + (h/6)*(k1 + 2*k2 + 2*k3 + k4)
```

con `f = ode_descent` e il controllo `u` **tenuto costante** su tutto l’intervallo – questo e’ il significato di *zero-order hold*. In produzione `main_task2.m` usa `n_sub = 2` (riga 37) e `dt = tf_nd/(N-1)` con `N = 50`.

Nota di convenzione, cruciale e ricorrente: qui e’ la **spinta T** a essere tenuta costante, non l’accelerazione. La variante GFOLD (`lti_zoh.m` + `ode_descent_uacc.m`) tiene costante `u = T/m`. Sono due ZOH *diversi*: producono traiettorie diverse sullo stesso intervallo finito e coincidono solo nel limite  $dt \rightarrow 0$ . Ci torniamo alla fine, perche’ e’ il motivo per cui in questa suite **non** esiste un test “`rk4_zoh` riproduce `lti_zoh`”.

I tre test coprono, nell’ordine: accuratezza contro un riferimento fidato, esattezza strutturale sulla riga di massa, e – il piu’ importante – **ordine di convergenza**.

---

## Proprieta’ costanti (righe 5-10)

```
properties (Constant)
    Vc = 0.0777 % V_ref/c
    x0 = [0.3; 1; -0.2; -0.6; 1] % non-dim initial state
    u = [0.5; 1.0] % non-dim ZOH control
    dt = 0.4 % non-dim ZOH interval
end
```

- Riga 6: `Vc = 0.0777` e’ il valore di produzione (numero di Tsiolkovsky, `V_ref/c`, dai dati di Tabella 1 della traccia).
- Riga 7: stato iniziale con **tutte le componenti non banali** (posizione fuori asse, velocita’ orizzontale e verticale entrambe non nulle, massa 1). Serve per eccitare tutte e cinque le righe della RHS: un test con `vx = 0` non distinguerebbe un bug che azzerava la prima riga.

- Riga 8:  $u = [0.5; 1.0]$ , quindi  $\|u\| = 1.118$  e con  $m = 1$  l'accelerazione supera la gravita' (il veicolo decelera). Anche qui: entrambe le componenti non nulle, cosi' la norma nella riga di massa e' "viva".
- Riga 9:  $dt = 0.4$ . Attenzione: NON e' il  $dt$  di produzione, che vale  $tf\_nd/(N-1) = 0.0444$  (lo si legge, quel valore, in `descentDynamicsPerformanceTest.m` e in `gfoldLogMassTest.m`). E' circa 9 volte piu' grande, ed e' una scelta deliberata: serve a mantenere l'errore di troncamento di RK4 **ben sopra il rumore del riferimento** per tutti gli  $n\_sub$  testati. Se si usasse  $dt = 0.0444$ , con  $n\_sub = 8$  l'errore RK4 (che scala come  $h^4 = (dt/n\_sub)^4$ ) scenderebbe sotto la soglia di accuratezza di ode45 e il test di ordine misurerebbe il rumore del riferimento, non l'integratore. E' il principio base di ogni convergence study: bisogna restare nel **regime asintotico** ma **sopra il floor numerico**.

**Possibile domanda d'esame** – Perche' il test usa  $dt = 0.4$  invece del  $dt$  di produzione (0.0444)?

*Risposta:* Perche' il test di ordine ha bisogno che l'errore sia dominato dal troncamento di RK4, non dagli errori del riferimento. Con  $dt$  piccolo e  $n\_sub = 8$  il passo effettivo  $h = dt/n\_sub$  e' cosi' piccolo che l'errore RK4 ( $\sim h^4$ ) cade sotto la tolleranza di ode45 ( $1e-12$  relativa) e il rapporto  $err(n)/err(2n)$  perde significato: si misurerebbe rumore. Ingrandendo  $dt$  si tiene la successione degli errori in una finestra dove il termine  $C*h^4$  domina, e la stima  $\log_2$  del rapporto restituisce davvero 4. Il test di accuratezza e il test di ordine hanno esigenze opposte sul passo, e questo file risolve il conflitto scegliendo il passo per il test piu' esigente.

---

## TestClassSetup – addHm2ToPath (righe 12-17)

Identico a quello di `odeDescentTest.m`: due `filepath`s su `mfilename('fullpath')` risalgono da `tests/` alla cartella `HM2_powered_descent/`, e una `PathFixture` la aggiunge al path con `teardown` automatico a fine classe. Rende la suite lanciabile da qualunque working directory e non lascia residui nel path dell'utente.

---

## ode45Reference – metodo statico (righe 44-54)

```
opts = odeset('RelTol', 1e-12, 'AbsTol', 1e-14);
[~, Y] = ode45(@(t, x) ode_descent(x, rk4ZohTest.u, rk4ZohTest.Vc), ...
               [0 rk4ZohTest.dt], rk4ZohTest.x0, opts);
xRef = Y(end, :).';
```

- Riga 45: e' un metodo **Static**, quindi puo' essere invocato come `rk4ZohTest.ode45Reference()` senza istanza; per la stessa ragione dentro il corpo le costanti si leggono col nome della classe (`rk4ZohTest.u`, riga 50) e non con `testCase.u`.
- Riga 49: tolleranze **strette** (RelTol 1e-12, AbsTol 1e-14). Non e' pignoleria: il riferimento deve essere piu' accurato di cio' che si sta misurando di almeno qualche ordine di grandezza, altrimenti il test misura il riferimento.
- Righe 50-51: la closure `@(t, x) ode_descent(x, u, Vc)` **ignora t** e passa sempre lo stesso **u**. Questa e' la definizione operativa di ZOH: il controllo e' costante sull'intervallo, quindi la traiettoria "vera" contro cui confrontarsi e' il flusso della ODE autonoma con quel controllo congelato. Se il riferimento interpolasse il controllo (come fa la trascrizione trapezoidale, che e' PWL), il confronto sarebbe con un'altra dinamica e il test fallirebbe pur essendo RK4 corretto.
- Riga 52: si prende solo l'ultimo campione, `Y(end, :).'`, cioe' lo stato a  $t = dt$ : quello che RK4 deve riprodurre.
- Nota: `ode45Reference` viene ricalcolata da capo in ogni test che ne ha bisogno (righe 21 e 36), quindi due volte per run. E' una decina di millisecondi, del tutto irrilevante – ma tecnicamente sarebbe cachabile in una proprieta' di classe.
- **Onesta' sul riferimento:** per questa dinamica una soluzione analitica *esiste*. Con  $T$  costante,  $m(t) = m_0 - Vc*||T||*t$  e' lineare, quindi  $vx\_dot = Tx/m(t)$  si integra in forma chiusa ( $vx = vx_0 - (Tx/(Vc*||T||)) * \ln(m(t)/m_0)$ , una Tsiolkovsky per componente), e le posizioni seguono da `integral(ln)`, anch'esso elementare. Un riferimento analitico eliminerebbe del tutto il floor a 1e-12.



Il codice ha scelto ode45: piu' semplice da scrivere e da leggere, ma pone un limite inferiore artificiale all'accuratezza verificabile.

---

## testMatchesOde45Reference (righe 20-24)

```
xRef = rk4ZohTest.ode45Reference();
xRk4 = rk4_zoh(testCase.x0, testCase.u, testCase.dt, testCase.Vc, 8);
testCase.verifyEqual(xRk4, xRef, 'AbsTol', 1e-8);
```

- Riga 22: RK4 con **8 substep** su  $dt = 0.4$ , cioè  $h = 0.05$ .
- Riga 23: tolleranza  $1e-8$ . E' una soglia che vive nel mezzo fra due scale: l'errore del riferimento (ordine  $1e-12$ , fissato dalle tolleranze di ode45) e l'errore di troncamento globale di RK4 con  $h = 0.05$  (empiricamente sotto  $1e-8$ ). Passare significa: "l'RK4 a 8 substep e' accurato almeno a  $1e-8$ , e la differenza osservata e' dominata dall'RK4, non dal riferimento".
- **Perche' e' l'invariante giusto:** e' il test di *accuratezza* (il propagatore approssima davvero il flusso della ODE), ed e' la prima linea di difesa contro i bug grossolani – un **h** invece di **h/2** in **k2**, un pesi sbagliato, un segno. Ma da solo NON basta: un integratore rotto in modo sottile (per esempio con i pesi  $(1,1,1,1)/4$  invece di  $(1,2,2,1)/6$ ) puo' comunque restare entro  $1e-8$  su un intervallo cosi' corto, semplicemente perche' converge – solo piu' lentamente. Ecco perche' esiste il terzo test.

---

## testMassRowIsExact (righe 26-32)

```
xRk4 = rk4_zoh(testCase.x0, testCase.u, testCase.dt, testCase.Vc, 1);
mExpected = testCase.x0(5) ...
    - testCase.Vc*norm(testCase.u)*testCase.dt;
testCase.verifyEqual(xRk4(5), mExpected, 'AbsTol', 1e-14);
```

- Riga 29: **un solo substep** ( $n_{sub} = 1$ ), cioè il caso piu' brutale possibile: un unico passo RK4 su tutto  $dt = 0.4$ .
- Riga 30: il valore atteso e' la retta  $m(dt) = m_0 - Vc * ||u|| * dt$ .
- Riga 31: tolleranza  $1e-14$ , cioè *esattezza a meno di arrotondamenti*, e non un errore di troncamento.

**La matematica dietro.** Sotto ZOH sulla spinta, la riga di massa e'

$$m_{dot} = -Vc * ||u||$$

con  $u$  costante: il secondo membro **non dipende dallo stato**, e' una costante. Applicando RK4 a  $x_{dot} = a$  ( $a$  costante) si ha  $k_1 = k_2 = k_3 = k_4 = a$ , quindi

$$x + (h/6)*(a + 2a + 2a + a) = x + (h/6)*6a = x + h*a$$

cioe' RK4 degenera nel metodo di Eulero, che per una derivata costante e' la soluzione **esatta**. Non c'e' errore di troncamento sulla quinta riga, per qualunque  $n_{sub}$ , incluso  $n_{sub} = 1$ . Il test verifica proprio questo.

**Perche' e' l'invariante giusto.** Perche' la massa finale e' la *funzione obiettivo*: `fmincon` massimizza `m(tf)` (in `solve_zoh`, riga 646, si minimizza `-z(in_m)`). Se il propagatore introducesse anche solo un errore di troncamento sulla riga di massa, l'ottimizzatore lo sfrutterebbe: guadagnerebbe massa finale "fittizia" scegliendo profili di spinta che ingannano il discretizzatore invece di risparmiare propellente. Un test dedicato che pretende esattezza *strutturale* (non "vicinanza") sulla riga della funzione obiettivo e' quindi molto piu' di una verifica di accuratezza: e' una garanzia che il modello discreto non regali carburante.

**Il contrappunto che spiega tutta HM2.** Questa esattezza vale SOLO con la convenzione T-ZOH. Con la convenzione accelerazione-ZOH (`ode_descent_uacc.m`) la riga di massa diventa  $m_{dot} = -Vc * m * ||u||$ : dipende dallo stato, la soluzione e' un esponenziale, e RK4 **non** sarebbe piu' esatto. La mossa di GFOLD e'

proprio questa: passare a  $z = \ln m$ , per cui  $z_{\text{dot}} = -Vc \cdot ||u||$  torna costante – e infatti `lti_zoh.m` discretizza il sistema con un singolo `expm`, esattamente. Le tre varianti di HM2 sono tre modi diversi di preservare l'esattezza del canale di massa.

**Possibile domanda d'esame** – Perché testare la riga di massa separatamente, se il test contro `ode45` già confronta tutte e cinque le componenti? *Risposta:* Perché il test contro `ode45` verifica una *approssimazione* (entro  $1e-8$ ), mentre qui si verifica una *identità esatta* (entro  $1e-14$ , cioè arrotondamento macchina) valida per qualunque  $n_{\text{sub}}$ , incluso  $n_{\text{sub}} = 1$ . Sono affermazioni di forza diversa: la seconda è una proprietà strutturale del metodo applicato a quella specifica dinamica, e proteggerla è importante perché quella riga è la funzione obiettivo del NLP. Un errore di troncamento sulla massa si tradurrebbe in propellente immaginario, non in un piccolo errore numerico.

## testFourthOrderConvergence (righe 34-41)

```
xRef = rk4ZohTest.ode45Reference();
err = arrayfun(@(n) norm(rk4_zoh(testCase.x0, testCase.u, ...
    testCase.dt, testCase.Vc, n) - xRef), [1 2 4 8]));
order = log2(err(1:end-1) ./ err(2:end));
testCase.verifyGreaterThan(min(order), 3.5);
```

- Righe 37-38: si calcola l'errore (norma euclidea sul vettore di stato finale) per  $n_{\text{sub}} = 1, 2, 4, 8$ . Raddoppiare  $n_{\text{sub}}$  significa **dimezzare il passo**  $h = dt/n_{\text{sub}}$ .
- Riga 39: `order = log2(err(k)/err(k+1))` per le tre coppie consecutive.
- Riga 40: si pretende che il **minimo** dei tre esponenti stimati superi 3.5.

**La matematica dietro il test dell'ordine.** Un metodo a un passo di ordine  $p$  ha errore globale (sull'intervallo fissato  $dt$ ) della forma

$$e(h) = C * h^p + O(h^{(p+1)})$$

con  $C$  costante che dipende dalle derivate della soluzione ma **non** da  $h$ . Dimezzando il passo:

$$e(h/2) = C * (h/2)^p = C * h^p / 2^p = e(h) / 2^p$$

quindi il rapporto fra errori consecutivi è

$$e(h) / e(h/2) = 2^p \quad \Rightarrow \quad p = \log_2(e(h) / e(h/2))$$

Per RK4 classico  $p = 4$ , quindi il rapporto atteso è  $2^4 = 16$ : dimezzando il passo, l'errore deve calare di un fattore 16. È esattamente ciò che la riga 39 misura – il logaritmo in base 2 del rapporto restituisce direttamente l'ordine osservato.

**Perché è l'invariante giusto – anzi, il più importante dei tre.** Un test di accuratezza a passo singolo (“l'errore è sotto  $1e-8$ ”) verifica un *numero*. Il test di ordine verifica l'*algoritmo*. La differenza è sostanziale:

- Se si sbagliano i pesi della quadratura (per esempio  $(k_1+k_2+k_3+k_4)/4$  invece di  $(k_1+2k_2+2k_3+k_4)/6$ ), il metodo resta **consistente** e converge – ma cade a ordine 2. Su un intervallo corto con  $h$  piccolo l'errore assoluto può ancora passare sotto  $1e-8$  e il test di accuratezza **passerebbe lo stesso**. Il test di ordine no: misurerebbe un esponente vicino a 2 e fallirebbe.
- Analogamente, valutare  $k_4$  in  $x + 0.5*h*k_3$  invece che in  $x + h*k_3$  (un errore di battitura del tutto plausibile, righe 19-20 del sorgente sono quasi identiche) degrada l'ordine senza rompere nulla di visibile.

In altre parole: l'ordine di convergenza è la **firma** di un integratore. Se la firma è 4, allora i coefficienti di Butcher sono giusti, punto. È il test che distingue “il codice dà un numero plausibile” da “il codice implementa RK4”.

**Perché la soglia è 3.5 e non 4.** Ci sono quattro fonti di degrado che rendono l'esponente misurato leggermente diverso da 4, tutte legittime:

1. il riferimento non e' esatto: `xRef` ha il suo errore ( $\sim 1e-12$ ), che contamina soprattutto `err(4)`, il piu' piccolo – e quindi *abbassa* l'ultimo rapporto;
2. a `n_sub = 1` con `h = 0.4` non si e' pienamente nel regime asintotico: i termini  $O(h^5)$  non sono ancora trascurabili, quindi il primo rapporto puo' scostarsi;
3. la norma mescola cinque canali con costanti `C` diverse (e uno, la massa, con errore identicamente zero – vedi il test precedente);
4. rumore di arrotondamento in virgola mobile.

3.5 e' una soglia che accetta questo margine ma **rifiuta ordine 3** (e a maggior ragione ordine 2 o 1): e' abbastanza lasca da non essere fragile, abbastanza stretta da essere diagnostica.

**Un dettaglio spesso frainteso:** `verifyGreaterThan(min(order), 3.5)` chiede che **tutti e tre** i rapporti superino 3.5, non che la media lo faccia. E' una richiesta piu' forte: non basta che l'errore cali complessivamente, deve calare col tasso giusto a *ogni* raffinamento.

**Possibile domanda d'esame** – Se dimezzo il passo e l'errore cala di 16, che ordine ha il metodo? E se calasse di 4? *Risposta:*  $16 = 2^4$ , quindi ordine 4 (RK4 corretto). Un fattore  $4 = 2^2$  significherebbe ordine 2, cioe' un metodo consistente ma con i coefficienti sbagliati – tipicamente pesi errati nella combinazione finale o uno stadio valutato nel punto sbagliato. Un fattore 2 sarebbe Eulero esplicito. La relazione generale e'  $e(h)/e(h/2) = 2^p$ , da cui  $p = \log_2$  del rapporto, ed e' esattamente quello che calcola la riga 39.

---

## Cosa questa suite NON copre (onesta' sulla copertura)

- **Nessun confronto diretto fra `rk4_zoh` e `lti_zoh`.** Il brief lo cita come invariante desiderabile (“le due discretizzazioni devono essere consistenti”), ma **non esiste** in questo file – e va detto perche' la versione ingenua di quel test sarebbe *sbagliata*. `rk4_zoh` tiene costante la **spinta `T`**; `lti_zoh` discretizza il sistema log-massa in cui e' costante l'**accelerazione `u = T/m`**. Sono ZOH diversi: dentro l'intervallo, con `T` costante l'accelerazione *cresce* (la massa cala), mentre con `u` costante e' la spinta a *calare*. Le due traiettorie divergono a  $O(dt^2)$  e coincidono solo nel limite  $dt \rightarrow 0$ . Un test che pretendesse `rk4_zoh(x,u,dt) == [Abar Bbar cbar]*[...]` fallirebbe correttamente. La consistenza di `lti_zoh` e' invece verificata dove ha senso, in `gfoldLogMassTest.m`: contro la forma chiusa analitica delle matrici discrete (`testZohClosedForm`), contro un ode45 della *stessa* dinamica LTI (`testZohMatchesOde45`), e contro la replay nonlineare del canale di massa con `ode_descent_uacc` (`testMassRowConsistency`).
  - **Nessun test su `n_sub` non intero o nullo.** `rk4_zoh` non ha `arguments` block (scelta deliberata: hot loop). `n_sub = 0` darebbe `h = Inf` e il ciclo non girerebbe, restituendo `x` invariato: un fallimento silenzioso. La validazione e' al call site (`solve_zoh` in `main_task2.m`, riga 639, valida `n_sub` con `mustBeInteger`, `mustBePositive`).
  - **Nessun test di stabilita'.** Non si verifica che `dt` stia dentro la regione di stabilita' assoluta di RK4 per questa dinamica. Per il problema in esame non e' un rischio (la dinamica non e' stiff), ma non e' testato.
  - **Un solo punto di lavoro.** Tutti i test usano lo stesso `x0/u`. Un test parametrizzato (o `u = 0`, il caso di coast, dove la norma ha il cono) darebbe copertura piu' larga.
- 

## Possibili domande d'esame

**D: Quali invarianti verifica questa suite, e in che ordine di forza?** R: Tre, di forza crescente. (1) *Accuratezza:* RK4 a 8 substep riproduce una integrazione ode45 a tolleranze strette entro  $1e-8$  – prova che il propagatore approssima il flusso della ODE giusta (con `u` congelato, cioe' proprio la semantica ZOH). (2) *Esattezza strutturale sulla massa:* sotto T-ZOH `m_dot = -Vc*||u||` e' costante, quindi RK4 e' esatto sulla quinta riga per qualunque `n_sub` – e questo protegge la funzione obiettivo `max m(tf)`. (3) *Ordine di convergenza:* dimezzando il passo l'errore deve calare di  $2^4 = 16$ . Il terzo e' il piu' forte perche' verifica l'algoritmo (i coefficienti di Butcher) e non solo il numero prodotto.

**D: Spiega la matematica dietro il test di ordine.** R: L'errore globale di un metodo a un passo di ordine  $p$  sull'intervallo fissato  $dt$  si scrive  $e(h) = Ch^p + O(h^{p+1})$ , con  $C$  indipendente da  $h$ . Dimezzando  $h$ :  $e(h/2) = C(h/2)^p = e(h)/2^p$ , quindi  $\log_2(e(h)/e(h/2)) = p$ . Il test valuta l'errore per  $n_{\text{sub}} = 1, 2, 4, 8$  ( $h = dt/n_{\text{sub}}$ , quindi ogni passaggio dimezza  $h$ ), calcola i tre rapporti consecutivi e ne prende il  $\log_2$ : sono tre stime dell'ordine. Si pretende che il minimo sia  $> 3.5$ , cioè che ogni raffinamento mostri il tasso quartico. La soglia non è 4 esatto perché il riferimento ode45 ha il suo errore ( $\sim 1e-12$ ) che contamina l'errore più piccolo, perché a  $h = 0.4$  i termini di ordine superiore non sono ancora trascurabili, e per il rumore di arrotondamento.

**D: Perché RK4 è esatto sulla riga di massa? Vale anche per la variante GFOLD?** R: Perché sotto ZOH sulla spinta la derivata della massa è  $-Vc/|u|$ , una costante che non dipende dallo stato. Applicando RK4 a  $\dot{x} = a$  costante si ottiene  $k_1 = k_2 = k_3 = k_4 = a$  e l'aggiornamento diventa  $x + (h/6)6a = x + ha$ , che è la soluzione esatta. Nella variante GFOLD la convenzione è diversa: si tiene costante l'accelerazione  $u = T/m$ , quindi  $\dot{m} = -Vcm/|u|$  dipende dallo stato e la soluzione è esponenziale – RK4 non sarebbe più esatto. La contromossa è il cambio di variabile  $z = \ln m$ , che riporta  $\dot{z} = -Vc/|u|$  a essere costante: la dinamica diventa esattamente LTI e `lti_zoh.m` la discretizza con un solo `expm`, senza alcun errore di troncamento.

**D: Perché non c'è un test che confronta rk4\_zoh con lti\_zoh?** R: Perché non sarebbero d'accordo, e giustamente. Le due funzioni implementano due convenzioni ZOH diverse: `rk4_zoh` congela la spinta  $T$  (quindi dentro l'intervallo l'accelerazione  $T/m$  cresce man mano che la massa cala), `lti_zoh` congela l'accelerazione  $u = T/m$  (quindi è la spinta a calare). Le traiettorie sull'intervallo differiscono a ordine  $dt^2$  e convergono solo per  $dt \rightarrow 0$ . La consistenza sensata da verificare – ed è quella che `gfoldLogMassTest.m` verifica – è che `lti_zoh` coincida con l'integrazione ode45 della sua dinamica e che il suo canale di massa coincida con la replay nonlineare di `ode_descent_uacc`.

**D: Cosa fallirebbe se sbagliassi un peso di RK4, e quale test lo prenderebbe?** R: Il metodo resterebbe consistente (convergerebbe comunque a  $dt \rightarrow 0$ ) ma crollerebbe di ordine, tipicamente a 2 o 1. Il test contro ode45 potrebbe ancora passare, perché su un intervallo corto e con 8 substep anche un metodo di ordine 2 può finire sotto  $1e-8$ . Il test di ordine invece misurerebbe  $\log_2$  del rapporto degli errori e troverebbe circa 2 invece di 4, fallendo la soglia 3.5. È precisamente la ragione per cui in una suite di test di un integratore il convergence test non è ridondante rispetto al test di accuratezza: sono i due test *diversi* che servono.

# HM2\_\_powered\_\_descent/tests/gfoldLogMass'

## Ruolo del file nel progetto

E' la suite `matlab.unittest` che copre i **due kernel numerici della trascrizione GFOLD log-mass**: `ode_descent_uacc.m` (il right-hand side nonlineare con l'accelerazione tenuta costante) e `lti_zoh.m` (la discretizzazione ZOH esatta via matrice esponenziale). Sono i due file che il prototipo `proto_gfold_logmass.m` e la "variant (d)" dentro `main_task2.m` **condividono**: testarli una volta copre entrambi i chiamanti.

La suite ha 7 test: 4 sul RHS e 3 sulla discretizzazione. Presi insieme non verificano una cosa qualunque – verificano **la tesi centrale della trascrizione**, cioè che il cambio di variabile  $z = \ln(m)$ ,  $u = T/m$  rende la dinamica esattamente LTI e che quindi la mappa discreta e' esatta e non approssimata. Ogni test isola una delle tre proprietà che servono a quella tesi:

1. le righe di velocità **non contengono la massa** (`testAccelerationIsDirect`);
2. la riga di massa e' **lineare in  $z$**  – equivalentemente  $m'$  e' **proporzionale** `am(testMassFlowScalesWithMass)`;
3. con il cono lossless attivo (`sigma = |u|`), la riga discreta di  $z$  riproduce **esattamente** lo svuotamento nonlineare della massa (`testMassRowConsistency`).

Le proprietà 1 e 2 sono precisamente le due condizioni sotto cui **A** e **B** di `lti_zoh` sono matrici *costanti*. La 3 e' il ponte fra il modello convesso e la realtà nonlineare: e' quella che giustifica il replay `ode45` del prototipo.

Onesta' preliminare: la suite **non tocca l'ottimizzatore**. `solve_gfold_socp`, `solve_gfold_scvx`, il vincolo di spinta linearizzato, il glide-slope e la losslessness all'ottimo non sono testati qui (la losslessness e' solo *stampata* a runtime dal prototipo, `cone_gap`). Vengono testati i mattoni, non l'edificio.

---

## properties (Constant) (righe 6-9)

```
Vc = 0.0777;      % V_ref/c
dt = 0.0444;      % tf_nd/(N-1) with N = 50
```

- Righe 7-8: le due costanti del run nominale, **hard-coded e arrotondate a 3 cifre**. Non vengono ricalcolate dai dati di Tabella 1: se domani cambiassero `Isp`, `y0` o `tf`, il test continuerebbe a girare con i vecchi numeri. Questo **non e' un bug**, perche' tutte le identita' verificate sono algebriche e valgono per *qualunque* `Vc` e `dt` positivi: i valori servono solo a stare in un regime numerico realistico. Ma va saputo: la suite non e' una regression sui dati del problema.
- Per la cronaca, con i dati del prototipo (`y0 = 3000`, `g = 9.81`, `Isp = 225`, `tf = 38`, `N = 50`) i valori esatti sono `Vc = V_rif/(Isp*g0) ~ 0.0777` e `dt = tf/sqrt(y0/g)/49 ~ 0.0444`: coerenti.

---

## addHm2ToPath (righe 11-16)

```
hm2 = fileparts(fileparts(mfilename('fullpath')));
testCase.applyFixture(matlab.unittest.fixtures.PathFixture(hm2));
```

- Riga 13: `mfilename('fullpath')` dà il path di questo file (che sta in `tests/`); il doppio `fileparts` risale a `HM2_powered_descent/`.
- Riga 14: `PathFixture` aggiunge quella cartella al path **per la durata della classe** e la rimuove alla fine. E' cio' che permette di chiamare `ode_descent_uacc` e `lti_zoh` (che vivono nella cartella padre) senza sporcare il path globale dell'utente. Fixture di classe (`TestClassSetup`), quindi si paga una volta sola.

## testUaccDerivativeDefinition (righe 20-27)

```
x = [1; 2; 0.3; -0.4; 0.8];
uacc = [0.6; 0.8]; % ||u|| = 1
dx = ode_descent_uacc(x, uacc, testCase.Vc);
expected = [0.3; -0.4; 0.6; 0.8 - 1; -testCase.Vc * 0.8 * 1];
```

- Riga 24: chiama il RHS con stato  $[x; y; vx; vy; m] = [1; 2; 0.3; -0.4; 0.8]$  e accelerazione  $u = [0.6; 0.8]$ , scelta apposta con **norma unitaria** (terna pitagorica 3-4-5 scalata) cosi' che il termine  $|u|$  nel valore atteso sia esattamente 1 e non introduca errore di arrotondamento.
- Riga 25: il valore atteso e' **scritto a mano**, non ricalcolato con la stessa formula del codice sotto test – condizione necessaria perche' il test abbia senso. Componente per componente:
  - $dx(1) = vx = 0.3$ ,  $dx(2) = vy = -0.4$  – cinematica, banale;
  - $dx(3) = ux = 0.6$  – **niente** /m;
  - $dx(4) = uy - 1 = -0.2$  – gravita' non-dim pari a 1, diretta verso il basso. E' qui che si fissa la convenzione di segno:  $y$  cresce verso l'alto;
  - $dx(5) = -Vc * m * |u| = -Vc * 0.8 * 1$  – la riga di massa in **variabile m** (non  $z$ ):  $m' = -Vc * |T| = -Vc * m * |u|$ .
- Riga 26: `AbsTol 1e-15`, cioe' a livello di epsilon macchina. Legittimo: sono somme e prodotti esatti in floating point, non c'e' integrazione.

**Possibile domanda d'esame** – Perche' il RHS prende `uacc` e non la spinta  $T$ ? *Risposta:* Perche' e' la convenzione ZOH nativa del GFOLD: quello che si tiene costante sull'intervallo e' l'**accelerazione**  $u = T/m$ , non la spinta. Con  $u$  costante,  $T(t) = m(t) * u$  decresce durante l'intervallo insieme alla massa. E' esattamente questa scelta che rende  $v' = u + g$  indipendente dalla massa e quindi il sistema LTI. `ode_descent.m` (le altre trascrizioni) tiene costante  $T$  e quindi ha  $v' = T/m(t)$ , che dentro l'intervallo varia.

## testAccelerationIsDirect (righe 29-37)

```
dx1 = ode_descent_uacc([0;1;0;0;0.9], uacc, testCase.Vc);
dx2 = ode_descent_uacc([0;1;0;0;0.3], uacc, testCase.Vc);
testCase.verifyEqual(dx1(3:4), [0.5; 0.2], 'AbsTol', 1e-15);
testCase.verifyEqual(dx2(3:4), [0.5; 0.2], 'AbsTol', 1e-15);
```

- Righe 33-36: stesso `uacc`, **due masse molto diverse** (0.9 e 0.3, cioe' 1800 kg e 600 kg), stesse derivate di velocita'. E' il test **strutturale** della trascrizione: dimostra che le righe 3-4 del RHS non dipendono da  $x(5) = m$ .
- Perche' e' la proprieta' giusta: nel sistema  $xi' = A * xi + B * w + c$ , la colonna di  $B$  che moltiplica  $u$  e' **costante**. Se  $v'$  dipendesse dalla massa,  $B$  sarebbe  $B(xi)$  e il sistema non sarebbe LTI: nessun **expm** unatum, nessuna discretizzazione esatta, e il SOCP diventerebbe un NLP. Questo test e' quindi il guardiano della premessa su cui poggia tutta la variante (d).

- Il commento (righe 30-31) lo dice esplicitamente: “unlike ode\_descent.m where the acceleration is  $T/m$ ”.

## testMassFlowScalesWithMass (righe 39-46)

```
uacc = [0; 1]; % ||u|| = 1
dxA = ode_descent_uacc([0;1;0;0;1.0], uacc, vc); % m = 1.0
dxB = ode_descent_uacc([0;1;0;0;0.5], uacc, vc); % m = 0.5
% attesi: -Vc*1.0 e -Vc*0.5
```

- Righe 42-45: la portata di massa **scala linearmente con la massa**:  $m' = -Vc*m*|u|$ . Con  $|u| = 1$ , dimezzando  $m$  si dimezza  $m'$ .
- **Perche' questa e' la proprieta' giusta**: e' l'enunciato “in variabile  $m$ ” del fatto che la riga di massa e' **lineare in  $z = \ln(m)$** . Infatti  $m'/m = -Vc*|u|$  costante equivale a  $d(\ln m)/dt = -Vc*|u|$ , cioe'  $z' = -Vc*|u|$ : un'equazione **senza stato a destra**, che dopo l'introduzione dello slack diventa  $z' = -Vc*sigma$ , la quinta riga di B in lti\_zoh ( $B(5,3) = -Vc$ ). Il commento alla riga 40 lo scrive nero su bianco.
- Insieme, testAccelerationIsDirect e testMassFlowScalesWithMass **sono** la verifica che il cambio di variabile linearizza esattamente la dinamica: uno toglie la massa dal denominatore delle velocita', l'altro la toglie dalla riga di massa.

**Possibile domanda d'esame** – Perche' il logaritmo? Non bastava usare  $m$  come stato? *Risposta*: No. Con  $m$  come stato la riga di massa e'  $m' = -Vc*m*|u|$ , che e' **bilineare** in (stato, controllo): non e' LTI. Passando a  $z = \ln(m)$  la stessa equazione diventa  $z' = -Vc*|u|$ , che non contiene lo stato: la riga di massa e' lineare e completamente disaccoppiata. Il logaritmo e' esattamente la trasformazione che uccide quella bilinearita' – e in piu' rende lineare anche il costo, perche' massimizzare  $z_N$  equivale a massimizzare  $m_N = \exp(z_N)$ .

## testBallisticCoast (righe 48-52)

```
dx = ode_descent_uacc([0.5;1;0.2;-0.1;0.9], [0;0], testCase.Vc);
testCase.verifyEqual(dx, [0.2; -0.1; 0; -1; 0], 'AbsTol', 1e-15);
```

- Riga 50-51: caso limite  $u = 0$ . Attesi: caduta libera ( $v' = [0; -1]$ , cioe' solo gravita') e **portata di massa nulla** ( $dx(5) = 0$ ).
- Verifica due cose non banali: (i) il termine costante  $c = [0;0;0;-1;0]$  di lti\_zoh e' coerente con il RHS nonlineare; (ii)  $|u| = 0$  non produce NaN o divisioni per zero nel calcolo di  $umag$  ( $\sqrt{0+0} = 0$ , poi moltiplicato: nessuna patologia). Il coast e' l'arco centrale della soluzione ottima (profilo max-coast-max secondo il README), quindi non e' un caso accademico: e' meta' della traiettoria.

## testZohClosedForm (righe 55-69)

```
[Abar, Bbar, cbar] = lti_zoh(h, vc);
A_exp = eye(5); A_exp(1,3) = h; A_exp(2,4) = h;
B_exp = [h^2/2, 0, 0;
         0, h^2/2, 0;
         h, 0, 0;
         0, h, 0;
         0, 0, -vc*h];
c_exp = [0; -h^2/2; 0; -h; 0];
```

- Righe 59-65: i valori attesi sono le **formule del moto uniformemente accelerato**, scritte a mano. Il test confronta l'output di `expm` (van Loan) con la forma chiusa.
- **Da dove viene la forma chiusa.** La matrice `A` di `lti_zoh` ha solo  $A(1,3) = 1$  e  $A(2,4) = 1$ , quindi  $A^2 = 0$  (**nilpotente**: la riga 3 e la riga 4 di `A` sono nulle). La serie esponenziale si tronca dopo due termini:

$$e^{A \cdot h} = I + A \cdot h$$

e l'integrale di convoluzione:

$$\int_0^h e^{A \cdot s} ds = h \cdot I + (h^2/2) \cdot A$$

da cui

$$\text{Abar} = I + A \cdot h$$

$$\text{Bbar} = h \cdot B + (h^2/2) \cdot A \cdot B$$

$$\text{cbar} = h \cdot c + (h^2/2) \cdot A \cdot c$$

Facendo i prodotti: `A*B` ha 1 in posizione (1,1) e (2,2) – cioè l'accelerazione che si integra due volte in posizione – e `A*c` = [0; -1; 0; 0; 0]. Si ottiene esattamente `B_exp` e `c_exp`:  $x_{k+1} = x_k + vx_k \cdot h + ux_k \cdot h^2/2$ ,  $y_{k+1} = y_k + vy_k \cdot h + (uy - 1) \cdot h^2/2$ ,  $vx_{k+1} = vx_k + ux_k \cdot h$ ,  $vy_{k+1} = vy_k + (uy - 1) \cdot h$ ,  $z_{k+1} = z_k - Vc \cdot \sigma \cdot h$ . Il  $-h^2/2$  e il  $-h$  in `c_exp` sono il contributo della gravità unitaria.

- **Perche' e' la proprieta' giusta da testare:** certifica che la mappa discreta non e' un'approssimazione ma la **soluzione esatta** dell'intervallo. Se `A` non fosse nilpotente, `Abar` conterrebbe tutta la serie e non ci sarebbe forma chiusa con cui confrontarsi. E' anche un test dell'**implementazione** del trucco di van Loan (indici delle sotto-matrici, `E(1:5,1:5)`, `E(1:5,6:8)`, `E(1:5,9)`, e il blocco `zeros(4,9)` che corrisponde alle 3 righe del controllo piu' 1 riga della costante): un errore di slicing verrebbe preso qui.
- Riga 66-68: `AbsTol` `1e-12`. Piu' lasca di `1e-15` perche' `expm` fa scaling-and- squaring e non e' esatta a bit.

**Possibile domanda d'esame** – Perche' `lti_zoh` costruisce una matrice 9x9? *Risposta:* Perche' il sistema e' affine, non lineare puro: c'e' il termine costante di gravita' `c`. Il trucco di van Loan aumenta lo stato con il controllo (3 componenti) e con la costante (1 componente),  $5 + 3 + 1 = 9$ , e sfrutta l'identita' `expm([A B c; 0 0 0]*h) = [e^{A h}, integrale*B, integrale*c; 0, I]`: le sotto-matrici in alto a destra **sono** `Bbar` e `cbar`. Così `Abar`, `Bbar`, `cbar` escono da **un solo expm**, senza integrare nulla numericamente.

## testZohMatchesOde45 (righe 71-83)

```
xi0 = [0.3; 1; -0.2; -0.6; 0];           % [x;y;vx;vy;z]
w    = [0.4; 1.1; 1.3];                 % [ux;uy;sigma]
rhs = @(~, xi) [xi(3); xi(4); w(1); w(2)-1; -vc*w(3)];
[~, XI] = ode45(rhs, [0, h], xi0, opts);
testCase.verifyEqual(Abar*xi0 + Bbar*w + cbar, XI(end,:), 'AbsTol', 1e-9);
```

- Righe 76-82: un passo della mappa discreta contro un'integrazione `ode45` della **stessa** dinamica LTI con `w` costante sull'intervallo. Tolleranze `RelTol` `1e-12` / `AbsTol` `1e-14` sull'integratore, confronto a `1e-9` – che e' il floor realistico di `ode45`, non un margine generoso.
- Nota che `w(3) = sigma = 1.3` e' **volutamente maggiore** di  $|u| = \sqrt{0.4^2 + 1.1^2} \sim 1.17$ : qui il cono e' **inattivo**. Il test verifica la mappa discreta del **modello rilassato**, non della fisica. E' corretto così: deve certificare che `Abar/Bbar/cbar` riproducono la dinamica *che il SOCP crede*, con `sigma` come ingresso indipendente.
- **Debolezza dichiarata:** la RHS della riga 78 e' **riscritta a mano dentro il test**, non presa da un file sorgente (nessun file della repo espone la dinamica LTI in coordinate `xi`; `ode_descent_uacc` lavora in `m`, non in `z`). Se ci fosse un errore di segno *identico* in `lti_zoh.m` e in questa `lambda`, il test passerebbe lo



stesso. La copertura e' salvata dal test precedente (`testZohClosedForm`), che confronta con formule chiuse **indipendenti** dal codice: i due test insieme sono ridondanti nel modo giusto.

---

## testMassRowConsistency (righe 85-97)

```
uacc = [0.5; 1.0];    umag = norm(uacc);
[~, X] = ode45(@(~,x) ode_descent_uacc(x, uacc, vc), [0 h], [0;1;0;0;1], opts);
z_nl  = log(X(end,5));           % log-massa nonlineare
z_lti = -vc * umag * h;          % predizione LTI con sigma = ||u||
testCase.verifyEqual(z_nl, z_lti, 'AbsTol', 1e-9);
```

E' il test **piu' importante della suite**, e va capito bene perche'.

- Righe 92-94: si integra la dinamica **nonlineare in massa** ( $m' = -Vc*m*|u|$ , quindi  $m(h) = \exp(-Vc*|u|*h)$ ) partendo da  $m(0) = 1$ , poi si prende il logaritmo della massa finale.
- Riga 95: si calcola la predizione del **modello discreto LTI** per la variazione di  $z$ , ponendo  $\sigma = |u|$ .
- Riga 96: devono coincidere a  $1e-9$  (floor ode45).

**Cosa certifica davvero.** Certifica il **ponte** fra il modello convesso e la dinamica vera: *se e solo se il cono e' attivo* ( $\sigma = |u|$ ), la quinta riga della mappa LTI riproduce **esattamente** lo svuotamento nonlineare della massa. Il partire da  $m(0) = 1$  (quindi  $z(0) = 0$ ) rende il confronto diretto:  $z\_nl = \ln(m(h)) - 0 = z(h) - z(0)$ .

Ed e' esattamente la proprieta' giusta, perche' e' l'unica cerniera fragile dell'intera trascrizione. Ragionamento: le righe di posizione e velocita' del modello discreto sono esatte **sempre** (non contengono la massa, e la mappa e' la soluzione in forma chiusa). Quindi **l'unica** sorgente possibile di discrepanza fra modello discreto e realta' nonlineare e' la riga di massa, e la riga di massa e' esatta **se e solo se**  $\sigma = |u|$ . Da qui due conseguenze pratiche, entrambe visibili nel prototipo:

- il replay ode45 (`fwd_integrate_uacc`, `proto_gfold_logmass.m` righe 241-254) torna al floor dell'integratore **solo** perche' il rilassamento e' lossless;
- il "drift" di massa modello-vs-replay (`dmf`, riga 64 del prototipo) e il `cone_gap` (riga 71) misurano **la stessa cosa**.

Se il rilassamento non fosse lossless, il modello brucerebbe piu' propellente del replay ( $\sigma > |u|$ ), il costo sarebbe pessimistico e questo test sarebbe il sintomo. Il test **non** dimostra che all'ottimo il cono e' attivo (quello e' un risultato di ottimalita', via PMP): dimostra che **se** lo e', la trascrizione e' esatta.

**Possibile domanda d'esame** – Questo test verifica la losslessness del rilassamento? *Risposta:* No, e la distinzione e' importante. Verifica la **conseguenza** della losslessness, cioe' che *ponendo*  $\sigma = |u|$  la mappa discreta della massa e' esatta. Che  $\sigma = |u|$  valga **all'ottimo** e' un risultato di teoria del controllo ottimo (PMP:  $u$  compare in Hamiltoniana solo via `lambda_v . u`, quindi il minimo su  $\{|u| \leq \sigma\}$  sta sul bordo), e nel codice viene solo **verificato a posteriori** dal `cone_gap` stampato dal prototipo. Nessun test unitario controlla questa proprieta'.

---

## Cosa NON e' coperto (onesta')

1. **Nessun test sull'ottimizzatore.** `solve_gfold_socp` e `solve_gfold_scvx` sono funzioni *locali* dello script `proto_gfold_logmass.m` (e duplicate in `main_task2.m`): non sono raggiungibili dalla suite. Il vincolo di spinta linearizzato ( $\sigma \leq T_{\max} * e^{-z_{\text{ref}}} * (1 - (z - z_{\text{ref}}))$ ), il glide-slope, la trust region e il ratio test **non hanno test**.
2. **Nessun test della losslessness all'ottimo** (`cone_gap`): e' solo una stampa a runtime.
3. **Nessun test di conservativita' della tangente**, cioe' che il bound linearizzato implichi davvero  $|T| \leq T_{\max}$  (proprieta' garantita dalla convessita' di  $\exp(-z)$ , ma mai verificata numericamente).

4. Le costanti  $V_c/dt$  sono arrotondate e hard-coded: la suite non e' una regression sui dati di Tabella 1.
  5. La dinamica LTI in coordinate xi e' duplicata nel test (riga 78) invece di essere importata da un sorgente: rischio teorico di errori correlati, mitigato da `testZohClosedForm`.
  6. Nessun performance test (`matlab.perftest`) per questi kernel, a differenza di quanto prevedono le convenzioni della repo per altre feature.
- 

## Possibili domande d'esame

**D: I 7 test cosa dimostrano, messi insieme?** R: Dimostrano la tesi su cui poggia l'intera variante GFOLD: che il cambio di variabile rende la dinamica **esattamente LTI** e quindi la discretizzazione ZOH **esatta**. `testAccelerationIsDirect` dimostra che le righe di velocita' non contengono la massa (quindi B e' costante); `testMassFlowScalesWithMass` dimostra che  $m'/m$  non dipende dallo stato (quindi la riga di  $z$  e' lineare); `testZohClosedForm` dimostra che `expm` restituisce le formule del moto uniformemente accelerato (nessun errore di troncamento, perche'  $A^2 = 0$ ); `testZohMatchesOde45` chiude il cerchio contro un integratore; `testMassRowConsistency` dimostra che con il cono attivo il modello convesso e la dinamica nonlineare coincidono.

**D: Perche' testMassRowConsistency e' il test critico?** R: Perche' isola l'unico punto in cui il modello discreto puo' mentire. Le righe di posizione/velocita' sono esatte per costruzione (soluzione in forma chiusa di un moto uniformemente accelerato, con  $u$  costante per ipotesi ZOH). L'unica approssimazione concettuale dell'intera trascrizione e' aver sostituito  $|u|$  con lo slack  $\sigma$  nell'equazione della massa. Questo test verifica che, quando lo slack e' saturo ( $\sigma = |u|$ , cioe' quando il rilassamento e' lossless), la sostituzione non costa nulla:  $z(h) - z(0) = -V_c * |u| * h$  a precisione di integratore.

**D: Perche' testZohMatchesOde45 usa  $\sigma = 1.3 > |u| \sim 1.17$ ?** R: Perche' quel test riguarda la **mappa discreta del modello rilassato**, dove  $\sigma$  e' un ingresso indipendente da  $u$ . Deve funzionare per qualunque  $w$  ammissibile, cono attivo o no. Il caso "cono attivo" e' invece il soggetto specifico di `testMassRowConsistency`. Sono due domande diverse: "la mia matrice discreta e' giusta?" e "il mio modello coincide con la fisica quando il rilassamento e' stretto?".

**D: Perche' le tolleranze sono cosi' diverse (1e-15 contro 1e-9)?** R: I test su `ode_descent_uacc` (righe 20-52) confrontano una manciata di somme e prodotti in floating point con il loro valore atteso: l'errore ammissibile e' l'epsilon macchina, 1e-15. `testZohClosedForm` usa 1e-12 perche' `expm` implementa scaling-and-squaring e introduce un errore di qualche ulp. I due test che integrano (`testZohMatchesOde45`, `testMassRowConsistency`) sono limitati dal controllo di passo di `ode45`: anche con `RelTol 1e-12 / AbsTol 1e-14` il floor realistico e' 1e-9. Stringere di piu' produrrebbe test intermittenti.

**D: Se volessi testare davvero la losslessness, come faresti?** R: Servirebbe estrarre `solve_gfold_socp` in un file autonomo (oggi e' una funzione locale dello script) e scrivere un test che: (a) risolve il SOCP su un caso piccolo ( $N$  basso), (b) verifica  $\max_k |\sigma_k - |u_k|| < 1e-8$ , (c) verifica che la spinta ricostruita  $T = m * u$  rispetti  $|T| \leq T_{max}$  a tutti i nodi, (d) confronta il costo ottimo con quello di una delle altre trascrizioni entro qualche kg. Sarebbe un test di integrazione (richiede YALMIP + ECOS), quindi andrebbe marcato come tale e saltato con grazia se i pacchetti non ci sono.

# HM2\_\_powered\_\_descent/tests/descentDynam

## Ruolo del file nel progetto

Questo file non verifica *correttezza*: verifica *velocita'*. E' l'unico benchmark della repo HM2 e misura le tre funzioni che stanno sul cammino critico dell'NLP: `ode_descent` (la RHS continua del punto materiale), `rk4_zoh` (il propagatore a passo fisso usato dai defect ZOH) e una replica `ode45` di un singolo intervallo ZOH alle tolleranze usate dai forward-integrate di verifica.

Il motivo per cui esiste e' una scelta di design che attraversa tutta la repo. In HM2 il problema di powered descent e' trascritto come NLP e risolto con `fmincon` (SQP). Il vincolo non lineare `trap_nonlcon` (`main_task2.m` righe 759-781) chiama `ode_descent` una volta per nodo, e `zoh_nonlcon` (righe 783-803) chiama `rk4_zoh` una volta per intervallo. `fmincon` non riceve gradienti analitici (`fmincon_opts`, righe 731-735, non imposta `SpecifyConstraintGradient`), quindi ricostruisce lo Jacobiano dei vincoli per differenze finite: con  $N = 50$  nodi il vettore delle variabili ha  $7*N = 350$  componenti, e ogni Jacobiano costa ~351 valutazioni di `nonlcon`. Moltiplicando, `ode_descent` viene chiamata dell'ordine di  $1e7$  volte in una singola soluzione. E' un hot loop nel senso stretto del termine.

Da qui la regola dichiarata in `CLAUDE.md` e ripetuta come commento nei sorgenti: le RHS e i propagatori **non hanno blocchi arguments** di validazione (`ode_descent.m` righe 10-11, `rk4_zoh.m` righe 12-13), perche' il costo della validazione verrebbe pagato milioni di volte contro un corpo funzione che fa poche operazioni in virgola mobile. La validazione vive nei *boundary helper* chiamati una volta per run (`nondim`, `solve_*`, `fwd_integrate_*`, `lti_zoh`). Questo benchmark e' il presidio di quella scelta: se qualcuno "ripulisce" `ode_descent` aggiungendo validazione, o sostituisce l'indicizzazione esplicita con qualcosa di piu' elegante ma piu' lento, `runperf` deve mostrare il degrado.

Onesta': il file **non** misura `ode_descent_uacc.m` ne' `lti_zoh.m`, che pure fanno parte della famiglia. Vedremo piu' avanti perche' l'esclusione e' difendibile, ma va detto subito che la copertura di performance e' parziale.

---

## Il framework: `matlab.unittest.TestCase` vs `matlab.perftest.TestCase`

Prima del codice, il pezzo di teoria che l'orale puo' chiedere.

`matlab.unittest.TestCase` e' la classe base dei test funzionali: ogni metodo `Test` esegue del codice e fa asserzioni (`verifyEqual`, `verifyLessThan`, ...). Il framework riporta pass/fail. Il tempo di esecuzione viene misurato, ma solo come informazione accessoria: e' il tempo totale del metodo, setup incluso, misurato una volta sola.

`matlab.perftest.TestCase` (riga 1) eredita da `matlab.unittest.TestCase` e aggiunge la nozione di **confine di misura** (*measurement boundary*). Il punto e' che il tempo di un metodo di test *nella sua interezza* e' inutile come metrica: comprende la costruzione dell'oggetto test, il `TestMethodSetup`, l'allocazione delle variabili, le asserzioni finali. Se il codice che ti interessa dura 60 nanosecondi e il setup ne dura 200 microsecondi, stai misurando il setup. Serve un modo per dire al framework "cronometra **solo** questa regione".

Ci sono due API per farlo:

1. `startMeasuring` / `stopMeasuring`. Racchiudono esplicitamente la regione di interesse:

```
function testSomething(testCase)
    payload = buildBigThing();    % NON misurato
    testCase.startMeasuring();
    result = functionUnderTest(payload);
    testCase.stopMeasuring();
    testCase.verifyEqual(result, expected);    % NON misurato
end
```

Il framework esegue il metodo per intero, ma registra solo il tempo fra **start** e **stop**. Accettano un **label** opzionale per definire piu' confini distinti nello stesso metodo.

2. **keepMeasuring** (introdotta in R2018b), che e' quella usata da questo file. Si usa come condizione di un **while**:

```
while testCase.keepMeasuring
    <codice da misurare>
end
```

La differenza non e' cosmetica. **startMeasuring/stopMeasuring** cronometrano **una** esecuzione della regione: se quella regione dura meno della risoluzione del timer, il numero e' rumore puro. **keepMeasuring** invece dice al framework: *itera questo ciclo quante volte ti servono per ottenere una misura accurata*. Il framework decide autonomamente il numero di ripetizioni del **while** e normalizza il risultato. E' l'API giusta per codice velocissimo, ed e' esattamente il caso di **ode\_descent**.

## Perche' misurare solo la regione di interesse e' l'unico modo

Detto in modo operativo: qualsiasi tempo misurato e' **t\_setup + t\_utile**. Se vuoi rilevare una regressione del 20% su **t\_utile**, ma **t\_setup >> t\_utile**, il segnale che cerchi e' sepolto sotto una costante additiva enorme. Il rapporto segnale/rumore di un benchmark e'

$$\text{SNR} \sim t_{\text{utile}} / (t_{\text{setup}} + \text{jitter\_del\_sistema\_operativo})$$

e l'unica leva che hai sul numeratore e' escludere il setup dal confine di misura. In questo file l'esclusione avviene su due livelli: il **TestMethodSetup** (righe 29-35) sta fuori dal **while** per costruzione del framework, e le variabili locali (**x**, **u**, **vc**, **h**, **opts**) sono preparate prima del **while** dentro ciascun metodo.

## Come runperf fa statistica (e perche' un timeit non basta)

**runperf('descentDynamicsPerformanceTest')** costruisce un *time experiment* di tipo **limitingSamplingError**. I default documentati sono:

| Parametro                    | Default | Significato                                    |
|------------------------------|---------|------------------------------------------------|
| <b>NumWarmups</b>            | 5       | esecuzioni di riscaldamento, scartate          |
| <b>MinSamples</b>            | 4       | campioni minimi raccolti                       |
| <b>MaxSamples</b>            | 256     | tetto ai campioni                              |
| <b>RelativeMarginOfError</b> | 0.05    | obiettivo: 5% di margine di errore sulla media |
| <b>ConfidenceLevel</b>       | 0.95    | livello di confidenza                          |

La logica e':

- **Warm-up (5 esecuzioni scartate)**. La prima esecuzione di un file **.m** in MATLAB paga il parsing e la compilazione JIT, e i dati non sono ancora in cache. Includere quelle esecuzioni nella statistica falserebbe la media verso l'alto. Le si esegue e le si butta.
- **Campionamento adattivo**. Il framework continua a raccogliere campioni finche' la media campionaria non e' nota entro il 5% di margine relativo al 95% di confidenza, oppure finche' non arriva a 256 campioni. Se la macchina e' rumorosa (altri processi, throttling termico), servono piu' campioni; se e' quieta, il framework si ferma prima. E' un criterio di **arresto statistico**, non un conteggio fisso.

- **Riepilogo.** `sampleSummary(results)` produce una tabella con media, mediana, deviazione standard, min e max dei campioni. La **mediana** e' la statistica che conviene leggere per confrontare due run: e' robusta agli outlier, e gli outlier in un benchmark su un sistema operativo general purpose sono la norma (un context switch, il garbage collector di un altro processo, un interrupt).

Perche' un singolo `timeit` non basta: `timeit` restituisce **un** numero (esso stesso una mediana di ripetizioni interne, per inciso), senza margine di errore associato e senza criterio di arresto. Non ti dice se la differenza fra 61 ns e 66 ns e' una regressione o rumore. Un time experiment ti da' una distribuzione con un intervallo di confidenza dichiarato: e' l'unica base su cui puoi affermare che due misure differiscono davvero. Inoltre `timeit` non si integra con il resto della suite `matlab.unittest` (fixture, parametri, selezione dei test), che qui serve.

## classdef e proprieta' (righe 1-20)

```
classdef descentDynamicsPerformanceTest < ...
    matlab.perftest.TestCase
    properties (Constant)
        Vc = 0.0777      % V_ref/c (Table 1 data)
        dt = 0.0444      % one ZOH interval, non-dim
    end
    properties
        x0
        u0
    end
    properties (TestParameter)
        nSub = struct('one', 1, 'two', 2, 'eight', 8)
    end
```

- Riga 1: eredita da `matlab.perftest.TestCase`, non da `matlab.unittest.TestCase`. E' questa riga a rendere disponibili `keepMeasuring` / `startMeasuring` e a far si' che `runperf` accetti la classe. Nota che la classe resta comunque un `matlab.unittest.TestCase` per ereditarieta', quindi `runtests` la esegue lo stesso – eseguirebbe i metodi una volta sola, trattando i `verifySize` come normali asserzioni.
- Righe 8-11: le due costanti fisiche del benchmark, **hard-coded**. Verifichiamole contro la vera non-dimensionalizzazione (`main_task2.m`, `nondim`, righe 203-228):

```
L_ref = y0 = 3000 m
g_ref = 9.81 m/s^2
t_ref = sqrt(L_ref/g_ref) = sqrt(3000/9.81) = 17.4874 s
V_ref = sqrt(g_ref*L_ref) = sqrt(9.81*3000) = 171.55 m/s
c      = lsp*g0 = 225*9.80665 = 2206.50 m/s
Vc     = V_ref/c = 171.55/2206.50 = 0.077749
```

che arrotonda a 0.0777: il valore torna. E per `dt` (con `tf` = 38 s, `main_task2.m` riga 35, e `N` = 50, riga 36):

```
tf_nd = 38/17.4874 = 2.17299
dt     = tf_nd/(N-1) = 2.17299/49 = 0.044347
```

Attenzione: 0.044347 arrotondato a quattro decimali fa **0.0443**, non 0.0444. La costante hard-coded e' il risultato di un doppio arrotondamento (0.044347 -> 0.04435 -> 0.0444) ed e' alta di circa lo 0.1%. Per un benchmark e' innocuo – `dt` fissa solo il punto di lavoro, non un risultato fisico – ma va detto: il valore *non* e' l'arrotondamento corretto di `tf_nd/(N-1)`. Il commento “N = 50” e' invece esatto. **Limite onesto:** sono duplicati. La sorgente di verita' e' `nondim`, che qui non viene chiamata. Se qualcuno cambiasse lo schema di scale di riferimento (per esempio `L_ref` = `x0` invece di `y0`), il benchmark continuerebbe a girare felicemente su costanti stantie, misurando un punto di lavoro che non esiste piu' nel problema. Non e' un errore – e' un rischio di manutenzione, e va dichiarato.

- Righe 13-16: `x0` e `u0` sono proprietà normali (non `Constant`) perché vengono riscritte dal `TestMethodSetup` prima di ogni metodo.
- Righe 18-20: `nSub` è una `TestParameter`. Il framework **moltiplica** il metodo di test che la dichiara come argomento, generando tre casi indipendenti (**one**, **two**, **eight**). Non è zucchero sintattico: ogni caso ottiene la sua serie di campioni e la sua statistica, quindi si legge il *costo in funzione del numero di substep*. Vedi la sezione su `testRk4ZohPropagation` per cosa ci si compra con questo.

## addHm2ToPath (righe 22-27)

```
methods (TestClassSetup)
    function addHm2ToPath(testCase)
        hm2 = fileparts(fileparts(mfilename('fullpath')));
        testCase.applyFixture( ...
            matlab.unittest.fixtures.PathFixture(hm2));
    end
end
```

- Riga 22: `TestClassSetup` gira **una volta per classe**, non per metodo. Giusto: mettere il path è un'operazione idempotente e costosa, non ha senso ripeterla.
- Riga 24: `mfilename('fullpath')` dà il path di questo file (`.../HM2_powered_descent/tests/descentDynamicsPerform`). Il primo `fileparts` sale a `.../HM2_powered_descent/tests`, il secondo a `.../HM2_powered_descent`. Da lì `ode_descent.m` e `rk4_zoh.m` sono visibili. Il doppio `fileparts` è l'idioma standard per “la cartella padre della cartella dei test”.
- Riga 25: `PathFixture` non è un semplice `addpath`. È una *fixture*: il framework la installa prima e la **disinstalla dopo**, ripristinando il path originale anche se un test fallisce o solleva un'eccezione. È la ragione per cui non si usa `addpath` a mano – quello lascerebbe la sessione MATLAB sporca.

## setupState (righe 29-35)

```
methods (TestMethodSetup)
    function setupState(testCase)
        % Mid-descent state, near-hover control
        testCase.x0 = [0.1; 0.5; -0.2; -0.4; 0.8];
        testCase.u0 = [0.1; 0.9];
    end
end
```

- Righe 29-30: `TestMethodSetup` gira prima di **ogni** metodo di test. Sta fuori dal confine di misura per costruzione, quindi il suo costo non entra nel benchmark. Anche se qui è banale (due assegnazioni), la struttura è quella corretta: se domani lo stato iniziale richiedesse un calcolo, non si dovrebbe toccare nulla.
- Riga 32: lo stato è `[x; y; vx; vy; m]` non-dim. Ridimensionalizzando con le scale sopra:

```
x = 0.1 * 3000 = 300 m
y = 0.5 * 3000 = 1500 m
vx = -0.2 * 171.55 = -34.3 m/s
vy = -0.4 * 171.55 = -68.6 m/s
m = 0.8 * 2000 = 1600 kg
```

cioè effettivamente un punto a meta' discesa, in caduta, con circa il 20% di propellente già bruciato. Il commento non mente.

- Riga 33:  $u_0 = [0.1; 0.9]$  e' la spinta non-dim  $[T_x; T_y]$ , con  $T_{ref} = m_0 * g = 2000 * 9.81 = 19620$  N. Verifichiamo il commento “near-hover”: in unita' non-dim la gravita' vale 1 (vedi `ode_descent.m` riga 14, il -1 nella riga di  $v_y$ ), quindi la spinta di hover e'  $T_y = m = 0.8$ . Qui  $T_y = 0.9$ , appena sopra hover, e  $|T| = \sqrt{0.1^2 + 0.9^2} = 0.905$ , ben dentro il bound  $T_{max} = 70000/19620 = 3.57$ . Il commento e' accurato.

**Perche' questo punto di lavoro e non un altro.** Non e' indifferente: `ode_descent` contiene una divisione  $u/x(5)$  e una `sqrt`. Se si scegliesse  $u = [0; 0]$ ,  $T_{mag} = 0$  e la `sqrt(0)` potrebbe imboccare un percorso denormale/degenere; se  $m$  fosse vicino a zero, la divisione esploderebbe. Un punto “generico e sano” evita di misurare accidentalmente un caso patologico della FPU. Detto questo, il codice **non** documenta questa motivazione: e' una lettura, non una citazione.

**Possibile domanda d'esame** – Perche' il setup dello stato sta in un `TestMethodSetup` e non dentro il metodo di test, prima del `while`? *Risposta:* Funzionalmente sarebbe equivalente, perche' in entrambi i casi resta fuori dal confine di misura. La ragione e' di condivisione: i tre metodi di test usano lo **stesso** punto di lavoro, e centralizzarlo garantisce che confrontino la stessa cosa. Se un domani si misurasse un altro stato, si cambierebbe in un posto solo.

## testOdeDescentEvaluation (righe 38-48)

```
function testOdeDescentEvaluation(testCase)
    % One call (~60 ns) is below framework precision;
    % measure a batch of 1000 per sample
    x = testCase.x0; u = testCase.u0; vc = testCase.Vc;
    while testCase.keepMeasuring
        for k = 1:1000
            dx = ode_descent(x, u, vc);
        end
    end
    testCase.verifySize(dx, [5 1]);
end
```

- Riga 41: le tre variabili sono estratte dalle proprieta' **prima** del `while`. Non e' pedanteria: l'accesso a una proprieta' di un oggetto (`testCase.x0`) in MATLAB passa dal dispatch delle property, che non e' gratuito. Se `testCase.x0` fosse dentro il ciclo, si misurerebbe in parte il costo di accesso alla property invece della RHS. Questo e' il tipo di contaminazione che rende inutile un microbenchmark.
- Righe 42-46: il confine di misura. Il corpo del `while non` e' una singola chiamata: e' un batch di 1000. La ragione e' nel commento (righe 39-40): una chiamata sola sta sotto la risoluzione del framework. Il corpo di `ode_descent` (righe 13-14 del suo file) e' una `sqrt`, una radice di somma di quadrati, due divisioni e un assemblaggio di vettore  $5 \times 1$ : e' dell'ordine delle decine di nanosecondi, mentre l'overhead di una singola misura del framework e' ordini di grandezza sopra. Con il batch, il tempo misurato per campione e' ~1000 volte quello di una chiamata, e il rapporto segnale/overhead diventa favorevole.

**Attenzione al numero riportato.** `keepMeasuring` normalizza per iterazione del `while`, non per iterazione del `for` interno. Il valore che `runperf` stampa per questo test e' quindi il tempo di **1000 chiamate**. Per avere il costo unitario bisogna dividere a mano per 1000. Questo e' un trabocchetto reale, e peggiora nel test successivo (batch da 100): i numeri dei due test **non sono confrontabili direttamente**.

Nota sul ~60 ns del commento: e' un'affermazione dell'autore, non un valore misurato o verificato da questo test. Il test non contiene nessuna soglia che lo controlli.

- Riga 44:  $dx$  viene sovrascritta a ogni iterazione. Gli argomenti  $x, u, vc$  sono **identici** per tutte e 1000 le chiamate. Questo garantisce che i dati siano in cache L1 e che la predizione dei branch sia perfetta: e' una misura di **best case**. Dentro `fmincon`, invece, ogni valutazione a differenze finite perturba una componente

diversa del vettore, e i dati scorrono. Il benchmark misura il costo aritmetico della RHS, non il costo che la RHS ha realmente nel contesto di uso. Vedi la sezione sui limiti.

- Riga 47: `verifySize(dx, [5 1])` sta **dopo** il `while`, quindi fuori dal confine di misura. Non e' un test di correttezza (quella vive in `tests/odeDescentTest.m`): e' un guardrail contro il benchmark che misura una funzione che nel frattempo ha smesso di restituire un vettore 5x1, e contro l'ipotesi remota che l'engine elimini una chiamata il cui risultato non viene mai usato. Usare il risultato e' buona igiene da microbenchmark.

---

## testRk4ZohPropagation (righe 50-61)

```
function testRk4ZohPropagation(testCase, nSub)
    % Batch of 100 per sample: the 1-substep case is
    % sub-microsecond and noise-dominated measured singly
    x = testCase.x0; u = testCase.u0;
    vc = testCase.Vc; h = testCase.dt;
    while testCase.keepMeasuring
        for k = 1:100
            xn = rk4_zoh(x, u, h, vc, nSub);
        end
    end
    testCase.verifySize(xn, [5 1]);
end
```

- Riga 50: la firma prende `nSub` come secondo argomento. E' il meccanismo di parametrizzazione di `matlab.unittest`: il framework riconosce che `nSub` e' una `TestParameter` (righe 18-20) e istanzia tre test distinti con `nSub = 1, 2, 8`. Nei risultati compaiono come `testRk4ZohPropagation(nSub=one)`, `(nSub=two)`, `(nSub=eight)`.
- Righe 55-59: batch da 100 (non 1000). Un `rk4_zoh` con `n_sub = 1` fa 4 chiamate a `ode_descent` (righe 17-20 di `rk4_zoh.m`), quindi costa circa 4 volte una RHS piu' l'overhead di chiamata; con `n_sub = 2` (il valore di produzione) le valutazioni sono 8, con `n_sub = 8` sono 32. Resta sotto il microsecondo, come dice il commento, ma essendo da 4 a 32 volte piu' caro di una RHS singola basta un batch dieci volte piu' piccolo per superare la risoluzione del timer. Il costo di ogni caso e', in prima approssimazione:

$$\text{costo}(n\_sub) = a + b * n\_sub$$

dove `b` e' il costo di un substep (4 valutazioni di `ode_descent` piu' la combinazione lineare  $x + (h/6)*(k_1 + 2*k_2 + 2*k_3 + k_4)$ , riga 21) e `a` e' l'overhead fisso della chiamata di funzione MATLAB piu' la divisione  $h = dt/n\_sub$  (riga 15). **E' questo che compra la parametrizzazione:** con tre punti (1, 2, 8) si puo' separare `a` da `b`. Se un giorno il tempo del caso `nSub=8` non fosse circa  $a + 8b$  coerente con gli altri due, si saprebbe che qualcosa e' cambiato *dentro* il ciclo di substep e non nell'overhead di chiamata. E' diagnostica, non solo misura.

- Riga 57: nota che `n_sub = 2` e' il valore effettivamente usato in produzione (`main_task2.m` riga 37, `n_sub = 2`). I casi 1 e 8 sono li' per la diagnostica di cui sopra, non perche' il solver li usi.

**Possibile domanda d'esame** – Perche' il batch di questo test e' 100 e quello di `ode_descent` e' 1000? E' un problema? *Risposta:* Perche' `rk4_zoh` costa  $4*n\_sub$  valutazioni di RHS piu' l'overhead di chiamata – da 4 volte una singola RHS (`n_sub = 1`) a 32 volte (`n_sub = 8`) – quindi bastano meno ripetizioni per superare la risoluzione del timer. E' pero' un problema di leggibilita': `runperf` riporta il tempo del corpo del `while`, cioe' del batch intero, quindi i numeri dei due test sono in "unita'" diverse (tempo di 1000 RHS contro tempo di 100 intervalli RK4). Per confrontarli bisogna normalizzare a mano. Un design piu' pulito userebbe la stessa dimensione di batch, o normalizzerebbe esplicitamente.



## testOde45ZohReplayInterval (righe 63-71)

```
function testOde45ZohReplayInterval(testCase)
    x = testCase.x0; u = testCase.u0;
    vc = testCase.Vc; h = testCase.dt;
    opts = odeset('RelTol', 1e-10, 'AbsTol', 1e-12);
    while testCase.keepMeasuring
        [~, Y] = ode45(@(t, xx) ode_descent(xx, u, vc), ...
            [0 h], x, opts);
    end
    testCase.verifySize(Y(end,:), [1 5]);
end
```

Questo e' il test di **livello sistema** annunciato nel commento di classe (riga 5). Non misura una funzione, misura un'operazione dell'utente finale: integrare un intervallo ZOH con `ode45` alle tolleranze di verifica.

- Riga 66: `odeset` sta **fuori** dal `while`. Corretto: costruire una struct di opzioni e' relativamente costoso e non fa parte di cio' che si vuole misurare. Le tolleranze `RelTol = 1e-10`, `AbsTol = 1e-12` non sono arbitrarie: sono **esattamente** quelle di `fwd_integrate` (`main_task2.m` riga 839) e `fwd_integrate_uacc` (riga 1295), le funzioni che replicano la soluzione ottimizzata per misurarne la fedelta'. Il benchmark quindi riproduce il carico reale della fase di verifica, non un carico inventato. Sono anche coerenti con la convenzione della repo per il lavoro indiretto/shooting (`RelTol=1e-10`, `AbsTol=1e-12`).
- Riga 67: nessun batch. Un `ode45` a tolleranza `1e-10` su un intervallo fa molti passi adattivi, ognuno con 6 valutazioni della RHS (Dormand-Prince): e' abbastanza costoso da essere misurabile singolarmente. Quindi qui il numero riportato e' il tempo di **una** replica di intervallo – di nuovo, un'unita' diversa dai due test precedenti.
- Riga 68: **divergenza dal codice di produzione**. Qui la RHS e' `@(t, xx) ode_descent(xx, u, vc)`, che cattura `u` direttamente. In `fwd_integrate` (righe 844-852) la RHS e' invece `@(tt, x) ode_descent(x, u_fcn(tt), d.Vc)`, dove `u_fcn` e' a sua volta una anonymous function. La produzione paga quindi **una dispatch di anonymous function in piu'** a ogni valutazione della RHS, che il benchmark non paga. Il benchmark e' cioe' sistematicamente ottimista rispetto al percorso reale 'zoh' di `fwd_integrate`. La differenza e' probabilmente piccola in rapporto al costo di `ode45`, ma non e' zero e non e' dichiarata da nessuna parte nel codice.
- Riga 68 (secondo punto): la anonymous function viene **costruita dentro** il `while`, quindi il suo costo di costruzione entra nella misura. E' un'inconsistenza rispetto a `opts`, che e' stato accuratamente issato fuori. Piccola, ma e' una contaminazione del confine di misura.
- Riga 70: `Y(end,:)` e' `1x5` (`ode45` restituisce le righe come istanti di tempo), da cui `verifySize(..., [1 5])` e non `[5 1]`. Consuma il risultato per evitare che venga considerato morto.

**Possibile domanda d'esame** – Perche' misurare un `ode45` in un test di performance, se `ode45` non compare mai dentro l'NLP? *Risposta:* Perche' non e' sul cammino critico del solver, ma e' sul cammino critico della **verifica**: `fwd_integrate` replica la soluzione intervallo per intervallo (`main_task2.m` righe 840-855) per calcolare il node error, ed e' l'operazione che produce le metriche di fedelta' riportate nel README (errore  $1.4e-8$  non-dim per la variante ZOH,  $7.3e-12$  per GFOLD). Con  $N = 50$  nodi si fanno 49 di queste integrazioni per ogni soluzione da validare, e ce ne sono quattro varianti piu' uno sweep. Se il costo per intervallo raddoppiasse, raddoppierebbe il tempo di tutta la fase di post-processing.

---

## Il nesso con il design: perche' le RHS non hanno arguments – e perche' `lti_zoh` invece ce l'ha

E' il punto piu' probabile all'orale, perche' sembra un'incoerenza. Non lo e'.

## Le funzioni hot loop: nessuna validazione

ode\_descent.m (righe 10-11) e rk4\_zoh.m (righe 12-13) portano lo stesso commento:

```
% No arguments validation by design: hot-loop RHS inside  
% ode45/fmincon; validate at the call site.
```

Lo stesso commento compare su trap\_nonlcon (riga 769), zoh\_nonlcon (riga 794), path\_ineq (riga 812) e sulla RHS aumentata dell'Appendix A (riga 288). Il conteggio delle chiamate, ricavato dal codice:

**Percorso trapezoidale** (solve\_trap -> trap\_nonlcon, righe 759-781):

```
variabili di decisione    = 7*N = 7*50 = 350  
Jacobiano a diff. finite = 350 + 1 = 351 valutazioni di nonlcon  
ode_descent per nonlcon   = N = 50                (righe 772-774)
```

```
-----  
ode_descent per Jacobiano ~ 351 * 50 = 1.8e4  
MaxIterations = 1000      (riga 733)
```

```
-----  
totale ~ 1.8e7 chiamate (piu' quelle della line search SQP)
```

**Percorso ZOH** (solve\_zoh -> zoh\_nonlcon, righe 783-803):

```
rk4_zoh per nonlcon      = N-1 = 49                (righe 797-800)  
ode_descent per rk4_zoh  = 4*n_sub = 4*2 = 8  
ode_descent per nonlcon  = 49*8 = 392
```

```
-----  
ode_descent per Jacobiano ~ 351 * 392 = 1.4e5  
totale al cap di 1000 iter ~ 1.4e8 chiamate
```

Contro un corpo funzione che fa una sqrt e due divisioni, un blocco arguments con validatori (mustBeFinite, controllo di size, ecc.) e' un costo dello stesso ordine o maggiore del calcolo stesso, pagato 1e7-1e8 volte. **Attenzione all'onesta'**: questa e' l'argomentazione di design, ma la repo **non contiene** un benchmark A/B che confronti ode\_descent con e senza arguments. Questo file misura la funzione **com'e'**; non dimostra che la scelta fosse corretta, la presidia soltanto contro regressioni future. Se all'orale ti chiedessero "e di quanto sarebbe piu' lenta con la validazione?", la risposta onesta e' "non l'ho misurato".

## lti\_zoh ha un blocco arguments (righe 19-22): non e' un'incoerenza

```
function [Abar, Bbar, cbar] = lti_zoh(dt, Vc)  
    arguments  
        dt (1,1) double {mustBePositive, mustBeFinite}  
        Vc (1,1) double {mustBeFinite}  
    end  
    ...  
    E = expm([A, B, c; zeros(4,9)] * dt);
```

La discriminante non e' "che tipo di funzione e'", ma **quante volte viene chiamata**. E il codice risponde in modo netto: in solve\_gfold\_scvx, lti\_zoh e' invocata alla **riga 1218**, mentre il ciclo SCvx comincia alla **riga 1239** (for iter = 1:max\_iter). Cioe' e' **fuori dal ciclo**, una volta sola per risoluzione, con tanto di commento in linea: % exact LTI ZOH, once.

La ragione fisica per cui puo' stare fuori dal ciclo e' il cuore del metodo GFOLD: il cambio di variabili  $z = \ln(m)$ ,  $u = T/m$  (Acikmese & Blackmore) rende la dinamica **esattamente LTI** (vedi l'intestazione di lti\_zoh.m, righe 3-5). Se il sistema e' tempo-invariante, le matrici discrete Abar, Bbar, cbar sono **costanti su tutta la griglia** e non dipendono dalla traiettoria di riferimento. Quindi non c'e' nulla da ricalcolare a ogni iterazione SCvx. Confronta con compute\_ltv\_zoh (riga 305) del percorso LTV, che invece deve reintegrare l'ODE aumentata dell'Appendix A a **ogni** iterazione, perche' li' la linearizzazione dipende dal riferimento corrente.

Conseguenza: `lti_zoh` viene chiamata  $O(1)$  volte per risoluzione, e il suo costo e' comunque dominato da una `expm` di una matrice  $9 \times 9$ . La validazione degli argomenti e' rumore in confronto, e in cambio protegge da un `dt <= 0` o non finito, che produrrebbe una `expm` silenziosamente sbagliata. **E' esattamente la regola "la validazione vive nei boundary helper chiamati una volta per run"**, applicata correttamente. Nessuna incoerenza.

Coerentemente, anche `solve_gfold_scvx` (righe 1211-1217), `nondim` (righe 210-212), `fmincon_opts` (righe 728-730) e `fwd_integrate_uacc` (righe 1289-1292) hanno blocchi `arguments`: sono tutti `boundary`.

### E `ode_descent_uacc`?

Il suo commento (riga 16) dice `No arguments validation by design: hot-loop RHS inside ode45`, e il codice conferma: e' chiamata solo da `fwd_integrate_uacc` (`main_task2.m` riga 1298) e da `proto_gfold_logmass.m` (riga 249), entrambe dentro un `ode45`. Quindi e' hot **rispetto a `ode45`** (migliaia di valutazioni per intervallo), ma **non** e' sul cammino critico dell'NLP: nel percorso GFOLD la dinamica dentro il ciclo SCvx e' gestita da `Abar/Bbar/cbar`, non da valutazioni della RHS. E' un "warm loop": chiamato molto, ma una volta sola per soluzione, in fase di verifica. Che non sia benchmarkato in questo file e' difendibile su questa base; resta il fatto che **non e' benchmarkato**, e il commento di classe (righe 2-5) non spiega l'esclusione.

---

## Cosa significherebbe una regressione qui

Il valore di questo file sta tutto nel fattore di amplificazione. Usando i conteggi ricavati sopra, una regressione di **100 ns per chiamata** di `ode_descent` (cioe' circa un raddoppio, se il commento dei ~60 ns e' nel giusto ordine di grandezza) si propaga cosi':

| Percorso                            | Chiamate <code>ode_descent</code> | Costo aggiunto @100 ns |
|-------------------------------------|-----------------------------------|------------------------|
| <code>solve_trap</code> (Task 1/2)  | ~1.8e7                            | ~1.8 s per solve       |
| <code>solve_zoh</code> (variante a) | ~1.4e8                            | ~14 s per solve        |

E se qualcuno aggiungesse un blocco `arguments` che costasse, poniamo, 1 microsecondo per chiamata (ordine di grandezza **plausibile ma non misurato in questa repo** – lo dico esplicitamente per non spacciare un numero inventato per un risultato), gli stessi conteggi darebbero ~18 s e ~140 s di overhead puro per singola risoluzione. Il README dichiara ~1-1.5 min per valore di `tf` per il Task 1: significa che una regressione del genere **raddoppierebbe o triplicherebbe** il tempo di soluzione. E lo sweep di sensitivita' risolve tre valori di `tf`, piu' uno studio di convergenza di griglia a  $N = 25/50/100$  – il fattore si moltiplica ancora.

C'e' un secondo effetto, piu' sottile. Le corse a `tf` corto terminano al cap `MaxIterations = 1000` con optimality di  $1e-3/1e-4$  (README, e riga 733 per il cap). Un solver che gira piu' lentamente non converge peggio, ma rende piu' costoso *provare* a farlo convergere: alzare `MaxIterations` per curare quel problema noto diventa proibitivo se ogni iterazione costa il doppio. La performance qui non e' comoda', e' cio' che rende praticabile l'iterazione sul metodo.

---

## Limiti onesti del benchmark

Elenco senza sconti.

1. **Non misura una chiamata isolata, misura un batch.** 1000 chiamate per `ode_descent`, 100 per `rk4_zoh`, 1 per `ode45`. Tre unita' diverse. I numeri stampati da `runperf` **non sono confrontabili fra i tre test** senza normalizzare a mano. Nulla nel codice fa questa normalizzazione e nulla la documenta oltre ai due commenti alle righe 39-40 e 51-52.

2. **Non e' rappresentativo del carico dentro fmincon.** Il batch ripete la stessa chiamata con **gli stessi identici argomenti**. Dati caldi in cache, branch predetti, nessuna variazione. Dentro `fmincon` le valutazioni a differenze finite perturbano ogni volta una componente diversa di un vettore da 350 elementi, e la RHS e' chiamata da dentro `trap_nonlcon` che fa `reshape`, alloca `f = zeros(5,N)` e cicla. Il costo *reale* per chiamata nel contesto d'uso include effetti di memoria e overhead del chiamante che questo benchmark **non** cattura. E' un microbenchmark di best case, e va letto come tale.
3. **Non misura il vero hot loop.** Cio' che `fmincon` chiama non e' `ode_descent`: e' `trap_nonlcon` / `zoh_nonlcon`. Quelle funzioni fanno `reshape`, allocano matrici, costruiscono i defect e chiamano `path_ineq`. Un benchmark di `trap_nonlcon(z, N, dt, d)` sarebbe molto piu' vicino alla realta' e catturerebbe regressioni (per esempio una allocazione inutile nel ciclo dei defect) che questo file lascia passare del tutto. La scelta di stare al livello della RHS e' difendibile (isola la primitiva), ma lascia scoperto lo strato dove sta la maggior parte del codice.
4. **Copertura parziale.** Non sono benchmarkati: `lti_zoh` (giustificabile:  $O(1)$  per solve), `ode_descent_uacc` (giustificabile: fuori dal cammino critico dell'NLP), ma soprattutto `compute_ltv_zoh` e la RHS aumentata `ltv_aug_rhs` (riga 279), che integrano un sistema a **70 stati** a ogni iterazione SCvx e sono plausibilmente la parte piu' costosa delle varianti b/c. Quella e' un'omissione senza giustificazione ovvia.
5. **Nessuna baseline, nessuna soglia, nessun gate.** `runperf` restituisce un array di `TimeResult`, ma la repo non memorizza nessun valore di riferimento e non contiene nessun confronto automatico (`comparisonPlot` o `assert` su soglia). In pratica una regressione viene "rilevata" solo se un umano lancia `runperf` a mano e si ricorda i numeri di prima. Chiamarlo *regression test* e' generoso: e' uno **strumento di misura**, non un allarme.
6. **Costanti duplicate.** `Vc` e `dt` (righe 9-10) sono ricopiate a mano da `nondim`. Verificate oggi: `Vc = 0.0777` torna, `dt = 0.0444` e' invece un doppio arrotondamento di 0.044347 (alto dello 0.1% circa). Non c'e' nulla che le tenga sincronizzate con `nondim`.
7. **Contaminazione minore del confine di misura.** La anonymous function della riga 68 e' costruita a ogni iterazione misurata, mentre `opts` (riga 66) e' stato issato fuori. E la RHS del benchmark non replica la doppia anonymous function (`u_fcn(tt)`) del vero `fwd_integrate`, quindi sottostima quel percorso.
8. **Dipendenza dalla macchina.** Nessun benchmark e' portabile fra CPU diverse, frequenze di turbo, stati termici. I numeri hanno senso solo come confronto *prima/dopo sulla stessa macchina, nella stessa sessione*. Il file non lo dice.

---

## Possibili domande d'esame

**D: Che differenza c'e' fra `matlab.unittest.TestCase` e `matlab.perftest.TestCase`, e perche' questo file eredita dalla seconda?** R: `matlab.perftest.TestCase` estende `matlab.unittest.TestCase` aggiungendo i *confini di misura*: `startMeasuring/stopMeasuring` per delimitare esplicitamente una regione, e `keepMeasuring` per farla ripetere automaticamente dal framework finche' la misura non e' accurata. Serve perche' il tempo di un metodo di test *intero* e' inutile: include setup, allocazioni e asserzioni, che possono dominare il codice di interesse di ordini di grandezza. Solo delimitando la regione si ottiene un numero che significa qualcosa. `runperf` inoltre accetta solo classi `perftest` per fare l'esperimento statistico.

**D: Perche' il test misura un ciclo `for k = 1:1000` invece di una singola chiamata a `ode_descent`?** R: Perche' una chiamata singola (dell'ordine delle decine di nanosecondi, il commento dice ~60 ns) sta sotto la risoluzione temporale del framework: misurarla darebbe rumore. Ripetendola 1000 volte dentro il confine di misura, il tempo totale diventa misurabile con precisione e l'overhead della misura diventa trascurabile. Il prezzo e' che il numero riportato e' il tempo di 1000 chiamate, non di una, e che il benchmark diventa un best case: 1000 chiamate identiche con dati caldi in cache non riproducono il pattern d'accesso reale dentro `fmincon`.

**D: Perche' `ode_descent` e `rk4_zoh` non hanno un blocco `arguments`, mentre `lti_zoh` ce l'ha? Non e' un'incoerenza?** R: No, il criterio e' la frequenza di chiamata. `ode_descent` e' invocata dentro

`trap_nonlcon/zoh_nonlcon`, che `fmincon` chiama a ogni valutazione a differenze finite: con 350 variabili e fino a 1000 iterazioni si arriva a  $1e7$ - $1e8$  chiamate, e un validatore costerebbe quanto o piu' del corpo funzione stesso. `lti_zoh` invece e' chiamata **una volta per risoluzione**, alla riga 1218 di `main_task2.m`, cioe' **prima** del ciclo SCvx che parte alla riga 1239: la dinamica GFOLD in variabili  $z = \ln(m)$ ,  $u = T/m$  e' esattamente LTI, quindi le matrici discrete sono costanti sulla griglia e non vanno ricalcolate. Essendo un boundary helper  $O(1)$ , paga volentieri la validazione. La regola della repo e' proprio "hot loop senza validazione, boundary con validazione", e i due casi la rispettano entrambi.

**D: Come fa `runperf` a darti un numero affidabile, e cosa aggiunge rispetto a un `timeit`?** R: `runperf` costruisce un time experiment `limitingSamplingError`: scarta 5 esecuzioni di warm-up (che pagano JIT e cache fredde), poi raccoglie campioni in modo **adattivo** finche' la media campionaria non e' nota entro un margine di errore relativo del 5% al 95% di confidenza, fra un minimo di 4 e un massimo di 256 campioni. `sampleSummary` riepiloga media, mediana, deviazione standard, min e max. `timeit` da' invece un singolo numero senza intervallo di confidenza e senza criterio d'arresto statistico: non ti permette di distinguere una regressione vera dal rumore del sistema operativo, che e' esattamente la domanda a cui un performance test deve rispondere.

**D: Cosa comporterebbe, in concreto, una regressione di prestazioni su `ode_descent`?** R: Un'amplificazione enorme, per via del conteggio delle chiamate. Sul percorso trapezoidale servono  $\sim 1.8e7$  valutazioni per risoluzione, su quello ZOH (con `n_sub` = 2, quindi 8 RHS per intervallo)  $\sim 1.4e8$ . Un rallentamento di 100 ns per chiamata costerebbe rispettivamente  $\sim 2$  s e  $\sim 14$  s per singola soluzione; e lo script risolve tre valori di `tf` piu' uno studio di convergenza a  $N = 25/50/100$ . Su un runtime nominale di  $\sim 1$ - $1.5$  min per `tf`, un'aggiunta di overhead per chiamata dell'ordine del microsecondo raddoppierebbe o triplicherebbe il tempo totale, e renderebbe proibitivo alzare `MaxIterations` = 1000 per curare la convergenza stallata a `tf` corto.

**D: Questo benchmark e' rappresentativo del carico reale dentro `fmincon`?** R: Solo parzialmente, e va detto. Misura la primitiva in condizioni ideali: stessi argomenti a ogni chiamata, dati in cache, nessuna variazione. Il vero hot loop chiamato da `fmincon` e' `trap_nonlcon/zoh_nonlcon`, che oltre alla RHS fanno `reshape`, allocano matrici e valutano `path_ineq`, e che ricevono ogni volta un vettore perturbato diverso. Un benchmark di `nonlcon` sarebbe piu' fedele. Inoltre il test `ode45` usa una RHS piu' semplice di quella vera di `fwd_integrate` (che passa per una anonymous function `u_fcn(tt)` in piu' a ogni valutazione), quindi sottostima quel percorso. Il file misura la primitiva, non il sistema.

**D: Perche' c'e' un `verifySize` dentro un test di performance?** R: Sta **fuori** dal confine di misura (dopo il `while`), quindi non inquina il numero. Serve a due cose: assicurare che il risultato venga effettivamente consumato (igiene da microbenchmark, evita che il calcolo possa essere considerato codice morto) e fare da guardrail minimo – se `ode_descent` smettesse di restituire un  $5 \times 1$ , staresti misurando la velocita' di una funzione sbagliata. Non e' un test di correttezza: quella vive in `tests/odeDescentTest.m` e `tests/rk4ZohTest.m`.

# HM3/tests/hm3PlantTest.m

## Ruolo del file nel progetto

E' la suite `matlab.unittest` che **congela la fisica** di HM3. Copre tre file: `load_hw3_params.m` (i coefficienti di Tabella 1 letti a  $t = 72$  s dal data set LPV), `build_plant_rigid.m` (il modello a 4 stati del Task 1) e `build_plant_full.m` (il modello a 6 stati con il primo modo di bending, Task 2/3). Nessuna di queste funzioni fa controllo: costruiscono soltanto la matrice **A**, **B**, **C** dell'Eq. (1) e dell'Eq. (2) della traccia. Se sono sbagliate, tutto il resto dell'homework (Nichols, notch, margini, Monte Carlo) e' costruito su sabbia – da qui la scelta di testarle per prime e in modo molto letterale.

La domanda che questa suite si pone e': *il plant che ho in mano e' davvero il lanciatore Greensite a max-qbar, o e' un sistema qualunque a 4/6 stati?* La risposta viene data pinnando quattro classi di invarianti:

1. **Invarianti sui dati** – i valori interpolati dal file `General/hw3-v3/GreensiteLPV_DATA.mat` a  $t = 72$  s coincidono con i letterali di Tabella 1 (test 1), e il carico dinamico  $qbar$  e' coerente con l'atmosfera esponenziale (test 2).
2. **Invarianti di interfaccia** – dimensioni e, soprattutto, i **nomi** dei segnali (test 4). Non e' cosmetica: `assemble_loop.m` chiude l'anello con `connect(...)`, che collega i blocchi **per nome**. Un `InputName` rinominato non rompe `build_plant_rigid`, rompe silenziosamente il loop.
3. **L'invariante fisico centrale** – il polo instabile del corpo rigido cade a `+sqrt(A_6)` (test 5). E' la firma analitica dell'instabilita' aerodinamica ed e' il motivo per cui esiste l'homework.
4. **Invarianti sulla struttura di misura** – la contaminazione INS del bending (Eq. 2) entra con i segni giusti (test 8), e sparisce nella variante `'true'` (test 9). E' la contaminazione che destabilizza l'anello a `omega_BM` e che motiva il notch del Task 2.

La suite gira in due minuti scarsi e non richiede nulla oltre alla Control System Toolbox.

---

## Setup di classe (righe 7-20)

```
properties
    p      % nominal parameter struct, loaded once per class
end

methods (TestClassSetup)
    function addHm3ToPath(testCase)
        hm3 = fileparts(fileparts(mfilename('fullpath')));
        testCase.applyFixture( ...
            matlab.unittest.fixtures.PathFixture(hm3));
    end
```

- Righe 7-9: la property `p` tiene lo struct dei parametri nominali. Essendo popolata in `TestClassSetup` (non in `TestMethodSetup`), `load_hw3_params()` viene chiamata **una volta sola per classe**, non una volta per test. E' corretto perche' lo struct e' immutabile e i test non lo modificano: quelli che vogliono parametri diversi (test 3) ricostruiscono uno struct locale.

- Righe 12-15: `mfilename('fullpath')` e' il path di questo file di test (`.../HM3/tests/hm3PlantTest.m`); il doppio `fileparts` risale di due livelli e restituisce `.../HM3`. La `PathFixture` aggiunge quella cartella al path MATLAB e la **rimuove alla fine della classe**. E' il modo pulito di rendere la suite eseguibile da qualunque cwd (`runtests('HM3/tests')` funziona anche dalla root della repo) senza sporcare il path dell'utente.
- Righe 17-19: caricamento dei parametri nominali (scala di incertezza = 1).

**Possibile domanda d'esame** – Perche' `PathFixture` e non un `addpath` secco? *Risposta:* `addpath` e' un effetto collaterale globale: se la suite fallisce a meta', il path resta sporco. La `PathFixture` e' gestita dal framework, che garantisce il ripristino anche in caso di eccezione (e' lo stesso principio dell'`onCleanup` usato in `design_controller.m` per le warning). Nei test serve isolamento: uno stato residuo puo' far passare o fallire il test successivo per ragioni che non c'entrano nulla col codice sotto test.

## testParamsMatchTable1 (righe 23-31)

```
pp = testCase.p;
testCase.verifyEqual(pp.A6, 3.3818, 'AbsTol', 5e-3);
testCase.verifyEqual(pp.K1, 4.5647, 'AbsTol', 5e-3);
testCase.verifyEqual(pp.V, 937.70, 'AbsTol', 0.5);
testCase.verifyEqual(pp.wBM, 18.9, 'AbsTol', 0.05);
testCase.verifyEqual(pp.a4, -27.2710, 'AbsTol', 5e-2);
```

- Righe 26-30: cinque valori attesi hard-coded. Sono i **letterali di Tabella 1** della traccia:

|                             |                              |                                        |
|-----------------------------|------------------------------|----------------------------------------|
| <code>A_6 = mu_alpha</code> | <code>= 3.3818 s^-2</code>   | (momento aerodinamico destabilizzante) |
| <code>K_1 = mu_c</code>     | <code>= 4.5647 s^-2</code>   | (efficacia del TVC)                    |
| <code>V</code>              | <code>= 937.70 m/s</code>    |                                        |
| <code>omega_BM</code>       | <code>= 18.9 rad/s</code>    |                                        |
| <code>a_4</code>            | <code>= -27.2710 s^-2</code> |                                        |

Il punto sottile e' *perche' questo test non e' una tautologia*. In `load_hw3_params.m` (righe 40-48) quegli stessi numeri sono scritti come letterali, ma alle righe 59-80 **vengono sovrascritti** dal data set LPV (il ramo LPV riscrive i soli coefficienti tempo-varianti – `A6`, `K1`, `a1`, `a3`, `a4`, `V`, `Tc`, `sigma_ins`, `phi_ins`, `phi_tvc`, `wBM`; `zBM`, `wTVC`, `zTVC` e `tau` restano i letterali di Tabella 1):

```
p.A6 = interp1(L.A6.Time, squeeze(L.A6.Data), 72)
```

Quindi il test confronta il valore **interpolato a  $t = 72$  s** con il valore **stampato in tabella**. E' un cross-check fra due fonti indipendenti: se il file `.mat` non c'e', se `t_ref` fosse sbagliato, se il mapping dei campi LPV fosse invertito (`L.a3` scambiato con `L.a4`), o se l'interpolazione cadesse su un istante diverso, i numeri divergerebbero e il test fallirebbe. Verificato in MATLAB: la sorgente effettiva e' `GreensiteLPV_DATA.mat @ t=72 s` e i valori interpolati sono `A6 = 3.3818`, `K1 = 4.5647`, `V = 937.71`, `wBM = 18.900`, `a4 = -27.2710`. Combaciano.

- Le tolleranze sono asimmetriche e questo e' voluto: `5e-3` sui coefficienti ( $\sim 0.1\%$ ) perche' la tabella li stampa a 4 decimali, `0.5` su `V` ( $\sim 0.05\%$ ) perche' la tabella arrotonda a `937.70` mentre il dato LPV da' `937.71`, `0.05` su `omega_BM` che in tabella e' a una sola cifra decimale. Le tolleranze seguono la **precisione di stampa della fonte**, non un criterio numerico.
- Nota di onesta': `a_4` merita attenzione. Il commento in `load_hw3_params.m` (righe 36-38) segnala che dai valori di massa e spinta della tabella si ricaverebbe  $-(T_t - D)/m = -23.17 \text{ s}^{-2}$ , mentre la tabella dichiara `a_4 = -27.2710`. Il data set LPV concorda con `-27.2710`, e **quello e' il valore che entra nella dinamica**. Il test pinna `-27.2710`, cioe' pinna l'incoerenza cosi' com'e'. E' la scelta giusta (il test deve riflettere il codice), ma all'orale bisogna saperla difendere: e' una discrepanza nota della traccia, non un errore di implementazione.

**Possibile domanda d'esame** – Il tuo test verifica  $a_4 = -27.2710$ , ma dai dati di Tabella 1 verrebbe -23.17. Quale usi e perché? *Risposta:* Uso -27.2710, perché è il valore che compare sia nella riga  $a_4$  di Tabella 1 sia nel data set LPV di riferimento del professore, che sono due fonti indipendenti in accordo fra loro. Il -23.17 nasce dal ricalcolare  $-(T_t - D)/m$  con i valori di massa e spinta arrotondati della stessa tabella, e la discrepanza (~15 %) è plausibilmente dovuta a quegli arrotondamenti o a un istante di riferimento leggermente diverso per  $T$  e  $D$ .  $a_4$  entra solo nella riga della dinamica laterale (accelerazione  $zddot$ ), non nel momento di beccheggio, quindi non tocca il polo instabile né i margini: sposta solo leggermente la deriva laterale.

---

## testDynamicPressureSelfConsistent (righe 33-38)

```
qbarExpected = 0.5 * 1.225 * exp(-pp.Alt/8000) * pp.V^2;
testCase.verifyEqual(pp.qbar, qbarExpected, 'AbsTol', 1e-6);
```

- Righe 36-37: ricalcola la pressione dinamica con l'**atmosfera esponenziale**

```
rho(h) = rho_0 * exp(-h / H_scale),    rho_0 = 1.225 kg/m^3, H_scale = 8000 m
qbar    = 0.5 * rho(h) * V^2
```

A  $h = 15143$  m e  $V = 937.7$  m/s viene  $qbar = 81.1$  kPa (verificato: 81129 Pa), che è il valore di max-qbar citato nel README.

- **Onesta' totale:** questo test è semi-tautologico. Riscrive la **stessa formula** delle righe 83-84 di `load_hw3_params.m`, con le stesse costanti hard-coded (1.225 e 8000). Non può quindi accorgersi di un errore *fisico* (per esempio: se `H_scale` fosse 7500 m, il test verrebbe aggiornato allo stesso modo). Cattura solo due cose: (a) che `p.qbar` sia effettivamente derivato e non un letterale scollegato da `V` e `Alt`, (b) che una futura modifica alla formula in `load_hw3_params.m` sia deliberata e non accidentale. Va classificato come *change-detector*, non come test di correttezza. Da notare inoltre che `p.Alt` resta un letterale (15143 m): non è letto dall'LPV, quindi la quota non è cross-checkata da nessun test.
- La `AbsTol` di  $1e-6$  su un numero dell'ordine di  $8e4$  è di fatto una richiesta di identità bit-a-bit modulo l'ordine delle operazioni in floating point.

---

## testUncertaintyScalingAppliesToA6K1 (righe 40-47)

```
ps = load_hw3_params('mu_alpha_scale', 1.3, 'mu_c_scale', 0.7);
testCase.verifyEqual(ps.A6, 1.3*pp.A6, 'AbsTol', 1e-12);
testCase.verifyEqual(ps.K1, 0.7*pp.K1, 'AbsTol', 1e-12);
testCase.verifyEqual(ps.a3, pp.a3, 'AbsTol', 1e-12);
```

- Riga 43: usa esattamente il vertice **V3** del box di incertezza del Task 3 ( $\mu_{\alpha} +30\%$ ,  $\mu_c -30\%$ ), che il README identifica come il caso peggiore (0.91 dB / 18.0 deg). Non è una scelta casuale di numeri.
- Righe 44-46: l'invariante è **quali coefficienti l'incertezza tocca e quali no**.  $A_6$  ( $= \mu_{\alpha}$ ) e  $K_1$  ( $= \mu_c$ ) scalano;  $a_3$  (efficacia della forza laterale del TVC) **resta identico**. La terza asserzione è quella interessante: è un test *negativo*, garantisce che lo scaling non sia stato applicato con un ciclo su tutti i campi dello struct.
- Perché è l'invariante giusto: la traccia chiede di perturbare  $\pm 30\%$  *i due parametri aerodinamico e di controllo*, non l'intero modello. Se lo scaling si propagasse anche ad  $a_1$ ,  $a_3$ ,  $a_4$ , lo studio di robustezza del Task 3 risponderebbe a una domanda diversa da quella posta, e i risultati non sarebbero confrontabili con quelli attesi. Nota fisica:  $\mu_c$  scala  $K_1$  (il momento di controllo) ma **non**  $a_3$  (la forza laterale prodotta dalla stessa deflessione), il che è fisicamente discutibile – entrambe derivano dalla stessa spinta deviata. Il test pinna la scelta della traccia, non una verità fisica.



- AbsTol 1e-12 perche' e' una moltiplicazione esatta: 1.3\*p.A6 calcolato nel test e nel codice deve dare lo stesso double.

**Possibile domanda d'esame** – Perche' lo scaling di robustezza non tocca a\_1, a\_3, a\_4? *Risposta:* Perche' la traccia definisce la scatola di incertezza sui due parametri che dominano l'anello di beccheggio: mu\_alpha (l'instabilita' aerodinamica, che fissa il polo a +sqrt(A\_6) e quindi il margine di guadagno aerodinamico) e mu\_c (l'autorita' di controllo, che fissa il guadagno d'anello in alta frequenza). Sono anche i due piu' incerti in volo reale, perche' dipendono dal centro di pressione e dalla spinta effettiva. I coefficienti laterali influenzano la deriva, non la stabilita' dell'anello di assetto, quindi restano nominali.

## testRigidPlantDimensionsAndNames (righe 49-55)

```
G = build_plant_rigid(testCase.p);
testCase.verifySize(G.A, [4 4]);
testCase.verifySize(G.B, [4 2]);
testCase.verifyEqual(G.InputName, {'delta'; 'alpha_w'});
testCase.verifyEqual(G.OutputName(1), {'theta_m'});
```

- Righe 51-52: 4 stati [z, zdot, theta, thetadot] e 2 ingressi [delta, alpha\_w]. Il secondo ingresso non e' un comando: e' il **disturbo di vento**, trattato come un ingresso esogeno del plant. E' la scelta che permette poi a assemble\_loop di restituire una T con ingressi {alpha\_w, theta\_ref} e di simulare la raffica con un semplice lsim.

- Righe 53-54: **questo e' il test importante della funzione**, ed e' facile scambiarlo per cosmesi. assemble\_loop.m (righe 26-27, 31-32, 35) costruisce l'anello con

```
T = connect(G, Kc, Wa, {'alpha_w', 'theta_ref'}, ...
            {'theta', 'z', 'zdot', 'delta'}, {'delta'});
```

e connect collega i blocchi **per nome di segnale**. Se build\_plant\_rigid rinominasse theta\_m in thetam, il plant continuerebbe a essere corretto, ma connect non troverebbe piu' l'ingresso corrispondente del controllore e l'anello si chiuderebbe male (o connect lascerebbe un ingresso scollegato, producendo un L privo del ramo di feedback). Il test protegge il **contratto di interfaccia** fra il plant e tutto il resto della catena.

- Nota: verifyEqual(G.InputName, {'delta'; 'alpha\_w'}) usa un cell **colonna** (punto e virgola), perche' e' cosi' che ss normalizza InputName. Un cell riga farebbe fallire il confronto.

## testRigidAirframeUnstablePole (righe 57-63)

```
% Aerodynamically unstable airframe: dominant pole at ~ +sqrt(A6)
% (the a1/a4 drift coupling shifts it ~1% off the pitch-only value)
G = build_plant_rigid(testCase.p);
testCase.verifyEqual(max(real(pole(G))), sqrt(testCase.p.A6), ...
    'RelTol', 0.02);
```

Questo e' il test della suite: cattura in una riga la ragione d'essere di tutto HM3.

**Da dove viene +sqrt(A\_6).** La riga di beccheggio dell'Eq. (1) e'

$$\theta_{ddot} = A_6 \theta + (A_6/V) \dot{z} + K_1 \delta - A_6 \alpha_w$$

Se si isola la sola rotazione (si congela la deriva laterale:  $\dot{z} = 0$ ,  $\alpha_w = 0$ ,  $\delta = 0$ ), resta

$\theta_{ddot} = A_6 * \theta \rightarrow s^2 - A_6 = 0 \rightarrow s = \pm \sqrt{A_6}$

cioe' **una coppia di poli reali simmetrici rispetto all'asse immaginario**, uno dei quali nel semipiano destro. Non e' un'oscillazione smorzata male: e' una divergenza esponenziale pura.

**Perche'  $A_6$  e' positivo.**  $A_6 = \mu_{\alpha} = N_{\alpha} * l_{\alpha} / I_{yy}$ . E' positivo perche' sul lanciatore **il centro di pressione sta davanti al centro di massa** ( $l_{\alpha} > 0$  misurato in avanti). Un incremento di incidenza  $\alpha$  genera una forza normale che, applicata davanti al baricentro, produce un momento che **aumenta ulteriormente  $\alpha$** . E' l'esatto opposto di un aereo (che ha il CP dietro il CG ed e' staticamente stabile): il lanciatore e' un pendolo rovesciato aerodinamico. Con  $A_6 = 3.3818 \text{ s}^{-2}$ :

$\sqrt{A_6} = 1.839 \text{ rad/s} \rightarrow \text{tempo di raddoppio} = \ln(2)/1.839 = 0.377 \text{ s}$

In meno di mezzo secondo l'errore d'assetto raddoppia: **il feedback non e' un miglioramento delle prestazioni, e' una condizione di sopravvivenza**. E questo fissa anche l'ordine di grandezza della banda: il crossover di controllo deve stare comodamente sopra  $1.84 \text{ rad/s}$  (il progetto lo mette a  $2.45 \text{ rad/s}$ ).

**Perche' la tolleranza e' relativa e vale 2 %.** Il plant completo non e' la sola rotazione: la matrice  $A$   $4 \times 4$  accoppia beccheggio e deriva (il termine  $A_6/V$  sulla riga 4, i termini  $a_1$  e  $a_1*V + a_4$  sulla riga 2). L'autovalore esatto quindi **non** e' esattamente  $\sqrt{A_6}$ . Verificato in MATLAB, i poli del plant rigido sono

0, -1.8610, +0.0291, +1.8165

Il polo dominante e' +1.8165 contro  $\sqrt{A_6} = 1.8390$ : uno scarto dell'**1.22 %**, compatibile con la `RelTol` di 0.02 e con quanto dichiara il commento alle righe 58-59. La `RelTol` (non `AbsTol`) e' la scelta giusta perche' lo scarto e' proporzionale, non additivo: se nel Task 3 si scala  $A_6$  di +30 %, il polo si sposta a  $\sim \sqrt{1.3*A_6}$  e la stessa tolleranza relativa continua a valere.

**Il dettaglio che vale all'orale.** Nella lista dei poli ci sono **due poli a parte reale positiva** (+1.8165 e +0.0291) e **uno nell'origine** (lo stato di posizione  $z$ , che e' un integratore puro: nessuna forza richiama il lanciatore verso la traiettoria nominale). L'accoppiamento con la deriva ha spezzato la coppia simmetrica  $\pm \sqrt{A_6}$  in una quaterna. Che il loop abbia due poli instabili in anello aperto e' esattamente il motivo per cui il sistema e' **condizionatamente stabile** e per cui un singolo numero di `margin()` non significa nulla – vedi `classify_margins.m` e la pagina su `hm3LoopTest`.

**Possibile domanda d'esame** – Perche' verifichi `max(real(pole(G)))` e non confronti tutti i poli con valori attesi? *Risposta:* Perche' l'invariante fisico che voglio proteggere e' *uno solo*: il polo dominante instabile, che e' la firma dell'instabilita' aerodinamica ed e' l'unica quantita' che ha una forma chiusa ( $+\sqrt{A_6}$ ) derivabile dalla teoria. Gli altri tre poli (l'integratore in zero, il polo stabile a -1.86, il polo lento a +0.029) sono artefatti dell'accoppiamento con la deriva laterale, non hanno un'espressione analitica pulita, e pinnarli significherebbe scrivere dei golden value senza contenuto teorico. Il test resta cosi' leggibile: dice "l'aereo diverge a  $\sqrt{A_6}$ ", che e' la frase che voglio poter difendere.

## testRigidMeasurementsEqualTrueStates (righe 65-70)

```
G = build_plant_rigid(testCase.p);
testCase.verifyEqual(G.C(1,:), G.C(5,:), 'AbsTol', 1e-15); % theta
testCase.verifyEqual(G.C(3,:), G.C(6,:), 'AbsTol', 1e-15); % z
```

- Righe 68-69: la  $C$  di `build_plant_rigid` ha 7 righe, nell'ordine `[theta_m, thetadot_m, z_m, zdot_m, theta, z, zdot]`. Le prime quattro sono le **misure** che vanno al controllore; le ultime tre sono segnali di **plot** (gli stati veri). Il test verifica che, in assenza di bending, riga 1 (`theta_m`) e riga 5 (`theta`) siano identiche, e cosi' riga 3 (`z_m`) e riga 6 (`z`).
- Perche' e' un invariante e non una banalita': stabilisce il **punto di riferimento** rispetto a cui il Task 2 e' un cambiamento. Nel modello rigido misura e stato coincidono; nel modello completo con INS **non coincidono piu'** (test 8), ed e' proprio quello scarto che destabilizza l'anello a  $\omega_{BM}$ . Il test fissa il "prima" del confronto prima/dopo.

- AbsTol 1e-15: sono zeri e uni esatti, non numeri calcolati. Si poteva usare `isequal`; la tolleranza e' innocua.

## testFullPlantBendingMode (righe 72-80)

```
G = build_plant_full(pp);
[wn, zeta] = damp(G);
[~, k] = min(abs(wn - pp.wBM));
testCase.verifyEqual(wn(k), pp.wBM, 'AbsTol', 1e-9);
testCase.verifyEqual(zeta(k), pp.zBM, 'AbsTol', 1e-9);
```

- Righe 76-77: `damp(G)` restituisce frequenze naturali e smorzamenti di **tutti** i modi; la riga 77 seleziona quello con `wn` piu' vicina a `omega_BM = 18.9` rad/s. E' un modo robusto di dire "prendi il modo di bending" senza dipendere dall'ordinamento con cui `damp` restituisce gli autovalori (che non e' garantito). Funziona perche' gli altri modi stanno a 0 / 1.8 / 1.9 rad/s: sono un ordine di grandezza sotto, non c'e' ambiguita'.
- Righe 78-79: **l'invariante e' l'esattezza**, con AbsTol 1e-9. Non "il modo sta circa a 18.9", ma "sta a 18.9 con nove cifre". Perche' si puo' pretendere tanto? Perche' la matrice A di `build_plant_full` (righe 20-25) e' **diagonale a blocchi**:

$$A = \begin{bmatrix} A_{\text{rigid}}(4 \times 4) & 0(4 \times 2) \\ 0(2 \times 4) & A_{\text{bend}}(2 \times 2) \end{bmatrix}$$

Le righe 1-4 hanno colonne 5-6 nulle (il corpo rigido non "sente" eta); le righe 5-6 hanno colonne 1-4 nulle (il bending non e' forzato dagli stati rigidi). Quindi **lo spettro del plant e' l'unione esatta dei due spettri**, e il blocco di bending e' l'oscillatore canonico

$$A_{\text{bend}} = \begin{bmatrix} 0 & 1 \\ -w^2 & -2*z*w \end{bmatrix} \rightarrow s^2 + 2*zeta_{\text{BM}}*omega_{\text{BM}}*s + omega_{\text{BM}}^2 = 0$$

i cui autovalori sono per costruzione a frequenza naturale `omega_BM` e smorzamento `zeta_BM`. Il test verifica quindi che `build_plant_full` scriva correttamente quel blocco 2x2, e la tolleranza 1e-9 lascia margine solo all'errore numerico di `eig`. Verificato: `damp` restituisce esattamente 18.9000 / 0.0050.

- **Il punto concettuale, che e' il piu' importante di tutta la pagina:** il bending **non** e' accoppiato al corpo rigido nella matrice A. L'accoppiamento passa da altre due strade:
  - via B:  $B_d(6) = -\phi_{\text{tvc}} * T_c$  (riga 27) – il TVC, deviando la spinta, eccita il modo flessionale;
  - via C: eta contamina le misure INS (righe 33-36) – il sensore, montato sul corpo flessibile, legge assetto vero **piu'** deformazione.

In **anello aperto** questi due percorsi non si chiudono, e infatti il plant completo ha esattamente gli stessi autovalori del plant rigido piu' la coppia di bending: il modo flessionale, con `zeta_BM = 0.005`, e' *poco smorzato ma stabile*. E' **la chiusura dell'anello** (delta -> eta -> misura -> delta) che crea il percorso di retroazione a `omega_BM` e lo rende instabile. Questo spiega perche' il test sul plant non puo' rilevare l'instabilita' di bending: quella vive in `hm3LoopTest` (`testBareFullModelIsBendingUnstable`).

**Possibile domanda d'esame** – Se il modo di bending e' stabile nel plant, perche' l'anello chiuso e' instabile? *Risposta:* Perche' l'instabilita' nasce dall'anello, non dal modo. In anello aperto la matrice A e' diagonale a blocchi e il bending e' un oscillatore stabile a `zeta = 0.005`. Chiudendo l'anello si crea il percorso delta -> eta (forzamento TVC,  $B(6,1) = -\phi_{\text{tvc}}*T_c$ ) -> `theta_m` (contaminazione INS,  $C(1,5) = \sigma_{\text{ins}}$ ) -> `u_pd` -> delta. A `omega_BM` lo smorzamento e' talmente basso che il guadagno d'anello ha un picco di +29 dB: se la fase in quel punto e' sfavorevole, il criterio di Nyquist da' instabilita'. La cura e' abbassare quel picco sotto 0 dB con un notch (gain stabilisation), che e' quello che fa il Task 2.

## testInsMeasurementsContaminatedByBending (righe 82-88)

```
G = build_plant_full(pp, 'ins');
testCase.verifyEqual(G.C(1,5), pp.sigma_ins, 'AbsTol', 1e-15);
testCase.verifyEqual(G.C(3,5), -pp.phi_ins, 'AbsTol', 1e-15);
```

- Righe 86-87: pinna l'Eq. (2) della traccia, cioè il modello di sensore:

```
theta_m = theta + sigma_ins * eta      (sigma_ins = 0.178 rad/m)
z_m      = z      - phi_ins  * eta      (phi_ins   = 0.8)
```

$\sigma_{ins}$  e' la **pendenza** della forma modale nel punto in cui e' montata la piattaforma inerziale (rad di rotazione locale per metro di deformazione generalizzata): il giroscopio non misura l'assetto del corpo rigido, misura l'assetto **della sezione in cui e' avvitato**, che ruota anche per flessione.  $\phi_{ins}$  e' invece l'**ampiezza** della forma modale nello stesso punto (spostamento laterale per unita' di  $\eta$ ).

- **L'invariante e' il segno**, e non e' un dettaglio contabile. Il segno di  $C(1,5)$  decide la **fase** con cui il lobo di bending compare nel guadagno d'anello  $L$ , e quindi se il modo si presenta sulla carta di Nichols dalla parte "buona" o "cattiva" del punto critico. Un errore di segno qui produrrebbe un anello che sembra stabilizzarsi senza notch (o che si destabilizza in modo diverso), e tutto il trade dei quattro filtri del Task 2 risponderebbe a un problema che non esiste. I due segni opposti (+ su  $\theta$ , - su  $z$ ) sono l'unica cosa che questi due `verifyEqual` proteggono, ed e' esattamente cio' che serve.
- Nota: il test verifica solo le colonne di  $\eta$  (colonna 5). Le colonne di  $\dot{\eta}$  (colonna 6, righe 2 e 4 della C:  $\dot{\theta}_m = \dot{\theta} + \sigma_{ins}\dot{\eta}$ ,  $\dot{z}_m = \dot{z} - \phi_{ins}\dot{\eta}$ ) **non sono testate**. E' una lacuna minore ma reale: un errore di segno su  $C(2,6)$  passerebbe. Dato che il feedback derivativo  $K_d \cdot \dot{\theta}_m$  e' proprio quello che tocca il bending in alta frequenza, sarebbe il termine da coprire per primo se si volesse rinforzare la suite.
- Nota 2: nemmeno il termine di forzamento  $Bd(6) = -\phi_{ins} \cdot T_c$  (riga 27 di `build_plant_full.m`) e' coperto da un test. E' l'altra meta' dell'accoppiamento di bending.

---

## testTrueMeasurementsBypassBending (righe 90-94)

```
G = build_plant_full(testCase.p, 'true');
testCase.verifyEqual(G.C(1:4, 5:6), zeros(4, 2), 'AbsTol', 1e-15);
```

- Riga 93: nella variante 'true' il blocco di misura (righe 1-4 della C) ha **colonne di bending identicamente nulle**: il controllore riceve gli stati veri, come se il sensore fosse montato su un corpo rigido ideale.
- A cosa serve questa variante: e' il **controfattuale diagnostico**. Permette di dimostrare che l'instabilita' del Task 2 e' colpa della *contaminazione del sensore* e non della *dinamica flessibile in se'*. Con 'true' il bending c'e' ancora nel plant (viene ancora eccitato dal TVC via  $B(6,1)$ ), ma non torna indietro nel controllore, l'anello non si chiude su  $\omega_{BM}$  e resta stabile. E' il modo pulito di isolare la causa. Un solo `verifyEqual` su una sottomatrice 4x2 basta a pinnare l'intera semantica dell'opzione.

---

## testFullPlantRejectsUnknownMeas (righe 96-99)

```
testCase.verifyError(@( ) build_plant_full(testCase.p, 'bogus'), ...
    'build_plant_full:meas');
```

- Righe 97-98: verifica che un valore non riconosciuto di `meas` sollevi l'errore con **identificatore** `build_plant_full:meas` (riga 43 del sorgente), non un errore generico. L'`arguments` block (righe 13-16) valida solo che `meas` sia testo scalare; e' lo `switch ... otherwise` a rifiutare i valori fuori dominio.
- Perche' conta: senza il ramo `otherwise`, uno `switch` MATLAB su una stringa sconosciuta **non fa nulla** e `Cm` resterebbe indefinita -> errore criptico piu' a valle, oppure (peggio, se `Cm` fosse preinizializzata) un plant silenziosamente sbagliato. Testare l'**identificatore** e non il messaggio e' la scelta corretta: i messaggi si riscrivono, gli ID sono il contratto d'errore.

## Cosa NON e' coperto (limiti noti)

Per onesta' e perche' all'orale puo' venire chiesto "quanto ti fidi di questa suite":

- **Coefficienti `a_1`, `a_3` nella matrice `A`.** Nessun test verifica che le righe 2 e 4 della `A` (l'accoppiamento deriva-beccheggio) siano scritte correttamente. Un `a_1*V + a_4` diventato `a_1*V - a_4` passerebbe tutti i test di questa suite (sposterebbe il polo dominante, ma probabilmente ancora entro la `RelTol` del 2 %). L'unico presidio indiretto e' che i margini del Task 1 cambierebbero e farebbe fallire `hm3LoopTest`.
- **Colonna del disturbo `Bw`.** Nessun test tocca `Bw = [0; -a1*V; 0; -A6]`. Il segno negativo davanti ad `A6` implica che l'incidenza aerodinamica interna al plant sia `alpha = theta + zdot/V - alpha_w`; vedi la nota di onesta' nella pagina su `hm3LoopTest`, dove questa convenzione **non** coincide con quella usata da `simulate_gust_response.m`.
- `Bd(6) = -phi_tvc * T_c` e le colonne di `etadot` nella `C`, come detto sopra.
- `p.Alt` e' un letterale mai cross-checkato con l'LPV.

## Possibili domande d'esame

**D: Il lanciatore ha un polo a +1.84 rad/s. Che cosa significa in pratica e come lo hai verificato?**

R: Significa che, senza controllo, un errore di assetto cresce come  $e^{(1.84*t)}$ : raddoppia in  $\ln(2)/1.84 = 0.38$  s. E' l'instabilita' aerodinamica dovuta al centro di pressione davanti al centro di massa ( $\mu\_alpha = A\_6 > 0$ ). Analiticamente, congelando la deriva laterale la dinamica di beccheggio si riduce a  $\theta\_ddot = A\_6*\theta$ , con poli  $\pm \sqrt{A\_6}$ . Il test `testRigidAirframeUnstablePole` confronta  $\max(\text{real}(\text{pole}(G)))$  con  $\sqrt{A\_6}$  a meno del 2 %: il plant reale, che include l'accoppiamento con la deriva, da' +1.8165 contro  $\sqrt{A\_6} = 1.8390$ , cioe' l'1.2 % di scarto. Il polo instabile esiste, sta dove la teoria dice, e fissa la banda minima di controllo.

**D: Perche' il tuo test sul modo di bending pretende una tolleranza di 1e-9, mentre quello sul polo rigido si accontenta del 2 %?** R: Perche' i due invarianti hanno natura diversa. Il modo di bending e' **esatto per costruzione**: la matrice `A` e' diagonale a blocchi, il blocco flessionale e' l'oscillatore canonico  $[0 \ 1; -\omega\_BM^2, -2\zeta\_BM\omega\_BM]$ , e i suoi autovalori sono per definizione a frequenza  $\omega\_BM$  e smorzamento  $\zeta\_BM$ . Non c'e' approssimazione, solo errore di `eig`, quindi posso pretendere 1e-9. Il polo rigido invece e'  $\pm\sqrt{A\_6}$  solo nel **limite disaccoppiato**: nel plant a 4 stati l'accoppiamento con la deriva laterale (i termini `a_1`, `a_4`, `A_6/V`) lo sposta dell'1.2 %. La tolleranza del 2 % e' la larghezza di quella approssimazione, non sciatteria.

**D: Che cosa rende instabile l'anello nel Task 2, se il modo di bending nel plant e' stabile?** R: La retroazione. In anello aperto il bending e' disaccoppiato dal corpo rigido nella matrice `A` e ha smorzamento  $0.005 > 0$ , quindi e' stabile. Chiudendo l'anello si crea il ciclo delta -> eta (il TVC eccita il modo, `B(6,1) = -phi_tvc*T_c`) -> `theta_m` (l'INS misura la deformazione, `C(1,5) = sigma_ins = 0.178`) -> `u_pd` -> delta. A  $\omega\_BM = 18.9$  rad/s lo smorzamento bassissimo produce un picco di risonanza di +29 dB nel guadagno d'anello, ben sopra 0 dB, e con fase sfavorevole: Nyquist da' instabilita'. `hm3PlantTest` pinna gli ingredienti (il forzamento e la contaminazione), `hm3LoopTest` pinna la conseguenza (l'anello nudo e' instabile).

**D: A cosa serve l'opzione `meas = 'true'` di `build_plant_full`, visto che il lanciatore reale ha sempre un INS contaminato?** R: E' uno strumento diagnostico, non un modello di volo. Serve a isolare la causa dell'instabilita': con `'true'` il bending e' ancora nel plant e viene ancora eccitato dal TVC, ma non torna nel

controllore, e l'anello resta stabile. Questo dimostra sperimentalmente che la colpa e' della **contaminazione della misura** (Eq. 2) e non della dinamica flessibile in se'. Il test `testTrueMeasurementsBypassBending` verifica che nella variante `'true'` le colonne di `eta` ed `etadot` nel blocco di misura siano identicamente nulle.

**D: I test sui nomi dei segnali sembrano cosmetici. Perche' ci sono?** R: Non sono cosmetici: sono un test di **contratto**. `assemble_loop.m` chiude l'anello con `connect()`, che collega i blocchi **per nome** dei segnali (`'delta'`, `'theta_m'`, `'alpha_w'`, ...). Se rinominassi un ingresso del plant, `build_plant_rigid` continuerebbe a restituire un `ss` corretto e tutti i test numerici passerebbero, ma `connect` non troverebbe piu' il segnale, e l'anello si chiuderebbe in modo sbagliato o incompleto. Testare i nomi e' l'unico modo di proteggere quell'accoppiamento implicito.

**D: `testDynamicPressureSelfConsistent` non e' una tautologia? Riscrive la stessa formula del codice.**

R: In buona parte si', ed e' giusto ammetterlo: e' un *change-detector*, non un test di correttezza fisica. Se `H_scale` fosse sbagliato, il test non se ne accorgerebbe, perche' la costante 8000 e' hard-coded in entrambi i posti. Quello che il test garantisce e' che `p.qbar` sia effettivamente **derivato** da `V` e `Alt` e non sia un letterale scollegato: se domani cambio la quota di riferimento o riscalo `V`, `qbar` si muove di conseguenza. Ha valore perche' `qbar` entra nell'indicatore di carico `qbar*alpha` del Task 1, che e' il numero che uso per giustificare il progetto a `max-qbar`.

# HM3/tests/hm3FilterTest.m

## Ruolo del file nel progetto

E' la suite `matlab.unittest` che congela i **due blocchi di compensazione** inseriti in serie fra il controllore PD e il plant: `build_tvc.m` (attuatore TVC del secondo ordine + ritardo di trasporto approssimato con Pade – Eq. 3 della traccia) e `build_notch_filter.m` (la sezione lead-lag / notch – Eq. 4). Sono i due elementi che nel Task 2 trasformano un problema risolto (il PD rigido del Task 1) in un problema difficile: l'attuatore e il ritardo **mangiano fase** alla frequenza di crossover, il notch **abbassa il guadagno** alla frequenza di bending. Il compromesso fra queste due cose e' l'intero Task 2.

La difficolta' di testare questi blocchi e' che non hanno un "risultato" ovvio da confrontare: sono funzioni di trasferimento. La suite risolve il problema verificando **proprieta' analitiche in forma chiusa**, cioe' quantita' che si sanno derivare a mano dalla struttura della `tf` e che si possono quindi pretendere con tolleranze severissime (1e-9), invece che golden value numerici. Le proprieta' scelte sono quattro:

1. **Guadagno statico unitario** (TVC e notch): un blocco di compensazione non deve riscalare il guadagno d'anello in continua, altrimenti sposterebbe silenziosamente tutti i margini.
2. **Ordine** della catena: 2 (attuatore) + n (Pade), senza cancellazioni accidentali.
3. **Il Pade riproduce davvero un ritardo**: la fase in eccesso rispetto al solo attuatore vale esattamente  $-\omega\tau$  nella banda di lavoro.
4. **Profondita' e fase del notch**:  $|H(j\omega_x)| = \zeta_N / \zeta_D$  e i **zeri nel semipiano giusto** – il discrimine fra la variante a fase minima (gain stabilisation) e quella a fase non minima (phase stabilisation).

Piu' tre test negativi sull'`arguments` block.

---

## Setup di classe (righe 7-19)

Identico a `hm3PlantTest`: una `PathFixture` che aggiunge HM3/ al path (righe 11-14, con `fileparts(fileparts(...))` che risale da HM3/tests/ a HM3/) e un caricamento unico dei parametri nominali (righe 16-18). I parametri usati qui sono  $w_{TVC} = 70$  rad/s,  $z_{TVC} = 0.7$ ,  $\tau = 0.020$  s,  $w_{BM} = 18.9$  rad/s.

---

## testTvcUnityDcGain (righe 22-26)

```
Wtvc = build_tvc(testCase.p);  
testCase.verifyEqual(dcgain(Wtvc), 1, 'AbsTol', 1e-9);
```

- Riga 25: il guadagno statico dell'intera catena TVC deve essere **esattamente 1**.

**Derivazione.** `build_tvc.m` (righe 15-21) costruisce

$$W_{act}(s) = w_{TVC}^2 / (s^2 + 2*z_{TVC}*w_{TVC}*s + w_{TVC}^2)$$
$$W_{delay}(s) = \text{pade}(\tau, n)$$

$$W_{\text{tvc}}(s) = W_{\text{act}}(s) * W_{\text{delay}}(s)$$

A  $s = 0$ :  $W_{\text{act}}(0) = w_{\text{TVC}}^2 / w_{\text{TVC}}^2 = 1$ . Il Pade approssima  $\exp(-\tau s)$ , che in  $s = 0$  vale 1, e l'approssimante di Pade **preserva esattamente** il valore in  $s = 0$  (e' costruito facendo coincidere i primi  $2n$  termini dello sviluppo di Taylor, a partire dal termine di ordine zero). Quindi  $W_{\text{delay}}(0) = 1$  e il prodotto vale 1. Verificato in MATLAB: `dcgain(Wtvc) = 1.000000000000`.

**Perche' e' l'invariante giusto.** Un attuatore e' un dispositivo che *segue* il comando: se gli chiedo 1 grado di deflessione, a regime deve darmi 1 grado, non 0.9. Se  $W_{\text{tvc}}(0) \neq 1$ , l'attuatore introdurrebbe un **guadagno nascosto** nell'anello: tutti i margini calcolati sul Task 2 sarebbero traslati di  $20 \cdot \log_{10}(W_{\text{tvc}}(0))$  dB rispetto a quelli del Task 1, e il confronto rigido/full del Task 2 misurerebbe in parte un artefatto di normalizzazione invece dell'effetto fisico del ritardo. E' il tipo di bug che non fa esplodere niente e falsa tutte le conclusioni.

Nota: il fatto che il guadagno statico sia unitario **non** dice nulla sul guadagno alla frequenza di crossover – li' l'attuatore attenua e sfasa, ed e' esattamente quello che il test successivo va a misurare.

## testTvcOrderIsActuatorPlusPade (righe 28-32)

```
testCase.verifyEqual(order(build_tvc(testCase.p)), 5); % default n = 3
testCase.verifyEqual(order(build_tvc(testCase.p, 2)), 4);
```

- Righe 30-31: l'ordine della catena e'  $2 + n$ , con  $n = 3$  di default (5 stati) e  $n = 2$  se passato esplicitamente (4 stati). Verificato: 5 e 4.
- Che cosa protegge davvero.** Sembra contabilita', ma cattura due errori reali:
  - una **cancellazione polo-zero accidentale** (se l'attuatore e il Pade avessero un fattore comune, l'ordine scenderebbe e la fase non sarebbe piu' quella attesa);
  - un cambio silenzioso dell'ordine di default del Pade. Il punto (b) e' importante perche' l'ordine del Pade determina **fino a che frequenza l'approssimazione e' fedele**: un Pade di ordine  $n$  riproduce bene  $\exp(-j\omega\tau)$  per  $\omega\tau$  piccolo, e degrada oltre. Con  $\tau = 0.020$  s, la scala di frequenza del ritardo e'  $1/\tau = 50$  rad/s. Il modo di bending sta a  $\omega_{\text{BM}} = 18.9$  rad/s, cioe'  $\omega\tau = 0.378$ : non e' banalmente piccolo. Il commento di `build_tvc.m` (righe 5-6) dice esplicitamente che un ordine piu' alto da' una fase migliore vicino a  $\omega_{\text{BM}}$ . Se qualcuno abbassasse il default a 1, il notch verrebbe progettato contro una fase sbagliata: il test lo blocca.
- La riga 31 e' anche l'unico posto della suite che esercita il **secondo argomento opzionale** di `build_tvc` con un valore valido (la riga 83 lo passa solo per farlo rifiutare dal validatore).

**Possibile domanda d'esame** – Perche' Pade di ordine 3 e non un ritardo esatto? *Risposta:* Perche' tutta l'analisi di HM3 e' basata su strumenti razionali: `allmargin`, `pole`, `isstable`, `minreal`, `connect`. Un ritardo puro  $\exp(-\tau s)$  non e' razionale, non ha una realizzazione in spazio di stato finita, e la Control System Toolbox lo tratterebbe come `InternalDelay`, rendendo impossibile `isstable(T)` sull'anello chiuso e complicando `allmargin`. Il Pade lo rende una `tf` a coefficienti reali, al prezzo di un errore che cresce con la frequenza. L'ordine 3 e' un compromesso: e' esatto in fase entro  $1e-6$  rad a 2 rad/s (dove leggo il margine di fase rigido) e ancora ragionevole a 18.9 rad/s (dove valuto il bending). Nota che il Pade sposta anche il **modulo** in modo lieve, mentre il ritardo vero e' all-pass esatto: e' un'approssimazione, non una riscrittura.

## testTvcDelayPhaseAtLowFrequency (righe 34-43)

```
wTest = 2.0; % rad/s, low frequency
h = freqresp(build_tvc(pp), wTest);
hA = freqresp(tf(pp.wTVC^2, ...
    [1, 2*pp.zTVC*pp.wTVC, pp.wTVC^2]), wTest);
```



```
phaseDelay = angle(h) - angle(hA);
testCase.verifyEqual(phaseDelay, -wTest*pp.tau, 'AbsTol', 1e-6);
```

E' il test piu' istruttivo del file: **verifica che il ritardo sia un ritardo.**

- Riga 38: la frequenza di prova e' 2.0 rad/s. **Non e' arbitraria:** e' la banda in cui vive il crossover di controllo del Task 1 (2.45 rad/s secondo il README, 2.455 rad/s verificato). Il test controlla la fedelta' dell'approssimazione **proprio dove viene letto il margine di fase**, che e' l'unico posto dove conta davvero.
- Righe 39-40: si calcola la risposta in frequenza della catena completa (h) e quella del **solo attuatore** (hA), ricostruito a mano dalla stessa formula usata in build\_tvc.m riga 16.
- Riga 41: la differenza di fase **isola il contributo del Pade:**

```
arg(W_tvc) = arg(W_act) + arg(W_delay)
-> arg(W_delay) = arg(W_tvc) - arg(W_act)
```

E' l'unico modo di testare il fattore di ritardo senza risporlo dall'interno della funzione: si sfrutta il fatto che la fase di un prodotto e' la somma delle fasi. Trucco pulito e riusabile.

- Riga 42: il valore atteso e' **la definizione stessa di ritardo di trasporto:**

```
exp(-j*omega*tau) -> modulo 1, fase = -omega*tau
```

A  $\omega = 2$  rad/s e  $\tau = 0.020$  s:  $-\omega\tau = -0.04$  rad =  $-2.29$  deg. Verificato in MATLAB:  $\text{phaseDelay} = -0.04000000$  rad, cioe' l'errore del Pade di ordine 3 a questa frequenza e' sotto  $1e-8$  rad. La  $\text{AbsTol}$  di  $1e-6$  e' quindi onestamente raggiungibile e non e' una tolleranza gonfiata.

- **Perche' proprio questo e' l'invariante giusto.** Il ritardo e' pericoloso non perche' attenua (non attenua:  $|\exp(-j\omega\tau)| = 1$ ), ma perche' **sfasa in modo proporzionale a omega**. In un progetto in frequenza il ritardo e' un consumatore puro di margine di fase, e il suo costo cresce linearmente con la banda: e' la ragione per cui non si puo' semplicemente "alzare il guadagno" per gestire il bending. Il test verifica proprio la costante di proporzionalita' di questo consumo. Se il Pade fosse costruito con  $\tau$  in millisecondi invece che in secondi, l'anello sembrerebbe avere 1000 volte piu' margine di ritardo e il test fallirebbe immediatamente.
- Contesto quantitativo utile all'orale: a  $\omega = 2.45$  rad/s (crossover rigido) il ritardo toglie  $2.45 * 0.020 = 0.049$  rad =  $2.8$  deg; l'attuatore, misurato, ne toglie altri  $\sim 2.3$  deg a 2 rad/s. Insieme sono  $\sim 5$  deg su un margine di 30 deg: gestibili. E' il **notch** che, aggiungendo il suo ritardo di fase, fa crollare il margine da 30 a 14.6 deg (README, tabella Task 2) e costringe al re-tuning del PD.
- Piccola nota di onesta': il test ricostruisce  $W_{\text{act}}$  **riscrivendo la stessa formula** del sorgente. La parte "attuatore" del confronto e' quindi tautologica; il contenuto informativo sta tutto nella differenza di fase, che isola il Pade. Va bene cosi', ma va detto.

**Possibile domanda d'esame** – Perche' verifichi la fase del ritardo a 2 rad/s e non a  $\omega_{\text{BM}} = 18.9$  rad/s, dove il ritardo e' molto piu' grande? *Risposta:* Perche' a 2 rad/s l'approssimante di Pade e' praticamente esatto (errore  $< 1e-8$  rad), quindi posso scrivere un'asserzione stringente con un valore atteso in forma chiusa ( $-\omega\tau$ ) e nessun golden value. A 18.9 rad/s il Pade di ordine 3 devia visibilmente dal ritardo ideale ( $\omega\tau = 0.378$  non e' piccolo), e l'unico valore atteso possibile sarebbe un numero pinnato, cioe' un golden value senza contenuto teorico. Ho scelto di testare la proprieta' dove e' esatta e dove serve: 2 rad/s e' la banda del crossover rigido, che e' esattamente dove leggo il margine di fase.

---

## testNotchUnityGainFarFromCentre (righe 45-50)

```
Hx = build_notch_filter(wx, 0.002, 0.7, +1);
testCase.verifyEqual(dcgain(Hx), 1, 'AbsTol', 1e-9);
testCase.verifyEqual(abs(freqresp(Hx, 1e4)), 1, 'AbsTol', 1e-3);
```

- Righe 48-49: il notch ha guadagno unitario a **entrambi gli estremi** dello spettro: in continua e a 1e4 rad/s (tre ordini di grandezza sopra omega\_BM).

**Derivazione.** Il filtro (righe 20-23 di build\_notch\_filter.m) e'

$$H(s) = \frac{(s^2 + \text{sgn} * 2 * \text{zeta}_N * \omega_x * s + \omega_x^2)}{(s^2 + 2 * \text{zeta}_D * \omega_x * s + \omega_x^2)}$$

- A  $s \rightarrow 0$ : numeratore e denominatore tendono entrambi a  $\omega_x^2 \rightarrow H(0) = 1$  **esattamente**, per qualunque  $\text{zeta}_N$  e  $\text{zeta}_D$ . Da qui la AbsTol di 1e-9.
- A  $s \rightarrow \infty$ : numeratore e denominatore sono entrambi dominati da  $s^2 \rightarrow |H| \rightarrow 1$ , ma solo **asintoticamente**. Il residuo si ricava in forma chiusa dal modulo quadro:

$$|H(j * \omega)|^2 = \frac{(\omega^2 - \omega_x^2)^2 + (2 * \text{zeta}_N * \omega_x * \omega)^2}{(\omega^2 - \omega_x^2)^2 + (2 * \text{zeta}_D * \omega_x * \omega)^2}$$

Per  $\omega \gg \omega_x$  lo scostamento e' **quadratico**, non lineare:  $1 - |H| \sim (1/2) * (2 * \text{zeta}_D * \omega_x / \omega)^2$ . A  $\omega = 1e4$ , con  $\text{zeta}_D = 0.7$  e  $\omega_x = 18.9$ :  $0.5 * (2 * 0.7 * 18.9 / 1e4)^2 = 3.5e-6$ , e infatti  $\text{abs}(\text{freqresp}(Hx, 1e4)) = 0.9999965$ . La AbsTol di 1e-3 e' quindi **abbondantemente lasca** (basterebbe 1e-5): non e' calibrata sull'asintoto, e' solo un margine prudenziale. E' l'unica tolleranza del file che non sia stretta al limite numerico, e conviene saperlo dire prima che lo chieda il professore.

**Perche' e' l'invariante giusto.** E' la proprieta' che rende il notch **usabile**: e' una modifica **locale in frequenza**. Deve scavare un buco attorno a  $\omega_{BM}$  e **non toccare niente altrove** – in particolare non deve alterare il guadagno d'anello attorno al crossover rigido (2-3 rad/s), dove vive il margine di fase, ne' il guadagno in bassa frequenza, dove vive il margine di guadagno aerodinamico. Se il notch avesse un guadagno statico diverso da 1, tutti i margini di bassa frequenza del Task 2 sarebbero traslati e il confronto con il Task 1 non significherebbe piu' nulla.

Attenzione pero': "guadagno unitario **in modulo** lontano dal centro" non vuol dire "invisibile lontano dal centro". Il notch continua a introdurre **fase** anche lontano dal buco – ed e' proprio quella fase residua a far crollare il margine da 30 a 14.6 deg nel Task 2. Il test verifica il modulo, non la fase, e va detto esplicitamente: **il test non cattura il costo reale del notch**, che e' in fase, non in guadagno. Quel costo emerge solo a valle, in hm3LoopTest e nei main\_task2.m.

## testNotchDepthIsZetaRatio (righe 52-59)

```
wx = testCase.p.wBM; zN = 0.002; zD = 0.7;
Hn = build_notch_filter(wx, zN, zD, +1);
Hl1 = build_notch_filter(wx, zN, zD, -1);
testCase.verifyEqual(abs(freqresp(Hn, wx)), zN/zD, 'AbsTol', 1e-9);
testCase.verifyEqual(abs(freqresp(Hl1, wx)), zN/zD, 'AbsTol', 1e-9);
```

E' il test che pinna il **buco**: dove sta, quanto e' profondo, e che non dipende dal segno.

**Derivazione (da saper fare alla lavagna).** Valutiamo  $H(s)$  in  $s = j * \omega_x$ , cioe' esattamente al centro:

$$\begin{aligned} \text{numeratore: } & (j * \omega_x)^2 + \text{sgn} * 2 * \text{zeta}_N * \omega_x * (j * \omega_x) + \omega_x^2 \\ & = -\omega_x^2 + \text{sgn} * 2 * \text{zeta}_N * \omega_x^2 * j + \omega_x^2 \\ & = j * \text{sgn} * 2 * \text{zeta}_N * \omega_x^2 \end{aligned}$$

$$\begin{aligned}\text{denominatore: } & (j*w_x)^2 + 2*zD*w_x*(j*w_x) + w_x^2 \\ & = j^2 * 2 * zD * w_x^2\end{aligned}$$

I due termini reali ( $-\omega_x^2$  e  $+\omega_x^2$ ) si **cancellano esattamente** in entrambi. Resta un rapporto puramente immaginario:

$$H(j*w_x) = (j*sgn*2*zN*w_x^2) / (j*2*zD*w_x^2) = sgn * zN/zD$$

$$\rightarrow |H(j*w_x)| = zN / zD \quad (\text{indipendente da } sgn \text{ e da } w_x)$$

Ecco perché i due `verifyEqual` alle righe 57-58 confrontano **entrambe le varianti di segno** con lo stesso valore: il segno cambia la **fase** (0 oppure 180 deg al centro), non il **modulo**. E' un invariante forte, in forma chiusa, che giustifica una `AbsTol` di  $1e-9$ .

**La formula della profondita'** e' quindi:

$$\text{profondita' [dB]} = 20*\log_{10}(zeta_N / zeta_D)$$

Con i valori del progetto ritenuto ( $zeta_N = 0.002$ ,  $zeta_D = 0.7$ ):

$$zN/zD = 2.857e-3 \rightarrow 20*\log_{10}(2.857e-3) = -50.9 \text{ dB}$$

Verificato in MATLAB: -50.88 dB, che e' il “-51 dB depth” citato nel README. **Il notch e' un rapporto di smorzamenti, non un guadagno da tarare:** la profondita' si progetta scegliendo  $zeta_N$  molto piccolo (zero quasi sull'asse immaginario, che quasi cancella il polo di bending) e  $zeta_D$  dell'ordine dell'unita' (polo ben smorzato, che non aggiunge risonanza propria). Questa e' la ragione fisica per cui si chiama “notch”: lo zero poco smorzato scava, il polo smorzato riempie e richiude.

**Cosa il test NON dice.** Verifica il valore di  $|H|$  in  $\omega_x$ , non che  $\omega_x$  sia il **punto di minimo** di  $|H|$ . Per  $zeta_N < zeta_D$  il minimo cade (essenzialmente) in  $\omega_x$ , quindi in pratica coincide, ma formalmente l'asserzione e' piu' debole di “il buco e' alla frequenza giusta”. Inoltre il test passa `wx = p.wBM` ma **la funzione non sa nulla di  $\omega_{BM}$** : e' il chiamante a doverla centrare. Che il notch sia effettivamente centrato sul bending **nel loop reale** e' verificato altrove, in `hm3LoopTest.testDeepNotchStabilisesFullModel`, dove si controlla  $|L(j*\omega_{BM})| < -10 \text{ dB}$ .

**Il prezzo del notch profondo (onesto').** Il README lo dice e va ripetuto: la larghezza del buco e' proporzionale a  $zeta_D * \omega_x$ , ma la sua **parte utile** (dove l'attenuazione e' davvero forte) e' strettissima con  $zeta_N = 0.002$ . Il risultato e' che il progetto tollera -10 % di detuning di  $\omega_{BM}$  ma diventa instabile a +5 %. Un notch a -51 dB e' un notch che **pretende di conoscere  $\omega_{BM}$** . Nessun test in questo file cattura quella fragilita': e' verificata solo dai sweep di `main_task2.m` e dal Monte Carlo.

**Possibile domanda d'esame** – Come scegli la profondita' di un notch? *Risposta:* La profondita' e' fissata analiticamente dal rapporto degli smorzamenti,  $20*\log_{10}(zeta_N/zeta_D)$  dB, e va scelta in modo che il picco di risonanza del bending finisca sotto il requisito di gain margin flessionale. Nel mio caso il picco nudo e' +29 dB e il requisito tipico e' -12 dB (cioe' 12 dB di margine sotto lo 0 dB): serve almeno ~41 dB di attenuazione. Con  $zeta_N = 0.002$  e  $zeta_D = 0.7$  ottengo -51 dB, che porta il lobo a  $|L(\omega_{BM})| = -21.9 \text{ dB}$  con i guadagni del Task 1 e a -18 dB dopo il re-tuning: sopra i 12 dB richiesti. Il prezzo e' la sensibilita' al detuning di  $\omega_{BM}$  e la fase persa attorno al crossover.

## testDefaultVariantIsNonMinimumPhase (righe 61-65)

```
% sgn = -1 (assignment Eq. 4 as printed): both zeros in the RHP
Hx = build_notch_filter(testCase.p.wBM, 0.2, 0.5);
testCase.verifyTrue(all(real(zero(Hx)) > 0));
```

## testMinimumPhaseVariantHasLhpZeros (righe 67-70)

```
Hx = build_notch_filter(testCase.p.wBM, 0.2, 0.5, +1);
testCase.verifyTrue(all(real(zero(Hx)) < 0));
```

Questi due test vanno letti insieme: sono la **distinzione concettuale piu' importante di tutto il Task 2**.

**Derivazione.** Gli zeri sono le radici di  $s^2 + \text{sgn} \cdot 2 \cdot \text{zeta}_N \cdot \omega_x \cdot s + \omega_x^2$ :

$$s = \omega_x \cdot ( -\text{sgn} \cdot \text{zeta}_N \pm j \cdot \sqrt{1 - \text{zeta}_N^2} )$$

$$\rightarrow \text{Re}(s) = -\text{sgn} \cdot \text{zeta}_N \cdot \omega_x$$

Quindi: -  $\text{sgn} = -1$  (default, l'Eq. 4 come stampata nella traccia):  $\text{Re}(s) = +\text{zeta}_N \cdot \omega_x > 0 \rightarrow$  **entrambi gli zeri nel semipiano DESTRO**  $\rightarrow$  sistema a fase non minima (NMP). Verificato con  $\text{zeta}_N = 0.2$ ,  $\omega_x = 18.9$ : zeri in  $+3.78 \pm j 18.52$ , e infatti  $+0.2 \cdot 18.9 = 3.78$ . -  $\text{sgn} = +1$ :  $\text{Re}(s) = -\text{zeta}_N \cdot \omega_x < 0 \rightarrow$  **zeri nel semipiano SINISTRO**  $\rightarrow$  fase minima. Verificato con la parametrizzazione del progetto ritenuto ( $\text{zeta}_N = 0.002$ ): zeri in  $-0.0378 \pm j 18.9$ , e infatti  $-0.002 \cdot 18.9 = -0.0378$ .

**Perche' questa distinzione e' il cuore del Task 2.** Un sistema a fase minima ha la fase **univocamente determinata dal modulo** (relazione di Bode guadagno-fase): se il modulo scende, la fase scende di conseguenza, ma non c'e' "lag extra". Un sistema a fase **non** minima aggiunge fase negativa **a parita' di modulo**: gli zeri RHP si comportano, in fase, come poli. Questo si traduce nelle due strategie classiche di stabilizzazione dei modi flessionali:

- **Gain stabilisation** (quella ritenuta): si usa il notch a fase minima ( $\text{sgn} = +1$ ) per **abbassare il modulo** del lobo di bending sotto 0 dB con ampio margine. Non importa piu' che fase abbia l'anello a  $\omega_{\text{BM}}$ : se  $|L| \ll 1$ , la curva di Nichols non puo' circondare il punto critico li'. Richiede pero' di **conoscere  $\omega_{\text{BM}}$**  con precisione (il buco e' stretto).
- **Phase stabilisation**: si lascia il lobo sopra 0 dB e si **modella la fase** perche' il lobo passi dalla parte giusta del punto critico. E' quello che fa la sezione lead-lag a fase **non minima** dell'Eq. 4 come stampata. E' piu' robusta al detuning in frequenza (non c'e' un buco stretto da centrare) ma molto piu' delicata: se la fase e' sbagliata di poco, il modo diventa instabile invece che stabilizzarsi.

La tabella del Task 2 nel README mostra il trade quantitativo: il lead-lag da solo (Eq. 4 as printed, migliore di 75 combinazioni provate) porta il bending a +23 dB e lascia un margine di fase rigido di 11.4 deg – "barely" stabile. Il notch profondo lo porta a -21.9 dB. Vince il notch.

**Onesta' – che cosa i due test coprono davvero.** Coprono la **struttura** della *tf* (dove finiscono gli zeri in funzione del segno), **non** la parametrizzazione del filtro effettivamente montato nel progetto. Usano infatti  $\text{zeta}_N = 0.2$ ,  $\text{zeta}_D = 0.5$ , che sono i valori di **linea guida** citati nel commento di *build\_notch\_filter.m* (righe 6-7: "guideline 0.1-0.3" e "0.4-0.6"), non i 0.002 / 0.7 del notch ritenuto. La scelta e' comunque sensata: con  $\text{zeta}_N = 0.002$  gli zeri stanno a -0.0378 di parte reale, cioe' quasi sull'asse immaginario, e un test di segno stretto ( $> 0 / < 0$ ) sarebbe numericamente al limite. Nota inoltre il caso degenerare: con  $\text{zeta}_N = 0$  gli zeri sarebbero **esattamente sull'asse immaginario** e  $\text{real}(\text{zero}(H_x)) < 0$  fallirebbe – eppure *build\_notch\_filter* accetta  $\text{zeta}_N = 0$  (*mustBeNonnegative*, riga 15). E' un buco nella validazione che nessun test copre; nella pratica non si presenta perche' il progetto usa 0.002.

**Possibile domanda d'esame** – L'Eq. 4 della traccia, cosi' com'e' scritta, e' a fase non minima. Perche' l'hai riscritta a fase minima? *Risposta:* Non l'ho riscritta: *build\_notch\_filter* implementa **entrambe** le varianti tramite il parametro *numSign*, e il **default** e' proprio -1, cioe' l'Eq. 4 come stampata (zeri RHP). Ho poi **confrontato** le due nel trade dei filtri del Task 2. La sezione lead-lag NMP e' una tecnica di *phase stabilisation*: lascia il lobo di bending sopra 0 dB e cerca di ruotarne la fase perche' passi dalla parte sicura del punto critico. Nel mio caso, alla migliore delle 75 parametrizzazioni provate, lasciava il bending a +23 dB e collassava il margine di fase rigido a 11.4 deg: stabile ma inaccettabile. Il notch a fase minima (*numSign* = +1) fa *gain stabilisation*, porta il lobo a -21.9 dB, ed e' il progetto che ho ritenuto. Il prezzo, che dichiaro nel report, e' la sensibilita' al detuning di  $\omega_{\text{BM}}$ .

---

## Test negativi sull'arguments block (righe 72-85)

```
testCase.verifyError(@() build_notch_filter(-1, 0.2, 0.5), ...
    'MATLAB:validators:mustBePositive');
testCase.verifyError(@() build_notch_filter(18.9, 0.2, 0.5, 0), ...
    'MATLAB:validators:mustBeMember');
testCase.verifyError(@() build_tvc(testCase.p, 2.5), ...
    'MATLAB:validators:mustBeInteger');
```

- Righe 72-75: frequenza di centro negativa -> `mustBePositive` (riga 14 di `build_notch_filter.m`). Una  $\omega_x$  negativa produrrebbe una `tf` con  $\omega_x^2$  invariato ma il termine lineare col segno sbagliato: silenziosamente scambierebbe la variante NMP con quella a fase minima. Rifiutarla e' giusto.
- Righe 77-80: `numSign = 0` -> `mustBeMember(numSign, [-1, 1])` (riga 17). Con `numSign = 0` il numeratore diventerebbe  $s^2 + \omega_x^2$ , cioè zeri **esattamente sull'asse immaginario**: una cancellazione ideale del modo di bending. E' un progetto matematicamente elegante e **fisicamente indifendibile** (richiede conoscenza esatta di  $\omega_{BM}$  e sarebbe instabile a qualunque detuning). Il validatore lo esclude per costruzione.
- Righe 82-85: ordine di Pade non intero -> `mustBeInteger` (riga 12 di `build_tvc.m`). `pade(tau, 2.5)` non ha senso.
- **Osservazione di metodo**: si verifica l'identificatore dell'errore ('MATLAB:validators:mustBePositive'), non il messaggio. E' la pratica corretta: i messaggi cambiano con la versione di MATLAB e con la lingua dell'installazione, gli ID no. Questi tre test sono anche l'unica documentazione eseguibile del **contratto d'uso** delle due funzioni.

---

## Cosa NON e' coperto (limiti noti)

- **La fase del notch attorno al crossover rigido.** E' il vero costo del notch (fa crollare il margine da 30 a 14.6 deg), e nessun test di questo file la misura. Solo il modulo e' verificato.
- **La larghezza del buco.** Nessun test lega `zeta_D` alla banda di attenuazione, quindi nessun test cattura la fragilita' al detuning (-10 % OK, +5 % instabile).
- `zeta_N = 0` e' accettato dall'arguments block e produrrebbe zeri sull'asse immaginario; nessun test lo documenta.
- **Fedelta' del Pade a  $\omega_{BM}$** , dove  $\omega \cdot \tau = 0.378$  e l'errore non e' trascurabile: testata solo a 2 rad/s.

---

## Possibili domande d'esame

**D: Da dove viene la formula della profondita' del notch, e quanto vale nel tuo progetto?** R: Valutando  $H(s)$  in  $s = j\omega_x$  i termini reali  $-\omega_x^2$  e  $+\omega_x^2$  si cancellano sia al numeratore sia al denominatore, e resta il rapporto di due immaginari puri:  $H(j\omega_x) = \text{sgn} * zeta_N/zeta_D$ . Quindi il modulo al centro e' esattamente  $zeta_N/zeta_D$ , indipendente da  $\omega_x$  e dal segno, e la profondita' in dB e'  $20 \cdot \log_{10}(zeta_N/zeta_D)$ . Con  $zeta_N = 0.002$  e  $zeta_D = 0.7$  il notch ritenuto vale  $2.857e-3$ , cioè **-50.9 dB**. Il test `testNotchDepthIsZetaRatio` verifica questa identita' su entrambe le varianti di segno, con tolleranza  $1e-9$ , perche' e' esatta in forma chiusa.

**D: Che differenza c'e' fra gain stabilisation e phase stabilisation di un modo flessionale, e quale hai usato?** R: Nella gain stabilisation si abbassa il modulo del guadagno d'anello alla frequenza del modo ben sotto 0 dB (tipicamente -12 dB o meglio), così che il lobo non possa circondare il punto critico qualunque sia la sua fase; serve un notch a **fase minima** e la conoscenza precisa di  $\omega_{BM}$ . Nella phase stabilisation si lascia il

lobo sopra 0 dB e si modella la **fase** perche' passi dalla parte sicura; e' quello che fa la sezione lead-lag a **fase non minima** dell'Eq. 4 come stampata nella traccia. Io ho usato la gain stabilisation: il notch a fase minima porta il bending da +29 dB a -21.9 dB. Il lead-lag da solo, alla migliore delle 75 parametrizzazioni provate, lasciava +23 dB e un margine di fase rigido di 11.4 deg.

**D: Come fai a distinguere, con un test, un filtro a fase minima da uno a fase non minima?** R: Guardando i segni delle parti reali degli zeri. Per il mio biquad,  $\text{Re}(\text{zero}) = -\text{numSign} * \text{zeta\_N} * \text{omega\_x}$ : con  $\text{numSign} = -1$  (default, Eq. 4 come stampata) gli zeri stanno nel semipiano destro e il filtro e' NMP; con  $\text{numSign} = +1$  stanno nel semipiano sinistro ed e' a fase minima. I due test `testDefaultVariantIsNonMinimumPhase` e `testMinimumPhaseVariantHasLhpZeros` verificano esattamente questo con `all(real(zero(Hx)) > 0)` e `< 0`. Fisicamente conta perche' uno zero RHP aggiunge ritardo di fase a parita' di modulo: si comporta, in fase, come un polo.

**D: Perche' il ritardo di 20 ms e' un problema, se non attenua nulla?** R: Proprio perche' non attenua: e' all-pass,  $|\exp(-j*\text{omega}*\text{tau})| = 1$ . Il suo effetto e' interamente in fase,  $-\text{omega}*\text{tau}$ , e cresce **linearmente con la frequenza**. Quindi consuma margine di fase in proporzione alla banda che voglio usare, e sul lanciatore la banda non e' negoziabile verso il basso (il polo instabile a +1.84 rad/s impone un crossover ben sopra 1.84). Al mio crossover di 2.45 rad/s il ritardo costa 2.8 deg, che sono gestibili; a 18.9 rad/s costerebbe 21.7 deg, ed e' una delle ragioni per cui il bending si stabilizza in guadagno e non in fase. Il test `testTvcDelayPhaseAtLowFrequency` verifica che il Pade riproduca esattamente questa fase, isolandola dalla fase dell'attuatore per differenza.

**D: Perche' il guadagno statico dell'attuatore e del notch deve essere esattamente 1?** R: Perche' altrimenti introdurrebbero un guadagno d'anello nascosto. Tutto il progetto e' letto sui **margini**, che sono distanze in dB e in gradi rispetto al punto critico: se un blocco "di compensazione" moltiplicasse L per una costante diversa da 1, tutti i margini di bassa frequenza (in particolare il gain margin aerodinamico, letto a 0.59 rad/s) si sposterebbero di  $20*\log_{10}(k)$  dB, e il confronto fra il modello rigido del Task 1 e il modello completo del Task 2 misurerebbe in parte un artefatto di normalizzazione invece dell'effetto fisico dell'attuatore e del bending. E' un errore che non fa esplodere nulla e falsa tutte le conclusioni: da qui i due test con tolleranza  $1e-9$ .

# HM3/tests/hm3LoopTest.m

## Ruolo del file nel progetto

E' la suite di **regressione sulle conclusioni dell'homework**. Mentre `hm3PlantTest` congela la fisica e `hm3FilterTest` congela i blocchi di compensazione, questo file congela i **risultati**: che il PD del Task 1 raggiunga i target di margine assegnati, che il modello completo senza filtro sia **instabile** per colpa del bending, e che il notch profondo lo **stabilizzi in guadagno** (Task 2). Sono esattamente le tre affermazioni che l'homework va a difendere.

Copre quattro file: `load_wind_profile.m`, `assemble_loop.m`, `design_controller.m` e `simulate_gust_response.m`. La struttura logica dei test segue l'ordine narrativo del report:

1. **Il disturbo** (test 1-4): la raffica 1-coseno ha la forma giusta e l'ampiezza giusta.
2. **Il progetto del Task 1** (test 5-7): i guadagni riproducono  $|GM_{aero}| = 6$  dB e  $PM_{rigido} = 30$  deg, e l'auto-tuner li ritrova da solo.
3. **Il problema del Task 2** (test 8): il modello completo nudo e' instabile.
4. **La soluzione del Task 2** (test 9): il notch lo stabilizza.
5. **Il post-processing** (test 10): il bilancio di incidenza aerodinamica.

**Attenzione, questo e' il file dove vivono i golden value.** Diversi test confrontano contro numeri hard-coded ( $K_{ref}$ , 6.0 dB, 30.0 deg, 6.38 m/s) che **non hanno derivazione analitica**: sono l'output di un `fminsearch` su un progetto specifico. Vanno capiti per quello che sono – vedi la sezione dedicata piu' sotto.

---

## Kref – i golden value del progetto (righe 7-13)

```
properties (Constant)
    % Task-1 PD design: gains re-tuned on the FULL loop so the classified
    % Aero GM = 6 dB / Rigid PM = 30 deg (canonical D'Antuono start, then the
    % lateral-drift aero-GM erosion is compensated). Pinned for regression.
    Kref = struct('Kp_th', 1.7845, 'Kd_th', 0.4433, ...
                  'Kp_z', -1e-3, 'Kd_z', -1e-3)
end
```

- Righe 11-12: quattro numeri, usati da **cinque** dei dieci test. Sono i guadagni del **progetto del Task 1**, e coincidono con i valori del README ( $K_p = 1.78$ ,  $K_d = 0.44$ ). Verificato in MATLAB: `design_controller` restituisce oggi  $K_{p\_th} = 1.784455$ ,  $K_{d\_th} = 0.443323$  (entrambi arrotondano ai valori pinnati).
- $K_{p\_z} = K_{d\_z} = -1e-3$  sono i guadagni di **deriva laterale**, fissati (non ottimizzati) come prescrive la traccia. Sono **negativi e piccoli**: e' il cosiddetto *load relief*. Un guadagno positivo su  $z$  cercherebbe di riportare il lanciatore sulla traiettoria nominale contro il vento, aumentando l'incidenza e quindi il carico  $q_{bar\alpha}$ ; un guadagno **negativo** lascia che il veicolo derivi con\* il vento, riducendo l'incidenza. E' il classico compromesso “seguire la traiettoria” contro “sopravvivere al carico”, e a max- $q_{bar}$  vince il secondo.
  - **Ma attenzione a non sovravendere il load relief.** Quella e' l'*intenzione* del segno; l'*effetto*, a  $1e-3$ , e' trascurabile. Misurando il bilancio di incidenza sull'anello effettivo (vedi `testGustResponseAngleOfAttackBudget` piu' sotto) il picco di  $|\alpha|$  vale **0.577 deg** contro i



**0.390 deg** della raffica da sola: l'anello chiuso **aggrava** il carico invece di alleggerirlo. Il PD d'assetto e' dominante e becca il muso dentro il vento relativo; i -1e-3 sulla deriva non bastano a invertire il bilancio. Va detto cosi', senza abbellirlo.

**Da dove vengono  $K_p = 1.78$  e  $K_d = 0.44$  (la derivazione che serve all'orale).** Non sono numeri magici: `design_controller.m` (riga 55) parte da una coppia in **forma chiusa** e poi la raffina. Congelando la deriva laterale, la dinamica di beccheggio e'  $\theta_{ddot} = A_6\theta + K_1\delta$ ; con la legge PD  $\delta = -K_p\theta - K_d\dot{\theta}$  si ottiene

$$\theta_{ddot} + K_1K_d\dot{\theta} + (K_1K_p - A_6)\theta = 0$$

da cui, per confronto con  $s^2 + 2\zeta\omega_n s + \omega_n^2$ :

$$\omega_n = \sqrt{K_1K_p - A_6} \quad \zeta = K_1K_d / (2\omega_n)$$

**Il termine  $-A_6$  e' tutto il problema:** l'instabilita' aerodinamica **sottrae** rigidita all'anello chiuso. Serve  $K_1K_p > A_6$  solo per non divergere. Le gains canoniche di D'Antuono (Eq. 3.6-3.7, citate alla riga 55 del sorgente) sono

$$K_{p0} = 2A_6/K_1 \quad K_{d0} = \sqrt{A_6}/K_1$$

e sostituendo si vede subito cosa fanno:

$$\begin{aligned} \omega_n &= \sqrt{2A_6 - A_6} = \sqrt{A_6} = 1.839 \text{ rad/s} \\ \zeta &= K_1(\sqrt{A_6}/K_1) / (2\sqrt{A_6}) = 1/2 = 0.5 \end{aligned}$$

cioe' **piazzano la coppia di beccheggio in anello chiuso esattamente alla frequenza del polo instabile, con smorzamento 0.5.** E' una scelta elegante: la banda di controllo e' fissata dalla velocita' dell'instabilita' che deve combattere. Numericamente:  $K_{p0} = 1.4816$ ,  $K_{d0} = 0.4029$ .

Da li' `fminsearch` raffina **sull'anello completo** (che include la deriva laterale, che erode il margine aerodinamico) e arriva a  $K_p = 1.7845$ ,  $K_d = 0.4433$ . La coppia equivalente di beccheggio risultante e'

$$\omega_n = \sqrt{4.5647 \cdot 1.7845 - 3.3818} = 2.18 \text{ rad/s} \quad \zeta = 0.46$$

che e' il "equivalent pitch pair  $\omega_n = 2.18$ ,  $\zeta = 0.46$ " del README (verificato). Restare in banda 1-4 rad/s e' il criterio tipico del corso.

**Possibile domanda d'esame** – Perche' i guadagni canonici non bastano e devi raffinarli? *Risposta:* Perche' sono derivati sulla dinamica di beccheggio **disaccoppiata**, mentre l'anello reale include la deriva laterale (i termini  $a_1, a_4, A_6/V$  della matrice  $A$ ) e i guadagni  $K_{p_z}, K_{d_z}$ . Il feedback di deriva **erode il margine di guadagno aerodinamico**: come dice il commento di `design_controller.m` (righe 8-10), i 6 dB canonici della coppia disaccoppiata scendono a ~4 dB sull'anello completo. Il `fminsearch` li ricompensa alzando  $K_p$  da 1.48 a 1.78 e  $K_d$  da 0.40 a 0.44, e riporta la coppia classificata a 6 dB / 30 deg.

## Setup (righe 15-35)

```
methods (TestMethodSetup)
    function muteConditionallyStableMarginWarning(testCase)
        ws = warning('off', 'Control:analysis:MarginUnstable');
        testCase.addTeardown(@() warning(ws));
    end
end
```

- Righe 15-28 (il resto del setup, non riportato sopra): la proprieta' `p` (righe 15-17) e il blocco `TestClassSetup` (righe 19-28), eseguito **una volta per classe**. `addHm3ToPath` aggiunge HM3/ al path con una `PathFixture` (che il framework rimuove da sola a fine suite, anche in caso di errore), e `loadNominalParams` riempie `testCase.p` con `load_hw3_params()`. E' da li' che arriva il `pp = testCase.p` usato da tutti i test.



- Righe 30-35: **questo blocchetto e' un indizio fisico**, non un dettaglio igienico. `Control:analysis:MarginUnstable` e' l'avviso che la Control System Toolbox emette quando le si chiede un margine su un anello aperto **instabile**. Il nostro anello aperto e' instabile per costruzione (due poli a parte reale positiva, vedi `hm3PlantTest`), quindi la warning arriverebbe a ogni singola chiamata di `margin/allmargin`, riempiendo l'output dei test. Va mutata, ma con `addTeardown`, che **ripristina lo stato precedente anche se il test fallisce**.
- La warning e' il sintomo della **stabilita' condizionata**: il sistema e' stabile in anello chiuso solo se il guadagno sta **dentro una banda**, non semplicemente "sotto un massimo". Abbassando troppo il guadagno l'anello torna instabile, perche' smette di controbilanciare il polo aerodinamico. E' il motivo per cui esiste un **gain margin di riduzione** (`aeroGM`, negativo in dB) oltre al classico gain margin di aumento.

## testGustProfileShape (righe 38-47)

```
w = load_wind_profile(pp, 'Vg', 8.0, 'Tg', 3.0, 't0', 1.0);
testCase.verifyEqual(max(abs(w.vw(w.t < 1.0))), 0, 'AbsTol', 1e-15);
[vwPeak, iPeak] = max(w.vw);
testCase.verifyEqual(vwPeak, 8.0, 'AbsTol', 1e-6);
testCase.verifyEqual(w.t(iPeak), 1.0 + 1.5, 'AbsTol', w.t(2)-w.t(1));
testCase.verifyEqual(w.alpha_w, w.vw/pp.V, 'AbsTol', 1e-15);
```

- Righe 42-46: quattro invarianti della raffica **1-coseno** (`load_wind_profile.m` riga 56):

```
v_w(t) = 0.5*V_g * (1 - cos(2*pi*(t - t0)/T_g)),    t0 <= t <= t0 + T_g
v_w(t) = 0                                           altrove
```

- **Causalita'** (riga 42):  $v_w = 0$  prima dell'istante di innesco  $t_0$ . Se il vento fosse non nullo a  $t = 0$ , il sistema partirebbe da una condizione forzata e la risposta al transitorio si mescolerebbe con quella alla raffica.
- **Ampiezza di picco** (riga 44): il massimo vale esattamente  $V_g$ . Deriva dal fatto che  $1 - \cos(x)$  ha massimo 2, quindi  $0.5*V_g*2 = V_g$ . Il fattore 0.5 e' quello che rende  $V_g$  interpretabile come *velocita' di picco* e non come ampiezza da raddoppiare – un classico errore di fattore 2.
- **Istante di picco** (riga 45): a meta' raffica,  $t_0 + T_g/2 = 2.5$  s. La tolleranza e'  $w.t(2)-w.t(1)$ , cioe' **un passo di campionamento** ( $dt = 5$  ms): e' la scelta corretta, perche' il picco viene individuato con `max` su una griglia discreta e la sua posizione e' nota solo a meno di un campione.
- **Definizione di  $\alpha_w$**  (riga 46):  $\alpha_w = v_w / V$ . E' la linearizzazione a piccoli angoli dell'angolo di incidenza indotto da una componente di vento **laterale**  $v_w$  su un veicolo che avanza a  $V$ :  $\alpha_w = \text{atan}(v_w/V) \sim v_w/V$  per  $v_w \ll V$ . Con  $V_g = 6.38$  m/s e  $V = 937.7$  m/s,  $\alpha_w = 6.8e-3$  rad = 0.39 deg: siamo profondamente in regime lineare, l'approssimazione e' irreperibile.

## testStepProfileShape (righe 49-53) e testWindProfileRejectsUnknownProfile (righe 61-65)

- Righe 51-52: il profilo '`step`' e' zero prima di  $t_0$  e vale  $V_g$  a fine orizzonte. E' il disturbo "peggiore" in senso di persistenza (non rientra mai), utile come controllo diagnostico.
- Righe 62-64: un profilo sconosciuto ('`sinusoid`') solleva `load_wind_profile:profile` (riga 64 del sorgente). Come in `build_plant_full`, si testa l'**identificatore**, non il messaggio.

## testDefaultGustAmplitudeFromDrywind (righe 55-59)

```
% Severe dry-wind dispersion at 15.1 km is the documented default
w = load_wind_profile(testCase.p);
testCase.verifyEqual(w.Vg, 6.38, 'AbsTol', 0.05);
```

- Riga 58: **golden value numero uno**. 6.38 m/s non e' una scelta di progetto: e' il risultato dell'interpolazione della dispersione di vento **severe** del file General/hw3-v3/drywind.mat alla quota Alt/1000 = 15.143 km (load\_wind\_profile.m righe 39-48). Verificato: Vg = 6.3785, e il picco di alpha\_w che ne deriva vale 0.3897 deg (il "0.39 deg" del README).
- **Questo test e' anche, implicitamente, un test di presenza del data file**. Se drywind.mat mancasse, load\_wind\_profile cadrebbe sul fallback Vg = 8.0 (riga 47) **senza errori**, e il test fallirebbe (8.0 contro 6.38 +/- 0.05). E' un effetto collaterale desiderabile: il fallback silenzioso e' pericoloso, perche' produrrebbe un'analisi di raffica del 25 % piu' severa senza avvisare nessuno. Il test lo intercetta. Vale la pena saperlo dire all'orale.
- La AbsTol di 0.05 e' larga rispetto al valore (0.8 %): assorbe eventuali differenze nella griglia di interpolazione ma non un cambio di severita' (light/moderate darebbero valori ben diversi).

---

## testRigidLoopMeetsMarginTargets (righe 67-76)

```
G = build_plant_rigid(testCase.p);
[L, T] = assemble_loop(G, testCase.Kref);
mm = classify_margins(minreal(L, 1e-6), ...
    'w_drift', 0.3*sqrt(testCase.p.A6));
testCase.verifyTrue(isstable(T));
testCase.verifyEqual(abs(mm.aeroGM_dB), 6.0, 'AbsTol', 0.3);
testCase.verifyEqual(mm.rigidPM_deg, 30.0, 'AbsTol', 0.7);
```

E' il test centrale del Task 1, e il piu' denso di contenuto teorico.

**Riga 72 – perche' w\_drift = 0.3\*sqrt(A\_6)**. E' il confine fra la banda “deriva laterale” e la banda “corpo rigido” passato a classify\_margins. Non e' una costante arbitraria: e' **scalato sulla frequenza dell'instabilita'**. La logica e': il crossover di controllo deve stare all'ordine di sqrt(A\_6) (altrimenti non batte il polo instabile), mentre i fenomeni di deriva laterale sono molto piu' lenti (l'integratore di posizione z). Prendere il 30 % di sqrt(A\_6) mette la soglia a 0.3\*1.839 = 0.552 rad/s, comodamente sopra le crossings di deriva e sotto il crossover rigido. Ed e' esattamente lo stesso valore che design\_controller.m (riga 46) calcola internamente: i due devono coincidere, altrimenti il test misurerebbe una cosa e il tuner ne ottimizzerebbe un'altra.

**Perche' classify\_margins e non margin()**. Questo e' il punto piu' importante da saper difendere. L'anello aperto ha **due poli instabili** e un integratore, quindi la curva di Nichols passa **fra** i due punti critici invece che sotto (stabilita' condizionata) e ci sono **piu' attraversamenti** dello 0 dB e dei -180 deg. Verificato con allmargin sull'anello progettato:

```
attraversamenti di fase (0 dB crossings):
-133.12 deg @ 0.161 rad/s    <- artefatto di deriva
-40.53 deg @ 0.222 rad/s    <- artefatto di deriva
+30.00 deg @ 2.455 rad/s    <- IL margine di fase rigido

attraversamento di guadagno (-180 deg crossing):
-6.00 dB @ 0.593 rad/s      <- margine di guadagno AERODINAMICO
```

Le due crossings a 0.161 e 0.222 rad/s hanno margini di fase **negativi**, ma il sistema e' **stabile** (isstable(T) = 1, allmargin.Stable = 1): sono artefatti del lobo di deriva, non indicatori di instabilita'. classify\_margins le esclude con la maschera pf > w\_drift (riga 49 del sorgente) e poi seleziona con la strategia 'maxv'. mm.drift\_w le conserva comunque, etichettate come tali.

Il margine di guadagno e' **negativo** (-6 dB) ed e' una **riduzione**: dice che l'anello si destabilizza se il guadagno **scende** di 6 dB. E' la firma della stabilita' condizionata, e fisicamente e' il *margine aerodinamico*: se il controllo perde meta' della sua autorita', non riesce piu' a contrastare il polo a +1.84 rad/s. `classify_margins` lo isola con la maschera `gmdb < 0` (riga 44).

Nota di onesta': su **questo specifico** progetto, `margin()` grezza restituisce per caso gli stessi numeri ( $G_m = 0.5012 = -6.00$  dB @ 0.593,  $P_m = 30.00$  deg @ 2.455) – verificato. Il commento di `classify_margins.m` (righe 13-15) sostiene che `margin()` “prenderebbe una delle crossings di deriva”, e su questo punto e' troppo assertivo. Ma la robustezza del ragionamento non cambia: la scelta di `margin()` **dipende dal progetto**. Con un'altra coppia di guadagni – per esempio  $K_p = 1.98$ ,  $K_d = 1.40$ , che sono i guadagni **pre-ritaratura** rimasti a lungo pinnati per sbaglio nel perf-test (vedi la pagina su `hm3LoopPerformanceTest`) – `margin()` restituisce  $P_m = -30.00$  deg su un anello che e' **stabile**: verificato. Cioe' `margin()` e' inaffidabile su questo tipo di anello e a volte da' la risposta giusta per fortuna. `classify_margins` esiste per **non dipendere dalla fortuna**, e questo va detto cosi'.

**Righe 74-75 – i target 6 dB / 30 deg.** Sono i valori richiesti dalla traccia, non un'ottimizzazione. Le tolleranze (0.3 dB, 0.7 deg) sono **strette**: sono tolleranze di **regressione**, non di ingegneria. Servono a far fallire il test se cambia una qualunque cosa a monte – il seed del tuner, `TolX/TolFun` di `fminsearch`, la logica di banda del classificatore, i coefficienti letti dall'LPV. Non dicono “il progetto e' buono entro 0.3 dB”: dicono “il progetto e' **questo**”.

**Riga 73 – `isstable(T)`.** E' il verdetto vero, ed e' logicamente **prioritario** rispetto ai margini: su un anello condizionatamente stabile, l'unica affermazione non ambigua e' “tutti i poli dell'anello chiuso hanno parte reale negativa”. I margini sono **metriche di robustezza** rispetto a quel verdetto, non sostituti.

---

## testDesignControllerMeetsTargets (righe 78-86)

```
[K, m] = design_controller(G, [], 'verbose', false);
testCase.verifyEqual(abs(m.aeroGM_dB), 6.0, 'AbsTol', 0.3);
testCase.verifyEqual(m.rigidPM_deg, 30.0, 'AbsTol', 0.7);
testCase.verifyTrue(m.stable);
testCase.verifyEqual(K.Kp_th, testCase.Kref.Kp_th, 'AbsTol', 1e-2);
testCase.verifyEqual(K.Kd_th, testCase.Kref.Kd_th, 'AbsTol', 1e-2);
```

- E' il test **piu' forte** della suite, e il piu' fragile. Le righe 81-83 verificano che il tuner **raggiunga i target**; le righe 84-85 verificano che ci arrivi **passando esattamente per Kref**.
- La differenza e' sostanziale. Le prime tre asserzioni testano una **proprieta'** (l'ottimizzatore converge dove deve). Le ultime due pinnano il **percorso**: dato che il costo (righe 78-90 di `design_controller.m`) e'

$$c = (|aeroGM| - 6)^2 + (rigidPM - 30)^2 \quad [+ 1e4 \text{ se instabile}]$$

e' un problema a 2 incognite e 2 obiettivi: la soluzione e' (localmente) unica e `Kref` e' quella soluzione. Pinnarla a  $1e-2$  significa pinnare **il seed D'Antuono, la tolleranza di `fminsearch`, e la definizione delle bande** tutti insieme. Se domani si cambiasse il seed (per esempio partendo da `[1 1]`), `fminsearch` potrebbe convergere a un minimo locale diverso che soddisfa comunque i target: le prime tre asserzioni passerebbero, le ultime due no. Il test e' progettato per accorgersene, ed e' la scelta giusta in un contesto di regressione.

- Nota sulla parametrizzazione: il tuner ottimizza in `log` (riga 55:  $x_0 = \log([2*A_6/K_1, \sqrt{A_6}/K_1])$ ). E' un dettaglio importante – garantisce che i guadagni restino **positivi** durante la ricerca, senza vincoli espliciti, e rende la ricerca naturalmente **moltiplicativa** (un passo di `fminsearch` e' una variazione percentuale, non assoluta), che e' la scala giusta per dei guadagni. `fminsearch` e' Nelder-Mead, senza vincoli: senza il `log` potrebbe proporre guadagni negativi e produrre anelli assurdi.

## testDesignControllerRestoresWarningState (righe 88-96)

```
warning('on', 'Control:analysis:MarginUnstable');
G = build_plant_rigid(testCase.p);
design_controller(G, [], 'verbose', false);
st = warning('query', 'Control:analysis:MarginUnstable');
testCase.verifyEqual(st.state, 'on');
```

- E' un **test di regressione su un bug di igiene**, non sulla fisica. Il commento (righe 89-90) e' esplicito: `design_controller` muta la warning al proprio interno (righe 51-52 del sorgente) e **deve ripristinare lo stato del chiamante** all'uscita.
- Perche' e' importante e non pedanteria: `warning('off', ...)` e' uno **stato globale della sessione MATLAB**. Se `design_controller` lo lasciasse spento, ogni successiva chiamata a `margin()` in **tutta la sessione dell'utente** smetterebbe di avvisare che l'anello e' instabile. Su un progetto dove la stabilita' condizionata e' il fenomeno centrale, sopprimere silenziosamente quell'avviso e' esattamente il tipo di errore che porta a conclusioni sbagliate.
- Il sorgente lo fa correttamente con `onCleanup` (riga 52), che garantisce il ripristino anche se `fminsearch` solleva un'eccezione. Questo test verifica che il pattern funzioni davvero.
- Sottigliezza: il test **si affida** al fatto che `TestMethodSetup` (righe 30-35) spenga la warning e la riaccenda in `teardown`. La riga 91 la riaccende esplicitamente *dentro* il test, perche' altrimenti il setup l'avrebbe appena spenta e il test verificherebbe `'off' == 'on'`.

---

## testBareFullModelIsBendingUnstable (righe 98-104)

```
% Task 2, Step B: TVC + delay with no bending filter -> unstable
Gf = build_plant_full(pp, 'ins');
[~, T] = assemble_loop(Gf, testCase.Kref, build_tvc(pp));
testCase.verifyFalse(isstable(T));
```

- Riga 103: un **verifyFalse** che vale come mezzo homework. E' la *dimostrazione eseguibile* che il Task 2 e' un problema vero: si prende il progetto del Task 1 (stesso Kref), lo si mette sul modello a 6 stati con INS contaminato e con il TVC reale, **senza filtro**, e l'anello chiuso e' **instabile**. Verificato: `isstable(T) = 0`.
- La causa e' il picco di risonanza di **+29 dB** a  $\omega_{BM} = 18.9$  rad/s (README). Il modo di bending, di per se', e' stabile ( $\zeta_{BM} = 0.005 > 0$ ): e' la **chiusura dell'anello** attraverso il percorso `delta -> eta` (forzamento TVC) e `eta -> theta_m` (contaminazione INS) a renderlo instabile, come spiegato nella pagina su `hm3PlantTest`.
- Un **test negativo di questo tipo e' prezioso** e spesso trascurato: senza di esso, si potrebbe aggiungere il notch "per sicurezza" e non avere alcuna prova che serva. Qui la prova c'e', ed e' automatica.

---

## testDeepNotchStabilisesFullModel (righe 106-114)

```
Gf = build_plant_full(pp, 'ins');
Hn = build_notch_filter(pp.wBM, 0.002, 0.7, +1);
[L, T] = assemble_loop(Gf, testCase.Kref, build_tvc(pp)*Hn);
testCase.verifyTrue(isstable(T));
testCase.verifyLessThan(20*log10(abs(freqresp(L, pp.wBM))), -10);
```

- Riga 110: il **notch ritenuto** del Task 2, con la parametrizzazione esatta del progetto: centrato su  $\omega_{BM}$ ,  $\zeta_N = 0.002$ ,  $\zeta_D = 0.7$ , a **fase minima** (+1). Profondita' -50.9 dB (vedi la pagina su `hm3FilterTest`).

- Riga 112: l'anello ora **e' stabile**. Verificato.
- Riga 113: **questa e' l'asserzione con contenuto ingegneristico**. Non basta che l'anello sia stabile: si verifica che lo sia per **gain stabilisation**, cioe' che il guadagno d'anello alla frequenza di bending sia sceso **sotto -10 dB**. Verificato:  $|L(j\omega_{BM})| = -21.87$  dB con questi guadagni.
  - La soglia -10 dB e' un requisito ingegneristico, non un numero preciso: la letteratura chiede tipicamente  **$\geq 12$  dB** di attenuazione per un modo gain-stabilised (il README lo dice). Il test usa 10 dB come soglia conservativa, cioe' verifica una condizione **piu' debole** di quella dichiarata nel report. Va bene per un test di regressione (non deve essere fragile), ma bisogna sapere che il requisito vero e' piu' stringente.
  - Perche' e' l'invariante giusto: **isstable** da solo non distingue una stabilizzazione **in guadagno** (robusta: il lobo e' cosi' in basso che la sua fase e' irrilevante) da una **in fase** (fragile: dipende dal passare dalla parte giusta del punto critico). Aggiungere la soglia sul modulo pinna **quale strategia** e' stata usata, non solo che ha funzionato.

**Nota di onesta' importante – questo test NON pinna il progetto finale.** Usa  $K_{ref}$ , cioe' i guadagni del Task 1 (1.7845 / 0.4433), non i guadagni **ri-tarati** del Task 2 che il README dichiara come progetto consegnato ( $K_p = 1.73$ ,  $K_d = 0.69$ ). La configurazione testata e' quella intermedia della tabella del report ("deep notch (retained)" con i guadagni del Task 1: **aero GM 6.08 dB**, **rigid PM 14.6 deg**,  $|L(\omega_{BM})| = -21.9$  dB), cioe' proprio la configurazione con il **marginale di fase collassato** che ha costretto al re-tuning. **Non esiste alcun test che pinni i guadagni finali del Task 2**, ne' i suoi margini (30 deg @ 3.2 rad/s, DM 165 ms,  $|L(\omega_{BM})| = -18$  dB). E' la lacuna di copertura piu' significativa della suite.

## testGustResponseAngleOfAttackBudget (righe 116-128)

```
% alpha = theta + zdot/V - alpha_w (the plant's own convention:
% Eq. (1) has Bw = [0; -a1*V; 0; -A6]), and peaks match the histories
r = simulate_gust_response(T, w);
testCase.verifyEqual(r.alpha, r.theta + r.zdot/pp.V - r.alphaw, ...
  'AbsTol', 1e-15);
testCase.verifyEqual(r.peak_theta, max(abs(r.theta)), 'AbsTol', 1e-15);
testCase.verifyEqual(r.theta(1), 0, 'AbsTol', 1e-15);
```

Le tre asserzioni sono, nell'ordine, debolissime. Vanno esaminate con onesta'.

- Righe 124-125: verifica che  $r.alpha == r.theta + r.zdot/V - r.alphaw$ . Ma quella **e' letteralmente la riga 29 di simulate\_gust\_response.m**. Il test **ricopia la formula** e verifica che la formula sia se stessa: e' una tautologia. Il suo unico valore e' da *change-detector* (se qualcuno riscrive la riga 29, il test fallisce e lo costringe a giustificarsi).
- Riga 126:  $peak\_theta == \max(abs(theta))$  – di nuovo, e' la riga 31 del sorgente riscritta. Tautologia.
- Riga 127:  $theta(1) == 0$ . Sembra un test di condizione iniziale nulla, ma passa **automaticamente**: `lsim` parte da stato nullo per default,  $D = 0$  (nessun feedthrough) e in ogni caso  $alphaw(1) = 0$  perche' la raffica comincia a  $t_0 = 1$  s. Non puo' fallire se non con un errore grossolano.

## Perche' il segno e' il MENO (il punto da difendere all'orale)

Il segno di  $alpha_w$  non e' una convenzione libera: **lo detta il plant**. Guardiamo la colonna di disturbo di `build_plant_rigid.m` (riga 17), che ricopia l'Eq. (1) della traccia:

$$Bw = [0; -a_1*V; 0; -A_6]$$

La riga di beccheggio del plant e' quindi

$$\begin{aligned} \theta_{ddot} &= A_6*\theta + (A_6/V)*z_{dot} + K_1*\delta - A_6*alpha_w \\ &= A_6 * ( \theta + z_{dot}/V - alpha_w ) + K_1*\delta \end{aligned}$$

e la riga laterale, sviluppata, da'

$$z\_ddot = a\_1*V*(\theta + z\_dot/V - \alpha_w) + a\_4*\theta + a\_3*\delta$$

**Entrambe le righe forzano sulla stessa incidenza aerodinamica**

$$\alpha = \theta + z\_dot/V - \alpha_w \quad (\text{segno MENO})$$

Il significato fisico e' preciso:  $\alpha$  e' l'incidenza rispetto alla velocita' **relativa all'aria**, non rispetto al suolo.  $\theta + z\_dot/V$  e' l'orientamento del veicolo rispetto alla traiettoria inerziale;  $\alpha_w = v_w/V$  e' la rotazione del vettore velocita' relativa prodotta dal vento laterale. Un vento che spinge nella stessa direzione in cui il muso e' gia' inclinato **riduce** l'incidenza vista dal veicolo: da qui la sottrazione. Un + descriverebbe un plant diverso da quello che si sta integrando – sarebbe in contraddizione con l'Eq. (1) stessa.

Il codice usa oggi il meno **in tutti i siti**: `simulate_gust_response.m` riga 29, questo test alla riga 124, e il post-processing LPV di `LTV_FULL_ASCENT/main_full_ascent.m` riga 153. Il plant (sia `build_plant_rigid`, sia `ode_lpv_ascent`) lo ha sempre avuto.

**Nota onesta, da raccontare com'e' andata.** Fino a poco fa il **post-processing** aveva un `r.alpha = theta + zdot/V + alpha_w`. Era un bug di post-processing, non di modello – il plant integrato era ed e' sempre stato quello giusto, quindi  $\theta$ ,  $z$  e  $\delta$  non ne erano toccati; a sbagliare era **solo** il canale diagnostico  $\alpha$  (e quindi  $\bar{q}\alpha$ ). Ora e' corretto ovunque. L'effetto sui numeri e' grosso:

$$\begin{array}{ll} \text{picco } |\alpha| \text{ col vecchio } \text{'+'} = 0.255 \text{ deg} & \rightarrow \bar{q}\alpha = 20.7 \text{ kPa*deg} \\ \text{picco } |\alpha| \text{ col corretto } \text{'-' } = 0.577 \text{ deg} & \rightarrow \bar{q}\alpha = 46.8 \text{ kPa*deg} \end{array}$$

**Un fattore 2.3 sull'indicatore di carico**, ed e' il numero con cui si argomentava che il progetto fosse load-relieving.

## La conseguenza fisica: l'attitude hold e' *load-aggravating*

Questo e' il punto d'orale vero, e vale la pena saperlo enunciare da solo.

Il picco di incidenza vale **0.577 deg**, mentre la raffica **da sola** produrrebbe solo  $\alpha_w$  di picco = **0.390 deg**. Cioe' l'anello chiuso **peggiora** il bilancio di incidenza invece di alleggerirlo: il contributo dell'assetto non elide il vento, **ci si somma**.

La ragione e' immediata una volta scritto il segno giusto. Il vento fa ruotare il vettore velocita' relativa; per **tenere l'assetto** ( $\theta_{ref} = 0$ ) il loop deve opporsi al momento aerodinamico, e cosi' facendo becca il muso **dentro** il vento relativo. Il termine  $\theta + z\_dot/V$  cresce nello stesso verso in cui  $-\alpha_w$  gia' spinge, e i due si sommano in modulo. **Una legge di puro attitude-hold e' load-aggravating, non load-relieving** – il contrario di quello che il segno sbagliato lasciava credere.

Non c'e' effetto banderuola che salvi la situazione: con  $A_6 > 0$  il centro di pressione sta **davanti** al baricentro, quindi il momento aerodinamico e' **divergente**. Il veicolo non si allinea da solo al vento – ci diverge contro, e il controllo deve spendere autorita' proprio per impedirglielo.

**Possibile domanda d'esame** – Perche' nel bilancio di incidenza  $\alpha_w$  entra con il segno meno, e cosa ti dice il numero che ne esce? *Risposta*: Perche' l'Eq. (1) ha la colonna di vento  $B_w = [0; -a_1*V; 0; -A_6]$ , quindi entrambi i termini aerodinamici del plant forzano su  $\alpha = \theta + z\_dot/V - \alpha_w$ :  $\alpha$  e' l'incidenza rispetto alla velocita' **relativa all'aria**, e il vento ruota quel vettore. Scrivere un + descriverebbe un plant diverso da quello che sto integrando. Il numero che ne esce e' istruttivo: il picco di  $|\alpha|$  e' **0.577 deg** contro i **0.390 deg** del vento da solo. L'assetto **non** compensa il vento, **si somma**: per tenere  $\theta \sim 0$  il loop becca il muso dentro il vento relativo. Quindi il mio Task 1 e' un attitude-hold **load-aggravating**, e i guadagni di deriva a  $-1e-3$  sono troppo piccoli per invertire il bilancio. Con  $A_6 > 0$  il centro di pressione e' davanti al baricentro: nessuna banderuola, il momento e' divergente. Un vero load relief richiederebbe guadagni di deriva molto piu' aggressivi (o un accelerometro), pagati in deriva laterale.

**Nota metodologica: il test, per come e' scritto, non poteva accorgersene.** Congelando la formula invece di confrontarla con la dinamica, la tautologia ha pinnato il segno sbagliato per tutto il tempo in cui c'e' stato. Un test non tautologico avrebbe confrontato `r.alpha` con il segnale che **effettivamente guida** la dinamica – per esempio verificando che  $\theta\_ddot - K_1*\delta = A_6*\alpha$  sulla risposta simulata. Questa e' la lezione

generale da portare all'orale: **un change-detector non e' un test di correttezza**, e va detto quando se ne scrive uno.

## I golden value di questo file, in chiaro

| Valore              | Riga   | Da dove viene                                                                                        | Cosa succede se il progetto cambia                            |
|---------------------|--------|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Kp_th = 1.7845      | 11     | output di <code>fminsearch</code> in <code>design_controller</code> sul plant rigido, seed D'Antuono | falliscono 5 test (tutti quelli che usano <code>Kref</code> ) |
| Kd_th = 0.4433      | 11     | idem                                                                                                 | idem                                                          |
| Kp_z = Kd_z = -1e-3 | 12     | <b>fissati dalla traccia</b> (load relief), non ottimizzati                                          | non cambiano                                                  |
| 6.0 dB              | 74, 81 | <b>target della traccia</b> (PM rigido)                                                              | GM                                                            |
| 30.0 deg            | 75, 82 | <b>target della traccia</b> (PM rigido)                                                              | idem                                                          |
| 6.38 m/s            | 58     | interpolazione di <code>drywind.mat</code> (severe) a 15.143 km                                      | cambia se cambia quota o severita'                            |
| -10 dB              | 113    | soglia conservativa di gain stabilisation (il requisito vero e' $\geq 12$ dB)                        | soglia scelta a mano                                          |
| 0.002 / 0.7         | 110    | parametri del notch ritenuto                                                                         | scelti dal trade a 4 filtri del Task 2                        |

**Come leggere i golden value.** I 6 dB e i 30 deg sono **requisiti**: se il progetto cambia, restano. `Kref`, 6.38, 0.002/0.7 sono **fotografie di un progetto specifico**: se il progetto cambia (altro seed, altra tolleranza del tuner, altro data set, altro notch), **devono** cambiare, e la giusta reazione a un fallimento non e' allargare la tolleranza ma **capire perche' il progetto si e' spostato** e, se lo spostamento e' voluto, aggiornare il numero pinnato **insieme al report**. Un golden value non e' una verita': e' un contratto con la versione del progetto documentata nel report.

## Possibili domande d'esame

**D: Perche' non usi semplicemente `margin()`? Cosa aggiunge `classify_margins`?** R: Perche' l'anello aperto ha due poli instabili e un integratore, quindi e' **condizionatamente stabile**: la curva di Nichols passa fra i due punti critici e ci sono piu' attraversamenti dello 0 dB. Su questo anello `allmargin` restituisce tre margini di fase: -133 deg a 0.161 rad/s, -40 deg a 0.222 rad/s e +30 deg a 2.455 rad/s – eppure l'anello chiuso e' stabile. I primi due sono artefatti del lobo di deriva laterale. `margin()` restituisce **un** numero e quale numero scelga dipende dal progetto: sui guadagni del Task 1 azzecca (-6 dB / 30 deg), su altri guadagni che ho provato restituisce  $P_m = -30$  deg su un anello stabile. `classify_margins` separa i margini **per banda fisica** – GM aerodinamico alla crossing di bassa frequenza (dove il gain margin e' di **riduzione**, cioe' negativo in dB), PM rigido al crossover di controllo, GM/PM flessionale attorno a  $\omega_{BM}$  – e non dipende dalla fortuna.

**D: Perche' il margine di guadagno e' negativo (-6 dB)? Non dovrebbe dire di quanto posso alzare il guadagno?** R: Su un sistema condizionatamente stabile ci sono **due** limiti al guadagno: uno superiore e uno inferiore. Il -6 dB dice che l'anello si destabilizza se il guadagno **scende** di 6 dB, cioe' se il controllo perde



meta' della sua autorita'. Fisicamente e' il **margin** aerodinamico: sotto quella soglia il controllo non riesce piu' a contrastare il polo instabile a +1.84 rad/s e il lanciatore diverge. E' l'unico margine di guadagno che esiste sul modello rigido del Task 1 (con attuatore ideale non c'e' crossing di aumento: `rigidGM_dB = NaN`, verificato). Nel Task 2, con TVC e ritardo, ricompare anche il margine di aumento (7.56 dB secondo il README), perche' l'attuatore aggiunge fase negativa in alta frequenza.

**D: Che cosa sono i tuoi golden value e cosa fai se un test fallisce?** R: Sono numeri pinnati che fotografano un progetto specifico:  $K_{ref} = (1.7845, 0.4433)$  e' l'output di `fminsearch` a partire dal seed D'Antuono sul plant rigido; 6.38 m/s e' l'ampiezza di raffica interpolata dal file `drywind.mat` alla mia quota. Non hanno derivazione analitica. I 6 dB e i 30 deg invece **non** sono golden value: sono i **requisiti** della traccia. Se un test su  $K_{ref}$  fallisce, la reazione giusta **non** e' allargare la tolleranza: e' capire cosa e' cambiato a monte (il seed, la tolleranza del tuner, il data set LPV, le bande del classificatore) e, se il cambiamento e' voluto, aggiornare il numero pinnato **insieme al report**, perche' i due devono raccontare la stessa storia.

**D: Come dimostri che il notch serve, e non l'hai messo per sicurezza?** R: Con un test negativo. `testBareFullModelIsBendingUnstable` prende i guadagni del Task 1, li mette sul modello a 6 stati con INS contaminato e TVC reale **senza filtro**, e verifica che l'anello chiuso sia **instabile** (`verifyFalse(isstable(T))`). Poi `testDeepNotchStabilisesFullModel` aggiunge il notch a fase minima e verifica due cose: che l'anello torni stabile, e che il guadagno d'anello a  $\omega_{BM}$  sia sceso sotto -10 dB (misurato: -21.9 dB). La seconda asserzione e' quella che qualifica la strategia: non e' stabile "per fase", e' stabile "per guadagno", cioe' il lobo di bending e' cosi' basso che la sua fase e' irrilevante.

**D: Perche' il tuner ottimizza il logaritmo dei guadagni?** R: Per due ragioni. Primo, garantisce che i guadagni restino **positivi** durante tutta la ricerca senza dover imporre vincoli: `fminsearch` e' Nelder-Mead, un metodo **non vincolato**, e senza il `log` potrebbe proporre  $K_p$  o  $K_d$  negativi, producendo anelli con feedback positivo. Secondo, rende la ricerca **moltiplicativa**: un passo del simplex e' una variazione percentuale, non assoluta, che e' la scala naturale per dei guadagni (passare da 0.4 a 0.5 e da 4 a 5 sono la stessa cosa in `log`). Il seed e' `log([2*A6/K1, sqrt(A6)/K1])`, cioe' la coppia canonica di D'Antuono.

**D: I guadagni di deriva sono negativi. Non e' un errore di segno?** R: No, e' *load relief*, ed e' voluto (la traccia li fissa a  $-1e-3$ ). Un guadagno **positivo** su  $z$  cercherebbe di riportare il lanciatore sulla traiettoria nominale contro il vento, e per farlo dovrebbe **aumentare l'incidenza** aerodinamica, cioe' aumentare il carico  $q_{bar\alpha}$  – *proprio nel punto di volo in cui il carico e' massimo*. Un guadagno **negativo** lascia che il veicolo *derivi* con\* il vento, riducendo l'incidenza al prezzo di qualche metro di deriva laterale. A max- $q_{bar}$  il carico strutturale e' il vincolo dominante e la deriva no, quindi si accetta il compromesso. **Ma il segno e' l'intenzione, non il risultato**: a  $1e-3$  i guadagni sono cosi' piccoli che il bilancio di incidenza resta dominato dal PD d'assetto, e misurando si trova un picco di  $|\alpha|$  di 0.577 deg contro i 0.390 deg del vento da solo – cioe' l'anello **aggrava** il carico. E' una correzione debole, non un anello di guida, e non va spacciata per un load relief che funziona.

**D: Il picco di incidenza e' piu' grande del vento da solo. Non e' un errore?** R: No, ed e' il punto fisico piu' interessante dell'homework. Il bilancio corretto e'  $\alpha = \theta + \dot{z}/V - \alpha_w$  (lo impone l'Eq. (1) con  $B_w = [0; -a_1*V; 0; -A_6]$ ):  $\alpha$  e' l'incidenza rispetto alla velocita' **relativa all'aria**. Per tenere l'assetto ( $\theta_{ref} = 0$ ) il loop deve opporsi al momento aerodinamico, e cosi' facendo **becca il muso dentro il vento relativo**: il termine d'assetto si **somma** a quello di vento invece di eliderlo. Risultato misurato: picco  $|\alpha| = 0.577$  deg contro  $\alpha_w$  di picco = 0.390 deg, e  $q_{bar}*\alpha = 46.8$  kPadeg. **Un puro attitude-hold e' load-aggravating, non load-relieving**. E non c'e' effetto banderuola a salvarlo: con  $A_6 > 0$  il centro di pressione sta davanti al baricentro, quindi il momento aerodinamico e' **divergente** – il veicolo non si allinea al vento, ci diverge contro. (Onesta' storica: fino a poco fa il post-processing aveva un + invece del - e dava 0.255 deg / 20.7 kPadeg, facendo sembrare il progetto load-relieving. Era un bug di post-processing – il plant integrato aveva sempre il segno giusto, quindi  $\theta$ ,  $z$  e  $\delta$  non erano toccati – ed e' stato corretto.)



# HM3/tests/hm3LoopPerformanceTest.m

## Ruolo del file nel progetto

E' l'unica suite di HM3 che **non verifica la correttezza ma il costo**. Estende `matlab.perftest.TestCase` (non `matlab.unittest.TestCase`) e misura il tempo di esecuzione dei blocchi che stanno **dentro i cicli caldi** dell'homework. Si lancia con `runperf('hm3LoopPerformanceTest')`: e' l'unico modo per ottenere i **tempi**. Attenzione pero' – `matlab.perftest.TestCase` e' una sottoclasse di `matlab.unittest.TestCase`, quindi anche `runtests('HM3/tests')` **scopre ed esegue** questa classe (eseguendo una volta i corpi dei cicli e valutando le `verify*`); semplicemente non produce misure.

**Perche' un perf-test in un homework di controlli.** Perche' HM3, a differenza di HM1 e HM2, non risolve *un* problema: ne risolve **migliaia**. Il progetto non e' un singolo anello, e' una **ricerca** su uno spazio di parametri, e la ricerca e' costruita interamente attorno a una funzione, `assemble_loop.m`, che viene richiamata da tre punti diversi:

1. **L'auto-tuner PD** (`design_controller.m`, righe 56-57): `fminsearch` valuta la funzione di costo fino a `MaxFunEvals = 400` volte, e **ogni valutazione** chiama `assemble_loop + minreal + classify_margins + isstable` (righe 82-89).
2. **Il trade dei filtri del Task 2**: lo sweep sulle parametrizzazioni del lead-lag (il README parla di “best of 75”) e sulle varianti di notch richiede un riassettaggio dell'anello per ogni candidato.
3. **Il Monte Carlo** (`main_montecarlo.m`, riga 27: `N = 1500`; riga 87: `[L, T] = assemble_loop(Gf, K, Wf)` **dentro il parfor**): 1500 riassettaggi dell'anello completo, uno per estrazione.

`assemble_loop` **non e' una funzione economica**. Fa `connect()` (interconnessione per nome, che costruisce una realizzazione in spazio di stato allargata e poi elimina i segnali interni), `getLoopTransfer`, una conversione `ss -> tf` e un `minreal`. Misurato su questa macchina:

| Operazione                                                               | Costo misurato    |
|--------------------------------------------------------------------------|-------------------|
| <code>assemble_loop</code> (rigido, 4 stati)                             | ~10.5 ms/chiamata |
| <code>assemble_loop</code> (completo + TVC + notch, 13 stati)            | ~9.1 ms/chiamata  |
| <code>classify_margins</code> (via <code>allmargin</code> )              | ~3.7 ms/chiamata  |
| costo del tuner ( <code>assemble_loop + margin + isstable</code> )       | ~15.0 ms/chiamata |
| <code>simulate_gust_response</code> ( <code>lsim</code> , 2401 campioni) | ~5.8 ms/chiamata  |
| <code>design_controller</code> (una ricerca completa)                    | ~830 ms           |

Dieci millisecondi sembrano nulla. Moltiplicati per il Monte Carlo diventano **~14 secondi solo di assemblaggio** (1500 x 9.1 ms), a cui vanno aggiunti i margini: la stima realistica e' 20-25 s in seriale. E' esattamente la ragione per cui `main_montecarlo.m` usa `parfor` (riga 75). Se `assemble_loop` rallentasse di un fattore 3 – per esempio perche' qualcuno aggiunge un `minreal` in piu' o sostituisce `connect` con una costruzione meno efficiente – il Monte Carlo passerebbe da “si fa mentre prendi un caffe'” a “si fa stanotte”, e la cosa verrebbe scoperta **al momento sbagliato**. Questa suite serve a scoprirlo prima.

**Questa suite ha ospitato, fino a poco fa, un golden value obsoleto che la rompeva.** Ora e' allineato e la suite HM3 passa **35/35**, ma la storia e' troppo istruttiva per cancellarla: vedi la sezione su [Kref](#) qui sotto.

---

## Kref – un golden value che era diventato STALE (righe 8-13)

```
properties (Constant)
    % Task-1 PD design (pinned), same as hm3LoopTest. Keep the two in sync:
    % testDesignControllerSearch asserts the tuner converges back to Kp_th.
    Kref = struct('Kp_th', 1.7845, 'Kd_th', 0.4433, ...
                  'Kp_z', -1e-3, 'Kd_z', -1e-3)
end
```

Oggi (righe 11-12) Kref vale Kp\_th = 1.7845, Kd\_th = 0.4433, **esattamente** come hm3LoopTest.m (riga 11), ed e' il progetto che design\_controller produce davvero: Kp\_th = 1.784455. Il commento delle righe 9-10 e' stato riscritto perche' dica una cosa in piu' del semplice "same as hm3LoopTest": **avverte di tenere i due file in sync** e spiega *perche'* (testDesignControllerSearch asserisce che il tuner riconverga su quel numero).

Quell'avvertimento e' li' perche' e' stato pagato.

### Com'era, e perche' era rotto

Fino a poco fa questa property pinnava

```
Kref = struct('Kp_th', 1.9800, 'Kd_th', 1.3997, ...)
```

cioe' i guadagni **PRE-ritaratura**, e il commento sosteneva – falsamente – di essere "same as hm3LoopTest". hm3LoopTest.m pinnava gia' 1.7845 / 0.4433. **I due Kref non coincidevano**, e il secondo non corrispondeva a nessun progetto prodotto dal codice.

Conseguenza, **osservata eseguendo runperf**: il test testDesignControllerSearch (righe 91-98) **falliva**.

```
verifyEqual failed.
  Actual Value:    1.784454942663017
  Expected Value:  1.9800000000000000
  AbsoluteTolerance: 0.005
Totals: 0 Valid, 1 Invalid.
```

Il 1.9800 / 1.3997 era il residuo di **una versione precedente del progetto**, antecedente al re-tuning sull'anello completo – quello che ha portato i guadagni canonici di D'Antuono a essere raffinati contro i **margini classificati**. Quando la metodologia dei margini e' stata riscritta, il numero pinnato in hm3LoopTest e' stato aggiornato e **questo e' rimasto indietro**.

**Perche' era peggio di un semplice test rosso**. In matlab.perftest, una verifica fallita **invalida la misura**: runperf riportava *0 Valid, 1 Invalid*. Il test non era solo "rosso", **non produceva alcun dato di tempo** per il livello workflow. Cioe' il benchmark piu' importante della suite – quanto costa una ricerca completa del tuner – non misurava niente.

Gli altri quattro test **passavano lo stesso**, ed e' proprio questo che ha reso il bug longevo: usano Kref solo come *insieme di guadagni con cui chiudere l'anello per misurare i tempi*, e il tempo di **assemble\_loop non dipende dal valore numerico dei guadagni** (la struttura e l'ordine dell'anello sono gli stessi). Con 1.98 / 1.3997 l'anello rigido era comunque stabile e **margin()** restituiva valori finiti, quindi anche le asserzioni di sanita' delle righe 78-79 reggevano. Il valore stale era **invisibile** a tutto tranne che all'unico test che lo confrontava con l'output del tuner.

### La correzione, e la lezione

La correzione e' stata **una riga** – allineare Kref a 1.7845 / 0.4433 – piu' il commento di avvertimento. La correzione *strutturale*, quella che il codice non ha ancora fatto, sarebbe far derivare entrambe le suite da un'**unica sorgente di verita'** (una funzione, un .mat, una property condivisa) invece di duplicare i numeri in due file. **La**

**uplicazione e' la causa prima del bug**, non il numero sbagliato: finche' il valore vive in due posti, prima o poi i due posti divergono.

**Possibile domanda d'esame** – Come e' potuto succedere che un test pinni dei guadagni che il codice non produce piu'? *Risposta:* E' un classico *stale golden value*, e la meccanica e' istruttiva. Il valore era **uplicato in due file** (`hm3LoopTest.m` e `hm3LoopPerformanceTest.m`); quando ho ri-tarato il progetto sull'anello completo – riscrivendo la metodologia dei margini da `margin()` a `classify_margins` – ho aggiornato il test unitario e **non** il perf-test, che e' rimasto sui guadagni pre-ritaratura (1.98 / 1.40 invece di 1.78 / 0.44). Il fallimento e' rimasto nascosto perche' 4 test su 5 usano `Kref` solo come guadagni con cui chiudere l'anello per **cronometrarlo**, e il tempo non dipende dal loro valore: solo `testDesignControllerSearch`, che confronta l'output del tuner con il numero pinnato, se ne accorgeva – e in `matlab.perftest` una `verify` fallita **invalida la misura**, quindi il benchmark di livello workflow non produceva piu' alcun dato (*0 Valid, 1 Invalid*). **Attenzione a non giustificarlo dicendo che i perf-test “sfuggono a runtests”**: `matlab.perftest.TestCase` e' una sottoclasse di `matlab.unittest.TestCase`, quindi `runtests('HM3/tests')` – il comando prescritto dalle convenzioni del repo – scopre ed esegue anche questa classe, e il `verifyEqual` falliva li' esattamente come sotto `runperf`. La causa vera e' che non ho rilanciato la suite per intero dopo il re-tuning. Ora e' allineato (35/35), ma **la lezione e' che un golden value non va duplicato**: va derivato da un'unica sorgente. Un golden value non e' una verita' eterna, e' una **fotografia di un progetto** – e invecchia esattamente quando il progetto cambia, cioe' nel momento in cui sei meno propenso a rileggere i test.

---

## TestClassSetup e TestMethodSetup (righe 24-45)

```
methods (TestMethodSetup)
function buildModels(testCase)
    % Model construction outside the measurement boundary; the
    % conditionally stable loop makes margin() warn on every call
    ws = warning('off', 'Control:analysis:MarginUnstable');
    testCase.addTeardown(@() warning(ws));
    testCase.p      = load_hw3_params();
    testCase.Grigid = build_plant_rigid(testCase.p);
    testCase.Gfull  = build_plant_full(testCase.p, 'ins');
    testCase.Wchain = build_tvc(testCase.p) * ...
        build_notch_filter(testCase.p.wBM, 0.002, 0.7, +1);
    [~, testCase.Trigid] = assemble_loop(testCase.Grigid, testCase.Kref);
    testCase.wind = load_wind_profile(testCase.p);
end
end
```

- Righe 32-44: **tutta la costruzione dei modelli avviene nel setup**, cioe' fuori dal confine di misura. E' la decisione metodologicamente piu' importante del file, ed e' dichiarata nel commento (riga 33).

Il motivo: `matlab.perftest` misura **solo** cio' che sta dentro il ciclo `while testCase.keepMeasuring` (o fra `startMeasuring` e `stopMeasuring`). Se `load_hw3_params()`, `build_plant_full()` e `build_tvc()` stessero dentro il ciclo, il tempo misurato includerebbe il caricamento del `.mat` LPV e la costruzione delle `tf`, che **non fanno parte del ciclo caldo reale**: nel Monte Carlo e nel tuner il plant e la catena attuatore sono costruiti **una volta sola**, fuori dal loop, e solo `assemble_loop` viene richiamata. Un benchmark che includesse il setup misurerebbe il ciclo sbagliato e le ottimizzazioni verrebbero fatte nel posto sbagliato.

- Righe 35-36: la solita muta della warning `Control:analysis:MarginUnstable`, che su questo anello condizionatamente stabile scatterebbe **a ogni singola chiamata** di `margin`. Qui la motivazione e' anche **prestazionale**: emettere una warning ha un costo non nullo, e in un ciclo `keepMeasuring` da migliaia di iterazioni contaminerebbe la misura con il tempo di formattazione del messaggio. Il ripristino e' via `addTeardown`, come altrove.

- `TestMethodSetup` (non `TestClassSetup`): i modelli sono ricostruiti **prima di ogni test**. Costa qualche decina di millisecondi in piu' ma garantisce che ogni benchmark parta da oggetti freschi, senza cache interne della Control System Toolbox eventualmente scaldate dal test precedente.

---

## testAssembleLoopRigid (righe 48-57) – livello *unit*

```
% fminsearch cost kernel: close the rigid loop (connect +
% getLoopTransfer + minreal)
G = testCase.Grigid; K = testCase.Kref;
while testCase.keepMeasuring
    [L, T] = assemble_loop(G, K);
end
testCase.verifyEqual(order(T), 4);
testCase.verifyNotEmpty(L);
```

- Righe 52-54: **il ciclo keepMeasuring e' l'unita' di misura**. Il framework esegue il corpo del `while` ripetutamente (con dei *warmup* iniziali scartati), raccoglie campioni finche' non raggiunge una confidenza statistica prefissata, e riporta media e dispersione. E' il pattern giusto per operazioni **brevi** (~10 ms): una singola esecuzione sarebbe dominata dal rumore dello scheduler, dalla cache, dal JIT.
- Riga 51: le variabili sono **copiate in locali** prima del ciclo (`G = testCase.Grigid`) invece di essere lette dalle property dentro il ciclo. Non e' pignoleria: l'accesso a una property di un oggetto MATLAB passa per il meccanismo di dispatch della classe e costa qualcosa. Metterlo dentro il loop misurerebbe anche quello. E' un dettaglio idiomatrico dei perf-test scritti bene.
- Righe 55-56: **le asserzioni stanno FUORI dal ciclo di misura**. Anche questo e' deliberato: `verifyEqual` e la macchina di diagnostica del framework hanno un costo, e se stessero dentro il `while` verrebbero cronometrate insieme al codice sotto test. Le due verifiche servono solo come **sanity check** (il benchmark ha effettivamente costruito l'oggetto giusto), non come test.
- `order(T) == 4`: l'anello chiuso rigido ha 4 stati. Il controllore e' un **guadagno statico** (`Kc = ss(...)`) in `assemble_loop.m` riga 25, senza stati) e l'attuatore e' `tf(1)` (ideale): l'unica dinamica e' quella del plant.
- **Perche' questo e' il kernel giusto da misurare**. E' *esattamente* l'operazione che `fminsearch` ripete a ogni valutazione della funzione di costo (`design_controller.m` riga 82). Misurando questa, si misura il collo di bottiglia del tuner.

---

## testAssembleLoopFullChain (righe 59-67) – livello *unit*, caso pesante

```
G = testCase.Gfull; K = testCase.Kref; Wa = testCase.Wchain;
while testCase.keepMeasuring
    [L, T] = assemble_loop(G, K, Wa);
end
testCase.verifyEqual(order(T), 13); % 6 + 5 (TVC+Pade) + 2 (notch)
```

- Riga 65: il conteggio degli stati e' una **verifica di composizione della catena**, e il commento esplicita la decomposizione:  
 6 stati del plant completo (`z`, `zdot`, `theta`, `thetadot`, `eta`, `etadot`)
  - 5 stati della catena TVC (2 attuatore + 3 Pade)
  - 2 stati del notch (biquad) = 13

Verificato: `order(T) = 13`. Se un `minreal` interno cancellasse qualcosa, o se l'ordine del Pade cambiasse, il conteggio non tornerebbe. E' lo stesso invariante di `hm3FilterTest.testTvcOrderIsActuatorPlusPade`, ma **a valle**, sull'anello assemblato: verifica che `connect` non abbia perso pezzi per strada.

- **Il risultato sorprendente:** l'anello completo a 13 stati costa **~9.1 ms**, cioè *meno* dell'anello rigido a 4 stati (~10.5 ms). E' controintuitivo e vale la pena averlo notato. La spiegazione plausibile e' che il costo di `assemble_loop` sia dominato non dalla dimensione del sistema (13 contro 4 stati sono entrambi minuscoli per un solver denso) ma dall'**overhead fisso** di `connect` (parsing dei nomi, costruzione della matrice di interconnessione) e di `minreal`, e che la tolleranza `1e-6` di `minreal` faccia lavori diversi nei due casi. Comunque sia, e' proprio questo il genere di conclusione che un perf-test serve a produrre: **l'intuizione "piu' stati = piu' lento" e' sbagliata qui**, e ottimizzare riducendo l'ordine del modello non porterebbe da nessuna parte. Se si volesse davvero accelerare il Monte Carlo, il bersaglio sarebbe l'`overhead` di `connect/minreal` (per esempio precostruendo l'interconnessione una volta e aggiornando solo i guadagni), non il numero di stati.

---

## testTunerCostEvaluation (righe 69-80) – livello *system*

```
% One full cost evaluation as design_controller performs it:
% assemble_loop + margin + isstable on the rigid loop
G = testCase.Grigid; K = testCase.Kref;
while testCase.keepMeasuring
    [L, T] = assemble_loop(G, K);
    [Gm, Pm] = margin(L);
    stable = isstable(T);
end
testCase.verifyTrue(stable);
testCase.verifyTrue(isfinite(Gm) && isfinite(Pm));
```

- Righe 73-77: misura **una valutazione completa della funzione di costo**, non solo il suo pezzo piu' costoso. E' il salto dal livello *unit* al livello *system*: quantifica il costo dell'unita' atomica che `fminsearch` ripete.
- Misurato: **~15.0 ms**, contro i ~10.5 ms del solo `assemble_loop`. Quindi l'assemblaggio pesa circa il 70 % del costo, e l'analisi dei margini il restante 30 %. **Questa e' l'informazione azionabile:** se si volesse accelerare il tuner, bisogna guardare `assemble_loop`, non `margin`.
- Righe 78-79: sanity check fuori dal ciclo. `isfinite(Gm) && isfinite(Pm)` verifica che l'anello abbia effettivamente degli attraversamenti da misurare: se `margin` restituisse `Inf` (nessun crossing), la funzione di costo di `design_controller` sarebbe degenerare e la misura non sarebbe rappresentativa.
- **Nota di onesta'.** Il test replica la funzione di costo usando `margin(L)`, ma `design_controller` usa in realta' `classify_margins(Lt, bands{:})` (riga 84 del sorgente), che a sua volta chiama `allmargin`. Non e' la stessa cosa: misurato, `classify_margins` costa ~3.7 ms, un valore simile a `margin`, quindi la stima complessiva regge. Ma il test **non sta misurando esattamente il codice che gira nel tuner**, e chiamarlo "one full cost evaluation as design\_controller performs it" (riga 70) e' una lieve sovra-affermazione. Manca inoltre il `minreal(Lt, 1e-6)` che `design_controller` esegue alla riga 83.
- Coerenza dei numeri: ~15 ms per valutazione x ~55 valutazioni = ~830 ms, che e' esattamente il costo misurato di una ricerca completa. I conti tornano, il che conferma che il modello di costo e' corretto.

---

## testGustResponseLsim (righe 82-89) – livello *system*

```
% Time-domain replay: lsim over the 2401-point severe gust
T = testCase.Trigid; w = testCase.wind;
while testCase.keepMeasuring
```

```

    r = simulate_gust_response(T, w);
end
testCase.verifySize(r.theta, [numel(w.t) 1]);

```

- Righe 85-87: misura la simulazione temporale. `simulate_gust_response` e' un wrapper attorno a un solo `lsim` (riga 21 del sorgente) su una griglia di **2401 punti** (12 s a  $dt = 5$  ms, piu' l'istante iniziale – verificato: `numel(w.t) = 2401`, coerente con il commento della riga 83).
- Misurato: **~5.8 ms**. E' l'operazione **piu' economica** del gruppo, il che e' un risultato in se': la simulazione temporale, che intuitivamente sembra la cosa costosa (2401 passi di integrazione!), costa **meno della meta'** di un assemblaggio d'anello. Il motivo e' che `lsim` su un sistema lineare tempo invariante non integra numericamente: **discretizza una volta** (matrice esponenziale) e poi itera una ricorsione matrice-vettore, che su 4 stati e' quasi gratis. Il collo di bottiglia di HM3 e' l'**algebra dei sistemi**, non l'**integrazione**.
- Riga 88: `verifySize(r.theta, [numel(w.t) 1])` – vettore colonna della lunghezza giusta. Sanity check, fuori dal ciclo.
- Perche' vale la pena misurarlo comunque: il Monte Carlo (`main_montecarlo.m`) e i corner del Task 3 producono anche **risposte temporali** per ogni caso, non solo margini. Con  $N = 1500$  anche 5.8 ms diventano ~9 s.

---

## testDesignControllerSearch (righe 91-98) – livello *workflow*

```

% Workflow level: one complete PD margin-matching search
G = testCase.Gridid;
testCase.startMeasuring();
K = design_controller(G, [], 'verbose', false);
testCase.stopMeasuring();
testCase.verifyEqual(K.Kp_th, testCase.Kref.Kp_th, 'AbsTol', 5e-3);

```

- Righe 94-96: **cambia la API di misura**. Qui non c'e' `keepMeasuring` ma la coppia esplicita `startMeasuring / stopMeasuring`. La differenza e' sostanziale:
  - `keepMeasuring` ripete il corpo **molte volte** finche' la statistica non converge. Va bene per operazioni da millisecondi.
  - `startMeasuring/stopMeasuring` cronometra **una singola esecuzione** del codice fra le due chiamate. E' il pattern per operazioni **costose**, dove ripetere centinaia di volte sarebbe proibitivo: qui una ricerca completa costa **~830 ms**, quindi anche solo i warmup del framework portano il test a durare una quindicina di secondi (misurato: `runperf` su questo singolo test ha impiegato 15.7 s).
- **Perche' esiste il livello workflow**. I tre test precedenti misurano i *mattoni*; questo misura il *risultato*. Serve a rispondere a una domanda che i micro-benchmark non possono rispondere: “quante volte posso permettermi di ri-tarare il controllore?”. La risposta – 830 ms – e' quella che rende praticabile il Task 3 (ri-tarare a ogni vertice del box di incertezza) e il Monte Carlo (dove pero' il controllore e' **fisso**, riga 42 di `main_montecarlo.m`: `K = design_controller(...)` viene chiamato **una volta sola**, fuori dal `parfor`, e poi congelato – che e' anche il senso fisico dello studio di robustezza: il controllore non si adatta, subisce).
- **Riga 97: e' la riga che era rotta, e ora passa**. `testCase.Kref.Kp_th` vale 1.7845 e `design_controller` restituisce 1.784455: lo scarto e'  $4.5e-5$  contro una `AbsTol` di  $5e-3$ , comodamente dentro. Fino a poco fa `Kref` pinnava pero' 1.9800, e il test falliva con uno scarto di 0.196 – **invalidando la misura** (0 Valid, 1 Invalid), cioe' azzerando il benchmark di livello workflow. Vedi la sezione su `Kref` per la storia completa.
- Nota metodologica: usare una `verifyEqual` sui guadagni **dentro un perf-test** e' discutibile in se', e questo episodio lo dimostra. Un perf-test dovrebbe misurare; la correttezza e' compito di `hm3LoopTest`, che gia' pinna gli stessi guadagni (`testDesignControllerMeetsTargets`, righe 84-85). La duplicazione qui non aggiunge copertura e ha aggiunto un modo di rompersi – **quello che poi si e' rotto davvero**. Il valore residuo dell'asserzione e' garantire che il benchmark non stia cronometrando una ricerca **divergente** (un `fminsearch` che sbatte contro `MaxFunEvals` senza convergere costerebbe di piu' e falserebbe la misura): un controllo piu' robusto e meno fragile sarebbe verificare i **margini** (`m.stable`,  $|GM| \sim 6$ ,  $PM \sim 30$ ) invece

dei guadagni, perché i margini sono i **requisiti della traccia** e non si spostano quando il progetto si ri-tara.

---

## Cosa questa suite NON fa (limiti noti)

E' il punto più importante da saper dire all'orale, perché un perf-test mal capito dà un falso senso di sicurezza.

- **Non c'è nessuna soglia.** Nessun test asserisce “questa operazione deve durare meno di X ms”. **runperf misura e riporta**, non boccia. Cioè se domani `assemble_loop` diventasse 10 volte più lenta, **tutti i test passerebbero comunque**: il rallentamento comparirebbe solo nella tabella dei tempi, e solo se qualcuno la guarda. Un perf-test senza soglia è una **misura**, non un **test di regressione**. Per farne un gate servirebbe salvare una baseline e confrontarsi con quella (`matlab.perftest` lo supporta, ma qui non è usato).
  - **Il Monte Carlo (N = 1500) non è benchmarkato**, pur essendo il ciclo caldo citato nel commento di testa. Il perf-test misura il **mattone** (`assemble_loop`) ma non il **ciclo** che lo contiene, quindi non cattura l'overhead del `parfor`, del trasferimento dei dati ai worker, né la degradazione a seriale in assenza di Parallel Computing Toolbox.
  - **Il trade dei filtri del Task 2 non è benchmarkato**, idem.
  - **classify\_margins non è misurata direttamente**, pur essendo dentro la funzione di costo del tuner (`testTunerCostEvaluation` usa `margin` al suo posto).
  - **Non si misura la memoria**, solo il tempo.
  - Piccola tensione con la convenzione del repo: il `CLAUDE.md` prescrive che le funzioni di ciclo caldo **non** portino validazione `arguments`, perché stanno dentro loop da milioni di chiamate. `assemble_loop.m` ha un blocco `arguments` (righe 15-19) ed è una funzione di ciclo caldo. In questo caso la deroga è innocua – il costo di `mustBeA` è del tutto trascurabile rispetto ai ~10 ms di `connect` + `minreal` – ma vale la pena notare che **è proprio questo perf-test lo strumento che permette di affermarlo con dati invece che per fede**.
- 

## Il commento “~400x” (righe 3-4)

```
% Unit: one assemble_loop call (fminsearch hits it ~400x, the Task-2
% sweeps ~150x). System: full tuner cost (assemble_loop + margin +
% isstable) and the lsim gust replay. Workflow: one design_controller
% search. Run: runperf('hm3LoopPerformanceTest')
```

Il “~400x” è il **limite superiore**, non il valore tipico: 400 è il `MaxFunEvals` passato a `optimset` in `design_controller.m` (riga 57). Misurando, una ricerca completa costa ~830 ms a ~15 ms per valutazione, cioè **~55 valutazioni effettive**: `fminsearch` converge molto prima del tetto, grazie al buon seed di D'Antuono e alle tolleranze (`TolX` = 1e-4, `TolFun` = 1e-3). Il commento non è sbagliato come *worst case*, ma va letto per quello che è.

Questo, incidentalmente, è un buon argomento a favore del seed analitico: partire dalla coppia canonica  $Kp0 = 2 \cdot A_6 / K_1$ ,  $Kd0 = \sqrt{A_6} / K_1$  invece che da un punto arbitrario riduce di un ordine di grandezza il numero di valutazioni. **Il seed teorico non è eleganza, è performance.**

---

## Possibili domande d'esame

**D: Perché hai scritto dei performance test per un homework di controlli? Non basta che il risultato sia giusto?** R: Perché HM3 non calcola un anello, ne calcola migliaia. `assemble_loop` viene chiamata fino a 400 volte per ogni ricerca dell'auto-tuner, ~150 volte nel trade dei filtri del Task 2, e **1500 volte** nel Monte



Carlo (una per estrazione, dentro il `parfor` di `main_montecarlo.m`). Misurata, costa ~10 ms a chiamata: solo il Monte Carlo sono ~14 s di assemblaggio, piu' i margini. Se qualcuno la rallentasse di un fattore 3 – aggiungendo un `minreal` di troppo, per dire – il Monte Carlo diventerebbe impraticabile e me ne accorgerei nel momento sbagliato. Il `perf-test` quantifica il costo dei mattoni e mi dice **dove** ottimizzare se serve: ho misurato che l'assemblaggio pesa il 70 % di una valutazione della funzione di costo e l'analisi dei margini il 30 %, quindi il bersaglio e' `connect/minreal`, non `margin`.

**D: Perche' un test usa `keepMeasuring` e un altro `startMeasuring/stopMeasuring`?** R: Dipende dal costo dell'operazione. `keepMeasuring` ripete il corpo del ciclo molte volte, scartando dei `warmup`, finche' la misura non e' statisticamente affidabile: e' il pattern giusto per operazioni brevi (~10 ms), dove una singola esecuzione sarebbe sepolta dal rumore di scheduler e cache. `startMeasuring / stopMeasuring` cronometra invece **una singola esecuzione**, ed e' il pattern per operazioni costose: una ricerca completa di `design_controller` dura ~830 ms, e ripeterla centinaia di volte per convergenza statistica renderebbe il test inutilizzabile (gia' cosi', con i soli `warmup`, il test impiega ~16 s). In entrambi i casi le `verify*` stanno **fuori** dal confine di misura, altrimenti cronometrerei anche il framework di test.

**D: Che cosa hai scoperto misurando, che non avresti indovinato?** R: Due cose. Primo, che l'anello **completo** a 13 stati (`plant + TVC + Pade + notch`) si assembla in ~9 ms, cioe' **piu' velocemente** dell'anello rigido a 4 stati (~10.5 ms). L'intuizione “piu' stati = piu' lento” e' falsa qui: il costo e' dominato dall'overhead fisso di `connect` e `minreal`, non dalla dimensione del sistema. Ridurre l'ordine del modello non accelererebbe nulla. Secondo, che `lsim` sulla raffica da 2401 punti costa ~5.8 ms, **meno della meta'** di un assemblaggio d'anello: la simulazione temporale non e' il collo di bottiglia, perche' su un sistema LTI `lsim` discretizza una volta e poi itera una ricorsione matrice-vettore, non integra numericamente. Il collo di bottiglia di HM3 e' l'algebra dei sistemi, non l'integrazione.

**D: I tuoi `perf-test` possono fallire se il codice rallenta?** R: No, ed e' un limite che riconosco. Nessuno dei test asserisce una **soglia** di tempo: `runperf` misura e riporta, non boccia. Se `assemble_loop` diventasse dieci volte piu' lenta, i test passerebbero comunque e il rallentamento comparirebbe solo nella tabella dei tempi. Per farne un vero gate di regressione servirebbe salvare una **baseline** e confrontarsi con quella (`matlab.perftest` lo supporta). Allo stato, questa suite e' uno strumento di **profiling riproducibile**, non un test di non-regressione prestazionale.

**D: C'e' stato qualcosa che non andava in questa suite?** R: Si', ed e' l'aneddoto che racconto piu' volentieri perche' e' un errore *strutturale*, non di distrazione. La property `Kref` di `hm3LoopPerformanceTest` pinnava `Kp_th = 1.9800`, `Kd_th = 1.3997` – i guadagni **pre-ritaratura** – mentre il commento sosteneva di essere “same as `hm3LoopTest`”, che invece pinnava `1.7845 / 0.4433`. I due non coincidevano, e `design_controller` produce `1.784455`. Risultato: `testDesignControllerSearch` **falliva**, e siccome in `matlab.perftest` una verifica fallita **invalida la misura**, il benchmark di livello workflow non produceva piu' alcun dato (`0 Valid, 1 Invalid`) – cioe' il numero piu' importante della suite era silenziosamente sparito. Era uno *stale golden value* nato dalla **duplicazione dello stesso numero in due file**: quando ho ri-tarato il progetto sull'anello completo (passando da `margin()` a `classify_margins`) ho aggiornato il test unitario e non il `perf-test`, e non ho rilanciato la suite per intero. Ed e' rimasto nascosto a lungo perche' gli altri quattro test usano `Kref` solo per **cronometrare** l'anello, e il tempo non dipende dal valore dei guadagni. Non e' che il `perf-test` sfugga a `runtests: matlab.perftest.TestCase` deriva da `matlab.unittest.TestCase`, quindi `runtests('HM3/tests')` – il comando prescritto dalle convenzioni del repo – scopre anche questa classe e il fallimento sarebbe emerso li'. **Ora e' allineato e la suite HM3 passa 35/35**. La correzione vera pero' non e' il numero: e' non duplicarlo affatto, derivandolo da un'unica sorgente. La lezione generale e' che un *golden value* e' la **fotografia di un progetto**, e invecchia esattamente quando il progetto cambia – cioe' quando sei meno propenso a rileggere i test. Per questo, in un `perf-test`, avrei fatto meglio ad asserire i **margini** (che sono i requisiti della traccia e non si spostano) invece dei guadagni (che sono un risultato e si spostano).